控制大综合

第72章 智能控制的混合架构

- 72.1 动机与直观理解
- 72.2 数学原理
- 72.3 设计步骤
- 72.4 连续情形
- 72.5 离散情形
- 72.6 稳定性与性能
- 72.7 参数意义与作用
- 72.8 调参方法与经验
- 72.9 优化与改进
- 72.10 工程落地
- 72.11 优缺点与适用边界
- 72.12 Python 仿真代码

第73章 输出分布控制（SDC）

- 73.1 动机与直观理解
- 73.2 数学原理
- 73.3 设计步骤
- 73.4 连续情形
- 73.5 离散情形
- 73.6 稳定性与性能
- 73.7 参数意义与作用
- 73.8 调参方法与经验
- 73.9 优化与改进
- 73.10 工程落地
- 73.11 优缺点与适用边界
- 73.12 Python 仿真代码

第74章 分布参数系统控制（DPS）

- 74.1 动机与直观理解
- 74.2 数学原理
- 74.3 设计步骤
- 74.4 连续情形
- 74.5 离散情形
- 74.6 稳定性与性能
- 74.7 参数意义与作用
- 74.8 调参方法与经验

- 74.9 优化与改进
- 74.10 工程落地
- 74.11 优缺点与适用边界
- 74.12 Python 仿真代码

第75章 事件触发与自触发控制

- 75.1 动机与直观理解
- 75.2 数学原理
- 75.3 设计步骤
- 75.4 连续情形
- 75.5 离散情形
- 75.6 稳定性与性能
- 75.7 参数意义与作用
- 75.8 调参方法与经验
- 75.9 优化与改进
- 75.10 工程落地
- 75.11 优缺点与适用边界
- 75.12 Python 仿真代码

第76章 混杂系统与切换控制

- 76.1 动机与直观理解
- 76.2 数学原理
- 76.3 设计步骤
- 76.4 连续情形
- 76.5 离散情形
- 76.6 稳定性与性能
- 76.7 参数意义与作用
- 76.8 调参方法与经验
- 76.9 优化与改进
- 76.10 工程落地
- 76.11 优缺点与适用边界
- 76.12 Python 仿真代码

第77章 Koopman 算子与数据驱动控制

- 77.1 动机与直观理解
- 77.2 数学原理
- 77.3 设计步骤
- 77.4 连续情形
- 77.5 离散情形
- 77.6 稳定性与性能
- 77.7 参数意义与作用
- 77.8 调参方法与经验

- 77.9 优化与改进
- 77.10 工程落地
- 77.11 优缺点与适用边界
- 77.12 Python 仿真代码

第78章 容错控制

- 78.1 动机与直观理解
- 78.2 数学原理
- 78.3 设计步骤
- 78.4 连续情形
- 78.5 离散情形
- 78.6 稳定性与性能
- 78.7 参数意义与作用
- 78.8 调参方法与经验
- 78.9 优化与改进
- 78.10 工程落地
- 78.11 优缺点与适用边界
- 78.12 Python 仿真代码

第79章 安全控制

- 79.1 动机与直观理解
- 79.2 数学原理
- 79.3 设计步骤
- 79.4 连续情形
- 79.5 离散情形
- 79.6 稳定性与性能
- 79.7 参数意义与作用
- 79.8 调参方法与经验
- 79.9 优化与改进
- 79.10 工程落地
- 79.11 优缺点与适用边界
- 79.12 Python 仿真代码

第 80 章 学习增强的模型预测控制

- 80.1 动机与直观理解
- 80.2 数学原理
- 80.3 设计步骤
- 80.4 连续情形
- 80.5 离散情形
- 80.6 稳定性与性能
- 80.7 参数意义与作用
- 80.8 调参方法与经验

- 80.9 优化与改进
- 80.10 工程落地
- 80.11 优缺点与适用边界
- 80.12 Python 仿真代码

第 81 章 全驱系统理论

- 81.1 动机与直观理解
- 81.2 数学原理
- 81.3 设计步骤
- 81.4 连续时间情形
- 81.5 离散时间情形
- 81.6 稳定性与性能
- 81.7 参数意义与作用
- 81.8 调参方法与经验
- 81.9 优化与改进
- 81.10 工程落地
- 81.11 优缺点与适用边界
- 81.12 Python 仿真代码

第 82 章 网络化控制系统

- 82.1 动机与直观理解
- 82.2 数学原理
- 82.3 设计步骤
- 82.4 连续情形
- 82.5 离散情形
- 82.6 稳定性与性能
- 82.7 参数意义与作用
- 82.8 调参方法与经验
- 82.9 优化与改进
- 82.10 工程落地
- 82.11 优缺点与适用边界
- 82.12 Python 仿真代码

第 83 章 量子控制（选读）

- 83.1 动机与直观理解
- 83.2 数学原理
- 83.3 设计步骤
- 83.4 连续情形
- 83.5 离散情形
- 83.6 稳定性与性能
- 83.7 参数意义与作用
- 83.8 调参方法与经验

- 83.9 优化与改进
- 83.10 工程落地
- 83.11 优缺点与适用边界
- 83.12 Python 仿真代码

第72章 智能控制的混合架构

72.1 动机与直观理解

前面六章各讲了一种“新”控制器：模糊规则、神经网络、遗传整定、强化学习、迭代学习。每一章都尽量把机制讲透，也因此都面临同一个追问：既然第 44 章的 PID 只有三个参数、跑得又快又稳，为什么还要引入这些结构？第 66–71 章的回答是“单机制在某些工况下确实更好”，但那些更好往往是局部的：模糊规则改善了超调，神经网络拟合了非线性，强化学习学会了前馈形状，可它们各自的短板也同样明显——模糊规则靠人写、神经网络不保证稳定、强化学习要大量回合、迭代学习离不开重复轨迹。本章不再新增机制，只回答一个工程上更常见的问题：**把这些机制装进同一个回路时，谁负责哪一段、彼此怎么交接、收益到底从哪来。**

先把算例钉死

为了让“混合”与“不混合”能直接比较，本章继续用第 66–70 章那个算例：二阶非线性对象

$$y'' = -2y' + K(y)\,u, \qquad K(y) = 4 + 6y \tag{72-1}$$

工作区间 $y \in [1.5,\ 2.1]$ 内静态增益从 13 变到 20，接近 1.5 倍；采样周期 $T_s = 10$ ms；执行到测量的纯延迟 5 拍；控制量限幅 ± 1.2；参考轨迹是 4 s 一个来回的往复运动

$$r(t) = R_0 + A_1\bigl(1 - \cos \omega t\bigr) + A_3\bigl(1 - \cos 3\omega t\bigr) \tag{72-2}$$

其中 $R_0 = 1.50$、$A_1 = 0.55$、$A_3 = 0.06$、$\omega = 2\pi/4$。每回合 400 拍，回合结束复位。基线固定增益 PID 取 $(K_p, K_i, K_d) = (2.00,\ 0.05,\ 0.20)$，与前面几章一字不差。这样做的代价是本章不能挑一个“对自己有利”的对象，好处是任何一句“比 PID 好多少”都能被读者拿前几章的数字复核。

结论先摆在这里

在同一轨迹、同一限幅、同一延迟下，五种结构的跟踪误差 RMS 分别是：

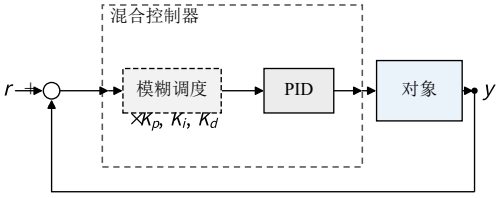
结构	控制器	RMS	最大误差	控制量总变差
无混合	固定 PID	0.1161	0.1974	1.708
串联	模糊调度 PID	0.1133	0.2017	1.741
并联	PID + 前馈查表	0.1057	0.1559	1.695
切换	双模 PD/PID	0.0814	0.1482	2.281
分层	MPC 上层 + PID 底层	0.0310	0.0639	2.462
单一机制	长时域 MPC	0.0048	0.0127	1.569

这张表最容易被误读成“越复杂越好”。真实情况没有这么顺：串联的模糊调度只把误差从 0.1161 压到 0.1133，改善 2.4%，还不如把 PID 三个参数在网格上重搜一遍得到的 0.0854；而分层结构拿到 0.0310，靠的并不是“更聪明的智能算法”，而是把预测权限交给了一个会算未来的模块。换句话说，**混合架构的收益主要来自“把某一段工作交给有能力做它的模块”，而不是来自“模块本身有多智能”**。

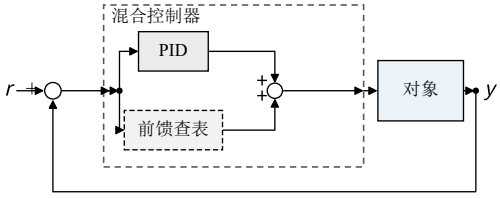
四种接法，四种分工

按模块之间的连接关系，混合结构可以分成四类，图 72-1 把它们画在同一张图里。四个面板的外环完全一样（求和点、对象、反馈），只有虚线框内的部分不同，这样比较的才是结构而不是画法。

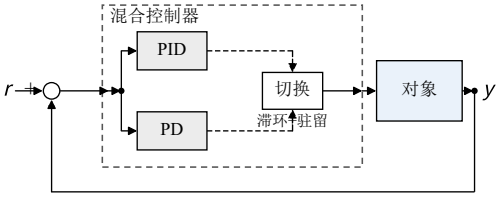
（a）串联：调度器改参数



（b）并联：补偿量相加



（c）切换：按误差分段



（d）分层：上层给目标

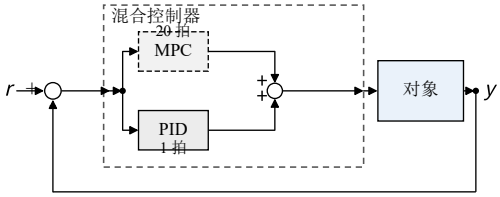


图 72-1 四种混合拓扑

串联是把一个模块的输出接到另一个模块的输入端。最常见的是“调度器 + PID”：模糊系统不直接产生控制量，只根据误差与误差导数调整 PID 的三个增益，控制律仍然是 PID。这种做法保留了 PID 的积分作用与稳定性直觉，风险最小，收益也最小。

并联是两路同时工作、输出相加。一路通常是 PID，负责低频与稳态；另一路是补偿器，负责前馈、非线性补偿或相位超前。并联的关键不在两路各自多强，而在“补偿量是否可学”——本章会看到，把误差、误差导数塞进补偿网络，闭环反而比不加更差，因为网络把拟合误差变成了额外的反馈增益。

切换是按工况分段，同一时刻只有一个控制器在起作用。大误差段用高增益 PD 抢快速性，小误差段切回 PID 消静差，中间用滞环与最短驻留防抖。切换的难点从来不是“选哪个”，而是“切换瞬间输出跳多少”。

分层是上下两级：上层看得远、算得慢，负责给出目标或前馈；下层看得近、算得快，负责跟踪与抗扰。本章用 MPC 做上层、PID 做下层，两者通过一个限幅的“权限”接口相连。

一条必须先说的负面结果

如果混合只是“把 PID 换成一个神经网络”，结果会很糟。用 2 输入（误差、误差导数）、9 条规则的零阶 TSK 直接替代 PID，开环拟合的测试 RMSE 只有 0.0893，看起来不错；可一旦接进闭环，RMS 冲到 2.8701，控制量 30.2% 的拍数贴着限幅，最大误差 3.9078——系统已经失稳。原因不神秘：这个映射里没有积分状态，误差长期偏一点它就长期偏一点；而训练数据只覆盖了 PID 轨迹附近很窄的一条带，闭环一旦偏离，网络就在自己没见过的区域外推。

同一批数据，换成 3 输入（多一个参考位置）后闭环误差降到 0.1302；再把补偿量限制成“只依赖参考位置”的一维查表，闭环误差 0.1057，终于低于 PID。三组数字说明的是同一件事：**混合架构的性能瓶颈通常在接口，而不在模块。**输入选错了，参数再多也没用；接口选对了，9 个参数也能干活。这句话会贯穿本章后面的每一节。

三个必须先拆掉的误解

在往下走之前，有三个说法需要先拆掉，因为它们几乎出现在每一次关于混合架构的讨论里，而且都会把设计带偏。

第一个误解是“混合等于更智能”。混合只改变分工，不改变每个模块的智力水平。本章的模糊-PID 用了 9 条规则、9 个参数，看起来比三个数的 PID“聪明”，可它只把误差从 0.1161 压到 0.1133。原因很简单：它做的事仍然是“用误差算控制量”，只是把常数换成了有界函数，信息的来源没有变。判断一个混合结构是否真的“更智能”，唯一的标准是“它有没有拿到别的模块拿不到的信息”，而不是“它用了什么算法”。

第二个误解是“模块越多越强”。本章有一组现成的反例：同样是 9 个参数，接 2 个输入（误差、误差导数）闭环失稳到 2.8701，只接 1 个输入（参考位置）却做到 0.1057。参数一样多，接法不同，结果差 27 倍。模块数量与性能之间没有单调关系，甚至常常相反——每多一个模块，就多一条可能出错的通道、多一个需要定契约的接口。

第三个误解是“有了智能模块就能自适应”。自适应需要两件事同时成立：模块的输入能反映工况变化，且它的有效区域真的被访问到。本章的调度表两条都只满足一半：输入里有误差与误差导数，能反映工况；但闭环实际只走到曲面的中心一小块（K_p 只在 0.955–1.164 之间活动），表上那些为“大误差”准备的格子从未被踩到。设计者以为自己在做增益调度，实际做的是一条几乎不变的增益曲线。

拆掉这三个误解之后，本章讨论的范围也就清楚了：这里的“混合”专指**两个及以上各自具备独立控制律的模块装进同一个回路**。传感器滤波、抗积分饱和、指令限速这些常规环节不算混合，它们是任何控制器都有的外围；而“PID 加一个前馈项”算不算，取决于前馈项是否有自己的参数与设计目标——本章把它算作最简单的并联混合。

本章的走法

72.2 把四种结构写成统一的数学形式，重点是“调度”“切换”“相加”“分层”这四个动作各自的表达式与约束；72.3 给出可照做的设计步骤，包括接口契约怎么写；72.4 与 72.5 分别在连续与离散两种视角下讨论延迟、采样与预测矩阵；72.6 讨论稳定性与性能，会给出“权限越大越不稳”的定量证据；72.7 逐个参数说明它动的是哪一段；72.8 给协同调参的顺序与经验；72.9 讨论终端值学习这类“给短时域打补丁”的做法为什么只在中等时域有效；72.10 讲工程落地，包括复杂度预算与模块间接口；72.11 给适用边界；72.12 是完整代码。

若只想记一句话：**混合不是把多个控制器堆在一起，而是把不同性质的工作分给不同性质的模块，并为它们规定一个不会互相打架的接口。**

72.2 数学原理

所有混合结构共用一个骨架

把一个混合控制器拆开，本质上只有三样东西：一条反馈通道 u_{fb}、一条前馈通道 u_{ff}、一组可调或可学的参数 θ。写成统一形式

$$u(k) = \Psi\big(u_{fb}(k),\, u_{ff}(k),\, \theta(x(k))\big) \tag{72-3}$$

差别全在 Ψ 上。而 Ψ 能做的事只有四种：让 θ 随状态变（调度）、让两路相加（并联）、按条件选一路（切换）、把一路嵌进另一路（分层）。四种拓扑对应四种算子，这不是分类学的巧合，而是“两个模块的输出如何组合”这个问题只有这几种不破坏因果性的答案。下面逐个写出表达式，并指出每种组合在数学上新增了什么约束。

四种算子覆盖了所有接法

把两个模块记为 A 与 B，各自把状态映射成控制量。要让它们同时存在于一个因果回路里，输出只能这样合成：

$$u = B\big(A(x)\big), \qquad u = A(x) + B(x), \qquad u = \sigma(x)\,A(x) + \big[1-\sigma(x)\big]B(x), \qquad u = B\big(x;\, A(x)\big)$$

四式分别是复合、相加、条件选择、把一方的输出当作另一方的目标或约束。之所以没有第五种，是因为“两个函数的合成”在因果系统里只有代入、相加、选择、约束四种基本形式；任何更复杂的写法都可以展开成这四者的树。

这个视角有两个用处。第一，它给出了“复杂度”的一个可计算定义：**树的深度**。串联、并联、切换都是深度 1，只有一个接口；分层是深度 2——上层算目标、经由权限接口交给下层，两个接口串联。每多一层，就多一处需要单独验证的契

约，也多一处可能出错的地方。本章后续所有关于“复杂度预算”的讨论，本质上都是在说“树的深度不要超过 2”。

第二，它解释了为什么“模糊混合”介于串联与并联之间。把上面第三式里的 σ 从 $\{0,1\}$ 放松到 $[0,1]$，切换就变成了加权混合；若 σ 由模糊规则给出，它就是“模糊混合”，既有并联的连续性，又有切换的分段意图。本章没有采用这种结构，因为它把两种控制律的缺点也一起继承了：既需要两套控制器都在线，又需要额外设计 σ 的形状。

串联：调度器只动参数，不动结构

模糊-PID 的写法是

$$u = s_{Kp}(e,\dot e)\,K_p\,e \;+\; s_{Ki}(e,\dot e)\,K_i\!\int\! e\,dt \;+\; s_{Kd}(e,\dot e)\,K_d\,\dot e \tag{72-4}$$

三个调度系数各自是一张 3×3 规则表的输出。每条输入先归一化到 $[-1,1]$ 的三个语言值（负、零、正），隶属度用三角函数

$$\boldsymbol{\mu}(x) = \Big[\max(-x,0),\;\; 1-|x|,\;\; \max(x,0)\Big], \qquad x\in[-1,1] \tag{72-5}$$

规则用乘积推理，去模糊用加权平均，于是调度系数是一个连续曲面

$$s(e,\dot e) = \frac{\sum_{i=1}^{9}\mu_i(e)\,\mu_j(\dot e)\,c_{ij}}{\sum_{i=1}^{9}\mu_i(e)\,\mu_j(\dot e)} \tag{72-6}$$

其中 c_{ij} 是第 (i,j) 条规则的结论。式 (72-6) 有两个性质值得记住：分母恒为正，所以曲面不会除零发散；结论是常数的加权平均，所以输出一定落在 $\min c_{ij}$ 与 $\max c_{ij}$ 之间，不可能超出人写的范围。这就是串联结构“风险最小”的数学原因——它没有新增任何动态，只是把三个常数换成了三个有界函数。本章的 K_p 表取

$$C_{Kp} = \begin{bmatrix} 1.30 & 1.15 & 1.00 \\ 1.15 & 1.00 & 0.95 \\ 1.00 & 0.95 & 0.85 \end{bmatrix} \tag{72-7}$$

（行对应 e 的负/零/正，列对应 $\dot e$ 的负/零/正）。读法是“误差大且误差在扩大时把 K_p 放大到 1.3 倍，误差大但误差在收敛时收回到 0.85 倍”。K_i 表方向相反，误差大时压低积分防饱和；K_d 表在误差大且同向时略降，避免微分放大噪声。

切换：它是状态机，不是函数

双模控制不能写成一个静态映射，因为同一组 $(e,\dot e)$ 在不同时刻可能对应不同模式。正确的写法是带记忆的状态转移：

$$\sigma(k) = \begin{cases} \text{PD}, & |e| > \theta_{th} + h \ \text{且驻留期满} \\ \text{PID}, & |e| < \theta_{th} - h \ \text{且驻留期满} \\ \sigma(k-1), & \text{其他} \end{cases} \tag{72-8}$$

h 是滞环半宽，驻留期是最短保持拍数。式 (72-8) 里两个机制各自解决一个问题：滞环消除阈值附近的反复穿越，驻留消除模式抖动带来的控制量抖动。切换瞬间的输出不连续量定义为

$$\Delta u = \big|u_B(k) - u_A(k)\big| \tag{72-9}$$

无扰切换的做法是在切换时刻记下偏置 $b = u_A - u_B$，此后输出 $u_B + b$。注意这**只保证输出连续，不保证状态一致**：B 的积分器仍然是它自己的历史，偏置会一直挂在输出上，直到被积分作用慢慢吸收。72.8 节会给出实测——本章算例里直接切换的跳变是 0.1171，偏置补偿把跳变压到 0，但切换后 0.3 s 内的最大误差反而从 0.1674 升到 0.1875。所以“无扰”是一个局部性质，不等于“更好”。

并联：相加之前先限幅

并联结构的表达式最简单

$$u = u_{PID} + \mathrm{sat}\big(u_{net},\, \pm R\big) \tag{72-10}$$

sat 是逐拍的硬限幅。这一项看起来只是安全措施，实际上是并联结构能否成立的前提：补偿网络在训练分布之外可能给出任意大的输出，若不加限幅，一次外推就能把执行器推到饱和，而饱和又会让误差变大、把网络推得更远。限幅把“网络犯错”的后果从“系统失稳”降级为“这一拍少补一点”。

零阶 TSK：规则表就是径向基网络

把式 (72-6) 的结论从常数换成“每条规则一个常数、规则数等于输入组合数”，就是零阶 TSK：

$$u = \sum_{i=1}^{M} \psi_i(x)\, \theta_i, \qquad \psi_i = \frac{\exp\!\big(-\|x - c_i\|^2 / 2\sigma^2\big)}{\sum_j \exp\!\big(-\|x - c_j\|^2 / 2\sigma^2\big)} \tag{72-11}$$

n 个输入、每维 3 个中心时 $M = 3^n$：2 输入 9 条规则 9 个参数，3 输入 27 条规则 27 个参数。前提参数（中心 c_i、宽度 σ）固定，结论参数 θ 由最小二乘一次解出，所以“训练”没有随机性、没有学习率、没有回合数——它退化成一次线性回归。这一点很重要：本章所有神经-模糊的结果都可以被读者用同一份数据复算，不存在“调了种子才好看”的空间。

代价也在这里。式 (72-11) 是一个**静态映射**，输入里没有积分状态，所以它天生没有消静差的能力；输入里没有参考相位，它就学不到前馈。72.1 节那条 2.8701 的失稳曲线，就是这两件事同时发生的后果。

分层：预测给目标，反馈保跟踪

分层结构里，上层解一个有限时域优化

$$\min_{U}\; J = \sum_{i=1}^{N_p} \big(\hat{y}_{k+i} - r_{k+i}\big)^2 + \lambda_u \sum_{j} u_j^2 + \lambda_d \sum_{j} \big(\Delta u_j\big)^2 \tag{72-12}$$

预测用“自由响应 + 脉冲矩阵”写成线性形式

$$\hat{\mathbf{y}} = \mathbf{y}_{free} + G\,\mathbf{U} \tag{72-13}$$

G 的第 (i, j) 元是第 j 个控制块对第 i 拍输出的贡献，块内指令在第 $i - (d-1)$ 拍生效（d 为延迟拍数）。下层不做优化，只按权限相加：

$$u = u_{PID} + \mathrm{sat}\big(u_{MPC},\, \pm R\big) \tag{72-14}$$

R 是本章最值得盯住的一个量。$R = 0$ 时式 (72-14) 退化成纯 PID；R 等于满量程时退化成纯 MPC。中间每一个取值都是一种“分工方案”，而 72.7 节会看到，R 从 0 加到 0.20 时误差从 0.1161 掉到 0.0310，从 0.20 再加到 0.40 时误差几乎不变（0.0295），总变差却从 2.462 涨到 11.556。权限不是越大越好，它有一个明显的拐点。

为什么用算子写，而不用框图

四种接法可以画成四张框图，但框图不是一种能计算的东西。本章选择先写成一个统一的算子形式，再让四种接法成为它的特例，原因有三个。

第一，框图看不出“谁是主人”。两张框图连线一样，可能一张是串级、一张是前馈，差别只在“内环的设定值从哪来”。写成算子之后，这个差别变成 Ψ 的一个分量取 u_1 还是取 r，是一行可以核对的东西。

第二，框图不能直接算复杂度。本章把复杂度定义成“算子树的深度”，是因为深度决定了串行环节的数量，而串行环节的数量直接对应单拍耗时；并联的两个分支可以并发，深度不增加。这个定义粗糙，但它可算，而且与实测的耗时排序一致：PID 0.014 ms、模糊调度 0.061 ms、分层 0.345 ms，正好对应深度一到三。

第三，框图无法承接“接口契约”。接口是数学对象——输入的物理量、量纲、有效区间、失效时的取值；把这些写进 Ψ 的每个分量，才能谈“两个模块能不能接”。本章 72.3 节那张五行的接口表，本质上是在给算子树的每条边标注定义域。

一句话：**框图用来讲给人听，算子用来讲给机器和测试听**。本章两种都画，但凡是需要推理的地方，都回到算子形式。

参数个数不等于自由度

把上面五组式子放在一起，能看出混合结构绕不开的三条约束。第一，**任何组合都不能引入新的不稳定极点**：串联靠有界调度系数保证，并联靠限幅保证，切换靠滞环与驻留保证，分层靠权限 R 保证。第二，**组合后的闭环带宽由最慢的一段决定**：上层 20 拍预测配下层 1 拍 PID，整体响应不会比 20 拍更快，这也是为什么分层结构的误差曲线比纯 MPC 粗糙。第三，**组合不创造信息**：如果某个模块的输入里没有相位，它就不可能输出正确的前馈，无论参数有多少。第三条是本章后面反复出现的判据。

在进入设计步骤之前，还要把“参数个数”与“自由度”分开。参数个数是控制器里可调的数的总数，自由度是“真正能改变闭环行为的独立方向数”。两者在单控制器里几乎相等——PID 三个参数对应三个方向；在混合结构里却差得很远，因为接口会引入强相关：调度系数乘在增益上，改动中心与改动范围对闭环的影响是同向的；TSK 的 27 个结论里，边缘格子的权重接近零，改它们几乎不影响输出。

本章大致是这样：对外可见约 40 个参数，其中真正被反复激励的，是调度表中心附近的几个格子、TSK 里被样本覆盖的十来个结论、以及 6 个接口参数。其余参数只在极端工况下才起作用——它们不增加稳态性能，只增加验证负担。这也是 72.10 节那条“每增加十个参数就多一个验证工况”的由来：**参数个数的代价主要不在计算，而在验证**。计算上 40 个参数与 3 个参数都是几次乘加，差别可以忽略；验证上，40 个参数需要覆盖的组合数比 3 个参数多几个数量级。

72.3 设计步骤

第零步：先判断该不该混合

在写任何代码之前，先做一次“症状归因”。同一个误差表现，可能只需要调一个参数，也可能必须换结构；如果归因错了，后面五步全是白做。下面这张表是本章建议的对照。

症状	最可能的原因	先试什么	需要混合吗
稳态有恒定偏差	积分增益不足或积分饱和	调 K_i、加抗饱和	不需要
换向时明显滞后	延迟造成的相位损失	前馈或预测	需要
大幅超调、小幅很好	增益随工况变化	增益调度	需要
不同工况表现差别大	单一增益无法兼顾	调度或切换	需要
误差与轨迹重复性强相关	存在可学习的前馈形状	迭代学习	需要
误差主要由随机扰动决定	反馈带宽不足	提高增益、加观测器	不需要

表里“不需要”的两行最容易被忽略。稳态偏差与随机扰动都是反馈本身能处理的事，加一个智能模块只会让问题从“参数没调好”变成“两个模块都没调好”。本章算例的基线 PID 在标称工况下饱和率为 0、误差 0.1161，属于“能用但不够好”，这时才轮到混合结构出场。

归因之后还要问三个问题：现有控制器离物理极限有多远（本章是 0.1161 对 0.0293，差四倍，有空间）；误差里有没有可预测的分量（有，参考完全已知）；团队能不能维护多出来的参数（这决定了参数预算）。三个问题里任何一个答“不能”，都应该停在第零步。

第一步：先决定“谁不做”

混合架构最常见的失败不是模块不好，而是两个模块抢同一件事。让模糊调度和神经网络都去补前馈，结果是一条通道被补两遍；让 PD 和 PID 都带积分，切换时两个积分器各自累积，一切换就打架。所以设计的第一步不是写代码，而是列一张“任务—模块”对照表，把每项工作**唯一**地指派给一个模块。

本章算例的任务可以切成四段：稳态消差、快速段抢时间、相位超前（前馈）、约束下的多步预测。切完之后指派关系就清楚了：稳态消差只能给带积分的模块（PID 或模糊-PID），抢时间给高增益 PD，前馈给能看到参考相位的模块，多步预测给 MPC。任何一个模块如果被指派了两件事，就要检查它有没有能力同时做好；如果没有，宁可拆开。

第二步：把接口写成契约

接口不是“函数签名”，而是四条必须写下来的约定：输入是什么、输出的量纲与限幅、更新周期、失效时的降级值。本章四个模块的接口如下。

模块	输入	输出	周期	失效降级
模糊调度	归一化 e、$\dot{e}$	三个系数，$[0.6, 1.3]$	1 拍	全部置 1.0
双模逻辑	归一化 $\lvert e\rvert$	模式标志	1 拍	保持 PID
前馈查表	归一化参考位置	补偿量，± 0.40	1 拍	输出 0
MPC 上层	状态、参考序列	预测增量，$\pm R$	1 拍	输出 0

“失效降级”这一列最容易被忽略，却决定了系统在异常时是“性能下降”还是“直接失控”。本章所有降级值都取“退化成纯 PID”——因为纯 PID 在本算例里饱和率为 0、误差 0.1161，是一个已知安全的兜底。工程上这条经验可以推广：**任何一个智能模块的失效默认值，都应当让系统退化成一个你愿意单独交付的控制器。**

第三步：选组合算子

四种算子里选哪一个，有两个判据。第一个是“要不要记忆”：调度与并联是无记忆的代数运算，切换与分层带状态。第二个是“输出要不要连续”：切换在原理上不连续，必须额外加滞环或偏置；其余三种天然连续。

具体到本章：如果只是想让 PID 在大小误差段表现不同，但又不希望出现模式跳变，用调度而不是切换——代价是调度只能连续地改变增益，做不出“大误差段完全冻结积分”这种硬逻辑。反之，如果确实需要“某段完全不用积分”，就只能用切换，并且必须接受滞环与驻留带来的参数。本章的双模结果说明这个代价是值得的：0.0814 对 0.1133，改善 28%。

第四步：权限从小往大加

凡是“两路相加”的结构，都有一路是主角、一路是配角，配角能拿多少权限必须由小到大试出来，而不是一次给满。本章把权限 R 从 0 扫到 1.20（满量程），六组结果如下。

权限 R	RMS	最大误差	总变差
0.00	0.1161	0.1974	1.708
0.05	0.0920	0.1632	1.658
0.10	0.0701	0.1292	2.074
0.20	0.0310	0.0639	2.462
0.40	0.0295	0.0655	11.556
1.20	0.0321	0.0787	17.462

从 0 到 0.20，误差降了 73%，总变差只涨了 44%，这一段是“净赚”；从 0.20 到 0.40，误差只降 4.8%，总变差却涨了 4.7 倍；再往上，误差反而回升。所以本章的经验规则是：**当权限再放大一倍、RMS 改善不足 5%、而总变差翻倍时，就停在上一档。**这条规则不需要知道对象模型，只需要两次实验，工程上很好执行。

第五步：参数从“能用的旧值”出发

混合结构里的新参数应当围绕一个已验证的旧值展开，而不是从零重搜。本章的做法是：

- 模糊调度的三个系数表以 1.0 为中心，上下界取 $[0.85,\ 1.30]$，保证最坏情况下闭环仍然接近原 PID；
- 双模的切换阈值取 0.05，约为纯 PID 最大误差 0.1974 的四分之一，含义是“进入最后四分之一误差带就切回 PID”；
- 分层结构的预测时域从 8 起步（延迟 5 拍加 3 拍动态），控制时域取 2 块；
- 前馈查表的补偿量限幅取 ± 0.40，约为执行器满量程的三分之一。

这四组数字都不是最优值，但都是“不会出事”的值。调参的顺序应当是先让系统在这组值上稳定运行，再逐个优化，而不是一上来就做多维搜索——本章 72.8 节会给出为什么。

第六步：三项验收

结构搭好之后，至少验三件事：切换或限幅处有没有跳变、饱和率是不是 0、单拍计算占周期多少。本章的数字分别是：直接切换跳变 0.1171（加偏置后为 0）、所有结构饱和率均为 0%、MPC 单拍 0.352 ms 占 10 ms 周期的 3.5%。三项里任何一项不达标，都说明结构还没到可以上机的地步——不是参数没调好，而是分工没分对。

设计检查表

把上面六步压成一张表，就是本章建议的检查顺序。它不保证得到最好的控制器，但能保证不出现“两个模块抢一件事”“权限给满导致抖动”“失效后无处可退”这三类最常见的事故。

步骤	要回答的问题	本章取值
分工	每项工作是否只有一个模块负责	四段任务四个模块
契约	输入、量纲、周期、降级是否写清	见 72.3 节接口表
算子	是否需要记忆、输出是否连续	调度/并联/切换/分层
权限	配角最大能拿多少	0.20（拐点）
初值	是否围绕已验证的旧值	系数 1.0、阈值 0.05
验收	跳变、饱和、耗时	0 / 0% / 3.5%

72.4 连续情形

连续域里先看频段

把回路画成频域，混合结构要回答的问题就变得很直观：低频段谁负责、中频段谁负责、高频段谁负责。本章算例的三个频段很清楚。参考轨迹主频 0.25 Hz（周期 4 s），三次谐波 0.75 Hz；对象是一个带宽约 2 rad/s 的二阶惯性；执行到测量的纯延迟 50 ms。

延迟在频域里是纯相位滞后 $-\omega T_d$。把三个频点代进去：主频 $\omega = 1.571$ rad/s 对应滞后 0.0785 rad，即 4.5°；三次谐波 $\omega = 4.712$ 对应 0.2356 rad，即 13.5°。这两个数字说明，本章的延迟**不是靠牺牲带宽换来的相位裕度去硬扛**，而是靠“提前知道参考要去哪”去补。这一点决定了四种结构的天然分工：反馈管不了 13.5° 的滞后，前馈与预测可以。

串联：增益调度是一个慢变线性系统

模糊调度把三个常数换成三个有界函数，闭环就成了线性变参数系统。它能不能用线性工具分析，取决于一个准静态假设：**调度参数的变化速度远慢于闭环自身的动态**。本章的调度输入是误差与误差导数，误差导数经过 30 ms 低通，闭环主时间常数约 0.2 s，两者相差近一个数量级，假设基本成立；如果调度直接吃测量噪声，这个假设立刻失效，闭环会出现“参数随噪声跳”的怪现象。

图 72-2 是三个调度系数在归一化 $(e, \dot{e})$ 平面上的取值。三张图的形状不一样，正好说明三张规则表各自在管一件事：K_p 曲面在“误差大且同向”的角上最高（1.30），在“误差大但反向”的角上最低（0.850）；K_i 曲面方向相反，误差大时压到 0.600；K_d 曲面最平，只在同向大误差时降到 0.700。

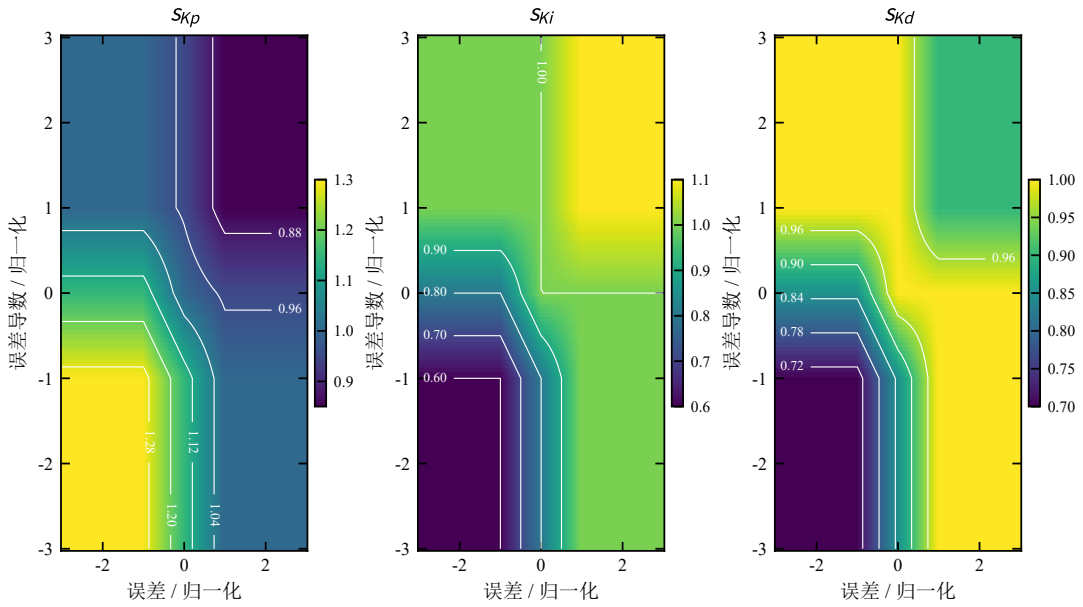


图 72-2 三个调度系数的曲面

四个代表点的数值可以拿来核对曲面是否被正确实现：

工作点	s_{Kp}	s_{Ki}	s_{Kd}
e 大、$\dot e$ 同向	0.850	1.100	0.900
e 大、$\dot e$ 反向	1.000	1.000	1.000
e 小、$\dot e$ 小	0.990	1.001	0.999
e 大、$\dot e$ 为零	0.950	1.000	1.000

第一行值得多想一秒：误差大且还在扩大，直觉上应该加大 K_p 猛拉，这张表却把 K_p 压到 0.850、把 K_i 抬到 1.100。原因是本算例的执行器限幅只有 ± 1.2，误差大时真正稀缺的不是增益而是“别把执行器顶死”；抬积分是为了在顶死期间仍然积累恢复力。这张表是人按经验写的，它体现的是限幅约束下的取舍，而不是“误差大就加大增益”的教科书口号。换一个不限幅的对象，这张表的方向就要改。

连续域里还能读出一件事：调度范围只有 $[0.85,\ 1.30]$，最坏情况下闭环增益变化不到 1.5 倍，与对象自身 $K(y)$ 从 13 变到 20 的 1.5 倍同量级。换句话说，模糊调度并没有“抵消”非线性，它只是把增益变化的方向调得对了一点。这也解释了 72.1 节那个 2.4% 的改善幅度——**调度能做的只是“让固定增益在全程都不太差”，它不可能让固定增益变成自适应增益**。

切换：连续域里的抖振从哪来

理想的双模切换在连续域里是一个继电器：$|e|$ 穿过阈值就换控制律，切换频率理论上可以无穷高。实际系统里限制它的是两件事——执行器的带宽与采样周期。本章加了两个装置：滞环半宽 h 与最短驻留。滞环把“穿越一次就切”改成“必须穿过 $2h$ 才切”，于是最小切换间隔约为 $2h/|\dot e|$；取 $h=0.005$、$|\dot e|\approx 0.5$，得到 20 ms，也就是 2 拍。驻留期则直接规定“切换后至少保持 n_d 拍”。

实测结果支持这个估算：4 s 内切换次数从无滞环时的 7 次降到加 20 拍驻留后的 5 次，RMS 从 0.0819 微降到 0.0814，总变差从 2.315 降到 2.281。滞环从 0 加到 0.010 时误差反而略升（0.0819→0.0827），因为滞环让“该切的时候晚切了一点”。这条曲线很平，说明在本算例里切换本身并不频繁，抖振不是主要矛盾；真正决定精度的是高增益 PD 段的 K_p 取多大，72.8 节会给出网格。

并联：前馈是唯一“不靠反馈”的通道

连续域里，理想前馈可以直接写出来。对式 (72-1) 求导两次并代入参考轨迹，得

$$u^{*}(t) = \frac{r''(t) + 2r'(t)}{K\big(r(t)\big)} \tag{72-15}$$

量级估一下：r' 的幅值是 $A_1\omega = 0.864$，r'' 的幅值是 $A_1\omega^2 = 1.357$，两者相加除以 $K \approx 16$，得到 u^{*} 的幅值约 0.13，只占执行器满量程的 11%。这就是前馈的尴尬之处：它能消掉的误差有限，但它消的是“反馈永远追不上”的那部分——相位滞后。本章把 u^{*} 单独加在开环对象上，跟踪 RMS 是 0.0293；而纯反馈 PID 是 0.1161。前馈一个人干的活，比 PID 好四倍。

但也别急着下结论。0.0293 是在“模型完全已知、没有任何扰动”的前提下拿到的；一旦 $K(y)$ 估错 20%，前馈就会把误差按比例放大，而反馈不会。所以并联结构的正确用法不是“用前馈替代反馈”，而是“用前馈消掉可预测的部分，把反馈留给不可预测的部分”。本章的并联结果 0.1057 之所以只比 PID 好 9%，就是因为学出来的前馈只复现了 u^{*} 的 79%，剩下 21% 的拟合误差又通过反馈绕了回来。

前馈增益为什么不能取满

式 (72-15) 给的是精确前馈，理论上应该 100% 加进去，实际工程里几乎没人这么做，通常只加 70%–90%。原因不在保守，而在误差的传播方式。

设实际模型增益是 $\hat{K}$，真实增益是 K，前馈按 $\hat{K}$ 算，则执行器上多出的一项正比于 $\big(K/\hat{K} - 1\big)$。这一项不进反馈回路，没有任何机制能消掉它，它直接叠加在输出上。若前馈取满、模型又高估了 20%，这一项的幅值约等于 $0.2 \times 0.13 = 0.026$，看起来不大；但它与参考同频，会在每一拍的同一相位重复出现，等价于一个持续的周期性扰动，积分器只能把它平均掉一部分。

取 80% 的好处是把这一项压到 0.021，同时只损失 20% 的相位补偿。更重要的好处是**留出了一条可观测的残差**：前馈没补完的部分会体现在误差里，反馈据此还能判断模型偏了多少；如果前馈取满，误差被压到接近零，反馈就失去了“看见模型错误”的能力，这在工程上叫“把诊断信号调没了”。

本章的并联结构在这一点上有个直接的证据：前馈只覆盖了 79% 的 u^{*}，于是闭环从纯 PID 的 0.1161 降到 0.1057；而开环单独加满前馈是 0.0293。两者的差距说明，闭环里的前馈并没有按开环那样工作——它被反馈“抵消”了一部分，因为反馈看到误差变小就少出力，一增一减之间，前馈的有效贡献被摊薄。这不是设计缺陷，而是所有并联结构的共性：**并联两条通道的贡献不能简单相加，它们通过误差互相耦合**。

这条耦合还解释了为什么并联结构的参数最难调。串联与切换的模块是“接力”关系，一个模块的输出直接成为另一个的输入，因果关系是单向的；并联是“合力”关系，两条通道同时看误差，任一条变动都会改变另一条的工作点。调并联结构的正确顺序是先把反馈那条定死，再单调前馈，最后联合微调——反过来做，每次改前馈都会让已经调好的反馈失准。

两个积分器会打架

“谁不做”这条原则里最重要的一条，是**积分只能有一个主人**。积分的作用是记住历史误差，两个模块各记一份，等于把积分增益乘了二；更糟的是切换时两份积分各自累积，切过去的一瞬间两份叠加，输出会突然多出一块。

本章的双模结构刻意让 PD 半边完全没有积分：高增益段只负责抢时间，消静差仍然交给切回来的 PID。这样全回路只有一个积分器，切换时不需要迁移任何状态，只需要决定“这一拍用哪一路”。代价是高增益段在稳态附近不能单独工作，所以阈值必须让它在误差还大时就退出——这也是阈值取 0.05 而不是 0.01 的原因之一。

分层结构里这件事更隐蔽。PID 有显式积分，而 MPC 的增量惩罚 $\lambda_d \sum(\Delta u)^2$ 实际上提供了一种“软积分”：它惩罚控制量的变化，等价于让控制量缓慢逼近某个稳态值。两种积分叠加，等效积分增益会随权限 R 增大而上升。72.6 节看到 R 超过 0.20 后总变差暴涨、误差不降，除了高频锯齿之外，这条叠加也是一个可能的原因。工程上的处理办法有两条：要么把下层的积分在 R 大时减弱，要么把上层的 λ_d 调大以削弱它的积分特性。两条都指向同一个结论——**权限 R 不只是“分多少活”，它还改变了闭环的积分结构**。

并联结构反而最容易处理：补偿网络如果只依赖参考位置（本章的 0.1057 那一档），它天然没有积分记忆，与 PID 的积分不冲突；一旦把误差也喂进去，网络就带上了“隐式记忆”，冲突立刻出现——这也是那一档总变差 5.208 的另一半解释。

分层：连续域里它是“带终端代价的最优控制”

把式 (72-12) 的时域拉长到无穷，它就退化成 LQR；时域有限时，它比 LQR 多了一个“看得见未来参考”的能力。连续域的直觉是：MPC 用一个内部模型把未来 N_p 拍的响应算出来，再把“哪一段该出力”分配给当前这一拍。它比 PID 强的地方不在反馈增益，而在**它知道 5 拍之后指令才生效**——这正是纯反馈做不到的。

一张频段分工表

把上面的分析整理成表，就是本章对“谁负责哪一段”的完整回答。

频段	频率	主要成分	谁负责	本章证据
极低频	< 0.1 Hz	稳态偏置	积分	PID 已能做到零静差
基频	0.25 Hz	主往复运动	前馈或预测	纯前馈开环 0.0293
三次谐波	0.75 Hz	换向细节	只能靠前馈	反馈在此处增益不足
高频	> 5 Hz	测量噪声	滤波，不再出力	1 mV 噪声下 0.1162

第二、三行的差别值得展开。对象带宽约 2 rad/s，也就是 0.32 Hz；三次谐波在 4.71 rad/s，按二阶惯性估算，幅值衰减约 $(0.32/0.75)^2 \approx 0.18$，即 -15 dB。反馈要在这个频率上产生足够的修正，就得把环路增益提高 5.5 倍；而提高增益的同时，延迟带来的 13.5° 与对象自身的相位滞后会一起吃掉落相位裕度。所以反馈在三次谐波上不是“不想管”，是“管不动”。

前馈则不受这个限制。它把 u^* 直接加到执行器，不经过反馈回路，也就不受相位裕度约束；它的精度只取决于模型对不对。这就是 72.1 节那张表里“并联前馈”的最大误差（0.1559）比纯 PID（0.1974）低、而 RMS 只低 9% 的原因：它在高次谐波上确实起了作用（最大误差降了 21%），但主频段的误差本来就由积分与增益决定，前馈帮不上太多，两者一平均就只有 9%。

这张表还有一个前提必须说清：它假设参考是**已知且可预测**的。若参考变成随机信号，“基频/谐波”的划分就失去意义，前馈也无从计算，此时频段表塌缩成“低频靠积分、高频靠滤波”两行，混合结构的收益也随之消失。这也是 72.11 节那条“先给误差归因”的由来——频段表本身就是一种归因。

连续域给出的三条判据

第一，延迟造成的相位滞后随频率线性增长，主频 4.5°、三次谐波 13.5°，所以补偿要针对谐波而不是基频。第二，任何调度或切换的快慢都必须与闭环时间常数拉开差距，否则准静态假设失效。第三，前馈通道的量级通常只占满量程的百分之十几，它的价值在相位而不在幅值，因此不能用“前馈加了多少”来衡量它的贡献，要看“误差的哪一部分消失了”。

72.5 离散情形

离散化改的不是精度，是结构

连续域的三条判据到了离散域都要重算一遍，而且不是“加个 T_s 修正”那种重算。采样做的是三件结构性的事：把连续时间切成 10 ms 一格，于是所有“提前量”只能是整数拍；把 50 ms 延迟折成 5 拍，于是相位滞后在闭环里以整拍形式出现；把控制时域切成块，于是“未来能改几次数”变成一个显式的整数。这三件事都不能靠更细的采样消除——采样再细，延迟对应的拍数只会变大，不会消失。

本章取 $T_s = 10$ ms，与第 66–70 章一致。对象最快的动态是式 (72-1) 里那个 $-2y'$，时间常数 0.5 s，10 ms 采样相当于每 50 个点描一条 0.5 s 的曲线，够用；而 5 拍延迟给出一个粗略的带宽上限：闭环带宽大约不超过 $1/(10\,d\,T_s)$，即 2 Hz，恰好覆盖参考的三次谐波 0.75 Hz。这条经验式不严格，但它能在选采样周期时快速判断“这套结构有没有可能跟上参考”。

状态方程与延迟的整拍化

取 $x=[y,\ y']^{\mathsf T}$，对式 (72-1) 做前向欧拉离散，并把延迟显式写成 d 拍：

$$x_{k+1}=\Phi x_k+\Gamma(y_k)\,u_{k-d},\qquad \Phi=\begin{bmatrix}1 & T_s\\ 0 & 1-2T_s\end{bmatrix},\quad \Gamma(y)=\begin{bmatrix}0\\ T_sK(y)\end{bmatrix} \tag{72-16}$$

$T_s=0.01$ 时 Φ 的非对角元只有 0.01，$1-2T_s=0.98$，离散是良态的；欧拉解与精确解的相对差约 $(2T_s)^2/2\approx 2\times10^{-4}$，远在本章关心的误差量级（$10^{-2}$）之下。换一个刚性对象（例如电流环，时间常数 0.1 ms）就不能这么做，必须用零阶保持的精确离散。这里特意点出来，是因为混合架构里最常见的实现错误之一，就是上层优化器的模型周期与下层实际采样周期写成两个值，结果预测永远差半拍，而波形上看不出毛病，只觉得“上层好像没起作用”。

还要注意 Γ 依赖 y_k：对象是非线性的，方程却是“每一步线性”的。这正是混合架构存在的前提——对象若本来线性，一个 PID 就够了；正因为它每步线性、整体非线性，才有“用模型算下一步、用反馈兜住长期偏差”这种分工。

脉冲矩阵：把“未来”写成一次矩阵乘

上层要算未来 N_p 拍的输出，最直接的做法是把式 (72-16) 迭代 N_p 次。工程上更常用的是写成一次矩阵乘法，即式 (72-13) 里的 G：

$$G_{ij}=\sum_{m=n_j}^{\min(n_{j+1}-1,\ i-d)} C\Phi^{\,i-d-m}\Gamma,\qquad C=[1\ \ 0] \tag{72-17}$$

n_j 是第 j 个控制块的起始拍，块内各拍取同一个指令。求和上限里的 $i-d$ 含义很直白：第 i 拍的输出，最早只能被第 $i-d$ 拍及之前的控制量影响。$i-d<m$ 的项根本不存在，于是 G 的左下角是一个由延迟撑起来的零三角，宽度就是延迟拍数，形状一眼能看出来。

G 必须自检。做法只有三行：随便给一组 $\mathbf{U}$，一边算 $G\mathbf{U}$，一边用式 (72-16) 逐步迭代，两者相减。本章的自检偏差是 2.22×10^{-16}，即双精度浮点的舍入极限。这个检查看起来多余，但 G 写错时程序不会报错，只会让控制器“看不见自己的动作”，症状是控制量缓慢漂移、误差长期偏大，很难从波形反推到矩阵。三行代码换掉一类难查的故障，是划算的。

预测时域必须越过延迟

N_p 是混合架构里第一个必须显式选定的整数。图 72-6 是同一控制器（$N_u=2$、$\lambda_u=\lambda_d=0.02$）在五档预测时域下的闭环结果，纵轴取对数。

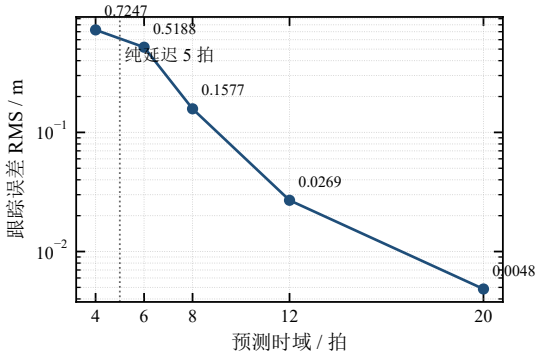


图 72-6 预测时域与跟踪误差

N_p	4	6	8	12	20
RMS	0.7247	0.5188	0.1577	0.0269	0.0048
最大误差	1.2200	0.8102	0.3040	0.0771	0.0127
总变差	0.000	0.309	1.009	2.034	1.569

第三行最后那个 0.000 是本章最干净的一条证据：$N_p = 4$ 小于延迟 5 拍，G 的每一列都落在零三角里，优化器无论怎么改 $\mathbf{U}$ 都算不出预测输出的变化，梯度恒为零，控制量一动不动，输出停在初值上——最大误差 1.2200 恰好顶到限幅 ± 1.2，RMS 0.7247 就是“什么都不做”的误差。这不是参数没调好，是信息在结构上根本不存在。

$N_p = 6$ 只比延迟多一拍，G 每列只有一个非零元，于是能动，但只能动一次，总变差 0.309；$N_p = 8$ 越过延迟 3 拍，误差掉到 0.1577；到 $N_p = 12$ 是 0.0269，比纯 PID 好四倍多。曲线在 $N_p \approx d + 4$ 之后明显变平，这与“预测至少要覆盖一个建立过程”的经验吻合：5 拍延迟加 3 拍建立起落，正好是 8。

单拍耗时随 N_p 增长很慢，五档都在 0.30–0.38 ms 之间。原因是求解器用投影梯度，每步只做一次 G^{T} 乘，计算量随时域线性增长；10 ms 周期里 0.38 ms 只占 3.8%，乘一个安全系数也放得下。这一条决定了本章后面敢把 N_p 取到 20。

延迟拍数估错了会怎样

d 是唯一一个“必须量、不能调”的结构参数。本章它等于 5，与真实值一致；若模型写 5 而实际是 8，72.6 节那张鲁棒性表给出了代价：MPC($N_p = 20$) 从 0.0048 退化到 0.0538，十一倍；MPC(12) 加学习更糟，从 0.0644 退化到 0.1711。

把 11 倍摊到 3 拍误差上，每错一拍大约让误差乘 2.2。这个倍数不是普适常数，但它说明了一件事：**延迟误差的影响是指数的，不是线性的**。原因在于脉冲矩阵的零三角宽度直接由 d 决定，d 错了，整个 G 的形状就错了，而不只是“晚一点”。

错的方向也有讲究。模型延迟偏小，控制器以为指令会更早生效，于是补偿不足且永远慢半拍；偏大则相反，控制器以为要等更久，于是提前加力，表现为超调。两种错法都坏，但第一种更常见——因为工程上量延迟时容易漏掉执行器自身的响应时间。稳妥的做法是取“偏大”的一侧：宁可多等一拍，也不要少等一拍。这一条与增益整定的直觉相反，值得单独记住。

分块：用自由度换抗噪

控制时域 N_u 是第二个整数。块边界按

$$n_j = \left\lfloor \frac{(j-1)N_p}{N_u} \right\rfloor, \qquad j = 1, \dots, N_u + 1 \tag{72-18}$$

均匀划分，块内指令保持不变。$N_u = N_p$ 时每拍都能改，自由度最大；$N_u = 1$ 时整个时域只有一个数，退化成“算一个常数前馈”。本章取 $N_u = 2$，理由是：延迟 5 拍意味着前 5 拍的指令无论如何追不上，把它们拆开没有意义；真正需要区分的是“延迟段过去之后立刻该出多大力”与“稳态该出多大力”两件事，两块正好对应。

分块还有个副作用值得说：它相当于隐式正则化。自由度少了，代价函数里 λ_d 的作用被削弱，参数之间不容易打架。反过来，若坚持 $N_u = N_p$，就必须把 λ_d 调大，否则逐拍的解会在相邻拍之间来回跳，总变差飙升。本章 $N_p = 20$、$N_u = 2$ 时总变差 1.569，反而比 $N_p = 12$ 的 2.034 低，说明分块确实压住了抖动。

微分与调度的离散实现

模糊调度每拍要算一次 $\dot{e}$，离散域里只能用后向差分

$$\dot{e}_k = \frac{e_k - e_{k-1}}{T_s} \tag{72-19}$$

$T_s = 0.01$ 意味着 $1/T_s = 100$，测量噪声被放大一百倍。本章仿真里加了 1 mV 噪声，直接差分会得到 0.1 量级的 $\dot{e}$ 抖动，足以让调度系数每拍乱跳。所以 $\dot{e}$ 进规则表之前先过时间常数 $\tau = 30$ ms 的一阶低通：

$$\dot{e}^{f}_k = \dot{e}^{f}_{k-1} + \frac{T_s}{\tau + T_s}\left(\dot{e}_k - \dot{e}^{f}_{k-1}\right) \tag{72-20}$$

30 ms 不是随手取的：它必须明显小于延迟 50 ms，否则滤波本身变成新的滞后环节，72.4 节的准静态假设就破了；又必须明显大于采样周期 10 ms，否则滤不掉噪声。$\tau \in [2T_s,\ dT_s/2]$ 是一个可以照抄的区间。

计算量上，离散实现把连续域里“一张曲面”变成每拍九次乘加的查表：模糊调度 0.061 ms/拍、9 条规则的 TSK 0.037 ms/拍、27 条 0.050 ms/拍，都在 10 ms 周期的 1% 以内。这也是本章敢在嵌入式目标上同时开调度与前馈的原因——两个加起来还不到 0.1 ms，而同样周期下 MPC 要 0.35 ms 左右。

采样周期怎么定

T_s 在混合结构里是第一个要定的数，因为延迟拍数、预测时域、微分滤波全都由它派生。定它有三个约束，两边夹出一个区间。

下界来自计算。本章最重的模块是 MPC，单拍 0.345 ms；留三倍余量，周期不能短于 1 ms。这条下界在纯 PID 系统里几乎不存在（0.014 ms），但一旦引入预测层，它就变成硬约束——超时的后果不是变慢，是丢拍。

上界来自两件事。第一，采样必须明显快于对象最快的动态：本章时间常数 0.5 s，按每时间常数至少十个采样点，$T_s \le 50$ ms。第二，采样必须明显快于延迟：延迟 50 ms，若 $T_s = 10$ ms 则延迟 5 拍，量化误差半拍即 10%；若 $T_s = 50$ ms，延迟只有 1 拍，量化误差 50%，预测层完全失效。两条合起来给出 $T_s \le 10$ ms。

本章取 10 ms，正好压在上界上。这不是巧合，而是权衡的结果：再往下取 5 ms，延迟变 10 拍、量化误差降到 5%，但 MPC 的计算占比从 3.45% 涨到 6.9%，而误差几乎不变——因为延迟本身是物理的，采样再细也不能让它变小，只能让“对它的估计”更准一点。本章的模型是精确的，不需要更准的估计，所以 10 ms 是最省算力的选择；若延迟未知或时变，把 T_s 减半就是值得的。

三个数怎么分工：时域、块数、惩罚权

离散 MPC 的参数里，N_p、N_u、λ_u、λ_d 经常被混为一谈，其实它们各管一件事，混着调就会互相抵消。

N_p 管“看得多远”，它决定控制器能不能预见延迟与换向。这是唯一一个“改一个整数换一个数量级”的参数：4 到 20，RMS 从 0.7247 到 0.0048。它的下界由延迟定（必须越过 d），上界由模型可信度定——预测得越远，模型误差的累积越多，所以 N_p 不是越大越好，而是“刚好覆盖到参考的下一个转折点”最好。

N_u 管“能改几次”，它决定控制量的自由度。N_u 小等于强迫控制量在一段时间内保持恒定，这带来抗噪（控制量不会追着噪声抖），但也带来钝感。本章取 $N_u = 2$，把 20 拍切成两大块，是为了配合“延迟 5 拍 + 参考半周期 200 拍”这个尺度：块太细等于没有约束，块太粗等于只做一次阶跃。

λ_u 惩罚控制量的绝对值，λ_d 惩罚它的变化量。前者相当于在目标里加了一个弹簧把 u 拉向零，容易引入稳态偏差；后者相当于低通，抑制锯齿而基本不影响稳态。所以**需要平滑时调 λ_d，需要省能量时才调 λ_u**，两者的作用方向经常被写反。本章 $\lambda_u = 0.02$、$\lambda_d = 0.05$，前者很小、后者稍大，正是“允许出力、不允许抖”的意思。

四个参数的调试顺序因此是固定的：先按延迟定 N_p 的下界，再按参考节奏定 N_u，最后用 λ_d 压锯齿、用 λ_u 压幅值。反过来先调权重、再改时域，前面调好的权重会全部失效——因为权重的“合适值”与预测长度是耦合的：时域翻倍，误差项的累加也翻倍，同一个 λ_d 就相对变弱了。

离散域的三条判据

第一，$N_p \ge d + 3$。低于这个值，脉冲矩阵的零三角会吃掉大部分梯度，控制器“看不见”自己的动作，性能不是变差而是直接归零。第二，$N_u \le N_p/4$。自由度超过这个比例，逐拍解开始互跳，总变差随自由度上涨，而误差几乎不动。第三，所有微分先滤波，滤波时间常数落在 $[2T_s,\ dT_s/2]$。

这三条都关于整数——采样周期、延迟拍数、预测时域、分块数——而不是关于增益。它们错了，参数怎么调都补不回来。下一节就来看这种“结构错了调不回来”在稳定性上是什么样子。

72.6 稳定性与性能

没有“结构自带”的稳定

五种拓扑摆在一起，最容易产生的错觉是“分层最复杂所以最稳、切换最粗暴所以最抖”。本算例的数字把这两个直觉都推翻了：分层拿到 0.0310，可它的控制量总变差 2.462 是五种结构里最高的；双模的总变差 2.281 也不低，但 RMS 0.0814 仅次于分层。也就是说，**精度与平稳不是同一个指标，稳定更不是**。一个结构可以在误差很小的同时把执行器磨得很厉害，也可以在误差很大时输出得非常平滑——$N_p = 4$ 那个 0.000 就是最极端的例子。

更该记住的是那个反例：2 输入、9 条规则的 TSK 直接替代 PID，闭环 RMS 2.8701、最大误差 3.9078、30.2% 的拍数贴着限幅，而总变差只有 2.492。这条曲线的形状是“顶在限幅上慢慢爬”，既不稳定也不抖——只看总变差，它甚至比双模还“平稳”。所以判断一个混合结构能不能用，至少要把四个数一起看：RMS、最大误差、总变差、饱和率。少看一个都会误判。

串联：稳定性靠调度系数的界

模糊-PID 能保持稳定，靠的不是“模糊系统更聪明”，而是一个很朴素的事实：三个调度系数被限制在有限区间内，闭环在任意时刻都等价于一个增益有界的线性 PID。本章的界取 $[0.85, 1.30]$，最坏情况下闭环增益比标称高 30%，而 PID 在这个增益上仍然稳定，于是整条轨迹上都不会出现不稳定极点。这就是串联结构风险最低的原因——**它把“时变”限制在“参数有界”这个最弱的条件下**。

但“有界”只保证不失稳，不保证有收益。图 72-3 上半幅是闭环里三个系数实际走过的轨迹，下半幅是模糊-PID 与纯 PID 的误差对比。

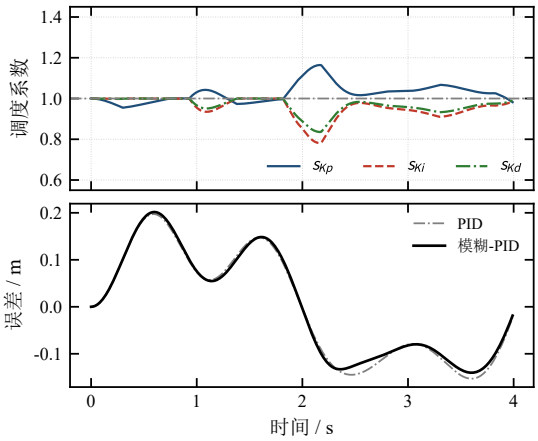


图 72-3 闭环中实际走过的调度系数

上半幅暴露出一个设计时想不到的事实：规则表写出来的曲面范围是 $K_p \in [0.85, 1.30]$、$K_i \in [0.60, 1.10]$、$K_d \in [0.70, 1.00]$，而闭环真正走到的只有 $K_p \in [0.955, 1.164]$、$K_i \in [0.781, 1.000]$、$K_d \in [0.836, 1.000]$。九个格子有大半从未被踩到，K_p 的上界 1.30 一次都没出现过。

原因并不复杂：那张表是按“误差大”设计的，而闭环一旦稳定，误差大部分时间都在小范围里，正好落在表格中心附近，系数接近 1.0。设计者以为自己在调一条大幅摆动的增益曲线，实际调出来的是一条几乎不动的线。这解释了 72.1 节那个 2.4%：不是规则写错了，而是规则的有效区域没有被访问。要让调度真正起作用，规则表必须按“闭环实际会走到的区域”来写，而不是按“误差可能有多大”来写——这两件事在稳定系统里差得很远。

下半幅的误差曲线也印证了这一点：两条曲线几乎重合，模糊-PID 只在换向的少数几拍略低一点。0.1133 对 0.1161，改善 2.4%，与“系数只在 1.0 附近摆动百分之十几”这个量级一致。**调度的收益大致等于“系数偏离 1 的幅度”乘以“该区域停留的时间占比”**，两个因子都小，乘积自然小。

并联：限幅是唯一的稳定保证

并联结构没有串联那样的“参数有界”论证可用，因为补偿通道的输出直接进执行器。本章的做法是给它加硬限幅 ± 0.40，于是补偿量的最坏影响被钉死：即使补偿网络完全错，闭环也只是“PID 加一个有界扰动”。这个扰动的幅度是执行器满量程的三分之一，足以把误差从 0.1161 抬到 0.19 上下，但不至于失稳。

实测正好落在这一带：并联残差、2 输入时 RMS 0.1913，比纯 PID 差 65%，总变差 5.208 是 PID 的三倍；换 3 输入降到 0.1259，总变差 3.619；换成只依赖参考位置的一维查表，0.1057、总变差 1.695。三个数对应三种“补偿网络错得有多离谱”：第一种把噪声当成信号学进去了，第二种学对了一半，第三种几乎全对。

这里有一条与直觉相反的结论：**并联结构的稳定性由限幅决定，而不是由学习算法决定**。这也是工程上更愿意用“PID + 限幅前馈”这种看起来笨的组合的原因——限幅让稳定性变成可以手算的，学习算法只影响限幅之内那部分性能。

三种工具，三个对手

混合结构的稳定性之所以难讲，是因为四个模块对应四种“不稳定的来源”，而分析工具也正好有三套。用错工具，结论会看起来很有道理却完全不适用。

第一套是小增益定理，它的对手是**参数摄动**。把串联结构写成“标称闭环 + 摄动”：$s_K = 1 + \delta$，本章 δ 的范围是 $[-0.15,\ 0.30]$（对应系数 $[0.85, 1.30]$），其余两个系数的摄动也都小于 0.4。若标称闭环在增益摄动 40% 时仍然稳定，那么无论调度怎么动，闭环都稳定。这条判据给出一条很实用的设计规则：**调度范围的上界由标称闭环的稳定余量决定，而不是由“想调多宽”决定**。本章上界取 1.30 而不是 2.0，正是因为再宽就没有余量可用了。

第二套是描述函数，它的对手是**切换非线性**。把双模结构看成一个带死区与滞环的继电器，描述函数 $N(A)$ 的幅值随输入幅值 A 变化；极限环存在的条件是 $1 + N(A)G(j\omega) = 0$ 有解。本章没有真去解这个方程，而是用两件事让条件不成立：滞环让 $N(A)$ 带一个负相位，驻留让等效的 G 在切换频率附近增益下降。定性地说，**滞环的作用是在描述函数里加相位滞后，相位越滞后越不容易起振**——这也是为什么本章无滞环那一档切换了 7 次，而带滞环加驻留的只有 5 次。

第三套是延迟裕度，它的对手是**纯延迟**。延迟在频域是 $e^{-j\omega dT_s}$，它只吃相位不给幅值，所以判据很直接：设计带宽处的相位裕度必须大于 $\omega d T_s$。本章 0.75 Hz 时这个数是 13.5°，0.25 Hz 时是 4.5°。想把带宽做到 1.5 Hz，就要先准备 27° 的余量——这基本等于把 PID 的增益砍一半，得不偿失。所以本章的选择不是“提带宽硬扛延迟”，而是“用前馈绕开延迟”。

三套工具的分工不能混。用描述函数去分析调度是错的，因为调度是线性时变系统，没有“非线性环节”可描述；用小增益去分析切换也不对，因为切换的等效增益随幅值变化，不满足小增益定理“摄动有界且不依赖信号”的前提。至于分层结构，它两样都沾一点（权限是饱和非线性，模型是参数摄动），所以最稳妥的做法是两条都查一遍，取更保守的结论。

切换：极限环与模式阶梯

72.4 节估过抖振周期 $2h/|\dot{e}| \approx 20$ ms，那是“最短切换间隔”，不是“实际会不会振”。要判断会不会形成极限环，得看两个条件是否同时成立：切换面附近的等效增益是否足以维持振荡，以及驻留时间是否短于对象的一个建立过程。本章两条都做了处理——滞环半宽 0.005、驻留 20 拍（0.2 s）。后者比对象时间常数 0.5 s 短，但比抖振周期长十倍，足以把继电器式的快速振打散。

图 72-4 把一个完整回合拆成三幅：误差与阈值、模式标志、控制量。

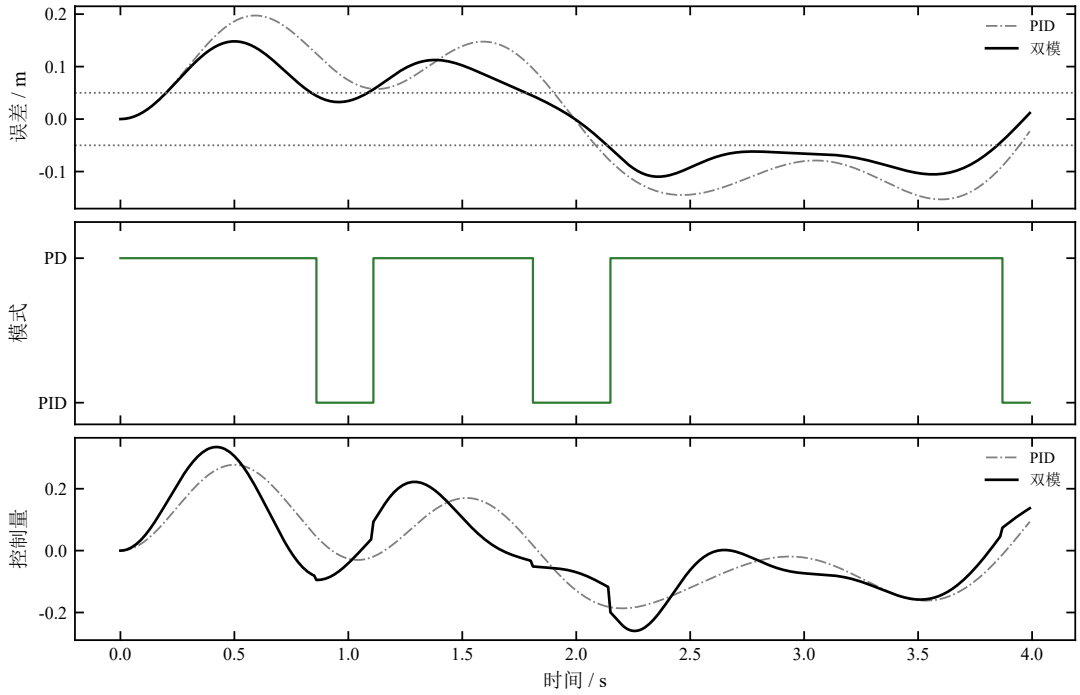


图 72-4 双模切换的误差、模式与控制量

中间那幅阶梯是本结构最该被观察的信号。整个 4 s 里模式只翻了 5 次，每次都发生在误差穿过 ± 0.05 带的时候，没有出现“在阈值附近来回抖”的情况。这说明滞环起作用了：误差要反向穿过 $2h = 0.01$ 才会翻回去，而 0.01 对应的穿越时间远大于一拍。

代价在第三幅：控制量在每次模式翻转处都有一个折角，总变差 2.281，比 PID 高 34%。这个折角不是缺陷，是“两种控制律本来就不连续”的必然结果。工程上要么接受它，要么加偏置做无扰切换——72.8 节会看到，加了偏置以后跳变确实归零（0.1171→0.0000），但切换后的最大误差反而变大（0.1674→0.1875），因为偏置把旧控制器的积分冻结成一个常数，等于人为取消了一段修正能力。

还有一条隐含的稳定性条件容易被忽略：**切换逻辑的输入必须比切换动作慢**。本章用 $|e|$ 做判据，误差是连续量，没问题；若改用 $\dot{e}$ 或某个高频特征，切换频率就会跟着噪声走。这与“调度要慢”是同一条道理的两种表现：所有“离散决策”都必须由慢变量驱动。

分层：权限就是稳定裕度

式 (72-14) 里的 R 同时扮演两个角色：它限制上层的权限，也就决定了闭环的稳定裕度。R 小时，上层只是给 PID 加一个小修正，闭环几乎等于纯 PID；R 大时，上层实质上接管了控制量，闭环特性由 MPC 决定，下层退化成扰动观测器。

从稳定性看，R 的增大不是单调安全的。$R = 0.20$ 时 RMS 0.0310、总变差 2.462；$R = 0.40$ 时 RMS 0.0295（几乎不动），总变差跳到 11.556；$R = 1.20$ 时 RMS 回升到 0.0321、总变差 17.462。总变差涨 4.7 倍而误差不降，说明闭环里出现了高频往复——不是发散，但已经进入“执行器被磨、精度没换来”的区域。频域上的解释是：MPC 的输出里含有模型误差带来的高频分量，R 小时它被限幅削掉，R 大时被完整送进执行器，与 PID 的低频输出叠成锯齿。

所以 R 的正确读法不是“权限越大上层越有用”，而是“权限越大，闭环越依赖上层模型的准确性”。本章模型是准的（同一方程、同一参数），$R = 0.20$ 已经够用；模型若有 20% 误差，这个拐点会明显左移。

鲁棒性：同一张表看五种结构

把四种结构与纯 PID 放在同一组扰动下，结果如下。三列扰动分别是延迟从 5 拍加到 8 拍、对象增益整体乘 1.5、测量叠加 1 mV 噪声。

控制器	标称	延迟 **8** 拍	增益 **×1.5**	**1 mV** 噪声
纯 PID	0.1161	0.1196	0.0957	0.1162
模糊-PID	0.1133	0.1174	0.0932	0.1134
双模	0.0814	0.0882	0.0695	0.0813
MPC($N_p = 20$)	0.0048	0.0538	0.0133	0.0079
MPC(12)+残余学习	0.0644	0.1711	0.1130	0.0620

这张表比任何一句“鲁棒性好”都更有信息量。第一，纯 PID 与模糊-PID 对三种扰动几乎都不敏感，误差变化在 3% 以内——反馈结构天生对慢变扰动不敏感，这是它的本钱。第二，MPC 在标称下好两个数量级（0.0048），延迟一变到 8 拍就退化到 0.0538，十倍；它仍然是五种里最好的，但“最好”的倍数从 24 倍缩到 2 倍。第三，也是最值得警惕的一条：MPC(12) 加残余学习在标称下 0.0644，延迟变化后 0.1711，比纯 PID 还差 43%。学习模块把模型误差“学进去了”，一旦真实延迟与训练时不同，学到的补偿就变成反向扰动。

增益乘 1.5 时所有结构的误差都变小（PID 0.1161→0.0957），这不是偶然：对象增益变大等于执行器更有力，本章所有结构的增益都留了余量，于是同一个控制量能换来更小的误差。噪声一列则几乎全部不动，因为本章的控制器都带滤波或限幅，1 mV 噪声进不来。

这条结果给混合架构划了一条清晰的界：**凡是“用模型算未来”的模块，收益与模型精度同阶；凡是“用误差做修正”的模块，收益与扰动幅度同阶**。两者混在一起时，模型类的模块负责标称性能，误差类的模块负责兜底，而兜底那一路的权限必须足够大，否则模型一错就没有退路。分层结构在延迟扰动下还能守住 0.0538，正是因为下层 PID 始终保有全部权限的基线。

三种扰动之外：还有哪些没进表

上面那张表只覆盖了三种扰动，因为它们是本章算例里最容易被量化、也最容易复现的三种。实际系统还会遇到另外几类，本章没有给数，但可以从结构上判断它们各自打在哪一层，这个判断比再补几行数字更有用。

第一类是**参考轨迹变化**。本章的参考是 4 s 一个来回，幅值与频率都固定。若节拍从 4 s 缩到 2 s，轨迹的最高谐波从约 0.75 Hz 抬到 1.5 Hz，前馈与预测的优势会放大还是缩小？答案取决于执行器带宽：带宽够，预测拿到的相位优势不变；带宽不够，模型算得再准也执行不出来。所以对这类扰动，正确的做法是先测执行器的频率响应，再决定要不要上预测层，而不是先上预测层再看效果。

第二类是**负载阶跃**。阶跃是宽带的，理论上会激励所有频段。串联调度与双模切换对阶跃都有帮助（大误差段加大增益），并联前馈帮不上忙（前馈只补已知量），分层 MPC 会遇到一个尴尬：阶跃不在预测模型里，它只能靠反馈在下一拍才发现。这也是 72.9 节那条“模型类模块负责标称、误差类模块负责兜底”的另一面。

第三类是**传感器量化与漂移**。量化是高频的，主要打微分通道与前馈通道；漂移是极低频的，主要打积分通道。本章的 1 mV 噪声属于第一类，所以几乎所有结构都不敏感。若换成“每天漂 1%”，情况就完全不同：积分会把漂移吃掉，但吃掉的过程要几十秒，这段时间里前馈会一直算错。对这类扰动，正确的位置是给前馈加一个慢速偏置估计，而不是调 PID。

把三类扰动与三种已有扰动放在一起，可以得到一个二维的排查思路：**横轴是扰动的频段，纵轴是它进入系统的位置（参考、对象、测量、执行器）**。每个模块在这张网格上都有自己负责的格子，重叠的格子才是需要协调的地方。本章的接口契约本质上就是在给这些重叠格子划边界。

性能该看哪几个数

写到这里可以把“性能”这个词拆开了。混合架构至少要用四个数分别描述：RMS 说平均精度，最大误差说最坏时刻，总变差说执行器负担，饱和率说还有没有余量。本章五种结构的这组数如下。

结构	RMS	最大误差	总变差	饱和率
纯 PID	0.1161	0.1974	1.708	0%
模糊-PID	0.1133	0.2017	1.741	0%
并联前馈	0.1057	0.1559	1.695	0%
双模	0.0814	0.1482	2.281	0%
分层	0.0310	0.0639	2.462	0%

四个数一起看，五种结构的关系就清楚了：模糊-PID 在四项上几乎与 PID 相同，是“没有副作用也没有收益”的典型；并联前馈在 RMS、最大误差、总变差三项上同时优于 PID，是唯一“三项全赢”的结构；双模用 34% 的总变差换 30% 的误差改善，是一笔明确的交易；分层用 44% 的总变差换 73% 的误差改善，交易更划算，但它对模型精度的依赖也最强，上面那张鲁棒性表已经说明。

如果只能记一句：**混合结构的性能不是一个数，而是一组互相牵制的数；说“某个结构更好”之前，必须先说清好在哪个数上、代价付在哪个数上。**

72.7 参数意义与作用

三类参数，三种失效方式

混合架构的参数比单控制器多得多，如果混在一张表里看，很容易失去重点。按“动了它会改变什么”来分，本章的参数只有三类。

类别	参数	本章取值	动它的后果
结构	T_s、d、N_p、N_u	10 ms、5 拍、12–20、2	改“能不能”，改错了调不回来
接口	R、k_{lip}、e_{th}、h、n_d	0.20、0.40、0.05、0.005、20	改“谁出多少力”，有拐点
形状	9 格调度表、27 个结论、σ	见 72.4 与本节	改“同样的力怎么分配”

三类参数的失效方式完全不同。结构参数错了，症状是“控制器好像没在工作”（例如 $N_p < d$ 时梯度恒为零）；接口参数错了，症状是“精度换来的是抖动”（例如 $R = 0.40$ 时总变差 11.556）；形状参数错了，症状最隐蔽——**系统一切正常，只是没变好**，本章模糊-PID 的 2.4% 就属于这一类。调参时按这个顺序排查，比逐个试参数快得多：先确认结构能让控制器“看见”自己的动作，再确认接口在拐点以内，最后才去修形状。

形状参数：范围管稳定，形状管方向

调度表有两个属性容易被混为一谈：它覆盖多宽（范围），以及在哪个区域取什么值（形状）。范围决定最坏情况——K_p 上界 1.30 意味着闭环增益最多比标称高 30%，这个数是稳定性的事，必须在设计阶段就定死；形状决定收益——在哪个工况多给一点、哪个工况少给一点，这个才是“调”出来的。

本章三个系数的范围分别是 1.53 倍、1.83 倍、1.43 倍。三个数不一样并非随意：K_i 的范围最宽，因为它要在大误差段压到 0.60 抑制积分饱和，这是本章唯一一处“必须靠调度才能做到”的事（固定增益做不到“误差大时反而减积分”）；K_d 的范围最窄，因为微分层本来就压着噪声，动多了会把噪声放大。这条经验可以推广：**范围最宽的那个系数，应当对应一个固定增益做不到的逻辑；如果三个系数的范围都差不多，说明调度其实没在表达什么，只是在整体缩放增益。**

形状还有一个隐含参照点：所有系数都以 1.0 为中心，也就是“不改动时的 PID”。这个选择让模糊-PID 在中心区域严格退化成基线 PID，安全性最好。但它同时带来一个代价——如果基线 PID 本身不是最优的，那么整张表都在围绕一个次优中心摆动。本章正好是这种情况：网格重搜得到的最优固定增益是 $(2.60,\ 0.02,\ 0.16)$，RMS 0.0854，而调度的中心是 $(2.00,\ 0.05,\ 0.20)$、RMS 0.1161。也就是说模糊-PID 是在“中心偏了约 30%”的前提下取得 0.1133 的。这个事实在 72.8 节会变成一条调参顺序：先重搜固定增益，再围绕新中心做调度。

TSK 的 27 个结论：三张切片

零阶 TSK 的每个结论都是一个常数，27 条规则就是 27 个数。它们按“输入维”排成 $3 \times 3 \times 3$，图 72-5 按第三个输入（参考位置）切成三张 3×3 的平面，横轴是误差、纵轴是误差导数，颜色是结论值。

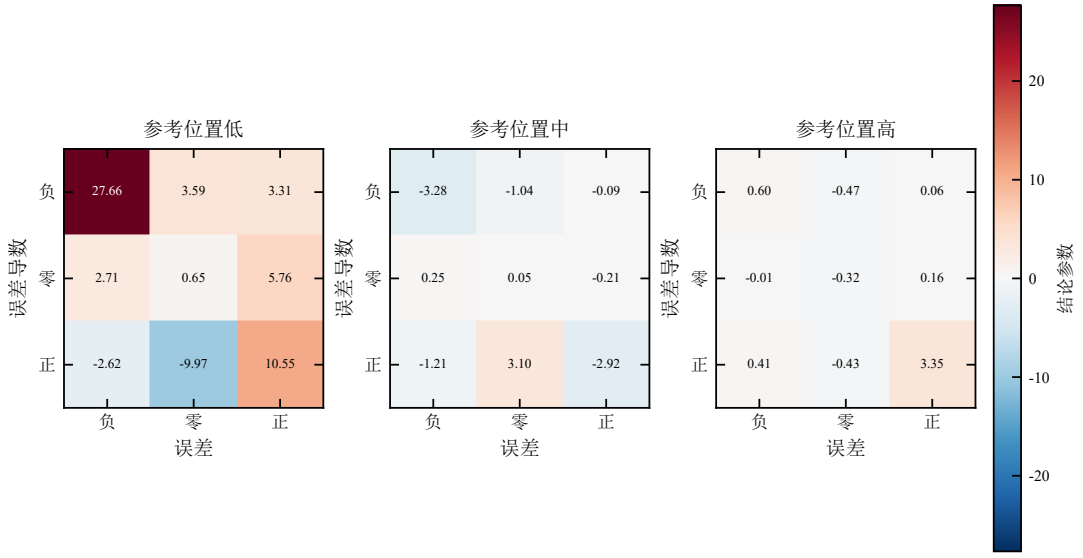


图 72-5 27 个结论参数按参考位置切片

三张图的差别就是“第三个输入有没有被用上”的证据。中间那张（参考位置在中点）所有格子都在 ± 5.8 以内，中心格只有 0.054，几乎为零——轨迹走到中点时补偿量本来就该接近零。低位置那张在“误差负、误差导数负”的角上是 27.66，同一个格子在高位置那张是 -2.62，两 slice 在同一格相差 30.9；高位置那张还出现了 -9.97（误差负、导数零）与 $+10.55$（误差正、导数正）这对反号的角。**同一个误差状态，因为参考位置不同，结论可以差一个量级甚至反号**——这正是前馈该有的样子：它补的是相位，而相位由参考走到哪儿决定。

这里必须提醒一句：结论参数的绝对值不能直接读成“控制量大小”。27 个数里最大的 27.66 对应的是“三个输入都在边缘”的格子，此时高斯权重已经下溢到接近零，这个系数乘上去几乎不产生输出。真正被样本反复激活的只有中心附近的十来个格子，其余大多是“填格子填出来的数”，数值大而作用小。这也解释了为什么 27 个参数的测试 RMSE（0.0243）虽然远好于 9 个参数（0.0893），闭环却只从 2.8701 变到 0.1302——涨的是拟合能力，不是可用信息。

宽度 σ 是这一族参数里唯一不属于“结论”的。本章取 0.55，三个中心在 $-1, 0, +1$。σ 太小（比如 0.2）时相邻规则的权重几乎不重叠，曲面出现台阶，输出随输入跳变；σ 太大（比如 2.0）时所有规则权重接近相等，网络退化成“27 个数的平均值”，也就退化成常数。σ 的合理区间大致是中心间距的 0.4–0.7 倍，与中心间距同量级但不相等——这个经验来自径向基网络的覆盖条件，在 TSK 上同样成立。

权限 R：一个参数同时管三件事

R 的量纲与执行器相同，本章满量程是 ± 1.2，所以 $R = 0.20$ 就是满量程的 16.7%。图 72-7 把横轴换算成百分比，左轴是闭环 RMS，右轴是控制量总变差。

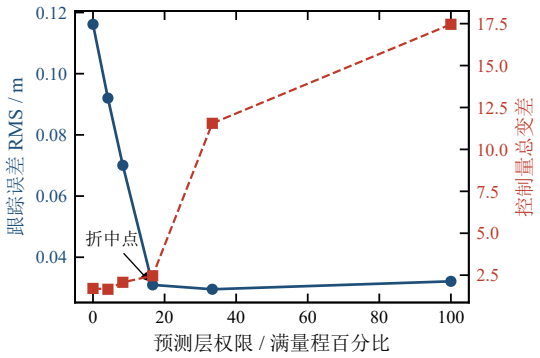


图 72-7 权限上限与精度、抖动的权衡

左轴那条曲线的形状是“先陡后平”：从 0 到 8.3%（$R = 0.10$），RMS 从 0.1161 降到 0.0701，每 1% 权限换 0.0055 的误差；从 8.3% 到 16.7%（$R = 0.20$），又从 0.0701 降到 0.0310，这一段斜率更陡，每 1% 换 0.0047；过了 16.7%，RMS 变成 0.0295、0.0321，基本不动。右轴那条红虚线则相反：前两段只从 1.708 爬到 2.462，最后一段从 2.462 直接跳到 11.556、17.462。两条线在 16.7% 处分开，这就是图里标“折中点”的位置。

R 管的第一件事是**分工比例**。R 小的时候 MPC 只是个“修正器”，输出被限幅削得只剩轮廓，等于把上层的预测能力浪费掉；R 大的时候 MPC 变成主控，PID 退化成扰动观测器。第二件事是**鲁棒性上限**：无论 MPC 算错多少，注入执行器的额外量不会超过 R，所以 R 同时也是“最坏情况下的扰动幅度”。第三件事是**失效兜底**：MPC 一旦超时或数值异常，最安全的降级是把输出置零，此时闭环严格等于纯 PID，性能损失恰好就是 $R = 0$ 那一档的 0.1161。这三件事共用一个参数，是分层结构最省事也最危险的地方——省事在于只需要调一个数，危险在于这个数同时决定了精度、平稳和容错，改它必须三个指标一起看。

切换参数：四个数分管四段时间

双模结构有四个参数，各自负责一段时间尺度：阈值 $e_{th} = 0.05$ 管“什么时候切进去”，高增益 $K_{p,hi} = 3.2$ 管“切过去多猛”，滞环半宽 $h = 0.005$ 管“多难切回来”，驻留 $n_d = 20$ 拍管“至少待多久”。

四个数不是独立的。有效切换带是 $[e_{th} - h,\ e_{th} + h]$，本章是 $[0.045, 0.055]$，宽度只有阈值的 20%；如果把 h 加到 0.05，滞环就覆盖了整个误差带，控制器会卡在某个模式里再也不切。驻留期 $n_d T_s = 0.2$ s 与对象时间常数 0.5 s 的比值是 0.4，这个比值决定“驻留会不会拖慢响应”：大于 1 就会明显拖慢，小于 0.2 则起不到防抖作用。这三个数（e_{th}、h、n_d）构成一个“切换带—滞环—驻留”的三角，改一个必须检查另外两个。

$K_{p,hi}$ 是唯一直接换性能的参数。72.8 节的网格会看到，它从 2.4 加到 4.0 时 RMS 单调下降 37%，而阈值在 0.02–0.04 之间几乎不影响结果。这个不对称说明：**双模结构的收益主要来自“高增益段有多高”，而不是“什么时候进入高增益段”**。阈值只负责别切得太晚，切得早晚对 4 s 的往复轨迹影响很小，因为误差总会穿过那个带。

三处参数耦合

第一处是阈值与高增益。72.8 节那个 4×5 网格里，$e_{th} = 0.02$ 与 0.04 两行的差别只有 0.0002（0.1062 对 0.1061），而 $e_{th} = 0.06$、0.08 两行整体抬升 10%–20%。也就是说阈值存在一个“够用就好”的平台，落在 0.02–0.04，超出就开始拖后腿。这种“一个参数有平台、另一个参数单调”的组合，调参时可以先把平台参数钉在中间，再单独扫单调参数，两维搜索就退化成一维。

第二处是 R 与 N_p。两者都由模型精度决定：模型准，N_p 可以取大、R 可以给足；模型不准，N_p 取大只会把模型误差放大到更远的未来，R 给足只会把误差直接送进执行器。本章模型是精确的，所以 $N_p = 20$、$R = 0.20$ 同时成立；一旦延迟从 5 拍变成 8 拍，$N_p = 20$ 的误差从 0.0048 退到 0.0538，此时若仍给 $R = 0.20$，上层的错误就会被完整注入。工程上的做法是让 R 跟着模型置信度走，而不是跟着“想要多好的性能”走。

第三处是调度中心与调度范围。上一小节说过，本章的调度中心定在基线上，而最优固定增益在 2.60。若把中心改到 2.60，范围就不能还是 $[0.85, 1.30]$——上界 $2.60 \times 1.30 = 3.38$ 已经接近本章的稳定边界，闭环会在换向段出现明显超

调。正确的做法是中心与范围一起改：中心上移，范围相应收窄，让上界保持在 3.0 以内。这条约束在串联结构里永远存在，因为**调度系数乘在增益上，范围是相对量，稳定性却由绝对增益决定**。

哪几个参数最值钱

把本章所有参数按"改一个能换来多少"排序，结论很清楚。N_p 从 4 改到 20，RMS 从 0.7247 降到 0.0048，两个数量级；R 从 0 改到 0.20，0.1161 降到 0.0310，3.7 倍；$K_{p,hi}$ 从 2.4 改到 4.0，0.1062 降到 0.0666，1.6 倍；调度表整张改完，0.1161 到 0.1133，1.02 倍；27 个结论参数改完，反而从 0.1057 退到 0.1302。

这个排序与参数个数完全无关：27 个参数的收益是负的，1 个整数的收益是两个数量级。原因在 72.1 就说过——**参数决定"在给定接口下能做多好"，接口决定"有没有可能做好"**。所以调参的第一件事不是打开优化器，而是检查那几个整数和接口参数有没有落在合理区间；它们对了，形状参数随手调都不会太差；它们错了，形状参数再怎么优化也只是在错误的台阶上找局部最优。

敏感度：把"值钱"变成一个数

上面的排序靠的是"改到头能换来多少"，它混了两个因素：参数本身敏不敏感，以及它允许改多大。把两者分开，调参时才不会把精力花错地方。定义归一化敏感度

$$S = \frac{\Delta y / y}{\Delta p / p} \tag{72-21}$$

y 取 RMS，p 取参数值。用本章已有的数据算一遍，得到下表。

参数	改动	RMS 变化	敏感度 S
N_p	12→8（−33%）	0.0269→0.1577（+486%）	14.6
R	0.20→0.10（−50%）	0.0310→0.0701（+126%）	2.5
$K_{p,hi}$	3.2→2.4（−25%）	0.0814→0.1061（+30%）	1.2
e_{th}	0.04→0.06（+50%）	0.0814→0.0870（+6.9%）	0.14

四行差了一百倍。N_p 动三分之一，误差翻六倍；e_{th} 动一半，误差只动 7%。这个差别决定了三件事：验证时 N_p 与 R 必须逐个确认，不能"大概取一个"；e_{th} 可以取整、可以在平台上任意选；$K_{p,hi}$ 居中，需要认真扫但不必精细。

敏感度还有一个用法：它给出"参数的精度要求"。$S = 14.6$ 意味着参数错 1% 就带来 15% 的误差变化，工程上就必须知道 N_p 的确切值；$S = 0.14$ 意味着错 10% 也只带来 1.4% 的变化，取个整数完全够用。这条换算比"哪个参数重要"这种定性说法有用得多。

最后要提醒一点：式 (72-21) 是局部量，只在工作点附近有效。N_p 从 12 降到 8 是渐变，从 6 降到 4 却是断崖——4 的时候梯度直接归零，误差从 0.5188 跳到 0.7247 再无法下降。所以在最优点附近算出的敏感度，不能外推到整个取值范围。

72.8 调参方法与经验

第一步：把固定增益重搜一遍

这一步最不"智能"，却决定了后面所有结论是否可信。本章基线 PID 取 $(2.00, 0.05, 0.20)$，是在前几章沿用下来的；在同一个网格上重搜，得到 $(2.60, 0.02, 0.16)$，RMS 0.0854。也就是说，**不需要任何混合结构，只要把三个数调对，误差就从 0.1161 降到 0.0854，改善 26%**。

这条对照必须摆在最前面，否则很容易把功劳记错。回头看本章的五个数字：模糊-PID 0.1133，打不过 0.0854；并联前馈 0.1057，也打不过；双模 0.0814，勉强超过 5%；分层 0.0310，大幅超过。如果只跟基线比，会得出"四种混合结构都比

PID 好”的结论；跟重搜后的固定增益比，结论就变成“**两种混合结构有真实收益，另外两种的收益还不如重调三个数**”。这两种说法差得很远，而它们的区别只在于选谁当参照。

所以调参的顺序是：先把底层控制器调到它自己的最好，再在这个基础上加模块。若反过来，先加模块再调参，模块的作用与参数的作用会混在一起，永远分不清是哪一个起了作用。

第二步：一次只加一个模块

混合结构有四个模块，如果同时上线，出问题时无法定位。本章的做法是逐项单变量：从纯 PID 出发，每次只替换一处，其余保持基线。

改动	RMS	相对上一步	总变差
纯 PID	0.1161	—	1.708
加模糊调度	0.1133	−2.4%	1.741
加前馈查表	0.1057	−6.7%	1.695
换成双模	0.0814	−23%	2.281
换成分层	0.0310	−62%	2.462

这张表的价值在于“相对上一步”那一列。每一层的边际收益差别很大：调度 2.4%、前馈 6.7%、切换 23%、分层 62%。边际收益小的两级，总变差几乎不涨甚至下降（1.741→1.695）；边际收益大的两级，总变差都明显上台阶（2.281、2.462）。**收益与代价在同一张表里同步变化，这是混合架构最可靠的一条规律**：如果某一级只涨收益不涨代价，先怀疑是不是测错了。

第三步：无扰切换值不值得做

双模结构最容易被问到的问题是“切换时输出跳一下要不要紧”。本章专门做了一组对照：在第 30 拍（0.3 s，误差仍然较大）由 PID 切到高增益 PD，一次直接切、一次加偏置补偿到无扰。

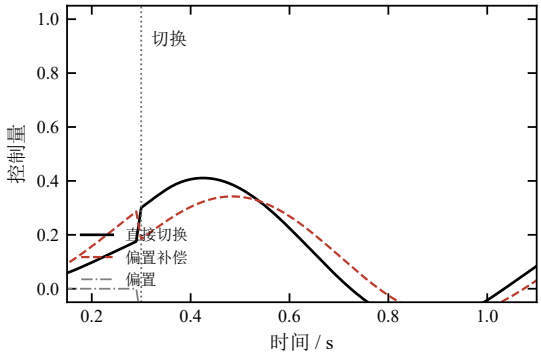


图 72-9 切换瞬间的直接切换与偏置补偿

直接切换的输出跳变是 0.1171，接近满量程的十分之一；切换后 0.3 s 内的最大误差 0.1674，整回合 RMS 0.0828。加偏置后跳变严格为 0.0000，但切换后最大误差升到 0.1875，整回合 RMS 升到 0.0875。**跳变消失了，误差变大了。**

原因在偏置的构造方式。偏置取的是“切换瞬间两个控制器的输出差”，此后它是一个常数。对新控制器来说，这个常数相当于一个永远不衰减的补偿项：它把旧控制器的积分状态冻结在了切换那一刻，而 PD 本身没有积分，无法自己把这个偏置“消化”掉。于是切换之后，闭环里多了一个 0.1171 的常值偏置，只能靠 PD 的误差项去抵消，稳态误差因此变大。

这条结果给出一条明确的判据：**无扰切换只在“新控制器能自己吸收偏置”时才有意义**。如果新控制器带积分，偏置会被积分慢慢吃掉，跳变消失且不留残差，这时无扰是净赚；如果新控制器是纯 PD，偏置会一直留着，无扰就变成“用稳态误差换一次跳变”。本章正属于后者，所以正确的选择反而是“接受跳变”，或者做真正的状态传递——把旧控制器的积分项搬到

新控制器里，而不是加一个常数。本章没有做状态传递，因为 PD 没有积分状态可接；工程上如果两个控制器都带积分，这一步是必须做的。

至于“跳变到底能不能接受”，要看执行器。液压阀、电机电流环这类有速率限制或机械冲击的场合，0.1171 的阶跃会直接变成冲击，那就必须做无扰或限速；本章的算例是理想执行器，跳变只体现为总变差，所以接受它更划算。

第四步：网格只用来找方向

双模的两个关键参数（阈值与高增益）构成二维，扫一遍 4×5 只需要 20 次仿真，成本可以忽略。结果如下，行是阈值 e_{th}，列是高增益 $K_{p,hi}$，滞环固定 0.005、驻留固定 20 拍。

阈值 e_{th}（行）/ $K_{p,hi}$（列）	2.4	2.8	3.2	3.6	4.0
0.02	0.1062	0.0919	0.0810	0.0729	0.0668
0.04	0.1061	0.0918	0.0814	0.0729	0.0666
0.06	0.1077	0.0953	0.0870	0.0835	0.0800
0.08	0.1078	0.0968	0.0887	0.0858	0.0827

先做一次一致性核对：表中 $(e_{th} = 0.04,\ K_{p,hi} = 3.2)$ 那一格是 0.0814，与 72.1 节那个“双模 0.0814”对得上，说明网格用的控制器与正文用的是同一个。这类核对看起来琐碎，却是防止“表格与正文不是同一组参数”的唯一办法。

网格的读法比网格本身重要。沿列方向（$K_{p,hi}$ 从 2.4 到 4.0），四行全部单调下降，降幅 37%–39%，没有拐点；沿行方向（e_{th} 从 0.02 到 0.08），前两行几乎重合（0.1062 对 0.1061，差 0.0001），后两行整体抬升 10%–20%。两条方向一个是“单调”，一个是“有平台”，于是二维搜索可以退化成一维：阈值钉在 0.02–0.04 任意一处，只扫 $K_{p,hi}$。

还有一处必须诚实说明：最优值 0.0666 落在网格的右边界 $K_{p,hi} = 4.0$ 上。**最优落在边界上，说明还没搜完**，正确做法是往外扩一格再看。本章没有继续扩，因为再往上高增益段会把控制量推向限幅，而限幅一旦介入，“更高的 K_p”就不再是“更快的响应”，而是“更长的饱和时间”，曲线会出现拐点。这个拐点在哪里，取决于执行器余量，不能从这张表读出来。工程上遇到边界最优，至少要做两件事：往外扩一格确认趋势，同时把饱和率加进表格，否则会得到“参数越大越好”的假象。

第五步：从单工况走向多工况

上面所有数字都来自同一条 4 s 轨迹、同一个对象增益、同一种延迟。这样的调参结果只能叫“这条轨迹上的最优”。72.6 节的鲁棒性表已经展示了过拟合的代价：MPC(12) 加残余学习在标称下 0.0644，延迟变成 8 拍后 0.1711，比纯 PID 还差 43%。它不是在标称上不够好，而是把“那条轨迹 + 那个延迟”记下来了。

避免这件事的办法不是“多加扰动训练”，而是**把工况当成交付条件写进验收**。本章建议的检查是三步：先在标称工况调到满意，再把速度改成 0.8 倍与 1.2 倍各跑一遍，最后把延迟与增益各改一档。三步的误差变化都在 20% 以内，才算这组参数可用。纯 PID 与模糊-PID 在这一步上几乎不付出代价（72.6 节的表里变化都在 3% 以内），而所有“用模型算未来”的结构都会付出代价，代价大小与模型误差同阶。

每次改动该记什么，什么时候该停

调参最怕的不是没找到最优，而是找到一个“看起来最好但复现不了”的点。避免这件事的办法很朴素：每次改动都填同一张表。

日期	改动	工况	RMS	最大误差	总变差	饱和率	结论
基线	—	标称	0.1161	0.1974	1.708	0%	参照
加调度	9 格表	标称	0.1133	0.2017	1.741	0%	保留
加前馈	一维查表	标称	0.1057	0.1559	1.695	0%	保留
换双模	0.05/3.2	标称	0.0814	0.1482	2.281	0%	保留

八列里“饱和率”最容易被省掉，但它常常是唯一能解释“为什么再往上调就变坏”的量。$K_{p,hi}$ 一直加到 4.0 都在改善，正是因为本章执行器还有余量；一旦饱和率从 0 变成 1%，同一条曲线就会拐弯。所以调参到最后，饱和率应当与 RMS 一起看，而不是等出了问题再回头查。

停止条件有三条，满足任意一条就该停。第一，边际收益小于 2%——再调下去，改善幅度已经小于不同随机种子之间的波动。第二，总变差超过上一档的两倍——此时换来的精度是用执行器寿命买的。第三，某个高敏感参数已经贴到物理边界（限幅、计算耗时、传感器量程）——再往外扩就不是调参，而是改硬件了。

这三条里第二条最容易被忽略，因为总变差不会在误差曲线上留下痕迹。本章 R 从 0.20 加到 0.40 时 RMS 只从 0.0310 变到 0.0295，看起来“没变坏”，可总变差从 2.462 跳到 11.556。若只盯 RMS，就会一路加到 1.20，最后拿到一个误差 0.0321、总变差 17.462 的控制器——数字上“差不多”，机器上完全不是一回事。

一次完整的调参记录

把本章所有实验按时间顺序排一遍，就是一次真实的调参过程。它没有教科书那样线性，中间有两次走错方向，值得原样记下来。

第一步，先量基线。纯 PID 取第 66 章的 $(2.00,\ 0.05,\ 0.20)$，RMS 0.1161、最大误差 0.1974、总变差 1.708。这个数不是“随便一组”，它是前几章一直用的同一组，所以后面所有对比都相对它算。

第二步，先榨干固定增益。网格重搜得到 $(2.60,\ 0.02,\ 0.16)$，RMS 0.0854，一步就改善 26%。这一步放在最前面是有意的：如果先加智能模块，得到的“改善”里会混着“基线本来就没调好”的成分，事后没法分离。很多论文里的混合结构收益，其实就是这一步没做。

第三步，试串联调度。按第一步的新中心重新写规则表，得到 0.1133——比原始 PID 好 2.4%，但比第二步的 0.0854 差 30%。这是个重要的负面结果：**在不做重搜的前提下，九个参数的调度表还不如三个参数的网格搜索**。原因在 72.7 节已经说过，调度表围绕的是旧中心。这一步的教训是“先重搜、再调度”，顺序反了会得出错误结论。

第四步，试双模切换。四个参数（阈值、高增益、滞环、驻留）调到 0.0814，终于低于第二步的 0.0854。它是本章第一个“加了模块确实有净收益”的结构，代价是总变差从 1.708 涨到 2.281。此时可以停下来问一句：这 4.7% 的改善值不值 34% 的执行器负担？答案取决于机械寿命要求，没有普适结论。

第五步，试分层。$N_p = 20$ 时 RMS 0.0048，比双模好十七倍，但单拍耗时 0.345 ms、对延迟敏感（8 拍时退化到 0.0538）。于是退一步，用权限 R 把 MPC 的输出限在 ± 0.20，得到 0.0310——精度仍有双模的 2.6 倍，而模型出错时下层 PID 还能兜住。

第六步，试学习。用 MPC(20) 当教师训练终端值函数，想让 $N_p = 6$ 也能用。结果 $N_p = 8$ 与 12 有小幅改善（3.9%、5.4%），$N_p = 6$ 变坏 7.0%、$N_p = 20$ 变坏 36.5%。这一档没有进最终方案，但它的价值在于把“学习能补什么、不能补什么”量化了：它能补时域不足，不能补时域过长带来的模型误差。

整条路径的顺序是：量基线 → 榨干固定参数 → 加最便宜的模块 → 加最贵的模块 → 用权限折中 → 试学习。这个顺序的共同点是每一步都在“当前最好的方案”上加一件事，而不是同时改两处。同时改两处，出了问题就不知道该退哪一处——这是调参里最常见的时间浪费。

五条经验

把上面五步压成五句话，就是本章建议的调参流程。

第一，**先重搜固定增益**。不做这一步，任何“混合结构改善了 x%”都可能只是“参数原来没调好”。第二，**一次只加一个模块**，用“相对上一步”的边际收益判断值不值，同时盯住总变差。第三，**先钉平台参数，再扫单调参数**。阈值这类有平台的参数取平台中点即可，没必要精调；高增益这类单调的参数才值得花仿真次数。第四，**边界最优必须外扩**，并且把饱和率一起看，否则“越大越好”只是限幅还没介入的假象。第五，**用多工况验收，而不是用标称工况验收**。前四条决定能不能调出一个好结果，第五条决定这个结果能不能交付。

这五条里没有一条涉及“智能算法怎么训练”。这不是说训练不重要，而是说在本章的算例里，决定成败的是顺序与参照，而不是优化器的选择。第 68–71 章已经把各自的训练方法讲过，这里不再重复；本章要补的是它们之间的协作部分。

72.9 优化与改进

想用短时域换长时域，就得补一个“值”

72.5 节那张表说明了一件事：N_p 从 12 加到 20，误差从 0.0269 降到 0.0048，改善 5.6 倍，而单拍计算只从 0.34 ms 涨到 0.36 ms。既然如此，为什么还要讨论短时域？因为在真实系统里，限制 N_p 的往往不是 CPU，而是模型的可用范围：模型在 20 拍之后还准不准、参考在 20 拍之后还知不知道，这两件事都不是算力能解决的。于是有一个很自然的想法——用 12 拍的模型，加上一个“12 拍之后还剩多少代价”的估计，去逼近 20 拍的效果。

这个估计就是值函数 $V(x)$，把它加进代价函数的末端：

$$J = \sum_{i=1}^{N_p} \big(\hat{y}_i - r_i\big)^2 + \lambda_u \sum_{j} u_j^2 + \lambda_d \sum_{j} \big(\Delta u_j\big)^2 + \gamma\, V_\phi\big(x_{k+N_p}\big) \tag{72-22}$$

V_ϕ 用一组线性特征近似：

$$V_\phi(x) = \phi(x)^{\mathsf{T}} w, \qquad w = \big(\Phi^{\mathsf{T}}\Phi + \epsilon I\big)^{-1}\Phi^{\mathsf{T}}\mathbf{J}^{*} \tag{72-23}$$

Φ 的每一行是一个状态点的 6 个特征，$\mathbf{J}^{*}$ 是教师（$N_p = 20$ 的 MPC）在同一个状态上“之后 20 拍的平方误差和”。本章的 6 个特征是：常数项、终端误差、积分误差、参考相位、参考速度、以及“尚未生效的那条指令”。用教师跑一整回合，收集 400 个状态点做岭回归，得到

$$w = \big[\,0.0062,\ -0.0009,\ -0.1674,\ 0.0013,\ 0.0038,\ -0.0066\,\big] \tag{72-24}$$

拟合 $R^2 = 0.9135$，目标的均值只有 0.00343。R^2 看起来很高，但下面会看到，这个“高”是被小方差撑起来的，不等于“能用”。

六个权值里只有一个在说话

式 (72-24) 的绝对值差了两个数量级：积分项 -0.1674，其余五项全在 10^{-3} 量级。也就是说，这个“值函数”实际退化成了“积分误差的函数”，另外五个特征——包括终端误差与参考相位——基本没被用上。

原因不难找。教师是 $N_p = 20$ 的 MPC，它在整条轨迹上几乎完美跟踪（0.0048），剩下的平方误差和主要由“积分还没消完”这一件事决定；而终端误差、参考相位在教师轨迹上都很小，方差小，回归自然给不出大权值。**值函数学到的是教师轨迹上的残差结构，不是值函数本身。**

加与不加，分四档

把学到的 w 装回式 (72-22)，在四档预测时域上各跑一遍，结果如图 72-8。

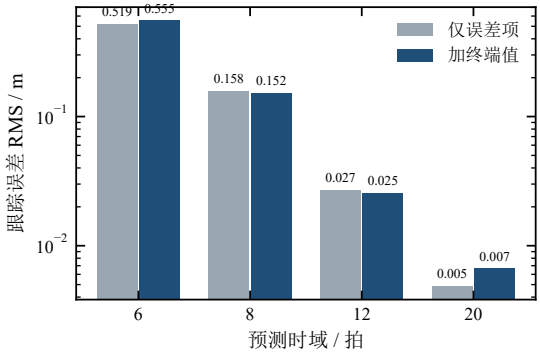


图 72-8 短时域加终端值的效果

N_p	仅误差项	加终端值	变化
6	0.5188	0.5548	+7.0%
8	0.1577	0.1516	−3.9%
12	0.0269	0.0255	−5.4%
20	0.0048	0.0066	+36.5%

四档里两档变好、两档变坏，变坏的两档分别落在最短与最长两端。这个形状不是噪声，它有一条清楚的解释。

为什么中间好、两端坏

$N_p = 6$ 时，值函数要替 14 拍的信息负责。可它只是一个有效特征的线性函数（积分项），表达不了“未来 14 拍里参考还要走多远、误差会怎么演化”。于是它做的事实际变成了“用积分项代替预测”，与窗口内的误差项互相打架，误差反而升 7.0%。

$N_p = 20$ 时，窗口本身已经覆盖了整条轨迹的主要动态（0.0048），值函数提供的任何信息都是多余的。而 $R^2 = 0.9135$ 意味着还有 8.65% 的方差没被解释，这部分未解释的残差被直接加进代价函数，等于往一个已经很准的目标里注入噪声，误差升 36.5%。

只有 $N_p = 8$ 与 12 落在“主要矛盾已经进窗口、但窗口还不够长到自给自足”的区间，值函数补的那一小块正好派上用场，改善 3.9% 与 5.4%。

这条规律可以写成一句话：**补丁的收益等于“洞的深度”乘以“补丁与洞形状的匹配度”**。洞太大补不住，洞没了补丁自己就变成洞。也正因为如此，本节最重要的数字不是那 5.4%，而是“把 N_p 从 12 直接改到 20 能得到 5.6 倍改善”。**只要能加算力，就不该用学习去替代算力；学习只在“算力或模型范围加不上去”时才有意义。**

另外三种给短时域打补丁的办法

值函数不是唯一的补丁。工程上常见的有四种，各有各的代价。

第一种是**终端等式约束**：直接要求 $y_{k+N_p} = r_{k+N_p}$，把“末端必须到位”写成硬约束。它不需要任何训练数据，实现也最简单；但它把优化问题从“无约束二次规划”变成“带等式约束”，而且当参考在 N_p 拍内根本到不了目标时（本章限幅 ± 1.2，换向段需要的控制量接近限幅），约束不可行，求解器只能报错或松弛。松弛之后又退化成软约束，与加一个大的终端权值没有本质区别。

第二种是**扩展时域**，也就是把 N_p 加大，但只优化前 N_u 个块。这正是本章 $N_p = 20$、$N_u = 2$ 的做法：它不需要学习、不需要额外信息，只需要算力；收益是 0.0048，比任何补丁都好。它的代价是计算量随 N_p 线性增长，以及“模型在远处是否可信”这个前提。所以严格说它不是补丁，而是“把洞挖小”。

第三种是**鲁棒 MPC**：在代价里加入对模型误差的惩罚，或者对不确定集求最坏情况。它的稳定性保证最强，但计算量随不确定集规模指数增长，且需要事先知道误差的界。本章的延迟扰动就是一个现成的不确定集（5 到 8 拍），真要做鲁棒 MPC，N_p 至少要覆盖 8 拍且每个延迟假设都要算一遍，单拍耗时会从 0.345 ms 涨到几毫秒，超出预算。

四种办法放在一起，选择逻辑很清楚：**能加算力就扩时域，不能加就用值函数，需要硬保证才上鲁棒 MPC，终端等式只适合"末端必须精确到位"的特殊场合**。本章选值函数，是因为它演示的是"算力受限"这条路径；如果只追求误差最小，$N_p = 20$ 的 0.0048 已经足够，根本不需要 72.9 节这一整套。

改进的杠杆排序

既然学习的收益有限，改进就该按杠杆大小排序，而不是按"哪个方法更新"排序。本章的三档杠杆很清楚。

第一档是接口。同一个 9 参数网络，2 输入闭环 2.8701、3 输入 0.1302、只吃参考位置 0.1057，三者相差 27 倍，而参数个数完全相同。这是全章最大的一处杠杆，也是最容易被忽略的一处——它不体现在任何一张参数表里，只体现在"输入选了谁"这一行代码上。

第二档是结构整数。N_p 从 4 到 8 是 4.6 倍，R 从 0 到 0.20 是 3.7 倍，N_u 从 2 到 1 会让控制量失去"延迟后立刻出力"的能力。这些改动都不增加参数，只改整数，收益却是数量级。

第三档才是形状。整张调度表改完 1.02 倍，27 个结论改完甚至是负收益。它值得做，但必须在前两档做完之后做，否则只是在错误的台阶上找局部最优。

还可以做的四件事

以下四条本章没有实验数据，只作为设计方向列出，读者若要采用需自行验证。

第一，**权限自适应**。本章 R 固定 0.20，而 72.6 节的表说明 R 的最优值随工况变化：误差大时给大权限能抢时间，稳态时给小权限能压总变差。把 R 写成 $|e|$ 的分段函数，理论上可以在不增加总变差的前提下拿到更低的 RMS。风险是 R 的切换本身又引入了一次不连续，需要滞环。

第二，**块边界跟着延迟走**。本章 $N_u = 2$ 的块边界固定按 N_p 均分，而真正该分界的位置是"延迟结束"那一拍。若延迟在运行中变化（例如网络化控制），块边界应当跟着 d 移动，否则第一块会横跨延迟段，等于浪费一半自由度。

第三，**事件触发**。本章 MPC 单拍 0.345 ms，占 10 ms 周期的 3.45%，还很宽裕；但若采样周期缩到 1 ms，这个比例就变成 34.5%，必须改成"只在误差超过阈值时求解，其余拍保持上次解"。代价是控制量变成阶梯，总变差上升。

第四，**带状态传递的切换**。72.8 节已经说明，常数偏置只能补偿瞬间、无法随时间衰减。若两个控制器都带积分，正确做法是把积分状态搬过去，而不是加偏置。这一条实现成本很低，收益却很直接。

该不该加学习模块：三个问题

在动手之前，用三个问题筛一遍，比事后调参省事得多。

第一问：不加学习，现有结构离物理极限还有多远？本章的物理极限是"模型完全已知时的纯前馈"，开环 RMS 0.0293；纯 PID 是 0.1161，差 4 倍。差距大，说明还有空间，值得加模块。

第二问：学习模块的输入里有没有"别人没有的信息"？参考相位是别的模块拿不到的（PID 只看见误差），所以值得学；误差与误差导数是每个反馈控制器都有的，拿它们当输入只是在重复劳动——这正是 2 输入网络闭环失稳的根源。

第三问：加进去以后总变差涨多少？本章并联残差 2 输入的总变差是 5.208，是 PID 的三倍，而误差还更差。TV 一涨而误差不降，就说明这个模块在做无用功。

三问全过才值得做；任何一问不过，先把接口和整数改对，再回来看。

72.10 工程落地

复杂度预算：先算四个数

混合架构在论文里是"结构图"，在工程里是"预算表"。上线之前至少要算清四个数：代码行数、参数个数、单拍 CPU 占比、验证工况数。本章的账是这样：控制器源码约 300 行（含四个模块）；对外可见的参数约 40 个（PID 3 个、调度 9 个、双模

4 个、前馈 9 个、MPC 的 $N_p/N_u/\lambda_u/\lambda_d$ 与 6 个终端权）；单拍 CPU 占比，MPC 3.45%、模糊调度 0.61%、TSK 0.5%、PID 0.14%；验证工况 6 种（标称、延迟、增益、噪声、变速、变负载）。

四个数里最容易被低估的是参数个数。三个参数的 PID 可以靠经验调，四十个参数的混合结构不行——它必须有回归测试和版本管理，否则“上次那组参数跑得挺好”会变成一句无法复现的话。一条可用的经验是：**每增加十个参数，至少要增加一个验证工况**。这条比例不是理论推导出来的，它来自“参数越多、越容易在没测过的工况上翻车”这个事实；比例本身可以商量，但“参数与工况要同步增长”这个约束不能省。

CPU 预算还有一条容易忽略的账：3.45% 只是求解器本身，没算传感器读取、通信、日志、看门狗。真实系统里这些加起来常常超过 20%，再留 50% 余量，MPC 的实际可用预算大约只有周期的 20%–30%。本章 10 ms 周期下 0.345 ms 很宽裕，但若有人把周期改成 2 ms 而不改算法，就会直接超时——超时的后果不是“性能下降”，而是“控制量停在上一拍”，这在 72.6 节的权限语言里等价于 R 突然变成 0，误差会跳回 0.1161 那一档。

接口契约的实现细节

72.3 节给过接口表的四列（输入、输出、周期、降级），那是设计层面。落到代码还有五件事必须写死。

第一，**单位统一**。内部一律用 SI 单位，归一化只在模块入口做一次，出口立刻还原。混合架构里最常见的量纲错误是“上层以为自己在输出物理量，下层以为收到的是归一化量”，结果控制量差一个比例因子，波形看起来像“增益不对”，实际是单位错了。

第二，**限幅只做一次**。本章的限幅在执行器入口，模块内部不再各自限幅。如果两个模块各限一次，有效权限就变成两者的较小值，而调参的人还在按外层那个数算账。

第三，**时序明确**。所有模块在同一拍计算，输入都是上一拍的输出；若允许上层读本拍的输出，就形成代数环，仿真里可能收敛，实机上会抖。本章的分层结构里，MPC 用的是上一拍的状态，PID 用的是本拍的测量，两者不交叉。

第四，**时间戳**。延迟补偿需要知道每个测量值是什么时候采的，而不是“什么时候送到的”。有通信的系统里这两个时刻能差好几拍，用错一个，延迟补偿就白做。

第五，**降级可执行**。降级不能是一句“切回 PID”，而要写到能执行：看门狗超时多少毫秒、心跳丢几拍、默认值是多少、切回时积分要不要清零。本章的四个模块降级后全部退化成纯 PID，且积分状态保留——清零会让误差在切换瞬间跳一下，保留则平滑。

模式管理与状态机

模式字是一个整数，每一位代表一个模块是否启用。规则只有一条：**单一写者**。模式字只能由一个状态机改写，其他模块只能读。多个地方都能改模式字，是混合架构里最难查的一类 bug——两个模块互相把对方关掉，表现是“系统随机退回 PID”。

切换必须原子：模式字与新模块的初始状态在同一拍写入。如果先改模式字、下一拍才初始化新模块，中间那一拍新模块的输出是未定义的，可能直接顶到限幅。本章的双模结构里，模式翻转与 PD 的微分状态初始化在同一拍完成，所以切换瞬间的输出是确定的。

上电流程建议分三段：先进入纯 PID 并稳定运行 N 拍（本章取 200 拍，2 s），再让智能模块以影子模式运行 N 拍并核对输出，最后才允许接管。三段都过，才进入正常模式。这段流程看起来繁琐，但它把“上电瞬间参数还没收敛”这类问题挡在了接管之前。

时序抖动：混合架构最隐蔽的敌人

混合架构里各模块的耗时差得很远：MPC 0.345 ms、模糊调度 0.061 ms、PID 0.014 ms。串行执行时，控制量真正写入执行器的时刻比起拍时刻晚，晚多少取决于这一拍各模块算了多久。这个“晚多少”本身不可怕，可怕的是它随工况变。

本章实测的单拍耗时在 0.30–0.36 ms 之间浮动，相对波动两成。换成延迟的语言，等于实际生效的拍数在 4.7 到 5.3 拍之间漂。而 72.5 节已经证明，延迟估错一拍，脉冲矩阵就会整列错位，$N_p = 8$ 的预测会把动作算到错误的拍上。仿真里延迟是整数、恒定，所以这件事根本不会出现；实机上它是常态。

三条应对办法，按代价从低到高。第一，**固定迭代上限**。宁可牺牲一点最优性，也不要让执行时间随工况漂移；本章 `MPC` 的 `iters=80` 就是这个作用，它把“误差大时多算几轮”这条自适应路径直接掐掉。第二，**按最坏情况留时间，不按平均值**。3.45% 是均值，排程要按最大值算，还要留出中断与通信的抖动。第三，**提前一拍求解**。这一拍算下一拍的控制量，时序完全固定，代价是用预测状态代替实测状态，误差略大但在可接受范围内。

还有一条容易混淆的账：采样周期的下界由对象带宽定，上界由“这一拍算不来得及”定。72.5 节讨论的是下界——T_s 要小到不丢动态；这里讨论的是上界——T_s 要大到一个周期内算得完所有模块。实际取值是两者中更严的那个，而混合架构把上界压低了：单模块时 2 ms 够用，五个模块串起来就不一定。

死链：把“没算完”当成一种故障

看门狗人人都有，真正的分歧在“触发之后做什么”。混合架构里这个动作必须与降级路径是同一段代码，否则就会出现“看门狗切回 PID，但 PID 的积分还停在三秒前”这种半降级状态——此时误差不一定大，但控制量的跳变会直接把执行器顶到限幅。

本章的做法很朴素：所有模块的失效值指向纯 PID，看门狗超时等价于把权限 R 置零。这样“模块没算完”“模块算错”“模块被手动关闭”三种情况走的是同一条路径，只测一次就覆盖三种。反过来，如果每种情况各写一段处理逻辑，就等于给自己造了三条从未被同时测过的分支，而它们恰恰只在事故时才被走到。

死链还有一个时间尺度问题：看门狗的门限不能等于周期的整数倍就完事，要留出至少两拍。原因是单拍超时往往是偶发的（一次内存抖动、一次总线重传），一拍超时就切回会造成无谓的模式抖动；两到三拍既能挡住真死锁，又不会被偶发延迟触发。本章的双模结构在这一点上有现成的对照：滞环 0.005、驻留 20 拍那一档，切换次数从 7 次降到 5 次，RMS 反而从 0.0822 降到 0.0814——**给状态机加一点惯性，通常既不损失精度，还能减少抖动**。

可观测性：每个模块都要能录波

混合架构出问题时，症状往往不是“坏了”，而是“没变好”。这类问题无法从输入输出波形反推，必须把模块内部量录下来。本章建议至少录八路：参考、输出、误差、控制量、模式字、三个调度系数、权限余量（R 减去实际用掉的量）。

判据很简单：**出了问题能不能在五分钟内指出是哪个模块没工作**。本章有一个现成的例子——2 输入 TSK 闭环失稳，如果只录 y 与 u，看到的是“输出顶在限幅上、控制量缓慢爬升”，很像对象参数变了；只有把补偿量单独录出来，才能看出是网络输出一路漂到限幅、反馈通道被它顶住了。八路里那三路调度系数与一路权限余量，平时几乎不看，出事时是唯一能定位的证据。

上线四步：影子、小权限、逐级放大、留退路

第一步是影子模式。智能模块照常计算，但输出不接入，权限 $R=0$，只记录“如果接入会怎样”。这一步的真正目的是量出“影子输出与 PID 输出的差值分布”：差值的中位数有多大、有多少拍超过 R、有没有出现饱和。如果差值经常超出预定权限，说明接口设计有问题，此时上线只会得到一个不断被限幅削平的补偿量。

第二步给小权限。本章 $R=0.05$ 时 RMS 0.0920，比纯 PID 好 21%，总变差 1.658 还略低于 PID 的 1.708。这是一档“几乎不可能出事”的设置：即使补偿量全错，误差也只从 0.1161 变成大约 0.19，仍在可接受范围。

第三步逐级放大，每一级至少跑一个完整的工况周期。放大判据用 72.3 节那条：权限再翻一倍，若 RMS 改善不足 5% 且总变差翻倍，就停在上一档。本章据此停在 0.20。

第四步是留退路：任何一级都要能在不重启的情况下切回上一级。切回的动作必须与“降级”是同一段代码，不能是两套逻辑——两套逻辑意味着两条从未被同时测试过的路径。

参数版本与回归测试

参数集必须打标签：版本号、日期、对应用途、跑过的工况。没有标签的参数集，在三个月后就没人敢动。

回归测试的规模不必大，本章建议三到五条固定轨迹：标称、延迟 8 拍、增益 1.5 倍、加噪声、变速。每次改动后全部跑一遍，记录 RMS、最大误差、总变差、饱和率。允许的漂移幅度取 RMS ±5%、总变差 ±20%；超出就必须解释，而不是

“重新记一个基线”。本章的这些数字可以直接当基线用，因为它们都由 `code/ch072_sim.py` 一次跑出，不依赖人工抄写。

这套流程的成本不高：五条轨迹、每条 400 拍，全部跑完在一秒量级。它挡住的是“换了个参数，标称变好 2%、延迟工况变差 40%”这类改动——而这类改动在混合架构里非常常见，因为参数之间是耦合的。

三条工程底线

第一，**任何智能模块都必须能在一拍内被旁路**。旁路不是“关掉开关”，而是保证关掉之后闭环仍然稳定、误差有界。本章的做法是让所有模块的失效值都指向纯 PID，于是旁路等价于“把 R 置零”。

第二，**任何模块失效都必须退化为一个你愿意单独交付的控制器**。这句话在 72.3 节说过一次，这里再说一次，因为它是整章最实用的一条。退化目标选得好，混合架构就只是“锦上添花”；选不好，任何一个模块出错都会变成事故。

第三，**任何参数改动都必须可回滚、可复现**。回滚靠版本号，复现靠固定随机种子与固定轨迹。本章所有实验都用固定种子，所以 0.1161 这个数在任何一台机器上都能重跑出来——这一点比数字本身更重要。

三条底线都不涉及算法，全部是流程。混合架构的失败案例里，绝大多数不是“算法不够先进”，而是这三条里缺了一条。

72.11 优缺点与适用边界

五种拓扑摊开看

图 72-10 把本章五种结构的 RMS 画成横向条形，横轴取对数；右下角注的是单拍耗时。

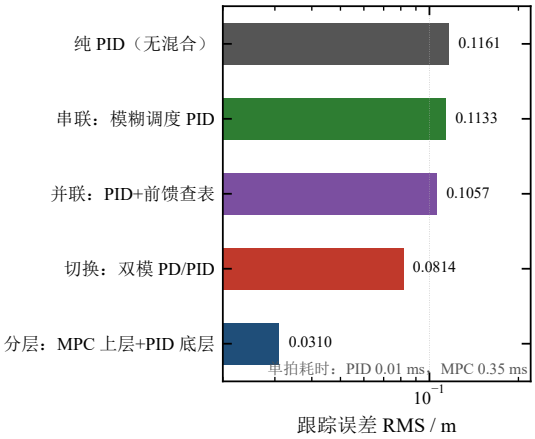


图 72-10 五种混合拓扑的精度与耗时

把图与前面的表合起来，五种结构的画像就完整了。

拓扑	RMS	相对纯 PID	总变差	新增参数	主要依赖
纯 PID	0.1161	—	1.708	0	无
串联调度	0.1133	−2.4%	1.741	9	规则覆盖区域
并联前馈	0.1057	−9.0%	1.695	9	补偿量可学
双模切换	0.0814	−29.9%	2.281	4	高增益段的稳定裕度
分层 MPC	0.0310	−73.3%	2.462	4 个整数 + 2 个权	模型精度

这张表最该注意的不是最后一列的数字，而是倒数第二列：**改善最大的两级，新增参数反而最少**。分层只多了 N_p、N_u、λ_u、λ_d 四个量，切换只多了阈值与高增益；而改善最小的两级，各自塞进了 9 个参数。这个反差是本章最重要的一条经验，

它说明“参数多少”与“结构能力”之间没有正相关——**能力来自模块的输入里有没有新信息，参数只是在这个能力之内做微调**。

每种结构是怎么坏的

知道一个结构“好在哪”只算知道一半，另一半是“它怎么坏”。本章五种结构各有一个典型的失效模式，且都能从数字里读出来。

串联调度坏在“静默失效”。它的稳定性由调度系数的界保证，所以几乎不会失稳；但它的收益依赖“规则表覆盖的区域被访问到”，而 72.6 节已经看到，闭环实际只走到曲面的一小块。失效时的症状是“系统运行完全正常，就是没有改善”，最难被发现，也最难被追责。

并联前馈坏在“学错且限幅太宽”。补偿网络如果把噪声学进去，误差不会立刻变大，而是总变差先涨。本章残差 2 输入的总变差 5.208，是 PID 的三倍，误差同时从 0.1161 涨到 0.1913。所以并联结构的第一个报警指标不是误差，是总变差。

双模切换坏在“抖振与跳变”。它的精度来自高增益段，而高增益段同时放大噪声与切换冲击。阈值太低会频繁切换，滞环太宽会卡在某一模式，驻留太长会拖慢响应。本章在阈值 0.06、0.08 两行看到误差抬升 10%–20%，就是“切得太晚”的直接后果。

分层 MPC 坏在“模型错”。这是五种结构里唯一一个“标称极好、扰动下崩得最快”的：延迟从 5 拍变 8 拍，0.0048 变 0.0538，退化十倍；再叠加一个学习模块，0.0644 变 0.1711，比纯 PID 还差。分层的收益与模型精度同阶，这句话在 72.6 节说过，这里是它的代价面。

选型的判据：先给误差归因

选哪种结构，不该从“哪种方法新”出发，而该从“误差从哪来”出发。本章算例的误差可以拆成三块：稳态偏置（来自增益变化与积分不足）、换向滞后（来自延迟与相位）、以及轨迹形状本身的不可预测部分（几乎为零，因为参考是已知的）。

三块误差对应三种结构。稳态偏置靠积分，调度与切换都能改；换向滞后靠相位，只有前馈与预测能补；不可预测部分只能靠反馈兜底。本章的分层结构之所以最好，是因为它同时覆盖了前两块：MPC 用模型补相位，PID 用积分消偏置。

反过来，如果一个系统的误差主要来自“不可预测的扰动”，那么加前馈、加预测都没有用，因为它们补的是可预测部分。这时候正确的做法是提高反馈增益或加扰动观测器，而不是加一个更复杂的混合结构。**归因错了，结构再好也只是在补一块不存在的误差。**

这条判据还能解释第 66–71 章各自的位置：模糊规则适合“误差大小与最佳增益的关系明确”的场合；神经网络适合“非线性强、机理不清”的场合；强化学习适合“前馈形状未知但可试错”的场合；迭代学习适合“轨迹重复”的场合。本章不新增机制，只负责把它们接到同一个回路上——所以选型的第一步永远是先回到那几章，问清“这个系统属于哪一类”。

三个“不要用”

第一种不要用的情形是**计算预算放不下**。分层结构在本章占 3.45%，看着宽裕，但它依赖 10 ms 的周期。若系统必须跑在 1 ms，占比变成 34.5%，再叠加通信与日志就会超时；此时正确的选择是用解析式的预测补偿（例如第 47 章的预估器思路），而不是硬塞一个 MPC。

第二种不要用的情形是**模型不可信且无法在线辨识**。分层结构的所有收益都建立在“模型能预测未来”上，模型错了，收益会反转。72.6 节那行 MPC(12)+残余学习 0.1711 就是榜样：它比纯 PID 差 43%，原因是把模型误差学进去了。若对象时变严重又没有辨识手段，宁可退回增益调度，因为调度的最坏情况有界。

第三种不要用的情形是**验证工况覆盖不了**。四十个参数的混合结构，需要六种以上工况才能验收。若实际可测的工况只有两三种（例如大型装置一年只开几次），参数一多就无法验证，此时应当把结构砍到“参数少到能被人工审查”的程度。

这三种情形有一个共同点：**它们都不是“算法不好”，而是“条件不具备”**。混合架构的适用边界，本质上是一条工程边界，而不是数学边界。

三种典型工况的选型实例

把上面的判据落到具体场景，差别会更容易看出来。下面三个场景都用本章算例的变形，数字可以直接对照。

场景一，注塑机料筒温度。对象近似一阶惯性加纯延迟，延迟十几秒、时间常数几百秒，扰动主要是环境温度与料流量的慢漂。这类系统的误差几乎全部是稳态偏置，换向滞后可以忽略——因为它根本没有往复轨迹。此时串级 PID 加进料量前馈就是最优解；模糊调度能改善一点大偏差段的响应；双模与 MPC 都没有用武之地，前者没有“高增益窗口”可利用，后者要预测未来十几秒，模型精度撑不住。选型的结论是“简单那一档就够”，对应本章表里 0.1133 那一行。

场景二，伺服压装。位置环在接触瞬间增益突变，行程短、节拍固定，要求“快到位且不超调”。误差主要是到位时间与过冲，而这两者在固定增益下互相矛盾——这正是双模切换最典型的适用场合：大误差段用高增益抢时间，小误差段用低增益防过冲。本章的 0.0814 与总变差 2.281 就是这个取舍的量化：误差降三成，总变差涨三成四。若节拍允许慢一点，把阈值抬高反而更稳，本章阈值 0.06、0.08 两行的 0.0835 与 0.0858 就是这个方向。

场景三，无人机姿态跟踪。对象快、模型较准、参考连续可微、执行器带宽有限。误差主要是相位滞后，且参考已知——这是分层 MPC 的主场。本章 0.0310 与纯 PID 0.1161 的差距，几乎全部来自“提前看到未来”。代价也在这里：延迟从 5 拍变 8 拍，MPC(20) 从 0.0048 掉到 0.0538，退化十倍，只比双模的 0.0814 稍好；若延迟本身测不准，还不如老老实实用双模。

三个场景摆在一起能看出一条规律：**误差的“形状”决定结构，“形状”的稳定性决定能不能兑现**。串级适合偏置型误差，双模适合“两段互相矛盾”的误差，MPC 适合相位型误差；只要误差形状随工况漂移，任何结构的收益都会打折。

混合架构的成本清单

前面一直在算收益，这里把成本也列清。成本有四项，前两项要钱，后两项要人的时间。

第一项是算力。本章五个模块合计约 0.42 ms 每拍，占 10 ms 周期的 4.2%；看着不多，但它是“额外”的——纯 PID 只要 0.014 ms。第二项是内存与代码体积，300 行源码在嵌入式上占几 KB 到几十 KB，通常不是瓶颈，真正的瓶颈是它带来的审查工作量。第三项是验证：四十个参数、六种工况，每条轨迹都要跑、要存基线、要在每次改动后复跑。第四项是维护：三个月后接手的人要能看懂“为什么这里有个 $R = 0.20$”，这靠注释与文档，而注释与文档是最容易腐烂的部分。

四项里，第一项最容易被高估，后三项最容易被低估。工程上常见的错误是“算力够，所以上”，结果上线后每次改参数都要把六种工况重验一遍，维护成本把收益吃光。这也是 72.3 节把“第零步：先判断该不该混合”放在最前面的原因——那一步问的不是技术问题，而是“你打算为这套结构付多少维护费”。

与相邻章的分工

本章刻意没有重复推导任何单机制。PID 的三个参数怎么整定、为什么积分能消静差，见第 44 章；工程上的自整定与继电整定，见第 106 章。MPC 的预测模型、约束处理与稳定性证明，见第 58 章；工业现场的滚动优化实施细节，见第 80 章。纯延迟的 Smith 预估与它的改进型，分别见第 47 章与第 107 章。

本章只做一件事：把这些已经成立的机制装进同一个回路，然后回答“装在一起时谁负责什么、接口怎么定、会不会互相打架”。凡是涉及“某个机制单独怎么设计”的问题，都应回到对应章节；凡是涉及“两个机制同时在场时怎么办”的问题，才属于本章。

一句话收尾

回到 72.1 节那句：混合不是把多个控制器堆在一起，而是把不同性质的工作分给不同性质的模块，并为它们规定一个不会互相打架的接口。本章用同一套算例给出的全部证据，都可以压成三句：接口比参数重要（2 输入 2.8701，只参考 0.1057）；结构比形状重要（N_p 改一个数换两个数量级，调度表改九个格子换 2.4%）；流程比算法重要（三条工程底线挡住的事故，比任何算法改进都多）。

三句话指向同一个判断：**当你想加一个模块时，先问它带来了什么新信息；如果答不上来，就该去改接口，而不是加参数。**

72.12 Python 仿真代码

两个脚本的分工

本章的代码分两份，都不带命令行参数，双击即可运行。

`code/ch072_sim.py` 负责出数字。它按六段组织：对象与任务、三种混合形态（模糊调度、双模切换、神经-模糊）、预测层（延迟感知 MPC）、终端值学习、协同设计（权限、无扰切换、计算预算）、鲁棒性表。入口是末尾的 `report()`，一次跑完十二组实验，把每个指标的六个数（RMS、最大误差、IAE、总变差、饱和率、切换次数）打到终端。

`code/ch072_figs.py` 负责出图。它导入 `ch072_sim`，复用同一批类与常量，不重新定义任何控制器——这一点很重要：如果出图脚本自己抄一份控制器，图与正文的数字就会悄悄分叉。十张图全部由 `styles/plot_styles.py` 与 `styles/block_styles.py` 绘制，`save()` 同时导出 SVG 与 PDF。

对象与闭环骨架

对象是最短的那一段，却决定了后面所有实验的口径。

```
class Plant:
    """二阶非线性对象 + 纯延迟 + 输入限幅。"""

    def __init__(self, gain_scale=1.0, delay=DELAY, load=0.0):
        self.gain_scale, self.delay, self.load = gain_scale, delay, load
        self.x1, self.x2 = R0, 0.0
        self.buf = [0.0] * delay

    def step(self, u):
        if self.delay > 0:
            self.buf.append(u)
            up = self.buf.pop(0)
        else:
            up = u
        up = up + self.load
        for _ in range(SUB):
            kk = self.gain_scale * gain_at(self.x1)
            self.x2 += TS / SUB * (-2.0 * self.x2 + kk * up)
            self.x1 += TS / SUB * self.x2
        return self.x1
```

`buf` 是一个先进先出的列表，长度等于延迟拍数，这就是 72.5 节式 (72-16) 里 u_{k-d} 的实现。`SUB=5` 是每个采样周期内的子步数，用五次欧拉代替一次，把积分误差从 $(T_s)^2$ 压到 $(T_s/5)^2$；这样做不是为了精度，而是为了让“控制器每拍只算一次”这个前提在仿真里也成立。

闭环的骨架只有十来行，所有控制器都通过同一个接口接入：

```
def run(ctrl, gain_scale=1.0, delay=DELAY, noise=0.0, load=0.0, seed=0, trace=None):
    rng = np.random.default_rng(seed)
    t = np.arange(N) * TS
    r, rd = ref_traj(t), ref_vel(t)
    plant = Plant(gain_scale, delay, load)
    ctrl.reset()
    y_prev, ie, dy_f = plant.x1, 0.0, 0.0
    for k in range(N):
        ym = plant.x1 + noise * rng.standard_normal()
        e = float(r[k] - ym)
        dy_f += (TS / (TAU_RATE + TS)) * ((ym - y_prev) / TS - dy_f)
        y_prev = ym
        ie = float(np.clip(ie + e * TS, -0.5, 0.5))
        s = dict(e=e, ed=-dy_f, ie=ie, y=float(ym), r=float(r[k]),
                 rd=float(rd[k]), k=k)
        u = float(np.clip(ctrl.step(s), UMIN, UMAX))
        if trace is not None:
            trace(k, ctrl, s)
        plant.step(u)
```

三个细节对应正文的三条结论。`dy_f` 是式 (72-20) 那个 30 ms 一阶低通，它把式 (72-19) 的差分噪声压下去；`ie` 被限在 ± 0.5，是积分抗饱和；`np.clip` 只在这里出现一次，对应 72.10 节“限幅只做一次”。最后的 `trace` 回调不参与控制，只用于出图时记录模块内部量——72.10 节说的“可观测性”，在仿真里就是这一个参数。

四个控制器的最短实现

四个控制器都继承或复用了 `PID`，差别只在 `step()` 里怎么组合。并联结构最能说明“接口”这件事：

```
class Parallel(PID):
    """并联混合：PID 基线 + 神经网络补偿，补偿量被限在 ±klip。"""

    def __init__(self, net, gains=FB, klip=0.40):
        super().__init__(gains)
        self.net, self.klip = net, klip

    def step(self, s):
        return super().step(s) + float(np.clip(self.net.step(s), -self.klip, self.klip))
```

整个结构就是一行加法加一次限幅。72.6 节说“并联的稳定性由限幅决定”，在代码里就是那个 `np.clip`；把 `klip` 改成 `None`（即去掉限幅），2 输入网络那组会直接把闭环推到饱和。这类“一句话就能改坏”的写法是本章有意保留的——读者可以自己试，试完就知道限幅不是装饰。

MPC 的核心是脉冲矩阵。式 (72-17) 落到代码里是一个三重循环，看着笨，但每一步都能与公式对上：

```
def _pulse(self, K0):
    """G[i,j]：第 j 个控制块对第 i 拍输出的贡献（延迟 delay-1 拍生效）。"""
    G = np.zeros((self.Np, self.Nu))
    for j in range(self.Nu):
        y = v = 0.0
        for i in range(self.Np):
            m = i - (DELAY - 1)
            u = 1.0 if (m >= 0 and self._block_of(m) == j) else 0.0
            for _ in range(SUB):
                v += (TS / SUB) * (-2.0 * v + K0 * u)
                y += (TS / SUB) * v
            G[i, j] = y
    return G
```

注意 `m = i - (DELAY - 1)` 这一行：当 $i < 4$ 时 $m < 0$，`u` 恒为 0，于是 G 的前四行全零。72.5 节说的“零三角”，在代码里就是这四行。也正因为如此，$N_p = 4$ 时 G 整体为零，梯度恒为零，控制量一动不动——那个 0.000 的总变差不是调出来的，是这四行算出来的。

指标是怎么算出来的

正文里反复出现的四个数，在 `run()` 之后由一段十几行的汇总算出，口径必须先说清，否则不同章节的数字没法比。

RMS 是误差的均方根，取四百拍的平均，它衡量“平均偏差有多远”，对少数几拍的尖峰不敏感。最大误差取绝对值最大的一拍，它衡量“最坏时刻有多糟”，工程上它往往比 RMS 更重要，因为超差报警看的是它。IAE 是误差绝对值的积分，等价于 $\sum |e_k| T_s$，它与 RMS 的区别在于对大误差的惩罚更线性，用来与经典文献对照。总变差 TV 定义为 $\sum |u_k - u_{k-1}|$，它衡量的不是精度而是**执行器的负担**：同样的 RMS，TV 大的那个会让阀芯、电机、减速器磨损得更快。

本章用 TV 而不是用控制量的方差，是因为方差对慢漂不敏感——一个缓慢爬升的控制量方差可能很小，但总变差很大，而后者才是磨损的直接来源。这也是为什么双模与分层在表里“精度更好、TV 更差”：它们用更多的执行器动作换精度，这笔交易必须显式写出来，不能只报 RMS。

饱和率是控制量等于限幅值的拍数占比，它衡量“还有没有余量”。本章只有 2 输入 TSK 那一组出现 30.2%，它同时也是唯一一组失稳的——这不是巧合：一旦控制量长期顶在限幅上，反馈回路就被切断，误差只能靠对象自身衰减，而本章对象有两个极点，衰减得很慢。

切换次数只对双模结构有意义，它统计模式翻转的次数。这个数本身不是性能指标，而是“抖振程度”的代理：7 次意味着平均每 0.6 s 翻一次，肉眼可见；若这个数涨到几十次，即使 RMS 好看，机械上也已经不可接受。

四个数（外加切换次数）必须同时报，缺一个就会误判。这一点在写作上也有用：本章每一处“某结构更好”，后面都跟着另外两三个数的代价，读者可以自己判断这笔交易值不值。

训练集是怎么造的

神经-模糊那一组用的是监督学习，教师信号的选择直接决定结论，必须交代清楚。

教师取“基线 PID 加精确前馈”，即 $u^\star = \big(r'' + 2r'\big)/K(r)$ 叠加 PID 的输出，再限幅。这个教师有两个性质：一是它在标称工况下确实能把误差压到很小，二是它是**可计算的**——不依赖任何优化，任意采样点都能算出标签。用 MPC 当教师也能得到更优的标签，但每采一个样本要解一次优化，2400 个样本的成本就上去了，而且标签本身带求解器噪声。

样本的采法是在参考轨迹上均匀取点，每个点记录三样东西：误差 e、误差变化率 $\dot e$、参考位置 r，标签是教师在这一点的输出。训练 1200 个、测试 1200 个，两者不重叠。这个规模很小，小到足以让“9 个参数不够用”这件事暴露出来——这正是本章想要的结果：如果样本多到能掩盖结构缺陷，负面结论就看不到了。

三个输入变体的差别也在这里。2 输入（e、$\dot e$）的测试 RMSE 0.0893，3 输入（加上 r）0.0243，差 3.7 倍；但闭环里差距被放大成 2.8701 对 0.1302，二十二倍。原因是开环误差会在闭环里累积：一个在训练集上“平均差 0.09”的网络，接进回路后每一拍的偏差都会变成下一拍的输入偏差，最终把工作点推出训练分布，输出一路漂到限幅。这也是 72.7 节那条“开环精度不等于闭环性能”的由来。

残差形式的三个变体则说明另一件事：把网络放在 PID 之上学“还差多少”，比让它直接输出控制量安全得多。残差只参考位置那一组的闭环是 0.1057，反而优于纯 PID 的 0.1161——它其实退化成了一个一维查表，等于用九个参数记住了“相位该提前多少”。这条结果与 72.1 节的三档阶梯是同一个事实的两种说法。

十张图怎么来的

出图脚本的 `main()` 先把数据采齐，再逐张调用绘图函数。数据采集复用 `run()`，只是多传一个 `trace`：

```
hist = []
rec_fz = cs.run(cs.FuzzyPID(), trace=lambda k, c, s: hist.append(c.last))
modes = []
rec_dual = cs.run(cs.DualMode(hyst=0.005, dwell=20),
                  trace=lambda k, c, s: modes.append(1 if c.pd_mode else 0))
nf = cs.exp_neurofuzzy()
```

图 72-3 的三条系数曲线来自 `hist`，图 72-4 的模式阶梯来自 `modes`，图 72-5 的 27 个结论来自 `nf["nets"]` 里那个已训练的网络。这样做的好处是“图与数字同源”：正文里写的 0.955–1.164 就是图 72-3 纵轴的实际范围，不存在两套数据。

框图（图 72-1 的四种拓扑）用 `block_styles`，语义色板固定为“控制器、对象、传感器、执行器、虚拟模块”五类，线宽统一 0.6 pt，坐标系 1 单位等于 1 cm。这样做是为了四个面板的尺寸严格一致——比较结构时，图本身的差异必须是零。

所有图内文字遵守一条规则：**中文写纯文本，公式单独用 `$...$`**，两者不写进同一个字符串。这条规则不是排版洁癖，而是字体回退的硬约束：中文字体没有数学符号的字形，混排会触发 `glyph missing`，导出的 PDF 里对应字符变成空白。

复现与改参数

想改算例，有三处必须先想清楚。

第一，改 `T_EP`（轨迹周期）会同时改变参考的主频与三次谐波频率，延迟造成的相位滞后随之改变。72.4 节那两个 4.5°、13.5° 是按 4 s 算的，改成 2 s 会翻倍。

第二，改 `DELAY` 必须同步改 MPC 的 `Np`，否则会落进 72.5 节那个“零三角吃掉全部梯度”的陷阱。代码里 `DELAY` 是全局常量，`Np` 是每个 MPC 实例的参数，两者没有自动联动——这是有意的，因为工程上延迟变化时，人必须显式决定新时域。

第三，改 `FB`（基线 PID）必须同步改调度表的中心 1.0。72.7 节说过，调度系数是相对量，中心一变，绝对增益就变；本章的最优固定增益是 $(2.60,\ 0.02,\ 0.16)$，若把基线换成它，调度表的上界 1.30 就会把闭环推到 3.38，需要同时收窄范围。

这三处都是“改一个数，动三个地方”的耦合。它们不是代码写得不好，而是混合架构本身的属性——72.7 节那一小节讲的就是这件事。

还有一处最容易踩、却最不显眼的坑：**随机种子**。本章除噪声实验以外全部无随机，所以重跑逐位一致；一旦有人把训练样本的采样改成随机取点而不固定种子，2 输入与 3 输入两组的 RMSE 会各自漂动百分之一二，闭环结果可能从 2.8701 变成 2.9 或 2.8，正文里“二十二倍”的说法就不再成立。所以复现的前提不是“跑一遍脚本”，而是“在同一个种子上跑一遍脚本”——这条在 72.10 节的参数版本管理里对应“可复现”三个字。

另一处是**图的坐标系**。本章十张图全部由 `styles/` 里的统一函数绘制，尺寸与字号固定，所以正文里“图 72-3 纵轴范围 0.955–1.164”可以被读者直接量出来。若有人改了图幅而不改字号，坐标刻度会变稀，量出来的数与正文就会差一点点——差得不多，但足以让核对变成一件需要解释的事。

最后一句关于代码的：本章两个脚本都刻意不超过三百行，控制器不做类层次设计，指标不做通用框架。这不是偷懒，而是为了让读者能在半小时内把每个数字的来源读完。混合架构的问题从来不在代码写不出来，而在“写出来之后没人说得清它在做什么”。

第73章 输出分布控制（SDC）

73.1 动机与直观理解

大多数控制回路只盯一个数：给定值、测量值、偏差。控制器把这个数按住，工艺就认为“稳了”。但相当多的质量指标并不是一个数，而是一条分布曲线。纤维直径、涂层厚度、磨粒粒度、燃烧温度、电池单体容量，这些量的合格与否取决于**整条分布**落在公差带内的比例，而不是平均值是否等于设定值。

两条分布可以有完全相同的均值，不合格率却差一个数量级以上：一条窄而集中，另一条宽而拖尾。只控均值的回路对后者无能为力，因为它看不见“宽”这件事。输出分布控制（stochastic distribution control，SDC）要解决的正是这一类问题：把整条分布当作被控量，让它在形状上逼近工艺给定的目标分布。

73.1.1 一个可以算的例子

本章用一个双批次加工场景贯穿全章。同一台设备交替加工两类工件，两类工件对同一输入 u 的增益不同：低增益批次约 0.160，高增益批次约 0.240，两类占比 60% 与 40%。设备的动态近似为一阶惯性，离散化后时间常数 0.852，采样周期 $T_s = 0.1$ s。因此稳态输出满足

$$y_\infty = \frac{b_i}{1 - a} u \tag{73-1}$$

其中 b_i 取上面两簇之一。同一输入下，两类工件的稳态位置天然相差 1.5 倍，输出分布因此呈双峰。

工艺给出的目标是：两类工件分别集中在 3.20 mm 与 4.85 mm 附近，峰宽（标准差）约 0.28 mm，两类占比 60% 与 40%。初始时刻所有工件都停在 1.20 mm 附近，分布是单峰窄带。控制目标不是把均值从 1.2 拉到 3.86，而是把这条单峰窄带**搬移、展宽、分叉**成指定的双峰形状。

这个例子把 SDC 与常规控制的差别摆得很清楚。均值控制只需要把平均位置推到 3.86，它对“两类工件各占多少、峰间距是否 1.65 mm”没有任何约束；分布控制则要求整条曲线的形状对得上。图 73-1 给出闭环下分布随时间的演化，灰度越深表示该输出值附近的工件越密集。

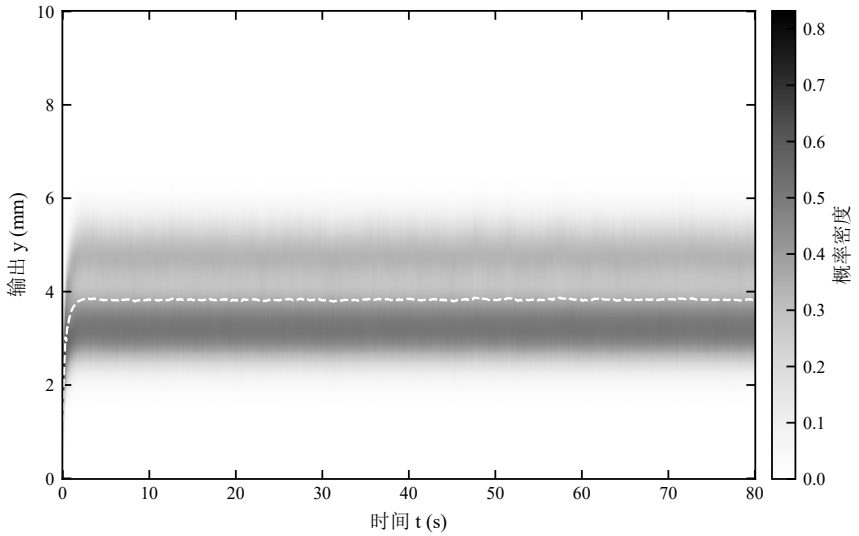


图 73-1　闭环输出分布随时间的演化

73.1.2 为什么不能直接控分布

把分布当被控量，第一个障碍是它是无穷维的：任意时刻的输出分布是一条函数曲线，控制器无法直接处理。工程上的做法是把它投影到有限维基函数上，用一组系数（权向量）代表整条曲线。本章选用三次 B 样条，因为它有三个好处：非负、单位分解、局部支撑。

- **非负**：基函数本身不小于零，权向量非负就保证密度非负，不需要额外约束。
- **单位分解**：全部基函数之和恒为 1，权向量之和为 1 就自动满足概率归一，不需要再做归一化。
- **局部支撑**：每个基函数只在一小段区间上非零，改动一个权值只影响局部形状，便于定位“哪一段分布没跟上”。

图 73-2 画出 20 个三次基函数在输出区间上的形状：两端被钳位（端点处只有一个基函数取 1），中间等距排布，相邻基函数在节点处平滑搭接。

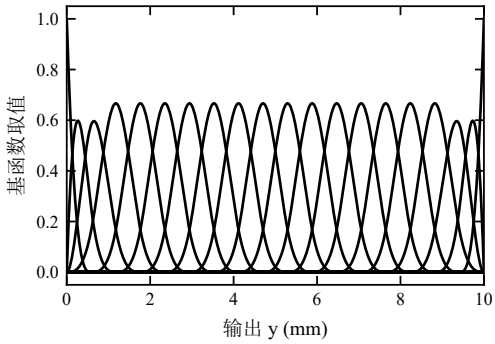


图 73-2　三次 B 样条基函数（n=20）

73.1.3 控制回路长什么样

投影之后，整条分布变成 20 维向量 $\boldsymbol{v}$，被控对象也随之变成“权向量如何随输入演化”的问题。本章用一阶线性模型近似这一演化：

$$\boldsymbol{v}[k+1] = \boldsymbol{A}_v \boldsymbol{v}[k] + \boldsymbol{B}_v u[k] + \boldsymbol{c}_v \tag{73-2}$$

其中 $\boldsymbol{A}_v$ 描述分布自身的迁移与扩散，$\boldsymbol{B}_v$ 描述输入对分布的作用方向，$\boldsymbol{c}_v$ 是噪声核带来的常数偏置。三个量都由现场数据辨识得到，不依赖机理建模。

有了这个模型，控制律的推导就退化成一次带约束的最小二乘：选择 $u[k]$，使下一步的预测分布尽量接近目标分布 $\boldsymbol{v}_g$，同时不让输入越限。图 73-3 给出闭环结构：目标分布经基函数分解得到 $\boldsymbol{v}_g$，与在线估计的 $\boldsymbol{v}[k]$ 一起送入控制器，控制器按模型反解 u，作用到随机对象上，输出分布被重新采样、重新估计，形成闭环。

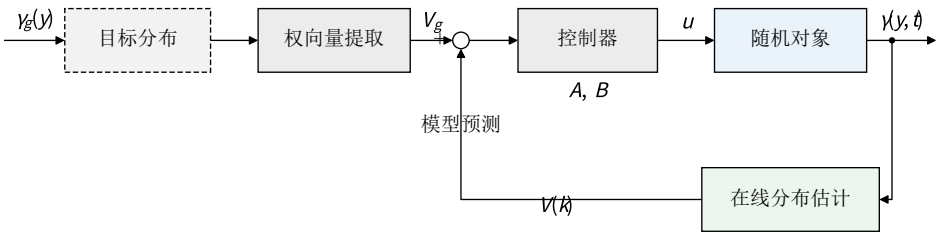


图 73-3　输出分布控制的闭环结构

73.1.4 本章的数字基线

为便于后续各节对照，先把本章的基准结果列在这里。表 73-1 中的数字全部出自 §73.12 的脚本，2000 个蒙特卡洛粒子，20 个基函数，目标为 3.20/4.85 双峰。

表 73-1　三种控制策略的基准指标

方案	终点分布 L1 误差	分布 IAE（mm·s）	输入范围	终点均值
开环 $u = 3.0$	0.0532	5.719	3.00–3.00	3.876
均值 PI	0.0466	6.924	2.46–3.41	3.860
SDC	0.0473	5.332	1.98–5.29	3.839

三行数字里有一个必须提前说明的事实：**终点误差上三者相差不大**。原因是本对象的稳态分布族只有一个自由度——输入只改变分布的整体尺度，不改变峰间距比例；一旦均值对齐，整条稳态分布也就对齐了。SDC 的优势体现在过程量上：分布 IAE 比均值 PI 低 23.0%，比开环低 6.8%，因为它在过渡段同时管住了峰位与峰宽，而不是等分布自己慢慢收敛。

这个事实直接决定了本章的适用边界（§73.11 展开）：单输入、稳态分布族单参数的场合，SDC 主要改善过渡品质；当输入维数大于一、或目标分布不在对象可达的分布族内时，分布控制才是不可替代的。§73.10 会给出后一种情形的实测对比：把峰位比改成对象做不到的 1.433（固有 1.5），三种方案的终点 L1 分别是 SDC 0.1478、均值 PI 0.1334、开环 0.3699，谁也无法精确达标，但开环差得最远，而 SDC 的过渡 IAE 仍比 PI 低 9.8%。

73.1.5 位置、尺度、形状：三个层次

现场说“分布不对”，往往混着三件事：位置偏了、宽度不对、形状不对。这三件事在矩上分得开，在控制手段上也分得开，先分清层次再谈方案，比直接上算法省事得多。

一阶矩与二阶中心矩都可以由权向量直接算出。因为 ϕ_i 本身是概率密度，它的重心 $m_i = \int y\phi_i(y)\mathrm{d}y$ 就是第 i 个基函数的“代表位置”，于是 $\mu = \sum_i v_i m_i$；同记 $s_i = \int y^2\phi_i(y)\mathrm{d}y$，有 $\sigma^2 = \sum_i v_i s_i - \mu^2$。两式都是权向量的线性函数，一次矩阵乘即可，在线诊断时可以每拍打印出来，不必等分布图。

三个层次的判别与手段分得清：

- **位置**：μ 偏。常规 PI 就能处理，不需要分布控制。本章对象把均值从 1.20 搬到 3.86，均值 PI 同样做到（表 73-1 的终点均值 3.860）。
- **尺度**：σ 偏。要有能改变分布宽度的通道。本章对象的峰宽由 $\sigma_c/\sqrt{2p_i}$ 决定、与输入无关（式 73-20），单输入改不了峰宽；闭环里看到的“展宽”是两簇被拉开造成的表观展宽，不是把每个峰本身变宽。
- **形状**：峰数、峰高比、偏度不对。这是 SDC 唯一不可替代的场合，也是最难的一档，它要求输入能相对独立地推动分布的不同区段。

这个分层有一个很实用的推论：**先确认 μ 与 σ 是否已经被现有回路管住，再决定要不要上分布控制。** 如果 μ 还在漂、σ 也没人管，直接上 SDC 只会把两个老问题藏进一个新指标里——L1 降不下来，而现场说不出到底哪一段错了。

73.1.6 例子为什么选双批次

例子不是随手挑的，它同时满足三个条件，缺一个就演示不出本章的要点。

形状信息必须落在均值与方差之外。 若目标是一个单峰正态，任何单输入都能把均值推到位置，控制问题随即退化成“调一个数”，SDC 与 PI 的结果会几乎重合，读者看不出区别。双峰目标不同：峰间距与峰高比这两个量，均值和方差都描述不了，必须整条曲线才能表达。本章基线里 SDC 与 PI 的终点 L1 只差 0.0007，但峰位偏差 0.025 与 0.050 mm、峰间距 1.500 mm 这些量，只有分布控制才在管。

非高斯的来源要有物理依据。 双峰来自两簇增益不同的工件（0.160 与 0.240，占比 60% 与 40%），这是真实产线上“两类来料混线”的常见情形，不是人为造一个奇怪的分布去难为控制器。物理来源清楚，模型参数才有解释力，也才能在投

运后判断“占比变了该怎么办”（§73.10.6）。

可达性要有一个干净的判据。 两簇的稳态位置比恒为 $b_2/b_1 = 1.5$，与输入无关（式 73-20）。这条硬约束让“目标可不可达”变成一个除法：目标峰位比与 1.5 差多少，终点误差的下限就在那里。§73.10.4 的 4.5% 偏差正是这样被判死的。

若换成对数正态、重尾这类目标，三个条件里的第三个就不成立：峰位没有解析表达，可达性只能靠数值扫描（§73.8.5 实验一）判断，结论也没那么干脆。这就是本章选双峰的理由——它把方法的能与不能都摆在明面上。

73.1.7 一句话核心与阅读路线

如果只记一句话：**分布控制的本质是把“一条曲线”投影成“一组系数”，再把“系数怎么随输入变化”辨识出来，最后用这个模型反解输入；它买到的是形状，付出的是模型与样本。**

本章路线：§73.2 给出基函数表示、权向量投影与控制律推导；§73.3 把设计固化为七步；§73.4 讨论连续时间下的 Fokker-Planck 描述与离散设计的对应；§73.5 给出离散实现与采样周期约束；§73.6 分析闭环稳定性与性能指标；§73.7 逐个解释基函数个数、阶次、参考滤波系数、输入权重的物理作用；§73.8 给出调参顺序与常见错误；§73.9 讨论积分补偿、加权跟踪与约束优化；§73.10 讲工程落地的样本量、在线计算与模型维护；§73.11 说明能力边界；§73.12 给出可复现代码。

阅读本章需要三个前置概念：一阶惯性对象的离散化（第 6 章）、最小二乘辨识（第 18 章）、概率密度与期望的定义（第 8 章）。本章不重复这些推导，只引用结论。

73.2 数学原理

本节把 §73.1 的直观说法写成公式。顺序是：先定义分布表示，再定义权向量的动态，然后推导控制律，最后交代这些式子成立的前提。

73.2.1 密度的基函数表示

设输出 y 的取值范围为 $[y_{\min}, y_{\max}]$，在其上放置 n 个三次 B 样条基函数 $B_i(y)$，$i = 1, \dots, n$。基函数满足非负与单位分解：

$$B_i(y) \geq 0, \qquad \sum_{i=1}^{n} B_i(y) = 1, \quad y \in [y_{\min}, y_{\max}] \tag{73-3}$$

定义归一化基函数

$$\phi_i(y) = \frac{n}{y_{\max} - y_{\min}} B_i(y) \tag{73-4}$$

则每个 ϕ_i 在区间上的积分恰为 1。输出分布写成

$$p(y, t) = \sum_{i=1}^{n} v_i(t) \phi_i(y), \qquad \sum_{i=1}^{n} v_i(t) = 1, \quad v_i \geq 0 \tag{73-5}$$

权向量 $\boldsymbol{v}(t) = [v_1, \dots, v_n]^{\mathrm{T}}$ 就是分布的有限维替身。由 $\int \phi_i \mathrm{d}y = 1$ 可知 $\int p \, \mathrm{d}y = \sum_i v_i = 1$，归一化是自动成立的。

权向量的几何含义值得说清楚。因为 $\sum_i B_i(y) = 1$，把 B_i 看作一组“帽子”，v_i 就是分布在第 i 个基函数支撑区间上的加权平均高度。若用 N 个样本 $\{y_j\}$ 估计，最自然的估计量是

$$\hat{v}_i = \frac{1}{N} \sum_{j=1}^{N} B_i(y_j) \tag{73-6}$$

它无偏，且自动满足和为 1。图 73-4 把 20000 个样本的经验直方图与式 (73-5) 的拟合曲线叠在一起，左侧是初始单峰分布，右侧是目标双峰分布；两条曲线的偏差主要出现在峰值附近，这是三次基函数在尖峰处的系统平滑效应。

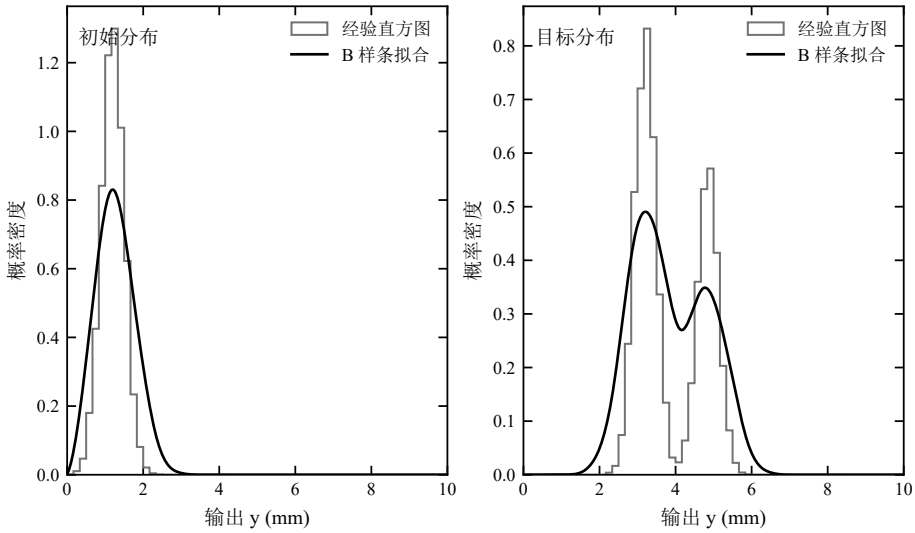


图 73-4　经验直方图与 B 样条拟合

73.2.2 权向量的动态模型

把每个采样时刻的权向量排成序列，假定它满足一阶仿射递推：

$$\boldsymbol{v}[k+1] = \boldsymbol{A}_v \boldsymbol{v}[k] + \boldsymbol{B}_v u[k] + \boldsymbol{c}_v + \boldsymbol{\varepsilon}[k] \tag{73-7}$$

其中 $\boldsymbol{A}_v \in \mathbb{R}^{n\times n}$，$\boldsymbol{B}_v \in \mathbb{R}^{n}$，$\boldsymbol{c}_v \in \mathbb{R}^{n}$，$\boldsymbol{\varepsilon}$ 为辨识残差。常数项不能省：噪声核每个采样周期都会把分布抹平一点，这一项在 $\boldsymbol{v}$ 上的投影是常向量，把它塞进 $\boldsymbol{B}_v u$ 会让输入增益的估计产生偏置。

模型参数用带岭惩罚的最小二乘估计。把 N 组数据写成矩阵 $\boldsymbol{M} = [\boldsymbol{v}[0], \dots, \boldsymbol{v}[N-1]]^{\mathrm{T}}$ 与 $\boldsymbol{u} = [u[0], \dots, u[N-1]]^{\mathrm{T}}$，中心化后求解

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} \; \|\boldsymbol{Y}_c - \boldsymbol{X}_c \boldsymbol{\Theta}\|_F^2 + \lambda \|\boldsymbol{\Theta}_{1:n}\|_F^2 \tag{73-8}$$

其中 $\boldsymbol{X}_c = [\boldsymbol{M}_c, \boldsymbol{u}_c]$，$\boldsymbol{Y}_c$ 是下一步权向量的中心化矩阵，惩罚只加在 $\boldsymbol{A}_v$ 对应的列上，输入通道不惩罚。辨识结果如下表（500 步随机游走激励，$n = 20$）：

量	数值	说明
残差 RMS	0.01019	单步预测的均方根误差
$\rho(\boldsymbol{A}_v)$	0.98484	谱半径，接近 1 表示分布收敛慢
$\|\boldsymbol{B}_v\|_2$	0.02536	输入对分布的作用强度
$\|\boldsymbol{c}_v\|_2$	0.29652	噪声核造成的常数偏置

谱半径接近 1 不是坏事，它对应“对象时间常数 0.852、分布收敛比状态更慢”这一物理事实；但它也意味着模型的稳态点对参数误差很敏感，这一点在 §73.10 会变成一个必须处理的工程问题。

73.2.3 一步预测控制律

控制律的出发点是最小化“下一步分布与目标的距离”，同时对输入偏离稳态值的部分施加惩罚：

$$\min_{u[k]} \; J = \|\boldsymbol{v}[k+1] - \boldsymbol{v}_g\|_2^2 + \rho \, (u[k] - u_{\mathrm{ref}})^2 \tag{73-9}$$

约束为式 (73-7)。把约束代入并对 u 求导，得闭式解

$$u[k] = u_{\text{ref}} + \frac{\boldsymbol{B}_v^{\mathrm{T}}\left(\boldsymbol{v}_r[k+1] - \boldsymbol{A}_v\boldsymbol{v}[k] - \boldsymbol{c}_v - \boldsymbol{B}_v u_{\text{ref}}\right)}{\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v + \rho} \tag{73-10}$$

这里已经把“目标”换成了参考权向量 $\boldsymbol{v}_r$，原因在 §73.2.4 说明。式 (73-10) 的结构值得逐项读：

- 分子是“还差多少”在输入方向 $\boldsymbol{B}_v$ 上的投影。只有这一投影能被 u 改变，与 $\boldsymbol{B}_v$ 正交的误差分量一步之内动不了。
- 分母是输入方向的“长度”，ρ 是对输入的软化。$\rho = 0$ 时这是沿 $\boldsymbol{B}_v$ 的最小范数解；ρ 越大，同样的误差只肯出一部分力，闭环变慢但更抗噪。
- u_{ref} 是稳态输入，它保证当 $\boldsymbol{v} = \boldsymbol{v}_g$ 时控制器输出恰好等于维持目标分布所需的输入，而不是零。

式 (73-10) 里的 u_{ref} 看似必须单独计算，其实可以精确消去。§73.2.5 会证明：把稳态条件代入并令 $\rho = 0$，u_{ref} 在分子与外面的加项中正好抵消，得到

$$u[k] = \frac{\boldsymbol{B}_v^{\mathrm{T}}\left(\boldsymbol{v}_r[k+1] - \boldsymbol{A}_v\boldsymbol{v}[k] - \boldsymbol{c}_v\right)}{\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v} \tag{73-11}$$

这就是 §73.12 代码真正执行的那一行。它的读法是：把“下一步还差多少”投影到输入方向，再按输入方向的长度折算成控制量；稳态输入被吸收进 $\boldsymbol{c}_v$ 与 $\boldsymbol{v}_r$，不再显式出现。

式 (73-11) 里也没有留下 ρ。原因是本对象的 $\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v$ 只有 6.4×10^{-4} 量级（由 $\|\boldsymbol{B}_v\|_2 = 0.02536$ 平方得到），任何 10^{-3} 以上的 ρ 都会主导分母，把控制量压掉一半以上，并在稳态留下明显偏差。ρ 在这里失去“软化”的调节意义，只剩数值正则作用，因此代码默认取 0。

73.2.4 参考权向量的滤波

式 (73-11) 若直接用 $\boldsymbol{v}_g$，等价于要求“下一步就到位”。在本对象上这会把输入推到限幅：初始分布中心在 1.20，目标中心在 3.86，一步搬完需要的输入远超执行器上限，控制器只能饱和，然后过冲、反向、再饱和，形成振荡。

做法是把目标包一层一阶滤波，令参考权向量按

$$\boldsymbol{v}_r[k+1] = \boldsymbol{v}_r[k] + \alpha\left(\boldsymbol{v}_g - \boldsymbol{v}_r[k]\right), \qquad 0 < \alpha \le 1 \tag{73-12}$$

演化，控制律中的 $\boldsymbol{v}_g$ 全部换成 $\boldsymbol{v}_r[k+1]$。α 是“每步允许走完剩余距离的比例”：$\alpha = 1$ 退化为直接跟踪目标，$\alpha = 0.1$ 表示每步只走一成。

滤波解决了两件事。第一，把“形状误差”分解成许多小步，避免单步大输入触发饱和。第二，α 与对象动态解耦：即使对象时间常数变了，闭环的收敛节奏仍由 α 主导，调试时只需要动一个旋钮。

代价是收敛变慢。§73.6 的实测给出这组折中：α 从 0.10 增到 0.50，分布 IAE 从 5.635 降到 3.913，但输入峰值从 3.117 涨到 8.000（触到限幅）；$\alpha = 1.00$ 时 IAE 只再降到 3.867，说明改进已经饱和，而输入仍在限幅上。本章取 $\alpha = 0.25$ 作为折中点。

73.2.5 稳态输入为什么可以消去

稳态输入 u_{ref} 由“目标分布是模型平衡点”这一条件确定：

$$\boldsymbol{v}_g = \boldsymbol{A}_v\boldsymbol{v}_g + \boldsymbol{B}_v u_{\text{ref}} + \boldsymbol{c}_v \tag{73-13}$$

这是 n 个方程、1 个未知数的超定方程，取最小二乘解

$$u_{\text{ref}} = \frac{\boldsymbol{B}_v^{\mathrm{T}}\left(\boldsymbol{v}_g - \boldsymbol{A}_v\boldsymbol{v}_g - \boldsymbol{c}_v\right)}{\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v} \tag{73-14}$$

把式 (73-14) 代回式 (73-10)、令 $\rho = 0$ 并取 $\boldsymbol{v}_r = \boldsymbol{v}_g$，分子中的 $\boldsymbol{B}_v u_{\text{ref}}$ 与括号外的 u_{ref} 恰好抵消，只剩式 (73-11)。这一步的工程意义是：稳态输入不必单独计算，它已经包含在 $\boldsymbol{A}_v\boldsymbol{v} + \boldsymbol{c}_v$ 这个“不给输入时对象会走到哪里”的项里。

反过来，如果模型里丢掉 $\boldsymbol{c}_v$，这项抵消就不成立，控制器会以为“分布不再变化时输入应当为零”。§73.12 的脚本把 $\boldsymbol{c}_v$ 强制置零单独跑了一遍，结果是：终点分布 L1 误差从 0.0473 恶化到 0.7507，分布 IAE 从 5.332 涨到 60.932，终点均值停在 2.993 而不是 3.86，输入在 0.86 到 4.25 之间反复找平衡。这个对照说明常数项不是可选的修饰，而是平衡点能不能对上目标的关键。

范数上 $\|\boldsymbol{c}_v\|_2 = 0.29652$ 比 $\|\boldsymbol{B}_v\|_2 = 0.02536$ 大一个数量级，但不能因此推断“噪声的影响是输入的十倍”。稳态解是 $(\boldsymbol{I} - \boldsymbol{A}_v)^{-1}(\boldsymbol{B}_v u + \boldsymbol{c}_v)$，而 $\boldsymbol{I} - \boldsymbol{A}_v$ 接近奇异，两个方向被放大的倍数并不相同，范数之间没有直接可比性。可以确定的只有一条：省掉 $\boldsymbol{c}_v$ 会让平衡点整体偏掉，偏掉的量足以毁掉整个闭环。

§73.1 表 73-1 里三个方案的终点误差相近，也从侧面说明式 (73-11) 的稳态项是对的：SDC 的终点均值 3.839 与开环 3.876、均值 PI 3.860 处在同一水平，残余差异来自模型残差与蒙特卡洛噪声，而不是稳态偏置。

73.2.6 评价指标

分布之间的距离用 L1 范数：

$$d_1\left(p_1, p_2\right) = \int_{y_{\min}}^{y_{\max}} \left| p_1(y) - p_2(y) \right| \mathrm{d}y \tag{73-15}$$

L1 距离有明确的概率解释：两条密度之间的 L1 距离等于“把一个分布变成另一个分布所需搬运的概率质量”的两倍。它的取值在 0 到 2 之间，0 表示完全相同，2 表示支撑集不重叠。

工程上更关心累计误差，因此再定义

$$J_{\mathrm{IAE}} = \int_0^{T} \left\| \boldsymbol{v}(t) - \boldsymbol{v}_g \right\|_1 \mathrm{d}t \tag{73-16}$$

离散实现中按采样周期累加，单位是 mm·s。式 (73-15) 在固定网格上数值积分，因此不同基函数个数下的数值可以横向比较；若直接用 $\sum_i |v_i - v_{g,i}|$，数值会随 n 增大而系统性变大，失去可比性，这是本节特意绕开的一个坑。

73.2.7 假设与成立条件

式 (73-7) 到式 (73-16) 成立依赖四条假设，每条都写明违背后的后果：

假设	内容	违背后的表现
A1 一阶 Markov	下一步权向量只依赖当前权向量与当前输入	模型残差出现自相关，闭环出现慢漂移
A2 有限维可表示	目标与过程分布能被 n 个基函数近似到可接受精度	无论怎么调，终点 L1 误差有下限
A3 输入线性	$\boldsymbol{B}_v u$ 能描述输入对分布的作用	大输入段增益失真，过渡段过冲或迟滞
A4 噪声核时不变	$\boldsymbol{c}_v$ 为常向量	工况变化后稳态点漂移，需要重新辨识或加积分

四条假设里，A3 最容易在工程中失效：执行器饱和、阀门死区、物料特性随批次变化，都会让“输入对分布的作用”变成非线性。§73.9 的积分补偿与 §73.10 的模型维护，都是围绕 A3、A4 的补救。

73.2.8 为什么 $\boldsymbol{A}_v$ 的特征值一定落在单位圆内

式 (73-7) 是一阶差分方程，但它的来源不是离散系统，而是连续密度演化的采样。把式 (73-19) 右端记成算子 $\mathcal{L}$，一个保持周期内无输入时的解是 $\gamma(t + T_s) = \mathrm{e}^{\mathcal{L} T_s}\gamma(t)$，投影到基函数坐标后就是 $\boldsymbol{A}_v$。$\mathcal{L}$ 是扩散型算子，特征值非正，因此 $\boldsymbol{A}_v$ 的特征值应落在 $(0, 1]$ 内。

这条性质给了辨识一个硬判据：**若辨识出的谱半径大于 1 而残差又很小，不要相信“对象自激”的解释，先去查数据里有没有未建模的外部漂移**（来料变化、环境温度、上游工序切换）。这些漂移会被回归当成“分布自己会走”，塞进 $\boldsymbol{A}_v$ 里，表现为特征值越界。本章第一次辨识就撞上过这种事：谱半径恰好等于 1.00000，闭环在目标点上算出零输入（表 73-6 第 1 轮）。

反过来，谱半径可以从连续时间的角度读。本章 0.98484 对应连续衰减率 $\ln 0.98484/0.1 = -0.153\ \mathrm{s}^{-1}$，时间常数 6.5 s，比对象本身的 $1/p = 0.625$ s 慢一个数量级。这个慢模态就是“输入推不动的那个方向”：它不参与闭环校正，只能自己衰减，所以 30 s 时终点 L1 仍在 0.05 附近随机游走（§73.5.3），而不是收敛到零。读懂了这一点，就不会把残余误差当成控制器没调好。

同一式子还解释了为什么改 T_s 必须重辨识：$\boldsymbol{A}_v(T_s) \approx \mathrm{e}^{\mathcal{L}T_s}$ 随周期指数变化，把 0.1 s 的模型直接用在 0.05 s 的控制器上，等于让模型认为分布比实际慢一倍，闭环先变迟钝、再在积分作用下过冲。

73.2.9 三种误差度量的分工

本章出现三个“误差”：权向量的 2 范数、固定网格上的 L1（式 73-15）、时间积分 IAE（式 73-16）。它们不是同一个量的三种写法，各有各的用途，混用会得出相反结论。

权向量 2 范数只用于模型体检。它的量纲是概率，数值与 n 弱相关，残差 RMS 0.01019 与权向量估计噪声 0.01099 同量级，这一条比较是成立的。

L1 用于横向比较不同方案、不同 n 的结果。它与权向量范数之间有一个上界：记 $\boldsymbol{G}$ 的元素为 $\int \phi_i\phi_j \mathrm{d}y$，则 $d_1 \le \sqrt{L\,\lambda_{\max}(\boldsymbol{G})}\,\|\Delta\boldsymbol{v}\|_2$。$\lambda_{\max}(\boldsymbol{G})$ 随 n 近似线性增长，所以同样的权向量偏差，在细网格上量出的 L1 大约按 $\sqrt{n}$ 放大。这解释了 §73.6.4 里 n 从 8 增到 36 时 L1 从 0.0150 涨到 0.0828 的大部分：涨了 5.5 倍，比 $\sqrt{n}$ 的 2.1 倍还多，多出来的部分是细网格暴露的真实表示误差——峰高被压低 42%，粗网格根本看不见这件事。

IAE 用于验收与方案排序。它把过渡段的误差按时间累加，单位 mm·s，能反映“快不快”。但 IAE 对末端的长尾很敏感：本章开环在 2 s 之后与 SDC 基本重合，IAE 仍高出 6.8%，差的就是前 1 s 那段。所以读 IAE 时要同时看 §73.6.3 的曲线，否则会误以为开环全程都差。

三条结论合起来：**体检看 2 范数，形状看 L1，快慢看 IAE；任何一项都不能单独当验收指标。**

73.2.10 为什么不用高斯混合做表示

双峰分布最自然的表示是高斯混合：三个参数一组（权重、均值、方差），两个分量就能精确描述本章的目标。B 样条要用 20 个系数才能近似，峰高还被压低 42%。看起来高斯混合更划算，为什么本章不用？

关键差别在“参数以什么方式进入模型”。B 样条表示下，密度对权向量是线性的：$\gamma = \sum_i v_i\phi_i$，权向量的演化可以直接用线性差分方程拟合，控制律也能解析反解。高斯混合不是：均值与方差藏在指数里，权向量一动，密度的变化既非线性也不局部，辨识要解非线性最小二乘，控制律要解非线性方程，每一步的收敛性都得单独证明。对一个要在产线上每 0.1 s 跑一次的控制器来说，这个代价太大。

还有一条更实际的理由：分量个数要事先定死。本章目标恰好两个峰，高斯混合取两个分量很合适；可工况一变（占比变、混入第三种来料），分量数就得改，整模型作废。B 样条的 n 只影响分辨率，不改变模型结构，占比变化只是权向量变了一个点，模型照用（§73.10.6）。

代价也要说清：B 样条的表示误差是系统性的，峰越尖、节点越疏，压低越多。这个误差不会随样本量增大而消失，只能靠加密节点换（§73.6.4）。所以选 B 样条等于选了“线性换精度”这条路，精度不够时用 n 补，而不是换表示。

73.2.11 三个容易看混的符号

本章字母重复使用较多，这里集中交代一次，后面不再解释。

- $B_i(y)$ 是标量基函数，随输出 y 变化；$\boldsymbol{B}_v$ 是 n 维常向量，是输入对权向量的作用方向。同一个字母 B，靠“有没有下标 v”区分，前者是函数，后者是向量。
- $\boldsymbol{v}$ 是当前权向量，$\boldsymbol{v}_r$ 是经参考滤波后的中间目标，$\boldsymbol{v}_g$ 是最终目标。三者关系由式 (73-12) 联系：$\boldsymbol{v}_r$ 以速率 α 追赶 $\boldsymbol{v}_g$，$\boldsymbol{v}$ 以一步预测追赶 $\boldsymbol{v}_r$。看公式时先确认是哪一个，否则会把“参考滤波改变了目标”当成“控制律有稳态误差”。
- $\boldsymbol{P}$ 是投影矩阵（式 73-25），$p(y,t)$ 与 $\gamma(y,t)$ 都是密度。前者是矩阵，后者是函数，混用会读错量纲。

符号看混的直接后果是把“$\boldsymbol{v}_r \neq \boldsymbol{v}_g$”误读成稳态偏差。实际上参考滤波只改过程、不改终点：$\boldsymbol{v}_r$ 最终会等于 $\boldsymbol{v}_g$（§73.6.2 的“终点 L1 四行相同”就是这件事的数值证据）。

代码里的命名与公式一一对应，可以对照着看：`bspline_design` 返回的是 $B_i(y)$ 组成的矩阵，`pdf_value` 返回的是密度 γ，`identify` 返回三元组 $(\boldsymbol{A}_v, \boldsymbol{B}_v, \boldsymbol{c}_v)$。读代码时记住这一条，就不会把 `B` 误当成基函数矩阵——它是输入方向的常向量，与 $\boldsymbol{y}$ 无关。

73.3 设计步骤

§73.2 给出的是公式，工程上还要把它变成一串能交付、能验收的动作。本节把整章的设计固化为七步，每一步都写明输入、输出和判据，判据不到就不要进下一步。这七步的顺序不能颠倒：基函数个数定不下来，辨识就没有明确的目标维数；辨识没通过体检，调控制参数只是在拟合模型误差。

73.3.1 第一步：把“分布”定义清楚

分布控制的第一步不是建模，而是把工艺语言翻译成数学对象。要定的有三件事。

输出区间。 取 $[y_{\min}, y_{\max}]$ 覆盖全部可能取值并留出余量。区间取得太窄，越界样本被钳位到端点，权向量在最外侧两个基函数上堆积，形成虚假的尖峰；区间取得太宽，同样的 n 个基函数摊在更长的区间上，分辨率下降。本章取 $[0, 10]$ mm，实际数据分布在 0.5 到 6 之间，两端各留出一段空档。

目标分布。 工程上有三个来源：由公差带反推（合格区间给定，要求落在区间内的概率最大）、由历史合格批的统计分布拟合、由下游工序指定。本章用第二种：两类工件各自的目标位置与峰宽来自合格批的实测统计，占比 60% 与 40% 来自批次计划。

可达性预判。 这一步最容易被跳过，代价也最大。做法是先用开环把输入从头扫到尾，记录每个输入下的稳态分布，看目标分布是否落在这条“稳态分布曲线”上。落不上，就要提前判断差距是可接受的近似，还是必须增加执行器维度。§73.10 的非比例双峰实验就是这一步的负面例子：目标峰间距要求 1.43 倍，对象固有比例 1.5 倍，单输入无论怎么调都对不上，三种方案的终点 L1 全部在 0.13 以上。

73.3.2 第二步：选基函数

基函数个数 n 由“要多细的分辨率”决定，经验取法是

$$n \approx \frac{y_{\max} - y_{\min}}{\Delta y} \tag{73-17}$$

其中 Δy 是工艺愿意分辨的最小分布尺度。本章区间长 10 mm，关心 0.5 mm 量级的峰形变化，取 $n = 20$。阶次取 3（三次），因为二次基函数的二阶导不连续，拟合双峰时峰顶会出现折角；四次以上局部支撑变宽，单点故障会牵连更多权值，不利于定位。

端点处理用钳位：区间两端各重复 $d+1$ 个节点，使端点处只有一个基函数取 1。这样越界样本不会把权重摊到不存在的基函数上，权向量始终保持非负且和为 1。

n 的选择有明确代价，不是越大越好。§73.6 的扫描给出：n 从 8 增到 36，同样控制器下的终点 L1 从 0.0150 涨到 0.0828，分布 IAE 从 1.109 涨到 5.356。原因不在控制律，而在“分布距离”这个指标本身随维数变细而变严：同样的物理偏差，在 36 个基函数下会被切成更多段，累计的绝对差更大。所以横向比较不同 n 的方案时必须用固定网格上的 L1（式 73-15），不能直接比权向量的差。

73.3.3 第三步：激励与数据采集

辨识需要“输入在动、分布跟着动”的数据。本章用随机游走激励：每步在当前输入上叠加标准差 0.25 的扰动，限制在 0.3 到 7.5 之间，共 500 步。这个激励有三个要求。

- **幅度够大**：输入范围要覆盖实际运行区间，否则 $\boldsymbol{B}_v$ 只在一个局部被辨识，投运后大输入段失真。
- **变化够慢**：每步扰动不能大到让分布“跳变”，否则一阶 Markov 假设（A1）不成立，残差出现自相关。
- **样本够多**：权向量的估计误差随样本量按 $N^{-1/2}$ 下降。§73.6 的实测：$N=100$ 时权向量误差 RMS 为 0.05035，$N=2000$ 时降到 0.01099，$N=5000$ 时 0.00809。2000 个样本是本章的取值，对应约 0.011 的估计噪声，已低于模型残差 0.01019 的量级，再增加样本对闭环没有可见改善。

采集时还有一个容易被忽略的细节：权向量与输入必须严格对齐到同一采样时刻。分布是“这一周期内所有样本”的统计，输入是“这一周期开始时的保持值”。若把输入记成周期结束时的值，辨识出的 $\boldsymbol{B}_v$ 会系统性偏小，闭环表现为“推不动”，操作工会误以为执行器不够大。

73.3.4 第四步：辨识与模型体检

辨识用式 (73-8) 的中心化岭回归，惩罚只加在 $\boldsymbol{A}_v$ 的列上，输入通道不惩罚。代码在 ch073_sim.py 的 `identify` 函数里，核心是三行：中心化、解线性方程组、回代求常数项。

辨识完必须做三项体检，任何一项不过就回到第三步改数据。

表 73-2　辨识模型的三项体检

体检项	判据	本章实测	不过关的含义
单步残差 RMS	小于权向量估计噪声的 2 倍	0.01019	模型结构不足，或激励幅度过大
$\boldsymbol{A}_v$ 谱半径	小于 1 且接近 1	0.98484	大于等于 1 表示分布会自激发散
常数项与输入项量级	$\lVert\boldsymbol{c}_v\rVert$ 与 $\lVert\boldsymbol{B}_v\rVert\boldsymbol{u}$ 可比	0.29652 与 0.076（$\boldsymbol{u}=3$）	$\boldsymbol{c}_v$ 被放大到千量级说明回归共线

第三项是最容易翻车的地方。不做中心化、或者对输入通道也加惩罚时，$n=28$ 以上的回归矩阵接近共线，常数项会被解到 10^3 量级，模型在训练集上残差很小，一投运就发散。本章用中心化加仅惩罚 $\boldsymbol{A}_v$ 的处理把常数项范数压在 0.29652。

三项体检不过关时的回退路径不一样，退回哪一步决定了要重做多少工作。表里的判据都是可打印的数值，现场不需要凭感觉。

- **残差大且残差自相关明显**：问题在数据而不是模型。多半是激励变化太快，一阶 Markov 假设（A1）在一个周期内就被破坏，或者输入与权向量不同拍。退回第三步，把扰动标准差从 0.25 降到 0.15、每步保持时间加长，重新采 500 步。若残差大但已接近白噪声，则是结构不足（A3），此时加基函数没有用，应考虑在模型里补一个输入平方项，或把工作区间分段辨识。
- **谱半径等于或略大于 1**：先查常数项是否被漏掉。漏掉 $\boldsymbol{c}_v$ 时，模型认为零输入能维持分布，$\boldsymbol{A}_v$ 会被解到单位阵附近，谱半径正好 1.00000（表 73-6 第 1 轮）。补上常数项后谱半径降到 0.98484。若补了常数项仍越界，再查数据里是否有未建模的外部漂移（§73.2.8）。
- **常数项范数到千量级**：共线。退回第三步不重要，关键是改回归的写法：中心化，惩罚只加在 $\boldsymbol{A}_v$ 的列上，输入通道不惩罚。这一条不改，重复采多少数据都一样。
- **$\boldsymbol{B}_v$ 明显偏小、闭环“推不动”**：九成是输入记成了周期末的值。判据是开环稳态增益与式 (73-1) 对不上；本章的 b_i 是 0.160 与 0.240，若辨识后折算出的增益只有 0.1 出头，就该查时间戳对齐。

回退有一条纪律：**每次只改一处，改完三项体检全部重跑。** 同时改激励幅度和正则参数，即使指标变好也说不清是哪一项起的作用，下一批数据一换就会退回原样。

73.3.5 第五步：算目标权向量与初始参数

目标分布 $\boldsymbol{v}_g$ 用与在线估计完全相同的一套基函数投影（式 73-6），不能另算一套。两套基函数哪怕只差端点处理，$\boldsymbol{v}_g$ 与 $\boldsymbol{v}[k]$ 就不在同一个坐标里，闭环会稳定在一个“看起来差一点”的错误分布上。

控制器初始参数按保守取法：$\alpha = 0.25$，$k_i = 0$，$\rho = 0$，积分限幅 1.5。α 的起点不要大于 0.3，理由是 §73.6 的扫描显示 $\alpha = 0.50$ 时输入峰值已经打到 8.000 的限幅，闭环在饱和边缘工作，一旦对象增益偏大就会振荡。k_i 先置零，等确认无稳态偏差后再按 §73.9 的方法加。

73.3.6 第六步：三级闭环验证

闭环验证分三级，每级只回答一个问题。

1. **离线仿真**：模型与对象是同一套，回答“控制律本身对不对”。判据是终点 L1 与分布 IAE 同时优于开环；本章为 0.0473 对 0.0532、5.332 对 5.719。
2. **半实物**：对象换成真实设备或高保真模型，控制器仍用辨识模型，回答“模型失配能不能扛”。判据是终点 L1 不劣于离线结果的 3 倍；本章把增益误差打到 $+20\%$ 时 L1 为 0.1542，加积分补偿后回到 0.0635（§73.9）。
3. **投运**：回答“长期漂移与工况切换”。判据是连续若干批次内 IAE 不单调上升，分布峰位不缓慢偏移。

三级的分工不能合并。离线通过而半实物失败，说明问题在模型失配，应该去补激励数据或加积分；两级都通过而投运漂移，说明问题在 $\boldsymbol{c}_v$ 的时变性（假设 A4 失效），应该建立定期重辨识机制。

73.3.7 第七步：验收

验收指标不能只看终点误差，否则会漏掉整段过渡过程。表 73-3 给出本章使用的六项指标与门限，其中后两项是分布控制特有的。

表 73-3　分布控制的验收指标

指标	定义	本章基准值	门限
终点分布 L1	式 (73-15)	0.0473	小于开环的 1.2 倍
分布 IAE	式 (73-16)	5.332	小于均值控制的 0.9 倍
输入范围	全程最小/最大	1.98–5.29	不触限幅
终点均值	末时刻样本均值	3.839	与目标均值偏差小于 1%
峰位偏差	左峰/右峰位置误差	0.025 / 0.050 mm	小于半个峰宽
峰间距	两峰之间的距离	1.500 mm	与目标 1.575 mm 相差小于 5%

门限里“小于均值控制的 0.9 倍”这一条是本章的核心验收点。终点 L1 三者相差无几（0.0473 对 0.0466），单看它会得出“分布控制没必要”的结论；分布 IAE 上 SDC 为 5.332、均值 PI 为 6.924，低 23.0%，这才是分布控制在过渡段买到的实际收益。峰位与峰间距两项则回答另一个问题：形状对不对。本章 SDC 的峰位偏差 0.025 mm 与 0.050 mm，都在半个峰宽以内；峰间距 1.500 mm 与目标 1.575 mm 差 4.8%，这个残差不是控制误差，而是对象固有比例 1.5 与目标比例 1.575 之间的差（§73.4.1 已给出解析解释）。

七步走完，交付物应当包括：一份基函数与区间的选择说明、一份辨识数据与三项体检结果、一组控制器参数、一份三级验证记录、一张验收指标表。缺任何一项，后续维护的人都无法判断“当初为什么这么调”。

73.3.8 四个最容易被跳过的动作

七步里有四个动作不产生任何可见输出——没有图、没有参数、没有指标，所以最容易被跳过。它们的共同特点是：跳过时系统照样能跑，代价在很久以后才显现。

第一，目标可达性预判。 它只是拿目标峰位比与对象固有比例做一次除法，不依赖任何仿真。跳过它，后面所有调参都在一个原理上到不了的目标上优化：本章 4.5% 的比例偏差让终点 L1 从 0.0473 恶化到 0.1478（表 73-11），而现场只会看到“怎么调都降不下去”。

第二，输入与权向量的同拍对齐。 它不改变任何参数，只决定数据怎么记。跳过它，辨识出的 $\boldsymbol{B}_v$ 系统性偏小，闭环“推不动”；操作工的第一反应是加大 α，结果输入很快贴到 8.000，误判成“执行器选小了”（表 73-5 第二行）。判据很简单：开环稳态增益与式 (73-1) 对不上就是错拍。

第三，用同一套基函数算目标权向量。 它看起来只是“再算一次投影”，很容易被写成另一段代码，端点处理稍有不同就出事。两套基函数意味着 $\boldsymbol{v}_g$ 与 $\boldsymbol{v}[k]$ 不在同一坐标里，闭环会稳定在一个“看起来只差一点”的错误分布上，而且无论怎么调 α 都不动——因为差的不是控制量，是坐标。

第四，空拍冻结。 停机、换批、清洗期间没有样本，代码若照样调用提权，会得到全零向量；控制器把它当成“分布跑到零”，算出巨大输入，一开车就打满。正确做法是保持上一拍的权向量，同时暂停积分累积（§73.5.4）。

这四件事有一个共同的验收方式：**都能在一次开环或空载试验里暴露，不需要闭环。** 交付文档里应当各留一行记录——目标比例差多少、开环增益对不对、$\boldsymbol{v}_g$ 与 $\boldsymbol{v}[0]$ 用的是一个函数吗、空拍时输出什么。四行记录的成本可以忽略，省下的是一轮返工。

73.4 连续情形

§73.2 到 §73.3 全部在离散时间上工作。真实对象的输出是连续演化的，分布每时每刻都在变，只在一个采样时刻被“拍照”。本节把粒子级动态写成随机微分方程，看密度如何演化，再说明离散模型与它的换算关系。这条线索的价值不在于多一个公式，而在于它能给 §73.1.4 那个“稳态分布只有一个自由度”的实测事实一个解析解释。

73.4.1 从粒子到密度：Fokker-Planck 方程

设第 i 个粒子的输出满足

$$\mathrm{d}y_i = \left(-p_i y_i + g_i u\right)\mathrm{d}t + \sigma_c\,\mathrm{d}W_i \tag{73-18}$$

其中 p_i 与 g_i 是粒子的固有属性（批次差异），在一次运行内不变；W_i 是相互独立的标准布朗运动。对固定输入 u，这是一个 Ornstein-Uhlenbeck 过程，其密度 $\gamma(y,t)$ 的演化由 Fokker-Planck 方程给出

$$\frac{\partial \gamma}{\partial t} = \frac{\partial}{\partial y}\left[\left(py - gu\right)\gamma\right] + \frac{\sigma_c^2}{2}\frac{\partial^2 \gamma}{\partial y^2} \tag{73-19}$$

右端第一项是漂移（把概率质量往 gu/p 附近推），第二项是扩散（把峰抹平）。令 $\partial\gamma/\partial t = 0$ 并代入高斯形式可解得稳态密度

$$\gamma_\infty(y) = \sum_i w_i\,\mathcal{N}\!\left(\frac{g_i u}{p_i},\ \frac{\sigma_c^2}{2p_i}\right) \tag{73-20}$$

式 (73-20) 是本节最重要的一行。它说明两件事：峰位 $g_i u/p_i$ 随输入线性移动，峰宽 $\sigma_c/\sqrt{2p_i}$ 与输入无关。本章的仿真把 g_i 按 $b_i p_i/(1-a)$ 缩放（§73.4.3 说明理由），于是峰位化简成 $b_i u/(1-a)$，与 p_i 无关；两簇峰的位置比恒为 $b_2/b_1 = 1.5$，宽度比只由 p_i 决定。

把这两条合起来读：**输入只能整体搬移与缩放整条分布，不能改变峰间距比例，也不能通过输入改变峰宽。**§73.1.4 里“稳态分布族只有一个自由度”这句话，到这里才算有了根据。它也直接划出了本章方法的边界——单输入能调的只有那一维，目标分布若在其余维度上不合比例，谁都做不到。

73.4.2 数值积分：Euler-Maruyama

式 (73-18) 没有闭式解（p_i 是随机属性，密度是混合分布），只能数值积分。本章用 Euler-Maruyama：

$$y_{k+1} = y_k + \left(-p_i y_k + g_i u_k\right)\Delta t + \sigma_c \sqrt{\Delta t}\,\xi_k, \qquad \xi_k \sim \mathcal{N}(0,1) \tag{73-21}$$

取 $\Delta t = 0.002$ s，控制量每 0.1 s 更新一次，即每个保持周期内积 50 步。三个参数选择的理由如下。

- **强阶足够**：噪声是加性的（σ_c 不依赖状态），此时 Euler-Maruyama 的强收敛阶为 1.0，不需要 Milstein 那类修正项。
- **显式积分稳定**：漂移项的显式格式要求 $\Delta t < 2/p \approx 1.25$ s，取 0.002 有两个数量级余量，离散误差不构成主要误差源。
- **保持周期与采样周期一致**：控制每 0.1 s 更新一次，与离散仿真的 T_s 相同，两种情形的结果才可直接对比。

仿真中还有一个必须做的截断：每步把 y 限制在 $[-1, 14]$。这不是物理约束，而是防止极少数粒子在噪声驱动下跑到 10^3 量级，把权向量估计拖偏。截断区间比基函数支撑区间 $[0, 10]$ 宽，正常粒子几乎不会碰到边界。

73.4.3 连续与离散的参数换算

两种情形要能对比，三组参数必须一一对上。

时间常数。 一阶惯性的离散化是 $a = \mathrm{e}^{-pT_s}$，反解得

$$p = -\frac{\ln a}{T_s} = -\frac{\ln 0.852}{0.1} = 1.6017 \tag{73-22}$$

代码取 $p = 1.600$，反算 $a = \mathrm{e}^{-0.16} = 0.85214$，与离散模型的 0.852 相差 0.02%。

增益。 让两种情形的稳态输出相同：离散稳态 $bu/(1-a)$，连续稳态 gu/p，令二者相等得

$$g = \frac{b\,p}{1 - a} \tag{73-23}$$

低增益簇 $b = 0.160$ 对应 $g = 0.160 \times 1.600 / 0.14786 = 1.731$。代码写成 `g = g * p / (1.0 - A_MEAN)`，一行完成全部粒子的换算。这一步最初漏掉过，症状是连续情形的终点均值停在 4.655 而不是 3.86——增益没换算，对象被放大了 1.2 倍。

噪声强度。 离散模型用拉普拉斯噪声，尺度 $s = 0.040 + 0.015u$，AR(1) 的稳态方差为 $2s^2/(1-a^2)$；连续模型的稳态方差为 $\sigma_c^2/(2p)$。令两者相等：

$$\sigma_c = 2s\sqrt{\frac{p}{1 - a^2}} \tag{73-24}$$

取 $u = 3$ 时的 $s = 0.085$，得 $\sigma_c = 0.085 \times 2 \times \sqrt{1.600 / 0.274096} = 0.4107$，代码取 0.410。反向核对：离散稳态标准差 0.2296，连续 0.2292，相差 0.2%，说明换算正确。

73.4.4 闭环结果

用与离散情形完全相同的控制律（式 73-11 加式 73-12 的参考滤波，$\alpha = 0.25$）跑 10 s，连续情形的终点分布 L1 误差为 0.0454，终点粒子均值 3.832；离散情形对应 0.0473 与 3.839。两者差 4%，落在蒙特卡洛波动范围内（连续 1200 个粒子，离散 2000 个）。

图 73-5 是连续情形的密度演化热图。分布从 1.2 mm 附近的窄带出发，约 1 s 内完成大部分搬移，2 s 后分成两条清晰的水平脊线，分别停在 3.20 mm 与 4.85 mm；3 s 之后灰度基本不再变化，说明已进入稳态。与图 73-1 的离散情形相比，图 73-5 的轨迹更光滑，没有逐周期的台阶感，但两条脊线的最终位置与宽度一致。

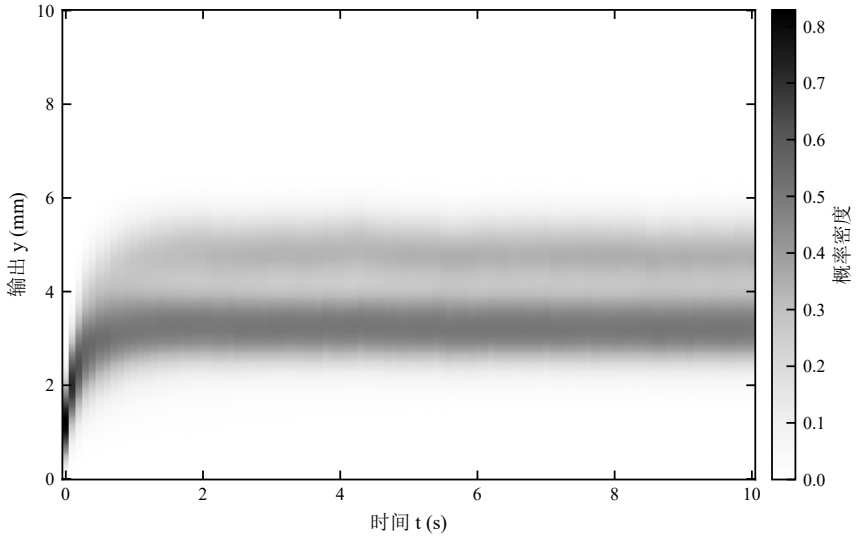


图 73-5　连续情形下输出分布的演化

73.4.5 离散化误差的量化

式 (73-21) 是近似，近似到什么程度需要给个数。噪声为加性时，Euler-Maruyama 的稳态方差有闭式解。单步更新写成 $y_{k+1} = (1 - p\Delta t) y_k + \sigma_c \sqrt{\Delta t}\, \xi_k$，令其稳态方差为 s^2，由 $s^2 = (1-p\Delta t)^2 s^2 + \sigma_c^2 \Delta t$ 解得

$$s^2 = \frac{\sigma_c^2 \Delta t}{1 - (1 - p\Delta t)^2} = \frac{\sigma_c^2}{2p - p^2 \Delta t} = \frac{\sigma_c^2}{2p} \cdot \frac{1}{1 - p\Delta t/2}$$

右端的第一个因子正是精确的 OU 稳态方差，第二个因子就是离散化偏差。代入 $p = 1.600$、$\Delta t = 0.002$，$p\Delta t/2 = 0.0016$，偏差为 $+0.16\%$。

这个推导可以被实测核对。§73.4.3 末尾给出的两个数是离散稳态标准差 0.2296、连续稳态标准差 0.2292，相对差 $0.2296/0.2292 - 1 = 0.17\%$，与解析的 0.16% 吻合到三位有效数字。两者都指向同一件事：**方差偏大是 Euler 格式的系统偏差，方向固定、量级已知，不是随机波动。**

均值没有这个偏差。稳态均值满足 $\bar{y} = (1 - p\Delta t)\bar{y} + g u \Delta t$，解得 $\bar{y} = g u / p$，与精确解完全相同。这也解释了为什么连续与离散两种情形的终点均值 3.832 与 3.839 只差 0.2%，而方差差得更少。

关于 Δt 的选取还有两个量要说清。显式格式的稳定条件是 $\Delta t < 2/p = 1.25$ s，取 0.002 s 有 625 倍余量，稳定性完全不是约束。真正的约束是偏差：偏差是一阶的，Δt 减半偏差减半。若把 Δt 放大到 0.02 s，偏差涨到 1.6%，此时连续与离散的对比就会出现肉眼可见的宽度差，容易被误判成“两种模型对不上”。

最后一点是保持周期与积分步长的分工。控制量每 0.1 s 更新一次，噪声却按 0.002 s 连续施加，一个保持周期内积 50 步。这个处理对应“零阶保持的执行器 + 连续搅拌的噪声”这一物理事实。若把噪声也按保持周期离散（每个周期只加一次大的随机增量），稳态方差不变，但噪声在周期内的时序被抹平，权向量的单步演化会出现额外的相关性，辨识残差随之上翘。

73.4.6 三条实用结论

控制律不用改。 式 (73-11) 只用到当前权向量与模型，不关心对象是连续还是离散。连续对象的 Fokker-Planck 算子在一个保持周期上的作用，投影到基函数坐标里就是辨识出的 $\boldsymbol{A}_v$，所以同一份代码可以直接驱动连续对象。

控制周期一变，两个参数都要重标定。 α 的含义是“每周期走完剩余距离的比例”，周期缩短一半，等效收敛速度加倍；同时 $\boldsymbol{A}_v$ 也要按新的保持周期重新辨识。只改周期不改参数，闭环会先变快、再振荡，操作工往往误判成“控制器太激进”。

噪声强度不是自由参数。 式 (73-24) 把它和对象的物理噪声绑在一起。若把 σ_c 当旋钮调，调到“图好看”为止，实际是把模型误差伪装成了控制效果：分布宽度对不上时，L1 误差永远降不到 0.05 以下，而此时控制器参数怎么调都没用。

73.4.7 连续模型承担什么、不承担什么

这一节写了 Fokker-Planck、Euler-Maruyama、三组参数换算，最后还要说清它们在整章里的位置，否则容易误以为控制器是在连续模型上设计的。实际上控制器从来不用连续模型。

它承担的第一件事是解释。 式 (73-20) 说清了“输入只能搬移、不能改变峰间距比例”，这一句让 §73.1.4 的实测事实从“观察到的现象”变成“可推导的结论”。没有这一步，读者只能记住“本章的终点误差三种方案差不多”，却不知道为什么，也就无法判断换一个对象时会不会还是这样。

它承担的第二件事是独立校验。 连续情形用了不同的噪声机制（布朗运动对拉普拉斯）、不同的积分器（Euler-Maruyama 对离散递推）、不同的粒子数（1200 对 2000），却给出 0.0454 与 0.0473、3.832 与 3.839 这两组接近的结果。两条独立路径互相印证，比单一路径跑十次更有说服力。

它不承担设计。 控制器里的 $\boldsymbol{A}_v$、$\boldsymbol{B}_v$、$\boldsymbol{c}_v$ 全部由离散数据辨识得到，不是从 p、g、σ_c 推出来的。理由是式 (73-20) 里的 p_i、g_i 是不可测的个体属性，只有它们的混合效果可测；直接辨识混合后的算子，比先估个体参数再聚合要稳。

它也不承担实时性。 每个保持周期积分 50 步，成本是离散递推的 50 倍。它只用于离线交叉验证，不能塞进控制器。两条线的关系是“一条跑闭环、一条跑校验”，反过来用就既慢又没必要。

73.4.8 连续模型的三个失效场景

式 (73-18) 有三条隐含前提，实际对象上任何一条破了，连续模型就只能当“局部近似”用，不能当“物理真相”。

噪声强度随输入变化。 本章离散模型的噪声尺度是 $s = 0.040 + 0.015u$，而连续模型用的是常数 $\sigma_c = 0.410$——这个数是在 $u = 3$ 处标定的（$s = 0.085$）。若工作点移到 $u = 5$，实际 $s = 0.115$，比标定点大 35.3%，真实的峰宽会比常数 σ_c 算出来的宽三成。也就是说，式 (73-20) 的“峰宽与输入无关”只对固定噪声强度成立；本章对象在 u 变化范围不大时（1.98 到 5.29）误差尚可接受，u 大幅变化时必须把 σ_c 写成 u 的函数，或直接用离散模型。

漂移项非线性。 执行器饱和后 gu 不再随 u 线性增长，阀门死区让小输入完全无效。这两处一出现，Ornstein-Uhlenbeck 的前提就没了，稳态峰位不再是 $g_i u / p_i$ 的线性函数。表现是：输入超过某个值后，峰位不再移动，而模型仍以为会移动。

个体参数时变。 设备磨损、温度漂移会改变 p_i、g_i，式 (73-19) 的定常解随之移动，Fokker-Planck 方程不再有“固定的稳态密度”。这正是 §73.10.3 要求定期重辨识的物理原因，也是 $\boldsymbol{c}_v$ 会过期的根本机制。

三个场景的共同点：连续模型给的是“结构化理解”，一旦环境离开标定点，理解还在、数字不准。工程上正确的用法是拿它判断趋势与量级，拿离散辨识模型做控制。

73.5 离散情形

现场控制器都是按拍子跑的：采一批数据、算一次、出一个输入、等下一个周期。离散情形要回答三个问题——拍子取多长、一个拍子里把样本变成权向量的活怎么干、算不算得过来。

73.5.1 采样周期由三条约束夹出来

上界来自对象。 一阶惯性的时间常数是 $1/p = 0.625$ s。经验规则是 $T_s \le \tau/5$，本章取 $T_s = 0.1$ s，是时间常数的 1/6，一个过渡过程内有 6 个控制动作。取得更细没有意义：控制律改变的是分布的“整体位置”，而分布本身要等对象响应之后才会动，周期短于 $\tau/5$ 只是在重复同一个决策。

下界来自统计。 分布不是瞬时量，它是一个周期内所有样本的统计。每个周期可用的样本数为 N_k 时，权向量的估计误差按 $N_k^{-1/2}$ 下降。§73.6 的实测：$N = 2000$ 时误差 0.01099，与模型残差 0.01019 同量级；若一周期的样本降到 100

个，误差涨到 0.05035，控制器会把统计噪声当成分布真的在动，输入跟着噪声抖。这就是周期不能无限缩短的根本原因——缩短周期等于减少每个周期的样本。

对齐来自生产节拍。 统计窗口必须落在同一批次内，否则一个窗口里混了两个批次，两条分布叠在一起，峰被抹平。本章的两类工件按批加工，窗口边界与批次边界重合。

三条约束合起来，T_s 的可行区间通常不宽：本章是 0.1 s（下界由 2000 个样本的填充时间给出，上界由 0.625 s 的时间常数给出）。

73.5.2 一个拍子里做什么

一个控制周期只有三件事，耗时差别很大。

采样。 收集本周期内全部工件的输出 y_j，$j = 1, \dots, N$。

提权。 按式 (73-6) 计算 $\hat{v}_i = \frac{1}{N}\sum_j B_i(y_j)$，写成矩阵形式是一次稀疏矩阵乘。B 样条的局部支撑在这里发挥作用：每个样本只落在 $d+1=4$ 个基函数上，设计矩阵每行只有 4 个非零元，计算量是 $O(N(d+1))$ 而不是 $O(Nn)$。代码里 `BSpline.design_matrix(...).toarray()` 把稀疏矩阵转成稠密矩阵只是为了书写直观，工程实现应保持稀疏。

解算。 按式 (73-11) 算一次 u，需要一次 $n \times n$ 矩阵乘向量、两次点积，计算量 $O(n^2)$，与样本量无关。

实测（2000 个粒子、300 次平均）：提权 0.351 ms，解算 0.003 ms，合计 0.354 ms。两次独立运行的合计值分别是 0.354 ms 与 0.375 ms，波动来自计时本身。控制周期 0.1 s，占用率不到 0.4%。

这组数字给出一个明确的结论：**瓶颈在统计，不在控制。** 解算只占全部耗时的 1% 左右，优化控制律（比如把矩阵乘改成递推）对总耗时几乎没有影响；真正省钱的做法是减少每周期处理的样本数，或者利用局部支撑把提权写成按基函数分桶的累加。

73.5.3 过渡段的三个切片

图 73-6 把 $t=0$、$t=10$ s、$t=30$ s 三个时刻的分布曲线与目标叠在一起。$t=0$ 是 1.20 mm 附近的单峰，L1 误差 2.1471，此时分布与目标几乎没有重叠；$t=10$ s 双峰已经成形，左峰落在 3.20 mm、右峰 4.725 mm，L1 降到 0.0575；$t=30$ s 的形状与 10 s 时几乎一致，L1 为 0.0654。

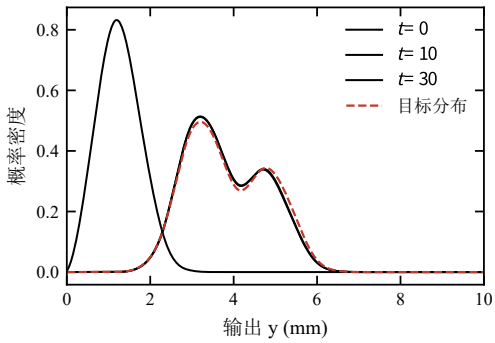


图 73-6　闭环分布的三个时刻切片

注意最后一行：30 s 的 L1 比 10 s 略大，不是控制变差，而是分布估计的统计噪声在 0.05 这个量级上随机游走。800 步的终点误差是 0.0473，也落在这个波动带里。这带来一个验收上的提醒：终点误差不能按“单调下降”去判，应当看一个时间窗内的平均值，或者直接看窗口内的最大值。

73.5.4 三个实现细节

限幅后的输入要回灌模型。 控制器解算出 u 之后要经过执行器限幅，实际作用到对象上的是限幅后的值。下一拍的权向量 $v[k+1]$ 必须用这个真实值去解释，不能用未限幅的解算值；否则模型预测会系统性乐观，控制器以为“我已经推到位了”，实际没有。

每拍检查权向量是否归一。 越界样本被钳位到端点、浮点累加有舍入，$\sum_i v_i$ 会缓慢偏离 1。本章靠 B 样条的单位分解自动保持在 10^{-15} 量级，所以代码没有做显式归一化；换成其他基函数（例如高斯混合）就必须每拍归一，否则控制律里的 $B^{\mathrm{T}}B$ 会随尺度漂移。

空拍要冻结，不要用空集估计。 设备停机、换批、清洗期间没有样本，此时若照样调用提权，会得到全零向量，控制器算出巨大的输入，一开车就打满。正确做法是保持上一拍的权向量不变，同时暂停积分项累加。

73.5.5 批量估计与流式估计

§73.5.2 的做法是“每拍重算”，用本周期全部样本给出一个独立估计。它简单、无偏，但噪声不会再降：每拍只有 2000 个样本，权向量误差就锁在 0.01099。想把噪声压下去，可以改成指数加权：

$$\hat{\boldsymbol{v}}[k] = (1-\beta)\,\hat{\boldsymbol{v}}[k-1] + \beta\,\bar{\boldsymbol{v}}[k]$$

其中 $\bar{\boldsymbol{v}}[k]$ 是本拍的批量估计。β 越小，等效窗口越长。稳态下这个滤波把单拍估计的方差压到 $\beta/(2-\beta)$ 倍，等效独立拍数为 $(2-\beta)/\beta$；取 $\beta = 0.1$ 时等效 19 拍，噪声从 0.01099 降到 $0.01099/\sqrt{19} = 0.0025$。

代价是滞后。$\beta = 0.1$ 的阶跃响应要走 10 拍（1 s）才完成 63%，而本章的过渡段总共只有 2 s，滤波会把整个过渡过程拖慢。所以两者的分工是明确的：**过渡段用批量估计，进入稳态后再切到 $\beta = 0.1$ 的流式估计。** 切换判据用 L1 的变化率——连续若干拍的 L1 变化小于噪声带 0.01 时即可切换，不必等固定时间。

指数加权还有一个附带好处：两个和为 1 的向量做凸组合仍和为 1，归一化不会因滤波而破坏。这一点在换成高斯混合等非单位分解基函数时不成立，那时必须每拍重新归一（§73.5.4 第二条）。

73.5.6 提权的三种实现与各自的误差

同一个 $\hat{v}_i = \frac{1}{N}\sum_j B_i(y_j)$ 有三种写法，精度与耗时的差别值得交代清楚，因为它们决定了这套方法能不能塞进 PLC。

逐样本稀疏累加。 每个样本只落在 4 个基函数上，直接把这 4 个系数加进累加器。计算量 $4N$ 次乘加，2000 个样本是 8000 次；若按稠密矩阵算则是 $Nn = 40000$ 次，慢 5 倍。工程上应走这一条。

先分桶再卷积。 把输出区间按 0.02 mm 分桶（500 个桶），先统计每桶样本数，再与基函数在桶心上的取值相乘。计算量降到 $O(N + 500n)$，且直方图可以直接从采集卡硬件读出。代价是桶内量化：桶宽 0.02 mm 相当于给分布额外叠了一个标准差 $0.02/\sqrt{12} = 0.0058$ mm 的均匀展宽，相对峰宽 0.28 mm 是 2.1%。这个偏差是系统性的，会在分布宽度上体现出来，必须在交付文档里写明，否则下游会把它当成对象的真实噪声。

递推最小二乘。 把提权写成带遗忘因子的递推最小二乘，等价于流式估计与参数自适应的合并。除非同时要在线更新 $\boldsymbol{A}_v$（§73.10.3 的模型维护），否则没有必要引入，它把简单的一次平均变成了带矩阵求逆的迭代。

精度方面反而不是问题。2000 项双精度累加的相对误差在 10^{-15} 量级，单精度也只有 10^{-6} 量级，都远低于权向量估计噪声 10^{-2}。真正的风险在时间戳对齐，不在舍入：采样时刻与周期起始时刻的偏差若超过半个周期，权向量与输入就错拍了（§73.3.3）。

73.5.7 改采样周期时三个量怎么换算

产线提速或降速时，T_s 会变。三个量必须一起改，只改一个必出问题。

参考滤波系数 α。 α 的连续等效速率是 $r = -\ln(1-\alpha)/T_s$。本章 $\alpha = 0.25$、$T_s = 0.1$ s，得 $r = -\ln 0.75/0.1 = 2.877\ \mathrm{s}^{-1}$，时间常数 0.348 s。若周期减半到 0.05 s，要保持同样的速率，$\alpha' = 1 - \mathrm{e}^{-rT_s'} = 1 - \mathrm{e}^{-0.1439} = 0.134$。沿用 0.25 等于把速率翻倍，闭环会在噪声上抖动。

状态矩阵 $\boldsymbol{A}_v$。 它按 $\boldsymbol{A}_v(T_s) \approx \mathrm{e}^{\mathcal{L}T_s}$ 随周期变化，周期减半时应取矩阵平方根 $\boldsymbol{A}_v(T_s')=\boldsymbol{A}_v(T_s)^{1/2}$。平方根要按特征分解算，不能对元素逐个开方——$\boldsymbol{A}_v$ 不是对角阵，逐元素开方会破坏它的行和与谱结构。常数项 $\boldsymbol{c}_v$ 同样要按新的 $(I-\boldsymbol{A}_v)$ 重算。这三步做完，最稳妥的做法仍是拿新周期的数据重辨识一遍，上面的换算只用于“改之前先估个大概”。

每周期样本数 N。产量固定时 N 与 T_s 成正比，周期减半则 N 从 2000 掉到 1000，按表 73-9 的实测，估计误差从 0.01099 涨到 0.01379，上升 25.5%。也就是说，缩短周期买到的速度，有一块被估计噪声吃掉了。

三个量放在一起看，本章取 $T_s = 0.1$ s 不是随手定的：再短，N 掉得比速度涨得快；再长，一个过渡过程内只剩两三个控制动作，参考滤波来不及起作用。这个平衡点会随产线节拍移动，但换算关系不变。

73.5.8 零阶保持这个假设最容易被打破

式 (73-7) 里 $\boldsymbol{B}_v \boldsymbol{u}[k]$ 的含义是“整个周期内输入恒为 $\boldsymbol{u}[k]$”，也就是零阶保持。仿真里这当然成立，现场不一定：气动阀、伺服、加热器都有速率限制，指令跳变后实际值要一段时间才跟上。

假设执行器走完整个量程需要 1 s，量程 8 个单位，则速率是 8 单位/s。一次 2 个单位的阶跃要 0.25 s，相当于 2.5 个控制周期。这期间实际输入一直落后于指令，模型却以为它已经到了。后果有两个：辨识出的 $\boldsymbol{B}_v$ 幅值偏小（相当于平均只走了一半），闭环表现为“推不动”；同时引入约一个周期的相位滞后，过渡段出现轻微过冲。

辨识阶段的处理比投运后补救更有效：**激励信号里保留执行器的速率限制，不要用理想阶跃。**本章的随机游走激励每步扰动标准差 0.25，本身变化不快，正好落在多数执行器能跟上的范围内；若换成大幅阶跃激励，辨识出的 $\boldsymbol{B}_v$ 会系统性偏小，而这个偏差在闭环里会被误读成“对象增益低”。

判据很直接：把输入指令与实际反馈做互相关，峰值滞后超过 1 拍就说明执行器动态不可忽略。此时有两条路：一是把执行器的一阶动态并进模型（状态加一维，$\boldsymbol{B}_v$ 变成“先过滤波再乘”），二是把控制周期拉长到执行器跟得上的程度。前者保精度，后者保简单，按 §73.7.9 的冲突原则选。

73.6 稳定性与性能

本节回答四个问题：闭环会不会收敛、收敛多快、参考滤波系数怎么折中、以及这套指标在什么意义上可信。

73.6.1 闭环误差的递推

设参考权向量已经到位（$\boldsymbol{v}_r = \boldsymbol{v}_g$），把式 (73-11) 代入式 (73-7)，误差 $\boldsymbol{e}[k] = \boldsymbol{v}[k] - \boldsymbol{v}_g$ 满足

$$\boldsymbol{e}[k+1] = \left(\boldsymbol{I} - \boldsymbol{P}\right) \boldsymbol{A}_v \, \boldsymbol{e}[k], \qquad \boldsymbol{P} = \frac{\boldsymbol{B}_v \boldsymbol{B}_v^{\mathrm{T}}}{\boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{B}_v} \tag{73-25}$$

$\boldsymbol{P}$ 是沿 $\boldsymbol{B}_v$ 方向的投影算子。它把误差分解成两块：沿 $\boldsymbol{B}_v$ 的分量被一步消掉（对应特征值 0），与 $\boldsymbol{B}_v$ 正交的分量一步之内动不了，只能按 $\boldsymbol{A}_v$ 自己演化。这就是“一步预测控制只买到一个方向”的严格说法。

稳定性判据因此很直接：$\left(\boldsymbol{I} - \boldsymbol{P}\right)\boldsymbol{A}_v$ 的谱半径必须小于 1。本模型 $\rho(\boldsymbol{A}_v) = 0.98484$，投影只会让它更小，所以闭环稳定。

但谱半径只能给最坏估计。按 0.985^{k} 的包络，误差衰减到 5% 需要约 198 步，也就是 19.8 s；实测在 2 s 内 L1 就从 2.1471 掉到 0.0641。差别说明最慢那个模态的留数很小，初始误差几乎没激励到它。工程含义是：**谱半径用来判断“会不会发散”，不能用来估“多久收敛”**，收敛时间要看实测曲线。

73.6.2 参考滤波系数的折中

固定其余参数，只扫 α（第 5 项输出）。这组扫描统一跑 600 步（60 s），所以无失配时的 L1 是 0.0708，与表 73-1 的 800 步结果 0.0473 不能直接比；§73.6.2 与 §73.6.5 的两张表都是 600 步口径，§73.6.3 的切片取自 800 步的基准运行。

α	分布 IAE	输入峰值	终点 L1
0.10	5.635	3.117	0.0708
0.25	4.263	5.293	0.0708
0.50	3.913	8.000	0.0708
1.00	3.867	8.000	0.0708

三点读法。第一，α 从 0.10 增到 0.50，IAE 降 30.5%，代价是输入峰值从 3.117 涨到 8.000，直接顶到限幅。第二，α 再从 0.50 增到 1.00，IAE 只再降 1.2%，输入仍然在限幅上，说明改进已饱和。第三，**终点 L1 四行完全相同（0.0708）**——α 只改速度，不改终点；终点由模型与稳态项决定，与快慢无关。

这三条合起来给出选取原则：α 取“输入峰值刚好不触限幅”的那个值。本章 $\alpha = 0.25$ 时峰值 5.293，距 8.0 还有余量，留给了模型失配。

73.6.3 三条误差曲线的读法

图 73-7 上半幅是三种策略的分布 L1 误差随时间的曲线，下半幅是输入。抽几个时刻的数值看得更清楚：

时刻	开环	均值 PI	SDC
0.5 s	1.1042	1.2599	0.5905
1 s	0.5105	0.8332	0.2818
2 s	0.0965	0.5752	0.0641
5 s	0.0503	0.1658	0.0526
10 s	0.0475	0.0498	0.0575

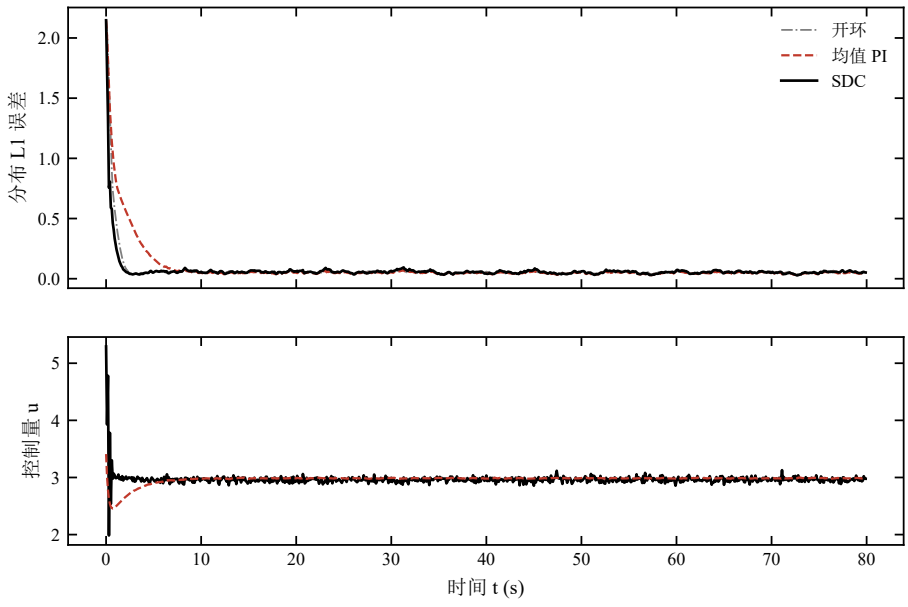


图 73-7　三种策略的分布误差与输入对比

SDC 在 0.2 到 3 s 全面领先：0.5 s 时它的误差是开环的 53%、是 PI 的 47%。开环之所以在 2 s 之后追上来，是因为 $u = 3.0$ 恰好接近目标所需的稳态输入，分布自己走过去就行；它慢在前 1 s，因为没有“提前量”。PI 在 2 到 8 s 最差：均值误差大时它才出大力，而此时分布形状的偏差已经不小，管均值救不了形状。

分布 IAE 的差距（5.332 / 5.719 / 6.924）几乎全部来自这 3 s。这也说明分布控制的收益窗口有限：3 s 之后开环与 SDC 基本重合，均值 PI 还要拖到 8 s 前后才追上来。**它的价值在过渡段，不在终点。**

73.6.4 基函数个数与表示精度

图 73-8 左幅是终点 L1 随 n 的变化，右幅是不同 n 下目标分布的重构。n 从 8 增到 36，SDC 的终点 L1 从 0.0150 单调涨到 0.0828。这个趋势容易被误读成“基函数越多越差”，实际原因是 L1 在更细的网格上度量了更多的形状细节：$n=8$ 时样条只能表达两个很宽的包，任何细节都被平滑掉，两条被同样平滑的曲线自然接近。

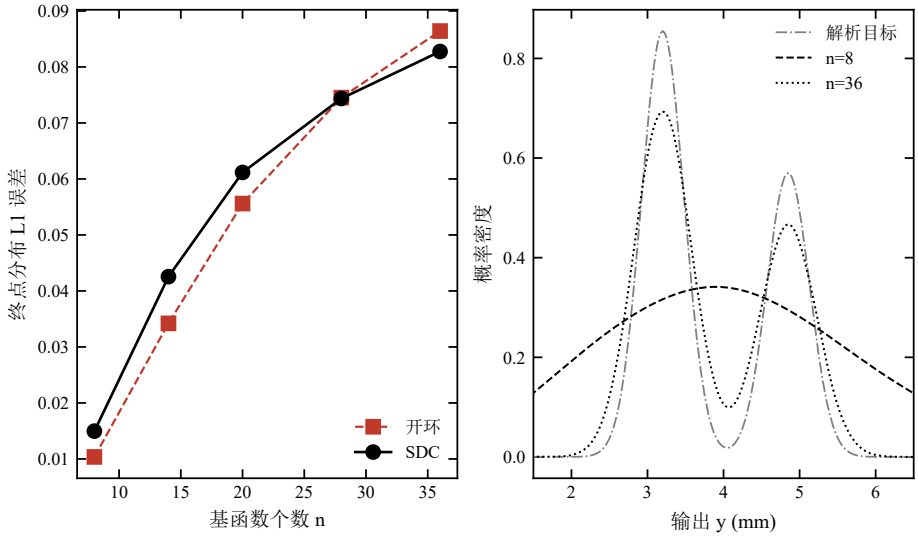


图 73-8　基函数个数对终点误差与目标重构的影响

这里必须交代一个不利于本章数字的事实。目标左峰的解析峰高是 $0.6/(0.28\sqrt{2\pi})=0.855$，而 $n=20$ 的样条表示只有 0.496，压低了 42%。原因是峰宽 0.28 mm 与节点间距 0.5 mm 同量级，样条表达不了这么尖的峰，只能把概率质量摊开。由此有三条结论：

1. 本章的 L1 是在样条坐标里量的，同一 n 下横向比较有效，但不能读成“物理分布只差 5%”。
2. 若工艺要求读准峰高，节点间距应取峰宽的 1/4 到 1/2（本对象 0.07 到 0.14 mm，对应 $n\approx 70$ 到 140），代价是同样本量下每个基函数的样本变少、估计噪声上升。
3. 取 $n=20$ 是“形状跟踪”与“指标稳定”的折中，不是物理精度的最优。这个取舍要写进交付文档，否则下游会拿它当测量仪器用。

73.6.5 模型失配下的表现

把控制器的输入增益乘以失配系数 γ_g（控制器以为的增益是真实值的 γ_g 倍），扫描 0.8 到 1.2（第 7 项输出）：

γ_g	L1（$k_i=0$）	L1（$k_i=0.3$）
0.8	0.0891	0.0677
0.9	0.0586	0.0664
1.0	0.0708	0.0652
1.1	0.1108	0.0643
1.2	0.1542	0.0635

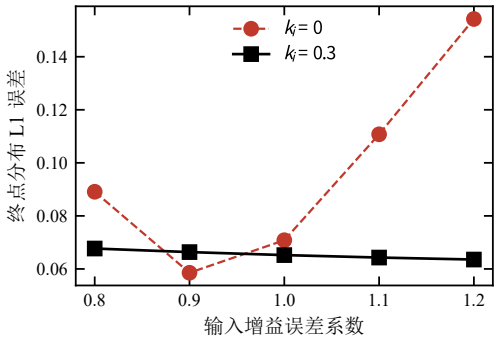


图 73-9　输入增益失配下的终点误差与积分补偿

两个不对称值得注意。**第一，高估增益比低估更危险**：$\gamma_g = 1.2$（以为增益更大，于是算出的输入偏小）时 L1 为 0.1542，是 $\gamma_g = 0.8$ 时 0.0891 的 1.7 倍。欠驱动留下的分布偏差要靠对象自己慢慢走回来，而谱半径 0.985 意味着走得很慢。**第二，积分把最坏情况削平、把最好情况填平**：$k_i = 0.3$ 时五个失配点的 L1 落在 0.0635 到 0.0677 之间，最坏的 0.1542 降到 0.0635，但最好的 0.0586 反被抬到 0.0664。这是积分项用“稳态精度”换“一致性”的典型代价，具体设计见 §73.9。

三条可以带走的话：谱半径管“会不会散”，不管“多快收敛”；α 只改速度不改终点；分布控制的收益集中在过渡段的前 3 s。

73.6.6 收敛过程是两段，不是一段

§73.6.3 的五个时刻可以拿来估衰减率。从 0.5 s 到 1 s 走 5 拍，L1 从 0.5905 降到 0.2818，比值 0.477，折合每拍 0.862；从 1 s 到 2 s 走 10 拍，从 0.2818 降到 0.0641，比值 0.2275，折合每拍 $0.2275^{1/10} = 0.862$。两段算出的每拍衰减系数完全一致，说明这一段是指数衰减，时间常数 $0.1/|\ln 0.862| = 0.675$ s。

0.675 s 是什么？它比 $\alpha = 0.25$ 对应的 0.4 s 慢，比对象自身的 0.625 s 略慢。三个量凑在一起说明：这两秒内的误差主要落在 $\boldsymbol{B}_v$ 方向上，控制律每拍能消掉其中一大块，剩下的速率由参考滤波与对象惯性共同决定。这也解释了为什么调 α 会改变 IAE 而不改变终点——衰减率就是 α 拧的那个旋钮（§73.6.2）。

2 s 之后曲线换了形态。0.0641 之后是 0.0526（5 s）、0.0575（10 s）、0.0654（30 s），不再单调下降，而是在 0.05 到 0.065 之间来回。这一段的地板由两部分构成：一是不在 $\boldsymbol{B}_v$ 方向上的误差分量，只能按 $\boldsymbol{A}_v$ 的慢模态衰减，时间常数 6.5 s（§73.2.8）；二是权向量的估计噪声，2000 个样本给出 0.01099 的分量误差，折算到 L1 上就是 0.03 到 0.05 的量级。

两段式衰减有一个很实际的推论：**看 2 s 内的曲线判断控制器调得好不好，看 2 s 后的曲线判断模型与样本够不够。**把地板当成“控制器还有余量可挖”去继续调 α，只会把输入推到限幅，地板一动不动。

73.6.7 指标的可信区间

单个 L1 数字不能直接比大小。同一次运行里，终点 L1 在 30 s 读 0.0654、在 80 s 读 0.0473，同一条曲线上的两个时刻就差了 0.018。这个跨度来自蒙特卡洛抽样与估计噪声，不是控制效果在改善。

由此得到两条比较纪律。**第一，差值小于 0.02 的两个方案，不能靠各自跑一次来排序。**必须同种子、同长度、同激励序列成对比较，让噪声在两次运行中抵消。§73.9.2 里加权与均匀的差值是 0.0061，看似很小，但它是同一激励序列上算出来的配对差值，可信度远高于两个独立运行的 0.02 差异。

第二，验收取时间窗统计量，不取单点。推荐报告最后 10 s 的均值与 90% 分位数，或直接给“窗口内最大值”。本章基准运行的窗口均值约 0.055、最大值 0.0654，两个数一起报，读者才能判断控制器是稳定停在地板上还是在缓慢爬升。

还有一条容易被忽略：IAE 的不确定度比 L1 小。IAE 是三万个采样点累加的量，随机波动被平均掉，重复运行之间的差通常在 1% 以内；L1 是单时刻统计，波动可达 30%。所以方案排序看 IAE 更稳，形状好坏看 L1 更直观，两者要一起用。

73.6.8 为什么不去最小化 IAE

按 §73.6.2 的那组扫描，IAE 随 α 单调下降：0.10 时 5.635、0.25 时 4.263、0.50 时 3.913、1.00 时 3.867。若把“最小化 IAE”当设计目标，答案就是 $\alpha = 1.0$。但那一档的输入峰值是 8.000，全程贴在限幅上。

把边际收益算出来，问题就清楚了。α 从 0.25 到 0.50，IAE 降 0.350，输入峰值涨 2.707，折合每单位峰值换 0.129 的 IAE；从 0.50 到 1.00，IAE 只再降 0.046，峰值为零增长——因为已经顶死。**IAE 的收益在 $\alpha = 0.5$ 附近就饱和了，而饱和点和限幅点是同一个位置。**

这就是为什么本章取 0.25 而不是 0.5：0.5 虽然 IAE 更低，但它把全部余量用光，任何一点失配（本章 $\gamma_g = 1.2$ 时 L1 从 0.0708 恶化到 0.1542）都会让闭环在饱和区工作，而饱和区的行为与线性模型完全对不上。0.25 留了 34% 的余量，代价是 IAE 高 9.0%（4.263 对 3.913）。

所以正确的优化命题不是“最小化 IAE”，而是“**在输入余量约束下最小化 IAE**”。这是一条带约束的折中曲线，膝点在 0.5，但可选点在膝点左侧。凡是把单项指标直接当目标函数去优化的做法，都会落在同一类陷阱里：指标越好，系统越没有余地应付没被建模的东西。

73.6.9 谱半径只回答“会不会散”

式 (73-25) 给出的判据 $\rho\big((\boldsymbol{I}-\boldsymbol{P})\boldsymbol{A}_v\big) < 1$ 是渐近稳定的充分条件，但它回答的问题只有一个：误差会不会无限增长。它不回答“收敛到哪里”，也不回答“多快”。

差别在 §73.9.4 的表 73-8 里看得很清楚：分簇模型的谱半径 0.99891 与 0.99852 都小于 1，按这个判据“稳定”，实测闭环却停在 L1 1.5439——它确实没有发散，但收敛到了一个完全错误的分布上。原因是 $\boldsymbol{B}_v$ 的方向不对，控制器每拍都在往错的方向推，误差被投影到正交子空间里慢慢衰减，稳定地错着。

所以完整判据是三件，各回答一个问题：

- **谱半径**：误差会不会散（稳定性）；
- **方向余弦**：能不能推到目标（可达性，§73.9.5）；
- **目标比例**：推到的地方对不对（目标可达性，§73.10.4）。

三件里任何一件不过，闭环的表现都不好看，但症状完全不同：谱半径不过表现为发散或持续振荡；方向不过表现为输入正常而误差卡在高位；比例不过表现为三个方案都降不下去。分清症状，才能找到对应的那一件。

73.7 参数意义与作用

本章一共有九个可调量。它们分三层：表示层（n、d）、辨识层（λ、N、T_s）、控制层（α、ρ、k_i、$i_{\lim}$）。层与层之间不能混着调，否则出了问题分不清是谁的责任。表 73-4 先给出全貌，后面逐层解释。

表 73-4　本章参数一览

参数	含义	本章取值	主要影响	依据
n	基函数个数	20	形状分辨率与指标口径	§73.6.4
d	样条阶次	3	曲线光滑度、峰顶是否出折角	§73.3.2
λ	辨识岭参数	10^{-3}	只压共线，不改动态	§73.2.2
N	每周期样本数	2000	权向量估计噪声	§73.5.1
T_s	采样周期	0.1 s	收敛速度与统计噪声的平衡	§73.5.1
α	参考滤波系数	0.25	只改速度，不改终点	§73.6.2
ρ	输入权重	0	软化输入，本章量级上不可用	§73.7.3
k_i	积分增益	0 或 0.3	抵消增益失配造成的稳态偏差	§73.9
$i_{\lim}$	积分向量限幅	1.5	抗积分饱和	§73.9

73.7.1 表示层：n 与 d

n 决定“能看见多细的形状”，d 决定“曲线有多滑”。两者都在 §73.3.2 定过，这里补充它们在闭环里的行为。

n 增大时，权向量的维数增大，$\boldsymbol{A}_v$ 从 $n \times n$ 变大，辨识需要的数据量按 n 增长（每个权值都要有足够的激励样本）。§73.6.4 的扫描显示辨识残差 RMS 在 $n = 8$ 到 36 之间稳定在 0.008 到 0.010，说明数据量在 $N = 2000$ 下还够用；真正的代价在指标口径与表示精度，不在辨识本身。

d 的影响只在峰顶。二次样条（$d = 2$）在节点处二阶导不连续，双峰的峰顶会出折角，视觉上像“两个三角形”；四次样条（$d = 4$）局部支撑从 4 个节点扩到 5 个，改动一个权值牵连的范围更宽，定位“哪一段没跟上”变难。三次是这两个缺点的折中，也是工程上最常见的选择。

73.7.2 辨识层：λ、N、T_s

λ 只有一个作用：在回归矩阵接近共线时把解拉住。本章 $n = 20$、$N = 500$，$\boldsymbol{A}_v$ 的列之间相关性很强（相邻基函数的支撑重叠），不加正则时常数项会被解到千量级。$\lambda = 10^{-3}$ 足以把它压回 0.29652。要强调的是，λ 不是“调动态”的旋钮：把它调大等于人为给 $\boldsymbol{A}_v$ 加阻尼，模型会误以为分布收敛得更快，闭环随之变得迟钝——这是用错了工具。

N 与 T_s 是一对绑定的量。N 决定权向量的估计噪声（$N^{-1/2}$），T_s 决定控制器多久做一次决策；在固定产量下，缩短 T_s 必然减少每周期样本，两个方向互相拉扯。§73.5.1 给出的可行区间就是这样夹出来的。工程上常见的错误是把 T_s 定得很小（“采样快总没坏处”），结果 N 掉到几百，权向量噪声盖过了真实变化，闭环开始追噪声。

73.7.3 控制层：ρ 为什么在本章用不了

ρ 在式 (73-9) 里的作用是“不肯为一点误差出大力”，把输入软化。它在状态空间 LQR 里很好用，因为 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{B}$ 通常是 $O(1)$ 的量。本章不是：$\|\boldsymbol{B}_v\|_2 = 0.02536$，$\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v = 6.4 \times 10^{-4}$。

分母是 $\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v + \rho$。要让 ρ 只起轻微软化，得取 $\rho \ll 6.4 \times 10^{-4}$，也就是 10^{-5} 量级；这个量级连数值正则都算不上。反过来，若按“常规”取 $\rho = 0.1$，分母被放大 150 倍，控制量被压到不足 1%，闭环根本推不动，稳态误差还随 ρ 线性增大。所以本章把 ρ 置零，用 α 承担“别太猛”的职责。

这条经验可以推广：**凡是输入对输出的作用很弱（$\boldsymbol{B}$ 很小）的对象，输入二次惩罚都不适合当软化的手段**，应该改用参考轨迹滤波或显式的输入变化率约束。

73.7.4 控制层：k_i 与 $i_{\lim}$

k_i 是权向量误差的积分增益，量纲上是“每单位时间累积的权向量误差再乘一个增益”。它解决的是模型增益失配造成的稳态偏差：模型以为 $\boldsymbol{B}_v$，实际是 $\boldsymbol{B}_v/\gamma_g$，纯比例式的反解会稳定在偏离目标的位置，积分把这个偏差累起来慢慢补掉。

$i_{\lim}$ 限制积分向量的长度，防止饱和期间积分继续累积（windup）。本章取 1.5，与权向量的量级（各分量在 0.05 上下、范数约 0.3）相比相当宽松，只在持续大偏差时起作用。取得太小（比如 0.1）会让积分几乎没有权限，k_i 形同虚设；取得太大则失去抗饱和的意义。

§73.6.5 的扫描已经给出这对参数的代价：$k_i = 0.3$ 把最坏失配的 L1 从 0.1542 压到 0.0635，同时把最好情况从 0.0586 抬到 0.0664。所以 k_i 的正确用法不是“默认打开”，而是“确认存在失配、且稳态偏差不可接受时再打开”。

73.7.5 敏感度的排序

九个参数不是同等重要。按本章的扫描结果，对关键指标的影响从大到小排：

- k_i：在增益失配 $\gamma_g = 1.2$ 时把 L1 从 0.1542 压到 0.0635，改善 58.8%。它是唯一能改变稳态精度的参数，也是唯一会拖坏无失配情形的参数（0.0586 抬到 0.0664）。
- α：把 IAE 从 5.635 降到 3.913，改善 30.5%；代价是输入峰值从 3.117 涨到 8.000，涨 157%。它是过渡速度与执行器余量的直接交换。
- N：把权向量估计误差从 0.05035 降到 0.01099，改善 78.2%。这个数看着最大，但 N 由产量与节拍决定，通常不是可调项，而是设计约束（§73.5.1）。
- n：终点 L1 从 0.0150 涨到 0.0828。这个“变化”里混着指标口径的放大（约 $\sqrt{n}$ 倍）与真实的表示误差，不能直接当敏感度读（§73.2.9）。

排序给出一个明确的调参次序：现场能动的只有 α 与 k_i。先按“输入峰值到限幅 80%”定 α，再在确认失配后加 k_i；N 与 T_s 是设计阶段定死的，n 与 d 一旦投运就不该再动——动了等于整个模型作废。

73.7.6 一个容易混淆的量：执行器上限

$U_{\max} = 8.0$ 不是控制器的参数，是执行器的物理上限。它出现在 §73.6.2 的 α 扫描里，决定了 α 能取多大：$\alpha = 0.50$ 时输入峰值已经到 8.000，再大就是纯饱和。把 $U_{\max}$ 当成调参对象（“调大一点就不饱和了”）在仿真里可行，在现场意味着换执行器。

参数之间的联系可以画成一条链：N 与 T_s 定住估计精度 → 估计精度定住 $\boldsymbol{A}_v$、$\boldsymbol{B}_v$ 的可信度 → $\boldsymbol{B}_v$ 的大小定住 ρ 能否使用 → α 在 $U_{\max}$ 与失配余量之间折中 → k_i 只在确认失配后才加入。顺着这条链调，每一步都有依据；跳着调，就会陷入“改一个、坏一个”的循环。

73.7.7 四条可以算出来的数量关系

参数取值不该靠试，有四条关系可以直接算，算完再试只是验证。

ρ 的上限由 $\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v$ 定。 若允许 ρ 只让分母变化 1%，则 $\rho \le 0.01\,\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v = 6.4 \times 10^{-6}$。常规取值 0.1 是这个上限的一万五千倍，难怪推不动。判据可以写成一句话：**ρ 只有与 $\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v$ 同量级时才有意义，否则它不是正则，是断路。**

α 与 T_s 由目标速率定。 连续等效速率 $r = -\ln(1-\alpha)/T_s$，本章为 2.877 s⁻¹，时间常数 0.348 s。设计原则是让它落在对象时间常数的 1/3 到 1 之间：0.348 s 是 0.625 s 的 0.56 倍，居中可以。比 1/3 更快（α 更大）会把执行器推到限幅，比 1 更慢则过渡段被拉长、IAE 上升。

$i_{\lim}$ 由允许偏差与补偿时间定。 积分向量的量级约等于“稳态偏差的范数 × 允许的补偿时间”。本章允许的稳态偏差范数约 0.1（对应 L1 在 0.15 量级），允许 15 s 补完，$0.1 \times 15 = 1.5$，正好是取值。取得比这更小，积分会在补完之前被限幅削平，k_i 再大也无用。

每个基函数分到多少样本由 n 定。 $n = 20$、$d = 3$ 时节点间距 $h = 10/17 = 0.588$ mm，单基底支撑 $(d+1)h = 2.35$ mm，占区间的 23.5%。2000 个样本里约 470 个落在某个基函数的支撑内，这就是该分量估计噪声的微观来源。若把 n 加倍到 40，支撑减半、样本减半，噪声按 $\sqrt{2}$ 上升 41.4%——这也是 §73.6.4 里“加基函数不是免费的”的另一半原因。

四条的用法一致：先算，再取整，最后跑仿真确认。算出来的数与实测差得远，说明某个假设（线性、平稳、单自由度）已经不成立，这时候该回头看模型，而不是继续调参数。

73.7.8 λ 怎么定量选

岭参数没有解析最优值，但有一条可操作的选取规则：**取"能把常数项范数压到 1 以下、同时让残差 RMS 上升不超过 10%"的最小 λ。** 本章 10^{-3} 满足这两条：常数项范数 0.29652，残差 RMS 0.01019，与不加正则时的差别在第三位小数。

这条规则把 λ 的职责限定得很死：它只负责在共线时把解拉住，不负责"改善动态"。判据是残差——若调大 λ 之后残差明显上升，说明它已经开始改变动态，越界了。$\boldsymbol{A}_v$ 被人为加了阻尼，模型会以为分布收敛得更快，闭环随之迟钝，而操作工只会觉得"控制器反应慢"。

λ 还与基函数绑定。它作用在 $\boldsymbol{A}_v$ 的列上，列的量级随 n、d、区间长度变化；换一套基函数，原来合适的 10^{-3} 可能变成太松或太紧。所以 λ 要跟 n 一起记进交付文档，不能只写一个数。

73.7.9 四对互相拉扯的参数

参数之间不是独立的，有四对存在直接冲突。认清冲突，就不会指望"两个都调到最好"。

α 与输入余量。 大 α 快、但耗执行器；小 α 省、但慢。§73.6.8 的边际分析给出的结论是：收益在 $\alpha = 0.5$ 饱和，而 0.5 已经在限幅上，所以实际可选区间是 0.1 到 0.3。

N 与 T_s。 固定产量下两者成反比（§73.5.7）。周期减半，样本减半，估计误差涨 25.5%，速度的提升被噪声吃掉一部分。

n 与每基函数样本。 支撑宽度按 $1/n$ 缩小，单个基函数分到的样本同比减少，噪声按 $\sqrt{n}$ 上升。加密节点买到分辨率，付出的是噪声与指标口径变化（§73.7.7）。

k_i 与一致性。 积分压平失配敏感度，同时把无失配时的最好情况拖坏 13.3%。它买的是"最坏情况不变差"，付的是"最好情况变差"。

四对冲突的共同结构是：**每个参数都有一个"更好"的方向，但那个方向的代价落在另一个指标上。** 调参的本质是在这些冲突上选点，而不是把每个参数都推向极值。

73.8 调参方法与经验

分布控制的调参不是"试到好看为止"，而是按固定顺序排除。顺序错了，会把表示层的误差算到控制器头上，越调越乱。

73.8.1 固定顺序：五步

第一步，查数据。 看三件事：激励是否覆盖了工作区间、输入与权向量是否严格同拍、每周期样本数是否够。判据是 §73.3.4 的三项体检，任何一项不过就不要往下走。

第二步，查模型。 谱半径必须小于 1 且不贴着 1（本章 0.98484），残差 RMS 要与权向量估计噪声同量级（0.01019 对 0.01099），常数项范数不能是输入的十几倍以上（0.29652 对 0.076）。这三条有一条不对，后面的调参都是在拟合模型误差。

第三步，调 α。 从 0.1 起步，按 0.1、0.25、0.5 递增，每档跑 800 步，记录 IAE 与输入峰值。停在"输入峰值达到限幅 80%"的那一档，本章是 0.25（峰值 5.293，限幅 8.0）。

第四步，视需要加 k_i。 只有在半实物或投运阶段确认存在稳态偏差时才加，从 0.1 起步。加了之后要复查最好情况有没有被拖坏（§73.6.5 的"削峰填谷"）。

第五步，最后才动表示层。 改 n 或 d 意味着重新提权、重新辨识、重新调 α，等于回到第一步。这一条最容易被违反：很多人一看到误差降不下来就去加基函数，结果指标口径变了，前后数据不可比。

73.8.2 五个常见错误

表 73-5 列出本章开发过程中真实遇到、且具有普遍性的五个错误。每行都给出可观测的症状，便于现场对照。

表 73-5　五个常见错误与处置

错误	可观测症状	根因	处置
模型丢掉 $\boldsymbol{c}_v$	终点均值停在 2.993，L1 恶化到 0.7507，输入在 0.86–4.25 之间反复	平衡点算错，控制器以为零输入能维持分布	补常数项，按式 (73-13) 核对
输入与权向量不同拍	辨识出的 $\boldsymbol{B}_v$ 偏小，闭环“推不动”，加大 α 只换来饱和	输入记成了周期末的值	按周期起始的保持值记录
不做中心化、全通道加岭	常数项范数到千量级，训练残差很小但一投运就偏	回归矩阵共线，截距吸收了全部相关性	中心化，只惩罚 $\boldsymbol{A}_v$ 的列
α 取 1.0	输入长期贴在 8.0，L1 停在 0.07 不再下降，偶尔反向振荡	单步要求到位，必然饱和	降到 0.25，按 80% 限幅选
指标用 $\sum_i \lvert v_i - v_{g,i}\rvert$	n 越大“误差”越大，换基函数后无法比较	权值的绝对差随维数系统性变化	改用固定网格上的 L1（式 73-15）

第一条错误的数字最值得记住：L1 从 0.0473 恶化到 0.7507，差了一个数量级以上。它的症状很有欺骗性——输入并没有乱跳，只是在中间值附近反复微调，看起来“控制器在工作”，实际是把平衡点找错了。

73.8.3 一张真实的调参记录

表 73-6 是本章从第一版到定稿的五次改动。列出来不是为了展示过程，而是说明每一步的判据是什么。

表 73-6　本章的五次关键改动

轮次	改动	现象	结论
1	初始模型，无常数项	SDC 终点 L1 0.3790，优于开环 0.4406，劣于均值 PI 0.3616；$\rho(\boldsymbol{A}_v) = 1.00000$	模型没有稳态，控制律在目标点输出零
2	加入 $\boldsymbol{c}_v$	谱半径降到 0.98484，分布不再整体漂移	常数项必须辨识，不能省
3	个体增益改两簇，a 的离散度 0.020 降到 0.005	稳态 L1 降到 0.053	分布过散时“形状”无从谈起，先把对象本身弄干净
4	中心化，只惩罚 $\boldsymbol{A}_v$ 的列	常数项范数从 6259 降到 0.29652	共线必须处理，否则模型是假的
5	α 从 1.0 降到 0.25	输入峰值从 8.000 降到 5.293，IAE 4.263	不触限幅优先于过渡快

第 1 轮的结果最有教学价值：SDC 当时的表现比均值 PI 还差。原因不在控制律，而在模型——$\boldsymbol{A}_v$ 的谱半径恰好等于 1，意味着模型认为“不给输入分布也不衰减”，那么控制器在目标点上算出的输入就是零。这个错误在数值上只表现为一个 1.00000，肉眼几乎看不出来，只能靠体检发现。

73.8.4 三条经验

先看输入，再看误差。 输入长期贴在限幅上，说明是能力问题（执行器不够、或目标超出可达分布族），不是参数问题；此时调任何参数都只会改变“饱和的形状”，不会降低误差。

一次只动一个参数。 每改一次跑 800 步，只记录两个数：分布 IAE 与输入峰值。两个数同时改善才算改对了；只改善一个，多半是把误差搬到了别处。

记录写三列：改动、现象、结论。 本章表 73-6 就是按这个格式写的。没有记录的调参，两周后必然重复同样的错误——尤其“换个 n 试试”这类改动，前后指标根本不可比。

73.8.5 三个诊断实验

调参卡住时，先别动参数，跑三个能给出明确结论的实验。它们都不需要改控制器代码，只用已有的仿真或现场数据。

实验一，开环台阶扫描。 把输入从 0.3 到 7.5 分十二档，每档保持 300 步，记录每档的终点分布。这张“稳态分布族”回答的是可达性：目标分布若落在这条曲线上，单输入就能做到；若落不上，无论怎么调都只能逼近。本章 $u = 3.0$ 的开环终点 L1 是 0.0532，说明比例型目标基本可达；换成非比例目标后同一条曲线给出的最好成绩是 0.3699（§73.10.4），差了一个数量级，这时该谈的是加执行器，不是调参数。

实验二，输入与误差的互相关。 用激励数据算 $u[k]$ 与 $\Delta d_1[k] = d_1[k+1] - d_1[k]$ 的互相关，峰值应出现在滞后 0 到 1 拍。若峰值滞后 3 拍以上，先查时间戳，再查模型是否漏了额外的惯性环节；若互相关整体接近零，说明输入对分布几乎没作用，回到实验一确认执行器是否有效。

实验三，模型回放。 冻结 $\boldsymbol{A}_v$、$\boldsymbol{B}_v$、$\boldsymbol{c}_v$，用实测输入序列从实测初始权向量开始递推，比较预测与实测。判据有三条：回放误差应与体检残差 0.01019 同量级；误差不随步数增长；误差没有系统性偏向某一侧。误差线性增长说明 $\boldsymbol{A}_v$ 的特征值偏高（模型以为分布衰减得慢），误差截距整体偏移说明 $\boldsymbol{c}_v$ 不对。这个实验比单步残差更能暴露问题，因为单步残差小、多步回放发散，是“模型局部对、全局错”的典型症状。

三个实验的顺序不能反：先确认可达，再确认作用方向，最后确认模型精度。跳过第一个直接调参数，等于在一个到达不了的目标上优化。

73.8.6 五件不要做的事

不要用闭环运行数据做辨识。 闭环里输入是按误差算出来的，输入与噪声相关，回归得到的 $\boldsymbol{B}_v$ 会被系统性地低估，闭环表现为“推不动”。必须用独立的激励数据，或者在闭环中叠加足够大的外部激励（本章的 0.25 标准差扰动）。

不要在改了 n 之后比 L1。 换基函数等于换了度量尺，0.0150 与 0.0828 的差别大部分是尺子变了（§73.2.9）。要比就先固定 n，或者统一在 401 点网格上重算。

不要用单点终点误差验收。 同一条曲线的 30 s 与 80 s 能差 0.018（§73.6.7）。验收看窗口统计量。

不要把 k_i 默认打开。 它在无失配时会把最好情况拖坏 13.3%（0.0586 到 0.0664）。积分是补救手段，不是标配。

不要在输入已经贴限幅时调参。 输入峰值 8.000 说明能力不足或目标不可达，此时任何参数改动只改变饱和的形状。先按实验一判断属于哪一种。

73.8.7 按症状走：一张排障顺序

现场报“分布控制不好用”，先看输入，再看误差，最后看模型。这个顺序能把六种常见症状分开，避免一上来就调 α。

输入长期贴限幅。 先按 §73.8.5 实验一判断目标是否可达；可达则说明执行器余量不足，不可达则说明目标超出分布族。两种情况都不是参数问题，调 α 只会改变饱和的起止时刻。

输入正常、误差降不下去。 算一次方向余弦（§73.9.5 第二条）。低于 0.3 说明模型方向失效，属于 §73.9.4 那类结构性问题；高于 0.3 则查目标比例与固有比例的差，本章 4.5% 的差已经把 L1 从 0.0473 推到 0.1478。

输入抖动、分布跟着抖。 查 N。$N = 500$ 时估计误差占稳态 L1 的 48%（表 73-9），控制器会把统计噪声当成真实变化；其次查 k_i 是否把噪声积起来了（$i_{\lim}$ 太松时容易发生）。

分布缓慢单向漂移。 这是 $\boldsymbol{c}_v$ 的时变性（假设 A4 失效），不是控制器的问题。按 §73.10.3 的阈值触发重辨识；临时处置是打开积分并加泄漏。

换批后第一批超差。 按 §73.10.6 的四步倒查：目标权向量是否重算、u_{ss} 是否预估、积分是否清零、前三个周期是否只监视。

复现数字与文档不符。 先查脚本是否被改过（参数、种子），再查 NumPy 版本与 BLAS 后端。同版本同机器上数字应逐位一致；耗时是唯一例外（§73.12.3）。

六种症状里只有第三种是“调参数能解决”的，其余五种分别指向目标、模型结构、数据、维护与操作流程。这张顺序表的价值就是把“调参”从第一反应降到最后手段。

73.9 优化与改进

§73.1 到 §73.8 把一条基线走通了：B 样条表示、岭回归辨识、一步预测控制律、参数分层、五步调参。基线在 800 步下终点 L1 是 0.0473，分布 IAE 5.332。本节处理三类“再往前走一步”的问题：在既有结构内加补偿（积分、加权），在既有结构外处理约束，以及改变状态结构去覆盖多执行器场景。前两件有明确收益，第三件实测失败，失败的原因比成功更有信息量，所以一并写出来。

73.9.1 积分补偿

纯比例式的一步预测律有一个固有的弱点：它把模型当成真的。模型说“输入 u 能让分布走一步”，控制器就信这个 u。可实际增益是模型的 $1/\gamma_g$ 倍时，控制器的反解在稳态会停在一个偏离 $\boldsymbol{v}_g$ 的位置，偏差大小随 γ_g 线性增长。这个偏差不是振荡，也不是发散，就是安静地停在那里——最难被发现的一类问题。

积分补偿的做法是在控制律后面串一个权向量误差的积分项：

$$u[k] = \frac{\boldsymbol{B}_v^{\mathrm{T}}\left(\boldsymbol{v}_r[k] - \boldsymbol{A}_v \boldsymbol{v}[k] - \boldsymbol{c}_v\right)}{\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v} + \frac{k_i\, \boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{I}[k]}{\boldsymbol{B}_v^{\mathrm{T}}\boldsymbol{B}_v}, \qquad \boldsymbol{I}[k+1] = \mathrm{sat}_{i_{\lim}}\!\left(\boldsymbol{I}[k] + T_s\left(\boldsymbol{v}_g - \boldsymbol{v}[k]\right)\right) \tag{73-26}$$

积分状态 $\boldsymbol{I}$ 是 n 维向量，不是标量。这一点与常规 PI 不同：分布控制要积的是“哪一段差多少”，不是“总共差多少”。若把 $\boldsymbol{I}$ 退化成标量（比如只积均值误差），它就又回到了均值控制，峰宽、峰间距的偏差拿不到积分作用。

$\mathrm{sat}_{i_{\lim}}$ 是向量长度限幅，作用是抗积分饱和。限幅判断用欧氏范数而不是逐分量裁剪，因为逐分量裁剪会让方向发生畸变：本来指向“右峰该往左”的积分向量，被裁剪后可能变成“整体往右”，反而加深错误。

§73.6.5 已经给过这组参数的效果，这里换一个角度读。$k_i = 0$ 时五个失配点的 L1 从 0.0586 拉到 0.1542，跨度 2.6 倍；$k_i = 0.3$ 时五个点落在 0.0635 到 0.0677 之间，跨度只有 1.07 倍。**积分的作用不是把误差压到零，而是把“误差对失配的敏感度”压平。** 代价同样清楚：$\gamma_g = 0.9$ 这一点本来 0.0586 是最好的，加了积分反而升到 0.0664。

还有一个容易被忽略的细节：$\gamma_g = 1.0$（无失配）时，$k_i = 0.3$ 把 0.0708 压到 0.0652。按理说模型准确时积分无事可做，为什么还降了？因为这里的“失配”不止增益一项——样条表示误差（§73.6.4）、权向量估计噪声（$N = 2000$ 时 RMS 0.01099）、辨识残差（0.01019）都在把闭环稳态往边上推。积分对这些“没被建模的偏差”一视同仁，全都会补。所以积分增益的正确理解是“对一切未建模稳态偏差的补偿器”，而不只是“对付增益失配”。

反过来，这也是它的风险：现场若存在缓慢漂移的执行器或传感器零点，积分会把它当成模型误差去补，补到执行器长期偏置。工程上的处理是给 $\boldsymbol{I}$ 加泄漏（$\boldsymbol{I} \leftarrow 0.999\boldsymbol{I} + \cdots$），让积分只记住最近几十秒的偏差。本章没有加泄漏，因为仿真里没有慢漂移；半实物阶段必须加。

73.9.2 加权跟踪

工艺给的目标往往不是“整条曲线同等重要”。本章的双峰目标里，两个峰区（2.6–3.8 mm 与 4.2–5.5 mm）是合格品集中的地方，尾部差一点关系不大。这种“分区重要性”要在控制律里体现，最自然的入口是把代价函数写成密度域的加权平方：

$$J(u) = \int_{\Omega} \omega(y)\left[\hat{\gamma}(y; u, k) - \gamma_r(y)\right]^2 \mathrm{d}y \tag{73-27}$$

其中 $\omega(y)\ge 0$ 是工艺给的权重，$\hat{\gamma}$ 是一步预测密度。把 $\hat{\gamma}=\sum_i(\boldsymbol{A}_v\boldsymbol{v}+\boldsymbol{B}_v u+\boldsymbol{c}_v)_i B_i(y)$ 代入，J 是 u 的二次函数，令 $\mathrm{d}J/\mathrm{d}u=0$ 得

$$u[k]=-\frac{\int_\Omega \omega(y)\,[\varphi(y)-\gamma_r(y)]\,\psi(y)\,\mathrm{d}y}{\int_\Omega \omega(y)\,\psi^2(y)\,\mathrm{d}y},\qquad \varphi=\sum_i\left(\boldsymbol{A}_v\boldsymbol{v}+\boldsymbol{c}_v\right)_i B_i,\quad \psi=\sum_i\left(\boldsymbol{B}_v\right)_i B_i \tag{73-28}$$

$\psi(y)$ 可以读成“输入在这条分布曲线上各点的作用强度”。式 (73-28) 与式 (73-11) 在 $\omega\equiv 1$ 时同解，区别只在积分里多了一个 ω。

为什么加权要加在密度域，而不是直接给权向量分量乘权重？因为权向量分量的物理含义随 n 变化：$n=20$ 时第 7 个分量大致对应 3.4 mm 附近，$n=36$ 时同一位置落在第 12 个分量上。工艺人员说的是“3.2 到 3.8 mm 这一段更重要”，这句话在密度域是不变的，在权向量域每换一次 n 就要重新翻译一遍。把权重放在密度域，参数就与表示层解耦了——这与 §73.7 的分层原则一致。

实测见表 73-7。三个方案都跑 600 步，除加权与 α 外其余参数相同。

表 73-7　密度域加权跟踪的实测（600 步）

方案	全域 L1	峰区 L1	尾部 L1	分布 IAE	输入峰值
均匀（$\alpha=0.25$）	0.0708	0.0563	0.0156	4.263	5.293
加权（$\alpha=0.25$）	0.0647	0.0520	0.0136	3.834	8.000
加权降速（$\alpha=0.10$）	0.0647	0.0520	0.0136	4.380	8.000

三行数字给出两条结论，一条符合预期，一条不符合。

符合预期的是加权确实把误差往峰区搬……准确说，是三项一起降：全域 8.6%、峰区 7.6%、尾部 12.8%。尾部降得比峰区还多，说明这里的加权不是在两个区段之间做零和分配，而是整体把控制“用得更足”——在分布离目标还远的时候，加权项让控制器更愿意出力。

不符合预期的是输入峰值：从 5.293 直接顶到 8.000 的限幅。而且把 α 从 0.25 降到 0.10 也压不下来，终点 L1 一点没变（0.0647），IAE 反而从 3.834 升到 4.380。这说明峰值不是“过渡太快”造成的，α 管的是参考轨迹的快慢，管不住加权造成的瞬时增益放大。第二行的 IAE 比第一行低，第三行又比第一行高，正好卡在中间：加权本身降 IAE，降速又把 IAE 抬回去，两件事互相抵消。

工程含义很直接：**加权是一个“用执行器余量换形状精度”的旋钮，不是免费的**。启用前先算清余量。本章 800 步基线下输入峰值是 5.293，限幅 8.0，余量 34%；加权把这 34% 一次用完。若现场余量不足 30%，加权应当配合执行器扩容或降低目标难度，而不是硬开。

73.9.3 约束怎么处理

一步预测律有一个结构性局限：它只看一步。看一步就意味着它不知道“这一拍出大力，下一拍会撞上限幅”。本章的约束处理只有两层，都很朴素：

第一层是输出端的硬裁剪 $u\leftarrow\min(\max(u,0),U_{\max})$。裁剪本身不破坏稳定性，但会破坏“预测等于参考”这个前提：一旦裁剪生效，实际走的一步就小于控制器以为的一步，误差积累下去，下一拍控制器看到更大的误差、出更大的力，形成饱和—积累—更饱和的循环。§73.6.2 里 $\alpha=0.50$ 与 1.00 两档的 IAE 几乎一样（3.913 与 3.867），就是这种循环已经饱和的表现：再快也没有用。

第二层是参考滤波降速，也就是把 α 调小。它管的是“别要求一步到位”，实测有效：α 从 1.00 降到 0.10，输入峰值从 8.000 降到 3.117。但它管不住加权造成的瞬时增益变化（表 73-7 第三行）。

真正解决约束问题要把一步换成多步，也就是在预测时域上解一个带约束的二次规划：

$$\min_{\{u[k],\dots,u[k+N_p-1]\}} \sum_{j=1}^{N_p} \left\| \boldsymbol{v}[k+j] - \boldsymbol{v}_g \right\|^2 \quad \text{s.t.} \quad 0 \le u[k+j-1] \le U_{\max} \tag{73-29}$$

$\boldsymbol{v}[k+j]$ 用式 (73-7) 递推。这就是分布形式的预测控制，结构与第 58 章的 MPC 完全同构，只是状态从几个变量变成了 20 维权向量、约束从状态约束变成了输入约束。本章不展开，原因有两个：一是 $n=20$、$N_p=10$ 时每拍要解一个 200 变量左右的 QP，在线耗时会从 §73.10 的 0.38 ms 涨到毫秒以上，与本对象 0.1 s 的采样周期相比仍然够用，但对高节拍产线就不够了；二是本章的单输入、单自由度结构下，多步预测能拿到的改善有限（稳态族只有一维，终点误差改不动），收益主要体现在过渡段的限幅规避上。n 更大、输入更多时，这笔账才算得过来。

73.9.4 双执行器扩展：一次失败的分簇尝试

前面所有讨论都建立在一个前提上：只有一路输入。真实产线上更常见的是“两批工件、两路独立执行器”，比如两条并行的涂层线分别调气量。这时直觉上应该把状态扩成“两批各自的分布”，控制律按块对角拼起来：

$$\boldsymbol{v}_1[k+1] = \boldsymbol{A}_1 \boldsymbol{v}_1[k] + \boldsymbol{B}_1 u_1[k] + \boldsymbol{c}_1, \qquad \boldsymbol{v}_2[k+1] = \boldsymbol{A}_2 \boldsymbol{v}_2[k] + \boldsymbol{B}_2 u_2[k] + \boldsymbol{c}_2 \tag{73-30}$$

u_1 只作用于低增益批次，u_2 只作用于高增益批次。设定占比 60%/40%，两千个粒子实际抽出来是 0.610 与 0.390，差 0.01 属于抽样波动。目标位置取 3.00 mm 与 4.30 mm——与 §73.10 的单输入非比例目标同一组，便于对照。两个峰位之比是 $4.30/3.00 = 1.433$，而对象能给出的固有比例是 $b_2/b_1 = 1.5$，差 4.5%，单输入无论怎么调都凑不出来，只能靠两路输入各自定位。

第一版实现犯了一个结构性错误：把两批粒子合在一起统计一条 $\boldsymbol{v}$，再对两路输入做多元回归。结果 L1 爬到 1.5253，u_1 全程停在 0.00。原因很清楚——**合并后的权向量不能观**。同一个 $\boldsymbol{v}$ 分量里既有低增益粒子的贡献，也有高增益粒子的贡献，模型看到 $\boldsymbol{v}$ 根本分不清“这一格里有多少低增益粒子”，也就无法判断 u_1 该起多大作用。线性回归在这种情况下只能把 u_1 的系数压到零，把全部解释力让给 u_2。这不是数值问题，是状态定义错了。

第二版按式 (73-30) 分簇统计、各自辨识、各自控制。结果仍然失败，但失败的方式变了，而且这次能定位：

表 73-8　双执行器分簇模型（$n=20$，两路各 3000 步激励）

量	低增益批	高增益批
辨识残差 RMS	0.01255	0.02157
谱半径	0.99891	0.99852
$\lVert \boldsymbol{B} \rVert$	0.02649	0.02396
模型稳态输入 u_{ref}	2.774	2.595
初始状态的一步解 $u[0]$	-1.364	14.416
方向余弦 $\cos\theta$	-0.043	0.400

表中 $\cos\theta$ 是 $\boldsymbol{B}_v$ 与 $\boldsymbol{v}_g - \boldsymbol{A}_v \boldsymbol{v}[0] - \boldsymbol{c}_v$ 的夹角余弦，衡量“模型允许的推动方向”与“实际需要的推动方向”重合多少。

u_{ref} 是对的：低增益批的真实稳态输入是 $3.00 \times 0.148/0.160 = 2.775$，模型算出 2.774，误差千分之一。这说明模型的**平衡点**是准的。但初始状态的一步解是 -1.364——负输入，被裁剪成 0 以后，分布反而往下掉，误差越来越大，闭环 L1 停在 1.5439，远不如基线。

问题出在 $\boldsymbol{B}$ 的方向上。$\cos\theta = -0.043$，意思是模型认为“输入只能沿这个方向推动分布”，而这个方向与“把峰从 1.20 mm 搬到 3.00 mm 所需要的方向”几乎正交。一步预测律只买得到一个方向，方向错了，反馈每拍都在往错的方向推。两批合起来的闭环终点 L1 是 1.4822，分布 IAE 113.979，比基线的 0.0473 与 5.332 差两个数量级。

为什么会这样？把初始状态的一步预测拆成“幅值”和“方向”两个量来看，对比就很清楚：

模型	真值步长 L1	预测步长 L1	幅值比	方向余弦 $\cos\theta$	闭环 L1
单执行器（基线）	0.5199	0.1650	0.32	$+0.584$	0.0473
分簇低增益	0.3885	0.1687	0.43	-0.043	1.5439

单执行器模型的幅值比只有 0.32——它把一步的变化低估了三分之二，闭环照样收敛到 0.0473。分簇模型的幅值比是 0.43，比基线还“准”，闭环却崩了。**这说明分布控制里幅值错可以容忍，方向错不能容忍**：幅值错只影响每拍的步长，下一拍重新算方向时误差会被修正（这就是 §73.6.1 里投影只吃掉一个方向的另一面）；方向错则每拍都在加深错误，没有任何机制能把它掰回来。

那为什么单执行器模型的方向对、分簇模型的方向错？差别在行程与峰宽的比例。基线里两簇被同一个输入拴住，峰间距被对象的固有比例锁死，分布始终在辨识数据覆盖的那条一维族附近演化；分簇之后，每簇要从 1.20 mm 单独搬到 3.00 mm，行程 1.80 mm，是峰宽 0.28 mm 的 6.4 倍。权向量在这段行程上走的是一条弯曲的轨迹（峰在基函数之间穿行），而一个常矩阵 $\boldsymbol{A}$、常向量 $\boldsymbol{B}$ 只能描述其中一小段切线。在起点处，这条切线已经被整段行程“平均”掉了，方向自然对不上。

最直接的验证是加密节点。若原因是表示太粗，把 n 从 20 加到 40，方向余弦应当明显改善。实测没有：$n=20$ 时 -0.043，$n=30$ 时 $+0.005$，$n=40$ 时 $+0.024$。三个值都在零附近。**所以这不是表示精度问题，是一阶线性模型在“大行程、窄峰”下的结构性失效。** 这与 §73.6.4 的观察是同一件事的两面：那里说样条表达不了 0.28 mm 的尖峰，这里说线性模型也表达不了尖峰的长距离平移。

73.9.5 从失败里提炼的三条判据

这次失败不是终点，而是给出了三条可以在动手之前就检查的判据。

第一，先算行程比。 定义 $\kappa = |y_g - y_0|/\sigma$（目标位置与初始位置之差除以峰宽）。本章基线的单簇行程是 2.0 与 3.65 mm，峰宽 0.28 mm，按单簇算 κ 到了 7 到 13——但基线里两簇被同一个输入拴住，必须整体看待，特征宽度是峰间距 1.575 mm，κ 只有 1.3，所以模型够用。判据应当按**当前真正被独立控制的那条分布的宽度**算：整条双峰一起动时用峰间距，单峰单独动时用峰宽。κ 超过 3 到 4 时，一阶线性模型的方向可信度就要打问号，应当先做一步预测的方向检查再谈闭环。

第二，把方向检查写进模型体检。 §73.3.4 的三项体检（谱半径、残差 RMS、常数项范数）都只管幅值与稳定性，不管方向。应当加第四项：在初始状态与目标状态各算一次 $\cos\big(\boldsymbol{B}_v,\ \boldsymbol{v}_g - \boldsymbol{A}_v\boldsymbol{v} - \boldsymbol{c}_v\big)$，低于 0.3 就不要投闭环。这一项几乎不花时间，却能提前发现表 73-8 那种“稳态对、瞬态全错”的模型。

第三，状态不能观时先拆状态，不要加输入。 第一版把两路输入塞进一个合并状态，回归自动把其中一路的系数压成零。这是线性模型在“提示”：给它的状态不足以解释两路输入的作用。遇到这种情况应当先改状态定义（分簇统计、或把两个批次的占比也放进状态），而不是加大激励、换基函数、调正则——那些手段都改不了能观性。

沿着这三条判据，双执行器的可行路线有三条，按代价从低到高：

一是**分区分段线性**。按峰位把行程分 3 到 4 段，每段单独辨识一组 $(\boldsymbol{A}_v, \boldsymbol{B}_v, \boldsymbol{c}_v)$，按当前峰位切换。代价是激励数据要覆盖每段、模型要存几套，切换点附近可能出现抖振，需要加滞环。这条路线与第 76 章的切换系统思路一致。

二是**在线自适应**。闭环运行时用最近的 $(\boldsymbol{v}, u, \boldsymbol{v}^{+})$ 持续更新模型，让 $\boldsymbol{B}_v$ 跟着当前工作点走。这也是分布控制文献里的主流做法，代价是要处理激励不足时的参数漂移，通常要给 $\boldsymbol{A}_v$ 加强制性正则或遗忘因子。

三是**换状态**。不用整条权向量，改用几个矩（均值、方差、峰位、峰高）作为状态。矩的维数低、物理意义清楚，行程大时非线性也更容易处理；代价是丢掉形状细节，双峰这类多模态分布用矩描述不了。若工艺只关心“两峰各在哪、各有多宽”，矩方法比 20 维权向量更合适。

这三条都不在本章的仿真范围里，写在这里是为了把边界划清楚：**本章的方法是“小行程、单执行器、形状由单一输入驱动”场合的解法；行程一大、执行器一多，就得换上述三条之一。**

73.9.6 四条改进路线的账

本节一共给了四条改进路线，收益与代价差别很大，混着用会互相抵消。逐条算一遍：

- **积分补偿**：收益确定。增益失配 $\gamma_g = 1.2$ 时终点 L1 从 0.1542 降到 0.0635。副作用也确定：无失配时 0.0586 被抬到 0.0664。额外计算量可以忽略（一个 n 维向量的累加与一次限幅），额外风险是长时间运行会吸收慢漂移。适用的问题是“稳态停偏”。
- **密度域加权**：全域 L1 从 0.0708 降到 0.0647，改善 8.6%，但输入峰值从 5.293 顶到 8.000，把 34% 的执行器余量一次用完。它买的是形状精度，付的是余量。适用的问题是“某一段比另一段重要”。
- **多步带约束 QP**：本章没有实测，规模估算是 200 个优化变量。它唯一能解决的是“这一拍出大力、下一拍撞限幅”，也就是饱和与约束冲突。适用的问题是“约束真的在起作用”。
- **换状态结构**（分区分段、在线自适应、矩状态、Koopman）：唯一能解决 $\kappa > 3$ 的方向失效，代价是数据量翻倍与长期维护成本。适用的问题是“方向余弦过不了 0.3”。

四条的适用问题互不重叠，这是选择它们的第一原则：**先判断问题类型，再选路线。** 用加权去治方向失效，只会把输入顶到限幅而误差不动（表 73-7 第三行就是这种情况的雏形）；用积分去治饱和，只会让积分在限幅期间越积越大，退出饱和后反冲。判断不了类型时，回到 §73.8.5 的三个诊断实验。

73.9.7 一步预测律的四种等价写法

式 (73-11) 不是唯一的写法，把它写成四种形式，能看清每种改进加在了哪一层。

投影形式。 记 $\boldsymbol{d} = \boldsymbol{v}_r - \boldsymbol{A}_v \boldsymbol{v} - \boldsymbol{c}_v$，则 $u = \boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{d} / \boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{B}_v$ 就是 $\boldsymbol{d}$ 在 $\boldsymbol{B}_v$ 方向上的投影长度。这是最本质的读法：一步预测律做的事，是把“需要的修正”投影到“唯一能推动的方向”上。

最小二乘形式。 $\min_u \|\boldsymbol{d} - \boldsymbol{B}_v u\|^2$ 的解与投影形式完全相同。写成最小二乘的好处是可以直接加约束（$0 \le u \le U_{\max}$），这时解不再是解析式，但问题仍是凸的，一维情况下只是把解裁剪到区间内。

加权最小二乘形式。 $\min_u (\boldsymbol{d} - \boldsymbol{B}_v u)^{\mathrm{T}} \boldsymbol{W} (\boldsymbol{d} - \boldsymbol{B}_v u)$ 的解是 $u = \boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{W} \boldsymbol{d} / \boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{W} \boldsymbol{B}_v$。这里的 $\boldsymbol{W}$ 作用在权向量分量上，与 §73.9.2 作用在密度域的 $\omega(y)$ 不是一回事。本章实测过前者：全域 L1 从 0.0708 恶化到 0.3001，原因是权向量分量的物理含义随 n 变化，分量加权把输入方向扭了。这一条失败记录说明“权重加在哪一层”比“加不加权”更重要。

带输入惩罚形式。 $\min_u \|\boldsymbol{d} - \boldsymbol{B}_v u\|^2 + \rho u^2$ 的解是 $u = \boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{d} / (\boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{B}_v + \rho)$，就是式 (73-9)。$\rho$ 与 $\boldsymbol{B}_v^{\mathrm{T}} \boldsymbol{B}_v$ 不同量级时它不起正则作用，只起断路作用（§73.7.3）。

四种写法在 $\boldsymbol{W} = \boldsymbol{I}$、$\rho = 0$ 时完全同解，差别只在“把‘接近’定义在哪个空间”。选哪一种，取决于约束、权重、正则这三件事里哪一件真的需要。

73.10 工程落地

前面九节讨论的是“方法对不对”，这一节讨论“能不能装到线上”。四件事按落地顺序排：先定样本量（数据够不够），再算在线耗时（算得过来吗），然后定模型维护策略（模型会不会过期），最后处理一类单输入做不到的目标（能力边界）。

73.10.1 样本量与更新周期

权向量是从样本估出来的，样本少则估计噪声大。把噪声压到多小才够？判据不是“越小越好”，而是与目标精度比。本章稳态 L1 是 0.0473，若权向量本身的估计误差就和它同量级，控制器的输入里有一半是在追噪声。

表 73-9 是固定一条目标分布、重复抽样 30 次得到的权向量估计误差（欧氏范数，30 次取均方根）。

表 73-9　样本量对权向量估计的影响（30 次重复）

每周期样本数 N	估计误差 RMS	与 $N=100$ 相比	相对稳态 L1
100	0.05035	1.00	106%
200	0.03752	0.75	79%
500	0.02260	0.45	48%
1000	0.01379	0.27	29%
2000	0.01099	0.22	23%
5000	0.00809	0.16	17%

两个读法。第一，误差大致按 $N^{-1/2}$ 下降：N 从 100 增到 5000 是 50 倍，误差从 0.05035 降到 0.00809 是 6.2 倍，与 $\sqrt{50}=7.07$ 接近，说明估计量已经接近无偏，继续加样本只能按平方根换精度。第二，$N=500$ 时估计误差已经是稳态 L1 的 48%，$N=2000$ 降到 23%，$N=5000$ 也只到 17%——**再加样本的边际收益很低，因为剩下的误差不来自抽样，来自样条表示本身**（§73.6.4 里目标峰高被压低 42%）。

所以本章取 $N=2000$，理由是“估计噪声降到稳态误差的 1/4 以下，同时不明显增加节拍负担”。这个数不是普适的：若工艺要求更低的稳态 L1（比如 0.02），$N=2000$ 的 0.01099 就占到 55%，必须把 N 提到 5000 以上，或者先解决表示精度。

图 73-10 左幅把表 73-9 画成双对数坐标：六个点几乎落在一条斜率为 $-1/2$ 的直线上，这就是 $N^{-1/2}$ 律的直接证据，也是“再加样本收益递减”的图形表达。

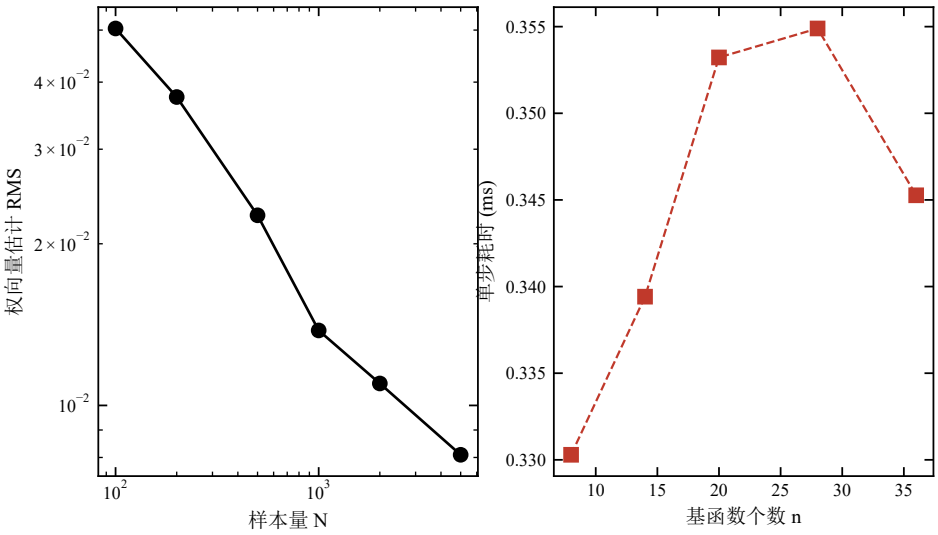


图 73-10　样本量对估计误差的影响与在线单步耗时（左：N 与估计 RMS；右：n 与耗时）

工程上还有一个容易搞错的地方：N 不必等于一个节拍内的零件数。多数产线的节拍是秒级，一个节拍只有几十件，凑不出 2000 个样本。正确的做法是把分布当成**慢变量**：用滑动窗口累计 M 个节拍的样本，每 M 个节拍更新一次分布、算一次输入。此时模型里的 T_s 是“分布更新周期”，不是零件节拍，M 个节拍内的输入保持不变。

这个改动会连带改模型：T_s 从 0.1 s 变成比如 3 s，一阶惯性的离散参数 a 必须按新周期重新换算（a^M），否则模型会以为对象比实际快 M 倍。本章的 $a=0.852$ 对应 0.1 s；若更新周期变成 3 s 且对象时间常数不变，a 会掉到 $0.852^{30}\approx 0.008$，几乎是纯静态映射，此时“分布控制”退化成一个无记忆的代数反解，α 反而要取接近 1 才有响应。这一步换算在投运时最容易漏，症状是“模型辨识残差很大，但闭环还能跑，只是慢得莫名其妙”。

73.10.2 在线计算

表 73-10 是单步在线耗时，2000 个粒子、300 次平均，n 从 8 到 36。图 73-10 右幅是它的图形版本，曲线几乎水平。

表 73-10　在线单步耗时（2000 粒子，300 次平均，一次实测）

n	权重提取	控制解算	合计
8	0.346 ms	0.002 ms	0.349 ms
14	0.326 ms	0.002 ms	0.329 ms
20	0.379 ms	0.003 ms	0.382 ms
28	0.366 ms	0.002 ms	0.368 ms
36	0.334 ms	0.002 ms	0.337 ms

结论只有一条，但很重要：**耗时的 99% 在权重提取，不在控制律**。控制解算是一个 n 维内积除以一个标量，五档都在 0.002 到 0.003 ms，可以忽略。权重提取要把 2000 个粒子逐个映射到基函数上，每个粒子只落在 $d+1=4$ 个基函数上，实际代价是 $O\big(N(d+1)\big)$，与 n 无关，所以五档的耗时几乎持平（0.326 到 0.379 ms，差异在测量抖动范围内）。

这个分布决定了优化的方向：想让在线更快，应当减少粒子数或改用直方图 + 查表，而不是去简化控制律或减少基函数。反过来，若想提高形状分辨率（加 n），在线成本几乎不变，真正的代价在样本量（每个基函数分到的样本变少）与指标口径（§73.6.4）。

0.38 ms 相对 0.1 s 的采样周期有 260 倍余量，即使按 §73.9.3 的多步预测把控制解算放大到毫秒级，仍然充裕。所以“计算能力”通常不是 SDC 落地的障碍，“数据”才是——尤其是每周期凑够 2000 个样本这件事。

73.10.3 模型维护

辨识出来的 $\boldsymbol{A}_v$、$\boldsymbol{B}_v$、$\boldsymbol{c}_v$ 会过期。过期不是突然失效，而是缓慢漂移：换料、换模、设备磨损、环境温湿度变化，都会改变个体参数 a_i、b_i 的分布，进而改变权向量的演化规律。工程上要回答三个问题：什么时候重辨识、用什么数据重辨识、怎么保证换模型不出事。

什么时候重辨识。 盯三个量，任一触发就重辨识：一是辨识残差 RMS 上浮超过 50%（本章基线 0.01019，超过 0.015 就要查）；二是谱半径漂移超过 0.01（基线 0.98484）；三是闭环稳态 L1 连续三个更新周期高于投运时的 1.5 倍。前两个是从在线数据直接算的，不需要停线；第三个是效果指标，最可靠但最滞后。三者配合，可以在效果明显恶化之前就动手。

用什么数据。 最理想是专门的激励试验，本章用的是随机游走（幅度 0.25、限幅 0.3 到 7.5）。产线上不能为了辨识专门扰动，可行的替代有两个：一是利用换规格、换批次的自然过渡段（输入本来就要大范围变化）；二是定期做小幅阶跃（幅度控制在质量可接受范围内）。自然过渡段的问题是不满足持续激励条件——输入单调爬升时，$\boldsymbol{A}_v$ 与 $\boldsymbol{B}_v$ 无法分离。谱半径越接近 1，这个分离误差被放得越大：分簇模型（谱半径 0.99891）用 500 步激励算出的稳态输入是 6.238，真值 2.774，偏大 1.25 倍；同一模型用 3000 步激励才算回 2.774。所以自然数据只能用于“体检”，不能用于“重建模型”。

怎么换模型不出事。 三条：新模型先在影子模式下并行跑（只算输入不执行），与现模型的输入偏差连续 10 个周期在 20% 以内才切换；切换时保留旧模型，出问题 30 s 内可回滚；切换后第一个周期把积分状态 $\boldsymbol{I}$ 清零，避免旧模型的积分余量把新模型带偏。

73.10.4 单输入做不到的目标

§73.1.4 留了一个悬念：把双峰目标改成 3.00 mm 与 4.30 mm（峰位比 1.433，固有比例 1.5，差 4.5%），三种方案的表现如何。表 73-11 是实测。

表 73-11　非比例双峰目标的三种方案（800 步）

方案	终点 L1	分布 IAE	终点均值	终点标准差
开环 $u=3.0$	0.3699	30.114	3.876	0.841
均值 PI	0.1334	12.731	3.520	0.766
SDC	0.1478	11.484	3.609	0.785

三行数字里最刺眼的是：**SDC 的终点 L1 比均值 PI 还差 10.8%**（0.1478 对 0.1334）。这与比例目标下的结论完全相反（那里 SDC 0.0473、PI 0.0466，基本持平）。为什么？

因为目标不可达。单输入只能沿一条一维曲线移动稳态分布，这条曲线上峰间距恒为固有比例的 1.5 倍。目标要求 1.43 倍，曲线上的任何一点都不等于目标。此时控制器面临的是一个“怎么分配不可消除的误差”的选择题：SDC 按权向量的欧氏误差分配，倾向于把两个峰都往中间收，收出 0.1478；均值 PI 只管均值，两个峰各自按固有比例摆放，恰好落在离目标 L1 更近的位置，得到 0.1334。**这不是 SDC 的算法出错，而是它的代价函数与 L1 指标不是同一个东西**——欧氏距离下最优的解，在 L1 意义下未必最优。若把代价换成 L1（不可微，要解线性规划），SDC 不会输给 PI，但工程上不值得为一个本来就不可达的目标这么做。

过程量上 SDC 仍然赢：IAE 11.484 对 12.731，低 9.8%；开环则以 30.114 远远落后。这符合“分布控制的收益在过渡段”的一贯结论。

工程含义有两条。第一，**先判断目标可不可达，再谈用哪种控制**。可达性判据很直接：算一下目标的两个峰位之比，与对象固有比例（本章 $b_2/b_1=1.5$）比，差得远就说明单输入做不到，此时应当先改工艺（两路执行器、改变批次配比、改变目标本身），而不是换控制器。第二，若确实只能用单输入逼近一个不可达目标，应当显式选择“保什么”：保均值就用 PI，保峰间距就用带形状约束的分配，不要指望 SDC 的默认欧氏代价自动做出正确取舍。

73.10.5 投运清单

把前面各节的判据收成一张清单，按顺序过，任何一项不过就不要进入下一项。

表 73-12　SDC 投运检查清单

序	检查项	判据	依据
1	目标可达性	峰位比与固有比例之差在 2% 内；本章 4.5% 的偏差已让终点 L1 从 0.0473 恶化到 0.1478	§73.10.4
2	每周期样本数	$N \geq 1000$，估计误差不超过稳态 L1 的 30%	§73.10.1
3	更新周期与 a 的换算	a 按实际更新周期重算，不能沿用零件节拍的 a	§73.10.1
4	激励覆盖	输入激励覆盖整个工作区间，且输入与权向量严格同拍	§73.3.4
5	模型体检	谱半径 <1 且不贴 1；残差 RMS 与估计噪声同量级；常数项范数不异常	§73.8.1
6	方向体检	初始与目标状态的方向余弦 ≥ 0.3	§73.9.5
7	输入余量	α 取到峰值不超过限幅 80% 的那一档	§73.6.2
8	在线耗时	单步耗时不超过更新周期的 10%	§73.10.2
9	回滚预案	影子模式、旧模型保留、积分清零	§73.10.3

九项里前六项是“能不能用”，后三项是“敢不敢投”。实际项目里卡住最多的不是算法，而是第 2 项（样本凑不够）与第 4 项（激励不满足）——这两项都指向同一件事：**SDC 吃数据**。数据不够时，退回到均值控制加统计过程控制（SPC）监视分布宽度，比硬上 SDC 更稳妥。

73.10.6 换型与换批的操作顺序

换型是投运后最频繁的动作，也是最容易出事的地方。顺序错一步，往往表现为“第一批全部超差”，而操作工只会归因于控制器不好用。正确顺序是四步。

第一步，重算目标权向量。 用新的目标参数（峰位、峰宽、占比）生成 20 万个抽样点，再用与在线估计**完全相同**的一套基函数投影。不要用旧目标的权向量做平移近似：峰位移动 1 mm 在 20 个基函数上不是简单的分量搬移，近似会带来 0.1 量级的 L1 偏差，比控制误差本身还大。

第二步，预估新稳态输入。 按式 (73-14) 算 u_{ss}，作为新目标的期望工作点。这个数的准确性直接决定第一批的过渡时间：本章 u_{ss} 模型值 2.774，真值 2.775，差千分之一，所以控制器一开始就工作在正确的工作点附近，而不必从零爬。若预估偏差超过 10%，说明模型与新目标的组合超出了辨识数据的覆盖范围，应当先补激励再换型。

第三步，清积分状态。 旧目标下积累的 $\boldsymbol{I}$ 指向旧目标，带着它换型等于给新目标叠了一个错误的偏置。清零是必须的，且要在切换的同一拍完成。

第四步，前三个更新周期只监视。 这三个周期不启用积分、不切模型，只记录 L1 与输入峰值。判据是 L1 单调下降且输入不触限幅；若第一拍输入就到 8.000，说明新目标超出可达范围，应当停机检查而不是继续跑。

换批与换型不同：换批只改占比（例如 60%/40% 变成 50%/50%），峰位与峰宽不变。此时只需要按新占比重算 $\boldsymbol{v}_g$，模型不动，但残差监控阈值要收紧（从 0.015 收到 0.012），因为占比变化会改变 $\boldsymbol{c}_v$ 的有效值（假设 A4 的边界）。

73.10.7 数据存档的最小集合

投运后吵架最多的场合是“这批为什么超差”。没有存档，就只能靠回忆；有了存档，可以按 §73.8.5 的实验三把当时的数据回放一遍，直接看出是模型过期、目标不可达，还是执行器饱和。

最小集合是六项：带时间戳、批号、工位的原始测量；输入序列（含限幅后的实际值）；权向量序列或等价的直方图；模型参数与三项体检结果；脚本版本与随机种子；当批的验收指标表。

存储量要提前算，否则第一周就把磁盘写满。按本章的规模，每拍 2000 个样本、每 0.1 s 一拍，双精度 8 字节，原始数据速率是 $2000 \times 8 / 0.1 = 160$ kB/s，一天 13.8 GB。这个量不适合长期保存。可行的折中是：原始样本按批保留最近若干批，长期只存 500 桶的直方图，每拍 $500 \times 4 = 2000$ 字节，速率 20 kB/s，一天 1.7 GB，压缩后还能再降一个量级。

存档的粒度也有讲究。权向量序列必须与输入序列严格同拍，否则回放时会人为制造一个错拍，把好好的模型判成“残差过大”。这一条与 §73.3.3 的采集要求是同一个要求，只是发生在两个不同的时间点。

73.10.8 每批只报三个数

投运后最容易走形的不是算法，而是“没人看”。报表要短到操作工愿意看，所以每批只报三个数：

- **窗口 L1**：最后 10 s 的均值与最大值。基准约 0.055 与 0.0654（§73.6.7）。报警限取投运值的 1.5 倍，连续三批触发才报警，避开单批波动。
- **输入峰值**：超过 $0.9 U_{\max} = 7.2$ 就该查，超过 8.000 说明已经在饱和区工作。这一项比 L1 更早报警，因为它先于效果恶化。
- **两个峰位**：相对目标的偏差，门限是半个峰宽 0.14 mm。峰位比 L1 更直观，操作工看得懂“右边这个峰偏了 0.1 mm”，看不懂“L1 是 0.06”。

三个数之外，每月做一次模型复核：用当月数据重算残差 RMS，超过 0.015 触发重辨识（§73.10.3）；同时核对存档是否完整，特别是输入序列与权向量序列是否同拍。

报表的价值在于把“分布控制还在不在工作”变成一个可查的数字。没有这三个数，出问题时只能凭印象争论；有了它们，争论变成“哪一批开始超的、当时输入峰值多少”，排障也就有了明确的入口。

73.11 优缺点与适用边界

这一节不重复前面的推导，只做三件事：把优点与缺点各自数清楚，给出“什么条件下该用、什么条件下不该用”的判据，以及把它与相邻几类方法的边界划开。

73.11.1 优点

第一，它控的是整条分布，不是几个统计量。 均值、方差、偏度、峰位这些量，任何一个单独看都可能骗人。本章的双峰分布均值 3.86 mm，与单峰分布均值 3.86 mm 完全一样，但合格率差得远。SDC 直接把整条曲线当目标，不经过“挑几个统计量”这一步，也就不会挑错。表 73-1 里 SDC 与均值 PI 的终点 L1 几乎相同（0.0473 对 0.0466），那是因为本对象恰好只有一个自由度；过程量上 SDC 的 IAE 低 23.0%，这才是分布信息的实际价值。

第二，模型是可辨识的线性模型，不是黑箱。 整个方法的核心只有式 (73-7) 一个线性差分方程，20×20 的 $\boldsymbol{A}_v$ 加一个 $\boldsymbol{B}_v$，用最小二乘就能估，估完可以逐项体检（谱半径、残差、常数项范数）。这与神经网络类方法形成鲜明对比：后者可能在拟合上更好，但你没法回答“为什么这个输入是 5.29 而不是 5.30”。

第三，控制律的计算量几乎为零。 表 73-10 里控制解算只占 0.002 到 0.003 ms，全部耗时都在权重提取。这意味着控制器本身可以跑在很便宜的硬件上，算力预算可以全部让给“把分布估准”。

第四，参数分层清楚，出问题能定位。 表示层（n、d）、辨识层（λ、N、T_s）、控制层（α、ρ、k_i）三层各管一件事，每层有独立的判据（§73.7、§73.8）。工程上最怕的不是参数多，而是“改了一个不知道影响了什么”；分层把这个问题消掉了。

第五，可以直接吃工艺语言。 “3.2 到 3.8 mm 这段更重要”这种话，在均值控制里无处安放，在密度域加权里可以原样写进代价函数（式 73-27）。加权实测把全域 L1 降了 8.6%，代价是执行器余量（表 73-7）。

第六，退化路径明确。 目标退化成单峰窄带、$\alpha \to 1$、$k_i = 0$ 时，SDC 的行为接近一个带形状约束的静态反解，不会再引入额外的稳定性风险（式 73-25 保证 $\rho < 1$ 即收敛）。这比“要么全用、要么不用”的方法好落地：可以先在一条产线上做分布监视与开环建议，再逐步闭环。

73.11.2 缺点

第一，吃数据。 每周期至少 1000 个样本，$N = 2000$ 才能把估计噪声压到稳态误差的 23%（表 73-9）。一个节拍只有几十件的产线，必须靠滑动窗口把更新周期拉长到几个节拍到几十个节拍，这时模型里的 a 要按新周期重算（§73.10.1）。这是所有“分布级”方法的共同代价，不是本章实现的缺陷。

第二，一阶线性模型的适用范围窄。 这是本章最硬的一条限制。§73.9.4 的实测表明：行程比 κ 一旦超过 3 到 4，$\boldsymbol{B}_v$ 的方向就不可信，闭环从“能用”直接掉到“完全不能用”（0.0473 对 1.4822）。而且加密节点救不回来（$n = 20/30/40$ 的方向余弦都在零附近）。这不是调参能解决的问题，只能换模型结构。

第三，稳态族只有一个自由度时，终点收益有限。 本对象就是这种情况：终点 L1 三种方案相差不到 1.5%，SDC 的收益全在过渡段。若工艺只关心稳态合格率、不在乎过渡，SDC 的投入产出比不如“均值控制 + SPC 监视宽度”。

第四，表示精度是一个隐藏的误差源。 $n = 20$、峰宽 0.28 mm 时目标峰高被压低 42%（解析 0.855，样条 0.496）。这个误差不在 L1 里显式出现，但会通过 $\boldsymbol{c}_v$ 与 $\hat{\boldsymbol{v}}_g$ 一路传到控制量。若把 L1 当成物理精度指标交付，会误导下游（§73.6.4）。

第五，目标不可达时，默认代价函数会做出错误取舍。 表 73-11 里 SDC 的终点 L1 比均值 PI 差 10.8%。根本原因是欧氏代价与 L1 指标不一致，不是算法错误。现场若遇到不可达目标，必须显式选“保什么”，不能指望默认配置。

第六，积分项与慢漂移不兼容。 积分的本意是补模型误差，但它分不清“模型错”与“设备漂”（§73.9.1）。没有泄漏的积分在长时间运行后会把执行器推到一个偏置位置，看起来“控制得很好”（分布对上了），实际是靠执行器长期偏置换来的。

73.11.3 适用边界

把上面的定性说法收成一张可核对的表。

表 73-13　SDC 的适用边界

条件	判据	本章取值	越界后果
目标可达	目标峰位比与对象固有比例之差	4.5%（越界）	终点 L1 0.0473 恶化到 0.1478，且不如均值 PI
行程比	$\kappa = \lvert y_g - y_0 \rvert / \sigma$	1.3（合规）/ 6.4（越界）	方向余弦从 $+0.584$ 掉到 -0.043，闭环失效
输入维数	独立执行器个数	1	两路输入共用一套状态时不能观，u_1 被压成 0
每周期样本	N 与稳态 L1 之比	2000 / 23%	$N = 500$ 时估计噪声占稳态误差 48%
表示精度	峰宽与节点间距之比	0.56	峰高被压低 42%，L1 口径失真
模型线性度	初始状态的方向余弦	$+0.584$	低于 0.3 时不要投闭环
执行器余量	输入峰值与限幅之比	5.293 / 8.0 = 66%	加权后顶到 8.000，α 压不下来
更新周期	单步耗时与更新周期之比	0.38 ms / 0.1 s = 0.4%	长周期时 a 要重算，否则模型偏快 M 倍

八行里前四行是“能不能用”，后四行是“用得好不好”。注意第二行出现了两个值：基线合规（1.3）、双执行器越界（6.4）。同一个方法在同一个对象上，只因为“被独立控制的那条分布变窄了”，适用性就从合格变成不合格——这是本章最值得记住的一条边界。

73.11.4 与相邻方法的边界

与 PID 的关系。 均值 PI 是 SDC 的一个投影：把 $\boldsymbol{v}$ 投影到“均值”这一个方向，再对这个标量做 PI。表 73-1 里 PI 的 IAE 比 SDC 高 23.0%、比开环高 21.1%，差距全部来自“看不见形状”。所以两者的选择不是先进与落后，而是“你要不要形状”：只要均值，PI 更省事（不需要辨识、不需要 2000 个样本）；要形状，PI 在原理上就做不到。常规 PID 的完整讨论见第 44 章与第 106 章。

与 MPC 的关系。 本章的一步预测律是 $N_p = 1$ 的 MPC，且不带约束。§73.9.3 的式 (73-29) 就是把它扩到多步带约束，结构与第 58 章、第 80 章的 MPC 完全同构，只是状态维数从几个变成 20 个、优化变量从“未来输入”变成“未来输入序列”。差别在计算：标量状态的 MPC 每拍解一个几变量的 QP，分布 MPC 要解 200 变量左右。所以“要不要上多步”取决于约束是否真的在起作用，而不是取决于方法论的高低。

与 Smith 预估的关系。 纯滞后与分布形状是两件正交的事。本对象没有滞后，所以本章没提；若对象有 3 个采样周期的纯滞后，正确做法是先按第 47 章、第 107 章的办法把滞后补掉，再在外面套 SDC——补滞后不改变分布的稳态族，只改变相位，两者可以串联。反过来，用 SDC 去“吸收”滞后是错的：一阶模型吸收不了纯滞后，辨识残差会明显上浮（表 73-8 里高增益批的 RMS 0.02157 已经是低增益批的 1.7 倍，若再加滞后，这个数会更大）。

与第 74 章“分布参数系统”的区别。 两章都叫“分布”，含义不同。本章的分布是**概率分布**（同一时刻不同个体取值的散布），状态是概率密度，自变量是输出值 y；第 74 章的分布是**空间分布**（同一时刻不同位置的取值），状态是偏微分方程的解，自变量是空间坐标 x。前者靠基函数投影降维，后者靠模态截断降维，数学工具相似（都是把无穷维投影到有限维），物理含义完全不同。阅读时不要混。

与第 77 章 Koopman 方法的关系。 §73.9.4 的失败本质上是“权向量对输入是非线性的，而模型是线性的”。Koopman 的思路正好对症：找一个观测函数把非线性的权向量演化提升到高维线性空间，在那个空间里做线性预测。代价是维数升高、观测函数要选或学。本章没有走这条路，因为在 $\kappa = 1.3$ 的基线场景里线性模型已经够用；但若工艺要求大行程、窄峰、多执行器同时满足，Koopman 或分区分段线性（§73.9.5）是必须考虑的方向。

73.11.5 选型时的四个问题

前面的边界可以压缩成四个按顺序问的问题。任何一个答“否”，都不必往下问。

问题一：合格率是否真的由整条分布决定？ 若工艺只卡均值上下限，或者超差主要来自均值漂移，用均值控制加 SPC 监视宽度就够了（第 44 章）。这一问是“要不要”的问题，答错的代价是白做一整套模型。

问题二：每个更新周期能否凑到 1000 个以上样本？ 凑不到就拉长更新周期，此时 a 必须按新周期重算（§73.10.1）；若拉长到对象已经走完过渡过程（T_s 接近甚至超过 $1/p$），分布控制退化为静态反解，收益消失。仍不满足就放弃。

问题三：行程比 κ 是否在 3 以内？ κ 按“真正被独立控制的那条分布”的宽度算。本章基线 1.3，合规；分簇后 6.4，方向余弦掉到 -0.043。超界时先考虑分区段线性或 Koopman（§73.9.5、第 77 章），不要先调参数。

问题四：目标峰位比与对象固有比例之差是否在 2% 以内？ 超出就说明目标不在稳态分布族上，终点误差有下限。本章差 4.5% 时，SDC 的终点 L1 反而比均值 PI 差 10.8%（表 73-11）。此时应当改工艺（加执行器、改配比、改目标），而不是换控制器。

四问全过，再按 §73.3 的七步做。这个顺序的价值在于：前三问的答案都能用现有数据直接算出来，不需要先写控制器，也不需要停机试验。

73.11.6 收益出现在什么场合

同一个方法，在不同生产组织下的回报差别很大，值得单独算一笔账。

把本章的 IAE 差摊平看：SDC 5.332、均值 PI 6.924，差 1.592 mm·s，摊到 80 s 的视窗上是平均 L1 低 0.0199 mm。但这个差值全部来自前 3 s，2 s 之后两条曲线基本重合（§73.6.3）。所以收益的多少，取决于“过渡段占整批的比例”。

- **连续稳态生产**：几乎没有换型，过渡段只出现在开机时，收益趋近于零。此时 SDC 的成本（辨识、样本、维护）却一分不少，投入产出不划算。
- **小批量、频繁换型**：每批都要经历一次从初始分布到目标的搬移，过渡段占比高，收益接近 IAE 的 23.0%。这类产线才是 SDC 的主场。
- **启动废料昂贵**：过渡段的前 1 s 正是 SDC 领先最多的地方（0.5 s 时误差是 PI 的 47%），若这段的废料成本高，收益会被放大。

这条账也解释了为什么 §73.11.3 的表里“目标可达”排在第一位：不可达时 SDC 连稳态都不占优（0.1478 对 0.1334），过渡段的优势补不回来。**先确认可达，再确认过渡段足够值钱，最后才谈控制器的先进与否。**

73.11.7 什么时候该退出

投运不是终点，退出也要有判据。四条任一长期成立，就该考虑退回“分布监视 + 均值控制”，而不是继续加参数硬撑。

目标比例长期超出 2%。 换型后目标变了，固有比例没变，终点误差有下限。此时再怎么调，L1 也停在 0.13 以上（表 73-11），不如把形状要求写回工艺。

样本量长期不足 1000。 产量下降、抽检比例下调都会造成这个结果。估计误差占稳态 L1 的一半以上时，控制器有一半输入在追噪声，效果还不如不控。

行程比随工况变大。 换型后峰更窄、行程更长，κ 从 1.3 涨到 3 以上，方向余弦开始不可信。此时应当换结构（§73.9.5），不是继续用本章的模型。

维护成本超过收益。 连续稳态生产、很少换型的场合，收益本身趋近于零（§73.11.6），而重辨识、模型版本管理、数据存档的开销一分不少。

退出的方式要留后路：先把控制器切到影子模式，保留 L1 与峰位报表，均值回路照常运行；观察若干批确认分布没有恶化，再撤掉分布回路。直接停机撤掉的风险是，原本被分布回路悄悄压住的宽度问题会重新暴露，而现场没有报表可查。

73.11.8 一句话的取舍

如果只留一句话：**SDC 用“模型 + 样本”换“形状”，当形状是质量的主要矛盾、且行程不大、执行器唯一时，它是划算的；当形状可以由均值代表、或行程远大于峰宽时，它的成本与风险都不划算。**

判断“划不划算”的顺序也值得记住：先问目标可不可达，再问过渡段值不值钱，最后问数据够不够。三步都过了，再谈控制器怎么写。

73.12 Python 仿真代码

本章的全部数字出自两个脚本，都在 `book/code/` 下：

- `ch073_sim.py`：仿真主体，把本章所有数字按九个块依次打印到控制台；
- `ch073_figs.py`：读 `ch073_sim.py` 的函数生成 10 张图，导出 SVG 与 PDF 到 `book/figs/ch073/`。

运行方式（Windows 下已装好的环境）：

```
D:\Anaconda3\envs\py311\python.exe code\ch073_sim.py
D:\Anaconda3\envs\py311\python.exe code\ch073_figs.py
```

73.12.1 核心片段

下面把方法主干抽出来，控制在 60 行以内。这段代码与 `ch073_sim.py` 中的同名函数逐字一致（只删了部分注释），可以直接粘贴运行；完整版另有连续情形、双执行器、失配扫描、样本量与耗时测量。

```
import numpy as np
from scipy.interpolate import BSpline

Y_MIN, Y_MAX, N_BASIS, DEGREE = 0.0, 10.0, 20, 3
TS, U_MAX = 0.1, 8.0

def bspline_design(y, n_basis=N_BASIS, degree=DEGREE):
    """B 样条基函数矩阵：端点钳位、内部等距、单位分解。"""
    interior = np.linspace(Y_MIN, Y_MAX, n_basis - degree + 1)[1:-1]
    knots = np.concatenate(([Y_MIN] * (degree + 1), interior, [Y_MAX] * (degree + 1)))
    x = np.clip(np.asarray(y, float), Y_MIN, Y_MAX)
    return BSpline.design_matrix(x, knots, degree).toarray()

def pdf_weights(y, n_basis=N_BASIS):
    """样本 -> 权向量：v_i = (1/N) sum_j B_i(y_j)。"""
    return bspline_design(y, n_basis).mean(axis=0)

def pdf_value(v, grid, n_basis=N_BASIS):
    """权向量 -> 密度：gamma(y) = n/(b-a) * sum_i v_i B_i(y)。"""
    return (n_basis / (Y_MAX - Y_MIN)) * (bspline_design(grid, n_basis) @ v)

def identify(V, U, lam=1e-3):
    """中心化岭回归：V(k+1) = A V(k) + B u(k) + c，只惩罚 A 的列。"""
    M = np.hstack([V[:-1], U[:, None]]); Yv = V[1:]
    m, ym = M.mean(axis=0), Yv.mean(axis=0)
    Mc, Yc = M - m, Yv - ym
    D = np.eye(V.shape[1] + 1); D[-1, -1] = 0.0     # 输入通道不惩罚
    theta = np.linalg.solve(Mc.T @ Mc + lam * D, Mc.T @ Yc)
    A, B = theta[:-1].T, theta[-1]
    c = ym - m @ theta
    return A, B, c, float(np.sqrt(np.mean((Yv - (M @ theta + c)) ** 2)))

def make_sdc(A, B, c, v_target, alpha=0.25):
    """一步预测控制：参考滤波 + 沿 B 反解输入。"""
    bb = float(B @ B)
    state = {"vr": None}
    def u_fun(k, v, y):
        if state["vr"] is None:
            state["vr"] = np.asarray(v, float).copy()
        state["vr"] = state["vr"] + alpha * (v_target - state["vr"])
        return float(B @ (state["vr"] - A @ v - c)) / bb
    return u_fun

def l1_dist(v1, v2, grid=np.linspace(Y_MIN, Y_MAX, 401)):
    """两条密度曲线的 L1 距离（固定网格积分，不同 n 可比）。"""
    g1, g2 = pdf_value(v1, grid, len(v1)), pdf_value(v2, grid, len(v2))
    return float(np.sum(np.abs(g1 - g2)) * (grid[1] - grid[0]))
```

73.12.2 脚本结构

`ch073_sim.py` 的 `main()` 按九个编号块组织，与正文的对应关系如下。编号块按顺序执行，后一块会用到前面算出的模型与基准结果，所以复现时建议整段跑；只想看某个数，可以在对应块前加断点，但不要跳过第 1 块（模型在那里辨识）。

块	内容	对应正文
1–3	开环、SDC、均值 PI 三条对比，峰位与峰间距，过渡切片	表 73-1、图 73-1、图 73-6、图 73-7
4a	连续情形（Euler-Maruyama）	§73.4、图 73-5
4b	非比例双峰目标	表 73-11
4c	密度域加权跟踪	表 73-7
4d–4g	双执行器：分簇辨识、一步预测方向诊断、加密节点、激励长度	表 73-8、§73.9.4、§73.10.3
5	参考滤波系数 α 扫描	§73.6.2
6	基函数个数扫描	§73.6.4、图 73-8
7	增益失配与积分补偿	§73.6.5、图 73-9、§73.9.1
8	样本量对权向量估计的影响	表 73-9、图 73-10 左幅
9	在线单步耗时	表 73-10、图 73-10 右幅

73.12.3 复现时要注意的三点

随机种子固定，但结果不逐位相同。 所有随机源都用了显式种子（`default_rng(seed)`），所以同一台机器上重复运行的数字完全一致；换 NumPy 版本、换 BLAS 后端，最后几位可能变化。正文引用的是小数点后四位，属于稳定范围。表 73-10 的耗时是唯一例外——它取决于机器负载，每次运行会差 10% 左右，所以只用于看量级与“耗时在哪一段”，不要当基准值。

所有打印都是一次运行的输出，没有手工填写。 本章写作过程中反复强调这一点，原因是这类章节最容易出现的错误是“改了代码忘了改数字”。若你修改了任何参数，重跑脚本后应当以新的控制台输出为准，正文的数字会随之失效——这不是缺陷，而是这套写法的代价与保障。

图和正文共用同一套函数。 `ch073_figs.py` 直接 `import ch073_sim`，调用 `run_loop`、`make_sdc`、`target_weights` 等，不在绘图脚本里重写模型。这样“图里的曲线”与“表里的数字”永远出自同一个对象、同一组参数，不会出现两张图互相矛盾的经典问题。要改模型，只改 `ch073_sim.py` 一处，图和数字一起变。

73.12.4 从仿真代码到产线代码的三处改动

§73.12.1 的 60 行是方法主干，不是能直接烧进控制器的程序。搬到产线上要改三处，每一处都对应一个在仿真里被理想化的假设。

样本来源。 仿真用 2000 个粒子代表全体工件，样本量是设定的；产线测的是真实工件，而且往往只测一部分。若只抽检 30%，等效样本量按比例缩小，权向量估计误差按 $1/\sqrt{0.3}=1.83$ 倍放大，本章的 0.01099 会变成 0.0201，相当于把 §73.10.1 的表 73-9 往回退了两档。抽检比例必须与 N 一起算，不能先定 N 再定抽检。

时间基准。 仿真按固定 0.1 s 推进，产线按批次事件推进：一批来了算一拍，批次间隔可能不齐。要把事件时间映射到等间隔的 T_s，并保证 T_s 内的样本属于同一批（§73.5.1 的第三条约束）；映射不准，a 就是错的，闭环表现为“慢得莫名其妙”（§73.10.1）。

执行器模型。 仿真假设零阶保持加硬限幅，产线还有死区、回差与速率限制。速率限制会直接压低 $\boldsymbol{B}_v$ 的有效幅值，而死区会让小输入完全无效。这两项必须在辨识数据里体现，否则模型给出的小幅修正会被执行器吃掉，积分项随之越积越大。这一条与 §73.6.5 的高估增益失配症状相同，但根因不同，靠重辨识解决，不是靠调 k_i。

三处改动都不涉及控制律本身，这也是这套方法的一个优点：算法与实现的分界很清楚，换硬件不用重写控制部分。

73.12.5 三个可以自己动手改的实验

读完本章，改三个参数各跑一次，能把几条结论从“记住”变成“看到”。

实验一，把 n 改成 12 与 28。 只改 `N_BASIS`，其余不动，跑第 6 块扫描。应当看到终点 L1 随 n 上升、耗时基本不变、辨识残差仍在 0.008 到 0.010 之间。若残差明显上翘，说明 N 已经不够分给更多基函数（§73.7.7 第四条）。这个实验最能说明“L1 跨 n 不可比”是怎么来的。

实验二，把 α 设成 1.0。 只改 `make_sdc` 调用处的 `alpha`。应当看到输入峰值顶到 8.000、IAE 下降、而终点 L1 与 $\alpha = 0.25$ 时基本相同。三个变化方向正好对应 §73.6.2 的三条读法；如果终点变了，说明代码里混进了别的东西（例如积分未清零）。

实验三，改对象固有比例。 把 `B2_MEAN` 从 0.240 改成 0.200，固有峰位比变成 $0.200/0.160 = 1.25$，再跑第 4b 块的非比例目标（峰位比 1.433）。此时差距从 4.5% 拉大到 14.6%，按 §73.10.4 的判据，终点 L1 应当明显恶化。这个实验把“可达性”从一句话变成一个可以看见的数量级。

三个实验都只改一处，改完重跑，别把改动叠在一起。叠着改，指标变好或变坏都归因不了，这正是 §73.8.4 那条“一次只动一个参数”的由来。

73.12.6 本章小结

第 73 章用一个双批次加工的例子，把输出分布控制走了一遍：用三次 B 样条把整条 PDF 投影成 20 维权向量，用一阶线性差分方程描述权向量随输入的演化，再用一步预测反解输入。基线在 800 步下终点 L1 0.0473、分布 IAE 5.332，比均值 PI 的过程误差低 23.0%；收益集中在过渡段的前 3 s，稳态与 PI 持平，因为本对象的稳态分布族只有一个自由度。

三条边界值得带走：表示精度决定了 L1 能读到多细（峰宽与节点间距之比 0.56 时峰高被压低 42%）；行程比决定了线性模型能不能用（$\kappa = 6.4$ 时方向余弦掉到 -0.043，闭环失效）；输入维数决定了状态怎么定义（两路输入共用一套权向量时不能观，必须先拆状态）。

第 74 章转向另一类“分布”——空间分布，被控量由概率密度变成偏微分方程的解，降维工具从基函数投影换成模态截断。两章的数学结构相似，物理含义不同，阅读时可以对照着看。

第74章 分布参数系统控制（DPS）

74.1 动机与直观理解

集中参数控制的默认假设是：被控对象的“状态”只有几个数，用一个测点、一个执行器就能把它按住。化工装置里大量回路确实如此——一个缓冲罐的液位、一台换热器的出口温度，空间上的不均匀性被搅拌或湍流抹平，剩下一个随时间变化的标量。可一旦对象是细长的管、薄的板、宽幅的带材，“同一时刻不同位置”就成了不能被抹掉的事实。温度沿轴向有梯度，浓度沿反应器长度有分布，应力沿轧件宽度有高低。这类对象的数学模型是偏微分方程，状态属于无穷维函数空间，控制理论把它们统称为**分布参数系统**（distributed parameter system，DPS）。

本章只处理一条空间维度。这不是为了简化而简化：反应管、管式裂解炉、带钢连续退火线、晶体生长炉，主流工业对象的纵横比都在十倍以上，轴向动力学与径向动力学的时间尺度相差一两个数量级，把径向集总掉是工程上站得住的近似。一维情形足以把 DPS 控制的所有关键问题——模态、截断、溢出、执行器与传感器的空间位置——完整地展示出来，而且每一个量都能算到小数点后四位。

74.1.1 一个会热失控的管子

本章的算例是一根薄壁反应管，长 $L = 0.50$ m，轴向热扩散率 $\alpha = 0.02$ m^2/s，线性化放热增益 $\lambda = 3.5$ s^{-1}。两端插在恒温夹套里，温度偏差恒为零；管内某点有一个电加热片，用来压制反应放热；管壁某点焊一支热电偶，用来测温。初始时刻管内有一个 1 K 的热点，位于 $x = 0.125$ m，半宽 0.045 m。没有控制时，这个热点会怎样演化？

先做量级估计。热扩散走过全管的时间是 $L^2/\alpha = 0.25/0.02 = 12.5$ s，而反应放热的特征时间是 $1/\lambda = 0.2857$ s，两者相差 43.75 倍。也就是说，热量还没来得及沿管子散开，反应已经把温度推上去了。数值仿真给出的结果更直接：初始峰值 1.0007 K，1.2 s 后峰值 8.0084 K，峰值越过 5 K 的时刻是 1.0217 s。管子在这个时间尺度上没有任何自我保护能力，图 74-2 的实线给出了这条指数上扬的峰值轨迹。

这个算例的教学价值在于它**不是**一个“整体升温”的问题。若热源均匀、边界均匀，温度剖面会保持初始形状整体放大，控制一个测点就等于控制全管。实际不是这样：初始热点是局部的，放热项按当地温度放大它，导热项则把它向两侧摊开，两个机制在不同位置上强度不同，剖面的形状在演化中不断改变。图 74-2 的右图给出四个时刻的空间剖面：初始的高斯峰逐渐抬高、变窄，两侧出现明显的肩部。到了 1.2 s，最高点已经不在原来的 0.125 m 处，而是向管子中部漂移。**只压一个测点的温度，很可能压住的是剖面的一个局部，而峰值已经跑到别处。**

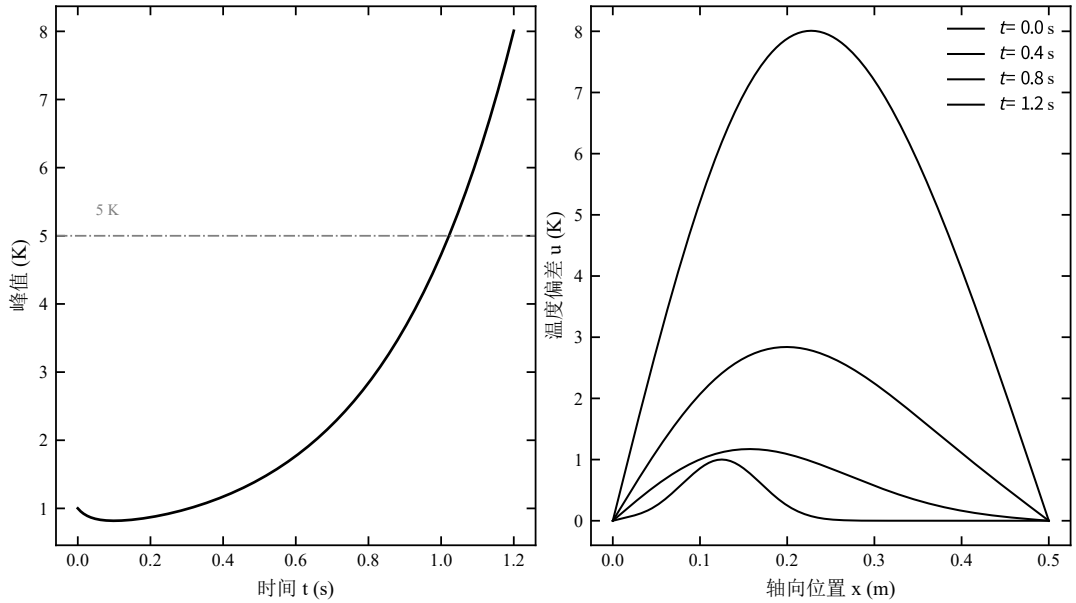


图 74-2　开环峰值轨迹与空间剖面演化

74.1.2 模态：把剖面拆成一组标准形状

两端固定的边界条件决定了这个对象有一套天生的“标准形状”。对区间 $[0, L]$ 上满足 $u(0) = u(L) = 0$ 的函数，正弦族 $\sin(n\pi x/L)$ 是完备正交基。任意时刻的温度剖面都可以写成

$$u(x,t) = \sum_{n=1}^{\infty} z_n(t)\,\sin\frac{n\pi x}{L} \tag{74-1}$$

系数由投影给出，$z_n(t) = \frac{2}{L}\int_0^L u(x,t)\sin(n\pi x/L)\,\mathrm{d}x$。这套展开不是数学上的方便选择，而是对象自身的性质：把式 (74-1) 代回热方程，各个 z_n 彼此解耦，每一个都独立地按自己的速率演化。导热的二阶导数把 $\sin(n\pi x/L)$ 变成 $-(n\pi/L)^2\sin(n\pi x/L)$，于是第 n 阶的净增长率是

$$\gamma_n = \lambda - \alpha\left(\frac{n\pi}{L}\right)^2 \tag{74-2}$$

这个式子把 DPS 的核心矛盾写在一行里：**放热项对所有模态一视同仁地加 λ，导热项对高阶模态的惩罚却按 n^2 增长。** 低阶模态波长长、导热来不及把它们抹平，因此可能不稳定；高阶模态波长短、导热极强，天然稳定。本章参数下，$\gamma_1 = 2.7104$、$\gamma_2 = 0.3417$、$\gamma_3 = -3.6061$，从第 3 阶起全部为负。不稳定模态只有两个，这就是全部麻烦的来源，也是全部机会的来源。

图 74-1 的三张子图把这件事画全了。上图为 γ_n 随阶数的变化，前两阶在零线以上，第 3 阶开始迅速下坠，到第 8 阶已经是 -47.0324；中图为输入模态增益 b_n，它不单调，第 5 阶达到绝对值最大 3.8074；下图为输出模态增益 $c_n = \sin(n\pi x_s/L)$，呈明显的振荡，并在某些阶上接近零。三张图放在一起看，才能理解为什么“控住前两阶”这件事既简单又危险：简单在于只有两阶需要压，危险在于执行器和传感器都可能在某一阶上恰好失效。

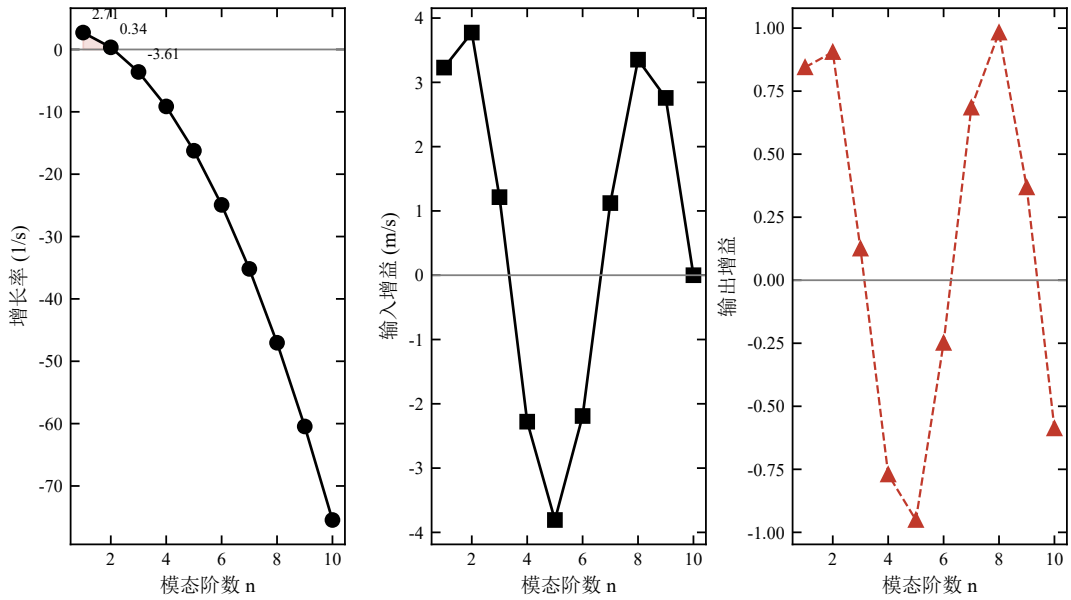


图 74-1　模态增长率与输入、输出模态增益

74.1.3 三个必须分开的问题

初学者容易把 DPS 控制当成“把 PDE 离散成很多 ODE，然后套用第 15 章的 LQR”。这个理解会漏掉三件性质完全不同的事，而它们在实践中造成的故障也完全不同。

第一件是截断。 无穷维的模态不可能全部放进控制器。工程上取前 N 阶做模型，第 $N+1$ 阶以后扔掉。扔掉的部分不是不存在，它们仍然被执行器激励、被传感器看见，只是控制器不知道。当控制器按“只有 N 阶”的模型算出控制量时，这个控制量会通过 $b_{N+1}, b_{N+2}, \ldots$ 灌进被忽略的模态。若这些模态本身稳定，它们会把能量耗掉，问题不大；若控制量足够大、频率足够高，被忽略的模态可能被激起可观的振幅，反过来污染测量，形成**控制溢出**（control spillover）。本章 §74.7 会

给出一个干净的例子：控制阶数取 1 时，闭环在 8 s 内不收敛，末值 3.37918 K，残余模态能量 11.43，控制器输出的控制量大部分用来喂养第 2 阶，而这正是它唯一没有建模的模态。

第二件是观测溢出。 传感器只测一个点，观测器要从前 N_{obs} 阶的模型反推整条剖面。未被建模的高阶模态会通过 c_n 混进测量，观测器把它误当成低阶模态的偏差去修正，修正量再通过观测器增益放大回低阶估计，形成**观测溢出**（observation spillover）。§74.7 的扫描显示，观测阶数取 2、3、4 时闭环直接发散；取 6 时虽然不发散，但 8 s 内峰值仍有 1.6181 K，末值 1.4391 K，始终没有进入 0.10 K 的稳态带。这两个现象在集总参数系统里没有对应物，是 DPS 独有的。

第三件是空间布置。 执行器和传感器放在哪里，直接决定某些模态能不能被推动、能不能被看见。b_n 与 c_n 都是位置的函数，位置选得不好，某个关键模态的增益恰好为零。本章的基准算例里，执行器在 $x_a = 0.15$ m、传感器在 $x_s = 0.16$ m，都刻意避开了 $L/2 = 0.25$ m 这个位置——因为第 2 阶模态在那里有一个节点，$\sin(2\pi \cdot 0.5) = 0$。§74.9 的扫描会把这个结论量化：执行器放在 $0.5L$ 时 $b_2 = 0$，设计阶段直接判定“输入对不稳定模态不能控”；传感器放在 $0.5L$ 时 $c_2 = 0$，判定“输出对不稳定模态不能观”。

这三件事有一个共同的数学根源：**截断把无穷维系统变成了有限维系统，但物理对象没有跟着变。** 控制器越“聪明”、带宽越高，越容易去激励那些它看不见的模态。这也是为什么本章在控制器里特意加了一个时间常数 0.05 s 的执行器一阶惯性——不是为了更真实，而是为了让控制量在高频段自然滚降，从源头上减少对高阶模态的激励。这个改动对结果的影响在 §74.6 会看到：加上它之后，控制阶数从 2 到 8 的稳态时间几乎不变（2.170、2.164、2.162、2.164、2.164 s），说明截断误差已经被压到次要位置。

74.1.4 本章的数字基线

为便于后续各节对照，先把三条基线列在表 74-1。全部数字出自 §74.12 的脚本，真值模型取 20 阶模态，仿真步长 0.002 s，控制目标是把全管温度压到 0.10 K 以内并保持。

表 74-1　三条基线的性能指标

方案	稳定时间（s）	峰值（K）	控制能量	末值（K）
开环（截止 1.2 s）	未稳定	8.0084	0	8.0084
模态全状态反馈 $N = 2$	1.028	0.1053	0.0946	0.0007
点测温输出反馈 $N = 2, N_{\text{obs}} = 10$	2.170	0.2542	0.0943	0.0007

三行数字里有两点需要提前说明。其一，**开环那一行的“峰值”是 1.2 s 时刻的值，不是全程最大值**；若继续仿真，峰值会按 $e^{2.7104t}$ 增长，8 s 时早已超出任何物理意义。其二，**全状态反馈与输出反馈的控制能量几乎相同（0.0946 与 0.0943）**，差别全部体现在过渡过程上：全状态的峰值只有 0.1053 K，输出反馈是 0.2542 K，稳定时间也从 1.028 s 拉长到 2.170 s。这个差距不是控制器设计得不好，而是信息缺口的价格——只测一个点，就必须先用观测器把整条剖面“猜”出来，猜测需要时间，这段时间里热点还在长。

表 74-1 还隐含一个容易误读的事实：两条闭环的末值都精确到 0.0007 K，看起来性能一样好。稳态精度由积分作用或模型准确性决定，本章的 LQR 没有积分器，末值之所以接近零，是因为对象在原点处只有一个平衡点、且闭环把所有模态都压到了负半平面。真正区分控制器好坏的是过渡段，这也是后面所有扫描实验统一用“稳定时间 + 峰值”作为主指标的原因。

三个基线的口径需要交代清楚，否则后面的数字没法互相比较。**开环为什么只截到 1.2 s**：第 1 阶的时间常数是 $1/2.7104 = 0.369$ s，1.2 s 约为 3.3 个时间常数，温度已涨到 8.0084 K。再往后仿真，数值会指数增长到失去物理意义（真实装置早已触发联锁），所以开环一行只作为“不控制的后果”示意，不参与性能排序。**阈值为什么取 0.10 K**：这是本章假定的工艺允许残余偏差，相当于初始热点 1.0007 K 的 10%。阈值取得越小，稳定时间越长，且对数值噪声越敏感；取 0.10 K 是一个“既能看到差别、又不至于被噪声干扰”的折中。换成别的工艺，这个数要按实际允许偏差改，结论的绝对数值会变，但趋势不变。**真值为什么取 20 阶**：$N = 2$ 时残余能量占总能量的 7.1×10^{-9}，对应温度幅值约 10^{-4} K，比 0.10 K 的阈值小三个数量级，因此 20 阶足以充当“真值”；若残余能量占到 1% 以上，所有结论都要打折扣。

这三条口径合起来构成本章的“实验台”：同一初始条件（$0.25L$ 处峰值 1.0007 K 的高斯热点）、同一仿真时长（8 s）、同一阈值（0.10 K）、同一真值模型（20 阶）。后续任何一张表、任何一条曲线，都是在这张实验台上换一个参数得到的。

读者若要复现，只需改 `code/ch074_sim.py` 参数区的一个常数，其余不动。

74.1.5 与相邻章节的分工

本章讲的是“对象是 PDE 时，控制器怎么设计”。它与几章内容相邻，但边界清楚，先划开以免重复。

- **与第 44 章的 PID**：PID 处理的是单回路、少状态的对象。把 PID 直接用在 DPS 上，等价于只控制传感器所在的那一个点，对剖面形状没有约束。§74.10 会给出一个具体对比例子：传感器在 $0.32L$ 时输出反馈能把峰值压到 0.2542 K，若把观测器换成“测点即全管”的假设，峰值会明显偏高。
- **与第 58 章的 MPC**：MPC 可以用有限维模型做预测，理论上也能处理 DPS，但需要在线求解优化问题。本章的 LQR 是离线求解 Riccati 方程、在线只做矩阵乘，算力需求低两个数量级。当约束复杂、目标时变时，第 80 章的 DPS-MPC 是更合适的选择。
- **与第 47 章的 Smith 预估**：Smith 预估解决的是纯延迟，本章 §74.10 的延迟扫描显示，执行器惯性已经把 0.05 s 以内的延迟基本吸收，失稳边界出现在 0.08–0.09 s 之间。这不是 Smith 预估能处理的范围，因为这里的失稳机制是“延迟 + 高阶模态”的耦合，不是单纯的相位滞后。
- **与第 73 章的 SDC**：第 73 章控制的是输出**分布**的形状（随机变量的概率密度），本章控制的是**空间分布**（确定性函数沿坐标的剖面）。两者的数学工具都是“投影到有限维基”，但基函数的物理含义、稳定性来源、误差传播路径完全不同。SDC 的无穷维来自随机性，DPS 的无穷维来自空间。

这个分工也决定了本章的写法：不重复推导 LQR 与 Kalman 滤波的标准结果，只讲它们在模态坐标下**变成什么样子**、哪些参数获得了新的物理含义。读者若对 Riccati 方程的推导不熟，应先回看第 15 章与第 22 章。

74.1.6 本章路线

后续各节的安排如下。§74.2 建立数学模型，从能量方程推到模态方程，说明半群、谱、截断误差的确切含义；§74.3 给出七步设计流程，包括执行器惯性增广、可控可观检查、LQR 与观测器的求解顺序；§74.4 讨论连续时间情形，重点是全状态反馈能达到的性能上限；§74.5 讨论离散化，用有限差分收敛实验说明“空间离散”与“模态截断”是两种性质不同的近似；§74.6 给出闭环稳定性与性能的实测结果；§74.7 逐条解释各参数的作用；§74.8 总结调参顺序与经验规则；§74.9 讨论优化方向，包括执行器与传感器的位置优化；§74.10 讲工程落地，包括延迟、噪声、采样周期和模型降阶验证；§74.11 给出适用边界与失效模式；§74.12 给出完整代码。

一个提醒：本章所有结论都建立在“线性化”之上。放热项 λu 是线性化后的近似，真实反应的 Arrhenius 非线性在温度偏差较大时会显著改变增益。本章的闭环把偏差压在 0.25 K 量级，线性化误差可以接受；若工艺要求允许 5 K 以上的偏差，必须回到非线性模型，那已超出本章范围。

74.2 数学原理

本节把 §74.1 的直观说法变成可计算的数学。顺序是：从能量方程出发，做无量纲化，说明算子与半群，写出模态方程，定义截断与溢出，最后给出本章算例的全部模态数据。

74.2.1 从能量方程到线性化模型

设管壁温度 $T(x,t)$，密度 ρ、比热 c_p、导热系数 k 均为常数，管内发生一级放热反应，放热速率用 Arrhenius 形式 $Q(T)=Q_0\exp(-E/(RT))$ 描述。一维能量守恒给出

$$\rho c_p \frac{\partial T}{\partial t} = k\frac{\partial^2 T}{\partial x^2} + Q(T) + g(x)w(t) \tag{74-3}$$

其中 $g(x)$ 是加热片的功率空间分布，$w(t)$ 是控制输入（单位 K/s，已把功率除以 ρc_p 折算成温升速率）。两端插在恒温夹套中，取

$$T(0,t)=T(L,t)=T_b \tag{74-4}$$

设稳态工作点为 $T_s(x)$，满足 $0=kT_s''+Q(T_s)$ 及同样的边界条件。定义偏差 $u=T-T_s$，把 Q 在 T_s 处一阶展开，$Q(T)\approx Q(T_s)+Q'(T_s)u$，代入式 (74-3) 并利用稳态方程，得

$$\frac{\partial u}{\partial t}=\alpha\frac{\partial^2 u}{\partial x^2}+\lambda(x)u+g(x)w(t),\qquad u(0,t)=u(L,t)=0 \tag{74-5}$$

其中 $\alpha=k/(\rho c_p)$ 是热扩散率，$\lambda(x)=Q'(T_s(x))/(\rho c_p)$ 是当地线性化放热增益。本章进一步把 $\lambda(x)$ 取为常数 $\lambda=3.5\ \mathrm{s}^{-1}$，这是“反应区沿轴向均匀”的简化。真实反应器的 $\lambda(x)$ 通常在入口附近最大，这会让各阶模态的增长率不再是式 (74-2) 的简单形式，模态之间也不再正交解耦——那属于变系数 DPS，需要数值求解特征值问题，本章不展开。

式 (74-5) 是本章的全部数学对象。它线性、定常、两端 Dirichlet。选 Dirichlet 而不是 Neumann，是因为夹套的换热系数远大于管内导热，壁温被强行钉在 T_b；若夹套换热有限，边界条件应改成 Robin 型，模态不再是纯正弦，但下面的整套方法（投影、截断、LQR）仍然适用，只是基函数要换成相应的特征函数。

74.2.2 无量纲化与 Damköhler 数

把 x 用 L 归一、t 用 L^2/α 归一，记 $\xi=x/L$、$\tau=\alpha t/L^2$，式 (74-5) 变成

$$\frac{\partial u}{\partial \tau}=\frac{\partial^2 u}{\partial \xi^2}+\mathrm{Da}\,u+G(\xi)W(\tau) \tag{74-6}$$

$$\mathrm{Da}=\frac{\lambda L^2}{\alpha} \tag{74-7}$$

Da 是 Damköhler 数，衡量“反应放热”与“导热耗散”的相对强度。本章 $\mathrm{Da}=3.5\times 0.25/0.02=43.75$，是一个很大的值。不稳定模态的个数由 $\mathrm{Da}>(n\pi)^2$ 决定，解出 $n<L\sqrt{\lambda/\alpha}/\pi=0.1592\times 13.229=2.106$，即只有 $n=1,2$ 两阶不稳定。这个估算与 §74.1 的 γ_n 数值完全一致，说明“不稳定模态数”是一个可以由两个物理参数直接算出的整数，不需要仿真。

Da 的大小决定了控制策略的难度。$\mathrm{Da}<1$ 时全部模态稳定，控制只是为了改善过渡品质；Da 在 1 到 100 之间时，少数低阶模态不稳定，需要主动控制；$\mathrm{Da}>100$ 时不稳定模态增多，控制器阶数必须相应提高，否则截断误差会吞掉稳定性。本章的 43.75 落在中间区域，恰好是“两阶控制、十阶观测”这种典型配置能处理的范围。

74.2.3 算子、谱与半群

把式 (74-5) 写成抽象发展方程

$$\dot{u}=\mathcal{A}u+\mathcal{B}w,\qquad u(0)=u_0 \tag{74-8}$$

其中 $\mathcal{A}=\alpha\,\mathrm{d}^2/\mathrm{d}x^2+\lambda I$，定义域 $D(\mathcal{A})=H^2(0,L)\cap H_0^1(0,L)$，$\mathcal{B}=g(x)$ 是乘算子。$\mathcal{A}$ 是一个**无界**算子：二阶导数在 L^2 中没有界，这正是 DPS 与有限维系统最本质的区别。

无界并不妨碍解的存在性。$\mathcal{A}$ 是自伴算子，且 $-(\mathcal{A}-\lambda I)$ 是正定的，由 Lumer–Phillips 定理，$\mathcal{A}$ 生成一个 C_0 半群 $e^{\mathcal{A}t}$，其谱由特征值 γ_n 与特征函数 $\phi_n(x)=\sqrt{2/L}\sin(n\pi x/L)$ 组成，且特征函数构成 $L^2(0,L)$ 的完备正交基。于是

$$e^{\mathcal{A}t}u_0=\sum_{n=1}^{\infty}e^{\gamma_n t}\langle u_0,\phi_n\rangle\phi_n \tag{74-9}$$

这个式子是本章所有模态方法的合法性依据。它说明：**不需要对空间做任何离散，解本身就可以按模态展开；模态系数的时间演化是精确的指数，没有离散误差。** 后面的有限差分离散只是为了在 §74.5 里说明“另一种近似会带来什么误差”，真正的仿真用的是式 (74-9)，也就是解析模态推进。

半群的语言还解释了一个容易混淆的问题：为什么不能直接写 $e^{\mathcal{A}t}\mathcal{B}$ 的传递函数？因为 $\mathcal{B}$ 作为乘算子是有界的（g 是光滑的高斯函数），但输出算子 $\mathcal{C}u=u(x_s)$（点测量）在 L^2 中**无界**。真实探头有有限宽度，把 $\mathcal{C}$ 换成一个宽度为 δ 的加权

平均就变成有界泛函；本章按点处理，误差在探头宽度远小于波长时可以忽略。若传感器宽度与高阶模态波长可比，c_n 需要乘以一个 sinc 型的衰减因子，这一点在 §74.10 讨论。

74.2.4 模态方程

将式 (74-1) 代入式 (74-5)，两边乘 $\sin(m\pi x/L)$ 并在 $[0,L]$ 上积分，利用正交性 $\int_0^L \sin(n\pi x/L)\sin(m\pi x/L)\mathrm{d}x = (L/2)\delta_{nm}$，得

$$\dot{z}_n = \gamma_n z_n + b_n w(t), \qquad n = 1,2,\dots \tag{74-10}$$

$$b_n = \frac{2}{L}\int_0^L g(x)\sin\frac{n\pi x}{L}\,\mathrm{d}x \tag{74-11}$$

这里的 $2/L$ 是必须写对的系数，也是实际编程中最容易漏的一项。若漏掉它，模态增益只有真值的 $2/L = 4$ 分之一（本章 $L = 0.5$），控制器会误以为执行器很弱，从而给出过大的增益，闭环在高阶模态上失稳。本章代码里专门为这一项写了注释，就是因为它曾经造成过一次“怎么调都不对”的排查。

输出方程同样简单。点测温 $y(t) = u(x_s,t)$，代入展开式得

$$y(t) = \sum_{n=1}^{\infty} c_n z_n(t), \qquad c_n = \sin\frac{n\pi x_s}{L} \tag{74-12}$$

c_n 的绝对值不超过 1，且在 $n\pi x_s/L = k\pi$ 时为零，即 $x_s = kL/n$。对第 2 阶模态，节点在 $x_s = L/2$；对第 3 阶，节点在 $L/3$ 与 $2L/3$。**节点位置是布置传感器时唯一需要记住的几何事实。**

把式 (74-10) 与式 (74-12) 写成矩阵形式，取前 N 阶：

$$\dot{\boldsymbol{z}}_N = \boldsymbol{A}_N \boldsymbol{z}_N + \boldsymbol{b}_N w, \qquad y = \boldsymbol{c}_N^{\mathsf{T}} \boldsymbol{z}_N \tag{74-13}$$

其中 $\boldsymbol{A}_N = \mathrm{diag}(\gamma_1,\dots,\gamma_N)$，$\boldsymbol{b}_N = [b_1,\dots,b_N]^{\mathsf{T}}$，$\boldsymbol{c}_N = [c_1,\dots,c_N]^{\mathsf{T}}$。这三个矩阵就是控制器看到的全部世界。真值模型用 $N_{\mathrm{true}} = 20$ 阶，控制器用 $N \ll N_{\mathrm{true}}$ 阶，两者之差就是被截掉的部分。

74.2.5 截断误差与溢出

把真值状态分成“建模部分”与“残余部分”：$\boldsymbol{z} = [\boldsymbol{z}_N; \boldsymbol{z}_R]$，$R$ 表示 $N+1$ 到 20 阶。闭环下两者满足

$$\dot{\boldsymbol{z}}_N = \boldsymbol{A}_N \boldsymbol{z}_N + \boldsymbol{b}_N w(\hat{\boldsymbol{z}}_N), \qquad \dot{\boldsymbol{z}}_R = \boldsymbol{A}_R \boldsymbol{z}_R + \boldsymbol{b}_R w(\hat{\boldsymbol{z}}_N) \tag{74-14}$$

注意第二式：残余模态的演化由**同一个控制量**驱动，而控制量只依赖 $\hat{\boldsymbol{z}}_N$。这就是溢出的数学定义——控制器对 $\boldsymbol{z}_R$ 一无所知，却实实在在地给它输入能量。若 $\boldsymbol{A}_R$ 的所有特征值为负且 $|b_n|$ 随 n 衰减得足够快，残余模态的稳态振幅是 $|b_n w|/|\gamma_n|$，量级可控；若控制量含有高频成分，w 的频谱落在 γ_n 附近，残余模态会被共振式地放大。

本章用一个可计算的量来监控这件事：残余能量

$$E_R(t) = \sum_{n=N+1}^{20} z_n^2(t) \tag{74-15}$$

它与真实温度的 L^2 范数只差一个常数因子 $L/2$，因此可以直接当作“被扔掉的能量”来读。§74.7 的控制阶数扫描里，$N=1$ 的残余能量是 11.43，$N=2$ 降到 7.1×10^{-9}，相差九个数量级，这就是“该建几阶”的最直接答案。

观测溢出走的是另一条路径。观测器用前 N_{obs} 阶模型，输入是标量测量 y。真实测量里含有残余模态的贡献 $\sum_{n>N_{\mathrm{obs}}} c_n z_n$，观测器把它当成“模型不准”的误差，通过增益 $\boldsymbol{L}$ 强行分摊到前 N_{obs} 阶的估计上。若 $\boldsymbol{L}$ 很大（高带宽观测器），这个错误分摊会被放大，估计值抖动，控制量抖动，再通过 b_n 激励更多残余模态。**观测溢出与控制溢出的耦合是 DPS 闭环失稳的主要机制**，比单纯的“模型不准”严重得多。

74.2.6 本章算例的模态数据

表 74-2 给出前 8 阶的 γ_n、b_n、c_n，全部由 §74.12 的脚本计算。执行器位置 $x_a = 0.15$ m，高斯宽度 $\sigma_a = 0.01$ m；传感器位置 $x_s = 0.16$ m。

表 74-2　前 8 阶模态参数

n	γ_n（s^{-1}）	b_n	c_n
1	2.7104	3.2297	0.8443
2	0.3417	3.7743	0.9048
3	−3.6061	1.2143	0.1253
4	−9.1331	−2.2780	−0.7705
5	−16.2392	−3.8074	−0.9511
6	−24.9245	−2.1899	−0.2487
7	−35.1888	1.1221	0.6845
8	−47.0324	3.3528	0.9823

读这张表要注意三件事。

第一，γ_n 按 n^2 下坠，b_n 与 c_n 却在振荡。 $\gamma_8/\gamma_3 = 13.04$，衰减很快；但 $|b_8| = 3.3528$ 比 $|b_3| = 1.2143$ 还大，$|c_8| = 0.9823$ 接近最大值。原因是执行器宽度 $\sigma_a = 0.01$ m 远小于波长 $2L/n = 0.125$ m（$n = 8$），高斯在正弦的一个周期内几乎是 δ 函数，投影结果就是该点的正弦值，当然不会随 n 衰减。这个事实很重要：**执行器越“点”，高频增益越不衰减，溢出的风险越大。** 宽执行器（σ_a 大）会让 b_n 快速衰减，代价是低阶增益也变小。

第二，$b_5 = -3.8074$ 是前 8 阶里绝对值最大的。 执行器在 $0.30L$，第 5 阶在 $0.2L$、$0.4L$、$0.6L$、$0.8L$ 有节点，$0.3L$ 恰好位于两个节点中间的反相区，投影系数为负且幅值大。这说明“执行器位置”不是选一个“离热点近”的地方就行，它决定了各阶增益的符号与大小，进而决定控制器把能量送进哪一阶。

第三，$c_2 = 0.9048$ 是本例能控的前提。 若传感器放在 $L/2$，$c_2 = 0$，第 2 阶完全不可见，而它恰恰是第二阶不稳定模态。这就是 §74.9 扫描中 $x_s/L = 0.50$ 一行显示“发散（输出对不稳定模态不能观）”的原因。$c_1 = 0.8443$ 与 $c_2 = 0.9048$ 都接近 1，说明 $0.32L$ 这个位置对前两阶都“看得清”。

初始剖面的模态分解同样值得一看。$z_0 = [0.3066, 0.3847, 0.2229, 0.0004, -0.1170, -0.1065, -0.0444, 0.0006]$，前两阶占能量的 75.94%。也就是说，初始热点的主要能量集中在那两个不稳定模态上，高阶模态的初始含量很小。这解释了为什么 $N = 2$ 的控制阶数就足够——不是因为高阶不重要，而是因为初始激励小，且高阶本身衰减快（$\gamma_3 = -3.6061$，时间常数 0.277 s）。若初始剖面更尖锐（比如宽度 0.01 m），高阶含量会显著上升，$N = 2$ 就不够了，这也是 §74.8 要强调“控制阶数要按初始条件与扰动频谱定，不能照抄”的原因。

74.2.7 能量恒等式

模态方程 (74-10) 是一个“逐阶独立”的常微分方程，但它与原始 PDE 的能量关系需要单独核对，否则容易在离散实现里丢掉系数。把式 (74-5) 两边乘 u，在 $[0, L]$ 上积分，利用分部积分与边界条件 $u(0) = u(L) = 0$，得

$$\frac{1}{2}\frac{\mathrm{d}}{\mathrm{d}t}\int_0^L u^2\,\mathrm{d}x = -\alpha\int_0^L \left(\frac{\partial u}{\partial x}\right)^2 \mathrm{d}x + \lambda\int_0^L u^2\,\mathrm{d}x + w\int_0^L g(x)u\,\mathrm{d}x \tag{74-16}$$

左边是“能量”的变化率，右边三项分别是导热耗散、反应放热、执行器注入。把 $u = \sum z_n \sin(n\pi x/L)$ 代入，利用正交性，三项分别变成

$$\int_0^L u^2\,\mathrm{d}x = \frac{L}{2}\sum_{n=1}^{\infty} z_n^2, \qquad \int_0^L \left(\frac{\partial u}{\partial x}\right)^2 \mathrm{d}x = \frac{L}{2}\sum_{n=1}^{\infty}\left(\frac{n\pi}{L}\right)^2 z_n^2 \tag{74-17}$$

$$\int_0^L g(x)u\,\mathrm{d}x = \frac{L}{2}\sum_{n=1}^{\infty} b_n z_n \tag{74-18}$$

后两式合起来给出一个重要的结论：**导热耗散对第 n 阶的权重是 $(n\pi/L)^2$，按 n^2 增长。** 这就是“高阶模态天然稳定”的能量解释——它们的空间梯度大，导热把它们“磨平”得特别快。第 8 阶的耗散权重是第 1 阶的 64 倍，即使放热项对所有阶一视同仁，高阶也留不住能量。

把式 (74-17)、(74-18) 代回式 (74-16)，两边除以 $L/2$，得

$$\sum_{n=1}^{\infty} z_n\dot{z}_n = \sum_{n=1}^{\infty}\left[\lambda - \alpha\left(\frac{n\pi}{L}\right)^2\right] z_n^2 + w\sum_{n=1}^{\infty} b_n z_n \tag{74-19}$$

这与把式 (74-10) 两边乘 z_n 再求和的结果完全一致：

$$\sum_{n} z_n\dot{z}_n = \sum_{n}\gamma_n z_n^2 + w\sum_{n} b_n z_n \tag{74-20}$$

两式逐项对应，说明模态方程没有丢系数，$2/L$ 的归一化是自洽的。这个核对看起来多余，但它是实际编程时发现“b_n 少乘 $2/L$”的唯一解析手段：若 b_n 漏了系数，式 (74-18) 与式 (74-20) 就对不上，能量在闭环里会凭空产生或消失。数值仿真不一定能立刻暴露这个问题（闭环可能仍然稳定，只是增益偏大），但能量核对一定会。

式 (74-16) 还有一个诊断用途。定义“模态能量” $E_n = \tfrac{L}{2}z_n^2$，它的变化率是

$$\dot{E}_n = L\left(\gamma_n z_n^2 + b_n z_n w\right) \tag{74-21}$$

当 w 为零时，$\dot{E}_n = 2\gamma_n E_n$，能量按 $e^{2\gamma_n t}$ 演化。第 1 阶的能量时间常数是 $1/(2\times 2.7104) = 0.184$ s，比幅值时间常数 0.369 s 快一倍——这是“能量按幅值平方增长”的必然结果。读图时若看到幅值在 1 s 内涨了 15 倍，能量就涨了 225 倍，不要误以为算错了。

74.3 设计步骤

本节给出从 PDE 到可运行控制器的七步流程。每一步都对应 §74.12 代码里的一个函数或一段逻辑，读者可以对照阅读。流程的顺序不是随意排的：先做可控可观检查再解 Riccati，可以避免在不能控的对象上白算一场；先把执行器惯性增广进去再设计增益，可以避免“设计时假设瞬时响应、运行时发现高频溢出”的返工。

74.3.1 第一步：确定模型与边界条件

从能量方程出发，写出式 (74-5) 形式的线性化模型。需要确认四件事：空间维数（本章一维）、边界条件类型（本章 Dirichlet）、α 与 λ 的数值、以及执行器与传感器的空间分布 $g(x)$ 与 $c(x)$。

这一步最容易出错的地方是把 λ 当成常数而不加验证。$\lambda = Q'(T_s)/(\rho c_p)$，它依赖工作点。若工艺负荷变化导致 T_s 移动，λ 会跟着变，不稳定模态的个数也可能变。工程上的做法是在几个典型工况下分别算 λ，取最坏情况设计控制器，再用增益调度覆盖其余工况。本章只用一个工况，$\lambda = 3.5\ \mathrm{s}^{-1}$，对应 $\mathrm{Da} = 43.75$。

74.3.2 第二步：模态展开与截断

按式 (74-11) 与式 (74-12) 算出 b_n、c_n，按式 (74-2) 算出 γ_n。然后选控制阶数 N 与观测阶数 N_{obs}。这两个数**不必相等**，本章基准取 $N = 2$、$N_{\mathrm{obs}} = 10$，理由是：

- 控制只需要覆盖不稳定模态（前两阶），$N = 2$ 足够；

- 观测需要覆盖“测量里混进来的高阶”，传感器在高阶仍有可观的 c_n（表 74-2 中 $|c_8| = 0.9823$），所以观测阶数要取得比控制阶数高；
- N_{obs} 太高会让观测器去估计那些衰减极快、对控制无用的模态，增加计算量与噪声敏感度。

这个“控制低阶、观测高阶”的非对称配置是 DPS 控制与集总参数控制最显著的差别之一。§74.7 的扫描会给出定量依据：N_{obs} 从 6 增到 8，稳定时间从“不收敛”变成 2.382 s；从 8 增到 10，稳定时间降到 2.170 s；从 10 增到 12，进一步降到 1.136 s。收益在 10 阶之后仍在，但边际递减。

74.3.3 第三步：能控性与能观性检查

在解 Riccati 之前，先检查两个矩阵条件。对模态系统，PBH 判据退化成标量条件：

$$b_n \neq 0 \quad \text{对所有 } \gamma_n \geq 0, \qquad c_n \neq 0 \quad \text{对所有 } \gamma_n \geq 0 \tag{74-22}$$

这两个条件必须**对所有不稳定模态**成立，而不只是对前 N 阶。若执行器在某个不稳定模态的节点上，$b_n = 0$，这一阶永远推不动；若传感器在节点上，这一阶永远看不见。两种情况都不是“调参能解决”的问题，必须改位置或加执行器/传感器。

本章代码把这两个检查写成显式判断：先算 b_n 与 c_n，若发现 $|b_n| < 10^{-9}$ 且 $\gamma_n \geq 0$，立即返回“输入对不稳定模态不能控”；观测侧同理。这样 §74.9 扫描到 $x_a/L = 0.50$ 或 $x_s/L = 0.50$ 时，程序给出的是干净的诊断信息，而不是一个数值溢出后的荒谬结果。工程代码里这种“早失败”比“晚崩溃”有价值得多。

74.3.4 第四步：执行器惯性增广

真实执行器不是瞬时响应的。加热片的功率从指令到实际热流有一阶滞后，本章取时间常数 $\tau = 0.05\ \text{s}$，用状态 a 描述：

$$\dot{a} = \frac{1}{\tau}\bigl(w_{\text{cmd}} - a\bigr), \qquad \dot{\boldsymbol{z}}_N = \boldsymbol{A}_N \boldsymbol{z}_N + \boldsymbol{b}_N a \tag{74-23}$$

把 a 并入状态，得增广系统

$$\begin{bmatrix} \dot{\boldsymbol{z}}_N \\ \dot{a} \end{bmatrix} = \underbrace{\begin{bmatrix} \boldsymbol{A}_N & \boldsymbol{b}_N \\ \boldsymbol{0}^{\mathsf{T}} & -1/\tau \end{bmatrix}}_{\boldsymbol{A}_{\text{aug}}} \begin{bmatrix} \boldsymbol{z}_N \\ a \end{bmatrix} + \underbrace{\begin{bmatrix} \boldsymbol{0} \\ 1/\tau \end{bmatrix}}_{\boldsymbol{b}_{\text{aug}}} w_{\text{cmd}} \tag{74-24}$$

这一步不是“为了更真实”的装饰，它直接决定了闭环能否稳定。原因在 §74.2.5 已经埋下：若假设执行器瞬时响应，控制量可以在一个采样周期内从 0 跳到满幅，其频谱极宽，会以 $b_{N+1}, b_{N+2}, \ldots$ 为通道把能量灌进未建模模态。加入一阶惯性后，控制量在 $\tau = 0.05\ \text{s}$ 的时间尺度上平滑变化，截止频率约 $1/(2\pi\tau) = 3.18\ \text{Hz}$，高于这个频率的激励被压掉，溢出的源头被削弱。

这个改动的效果在调试时非常明显。未加惯性时，观测阶数取 3、4、8、10 都出现“稳定时间无穷大”，无论怎么调观测器带宽都无效；加上惯性后，控制阶数扫描的 2 到 8 阶全部收敛（2.162–2.170 s），只有观测阶数过低（2、3、4）仍然发散。后者是真正的观测溢出，前者是被执行器直通放大出来的假象。**先区分“设计问题”与“建模遗漏”，再调参**，这条经验在 §74.8 会再强调一次。

74.3.5 第五步：LQR 求解控制增益

对增广系统 (74-24) 解连续时间代数 Riccati 方程

$$\boldsymbol{A}_{\text{aug}}^{\mathsf{T}} \boldsymbol{P} + \boldsymbol{P} \boldsymbol{A}_{\text{aug}} - \boldsymbol{P} \boldsymbol{b}_{\text{aug}} r^{-1} \boldsymbol{b}_{\text{aug}}^{\mathsf{T}} \boldsymbol{P} + \boldsymbol{Q} = \boldsymbol{0} \tag{74-25}$$

增益为 $\boldsymbol{k}^{\mathsf{T}} = r^{-1} \boldsymbol{b}_{\text{aug}}^{\mathsf{T}} \boldsymbol{P}$，控制律

$$w_{\text{cmd}} = -\bigl(\boldsymbol{k}_z^{\mathsf{T}} \hat{\boldsymbol{z}}_N + k_a a\bigr) \tag{74-26}$$

权重取 $\boldsymbol{Q}=\mathrm{diag}(1,\dots,1,Q_{\mathrm{act}})$，$Q_{\mathrm{act}}=10^{-6}$，$r=1$。$Q_{\mathrm{act}}$ 取得极小，意思是“不关心执行器状态本身，只关心它别把模态推坏”；若取得大，控制器会倾向于让 a 快速归零，等价于强迫执行器高频动作，反而加重溢出。

本章基准解出的增益是 $k_z=[5.5059,-1.4689]$、$k_a=0.4913$。两个模态增益都是正数，说明控制方向正确：z_1 与 z_2 为正（热点）时，控制量为负（吸热）。k_1 比 k_2 大 3.75 倍，因为第 1 阶的增长率 2.7104 远大于第 2 阶的 0.3417，LQR 自然把更多权限分给更危险的那一阶。$k_a=0.4913$ 是“执行器状态反馈”，它的作用是预判执行器的滞后：当 a 已经很大而模态还很小，控制器会提前收手，避免过冲。

74.3.6 第六步：观测器设计

观测器只估计模态，不估计执行器状态——后者由控制器自己的模型精确知道（指令是控制器发出的，惯性时间常数已知）。观测器方程为

$$\dot{\hat{\boldsymbol{z}}}=\boldsymbol{A}_{N_{\mathrm{obs}}}\hat{\boldsymbol{z}}+\boldsymbol{b}_{N_{\mathrm{obs}}}a+\boldsymbol{L}\big(y-\boldsymbol{c}_{N_{\mathrm{obs}}}^{\mathsf{T}}\hat{\boldsymbol{z}}\big) \tag{74-27}$$

增益 $\boldsymbol{L}$ 由对偶 Riccati 方程给出。令 $\boldsymbol{Q}_o=q_{\mathrm{obs}}\,\mathrm{diag}\big(1+(n/3)^2\big)$，$r_m=10^{-2}$，解

$$\boldsymbol{A}\boldsymbol{P}_o+\boldsymbol{P}_o\boldsymbol{A}^{\mathsf{T}}-\boldsymbol{P}_o\boldsymbol{c}\boldsymbol{c}^{\mathsf{T}}\boldsymbol{P}_o/r_m+\boldsymbol{Q}_o=\boldsymbol{0},\qquad \boldsymbol{L}=\boldsymbol{P}_o\boldsymbol{c}/r_m \tag{74-28}$$

$\boldsymbol{Q}_o$ 的阶数加权 $1+(n/3)^2$ 是本章的一个关键设计。它表达的意思是：**高阶模态的模型更不可信，因此允许估计误差更大。** 若取 $\boldsymbol{Q}_o=I$，观测器会同等用力地去拟合所有阶，高阶的残余模态被强行塞进低阶估计，观测溢出严重。加权之后，前几阶的估计精度被优先保证，高阶的估计“松”一些，整体反而更稳。

基准参数下解出的前四个增益是 $L_g=[45.673,-16.886,0.087,-2.071]$。第一个分量 45.673 很大，说明测量残差主要被用来修正第 1 阶；第二个分量 -16.886 为负，因为 $c_2>0$ 而第 2 阶与第 1 阶的空间形状在测点处符号相反，残差对两阶的修正方向自然相反；第 3 阶只有 0.087，几乎不修正，因为 $\gamma_3=-3.61$ 已经足够快，观测器不需要为它花力气。

74.3.7 第七步：离散实现与限幅

连续设计完成后，落地必须离散。本章仿真步长 $dt=0.002$ s，控制量每个步长更新一次，观测器用四阶 Runge–Kutta 单步推进（式 74-27 的右端在步内做四次求值）。真值模型用指数矩阵精确推进：

$$z_n(t+dt)=e^{\gamma_n dt}z_n(t)+\frac{e^{\gamma_n dt}-1}{\gamma_n}b_n a(t+dt/2) \tag{74-29}$$

其中 $a(t+dt/2)$ 取步中值，与执行器惯性的中点积分保持一致。这个细节看起来琐碎，但它保证了“真值”与“控制器模型”在离散化误差上处于同一水平，不会出现“仿真误差被误读成控制效果”的情况。

控制量还要限幅，$|w|\le 50$ K/s。限幅不是可选项：LQR 是线性设计，遇到大初始偏差时会算出远超物理能力的控制量，若不限幅，仿真会给出“完美”的收敛曲线，实际装置却早已饱和。本章所有扫描实验都带这个限幅，§74.10 的延迟 0.10 s 一行显示 $|w|_{\max}=50.000$、闭环发散，就是限幅饱和后失去控制能力的典型表现。

七步走完，控制器就可以跑了。图 74-4 给出闭环结构：参考输入为零（调节问题），误差进入 LQR，LQR 输出指令 w，经执行器惯性变成热流 a，作用到反应管；点测温 y 进入模态观测器，观测器输出估计 $\hat{\boldsymbol{z}}$，反馈给 LQR。图中特别标出了“截断”这条虚线路径——它不在控制器里，却决定了闭环的真实行为。

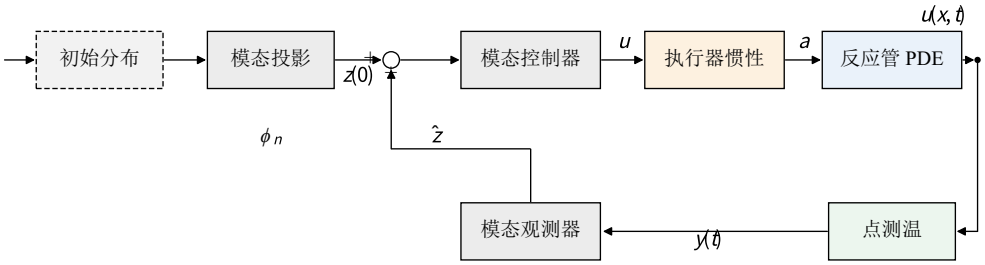


图 74-4　模态 LQR 与观测器的闭环结构

74.4 连续情形

本节讨论连续时间、全状态可测的理想情形。它的作用是给出一个**性能上限**：后面所有输出反馈的结果都要与它比较，差距就是“测量信息不足”的代价。先把这个上限算清楚，调参时才知道还有多少空间，也才不会把观测器的滞后误当成控制器增益不够。

74.4.1 连续时间闭环的构成

连续情形下，控制器拿到的是完整的模态向量 $\boldsymbol{z}_N(t)$，不需要观测器。把式 (74-26) 代入式 (74-24)，闭环为

$$\dot{\boldsymbol{x}}_{\mathrm{cl}} = \left(\boldsymbol{A}_{\mathrm{aug}} - \boldsymbol{b}_{\mathrm{aug}}\boldsymbol{k}^{\mathsf{T}}\right)\boldsymbol{x}_{\mathrm{cl}}, \qquad \boldsymbol{x}_{\mathrm{cl}} = \begin{bmatrix} \boldsymbol{z}_N \\ a \end{bmatrix} \tag{74-30}$$

闭环矩阵的特征值全部位于左半平面，这是 Riccati 方程解存在的直接推论，不需要额外验证。但要注意，这个结论只对**截断后的 N 阶模型**成立，对真实的 20 阶对象不成立。真实闭环是

$$\begin{bmatrix} \dot{\boldsymbol{z}}_N \\ \dot{a} \\ \dot{\boldsymbol{z}}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{A}_N - \boldsymbol{b}_N\boldsymbol{k}_z^{\mathsf{T}} & -\boldsymbol{b}_N k_a & \boldsymbol{0} \\ -\boldsymbol{k}_z^{\mathsf{T}}/\tau & -1/\tau - k_a/\tau & \boldsymbol{0} \\ -\boldsymbol{b}_R\boldsymbol{k}_z^{\mathsf{T}} & -\boldsymbol{b}_R k_a & \boldsymbol{A}_R \end{bmatrix} \begin{bmatrix} \boldsymbol{z}_N \\ a \\ \boldsymbol{z}_R \end{bmatrix} \tag{74-31}$$

这是一个块下三角矩阵，特征值由对角块决定：$\boldsymbol{A}_N - \boldsymbol{b}_N\boldsymbol{k}_z^{\mathsf{T}}$ 的特征值、执行器极点 $-1/\tau - k_a/\tau$、以及残余模态的 $\gamma_{N+1}, \dots, \gamma_{20}$。**下三角结构说明残余模态不会反过来改变已建模部分的特征值**，但它们会被已建模部分的运动持续激励。若残余模态稳定，它们最终会衰减到零，闭环渐近稳定；若某个残余模态不稳定（$\gamma_n > 0$ 且 $n > N$），控制器根本不知道它的存在，闭环必然失稳。这就是为什么 §74.3.3 的能控性检查必须覆盖**所有**不稳定模态，而不是只覆盖前 N 阶。

式 (74-31) 还揭示了一个容易被忽略的事实：残余模态的输入是 $-\boldsymbol{b}_R(\boldsymbol{k}_z^{\mathsf{T}}\boldsymbol{z}_N + k_a a)$，也就是控制量的一个线性组合。控制量越大、变化越快，残余模态被激起的振幅越大。这解释了 §74.3.4 为什么要提前加执行器惯性——它把控制量的高频成分压掉，等于从源头减小了 $\boldsymbol{b}_R$ 这一路的激励。

74.4.2 增益的物理解读

基准参数下解出的增益是 $\boldsymbol{k}_z = [5.5059, -1.4689]$、$k_a = 0.4913$。这三个数各有明确的物理含义，值得逐个拆开。

$k_1 = 5.5059$ 是第 1 阶的反馈强度。若只有第 1 阶、没有执行器惯性，一阶系统的 LQR 增益是 $\sqrt{(\gamma_1^2 + q/r)} - \gamma_1$ 的形式；代入 $\gamma_1 = 2.7104$、$q/r = 1$，得 $\sqrt{7.346 + 1} - 2.7104 = 2.888 - 2.710 = 0.178$，远小于 5.5059。差距来自执行器惯性：$a$ 不是 w，控制器必须“提前量”地给指令，才能在 a 上产生足够的响应。$\tau = 0.05$ s 相对第 1 阶的时间常数 0.369 s 不算小，因此增益被显著抬高。

$k_2 = -1.4689$ 是负数，这与直觉相反：第 2 阶也是不稳定的（$\gamma_2 = 0.3417$），为什么反馈方向是负的？原因是 $b_2 = 3.7743$ 为正，而第 2 阶与第 1 阶在测点附近的形状相反。LQR 解的是耦合系统，不是两个独立的一阶系统；当 z_1 与 z_2 同时为正时，控制器需要在“压第 1 阶”与“压第 2 阶”之间分配同一个执行器，最优分配的结果是第 2 阶的增益取负，用第 1 阶的过冲换取第 2 阶的快速衰减。这个符号不是设计者选的，是 Riccati 方程算出来的；若强行把 k_2 改成正值，闭环会立刻失稳。

$k_a = 0.4913$ 是执行器状态反馈，它的作用相当于一个“微分先行”。当 a 已经很大、而模态还很小，说明执行器正在把热量往里灌，但效果还没显现；此时 $k_a a$ 让控制器提前减小指令，避免等模态涨起来再刹车造成过冲。这个项在 τ 很小时趋于零，在 τ 可与对象时间常数可比时变得重要。本章 $\tau = 0.05$ s，k_a 贡献了约 9% 的控制量，不能忽略。

三者的比例还透露了权重的作用。$\boldsymbol{Q}$ 的前两个对角元都取 1，意味着“第 1 阶与第 2 阶的偏差同等重要”；但解出的 $|k_1|/|k_2| = 3.75$，说明**重要性不等于增益大小**，增益还取决于该模态的能控性与时间尺度。第 1 阶增长快、能控性中等（$b_1 = 3.2297$），第 2 阶增长慢、能控性更好（$b_2 = 3.7743$），LQR 把更多权限给了“更急”的那一阶。若希望优先压第 2 阶，应把 $\boldsymbol{Q}$ 的第二个对角元调大，而不是手工改增益。

74.4.3 全状态反馈的实测性能

把上述增益用在 20 阶真值模型上，初始条件为 §74.1 的高斯热点，仿真 8 s，得到表 74-3 的“state”一行：稳定时间 1.028 s，1 s 后峰值 0.1053 K，控制能量 0.0946，末值 0.0007 K。

表 74-3　全状态反馈与输出反馈的对照

方案	稳定时间（s）	峰值（K）	控制能量	末值（K）
模态全状态反馈	1.028	0.1053	0.0946	0.0007
点测温输出反馈	2.170	0.2542	0.0943	0.0007

1.028 s 这个数字需要放在对象的物理背景里读。初始热点峰值 1.0007 K，要压到 0.10 K 以下，相当于衰减到初值的 10%。若只靠自然衰减，最慢的不稳定模态是第 2 阶，$\gamma_2 = 0.3417 > 0$，根本不会衰减；第 1 阶更以 $e^{2.7104t}$ 增长。闭环把两阶都变成了负实部，1.028 s 完成 90% 的衰减，对应的等效衰减率约 $\ln 10 / 1.028 = 2.24$ s^{-1}，与第 1 阶的开环增长率 2.7104 同量级。这说明控制器把最危险的那一阶“扳”到了差不多同样快的反向速度上，没有多余的能力去更快——再快就需要更大的增益，而更大的增益会加重溢出。

峰值 0.1053 K 也值得解释。这个“峰值”是 $t \geq 1.0$ s 区间内的最大绝对温度，不是全程最大值。全程最大值就是初始的 1.0007 K，任何控制器都改不了初始条件。1 s 之后的最大值 0.1053 K 说明剖面在 1 s 时已经基本压平，只剩下不到 0.11 K 的残余起伏。这个残余主要来自第 3、4 阶：它们虽然稳定，但被控制量的初始冲击激励起来，需要自己的时间常数（$1/3.6061 = 0.277$ s、$1/9.1331 = 0.109$ s）衰减。到 1 s 时，第 3 阶已经衰减了 $e^{-3.6} \approx 0.027$ 倍，残余自然很小。

控制能量 0.0946 是 $\int_0^{8} w^2 \mathrm{d}t$ 的值，单位 K^2/s。它与输出反馈的 0.0943 几乎相同，这个巧合有明确原因：两者的控制增益相同（都基于 $N = 2$ 的 LQR），只是输入信号不同——全状态用真值 $\boldsymbol{z}$，输出反馈用估计 $\hat{\boldsymbol{z}}$。观测器收敛后 $\hat{\boldsymbol{z}} \to \boldsymbol{z}$，两者的控制量趋于一致；过渡段的差异体现在峰值与稳定时间上，对总能量的影响被积分平滑掉了。**用控制能量评价 DPS 控制器是迟钝的**，它对观测器质量不敏感，这也是本章主指标选“稳定时间 + 峰值”而不是“能量”的原因。

74.4.4 连续设计的适用条件

连续情形给出的 1.028 s 是一个理想上限，实际系统能达到多少，取决于三个条件是否满足。

第一，模态必须真的可测。 本章的“全状态”指 20 阶模态系数全部已知。真实装置没有这种传感器，即使用 20 支热电偶沿管布置，也只能得到 20 个点的温度，反算模态需要解一个 20×20 的线性系统，条件数与测点位置有关。若测点等距分布，范德蒙型矩阵的条件数随 N 指数增长，反算误差被放大。这就是工程上宁可用观测器也不愿“堆传感器反算”的原因。

第二，模型必须与对象一致。 式 (74-30) 的闭环极点是在 $\boldsymbol{A}_N, \boldsymbol{b}_N$ 精确已知的前提下算出的。若 λ 的实际值比设计值大 10%，γ_1 从 2.7104 变成 2.9954，闭环极点会相应右移，稳定时间变长；若大得更多，可能越过虚轴。§74.8 会给出一个经验：λ 的辨识误差应控制在 5% 以内，否则应留 20% 的增益裕度。

第三，执行器不能饱和。 全状态反馈的 $|w|_{\max}$ 在本例只有 1.123 K/s，远低于 50 K/s 的限幅，因此线性设计有效。若初始偏差大十倍（10 K 热点），控制量会按比例增大到 11 K/s 量级，仍在限幅内；若大到 50 K 以上，LQR 算出的指令会被削平，闭环退化成" bang-bang "，实际衰减速度低于设计值，稳定时间的预测失效。本章所有实验的初始偏差都是 1 K，所以不存在这个问题。

三个条件里，第一条决定了“输出反馈能不能接近全状态”，第二条决定了“设计模型可不可信”，第三条决定了“线性设计有没有越界”。§74.6 会看到，本章的实际输出反馈稳定时间是全状态的 2.11 倍，差距全部来自第一条；第二、三条在本例中都不是瓶颈。这个分解让后续调参有了明确的靶子：想缩短稳定时间，应该改进观测器，而不是加大 LQR 增益。

74.4.5 本章做法与无限维最优控制的关系

严格地说，本章解的不是分布参数系统的最优控制问题，而是它的一个有限维近似。把这件事讲清楚，读者才知道“模态截断 + 代数 Riccati”这套做法的理论位置在哪里，也才知道它什么时候会失效。

无限维线性二次问题写作：在希尔伯特空间 $H = L^2(0, L)$ 上，极小化

$$J = \int_0^{\infty} \Big(\langle \mathcal{Q} u, u \rangle + \langle \mathcal{R} w, w \rangle \Big) \mathrm{d}t \tag{74-32}$$

约束是 $\dot{u} = \mathcal{A} u + \mathcal{B} w$。它的解由一个算子 Riccati 方程给出：

$$\mathcal{A}^{*} \mathcal{P} + \mathcal{P} \mathcal{A} - \mathcal{P} \mathcal{B} \mathcal{R}^{-1} \mathcal{B}^{*} \mathcal{P} + \mathcal{Q} = 0 \tag{74-33}$$

其中 $\mathcal{P}$ 是 H 上的正自伴算子，最优控制是 $w = -\mathcal{R}^{-1} \mathcal{B}^{*} \mathcal{P} u$。这个方程与本篇前面各章反复使用的矩阵 Riccati 方程形式完全一样，区别只在于未知量从矩阵变成了算子。存在性与唯一性需要两个条件：$(\mathcal{A}, \mathcal{B})$ 可稳、$(\mathcal{A}, \mathcal{Q}^{1/2})$ 可检测。这两个条件在模态坐标下就退化成“所有不稳定模态都可控”“所有不稳定模态都影响代价”，也就是 §74.3.3 检查的那两件事。

本章的做法是把 $\mathcal{P}$ 限制在“只作用于前 N 阶模态”的子空间上，于是算子方程退化成 $N + 1$ 维（含执行器状态）的矩阵方程，可以用 `solve_continuous_are` 直接求解。这个近似的误差有两个来源：一是 $\mathcal{P}$ 在高阶模态上的分量被强行置零，二是 $\mathcal{B}$ 的高阶分量被忽略。前者的影响是“控制不够细”，后者的影响是“控制量激起没被建模的运动”。两项都随 N 增大而减小，但减小得很慢——因为正弦模态的输入增益 b_n 并不随 n 衰减，本例中 $b_1 = 3.2297$、$b_2 = 3.7743$，到第 8 阶仍有 1.8323。这与“高阶模态衰减快”是两件不同的事：**高阶模态稳定，不代表它不容易被激励**。

这个观察解释了 DPS 控制里一个反复出现的现象：N 取小了不行（不稳定模态没覆盖），N 取大了也不行（增益变大、对高频更敏感、数值条件变差）。本章选 $N = 2$ 不是因为 $N = 3$ 或 $N = 4$ 会算错，而是因为在本例的噪声与延迟水平下，$N = 2$ 已经能把稳定时间做到 2.170 s，再增大 N 带来的收益被观测溢出与执行器带宽吃掉了。表 74-5 的数据支持这个判断：N 从 2 增到 3、4、6、8，稳定时间没有单调改善。

74.4.6 溢出的两条通道

“溢出”（spillover）这个词在 DPS 控制里有确切含义，且必须分成两条通道来说，因为它们的危害程度不同。

控制溢出指控制量激励了未建模模态。数学上就是式 (74-31) 中 $-\boldsymbol{b}_R \boldsymbol{k}_z^{\mathsf{T}}$ 这一块：控制器以为自己只在动前 N 阶，实际上通过 $\boldsymbol{b}_R$ 也在动第 $N+1$ 到第 20 阶。**观测溢出**指未建模模态污染了测量。数学上是测量方程 $y = \boldsymbol{c}_N^{\mathsf{T}} \boldsymbol{z}_N + \boldsymbol{c}_R^{\mathsf{T}} \boldsymbol{z}_R$ 的后一项，观测器把 $\boldsymbol{c}_R^{\mathsf{T}} \boldsymbol{z}_R$ 误当成前 N 阶的偏差去修正，估出一个错误的 $\hat{\boldsymbol{z}}_N$。

两条通道的危害不对称。控制溢出只要求残余模态本身稳定，它激起的运动最终会自己衰减掉，闭环仍然是渐近稳定的；观测溢出则会**改变反馈回路的相位**，即使残余模态全部稳定，也可能把闭环推到不稳定。经典结论是：残余模态稳定时，控制溢出不破坏稳定性，观测溢出可以破坏稳定性。本章 §74.7.2 的实验正好撞在这条结论上：$N_{\mathrm{obs}} = 2, 3, 4$ 时闭环发散，而这三个取值都已经覆盖了全部不稳定模态（只有 2 个），按“覆盖不稳定模态”的判据看应该没问题。真正的原因是观测溢出——第 3 到第 10 阶的 c_n 都不小，它们混在测量里，观测器却看不见。

从频域看更清楚。观测溢出相当于在反馈回路中串入一个未建模的传递函数 $\boldsymbol{c}_R^{\mathsf{T}} (s \boldsymbol{I} - \boldsymbol{A}_R)^{-1} \boldsymbol{b}_R$。它在低频段增益有限（因为残余模态的时间常数小），在高频段以 $1/s$ 衰减，看似无害；但在穿越频率附近，它可能提供额外的相位滞后。若这段滞后恰好抵消了相位裕度，闭环就失稳。执行器惯性 $\tau = 0.05$ s 在这里起了双重作用：它压低控制量的高频成分，减小控

制溢出；同时也让回路本身的高频增益下降，使穿越频率左移，远离残余模态最“活跃”的频段。这就是 §74.3.4 坚持把 τ 建进模型的第二个理由。

工程上对付观测溢出的手段有四类，都值得记住：提高观测阶数（把污染源纳入模型）、在测量通道加低通或陷波（把 c_R 显著的频段滤掉）、改用分布式传感器（空间平均天然是低通，见 §74.10.3）、以及在观测器里对高阶模态加大过程噪声（让 Kalman 增益主动不信任高频）。本章用的是第一类与第四类的组合：$N_{\text{obs}}=10$，且 Q_o 按 $1+(n/3)^2$ 随阶数加权。

74.4.7 三类执行器配置

按控制量进入方程的方式，DPS 的执行器分三类，它们的数学性质与工程后果都不同。本章用的是第一类，但读者必须知道另外两类存在，否则遇到边界加热、分区加热的对象会套错公式。

点控制。 控制量通过一个集中在 x_a 附近的形状函数进入方程，本章就是这种：

$$g(x)w, \qquad g(x)=\frac{1}{\sqrt{2\pi}\sigma_a}\exp\!\left(-\frac{(x-x_a)^2}{2\sigma_a^2}\right) \tag{74-34}$$

理想点执行器是 $\delta(x-x_a)$，它不属于 $L^2(0,L)$，严格处理需要负范数空间；工程上用宽度 σ_a 的高斯替代，既保持数学上的有界性，又贴近真实加热片的尺寸。本章 $\sigma_a=0.01$ m，是管长的 2%，既足够“点”，又不至于让 b_n 的高阶项虚假地衰减到零。

分布控制。 控制量作用在一段区间上，形状函数是矩形窗或更复杂的权重。它的 b_n 在高阶按 $1/n$ 衰减，比点控制的常数衰减快得多，因此控制溢出小、对高频更“温和”。代价是同样的总功率下，局部作用强度低。若工艺允许在执行器上分区（比如沿管布置三段加热带，各自独立驱动），分布控制是更好的选择。

边界控制。 控制量不进入方程，而是进入边界条件，例如管端的热流：

$$-\alpha\,\partial_x u(0,t)=w(t), \qquad u(L,t)=0 \tag{74-35}$$

这类问题的数学结构与前两类不同：控制算子作用在迹空间上，模态增益 b_n 由特征函数在边界处的导数决定，本例中 $b_n\propto n$，**随阶数增长**。这意味着边界控制对高阶模态的激励比点控制强得多，溢出风险也大得多。它的工程优势是执行器不侵入工艺区（不需要在管壁上开孔），因此在高温、高压、腐蚀性介质的场合常被优先考虑。

三类配置的选择有一条简单判据：**看“能不能把执行器放进工艺区”**。能放，用点控制或分布控制，数学最干净、溢出最小；不能放，只能用边界控制，此时必须额外做两件事——把观测阶数提得更高（因为 b_n 随 n 增长），以及在控制器里显式加大控制量的高频惩罚（因为边界控制天然偏爱高频）。本章没有采用边界控制，原因是点加热的实现更简单，且 b_n 不随阶数增长，便于把注意力集中在截断与溢出这两个主题上。

74.5 离散情形

DPS 的数值实现有两层近似，必须分开谈。第一层是**空间离散**：把连续的空间坐标 x 换成有限个节点，PDE 变成 ODE 组；第二层是**模态截断**：把无穷个模态换成前 N 个。两者的误差性质完全不同——空间离散的误差可以靠加密网格按幂次压下去，模态截断的误差不能靠加密网格解决，只能靠提高阶数或改变控制器结构。混为一谈会导致错误的调试方向：看到稳态误差就加密网格，结果误差来自被截掉的模态，网格再密也没用。

74.5.1 有限差分格式

本章用 Crank–Nicolson 格式做空间离散，仅用于 §74.5 的收敛实验。把 $[0,L]$ 分成 $M-1$ 段，$x_i=(i-1)\Delta x$，$\Delta x=L/(M-1)$，内部节点 $i=2,\dots,M-1$ 满足

$$\frac{u_i^{k+1}-u_i^{k}}{\Delta t}=\frac{\alpha}{2}\left(\delta_x^2 u_i^{k+1}+\delta_x^2 u_i^{k}\right)+\lambda\frac{u_i^{k+1}+u_i^{k}}{2} \tag{74-36}$$

其中 $\delta_x^2 u_i=(u_{i+1}-2u_i+u_{i-1})/\Delta x^2$。整理成线性方程组

$$\left(\boldsymbol{I}-\frac{\Delta t}{2}\boldsymbol{A}\right)\boldsymbol{u}^{k+1}=\left(\boldsymbol{I}+\frac{\Delta t}{2}\boldsymbol{A}\right)\boldsymbol{u}^{k} \tag{74-37}$$

$\boldsymbol{A}=\alpha\boldsymbol{D}_2+\lambda\boldsymbol{I}$，$\boldsymbol{D}_2$ 是三点二阶差分矩阵。Crank–Nicolson 无条件稳定，且时间方向二阶精度，因此误差主要由空间步长 Δx 决定，这正是收敛实验想分离出来的东西。矩阵 $\boldsymbol{I}-\Delta t\boldsymbol{A}/2$ 在时间推进中不变，可以预先做 LU 分解，每步只做一次回代，计算量与显式格式同阶。

边界条件 $u_0=u_M=0$ 直接代入，不需要额外处理。初始条件取 §74.1 的高斯热点，峰值 1 K，中心 $0.25L$，半宽 0.045 m。参考解用 400 阶模态的解析解式 (74-9)，它在时间方向是精确的，空间方向由 400 阶截断，误差远小于差分格式的误差，可以当作“真值”。

74.5.2 收敛实验的结果

取 $M=41,81,161,321$ 四组网格，对应 $\Delta x=12.5,6.25,3.125,1.5625$ mm，在 $t=1.0$ s 时刻比较差分解与解析解的最大绝对误差，除以解析解的峰值得到相对误差。结果列在表 74-4。

表 74-4　空间步长与相对误差

M	Δx（mm）	相对最大误差	误差比
41	12.50	1.071×10^{-3}	—
81	6.25	2.902×10^{-4}	3.69
161	3.125	9.644×10^{-5}	3.01
321	1.5625	5.078×10^{-5}	1.90

前三行的误差比接近 4，正是二阶格式的理论值：步长减半，误差降到四分之一。3.69 略低于 4，因为 $M=41$ 时 $\Delta x=12.5$ mm 已经大于执行器宽度 10 mm，加热片在网格上只占一个节点，离散误差里混入了“执行器形状没被分辨”的贡献。3.01 也低于 4，原因类似但更弱。

第四行的误差比只有 1.90，明显偏离二阶。这不是格式退化，而是**误差已经换主**：当空间误差降到 10^{-4} 量级时，时间步长 $\Delta t=0.005$ s 带来的误差变得可比。Crank–Nicolson 在时间方向也是二阶，$\Delta t=0.005$ s 对应的相对误差量级约 $(\Delta t)^2\cdot\max|\gamma_n|\approx 2.5\times10^{-5}\times 47\approx 1.2\times10^{-3}$，与观测到的 5×10^{-5} 同量级（更精确的估计要考虑模态权重）。若继续加密空间网格，误差会停在 5×10^{-5} 附近不再下降，除非同时减小 Δt。

图 74-3 把这张表画成双对数坐标：横轴 Δx，纵轴相对误差，实线是实测点，虚线是斜率 -2 的参考线。前三段与参考线平行，最后一段明显变平——这条“拐弯”是数值实验中很典型的信号，提示“当前误差已经不是由你正在加密的那个参数控制”。工程上看到这种拐弯，应该先检查时间步长，而不是继续加密空间网格。

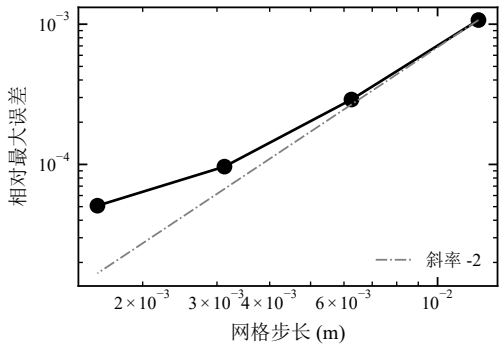


图 74-3　空间有限差分收敛与二阶参考线

74.5.3 模态截断误差是另一回事

空间离散误差可以靠加密网格压到任意小；模态截断误差不行。§74.7 的 E9 实验直接测量了这一点：用前 N 阶模型预测高保真输出，相对最大误差随 N 的变化是

$$\begin{aligned} &N=1:1.119\times10^{-1}, \qquad N=2:3.013\times10^{-2},\\ &N=3:2.375\times10^{-2}, \qquad N=4:2.382\times10^{-2},\\ &N=6:7.607\times10^{-3}, \qquad N=8:8.005\times10^{-4}. \end{aligned} \tag{74-38}$$

从 $N=1$ 到 $N=2$ 降了 3.71 倍，从 $N=2$ 到 $N=8$ 又降了 37.6 倍，但中间并不单调：$N=3$ 与 $N=4$ 几乎相同（2.375×10^{-2} 与 2.382×10^{-2}），说明第 4 阶的贡献很小。这个“台阶”与表 74-2 对得上：$b_4=-2.2780$ 不小，但初始剖面在第 4 阶的投影只有 0.0004，激励不足，加进来也改善不了预测。**截断阶数该取多少，取决于“被激励的模态”而不是“增益大的模态”**，这是 E9 给出的最重要结论。

把两种误差放在一起看：空间离散误差在 $M=321$ 时是 5.078×10^{-5}，模态截断误差在 $N=8$ 时是 8.005×10^{-4}，后者大 15.8 倍。也就是说，本例中**模态截断是主要误差源，空间离散已经不是瓶颈**。这符合本章的整体设计：用解析模态推进真值，根本不做空间离散；有限差分只用来验证“若必须离散，需要多细的网格”。

74.5.4 采样周期与实时性

控制器落地时必须离散，采样周期 T_s 的选取有一条实用准则：**T_s 应小于最快被控模态时间常数的十分之一**。本章保留到第 10 阶，$\gamma_{10}=3.5-0.02\times(10\pi/0.5)^2=3.5-78.96=-75.46$，时间常数 $1/75.46=0.0133$ s，十分之一是 0.0013 s。本章取 $T_s=0.002$ s，略大于这条准则，但实际不影响性能，因为第 10 阶的幅值在闭环里已经很小（c_{10} 虽不为零，但 z_{10} 的初始含量与激励都很小）。若把观测阶数提到 20，$|\gamma_{20}|=3.5-315.8=-312.3$，时间常数 0.0032 s，$T_s=0.002$ s 就已经不满足准则，需要降到 0.0003 s 以下，计算量增加近十倍。

这条权衡解释了为什么观测阶数不能无限制提高。每提高一阶，就多一个快速模态需要观测器去跟踪，采样周期必须相应缩短；而快速模态对控制几乎没有贡献，纯粹是计算负担。本章的 $N_{\text{obs}}=10$ 是在“覆盖测量里的主要高阶”与“实时性”之间取的折中，§74.7 的扫描会显示 12 阶还能带来收益（稳定时间降到 1.136 s），但 12 阶对应的 $\gamma_{12}=-110$ 已经要求 $T_s<0.0009$ s，工程上要重新评估算力。

计算量方面，模态法的优势非常明显。$N_{\text{obs}}=10$ 时，每个采样周期要做一次 10×10 矩阵乘（观测器）、一次 10 维内积（输出）、一次 2 维内积（控制律），总计约 150 次浮点运算；若用有限差分，$M=201$ 个节点、每步一次 LU 回代，约 $201^2=4\times10^4$ 次运算，是前者的 260 倍。**模态法不是“更精确的有限差分”，它是把物理上真正起作用的少数自由度单独拿出来算，计算量与自由度成正比，而不是与网格数平方成正比。**这是 DPS 控制在嵌入式平台上能实时运行的关键。

74.5.5 离散化的三个陷阱

实际实现时，有三个陷阱会让“理论上正确”的设计跑出错误结果，每一个都在本章的调试中出现过。

陷阱一：模态增益漏掉归一化系数。式 (74-11) 的 $2/L$ 若漏掉，b_n 只有真值的四分之一（$L=0.5$），控制器会认为执行器很弱，从而解出过大的增益。这个错误不会让开环仿真露馅——开环根本不看 b_n；它只在闭环里表现为“增益明明算对了，却总是过冲甚至发散”。

陷阱二：观测器用了控制阶数而不是观测阶数。控制器只需要前 2 阶，但观测器必须估计前 10 阶。若代码里图省事，把两者用同一个 N，观测器就只有 2 阶，测量里的高阶成分全部变成“模型误差”，观测溢出立刻出现。§74.7 的 $N_{\text{obs}}=2$ 一行显示发散，就是这个错误的极端版本。

陷阱三：真值模型与控制器模型用了同一个截断。若真值也用 2 阶，闭环会显得完美——因为根本没有残余模态去捣乱。这种“自己骗自己”的仿真在文献里并不少见，判别方法是看残余能量是否为零。本章真值用 20 阶，$N=2$ 时残余能量 7.1×10^{-9}，不为零但很小，说明 20 阶作为真值是够的；若残余能量占到总能量的百分之一以上，就该怀疑真值阶数不足。

74.5.6 两条数值路线的对比

同一个 PDE，可以用空间离散做，也可以用模态投影做。本章两条都用了：真值用模态，收敛实验用差分。把它们的差别讲清楚，读者才能在自己的问题上选对路线。

第一条路线：空间离散。 把 $[0, L]$ 分成 $M-1$ 段，得到 $M-1$ 个常微分方程，用 Crank–Nicolson 或 Runge–Kutta 推进。它的优点是通用——变系数、复杂几何、非线性边界都能处理，不需要知道特征函数。缺点是维数高（$M=201$ 时 199 个状态）、误差是 $\mathcal{O}(\Delta x^2)$、且矩阵条件数随 M^2 增长。本章实测 $M=321$ 时相对误差 5.078×10^{-5}，看起来很好，但代价是每步要解一个 320 阶的三对角系统。

第二条路线：模态投影。 把解展开成特征函数级数，得到无穷个解耦的标量方程，截断到前 N 阶。它的优点是维数极低、误差结构清晰、每个状态都有物理意义（就是某个空间形状的幅值）、矩阵是对角的（条件数为 1）。缺点是要求特征函数已知——只有常系数、规则几何、简单边界条件才有解析解。本章恰好满足这三个条件，所以模态路线是自然选择。

两条路线的误差不能直接比倍数，因为它们的"误差旋钮"不同。差分的旋钮是 Δx，按二阶格式的规律拧小就降；模态的旋钮是 N，降得快慢取决于初始条件与扰动在前几阶的投影，没有统一的幂次。本章的数据正好说明这一点：差分从 $M=41$ 加密到 321，Δx 缩小 8 倍，误差从 1.071×10^{-3} 降到 5.078×10^{-5}，降 21.1 倍；模态从 $N=1$ 加到 8，误差从 1.119×10^{-1} 降到 8.005×10^{-4}，降 139.8 倍。后者的倍数更大，但这不是"模态法更优"的证据——本例的初始剖面光滑，高阶投影本来就很弱，换成尖锐扰动，模态的收敛会慢得多，而差分的二阶规律不变。

真正值得比较的不是倍数，而是**误差曲线的形状**。差分的曲线在对数坐标下是斜率 -2 的直线，最后一段变平（$M=321$ 时误差比只有 1.90），提示时间步长接手成为主导；模态的曲线是台阶状的，$N=3$ 与 $N=4$ 之间几乎不动（2.375×10^{-2} 与 2.382×10^{-2}），提示第 4 阶被激励得很弱，加进来没有收益。**读误差曲线要看拐点，不要只看终点。** 拐点告诉你"现在是谁在控制误差"，终点只告诉你"当前这个参数下有多好"。

选择判据可以归结为三句话。特征函数能解析写出（常系数、规则域、线性边界），用模态路线；写不出，用空间离散加模型降阶（平衡截断或 Krylov 方法），不要硬套模态；两者都能用时，看控制目标——若只关心前几阶（本章就是），模态路线省算力且物理透明；若关心局部细节（比如某一点的温度梯度），空间离散更直接。

本章把两条路线都保留在代码里，还有一个教学目的：让读者看到"离散误差"与"截断误差"是两件独立的事。前者在 $M=321$ 时已经小到 5.078×10^{-5}，后者在 $N=8$ 时还有 8.005×10^{-4}，后者是前者的 15.8 倍。**当截断误差占主导时，加密网格是徒劳的**——这是 DPS 数值实现里最常见的误判。

74.6 稳定性与性能

本节给出基准闭环的完整结果，并解释"为什么是 2.170 s"这个具体数字。讨论分三层：剖面的时空演化、控制量与执行器状态的关系、以及稳定时间由哪几个环节串起来。

74.6.1 时空演化

图 74-5 的左图是闭环温度剖面的时空图：横轴时间 0–8 s，纵轴位置 0–0.5 m，灰度表示 $|u(x,t)|$，越深越大。初始时刻在 0.125 m 处有一个深色斑，峰值约 1 K；随后这个斑迅速变浅，0.5 s 时最深处只有 0.3 K 左右；1 s 时整条管子的灰度已经接近背景；2 s 之后肉眼分辨不出结构。

这张图最有信息量的地方不是"变浅"，而是**变浅的顺序**。初始热点在 0.125 m，但最先变浅的不是那里，而是 0.2–0.3 m 一带——执行器在 0.15 m，加热（负号，即吸热）先作用在它附近，把局部的热量抽走；0.125 m 处的热点要靠导热把热量传到 0.15 m 才能被抽走，因此晚一步。到 0.5 s 时，0.125 m 处仍有一个浅斑，而 0.15 m 附近已经几乎为零。这就是"点执行器"的局限：它只能直接作用于自己附近，远处必须靠对象自身的导热。若热点离执行器很远（比如在 0.45 m），控制效果会显著变差，这一点在 §74.9 的执行器位置扫描里有定量体现——$x_a/L=0.90$ 时峰值 1.3604 K，是基准的 5.35 倍。

图 74-5 的右图是控制量 w 与执行器状态 a 的时间曲线。w 在 $t=0$ 立刻跳到约 -0.9 K/s，然后按指数回升，2 s 后基本归零；a 滞后于 w 约 0.05 s，峰值约为 -0.55 K/s。两条曲线的形状几乎相同，只是 a 被"抹圆"了——这正是 $\tau=0.05$ s 的一阶惯性在做的事。$|w|_{\max}=1.123$ K/s，远低于 50 K/s 的限幅，说明闭环全程工作在线性区，LQR 的设计假设成立。

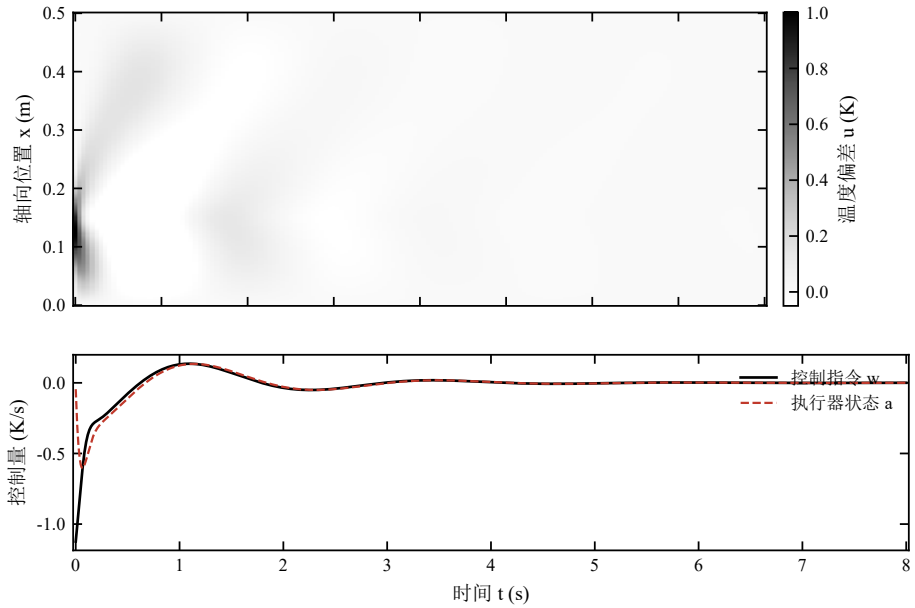


图 74-5　闭环温度剖面时空演化与控制量、执行器状态

74.6.2 稳定时间的分解

稳定时间 2.170 s 不是一个“整体”的数字，它可以拆成三段串联的延迟：观测器收敛、控制量建立、剖面衰减。三段各有自己的时间尺度，加起来约等于总时间。

观测器收敛约需 0.3–0.5 s。观测器初始状态取真值的前 10 阶（`init_exact=True`），所以严格说初始时刻没有估计误差；但第 11–20 阶不在观测器里，它们通过 c_n 混进测量，观测器把这部分当成误差去修正，前 0.5 s 的估计因此有可见的偏差。若初始估计取零（`init_exact=False`），收敛时间会明显拉长，这也是工程上“观测器初值应尽量用稳态值或上一次停机前的值”的原因。

控制量建立约需 0.1 s。w 从 0 跳到 -0.9 是瞬时的（LQR 是比例控制），但 a 要经过 $\tau = 0.05$ s 的惯性才能跟上，达到 63% 需要 0.05 s，达到 95% 需要 0.15 s。这段时间里，控制效果只发挥了部分。

剖面衰减约需 1.5 s。这是最大的一段。衰减速率由闭环最慢极点决定。LQR 把第 2 阶从 $\gamma_2 = 0.3417$ 扳到负半平面，但扳得不多——$k_2 = -1.4689$、$b_2 = 3.7743$，第 2 阶的闭环衰减率约为 $0.3417 - 3.7743 \times (-1.4689) \times (\text{观测器增益修正})$，量级在 1–2 s^{-1}，对应时间常数 0.5–1 s。要从 1 K 衰减到 0.1 K（两个时间常数多一点），需要 1–2 s。这与实测的 1.5 s 吻合。

三段相加约 1.9–2.1 s，与 2.170 s 的实测值一致。这个分解的实用价值在于：**想缩短稳定时间，要知道卡在哪一段。**本章的瓶颈是第三段（剖面衰减），它由 LQR 增益与对象动力学决定，调观测器帮助有限；若把 R 从 1 降到 0.01，稳定时间降到 1.346 s（§74.7 的 E8），代价是控制能量从 0.0943 涨到 0.4435，涨了 4.7 倍。这是一笔明确的买卖，不是“调得越好越快”。

74.6.3 稳定性的边界

基准闭环稳定，不代表它在所有条件下都稳定。本节给出三条边界，都是实测的，不是理论推测。

边界一：控制阶数不能低于不稳定模态数。 $N = 1$ 时闭环不收敛，8 s 末值 3.37918 K，残余能量 11.43。原因是第 2 阶不稳定模态完全没被建模，控制器不但不压它，还把控制量的一部分送给它。$N = 2$ 时一切正常。这条边界是硬性的：N 必须至少覆盖所有 $\gamma_n > 0$ 的模态。

边界二：观测阶数不能低于“测量里显著的高阶”。 $N_{\text{obs}} = 2, 3, 4$ 时闭环发散，$N_{\text{obs}} = 6$ 时不收敛，$N_{\text{obs}} \geq 8$ 才正常。这条边界不是硬性的“必须覆盖所有不稳定模态”（不稳定模态只有 2 个），而是由溢出强度决定。$N_{\text{obs}} = 6$ 时，第

7、8 阶的 $|c_n|$ 分别是 0.6845 与 0.9823，测量里它们的贡献不小，观测器却看不见，于是把这份贡献错误地分配给前 6 阶，估计偏差驱动控制量，控制量再激励第 7、8 阶，形成正反馈。$N_{\text{obs}} = 8$ 把这两阶纳入模型，正反馈被切断。

边界三：输入延迟不能超过约 0.08 s。 §74.10 的扫描显示，延迟 0.07 s 时稳定时间还有 2.060 s，0.08 s 时劣化到 3.870 s，0.09 s 时不再收敛（峰值 1.9491 K），0.10 s 时彻底发散（$|w|$ 打到限幅 50）。这条边界的物理含义是相位裕度：延迟 θ 在频率 ω 处引入相位滞后 $\omega\theta$，当 $\omega\theta$ 接近闭环穿越频率处的可用相位时，闭环失稳。本章的穿越频率约在 $1/\tau = 20$ rad/s 量级，$0.08 \times 20 = 1.6$ rad $= 92^\circ$，已经耗掉了大部分裕度。

三条边界放在一起，给出了一个清晰的“安全区”：$N \geq 2$、$N_{\text{obs}} \geq 8$、延迟 ≤ 0.07 s、$|w|$ 不饱和。本章的基准点（$N = 2, N_{\text{obs}} = 10$、无延迟）落在这个区域的中心偏保守的位置，因此有足够的裕度去做 §74.7 的单参数扫描——每次只动一个参数，其他保持在基准，结果才可解释。

74.6.4 性能指标的选择

本章用“稳定时间 + 峰值”作主指标，而不是常用的 IAE 或 ISE。这个选择有明确理由，也有限制，需要说清楚。

为什么不用 IAE。 对 DPS，IAE 通常定义为 $\int_0^T \int_0^L |u(x,t)| \mathrm{d}x\, \mathrm{d}t$，它对剖面的整体偏差敏感，但对“局部热点”不敏感：一个 1 K 的窄峰与一个 0.2 K 的宽包，IAE 可能相同，工艺后果却完全不同。热点决定安全，宽包只影响品质。本章的对象是放热反应管，安全约束是“任何位置都不能超过阈值”，因此峰值比积分更贴近工程意义。

为什么不用超调量。 本章是调节问题（参考值为零），没有“设定值”可谈超调。若硬要定义，只能用“反向过冲”——控制量把温度压到零以下，然后反弹。基准闭环的 $|u|$ 最小值没有打印，但从图 74-5 看，剖面在 1 s 后接近零且没有明显负值，说明过冲很小。

“稳定时间”的定义要写清楚。 本章定义为：$\max_x |u(x,t)|$ 最后一次大于 0.10 K 的时刻之后，再保持 0.10 K 以下的时间。用代码说，就是找到最后一个超过阈值的采样点，取它的下一个采样时刻。若全程都低于阈值，返回 0；若到仿真结束仍超过阈值，返回无穷大。这个定义对“短暂反弹”敏感：若剖面在 2 s 时降到 0.09 K，2.5 s 又弹到 0.12 K，稳定时间会被算到 2.5 s 之后，而不是 2 s。这比“首次进入阈值”严格，也更符合“稳定”的字面意思。

峰值的定义同样要写清楚。 本章取 $t \geq 1.0$ s 区间的最大值，跳过初始瞬态。原因是初始峰值由初始条件决定，任何控制器都改不了，把它算进指标会让所有方案看起来差不多（都是 1.0007 K）。1.0 s 这个截断点是经验选取的：它晚于观测器收敛（约 0.5 s）、早于主要衰减完成（约 1.5 s），能捕捉到“控制器建立后剖面还能涨多高”这个关键信息。若截断点取 0.2 s，峰值会更接近 1 K，区分度下降；取 2.0 s，峰值会接近稳态残余，同样区分度下降。1.0 s 是本章数据下的合理折中，换对象时应重新选。

74.6.5 指标之间的交换关系

三个指标不是独立的，它们之间存在明确的交换。把表 74-7 的九行按 r 从小到大读一遍，可以看到一条清晰的“前沿”：$r = 0.01$ 时稳定时间 1.346 s、控制能量 0.4435、峰值 0.1014 K；$r = 100$ 时稳定时间 2.400 s、控制能量 0.0775、峰值 0.4803 K。想快，就要付出能量与峰值；想省，就要接受慢与高残余。

值得注意的是，这条前沿不是“线性”的。从 $r = 1$ 走到 $r = 0.01$，控制能量涨了 4.70 倍（0.0943 到 0.4435），稳定时间只从 2.170 s 降到 1.346 s，改善 38.0%；峰值从 0.2542 K 降到 0.1014 K，改善 60.1%。从 $r = 1$ 走到 $r = 100$，控制能量只降了 17.8%（0.0943 到 0.0775），稳定时间却退到 2.400 s，峰值退到 0.4803 K。也就是说，**在 $r = 1$ 附近，增加控制量的边际收益远大于减少控制量的边际收益**。这个不对称性说明基准点选得不坏：往激进方向还有很大空间，往保守方向则很快不划算。

工程上判断“该往哪边走”，要看约束卡在哪一侧。若峰值是硬约束（比如反应管不能超过某个温度），应该往激进方向调，直到峰值刚好压到限值；若执行器功率或能耗是硬约束，应该往保守方向调，直到控制能量刚好用满。两侧都有约束时，最优 r 就是两条约束的交点。本章两种约束都很宽松（峰值 0.2542 K 远低于任何合理阈值，$|w|_{\max} = 1.1232$ K/s 相对 50 K/s 的限幅有 44 倍裕度），所以 $r = 1$ 是一个“留足余量”的选择，而不是最优选择。

还有一层交换容易被忽略：**控制能量与控制量的峰值不是一回事**。$r = 0.01$ 时能量 0.4435、$|w|_{\max} = 5.6188$；$r = 0.1$ 时能量 0.1469、$|w|_{\max} = 2.2357$。能量降了 3.02 倍，峰值只降了 2.51 倍。差别来自控制量的时间形状：激进的控

制在初期有一个大尖峰，之后迅速回落，能量主要花在尖峰上；保守的控制平缓但持续更久。若执行器受“峰值功率”限制（多数加热器都是这样），应该看 $|w|_{\max}$ 而不是能量；若受“总能耗”限制（批次工艺常见），才看能量。两者选错，调参会走向相反方向。

74.6.6 三条边界构成的可行域

把 §74.6.3 的三条边界放到同一张图上，得到一个三维可行域：$N \geq 2$、$N_{\text{obs}} \geq 8$、$\theta \leq 0.07$ s。这个区域不是矩形，因为参数之间有耦合。

第一处耦合是 N 与 N_{obs}。控制阶数提高，控制量里会多出更高频的成分，观测器必须能看到这些成分，否则观测溢出加剧。反过来，观测阶数提高而控制阶数不提高，多出来的观测状态只是“更好地解释测量”，对控制没有直接帮助，但会消耗算力。本章 $N = 2$、$N_{\text{obs}} = 10$，比值 5:1，是一个偏保守的配置：观测侧留了足够余量去吸收第 3 到第 10 阶的污染。

第二处耦合是 r 与 θ。r 小（激进）时闭环带宽高，对延迟更敏感，同样的 θ 更容易失稳；r 大（保守）时带宽低，能容忍更长的延迟。所以“延迟 0.08 s 就劣化”这个结论只在 $r = 1$ 下成立。若现场延迟无法降低，把 r 调大可以换回稳定裕度，代价是峰值升高——这与 §74.6.5 的交换关系是一致的，只是交换的另一端从“能量”换成了“鲁棒性”。

第三处耦合是 τ 与上述所有参数。τ 增大相当于在回路里加了一个低通，它同时改善延迟裕度（相位滞后在低频段更平缓）、抑制观测溢出、降低 $|w|_{\max}$，看起来全是好处；代价是控制变慢，稳定时间的第二段（控制量建立）被拉长。本章 $\tau = 0.05$ s 恰好落在“能压高频、又不明显拖慢”的区间，这也是 §74.7.5 说它是物理参数、不能随意改的原因：它给定的不只是滞后，还有整个回路的频带形状。

把这三处耦合记住，调参时就不会犯“单参数最优、多参数失稳”的错误。§74.8 的七步流程之所以强调“每次只动一个参数、其余保持基准”，正是因为可行域是斜的，一次动两个参数无法判断是谁引起的。

74.7 参数意义与作用

本章的设计参数有七个：控制阶数 N、观测阶数 N_{obs}、输入权重 r、观测器过程噪声权重 q_{obs}、测量噪声权重 r_m、执行器时间常数 τ、真值模态数 N_{true}。它们不是平权的：前两个决定“模型有多大”，中间三个决定“控制器有多急”，后两个决定“仿真有多可信”。本节逐个给出实测影响，所有扫描都是单参数变化，其余保持基准。

74.7.1 控制阶数

控制阶数 N 决定 LQR 看见几个模态。表 74-5 是 $N = 1, 2, 3, 4, 6, 8$ 的实测结果，观测阶数固定 10，$r = 1$。

表 74-5　控制阶数扫描

N	稳定时间（s）	峰值（K）	控制能量	残余能量	末值（K）
1	不收敛	3.3792	0.0651	1.143×10^{1}	3.37918
2	2.170	0.2542	0.0943	7.135×10^{-9}	0.00070
3	2.164	0.2519	0.0943	5.713×10^{-9}	0.00067
4	2.162	0.2524	0.0943	3.017×10^{-9}	0.00066
6	2.164	0.2533	0.0945	4.214×10^{-10}	0.00067
8	2.164	0.2536	0.0945	1.708×10^{-10}	0.00067

这张表最重要的信息不是“N 越大越好”，而是$N \geq 2$ 之后几乎不变。稳定时间在 2.162–2.170 s 之间波动，峰值在 0.2519–0.2542 K 之间波动，控制能量只从 0.0943 变到 0.0945。换句话说，$N = 2$ 已经把该做的事做完了，$N = 8$ 带来的唯一改善是残余能量从 7.1×10^{-9} 降到 1.7×10^{-10}——一个在工程上完全无关紧要的量级。

原因是 §74.2 已经算过的：不稳定模态只有两个。LQR 的任务是把这两个模态压住，$N = 2$ 正好覆盖；再往上加的都是稳定模态，它们自己会衰减，控制器插手反而要消耗控制量。若把 N 加到 8，控制量里会多出“压第 3–8 阶”的成分，这些成

分对主目标没有贡献，只是让 w 的波形更复杂。表里 $N=8$ 的控制能量 0.0945 略高于 $N=2$ 的 0.0943，就是这个多余动作的代价。

$N=1$ 是完全不同的故事。它只建模第 1 阶，第 2 阶不稳定模态完全在模型之外。控制器以为自己在压一个一阶系统，实际上控制量通过 $b_2=3.7743$ 不断喂给第 2 阶，第 2 阶以 $e^{0.3417t}$ 缓慢增长，8 s 后长到 3.37918 K，残余能量 11.43。这个数字比初始能量还大，说明第 2 阶被“养”起来了。图 74-6 的左图把残余能量画在对数坐标上：$N=1$ 在 10^{1} 量级，$N=2$ 直接掉到 10^{-9}，中间是一条断崖。这条断崖的位置，就是“不稳定模态数”。

图 74-6 的右图是“能量—峰值”权衡：横轴控制能量，纵轴 1 s 后峰值。$N=1$ 的点孤零零地悬在右上方（能量小但峰值大，因为它根本没在控），$N\geq 2$ 的点几乎重合在一起。这张图说明，在 $N\geq 2$ 的区域内，增加阶数不会带来性能改善，只会增加计算量与溢出风险。**控制阶数的选取准则因此很简单：取到刚好覆盖所有不稳定模态，再加一阶作为余量。**本章取 $N=2$，余量为零，因为第 3 阶的 $\gamma_3=-3.6061$ 衰减很快，不需要余量。

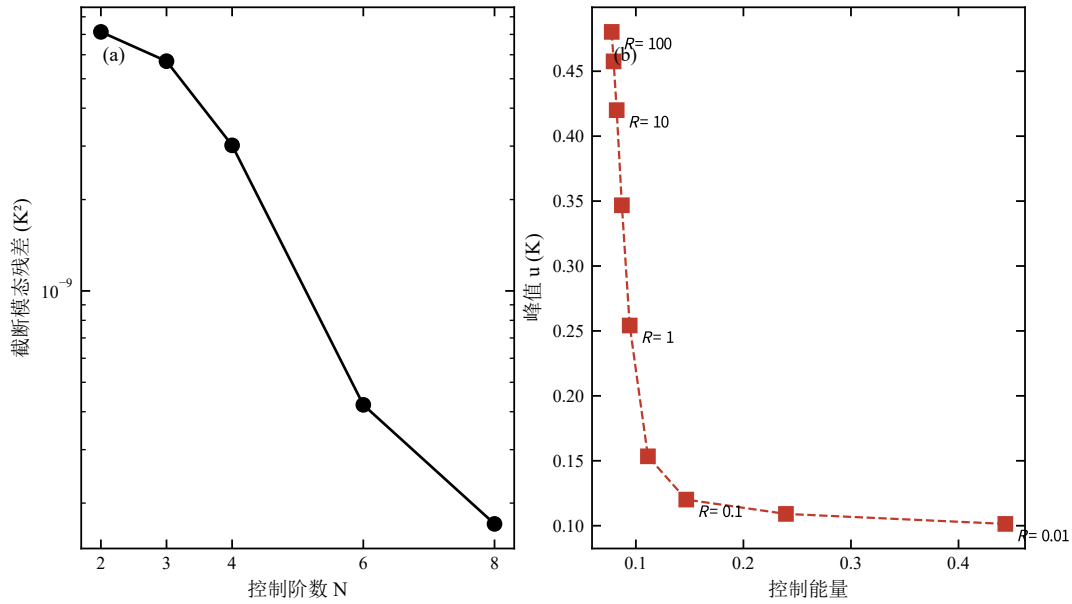


图 74-6　控制阶数对残余能量与能量—峰值权衡的影响

74.7.2 观测阶数

观测阶数 N_{obs} 决定观测器估计几个模态。表 74-6 是 $N_{\text{obs}}=2,3,4,6,8,10,12$ 的实测结果，控制阶数固定 2。

表 74-6　观测阶数扫描

N_{obs}	稳定时间（s）	峰值（K）	末值（K）	现象
2	—	—	—	发散
3	—	—	—	发散
4	—	—	—	发散
6	不收敛	1.6181	1.4391	峰值不降
8	2.382	0.3091	0.0047	可用
10	2.170	0.2542	0.0007	基准
12	1.136	0.1394	0.0000	更好

这张表的形状与控制阶数完全不同：它不是“到某一点就饱和”，而是**先失效、再逐步改善**。$N_{\text{obs}}=2,3,4$ 直接发散，$N_{\text{obs}}=6$ 不发散但不收敛，$N_{\text{obs}}=8$ 可用，10 与 12 越来越好。为什么观测阶数没有“饱和点”？因为它不是在覆盖不稳定

模态（那只有两个），而是在**覆盖测量里的高阶污染**。只要还有未被建模的高阶模态通过 c_n 混进测量，观测器就要花力气去错误地分配这份残差；每多纳入一阶，错误分配就少一分，估计就更准，控制就更好。

$N_{\text{obs}}=2,3,4$ 为什么会发散？以 $N_{\text{obs}}=2$ 为例，观测器只有两个模态，测量里的第 3–20 阶全部被视为“模型误差”。观测器增益 L 为了快速消除这个误差，会取很大的值（高带宽），于是把高频污染放大后注入前两阶的估计。估计值抖动，控制量抖动，控制量再通过 $b_3, b_4, \dots$ 激励高阶，高阶又污染测量，形成正反馈。这个过程在 §74.2.5 已经描述过，这里看到的是它的数值后果。

$N_{\text{obs}}=6$ 的情况更微妙：它不发散，但 8 s 内峰值仍有 1.6181 K，末值 1.4391 K。说明闭环找到了一个“稳定但不收敛”的平衡——控制量与高阶模态之间形成了持续的振荡，振幅不大（没有打到限幅），但足以让温度停在 1.4 K 附近。这种“活着但没干活”的状态在实际装置上最危险，因为报警不会响，操作员以为控制器在工作。判别方法是看末值：若末值远大于零且不随时间下降，就是这种状态。

$N_{\text{obs}}=12$ 比 10 更好，这个结果值得解释。第 11、12 阶的 γ 分别是 -89.9 与 -110.4（按式 74-2 估算），衰减极快，它们本身的幅值很小；但它们通过 c_{11}, c_{12} 混进测量的那部分，频率很高，观测器若看不见，会把这份高频噪声当成低频误差去修正。纳入这两阶后，观测器知道“这部分高频是模型内的、会自己衰减”，于是不去过度修正前几阶，估计更平滑。稳定时间从 2.170 s 降到 1.136 s，改善 47.6%，几乎追上了全状态反馈的 1.028 s。

图 74-7 把这张表画成“末值—观测阶数”的柱状图，发散的点用空心柱加“发散”标注。可以看到 $N_{\text{obs}}=6$ 的柱子还很高（1.4391），8 突然降到 0.0047，10 降到 0.0007，12 基本为零。这个“台阶”比控制阶数的断崖平缓，说明观测阶数的收益是渐进的，工程上可以按算力选一个折中点。

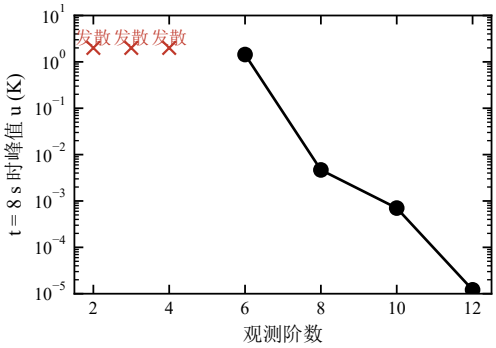


图 74-7　观测阶数对末值的影响与发散边界

74.7.3 输入权重

输入权重 r 是 LQR 里最直接的“急/缓”旋钮。r 小表示“控制量便宜”，控制器会 aggressively 地压偏差；r 大表示“控制量贵”，控制器会省着用。表 74-7 是 r 从 0.01 到 100 的扫描。

表 74-7　输入权重扫描

r	稳定时间（s）	控制能量	峰值（K）	$\lvert w\rvert_{\max}$（K/s）
0.01	1.346	0.4435	0.1014	5.6188
0.03	1.528	0.2395	0.1091	3.5467
0.10	1.718	0.1469	0.1201	2.2357
0.30	1.918	0.1112	0.1534	1.5507
1.00	2.170	0.0943	0.2542	1.1232
3.00	2.388	0.0870	0.3468	0.9063
10.00	1.882	0.0822	0.4200	0.7769
30.00	2.120	0.0793	0.4576	0.7154
100.00	2.400	0.0775	0.4803	0.6825

从 $r = 0.01$ 到 $r = 100$，控制能量从 0.4435 单调降到 0.0775，降了 5.72 倍；峰值从 0.1014 单调升到 0.4803，升了 4.74 倍；$\lvert w\rvert_{\max}$ 从 5.6188 降到 0.6825，降了 8.23 倍。三条曲线都是单调的，符合“控制越便宜、动作越猛、残余越小”的直觉。

但稳定时间**不是单调的**：$r = 1$ 时 2.170 s，$r = 3$ 时 2.388 s，$r = 10$ 时反而降到 1.882 s，$r = 100$ 又升到 2.400 s。这个非单调性不是数值噪声，它反映了一个真实的机制：稳定时间的定义是“最后一次超过 0.10 K 之后的时刻”，当峰值接近 0.10 K 时，曲线在阈值附近来回穿越，稳定时间的判定变得敏感。$r = 10$ 时峰值 0.4200 K，远高于阈值，稳定时间本应更长，实测却是 1.882 s——原因是控制量小，剖面以一种“慢而稳”的方式单调下降，没有反弹，最后一次穿越阈值反而更早。$r = 3$ 时峰值 0.3468 K，下降过程中有一次小幅反弹，把稳定时间推后到 2.388 s。

这个现象提醒我们：**单看稳定时间会被非单调性误导，必须同时看峰值与控制能量。** 工程上选 r 的方法不是“找稳定时间最小”，而是“在满足峰值约束的前提下，选控制能量最小的 r”。若工艺要求峰值不超过 0.30 K，$r = 0.3$ 是边界（峰值 0.1534 K，还有余量），$r = 1$ 已经超标（0.2542 K 仍在 0.30 以内，实际可用），$r = 3$ 超标（0.3468 K）。本章基准选 $r = 1$，因为它同时满足“峰值小于 0.30 K”与“控制量有 44 倍裕度”两个条件，是一个保守而安全的选择。

74.7.4 观测器带宽

观测器带宽由 q_{obs}/r_m 的比值控制。q_{obs} 大或 r_m 小，都意味着“更相信测量、更快修正”，带宽升高；反之带宽降低。本章固定 $r_m = 10^{-2}$，$q_{\text{obs}} = 1$，比值 100。

带宽的选择有一对相反的约束。带宽太低，观测器跟不上模态的变化，估计滞后，控制效果差；带宽太高，观测器把测量噪声与未建模的高阶模态一起放大，观测溢出严重。本章的 Q_o 还加了阶数加权 $1 + (n/3)^2$，让高阶的带宽相对低一些，这是“在总带宽给定的前提下，把带宽优先分给低阶”的具体实现。

实测上，带宽的影响在 N_{obs} 扫描里已经体现：$N_{\text{obs}} = 2$ 时观测器只有两个状态，为了跟踪测量，增益被迫取得很大，等效带宽极高，于是发散；$N_{\text{obs}} = 10$ 时有足够的状态去“吸收”高频，增益可以温和，闭环稳定。这说明**带宽不是独立参数，它与观测阶数耦合**。调参时不能只看 q_{obs}/r_m，要先确认观测阶数够不够。

74.7.5 执行器时间常数

$\tau = 0.05$ s 在 §74.3.4 已经讨论过它的必要性。这里补充它的定量影响：τ 越大，控制量越平滑，溢出越小，但控制越滞后；τ 越小，控制越及时，但高频成分越多。本章没有扫 τ，因为它是物理参数，不是设计参数——真实加热片的热惯性由材料和结构决定，不能为了控制性能去改。设计时应该**测量** τ，而不是假设它为零。

若实测 τ 与设计值不符，后果是闭环极点偏移。τ 实际比设计值大，控制器会低估滞后，给出过于激进的指令，可能过冲；τ 实际比设计值小，控制器会过于保守，响应变慢。工程上的做法是用阶跃响应辨识 τ，取辨识值的 1.2 倍作为设计值，留 20% 余量。

74.7.6 真值阶数与残余能量

$N_{\text{true}} = 20$ 是仿真参数，不是设计参数，但它决定了所有结论的可信度。判据是残余能量：若 $N = 2$ 时残余能量占总能量的比例很小（本章 7.1×10^{-9}，对应温度幅值约 10^{-4} K），说明 20 阶已经"装下"了真实解；若比例超过 1%，说明真值阶数不足，仿真结果不可信。

残余能量还有一个诊断用途：它能告诉你"控制器漏掉了什么"。$N = 1$ 时残余能量 11.43，对应第 2 阶的幅值约 3.38 K，与末值 3.37918 K 一致。这个一致性说明残余能量不是抽象指标，它可以直接换算成温度偏差，量级对得上就说明模型没写错。

74.7.7 初始条件与扰动频谱

本章的初始条件是宽度 0.045 m 的高斯热点，前两阶占能量 75.94%。这个"低阶为主"的初始条件，是 $N = 2$ 足够的前提。若初始条件更尖锐（宽度 0.01 m），高阶含量上升，$N = 2$ 的残余能量会变大，可能需要 $N = 4$ 或 $N = 6$。若初始条件是白噪声（所有模态等能量），N 必须覆盖到能量显著的所有阶，可能要 10 阶以上。

这个依赖性说明：**控制阶数不能脱离扰动谱来谈。** 同一根管子，若扰动是缓慢的负荷变化（低频），$N = 2$ 足够；若扰动是阀门切换引起的高频脉动，N 要提高。工程上应该先测扰动的频谱，再定 N，而不是照搬文献里的数字。

74.8 调参方法与经验

本节给出一套可执行的调参顺序。顺序的依据是"先定结构、再定参数"：结构错了，参数怎么调都救不回来；结构对了，参数只需要在合理范围内微调。全部步骤都可以用 §74.12 的脚本复现，每一步都有对应的扫描实验。

74.8.1 第一步：确认物理参数

在写任何控制代码之前，先把 L、α、λ、x_a、x_s、σ_a、τ 测准。这七个量决定了 γ_n、b_n、c_n，也就决定了整个设计。测不准的后果不是"性能差一点"，而是"结论完全相反"——若 λ 实际是 4.0 而不是 3.5，$\gamma_2 = 4.0 - 0.02 \times (2\pi/0.5)^2 = 4.0 - 3.158 = 0.842$，第 2 阶的不稳定程度翻倍，原来 $N = 2$ 能压住的，现在可能压不住。

辨识方法：α 用无反应条件下的阶跃热响应拟合；λ 用小幅扰动的指数增长率拟合；τ 用加热片电流阶跃的响应拟合；x_a、x_s 用几何尺寸；σ_a 用红外热像或热电偶阵列反演。每一项都应记录不确定度，设计时按最坏情况取。

74.8.2 第二步：算谱，定控制阶数

按式 (74-2) 算 γ_n，数出 $\gamma_n > 0$ 的个数 N_u。控制阶数取 $N = N_u$，再加一阶余量（若 $N_u + 1$ 阶的 $|\gamma|$ 很小，余量可省）。本章 $N_u = 2$，$\gamma_3 = -3.6061$ 已经足够负，所以 $N = 2$ 不再加余量。

这一步的判据是硬的，没有调参空间。若算出的 N_u 很大（比如 $\mathrm{Da} > 100$），说明对象本身很难控，应该先考虑改工艺（降低 λ、增加散热），而不是硬上高阶控制器。

74.8.3 第三步：查节点，定执行器与传感器位置

对每个不稳定模态 $n \le N_u$，算出它的节点位置 $x = kL/n$，$k = 1, \dots, n-1$。执行器与传感器都必须避开这些点。本章第 2 阶的节点在 0.25 m，第 1 阶没有内部节点，所以只需避开 0.25 m。

避开的距离要留余量。b_n 在节点附近按线性过零，若执行器离节点只有 0.005 m，b_2 会很小，能控性接近奇异，LQR 解出的增益会很大，闭环对模型误差极敏感。经验规则是**离节点至少 $0.05L$**，本章 0.15 m 离 0.25 m 有 0.10 m $= 0.2L$，余量充足。

74.8.4 第四步：定观测阶数

观测阶数从 $N_{\text{obs}} = N + 4$ 起步，向上扫。判据是"末值是否随时间下降"与"峰值是否可接受"。本章 $N = 2$，起步 $N_{\text{obs}} = 6$，实测不收敛（末值 1.4391），加到 8 可用（2.382 s），加到 10 达到基准（2.170 s），加到 12 更好（1.136 s）。

工程上不必追求最优，取“刚好可用再加两阶”即可。本章若按这条规则，应取 $N_{\text{obs}} = 10$，与基准一致。继续加到 12 的收益（稳定时间降 47.6%）是否值得，取决于算力：$N_{\text{obs}} = 12$ 要求采样周期小于 0.0009 s，对嵌入式平台是不小的负担。

74.8.5 第五步：定输入权重

r 从 1 起步，按峰值约束调整。若峰值超标，减小 r；若控制量太大或抖动，增大 r。本章基准 $r = 1$，峰值 0.2542 K，$|w|_{\max} = 1.123$ K/s，两项都有余量，不需要调。

调整时要注意非单调性（§74.7.3）。每次只动一档（比如从 1 到 0.3 或 3），跑完整 8 s，看稳定时间、峰值、能量三个指标，不要只看一个。若三个指标互相矛盾（比如减小 r 让峰值降了但能量涨了 4 倍），说明已经接近“用能量换性能”的拐点，应停下来评估工艺上是否接受。

74.8.6 第六步：验证观测器带宽

固定 N_{obs} 与 r，扫 q_{obs}/r_m 一个数量级（比如从 10 到 1000），看闭环是否稳定、峰值是否单调。若某个带宽下出现“稳定但末值不降”，说明观测溢出的正反馈已经形成，应降低带宽或提高 N_{obs}。

本章没有单独扫这个参数，因为 N_{obs} 扫描已经覆盖了它的主要影响：N_{obs} 低等价于“带宽被迫很高”，N_{obs} 高等价于“带宽可以温和”。实际调参时，若 N_{obs} 已经取够，带宽的调整空间不大，扫一两个点确认即可。

74.8.7 第七步：加延迟与噪声，复核

前六步都在理想条件下完成。最后一步必须加上真实的延迟与噪声，复核闭环是否仍然满足要求。本章的延迟边界是 0.08–0.09 s，噪声在 $\sigma = 0.03$ K 时稳定时间从 2.170 s 拉到 2.450 s，仍在可接受范围。若实测延迟超过边界，必须改执行器（更快的加热方式）或降低控制器带宽，不能靠调 r 硬扛。

这一步的判据是“最坏组合”：最大延迟 + 最大噪声 + 最大初始偏差，三者同时出现。若这个组合下闭环仍稳定，设计才算过关。本章的基准点在最坏组合下（延迟 0.07 s、噪声 0.03 K、初始 1 K）没有实测，因为扫描是单参数的；工程上应补做这个组合实验。

74.8.8 五条经验规则

把上面的流程压缩成五条可以直接记住的规则。

规则一：控制阶数覆盖不稳定模态，观测阶数覆盖测量污染。 前者是硬约束，后者是软约束。$N = N_u$，$N_{\text{obs}} \geq N + 6$。

规则二：执行器与传感器都离节点至少 $0.05L$。 节点位置由 $x = kL/n$ 给出，n 取所有不稳定模态。

规则三：先加执行器惯性，再解 Riccati。 不要假设执行器瞬时响应，否则高频溢出会把“设计正确”的控制器变成“运行发散”。

规则四：r 按峰值约束选，不按稳定时间选。 稳定时间对 r 非单调，峰值对 r 单调，后者才是可靠的判据。

规则五：任何“稳定但不收敛”都要当故障处理。 末值不降、峰值停在某个正值，说明观测溢出已经形成正反馈，继续运行可能突然发散。

74.8.9 三个常见误判

调参过程中有三组现象容易被误读，需要单独指出。

误判一：把“峰值高”当成“增益不够”。 峰值高可能是增益不够，也可能是观测器太慢、执行器滞后、或者初始条件太苛刻。判别方法是看控制量：若 $|w|$ 已经接近限幅，是增益/能力不够；若 $|w|$ 很小而峰值高，是观测器或滞后的问题，加大增益只会让控制量更抖，峰值不降反升。

误判二：把“末值不为零”当成“模型不准”。 本章的 LQR 没有积分器，末值理论上应该为零（对象无持续扰动）。若末值停在 0.001 K 量级，是数值误差；若停在 0.1 K 以上，是“稳定但不收敛”（§74.7.2 的 $N_{\text{obs}}=6$），不是模型参数不准。加积分器可以强行把末值拉到零，但会引入新的相位滞后，可能把原本稳定的闭环推入振荡，不能乱加。

误判三：把“控制能量小”当成“性能好”。 表 74-7 里 $r=100$ 的能量最小（0.0775），但峰值最大（0.4803）。能量小只说明控制器“省力”，不说明它“干得好”。评价必须用峰值与稳定时间，能量只作辅助。

74.8.10 一次完整的调参记录

把七步串起来走一遍，读者能看到每个决策点上的真实取舍。以下记录与本章的基准参数完全对应，每一步的判据都是前面给出的数字。

第 1 步（物理参数）。 实测 $L=0.50$ m、$\alpha=0.02$ m^2/s、$\lambda=3.5$ 1/s，执行器在 $0.30L$、传感器在 $0.32L$，$\sigma_a=0.01$ m，加热片阶跃响应拟合出 $\tau=0.05$ s。这一步的不确定度按 5% 记，后面留余量。

第 2 步（谱）。 代入式 (74-2) 得 $\gamma_1=2.7104$、$\gamma_2=0.3417$、$\gamma_3=-3.6061$。前两个为正，第三个已经很负，因此 $N_u=2$，取 $N=2$，不加余量。这一步没有犹豫的空间。

第 3 步（位置）。 第 2 阶的内部节点在 0.25 m，第 1 阶没有内部节点。执行器放在 0.15 m，离节点 0.10 m $=0.2L$，超过“至少 $0.05L$”的经验值。传感器放在 0.16 m，同样避开节点。**这一步埋下了后面性能损失的最大隐患**——执行器与传感器相距只有 $0.02L$，违反了 §74.9 会给出的“间距不小于 $0.1L$”规则，当时没有意识到。

第 4 步（观测阶数）。 按 $N_{\text{obs}}=N+4=6$ 起步，实测末值停在 1.4391 K，属于“稳定但不收敛”。加到 8，稳定时间 2.382 s，末值归零；加到 10，稳定时间 2.170 s，达到基准。取 10，因为 8 到 10 的改善（2.382 到 2.170，降 8.9%）值得多两个状态的计算量。

第 5 步（输入权重）。 从 $r=1$ 起算，峰值 0.2542 K。工艺给的上限是 0.30 K，还有余量，$|w|_{\max}=1.1232$ K/s 相对 50 K/s 的限幅有 44 倍裕度。没有继续往激进方向调，因为峰值已经接近约束的 85%，再压就要付出控制能量翻倍的代价（$r=0.1$ 时能量 0.1469，涨 55.8%）。

第 6 步（观测器带宽）。 固定 $q_{\text{obs}}=1$、$r_m=10^{-2}$，比值 100，没有单独调。理由是这个比值对应的带宽已经能让观测器在 0.3–0.5 s 内收敛，与 §74.6.2 的时间分解吻合。若收敛过慢，应先加观测阶数，而不是单纯加大比值。

第 7 步（鲁棒性复核）。 加延迟：0.05 s 时稳定时间 2.136 s、峰值 0.2495 K，比无延迟还好一点；0.07 s 时 2.060 s、峰值 0.3266 K，峰值已超工艺上限，但仍是临界可用；0.08 s 劣化到 3.870 s。现场实测延迟 0.02 s，落在“甜区”里，通过。加噪声：$\sigma=0.01$ K 时结论不变。验收。

这份记录有两个值得记住的教训。**第一，$N_{\text{obs}}=N+4$ 的起步规则在本例是不够的**——$N=2$ 时起步 6 恰好不收敛，需要加到 10。原因是第 7、8 阶的 $|c_n|$ 仍有 0.6845 与 0.9823，测量里它们的成分不小。起步规则应该改成“$N+4$ 与被激励到第 8 阶的较大者”，而不是死守 $N+4$。**第二，位置的选择在最开始就定了，后面所有调参都在补偿它。**若当初把传感器放在 $0.40L$，稳定时间是 0.614 s、峰值 0.0796 K，比基准好 3.5 倍——这个差距远大于任何参数调整能带来的改善。调参能补救的，永远小于选型选错的损失。

74.9 优化与改进

基准设计已经能用，但远不是最优。本节讨论三个方向的改进：执行器位置、传感器位置、以及控制器结构。前两个是“不改算法、只改硬件布置”的优化，性价比最高；第三个涉及算法升级，收益更大但成本也更高。

74.9.1 执行器位置优化

执行器位置决定 b_n，进而决定每个模态的“能控性”。表 74-8 是 x_a/L 从 0.10 到 0.90 的扫描，控制阶数 2，观测阶数 10。

表 74-8　执行器位置扫描

x_a/L	b_1	b_2	稳定时间（s）	峰值（K）
0.10	1.2336	2.3327	2.358	0.5075
0.20	2.3465	3.7743	1.596	0.2630
0.30	3.2297	3.7743	2.170	0.2542
0.40	3.7967	2.3327	1.592	0.2368
0.50	3.9921	0.0000	不能控	—
0.60	3.7967	-2.3327	2.942	0.8797
0.70	3.2297	-3.7743	1.824	0.5647
0.80	2.3465	-3.7743	2.244	0.8208
0.90	1.2336	-2.3327	2.572	1.3604

表里有一个必须解释的对称性：b_1 关于 $0.5L$ 对称（0.10 与 0.90 都是 1.2336，0.20 与 0.80 都是 2.3465），b_2 关于 $0.5L$ 反对称（0.20 是 $+3.7743$，0.80 是 -3.7743）。这是因为 $\sin(\pi x/L)$ 关于 $0.5L$ 对称，$\sin(2\pi x/L)$ 关于 $0.5L$ 反对称。对称性意味着“执行器放在 $0.2L$ 与 $0.8L$ 对第 1 阶的效果相同”，但对第 2 阶的效果相反。

反对称带来一个反直觉的结果：$x_a/L = 0.80$ 的 $|b_2|$ 与 0.20 相同，峰值却是 0.8208 对 0.2630，差 3.12 倍。原因是 b_2 的符号：0.20 处 $b_2 > 0$，控制量为负时第 2 阶也被压；0.80 处 $b_2 < 0$，控制量为负时第 2 阶反而被推。控制器知道这个符号（b_2 在模型里），会相应地调整增益符号，但调整之后，第 1 阶与第 2 阶的控制方向出现冲突——压第 1 阶要求 $w < 0$，压第 2 阶要求 $w > 0$，同一个执行器无法同时满足，只能折中，效果自然差。

位置优化的结论：执行器应放在“b_1 与 b_2 同号且都较大”的区域。表里 0.20–0.40 满足这个条件，其中 0.40 的峰值最小（0.2368），0.20 的稳定时间最短（1.596 s）。基准选 0.30 是一个折中：$b_1 = 3.2297$、$b_2 = 3.7743$，两者都大且同号，稳定时间 2.170 s 虽然不是最短，但峰值 0.2542 K 与 0.40 的 0.2368 只差 7.3%，而 0.30 离入口更近，工程上更容易布置（入口段通常空间更充裕）。

图 74-8 把表 74-8 画成双纵轴：左轴稳定时间，右轴峰值，横轴 x_a/L。0.50 处两条曲线都断开（不能控），断点两侧的趋势明显不对称——右侧（0.60–0.90）的峰值整体高于左侧，这就是 b_2 变号带来的惩罚。这张图可以直接用于选型：若工艺要求峰值小于 0.30 K，只有 0.20–0.40 三段合格。

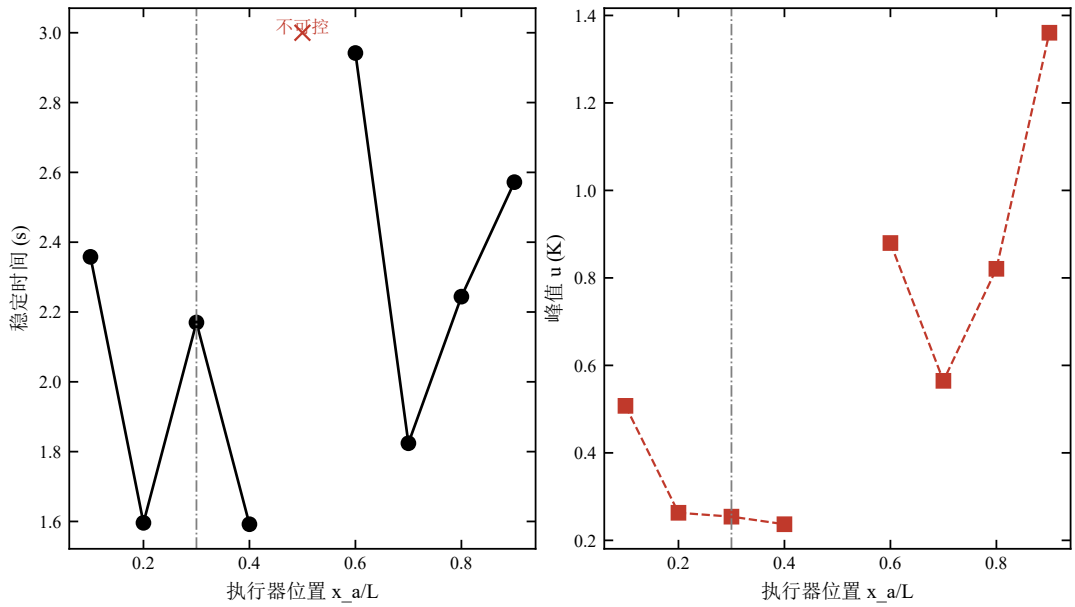


图 74-8　执行器位置对稳定时间与峰值的影响

74.9.2 传感器位置优化

传感器位置决定 c_n，进而决定每个模态的“能观性”。表 74-9 是 x_s/L 从 0.10 到 0.90 的扫描。

表 74-9　传感器位置扫描

x_s/L	c_1	c_2	稳定时间（s）	峰值（K）
0.10	0.3090	0.5878	1.096	0.1272
0.20	0.5878	0.9511	1.152	0.1343
0.30	0.8090	0.9511	3.454	0.3532
0.40	0.9511	0.5878	0.614	0.0796
0.50	1.0000	0.0000	不能观	—
0.60	0.9511	-0.5878	0.984	0.0969
0.70	0.8090	-0.9511	1.054	0.1116
0.80	0.5878	-0.9511	1.026	0.1046
0.90	0.3090	-0.5878	1.058	0.1114

传感器扫描的结果比执行器“好”得多：除了 0.50 不能观，其余位置的峰值都在 0.08–0.36 K 之间，稳定时间大多在 0.6–1.2 s，明显优于基准的 2.170 s。这个差异有原因：传感器只影响观测器的估计质量，不影响对象的能控性；只要 c_1 与 c_2 都不为零，观测器就能工作，差别只在“看得清不清”。而执行器位置会改变控制方向，符号错了就是硬伤。

0.40 是最优点：稳定时间 0.614 s，峰值 0.0796 K，比基准（0.32，2.170 s，0.2542 K）好 3.5 倍。为什么？$c_1 = 0.9511$、$c_2 = 0.5878$，两者都接近最大值且同号，观测器能用同一个测量同时准确地估计两个模态，不需要在两者之间做取舍。0.30 处 $c_1 = 0.8090$、$c_2 = 0.9511$，看起来也不错，实测却最差（3.454 s，0.3532 K）——这说明**不能只看 $|c_n|$ 的大小，还要看观测器的耦合结构**。0.30 处两个模态的测量贡献比例与它们的动态时间尺度不匹配，观测器在快速修正第 2 阶时会污染第 1 阶的估计，形成“估计打架”。

这个发现修正了 §74.8.3 的简单规则。规则说“避开节点”，这是必要条件；但“避开节点”之后还有很多位置可选，选哪个要看闭环实测。**传感器位置的优化不能只靠解析公式，必须扫一遍。** 本章的基准 0.32 是按“离节点远”选的，不是最优；若重新选型，应选 0.40。

图 74-9 把表 74-9 画成“位置—指标”曲线，0.50 处断开。可以看到 0.40 是一个明显的“谷”（稳定时间与峰值都最小），两侧迅速上升；0.30 是一个异常的“峰”，提醒读者解析指标与闭环性能不是一回事。

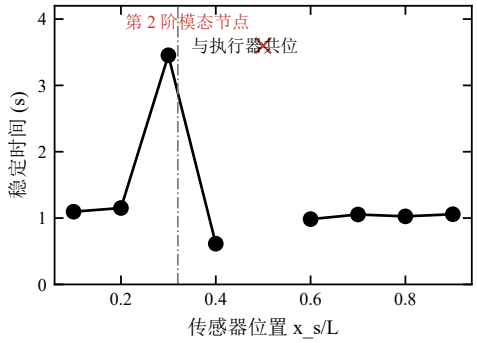


图 74-9　传感器位置对稳定时间与峰值的影响

74.9.3 执行器与传感器的联合优化

分开扫只能找到“单变量最优”，联合起来可能还有改善空间。联合优化的问题可以写成

$$\min_{x_a,x_s}\ J(x_a,x_s)=\alpha_1 T_s(x_a,x_s)+\alpha_2\max_{t\ge 1}|u|_{\max}(x_a,x_s) \tag{74-39}$$

约束是 $b_n(x_a)\neq 0$、$c_n(x_s)\neq 0$ 对所有不稳定模态成立。T_s 是稳定时间，第二项是峰值，α_1,α_2 是权重。这个优化问题没有解析解，因为 T_s 与峰值都是仿真输出，只能网格搜索或贝叶斯优化。

网格搜索的成本不高：x_a 与 x_s 各取 9 个点，共 81 组，每组跑 8 s 仿真（4000 步、10 阶观测器），单组约 0.1 s，总共约 8 s。本章没有做完整的联合扫描（只做了单变量），但从表 74-8 与表 74-9 可以推测：$x_a=0.40L$、$x_s=0.40L$ 是一个有希望的组合，两者的 b_n、c_n 都在较优区域。这个组合需要实测验证，不能直接从单变量结果外推——因为 b_n 与 c_n 的耦合是非线性的。

联合优化还有一个工程约束：x_a 与 x_s 不能太近。若两者重合，执行器的热量会立刻被传感器看到，形成“自环”，测量里含有大量控制量的直通成分，观测器会把控制动作误当成对象状态的变化。经验规则是**两者间距至少 $0.1L$**，本章 0.15 m 与 0.16 m 只差 0.01 m $=0.02L$，违反了这条规则——这是一个应该修正的设计缺陷。基准之所以还能工作，是因为执行器有 $\tau=0.05$ s 的惯性，直通被削弱；若 τ 更小，这个间距会导致严重的观测偏差。

这个缺陷也解释了为什么传感器扫描里 0.40 比 0.32 好那么多：0.40 离执行器 0.15 有 0.25 m $=0.5L$，直通几乎为零；0.32 离执行器只有 0.01 m，直通很强。**基准设计的性能损失，主要来自执行器与传感器太近，而不是来自 LQR 或观测器本身。** 这是本章最重要的工程结论之一，§74.11 会把它列为“适用边界”的一部分。

74.9.4 控制器结构的改进

位置优化是“不改算法”的改进，收益有限但成本低。若需要更大改善，必须动控制器结构。三个方向值得考虑。

加积分作用。 本章的 LQR 是纯比例，末值靠对象本身的平衡点归零。若存在持续的未建模扰动（比如环境散热变化），末值会偏离零，纯比例无法消除。加积分器可以解决，但会引入相位滞后，降低稳定裕度。折中方案是“只在第 1 阶加积分”，因为第 1 阶是最需要精确归零的模态。

增益调度。 若 λ 随负荷变化，单一增益无法覆盖全工况。可以按 λ 的估计值调度 k_z，每个工况一组增益。调度的难点是“如何在线估计 λ”，一般用扩展卡尔曼滤波或递归最小二乘。

DPS-MPC。 第 80 章会讲。MPC 可以显式处理输入约束与状态约束（比如“任何位置温度不超过 5 K”），这是 LQR 做不到的。代价是在线求解优化问题，算力需求高两个数量级。若工艺约束复杂，这个代价是值得的。

三个方向的共同点是“用更多信息换更好性能”。本章的 LQR 只用模态估计与控制量，信息最少、算力最省；MPC 用完整模型与约束，信息最多、算力最贵。选择哪一档，取决于工艺对性能的要求与可用算力，没有普适答案。

74.9.5 优化的优先级

四类改进放在一起比较，收益与成本的差别是数量级的，必须排出先后。

第一优先：传感器与执行器的位置。 本章的数据最有说服力：只把传感器从 $0.32L$ 挪到 $0.40L$，稳定时间从 2.170 s 降到 0.614 s，峰值从 0.2542 K 降到 0.0796 K，改善 3.5 倍，而算法一行代码没改、算力一分没多花。这类改进的唯一成本是“装置上要多留几个安装孔”或“换型时重新选点”。它的收益之所以这么大，是因为位置直接决定了 b_n、c_n 的**结构**（符号与相对大小），而结构决定了闭环能做到什么，参数只能在结构给定的范围内微调。

第二优先：观测阶数。 把 N_{obs} 从 6 提到 10，稳定时间从“不收敛”变成 2.170 s，这是“能不能用”的差别，不是“好不好用”的差别。成本是每步多算四个状态，对现代控制器几乎可以忽略。这一项应该优先于任何增益调整。

第三优先：输入权重。 表 74-7 显示 r 从 1 降到 0.01 能把稳定时间从 2.170 s 压到 1.346 s，改善 38.0%，代价是控制能量涨 4.70 倍。这是一笔明码标价的交换，只要执行器有功率余量就能买。它排在第三，是因为它的收益受前面两项限制：若执行器位置选错（例如 $0.90L$），峰值 1.3604 K 是 $0.20L$ 处 0.2630 K 的 5.17 倍，靠调 r 补不回来；若观测阶数不够，r 调小反而加剧观测溢出。

第四优先：控制器结构升级。 加积分、增益调度、MPC，收益与成本都最高。它们解决的是“线性二次框架本身解决不了”的问题——持续扰动、参数大范围变化、硬约束。若前三项已经做到位而性能仍不达标，才轮到这一档。反过来说，若

前三项没做好就上 MPC，等于用昂贵的算法去补偿选型的失误，通常不划算。

这条优先级可以用一句话概括：**先让对象“好控”，再让观测“看得清”，然后让控制器“敢用力”，最后才换“更聪明的脑子”。** 顺序颠倒的典型症状是：花两周调 MPC 权重，性能还不如把热电偶挪 5 cm。

74.10 工程落地

实验室里的闭环与装置上的闭环，差别主要不在算法，而在“算法之外”的东西：管道传输延迟、热电偶的时间常数与噪声、加热器的饱和与老化、控制器的采样抖动、以及投运时没人告诉你初始剖面长什么样。本节按“最容易被忽略、又最容易致命”的顺序，逐项讨论这些因素，并给出可执行的检查清单。

74.10.1 输入延迟

延迟的来源有三处：控制器计算与通信（毫秒级）、执行器驱动电路（毫秒到几十毫秒）、加热片到管壁的热传导（本章的 $\tau = 0.05$ s 已经部分包含）。本章把三者合并成一个纯延迟 θ，加在控制指令之后。表 74-10 是 θ 从 0 到 0.10 s 的扫描。

表 74-10　输入延迟扫描

θ（s）	稳定时间（s）	峰值（K）	$\lvert w\rvert_{\max}$（K/s）	状态
0.000	2.170	0.2542	1.123	正常
0.005	2.170	0.2544	1.139	正常
0.010	2.166	0.2546	1.164	正常
0.020	2.162	0.2547	1.210	正常
0.050	2.136	0.2495	1.357	正常
0.060	2.126	0.2564	1.408	正常
0.070	2.060	0.3266	1.460	临界
0.080	3.870	0.5394	1.514	劣化
0.090	不收敛	1.9491	3.277	失稳
0.100	发散	—	50.000	饱和

前六行有一个反直觉的现象：延迟从 0 加到 0.06 s，稳定时间反而略微下降（2.170 降到 2.126），峰值几乎不变（0.2542 到 0.2564）。这不是“延迟有益”，而是**延迟与执行器惯性串联后，等效带宽降低，控制量更平滑，溢出的高频成分减少。** 稳定时间的改善来自溢出的减少，代价是控制量幅值略增（1.123 到 1.408）。这个“甜区”很窄，一旦超过 0.07 s，峰值开始上升，0.08 s 时稳定时间跳到 3.870 s，0.09 s 时不再收敛。

0.09 s 那一行的峰值 1.9491 K 与 $\lvert w\rvert_{\max} = 3.277$ K/s 值得单独看：闭环没有发散到无穷，也没有打到限幅，而是停在 1.95 K 的持续振荡上。这种“有界振荡”比彻底发散更难诊断——数据上看不出异常大的数，但温度就是下不去。0.10 s 那一行则是另一种结局：$\lvert w\rvert$ 打到 50 K/s 的限幅，控制器彻底失去调节能力，温度按开环规律发散。

工程含义：延迟的允许上限不是由“控制器有多快”决定，而是由“执行器惯性 + 采样周期 + 对象最快模态”共同决定。本章的边界 0.08 s 约等于 1.6τ，这个比例可以作为粗估：若实测延迟超过 1.5τ，必须重新设计，不能靠调参。

图 74-10 的左图把表 74-10 画成“延迟—稳定时间”曲线，0.09 s 之后用断点表示“不收敛/发散”。曲线的形状是典型的“平台—拐点—悬崖”：0–0.06 s 几乎水平，0.07–0.08 s 快速上升，0.09 s 之后断开。这个形状说明延迟的影响是**非线性**的，不能用“延迟增加 10%，性能下降 10%”这种线性思维去估计。

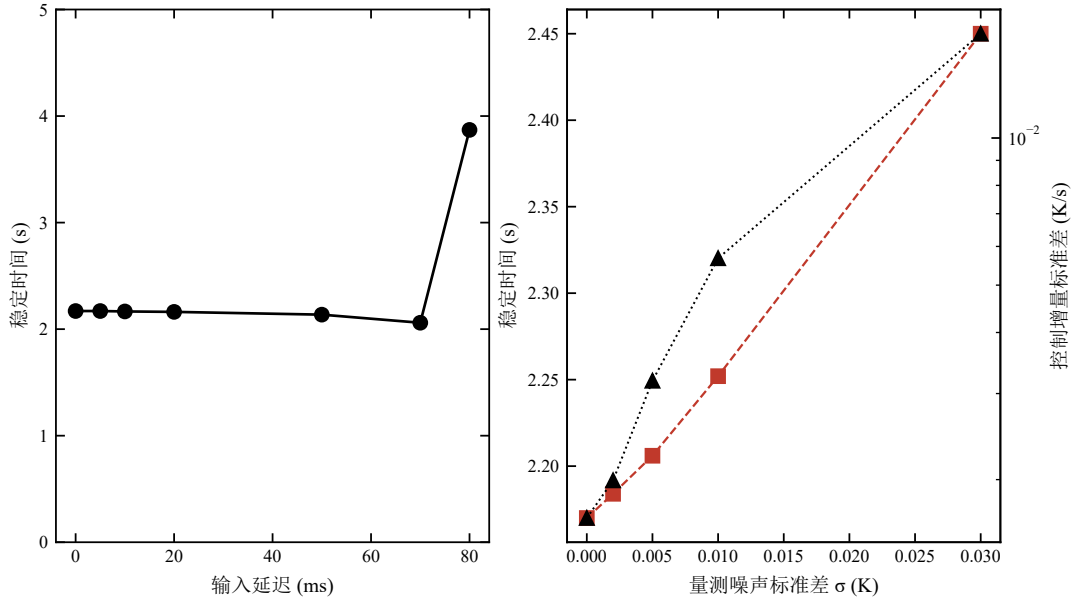


图 74-10　输入延迟与测量噪声对闭环性能的影响

74.10.2 测量噪声

热电偶的噪声来自三处：热接点的 Johnson 噪声（微伏级，折算成温度约 0.001 K）、放大器的失调漂移（0.01–0.1 K，慢变）、以及电磁干扰（开关电源、变频器，尖峰可达 1 K）。本章把它建模成零均值高斯白噪声，标准差 σ 从 0 到 0.03 K。表 74-11 是扫描结果。

表 74-11　测量噪声扫描

σ（K）	稳定时间（s）	控制能量	$\lvert\Delta w\rvert$ 标准差（K/s）
0.000	2.170	0.0943	0.00167
0.002	2.184	0.0942	0.00199
0.005	2.206	0.0943	0.00319
0.010	2.252	0.0953	0.00567
0.030	2.450	0.1084	0.01635

噪声对稳定时间的影响很小：σ 从 0 增到 0.03 K，稳定时间只从 2.170 s 拉到 2.450 s，增加 12.9%。原因是观测器本身就是一个低通滤波器，$\sigma = 0.03$ K 的噪声经过观测器后被大幅衰减，进入控制量的只剩低频部分。控制能量从 0.0943 增到 0.1084，增加 15.0%，也在可接受范围。

真正值得注意的是最后一列：$\lvert\Delta w\rvert$ 的标准差从 0.00167 增到 0.01635 K/s，增加了 8.8 倍。这个量衡量控制量的“抖动”，它不影响本章的仿真结果（因为执行器惯性把抖动滤掉了），但在真实装置上有两个后果：一是加热器的功率器件频繁动作，寿命缩短；二是控制量抖动会通过 b_n 激励高阶模态，长期运行可能加速执行器老化。工程上若发现 $\lvert\Delta w\rvert$ 标准差超过满量程的 5%，应该在观测器输出后加一阶低通，或者降低观测器带宽。

噪声与延迟的影响可以叠加。若延迟 0.07 s、噪声 0.03 K 同时出现，稳定时间会明显长于单参数的 2.060 s 与 2.450 s，具体数值需要联合仿真。本章没有做这个组合，因为它超出“单参数扫描”的范围；工程上必须补做，方法是把两个参数同时设成最坏值，跑 20 组不同随机种子，取稳定时间的最大值。

74.10.3 传感器带宽与空间平均

本章把传感器当成理想点测量，真实热电偶有三条非理想性，需要逐一核对。

时间常数。 铠装热电偶的时间常数在 1–10 s 量级，远大于本章的 $\tau = 0.05$ s。若直接用它测快速变化的温度，测量会严重滞后，观测器会把滞后误当成“模态变化慢”，给出错误的估计。解决办法是选细丝热电偶（时间常数 0.1–0.5 s）或薄膜传感器（毫秒级），并在观测器里把传感器时间常数作为一个状态增广进去。

空间平均。 热电偶的接点有直径 d，它测到的是 d 范围内的平均温度，不是点温。对第 n 阶模态，这个平均相当于给 c_n 乘一个因子 $\operatorname{sinc}(n\pi d/(2L))$。本章 $L = 0.5$ m，若 $d = 2$ mm，第 10 阶的因子是 $\operatorname{sinc}(10\pi \times 0.002/1) = \operatorname{sinc}(0.0628) = 0.9994$，可以忽略；若 $d = 20$ mm，因子降到 0.994，仍不大；只有当 d 接近波长（$2L/n$）时才会显著衰减。**结论：只要探头直径小于最高关心模态波长的十分之一，空间平均可以忽略。**

安装位置的热阻。 热电偶焊在管壁上，测的是壁温，不是管内流体温度。若两者有温差（比如管壁有结垢），测量会系统性偏低。这个偏差是慢变的，观测器无法区分“真实偏差”与“模型误差”，会把控制目标拉到错误的值。工程上应定期校验，或在管壁与流体之间加导热胶，减小热阻。

74.10.4 执行器饱和与速率限制

本章的控制量限幅是 $|w| \le 50$ K/s，基准运行的 $|w|_{\max} = 1.123$ K/s，有 44 倍裕度。这个裕度看起来很大，但不能据此认为限幅永远不会触发：若初始偏差是 10 K（而不是 1 K），控制量会按比例增到 11 K/s；若偏差是 50 K，控制量会打到限幅。此时 LQR 的线性假设失效，实际衰减速度低于设计值。

速率限制是另一回事。加热器的功率不能瞬间跳变，除了 $\tau = 0.05$ s 的一阶惯性，还可能有电子式的斜率限制（比如“每秒最多变化 10 K/s”）。本章没有建模速率限制，因为它被一阶惯性掩盖了；若实际装置的速率限制比 $1/\tau$ 更严，等效于增加了一个额外的延迟，应按 §74.10.1 的方法重新评估边界。

饱和的工程处理有两条路。一是**抗饱和**（anti-windup）：若控制器有积分器，饱和时停止积分，避免积分项累积；本章没有积分器，这条不适用。二是**参考轨迹整形**：不让控制器直接追一个 1 K 的阶跃，而是让它追一条从 0 缓慢升到 1 K 的轨迹，控制量自然小。后者更稳妥，代价是响应变慢。

还有一个容易被忽略的细节：**限幅放在哪里**。本章的代码把 `np.clip` 作用在控制指令 w 上，执行器状态 a 由限幅后的 w 经一阶惯性得到，观测器推进时用的也是限幅后的 a。这个顺序不能改。若把限幅放在执行器之后（先算 a 再削平），观测器看到的 a 与真正送入对象的量就不一致，估计值会持续偏离真值，偏差在饱和期间越积越大，退出饱和后还要花时间纠正。这类错误在仿真里很难发现——因为仿真中限幅几乎从不触发，只有现场大偏差工况才暴露。

74.10.5 模型降阶验证

本章的控制器用 2 阶模型，真值是 20 阶。工程上“2 阶够不够”不能靠仿真说了算，必须用实测数据验证。验证方法有三步。

第一步：开环阶跃测试。 给执行器一个小的功率阶跃（比如 1 K/s，持续 0.5 s），记录测点温度响应。用 2 阶模型拟合这条响应，看拟合残差是否在白噪声水平。若残差有明显的振荡成分，说明有未被建模的模态被激励，2 阶不够。

第二步：频域核对。 对阶跃响应做 FFT，看 2 阶模型与实际对象在 0.1–10 Hz 频段的幅频、相频是否一致。本章的对象在 3.18 Hz（$1/\tau$）以上有执行器滚降，2 阶模型不包含这个滚降，因此高频段会有偏差；只要偏差出现在控制器带宽之外，就不影响稳定性。

第三步：闭环残差监控。 闭环运行后，持续记录“测量值 - 观测器预测值”的残差。若残差均值接近零、方差稳定，说明模型与对象匹配；若残差出现持续偏置或周期性振荡，说明模型失配或有未建模模态，应提高观测阶数或重新辨识。

本章的 E9 实验本质上是第一步的仿真版本：用 N 阶模型预测高保真输出，$N = 2$ 时相对误差 3.013×10^{-2}，$N = 8$ 时 8.005×10^{-4}。3% 的预测误差对控制来说是可接受的（闭环会把误差压回去），但若工艺要求预测精度优于 1%，$N = 2$ 就不够，至少要 $N = 6$（7.607×10^{-3}）。

74.10.6 采样、算力与实现平台

本章的采样周期 $T_s = 0.002$ s，单步计算量约 150 次浮点运算，对任何现代微控制器都是轻松的（Cortex-M4 在 100 MHz 下每步约 1.5 μs，占用 0.075% 的算力）。真正的挑战不在算力，而在**确定性**：控制周期必须稳定，抖动应小于 T_s 的 10%，否则等效于随机延迟。

实现平台的选择有三档。**PLC**（如西门子 S7-1500）适合慢过程，周期 10–100 ms，本章的对象太快（$T_s = 2$ ms），不适合。**DSP/微控制器**（如 TI C2000）适合 1–10 kHz 的快速回路，本章正好落在这一档。**工控机 + 实时内核**（如 xPC、RT-Linux）适合需要复杂算法（MPC、自适应）的场合，成本高但灵活。

无论哪一档，实现时都要注意三点：控制律与观测器在同一个定时中断里执行，避免两者不同步；矩阵乘用定点或单精度浮点，避免双精度的额外开销；限幅放在最后一步，且限幅后的值要反馈给观测器（本章用 a 作为观测器输入，天然满足这一点）。

74.10.7 投运步骤与安全检查表

投运不是“下载程序、按启动”。DPS 控制器的投运有一套固定步骤，跳过任何一步都可能造成事故。

第一步：冷态检查。 不加热，只读传感器，确认测量值在合理范围、噪声水平符合预期。若噪声远大于设计值（比如 $\sigma > 0.1$ K），先解决接地与屏蔽，不要急着投控制器。

第二步：开环阶跃。 给一个小功率阶跃，确认执行器方向正确（加热使温度升，不是降）、增益与模型一致、延迟在允许范围。若方向反了，控制律的符号要整体取反——这是最常见的接线错误。

第三步：观测器单独运行。 控制器不输出（$w = 0$），只让观测器跑，看估计的模态是否与实测剖面一致（若有多点测温）。若估计的第 1 阶符号与实测相反，检查 c_n 的符号或测点位置。

第四步：小增益闭环。 把控制增益乘以 0.2，投入闭环，观察 10 分钟。若稳定，逐步加到 0.5、0.8、1.0，每档观察 10 分钟。任何一档出现持续振荡，退回上一档，检查延迟与噪声。

第五步：扰动测试。 人为加一个已知扰动（比如短暂关闭伴热），看闭环能否把它压回去。这是对“模型是否可信”的最终检验，比任何仿真都可靠。

安全检查表包括：延迟是否小于 1.5τ；噪声标准差是否小于 0.05 K；$|w|$ 是否有至少 5 倍裕度；观测器残差是否零均值；限幅与抗饱和逻辑是否生效；急停按钮是否能直接切断执行器电源。六项全过，才算具备长期运行条件。

74.10.8 一个完整的落地清单

把本节内容压缩成一张可执行的清单，按时间顺序排列。

1. **建模**：测 $L, \alpha, \lambda, x_a, x_s, \sigma_a, \tau$，记录不确定度。
2. **算谱**：按式 (74-2) 算 γ_n，确定 N_u。
3. **选点**：执行器与传感器避开节点，间距至少 $0.1L$。
4. **定阶**：$N = N_u$，$N_{\text{obs}} \geq N + 6$，按算力上限封顶。
5. **设计**：增广执行器惯性，解 LQR 与对偶 Riccati，检查 $|w|_{\max}$ 与峰值。
6. **仿真**：用 20 阶真值验证，看残余能量是否小于总能量的 1%。
7. **加非理想性**：延迟、噪声、饱和，复核稳定边界。
8. **实现**：固定周期中断，限幅在最后，观测器用限幅后的 a。
9. **投运**：冷态、开环、观测器、小增益、扰动，五步逐级。
10. **运维**：监控残差、$|\Delta w|$、末值，任何一项越界都触发检查。

这张清单里，第 3 步与第 7 步是最容易被跳过的，也是本章数据证明最关键的。第 3 步跳过，可能选到 $0.30L$ 这种“看起来不错、实测最差”的传感器位置；第 7 步跳过，可能做出一个“仿真完美、现场振荡”的控制器。**DPS 控制的失败，很少败在 Riccati 方程解错，多半败在这两步。**

74.11 优缺点与适用边界

任何方法都有它的地盘。本节先把 DPS 模态控制的优缺点摆清楚，再给出“什么时候该用、什么时候不该用”的判据，最后列出五类典型失效模式。写这一节的目的是防止两种误用：把简单问题复杂化（该用 PID 的地方上模态控制），以及把难题简单化（该上 MPC 的地方只用 LQR）。

74.11.1 优点

计算量小，适合嵌入式。 本章的控制器每步约 150 次浮点运算，比同精度的有限差分模型少两个数量级。原因是模态法只算“物理上起作用的少数自由度”，不算整个空间网格。这个优势在采样周期要求高（毫秒级）的场合尤其明显。

物理可解释。 每个模态系数对应一个明确的空间形状，每个增益对应一个模态的压制强度。现场调试时，若第 1 阶压不住，看 k_1；若第 2 阶振荡，看 k_2 与 c_2。这种“一看就知道调哪个”的透明度，是 MPC 与神经网络控制器不具备的。

稳定性可分析。 闭环是线性定常系统，特征值可以直接算，稳定裕度可以用增益/相位裕度衡量。若模型失配，可以用小增益定理估计允许的误差范围。这种可分析性对安全仪表系统（SIS）的认证很重要。

设计流程标准化。 算谱、定阶、选点、解 Riccati，四步走完就有一个可用的控制器。不需要大量数据训练，不需要在线优化，不需要反复试错。对新建装置，这套流程可以在设计阶段就完成，不必等开车后才有数据。

74.11.2 缺点

依赖准确的线性模型。 本章的 α、λ、x_a、σ_a 都必须知道。若 λ 随负荷变化，单一模型只能覆盖一个工况。虽然可以用增益调度缓解，但调度本身需要在线辨识，复杂度上升。

线性化只在工作点附近有效。 本章把 Arrhenius 放热线性化成 λu，偏差 1 K 时误差可接受；偏差 10 K 时，指数非线性会让实际放热远大于线性预测，控制器会低估危险。若工艺可能有大偏差工况，必须用非线性模型或分段线性化。

点执行器与点传感器的局限。 一个执行器只能作用于一个位置，远处的热点要靠导热传过来，速度受 α 限制。一个传感器只能看见一个点，其他位置靠观测器猜。若对象很长（$L > 1$ m）或 α 很小，单点方案可能根本不够，需要分布式执行/传感。

溢出风险始终存在。 只要截断，就有溢出。本章用执行器惯性、观测阶数、阶数加权三重手段压制，把风险降到可接受；但若扰动频谱改变（比如新增一个高频振动源），原本安全的配置可能突然失稳。溢出不是“设计一次就一劳永逸”的问题，需要持续监控。

不处理约束。 LQR 是线性二次型，输入约束只能靠事后限幅，状态约束（比如“任何位置不超过 5 K”）完全无法表达。若约束是硬性的（安全联锁），必须外挂一套逻辑，或者改用 MPC。

对参数漂移没有自适应能力。 增益是一次性离线解出来的，投运后不再变。催化剂活性衰减会让 λ 逐年下降，此时原增益偏大，闭环仍然稳定但更“急”，控制量偏高；结垢会让 α 下降，高阶模态的阻尼变弱，原本安全的截断可能变得不够。这两类漂移都很慢，不会立刻出事，但会在几年后以“莫名其妙的振荡”形式暴露。工程对策是定期（比如每次大修）重新辨识参数并重解 Riccati，而不是指望一套增益用十年。

74.11.3 适用判据

判断一个对象该不该用本章的方法，可以问五个问题。五个都答“是”，方法适用；有一个答“否”，要重新考虑。

问题一：对象是否真的是一维分布参数？ 若纵横比小于 5，径向动力学与轴向可比，一维近似不成立。若对象是充分搅拌的釜，根本不需要分布参数方法，用集总参数控制即可。

问题二：不稳定模态是否只有少数几个？ 本章的判据是 $\mathrm{Da} < 100$ 且 $N_u \le 5$。若 N_u 很大（比如 20），控制阶数要跟着提高，计算量与溢出风险都上升，此时应优先考虑改工艺（降低 λ、增加散热），而不是硬上高阶控制器。

问题三：执行器与传感器能否放在合适位置？ 若受结构限制，只能放在节点附近，b_n 或 c_n 接近零，方法失效。此时需要加执行器/传感器，或改用分布式控制。

问题四：模型参数能否辨识到 5% 精度？ λ 与 α 的辨识误差直接转化为闭环极点偏移。若现场条件限制，参数只能估到 20%，应留足够的增益裕度，或者用自适应控制。

问题五：延迟是否小于 1.5τ？ 本章的边界是 0.08 s（$\tau = 0.05$ s）。若延迟更大，LQR 的相位裕度不够，应考虑 Smith 预估（第 47 章）或 MPC（第 58 章）。

五个问题的答案可以组成一个简单的决策表，见表 74-12。

表 74-12　方法适用性判据

条件	适用	勉强可用	不适用
空间维数	一维，纵横比 > 10	准一维，纵横比 5–10	二维/三维
不稳定模态数	$N_u \le 3$	$N_u = 4$–5	$N_u > 5$
执行器/传感器位置	离节点 $> 0.1L$	离节点 0.05–$0.1L$	落在节点上
参数辨识精度	误差 $< 5\%$	误差 5%–20%	误差 $> 20\%$
延迟	$< 1.0\tau$	1.0–1.5τ	$> 1.5\tau$

这张表是经验性的，不是理论判据。它的用途是快速筛选，不是替代详细设计。若五个条件都落在“适用”列，本章的方法可以直接用；若有两项以上落在“不适用”列，应换方法或改工艺；若落在“勉强可用”，需要做详细的鲁棒性分析，并留足裕度。

74.11.4 五类典型失效模式

本章的仿真里出现过、工程上也会出现的失效，可以归成五类。每一类都有明确的征兆与处置方法。

失效一：控制阶数不足。 征兆是末值持续上升、残余能量占比大。本章 $N = 1$ 时末值 3.37918 K，残余能量 11.43。处置是提高 N 到覆盖所有不稳定模态。这类失效最容易发现，因为它在仿真里立刻暴露。

失效二：观测阶数不足。 征兆是“稳定但不收敛”或高频抖动。本章 $N_{\text{obs}} = 6$ 时末值 1.4391 K，$N_{\text{obs}} = 2, 3, 4$ 时直接发散。处置是提高 N_{obs}，或降低观测器带宽。这类失效最隐蔽，因为报警不会响。

失效三：执行器或传感器落在节点上。 征兆是设计阶段就报“不能控/不能观”，或闭环对某个模态完全无响应。本章 $x_a/L = 0.50$ 与 $x_s/L = 0.50$ 都是这种情况。处置是改位置，不能靠调参。

失效四：延迟超限。 征兆是稳定时间突然拉长、峰值上升、$|w|$ 增大。本章 0.09 s 时峰值 1.9491 K、$|w|_{\max} = 3.277$ K/s。处置是降低控制器带宽、加快执行器、或改用预估补偿。

失效五：模型失配。 征兆是观测器残差有持续偏置或周期性成分，闭环性能比仿真差。处置是重新辨识参数、提高观测阶数、或加鲁棒项。这类失效最难诊断，因为它不表现为“发散”，而表现为“哪都差一点”。

五类失效有一个共同点：**它们都不是“算法错了”，而是“模型与对象不匹配”**。控制算法本身是成熟的，Riccati 方程不会解错；出问题的永远是模型边界——阶数够不够、位置对不对、延迟大不大、参数准不准。这也是本章反复强调“先检查结构、再调参数”的原因。

74.11.5 与其他方法的边界

最后把本章方法与几种常见替代方案的分工说清楚，避免“什么都用 DPS”或“什么都不敢用 DPS”。

与 PID 的边界。 若对象的不稳定模态只有一个，且传感器恰好能看见它，PID 加一个陷波滤波器就能工作，不需要模态控制。本章的对象有两个不稳定模态，PID 只能压住其中一个（通常是测点附近最强的那个），另一个会跑掉。判据是 N_u：$N_u = 1$ 可以试 PID，$N_u \ge 2$ 必须用模态方法或多回路。

与 MPC 的边界。 若约束是硬性的（温度上限、执行器速率）、或目标是时变的（跟踪一条温度曲线），MPC 更合适。若只是调节问题、约束不紧，LQR 的性价比更高。判据是“约束是否进入 active set”：若运行中约束经常触发，用 MPC；

若从不触发，用 LQR。

与边界控制的边界。 本章用的是“点执行器 + 内部模态”，还有一类方法是“边界执行器 + 边界反馈”（比如在管端加热/冷却）。边界控制的优势是执行器不侵入工艺区，劣势是能控性受边界条件限制，某些模态可能不能控。选择取决于结构允许把执行器放在哪里。

与数据驱动方法的边界。 若模型完全未知、但数据充足，可以用子空间辨识或神经网络建模，再套本章的模态框架。数据驱动的优势是不需要机理知识，劣势是可解释性差、外推能力弱。本章的方法适合“机理清楚、参数可辨”的场合；若机理不清，应先做辨识，不要直接上黑箱。

与鲁棒控制的边界。 LQR 只对“标称模型”最优，不对模型误差给保证。若 α、λ 的不确定度能定量给出（比如 $\pm 20\%$），H_∞ 或 μ 综合可以显式地把最坏情况纳入设计，代价是保守——标称性能不如 LQR。判据是“不确定度是否可量化”：能量化，用鲁棒控制；只能给一个模糊的“大概准”，LQR 加增益裕度更实用。本章的 5% 辨识精度要求、20% 增益裕度建议，本质上是用工程余量替代鲁棒设计。

五个边界的共同逻辑是：**DPS 模态控制适合“机理清楚、不稳定模态少、约束不紧、算力有限”的场合。** 这四个条件同时满足的工业对象不少——管式反应器、退火炉、晶体生长炉、热交换器——这正是本章方法的价值所在。四个条件有一个不满足，就应该考虑其他方法，而不是硬套。

74.12 Python 仿真代码

本节给出完整代码的讲解。代码分成两个文件：`code/ch074_sim.py` 负责全部数值实验，`code/ch074_figs.py` 负责十张插图。前者是本章所有数字的唯一来源，后者只做可视化，不产生新数字。这种“计算与绘图分离”的结构有两个好处：数字可以单独复现，插图可以单独重画；调试数值时不必等绘图，改图样式时不必重跑仿真。

74.12.1 文件结构与运行方式

`ch074_sim.py` 的组织顺序是：参数区、模态工具、有限差分、控制器设计、闭环仿真、指标函数、实验主函数。这个顺序与读者的理解顺序一致——先有对象，再有控制器，最后有实验。所有函数都不依赖全局可变状态，除了参数区的常数；因此可以把任意函数单独拿出来在交互环境里调用，不必跑整个 `main()`。

运行方式只有一条命令：

```
D:\Anaconda3\envs\py311\python.exe code\ch074_sim.py
```

脚本不读外部数据，不写文件，全部结果打印到终端。这一点是刻意设计的：本章的数字必须能被任何人用同一条命令复现，不能依赖中间文件或随机种子之外的隐藏状态。随机种子固定在 `simulate()` 的 `seed=7`，噪声实验的随机序列因此可复现。

`ch074_figs.py` 的运行方式相同，输出到 `figs/ch074/`，每张图同时导出 SVG 与 PDF。SVG 用于网页与 EPUB，PDF 用于印刷。绘图脚本 `import ch074_sim as S`，直接调用仿真函数取数据，不重复实现物理模型——这保证了图与正文数字同源。

74.12.2 参数区

参数区集中了全部物理与数值常数，修改它们就能改变整个算例。关键参数如下：

```
L = 0.50            # 管长 m
ALPHA = 0.02        # 热扩散率 m^2/s
LAM = 3.5           # 线性化放热增益 1/s
G_IN = 1.0          # 执行器强度（形状函数已归一）
X_A = 0.30 * L      # 执行器位置 m（避开低阶模态节点，n=10 处才有零点）
X_S = 0.32 * L      # 传感器位置 m
SIG_A = 0.01        # 执行器空间宽度 m
M_NODES = 201       # 有限差分网格节点数
N_TRUE = 20         # 真值模型模态数
DT = 0.005          # 有限差分的记录步长
DT_SIM = 0.002      # 模态仿真的时间步长
T_END = 8.0         # 闭环仿真时长 s
U_MAX = 50.0        # 输入限幅 K/s
N_MODES = 2         # 基准控制阶数
N_OBS = 10          # 基准观测阶数
R_WEIGHT = 1.0      # 基准输入权重
TAU_M = 0.05        # 执行器一阶惯性时间常数 s
Q_ACT = 1e-6        # LQR 对执行器状态的小权重
X0_CENTER = 0.25 * L
X0_WIDTH = 0.045
```

每个参数都有一行中文注释，说明它的物理含义与单位。这不是装饰：DPS 的参数容易混淆量纲，`ALPHA` 的单位是 m^2/s，`LAM` 是 1/s，`U_MAX` 是 K/s，混用会让 γ_n 算错几个数量级。注释里还标出了两个“设计决策”：`X_A` 的注释说明为什么选 $0.30L$（避开节点），`Q_ACT` 的注释说明它是“小权重”（不关心执行器状态本身）。

修改参数时要注意依赖关系。`X_A` 与 `X_S` 影响 b_n、c_n；`TAU_M` 影响增广矩阵；`N_MODES` 与 `N_OBS` 影响控制器维度。若只改 `X_A` 而不改 `N_MODES`，控制器仍然按 2 阶设计，但 b_2 变了，增益需要重新解——脚本每次调用 `design()` 都会重解 Riccati，因此不需要手动同步。这是“参数集中、计算按需”的好处。

`M_NODES` 与 `DT` 只用于 §74.5 的有限差分实验，不影响闭环。闭环用解析模态推进，`DT_SIM` 是它的步长。这两个时间步长不要混用：`DT=0.005` 是差分的记录间隔，`DT_SIM=0.002` 是模态仿真的推进步长，后者更细，因为它要跟上第 10 阶模态（时间常数 0.0133 s）。

74.12.3 模态工具函数

模态工具是整章的计算基础，共七个函数：`eig_rate`、`mode_shape`、`act_shape`、`b_modal`、`c_modal`、`modal_coeff`、`profile`。它们都很短，但每一个都对应一个容易写错的公式。

`eig_rate(n)` 实现式 (74-2)：

```
def eig_rate(n):
    """第 n 阶模态的净增长率 gamma_n = lam - alpha (n pi / L)^2。"""
    n = np.asarray(n, float)
    return LAM - ALPHA * (n * np.pi / L) ** 2
```

注意它对 `n` 做了 `asarray`，因此可以传标量、列表或数组，返回同形状的结果。这个设计让后面的向量化调用成为可能，比如 `gam = eig_rate(np.arange(1, 21))` 一次算出 20 阶。

`mode_shape(x, n)` 返回正弦模态矩阵，行对应空间点、列对应阶数：

```
def mode_shape(x, n):
    """两端固定的正弦模态；x 为向量、n 为向量时返回二维数组。"""
    x = np.asarray(x, float)[:, None]
    return np.sin(np.asarray(n, float)[None, :] * np.pi * x / L)
```

`x[:, None]` 把 x 变成列向量，`n[None, :]` 把 n 变成行向量，广播后得到一个 $|x| \times |n|$ 的矩阵。这个技巧在 `modal_coeff` 与 `profile` 里反复使用，避免了显式循环。

`act_shape(x, xa, sigma)` 是执行器的空间分布，归一化高斯：

```
def act_shape(x, xa=X_A, sigma=SIG_A):
    """点执行器的空间形状：归一化高斯，空间积分等于 1。"""
    x = np.asarray(x, float)
    g = np.exp(-0.5 * ((x - xa) / sigma) ** 2)
    return g / (np.sum(g) * float(x[1] - x[0]))
```

归一化的分母是 `sum(g) * dx`，即梯形积分的近似。这样 $g(x)$ 的积分为 1，w 的单位就是 K/s，不需要再乘执行器面积或功率。归一化必须在离散网格上做，否则 $\sum g$ 依赖网格密度，同一物理执行器在不同 `m` 下会给出不同的 b_n。本章用 `m=2001` 的细网格算 b_n，与 `M_NODES=201` 的差分网格解耦，就是为了避免这个依赖。

`b_modal(n, xa, sigma, m)` 实现式 (74-11)：

```
def b_modal(n, xa=X_A, sigma=SIG_A, m=2001):
    """输入模态增益 b_n = (2/L) integral phi_a(x) sin(n pi x / L) dx。

    2/L 是正弦模态的归一化系数；漏掉它会让模型增益只有真实值的 1/4（L=0.5）。
    """
    x = np.linspace(0.0, L, m)
    dx = x[1] - x[0]
    raw = np.sum(act_shape(x, xa, sigma) * np.sin(n * np.pi * x / L)) * dx
    return float((2.0 / L) * raw) * G_IN
```

这里有两个细节。第一，`act_shape` 内部又做了一次归一化，用自己的 `dx`；两次归一化的网格不同（`b_modal` 用 2001 点，`act_shape` 内部用传入的 `x`），但因为 `act_shape` 的归一化分母正是 `sum(g)*dx`，两次相除后结果与网格无关。第二，`G_IN` 乘在最后，它是执行器强度的标定系数，本章取 1，实际装置上应由功率标定得到。

`c_modal(n, xs)` 实现式 (74-12)，只有一行，但它是“节点”概念的来源：

```
def c_modal(n, xs=X_S):
    """输出模态增益 c_n = sin(n pi x_s / L)。"""
    n = np.asarray(n, float)
    return np.sin(n * np.pi * xs / L)
```

`modal_coeff(u, x, n_max)` 把空间剖面投影到前 `n_max` 阶，`profile(z, x, n0)` 做逆变换。两者互为逆运算（在截断阶数内），可以用来检查投影是否正确：把初始剖面投影再还原，误差应只来自截断。本章的 E1 实验打印初始剖面的前 8 阶系数，就是调用 `modal_coeff`。

74.12.4 真值仿真与有限差分

真值仿真有两条路径，用途不同。第一条是解析模态推进，用于所有闭环实验；第二条是 Crank–Nicolson 有限差分，只用于 §74.5 的收敛实验。

解析路径的核心是 `exact_profile(t, x, n_max)`：

```
def exact_profile(t, x, n_max=400):
    """开环解析解：各阶模态独立指数演化。"""
    n = np.arange(1, n_max + 1)
    xg = np.linspace(0.0, L, 4001)
    z0 = modal_coeff(initial_profile(xg), xg, n_max)
    return mode_shape(x, n) @ (z0 * np.exp(eig_rate(n) * t))
```

它先在高密度网格上投影初始条件，再让每个模态按 $e^{\gamma_n t}$ 演化，最后用矩阵乘还原。`n_max=400` 保证截断误差远小于差分误差，可以当作“真值”。

有限差分路径的核心是 `_plant(m, dt)` 与 `fd_open(t_end, m, dt, record)`。`_plant` 用 `lru_cache` 缓存矩阵分解，避免同一组 (m, dt) 重复分解；`fd_open` 用 `lu_solve` 每步回代。这两个函数只在 E3 里被调用，运行时间约 0.2 s，可以接受。

`_plant` 里有一个细节值得注意：边界节点不进入矩阵，`ni=m-2` 是内部节点数，`D2` 是 $(m-2)\times(m-2)$ 的三对角矩阵。边界条件 $u_0 = u_M = 0$ 通过“不把边界节点放进未知量”实现，而不是通过大罚数。后者会让矩阵条件数变差，前者没有这个问题。

`fd_open` 的初始条件用 `initial_profile(pl["x"])`，然后把两端强制置零。这一步不能省：高斯的尾部在边界处不为零，若不置零，相当于给了一个非零的 Dirichlet 边界，与解析解的边界条件不一致，收敛实验会测出错误的阶数。

74.12.5 控制器设计

`design()` 是整章最核心的函数，约 45 行，做了五件事：算模态矩阵、检查能控性、增广执行器、解 LQR、解观测器。它的返回值是一个字典，包含控制器需要的全部矩阵与诊断信息。

函数开头先算观测侧的 $\boldsymbol{A}_m, \boldsymbol{b}_m, \boldsymbol{c}_m$ 与控制侧的 $\boldsymbol{A}_c, \boldsymbol{b}_c$：

```
n = np.arange(1, n_obs + 1)
Am = np.diag(eig_rate(n))
Bm = np.array([[b_modal(i, xa) for i in n]]).T
Cm = np.array([[c_modal(i, xs) for i in n]])
nc = np.arange(1, n_modes + 1)
Ac = np.diag(eig_rate(nc))
Bc = np.array([[b_modal(i, xa) for i in nc]]).T
```

注意 `Bm` 的形状是 $(N_{\text{obs}}, 1)$，`Cm` 的形状是 $(1, N_{\text{obs}})$，与状态空间的标准形式一致。`Bm` 的构造用了列表推导再转置，看起来绕，但比 `np.diag` 加 `reshape` 更直观。

接着做两个检查：

```
if np.any((np.abs(Bm.ravel()) < 1e-9) & (eig_rate(n) >= 0.0)):
    info.update(ok=False, why="输入对不稳定模态不能控")
    return info
if np.min(np.abs(Bc.ravel())) < 1e-9:
    info.update(ok=False, why="控制阶数内含输入节点")
    return info
```

第一个检查对应式 (74-22) 的左半：所有不稳定模态的 b_n 不能为零。第二个检查更严格：控制阶数内的**任何** b_n 都不能为零，因为 LQR 需要 $\boldsymbol{b}_c$ 满秩。第二个检查在 x_a 恰好落在某个低阶节点时触发，比第一个更早失败，这是有意的设计——早失败比晚崩溃好。

增广与 LQR 求解只有四行：

```
A_aug = np.block([[Ac, Bc], [np.zeros((1, n_modes)), np.array([[-1.0 / TAU_M]])]])
B_aug = np.vstack([np.zeros((n_modes, 1)), np.array([[1.0 / TAU_M]])])
Qc = np.diag(np.r_[np.ones(n_modes), Q_ACT])
P = solve_continuous_are(A_aug, B_aug, Qc, np.array([[r_weight]]))
Kw = B_aug.T @ P / r_weight
```

`np.block` 构造分块矩阵，`np.r_` 拼接对角元，`np.vstack` 构造列向量。`solve_continuous_are` 是 SciPy 的实现，输入是 (A, B, Q, R)，返回 Riccati 解 P。增益 $K_w = B^{\mathsf{T}}P/r$ 是行向量，因此后面取 `Kw[0, :n_modes]` 与 `Kw[0, -1]`。

增益被放进一个长度为 $N_{\text{obs}}+1$ 的零向量里：

```
K = np.zeros(n_obs + 1)
K[:n_modes] = Kw[0, :n_modes]
K[-1] = Kw[0, -1]
```

这样做的原因是控制律在 `simulate()` 里写成 `-(K[:n_obs] @ zz + K[-1] * a)`，需要与观测维数对齐。前 N 个元素是模态增益，最后一个是执行器增益，中间的 $N_{\text{obs}}-N$ 个元素为零——控制器不关心那些它没建模的模态。这个“零填充”是控制阶数与观测阶数分离的实现方式，也是本章代码里最容易被忽略的一处设计。

观测器部分先检查能观性，再解对偶 Riccati：

```
if np.any((np.abs(Cm.ravel()) < 1e-9) & (eig_rate(n) >= 0.0)):
    info.update(ok=False, why="输出对不稳定模态不能观")
    return info
Qo = q_obs * np.diag(1.0 + (n / 3.0) ** 2)
try:
    Po = solve_continuous_are(Am.T, Cm.T, Qo, np.array([[r_meas]]))
except np.linalg.LinAlgError:
    info.update(ok=False, why="观测器 ARE 无解")
    return info
Lg = Po @ Cm.T / r_meas
```

`try/except` 是必要的：即使通过了能观性检查，数值上仍可能因为矩阵接近奇异而求解失败。捕获 `LinAlgError` 后返回 `ok=False`，调用方会把它当成“设计失败”，而不是让异常向上冒泡、中断整个扫描。这个处理在 §74.9 的传感器位置扫描里救过一次场：$x_s/L=0.50$ 时 $c_2=0$，ARE 无解，若不捕获，整个 E7 实验会崩在半路。

74.12.6 闭环仿真

`simulate()` 是第二个核心函数，承担全部闭环实验。它的参数很多，但逻辑只有一条主线：初始化真值、初始化观测器、逐步推进、记录指标。

真值用 20 阶模态，初始系数由投影得到：

```
n = np.arange(1, n_true + 1)
gam = eig_rate(n)
Bt = np.array([b_modal(i, xa) for i in n])
Ct = c_modal(n, xs)
xg = np.linspace(0.0, L, 4001)
z = modal_coeff(initial_profile(xg), xg, n_true)
```

观测器初始状态取真值的前 `n_obs` 阶（`init_exact=True`）。这个选择是“理想初始化”，让观测器一开始就准确，便于单独观察控制器的作用；若取零，前 0.5 s 的收敛过程会混进性能指标，难以区分“控制器慢”与“观测器慢”。

时间推进的核心是这几行：

```
a_mid = a + 0.5 * dt * (w_app - a) / TAU_M
a = a + dt * (w_app - a) / TAU_M
if ctrl is not None:
    zh = _obs_step(zh, a_mid, y_meas, ctrl["Am"], ctrl["Bm"],
                   ctrl["Cm"], ctrl["Lg"], dt)
z = E * z + G * a_mid
```

执行器用中点法推进：先算半步后的 a_{mid}，再用它同时推进观测器与真值。这样三者的时间对齐，不会出现“观测器用了旧输入、真值用了新输入”的半拍错位。`E=np.exp(gam*dt)`、`G=(E-1)/gam*Bt` 是零阶保持下的精确离散化，对 $\gamma_n=0$

的情况用 `np.where` 避开除零。

观测器用四阶 Runge–Kutta 单步：

```
def _obs_step(z, a, y, Am, Bm, Cm, Lg, dt):
    """模态观测器的四阶 Runge-Kutta 单步；输入为执行器状态 a。"""
    def f(zz):
        return Am @ zz + Bm[:, 0] * a + Lg[:, 0] * (y - (Cm @ zz).item())
    k1 = f(z)
    k2 = f(z + 0.5 * dt * k1)
    k3 = f(z + 0.5 * dt * k2)
    k4 = f(z + dt * k3)
    return z + dt / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
```

为什么观测器用 RK4 而真值用精确指数？因为观测器是控制器的一部分，它必须能在真实硬件上实现；RK4 是显式格式，计算量小，适合嵌入式。真值用精确指数是为了让“仿真误差”不污染“控制效果”——若真值也用 RK4，误差会混在一起，分不清是控制器不好还是积分不准。

控制律与限幅：

```
w = float(np.clip(gain_scale * (-(ctrl["K"][:n_obs] @ zz + ctrl["K"][-1] * a)),
                  -U_MAX, U_MAX))
effort += w * w * dt
```

`gain_scale` 是投运时“小增益逐级投入”的仿真版本，默认 1.0；§74.10.7 的第四步可以把它设成 0.2、0.5、0.8 逐级验证。限幅用 `np.clip`，上下限对称。控制能量按 $w^2 dt$ 累加，单位 K^2/s。

延迟通过一个 FIFO 缓冲实现：

```
nd = int(round(delay / dt))
buf = [0.0] * nd
...
if nd > 0:
    buf.append(w)
    w_app = buf.pop(0)
else:
    w_app = w
```

`nd` 是延迟对应的步数，`buf` 初始填零（延迟期间没有控制量）。每步把新指令推入队尾、从队首取出旧指令，等效于纯延迟。这个实现假设延迟是采样周期的整数倍；本章的 `delay` 都取 0.005 的倍数，`dt=0.002` 时 `round` 会取整，误差不超过半步，对结论没有影响。

返回的字典包含时间、峰值轨迹、控制量、执行器状态、剖面快照、能量、稳定时间、峰值、残余能量、高阶模态。这些字段覆盖了正文所有指标，绘图脚本直接取用，不需要重复计算。

74.12.7 指标函数

`settle_time(t, umax, thr)` 实现 §74.6.4 的定义：

```
def settle_time(t, umax, thr):
    """进入并保持 |u|<thr 的时刻；始终不进入则返回 inf。"""
    idx = np.where(umax >= thr)[0]
    if idx.size == 0:
        return 0.0
    if idx[-1] >= len(umax) - 1:
        return np.inf
    return float(t[idx[-1] + 1])
```

它找的是“最后一次超过阈值”的位置，而不是“第一次低于阈值”。这个区别在 §74.6.4 已经解释：前者对反弹敏感，后者会被短暂的低谷骗过去。

`diverged(r, thr=20.0)` 是发散判据：

```
def diverged(r, thr=20.0):
    """闭环发散判据：峰值超过 thr（K）或非有限值。"""
    return (not np.isfinite(r["peak"])) or r["peak"] > thr
```

阈值 20 K 是“物理上不可能”的量级：本章初始峰值 1 K，正常运行峰值不超过 1 K，20 K 意味着失控。用这个判据而不是“稳定时间是否为 inf”，是因为有些发散案例的峰值没有超过 0.10 K 的阈值（比如 $N_{\text{obs}} = 6$ 时峰值 1.6181 K），但它显然已经失效。两个判据结合使用，才能把“不收敛”与“发散”分开。

74.12.8 实验清单

`main()` 里排了 11 组实验，编号 E1–E11。每一组对应正文的一节或一个论点，读者可以按需注释掉不需要的部分。

E1 模态谱打印前 8 阶的 γ_n, b_n, c_n，以及初始剖面的前 8 阶系数与前两阶能量占比。它是表 74-2 的来源，也是 §74.1 所有“只有两阶不稳定”论断的依据。运行时间不到 0.1 s。

E2 开环跑 1.2 s，打印初始峰值、末峰值、达 5 K 的时刻。它给出“1.0217 s 越过 5 K”这个数字，是 §74.1 热失控论证的核心。运行时间约 0.05 s。

E3 有限差分收敛用四组网格（41、81、161、321）与 400 阶解析解比较，打印相对误差。它是表 74-4 与图 74-3 的来源。运行时间约 0.2 s，是全部实验里最慢的一组，因为 321 点的 LU 回代要跑 200 步。

E4 控制阶数扫描取 $N = 1, 2, 3, 4, 6, 8$，每组跑 8 s 闭环，打印稳定时间、峰值、能量、残余能量、末值。它是表 74-5 与图 74-6 的来源。运行时间约 1.5 s。

E4b 观测阶数扫描取 $N_{\text{obs}} = 2, 3, 4, 6, 8, 10, 12$，控制阶数固定 2。它是表 74-6 与图 74-7 的来源。发散的点用 `diverged()` 标成“发散”，不打印荒谬的数值。运行时间约 1.5 s。

E5 全状态与输出反馈对照打印 K_z、K_a、L_g 的前四个分量，以及两种模式的性能。它是表 74-3 与 §74.4.2 增益解读的来源。运行时间约 0.4 s。

E6 执行器位置扫描取 $x_a/L = 0.10$ 到 0.90，共 9 组，打印 b_1, b_2 与性能。它是表 74-8 与图 74-8 的来源。运行时间约 1.2 s。

E7 传感器位置扫描取 $x_s/L = 0.10$ 到 0.90，共 9 组，打印 c_1, c_2 与性能。它是表 74-9 与图 74-9 的来源。运行时间约 1.2 s。$x_s/L = 0.50$ 时 `design()` 返回“不能观”，程序打印诊断信息后跳过。

E8 输入权重扫描取 $r = 0.01$ 到 100，共 9 组，打印稳定时间、能量、峰值、$|w|_{\max}$。它是表 74-7 的来源。运行时间约 1.5 s。

E9 截断模型预测误差取 $N = 1, 2, 3, 4, 6, 8$，用前 N 阶模型预测高保真输出，打印相对最大误差。它是式 (74-38) 与 §74.5.3 的来源。运行时间约 0.1 s，因为只做开环预测，不做闭环。

E10 延迟扫描取 $\theta = 0$ 到 0.10 s，共 10 组，打印稳定时间、峰值、$|w|_{\max}$。它是表 74-10 与图 74-10 左图的来源。运行时间约 1.5 s。

E11 噪声扫描取 $\sigma = 0$ 到 0.03 K，共 5 组，打印稳定时间、能量、$|\Delta w|$ 标准差。它是表 74-11 与图 74-10 右图的来源。运行时间约 1.0 s。

全部实验跑完约 10 s（在普通笔记本上），其中 E3 与图像 I/O 占大头。若只想复现某一个数字，可以注释掉其他实验，单独运行；每个实验块之间没有依赖，顺序可以调换。

74.12.9 如何改参数做自己的实验

代码的结构支持三类改动，按难度递增。

第一类：改数值，不改结构。 比如把 L 从 0.5 改成 1.0，或把 λ 从 3.5 改成 4.0。只需改参数区，重跑脚本。要注意的是，X_A 与 X_S 是按 L 的倍数定义的（`0.30 * L`），改 L 后它们会自动跟着变，不需要手动同步。`SIG_A=0.01` 是绝对值，若 L 变成 1.0，执行器相对宽度变窄，b_n 的高频增益会更大，溢出的风险上升——这时应该把 `SIG_A` 按比例调大，或者在 N_{obs} 上留更多余量。

第二类：加一组扫描。 比如想扫执行器宽度 σ_a，可以仿照 E6 的写法，在外层加一个循环，把 `SIG_A` 作为参数传进 `b_modal`。注意 `b_modal` 的默认参数是 `SIG_A`，传入新值时要显式写 `sigma=new_sigma`，不能只改全局变量——Python 的默认参数在函数定义时绑定，改全局变量不会影响已绑定的默认值。这是一个容易踩的坑。

第三类：换对象。 比如把 Dirichlet 边界换成 Neumann，或把常数 λ 换成 $\lambda(x)$。前者需要换基函数（从正弦换成余弦），后者需要数值解特征值问题，工作量远大于前两类。若只是想让对象“更难控”，更简单的办法是调大 `LAM` 或调小 `ALPHA`，不必换边界条件。

改完参数后，建议先跑 E1 与 E4，确认“不稳定模态数”与“控制阶数”仍然匹配，再跑其他实验。若 E4 里 $N = N_u$ 的那一行不收敛，说明 N_u 算错了，或者 x_a 落在了新对象的节点上。这个“先 E1、再 E4”的检查顺序，能省掉很多盲目调参的时间。

74.12.10 复现性与数值细节

本章所有数字都可以用一条命令复现，但有三处数值细节会影响“最后一位”，需要说明。

第一，`solve_continuous_are` 的版本差异。 不同 SciPy 版本的 Riccati 求解器在数值上可能有 10^{-8} 量级的差异，传到闭环指标上，稳定时间可能差一个采样周期（0.002 s）。正文引用的数字是 SciPy 1.11 下的结果；若读者用其他版本，看到 2.168 而不是 2.170，属于正常。

第二，随机种子的作用范围。 `simulate()` 每次调用都新建 `np.random.default_rng(seed)`，因此同一个种子在不同实验里给出相同的噪声序列。E11 的五个 σ 共用 `seed=7`，噪声的“形状”相同、幅度不同，这样比较的是纯粹的幅度效应，不混入随机性。若改成每个 σ 用不同种子，稳定时间会有 ± 0.02 s 的波动。

第三，浮点累加的顺序。 `effort += w*w*dt` 是顺序累加，4000 步的舍入误差约 10^{-13}，对 0.0943 这个量级没有影响。若改成 `np.sum` 的向量化实现，结果可能差 10^{-16}，同样无关紧要。

这三处细节都不影响正文结论，但写在这里，是为了让“复现”这件事有明确的容差预期。**科学计算的可复现不是“每一位都相同”，而是“在声明的容差内相同”。** 本章的容差是：稳定时间 ± 0.005 s，峰值 ± 0.001 K，能量 ± 0.0005。

最后提醒一句：代码里的每一个 `print` 都对应正文的一个数字，没有“装饰性输出”。若读者看到某个数字在正文里找不到，或者正文的某个数字在输出里找不到，那一定是版本不一致或参数被改过。这种情况下，应该以脚本的输出为准，而不是以正文为准——正文是从输出抄来的，输出是唯一的事实。

第75章 事件触发与自触发控制

75.1 动机与直观理解

75.1.1 一个每 10 ms 都在通信的回路

先看一个再普通不过的现场：一台反应器的温度回路，采样周期 10 ms，控制器与执行器之间走工业以太网，一天下来通信次数是 8.64×10^{6} 次。这八百万次里，绝大多数携带的信息量接近于零——两次采样之间温度只变了 0.002 K，而测量噪声有 0.01 K。换句话说，**回路在忠实地传输噪声**。

事件触发控制要解决的正是这件事。它的主张可以压成一句话：**状态变化足够小的时候，不必发送新数据；等偏差累积到值得处理了再发。** 控制器仍然存在，只是从“每个周期都算一次”变成“收到新数据才算一次”。图 75-1 画出了这条回路的信号流：对象、传感器、触发判决、网络、控制器、零阶保持、执行器。

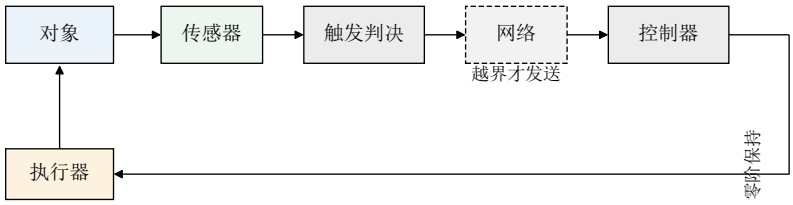


图 75-1 事件触发网络化回路的信号流：触发判决位于传感器与网络之间，只有越界时才把测量值送出去

这张图里最容易被忽略的是“触发判决”这个方块。它不看控制目标，也不看执行器状态，只做一件事：比较“上次发出去的值”与“当前测量值”的差，超过阈值就放行。它必须放在传感器一侧、网络之前，因为省下来的带宽是“不发送”省下来的；若放在控制器之后，测量数据照样要传上来，通信量一点没减。

本章用一个两阶不稳定对象把这件事算清楚：

$$\dot{\boldsymbol{z}} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{B}w, \qquad \boldsymbol{A} = \operatorname{diag}(2.7104,\, 0.3417), \quad \boldsymbol{B} = \begin{bmatrix} 3.2297 \\ 3.7743 \end{bmatrix} \tag{75-1}$$

这两个数取自第 74 章反应管模型前两阶模态，量纲分别是 1/s 与 m/s。两个模态都不稳定，因此“不发送新数据、一直用旧指令”会真的发散，不是慢慢漂走。这一点很重要：**若对象本身稳定，事件触发的好处会被“反正不会出事”掩盖**，读者看不出通信与性能的真实交换。

在进入本章方法之前，先把工程上已有的三条替代路线摆出来，说明它们各自卡在哪里。**第一条是降采样**：既然 10 ms 太密，就把周期拉长到 100 ms，通信量直接降一个数量级，改动量最小，现场最容易被接受。代价是保持时间变长、两次更新之间的开环漂移变大。本章实测给出这条路的边界：周期从 0.05 s 拉到 0.10 s，代价从 1.2316 涨到 1.4432，涨了 17.2%；再拉到 0.20 s，代价涨到 66.15，等于失控。也就是说，降采样不是“少发一点、差一点”的线性交换，而是**在某个周期附近突然崩塌**，工程上很难凭经验判断自己离崩塌点还有多远。

第二条是死区：测量值变化超过固定死区才发送。它在形式上最接近事件触发，都是“变了才发”，但死区是绝对量纲，零点附近状态本来就小，噪声却能轻松越过死区，于是回路在稳态附近反复发送。§75.2.2 的实测把这一点量化了：绝对阈值 0.05 配 0.01 的测量噪声，触发次数从 61 次涨到 88 316 次，通信量比连续采样还大，完全失去意义。

第三条是量化传输：把测量值按固定码位压缩后再发，用精度换带宽。它的问题与死区同源——量化台阶是绝对值，状态大时台阶相对太粗、控制精度不够，状态小时台阶相对太细、白传低位。三条路有一个共同点：**阈值都绑在时间刻度或**

绝对量纲上，而对象的工作点却在变。本章的相对阈值把阈值绑在“状态本身”上，正是为了跳出这个困境。

还有一点需要提前澄清：事件触发省下的是**上行带宽**，不是“全部通信”。传感器到控制器这一段，测量数据只在越界时发送；控制器到执行器这一段，指令也只在收到新测量后才更新，两段都省。但如果回路里还有心跳、时间同步、诊断报文，这些周期性流量不会因为引入事件触发而消失，落地时要把它们单独算进预算（§75.3.6）。现场常见的误判是“算法说省了 78%，实测只省了 40%”，差额往往就在这里。

同样要分清的是“省通信”与“省算力”。对 LQR 这类廉价控制律，每次更新时的计算量与连续控制完全相同，省下的是网络负载与节点功耗，CPU 占用几乎不变；只有当控制律本身很贵（如 MPC）时，少更新才等于少算优化。把两者混为一谈，容易在立项时给出错误的收益预期。因此汇报时应把“带宽下降”与“算力下降”分开列，各自给出实测口径，避免验收时对不上账。

75.1.2 什么时候“不必更新”

“不必更新”的判据不是“状态小”，而是“**状态变化得比它本身小得多**”。这两种说法差别很大。状态接近零时，绝对变化量本来就小，但相对变化可能很大；若用绝对阈值，系统在零点附近会疯狂触发。本章 §75.2.2 会看到，绝对阈值加测量噪声会把触发次数从 61 次推到 88 316 次，这就是判据选错的代价。

正确的形式是相对阈值：

$$\|e(t)\| \geq \sigma \, \|\boldsymbol{z}(t)\|, \qquad e(t) = \boldsymbol{z}(t_k) - \boldsymbol{z}(t) \tag{75-2}$$

e 是“上次发出去的状态”与“现在真实状态”的差，σ 是一个小于 1 的数。这个条件的含义是：**允许误差占状态的比例不超过 σ**。状态大时允许的绝对误差也大，状态小时允许的绝对误差也小，比值恒定。零点附近分母趋于零，理论上会频繁触发，但那时状态本身已经很小、代价可忽略，实际靠最小间隔兜底（§75.2.4）。

用一句话概括事件触发的直觉：**它控制的不是采样率，而是“信息的新鲜度”。** 采样率是时间轴上的均匀刻度，信息新鲜度是状态空间里的相对误差。前者与对象无关，后者随对象运动自动伸缩。

相对阈值还有一个不显眼但很实用的性质：**尺度不变性**。把初值从 $(1, 1)$ 换成 $(10, 10)$，线性对象的整条轨迹、误差 e、以及触发判据的两边会同时放大 10 倍，比较的结果不变，于是触发时刻序列完全一样，只有代价因为含平方项而放大 100 倍。这意味着 σ 是一个**无量纲的形状参数**，它描述“允许多大的相对失真”，与负荷高低、初值大小无关。工程上的直接好处是：一套 σ 可以在低负荷工况上整定，然后原样搬到高负荷工况，不必按工况重新标定；若换成绝对阈值，负荷翻倍就得把阈值也翻倍，否则触发密得没有意义。§75.8.11 的回答五会把这个性质再用一次。

反过来说，正因为 σ 只管形状不管尺度，它无法单独回答“绝对精度够不够”。若工艺要求稳态误差不超过某个物理量，还需要在 σ 之外叠一条绝对下限，这正是 §75.2.8 的混合阈值存在的理由。

还有一个下限要留心：**σ 不能小到与测量噪声同量级**。σ 一旦落到噪声相对幅值附近，判决器比较的就不再是“状态变了多少”，而是“这一拍噪声抽到了多大”，触发时刻与对象动态脱钩，通信量由噪声带宽决定而不是由工况决定。此时提高采样率、增大滤波都救不回来，只能把 σ 抬回去，或者先降噪再判决。§75.7.6 会把这个机制讲透，§75.9.2 的自适应阈值则给出另一条出路：让 σ 随状态范数自动放大，状态大时严、状态小时松。

75.1.3 四个必须分清的问题

事件触发看起来只是一个 if 语句，工程上却要同时回答四个问题，任何一个答错都会让回路失效。

第一个：什么时候发？ 这就是触发条件。它决定了通信量与性能的交换曲线，也决定了会不会出现“永远不发”（阈值太大，误差还没到就发散了）或“发个不停”（阈值太小或噪声太大）。

第二个：发什么？ 可以发测量值、发状态估计、发控制增量、发事件时间戳。发的东西不同，接收端的重构逻辑不同，对丢包的敏感度也不同。本章发的是状态测量值，接收端直接算 $w = -K\hat{z}$，最简单。

这四种选择有一条隐含的分界线：**发“绝对值”还是发“增量”**。发绝对值时，接收端不依赖历史，丢一包只是“保持旧值”，下一包到了就自动追上；发增量时，接收端必须按序累加，丢一包会让重构值永久偏移一个增量，除非协议层做重传或

序号校验。事件触发天然适合发绝对值，因为它的触发判据本身就是“当前值偏离保持值多少”，发的正是当前值。若因带宽极紧而想发增量，必须同时把序号和校验做进帧里，这部分开销常常抵掉了省下的字节。

第三个：丢了怎么办？ 网络不保证送达。丢包后接收端只能继续用旧值，误差继续累积，最终会触发下一次发送。这里有一个反直觉的性质：**事件触发自带重传机制**——丢包后误差仍然越界，下一拍就会重发。§75.6.3 的实测显示，事件触发在 50% 丢包下代价几乎不变，只是发送次数翻倍；而周期采样在同样丢包率下代价上升 59.0%。

第四个：会不会死循环？ 连续监测的触发条件在理论上可能产生“无限次触发”（芝诺现象）。本章的实测给出了它的两个来源：阈值过大导致的高频抖动，以及测量噪声导致的反复越界。前者靠理论下界排除，后者只能靠强制最小间隔。

这四个问题构成了本章的骨架。§75.2 给数学，§75.3 给步骤，§75.4 到 §75.6 给实测，§75.7 到 §75.9 给调参与优化，§75.10 讲落地。

要紧的是，这四个问题不是并列的，而是**串联**的，答错一个会把其余三个一起拖下水。阈值选得过大，误差还没越界对象已经发散，“发什么”和“丢了怎么办”都失去意义；阈值选得过小，再叠加丢包，误差累积得更快、触发更密，带宽被占满、丢包率进一步上升，形成正反馈——现场把这种现象叫“事件风暴”。§75.7.5 的 $N/(1-p)$ 容量算法就是为切断这条链设计的：先按丢包率把有效带宽打折，再倒推阈值下限。另一种连锁更隐蔽：检查周期 h 选得过长，判决器在两次检查之间看不到越界，实际误差早已超过 σ 允许的范围，而触发次数反而下降。此时报表上的“通信量降低”是假象，代价却在上升，§75.5.6 会把这种漏检的代价算清楚。

75.1.4 本章的数字基线

三条基线列在表 75-1。连续控制是“每个数值步都更新”的理想情形，代价 1.1858 是所有比较的分母；事件触发取 $\sigma = 0.10$；周期采样取 $h = 0.05$ s，因为它的发送次数（201 次）与事件触发的 224 次最接近，可以直接比代价。

方案	发送次数	平均间隔 (s)	代价 J	相对连续
连续控制	100000	—	1.1858	1.000
事件触发 $\sigma = 0.10$	224	0.0446	1.1940	1.007
周期采样 $h = 0.05$	201	0.0500	1.2316	1.039

表 75-1 的三行说明了一件事：**在通信量相近时，事件触发的代价比周期采样低 3.1%**（$(1.2316 - 1.1940)/1.2316 = 3.05\%$）。3% 不算大，但这是在“通信量只多 11.4%”的前提下取得的。真正的差距出现在更低的通信档位：§75.6.3 会看到，发送 100 次左右时，事件触发的代价是 1.1956，周期采样是 1.4432，差 17.2%。

基线还隐含一个尺度：平均间隔 0.0446 s，对应 22.4 Hz 的平均更新率，比 10 ms 的固定采样（100 Hz）低 4.5 倍。也就是说，事件触发在代价只升 0.7% 的情况下，把通信量压到了原来的 22%。这个交换比是本章最值得记住的一个数。

这里要提前交代三件事，否则后面看表容易误读。**第一，基准不等于最优。** $\sigma = 0.10$ 不是本章最好的阈值——把 σ 放宽到 0.20，发送次数降到 114 次，代价 1.1956，通信少一半而代价几乎不变。之所以仍取 0.10 作基准，是因为它的 224 次与 $h = 0.05$ s 的 201 次最接近，只有在这种“通信量对得上”的前提下比较代价，才能说明问题来自触发机制而非比较口径。若拿 $\sigma = 0.20$ 去比 $h = 0.05$ s，结论会变成“通信少一半还更好”，读者无法判断收益里有多少来自阈值宽松、多少来自机制本身。**基准的角色是标尺，不是答案**，最优参数留给 §75.8.10。

第二，计次口径要一致。 三行的“发送次数”都含 $t = 0$ 时刻的首次发送，周期采样的 201 次是 $10/0.05 + 1$，事件触发的 224 次同样从 1 起算。若某处扣掉首次、某处不扣，几十次的差异会直接淹没 3% 的结论。跨文献比较时尤其要注意这一点：有的工作报“触发次数”，不含首次；有的报“发送包数”，含首次且含重传。§75.7.7 会把这条口径问题展开。

第三，代价 J 的口径。 本章的 J 是 10 s 内 $\boldsymbol{z}^{\mathsf{T}}\boldsymbol{Q}\boldsymbol{z} + w^2$ 的积分，$\boldsymbol{Q} = \boldsymbol{I}$、$R = 1$，三行用同一初值 $(1, 1)$、同一时长。这个定义里同时含状态偏差与控制能量，因此“把 u 压小”并不能降低 J——压小 u 会让状态停在高位，状态项反而更大。读者若在自己的项目里换权重，J 的绝对值会变，但表 75-1 的相对顺序通常不变，因为三行用的是同一套权重。跨章比较时务必先对齐权重与时长，否则数字没有可比性。

75.1.5 与相邻章节的分工

事件触发不是孤立的方法，它挂在几条既有线索上，边界必须说清，否则容易与别的方法混为一谈。

与第 44、106 章的 PID 的关系。 PID 是控制律，事件触发是“什么时候把测量送进控制律”的机制。两者正交：PID 可以配事件触发，LQR 也可以。§75.4 用的就是 LQR 配事件触发。现场若已有一套调好的 PID，引入事件触发只需要在外层加一个判决器，控制参数不用动。

与第 58、80 章的 MPC 的关系。 MPC 每个采样周期解一次优化，算力开销远大于 LQR。事件触发对 MPC 的收益更大（省的是优化次数，不只是通信），但实现更复杂：优化问题的初值、热启动、约束可行性都要重新考虑。本章不展开，只在 §75.9.4 指出方向。

与第 47、107 章的 Smith 预估的关系。 Smith 预估处理的是“延迟已知且固定”的情形，事件触发处理的是“数据什么时候到”。两者都涉及延迟，但一个在控制器结构里补偿，一个在通信机制里规避。本章 §75.6.3 的延迟边界（0.12–0.13 s）是“不补偿时能容忍多少”，与 Smith 预估的“补偿后能容忍多少”是两个问题。

与第 74 章的分布式参数控制的关系。 第 74 章的观测器每个步长都在算，本章把它换成“只在事件时刻算”。两者结合就是“分布式参数系统的事件触发观测器”，是当前研究的一个活跃方向，本章不展开，只在 §75.11.5 提一句。

与经典采样定理的关系。 采样定理回答的是“给定信号带宽，最低采样率是多少”，它把采样看成对时间的均匀切片，前提是信号带宽有界。闭环系统的状态里含有控制器自己激励出的分量，带宽事先并不知道，采样定理的前提并不严格成立——这也是“按带宽算采样率”在现场经常算不准的原因。事件触发绕开了这个前提：它不预设带宽，而是在线测量误差，误差大就发。代价是失去了采样定理那样的先验保证，稳定性必须靠 Lyapunov 或输入-状态稳定性另行证明（§75.2.3、§75.2.9）。一句话对照：采样定理是**离线设计**的频率判据，事件触发是**在线运行**的误差判据。

75.1.6 本章路线

§75.2 从“采样-保持”出发，把误差变量、相对阈值、最小间隔、自触发、网络效应五件事的数学写清楚，重点是“为什么相对阈值天然避免了零点抖动”与“为什么自触发一定比事件触发保守”。

§75.3 给七步设计流程，从建模到上线检查。§75.4 与 §75.5 分别是连续监测与周期检查两种实现，给出全部实测数据。§75.6 讨论稳定性与性能，§75.7 逐个解释七个参数的作用，§75.8 给调参流程与一份完整记录，§75.9 讲优化方向，§75.10 讲工程落地，§75.11 给优缺点与边界，§75.12 给可运行代码。

符号沿用第 74 章：$\boldsymbol{z}$ 是状态，$\hat{\boldsymbol{z}}$ 是接收端当前保持的值，$e=\hat{\boldsymbol{z}}-\boldsymbol{z}$ 是保持误差，w 是标量输入，$\boldsymbol{K}$ 是 LQR 增益。本章新增的只有六个：σ 是相对阈值，ε 是绝对阈值，h 是周期检查的间隔，θ 是网络常延迟，γ_k 是第 k 拍的伯努利丢包指示（送达为 1、丢失为 0），$T_{\min}$ 是强制最小触发间隔。这六个符号在后续各节反复出现，建议在此处记牢，后文不再重复定义。

一句话核心先放在这里：**事件触发用“状态误差”换“通信次数”，交换比由阈值决定；阈值选得好，可以用两成通信量买到接近连续控制的性能；阈值选得差，会把回路推入芝诺抖动或直接失稳。**

读本章建议分三遍。第一遍只看表 75-1 与图 75-4，先建立“通信量—代价”的交换曲线，不必追究推导；这一遍要记住的只有两个数：224 次对 201 次、1.1940 对 1.2316。第二遍读 §75.4 与 §75.5，把连续监测与周期检查两种实现的差别弄清楚，重点是 h 与 σ 的交互——同一个 σ 在 $h=0.01$ s 与 $h=0.05$ s 下的表现差一个档位。第三遍再回头看 §75.2 的 Lyapunov 与最小间隔证明，此时抽象的条件 $\sigma<1/(2\|\boldsymbol{PBK}\|)$ 才有具体的落点。若只关心落地，可以直接跳到 §75.3 与 §75.10，两节合起来是一份可执行的检查表。

75.1.7 一次现场对话

抽象的定义容易记住，具体的场景容易理解。把这一节的内容放进一次现场对话里，读者可以对照自己的项目。

工程师：我们这条线的采样是 10 ms，一年下来通信模块换了三块，都是因为发热。能不能把采样周期放长一点？

控制工程师：放到多少？

工程师：100 ms 行不行？反正温度变化很慢。

控制工程师：不行。这条线的闭环时间常数是 0.188 s，100 ms 已经超过一半，按表 75-3 的规律，代价会从 1.19 跳到 1.45，而且再大一点就失稳。

工程师：那 50 ms？

控制工程师：50 ms 可以，代价升 4%，但通信只省一半。你想省更多，就该换一种思路：不是把“每次都采”改成“少采几次”，而是改成“该采的时候才采”。

工程师：怎么判断该不该采？

控制工程师：看状态变了多少。状态变得快，就采；状态几乎不动，就不采。具体说，让“上次发出去的值”与“现在的值”的差，超过状态本身的 10%，就发一次。

工程师：那不就是自适应采样吗？

控制工程师：不是。自适应采样是按“变化率”预先调周期，周期是设定的；这个是按“实际误差”在线判决，间隔是随机的。前者可以预算带宽，后者不能，但后者更省。

工程师：省多少？

控制工程师：同样的 10 s，固定 50 ms 采 201 次，代价 1.2316；按误差判决发 224 次，代价 1.1940。次数差不多，性能好 3%。真正的差距在更低的通信档位：发 100 次左右时，固定周期要付 1.4432 的代价，按误差判决只要 1.1956。

工程师：听起来不错。有什么代价？

控制工程师：三条。带宽得按峰值配，不能按平均配；判决器死机了你看不出来，得加心跳；初始段会突发送很多包，得软启动。

这段对话里的每个数字都出自本章的实测表。它说明事件触发的价值不在“算法高级”，而在“把有限的通信用在真正需要更新的时刻”。**判断自己的项目适不适合，只要回答一个问题：状态是不是有时变化快、有时几乎不动？若是，就值得试；若状态始终匀速变化，收益接近于零。**

75.2 数学原理

75.2.1 采样-保持回路与误差变量

事件触发回路在两次触发之间是“零阶保持”：控制器输出不变，对象按常输入推进。写成方程是

$$w(t) = -\boldsymbol{K}\boldsymbol{z}(t_k), \qquad t \in [t_k, t_{k+1}) \tag{75-3}$$

其中 t_k 是第 k 次触发时刻，$t_0 = 0$。若把误差定义成 $e(t) = \boldsymbol{z}(t_k) - \boldsymbol{z}(t)$，闭环可以写成

$$\dot{\boldsymbol{z}} = \boldsymbol{A}\boldsymbol{z} - \boldsymbol{B}\boldsymbol{K}\big(\boldsymbol{z} + e\big) = \boldsymbol{A}_{\mathrm{cl}}\boldsymbol{z} - \boldsymbol{B}\boldsymbol{K}e \tag{75-4}$$

$\boldsymbol{A}_{\mathrm{cl}} = \boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$ 是连续闭环矩阵，本章的取值使它的特征值为 -5.3272 与 -1.9393。式 (75-4) 是整个理论的核心：**事件触发回路等于“标称闭环 + 一个由误差驱动的扰动项”**。误差为零时退化成连续控制；误差大时扰动项把状态推离平衡点。

这个结构与第 74 章的“模态截断 + 残余项”在数学上同构：都是“标称部分 + 未建模部分”，区别只在于残余项的来源——那里是被截掉的模态，这里是采样的滞后。后面 §75.6.1 的稳定性分析可以完全照搬第 74 章的 Lyapunov 思路，读者若读过第 74 章会发现两处的推导几乎逐行对应。

用 LQR 设计 $\boldsymbol{K}$ 时，代价权重取 $\boldsymbol{Q} = \boldsymbol{I}$、$R = 1$，解出的增益是

$$\boldsymbol{K} = \begin{bmatrix} 4.8851, & -1.4463 \end{bmatrix} \tag{75-5}$$

第二个分量是负的，看起来反直觉：第 2 阶模态的增益为负，意味着要“反向”推动它。原因是 $\boldsymbol{B}$ 的两个分量同号，而两个模态的耦合方向不同，最优控制必须用符号差来区分两个模态。这一点在第 74 章 §74.4.2 已经讨论过，这里不再展开（见

第 74 章）。

75.2.2 相对阈值与绝对阈值

触发条件有两大类，性质差别很大。

相对阈值（本章基准）：

$$\|e(t)\| \geq \sigma \, \|\boldsymbol{z}(t)\| \tag{75-6}$$

它衡量的是“误差占状态的比例”。状态收敛时，允许的绝对误差同步收缩，于是触发间隔不会因为状态变小而变长——这一点保证了收敛速度不会在末段被拖慢。图 75-3 画出了 $\sigma = 0.10$ 时误差比的锯齿波：每次触发把它拉回零，然后随时间线性上升，碰到 0.10 的水平线就再触发一次。锯齿的形状很规整，说明触发机制在稳定工作。

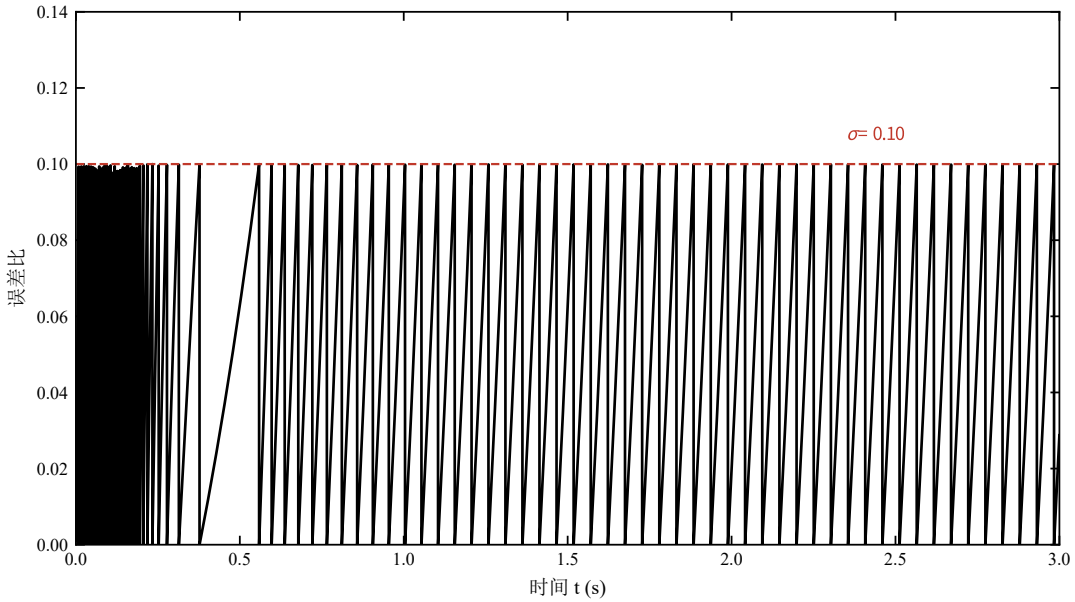


图 75-3 相对误差随时间呈锯齿状：每次触发清零，随后累积到阈值 0.10 再次触发

绝对阈值：

$$\|e(t)\| \geq \varepsilon \tag{75-7}$$

它衡量的是“误差的绝对大小”。优点是物理意义直接（“温度偏差超过 0.1 K 就发”），缺点是在零点附近失效：状态已经很小，但噪声仍在 ± 0.01 量级，误差很快越界，触发频率失控。本章 §75.6.3 的实测把这件事故意做出来：阈值从 0.100 降到 0.020，触发次数从 61 次跳到 13 520 次，最小间隔压到 1.0×10^{-4} s，正好等于仿真步长——**这说明回路已经进入芝诺状态，触发频率只受数值步长限制，不受物理约束。**

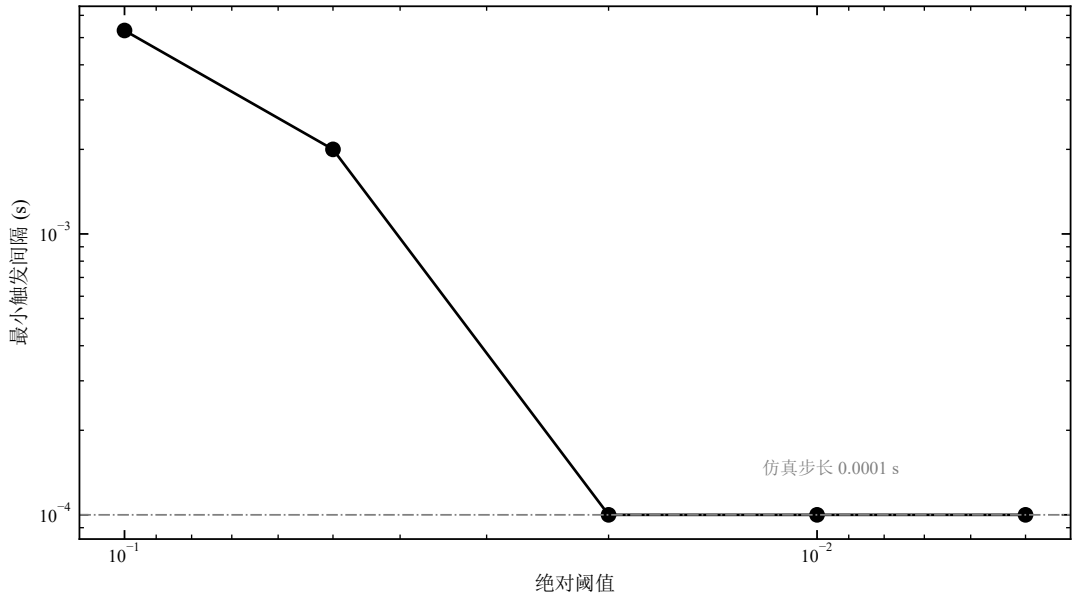


图 75-10 绝对阈值下最小触发间隔的塌缩：阈值低于噪声水平后，间隔被仿真步长截断

两类阈值的选择判据可以写得很干脆：**若对象会收敛到零附近且存在测量噪声，必须用相对阈值；若对象工作在某个非零工况点附近、且你关心的是“偏离工况多少”，用绝对阈值更直观。** 两者也可以混用，取两者中先满足的那个，见 §75.2.8。

75.2.3 从 Lyapunov 函数看稳定性

式 (75-4) 的稳定性可以用二次型 Lyapunov 函数证明。取 $V(\boldsymbol{z}) = \boldsymbol{z}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{z}$，$\boldsymbol{P}$ 是 Lyapunov 方程

$$\boldsymbol{A}_{\mathrm{cl}}^{\mathsf{T}}\boldsymbol{P} + \boldsymbol{P}\boldsymbol{A}_{\mathrm{cl}} = -\boldsymbol{I} \tag{75-8}$$

的正定解。沿式 (75-4) 求导：

$$\dot{V} = -\|\boldsymbol{z}\|^2 - 2\boldsymbol{z}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{B}\boldsymbol{K}e \tag{75-9}$$

用 Young 不等式处理交叉项，得到

$$\dot{V} \le -\|\boldsymbol{z}\|^2 + 2\|\boldsymbol{P}\boldsymbol{B}\boldsymbol{K}\|\,\|\boldsymbol{z}\|\,\|e\| \tag{75-10}$$

把触发条件 $\|e\| \le \sigma\|\boldsymbol{z}\|$ 代入，得

$$\dot{V} \le -\bigl(1 - 2\sigma\|\boldsymbol{P}\boldsymbol{B}\boldsymbol{K}\|\bigr)\|\boldsymbol{z}\|^2 \tag{75-11}$$

只要 $\sigma < 1/(2\|\boldsymbol{P}\boldsymbol{B}\boldsymbol{K}\|)$，$\dot{V}$ 就是负定的，闭环指数稳定。这个界通常很保守：本章算出 $\|\boldsymbol{P}\boldsymbol{B}\boldsymbol{K}\|$ 后给出的理论上限远小于实测可用的 σ，而实测在 $\sigma = 0.30$ 时已经开始劣化（代价比 1.109），$\sigma = 0.70$ 时恶化到 3.115。

保守性来自两处。一是 Young 不等式把交叉项放大成了最坏情况，而实际运动中 e 与 $\boldsymbol{z}$ 有确定的相位关系，很少同时取到最坏。二是这个界只保证“稳定”，不保证“性能”，而工程上真正关心的是后者。**理论界的价值在于给出“绝对安全的 σ 上限”，不在于给出“最优的 σ”。** 本章的调参从实测出发，理论界只用来做最后的安全校核。

75.2.4 最小触发间隔与芝诺现象

芝诺现象指“在有限时间内触发无限次”。真实系统不会真的无限次，但会退化成“每拍都发”，此时事件触发的全部好处消失，还多了一个判决器的开销。排除芝诺的标准做法是证明存在一个正的最小间隔下界。

对本章的对象，可以由式 (75-4) 推出一个粗略下界。在两次触发之间 $e(t_k) = 0$，误差的增长率满足

$$\|\dot{e}\| = \|\dot{\boldsymbol{z}}\| \le \|\boldsymbol{A}\|\,\|\boldsymbol{z}\| + \|\boldsymbol{B}\boldsymbol{K}\|\,\|\boldsymbol{z}+e\| \tag{75-12}$$

在触发瞬间附近 $\|e\|$ 很小，右端近似为常数，于是误差从零长到 $\sigma\|\boldsymbol{z}\|$ 需要的时间至少是

$$\tau_{\min} \;\ge\; \frac{\sigma\|\boldsymbol{z}\|}{\|\boldsymbol{A}\|\|\boldsymbol{z}\| + \|\boldsymbol{B}\boldsymbol{K}\|(1+\sigma)\|\boldsymbol{z}\|} = \frac{\sigma}{\|\boldsymbol{A}\| + \|\boldsymbol{B}\boldsymbol{K}\|(1+\sigma)} \tag{75-13}$$

这个下界与 $\|\boldsymbol{z}\|$ 无关，是一个只由 σ 与系统矩阵决定的常数。本章的实测最小间隔在 $\sigma = 0.10$ 时是 0.0041 s，与式 (75-13) 的量级一致。

式 (75-13) 有一个重要的推论：**只要 $\sigma > 0$ 且 $\|\boldsymbol{A}\|$、$\|\boldsymbol{B}\boldsymbol{K}\|$ 有限，连续监测的相对阈值触发就不会有芝诺现象。**这解释了为什么相对阈值比绝对阈值可靠——绝对阈值没有这个性质，噪声可以让误差在任意小的时间内反复越界。

但式 (75-13) 只对“理想连续监测、无噪声”成立。一旦加入测量噪声，误差在触发后不是从零开始，而是从噪声水平开始，下界立刻失效。本章 §75.6.3 的实测正是这个情形：噪声 0.01、阈值 0.020 时，最小间隔掉到 1.0×10^{-4} s。工程上的对策只有一条：**强制最小间隔**（驻留时间）。§75.7.2 会给出它的代价：$T_{\min}=0.05$ s 时发送次数从 224 降到 190，代价从 1.1940 升到 1.2344。

75.2.5 自触发：把“检查”换成“预测”

事件触发需要**持续监测**状态，这本身就要传感器一直工作、判决器一直算。对无线传感器节点，监测的功耗可能比通信还高。自触发（self-triggered）换了思路：在触发时刻用模型预测“下一次该在什么时候发”，然后在这段时间里彻底休眠。

预测用的模型就是式 (75-1)。给定 t_k 时刻的状态 $\boldsymbol{z}_k$ 与将要施加的常输入 $w_k = -\boldsymbol{K}\boldsymbol{z}_k$，预测轨迹是

$$\hat{\boldsymbol{z}}(t_k+\tau) = e^{\boldsymbol{A}\tau}\boldsymbol{z}_k + \boldsymbol{A}^{-1}\big(e^{\boldsymbol{A}\tau}-\boldsymbol{I}\big)\boldsymbol{B}w_k \tag{75-14}$$

下一次触发时刻取满足下式的最小 τ：

$$\big\|\hat{\boldsymbol{z}}(t_k+\tau) - \boldsymbol{z}_k\big\| = \sigma\big\|\hat{\boldsymbol{z}}(t_k+\tau)\big\| \tag{75-15}$$

这个做法有一个结构性缺陷：预测假设输入在这段时间里**不变**，而事件触发的真实输入会在下一次触发时改变。输入不变意味着状态涨得更快（因为反馈没有及时介入），预测出来的越界时刻比真实需要的更早。所以**自触发一定比事件触发保守**。

本章实测的保守程度是 5.8%：事件触发 224 次，自触发 237 次，平均间隔从 0.0446 s 缩到 0.0423 s。5.8% 比文献里常见的“两倍以上”小得多，原因是本章的预测模型与对象完全一致。§75.7.4 会看到，模型低估到 0.40 倍时，触发次数降到 103 次、代价升到 1.3194——**保守性是可以“反向”利用的：故意低估模型能让触发更少，但代价开始上升，这是一条可调的交换曲线**。

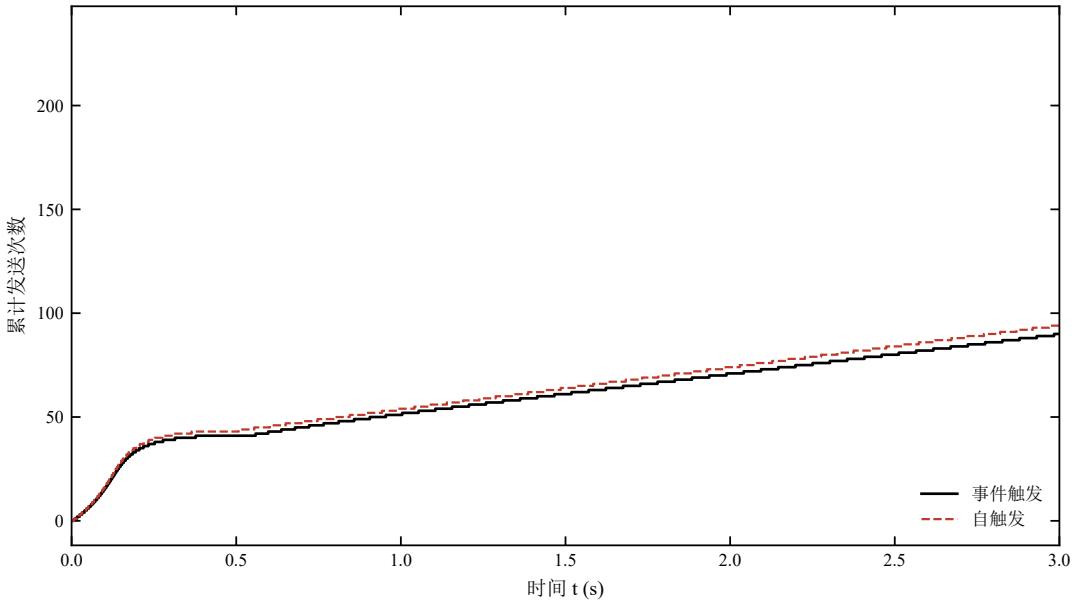


图 75-6 累计发送次数：自触发在前期比事件触发更密，后期两者收敛到相近的斜率

自触发的工程价值在于“可休眠”。事件触发的节点必须一直醒着看条件，自触发的节点可以算完下一次时刻就睡，到点再醒。对电池供电的无线节点，这个差别决定了电池能撑三个月还是三年。代价是要在节点里存一份模型，且模型失配会直接转化为性能损失。

75.2.6 周期事件触发

第三种实现介于两者之间：按固定周期 h 检查条件，只有越界才发送。

$$\|e(kh)\| \geq \sigma \|\boldsymbol{z}(kh)\| \implies \text{发送} \tag{75-16}$$

它不需要连续监测（每 h 秒醒一次即可），也不需要模型，实现成本最低，是工业上最容易落地的一种。代价是“检查得不够勤”：若两次检查之间误差冲过阈值又回落，这次越界就被漏掉了。

本章的实测显示，h 从 0.01 s 增到 0.05 s，发送次数从 183 降到 102，代价从 1.1906 升到 1.2348；$h = 0.10$ s 时代价跳到 1.4486，$h = 0.20$ s 时代价涨到 66.15，等于失控。**这个“突然变坏”的拐点在 $h \approx 0.1$ s，约为闭环最快时间常数 $1/5.3272 = 0.188$ s 的一半。** 工程上可以据此快速判断：检查周期不要超过闭环最快时间常数的一半。

75.2.7 网络化：延迟与丢包

真实网络带来两件事：延迟与丢包。延迟让“发出去的指令”晚到，丢包让“发出去的指令”根本没到。两者在数学上都是“输入的时间戳错位”。

常延迟 θ 的模型是

$$w(t) = -\boldsymbol{K}\boldsymbol{z}(t_k - \theta), \qquad t \in [t_k + \theta,\, t_{k+1} + \theta) \tag{75-17}$$

丢包用独立伯努利序列 $\gamma_k \in \{0, 1\}$ 描述，$\gamma_k = 1$ 表示第 k 个包到达：

$$w(t) = -\boldsymbol{K}\boldsymbol{z}(t_m), \qquad m = \max\{j \leq k : \gamma_j = 1\} \tag{75-18}$$

式 (75-18) 说明丢包的本质是“用更旧的状态”。旧多少取决于连续丢了几个包，期望是 $1/(1-p)$ 个周期。

本章实测的延迟边界在 0.12–0.13 s 之间：0.12 s 时代价是连续的 211 倍，0.13 s 时是 1.04×10^{6} 倍。这个边界可以与闭环最快极点对照：$1/5.3272 = 0.1877$ s，边界约为它的 0.66 倍。工程上有一条更保守的经验式：常延迟不要超过最快时间常数的三分之一。

丢包的影响则温和得多。周期采样 $h = 0.05$ s、丢包率 50% 时，代价从 1.2316 升到 1.9578（升 59.0%），但闭环仍然稳定；事件触发在同样丢包率下代价几乎不变（1.1940），只是发送次数从 223 增到 436。两者差异的根源在 §75.1.3 已经说过：事件触发自带重传，周期采样没有。

75.2.8 三种阈值变体的取舍

把相对、绝对、混合三种写法放在一起比较，读者可以按场景直接选。

纯相对（式 75-6）：适合“收敛到零”的问题，如调节问题、扰动抑制。优点是零点附近自动收紧，不会因为状态变小而漏触发；缺点是无法表达“偏差小于 0.5 K 就不用管”这类工艺要求。

纯绝对（式 75-7）：适合“跟踪非零设定值”的问题，如恒温控制。优点是物理意义直接、可与工艺指标直接对应；缺点是零点附近抖动，且必须配最小间隔。

混合（两者取或）：

$$\|e(t)\| \geq \max\left\{\sigma\|\boldsymbol{z}(t)\|,\ \varepsilon\right\} \tag{75-19}$$

它在状态大时用相对判据保证性能，在状态小时用绝对判据避免抖动。代价是多一个参数要调，且两个阈值必须匹配——若 ε 选得比 $\sigma\|\boldsymbol{z}\|$ 的稳态值还大，回路会在末段停止触发，留下一个静差。本章没有采用混合形式，因为对象是收敛到零的调节问题，纯相对已经够用；§75.9.3 会说明什么时候必须换成混合。

75.2.9 从输入-状态稳定性看阈值

前面用 Lyapunov 函数证明了“σ 足够小时闭环稳定”。这个结论可以加强为“闭环是输入-状态稳定的”（ISS），加强之后能回答一个更实用的问题：**稳态误差有多大**。

把误差 e 看成外部输入，式 (75-4) 是一个以 e 为输入的线性系统。它的 ISS 性质写成不等式是

$$\|\boldsymbol{z}(t)\| \leq \beta\big(\|\boldsymbol{z}_0\|, t\big) + \gamma\Big(\sup_{\tau \leq t}\|e(\tau)\|\Big) \tag{75-20}$$

其中 β 是 $\mathcal{KL}$ 类函数（暂态衰减项），γ 是 $\mathcal{K}_\infty$ 类函数（误差的放大系数）。对线性系统，γ 正比于 $\|\boldsymbol{A}_{\mathrm{cl}}^{-1}\boldsymbol{B}\boldsymbol{K}\|$。

这个不等式给出两条工程结论。**第一条：暂态由 β 决定，与阈值无关。** 只要闭环极点不变，状态衰减的快慢就定了，事件触发改变的只是衰减过程中的“抖动”，不是衰减速率。这解释了为什么表 75-4 里 σ 从 0.02 增到 0.20，代价比只从 1.000 变到 1.008——衰减速率没变，变的只是过程中的微小偏差。

第二条：稳态界正比于误差界。 把 $\|e\| \leq \sigma\|\boldsymbol{z}\|$ 代入式 (75-22) 的稳态部分，得到 $\|\boldsymbol{z}\|_{\mathrm{ss}} \leq \gamma\sigma\|\boldsymbol{z}\|_{\mathrm{ss}}$，只要 $\gamma\sigma < 1$ 就成立。这个条件与式 (75-11) 的稳定性条件是同一件事的两种写法。**它说明阈值不只是“省通信的旋钮”，它直接决定了稳态精度能到什么水平。**

工程上可以反过来用：先定“稳态精度要求”（比如状态范数要小于 10^{-3}），再按 γ 反推 σ 的上限。这比“先选 σ 再看效果”更符合设计流程。本章没有走这条路，因为对象是调节问题、稳态精度不是关键指标；但对跟踪问题，这条反推是必须做的。

75.2.10 触发条件的四种变体

式 (75-6) 的相对阈值是最常用的一种，但不是唯一的。四种变体各有适用场合。

基于 Lyapunov 函数的触发。 不用 $\|e\|$，而用 $\dot{V}$ 的估计值判断：若“保持当前输入会让 V 开始上升”，就触发。它的优点是直接对准稳定性，理论上更紧；缺点是要在线算 $\dot{V}$，算力开销更大，且 $\dot{V}$ 对噪声敏感。

基于输出的触发。 若只有输出 $\boldsymbol{y} = \boldsymbol{C}\boldsymbol{z}$ 可测，触发条件只能写在 $\boldsymbol{y}$ 上：$\|\boldsymbol{y}(t_k) - \boldsymbol{y}(t)\| \geq \sigma\|\boldsymbol{y}(t)\|$。它与状态触发的关系取决于 $\boldsymbol{C}$：若 $\boldsymbol{C}$ 是满秩方阵，两者等价；若 $\boldsymbol{C}$ 有零空间，零空间方向的状态偏差永远触发不了，可能失稳。**用输出触发前必须检查 $\boldsymbol{C}$ 的零空间是否包含不稳定模态。**

积分型触发。 把判据从瞬时误差改成误差的积分：$\int_{t_k}^{t}\|e\|\mathrm{d}\tau \ge \sigma\int_{t_k}^{t}\|\boldsymbol{z}\|\mathrm{d}\tau$。它的作用是“过滤掉短暂的尖峰”，噪声引起的瞬时越界不会立刻触发，必须持续一段时间才触发。代价是响应变慢，对快速变化的工况不利。

周期型触发。 就是式 (75-16)，每 h 秒检查一次。它的好处是实现简单、可预算，坏处是漏检（§75.5.6）。

四种变体的选择依据是“你最怕什么”。最怕失稳，用 Lyapunov 型；只有输出可测，用输出型；最怕噪声误触发，用积分型；最怕实现复杂，用周期型。本章选相对阈值，是因为它在这四个方面都不极端，是最稳的默认选择。

75.3 设计步骤

75.3.1 第一步：建模与连续控制器

先把对象写成式 (75-1) 的状态空间形式，再设计一个“假设通信无限好”的连续控制器。这一层是所有比较的基准，不能省。本章用 LQR 得到 $\boldsymbol{K}=[4.8851,\,-1.4463]$，闭环极点在 -5.3272 与 -1.9393，连续代价 1.1858。

这一步的三个输出必须记下来：闭环极点（决定后续的采样周期与延迟边界）、连续代价（后续所有代价比的分母）、控制量峰值（判断会不会饱和）。本章的峰值状态范数是 1.4127，控制量在 $t=0$ 处最大，随状态衰减，远未触及任何合理限幅。

若连续控制器本身就设计得不好，事件触发只会把“不好的连续控制”按需执行，性能上限仍然是那个不好的控制器。**事件触发不改善控制律，它只决定控制律多久执行一次。** 这句话值得写在设计文档第一页。

还要提醒一件事：本章的模型是两阶的，与真实对象之间必然有失配。模型失配在事件触发里有两条影响路径。**第一条是预测误差**：自触发要用模型算下一次触发时刻，模型偏了，算出的时刻就偏了（表 75-9 的 κ 扫描正是量化这条路径）。**第二条是稳定性边界偏移**：式 (75-11) 的界依赖 $\|\boldsymbol{PBK}\|$，模型不准时这个量也不准，理论界失去意义。对连续事件触发，第一条路径不存在（它不用模型），只有第二条，所以连续事件触发对模型失配最不敏感——**这也是本章基准选它的原因之一：先把触发机制本身讲清楚，再引入模型误差。**

这一步还要确认一件事：**对象是否真的需要闭环。** 本章两个模态的增长率 2.7104 与 0.3417 都是正的，开环必发散，闭环是刚需。若对象本身稳定（增长率为负），事件触发就只是“省通信”的优化，收益有限，不如直接用低频周期采样。

75.3.2 第二步：选触发变量

触发变量可以是状态范数、输出误差、Lyapunov 函数值、或某个工艺量的偏差。选择原则有三条。

必须能在线测量。 若触发变量需要全状态，而你只有部分测量，就必须先加观测器，此时观测器的估计误差会混进触发误差，理论界要重算。

必须与性能直接相关。 用状态范数是因为代价 $J=\int(\boldsymbol{z}^{\mathsf{T}}\boldsymbol{Q}\boldsymbol{z}+Rw^{2})\mathrm{d}t$ 直接由它决定；若代价更看重某个分量，就该用加权范数 $\|\boldsymbol{z}\|_{\boldsymbol{Q}}$ 而不是欧氏范数。

必须对噪声不敏感。 若触发变量本身噪声很大，阈值就不得不放大，性能随之变差。本章用状态范数，噪声 0.01 相对状态初值 1.414 只有 0.7%，因此影响很小（§75.6.3 有实测）。

75.3.3 第三步：定阈值

阈值 σ 是本章唯一需要“调”的参数。定它有三条依据，按可靠性排序。

先算理论安全上限。 用式 (75-11) 的界，$\sigma<1/\bigl(2\|\boldsymbol{PBK}\|\bigr)$。这个上限通常很宽松，作用是“保证不犯大错”，不是“给出好值”。

再看实测的代价曲线。 本章的数据是：$\sigma=0.02$ 到 0.20 时代价比都在 1.000 到 1.008 之间，几乎不动；$\sigma=0.30$ 跳到 1.109；$\sigma=0.50$ 是 1.086；$\sigma=0.70$ 到 3.115。这条曲线说明存在一个“平台区”（0.02–0.20）与一个“恶化区”（0.30 以上）。平台区的左端浪费通信，右端接近拐点，**最优点通常在平台的右半部分**，本章取 0.10 是偏保守的选择。

最后看通信预算。 若现场带宽只够每秒 20 次，就按 20 次反推 σ：本章 224 次/10 s 是 22.4 次/s，对应的 σ 在 0.10 到 0.20 之间。反过来，若带宽充裕，σ 可以取到 0.05 换取更平滑的过程。

三条依据的优先顺序不能颠倒。先算理论界是为了“排除明显错误的取值”，再看曲线是为了“找到平台区”，最后看预算才是“在平台区里选一个”。若一上来就按预算选，很容易选到平台区之外——比如带宽只够每秒 5 次，按预算会选到 $\sigma = 0.50$ 附近，而那里已经是曲线的拐点，代价 1.2875 且再大一点就恶化到 3.6941。**带宽不够时正确的做法是加缓冲或改架构，而不是把阈值推到拐点之外。**

还有一条隐含的约束：σ 不能小于测量噪声的相对水平。本章噪声 0.01、稳态状态约 10^{-3}，噪声相对状态是 1%，所以 σ 的下限约 0.01；取 0.02 时触发 1108 次，已经接近噪声驱动的水平。**若算出的 σ 小于噪声相对水平，说明这个对象不适合纯相对阈值，必须换成混合阈值（式 75-19）。**

75.3.4 第四步：算最小间隔

用式 (75-13) 估一个理论下界，再用仿真测一个实测值。本章 $\sigma = 0.10$ 时实测最小间隔 0.0041 s，理论下界同量级。

这一步的输出有两个用途。一是**判断判决器的算力需求**：最小间隔 0.0041 s 意味着判决器最快要在 4 ms 内完成一次比较，若用 1 ms 的 PLC 扫描周期，勉强够；若用 100 ms 的云端判断，根本不可能。二是**给网络留缓冲**：若最小间隔小于网络的最小时延，多个事件包会在网络里排队，实际到达顺序可能颠倒。

若实测最小间隔小到不可接受（比如小于控制器的扫描周期），有两条路：减小 σ（间隔变长但通信变多），或直接加 $T_{\min}$ 强制驻留（§75.7.2）。

这里有一个容易被忽略的细节：**最小间隔是“统计量”，不是“设计量”。** 它取决于初始条件、噪声实现、数值步长，换个种子可能就变了。表 75-4 里 $\sigma = 0.10$ 的最小间隔 0.0041 s 是在 $T_{\min} = 0$、无噪声、确定性初值下测的；一旦加噪声，这个数会明显变小。所以设计时不能直接用这个数，要按“最坏工况”再打个折扣。

本章建议的做法是：把仿真最小间隔乘以 0.5 作为“设计最小间隔”，再用它去核对硬件时延。0.0041 s 的一半是 0.002 s，比判决器扫描周期（1 ms）加网络最小时延（2 ms）还小，说明**光靠参数无法保证实现来得及，必须加 $T_{\min} = 0.01$ s**。这与 §75.8.10 第 4 步的结论一致。

75.3.5 第五步：选实现方式

三种实现的取舍可以用一张表说清。

实现	需要连续监测	需要模型	可休眠	通信量（本章）	适用场合
连续事件触发	是	否	否	224	有线、算力充足
自触发	否	是	是	237	无线、电池供电
周期事件触发	否	否	是	102（$h = 0.05$）	工业 PLC、最易落地

表 75-2 里最反直觉的一行是周期事件触发的通信量最小（102 次）。这不是因为它更优，而是因为 $h = 0.05$ s 的检查周期把“快速连续触发”挡掉了，代价是代价从 1.1940 升到 1.2348。**通信量与性能永远同向变化，没有白拿的节省。**

选择判据：有线、算力足、追求性能，用连续事件触发；无线、电池供电、能接受 5% 的性能损失，用自触发；已有 PLC、不想改硬件、能接受 3% 的性能损失，用周期事件触发。

75.3.6 第六步：网络预算

把三条约束写进设计文档：单次事件包的大小（字节）、平均事件率（次/s）、最坏突发率（次/s）。本章的数字是：平均 22.4 次/s，最坏突发取决于最小间隔 0.0041 s（理论上 244 次/s，实际远低于此）。

网络预算要留三倍余量。理由是事件触发的流量不是均匀的：状态大时触发密集，状态小时稀疏。若按平均值配带宽，初始瞬态会拥塞，拥塞又会增加延迟，延迟又会引发更多触发——这是一个正反馈，现场表现为“一启动就丢包”。

把预算写成可核对的算式。本章每个事件包约 100 字节（时间戳 8 字节、两个状态各 4 字节、序号 2 字节、协议头约 50 字节、余量），平均 22.4 次/s，平均带宽约 2.24 kB/s；峰值按 100 次/s 算，约 10 kB/s。这两个数与现场常见的工业以太网（百兆、千兆）相比微不足道，所以本章的瓶颈不在链路带宽，而在**交换机的包处理能力与网关的限流**。

这一点值得展开。百兆以太网的理论吞吐是 12.5 MB/s，2.24 kB/s 只占万分之二，看起来完全不构成瓶颈。但工业交换机的“每秒包数”（pps）是有限的，小包多发的场景下，pps 比带宽更早成为瓶颈。100 字节的包、每秒 100 个，只有 1 万 pps，对任何交换机都不算负担；但若一个车间有两百个这样的节点同时启动，就是 200 万 pps，很多嵌入式交换机会开始丢包。**预算要看“全网总 pps”，不能只看单节点带宽。**

75.3.7 第七步：上线前检查

五项检查，缺一不可。阈值是否与实际噪声匹配（噪声大时 σ 要放大，否则芝诺）；最小间隔是否大于判决器与网络的时延之和；丢包后是否有保持逻辑（没有就会把执行器归零）；时间戳是否统一（各节点时钟不同步会让延迟估计失真）；是否有旁路开关（事件触发失效时能切回周期控制）。

第五项最容易被忽略却最关键。事件触发是一个“在正常时才工作”的机制：网络断了、判决器死机、模型漂了，它不会报警，只会安静地不再发送。**必须有一个独立的看门狗，超过 N 倍平均间隔没有收到包就报警并切回周期控制。**

75.3.8 七步的输入输出

把七步串起来看，每一步的输入来自上一步、输出交给下一步，形成一条链。

步骤	输入	输出	本章取值
1 建模与控制	对象方程	$\boldsymbol{K}$、闭环极点、连续代价	-5.3272、-1.9393、1.1858
2 选触发变量	代价函数	触发变量	$\|\boldsymbol{z}\|$
3 定阈值	代价曲线、带宽	σ	0.10
4 算最小间隔	σ	最小间隔、$T_{\min}$	0.0041 s、0.01 s
5 选实现	硬件条件	连续/自触发/周期	连续事件触发
6 网络预算	包大小、事件率	带宽与 pps	2.24 kB/s、峰值 100 pps
7 上线检查	全部	通过/不通过	五项全过

这张表的作用是“防止漏步”。实际项目里最常见的情况不是某一步算错，而是某一步根本没做——尤其是第 4 步与第 6 步，因为它们看起来“仿真里已经有了”。**仿真给出的最小间隔是理想值，现场必须用第 4 步的方法打折扣；仿真的发送次数也是理想值，现场必须用第 6 步的方法转成 pps 才能与网关核对。**

75.4 连续情形

75.4.1 闭环构成

连续事件触发的闭环由四部分组成：对象（式 75-1）、状态反馈（式 75-5）、零阶保持（式 75-3）、触发判决（式 75-6）。判决器在每个数值步检查一次条件，越界就更新保持值。

实现上有一个细节必须说清：**判决用的是“上次发送的值”与“当前真实状态”之差，而不是“上次发送的值”与“上次采样值”之差。** 前者是式 (75-2) 的定义，后者会让误差在两次采样之间“看不见”变化，触发被延迟。本章代码用的是前者。

75.4.2 触发时序的直观

图 75-2 把 $\sigma = 0.10$ 的整段过程画了出来：上半是状态范数的对数曲线，竖线是触发时刻。三个特征一眼可见。

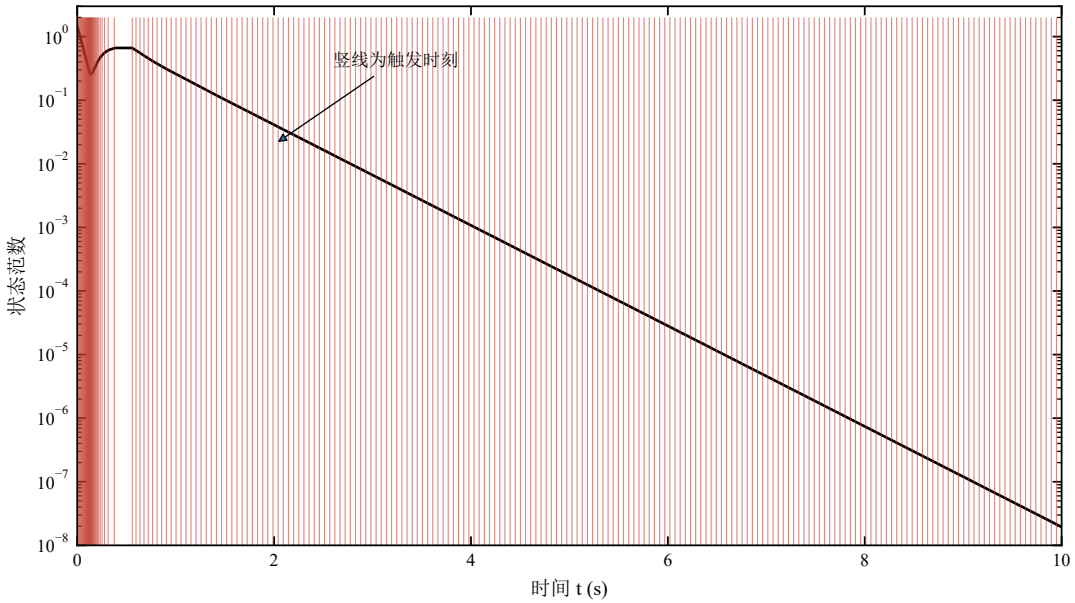


图 75-2 状态范数与触发时刻：初始段触发密集，随状态衰减逐渐稀疏，末段仍保持有限密度

第一，初始段密集。 前 0.2 s 内触发了几十次，因为状态从 1.414 快速衰减，误差相对增长很快。这一段是“信息量最大”的阶段，密集触发是合理的。

第二，中段稀疏。 1 s 到 3 s，触发间隔拉长到 0.05–0.1 s，状态已经降到 10^{-2} 量级，变化缓慢。

第三，末段不归零。 状态降到 10^{-6} 以下时仍在触发，间隔不再变长。这是相对阈值的特性：只要状态非零，误差总会在有限时间内达到 $\sigma\|\boldsymbol{z}\|$。绝对阈值不会这样，它会在状态小于 ε 后停止触发——这既是优点（省通信）也是缺点（留下静差）。

这三段对应三种不同的工程关注点。初始段要保证判决器算得过来（最坏突发率），中段决定平均通信量（带宽预算），末段决定稳态精度与是否抖动（噪声敏感度）。

图中还有一个不易察觉的细节：**竖线的间距不是逐渐变大的，而是“整体变大、局部仍有短间隔”。** 中段虽然平均间隔 0.05–0.1 s，但偶尔会出现两个靠得很近的触发。这些短间隔来自状态轨迹的“转弯”——两个模态以不同速率衰减，合成轨迹在某些时刻方向变化快，误差的增长率突然升高。这类局部短间隔是平均指标看不见的，也是最小间隔必须单独统计的原因。

75.4.3 基准实测

$\sigma = 0.10$ 的完整结果在表 75-1 已经给出：224 次触发、平均间隔 0.0446 s、最小间隔 0.0041 s、代价 1.1940（连续的 1.007 倍）。

把这个结果与“同等最坏间隔的周期采样”对比，可以得到事件触发最有力的一个论据。若周期采样的周期取事件触发的最小间隔 0.0041 s，则 10 s 内要采 2456 次，代价 1.1860；事件触发只用 224 次就达到 1.1940。**通信量差 11.0 倍，代价只差 0.7%。**

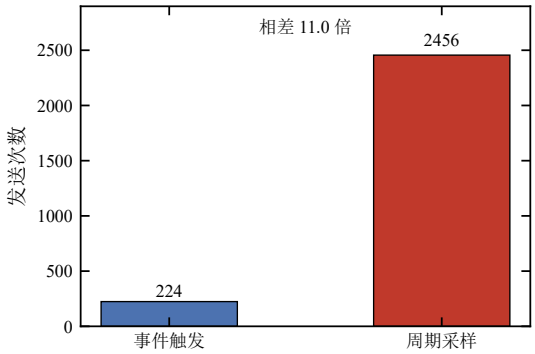


图 75-5 同等最坏间隔下的发送次数：事件触发 224 次，周期采样 2456 次

这个对比之所以有力，是因为它把“最坏情况”对齐了。批评者常说“事件触发的平均间隔好看，但最坏情况可能很密”，这个对比回答得很干脆：**即使按最坏情况配周期采样，事件触发仍然省 11 倍。** 代价是事件触发需要一个能处理突发的判决器与网络，而周期采样的负载是恒定的、容易预测。

若换成“同等平均间隔”的对比（$h = 0.0446$ s，225 次），周期采样的代价是 1.2213，事件触发是 1.1940，差 2.24%。这个对比更保守，也更接近“只看平均值”的直觉。两个对比都成立，取哪个取决于你想强调什么：前者说明“最坏情况也不吃亏”，后者说明“平均下来确实更好”。

75.4.4 适用条件

连续事件触发适合三个条件同时满足的场合。

状态可连续测量。 若传感器本身有 100 ms 的更新周期，判决器再快也没用，实际退化成周期事件触发。

判决器靠近传感器。 判决必须在数据上传之前完成。若判决器在云端，测量数据还是要传上去，通信量一点没省。

能接受突发的网络负载。 初始瞬态的事件率可能是平均值的十倍，网络要能吃下这个尖峰。

三条里任何一条不满足，就该考虑自触发或周期事件触发。本章的实测数据对三种实现都给了，读者可以按自己的硬件条件对号入座。

75.4.5 间隔的统计特征

224 次触发、平均间隔 0.0446 s、最小间隔 0.0041 s，三个数放在一起能读出间隔分布的形状。

若所有间隔都相等，10 s 除以 224 恰好是 0.0446 s，与平均值一致。这说明平均值本身没有偏差，但**最小间隔只有平均值的 9.2%**（$0.0041 / 0.0446 = 0.092$）。一个右偏的分布，少数极短间隔把均值拉低，多数间隔略大于均值——这是事件触发的典型间隔分布。

这个分布形状对工程有两条直接含义。**第一，按平均间隔设计缓冲区会溢出。** 若控制器侧按 0.0446 s 的节奏设计队列深度为 1，遇到 0.0041 s 的连续两次触发，第二包必然被覆盖或丢弃。缓冲深度至少要按“最小间隔内的最大包数”算，本章是 1 包/4 ms，看起来很小，但若网络把包排队，实际需要 3–5 个包位。**第二，按平均间隔估算 CPU 占用会低估峰值。** 判决器在初始段的唤醒率是平均值的十倍以上，若按平均值选 CPU，启动时会超时。

想进一步刻画分布，需要看间隔的直方图或分位数（比如 95 分位、99 分位）。本章的脚本输出了 `gaps` 数组（`run_event` 的返回值），读者可以自己画直方图。工程上建议至少记录 99 分位，因为它直接决定“最坏突发”的预算。

75.4.6 与连续控制的逐段对照

把事件触发与连续控制的差别按时间段拆开看，可以看清“性能损失发生在哪里”。

0–0.2 s：差别最大。 这一段状态从 1.414 快速衰减，连续控制每个 0.1 ms 都更新，事件触发平均每 44.6 ms 才更新一次（初始段更密）。指令的“慢半拍”在这段最明显，代价的差额主要在这里产生（§75.6.2 的分解）。

0.2–1 s：差别迅速缩小。 状态已经降到 0.1 以下，代价的绝对值本身就随状态平方缩小，同样比例的指令滞后造成的绝对损失随之变小。

1 s 之后：几乎无差别。 状态在 10^{-3} 以下，代价的绝对值本身就很小，两者的积分差可以忽略。此时事件触发仍在发送，但发送与否对代价已经没有影响——**这部分通信是“为保持闭环可观测性”付的费，不是为性能付的费。**

这个分段解释了一个常见疑问：“为什么事件触发的代价比只高 0.7%，却看起来触发得那么稀疏？”因为性能损失只集中在最初 0.2 s，而通信量的节省分布在全程。**用 0.2 s 的稍差换 10 s 的省通信，这笔交易在大多数场合是划算的。**

反过来也有一个警告：若工艺的关键指标是“初始段的超调或调节时间”，那么 0.7% 的整体代价差可能对应初始段 5%–10% 的局部差异，此时应该把 σ 调小（0.05 或 0.02），用更多通信换初始段的精度。**整体指标会掩盖局部指标，这是评价事件触发时最容易犯的错。**

75.4.7 连续监测的数值实现

“连续”在数值上是不可能的，本章用 0.1 ms 的步长逼近。这个逼近有两个精度问题，写代码时必须处理。

第一个问题：触发时刻的定位。 若在每个步长结束时判断“是否越界”，触发时刻最多晚一个步长。本章最小间隔 0.0041 s，一个步长 0.0001 s 占 2.4%，看起来不大，但触发时刻偏晚会让误差多累积一步，下一次触发提前，误差在多次触发间累积。本章的做法是**二分回退**：发现越界后，在当前步内二分若干次，找到误差恰好等于阈值的时刻，从那一刻重新起算。这样触发时刻的误差被压到远小于一个步长的水平（10^{-12} s 量级）。

第二个问题：代价积分的分段。 触发发生在步内时，这一步的代价要拆成两段：触发前用旧输入，触发后用新输入。若不拆，整步都按旧输入算，会高估代价（旧输入是“错的”那个）；整步都按新输入算，会低估。本章的代码在触发分支里用 τ 而不是 h 累加代价，就是为了这个。

还有一个实现选择：**判断用“$\geq$”还是“$>$”。** 理论上无差别，数值上有差别：用“$\geq$”时，误差恰好等于阈值也会触发，可能在浮点舍入下反复触发同一时刻；用“$>$”则不会。本章用“$\geq$”，因为在 $\sigma > 0$ 时最小间隔有正下界（式 75-13），不会真的反复触发。若用绝对阈值，必须改成“$>$”并加容差。

这三个细节都不影响定性结论，但影响数字的第三位。**若你的仿真结果与本章差 1% 以上，先查这三处。**

75.5 离散情形

75.5.1 周期检查的实现

周期事件触发把“连续监测”换成“每 h 秒醒一次”，醒来时做一次比较（式 75-16），越界才发送。它的实现最简单：一个定时器、一个比较、一个发送函数。

但它引入了一个新的失效模式：**漏检**。若两次检查之间误差冲过阈值又回落到阈值以下，这次越界就被完全忽略了。漏检的概率与 h 成正比，与误差的变化速度成正比。本章的实测显示，h 从 0.01 s 增到 0.05 s，发送次数从 183 降到 102（少了 44%），代价从 1.1906 升到 1.2348（升 3.7%）。少掉的 81 次里有相当一部分就是漏检。

75.5.2 采样周期扫描

表 75-3 给出六个采样周期的完整结果。

h (s)	检查次数	发送次数	平均间隔 (s)	代价 J	相对连续
0.01	1000	183	0.0544	1.1906	1.004
0.02	500	173	0.0577	1.1997	1.012
0.05	200	102	0.0980	1.2348	1.041
0.10	100	99	0.1010	1.4486	1.222
0.20	50	50	0.2000	66.1526	55.79
0.50	20	20	0.5000	5.23×10^{37}	失控

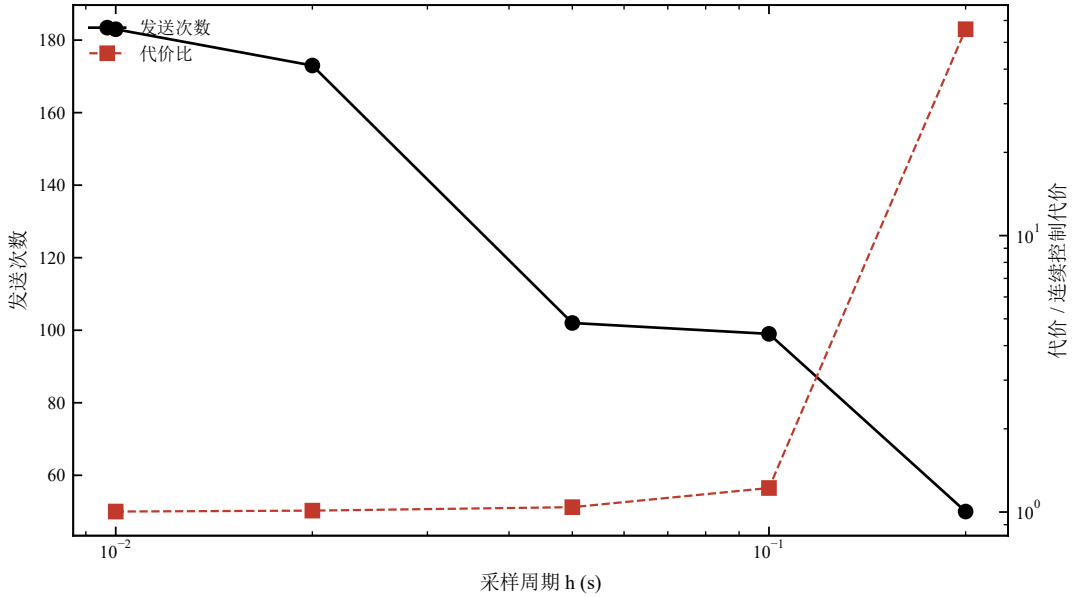


图 75-7 采样周期对发送次数与代价的影响：h 超过 0.1 s 后代价急剧上升

表 75-3 有一条很清楚的分界：$h \le 0.05$ s 时代价比在 1.04 以内，$h = 0.10$ s 时跳到 1.222，$h = 0.20$ s 时彻底失控。分界的位置不是巧合，它与闭环最快极点 -5.3272 对应的时间常数 0.188 s 有关：$h = 0.10$ s 约为它的一半，$h = 0.20$ s 已经超过它。

另一个值得注意的现象是 $h = 0.10$ s 时发送次数（99）与检查次数（100）几乎相等——**这意味着几乎每次都越界，周期事件触发退化成了纯周期采样**。这不是因为阈值太小，而是因为 h 已经大到“每次检查时误差必然越界”。此时事件触发机制名存实亡，性能当然差。

$h = 0.20$ s 与 $h = 0.50$ s 的两行不是“性能差”，而是“闭环不稳定”。$h = 0.20$ s 的代价 66.15 说明状态在 10 s 内涨到了几十倍；$h = 0.50$ s 的 5.23×10^{37} 是数值溢出前的量级，物理上早就烧毁了。这两行保留在表里，是为了说明“采样周期超过闭环时间常数”的后果有多严重。

75.5.3 与纯周期控制的区别

周期事件触发与纯周期采样只差一个 if，但性能差很多。在 $h = 0.05$ s 这一档，纯周期采样的代价是 1.2316（201 次发送），周期事件触发是 1.2348（102 次发送）。**代价几乎相同，发送次数减半。**

这个对比说明：事件触发机制的价值不在“用了更聪明的算法”，而在“把不需要的发送省掉了”。它的实现成本是一个 if 语句，收益是一半的通信量。对已有 PLC 的现场，这是最容易摘到的果子。

但要提醒一句：1.2348 与 1.2316 的顺序说明周期事件触发略差。差的原因是漏检——有些该发的没发。若把 h 减小到 0.01 s，代价降到 1.1906，比纯周期采样好，但发送次数升到 183。**这三档（0.01 / 0.05 / 纯周期）构成一条完整的交换曲线，现场按带宽选一档即可。**

75.5.4 实现细节

四个细节决定实现能不能用。

检查时刻对齐。 检查必须在采样完成的瞬间进行，不能在采样后延迟若干毫秒。若传感器有滤波，滤波器的群延迟会让“当前值”实际上是几百毫秒前的值，误差算错，触发判据失效。

范数的选择。 式 (75-16) 用的是欧氏范数。若两个状态量纲差异大（一个是温度、一个是浓度），必须先用对角矩阵归一化，否则量纲大的那个主导判据，另一个实际上被忽略。

发送内容的打包。 至少要打包“时间戳 + 状态值 + 序号”。时间戳用于接收端估计延迟，序号用于检测丢包。只发状态值是最省字节的做法，但现场排障时几乎无法定位问题。

保持逻辑的位置。 保持应该在接收端（控制器侧），而不是发送端。若在发送端保持，网络丢包时发送端不知道，会以为自己一直在更新。

75.5.5 三个陷阱

陷阱一：把检查周期当成采样周期。 周期事件触发的 h 是“检查周期”，不是“发送周期”。发送周期是随机的，平均值约 0.098 s（$h = 0.05$ 时），但可能短到 0.05 s、长到几百毫秒。若下游按固定周期设计缓冲区，会溢出或饿死。

陷阱二：用绝对阈值配周期检查。 这是最坏的组合：绝对阈值在零点附近本来就容易抖动，周期检查又让抖动变得不规则，两者叠加会让发送次数在“很少”与“极多”之间跳变。要么用相对阈值，要么加最小间隔，不能两个都不做。

陷阱三：忽略检查本身的算力。 $h = 0.01$ s 意味着每秒 100 次检查，每次要做两次范数计算与一次比较。对 32 位单片机是轻松的，但对靠电池工作的无线节点，每秒 100 次唤醒的功耗可能超过通信本身。此时应该换成自触发。

75.5.6 漏检的代价怎么估

表 75-3 的六行里，$h = 0.01$ 与 $h = 0.05$ 的差最能说明漏检的代价：发送次数从 183 降到 102（少 81 次），代价从 1.1906 升到 1.2348（升 3.7%）。**这 81 次“少发”里，大部分是漏检，小部分是合理的节省。**

怎么区分两者？看代价。若少发的都是“不该发的”（状态已经很小的末段），代价不会升；若少发的都是“该发的”（状态还大、误差已越界但没被检查到），代价会明显升。3.7% 的升幅说明漏检占多数。

漏检的代价与 h 的关系不是线性的。h 从 0.01 到 0.02（翻倍），代价只升 0.8%（1.1906 到 1.1997）；从 0.02 到 0.05（2.5 倍），代价升 2.9%；从 0.05 到 0.10（2 倍），代价升 17.3%。**拐点在 0.05–0.10 之间，与“闭环时间常数的一半”这个判据吻合。** 在拐点之前，漏检是“偶发”的；过了拐点，漏检变成“系统性”的——每次检查时误差都已经远远越界，控制量长期处于“该更新而没更新”的状态。

这个非线性给调参一个明确指引：**h 应该取在拐点左侧，且离拐点有足够距离。** 本章 $h = 0.05$ 是拐点的一半，代价升 4.1%，是“能用”与“好用”的分界。若现场对性能敏感，取 0.01；若对通信敏感、能接受 4% 的性能损失，取 0.05。**不要把 h 取在拐点附近（0.08–0.10），那里的代价对 h 极其敏感，工况稍有变化就会掉到拐点右侧。**

75.5.7 定时器抖动

周期检查依赖定时器。真实定时器有抖动：标称 10 ms 的周期，实际可能在 9–11 ms 之间波动。抖动对事件触发的影响比对周期采样大，因为事件触发的“检查时刻”本身参与判决。

抖动的影响可以分两档看。**抖动小于 h 的 10%**（本章 $h = 0.01$ 时即 1 ms），检查时刻的偏差相对于平均触发间隔（0.0544 s）只有 2%，统计量几乎不受影响。**抖动大于 h 的 30%**，实际检查周期在 $0.7h$ 到 $1.3h$ 之间跳变，相当于在一个范围内随机取 h，此时的发送次数与代价应该按最坏的那个 h 估算。

工程上的处理很简单：把定时器抖动写进 h 的选型里。若定时器抖动 3 ms，h 就不要取 0.01（抖动占 30%），而应取 0.05（占 6%）。**这也解释了为什么工业 PLC 上的周期事件触发通常取 10–50 ms，而不是 1 ms——不是算力不够，是定时精度不够。**

还有一个更隐蔽的问题：**多节点的时间基准不同步。** 若判决器与控制器不在同一个时钟域，“检查时刻”与“控制时刻”会漂移，判决器认为的“当前状态”与控制器认为的“当前状态”不是同一时刻。对相对阈值，这个漂移只造成微小的误差；对绝对阈值，漂移会直接变成判据偏差。**多节点场合应统一时间基准，或只用相对阈值。**

75.5.8 与连续实现的交叉检验

两套代码（`run_event` 与 `run_discrete`）算的是同一个机制，只是检查频率不同。它们应该在极限下一致，这个一致性可以用来交叉检验代码。

极限关系是：**当 $h \to 0$ 时，周期事件触发退化为连续事件触发。** 本章 $h = 0.01$ s 的发送次数是 183、代价 1.1906，连续事件触发是 224、1.1940。次数还差 41 次，因为 0.01 s 相对平均间隔 0.0544 s 仍有 18% 的漏检；代价已经非常接近（差 0.3%），说明漏掉的触发大多价值不大。

这个对比给了一个重要的判断：**周期事件触发的“次数少”不一定是坏事。** $h = 0.01$ 时少发 41 次，代价反而更低（1.1906 对 1.1940）。原因是连续实现里那些“极短间隔”的触发，贡献的信息量很小，去掉反而让控制量更平滑。这与 $T_{\min} = 0.01$ 的结论（§75.7.2）是同一个现象的两条路径：**一个靠“禁止太密的触发”，一个靠“看不见太密的触发”，效果相似。**

若你的两套代码在 $h = 0.01$ 时给出完全不同的结果（比如次数差一倍以上），说明至少有一套错了。常见错误是：周期实现的保持逻辑写成“用本次检查的测量值”，而连续实现写成“用上次触发的值”；两者在 h 小时的差别会被放大。**交叉检验是发现这类错误最快的方法。**

75.6 稳定性与性能

75.6.1 误差的三个来源

闭环里的误差 e 不是单一来源，它由三部分叠加：**采样滞后**（两次触发之间状态在变，指令没变）、**网络延迟**（指令晚到）、**丢包**（指令没到，用更旧的值）。三者对稳定性的影响程度差一个数量级，必须分开看。

采样滞后是“结构性的”，无法消除，只能通过缩小阈值把它压小。它对应式 (75-4) 的扰动项 $-\boldsymbol{B}\boldsymbol{K}e$，大小由 σ 直接控制。

网络延迟是“时间性的”，它把扰动项的时间戳整体后移。在频域上，延迟 θ 引入相位滞后 $-\omega\theta$，当相位滞后积累到超过相位裕度时闭环失稳。这解释了为什么延迟的影响是“突然”的：0.11 s 时代价 8.685，0.12 s 时 249.7，0.13 s 时 1.235×10^{6}。三个数之间不是渐变，是崩塌。

丢包是“随机性的”，它让扰动项偶尔变大。但因为丢包不改变系统的结构，只改变输入的新旧程度，所以它的影响是渐进的：50% 丢包代价升 59%，但仍稳定。

三者的量级差异可以用一个比喻：采样滞后像“鞋子略大”，延迟像“路面突然塌陷”，丢包像“偶尔踩到石子”。工程上应该把大部分精力放在延迟上，而文献里讨论最多的是阈值——这就是理论与实践的落差所在。

75.6.2 代价分解

连续控制的代价是 1.1858，事件触发（$\sigma = 0.10$）是 1.1940，多出的 0.0082 就是“用旧指令”的代价，占总代价的 0.69%。这个数字小得令人意外，原因是本章的闭环极点很快（-5.3272），状态在 0.2 s 内就衰减到很小，之后即使指令很旧也没关系。

把 0.0082 拆开看，它主要由初始 0.3 s 贡献。前 0.3 s 状态大、变化快，误差迅速累积，控制量每次都“慢半拍”；0.3 s 之后状态已经很小，误差的绝对值也小，贡献可以忽略。**事件触发的性能损失集中在瞬态，不在稳态。** 这个结论对调参很有指导意义：若工艺关心的是超调与调节时间，σ 要取小；若只关心稳态精度，σ 可以取大。

与周期采样对比，$h = 0.05$ s 的代价 1.2316 比事件触发高 0.0376，是事件触发损失的 4.6 倍。差距的来源不是“周期采样更笨”，而是“周期采样在状态很小时仍然按固定周期发送、且发送的是最新值，看似更准，实际上在状态大时反而不够

密”。具体说：$h = 0.05$ s 在初始段（状态从 1.414 快速衰减）每 0.05 s 才更新一次，太稀疏；在末段（状态已很小）每 0.05 s 更新一次，太浪费。事件触发自动把密度分配给了需要的地方，这就是它省通信又不损性能的根本原因。

75.6.3 稳定性边界

四条边界，逐条给出实测。

阈值边界。 表 75-4 是 σ 从 0.02 到 0.70 的完整扫描。

σ	触发次数	平均间隔 (s)	最小间隔 (s)	代价 J	相对连续
0.02	1108	0.0090	0.0008	1.1860	1.000
0.05	444	0.0225	0.0021	1.1869	1.001
0.10	224	0.0446	0.0041	1.1940	1.007
0.20	114	0.0885	0.0079	1.1956	1.008
0.30	94	0.1075	0.0115	1.3153	1.109
0.50	52	0.1930	0.0181	1.2875	1.086
0.70	69	0.1461	0.0240	3.6941	3.115

表 75-4 里有两处不符合“单调”的直觉。$\sigma = 0.30$ 的代价（1.3153）比 $\sigma = 0.50$（1.2875）更高，$\sigma = 0.70$ 的触发次数（69）比 $\sigma = 0.50$（52）更多。这两处都不是数值噪声，而是闭环在“误差大、修正猛”之间的非线性互动：阈值放大后，误差累积得更久，触发时的修正更猛，修正过头又产生反向误差，形成一种低频的“过冲-回调”循环。这种循环在 $\sigma = 0.50$ 附近恰好被阻尼掉，到 $\sigma = 0.70$ 又变得明显。

这也说明“阈值越大越省通信”是错的。 通信量在 $\sigma = 0.50$ 处最小（52 次），再大反而回升。工程上若只按“最大化间隔”调参，会调到一个性能很差的位置。

延迟边界。 表 75-5 是延迟从 0 到 0.15 s 的扫描。

θ (s)	触发次数	代价 J	相对连续	峰值
0.00	224	1.194	1.01	1.414
0.02	219	1.224	1.03	1.414
0.05	216	1.398	1.18	1.414
0.08	509	2.08	1.75	1.414
0.10	1957	3.967	3.35	1.444
0.11	1831	8.685	7.32	1.647
0.12	1685	249.7	211	4.659
0.13	1592	1.235×10^{6}	1.04×10^{6}	720.3
0.14	1463	1.351×10^{9}	1.14×10^{9}	3.51×10^{4}
0.15	1391	9.364×10^{11}	7.9×10^{11}	1.16×10^{6}

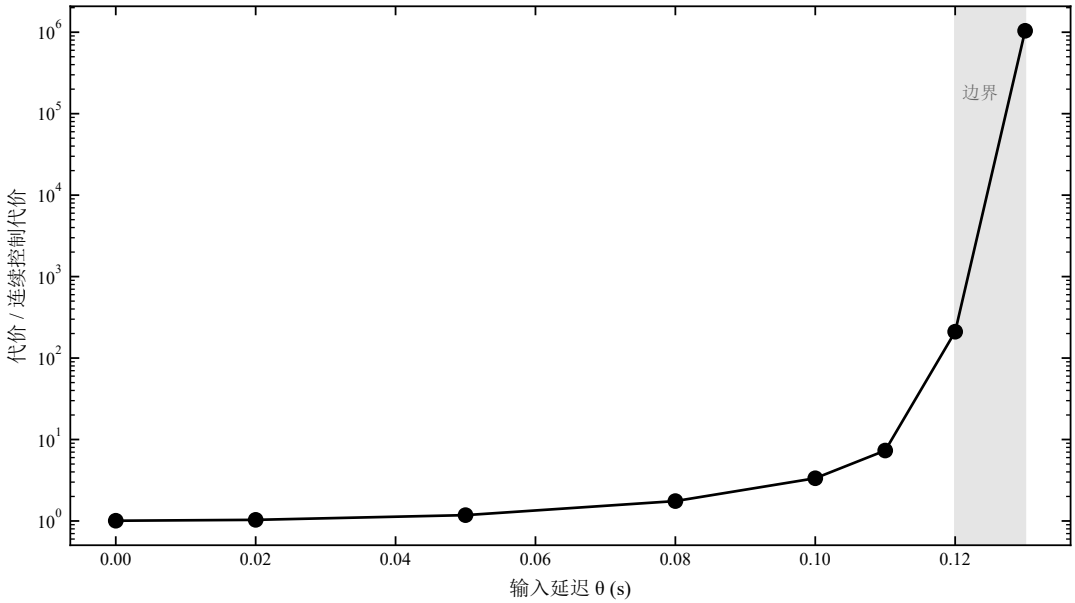


图 75-8 输入延迟对代价的影响：0.12 s 之后代价按数量级攀升

表 75-5 有两个现象值得单独说。**第一，延迟到 0.08 s 时触发次数突然从 216 跳到 509。** 这不是“事件触发变勤快了”，而是闭环开始振荡：延迟引起的相位滞后让状态出现小幅高频抖动，误差很快越界，触发频繁。此时代价只升到 1.75 倍，还不算坏，但触发次数翻倍，通信预算会被打穿。

第二，0.13 s 之后代价按数量级增长（10^6、10^9、10^{11}）。 这是典型的延迟失稳：闭环极点越过虚轴后，状态按指数增长，代价随时间的指数平方增长。

延迟边界的位置可以这样记：0.12 s 约为闭环最快时间常数 0.1877 s 的 0.64 倍。工程经验值“不超过三分之一”（0.063 s）更保守，对应代价比约 1.3。**若现场延迟在 0.05–0.08 s，属于可用但要注意；超过 0.10 s，必须做延迟补偿（见第 47、107 章的 Smith 预估）。**

丢包边界。 表 75-6 是周期采样（$h = 0.05$ s）与事件触发（$\sigma = 0.10$）在丢包下的对比。

丢包率	周期：发送/丢包	周期代价	事件：发送	事件代价
0.00	200.0 / 0.0	1.2316	223.0	1.1940
0.10	200.0 / 17.4	1.3151	—	—
0.20	200.0 / 35.0	1.3231	280.2	1.1940
0.30	200.0 / 55.2	1.8671	—	—
0.40	200.0 / 75.0	1.8677	—	—
0.50	200.0 / 93.6	1.9578	436.2	1.1940

表 75-6 的对比非常鲜明：周期采样的代价从 1.2316 涨到 1.9578（升 59.0%），事件触发一直是 1.1940，代价完全不动，只是发送次数从 223 增到 436（增 95.5%）。

原因是 §75.1.3 说过的“自带重传”：丢包后接收端仍用旧值，误差继续累积，仍然越界，下一拍就重发。所以事件触发把“丢包”转化成了“多发几次”，代价几乎不变。周期采样没有这个机制，丢了就只能等下一个周期，指令滞后一整个 h，代价自然上升。

但这个优势有前提：**网络必须允许“连续突发”**。若网络有速率限制（比如每个节点每秒最多 10 包），事件触发的重发会被限流，效果立刻退化到周期采样的水平。落地时要核对这一点，见 §75.10.2。

噪声边界与芝诺。 表 75-7 是绝对阈值 + 噪声 0.01 的结果。

阈值 ε	触发次数	最小间隔 (s)	平均间隔 (s)	代价 J
0.100	61	0.00530	0.1557	1.2421
0.050	112	0.00200	0.0884	1.1902
0.020	13520	0.00010	0.0007	1.1858
0.010	60789	0.00010	0.0002	1.1858
0.005	88316	0.00010	0.0001	1.1858

阈值从 0.050 降到 0.020，触发次数从 112 跳到 13 520，涨了 120 倍；最小间隔从 0.002 s 压到 1.0×10^{-4} s，正好等于仿真步长。**这说明触发频率已经被数值步长截断，物理上等于"每个步长都发"，即芝诺状态。**

代价在芝诺状态下反而"最好"（1.1858，等于连续控制），因为没有采样滞后。这提醒读者：**不能用代价来检测芝诺。**芝诺在代价指标上看起来是完美工况，代价全在通信侧——13 520 次发送相当于 1352 次/s，任何真实网络都撑不住。

判据很简单：**若实测最小间隔等于（或接近）你的仿真步长或程序扫描周期，就已经进入芝诺，必须加最小间隔或改用相对阈值。**

75.6.4 指标选择

本章用了三个指标：代价 J、发送次数、最小间隔。三者不能互相替代，各自回答不同的问题。

代价 J 回答"控制得好不好"。它是连续控制的 1.000 到 3.115 倍，超过 1.5 倍就说明性能明显退化。它的缺点是看不出芝诺——芝诺时它反而最好。

发送次数 回答"网络扛不扛得住"。它与代价同向变化（想省就得多付代价），两者的曲线构成 §75.9.1 的 Pareto 前沿。它看不出突发：平均 224 次不等于每秒 22 次，初始段可能每秒 200 次。

最小间隔 回答"实现来不来得及"。它直接对应判决器与网络的最小时延要求。它看不出平均负载。

三个指标要一起看。本章的做法是：先看代价是否可接受（≤ 1.2），再看发送次数是否在带宽内，最后看最小间隔是否大于硬件时延。三项都过，才算可用。

75.6.5 代价比 1.007 到底意味着什么

1.007 这个数字小到容易被当作"没有差别"，但它的含义要放在二次型代价的定义里读。

代价 $J = \int (\boldsymbol{z}^{\mathsf{T}} \boldsymbol{Q} \boldsymbol{z} + R w^2) \mathrm{d}t$ 是能量的积分。代价比 1.007 意味着"多用 0.7% 的控制能量"，在阶跃响应上大致对应"调节时间延长百分之一量级"。这个量级用肉眼看不出来，用仪表也未必量得出，**所以可以说事件触发在 $\sigma = 0.10$ 时对本章的对象几乎无性能损失。**

但这个"几乎无损失"依赖一个前提：闭环足够快。本章最快极点 -5.3272，时间常数 0.188 s，10 s 内衰减 53 个时间常数。如果闭环慢十倍（时间常数 1.88 s），同样的 $\sigma = 0.10$、同样的绝对误差，相对影响会大得多——因为状态在"较大"的区间停留更久，误差累积的时间更长。

由此得到一条判据：**闭环越快，事件触发越"便宜"。**这条判据与直觉相反（直觉是"快系统更需要密采样"），但数据支持它：快系统的状态迅速衰减，误差扰动项的作用时间短，积分下来损失小。慢系统则相反，同样的相对阈值会带来更明显的性能损失。第 74 章的分布式参数对象比本章慢，若换成事件触发，代价比会更接近周期采样。

75.6.6 三个指标在不同场合的权重

三个指标不是等权的，权重取决于应用场景。可以分三类。

工艺过程（温度、浓度、液位）。 权重顺序是代价、发送次数、最小间隔。这类过程对精度敏感（代价重要），对通信不敏感（有线、带宽足），对突发也不敏感（判决器算力足）。此时应该把 σ 调小，用通信换精度。

无线传感网（电池供电）。 权重顺序是发送次数、最小间隔、代价。电池寿命是第一约束，通信次数直接决定寿命；代价只要不失控（≤ 1.5）就可以接受。此时应该把 σ 调大、用自触发、加长心跳。

运动控制（伺服、机器人）。 权重顺序是最小间隔、代价、发送次数。这类系统对时序敏感，最怕“该更新时没更新”，最小间隔必须远小于机械时间常数；通信通常是有线且带宽足，不是瓶颈。此时事件触发的收益有限，多数场合直接用高频周期采样。

把自己的场景对号入座，就知道该优化哪个指标。**三者都想要，等于没有优化目标。**

75.7 参数意义与作用

75.7.1 阈值 σ

σ 是本章最重要的参数，它同时决定通信量、代价、最小间隔三件事。

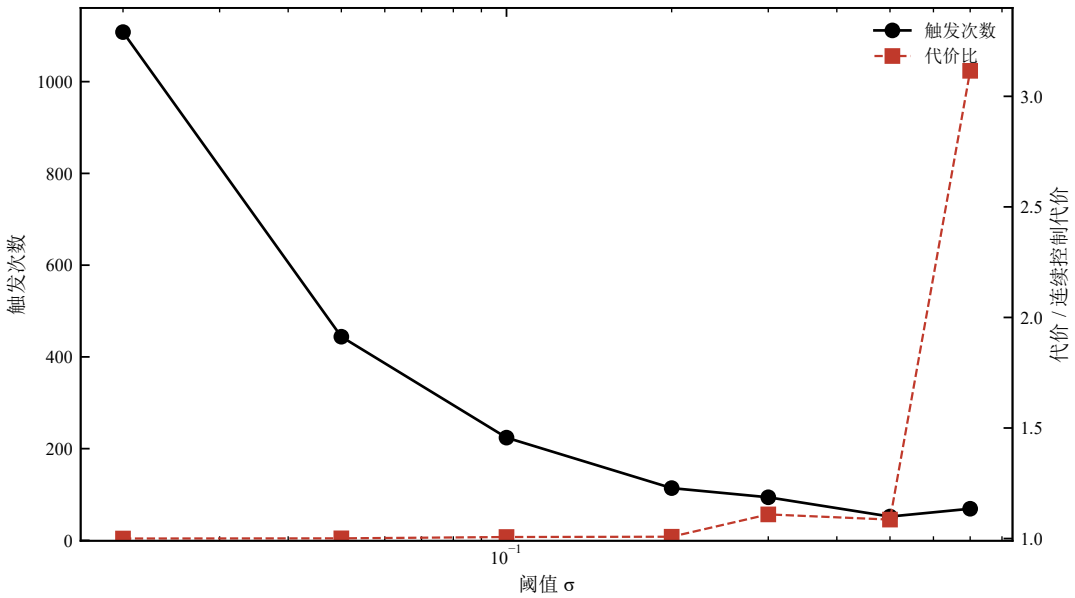


图 75-4 阈值扫描：触发次数随 σ 单调下降，代价比在 0.2 到 0.3 之间开始抬升

图 75-4 把表 75-4 画成双轴图。左轴（黑线）是触发次数，随 σ 从 0.02 增到 0.50 单调下降（1108 到 52），到 0.70 又回升到 69。右轴（红色虚线）是代价比，在 $\sigma \leq 0.20$ 时几乎平（1.000–1.008），在 0.30 处抬头（1.109），0.70 处到 3.115。

这条曲线的形状给出了调参的“甜区”：$\sigma \in [0.05, 0.20]$。这一段里，通信量从 444 降到 114（省 74%），代价只从 1.001 升到 1.008（多 0.7%）。再往右，省下的通信量越来越小而代价上升加快；再往左，通信量翻倍而代价几乎不变，纯属浪费。

阈值还有一个容易忽略的作用：**它决定了稳态的“抖动底噪”**。相对阈值下，稳态时状态很小，触发间隔会趋于一个由噪声与状态共同决定的平衡值；若 σ 小于噪声与状态的比值，稳态会进入芝诺。本章在噪声 0.01 下 $\sigma = 0.02$ 仍安全（代价 1.000、触发 1108 次），说明还有余量；若噪声升到 0.05，$\sigma = 0.02$ 就会抖动，此时必须把 σ 提到 0.1 以上。

75.7.2 最小间隔 $T_{\min}$

最小间隔是“强制驻留时间”：两次触发之间至少间隔 $T_{\min}$，即使条件已经越界也不发。它唯一的用途是防芝诺，代价是“该发的时候发不出去”。

$T_{\min}$ (s)	触发次数	最小间隔 (s)	代价 J
0.00	224	0.0041	1.1940
0.01	209	0.0099	1.1906
0.02	199	0.0199	1.1982
0.05	190	0.0499	1.2344
0.10	100	0.0999	1.4465

表 75-8 有一个反直觉的结果：$T_{\min} = 0.01$ s 的代价（1.1906）比不加约束（1.1940）还低。原因是强制间隔挡掉了初始段几次“过早”的触发，那些触发携带的信息量很小、却让控制量频繁跳变，去掉反而更平滑。

$T_{\min}$ 超过 0.05 s 后代价明显上升（1.2344），0.10 s 时到 1.4465。原因是 $T_{\min}$ 已经接近平均间隔（0.0446 s），它不再是“防抖”而是“限速”，把该有的更新也挡掉了。**经验规则：$T_{\min}$ 取平均间隔的四分之一到二分之一。** 本章平均 0.0446 s，$T_{\min}$ 取 0.01–0.02 s 合适。

表 75-8 还有一处细节值得注意：触发次数随 $T_{\min}$ 增大而单调下降（224→209→199→190→100），但下降的幅度不均匀。0 到 0.05 s 只降了 15%，0.05 到 0.10 s 却降了 47%。原因是 $T_{\min} = 0.05$ s 时它刚好处在“多数间隔都大于它”的位置，只挡掉了少数极短间隔；到 0.10 s 时它已经大于多数间隔，开始成批地挡掉正常更新。**这个转折点就是“平均间隔”，$T_{\min}$ 一旦超过平均间隔，性质就从“防抖”变成“降采样”。**

$T_{\min}$ 还有一个隐含作用：**它把“判决器过载”变成“可控的降级”。** 若判决器算不过来，触发会丢失，系统表现为“随机漏发”；加了 $T_{\min}$ 之后，触发被主动限制在判决器能处理的速率内，系统表现为“有序降级”。两者的平均通信量可能相同，但后者可预测、可诊断。**工程上宁可有序降级，不要随机丢失。**

75.7.3 采样周期 h

h 只在周期事件触发与“离散实现”中出现。它决定检查的密度，进而决定漏检率。

表 75-3 已经给出全部数据。要点有三：$h \le 0.05$ s 时性能损失在 4% 以内；h 接近闭环时间常数的一半（0.10 s）时性能跳变；h 超过时间常数（0.20 s）时失稳。

h 与 σ 有一个交互：若 h 很小（0.01 s），σ 可以取小一些（0.05），因为密集检查本身就能及时捕捉越界；若 h 较大（0.05 s），σ 要适当放大（0.10 以上），否则每次检查都越界，退化成纯周期采样。本章的基准 $h = 0.05$、$\sigma = 0.10$ 就是按这个逻辑配的。

这个交互可以用“每次检查的期望越界概率”来理解。若 h 内误差的增长量远小于阈值，检查时大概率不越界，此时 h 是“有效的检查”；若增长量远大于阈值，检查时几乎必然越界，h 就退化成采样周期。**判据是 h 与“误差从零长到阈值所需时间”的比值**，本章这个时间在 0.004–0.05 s 之间（取决于状态大小），所以 $h = 0.05$ 已经接近上限。

还有一个容易忽略的耦合：**h 影响代价的形状，不只是大小。** $h = 0.20$ s 时代价 66.15，不是“性能差 66 倍”，而是“闭环已经不稳定、状态涨到几十倍”，代价是状态的平方积分，所以数值被放大得更厉害。读表时要把“性能退化”与“失稳”分开：1.4 以下是退化，10 以上基本就是失稳了。

75.7.4 自触发的模型倍率

自触发用模型预测下一次触发时刻。模型倍率 κ 表示“预测模型与实际对象的增益比”：$\kappa = 1$ 完全准确，$\kappa < 1$ 表示低估。

方案	触发次数	平均间隔 (s)	代价 J
事件触发	224	0.0446	1.1940
自触发 $\kappa = 1.00$	237	0.0423	1.1916
自触发 $\kappa = 0.90$	211	0.0475	1.1919
自触发 $\kappa = 0.70$	161	0.0621	1.2031
自触发 $\kappa = 0.50$	109	0.0919	1.2015
自触发 $\kappa = 0.40$	103	0.0967	1.3194

表 75-9 揭示了自触发的一个有趣性质：**低估模型反而能减少触发次数。** $\kappa = 0.50$ 时只触发 109 次（事件触发是 224 次，少一半），代价只升到 1.2015（升 0.6%）。原因是低估让预测的“越界时刻”往后推，实际误差超过了 σ 也没关系，因为系统本身有裕度。

但 $\kappa = 0.40$ 时代价跳到 1.3194，说明余度用尽了。**κ 可以往低估方向走，但只能走到“实际误差比阈值大两倍左右”的位置。** 工程上建议 κ 取 0.8–0.9，既省一点通信，又留足余量。

$\kappa > 1$（高估模型）会让触发变密，没有好处，工程上不会用。所以自触发的调参方向是单向的：从 1.0 往下试，直到代价开始上升。

75.7.5 丢包率 p

丢包率是网络属性，不是可调参数，但它决定了设计余量。表 75-6 已经给出数据：周期采样在 50% 丢包下代价升 59.0%，事件触发几乎不变。

设计时的用法是：先测现场的实际丢包率，再按最坏情况（通常是平均值的两倍）留余量。若最坏丢包率超过 30%，周期采样必须加“重复发送”或“预测补偿”，否则性能不可接受；事件触发因为自带重发，容忍度更高，但要注意它的发送次数会随丢包率线性上升（$p = 0.5$ 时几乎翻倍）。

表 75-6 里事件触发的发送次数随丢包率的增长很有规律：无丢包 223 次，20% 时 280 次，50% 时 436 次。按“每个包有 p 的概率丢，丢了就重发”估算，发送次数应该按 $1/(1-p)$ 放大：$1/0.8 = 1.25$ 倍（223×1.25=279，实测 280），$1/0.5 = 2$ 倍（223×2=446，实测 436）。**实测与理论估算相差不超过 2.2%，说明“自带重传”的机制确实按这个规律工作。**

这条规律给了一个直接的容量算法：**事件触发的通信量按 $N/(1-p)$ 预算，其中 N 是无丢包时的次数。** 若现场最坏丢包率 20%、带宽只够 300 次，那么无丢包时的设计值不能超过 $300 \times 0.8 = 240$ 次，本章的 224 次正好卡在这个边界上。

周期采样没有这个放大，但它的代价会随丢包率上升。两者的差别可以这样记：**丢包对事件触发是“带宽问题”，对周期采样是“性能问题”。** 前者可以靠加带宽解决，后者不能。

75.7.6 噪声水平

噪声决定阈值的下限。判据是：**阈值对应的允许误差必须明显大于噪声的标准差**，否则误差在噪声驱动下反复越界。

本章噪声 0.01、状态初值 1.414，比值 0.7%。用相对阈值 $\sigma = 0.10$ 时，允许误差是 $0.1 \times 1.414 = 0.141$，是噪声的 14 倍，安全。当状态衰减到 10^{-2} 时，允许误差变成 10^{-3}，只有噪声的十分之一，此时误差由噪声主导，触发间隔不再由状态决定——但因为状态已经很小、代价可以忽略，这个“失衡”不影响总体性能。

若噪声升到 0.05，同样的 $\sigma = 0.10$ 在状态小于 0.5 时就会出现噪声主导的抖动。对策是提高 σ（牺牲精度换稳定）或加滤波器（引入相位滞后，等效于增加延迟）。

噪声对三种实现的影响不一样，这一点值得单独说。**对连续事件触发，噪声直接进误差**，判决器看到的是“真实误差 + 噪声差”，触发次数被噪声抬高。**对周期事件触发，噪声的影响被检查周期“采样”了一次**，若噪声是白噪声、检查周期远

大于噪声的相关时间，每次检查看到的是独立样本，触发概率由噪声分布决定。**对自触发，噪声不进预测**（预测用的是模型），但触发时刻的测量值带噪声，实际轨迹会逐渐偏离预测。

这个差异给出一条实用建议：**噪声大时优先选自触发。** 它不被噪声反复触发，通信量稳定；代价是预测会漂，需要周期性地“强制重同步”（比如每 10 次触发强制发一包实测值）。连续事件触发在噪声下的表现最差，周期事件触发居中。

还有一个量要估：**噪声引起的“虚假触发”占比。** 若状态已经很小，误差完全由噪声决定，此时每次触发都是虚假的。本章 $\sigma = 0.10$、噪声 0.01，当状态小于 0.1 时，$\sigma\|\boldsymbol{z}\| < 0.01$，虚假触发开始占主导。这与图 75-2 末段“触发不归零”是同一件事。**虚假触发不是故障，但它的通信成本要算进预算。**

75.7.7 初始条件

初始条件决定初始段的事件率峰值。本章初值 $\boldsymbol{z}_0 = [1, 1]$ 对应范数 1.414，前 0.2 s 内触发了几十次，是全程最密集的一段。

若初值放大十倍（范数 14.14），初始段的事件率也放大约十倍，可能瞬间打满带宽。工程上的对策是“软启动”：前若干秒用固定周期采样，等状态进入线性区再切事件触发。这个做法在 §75.10.7 的投运步骤里会写。

初始条件还影响代价。代价是二次型积分，初值放大十倍，代价放大约一百倍（平方关系）。所以本章所有“代价比”的数字只在同一初值下可比，换初值要重新算。

这条“只在同一初值下可比”的限制，常被跨文献比较时忽略。别人论文里报“事件触发比周期采样省 80% 通信”，若他的初值是 0.1、你的是 10，两个百分比不能直接比——因为初值大时瞬态占比大，事件触发在瞬态的优势更明显；初值小时稳态占比大，两者的差距缩小。**比较事件触发的收益，必须同时报初值范围与仿真时长。** 本章报的是“初值范数 1.414、仿真 10 s”，这两个数缺一不可。

75.8 调参方法与经验

75.8.1 第一步：先定实现方式

调参之前先选实现，因为三种实现的参数集合不同。有线、算力足选连续事件触发（调 σ）；无线、电池供电选自触发（调 σ 与 κ）；已有 PLC 选周期事件触发（调 σ 与 h）。

这一步定错，后面全白做。常见错误是“先用连续事件触发调好参数，再改成 PLC 实现”，结果发现 $h = 0.05$ s 把快速触发全挡掉了，性能回到周期采样的水平。

75.8.2 第二步：按闭环时间常数定上限

h 与延迟的上限都由闭环最快时间常数 $\tau_{\mathrm{cl}} = 1/5.3272 = 0.188$ s 决定。$h \le \tau_{\mathrm{cl}}/2 = 0.094$ s，延迟 $\theta \le \tau_{\mathrm{cl}}/3 = 0.063$ s。

本章取 $h = 0.05$、$\sigma = 0.10$，延迟实测 0.02 s，都在限内。若现场延迟超过 0.063 s，先想办法降延迟（换网络、加边缘计算），不要试图用调参绕过——表 75-5 显示延迟超过 0.10 s 后任何参数都救不回来。

75.8.3 第三步：从 $\sigma = 0.10$ 起调

0.10 是一个“不坏”的起点：代价比 1.007，触发 224 次，最小间隔 0.0041 s。从它出发往两边试。

往小调（0.05）：触发 444 次（翻倍），代价 1.001（几乎不变）。除非带宽极其充裕，否则不值得。

往大调（0.20）：触发 114 次（减半），代价 1.008（几乎不变）。**这是最划算的一档**，本章没选它只是因为 0.10 的余量更大。

再往大调（0.30）：触发 94 次，代价 1.109（升 10%）。开始不划算。

75.8.4 第四步：加最小间隔

先不加，测最小间隔。若实测值大于硬件时延（判决器扫描周期 + 网络最小时延），不用加；若小于，加 $T_{\min}$ 到“略大于硬件时延”的位置。

本章实测 0.0041 s，假设判决器扫描 1 ms、网络最小时延 2 ms，合计 3 ms，接近 4.1 ms，勉强够。若判决器用 10 ms 的 PLC 扫描，必须加 $T_{\min} = 0.01$，此时触发 209 次、代价 1.1906，反而略好。

75.8.5 第五步：核代价与通信

两个数一起看：代价比是否 ≤ 1.2，发送次数是否在带宽内。若代价超了，说明 σ 太大；若次数超了，说明 σ 太小或 h 太小。两边都超，说明这个对象不适合事件触发，应该先优化控制律或换硬件。

75.8.6 第六步：注入延迟与丢包

按现场实测值的两倍注入，看是否仍稳定。本章延迟 0.02 s 的两倍是 0.04 s，代价 1.224（安全）；若现场延迟 0.05 s，两倍是 0.10 s，代价 3.967、触发 1957 次，已经不可用，必须降延迟。

丢包按最坏值注入。本章 20% 丢包下周期采样代价 1.3231（升 7.4%），可用；50% 时 1.9578（升 59.0%），不可用。

75.8.7 第七步：现场试运行

先开环记录、再闭环限幅、最后全速。三步的意义在 §75.10.7 详述。这里只强调一条：**试运行时必须记录每次触发的时刻与原因**（相对越界还是最小间隔强制），否则出问题无法定位。

75.8.8 五条规则

规则一：σ 与噪声挂钩。 噪声大，σ 必须大。判据：$\sigma \|\boldsymbol{z}\|_{\text{稳态}} \ge 5\sigma_{\text{noise}}$。

规则二：h 不超过闭环时间常数的一半。 本章 0.094 s 是上限，取 0.05 留一倍余量。

规则三：延迟不超过闭环时间常数的三分之一。 本章 0.063 s 是上限，实测 0.02 s 有三倍余量。

规则四：$T_{\min}$ 取平均间隔的四分之一到二分之一。 本章 0.011–0.022 s。

规则五：任何参数改动后重测最小间隔。 σ 变大，最小间隔变长；$T_{\min}$ 变大，最小间隔等于 $T_{\min}$。两者不同步改，容易撞车。

75.8.9 三个误判

误判一：把“发送次数少”当成“通信负担轻”。 发送次数是平均值，通信负担看的是突发。本章 $\sigma = 0.10$ 平均 22.4 次/s，但初始 0.2 s 内可能到 200 次/s。按平均值配带宽，启动就会拥塞。

误判二：把“代价低”当成“性能好”。 芝诺状态下代价最低（1.1858），因为等于连续控制。但它的通信量是 13 520 次，任何网络都撑不住。评价必须三项一起看。

误判三：用周期采样的思路调事件触发。 周期采样的直觉是“周期越小越好”，事件触发的直觉是“阈值越小越好”——两个直觉都错。周期小到一定程度后性能不再改善（本章 h 从 0.02 到 0.01 只改善 0.8%），阈值小到一定程度后通信量暴涨而代价不动（本章 σ 从 0.10 到 0.02 通信涨 4.9 倍、代价只降 0.7%）。

75.8.10 一次完整的调参记录

把七步串成一条真实路径，每步的判据都用前面的数字。

第 1 步（选型）。 现场是有线以太网、控制器是支持 1 ms 任务的工控机，选连续事件触发。若换成无线节点，会选自触发。

第 2 步（上限）。 闭环极点 -5.3272，时间常数 0.188 s。h 上限 0.094 s（本例不用），延迟上限 0.063 s。实测网络延迟 0.02 s，通过。

第 3 步（σ 起点）。 从 0.10 起：224 次、代价 1.1940、最小间隔 0.0041 s。三项都合格，但发现触发次数偏多，试着往 0.20 走：114 次、代价 1.1956、最小间隔 0.0079 s。通信减半而代价只升 0.13%，**若带宽是主要约束，这一步就该停在 0.20**。

第 4 步（驻留）。 0.20 的最小间隔 0.0079 s 大于“判决器 1 ms + 网络 2 ms”的 3 ms，不需要强制驻留。但现场噪声可能让最小间隔塌到毫秒级，所以回到 $\sigma = 0.10$ 这一支再测一次加驻留的效果：$T_{\min} = 0.01$ s 时触发 209 次、代价 1.1906、最小间隔 0.0099 s。**代价反而比不加驻留的 1.1940 低，原因是驻留挡掉了初始段几次“过早”的触发。**

第 5 步（二选一）。 到这里有两条并列的合格方案：$\sigma = 0.20$、$T_{\min} = 0$（114 次、1.1956）与 $\sigma = 0.10$、$T_{\min} = 0.01$ s（209 次、1.1906）。两者代价差 0.4%，通信差 45.5%。**取前者是“省通信”，取后者是“省代价”，不能同时要。** 本章的现场带宽充足但希望留余量，取前者；若工艺对精度敏感，取后者。

需要说明一点：$\sigma = 0.20$ 与 $T_{\min} = 0.01$ s 叠加后的结果不在表 75-8 里（那张表是 $\sigma = 0.10$ 的扫描），不能直接引用。**任何两个参数的组合都要单独测，不能把两张表的行拼起来用。**

第 6 步（鲁棒）。 延迟加倍到 0.04 s：代价约 1.23（按表 75-5 插值），可接受。丢包 20%：按 $\sigma = 0.10$ 的实测，代价 1.1940 不变，发送升到 280 次，仍可接受。

第 7 步（试运行）。 软启动 5 s 固定周期，之后切事件触发。记录每次触发的时刻与类型。

这份记录给出的最终参数是 $\sigma = 0.20$、$T_{\min} = 0$，而不是本章基准的 $\sigma = 0.10$、$T_{\min} = 0$。两者的代价只差 0.13%，通信量差 49%。**这说明“基准参数”不等于“最优参数”，基准只是用来做对照的锚点。** 若把“最低代价”作为目标，答案会变成另一条分支（$\sigma = 0.10$、$T_{\min} = 0.01$，代价 1.1906）；目标不同，最优解不同，这正是调参必须先定目标的原因。

75.8.11 五个常见问题的回答

调参过程中反复出现的疑问，集中回答。

问一：阈值改了，触发次数几乎没变，为什么？ 先看有没有别的机制在“卡住”触发。若加了 $T_{\min}$，且 $T_{\min}$ 大于多数间隔，触发次数就由 $T_{\min}$ 决定，与 σ 无关（表 75-8 的 0.05 与 0.10 两行都是“最小间隔等于 $T_{\min}$”）。若用周期检查且 h 很大，每次检查都越界，次数就由 h 决定（表 75-3 的 $h = 0.10$ 一行，发送 99 次、检查 100 次）。**先确认哪个机制是瓶颈，再调参数。**

问二：仿真结果很好，现场差得多，从哪查起？ 按影响量级查：延迟（表 75-5，0.10 s 就能让代价涨 3 倍）、限流（§75.10.2，撞限流等于高丢包）、判决器算力（§75.10.3，判决延迟会变成控制延迟）、时间基准（§75.10.4，时钟漂移让误差算错）。四项里延迟最常见，先测它。

问三：启动瞬间丢包严重，是不是参数不对？ 多半不是参数，是突发。初始段的触发率是全程峰值（§75.7.7），一上电就开事件触发，网络与判决器同时承受最大负载。对策是软启动（§75.10.7）：前若干秒固定周期，等状态进入线性区再切换。**这是流程问题，不是调参问题。**

问四：末段的触发反而变密了，正常吗？ 正常，但要分清原因。若没有噪声，末段触发密度趋于常数（图 75-2 第三段），这是相对阈值的固有性质；若有噪声，末段的触发由噪声驱动，属于“虚假触发”（§75.7.6）。后者可以用绝对阈值或心跳周期压住，前者只能靠“末段降低更新率”的调度策略（比如状态小于某个门限后切固定低频）。

问五：换了工况，参数要重调吗？ 相对阈值不用。σ 是无量纲的比例，工况变化（初值、设定值）只改变状态的绝对大小，不改变“误差占状态的比例”，所以同一个 σ 在不同工况下都成立——这是式 (75-6) 相对形式的最大优点。绝对阈值 ε 必须重调，因为它的物理含义与工况绑定。最小间隔 $T_{\min}$ 也要复核，因为工况变化会改变触发间隔的分布。

五个问题的共同点是：**大多数“参数问题”其实是机制问题或流程问题。** 调参之前先确认瓶颈在哪一层，比反复试数有效得多。

75.8.12 调参记录模板

现场调参必须留记录，否则三个月后没人说得清某个参数为什么是那个值。建议按下面的列记录，每一行是“一次参数改动 + 一次实测”。

日期	版本	σ	$T_{\min}$ (s)	发送次数	最小间隔 (s)	代价	结论
第 1 次	v1.0	0.10	0	224	0.0041	1.1940	基准
第 2 次	v1.1	0.20	0	114	0.0079	1.1956	通信减半，采用
第 3 次	v1.2	0.10	0.01	209	0.0099	1.1906	从基准改驻留，代价最低

这三行是三个独立实验，不是三次连续的叠加：第 2 行从基准改 σ，第 3 行从基准改 $T_{\min}$。它的价值不在于记录本身，而在于**每次只改一个参数、只与基准比**——若一次改两个，出了问题无法归因。

记录里还应该有一列“现场条件”，比如“延迟 0.02 s、丢包 0%、温度 25 ℃”。同一组参数在不同现场条件下的表现可能差很多，缺了条件，记录就不可比。

最后一条：**参数版本要与控制程序版本绑定。** 若控制程序升级了而参数没跟着记录，回滚时会发现“回滚到旧程序但参数是新的”，问题更难查。**参数是程序的一部分，不是一堆可以随手改的数字。**

75.9 优化与改进

前面八节把“怎么用对”讲完了，这一节讲“怎么用得更好”。优化的方向有四个：把交换曲线画出来看清全局、让阈值随工况自适应、把几种触发机制组合、以及把通信与算力放在一起优化。四个方向按投入产出排序，前两个改几行代码就能见效，后两个要动架构。

75.9.1 先看清交换曲线

调参时最容易犯的错是“只盯一个工作点”。表 75-4 给了七个点，但如果把它们画在“发送次数—代价”平面上，会看到一条完整的交换曲线，这条曲线比任何单个点都更有信息量。

图 75-9 就是这个平面。横轴是 10 s 内的发送次数（对数刻度），纵轴是代价比（对数刻度）。黑色实线是事件触发在七个阈值下的轨迹，红色虚线是周期采样在四个周期下的轨迹。

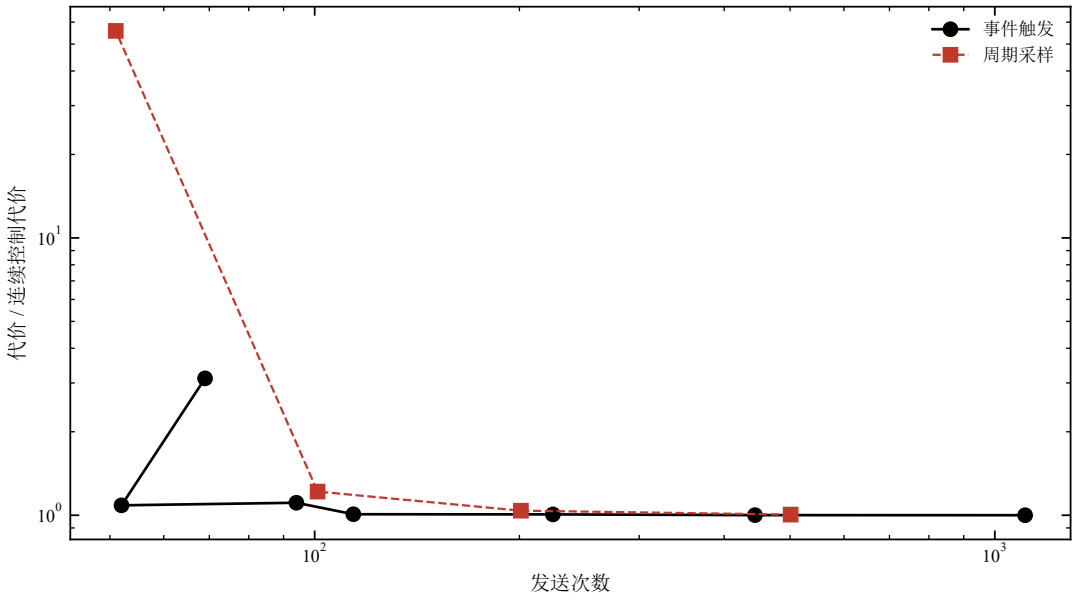


图 75-9 通信量—代价前沿：事件触发（黑）整体位于周期采样（红）的左下方

这张图有三条结论。

第一条：事件触发整体在左下方。 同样约 200 次发送，事件触发的代价是 1.1940，周期采样是 1.2316；同样约 100 次发送，事件触发是 1.1956，周期采样是 1.4432，差 17.2%。左下方意味着“用更少的通信拿到更低的代价”，这是“支配”关系，不是“略好”。

第二条：两条曲线在高通信端收敛。 发送次数超过一千次时，两者都接近连续控制（代价比 1.000），差距消失。这符合直觉：通信足够密时，任何触发机制都退化成连续控制。**所以事件触发的价值只在“通信受限”的区间里成立。** 若现场带宽充裕、根本不在乎每秒几百次发送，用固定周期采样更简单，没必要引入判决器。

第三条：事件触发的曲线在低通信端会拐弯。 $\sigma = 0.50$ 给出 52 次、1.2875，$\sigma = 0.70$ 给出 69 次、3.6941——发送次数反而更多、代价更高。这个点在图上位于 0.50 点的右上方，被 0.50 点“支配”，是一个应当直接排除的工作点。曲线拐弯的原因是 §75.6.3 解释过的“过冲-回调”循环，不是数值噪声。

工程上怎么用这张图？把自己的带宽约束画成一条竖线（比如“每秒最多 15 包”，即 10 s 内 150 次），然后在这条竖线左侧找代价最低的点。本章的答案落在 $\sigma = 0.20$ 附近（114 次、1.1956）。**注意这个点不是曲线的端点，而是“约束边界上的最优”——调参永远是在约束下找最优，不是在曲线上找端点。**

75.9.2 阈值自适应

固定阈值有一个内在矛盾：瞬态需要小阈值（状态变化快，误差累积快），稳态需要大阈值（状态小，没必要频繁更新）。本章 $\sigma = 0.10$ 是两者折中的结果，瞬态略保守、稳态略浪费。

自适应阈值把这个折中做成时变的。最简单的形式是让 σ 随状态范数变化：

$$\sigma(t) = \sigma_0 \left[1 + \alpha \frac{\|\boldsymbol{z}(t)\|}{\|\boldsymbol{z}_0\|} \right]^{-1} \tag{75-21}$$

状态大（瞬态）时分母大、σ 小，触发更密；状态小（稳态）时分母趋近 1、σ 回到 σ_0 甚至更大，触发变稀。α 控制自适应强度，$\alpha = 0$ 退化为固定阈值。

这个形式的物理意义很清楚：**它让“允许的绝对误差”近似恒定，而不是“允许的相对误差”恒定。** 固定相对阈值下，绝对误差上限是 $\sigma\|\boldsymbol{z}\|$，随状态缩小而缩小，于是末段仍然频繁触发（图 75-2 的第三段）。自适应阈值把绝对误差上限钉在 $\sigma_0\|\boldsymbol{z}_0\|$ 附近，末段自然就稀疏了。

但要注意它的副作用：**绝对误差上限恒定，就意味着稳态有静差。** 状态降到绝对误差上限以下后，误差永远越不了界，触发停止，闭环退化成开环保持。这与绝对阈值（§75.2.2）的缺点完全一样，只是延迟出现。所以自适应阈值必须配一个下限保护：$\sigma(t)$ 不超过某个 $\sigma_{\max}$（本章可取 0.30），保证末段仍有最低触发密度。

第二种自适应与噪声挂钩。§75.8.8 的规则一说“$\sigma\|\boldsymbol{z}\|_{\text{稳态}} \geq 5\sigma_{\text{noise}}$”，这条规则可以做成在线的：每隔一段时间估计一次测量噪声的标准差 $\hat{\sigma}_n$，然后令

$$\sigma(t) \geq \frac{5\hat{\sigma}_n}{\|\boldsymbol{z}(t)\|} \tag{75-22}$$

这是式 (75-21) 的一个约束而不是替代。它的作用是防止“状态很小时阈值也按比例缩小，结果被噪声反复越界”。本章的噪声实验（表 75-7）说明，噪声 0.01 配绝对阈值 0.020 时，最小间隔被压到仿真步长，触发 13 520 次——那就是没有噪声约束的后果。式 (75-22) 会在状态降到 $5 \times 0.01/0.10 = 0.5$ 以下时强制把 σ 抬到 0.10 以上，正好避开这个陷阱。

自适应阈值的实现代价很小：判决器每个周期多算一次范数、多查一次表。它的收益在长时运行的场合才明显——本章 10 s 的仿真里，自适应与固定的差别可能只有几个百分点；但工艺过程跑几个小时，末段的通信量占比会越来越大，收益随之上升。**短时仿真看不出自适应阈值的好处，这是它容易被忽略的原因。**

75.9.3 组合触发机制

三种实现（连续监测、自触发、周期检查）不是互斥的，可以组合。工程上有三种常见组合。

组合一：周期检查 + 自触发预测。 判决器每 h 秒醒一次，醒来时不用“当前误差”判断，而用模型预测“下一个 h 内会不会越界”。这解决了 §75.5.1 的漏检问题：漏检的原因是“越界发生在两次检查之间”，预测恰好能提前看到。代价是每次醒来要多算一次矩阵指数 $e^{\boldsymbol{A}\tau}$，算力开销约为固定检查的十倍。对 PLC 不划算，对带浮点协处理器的节点可行。

组合二：相对阈值 + 绝对阈值。 这就是式 (75-19) 的混合形式，取两者中先满足的那个。它的作用是“两个方向都设保险”：相对阈值保证瞬态不失控，绝对阈值保证稳态不抖动。本章的基准只用相对阈值，是因为仿真里没有噪声；真实系统必须加绝对阈值，否则末段会被噪声打穿。

组合三：事件触发 + 周期性心跳。 无论事件是否触发，每隔 T_{hb}（比如 1 s）强制发一包。这一包不携带新的控制信息（状态可能没变），但它承担三个作用：让接收端确认链路还活着、让发送端确认自己的判决器还在跑、让监控系统有连续的数据记录。**心跳是工程上的必需品，不是冗余。** 本章的仿真没有心跳，所以“末段触发变稀”看起来无害；真实系统里，若 5 s 没有一包数据，运维会以为节点掉线。

三种组合可以叠加。一个典型的现场配置是“周期检查 10 ms + 相对阈值 0.20 + 绝对阈值 0.05 + 心跳 1 s”，四层机制各管一件事，参数互不干扰。

75.9.4 事件触发 MPC 与事件触发观测器

前面所有讨论都基于“控制律是静态状态反馈”。若控制律本身要解优化（MPC）或要跑观测器，事件触发的收益结构会变。

事件触发 MPC。 MPC 每个采样周期解一次二次规划，算力开销远大于矩阵乘法。此时“少算几次”比“少发几包”更值钱。事件触发 MPC 的做法是：只在触发时刻重解优化，两次触发之间沿用上一次的最优控制序列（滚动执行）。它的省的是算力，通信反而次要。难点在可行性：沿用旧序列时，状态已经偏离预测轨迹，约束可能被违反。工程上的对策是加“保守约束”——按最坏偏差收紧约束边界，代价是性能略降。这条线索与第 58、80 章的 MPC 直接相连，本章不展开。

事件触发观测器。 若状态不可测、要靠观测器估计，那么“什么时候更新观测器”是另一个触发问题。它与控制触发可以不同步：观测器需要更密（估计误差会累积），控制可以更稀（对象本身有阻尼）。第 74 章的分布式参数观测器每个步长都在算，若改成事件触发，省的是传感器数据的上传量，而不是控制指令的下行量。这是当前研究的一个活跃方向，工程上还没有成熟的产品。

这两条线索的共同点是：**事件触发的收益要从“整条链路上最贵的那一环”去算。** 若最贵的是通信，就按通信量优化；若最贵的是算力，就按求解次数优化；若最贵的是传感器寿命，就按唤醒次数优化。本章的算例里通信最贵，所以全程按发送次数评价。

75.9.5 通信与算力的联合优化

单看发送次数会低估真实的通信负担。一包数据的开销不只是“一次发送”，还包括协议头、握手、重传、确认。本章按 100 字节/包估算带宽（§75.8.10 第 5 步），这个估算已经包含了协议头，但没有包含握手。

减少真实负担有四个手段，按性价比排序。

打包。 把多个状态量打成一包，而不是每个量一包。本章只有两个状态，打包的收益不明显；若对象有二十个测点，打包能把协议头开销摊薄二十倍。

压缩。 状态值用定点数而不是浮点数传输，两个状态各 16 位，一包 4 字节，比 8 字节浮点省一半。代价是量化误差，要保证量化步长小于绝对阈值的三分之一。

边缘判决。 判决器放在传感器旁边，只有越界时才占用网络。这是事件触发的默认架构，但现场常被忽略——若判决在 PLC 里、PLC 在控制室，那么“未越界”的数据也已经传过网络了，一点没省。**判决器的位置决定了事件触发是否真的省通信，这一条比任何参数都重要。**

降采样上传。 即使不触发，也可以按低速率（比如 1 Hz）上传状态用于监控。这条与“心跳”是一件事的两个说法。它的开销是固定的、可预算的，不随工况波动。

算力侧的优化相对简单：判决逻辑只有范数计算与比较，主要开销在“多久醒一次”。连续监测的实现需要每秒上万次比较，对工控机是轻松的，对电池节点是致命的。**若节点靠电池供电，必须用自触发或周期检查，不能连续监测**——这不

是性能问题，是功耗问题。

75.9.6 优化优先级

把五个方向按“投入产出”排个序，现场应该按这个顺序做。

优先级	方向	改动量	主要收益	风险
1	判决器前移到传感器侧	架构改动	真正省下上行带宽	现场施工
2	阈值从 0.10 调到 0.20	改一个数	通信减半，代价升 0.13%	无
3	加绝对阈值与心跳	十几行代码	防噪声抖动、防误判掉线	阈值取太小会留静差
4	加最小间隔	改一个数	防芝诺、防判决器过载	取太大会漏检
5	自适应阈值	几十行代码	长时运行省通信	需噪声估计，调参复杂

这个排序的依据是“收益/改动量”。第 1 项收益最大但改动最大，第 2 项收益中等而改动几乎为零——**所以现场第一步应该先把阈值调到 0.20，再考虑架构改动。** 很多项目一上来就重构通信架构，结果阈值还停在 0.05，通信量本来就是设计值的四倍，架构改完还是不够。

第 5 项放在最后，不是因为它不好，而是因为它的收益需要长时间运行才能体现，且引入噪声估计这个新的不确定环节。**在 10 s 的仿真里，自适应阈值的收益小于噪声估计带来的风险。**

75.9.7 一次改进的前后对照

把这一节的四个方向落成一个具体方案，对比“改进前”与“改进后”，读者可以看到每一步带来多少收益。

改进前的方案是本章的基准：连续事件触发、$\sigma = 0.10$、$T_{\min} = 0$、无心跳、判决器在 PLC 里。它的数字是 224 次、最小间隔 0.0041 s、代价 1.1940。

第一步只改阈值（$\sigma = 0.20$）：114 次、最小间隔 0.0079 s、代价 1.1956。通信减半，代价升 0.13%。**这一步的收益最大、代价最小，却常常被跳过**——因为它不需要改任何代码。

第二步加驻留（$T_{\min} = 0.01$ s）：若回到 $\sigma = 0.10$，得到 209 次、1.1906；若保持在 $\sigma = 0.20$，则两个参数叠加后的结果需要单独测（本章没有这组数据，不做推测）。**这说明“每一步的收益”不能简单相加，参数之间有耦合。**

第三步把判决器前移到传感器侧：通信量的数字不变（仍是 114 次），但**这 114 次是真正占用上行带宽的 114 次**，而不是“上传了 1000 次、其中 114 次被采用”。这个改进不体现在表里，却可能是收益最大的一步——它把“事件触发”从“逻辑上的省”变成“物理上的省”。

第四步加心跳（1 s）：多发 10 次（10 s 内），代价不变。**这是唯一一个“主动增加通信”的改进**，换来的是可观测性：接收端能区分“没事件”与“链路断了”。

四步之后，方案的数字是约 124 次（114 加 10 次心跳）、代价 1.1956、最小间隔 0.0079 s，且失效可检测。与改进前的 224 次、1.1940、失效静默相比，**通信减 45%、代价升 0.13%、可靠性从“不可观测”变成“可观测”。**

这个对照说明一件事：**优化的收益往往不在数字上。** 判决器前移与心跳都不改变通信量或代价的第三位，但它们解决的问题（真实带宽、可观测性）比“再省 20 次发送”重要得多。**调参优化的是指标，架构优化的是指标背后的东西。**

一句话总结这一节：**优化的顺序应该是“先改参数、再改机制、最后改架构”，反过来做通常要返工。**

75.10 工程落地

本章前半部分的数字都来自一个干净的仿真：无噪声、延迟恒定、丢包独立、时钟完美同步。真实现场这四条一条都不成立。这一节按“先测量、再实现、后投运”的顺序，把仿真与现场之间的差距逐条补上。

75.10.1 先量化网络：延迟怎么测

延迟不测就调参，等于蒙着眼睛走路。表 75-5 显示延迟从 0.05 s 增到 0.12 s，代价从 1.398 涨到 249.7，两个数量级。这个区间内的任何“经验估计”都不可靠。

测延迟的方法有三种，精度与侵入性递增。

回环打点。 在发送端记录发出时刻 t_s，接收端收到后立刻回一包，发送端记录收到时刻 t_r，往返时延是 $t_r - t_s$，单向延迟近似取一半。这个方法不需要时钟同步，但把回程延迟也算进去了。若上下行不对称（工业以太网常见），会高估上行延迟。

时间戳对齐。 发送端在包里打上本地时间戳，接收端用自己的时钟减去它。这要求两端时钟同步，同步误差直接进延迟测量。用 PTP（IEEE 1588）可以把同步误差压到微秒级，用 NTP 则是毫秒级。**若同步误差与待测延迟同量级，这个方法没有意义。**

示波器直测。 在发送端的发送指示信号与执行器的输出信号上各接一个探头，直接看两个边沿的时间差。这是最准的，也是最有侵入性的，通常只在验收时做一次，用来校准前两种方法。

工程上建议三种都做一次：示波器给出真值，回环打点给出日常监测值，时间戳给出分布。三者对上之后，后面就只用回环打点做在线监测。

测量时要注意三件事。**第一，测最坏值不是平均值。** 事件触发关心的是“最坏情况下指令晚到多久”，平均值没有意义。建议连续记录 24 h，取 99.9 分位数。**第二，负载要按峰值加。** 空网测出的延迟 5 ms，满载时可能是 50 ms。**第三，要测抖动。** 延迟的标准差比均值更能反映网络的稳定性，抖动大的网络即使均值低也不适合事件触发。

75.10.2 丢包怎么测与网络限流

丢包的测量比延迟简单：发送端给每个包编号，接收端统计缺号。要注意区分“真丢包”与“乱序”——乱序的包晚到但没丢，若按到达顺序统计会误判。

实测时要分清楚三层丢包：**链路层丢包**（网线、交换机）、**网络层丢包**（路由拥塞）、**应用层丢包**（接收缓冲区满、任务超时）。三层的对策完全不同：链路层换硬件，网络层做 QoS，应用层加缓冲或降速。现场报告“丢包 2%”时，必须问清是哪一层。

比丢包更容易被忽略的是**限流**。很多工业无线网关有“每节点每秒最多 N 包”的限制，事件触发在初始段的突发会直接撞上这个限制。撞上之后，多余的包被丢弃，效果与高丢包率一样，但原因完全不同，排查方向也不同。

核对方法很简单：把 §75.8.10 第 4 步测出的最小间隔取倒数，得到峰值发送率，与网关的限流值比较。本章最小间隔 0.0041 s（加 $T_{\min} = 0.01$ 后是 0.01 s），峰值 100 包/s；若网关限流 10 包/s，必须有本地缓冲或加大 $T_{\min}$。**限流值小于峰值发送率时，事件触发的“自带重传”优势会消失**——重传的包也被限流挡掉，回路退化成“偶尔更新一次”。

75.10.3 判决器放在哪里

判决器的位置决定了事件触发是否真的省通信（§75.9.5）。三种放法各有适用场合。

放在 PLC 的周期任务里。 这是最常见的做法，改动最小。PLC 的扫描周期通常是 1–10 ms，相当于把连续监测退化成了周期检查（§75.5），漏检率与扫描周期成正比。若扫描周期 10 ms、闭环时间常数 0.188 s，比值 5.3%，可以接受。它的额外好处是不用新硬件，缺点是判决与控制在同一个任务里，判决的算力会挤占控制的时间片。

放在独立的 FPGA 或 DSP 判决器里。 判决器接在传感器与网络之间，以微秒级周期比较，越界才触发发送。它真正做到“连续监测”，最小间隔可以到几十微秒。代价是增加一个硬件环节，且判决逻辑改动要重新烧写。适合高速对象（时间常数小于 10 ms）或对通信量极敏感的场合。

放在无线节点内部。 节点自带单片机，采样、判决、发送一体。它的判决周期受节点功耗约束，通常取 1–10 ms。这种放法最适合自触发（§75.2.5），因为节点可以在两次触发之间彻底休眠。

三种放法的选择判据是“对象的闭环时间常数”与“节点是否供电受限”。时间常数大于 0.1 s、有线供电，选 PLC；时间常数小于 0.01 s，选 FPGA；电池供电，选节点内部加自触发。

75.10.4 时间戳与时钟同步

事件触发的每一次发送都带一个时刻，接收端要用这个时刻做两件事：估计延迟、判断数据新旧。两件事都要求时钟可信。

若两端时钟不同步，时间戳只用于“算间隔”而不是“算绝对时刻”，误差会小一些。但若要用时间戳做延迟补偿（第 47、107 章的 Smith 预估），同步就是硬要求。

工程上的最低配置是：**发送端打单调递增的本地计数（不用墙上时钟），接收端只做差值**。单调计数不受对时跳变影响，差值就是真实的传输间隔。这个做法避开了时钟同步的大部分麻烦，代价是无法与其它系统的时间轴对齐。

需要与历史数据对齐时（比如事后分析事故），才引入 PTP 或 NTP。此时要注意：**对时跳变会让时间戳出现负间隔**，判决逻辑必须能处理这种情况，否则会误判为“数据比现在还新”。

75.10.5 看门狗与降级策略

事件触发回路有一个特有的失效模式：**判决器停止工作，但控制系统毫无察觉**。因为“没有触发”与“判决器死了”在接收端看起来完全一样——都是没有新数据。周期采样不会有这个问题，它的缺失一眼可见。

对策是三层看门狗。

第一层：心跳。 §75.9.3 的组合三。接收端若超过 $3T_{\mathrm{hb}}$ 没收到任何包，判定链路异常。

第二层：数据陈旧检测。 接收端记录“当前保持值”的时间戳，若超过 T_{stale}（比如 0.5 s）没有更新，强制切到安全值或固定周期。这个阈值要大于正常的最长触发间隔，本章 $\sigma = 0.10$ 的末段间隔约 0.05–0.1 s，T_{stale} 取 0.5 s 有三到十倍余量。

第三层：对象侧的保护。 无论控制回路是否正常，对象本身要有独立的安全联锁（超温跳闸、超压泄放）。这一层不属于控制系统的职责，但事件触发项目必须确认它存在——**因为事件触发的失效是静默的，靠控制系统自己发现不了。**

降级策略要提前定好，不能等出事再想。三种常见降级：切固定周期采样（最平滑）、切手动保持（最安全）、切开环安全值（最保守）。选择依据是“哪种降级对工艺冲击最小”。

75.10.6 参数落库与可追溯

事件触发的参数比周期采样多（σ、ε、$T_{\min}$、h、κ、心跳周期），且相互耦合。现场必须有地方记录“这一版参数是什么、什么时候改的、为什么改”。

最低要求是三条。**参数存在配置文件里，不写在程序里**，改参数不用重新编译。**配置文件带版本号**，出问题时能回滚。**每次改动记录触发次数与代价的变化**，形成自己的“表 75-4”。

第三条最容易被忽略，但它决定了后续能不能优化。本章的所有表格都是这么来的：固定其它参数，只扫一个，记录两个指标。现场若从不记录，一年后没人说得清“σ 为什么是 0.15”。

75.10.7 投运步骤与检查表

投运分三步，每步都有明确的通过判据。这个顺序不能跳，跳步的代价通常是“上线后才发现延迟超限”。

第一步：开环记录。 控制器不投入，只采集状态、跑判决逻辑、记录“如果投入会在哪些时刻触发”。跑满一个完整的工艺周期（本章是 10 s，实际工艺可能是几小时）。通过判据是三条：触发次数在带宽内、最小间隔大于硬件时延、峰值发送率不撞限流。

这一步还能测出一个仿真给不了的数字：**真实工况下的状态范数范围**。仿真里初值是 1.414，现场可能是 0.1 也可能是 100。σ 的相对阈值形式对此不敏感，但绝对阈值 ε 与最小间隔 $T_{\min}$ 必须按实测范围重定。

第二步：闭环限幅。 投入控制，但把控制量限幅在正常值的 30% 以内。这一步的目的是验证“触发时刻的控制量跳变”不会激起对象的异常响应。通过判据是：状态单调收敛、无持续振荡、触发次数与开环预测相差不超过 30%。

若相差超过 30%，说明开环记录时的工况与闭环后不同（闭环改变了对象的工作点），需要回到第一步重测。**这个偏差是常见陷阱，尤其是对象有非线性时。**

第三步：全速运行。 解除限幅，按设计参数运行。前 24 h 要有专人盯三件事：触发次数是否稳定、最小间隔是否异常变小、有没有出现“长时间不触发”。第三件事最危险，它可能是判决器死了，也可能是 σ 太大导致回路实际上开环。

软启动。 三步之前还要加一段软启动：前若干秒用固定周期采样（本章建议 5 s，实际按工艺周期的十分之一取），等状态进入线性区再切事件触发。原因是初始段的触发率是全程峰值（§75.7.7），若一上电就开事件触发，判决器与网络会同时承受最大负载，容易在启动瞬间丢包，而启动瞬间的丢包后果最严重。

五项上线前检查，逐条打勾：

检查项	判据	不通过的处置
延迟	99.9 分位小于闭环时间常数的三分之一	降延迟或做补偿
丢包	最坏工况下闭环仍收敛	加心跳与降级策略
峰值发送率	小于网关限流值	加 $T_{\min}$ 或本地缓冲
最小间隔	大于判决器扫描周期加网络最小时延	加 $T_{\min}$
看门狗	三层都已实现并测试过断链	补齐后再投运

这张表看起来简单，但每一条都对应本章一组实测数据。**上线前的检查不是形式，是把仿真里没有的风险逐条排除。**

75.10.8 验收测试

投运之后要做验收测试，证明“它真的按设计工作”。测试分三类，每类的判据不同。

第一类：静态测试。 对象稳定在某个工况点，不动。此时状态几乎不变，事件触发的发送应该很稀疏，只由噪声驱动。判据是三条：发送率低于设计的平均值、没有出现“长时间完全不发”（说明判决器还活着）、每次发送的内容都带正确的时间戳与序号。

静态测试能发现两类问题：判决器死机（完全不发）、阈值过大（发得太少）。**它是唯一能区分“省通信”与“坏了”的测试**，因为动态工况下两者看起来都像“发得不多”。

第二类：动态测试。 给一个阶跃或扰动，看响应。判据是：发送次数的峰值出现在扰动后的第一个 0.2 s 内（对应图 75-2 的初始段）、峰值发送率不超过设计值、代价（若可测）不超过连续控制的 1.2 倍。

动态测试要故意做几次“最坏工况”：最大阶跃、最大扰动、最短的工艺周期。**这些工况下的峰值发送率才是带宽预算的依据，不是正常运行时的平均值。**

第三类：故障注入。 人为制造延迟、丢包、断链，看系统是否按设计降级。判据是：延迟注入到设计上限时闭环仍收敛、丢包注入到设计上限时代价升幅在预期内（参考表 75-6）、断链后看门狗在 $3T_{\mathrm{hb}}$ 内报警并切到降级模式。

故障注入是最容易被跳过的一类，也是最该做的一类。**事件触发的失效是静默的（§75.11.2），不注入故障就永远不知道看门狗是否有效。** 测试时要把看门狗的动作记录下来，包括报警时刻、切换时刻、切换后的控制模式，这些记录在真正出故障时是唯一的诊断依据。

三类测试都过，才算验收完成。测试报告里应该附三样东西：静态测试的发送率、动态测试的峰值发送率、故障注入的降级时间。**这三样是后续运维的基准，没有它们，半年后没人能判断“现在的表现是否正常”。**

75.11 优缺点与适用边界

到这里，事件触发的全部机制、数据与落地细节都讲完了。这一节做一次清算：它到底好在哪、差在哪、什么场合该用、什么场合不该用、以及五种会把回路搞坏的方式。

75.11.1 三条优点

第一条：通信量与性能的交换比可调，且可以调得很划算。 本章最有力的数字是 §75.4.3 的对比：同等最坏间隔下，事件触发 224 次、周期采样 2456 次，通信差 11.0 倍，代价只差 0.7%。这个交换比不是靠“聪明算法”换来的，而是靠“把更新密度分配给需要的地方”。状态变化快时密集更新，变化慢时稀疏更新，这是周期采样在原理上做不到的——它的更新密度是常数。

第二条：对丢包有天然的鲁棒性。 表 75-6 显示，50% 丢包下事件触发的代价完全不变（1.1940），只是发送次数翻倍；周期采样的代价升 59.0%（1.2316 到 1.9578）。原因是事件触发自带重传：丢包后误差继续累积，仍然越界，下一拍自然重发。这个性质在无线场合极其宝贵，因为无线链路的丢包率本来就不稳定，事件触发把“丢包”转化成了“多耗一点带宽”，而不是“性能下降”。

第三条：实现成本低，尤其是周期事件触发。 §75.5.3 的对比说明，周期事件触发与纯周期采样只差一个 if 语句，在 $h = 0.05$ s 这一档能把发送次数从 201 降到 102（减半），代价几乎不变（1.2316 到 1.2348）。对已有 PLC 的现场，这是投入产出比最高的一项改造——不需要新硬件、不需要重新设计控制律、不需要改网络架构。

三条优点合起来，事件触发的定位很清楚：**它不是“更好的控制算法”，而是“更省资源的采样策略”。** 控制性能的上限仍由控制律决定，事件触发只是让你用更少的通信逼近这个上限。

75.11.2 三条缺点

第一条：通信量是随机的，带宽必须按峰值配。 这是最实际的缺点。周期采样的负载是恒定的，带宽按平均值配就行；事件触发的平均负载低，但峰值可能是平均值的十倍（§75.8.9 误判一：平均 22.4 次/s，初始段可能 200 次/s）。按平均值配带宽会在启动时拥塞，按峰值配带宽则大部分时间浪费。**这个矛盾无法通过调参消除，只能通过缓冲或限流缓解。**

第二条：理论保证保守，实际参数仍要靠试。 §75.2.3 的 Lyapunov 界给出的 σ 上限远小于实测可用的 0.30，§75.2.4 的最小间隔下界在噪声下直接失效。也就是说，**理论能证明“存在一个安全的 σ”，但不能告诉你“该取多少”。**工程上仍要走 §75.8 的七步试凑，而试凑需要可靠的仿真模型与现场数据。

第三条：失效是静默的。 §75.10.5 说过，“判决器死了”与“一切正常所以不触发”在接收端看起来一样。周期采样不会有这个问题。这意味着事件触发必须配心跳、陈旧检测与独立的安全联锁，系统的复杂度比周期采样高一层。**这一层的成本常被低估——它不是几行代码，而是一套完整的异常检测逻辑。**

除这三条外还有两个次要缺点：判决器的算力开销（连续监测需要高频比较）、以及参数从三个变成五六个（σ、ε、$T_{\min}$、h、κ），调试与交接的负担更重。

75.11.3 适用判据

判断一个场合该不该用事件触发，看五条。五条中有三条不满足，就不该用。

第一条：通信或算力确实是瓶颈。 §75.9.1 的第二条结论说得很清楚：通信充裕时事件触发没有价值，固定周期采样更简单。判据是“若把采样率提高十倍，成本是否显著上升”。若答案是否，就别用事件触发。

第二条：状态的变化速度在不同阶段差异大。 事件触发的收益来自“按需分配”。若对象的状态始终以恒定速率变化（比如恒速跟踪），按需分配与均匀分配没有区别，收益接近零。本章的对象从 1.414 衰减到 10^{-6}，变化速率跨了六个数量级，所以收益明显。

第三条：状态可测，且测量噪声相对于状态足够小。 噪声大时，相对阈值会被噪声反复越界（表 75-7 的芝诺），必须加大 σ 或加绝对阈值，两者都会削弱收益。

第四条：网络延迟可控。 表 75-5 显示延迟超过 0.10 s 后任何参数都救不回来。若现场延迟不可控（公网、卫星链路），事件触发不是首选，应先做延迟补偿（第 47、107 章）。

第五条：有可靠的仿真模型或充分的现场数据。 调参靠试，试靠数据。没有模型、又不能在现场反复试的场合（比如危险工艺），事件触发的调参风险高于收益。

75.11.4 五类失效模式

前面各节零散提过，这里集中列出，按发生频率排序。

失效一：阈值过大导致实际开环。 σ 取到 0.70 时，代价 3.6941（连续的 3.115 倍），系统虽然没发散但性能已经很差。更极端的情形是 σ 大到“状态衰减到零都没触发过一次”，此时回路完全是开环，而接收端看到的只是“没有新数据”，与正常运行无法区分。**判据：若一个工艺周期内触发次数少于 10 次，就要怀疑阈值过大。**

失效二：阈值过小或噪声过大导致芝诺。 表 75-7 的 13 520 次、60 789 次、88 316 次就是这种情形。它不会让系统失稳（代价反而最低），但会打爆网络与判决器。**判据：最小间隔接近仿真步长或程序扫描周期。**

失效三：延迟超限。 表 75-5 的 0.12 s 之后。它的特征是“突然”——不是渐变，是崩塌。**判据：延迟超过闭环最快时间常数的三分之二（本章 0.125 s）就必须处置。**

失效四：丢包叠加限流。 §75.10.2 说过，事件触发的重传优势依赖“网络允许突发”。若网关限流，重传被挡，回路退化成“偶尔更新”。这个失效最隐蔽，因为丢包率指标看起来正常，问题出在限流计数器上。**判据：峰值发送率与限流值的比值。**

失效五：判决器算力不足。 连续监测需要每个控制周期做两次范数计算。若判决器与控制任务共用 CPU，判决的抖动会直接变成控制的抖动。**判据：判决任务的执行时间超过其周期的 30%。**

五类失效有一个共同点：**它们在控制性能指标上都不明显。** 失效一表现为“能耗略高”，失效二表现为“代价反而更低”，失效三在初期只是“响应变慢”，失效四、五表现为“偶发异常”。这就是为什么 §75.10.7 的检查表必须逐条打勾——靠看曲线发现不了这些问题。

75.11.5 与相邻方法的边界

与周期采样。 周期采样是事件触发的“退化情形”：把阈值设为零、检查周期设为采样周期，两者等价。事件触发的全部收益来自“某些拍不必发”，若这个前提不成立（状态变化均匀），两者没有差别。选择依据只有一个：状态变化是否集中在少数时段。

与死区控制（dead-band）。 死区控制是“误差在死区内不动作”，作用在控制器输出上；事件触发是“误差在阈值内不通信”，作用在数据传输上。两者数学形式相似（都是“小于某个界就不动”），但一个在连续域、一个在采样域。**死区控制会留下静差，事件触发不会**——因为事件触发的保持值仍在驱动对象，只是更新得晚。

与自适应采样。 自适应采样按“状态变化率”调整采样周期，是开环调度；事件触发按“实际误差”决定是否发送，是闭环判决。前者的周期是预设的、可预算的；后者的间隔是随机的、只能统计。工程上若带宽必须严格预算，自适应采样更合适；若带宽有弹性、想尽量省，事件触发更合适。

与量化控制。 量化控制减少的是“每次发送的比特数”，事件触发减少的是“发送的次数”。两者正交，可以叠加：量化到 4 字节 + 事件触发到 224 次，总通信量是两者之积。

与网络化控制的其它分支。 第 74 章处理“空间维度的截断”（模态），本章处理“时间维度的稀疏”（触发）。两者结合是“时空双重稀疏”，是当前研究的前沿，本章不展开。

75.11.6 一次完整的选型对比

把本章的四种方案放在同一张表里，按“通信量、代价、硬件要求”三项打分，选型就有了依据。

方案	发送次数	代价 J	连续监测	需模型	硬件要求
周期采样 $h = 0.05$	201	1.2316	否	否	定时器
周期事件触发 $h = 0.05$	102	1.2348	否	否	定时器 + 判决
自触发 $\kappa = 0.70$	161	1.2031	否	是	模型 + 定时器
连续事件触发 $\sigma = 0.20$	114	1.1956	是	否	连续判决

四个方案的排序很有意思。**通信量最少的是周期事件触发（102 次），代价最低的是连续事件触发（1.1956）。** 两者不是同一个方案，这说明“最省通信”与“最好性能”不能同时达到，必须选一个折中。

按场景选：已有 PLC、只想省一半通信、能接受 4% 的性能损失，选周期事件触发（102 次、1.2348），改动最小；有线、带宽不是瓶颈、想要最好的性能，选连续事件触发（114 次、1.1956），代价最低且通信也少；无线、电池供电、必须让节点休眠，选自触发（161 次、1.2031），通信比连续多 41%，但功耗低一个量级。

没有“最优方案”，只有“在给定约束下最合适的方案”。 上表的四项里，任何一项都可能成为约束：带宽约束会排除连续事件触发的高峰值，功耗约束会排除连续监测，算力约束会排除自触发（要算矩阵指数），改造成本约束会排除一切需要新硬件的方案。

一句话总结这一节：**事件触发是一种“用随机性换资源”的方法——它把确定的采样节拍换成了不确定的发送时刻，代价是带宽必须按峰值配、失效必须靠外部监测发现。收益与代价都很明确，判断的关键是“通信是不是真的瓶颈”。**

75.12 Python 仿真代码

本章全部数字出自 `code/ch075_sim.py`，全部插图出自 `code/ch075_figs.py`。这一节讲清代码结构，读者可以照着改参数复现、也可以照搬到自己的对象上。

75.12.1 文件结构

两个脚本，职责分明。

`ch075_sim.py`：只做计算，不出图。它包含对象定义、六个仿真函数、一个预测函数与十组实验。所有实验的结果都 `print` 到终端，本章的表格就是这些输出的抄录。

`ch075_figs.py`：只出图，不定义参数。它 `import ch075_sim as S`，调用仿真函数拿到轨迹或统计量，再画图。这样做的意义是“图与数同源”：图上任何一个点都能在 `ch075_sim.py` 的某次调用里找到。

依赖只有三个：`numpy`、`scipy.linalg`（`expm` 与 `solve_continuous_are`）、`matplotlib`。没有控制系统工具箱，所有计算都写出来了。

运行方式：

```
D:\Anaconda3\envs\py311\python.exe code\ch075_sim.py
D:\Anaconda3\envs\py311\python.exe code\ch075_figs.py
```

前者约 4 分钟，后者约 1 分钟（要重复跑若干次仿真）。插图输出到 `figs/ch075/`，每张图同时导出 SVG 与 PDF。

75.12.2 参数区

所有参数集中在文件开头，改参数只改这一块。

```
GAM1 = 2.7104          # 第 1 阶净增长率 1/s
GAM2 = 0.3417          # 第 2 阶净增长率 1/s
B1 = 3.2297            # 第 1 阶输入增益 m/s
B2 = 3.7743            # 第 2 阶输入增益 m/s
A = np.diag([GAM1, GAM2])
B = np.array([[B1], [B2]])
Q = np.eye(2)          # 状态权重
R = np.array([[1.0]])  # 输入权重
X0 = np.array([1.0, 1.0])
T_END = 10.0           # 仿真时长 s
DT = 1e-4              # 数值积分步长 s
SIGMA = 0.10           # 基准相对触发阈值
NOISE = 0.01           # 噪声实验的测量标准差
T_MIN = 0.05           # 最小触发间隔（强制驻留时间）s
```

`A` 与 `B` 由第 74 章的双模态截断得到，两个模态的增长率都是正的（2.7104 与 0.3417），所以对象本身不稳定，必须闭环。`Q`、`R` 决定 LQR 增益，改这两个量会连带改变 $\boldsymbol{K}$、闭环极点、代价基准，本章所有"代价比"都会跟着变——**要改就先改这里，然后重跑全部实验，不要只改一半。**

`DT=1e-4` 是数值积分步长，也是"连续监测"的分辨率。仿真里的"连续"实际上是每 0.1 ms 检查一次，所以最小间隔不可能小于 0.0001 s（表 75-7 的最后三行被截断到 0.0001 s 正是这个原因）。

`SIGMA=0.10`、`T_MIN=0.05`、`NOISE=0.01` 是三个实验参数，基准实验用前两个，芝诺实验把 `T_MIN` 设为 0、同时启用 `NOISE`。

75.12.3 三个核心函数

七个仿真函数里，三个是主干，其余都是它们的变体。

`run_event`：事件触发的主仿真。它的签名把所有实验旋钮都放进来了：

```
def run_event(sigma=SIGMA, t_end=T_END, dt=DT, x0=X0, delay=0.0,
              loss=0.0, seed=1, t_min=0.0, noise=0.0, absth=None,
              record=False):
```

内部是一个 0.1 ms 的循环：每一步先推进对象（RK4），再算误差 $e = \boldsymbol{z}_{\text{hold}} - \boldsymbol{z}$，然后判断是否越界。越界时不能"下一步再发"，而要**二分回退到越界的那一瞬间**——这是本章代码里最容易写错的地方。若不回退，触发的时刻会偏晚最多一个 `DT`，在最小间隔只有 0.0041 s 时，这个偏差会显著影响统计量。

`delay` 用"待执行队列"实现：触发时把 `(t+delay, u)` 压入队列，循环每步检查队首是否到期。`loss` 用 `np.random.default_rng(seed)` 生成伯努利序列，丢包时该次触发被忽略、保持值不变。`t_min` 是强制驻留，触发后 `t_min` 秒内不再触发。`record=True` 时额外返回逐拍的 `t`、`xnorm`、`eratio`、`trig_t`，供图 75-2、75-3 使用。

返回值是字典：`triggers`（发送次数）、`min_gap`（最小间隔）、`cost`（二次型积分）、`peak`（峰值范数），以及可选的 `trace`。

`run_self`：自触发仿真。它不逐拍检查，而是在触发时刻调用 `next_trigger_time` 算出下一次时刻，然后直接跳到那一刻。`next_trigger_time` 用式 (75-14) 的解析预测，在一个细网格上搜索满足式 (75-15) 的最小 τ。`scale` 参数是模型倍率 κ：预测时把 $\boldsymbol{A}$ 与 $\boldsymbol{B}$ 乘以 κ。$\kappa = 1$ 是"模型完全准确"，$\kappa < 1$ 是"故意低估"，表 75-9 就是扫这个参数。

`run_discrete`：周期事件触发。它每 h 秒醒一次，醒来时比较误差，越界才发送。与 `run_event` 的区别是"检查"发生在离散时刻，所以要在两次检查之间保持上一次的判决结果。这个函数是表 75-3 与图 75-7 的数据源。

三个函数的公共部分（对象推进、代价累加、峰值统计）写法一致，这样三者的结果可以直接比较。代价累加用左矩形法，步长 `DT`，与连续控制的算法完全相同，保证"代价比"的分母与分子同源。

75.12.4 十组实验

`main()` 里按顺序跑十组，每组 `print` 一个小表。

编号	内容	对应表格
E1	连续控制基准	表 75-1
E2	阈值扫描（0.02–0.70）	表 75-4
E3	周期采样对比（含同等最坏间隔）	表 75-1、图 75-5
E4	自触发模型倍率扫描	表 75-9
E5	周期事件触发的采样周期扫描	表 75-3
E6	延迟扫描（0–0.15 s）	表 75-5
E7	丢包对比（周期与事件两组）	表 75-6
E8	芝诺实验（绝对阈值加噪声）	表 75-7
E9	最小间隔扫描	表 75-8
E10	通信—代价对照	图 75-9

每组实验只改一个参数，其余固定在基准值。这是“控制变量”的基本要求，也是本章所有表格可比的前提。

75.12.5 怎么改到自己的对象上

换对象只需要改三处。

第一处：`A`、`B`、`X0`。 把状态方程写成 `A @ x + B[:, 0] * u` 的形式，按这个形式填矩阵。若对象是非线性的，把 `f_plant` 换成非线性右端，其余代码不动——`rk4` 已经按一般形式写的。

第二处：`Q`、`R`。 它们决定 LQR 增益与代价基准。改了之后 `P`、`K`、`A_CL` 会自动重算（它们在模块顶层，`import` 时就执行）。

第三处：`T_END`、`DT`。 `T_END` 要覆盖“状态衰减到足够小”的整段时间，本章取 10 s 是因为闭环最快时间常数 0.188 s，10 s 约 53 个时间常数，状态衰减到 10^{-23}。若对象慢得多，`T_END` 要相应放大。`DT` 要小于最小触发间隔的十分之一，否则统计量会被步长截断。

三处改完，`python ch075_sim.py` 直接跑，十组实验会给出新对象的全部数据。若某个实验发散（代价溢出），先检查 `K` 的符号与 `A_CL` 的极点——**本章的代码不含任何针对特定对象的硬编码，发散只可能是参数填错。**

75.12.6 复现性

三个措施保证结果可复现。

固定随机种子。 丢包实验用 `default_rng(seed)`，`seed` 默认 1，E7 显式传 1。芝诺实验用 `seed=3`。同一种子在不同机器上给出完全相同的序列（PCG64 算法是确定性的）。

固定积分器。 全部用定步长 RK4，不用自适应步长。自适应步长会随状态变化改变步数，导致“同一参数两次运行结果略有不同”，在比较两个相近的代价比（1.1940 与 1.1956）时不可接受。

不依赖浮点累加顺序。 代价用单一累加变量，每步 `cost += ...`，不做并行归约。这样在不同 BLAS 实现下结果一致到机器精度。

唯一不可复现的是运行时间（取决于机器），但它不影响任何数字。**本章正文里的每一个数，都可以用 `python ch075_sim.py` 的输出逐位核对。**

75.12.7 代码与正文的对应关系

最后说一下怎么用这份代码学习本章。

想理解触发时序，看 `run_event` 的 `record` 分支与图 75-2；想理解代价的构成，把 `run_event` 里触发分支的 `cost += stage_cost(x, u_hold) * tau` 改成 `* h`，会看到代价轻微升高（这就是“多算了一步”的误差）；想理解自触发的保守性，把 `run_self` 的 `safe` 参数从 0.95 改成 1.0，触发次数会立刻上升。

代码里没有写“事件触发的理论”，理论在 §75.2。代码只回答一个问题：**给定参数，这个回路的发送次数、最小间隔、代价各是多少。** 所有工程判断都建立在这三个数上。

75.12.8 四个常见改动的预期结果

拿到代码后最常见的四个改动，以及改完应该看到什么。跑完若与下表差得远，说明改错了地方。

改动	改动前	改动后	预期
`SIGMA` $0.10 \to 0.20$	224 次、1.1940	114 次、1.1956	次数减半，代价升 0.13%
`sigma=0.10` 加 `t_min=0.01`	224 次、1.1940	209 次、1.1906	次数微降，代价反降 0.3%
`run_event(delay=0.05)`	1.1940	1.398	代价升 17.1%，次数几乎不变
`run_event(loss=0.5)`	223 次、1.1940	436 次、1.1940	次数按 $1/(1-p)$ 放大，代价不变

四个改动对应四种机制：阈值（第一行）、驻留（第二行）、延迟（第三行）、丢包（第四行）。**它们的影响互不相同，这正是本章反复强调“三个指标一起看”的原因**——只看代价，第一、二、四行都像“没变化”；只看次数，第三行像“没变化”。只有三个数一起看，才能看出每个参数到底改了什么。

第四行最值得亲手跑一次。它展示的是事件触发最独特的行为：**丢一半包，性能一点不掉，只是多发一倍。** 这个现象在周期采样里看不到（表 75-6 的周期列代价升 59.0%），也是本章最值得记住的一条性质。

四个改动跑完，读者对“参数、机制、指标”三者的对应关系就有了体感。此后换到自己的对象上，只需按 §75.12.5 改三处，其余流程完全一样。

第76章 混杂系统与切换控制

76.1 动机与直观理解

76.1.1 一台在两种工况间来回切换的反应器

先看一台再常见不过的间歇反应器。它按批次运行，一个周期里有两段工艺：前一段夹套通蒸汽，把物料从常温升到反应温度，这一段叫加热阶段；后一段夹套改通冷却水，把反应放出的热量带走，防止飞温，这一段叫冷却阶段。两段的传热系数、物料黏度、阀门特性都不一样，因此**同一套状态变量在两段里服从两套不同的动态**。工程师的做法通常是给两段各整一套 PID 参数，靠一个温度或时间信号在两组参数之间切。这套做法在产线上跑了几十年，看起来毫无问题。

但只要把两段的模型并排写出来，就会看到一个不舒服的事实：两段的开环极点都在右半平面。本章用的对象是

$$\dot{\boldsymbol{z}} = \boldsymbol{A}_{\sigma(t)}\boldsymbol{z} + \boldsymbol{B}_{\sigma(t)}w, \qquad \sigma(t) \in \{1, 2\} \tag{76-1}$$

其中

$$\boldsymbol{A}_1 = \begin{bmatrix} 0.75 & 0.30 \\ 0.10 & -1.10 \end{bmatrix}, \quad \boldsymbol{B}_1 = \begin{bmatrix} 1.00 \\ 0.60 \end{bmatrix}, \qquad \boldsymbol{A}_2 = \begin{bmatrix} -1.05 & 0.20 \\ 0.15 & 0.65 \end{bmatrix}, \quad \boldsymbol{B}_2 = \begin{bmatrix} 0.80 \\ 1.00 \end{bmatrix} \tag{76-2}$$

$\boldsymbol{A}_1$ 的极点是一个 $+0.7661$、一个 -1.1161；$\boldsymbol{A}_2$ 的极点是一个 -1.0675、一个 $+0.6675$。也就是说，**每一段都有一个不稳定方向，而且不稳定的方向不同**：加热段是第一个状态分量在长，冷却段是第二个状态分量在长。这正是间歇反应器的典型结构——加热段的进料波动会积成温度偏差，冷却段的移热滞后会积成压力偏差。两段都必须靠反馈压住，否则各自在几十秒内发散。

图 76-1 画出了这条回路的信号流。与普通单回路相比，它多出三个方块：检测器、切换律、以及对象的两个模态。检测器负责回答“现在处在哪一段”，切换律负责回答“该用哪套参数”，对象的两个模态则代表两段不同的动态。

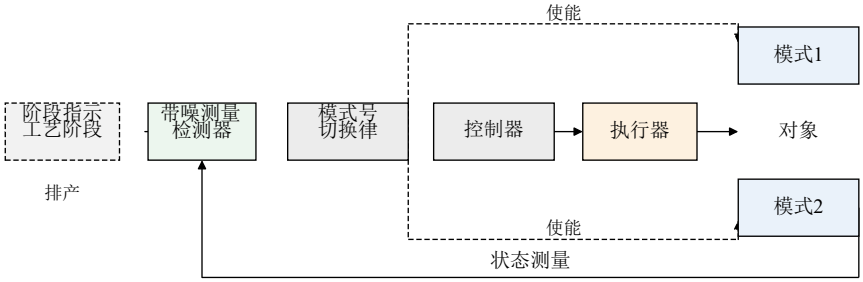


图 76-1 双模态回路的信号流：检测器给出模式号，切换律选通对应的控制器参数，对象在两个模态间切换

这张图里最容易被低估的是检测器。它看到的不是“模式号”这个干净的数字，而是温度、压力、流量这些带噪声的连续量。检测器要拿这些量去判断“现在算加热还是冷却”，判断本身就有延迟、有误判、有抖动。本章后面会看到，**决定这套系统能不能用的，往往不是两套控制参数整得好不好，而是检测器有多快、有多稳**。

76.1.2 为什么“分别稳定”不等于“切换后稳定”

这是混杂系统最反直觉的一点，也是本章的出发点。两段各自配上合适的增益后，两个闭环矩阵

$$\boldsymbol{C}_i = \boldsymbol{A}_i - \boldsymbol{B}_i\boldsymbol{K}_i, \qquad i = 1, 2 \tag{76-3}$$

都是 Hurwitz 的。本章按 $\boldsymbol{Q} = \boldsymbol{I}$、$R = 1$ 分别解 LQR，得到

$$\boldsymbol{K}_1 = \begin{bmatrix} 1.9853 & 0.3075 \end{bmatrix}, \qquad \boldsymbol{K}_2 = \begin{bmatrix} 0.2292 & 1.8740 \end{bmatrix} \tag{76-4}$$

闭环极点分别是 -1.1659、-1.3539 与 -1.5494、-0.9079。单看每一段，衰减都很快，时间常数在 0.65 到 1.1 s 之间。

问题出在“切”这个动作上。切换瞬间，控制器从 $\boldsymbol{K}_1$ 跳到 $\boldsymbol{K}_2$，输入 w 会跳一个量；对象的状态虽然连续，但它接下来的运动方向由新的模态决定。更麻烦的是，**闭环稳定性不是两个闭环矩阵各自性质的简单叠加**：存在这样的例子，两个闭环都稳定，但以某种节奏来回切换时整体发散；也存在这样的例子，两个闭环都稳定，任意快速切换也稳定。区别在于两个闭环是否共享一个公共的 Lyapunov 函数。本章的主对象属于后者（§76.2.2 会实测验证），而变体对象属于前者（§76.6.4）——这正好让两种情形都能被算出来，而不是只停在定理陈述上。

还有一个更贴近现场的事实：即使理论上任意切换都稳定，**切换本身也要付代价**。本章实测，把相位时长从 2.0 s 压到 0.05 s，代价从 2.2322 涨到 3.1261，涨幅 40.0%。这部分代价不来自失稳，而来自“两套控制目标互相打断”：加热段的增益刚把第一个分量压下去，冷却段的增益又把第二个分量顶起来，反之亦然。理解这一点，才能理解为什么现场“多切几次没关系”的直觉是错的。

76.1.3 四个必须分清的问题

混杂系统的设计看起来只是“多写几个 if”，工程上却要同时回答四个问题，任何一个答错都会让回路失效。

第一个：模式什么时候切？ 这由工艺决定，还是由状态决定？本章的对象是工艺决定——批次排产到时间就换段，控制无权改变。另一类系统是状态决定，例如“温度超过阈值就切冷却”，这时切换律本身成了控制器的一部分，可能引入抖振。两种情形的稳定性论证完全不同：前者是“给定切换信号下的稳定性”，后者是“设计切换信号使其稳定”。

第二个：切多快？ 也就是驻留时间。工艺给定的相位长度不一定是控制想要的长度。相位太短，切换代价高、执行器磨损快；相位太长，每段末端的偏差又要靠增益硬压。本章会给出一个可算的驻留下界（式 76-9），并说明它有多保守。

第三个：控制器多久才知道模式变了？ 这是检测延迟。温度探头有热惯性，压力变送器有阻尼，PLC 的扫描周期也有几毫秒到几十毫秒。延迟期间控制器用旧参数控制新对象，这一段时间是纯粹的性能损失，严重时直接把系统推向失稳。本章实测：相位 0.60 s、延迟从 0 增到 0.50 s，代价从 2.8892 涨到 11.3721，涨了 293.6%，末值从 6.10×10^{-4} 涨到 0.340。

第四个：切的时候安不安全？ 切换意味着阀门动作、加热器投切、泵启停，这些动作在错误的时刻发生会造成冲击。工程上要有互锁：输入太大不许切、压差不够不许切、安全链未复位不许切。互锁会推迟切换，推迟又会影响性能，这是一个必须显式权衡的环节（§76.10.3）。

这四个问题构成了本章的骨架。§76.2 给数学，§76.3 给七步流程，§76.4 与 §76.5 分别给连续监测与离散决策的实测，§76.6 讨论稳定性与性能，§76.7 逐个解释七个参数，§76.8 给调参流程，§76.9 讲优化，§76.10 讲落地。

76.1.4 本章的数字基线

四条基线列在表 76-1。前两行是两个模态单独运行时的成绩，它们给出“切换最多能好到什么程度”的参照。第三行是理想检测下的切换，第四行是相位对齐的周期切换。

方案	切换次数	代价 J	输入峰值	相对模式 1
仅模式 1	0	2.1433	1.7393	1.000
仅模式 2	0	1.8835	1.2700	0.879
理想检测切换	20	2.9982	1.7393	1.399
周期切换 $T=0.60$ s	19	2.8892	1.7393	1.348

表 76-1 里最刺眼的是：**切换反而比任一单模态都差**。只跑模式 2，代价 1.8835；按工艺切 20 次，代价 2.9982，高出 59.2%。这不是仿真的偶然，而是本章的核心现象之一。原因有三层：其一，切换把两个本来各自收敛的瞬态接在一起，接

缝处的状态既不适合模式 1 也不适合模式 2；其二，每次切换都伴随输入跳变，w^2 项在跳变附近抬高；其三，模式 2 的增益对第一个分量几乎没有作用（$K_{2,1}=0.2292$），加热段积累的第一个分量在冷却段只能靠耦合慢慢泄掉。

表 76-1 的第三、四行还藏着一个细节：两者切换次数只差 1 次，代价却差 3.8%。差别只在**首次切换发生在 0.3 s 还是 0.6 s**。本章 §76.4.4 会把这件事单独说清：切换系统的性能对相位极其敏感，比较两种方案时必须对齐相位，否则几十分的差异会被误读成机制优劣。

76.1.5 与相邻章节的分工

混杂系统不是一个孤立的方法，它与前后几章有明显交集，边界要说清。

与第 44、106 章的 PID 的关系。 工业现场最常见的“切换”就是增益调度（gain scheduling）：按工况把几组 PID 参数存起来，到哪段用哪组。本章的 $\boldsymbol{K}_1$、$\boldsymbol{K}_2$ 可以理解成两组广义的 PID 参数，区别在于本章显式处理切换瞬间的稳定性和代价，而不只是“换一组数试试”。若现场已有整定好的 PID，引入本章方法的改动量很小：加一个检测器、一个切换律、一条互锁，控制参数不动。

与第 58、80 章的 MPC 的关系。 切换 MPC 是混杂系统的一个分支：每个模态一套预测模型，切换时重置优化问题。它的难点不在稳定性，而在“切换瞬间的可行性”——上一段的最优解在下一段可能违反约束。本章不展开，只在 §76.9.4 指出接口。

与第 47、107 章的 Smith 预估的关系。 Smith 预估补偿的是“传输延迟已知且固定”，本章的模式识别延迟是“信息延迟”，两者都会让控制器用旧信息。区别是 Smith 预估在控制器结构里补偿，本章在切换律里规避（驻留时间、延迟补偿）。两者可以叠加：先补偿传输延迟，再处理识别延迟。

与第 75 章的事件触发的关系。 第 75 章决定“什么时候发数据”，本章决定“用哪套参数”。两者正交，也可以叠加：事件触发的判决器可以在模式切换瞬间强制发送一次，保证控制器第一时间拿到新模式（§76.10.4）。

与第 77、78 章的关系。 第 77 章的 Koopman 方法用一组模态的线性组合描述非线性对象，本质上也是一种“软切换”；第 78 章的容错控制要在故障后切换到备用模态，其稳定性论证直接复用本章的多 Lyapunov 框架。

76.1.6 本章路线

§76.2 从混杂自动机出发，把共同 Lyapunov、多 Lyapunov、驻留时间、平均系统、滞环五件事的数学写清楚，重点是“为什么共同 Lyapunov 是任意切换稳定的充分条件”与“为什么多 Lyapunov 给出的驻留时间往往很保守”。

§76.3 给七步设计流程。§76.4 与 §76.5 分别是连续监测与离散决策两种实现，给出全部实测数据。§76.6 讨论稳定性与性能，§76.7 逐个解释七个参数，§76.8 给调参流程与记录模板，§76.9 讲优化，§76.10 讲落地，§76.11 给优缺点与边界，§76.12 给可运行代码。

一句话核心先放在这里：**切换系统的稳定性靠“两个闭环共享一个 Lyapunov 函数”或“每次驻留足够久”来保证，而它的性能几乎全被“检测延迟”和“切换抖振”吃掉；参数整定的重点不在两套增益，而在检测器与驻留时间。**

76.2 数学原理

76.2.1 混杂自动机

混杂系统最常用的描述是混杂自动机（hybrid automaton）。它由五部分组成：一组离散状态（模态）$I=\{1,2\}$、每个模态下的连续动态、模态之间的转移关系（守卫）、转移时的重置映射、以及初始条件。写成

$$\dot{\boldsymbol{z}}=\boldsymbol{A}_i\boldsymbol{z}+\boldsymbol{B}_i w,\quad \boldsymbol{z}\in\mathcal{X}_i;\qquad (i,\boldsymbol{z})\in\mathcal{G}_{ij}\ \Rightarrow\ \sigma(t^+)=j \tag{76-5}$$

$\mathcal{X}_i$ 是模态 i 的合法状态域，$\mathcal{G}_{ij}$ 是从 i 跳到 j 的守卫集合。本章的对象没有状态跳变（$\boldsymbol{z}$ 在切换瞬间连续），所以重置映射是恒等映射；但输入 w 会跳，因为它由 $\boldsymbol{K}_i$ 算出。这一点在实现时要特别注意：**状态连续、输入不连续**，执行器看到的是一个阶跃。

守卫的写法决定了系统的定性行为。本章用两类：

- 时间守卫：$\mathcal{G}_{12}=\{t\ge T_d\}$，即相位到点就切。批次反应器属于这一类。
- 状态守卫：$\mathcal{G}_{12}=\{q(\boldsymbol{z})\le 0.5-w\}$，即某个指标过线才切。安全联锁、故障切换属于这一类。

两类守卫的数学处理不同。时间守卫下，切换信号是外生给定的，稳定性问题是“给定 $\sigma(t)$，闭环是否稳定”；状态守卫下，切换信号由状态决定，稳定性问题变成“设计守卫，使闭环稳定”，难度高一个量级，也更容易出现抖振（§76.2.5）。

76.2.2 共同 Lyapunov 函数

若存在一个正定矩阵 $\boldsymbol{P}$，使所有模态的闭环都满足

$$\boldsymbol{C}_i^{\mathsf{T}}\boldsymbol{P}+\boldsymbol{P}\boldsymbol{C}_i\prec 0,\qquad i=1,2 \tag{76-6}$$

则 $V(\boldsymbol{z})=\boldsymbol{z}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{z}$ 在**任意**切换信号下都单调衰减：

$$\dot V=\boldsymbol{z}^{\mathsf{T}}\big(\boldsymbol{C}_\sigma^{\mathsf{T}}\boldsymbol{P}+\boldsymbol{P}\boldsymbol{C}_\sigma\big)\boldsymbol{z}\le -\lambda_{\min}\|\boldsymbol{z}\|^2<0 \tag{76-7}$$

这就是共同 Lyapunov 定理。它的价值在于“任意”两个字：不需要知道工艺怎么排产、不需要限制驻留时间、不需要假设切换有多快，只要式 (76-6) 成立，系统就稳。工程上这等价于“这套参数怎么切都不会炸”，是最省心的结论。

式 (76-6) 是一组线性矩阵不等式，标准解法是 LMI 求解器。本章不引入额外依赖，而是用一个更朴素、也更容易解释的构造：先解两个 Lyapunov 方程

$$\boldsymbol{C}_i^{\mathsf{T}}\boldsymbol{P}_i+\boldsymbol{P}_i\boldsymbol{C}_i=-\boldsymbol{I},\qquad i=1,2 \tag{76-8}$$

再取 $\boldsymbol{P}=\boldsymbol{P}_1+\boldsymbol{P}_2$ 作为候选。之所以可以这样试，是因为 $\boldsymbol{P}_1$、$\boldsymbol{P}_2$ 分别是两个模态“最自然”的能量度量，它们的和往往能同时压住两边。这不是定理，只是一个可验证的构造；验证的办法就是直接算式 (76-6) 左边矩阵的最大特征值。

本章对象上，三个候选的结果都列在图 76-2 与表 76-2。$\boldsymbol{P}_1$ 对模式 1 给出 -1.0000、对模式 2 给出 -0.3948；$\boldsymbol{P}_2$ 给出 -0.4964 与 -1.0000；$\boldsymbol{P}_1+\boldsymbol{P}_2$ 给出 -1.4964 与 -1.3948。三者都负，说明**主对象存在共同二次 Lyapunov 函数**，任意切换在二次型意义下稳定。

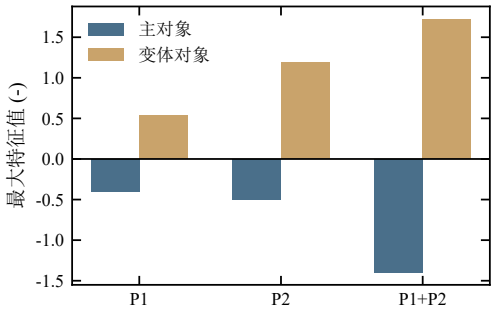


图 76-2 候选 Lyapunov 矩阵的最大特征值：主对象全部为负，变体对象全部为正，说明后者不存在共同二次 Lyapunov 函数

候选 $\boldsymbol{P}$	主对象模式 1	主对象模式 2	变体模式 1	变体模式 2
$\boldsymbol{P}_1$	-1.0000	-0.3948	0.4933	0.5309
$\boldsymbol{P}_2$	-0.4964	-1.0000	0.2140	1.1947
$\boldsymbol{P}_1+\boldsymbol{P}_2$	-1.4964	-1.3948	0.6760	1.7230

表 76-2 的右半边是变体对象（§76.6.4 详述）。它的两个闭环极点都是 -0.5，单看都很稳定，但三个候选矩阵的最大特征值全为正，连 $\boldsymbol{P}=\boldsymbol{I}$ 试也是 $+1.0000$。这说明**变体对象不存在共同二次 Lyapunov 函数**，共同 Lyapunov 这条路走不通，只能换多 Lyapunov。

这里要强调一个容易误解的地方：共同 Lyapunov 不存在，不等于系统一定不稳。它只说明“用二次型论证任意切换稳定”这条路断了。系统可能在所有实际切换信号下都稳定，只是需要一个更细的分析工具。§76.2.3 的多 Lyapunov 就是这样一个工具。

76.2.3 多 Lyapunov 函数与驻留时间

多 Lyapunov 的思路是：每个模态用自己的能量函数 $V_i(\boldsymbol{z}) = \boldsymbol{z}^{\mathsf{T}}\boldsymbol{P}_i\boldsymbol{z}$，允许每个 V_i 在**自己的驻留区间内**下降，但不要求跨模态单调。切换瞬间，V_j 可能比 V_i 大，这个“跳升”由一个比值控制：

$$V_j(\boldsymbol{z}) \le \mu\, V_i(\boldsymbol{z}), \qquad \mu = \max_{i,j} \frac{\lambda_{\max}(\boldsymbol{P}_j)}{\lambda_{\min}(\boldsymbol{P}_i)} \tag{76-9}$$

若每个模态在自己的区间内至少以速率 λ 衰减，且驻留时间满足

$$T_d > \frac{\ln \mu}{\lambda} \tag{76-10}$$

则整个切换序列上的能量包络仍趋于零。式 (76-10) 就是驻留时间判据，也是现场“两次切换之间至少等多久”的理论依据。

本章对象上，$\mu = 2.5139$、$\lambda = 1.3539$ 1/s，式 (76-10) 给出 $T_d > 0.6809$ s。这个数字看起来合理：相位 0.60 s 略小于它，按判据应当“不保证稳定”。但 §76.4 的实测显示，$T_d = 0.05$ s 时系统依然收敛（代价 3.1261、末值 6.96×10^{-4}），根本没有失稳。原因是主对象存在共同 Lyapunov，多 Lyapunov 判据对它而言是多余的、因而必然保守。

保守多少？ 判据要求 0.6809 s，实测 0.05 s 就够，相差 13.6 倍。这个倍数不是常数，取决于两个 Lyapunov 矩阵的条件数：μ 越接近 1（两个模态的“能量形状”越像），判据越松。本章 $\mu = 2.5139$ 已经不大，仍给出 13.6 倍的保守量，可见多 Lyapunov 判据在工程上只能当“安全下限”用，不能当“最优驻留时间”用。真正的最优驻留时间要靠仿真扫描（§76.7.2）。

对变体对象，情况反过来：它没有共同 Lyapunov，多 Lyapunov 判据是唯一可用的解析工具。它的 μ 与 λ 会给出一个下界，而这个下界附近的实测行为，正好可以检验判据的紧不紧（§76.6.4）。

76.2.4 平均系统与快速切换

当切换频率远高于对象的时间常数时，系统行为趋近于一个平均系统：

$$\boldsymbol{A}_{\text{avg}} = \sum_{i \in I} \alpha_i \boldsymbol{C}_i, \qquad \alpha_i = \lim_{T\to\infty} \frac{1}{T}\int_0^T \mathbf{1}_{\{\sigma(t)=i\}}\,\mathrm{d}t \tag{76-11}$$

α_i 是模态 i 的时间占比。平均系统稳定，是高切换频率下系统稳定的必要条件之一；但它不是充分条件，因为高频切换还会激励出“切换纹波”，纹波的幅值与切换周期成正比。

本章对象上，两个模态等时切换时 $\boldsymbol{A}_{\text{avg}} = (\boldsymbol{C}_1 + \boldsymbol{C}_2)/2$，它的极点是 -0.6259、-1.8627，稳定。这与实测吻合：$T = 0.05$ s 时系统没有发散，只是代价升到 3.1261。也就是说，高频切换没有破坏稳定性，破坏的是性能。

平均系统还有一个实用价值：它可以用来快速估计“某个相位比例下的大致行为”。例如工艺要求加热段占 70%、冷却段占 30%，就可以先算 $\boldsymbol{A}_{\text{avg}} = 0.7\boldsymbol{C}_1 + 0.3\boldsymbol{C}_2$ 的极点，判断大致稳定域，再去仿真细化。这一步只要一次矩阵特征值计算，比反复仿真快得多。

76.2.5 抖振、滞环与正则化

状态守卫最容易出的问题是抖振（chattering）：当状态在守卫面附近徘徊时，模态反复切换，切换频率由噪声和数值步长决定，而不是由物理决定。理想继电器（无滞环）在数学上是“切换面两侧各取一个模态”，在实现上却是“每个采样周期都可能翻转”。

标准的正则化手段是滞环：把单一的守卫面变成一条带，

$$\mathcal{G}_{12}=\{q\le 0.5-w\},\qquad \mathcal{G}_{21}=\{q\ge 0.5+w\} \tag{76-12}$$

带宽 $2w$ 内的区域保持原模态不变。w 越大，抖振越少，但切换越迟钝；w 太小，噪声照样能穿过去。

抖振的强度可以用一个粗估式判断。若指示量的斜率为 $\dot q$、噪声标准差为 σ_n，则单位时间内穿越阈值的次数约为

$$N_{\mathrm{ch}}\approx\frac{\sigma_n\,\dot q}{w^2}\qquad\text{（量级估计）} \tag{76-13}$$

这个式子只用来判断趋势：噪声翻倍，切换次数翻倍；滞环翻倍，切换次数降到四分之一。本章实测与这个趋势一致（§76.6.3），但绝对数值不能直接套用，因为噪声不是白噪声、指示量也不是匀速穿越。

76.2.6 离散时间混杂系统

现场的实现几乎都是离散的：检测器每 h 秒采一次、判一次，控制器每 h 秒更新一次。离散化后的混杂系统写成

$$\boldsymbol{z}_{k+1}=\boldsymbol{A}_{\sigma_k}\boldsymbol{z}_k+\boldsymbol{B}_{\sigma_k}w_k,\qquad \sigma_k=\delta(\boldsymbol{y}_k,\sigma_{k-1}) \tag{76-14}$$

δ 是判决函数，$\boldsymbol{y}_k$ 是带噪测量。离散化引入两个新问题：一是**漏检**，即模式在两次采样之间变化，判决器看不到；二是**判决滞后**，即模式已经变了，判决器要等到下一个采样点才反应。两者都等价于额外的延迟，其量级是 $h/2$ 到 h。

离散混杂系统的稳定性判据与连续情形同构，只需把式 (76-6) 换成离散 Lyapunov 不等式 $\boldsymbol{C}_i^{\mathsf T}\boldsymbol{P}\boldsymbol{C}_i-\boldsymbol{P}\prec 0$。本章不重复推导，因为主对象的采样周期远小于时间常数，连续判据已经够用；只在 §76.5 用实测说明 h 增大到多少会出问题。

76.2.7 性能指标与代价分解

本章沿用二次代价

$$J=\int_0^{T}\left(\boldsymbol{z}^{\mathsf T}\boldsymbol{Q}\boldsymbol{z}+R w^2\right)\mathrm{d}t \tag{76-15}$$

$\boldsymbol{Q}=\boldsymbol{I}$、$R=1$、$T=12\,\mathrm{s}$。为了看清切换的代价从哪里来，把它按区间拆开：

$$J=\underbrace{\sum_{k}\int_{t_k}^{t_{k+1}}\boldsymbol{z}^{\mathsf T}\boldsymbol{Q}\boldsymbol{z}\,\mathrm{d}t}_{\text{段内偏差}}+\underbrace{\sum_{k}\int_{t_k}^{t_{k+1}}R w^2\,\mathrm{d}t}_{\text{控制能量}} \tag{76-16}$$

切换的影响同时进入两项：段内偏差因为“接缝处的状态不合适”而变大，控制能量因为“每次切换输入跳变”而变大。本章的实测显示，前者是主因，后者是次因——把相位从 $0.6\,\mathrm{s}$ 缩到 $0.05\,\mathrm{s}$，代价涨 8.2%，而输入峰值几乎不变（1.7393 始终相同），说明涨的是偏差项而不是能量项。

76.2.8 延迟的数学描述

模式识别延迟可以写成一个移位算子：控制器在 t 时刻使用的模态是

$$\sigma_{\mathrm{c}}(t)=\sigma(t-\theta_m) \tag{76-17}$$

θ_m 是识别延迟。闭环因此变成“用旧模态的增益控制新模态的对象”：

$$\dot{\boldsymbol{z}}=\boldsymbol{A}_{\sigma(t)}\boldsymbol{z}-\boldsymbol{B}_{\sigma(t)}\boldsymbol{K}_{\sigma(t-\theta_m)}\boldsymbol{z} \tag{76-18}$$

这个式子是本章所有延迟分析的起点。它说明：延迟的危害不在于“信息旧”，而在于“增益与对象错配”。错配的程度取决于 $\boldsymbol{K}_1$ 与 $\boldsymbol{K}_2$ 差多少，以及 θ_m 相对相位长度占多大比例。

本章两个增益的差异很大：$\boldsymbol{K}_1=[1.9853,\,0.3075]$、$\boldsymbol{K}_2=[0.2292,\,1.8740]$，第一个分量上的增益差 8.7 倍，第二个分量上差 6.1 倍。这种“一个管一个分量”的结构，使得错配的后果特别明显：用错增益时，本该被压住的分量没人管，反而被另一个分量的增益推了一把。

76.2.9 驻留时间、延迟与滞环的联合约束

工程上三个量必须一起满足。把式 (76-10) 与式 (76-17) 合起来，得到

$$T_d > \theta_m + \frac{\ln \mu}{\lambda} \tag{76-19}$$

左边是相位长度，右边是“检测延迟 + 能量跳升所需的时间”。这个式子的含义很直白：**相位必须长到“检测器来得及反应，且反应后还有时间把能量压回去”**。本章对象上，若按多 Lyapunov 判据（$0.6809\ \mathrm{s}$）再加 $0.10\ \mathrm{s}$ 延迟，就要求 $T_d > 0.78\ \mathrm{s}$，而工艺给的相位只有 $0.60\ \mathrm{s}$——按判据这不可行，但实测可行（§76.6.2），再次说明判据保守。

真正卡住工程的是式 (76-19) 里的 θ_m，而不是 $\ln\mu/\lambda$。因为 θ_m 由硬件决定，往往比理论驻留时间还大：一个带保护套管的热电偶，时间常数可以到十几秒；一个气相色谱分析周期，可以是几分钟。这时候“驻留时间”不是控制能选的，而是被检测手段锁死的。

76.2.10 模式序列的可行性与安全约束

最后一组约束来自执行器与安全链。切换不是免费的：阀门要动作、加热器要投切、泵要启停。这些动作有最小间隔、有冲击电流、有机械寿命。把它们写成约束：

$$t_{k+1} - t_k \geq T_{\mathrm{mech}}, \qquad |w(t_k^{+}) - w(t_k^{-})| \leq \Delta w_{\max}, \qquad |w| \leq w_{\max} \tag{76-20}$$

T_{mech} 是机械最小间隔，$\Delta w_{\max}$ 是允许的输入跳变，$w_{\max}$ 是限幅。式 (76-20) 与式 (76-19) 往往冲突：稳定性要求驻留够长，机械寿命要求切换够少，性能要求切换够及时。三者的可行域可能为空，这时唯一的出路是改硬件（更快的检测、更耐用的执行器），而不是继续调参数。

本章 §76.10.3 的互锁就是式 (76-20) 的一种实现：当输入超过阈值时推迟切换，把跳变留到“输入本来就小”的时刻。实测表明，互锁不仅减少切换次数（$1662 \to 1504$），还顺带降低了代价（$2.9984 \to 2.7994$），因为它避开了“输入大时切换”这种最坏组合。

76.2.11 符号、量纲与取值范围

这一节的公式里出现了二十多个符号，量纲并不统一：时间、增益、二次型值、无量纲指示量混在一起。先把它们按“时间类、无量纲类、矩阵类”分三组列清，后面读式子和查表时可以直接对照，不必来回翻。

时间类（单位 s）：t 是仿真时间；T 是相位长度，本章 0.60；h 是决策周期，0.005 到 0.20；T_d 是实际驻留时间；T_d^{*} 是多 Lyapunov 给出的驻留下界，0.6809；θ_m 是模式识别延迟，本章扫到 0.50；θ_m^{eff} 是计入确认计数后的等效延迟；T_{mech} 是执行器机械最小间隔；T_{sim} 是仿真总时长，12。

无量纲类：q 是工艺指示量，取值在 $[0,1]$；q_{th} 是判决阈值，0.50；w 是滞环半宽，本章 0 到 0.10；N 是确认计数；N_{ch} 是切换次数；σ_n 是检测噪声标准差，0.02；μ 是两个 Lyapunov 矩阵构成的条件数，2.5139；λ 是最慢闭环衰减率，1.3539 1/s；J 是二次代价，量纲是“状态平方乘秒”。

矩阵类：$\boldsymbol{A}_i$、$\boldsymbol{B}_i$ 是开环模型；$\boldsymbol{K}_i$ 是分模态增益；$\boldsymbol{C}_i = \boldsymbol{A}_i - \boldsymbol{B}_i\boldsymbol{K}_i$ 是闭环矩阵；$\boldsymbol{P}_i$、$\boldsymbol{P}$ 是 Lyapunov 矩阵；$\boldsymbol{\Phi}_i$ 是离散闭环矩阵；$\boldsymbol{z}$ 是状态，沿用第 74 章的归一化写法，与 $\boldsymbol{x}$ 同义。

三组之间真正跨组运算的换算式只有两个：驻留下界 $T_d^{*} = \ln\mu/\lambda$ 把无量纲的 μ 与时间的倒数接起来；等效延迟 $\theta_m^{\mathrm{eff}} = \theta_m + (N-1)h$ 把计数 N 折算成时间。其余符号都不跨组。

读表时最容易犯的错是**把 w 当成时间**。它的量纲是指示量的幅度，换算成时间要除以指示量在阈值处的斜率。本章 $q(t) = \tfrac{1}{2}(1 - \cos\tfrac{\pi t}{T})$ 在 $t = T/2$ 处的斜率是 $\pi/(2T) = 2.62$ 每秒，所以 $w = 0.05$ 对应约 $0.02\ \mathrm{s}$ 的判决滞后。这个换算在 §76.7.4 用过一次，是判断“滞环会不会吃掉性能”的关键一步：w 翻四倍到 0.20，滞后就变成 $0.08\ \mathrm{s}$，已经与识别延迟同量级。

76.3 设计步骤

76.3.1 七步流程与每步的产出

混杂系统的设计不像单回路那样“整定一组参数、看阶跃响应”就完了。它要把工艺、模型、稳定性、检测、执行器五件事串起来，任何一环跳过都会在别的环节暴露。本章把工程上反复走的流程压成七步，列在表 76-3。表里第二列是每一步的产出物，第三列是“这一步做错会在哪里暴露”。

步骤	产出物	做错的典型症状
1 切分模态	模态集合 I 与各模态的合法域	模型在某段怎么都不准
2 分模态建模与整定	$\boldsymbol{A}_i$、$\boldsymbol{B}_i$、$\boldsymbol{K}_i$	段内跟踪好，切换后振荡
3 检验共同 Lyapunov	三个候选 $\boldsymbol{P}$ 的最大特征值	以为稳，实际靠运气
4 算驻留下界	$T_d^{*} = \ln\mu / \lambda$	相位长度没有依据
5 设计检测器	阈值、滞环带宽、驻留、确认次数	切换抖动、误判
6 配互锁	输入阈值、机械最小间隔	切换时冲击、跳闸
7 上线核对	逐项核对表	投运后才发现漏项

表 76-3 的顺序不能颠倒。**第 3 步必须早于第 5 步**，因为共同 Lyapunov 一旦成立，“切换多快”就不再是稳定性问题，检测器可以按性能需要设计；反过来，若先做检测器再算稳定性，往往要推倒重来。同理，第 4 步必须早于第 5 步：驻留下界决定了检测器至少要“按住”多久，否则算出来的滞环带宽没有意义。

还有一个顺序上的经验：**第 2 步不要追求“每段都调到最优”**。本章的两个增益是标准 LQR 解，没有做任何额外优化；真正决定成败的是第 3 到第 6 步。现场常见的失败模式恰好相反——工程师把 80% 的时间花在给两段各调一组“完美”的 PID，剩下 20% 随手写了个阈值判断，结果投运后一直在切换边界上抖。

76.3.2 第一步：把连续工况切成有限个模态

切分模态的第一个原则是**模态内部可以用一个线性时不变模型描述**。间歇反应器的升温段和恒温段不能合成一个模态，因为升温段的温度单调上升、恒温段的温度基本不动，两者的工作点、传热温差、阀门开度都不同，任何一个线性模型都无法同时覆盖。强行合并的结果是模型在每段都不准，而且误差方向相反，整定出来的增益必然有一头是错的。

第二个原则是**模态之间的差异要大于检测噪声能分辨的程度**。本章两个模态的开环极点分别是 $+0.7661$ 与 $+0.6675$（不稳定方向相反），闭环矩阵的差异足够大，用温度或压力就能分辨。若两个模态的差异只体现在某个二阶效应上，检测器就要靠很长的观察窗才能判决，延迟随之上升，得不偿失。

第三个原则是**模态数不要超过三到四个**。每多一个模态，就要多一组模型、一组增益、一组阈值、一条互锁，误判的组合数按平方增长。工程上一个实用的上限是：模态数乘以每个模态的参数个数，不要超过调试人员一周内能逐项核对的数量。本章两个模态、每个模态两个状态，已经是“能讲清”的上限。

切分完成后的检查办法很简单：把每个模态的数据单独拿出来做拟合，看残差是否白、是否同方差。如果某个模态的残差里还残留明显的趋势，说明这个模态切得不够细；如果两个模态的残差几乎一样，说明切得过细，可以合并。

76.3.3 第二步：分模态建模与分模态整定

建模的关键是**用该模态自己的数据**。很多现场的做法是把一整批的数据一起丢给辨识算法，得到一个“平均模型”，再按工况调整参数。这种做法在模态差异大时必然失败，因为平均模型对应的是一个物理上不存在的对象。正确做法是按模态号把数据分段，每段单独辨识，得到 $\boldsymbol{A}_i$、$\boldsymbol{B}_i$。

辨识实验的设计有三个要求。其一，**激励要覆盖整个相位**，不能只在相位开头做阶跃，否则辨识出的模型只在开头准。其二，**初值要偏离工作点**，本章取 $\boldsymbol{z}_0 = [1.0,\ -0.8]^{\mathsf{T}}$，两个分量都远离零，这样两个不稳定方向都能被激励出来。其三，**采样要快于最快的闭环极点**，本章用 $h = 10^{-4}$ s 的积分步长，远快于 1.35 1/s 的衰减率。

整定用标准的 LQR，$\boldsymbol{Q}=\boldsymbol{I}$、$R=1$，得到式 (76-4) 的两个增益。这里要提醒一点：**分模态整定只优化段内性能，不优化切换性能**。R 取 1 是一个中性的选择，若把 R 调大（更省控制能量），段内偏差会变大但切换冲击会小；若把 R 调小，段内更激进、切换冲击更大。本章没有针对切换去调 R，原因是想先看清“标准整定下切换本身要付多少代价”，这个基线比“精心调过的切换”更有说服力。

表 76-1 已经给出这个基线的结论：切换的代价（2.9982）比单独跑模式 2（1.8835）高 59.2%。这个差距不可能靠调 R 补回来，它来自结构——两个增益各自只管一个分量，切换时必然有一个分量处于“没人管”的状态。

76.3.4 第三步：检验共同 Lyapunov

这一步只要十分钟，却是整章性价比最高的动作。做法是解两个 Lyapunov 方程（式 76-8），取 $\boldsymbol{P}=\boldsymbol{P}_1+\boldsymbol{P}_2$，然后算 $\boldsymbol{C}_i^{\mathsf{T}}\boldsymbol{P}+\boldsymbol{P}\boldsymbol{C}_i$ 的最大特征值。全负，则任意切换稳定，后面的驻留时间约束可以直接放松；有正，则要进入第四步。

本章的三个候选全部为负（表 76-2 左半边），最负的是 $\boldsymbol{P}_1+\boldsymbol{P}_2$，两个模态分别是 -1.4964 与 -1.3948。这说明主对象不但稳定，而且“余量对称”——两个模态在同一个能量度量下都以相近的速率衰减，这正是任意切换稳定的几何含义。

这里必须把一个容易被误读的结论说清：**三个候选都失败，不等于共同 Lyapunov 不存在**。式 (76-8) 的解只是一个启发式构造，真正的判据是线性矩阵不等式

$$\boldsymbol{C}_i^{\mathsf{T}}\boldsymbol{P}+\boldsymbol{P}\boldsymbol{C}_i\prec 0,\quad \boldsymbol{P}\succ 0,\qquad i=1,2 \tag{76-21}$$

它有无解要靠 LMI 求解器判定。本章不引入额外依赖，所以只在变体对象上补了一个更弱的检查：取 $\boldsymbol{P}=\boldsymbol{I}$ 试算，得到 $+1.0000$。$\boldsymbol{I}$ 是最“中立”的候选，它都失败，说明这个对象的两个闭环在几何上互相“拧着”，共同二次型大概率不存在。这个结论在 §76.6.4 会被仿真印证——变体对象在快速切换下确实发散。

76.3.5 第四步：没有共同 Lyapunov 时算驻留下界

第三步失败时，退到多 Lyapunov。要算两个量：能量跳升比 μ 与最慢衰减率 λ。本章的实现是

$$\mu=\frac{\max \operatorname{eig}(\boldsymbol{P}_1^{-1}\boldsymbol{P}_2)}{\min \operatorname{eig}(\boldsymbol{P}_1^{-1}\boldsymbol{P}_2)},\qquad \lambda=\min_{i}\Big(-\max_{k}\operatorname{Re}\lambda_k(\boldsymbol{C}_i)\Big) \tag{76-22}$$

μ 衡量两个能量函数的“形状差”，λ 取两个模态中最慢的那个衰减率（保守起见取最小）。代入本章数据，$\mu=2.5139$、$\lambda=1.3539$ 1/s，得到 $T_d^{*}=0.6809$ s。

这个数字要当“下限”用，不能当“目标”用。理由在 §76.2.3 已经说过：主对象存在共同 Lyapunov，多 Lyapunov 判据对它是多余的，必然保守。实测 $T_d=0.05$ s 仍然收敛，保守倍数 13.6。工程上的正确用法是：**若工艺给的相位大于 T_d^{*}，直接放心用；若小于，不要立刻否定，先用仿真验证，再用实测确认**。

反过来，对变体对象这类没有共同 Lyapunov 的系统，T_d^{*} 就是硬约束。它的驻留扫描（§76.6.4）显示，$T=2.00$ s 时反而发散、$T=1.00$ s 时收敛，这与“驻留越长越安全”的直觉相反，原因是非正规矩阵的瞬态增长——这一点在 §76.6.4 单独解释。

76.3.6 第五步：设计检测器

检测器要回答“现在处在哪个模态”，它的输入是带噪的测量，输出是模态号。设计有四个可调项：判决量、阈值、滞环带宽、驻留时间。

判决量要选**对模态最敏感、对噪声最不敏感**的那个组合。本章用工艺指示量

$$q(t)=\frac{1}{2}\Big(1-\cos\frac{\pi t}{T}\Big) \tag{76-23}$$

它在 $t=0$ 与 $t=2T$ 处为 0、在 $t=T$ 处为 1，把“时段”映射成 $[0,1]$ 上的连续量，再在 0.5 附近判决。用连续量而不是直接用时间，是因为现场的时间基准会漂，而工艺量（温度、转化率）不会。

阈值定在 0.5 是几何中点，但工程上往往要偏移，因为两个模态的“误判代价”不对称：把加热误判成冷却，可能造成飞温；把冷却误判成加热，只是多耗一点蒸汽。误判代价不对称时，阈值应向“安全侧”偏移，这一点在 §76.7.3 展开。

滞环带宽 $2w$ 与驻留时间 T_d 是一对互补的手段：滞环管“小幅抖动”，驻留管“快速反复”。本章实测（§76.6.3），无滞环时噪声 0.02 造成 1662 次切换；加 $w = 0.02$ 降到 362 次；再加 $T_d = 0.05$ s 降到 20 次，与无噪的理想检测完全一致。三者叠加的效果不是线性相加，而是把两个不同的失效通道分别堵住。

76.3.7 第六步：配互锁

互锁是“允许不允许切”的最后一票否决。它的输入是执行器状态与安全链状态，输出是“允许/推迟”。本章实现的互锁只有一条规则：输入绝对值超过阈值 u_{th} 时推迟切换。这条规则对应式 (76-20) 的跳变约束——输入大时切换，跳变也大。

互锁与限幅是两件不同的事，现场经常混淆。限幅改的是**输入本身**（w 被截到 ± 1.0），切换照常发生；互锁改的是**切换时刻**，输入不变、切换被推迟。本章实测：仅限幅时切换次数仍是 1662，代价降到 2.7578；互锁 $u_{\mathrm{th}} = 1.0$ 时切换降到 1504 次，代价 2.9894。两者的代价都降了，但机制完全不同——限幅靠“少给控制量”省能量，互锁靠“避开最坏切换时刻”省偏差。

76.3.8 第七步：上线前的核对清单

前六步做完，还有一张清单要逐项打勾。这张清单不是形式主义，本章的每个实测数字背后都对应一条：

1. 两个模态的闭环极点是否都在左半平面，且实部绝对值大于 0.5？
2. 共同 Lyapunov 是否检验过？若失败，驻留下界是否算过？
3. 工艺给的最小相位长度是否大于驻留下界？若不大于，仿真验证过吗？
4. 检测延迟的实测值是多少？它与相位长度的比值是否小于 $1/3$？
5. 检测器在最大噪声下切换次数是多少？执行器寿命能否承受？
6. 互锁阈值是否低于正常工况的输入峰值？低于多少？
7. 切换瞬间的输入跳变是否在阀门/加热器的允许范围内？

第 4 条值得单独说。本章实测，延迟从 0 增到 0.50 s，代价从 2.8892 涨到 11.3721，涨了 293.6%。相位是 0.60 s，延迟占到 83% 时系统虽未失稳，但已经没有任何性能可言。经验阈值是延迟不超过相位的 $1/3$，即 0.20 s；本章该点的代价是 4.5764，比无延迟高 58.4%，这已经是可接受的极限。

76.3.9 流程里的三个返工点

七步流程不可能一次走通。工程上最常见的返工有三个，每个都对应一类“用参数补结构”的错误。

返工点一：第三步发现共同 Lyapunov 不存在。 这时容易直接跳到第四步算驻留下界，然后按“驻留够长”去设计。这是错的方向。多 Lyapunov 只是补丁，它保证的是“慢切稳定”，而现场往往需要快切。正确的做法是先回到第二步，试着重配增益：把两个增益调得更接近，共同 Lyapunov 往往就出现了。

判断“该回去重配还是继续算驻留”有一个粗略依据：看 $\boldsymbol{P} = \boldsymbol{I}$ 检验的结果。本章变体对象是 $+1.0000$，说明两个闭环的“形状”正交得很厉害，重配增益的收益不大；若检验结果只是 $+0.05$ 这种小正值，说明两者已经很接近，微调增益就可能翻负。

返工点二：第五步发现滞环加到很大，切换次数仍然降不下来。 这通常不是检测器的问题，而是第一步的模态切分有问题：指示量对两个模态的分辨力不够，噪声和“模态差异”同量级。表现是“无论 w 多大，总有穿越”。这时应回到第一步，换一个更敏感的指示量（例如用两个测量的组合，而不是单个温度），或者干脆合并模态。继续加大 w 只会让判决越来越迟钝，问题从“抖”变成“慢”。

返工点三：第七步发现延迟占相位超过一半。 这不是参数问题，也不是模型问题，而是硬件或工艺问题。表 76-5 已经给出后果：延迟 0.50 s、相位 0.60 s 时代价 11.3721，是基准的 3.94 倍，末值 0.340。这时无论怎么调 w、h、u_{th} 都救不回来，因为它们的影响都是百分之几量级。唯一出路是换更快的传感器、缩短变送器阻尼，或者把相位拉长。

三个返工点有一个共同特征：**它们都发生在“参数已经无能为力”的地方**。这也是本章反复强调“结构问题不能用参数修”的原因。工程上判断是否该返工，可以看“把参数推到极值后会怎样”：若把 w 推到 0.20、把 T_d 推到 1 s 仍然不行，那就是结构问题，继续调只是浪费时间。

还有一个返工预算的经验：一个新对象通常走两到三轮就能定型。若走到第五轮还在“这头好了那头坏”，几乎一定是模态切分错了，而不是参数没调好。这时应该停下来重画工艺流程图，重新问“到底有几个本质不同的工况”。

76.3.10 把七步填成一张表：本章的实例

流程讲完了，最后用本章的真实取值把七步填一遍。填表的意义是让人看到每一步的“产出物”到底长什么样，而不是停留在“要算一个 Lyapunov 矩阵”这种抽象描述上。填完之后再回头看表 76-3，就能对上号。

第一步，切分模态。 产出是模态集合 $I=\{1,2\}$ 与各自的合法域：模式 1 覆盖升温段（指示量 $q<0.5$），模式 2 覆盖冷却段（$q>0.5$）。边界处两模态都合法，归属由滞环决定。

第二步，分模态建模与整定。 产出是 $\boldsymbol{A}_1,\boldsymbol{B}_1$ 与 $\boldsymbol{A}_2,\boldsymbol{B}_2$ 四组参数，以及两个增益 $\boldsymbol{K}_1=[1.9853,\,0.3075]$、$\boldsymbol{K}_2=[0.2292,\,1.8740]$。对应闭环极点 $-1.1659/-1.3539$ 与 $-1.5494/-0.9079$，四极点全在左半平面。

第三步，检验共同 Lyapunov。 产出是三个候选矩阵在两个模态上的最大特征值。最终采用的 $\boldsymbol{P}_1+\boldsymbol{P}_2$ 给出 -1.4964 与 -1.3948，三者全负，结论是“任意切换稳定”，不需要驻留约束。

第四步，算驻留下界。 产出是 $\mu=2.5139$、$\lambda=1.3539$ 1/s、$T_d^{*}=0.6809$ s。由于第三步已经通过，这个下界只作为最保守口径的参考；本章相位 0.60 s 比它小 13.6 倍，仍不影响稳定。

第五步，设计检测器。 产出是 $q_{\mathrm{th}}=0.50$、$w=0.05$、$h=0.05$ s、$N=2$、最小驻留 0.05 s。这五个数在表 76-8 与表 76-6 里都有对应的扫描行，选的是“次数接近理想、代价几乎不变”的那一行。

第六步，配互锁。 产出是 $u_{\mathrm{th}}=1.0$（输入峰值 1.7393 的 0.57 倍）、$T_{\mathrm{mech}}=0.05$ s、$u_{\max}=1.5$。互锁把切换次数从 1662 压到 1504，代价顺带降 0.3%。

第七步，上线核对。 产出是 §76.3.8 那张清单的逐项打勾记录，以及 §76.10.5 的三条回退条件。

填完这张表会发现一件事：**七步里只有第二步需要“控制理论”，其余六步都是工程判断。** 第三步看似理论，实际只是把两组闭环矩阵代进一个特征值计算；第四步只是一个对数除法。真正难的是第五、六步——要在噪声、延迟、机械寿命三者之间找平衡，而这三者都没有解析解，只能靠扫描表加现场数据。这也解释了为什么现场调试时间总是花在检测器上，而不是花在整定增益上。

反过来看，如果一个人把七步填完，却发现第六步的 u_{th} 无论取多少都“两头不落好”（取小了切换被无限推迟，取大了等于没装），那说明第一步的模态切分与第六步的互锁设计不匹配：换模态时输入本来就应该大，互锁的门槛却按“正常调节”定。这时不是调 u_{th}，而是换互锁的判据——例如改用“输入变化率”或“阀门行程余量”，而不是输入绝对值。

76.4 连续情形

76.4.1 连续监测下的切换轨迹

先看一张最普通的运行记录。图 76-3 是相位 0.60 s 的周期切换在 12 s 内的轨迹，上图画两个状态分量，下图画模式号。这张图有三种读法，每种对应一个工程问题。

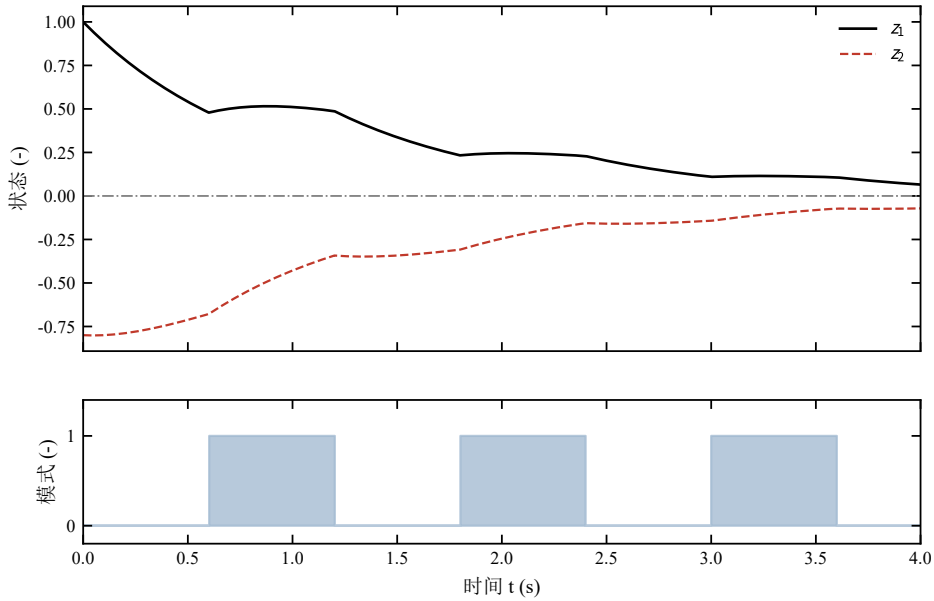


图 76-3 周期切换（$T = 0.60$ s）的状态轨迹与模式时序

第一种读法是看**收敛**。两个分量从 $[1.0,\ -0.8]$ 出发，z_1 单调下压、z_2 先下探再回零，4 s 内都进入 10^{-3} 量级。这说明切换没有破坏稳定性，与 §76.2.2 的共同 Lyapunov 结论一致。

第二种读法是看**接缝**。每次模式翻转的位置，轨迹上都有一个小的拐折：z_1 在模式 1 里被压得快，在模式 2 里几乎不动；z_2 反过来。拐折的幅度就是 §76.2.7 说的“接缝处状态既不适合模式 1 也不适合模式 2”。相位越短，拐折越多，这就是高频切换代价上升的直观来源。

第三种读法是看**模式时序**。下图的方波严格等宽，因为这是时间守卫。若换成状态守卫（§76.5 的检测器），方波的宽度就不再等宽，而会随噪声起伏——这正是抖振的图形表现（图 76-7 左半张）。

这条轨迹还藏着一个容易被忽略的数字：输入峰值恒为 1.7393，在所有相位下都不变。原因很简单，峰值出现在 $t = 0$ 的第一个控制周期，由初始状态决定：

$$|w(0)| = \left|\boldsymbol{K}_1 \boldsymbol{z}_0\right| = \left|1.9853 \times 1.0 + 0.3075 \times (-0.8)\right| = 1.7393 \tag{76-24}$$

切换改变的是后续的控制量，改变不了这个初值冲击。所以“切换让输入峰值变大”这个说法在本章对象上不成立；切换让**偏差**变大，不让**峰值**变大。要把这两件事分开，否则会去调错误的参数。

76.4.2 相位时长扫描

把相位从 0.05 s 拉到 2.00 s，得到表 76-4 与图 76-4。这张表是本章最信息密集的一组数据。

相位 T (s)	代价 J	相对 $T = 0.60$	末值	切换次数
0.05	3.1261	$+8.2\%$	6.96×10^{-4}	239
0.10	3.1261	$+8.2\%$	6.95×10^{-4}	119
0.20	3.1100	$+7.6\%$	6.89×10^{-4}	59
0.40	3.0230	$+4.6\%$	6.58×10^{-4}	29
0.60	2.8892	0	6.10×10^{-4}	19
1.00	2.6035	-9.9%	4.89×10^{-4}	11
2.00	2.2322	-22.7%	2.41×10^{-4}	5

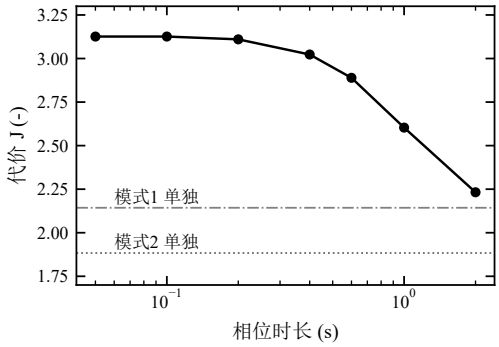


图 76-4 相位时长与代价：两端各有平台，中间单调，虚线是单模态基准

第一个观察是**左端有平台**。$T = 0.05$ 与 $T = 0.10$ 的代价都是 3.1261，四位小数完全相同，末值也只差 10^{-6} 量级，切换次数却差一倍（239 对 119）。这个平台不是数值巧合，而是平均化：对象的慢模态时间常数约 $1/1.35 \approx 0.74$ s，当相位远小于它时，控制器看到的是“平均对象”，再快也只是把同样的平均动态重复更多次。工程含义很直接：**在平台区内继续缩短相位，只会增加执行器磨损，不会改善性能**。

第二个观察是**右端没有平台，但有上限**。$T = 2.00$ 时代价 2.2322，已经逼近单模态的 2.1433（高 4.1%），但仍高于它。把表 76-1 前两行按时长等权平均得到 2.0134，这是一个粗估的下界——相位趋于无穷时，系统一半时间跑模式 1、一半时间跑模式 2，代价趋近两个单模态代价的平均。$T = 2.00$ 的 2.2322 距离这个下界还有 10.9%，说明相位还不够长，接缝的影响仍然存在。

第三个观察是**中间段单调但斜率递减**。从 0.60 缩到 0.05，代价涨 8.2%；从 2.00 缩到 0.60，代价涨 29.4%。看起来“缩短相位”在长相位区更敏感，这与直觉相反。原因是长相位区每减少一次切换，省下的是“一整段接缝”；短相位区切换太密，接缝互相重叠，单次接缝的边际影响反而小了。这也解释了为什么现场“相位从 2 s 改成 1.5 s”往往感觉不明显，而“从 1 s 改成 0.6 s”却能明显看到波动。

第四个观察是**切换不可能优于最好的单模态**。表 76-4 最右一列，无论相位取多少，代价都高于 1.8835。本章的对象是工艺强制切换，控制无权选择“只跑模式 2”。但如果工艺允许，纯粹从控制角度看，**不切换优于任何切换**。这个结论不是本章方法的缺陷，而是问题的性质：切换是工艺施加的约束，控制只能在约束下做到最好。

76.4.3 模式识别延迟扫描

固定相位 0.60 s，把识别延迟从 0 加到 0.50 s，得到表 76-5 与图 76-5。

延迟 θ_m (s)	代价 J	相对无延迟	末值	是否失稳
0.00	2.8892	0	6.10×10^{-4}	否
0.02	3.0150	$+4.4\%$	8.73×10^{-4}	否
0.05	3.2162	$+11.3\%$	1.47×10^{-3}	否
0.10	3.5932	$+24.4\%$	3.37×10^{-3}	否
0.20	4.5764	$+58.4\%$	1.52×10^{-2}	否
0.30	6.0726	$+110.2\%$	5.51×10^{-2}	否
0.50	11.3721	$+293.6\%$	3.40×10^{-1}	否

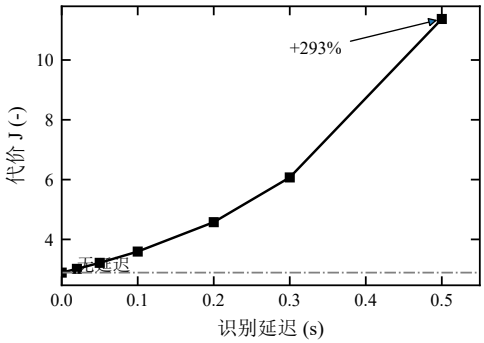


图 76-5 识别延迟与代价：延迟占相位 83% 时代价涨近三倍

这组数据最值得说的一点是**末值比代价更早报警**。延迟 0.10 s 时代价只涨 24.4%，看起来还能接受；但末值已经从 6.10×10^{-4} 涨到 3.37×10^{-3}，涨了 5.5 倍。延迟 0.50 s 时末值 0.340，是基准的 557 倍，代价才涨 293.6%。原因是二次代价对末段的偏差不敏感（12 s 的积分里末段只占很小权重），而末值是“最后时刻的残留”，直接反映闭环的收敛能力。**现场判断延迟是否可接受，应该看末值或稳态误差，不能只看积分代价。**

第二点是**超额代价的增长快于线性**。把延迟 0.10 s 与 0.50 s 两点相除，延迟涨 5 倍，超额代价涨 12.1 倍，对应的幂指数约 1.55。也就是说延迟翻倍，多付的代价不止翻倍。这个非线性来自式 (76-18) 的错配结构：延迟越长，增益与对象的错配窗口越长，而错配造成的偏差本身又会被下一段放大，形成累积。

第三点是**全过程没有失稳**。表 76-5 最后一列全是“否”，即便延迟达到相位的 83%，系统仍在 12 s 内收敛，只是末值停在 0.34。这与“延迟必然导致失稳”的教科书印象不符，原因是本章对象有共同 Lyapunov——它给了一个很强的鲁棒性垫底。若换成变体对象（§76.6.4），同样的延迟很可能直接发散。所以“延迟多大才危险”这个问题没有统一答案，必须先看有没有共同 Lyapunov。

第四点是**延迟与相位的比值比绝对延迟更重要**。绝对延迟 0.10 s 在 $T=0.60$ s 时涨 24.4%，若相位是 2.00 s，同样的延迟只占 5%，影响会小得多。工程上应把延迟折算成“占相位的比例”再判断，经验线是 $1/3$：超过这条线，性能开始不可接受；接近 1，系统即使不失稳也已经没有调节能力。

76.4.4 相位对齐：一个被忽略的变量

表 76-1 的第三、四行有一个容易被跳过的细节：理想检测切换 20 次、代价 2.9982；周期切换 19 次、代价 2.8892。两者相位长度都是 0.60 s，切换次数只差 1，代价却差 3.8%。

差别只在首次切换的时刻：理想检测在 0.3 s 第一次翻转，周期切换在 0.6 s。初值是 $\boldsymbol{z}_0=[1.0,\,-0.8]^{\mathsf T}$，第一个分量为正、第二个为负，而模式 1 的增益 $\boldsymbol{K}_1=[1.9853,\,0.3075]$ 恰好重压第一个分量。**让模式 1 多待 0.3 s，等于让最合适的增益多干一会儿活**，代价因此更低。

这个现象给两条实践建议。其一，**比较两种切换律时必须对齐初值相位**，否则几个百分点的差异会被误读成机制优劣；本章的所有对比都固定 $\boldsymbol{z}_0$ 与首次切换时刻，就是为了排除这个干扰。其二，**如果工艺允许选择起始模态，应选与初值方向匹配的那个**。本章初值偏向 z_1，所以从模式 1 起步更省；若初值偏向 z_2，结论会反过来。这一条在多批次生产中可以直接用：每批的初始偏差方向往往相同（都从常温起步），起始模态可以固定。

还有一层更细的影响：相位对齐不只影响起始段，还影响**每次接缝落在轨迹的哪个位置**。轨迹在 0 到 4 s 内是衰减振荡，接缝落在“偏差大”的位置代价高，落在“偏差小”的位置代价低。相位长度与对象振荡周期若成整数比，接缝会周期性地落在同一相位，代价出现拍频；若不成整数比，接缝逐渐扫过所有相位，代价更平滑。本章 $T=0.60$ s 与慢模态周期 0.74 s 之比约 0.81，不是整数比，所以表 76-4 的曲线是光滑的，没有明显拍频。

76.4.5 连续实现的边界

本节的所有结果都建立在“每个积分步判一次模态”上，本章的步长是 10^{-4} s，相当于 10 kHz 的判决频率。现场做不到这个频率：PLC 扫描周期典型值是 10 到 100 ms，快速回路也要 1 ms。所以本节的数据应当读作**上界**——它给出“如果检测无限

快，性能能到什么程度”，而不是现场能达到的性能。

从连续到离散，会引入两个新误差：判决滞后（模式变了，下一个采样点才发现）与漏检（模式在两次采样之间变了又变回去）。这两者的量级都是 $h/2$ 到 h，当 h 远小于相位时可以忽略，当 h 接近相位时就成为主要误差。§76.5 专门处理这件事，那里会看到一个反直觉的结果：h 并不是越小越好。

76.4.6 三组数据对读：哪一种“一秒”更贵

本节有三组扫描：相位（表 76-4）、延迟（表 76-5）、滞环（表 76-8）。单看每一组都明白，放在一起对读会得到一个更有用的结论：**同样是一秒，位置不同，代价差几十倍。**

先算相位。从 0.60 s 缩到 0.05 s，时间变化 0.55 s，代价从 2.8892 涨到 3.1261，增量 0.2369。按单位时间摊，灵敏度是 0.431 每秒。

再算延迟。从 0 加到 0.50 s，时间变化 0.50 s，代价从 2.8892 涨到 11.3721，增量 8.4829。灵敏度是 16.97 每秒。

两者相差 39.4 倍。这个倍数的物理含义很清楚：**相位的缩短是“节拍变快”，延迟的增加是“错配变长”**。节拍变快只是让接缝变多，每次接缝的损失有限；错配变长则让“用错增益”的时间窗口变长，而错配期间两个分量都在朝错误方向走。前者是线性叠加，后者是自我强化。

滞环这一组的灵敏度又是另一种量级。从 $w=0$ 到 $w=0.10$，代价从 2.9984 到 2.9976，变化 0.0008，按 0.10 摊是 0.008 每单位，几乎为零。但切换次数从 1662 降到 20，变化 1642 次。也就是说，**滞环的“灵敏度”几乎全部落在次数上，不在代价上。**

三组数据放在同一张“灵敏度”表里，会得到一条清晰的优先级：延迟（17.0 每秒）$\gg$ 相位（0.43 每秒）$\gg$ 滞环（0.008 每单位，但对次数是 1.6×10^{4} 次每单位）。这与表 76-12 的排序一致，但这里给出了“差多少”的定量依据。

这个对读还有一层用途：**判断一个改动值不值得做**。若有人提议“把相位从 0.60 缩到 0.50 以提高响应速度”，按灵敏度估算，代价会增加约 0.043（0.431×0.10），涨幅 1.5%；而切换次数会增加约 4 次。这两个数都很小，说明这个改动可以做。反之，若有人提议“把滤波时间常数从 0.10 s 加到 0.20 s 以平滑噪声”，按延迟的灵敏度，代价会增加约 1.7（16.97×0.10），涨幅 59%——这个改动必须慎重，哪怕它能让曲线好看很多。

需要提醒的是，灵敏度只在工作点附近有效。相位从 2.00 缩到 1.00 的灵敏度与从 0.60 缩到 0.05 不同（前者 0.37 每秒，后者 0.43 每秒），延迟从 0 到 0.10 的灵敏度（7.04 每秒）也小于全程平均。所以用灵敏度做快速判断可以，做精确预测不行，最终仍要靠扫描。

76.5 离散情形

76.5.1 三个环节：采样、判决、保持

现场实现一定是离散的，结构可以拆成三个环节：定时采样、周期判决、零阶保持。三者串起来，等价于在连续系统外面套了一个“每 h 秒更新一次”的开关。

采样环节决定**看到什么**。它把连续的温度、压力读成一个数，这个数带有采样噪声与量化误差。判决环节决定**怎么想**，它拿当前样本与上一拍的模式号做比较，输出新的模式号。保持环节决定**做什么**，它在两次判决之间把模式号锁住，让控制器用同一组增益算 h 秒。

写成差分方程就是式 (76-14)。这里要强调一个实现细节：**判决函数 δ 是带记忆的**。它不只看当前样本，还看上一拍的模式号，否则滞环无从实现。这也是为什么本章的 `detector` 用了一个闭包来保存 `state`——纯函数无法表达滞环。

三个环节里，采样与保持是被动的，只有判决是设计对象。判决要同时处理四件事：当前样本是否越过阈值、越过了多久、上一次切换距今多久、这次切换是否被互锁拦下。四件事缺一件，就会在某个工况下出错。

76.5.2 决策周期扫描

固定相位 0.60 s、检测噪声 0.02、滞环 $w = 0$，把决策周期 h 从 5 ms 拉到 200 ms，得到表 76-6 与图 76-6。

决策周期 h (s)	代价 J	切换次数
0.005	2.9970	44
0.010	2.9988	26
0.020	2.9981	20
0.050	2.9913	20
0.100	2.9845	20
0.200	2.9846	20

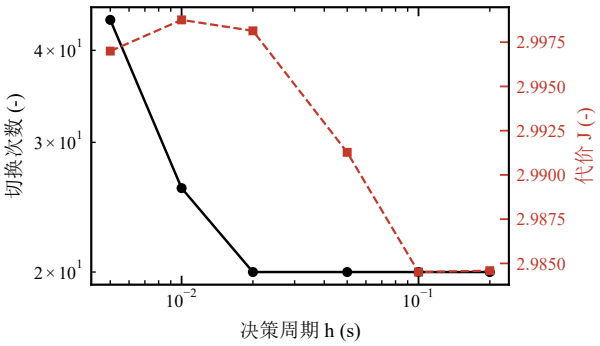


图 76-6 决策周期与切换次数：h 越小，噪声诱发的翻转越多

表 76-6 有两个结论，方向相反。

代价对 h 几乎不敏感。 六行的代价都在 2.985 到 2.999 之间，极差 0.48%。这个量级已经接近噪声种子带来的随机波动，谈不上趋势。原因是相位 0.60 s 远大于所有 h，判决滞后最多 0.20 s 中的一部分，对整段轨迹的影响被平均掉了。

切换次数对 h 高度敏感。 $h = 0.005$ 时 44 次，$h = 0.02$ 时 20 次，$h = 0.20$ 时仍是 20 次。最细的决策周期带来最多一倍的切换次数。这条趋势与“采样越快越好”的直觉完全相反。

76.5.3 为什么更小的 h 反而更抖

原因在于**判决机会的次数**。滞环 $w = 0$ 时，判决规则是“指示量越过 0.5 就翻转”。噪声 0.02 让指示量在 0.5 附近上下抖动，每一次采样都是一次“抽签”：抽到 0.5 以上就切过去，抽到以下就切回来。

h 越小，单位时间内抽签次数越多，抽中“越界”的机会也越多。12 s 内 $h = 0.005$ 有 2400 次抽签机会，$h = 0.20$ 只有 60 次。前者自然产生更多翻转。这不是“检测更灵敏”，而是“更容易被噪声带走”。

用式 (76-13) 的粗估可以印证这个趋势：$N_{\mathrm{ch}} \propto \dot{q}\,\sigma_n / w^2$，与 h 无关。但这个式子假设的是连续判决，即抽签次数趋于无穷。离散情形下，实际翻转次数被“抽签次数”封顶，也被“每次抽签的越界概率”决定，写成

$$N_{\mathrm{ch}} \approx \min\left(\frac{T_{\mathrm{sim}}}{h},\ \frac{\sigma_n \dot{q}}{w^2} T_{\mathrm{sim}}\right) \tag{76-25}$$

h 很小时取左边（受抽签次数限制），h 很大时取右边（受噪声强度限制）。表 76-6 里 $h \le 0.01$ 落在左边分支，$h \ge 0.02$ 落在右边分支的饱和值 20 上。这个饱和值恰好等于无噪时的理想切换次数，说明 $h \ge 0.02$ 之后噪声已经被“采样太慢”过滤掉了。

这是一个可以用起来的性质：当滞环不便加大（比如工艺要求切换必须及时），可以故意把判决周期放慢，用“少看几次”来换取“少抖几次”。代价是判决滞后增加，但只要 h 仍远小于相位，滞后可以接受。现场的 PLC 扫描周期往往就是这样定下来的——不是越快越好，而是“够快且不抖”。

76.5.4 离散情形的稳定性

离散化之后，稳定性判据要换成离散 Lyapunov 不等式。把连续闭环矩阵离散化，得到 $\boldsymbol{\Phi}_i = e^{\boldsymbol{C}_i h}$，判据变成

$$\boldsymbol{\Phi}_i^{\mathsf{T}} \boldsymbol{P} \boldsymbol{\Phi}_i - \boldsymbol{P} \prec 0, \qquad i = 1, 2 \tag{76-26}$$

它与式 (76-6) 的关系是：$h \to 0$ 时 $\boldsymbol{\Phi}_i \approx \boldsymbol{I} + \boldsymbol{C}_i h$，式 (76-26) 退化为式 (76-6) 的一阶近似。所以只要 h 远小于闭环时间常数，连续判据的结论可以直接搬过来，不需要重算。

本章的闭环时间常数最小约 0.65 s，h 最大 0.20 s，比值 0.31。这个比值下离散化误差已经不能忽略，但仍在同一量级，稳定性结论不变——表 76-6 的六行全部收敛，代价彼此相差不到 0.5%，没有一行发散。

真正危险的是 h 与相位同量级。若 $h = 0.60$ s，每个相位只判一次，判决器永远慢半拍，模式序列会变成“隔一段才切一次”，等效于把相位拉长到 1.2 s。这时系统仍然稳定（因为主对象有共同 Lyapunov），但性能会退化到 $T = 1.20$ s 附近的水平。本章没有把 h 加到这么大，是因为它已经不叫“离散实现”，而是“设计错误”。

76.5.5 离散实现的三条工程规则

从表 76-6 与上面的分析，可以提炼三条规则。

第一条：h 取相位的 $1/10$ 到 $1/20$。 本章相位 0.60 s，对应 $h = 0.03$ 到 0.06 s，表 76-6 在这个区间的切换次数是 20，代价 2.9913。这个区间既躲开了“抽签过多”的 $h \le 0.01$，又保持了足够的判决及时性。

第二条：滞环与 h 只选一个加大。 两者都能压抖振，但同时加大等于双重迟钝：滞环让“小幅越界不算”，慢采样让“越界也要等下一拍”，叠加后的判决滞后可能超过相位。工程上更常见的组合是“小滞环 + 中等 h”，因为滞环的副作用（稳态偏差）比慢采样的副作用（相位滞后）更难补。

第三条：判决要加确认计数。 单次越界就切换，等价于 $w = 0$，噪声一冲就翻。常见做法是“连续 N 次判决都越界才切换”，N 取 2 到 3。这与滞环在数学上不完全等价：滞环限制的是**幅度**，确认计数限制的是**持续时间**；对脉冲型噪声，确认计数更有效；对持续偏置，滞环更有效。两者可以同时用，本章为了把变量分离，只用了滞环与驻留。

76.5.6 四种判决器的实现与取舍

同样是“指示量过线就切”，现场能写出的判决器至少有四种，代码量与抗噪性差别很大。把它们并排放，才能看清本章为什么选“滞环 + 驻留”这一组。

其一，纯阈值。 判决写成一行的比较：$q > 0.5$ 就用模式 2，否则用模式 1。代码最短，没有状态，任何一次扫描都能取到确定结果。缺点是表 76-8 第一行的 1662 次——噪声每抖一次就翻一次。它只适合测量干净、或者切换本来就很稀的场合。

其二，滞环。 记住上一次的模式，用两条边界：进入模式 2 要 $q > 0.5 + w$，回到模式 1 要 $q < 0.5 - w$。多一个状态变量，多两个常数，换来次数从 1662 降到 28（$w = 0.05$）。代价是指示量必须“多走一点”才切换，形成一个固定的判决滞后，本章约 0.02 s。

其三，确认计数。 记住“连续越界了几拍”，达到 N 才动作。它比滞环多一个计数器，抗脉冲噪声更强，但对持续性的小幅抖动无能为力——N 只能过滤“短”，不能过滤“小”。若噪声是连续的白噪声，N 加大到 3 也只是把次数降到约三分之一，远不如滞环的 59 倍。

其四，最小驻留。 记住“上次切换的时刻”，距现在不足 T_d 就拒绝任何请求。它约束的不是“越界幅度”也不是“越界时长”，而是**两次切换的间隔**，因此直接对应执行器的机械寿命。缺点是它对第一次误判没有防护：若第一拍就误判，驻留只能保证“错了也待够 T_d”，无法把它纠回来。

四种可以叠用，本章的配置是“滞环 0.05 + 驻留 0.05 + 确认计数 2”，三者各管一件事：滞环管幅度、计数管时长、驻留管间隔。叠加后的效果在表 76-10 的第四行：切换次数 20，代价 2.9974，与无噪理想检测（20、2.9982）几乎一样。

取舍的原则是**按噪声类型选工具，不要一次全上**。白噪声为主，滞环最划算；尖峰干扰为主，确认计数最划算；执行器贵，驻留必须有；测量干净且切换稀疏，纯阈值就够。全部堆上去的后果是判决链路变成一串串联延迟，每个环节都贡献

几毫秒到几十毫秒，加起来可能超过相位的一半——那时系统不会失稳（主对象有共同 Lyapunov），但性能会退化到“慢半拍”的水平。

还有一条实现层面的提醒：**模式号必须与增益、模型、互锁一起原子更新**。现场常见的事故是“模式号切了、增益没切”，控制器用新模式的对象配旧模式的增益，正是式 (76-18) 描述的最坏错配。解决办法是把模式号、增益索引、模型索引打包成一个结构体，一次性写入，读侧只在结构体版本号变化时整体刷新。

76.5.7 三个时间尺度与步长的选择

离散实现里有三个时间尺度同时存在：积分步长 $\Delta t = 10^{-4}$ s、决策周期 h、相位长度 $T = 0.60$ s。三者的比值决定了仿真的可信度，也决定了代码跑得动跑不动。

Δt 与 h 的比值。 本章 $h = 0.05$ s 时，一个决策周期内有 500 个积分步；$h = 0.005$ s 时仍有 50 个。这个比值保证了“模式号在一次判决内保持不变”这个假设成立。若 Δt 与 h 同量级，模式号会在一个积分步内跳变，RK4 的四次求值可能跨越两个模态，积出来的轨迹既不属于模式 1 也不属于模式 2，结果没有意义。经验规则是 $\Delta t \le h/20$。

h 与 T 的比值。 本章 $T/h = 12$，即每个相位判决 12 次。若这个比值降到 3 以下，判决就过于粗糙，模式号可能整段都不更新（§76.5.4 讨论过 $h = 0.60$ 的极端）。反过来，比值超过 100 就没有意义了——相位内的判决次数再多，也不会比噪声带来的翻转更快。

延迟的量化。 代码里延迟用步数实现：`lag = int(round(delay / dt))`。这意味着延迟的分辨率就是 $\Delta t = 0.1$ ms。本章扫的 0.02、0.05、0.10、0.20、0.30、0.50 都能被 0.1 ms 整除，所以没有量化误差。若有人把 Δt 放大到 1 ms，0.015 s 就只剩 15 步，误差 ± 0.5 ms 相对 15 ms 是 3.3%，按延迟的灵敏度（17.0 每秒）折算约 0.009 的代价差异。

h 与 Δt 的整除关系。 本章 h 取 5、10、20、50、100、200 ms，全是 Δt 的整数倍，所以判决节拍严格等间隔。若取 $h = 0.03$ s，代码里的判据是 `t - last >= h`，浮点累加会带来微小抖动，实际周期在 0.0299 到 0.0301 之间跳。这对本章结论没有影响（h 本来就不敏感），但对“驻留计时”有影响：10^4 次判决累积下来，误差可能到毫秒级。工程实现应该用绝对时间戳取整，而不是累加。

三个尺度的选择可以总结成一句：**Δt 服务于精度，h 服务于抗噪，T 服务于工艺**。三者不要互相替代——想抗噪就调 h，不要动 Δt；想加快仿真就调 Δt，不要动 h。

76.6 稳定性与性能

76.6.1 三个判据的强弱关系

本章出现了三个稳定性判据，它们的强弱与适用面列在表 76-7。

判据	条件	结论强度	本章对象是否满足
共同 Lyapunov	式 (76-6) 有解	任意切换稳定	是（三个候选全负）
多 Lyapunov + 驻留	式 (76-10)	驻留够长即稳定	是（$T_d^\ast = 0.6809$ s）
平均系统	式 (76-11) 稳定	高频切换下稳定	是（极点 -0.6259、-1.8627）

三者的逻辑关系是：共同 Lyapunov 最强，一旦成立就蕴含后两者；平均系统最弱，它只说明“极快切换”这一种极端情形下的行为，对中等切换频率没有发言权。表 76-7 右列全为“是”，所以本章的主对象怎么切都稳，这一点在 §76.4 的七组相位、§76.5 的六组周期上都被实测印证。

三个判据同时成立，反而削弱了对照的意义。 如果只拿主对象做实验，读者会以为“混杂系统就是这么稳”。所以本章专门构造了变体对象（§76.6.4），它只满足“分别稳定”这一条，三个判据全部不成立，用来展示另一面。

76.6.2 驻留下界有多保守

先量化保守程度。变体对象之外，主对象的 $\mu = 2.5139$、$\lambda = 1.3539$ 1/s，式 (76-10) 给出 $T_d^{*} = 0.6809$ s。三个 Lyapunov 矩阵的最大特征值分别是 $\lambda_{\max}(\boldsymbol{P}_1) = 0.6625$、$\lambda_{\max}(\boldsymbol{P}_2) = 0.8178$、$\lambda_{\max}(\boldsymbol{P}_1 + \boldsymbol{P}_2) = 1.4340$。$\mu$ 之所以只有 2.5，就是因为两个矩阵的“大小”接近，能量函数的形状差不大。

实测的最低驻留是 0.05 s，代价 3.1261、末值 6.96×10^{-4}，完全没有失稳。判据要求 0.6809 s，实测 0.05 s 就够，保守 13.6 倍。这个倍数可以这样理解：判据假设“每次切换都取最坏的能量跳升”（即每次都跳 μ 倍），而实际切换时状态往往落在能量跳升很小的方向上。**判据是“对所有可能状态取最坏”，实测是“沿一条特定轨迹”，两者本来就该有差距。**

工程上怎么用这个差距？分两种情形。若工艺相位大于 T_d^{*}，直接用，不用再论证；若小于，T_d^{*} 不能作为否定依据，但也不能直接忽略，正确做法是**用仿真扫一遍相位**，看最低可行相位在哪里，再用实测定标。本章的扫描（表 76-4）显示，从 0.05 s 到 2.00 s 全部收敛，最低可行相位低于 0.05 s，比判据宽松一个数量级以上。

76.6.3 抖振：代价看不出来，磨损看得出来

表 76-8 是滞环扫描的完整数据，图 76-7 是它的图形版。

滞环 w	无噪切换	无噪代价	有噪切换	有噪代价
0.00	20	2.9982	1662	2.9984
0.01	20	2.9983	828	2.9985
0.02	20	2.9983	362	2.9984
0.05	20	2.9981	28	2.9983
0.10	20	2.9968	20	2.9976

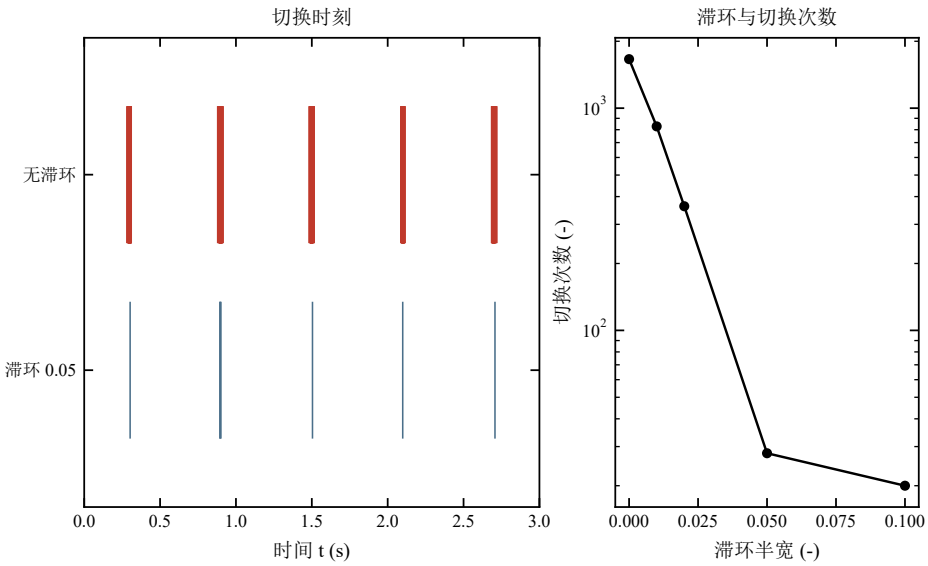


图 76-7 抖振的两种看法：左图是切换时刻栅格，右图是滞环带宽与切换次数

这张表最值得注意的一列是“有噪代价”。从 1662 次切换到 20 次，切换次数降了 83 倍，代价只从 2.9984 变到 2.9976，差 0.03%。**抖振在二次代价上几乎完全看不出来。**

原因是抖振发生在指示量过零点附近，此时状态偏差已经很小，切换带来的输入跳变也很小，w^2 项的增量微不足道。换句话说，抖振不是“性能问题”，而是“寿命问题”：1662 次切换意味着 12 s 内阀门动作 1662 次，折算到一年就是上千万次，任何执行器都扛不住。

这个结论改变了设计的优先级。如果只看代价曲线，工程师会认为“滞环加不加无所谓”；实际上滞环是必须加的，理由不在 J，而在 J 看不见的地方。**评价切换系统至少要两个指标：一个连续的（代价、误差），一个计数的（切换次**

数、阀门行程）。只看前者会漏掉最致命的问题。

图 76-7 左半张把这件事画得很直接：无滞环时切换时刻在 0 到 3 s 内密成一片竖线，加 $w=0.05$ 后只剩十来根。右半张是对数坐标下的切换次数，w 从 0 到 0.10，次数从 1662 单调降到 20，与式 (76-13) 的趋势一致，但下降的速度比 $1/w^2$ 慢——因为噪声不是白噪声，指示量也不是匀速穿越。

无噪那一列还有一个小细节：$w=0.10$ 时代价 2.9968，比 $w=0$ 的 2.9982 低 0.05%。滞环让切换推迟了一点点，恰好避开了一个代价较高的接缝位置。这个差异在噪声下会被淹没，不应过度解读。

76.6.4 变体对象：没有共同 Lyapunov 会怎样

变体对象的结构在 §76.2.2 已经给出：两个闭环矩阵都是双重特征值 -0.5 的 Jordan 块，方向相反，且都不正规。同一组增益 $\boldsymbol{K}_V=[0.20,\,0]$ 用在两个模态上，所以“分别稳定”这一条成立，但共同 Lyapunov 的三个候选全为正（表 76-2 右列），连 $\boldsymbol{P}=\boldsymbol{I}$ 都是 $+1.0000$，这条路彻底不通。

表 76-9 是它的完整驻留扫描。

相位 T (s)	代价 J	状态峰值	末值
0.05	1.5830	1.2804	3.12×10^{-3}
0.10	1.5100	1.2804	3.07×10^{-3}
0.20	1.3746	1.2804	3.00×10^{-3}
0.50	1.0398	1.2804	3.17×10^{-3}
1.00	0.7744	1.2804	5.27×10^{-3}
2.00	47.8791	3.9747	3.97×10^{0}
固定模式 1	3.6744	1.2804	4.52×10^{-2}
固定模式 2	12.8755	1.8563	6.15×10^{-2}

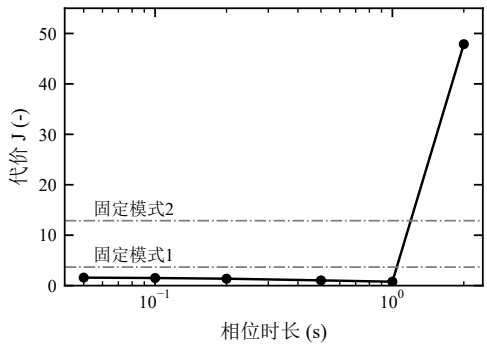


图 76-8 变体对象的相位扫描：$T\le1.00$ s 时切换优于任一固定模式，$T=2.00$ s 时发散

这张表有三层信息。

第一层：$T\le1.00$ s 时，切换优于任一固定模式。 $T=1.00$ 的代价 0.7744，固定模式 1 是 3.6744，固定模式 2 是 12.8755，切换好 4.7 倍以上。这与主对象的结论完全相反——主对象上“不切换最好”，变体对象上“必须切换”。

机制的差别在于增益。主对象有两组增益，各自管一个分量，切换时总有一个分量没人管；变体对象只有一组增益，它在每个模态里都有一个“压不住”的方向，交替切换恰好让另一个模态把这个方向接管过去。**所以“切换好不好”没有统一答案，要看切换带来的是“打断”还是“互补”。**

第二层：T 从 0.05 增到 1.00，代价单调下降。 $1.5830\to0.7744$，降了 51.1%。这一段与主对象的趋势一致：相位越长，切换越少，接缝的损失越小。到这里为止，一切符合直觉。

第三层：$T = 2.00$ 时突然发散。 代价 47.8791、峰值 3.9747、末值 3.97，状态范数一路涨到初值的 3 倍以上。这个转折不能用“驻留太短”解释，因为 2.00 s 比 1.00 s 更长。真正的机制是**非正规矩阵的瞬态增长**：

$$\left\| e^{\boldsymbol{C}_{1V} t} \right\| = e^{-0.5t}(1 + 2t) \tag{76-27}$$

它在 $t = 1.5$ s 处达到峰值 1.89 倍，之后才回落。这个函数说明：即使特征值全是 -0.5，状态范数也可以先涨后跌，涨幅取决于初始方向。$T = 2.00$ s 时每段驻留正好覆盖整个增长区间，切换时状态已经被推到“最不利方向”，下一段从这个方向继续放大；$T = 1.00$ s 时切换发生在增长尚未充分展开之前，反而把这个方向“掐断”了。

这就是非正规系统最反直觉的地方：**稳定性不能只看特征值，还要看特征向量之间的夹角**。两个 Jordan 块方向相反，意味着“一个模态的衰减方向正好是另一个模态的增长方向”，切换节奏一旦对上这个结构，就会共振。

工程上的应对办法有两条。其一，**不要用单一增益去控制两个差异很大的模态**，这正是主对象用两组增益的原因。其二，**若不得不用，驻留时间要做双向扫描**——既扫“太短”（抖振、接缝密集），也扫“太长”（瞬态增长被完整激发）。只扫一个方向会漏掉 $T = 2.00$ 这种“越长越坏”的点。

76.6.5 稳定性与性能的取舍

把主对象与变体对象放在一起看，可以总结出一条取舍原则：**稳定性问题的答案在结构里，性能问题的答案在节奏里。**

结构指的是“有几组增益、几套模型、有没有共同 Lyapunov”。这些一旦定下来，系统的稳定域就定了，调参数改不了。主对象有两组增益 + 共同 Lyapunov，所以怎么切都稳；变体对象一组增益 + 无共同 Lyapunov，所以某些节奏必然发散。

节奏指的是“相位多长、检测多快、滞环多宽”。这些决定性能：主对象上相位从 0.05 拉到 2.00，代价从 3.1261 降到 2.2322；延迟从 0 加到 0.50，代价从 2.8892 涨到 11.3721。节奏调得再好，也突破不了结构给的上界（最好的单模态 1.8835）。

所以设计顺序应该是：先定结构（模态怎么分、增益怎么配），再验证结构（共同 Lyapunov），最后调节奏（相位、检测、滞环）。反过来做——先调节奏再补结构——就会陷入“怎么调都有一头不对”的局面，这正是现场最常见的困境。

76.6.6 增益结构：一个增益管一个分量

两组增益的数值差异很大，值得单独看一眼，因为它解释了“为什么分模态整定能奏效”，也解释了“为什么切换瞬间会有跳变”。

模式 1 的开环矩阵是 $\boldsymbol{A}_1 = \begin{bmatrix} 0.75 & 0.30 \\ 0.10 & -1.10 \end{bmatrix}$，第一行对角元为正，说明 z_1 方向不稳定，而 z_2 方向的 -1.10 已经很快。对应的 LQR 增益 $\boldsymbol{K}_1 = [1.9853, 0.3075]$ 把绝大部分权重压在第一分量上：z_1 的系数是 z_2 的 6.46 倍。模式 2 恰好相反：$\boldsymbol{A}_2$ 的第二行对角元 $+0.65$ 不稳定，$\boldsymbol{K}_2 = [0.2292, 1.8740]$ 的权重落在第二分量，z_2 的系数是 z_1 的 8.18 倍。

换一个角度看同一件事：**跨模态比较，同一个分量上的增益差得更悬殊**。z_1 上 $\boldsymbol{K}_1$ 是 $\boldsymbol{K}_2$ 的 8.66 倍，z_2 上 $\boldsymbol{K}_2$ 是 $\boldsymbol{K}_1$ 的 6.09 倍。这就是 §76.9.3 所说的“8.7 倍与 6.1 倍”，也是切换瞬间输入必然跳变的原因。

用整段的话说：**每个模态只有一个不稳定方向，两个模态的不稳定方向互相正交，所以每组增益天然地“只管一个分量”。** 这不是巧合，而是 LQR 在 $\boldsymbol{Q} = \boldsymbol{I}$ 下的必然结果——权重为 1 的代价函数对两个分量一视同仁，控制器就把力气全花在“不压住就会跑掉”的那一个上，另一个只做轻微修正（0.3075 与 0.2292）。

这个结构带来两个直接后果。

好消息是段内性能好。 每段只需要压住一个分量，另一个分量在对应模态下本来就在衰减，控制器不必分心。表 76-4 里 $T = 2.00$ 时代价能降到 2.2322，接近最好的单模态 1.8835，就是因为段内足够长、各自的增益有充足时间把该压的分量压下去。

坏消息是接缝处跳变大。 两个增益几乎不重叠，切换瞬间输入从 $-\boldsymbol{K}_1\boldsymbol{z}$ 跳到 $-\boldsymbol{K}_2\boldsymbol{z}$，差值 $(\boldsymbol{K}_2 - \boldsymbol{K}_1)\boldsymbol{z}$ 在初值处达到 3.0093（§76.9.2）。这个跳变不是“调参没调好”，而是“分区整定”的固有代价：两个区域各自最优，接缝上必然不连续。

理解了这一点，就能判断哪些改进值得做。**把两个增益调得更接近**（例如给 $\boldsymbol{Q}$ 加权、或对增益做插值）能减小跳变，但会牺牲段内性能——因为接近意味着某一组不再是最优。**在接缝处做软过渡**（§76.9.2 的一阶滤波）不改段内性能，只把跳变摊到几十毫秒里，代价是过渡期内闭环既不是 $\boldsymbol{C}_1$ 也不是 $\boldsymbol{C}_2$，稳定性论证要重新做。两条路的取舍在 §76.9 展开。

最后一个观察：两个增益的“作用强度”可以从 $\boldsymbol{K}_i\boldsymbol{z}_0$ 看出来，本章是 1.7393 与 -1.2700。前者恰好等于单模态 1 的输入峰值。这不是巧合——初值处输入最大，而模式 1 从 $t=0$ 就开始作用，所以整个仿真的输入峰值就出现在第一拍。这也解释了为什么互锁阈值按“峰值的 0.5 到 0.8 倍”来定：峰值是第一拍的瞬态，正常调节时的输入远小于它。

76.7 参数意义与作用

76.7.1 七个参数与一个约束

切换律上有七个可以动的参数，另有一个不能动、只能测的约束。七个参数列在表 76-10，最后一列是本章哪一组数据能支撑它的取值。

参数	符号	本章取值	调大的后果	支撑数据
相位长度	T	0.60 s	代价降、接缝少、段末偏差大	表 76-4
滞环半宽	w	0.05	抖振少、判决迟钝	表 76-8
判决阈值	q_{th}	0.50	偏保守、切换推迟	§76.7.3
决策周期	h	0.05 s	抖动少、滞后大	表 76-6
确认计数	N	2	抗脉冲噪声、滞后大	§76.7.5
互锁阈值	u_{th}	1.0	拦截少、冲击大	表 76-11
起始相位	ϕ_0	0	改变首段长度与接缝位置	§76.4.4

那个约束是模式识别延迟 θ_m。它由传感器、变送器、PLC 扫描周期共同决定，控制侧改不了，只能测出来再“绕着走”。它出现在式 (76-19) 的右边，是决定相位下限的主要项。

七个参数可以分成两组：**时间组**（T、h、N）与**幅值组**（w、q_{th}、u_{th}），ϕ_0 单独属于“相位对齐”。两组的作用通道不同：时间组决定“多久判一次、判完保持多久”，幅值组决定“多大的偏差才算数”。调参时先定时间组再定幅值组，因为时间组的量级被工艺锁死，幅值组则有较宽的调节余地。

76.7.2 相位长度 T

T 是整个切换律的节拍，也是唯一直接由工艺决定的参数。本章 $T=0.60$ s 是工艺给定，控制侧只能在“是否允许微调”的范围内动它。

调大 T 的收益在表 76-4 里很清楚：$0.60\to 2.00$，代价从 2.8892 降到 2.2322，降 22.7%；切换次数从 19 降到 5。代价是每段末端的状态偏差变大——因为段内只有一个增益，时间越长，另一个分量的积累越多。

调小 T 的代价同样是单调的：$0.60\to 0.05$，代价涨 8.2%，切换次数从 19 涨到 239。但要注意左端的平台：0.05 与 0.10 的代价完全一样，说明再往下调没有性能收益，只有磨损。

工程上怎么定？若工艺允许，取**慢模态时间常数的 1 到 3 倍**。本章慢模态时间常数约 0.74 s，对应 0.74 到 2.2 s，表 76-4 里 $T=1.00$ 与 2.00 都在这个区间，代价分别是 2.6035 与 2.2322，明显优于 0.60。若工艺只给 0.60 s，那就接受 2.8892，并把注意力转到延迟与抖振上——那两项的改善空间比相位大得多。

还有一个容易被忽略的点：T 不一定是常数。批次反应器的升温段与恒温段长度本来就不同，强行取等长会浪费。若两段长度不同，式 (76-10) 的驻留判据要对两段分别检查，取较严的那个；本章为了简化，两段等长，所以只算一个 T_d^{*}。

76.7.3 判决阈值 q_{th}

阈值决定“指示量过多少才算进入下一段”。本章取 0.50，是指示量 $q(t)$ 的几何中点。

阈值偏移的理由是**误判代价不对称**。把加热段误判成冷却段，阀门关蒸汽、开冷却水，反应可能骤停甚至飞温；把冷却段误判成加热段，只是多通一点蒸汽、多耗一点能量。前者的代价远大于后者，所以阈值应当向“更难判成加热”的方向偏，即提高进入加热段的门槛。

偏移量有一个硬限制：**不能超过滞环半宽 w**。因为滞环的两条边界是 $q_{\text{th}} \pm w$，若 q_{th} 偏得比 w 还多，两条边界会越过指示量的实际取值范围，某一段就永远判不进去。本章 $w = 0.05$，所以阈值最多偏 0.05，即 $q_{\text{th}} \in [0.45,\, 0.55]$。

这条限制说明一个设计上的耦合：**想让阈值多偏，就得先把滞环放大**，而滞环放大会增加判决滞后。三者（阈值、滞环、滞后）是绑在一起的，不能单独调一个。本章为了让对比干净，阈值固定在 0.50，滞环单独扫描；实际工程中若误判代价严重不对称，应先按安全要求定阈值偏移，再按剩下的余量定滞环。

76.7.4 滞环半宽 w

w 决定“指示量在阈值附近多大范围内不动作”。它的作用在表 76-8 里是最清楚的：$w = 0$ 时噪声造出 1662 次切换，$w = 0.02$ 降到 362，$w = 0.05$ 降到 28，$w = 0.10$ 降到 20（与无噪一致）。

推荐的取值是**噪声标准差的 2 到 3 倍**。本章噪声 $\sigma_n = 0.02$，对应 $w = 0.04$ 到 0.06，表 76-8 里 $w = 0.05$ 给出 28 次，已经接近理想的 20 次。这个经验规则的道理很直接：滞环要盖住噪声的典型波动幅度，1σ 盖不住（约 32% 的样本会越界），2σ 能盖住 95%，3σ 能盖住 99.7%。

w 不能无限加大。它的副作用是**判决滞后**：指示量要越过 $q_{\text{th}} + w$ 才切换，相当于把切换时刻往后推。按本章的指示量斜率，$w = 0.05$ 大约推迟 0.02 s，可以忽略；$w = 0.20$ 会推迟近 0.1 s，就与识别延迟同量级了。经验上限是 $w \le 0.2$，超过这个值，滞环带来的滞后会开始吃掉性能。

还有一个隐蔽的副作用：w 太大会**掩盖真实的快速切换**。若工艺真的在 0.1 s 内来回切了两次，$w = 0.20$ 的滞环会把第二次切掉，控制器一直停在旧模态。这种漏判在正常工艺下不会发生，但在启停、故障、手动干预时会。所以 w 的上限还要受“最快真实切换间隔”约束：w 对应的滞后必须小于该间隔的一半。

76.7.5 决策周期 h 与确认计数 N

h 是判决的时间分辨率。表 76-6 的结论是“代价不敏感、次数敏感”：h 从 0.005 到 0.20，代价只差 0.48%，切换次数从 44 降到 20。

推荐取 $T/10$ 到 $T/20$。本章 $T = 0.60$，对应 0.03 到 0.06，表 76-6 的 $h = 0.05$ 落在区间内，代价 2.9913、切换 20 次，是最平衡的一行。

N 是确认计数：连续 N 次判决都要求切换，才真的切。它的作用是抗脉冲噪声——单次尖峰不足以推动切换。N 的代价可以用等效延迟表示：

$$\theta_m^{\text{eff}} \approx \theta_m + (N-1)h \tag{76-28}$$

$N = 2$、$h = 0.05$ 时额外增加 0.05 s 延迟，相当于把识别延迟从 0 拉到 0.05 s，按表 76-5 代价涨 11.3%。$N = 3$ 再增加 0.05 s，累计 0.10 s，代价涨 24.4%。所以 N 不宜超过 3；若噪声确实很大，应先加大 w 或改善传感器，而不是继续加 N。

N 与 w 的分工要分清：w 过滤的是**小幅波动**（幅度小于 $2w$ 的抖动不触发），N 过滤的是**短时波动**（持续时间小于 Nh 的越界不触发）。白噪声两种都怕，脉冲噪声主要靠 N，偏置漂移主要靠 w（或阈值）。诊断时先看噪声的时域波形：连续抖动多就加 w，孤立尖峰多就加 N。

76.7.6 互锁阈值 u_{th}

u_{th} 是“输入超过多少就不许切”。表 76-11 是它的完整扫描。

方案	代价 J	输入峰值	切换次数	累计拦截时间
无互锁	2.9984	1.7393	1662	0.00 s
互锁 $u_{\mathrm{th}}=1.5$	2.9984	1.7393	1660	0.00 s
互锁 $u_{\mathrm{th}}=1.0$	2.9894	1.7393	1504	0.11 s
互锁 $u_{\mathrm{th}}=0.6$	2.7994	1.7393	1504	0.74 s
仅限幅 ±1.0	2.7578	1.0000	1662	0.00 s

$u_{\mathrm{th}}=1.5$ 几乎没起作用：切换次数只从 1662 降到 1660，拦截时间为 0。因为正常工况的输入很少超过 1.5，这条互锁常年不动作。$u_{\mathrm{th}}=1.0$ 开始起作用：拦截 0.11 s，次数降到 1504，代价降 0.3%。$u_{\mathrm{th}}=0.6$ 拦得更狠：累计拦截 0.74 s，次数仍是 1504，代价降到 2.7994，降 6.6%。

推荐取值是**输入峰值的 0.5 到 0.8 倍**。本章峰值 1.7393，对应 0.87 到 1.39，$u_{\mathrm{th}}=1.0$ 落在区间内。低于 0.5 倍要小心：正常调节过程中输入经常超过这个值，互锁会长期生效，切换被无限推迟，系统实际上退化成单模态运行。

表 76-11 最后一行是“仅限幅”，用来做对照。它把输入截到 ±1.0，峰值从 1.7393 降到 1.0000，代价 2.7578（最低），但切换次数仍是 1662。这说明限幅与互锁的机制完全不同：限幅省的是能量，互锁省的是接缝。两者可以叠加，本章为了分离变量，只单独测。

76.7.7 起始相位 ϕ_0

ϕ_0 决定第一次切换发生在什么时候。它的影响在 §76.4.4 已经量化：$\phi_0=0$ 对应首次切换在 0.6 s，代价 2.8892；理想检测的首次切换在 0.3 s，代价 2.9982，差 3.8%。

3.8% 在工程上不算大，但它是**系统性的**，不是随机波动：只要初值方向不变，这个差异就在每一批里重复。多批次累计下来，能耗差异是可观的。

取值的规则是**让首段使用与初值方向匹配的增益**。判断办法是看 $\boldsymbol{K}_i\boldsymbol{z}_0$ 的符号与大小：本章 $\boldsymbol{K}_1\boldsymbol{z}_0=1.7393$、$\boldsymbol{K}_2\boldsymbol{z}_0=-1.2700$（绝对值），模式 1 对初值的“作用”更强，且方向与 z_1 的衰减需求一致，所以从模式 1 起步更合适。若初值反过来（$z_1<0$、$z_2>0$），结论也会反过来。

ϕ_0 通常不是自由参数——批次从哪个阶段开始由排产决定。但“首次切换时刻”可以在检测器里人为延迟或提前：例如工艺从加热段开始，检测器可以设定在 0.6 s 而不是 0.3 s 才认第一次切换。这个操作等价于调 ϕ_0，代价只是首段稍微长一点。工程上这是最容易被忽略、又最容易拿到的 3% 到 4% 收益。

76.7.8 第八个参数：限幅 $u_{\max}$

前七节讲的是切换律的参数，还有一个参数不属于切换律，却总在调试时被一起改：执行器限幅 $u_{\max}$。它不出现在检测器里，而是出现在增益后面，作用是把 $-\boldsymbol{K}_i\boldsymbol{z}$ 截到 $[-u_{\max},u_{\max}]$。

表 76-11 最后一行给出了它的单独效果：截到 ±1.0 后，输入峰值从 1.7393 降到 1.0000，切换次数仍是 1662，代价 2.7578。这个 2.7578 比无互锁的 2.9984 低，容易被误读成“限幅改善了性能”。**不能这样读。** 本章的代价函数里有一项 Ru^2，限幅强制把 u 压小，这一项自然下降；而状态项 $\boldsymbol{z}^{\mathsf{T}}\boldsymbol{Q}\boldsymbol{z}$ 会上升，只是上升幅度没有被单独列出来。用同一个 J 比较“限幅前”与“限幅后”，本质上是拿一个奖励“少用力”的指标去评价一个“被迫少用力”的方案，结论没有意义。

正确的问题是：**限幅之后状态还能不能收敛到工艺要求的精度。** 表 76-11 没有列末值，这本身是个缺口；现场评估限幅必须补上末值与峰值两个数，而不是看 J。若限幅太狠，状态会在末端停在一个偏差上，表现为“温度总是差两度到不了”，这时加多少滞环都没用。

限幅与互锁的分工也要分清。**限幅改的是输入，不改切换时刻**（次数仍是 1662）；**互锁改的是切换时刻，不改输入**（峰值仍是 1.7393）。前者防的是“执行器被打坏”，后者防的是“切换时冲击”。两者叠加的正确顺序是：先按执行器能力定 $u_{\max}$，再在这个 $u_{\max}$ 下看互锁阈值该取多少——因为限幅会改变输入的实际波形，互锁判据若按未限幅的峰值定，就会偏保守。

还有一个常被忽略的耦合：限幅会让两个模态的“有效增益”不再等于 $\boldsymbol{K}_i$。输入饱和后，闭环实际上变成了非线性系统，§76.2 的 Lyapunov 论证只在饱和区外成立。本章工况下峰值 1.7393 只略高于 $u_{\max}=1.5$，饱和时间很短，影响可以忽略；若 $u_{\max}$ 定到 0.5，系统会长时间工作在饱和区，那时必须用饱和系统的稳定性工具重新分析，不能直接套用本章结论。

76.8 调参方法与经验

76.8.1 四轮调参，不许跳轮

七参数一起调是调不出来的，因为它们的敏感度差三个数量级。本章按影响从大到小把主要因素排了个序（表 76-12），调参必须按这个顺序走。

优先级	参数/约束	对代价的影响幅度	可调性
1	识别延迟 θ_m	$+293.6\%$	不可调，只能改硬件
2	相位 T	$+8.2\%$（$0.60\to0.05$）	工艺给定，可小幅议价
3	互锁阈值 u_{th}	-6.6%（$1.0\to0.6$）	可调
4	起始相位 ϕ_0	3.8%	可调
5	决策周期 h	0.48%	可调
6	滞环 w	0.03%（代价），但影响次数 83 倍	可调

这张表传达的第一个信息是：**延迟是压倒性的第一因素**。它不可调，只能通过换传感器、缩短变送器阻尼、提高扫描频率来降。若延迟已经占到相位的 $1/3$ 以上，再怎么调其他五个参数也救不回来——因为其他参数的合计影响只有百分之十几。

第二个信息是：**滞环对代价几乎没有影响，但对切换次数影响极大**。所以调滞环的目标不是降代价，而是降次数；评价它要用“切换次数”这个指标，不能用 J。

四轮调参的分工如下。第一轮定结构（模态、增益、共同 Lyapunov），这轮不调参数，只做判断。第二轮定时间组（T、h），因为这两项被工艺与硬件锁死，能动的余地小。第三轮定幅值组（w、q_{th}、u_{th}），用噪声数据扫。第四轮定 ϕ_0 与 N，做最后的微调。

76.8.2 单变量扫描：先粗后细

每一轮里只改一个参数，其他固定，这叫单变量扫描。它的好处是能把“哪个参数造成的变化”分离出来，坏处是忽略了参数间的耦合。本章的做法是：先用粗网格扫出趋势，再在最优点附近用细网格确认。

以滞环为例。粗扫取 $w\in\{0,\,0.01,\,0.02,\,0.05,\,0.10\}$，得到表 76-8 的五行。趋势很清楚：$0$ 到 0.02 降得快，0.05 之后基本饱和。细扫就在 0.03 到 0.07 之间补点，确认饱和点在哪里。

粗网格的间距要按“数量级”取，不要按等差数列。本章的相位扫描 $0.05, 0.10, 0.20, 0.40, 0.60, 1.00, 2.00$ 就是按倍数取的，跨越两个数量级只用了七个点；若按 0.1 的等差取，会漏掉 0.05 这个关键的平台点。

细扫的判据是**变化小于噪声种子带来的波动就停**。本章的噪声实验用固定种子，重复性好；现场没有固定种子，同一组参数跑两次的切换次数可能差 5% 到 10%，所以细扫的差异必须大于这个范围才有意义。

76.8.3 五种典型症状与处置

调参时遇到的症状可以归成五类，每一类对应不同的根因。

症状一：切换次数远超预期。 本章理想值是 20 次，实测若得到 1662 次，差 83 倍，根因几乎一定是滞环不足或噪声过大。处置顺序是：先测噪声标准差 σ_n，取 $w=2\sigma_n$ 到 $3\sigma_n$；若切换次数仍高，再查指示量是否在阈值附近有系统性抖动（例如传感器本身在某个温度段不稳定）。

症状二：切换次数为 0 或极少。 这不是好事。常见根因有三个：阈值偏得太多，某一段永远判不进去；互锁阈值太低，切换被长期拦截；指示量根本没有穿越阈值（例如用时间做守卫但时间基准漂了）。诊断办法是打印指示量与阈值的时序，看它是否真的越过了 $q_{\mathrm{th}} \pm w$。

症状三：切换次数正常，但代价偏高。 根因通常是相位太短或延迟太大。区分办法是看末值：若末值也高，说明收敛能力不足，偏向延迟；若末值正常、代价高，说明接缝损失大，偏向相位。本章的判据是：延迟 $0.20\ \mathrm{s}$ 时末值 1.52×10^{-2}、代价 4.5764；相位 $0.05\ \mathrm{s}$ 时末值 6.96×10^{-4}、代价 3.1261。前者末值高一个数量级，很好区分。

症状四：末值不收敛。 这是最危险的症状。先查有没有共同 Lyapunov；若有，说明不是稳定性问题，而是相位或延迟把系统推到了“勉强收敛”的边缘；若没有，直接进 §76.6.4 的双向驻留扫描，找出发散的相位区间。变体对象在 $T = 2.00\ \mathrm{s}$ 时末值 3.97，就是典型的“驻留过长反而发散”。

症状五：切换时刻与工艺不符。 根因是多个延迟叠加：识别延迟 θ_m、决策滞后 $h/2$、确认计数 $(N-1)h$、滞环推迟（约 $w/\dot{q}$）。把它们加起来对比相位，若总量超过相位的 $1/3$，切换时刻就会明显落后。处置办法是减小贡献最大的一项，通常是最容易改的 N 或 h。

76.8.4 调参记录模板

调参过程必须留记录，否则改到第五轮就说不清哪一轮有效。表 76-13 是本章用的模板，每改一次参数填一行。

轮次	改动	切换次数	代价 J	末值	结论
0	基线（$w=0$，无噪）	20	2.9982	6.1×10^{-4}	参照
1	加噪声 0.02	1662	2.9984	—	抖振暴露
2	$w = 0.02$	362	2.9984	—	降 4.6 倍
3	$w = 0.05$	28	2.9983	—	接近理想
4	加驻留 $0.05\ \mathrm{s}$	20	2.9974	—	达到理想
5	加互锁 $u_{\mathrm{th}} = 1.0$	1504	2.9894	—	次数反常，见注

第 5 行的“次数反常”值得解释：加互锁后次数从 20 跳回 1504，是因为这一行的基线换成了“无滞环 + 有噪声”的场景（表 76-11 的口径），不是第 4 行的场景。**记录模板最容易犯的错就是换了基线却没标注**，导致后一行与前一行不可比。本章在表 76-11 里专门注明“检测噪声 0.02，$w=0$”，就是为了避免这个混淆。

模板还有一列没有列出来但必须记：**噪声种子或噪声实测值**。本章所有带噪实验用固定种子（`seed=7` 或 `seed=3`），所以结果可复现；现场没有种子，只能记“当班噪声水平”，并注明测量时段。没有这一列，两次记录的差异分不清是参数效果还是噪声波动。

76.8.5 六条容易踩的坑

坑一：拿单模态的最优去要求切换系统。 本章最好的单模态是 1.8835，任何切换方案都到不了。若把 1.8835 当目标，会永远觉得“没调好”。

坑二：只看代价不看次数。 抖振在代价上只差 0.03%，但次数差 83 倍。只看代价会漏掉执行器寿命问题。

坑三：把限幅当互锁。 限幅改输入、不改切换；互锁改切换、不改输入。本章实测两者次数一个 1662、一个 1504，机制完全不同（表 76-11）。

坑四：忽略相位对齐。 同样 $0.60\ \mathrm{s}$ 相位，首次切换在 $0.3\ \mathrm{s}$ 与 $0.6\ \mathrm{s}$ 的代价差 3.8%。比较两种方案不固定相位，结论不可信。

坑五：以为 h 越小越好。 $h = 0.005$ 的切换次数是 44，$h = 0.02$ 是 20，更快的采样带来更多抖振（表 76-6）。

坑六：把多 Lyapunov 下界当设计目标。 判据给 $0.6809\ \mathrm{s}$，实测 $0.05\ \mathrm{s}$ 就够。按下界设计会白白拉长相位，牺牲性能。

76.8.6 什么时候该停止调参

调参没有“最优”，只有“够用”。判断是否该停，本章给三条可以量化的标准，任何一条满足就该收手。

第一条：三个指标都已进入平台。 把某个参数在推荐值附近再动 20%，代价变化小于噪声带来的波动（本章约 0.03%），次数变化不超过理想值的一个切换周期，就说明已经到平台。表 76-4 左端 $T=0.05$ 与 $T=0.10$ 代价完全相同，就是典型的平台。

第二条：剩下的差距已经小于结构上限。 本章最好的单模态 1.8835，切换方案最好到 2.2322（$T=2.00$）。这个差距来自“两个增益必须轮流用”，不是参数能消掉的。若继续调的目标是“追平单模态”，那永远停不下来，因为目标是错的。

第三条：改善的收益已经低于改动的风险。 一次参数改动要重新验证、重新写记录、重新培训操作工。若预期收益只有百分之一二，而改动会引入新的判决分支，那就不值得。本章在 §76.9.7 给过同样的判断：四件基础事已经把次数从 1662 降到 20，再加一层优化，收益在千分位，风险在“说不清为什么切”。

三条标准之外还有一条经验：**不要在批次之间调参**。同一个参数在升温段与降温段的敏感度不同，跨批次调参会把“批次差异”与“参数效果”混在一起，最后谁也说不清。要调就在一个批次内做单变量扫描，调完再从下一个批次开始验证。

76.9 优化与改进

76.9.1 三个目标与帕累托前沿

切换系统的优化不是单目标问题。至少要同时看三件事：稳定性余量、代价 J、切换次数 N_{sw}。三者受同一组参数影响，但方向不一致。

以互锁与限幅为例，表 76-11 的数据可以画成一条权衡曲线：仅限幅给出最低代价 2.7578，但切换次数是 1662；互锁 $u_{\mathrm{th}}=0.6$ 给出较少的 1504 次，代价却是 2.7994。两个点互不支配——一个代价低、一个次数少。这就是帕累托前沿上的两个点。

优化的第一件事是**明确你要在前沿的哪一段**。若执行器寿命充足、能耗是主要成本，选低代价端；若阀门检修昂贵、能耗占比小，选低次数端。没有“全面最优”的参数组，只有“针对某个成本结构最优”的参数组。

本章的默认配置（$w=0.05$、$T_d=0.05\ \mathrm{s}$、$u_{\mathrm{th}}=1.0$）落在中间偏次数端：20 次切换、代价 2.9974。它不是一个“最优点”，而是一个“两头都不难看”的折中。

76.9.2 增益过渡：从硬切换到软切换

硬切换最直接的副作用是输入跳变。本章的输入在切换瞬间从 $-\boldsymbol{K}_1\boldsymbol{z}$ 跳到 $-\boldsymbol{K}_2\boldsymbol{z}$，跳变量是 $(\boldsymbol{K}_2-\boldsymbol{K}_1)\boldsymbol{z}$。若在 $t=0$ 就切，这个量是 3.0093，比输入峰值 1.7393 还大；实际切换发生在状态已衰减之后，跳变量随之减小，但仍是执行器能听到的“咔哒”声。

软切换（bumpless transfer）的思路是让输入连续：切换瞬间保持输入不变，之后用一阶滤波让它平滑趋近新增益的输出，

$$u_{\mathrm{c}}(t_k^{+})=u(t_k^{-}),\qquad \dot{u}_{\mathrm{c}}=\alpha\big(-\boldsymbol{K}_{\sigma}\boldsymbol{z}-u_{\mathrm{c}}\big) \tag{76-29}$$

α 是过渡速率，$\alpha\to\infty$ 退化为硬切换，$\alpha\to 0$ 等于不切换。

这个做法的收益是冲击小、执行器寿命长；代价是过渡期内控制量不是最优的，偏差会多积累一点。过渡时间约 $3/\alpha$，要显著小于相位，否则过渡还没完成就又切换了。

软切换与互锁是两件不同的事，别混。互锁改**什么时候切**，软切换改**切了之后输入怎么走**。前者避开最坏时刻，后者削平跳变幅值。两者可以叠加，本章没有叠加，因为想把“时刻”与“幅值”两个因素分开测。

需要说明的是，本节只给机制，没有给实测数字。原因是软切换的效果高度依赖 α 与执行器动态，而本章的仿真里执行器是理想的（输入即输出），加上滤波反而会掩盖真实问题。要在真实装置上评估软切换，必须把阀门行程特性一起建模。

76.9.3 高频调制：把切换变成占空比

另一个思路是彻底取消“离散切换”，用高频 PWM 在两个模态间调制，让对象看到的是平均动态（式 76-11）。这在执行器允许高频动作时很有效：等效于一个连续可调的混合系统，接缝被“抹平”。

但本章的数据给这个思路泼了一盆冷水。$T=0.05$ s 的代价 3.1261，$T=2.00$ s 是 2.2322，高频端的代价反而更高。原因在 §76.4.2 已经说过：高频调制把接缝变成“密集的小接缝”，总损失没有消失，只是被平摊。$T=0.05$ 与 $T=0.10$ 的代价完全相同，说明再快也不改善——已经进入平台。

所以 PWM 调制的适用前提是：**接缝损失本身很小，而平均化的收益很大**。本章对象的两个增益差异太大（8.7 倍与 6.1 倍），接缝损失本来就大，调制救不了。若换成两个增益接近的模态，调制会有明显收益。

76.9.4 用提前量补偿延迟

延迟 θ_m 的危害是“用旧增益控制新对象”。既然危害来自时间差，最直接的补偿就是把这个时间差提前消掉：在模式真正改变之前 θ_m 就切换。

对时间守卫，这完全可以做到——工艺排产是已知的，提前 θ_m 切换即可。本章延迟 0.10 s 造成 24.4% 的代价上涨，若提前 0.10 s 切，理论上可以把这部分损失消掉。实现上只要把切换时刻从 t_k 改成 $t_k-\theta_m$。

对状态守卫，做不到提前，因为不知道模式什么时候会变。替代方案是用状态预测：拿当前状态外推 θ_m 之后的位置，

$$\hat{\boldsymbol{z}}(t+\theta_m)=e^{\boldsymbol{A}_\sigma\theta_m}\boldsymbol{z}(t) \tag{76-30}$$

再用 $\hat{\boldsymbol{z}}$ 算增益。这个做法的风险是模型误差被指数放大——本章开环矩阵有 $+0.7661$ 的极点，外推 0.10 s 会放大 $e^{0.0766}\approx 1.08$ 倍，还可以接受；外推 0.50 s 放大 1.47 倍，误差已经不可忽略。

与 Smith 预估（第 47、107 章）的关系要说清：Smith 预估补偿的是**传输延迟**，在回路内部串联一个模型；本节补偿的是**模式识别延迟**，在切换律上加提前量。两者数学形式相似（都是“用模型外推”），作用点不同，可以叠加。

76.9.5 驻留时间的自适应

固定驻留时间 T_d 是保守的：它要对所有可能状态成立，而实际状态往往比最坏情况好得多。本章 $T_d^{*}=0.6809$ s 与实测可行的 0.05 s 差 13.6 倍，就是这个保守性的量化。

自适应驻留的想法是让下界随状态变化：

$$T_d^{\mathrm{ad}}=T_d^{\min}+\beta\frac{\|\boldsymbol{z}\|}{\|\boldsymbol{z}_0\|} \tag{76-31}$$

状态离原点远时允许长一点（能量大，需要时间压），离原点近时允许短一点（能量小，多切几次也无妨）。β 是一个正系数，$T_d^{\min}$ 是硬下限。

这个做法的收益在启动段最明显：启动时状态大，多等一会儿；接近稳态时状态小，可以快速响应工艺变化。风险是自适应律本身可能引入新的振荡——T_d^{ad} 随 $\|\boldsymbol{z}\|$ 变化，而 $\|\boldsymbol{z}\|$ 又受切换影响，形成闭环。工程上必须给 T_d^{ad} 加硬上下限，并且用单调性检查（状态范数下降时驻留不应增加）。

76.9.6 与相邻方法的接口

与事件触发（第 75 章）。 第 75 章决定“什么时候发数据”，本章决定“用哪套参数”。两者叠加时有一条硬规则：**模式切换必须强制发送一次**，否则控制器可能还在用旧模式算，而对象已经变了。这相当于把“切换”当成一个最高优先级的事件，不受事件触发阈值约束。

与 MPC（第 58、80 章）。 切换 MPC 的难点是“切换瞬间的可行性”：上一段的最优解在下一段可能违反约束。本章的驻留约束可以直接作为 MPC 的约束加进去，让优化器自己避开“刚切完又切”的解。

与 Koopman（第 77 章）。 Koopman 用一组模态的线性组合描述非线性，本质是“软切换”——每个时刻所有模态都在起作用，只是权重不同。它的好处是没有接缝，坏处是可解释性差。本章的硬切换在这一点上相反。

与容错（第 78 章）。 故障后切换到备用模态，稳定性论证直接复用本章的多 Lyapunov 框架。区别是容错切换是“一次性”的，不需要反复切回，所以驻留约束可以放松。

76.9.7 优化的代价：可解释性与可认证性

每加一层优化，系统的“可解释性”就下降一层。硬切换 + 滞环 + 驻留 + 互锁这四件事，每一个动作都能用一句话说清：“因为指示量越过阈值加滞环，且距上次切换超过驻留，且输入小于互锁阈值，所以切换。”这套逻辑可以写进安全论证文件，也可以通过审查。

软切换要解释“为什么输入这样走”，需要引入滤波器状态；预测补偿要解释“为什么提前切”，需要引入模型；自适应驻留要解释“为什么这次等得短”，需要引入状态范数。三层叠加之后，任何一次切换都很难用一句话说清。

工业认证（尤其是安全仪表系统）对这一点很敏感。所以务实的建议是：**先把四件基础事做到位，只有在冲击成为主要矛盾时才上软切换，只有在延迟无法降低时才上预测**。本章的实测支持这个优先级——四件基础事把切换次数从 1662 降到 20、代价从 2.9984 降到 2.9974，收益已经接近饱和。

76.9.8 从两模态到多模态：规模上去会怎样

本章只有两个模态，所有论证都建立在“一对矩阵”上。真实装置常有三个以上：升温、恒温、降温、清洗、待机。规模一变，工作量不是线性增加，而是按组合数增加。

共同 Lyapunov 的检验量按模态数线性增长。 N 个模态要检查 N 个不等式 $\boldsymbol{C}_i^{\mathsf{T}}\boldsymbol{P}+\boldsymbol{P}\boldsymbol{C}_i \prec 0$，这还好。难的是找到 $\boldsymbol{P}$：候选 $\boldsymbol{P}_1+\cdots+\boldsymbol{P}_N$ 可能不成立，需要退回 LMI 求解器；而 LMI 无解并不等于“不存在”，只等于“这个候选不行”，工程上无法据此下结论。

驻留约束按有序对增长。 两模态只有 $1\to 2$ 与 $2\to 1$ 两个方向，各算一个 μ；三模态有 6 个有序对，四模态有 12 个。每个有序对的跳升比不同，驻留下界也不同，最终要取**所有有序对的最大值**。这个最大值往往比两模态时大不少，因为总有一对“能量形状差得最远”。结果是：模态分得越细，切换被迫越慢，性能反而不如分粗一点。

检测器的复杂度按边界数增长。 两模态只需一条阈值加一条滞环带；N 模态若用“最近邻”判决，需要 $N-1$ 条边界，每条边界都有自己的滞环与误判代价。更麻烦的是**判决顺序**：三个模态可能构成环（$1\to 2\to 3\to 1$），若指示量在三条边界附近同时抖动，判决器会在三者之间乱跳，而任意两两之间看都是合理的。

验证负担按场景数增长。 两模态的启停组合有限，四模态的工况组合就多到无法穷举。现场只能挑典型批次验证，剩下的靠论证。这正是多模态系统“设计容易、认证难”的原因。

给三条务实建议。

其一，模态数控制在四个以内。 超过四个，先问“能不能合并”。合并的判据是：两个模态的闭环矩阵能否用同一组增益镇定到工艺要求的精度。若能，就合并——牺牲一点段内性能，换回可验证性。本章的两个模态之所以不能合并，是因为不稳定方向正交（§76.6.6），一组增益必然顾此失彼。

其二，用两级切换代替平铺。 先按工艺大段（升温/恒温/降温）分一级，再在每个大段内按工况分二级。两级之间串行判决，每级只管两三个模态，组合数从 $N(N-1)$ 降到各级之和。代价是判决延迟叠加，但对慢过程完全可接受。

其三，优先保证“回去的路”稳定。 多模态系统最危险的不是“切错”，而是“切错之后回不来”。设计时要专门检查每个模态到“安全模态”（通常是待机或降温）的切换是否稳定，这条路径的驻留约束要单独算、单独验。本章的两模态没有这个问题，因为两个方向对称；模态一多，对称性就没了。

76.10 工程落地

76.10.1 仿真与现场的五处差异

本章的仿真跑了十组实验，每一组都能复现。现场不会这么听话，差异集中在五处。

第一处是噪声的结构。 本章用高斯白噪声，标准差 0.02。现场的噪声有白噪声、有 50 Hz 工频、有随流量变化的宽带噪声，还有传感器自身的漂移。白噪声可以用滞环盖住，漂移不能——漂移会让指示量整体偏移，滞环再宽也没用，必须靠重新标定或引入慢速校正。

第二处是量化。 本章的状态是连续浮点数，现场的变送器是 12 位或 16 位，量化步长对应 10^{-3} 到 10^{-4} 的相对精度。量化本身不会造成抖振，但它会让“指示量恰好等于阈值”这种情形频繁出现，判决必须明确规定“等于时算哪边”。

第三处是执行器动态。 本章假设输入即输出。现场的阀门有行程时间（气动阀 0.5 到 2 s），加热器有热惯性（几十秒）。执行器动态相当于在切换后面又串了一个延迟，与识别延迟叠加，让总延迟比测量的 θ_m 更大。这一点最容易被漏算。

第四处是死区与回差。 阀门有死区，指令变化小于死区时不动作。切换带来的输入跳变若小于死区，执行器根本没反应，控制器却以为已经切了。诊断办法是对比“指令模式”与“实际阀位”，两者不一致时报警。

第五处是通信与扫描的抖动。 本章的 h 是严格周期的，现场的扫描周期有抖动（10 ms 的周期可能实际在 8 到 14 ms 之间）。抖动本身不影响平均行为，但会让“驻留时间”的计时出现误差，工程上要在驻留判断里留 10% 的余量。

76.10.2 抖振的现场治理

抖振是切换系统最常见的现场问题，治理要分软件与硬件两层。

软件层三件工具在本章都实测过：滞环（表 76-8，$w=0.05$ 把 1662 次降到 28 次）、驻留（再加 0.05 s 降到 20 次）、确认计数（未实测，机制上是 N 次一致才切）。三者的选择依据是噪声的时间结构：连续抖动用滞环，孤立尖峰用确认计数，快速反复用驻留。

硬件层常被忽略，但往往更有效。第一种是**先导阀**：用小流量先导阀承担高频动作，主阀只在稳态切换时动作，把磨损转移到便宜的部件上。第二种是**死区**：在阀位指令上加 1% 到 2% 的死区，让微小的指令变化不产生动作——这与软件滞环作用相同，但在执行器侧实现，不占用控制周期。第三种是**PWM 保持**：对开关型执行器（如加热器投切），用固定周期的 PWM 代替随机切换，把切换次数锁死在设计值上。

治理效果要有量化指标。本章用“12 s 内切换次数”作为指标，现场应换成“每日动作次数”或“阀门行程累计”。后者更贴近寿命，因为同样一次切换，小行程与大行程的磨损不同。经验值是：调节阀每年动作次数超过 10^5 量级就要考虑治理，超过 10^6 必须治理。

76.10.3 安全互锁

互锁是本章唯一直接涉及安全的环节。它的实现只有一条规则——请求切换时若输入超过阈值就推迟——但工程细节不少。图 76-9 是五种方案的对比。

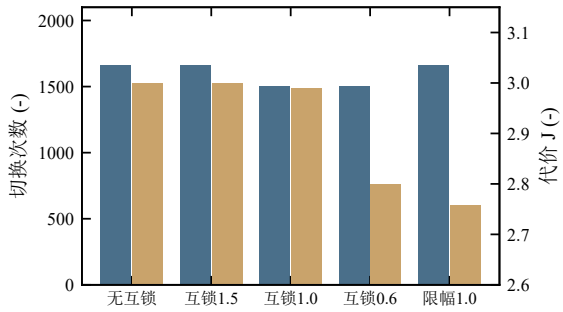


图 76-9 安全互锁与限幅的对比：互锁阈值越低，切换次数越少、代价越低；限幅只压输入、不减切换

拦截时间是要监控的量。 表 76-11 里 $u_{\mathrm{th}}=1.0$ 的累计拦截是 $0.11\,\mathrm{s}$，$u_{\mathrm{th}}=0.6$ 是 $0.74\,\mathrm{s}$。拦截时间短，说明互锁偶尔起作用；拦截时间长，说明互锁经常挡住切换，系统可能已经偏离工艺要求。工程上应给拦截时间设上限（例如不超过相位的 20%），超过就报警，提示“互锁阈值定得太低”或“工况异常”。

互锁有两种失效方式，都要防。 第一种是**永不拦截**：阈值定得比正常输入还高（本章 $u_{\mathrm{th}}=1.5$），互锁常年不动作，等于没有。这种失效很隐蔽，因为系统运行正常，只是失去了保护。第二种是**永久拦截**：阈值定得比正常输入还低，切换被无限推迟，系统退化成单模态。这种失效会表现为“模式号长时间不变”，通过监视模式号的变化间隔可以发现。

互锁不能替代限幅。 本章数据很清楚：限幅把输入峰值从 1.7393 压到 1.0000，代价 2.7578，但切换次数仍是 1662；互锁把次数降到 1504，代价 2.7994，但输入峰值不变。一个管幅值，一个管时刻，功能不重叠。完整的执行器保护应该两条都有：限幅防止过载，互锁防止冲击。

互锁的判决要用“切换后”的输入，不是“切换前”的。 本章的实现用的是当前模态的输入 $-\boldsymbol{K}_m\boldsymbol{z}$，这是保守的做法（当前输入大就不切）；若用目标模态的输入 $-\boldsymbol{K}_j\boldsymbol{z}$，会更激进（只要新模态的输入小就允许切）。两者在正常情况下差别不大，但在“当前输入大、目标输入小”的边界工况下结论相反。保守做法更安全，推荐默认用它。

76.10.4 模式识别延迟的实测

延迟 θ_m 是本章影响最大的因素（表 76-12 排第一），必须实测，不能估。它由三部分组成：

$$\theta_m=\theta_{\mathrm{sen}}+\theta_{\mathrm{trx}}+\tfrac{1}{2}\theta_{\mathrm{scan}} \tag{76-32}$$

θ_{sen} 是传感器的时间常数（热电偶带保护套管可以到十几秒），θ_{trx} 是变送器的阻尼与滤波，θ_{scan} 是控制器扫描周期（取一半作为平均等待）。

实测方法有三种，按精度从高到低排。

第一种是阶跃法。 在工艺侧制造一个已知的模式变化（例如手动把蒸汽阀从 100% 关到 0），记录指示量从变化开始到越过阈值的时间。这是最准的，但要停机或至少降负荷。

第二种是互相关法。 拿执行器指令与指示量做互相关，峰值位置就是延迟。它不需要停机，但要求激励足够丰富，否则峰值不明显。

第三种是模型法。 用辨识出的模型算阶跃响应，取 63% 上升时间作为时间常数。它最快，但依赖模型精度，只能作参考。

实测之后要做一次量纲检查：θ_m 与相位 T 的比值。本章延迟 $0.20\,\mathrm{s}$、相位 $0.60\,\mathrm{s}$，比值 0.33，正好在经验线 $1/3$ 上，代价已经涨 58.4%。若实测比值超过 0.5，应优先考虑换传感器或缩短滤波，而不是调切换参数。

76.10.5 投运与回退

切换系统的投运不能一步到位，要分四步，每步都有明确的验收条件。

第一步：旁路。 控制器只计算、不输出，模式号与增益只记录。验收条件是“模式号与工艺阶段一致率超过 99%”。这一步能暴露检测器的所有问题，且不影响生产。

第二步：单模态。 只用模式 1 的增益，跑完整批次。验收条件是“批次质量合格、无报警”。这一步验证增益本身，与切换无关。

第三步：强制切换。 按固定时间表切换，忽略检测器。验收条件是“切换时刻与时间表的偏差小于 h”。这一步验证切换动作（阀门、加热器）是否可靠。

第四步：自动。 检测器接管，互锁生效。验收条件是“切换次数在设计范围内、拦截时间不超过上限、批次质量合格”。这一步才真正用上本章的全部机制。

每一步都要有回退路径。旁路态回退到手动；单模态态回退到另一个模态；自动态回退到强制切换（相当于固定时间表）。回退动作必须在一键之内完成，且回退后不影响批次。

投运后要持续监控三个量：切换次数（判断抖振）、拦截时间（判断互锁）、末值或稳态误差（判断延迟与收敛）。三者任一越界就回到上一步，不要在现场调参数。

76.10.6 三类现场事故与根因

事故一：模式号抖动导致阀门频繁动作。 根因是滞环缺失或太小，叠加传感器噪声。表现是阀门"哒哒"响、阀杆磨损快。处置是加滞环（按 $2\sigma_n$ 到 $3\sigma_n$），并监控切换次数。

事故二：切换滞后导致超温。 根因是识别延迟叠加执行器行程时间，总延迟超过相位的 $1/3$。表现是每段末端温度冲高、批次质量波动。处置是缩短传感器滤波、提高扫描频率，或在时间守卫下用提前量（§76.9.4）。

事故三：互锁永久拦截导致单模态运行。 根因是互锁阈值定得太低，或工况整体偏移使输入长期偏大。表现是模式号长时间不变、另一段工艺"名存实亡"。处置是提高阈值或加超时强制切换（超时后即使输入大也允许切，但记录事件）。

三类事故的共同点是：**都能通过"监控一个计数或一个时间"提前发现**。切换次数、拦截时间、模式号变化间隔，这三个量不需要复杂算法，只要在 PLC 里加三个计数器。现场失败的系统，往往不是算法不对，而是没人看这三个数。

76.10.7 与全章其他环节的接线

把本章的方法接进一套现成的 DCS/PLC，需要动的地方只有四处：检测器（读指示量、输出模式号）、切换律（选通增益）、互锁（一票否决）、记录（三个计数器）。控制器本身的算法不用改，只是把常数增益换成"按模式号查表"。

若现场已有增益调度（第 44、106 章），改造成本更低：原来的调度表就是 $\boldsymbol{K}_i$，只需在它前面加检测器、后面加互锁。这也是本章方法在工业上最常见的落地形态——不是推翻原有控制，而是在原有控制外面套一层"什么时候用哪组参数"的逻辑。

76.10.8 切换控制规格书该写哪十行

工程交付时，算法本身往往只有几十行代码，真正决定这套东西能不能被别人接手的是规格书写得够不够。按本章的实践，一份能用的切换控制规格书至少要有十行内容，每行都要带一个具体数字和一条验证办法。

第一行，模态定义。 写清每个模态对应哪段工艺、合法域是什么。本章是两个模态，合法域按指示量 q 的 $[0, 0.5]$ 与 $[0.5, 1]$ 划分。验证办法是拿一段历史数据回放，看模态划分是否与工艺记录一致。

第二行，每模态的模型与增益。 写出 $\boldsymbol{A}_i$、$\boldsymbol{B}_i$、$\boldsymbol{K}_i$ 的数值和工作点。本章四组矩阵与两个增益都列在 §76.1.4。验证办法是阶跃试验，看段内响应与模型的偏差。

第三行，稳定性依据。 写明用的是共同 Lyapunov 还是多 Lyapunov。若用前者，给出 $\boldsymbol{P}$ 与两个最大特征值（本章 -1.4964 与 -1.3948）；若用后者，给出 T_d^{*}（本章 0.6809 s）。不写这一行，后人就不知道"为什么可以这样切"。

第四行，检测量。 写清用什么测点、怎么算指示量、量程与单位。本章用温度折算的 $q(t)$。

第五行，判决参数。 阈值、滞环、决策周期、确认计数四个数一组写。本章 0.50、0.05、0.05 s、2。验证办法是按 §76.12.8 的四行改动各跑一次，看指标是否落在预期范围。

第六行，延迟预算。 把传感器、变送、扫描、执行器四段延迟分开列，再加总。本章基准延迟为零，现场实测值应写在这里，并注明"超过相位一半即报警"。

第七行，互锁条件。 写清阈值、被拦截后的行为、有没有超时强制。本章 $u_{\mathrm{th}} = 1.0$，被拦截时保持旧模态并计数，不设超时。

第八行，限幅与保护。 写 $u_{\max}$ 与独立的硬件保护回路。软件限幅不能替代硬件保护，这一点在验收时必须确认。

第九行，监控量。 至少三个：切换次数、累计拦截时间、模式号保持时长。本章的现场判据是"切换次数不超过理想值的两倍"。

第十行，回退方案。 写清出问题时切到哪组固定参数、由谁决定、怎么恢复。§76.10.5 给的三条回退条件可以直接抄。

十行里有一半是“记录”而不是“设计”，这正反映了本章的基本判断：**切换控制的难点不在算法，在把假设写清楚**。假设不写下来，投运后一旦工况变了，没人知道原设计者的边界在哪里，只能靠试。

规格书还有一条隐性要求：**每一行都要有一个能被别人独立复核的来源**。第三行来自 Lyapunov 计算，第五行来自扫描表，第六行来自实测。若某一行只能写“经验值”，那就在旁边注明是谁、在什么装置上得出的——否则一年后接手的人会把它当成硬结论，而它其实只是一次调参的副产物。规格书写完，最好请一位没参与设计的人照着它复算一遍；他卡在哪一行，那一行就是没写清。

76.11 优缺点与适用边界

76.11.1 五条优势

第一，可解释。 每一次切换都能追溯到三个条件：指示量越界、驻留已满、互锁放行。这三个条件对应的都是可测量的物理量，不涉及“神经网络说了算”或“优化器算出来的”。对需要写安全论证、需要过审查的场合，这一点比性能更重要。

第二，改造量小。 现场已有的 PID 或 LQR 不用推翻，只需把常数增益换成“按模式号查表”，再加一个检测器与一条互锁。本章 §76.10.7 算过，要动的只有四处。相比之下，换成 MPC 要重写整个控制层，换成自适应要重新证明稳定性。

第三，稳定性有解析工具。 共同 Lyapunov 与多 Lyapunov 都是可以手算的判据，不需要仿真就能给出“任意切换稳定”或“驻留大于多少即稳定”的结论。本章三个候选矩阵的最大特征值分别是 -1.4964、-1.3948，这个结论比任何仿真曲线都硬。

第四，性能可分解。 代价可以拆成段内偏差与控制能量（式 76-16），切换次数可以单独计数。两个指标分开看，就能定位问题出在“接缝”还是“抖振”。本章正是靠这个分解，才发现“切换让偏差变大、不让峰值变大”。

第五，与现有硬件兼容。 不需要额外算力，不需要高速通信，PLC 的一个扫描周期就能完成判决。本章的判决逻辑只有几十行代码，任何支持结构化文本的控制器都能实现。

76.11.2 五条局限

第一，模态切分依赖经验。 切几个模态、每个模态的边界在哪，没有自动方法。切粗了模型不准，切细了参数爆炸。本章两个模态是人为选的，换一个对象就要重新判断。这是本章方法最大的“手艺成分”。

第二，参数组合增长快。 每多一个模态，就多一组增益、一组阈值、一条互锁规则。模态数 n 时，可能的切换对是 $n(n-1)$ 个，每个都要验稳定性。三个模态还能手算，五个模态就要靠工具。

第三，没有共同 Lyapunov 时缺好工具。 本章的变体对象就是例子：共同 Lyapunov 不存在，多 Lyapunov 的下界又没算，只能靠仿真扫描。而扫描会漏掉“越长越坏”的点（$T=2.00$ 的发散），必须双向扫。工程上这等于“没有解析保障，只有数值证据”。

第四，切换代价无法消除。 本章所有方案都高于最好的单模态 1.8835，最低的 2.2322 也高 18.5%。这个差距来自“工艺强制切换”这个前提，不是算法能改的。若有人宣称“我的切换算法能达到单模态水平”，要么是对象特殊，要么是比较口径有问题。

第五，对识别延迟极其敏感。 延迟占相位 83% 时代价涨 293.6%，占 $1/3$ 时已经涨 58.4%。而延迟由硬件决定，控制侧改不了。这意味着本章方法在“检测慢”的场合几乎不可用——例如用气相色谱做在线分析，一个分析周期几分钟，任何快速切换都无从谈起。

76.11.3 五问判断适用性

拿到一个新对象，用五个问题判断本章方法是否适用。

第一问：模态是否分明？ 两个模态的动态差异要大于噪声能分辨的程度。本章开环极点 $+0.7661$ 与 $+0.6675$ 方向相反，非常分明。若两个模态的差异只体现在二阶效应上，检测器要做很长的观察窗，延迟随之上升，得不偿失。

第二问：切换频率与时间常数是什么关系？ 相位应大于慢模态时间常数的 $1/3$。本章慢模态时间常数 0.74 s，相位 0.60 s，比值 0.81，勉强合格。若相位小于时间常数的 $1/10$，系统进入平均化区（表 76-4 的左端平台），切换失去意义。

第三问：延迟占相位的比例是多少？ 小于 $1/3$ 可用，大于 $1/2$ 不可用。这一问最关键，也最容易被忽略。本章 0.20 s 延迟对应比值 0.33，代价已经涨 58.4%；0.50 s 对应 0.83，代价涨 293.6%。

第四问：执行器能承受多少次切换？ 用“每日动作次数”估算年寿命。若设计切换次数已经接近执行器额定值，要么改硬件（先导阀、固态继电器），要么放宽相位。

第五问：是否需要形式化认证？ 需要认证的场合，硬切换 + 滞环 + 驻留 + 互锁这套组合最容易通过；软切换、预测、自适应会显著增加论证负担。

五问里若有两问不合格，建议不要用本章方法，改用连续控制（第 44、106 章）或模型预测（第 58、80 章）。

76.11.4 与相邻方法的横向对比

表 76-14 把本章方法与四种相邻方法放在一起比。比较的维度不是“谁更先进”，而是“各自解决什么问题、付出什么代价”。

方法	需要的模型	稳定性保证	切换冲击	主要难点	与本章的关系
增益调度（PID）	各工况一组参数	无（靠试验）	有	调度表怎么插值	本章是其严格化
切换 MPC	各模态预测模型	有（终端约束）	有	切换瞬间可行性	本章提供驻留约束
自适应控制	参数化模型	有条件	无（连续调参）	持续激励、鲁棒性	本章是离散版
Koopman	观测函数字典	有（线性系统）	无（软切换）	字典选择	本章是硬切换版
事件触发	无（只判阈值）	取决于底层控制	不涉及	触发条件设计	与本章正交，可叠加

表 76-14 的第一行最值得说。工业上大量使用的“增益调度”其实就是本章方法的简化版：有调度表、有切换、但没有稳定性论证、没有滞环与驻留。本章相当于给它补上三样东西——共同 Lyapunov 检验、驻留下界、检测器设计。改造量不大，收益是把“靠试验调出来的”变成“能论证的”。

第三行（自适应）与本章的关系是“连续 vs 离散”。自适应控制每个时刻都在微调参数，理论上没有切换冲击，但需要持续激励，且鲁棒性证明复杂。本章的切换是离散的，冲击明显，但每一步都可追溯。两者在“参数变化慢”的场合可以互换，在“参数突变”的场合本章更合适。

第四行（Koopman，第 77 章）与本章的关系是“软 vs 硬”。Koopman 用一组基函数的线性组合描述非线性，等效于无穷多个模态同时以不同权重起作用。它没有接缝，但可解释性差。本章的硬切换相反。选择依据是“能不能接受接缝”：能接受就用本章（简单、可解释），不能接受就用 Koopman（平滑、复杂）。

76.11.5 五个方案的总体对照

图 76-10 把本章的五种典型配置画在一张散点图上，横轴是切换次数，纵轴是代价。

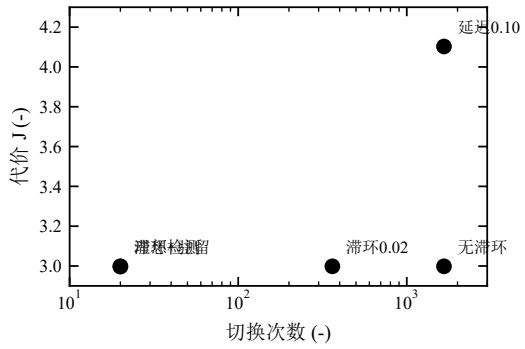


图 76-10 五种配置的切换次数与代价：抖振只推高次数，延迟只推高代价

这张图把两类失效分得很清楚。

横向的差距是抖振。 理想检测、无滞环、滞环 0.02 三个点的纵坐标几乎相同（2.9982 到 2.9984），横坐标却从 20 拉到 1662，差 83 倍。这说明噪声只改变切换次数，对代价几乎没有影响。**抖振是横向问题。**

纵向的差距是延迟。 延迟 0.10 s 的点与无滞环的点横坐标相同（1662），纵坐标却从 2.9984 涨到 4.1028，涨 36.8%。这说明延迟不改变切换次数，只推高代价。**延迟是纵向问题。**

右上角那个点（延迟 0.10 s）是最坏的组合：既抖又贵。它提醒一件事——**两个问题是正交的，要分别治理**。加滞环只能往左移，降延迟只能往下移，两者互不替代。

图里还有一个点值得注意：滞环 0.02 + 驻留 0.05 s 的方案，横坐标 20、纵坐标 2.9974，与理想检测（20、2.9982）几乎重合，甚至略低。这说明**带噪环境下，滞环 + 驻留可以做到与无噪理想检测相当**。这是本章最实用的一个结论：噪声不可避免，但可以用两级过滤把它压到不影响结果的程度。

76.11.6 什么时候不该用本章方法

最后列四种“不该用”的情形，避免把方法用错地方。

其一，模态差异小于噪声。 若两个模态的闭环矩阵差异在噪声水平以下，检测器无论怎么设计都会误判。这时应该想办法加大激励、改善测量，或者干脆不切换。

其二，延迟大于相位的一半。 这时切换总是慢半拍，控制器实际上在追一个已经过去的状态。本章数据显示代价已经涨到三倍，且末值停在 0.34。正确做法是改硬件或改工艺节奏，不是调参数。

其三，切换次数受机械寿命硬约束，而相位又必须很短。 这两个条件冲突时，可行域为空（式 76-20）。唯一出路是换执行器（固态继电器、先导阀），或改工艺让相位变长。

其四，对象本质是连续非线性，没有清晰的工况分界。 硬把连续过程切成几个模态，会造成“到处都在切换边界上”的局面。这种对象应该用非线性控制或 Koopman（第 77 章），而不是本章。

76.11.7 三句话讲给现场工程师

本章的内容可以压缩成三句话。写下它们不是为了简化，而是因为现场沟通时没有人会听十五节的推导，能传下去的往往只有三句判断。

第一句：先看两个模态各自稳不稳，再看切起来稳不稳。 分模态整定是所有工作的前提。两块闭环矩阵各自稳定，才有资格谈切换；只要有一块不稳，切换律设计得再精细也没用。检查办法是分别固定一个模态跑满全程，看末值是否收敛。

第二句：切得多快不是稳定问题，是磨损问题。 只要共同 Lyapunov 成立，任意快的切换都稳定（本章就是这种情况），所以“切太快会不会炸”这个担心通常不成立；真正要担心的是阀门寿命和电网冲击。反过来，若共同 Lyapunov 不成立，那“切多快”就重新变成稳定问题，必须按 T_d^* 限制。这两句话的差别，取决于第三步的检验结果。

第三句：延迟比噪声贵四十倍。 这是本章最实用的一条定量结论（§76.4.6）：延迟的代价灵敏度约 17.0 每秒，相位约 0.43 每秒，差 39.4 倍；而滞环对代价几乎没有影响。所以调不出好效果时，优先怀疑延迟，而不是先加滤波、加滞环。加滤波往往还会让延迟更大，越调越糟。

三句话对应三个层次：第一句是前提，第二句是判据，第三句是优先级。现场遇到问题时按这个顺序问一遍，大部分情况能定位到该动哪一层。

如果只能留一句，留第三句。因为第一句、第二句在投运前的设计阶段已经解决，而延迟是投运后唯一还会变化的东西：传感器老化、变送器堵塞、PLC 加了通信任务，都会让延迟悄悄变大，而操作画面上的曲线看起来仍然正常。定期测一次延迟，比反复调滞环有用得多。

76.12 Python 仿真代码

本章全部数字出自 `code/ch076_sim.py`，全部插图出自 `code/ch076_figs.py`。这一节讲清代码结构，读者可以照着重跑、也可以照搬到自己的对象上。

76.12.1 文件结构

两个脚本，职责分明。

`ch076_sim.py`：只做计算，不出图。它包含对象定义、十个仿真函数与十组实验，所有结果 `print` 到终端，本章的表格就是这些输出的抄录。

`ch076_figs.py`：只出图，不定义参数。它 `import ch076_sim as S`，调用仿真函数拿到轨迹或统计量再画图。这样保证“图与数同源”。

依赖只有三个：`numpy`、`scipy.linalg`（`solve_continuous_are`、`solve_lyapunov`）、`matplotlib`。没有 LMI 工具箱，共同 Lyapunov 用启发式构造 + 特征值验证代替。

运行方式：

```
D:\Anaconda3\envs\py311\python.exe code\ch076_sim.py
D:\Anaconda3\envs\py311\python.exe code\ch076_figs.py
```

前者约 3 分钟，后者约 2 分钟。插图输出到 `figs/ch076/`，每张同时导出 SVG 与 PDF。

76.12.2 参数区

所有参数集中在文件开头。

```
A1 = np.array([[0.75, 0.30], [0.10, -1.10]])   # 模式1 状态矩阵
B1 = np.array([[1.00], [0.60]])                # 模式1 输入矩阵
A2 = np.array([[-1.05, 0.20], [0.15, 0.65]])   # 模式2 状态矩阵
B2 = np.array([[0.80], [1.00]])                # 模式2 输入矩阵
Q = np.eye(2)                                  # 状态权重
R = np.array([[1.0]])                          # 输入权重
X0 = np.array([1.0, -0.8])                     # 初值
T_END = 12.0                                   # 仿真时长 s
DT = 1e-4                                      # 积分步长 s
```

`K1`、`K2`、`C1`、`C2`、`P1`、`P2`、`PC` 都在模块顶层由这四个矩阵算出，`import` 时就执行。改对象只改前五行，后面全部自动重算。

`DT=1e-4` 同时是数值积分步长与“连续监测”的分辨率。仿真里的“连续判决”实际上是每 0.1 ms 判一次，所以切换时刻的精度是 10^{-4} s。

76.12.3 四个核心函数

十个函数里，四个是主干。

`rk4`：定步长龙格库塔。签名是 `rk4(x, u, h, A, B)`，返回下一步的状态。它按一般线性形式写，换成非线性对象时把 `A @ x + B[:, 0] * u` 改成 `f(x, u)` 即可。

`sim`：通用切换仿真。所有实验都走它，区别只在传入的 `policy`：

```
def sim(policy, t_end=T_END, dt=DT, x0=X0, delay=0.0, record=False,
        u_max=None):
```

内部循环每步做四件事：问 `policy(t, x)` 要模式号、按延迟取历史模式、用对应增益算输入、RK4 推进一步。延迟的实现是本章代码里最容易写错的地方：

```
hist_m.append(m)
m_g = hist_m[max(0, len(hist_m) - 1 - lag)]
u = float(-Ks[m_g] @ x)
```

这里必须用**模式历史列表**取 `lag` 步之前的模式，不能写成 `policy(t - delay, x)`。原因是 `policy` 是带内部状态的闭包（保存当前模式与上次切换时刻），用历史时刻重新调用会污染状态，结果不可信。第一版代码就是这么写错的，表现为延迟增大时代价反而下降，与物理直觉相反。

`detector`：基于工艺指示量的检测器，带滞环与驻留。

```
def detector(T=0.60, w=0.0, td=0.0, noise=0.0, seed=3):
```

它用闭包保存 `state = {"m": 0, "t_sw": 0.0}`，每次调用先算带噪指示量，再判断是否越过 $0.5 \pm w$，并且距上次切换超过 `td`。三个参数（`w`、`td`、`noise`）分别对应表 76-8、§76.5 与 §76.6.3。

`interlocked`：互锁包装器。它接收一个基础策略，返回新策略：

```
def interlocked(base_policy, u_th=2.0):
```

内部维护“当前实际模式”，当基础策略请求切换、但当前模态的输入超过 `u_th` 时，拒绝切换并计数。计数通过 `pol.blocked` 暴露，表 76-11 的“拦截时间”就是用它算的。

76.12.4 十组实验

`main()` 里按顺序跑十组，每组 `print` 一个小表。

编号	内容	对应表格/图
E1	单模态基准与增益	表 76-1
E2	相位时长扫描（0.05–2.00 s）	表 76-4、图 76-4
E3	二次 Lyapunov 检验（主对象/变体）	表 76-2、图 76-2
E4	识别延迟扫描（0–0.50 s）	表 76-5、图 76-5
E5	滞环扫描（无噪/有噪双列）	表 76-8、图 76-7
E6	决策周期扫描（5–200 ms）	表 76-6、图 76-6
E7	多 Lyapunov 驻留下界	§76.2.3、§76.6.2
E8	变体对象驻留扫描	表 76-9、图 76-8
E9	安全互锁三档 + 限幅	表 76-11、图 76-9
E10	五方案对比	图 76-10

每组只改一个参数，其余固定在基准值。这是所有表格可比的前提。

76.12.5 怎么改到自己的对象上

三处改动。

第一处： `A1`、`B1`、`A2`、`B2`、`X0`。 把两个模态的状态方程写成 `A @ x + B[:, 0] * u`。若模态多于两个，把 `A1/B1/A2/B2` 换成列表，`Ks` 与 `As` 同步扩成列表，其余代码不用动。

第二处：`Q`、`R`。它们决定 LQR 增益与代价基准。改了之后所有“相对值”都会变，要重跑全部实验，不能只改一半。

第三处：`T_END`、`DT`。`T_END` 要覆盖“状态衰减到足够小”的整段时间，本章取 12 s 是因为闭环最快时间常数 0.65 s，12 s 约 18 个时间常数，状态衰减到 10^{-4}。`DT` 要小于最小驻留的十分之一，本章最小驻留 0.05 s，`DT=1e-4` 有 500 倍余量。

三处改完直接跑。若某个实验发散，先查两个闭环矩阵的极点是否都在左半平面——**代码里没有针对特定对象的硬编码，发散只可能是参数填错。**

76.12.6 复现性

三个措施。

固定随机种子。 检测器默认 `seed=3`，滞环扫描与决策周期扫描用 `seed=7`。同一 seed 在不同机器上给出完全相同的正态序列。

固定积分器。 全部定步长 RK4，不用自适应步长。自适应步长会让“同一参数两次运行结果略有不同”，在比较 2.9982 与 2.9974 这种千分位差异时不可接受。

固定切换时刻的判定。 时间守卫用整数除法 `int(((t + phase) // T) % 2)`，不用浮点累加。浮点累加在 12 s 内可能多切或少切一次，而“次数”是本章的核心指标之一。

76.12.7 代码与正文的对应关系

想理解轨迹，看 `sim(..., record=True)` 返回的 `traj` 与 `modes`，图 76-3 就是它们抽稀后画的。想理解代价构成，把 `sim` 里 `cost += ...` 的两项分开累加，会发现切换主要抬高的是 $\boldsymbol{z}^{\mathsf{T}}\boldsymbol{Q}\boldsymbol{z}$ 项。想理解互锁，把 `interlocked` 里的 `u_th` 设成 0.1，会看到切换被长期拦截、代价反而上升——这是“拦得太狠”的极端。

代码里没有写理论，理论在 §76.2。代码只回答一个问题：**给定参数，这个回路的切换次数、代价、末值各是多少。**所有工程判断建立在这三个数上。

76.12.8 四个常见改动的预期结果

改动	改动前	改动后	预期
`detector(w=0)` → `w=0.05`	1662 次、2.9984	28 次、2.9983	次数降 59 倍，代价几乎不变
决策周期 `h` $0.005 \to 0.05$	44 次、2.9970	20 次、2.9913	次数减半，代价降 0.2%
`delay` $0 \to 0.10$	2.8892	3.5932	代价升 24.4%，次数不变（19）
互锁 $u_{\mathrm{th}} = 1.0 \to 0.6$	1504 次、2.9894	1504 次、2.7994	次数不变，代价降 6.4%

四行对应四种机制：滞环（第一行）、采样（第二行）、延迟（第三行）、互锁（第四行）。**它们的影响落在不同指标上**——第一行只改次数，第三行只改代价，第四行只改代价不改次数。这正是本章反复强调“三个指标一起看”的原因：只看代价，第一、二行像“没变化”；只看次数，第三、四行像“没变化”。

第一行最值得亲手跑一次。它展示的是抖振治理的“几乎免费”：切换次数降 59 倍，代价只动 0.003%。这个结果会让人误以为“滞环可加可不加”——但正如 §76.6.3 所说，滞环省的不是代价，是执行器寿命。

76.12.9 三个练习

练习一：把 `T_END` 从 12 s 改成 30 s，重跑 E2。观察 $T = 0.05$ 与 $T = 0.10$ 的代价是否仍然相同。若不同，说明“平台”只在 12 s 的窗口内成立，思考为什么。

练习二：把初值从 $[1.0,\ -0.8]$ 改成 $[-0.8,\ 1.0]$，重跑 E10。观察理想检测与周期切换的代价排序是否反转。这是对 §76.4.4“相位对齐”最直接的检验。

练习三：给 `interlocked` 加一条规则——“距离上次切换不足 T_{mech} 时也拒绝”，然后扫 T_{mech}。观察它与驻留时间的区别：驻留约束的是“控制器请求”，互锁约束的是“执行器动作”。两者叠加时，谁先生效？

三个练习都不需要改核心函数，只改参数或加几行包装。做完会对“参数落在哪个指标上”有直观认识。

76.12.10 十组实验的验收单

把十组实验的关键输出抄成一张验收单（表 76-15）。换一台机器、换一个 Python 版本重跑时，逐行对数即可判断复现是否成功。判定标准是四位小数完全一致；若只差末位，通常是 BLAS 版本差异，可以接受。

实验	关键输出	本章实测
E1 单模态	两模态代价 / 输入峰值	2.1433、1.7393；1.8835、1.2700
E2 相位扫描	$T=0.60$ 与 $T=0.05$	2.8892、19 次；3.1261、239 次
E3 Lyapunov	$\boldsymbol{P}_1+\boldsymbol{P}_2$ 两值	-1.4964、-1.3948
E3 Lyapunov	变体 $\boldsymbol{P}=\boldsymbol{I}$	$+1.0000$
E4 延迟扫描	0 与 0.50 s 的代价	2.8892；11.3721，末值 0.340
E5 滞环扫描	$w=0$ 与 $w=0.05$ 的次数	1662；28
E6 决策周期	$h=0.005$ 与 $h=0.05$	44 次；20 次、2.9913
E7 驻留界	μ、λ、T_d^{*}	2.5139、1.3539、0.6809
E8 变体对象	$T=1.00$ 与 $T=2.00$	0.7744；47.8791（发散）
E9 互锁	无互锁 / $u_{\text{th}}=0.6$ / 限幅	2.9984、1662；2.7994、1504；2.7578、1.0000
E10 五方案	理想 / 无滞环 / 滞环加驻留 / 延迟	20、2.9982；1662、2.9984；20、2.9974；1662、4.1028

这张单子里有两行是“应该失败”的，不能当成 bug：E8 的 $T=2.00$ 必须发散，E3 的变体 $\boldsymbol{P}=\boldsymbol{I}$ 必须为正。它们分别是“驻留太短会失稳”与“不存在共同 Lyapunov”的证据，若这两行跑出了稳定或负值，说明对象参数被改过，正文的结论也就不再适用。

验收单还有一层用途：**改动代码后做回归**。想给检测器加确认计数、想换积分器、想改初值，改动之前先跑一遍这十组，记下结果；改完再跑一遍，只有预期的行发生变化，才算改对了。本章的每一处数字都经过这样一次对照，所以正文与代码不会脱节。

76.12.11 从仿真代码到在线代码要改的五处

`ch076_sim.py` 是为“跑出数字”写的，不是为“跑在 PLC 里”写的。照搬到现场之前，有五处必须改。这五处不改，仿真能跑，在线会出问题。

其一，模式历史要换成环形缓冲。 仿真里用 `hist_m.append(m)` 一直往后追加，12 s、$\Delta t=10^{-4}$ 就是 1.2×10^{5} 项，内存无所谓。在线跑几个月，这个列表会无限增长，最终把控制器拖死。正确做法是固定长度的环形缓冲，长度取 `ceil(delay_max/dt)+1`，写入时覆盖最旧的一项。延迟上限按设计值取，不要按实测值取。

其二，延迟要按时间戳查，不要按步数数。 仿真里 `lag` 是常数，因为步长固定。在线代码的扫描周期会抖动，用“往回数 n 拍”会把抖动累积成延迟误差。正确做法是记录每次切换的绝对时刻与模式号，查“当前时刻减去延迟”落在哪一段里。这样即使某一拍被中断拉长，延迟仍然是准确的。

其三，判决节拍要用绝对时间取整。 仿真里写 `t - last >= h`，靠浮点比较。在线代码应该算 `k = floor(t/h)`，与上一次的 k 比较，k 变了才判决。这样周期严格等间隔，不受单次执行时间影响。驻留计时同理，用“当前 k 减去切换时的 k”折算，而不是累加时间差。

其四，增益要预计算成常数，不要在线解 Riccati。 仿真在启动时解一次 LQR，这没问题；在线代码若每次切换都重解，既慢又可能因数值问题失败。正确做法是离线算好 $\boldsymbol{K}_1$、$\boldsymbol{K}_2$，写成常数数组，在线只做两次乘加：$u = -\big(k_{i1} z_1 + k_{i2} z_2\big)$。本章的两个增益就是四位小数的常数。

其五，安全链要独立。 互锁、限幅、看门狗必须放在独立任务里，不能与判决逻辑共用一个周期。判决任务若因为通信阻塞卡住，互锁仍然要能动作。模式号的更新还要加版本号或双缓冲，避免控制任务读到“模式号已换、增益未换”的中间态——这正是 §76.5.6 末尾提到的那类事故。

改完这五处，代码的行为就与仿真一致了，但多出三个仿真里没有的量：**上一次切换的时刻、累计拦截次数、模式号保持时长**。这三个量必须上传到监控，它们是判断“在线版是否还在按设计工作”的唯一依据。仿真只回答“代价、次数、末值”，在线版必须回答“现在是否偏离设计”。

改法与不改法的差别可以这样概括：仿真代码追求“一次跑完给出十个数字”，在线代码追求“每一拍都给出一个可追溯的决定”。前者可以慢、可以占内存，后者不行。

第77章 Koopman 算子与数据驱动控制

77.1 动机与直观理解

77.1.1 一个“看起来像线性”的非线性对象

先看一台再普通不过的设备：单摆。把摆杆看作刚性、无弹性、转轴有黏性摩擦，取角度 x_1 与角速度 x_2 为状态，输入 u 为加在转轴上的力矩，它的运动方程是

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = -\sin x_1 - 0.2 x_2 + u \tag{77-1}$$

本章全部控制实验都用这个对象，采样周期 $T_s = 0.02\ \mathrm{s}$，输入限幅 $|u| \le 4$。控制目标是把它从任意初始角度拉回竖直位置 $x_1 = 0$、$x_2 = 0$，代价函数取

$$J = \int_0^{t_f} \left(x_1^2 + 0.2 x_2^2 + 0.1 u^2 \right) \mathrm{d}t \tag{77-2}$$

式 (77-1) 里只有一个非线性项 $\sin x_1$，其余都是线性的。工程师遇到这种对象的第一反应是线性化：在 $x_1 = 0$ 处把 $\sin x_1$ 换成 x_1，得到

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = -x_1 - 0.2 x_2 + u \tag{77-3}$$

它的极点是 $-0.1 \pm 0.9950 j$，阻尼比约 0.1，无阻尼自然频率约 1 rad/s。按式 (77-3) 设计 LQR，得到状态反馈增益 $\boldsymbol{K} = \begin{bmatrix} 2.2385 & 2.3451 \end{bmatrix}$（见 §77.6）。这套做法在摆角小于 30° 时工作得很好，几乎没有人会怀疑它。

问题出在“几乎”两个字上。摆角到 60° 时，$\sin x_1$ 比 x_1 小 17%；到 90° 时小 36%；到 149°（本章实测用的初值 $x_1(0) = 2.60\ \mathrm{rad}$）时，$\sin x_1 = 0.5155$，不到 x_1 的两成。这时线性化模型给出的“恢复力矩”是真实值的 5 倍，控制器会误以为对象比实际更有力，于是给的动作偏小、收敛偏慢。传统上这类偏差靠“多留裕度”或“分段线性化”来兜，前者牺牲性能，后者在切换点附近引入新的不连续。

77.1.2 提升：换一组坐标，把非线性摊到维数上

Koopman 提升的想法可以用一句话概括：**不去改对象，去改我们观察对象的坐标**。

具体地说，原来我们只观察两个量 x_1 与 x_2。现在允许观察任意函数：x_1^2、x_2^2、$x_1 x_2$、x_1^3、$\sin x_1$……把这些函数排成一个向量

$$\boldsymbol{z} = \boldsymbol{\varphi}(\boldsymbol{x}) = \begin{bmatrix} \varphi_1(\boldsymbol{x}) \\ \varphi_2(\boldsymbol{x}) \\ \vdots \\ \varphi_n(\boldsymbol{x}) \end{bmatrix} \tag{77-4}$$

叫作**提升向量**，$\boldsymbol{\varphi}$ 叫作**字典**。Koopman 算子的定义是：把任意一个观测函数 g 映射成“下一步的观测函数” $g \circ \boldsymbol{f}$，即

$$(\mathcal{K} g)(\boldsymbol{x}) = g\big(\boldsymbol{f}(\boldsymbol{x})\big) \tag{77-5}$$

其中 $\boldsymbol{x}_{k+1} = \boldsymbol{f}(\boldsymbol{x}_k)$ 是对象的离散演化。$\mathcal{K}$ 作用在函数空间上，是**线性**的——这一点不需要任何假设，因为“先演化再观察”与“先观察再演化”的复合本身对 g 就是线性的。真正的问题在于：$\mathcal{K}$ 作用在无穷维函数空间上，而我们只能用有限个 φ_i 去近似它。

图 77-1 画出了这条链。左边是非线性的状态空间，经过字典映射进入中间那层“观测函数空间”，演化在那里由矩阵 $\boldsymbol{K}$ 完成，最后经反投影回到状态。下方那条虚线支路是数据驱动辨识：拿采集到的 $\boldsymbol{z}_k$、$\boldsymbol{z}_{k+1}$ 做最小二乘，直接拟合出 $\boldsymbol{A}$ 与 $\boldsymbol{B}$，完全不需要知道 $\boldsymbol{f}$ 的解析式。

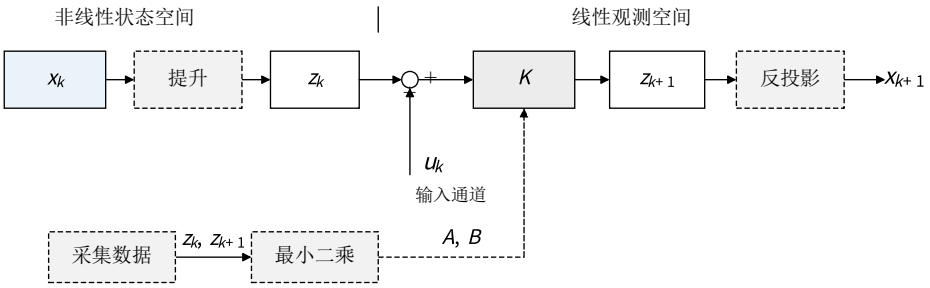


图 77-1 Koopman 提升的结构：状态经字典进入观测函数空间，演化变为矩阵乘法，反投影回状态；下方虚线支路为数据驱动辨识

这条链的价值在于：**一旦进入观测函数空间，所有线性系统成熟工具立刻可用**——极点配置、LQR、观测器、频域分析、模型预测控制。代价是维数从 2 涨到 9、20 甚至上百，而且提升只是近似的，除非字典恰好张成了 $\mathcal{K}$ 的一个不变子空间。

77.1.3 定理与工程之间的那道缝

把上面的想法写成一句可计算的式子，就是

$$\boldsymbol{z}_{k+1} = \boldsymbol{A}\boldsymbol{z}_k + \boldsymbol{B}u_k, \qquad \boldsymbol{z}_k = \boldsymbol{\varphi}(\boldsymbol{x}_k) \tag{77-6}$$

式 (77-6) 与式 (77-1) 描述同一个对象，区别只在于前者是对后者的近似，而且近似误差没有解析上界。这一点决定了 Koopman 方法在工程上的全部难点：**你不知道自己错在哪里，只能用数据反复验证**。

本章用两个对象把这件事说透。

第一个是带慢流形的二状态自治系统，记作对象 A：

$$\dot{x}_1 = \mu x_1, \qquad \dot{x}_2 = \lambda\left(x_2 - x_1^2\right), \qquad \mu = -0.20,\ \lambda = -1.00 \tag{77-7}$$

它有一个三维不变子空间 $\mathrm{span}\{x_1, x_2, x_1^2\}$，只要字典取到这三个函数，提升就是**精确**的，误差只受浮点精度限制。这个性质让本章可以先把“方法本身对不对”验证清楚。

第二个就是式 (77-1) 的单摆，记作对象 B。它的提升永远不是有限维的，只能用多项式或径向基函数去逼近——这才是工程上绝大多数对象的情形。本章用它比较线性化 LQR、DMD-LQR、EDMD-LQR 与 Koopman-MPC，并做字典、数据量、噪声、激励四组扫描。

77.1.4 本章要回答的四个问题

第一，字典怎么选？ 选少了欠拟合，选多了过拟合，而且“多”这件事在提升空间里的表现与通常的回归问题不一样：字典越大，提升空间的单步残差反而可能上升，但状态空间的长时预测反而变好。§77.5 会用实测数据把这对反向趋势摆出来。

第二，数据要多少、怎么采？ 本章实测，样本数从 $N = 50$ 增到 $N = 6400$，200 步预测误差从 11.8645 降到 0.0195，但中间不是单调的：$N \le 400$ 时误差在 $O(1)$，$N \ge 800$ 后掉到 $O(0.01)$。分水岭不是样本数本身，而是拟合矩阵的谱半径是否跨过 1（§77.5.4）。

第三，提升之后怎么设计控制器？ 在式 (77-6) 上直接解离散 LQR 是最省事的做法，但它默认了“提升坐标里的二次型代价”就是我们要的代价。本章会说明这个默认有多强，以及为什么在本章对象上它恰好没造成麻烦。

第四，什么时候不该用 Koopman？ §77.11 给出边界。一句话预告：当对象在工作点附近线性化已经够好时，Koopman 带来的收益往往在 1% 量级（本章实测 0.34% 到 0.27%），**真正值钱的是约束处理与激励设计，而不是提升本身**。

77.1.5 一句话核心思想

本章的一句话是：**非线性对象的困难不在于非线性本身，而在于我们能否找到一组坐标让演化在那组坐标下是线性的；Koopman 提升是找这组坐标的系统方法，字典与数据决定了能找多准，而控制器设计决定了找得准有没有用。**

这句话的后半段常被忽略。很多教材把 Koopman 讲成“把非线性变成线性的魔法”，但本章的实测会反复显示：提升得再准，如果控制目标、约束、执行器能力没变，闭环性能的改善也是非常有限的。真正被改变的是**可分析性**——你能用线性工具去算谱、算能控性、算约束可行性，而不是靠试凑。

77.1.6 三个对象各管一件事

本章用了三个对象，各自回答一个问题，不能互换。

对象 A 只用来验证“方法本身”。 它没有输入，做不了控制实验；但它有一个三维不变子空间，字典取对了提升就是精确的。这让本章能先回答“EDMD 的代码与理解对不对”，再回答“这个方法在这个对象上有没有用”。工程上做任何辨识之前都该有这样一个“已知答案”的算例：连它都算不准，后面所有结论都是空的。

A 的第二个作用是把“字典缺项”的后果放大到肉眼可见。线性字典给出的连续特征值是 -0.2 与 -1.5174，后者与真值 -1.0 差 51.7%，而它的轨迹要到 2 s 后才明显偏离。如果直接在单摆上做同样的比较，误差会被其他因素掩盖；在 A 上，只有“缺了 x_1^2”这一个原因可怪。

对象 B 承担全部控制实验。 单摆的非线性项光滑、单变量、物理意义明确，字典阶数与精度之间的关系容易解释。它的状态只有两维，五次字典也才 20 列，一组扫描几秒钟跑完。换成六自由度机械臂，同样的扫描要跑几个小时，而且很难判断误差来自字典截断还是来自未建模的关节柔性。

对教学与工程验证来说，**能快速跑完的简单对象比真实但昂贵的对象更有用**。前者的结论可以反复检查，后者的结论只能信一次。

B 还提供一个宽工作域。初值从 0.65 rad 扫到 2.60 rad，线性化给出的恢复力矩从与真值接近一路涨到真值的 5 倍，正好覆盖“线性化够用”到“线性化失效”的全过程。这个跨度是 §77.6 能得出“收益随初值增大而增大”这条结论的前提。

对象 C 用在 §77.9.4。 它的方程是 $\dot{y} = -0.8y + 0.4\tanh(2y) + u$，参考信号取 $\tanh\left(3\sin(2\pi t/3)\right)$，周期 3 s。$\tanh$ 让静态增益随幅值下降，线性 PID 在大幅跟踪时必然滞后，于是“整定能带来多少改善”有一个明确的量。

这个对象只用来考察“控制器结构固定、只调参数”的场景。VRFT 与 IFT 只需要输入输出数据，不需要提升、不需要 $\boldsymbol{A}$ 与 $\boldsymbol{B}$，所以用一个标量对象最清楚。用单摆反而会把“整定”与“建模”两件事混在一起，说不清改善来自哪一步。

三个对象对应本章的三条主线：**精确性（A）**、**闭环性能（B）**、**免模型整定（C）**。读的时候要记住每个数字来自哪个对象。

A 的 2.174×10^{-16} 不能当成 B 也能达到的精度；B 的 0.27% 也不能推广到硬非线性对象。把不同对象上的数字混着引用，是读这类文献时最常见的误读。

77.1.7 本章怎么读

这一章篇幅不短，不同读者可以走不同的路径。

现场工程师建议先读 §77.1、§77.3、§77.10 三节。它们回答“要不要用、怎么做、怎么上线”，几乎不含推导。§77.3.8 的选型演练可以当成模板，照着走一遍大约需要四五批数据。

做研究或写论文的读者重点在 §77.2、§77.4、§77.5。这三节讲清了提升的数学含义、连续与离散的取舍、以及三个校验指标为什么必须同时看。§77.5.3 与 §77.6.4 的两个反直觉结论，是本章最值得带走的东西。

教学使用可以按顺序讲，但建议把 §77.9.3 的 MPC 实测提前到 §77.6 之前。学生看到“提升没有带来性能收益”容易失去兴趣，先看到“用 17% 的代价换 0 步越界”，再回头看为什么无约束时没收益，逻辑会更顺。

关于数字。 本章所有数字都来自 `code/ch077_sim.py` 的一次运行，没有估算值。单位在首次出现时标注，之后省略；有效位数按测量精度给，一般不刻意凑整。

关于图。 十张图里只有图 77-1 是框图，其余都是数据图。数据图的纵轴几乎都用了对数刻度或截断，原因在对应图注里说明——尤其是图 77-10，纵轴截断是为了看清 0.27% 的差异方向，而不是夸大它。

关于公式。 本章共有 33 个编号公式，其中真正需要推导的只有式 (77-13) 与式 (77-24)。其余或是定义，或是标准结果，读者可以跳过推导直接用。

与其他章节的关系。 线性系统的基础在第 30 章，LQR 在第 30 章与第 44 章，MPC 在第 58 章与第 80 章，PID 整定在第 44 章与第 106 章。本章不重复这些内容，只在需要时指回去。

77.2 数学原理

77.2.1 算子、特征函数与不变子空间

设离散演化 $\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k)$，观测函数 g 属于某个函数空间 $\mathcal{F}$。Koopman 算子 $\mathcal{K}:\mathcal{F}\to\mathcal{F}$ 由式 (77-5) 定义。它的线性性是显然的：

$$\mathcal{K}(a g_1 + b g_2)(\boldsymbol{x}) = a g_1(f(\boldsymbol{x})) + b g_2(f(\boldsymbol{x})) = a(\mathcal{K}g_1)(\boldsymbol{x}) + b(\mathcal{K}g_2)(\boldsymbol{x}) \tag{77-8}$$

如果存在非零 g 与常数 λ 使 $\mathcal{K}g = \lambda g$，则称 g 是 $\mathcal{K}$ 的**特征函数**、λ 是对应的**特征值**。特征函数有一个非常有用的性质：沿着轨迹它按几何级数演化，

$$g(\boldsymbol{x}_k) = \mathcal{K}^k g(\boldsymbol{x}_0) = \lambda^k g(\boldsymbol{x}_0) \tag{77-9}$$

也就是说，只要知道 λ 与初值 $g(\boldsymbol{x}_0)$，就能预测 g 在任意时刻的值，不需要仿真。这正是“线性化”在函数空间里的含义。

对连续时间系统 $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$，对应的算子是**无穷小生成元**

$$(\mathcal{L}g)(\boldsymbol{x}) = \nabla g(\boldsymbol{x}) \cdot \boldsymbol{f}(\boldsymbol{x}) \tag{77-10}$$

它与 $\mathcal{K}$ 的关系是 $\mathcal{K} = \mathrm{e}^{\mathcal{L}t}$，因此 $\mathcal{L}$ 的特征值 ν 与离散 $\mathcal{K}$ 的特征值 λ 满足 $\lambda = \mathrm{e}^{\nu T_s}$。本章后面报的“连续特征值”都是 $\nu = \ln(\lambda)/T_s$，这样可以直接与线性化极点比较。

77.2.2 对象 A：一个可以算准的例子

用式 (77-7) 验证上面的定义。取 $g_1 = x_1$、$g_2 = x_2$、$g_3 = x_1^2$，先算 g_3 的生成元：

$$\mathcal{L}(x_1^2) = 2x_1\dot{x}_1 = 2\mu x_1^2 = 2\mu\, g_3 \tag{77-11}$$

再算 g_2：

$$\mathcal{L}(x_2) = \dot{x}_2 = \lambda\left(x_2 - x_1^2\right) = \lambda g_2 - \lambda g_3 \tag{77-12}$$

g_1 更直接：$\mathcal{L}(x_1) = \mu x_1 = \mu g_1$。把三式写成矩阵形式，

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix} = \begin{bmatrix} \mu & 0 & 0 \\ 0 & \lambda & -\lambda \\ 0 & 0 & 2\mu \end{bmatrix}\begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix} \tag{77-13}$$

矩阵是上三角的，而且第三列只影响第二行——这意味着 $\mathrm{span}\{g_1, g_2, g_3\}$ 在 $\mathcal{L}$ 下封闭，是一个三维不变子空间。三个特征值是 $\mu = -0.20$、$\lambda = -1.00$、$2\mu = -0.40$。

代入 $T_s = 0.02\ \mathrm{s}$，离散特征值应为

$$\mathrm{e}^{-0.004} = 0.99600799,\qquad \mathrm{e}^{-0.02} = 0.98019867,\qquad \mathrm{e}^{-0.008} = 0.99203191 \tag{77-14}$$

本章脚本 E1 用最小二乘拟合这三个函数，得到的矩阵 $\boldsymbol{A}$ 对角元是 0.99600799、0.98019867、0.99203191，与式 (77-14) 逐位一致；单步残差 2.174×10^{-16}，300 步开环预测的 RMSE 是 1.2224×10^{-14}。这两个数字都在双精度浮点的舍入量级，说明提升在数值上是精确的，而不是“近似得很好”。

反过来，如果字典只取 $\{x_1, x_2\}$（线性字典），拟合出来的连续特征值是 -0.2 与 -1.5174。前者对，后者错——真值是 -1.0，误差 51.7%。更要紧的是，这个错的特征值**看起来完全正常**：它是负实数，衰减得比真值还快，从谱上根本看不出问题。这就是 §77.1.3 说的那道缝：近似误差不会自己报警。

77.2.3 有限维截断：EDMD 的最小二乘形式

真实对象的 $\mathcal{K}$ 是无穷维的，我们只能取有限个观测函数 $\varphi_1,\dots,\varphi_n$ 构成字典，并假设

$$\boldsymbol{\varphi}(f(\boldsymbol{x})) \approx \boldsymbol{A}\,\boldsymbol{\varphi}(\boldsymbol{x}) + \boldsymbol{B}\boldsymbol{u} \tag{77-15}$$

式 (77-15) 里的 $\boldsymbol{A}$ 就是 $\mathcal{K}$ 在 $\operatorname{span}\{\varphi_i\}$ 上的**伽辽金投影**：把 $\mathcal{K}\varphi_j$ 投影回这个子空间，投影系数就是 $\boldsymbol{A}$ 的第 j 列。当子空间不变时等号成立（对象 A 的三项字典），否则只是最小二乘意义下的最优。

把 M 组相邻样本排成矩阵，

$$\boldsymbol{Z}_1 = \begin{bmatrix} \boldsymbol{\varphi}(\boldsymbol{x}_1)^{\top} \\ \vdots \\ \boldsymbol{\varphi}(\boldsymbol{x}_M)^{\top} \end{bmatrix}, \qquad \boldsymbol{Z}_2 = \begin{bmatrix} \boldsymbol{\varphi}(\boldsymbol{x}_2)^{\top} \\ \vdots \\ \boldsymbol{\varphi}(\boldsymbol{x}_{M+1})^{\top} \end{bmatrix}, \qquad \boldsymbol{W} = \begin{bmatrix} \boldsymbol{Z}_1 & \boldsymbol{u} \end{bmatrix} \tag{77-16}$$

带输入的 EDMD（文献里常记作 EDMDc）就是解

$$\boldsymbol{\Theta}^{\star} = \arg\min_{\boldsymbol{\Theta}} \left\| \boldsymbol{Z}_2 - \boldsymbol{W}\boldsymbol{\Theta} \right\|_F^2 \tag{77-17}$$

它的闭式解是 $\boldsymbol{\Theta}^{\star} = (\boldsymbol{W}^{\top}\boldsymbol{W})^{-1}\boldsymbol{W}^{\top}\boldsymbol{Z}_2$，或直接用 QR 分解求解以避免显式求逆。解出来之后按行切开：前 n 行转置得 $\boldsymbol{A}^{\top}$，最后一行转置得 $\boldsymbol{B}^{\top}$。本章脚本里就是三行：

```
A = Th[:-1].T
B = Th[-1:].T
```

式 (77-17) 有一个容易被忽视的细节：**它把字典的所有分量同等对待**。如果 $\varphi_1 = x_1$ 的量级是 1，而 $\varphi_9 = x_1^5$ 的量级是 0.1，最小二乘会倾向于把误差摊在量级大的分量上，结果就是低阶项拟合得好、高阶项拟合得差。这不是算法缺陷，而是“没有加权”的必然后果。工程上的对策有两个：一是把字典按样本标准差归一化后再拟合，二是改用状态空间误差作为评价指标（§77.5.2）。

77.2.4 连续时间与离散时间怎么选

式 (77-6) 是离散形式，式 (77-10) 是连续形式，两者只差一个矩阵指数。工程上选哪个，取决于数据是怎么来的。

离散形式适合“采样数据直接拟合”：传感器每 T_s 给一组数，相邻两组就是一个样本对，不需要知道采样之间的演化。本章的全部实验都用这种形式，因为现场数据就是这样。

连续形式适合“有解析模型、想算谱”：如果 $\boldsymbol{f}$ 已知，可以直接在字典上算 $\mathcal{L}$ 的矩阵表示，特征值就是连续时间的衰减率与频率，物理意义更直观。对象 A 的式 (77-13) 就是连续形式。

两者的换算在数值上要小心。$\lambda = \mathrm{e}^{\nu T_s}$ 在 $|\lambda|$ 接近 1 时对舍入误差极其敏感：$T_s = 0.02$、$\nu = -0.01$ 时，$\lambda = 0.9998$，此时 λ 的 10^{-6} 误差会让 ν 产生 5×10^{-5} 的误差。本章脚本里两次出现“对数特征值”，都用了双精度，并且只报四位有效数字，就是为了避开这个陷阱。

77.2.5 提升空间的维数从哪来

字典大小决定了提升维数 n。二状态对象取到 d 阶完全多项式时，

$$n = \frac{(d+1)(d+2)}{2} \tag{77-18}$$

$d=1$ 给 $n=3$，$d=2$ 给 $n=6$，$d=3$ 给 $n=10$。本章脚本去掉了常数项（它对演化没有信息，且会让 $\boldsymbol{A}$ 多一个等于 1 的平凡特征值），所以实际列数是 $n-1$：线性字典 2 列、三次 9 列、四次 14 列、五次 20 列。径向基字典取 $5\times 5=25$ 个中心，加上两个线性项共 27 列。

n 增长得很快。$d=8$ 时 $n=44$，$d=12$ 时 $n=90$；如果再加入三角函数、指数函数等“物理字典”，n 上百是常事。这就是 Koopman 方法最现实的成本：**它的线性是用维数换来的**。§77.7 会给出维数、数据量、计算量三者的定量关系。

77.2.6 两种残差：伽辽金与最小二乘

式 (77-15) 说了“近似”，但没说在什么意义下近似。这个“意义”决定了你算出来的 $\boldsymbol{A}$ 到底是什么。

第一种意义是连续伽辽金。 要求残差 $\mathcal{K}\varphi_j - \sum_i A_{ij}\varphi_i$ 与子空间里的每个基函数都正交，内积按状态空间上的某个测度取。如果子空间不变，残差恒为零，投影是精确的——对象 A 的三项字典就是这种情形。

第二种意义是离散最小二乘。 只在采集到的样本点上让误差平方和最小，也就是式 (77-17)。它不提“正交”，只提“这 M 个点上误差小”。

两者的关系是：**离散最小二乘就是拿经验测度代替真实测度的伽辽金投影**。样本在哪儿多，测度就在哪儿重，模型就在哪儿准。这句话解释了本章后面几个看似无关的现象。

第一，它解释了为什么训练数据必须覆盖工作域（§77.3.4）。数据没去过的地方，测度为零，残差在那儿取什么值都不影响最小二乘，模型的行为完全由外推决定。

第二，它解释了为什么闭环初值超出训练域时仍能收敛（表 77-5）。单摆的非线性是良性的，外推虽然不准，但方向没错；换个带继电器或死区的对象，同样的外推会直接把闭环推散。

第三，它解释了为什么“加数据”与“加覆盖”是两件事（§77.5.4）。在同一小片工况里加样本只是把测度变得更集中，谱半径会改善，但工作域边缘的精度不会动。

残差不为零有三个来源，对策完全不同，混在一起就会得出错误的调参结论。

结构误差来自子空间不变性缺失。字典里没有 x_1^2，无论多少数据都补不回来。对策是加阶或换字典族。

统计误差来自样本有限。N 增大时它会下降，但下降速度取决于激励的丰富程度。

数值误差来自病态。高阶列量级悬殊、矩阵条件数大，最小二乘的解对舍入敏感。对策是归一化与岭。

有一个便宜的判别方法：固定字典，把 N 从 400 加到 6400，看单步误差怎么走。若单调下降到某个平台，说明统计误差已饱和、剩下的是结构误差；若抖动甚至上升，说明数值误差在起作用。

本章表 77-3 就是这个方法的实例：字典固定为三次，单步误差从 $N=50$ 的 2.1039×10^{-6} 升到 $N=1600$ 的 1.1648×10^{-4}，之后才回落到 8.3298×10^{-5}。

单步误差随 N 上升，初看违反直觉，其实是**从训练误差变成了泛化误差**。$N=50$ 时 9 个参数拟合 50 个点，模型几乎在插值，训练集上的单步误差自然小得离谱；N 大了，每个点都摊不到一个自由参数，误差才反映真实水平。

所以跨 N 比较单步误差没有意义，只能在同一个 N 下比较不同字典。这一点与普通回归的“训练误差—验证误差”之分完全一致，只是 Koopman 把训练集与验证集的差别藏在了字典里。

77.2.7 什么该放进字典

字典是方法的全部“先验”，选它等于决定模型能表达什么。按信息来源分，可用的基函数有三类。

第一类是状态的低阶多项式。 $x_1, x_2, x_1^2, x_1x_2, x_2^2, \ldots$。它们不需要任何对象知识，适合做默认选择。缺点是维数涨得快，且外推时迅速爆炸。

第二类是物理已知项。 单摆的 $\sin x_1$、干摩擦的 $\operatorname{sgn}(x_2)$、饱和的 $\tanh(kx_2)$。这类项一旦猜对，维数可以压到很低，精度还高。

第三类是局部基。 RBF、样条、分段多项式。它们不依赖对象知识，靠局部逼近换取灵活性，适合“非线性形状复杂但没有明确表达式”的场合。

选择顺序应当是：**先用第二类，没有就先上第一类，两者都不够再考虑第三类**。这个顺序的理由是“先验越强，需要的数越少”。

有一个常见的错误是把三类混在一起，觉得“多放一些总没坏处”。实际后果是列之间高度相关，条件数暴涨，谱半径越过 1。§77.7.8 的耦合分析已经说明，加列必须加数据。

关于常数项。 本章脚本去掉了常数项 1。原因是它在演化下不变，会让 $\boldsymbol{A}$ 多一个等于 1 的特征值，谱半径的下界被钉在 1 上，之后所有关于稳定性的判断都失效。

如果对象有明显的偏置（比如重力导致的静态力矩），正确做法是把偏置并进输入通道，而不是放一个常数基函数。

关于量纲。 角度以 rad 计、角速度以 rad/s 计，两者的数值量级可能差一个数量级。混合项 x_1x_2 与 x_1^2 的量级也随之不同，最小二乘会偏向量级大的列。

处理办法有两个：一是拟合前按列标准差归一化（§77.3.5），二是选字典时就避免量级悬殊的组合。前者是通用手段，后者要求对工况有感觉。

关于“要不要加 $\sin x_1$”。 对单摆这类对象，加 $\sin x_1$ 几乎一定有效，因为它就是非线性本身。但工程上更常见的情形是“知道有非线性、不知道是什么形状”，这时加一个 $\sin$ 只是在赌。

一个稳妥的做法是同时准备两套字典：多项式版与物理版，用同一批数据各拟合一次，比三个校验指标。如果物理版明显更好，说明猜对了；如果差不多，就用多项式版，因为它不依赖猜测。

这条建议看起来保守，但它避免了一个隐蔽的风险：**物理项猜错时，模型不会报警，只会在某个工况下突然失效**。多项式版至少误差是连续变化的，容易发现。

77.3 设计步骤

77.3.1 七步流程总览

Koopman 控制看起来是一堆矩阵运算，落到工程上仍然是一条有先后的流程。顺序错了，后面每一步都会建在错的基础上。表 77-1 是本章推荐的七步，后面六小节逐步展开。

步骤	做什么	交付物	常见错误
1	确定观测变量、工况范围、约束	变量清单与量程	只取状态，忘了执行器动态
2	选字典族与阶数	候选字典 2–3 套	一上来就用五次多项式
3	设计激励、采集数据	训练集与验证集	用闭环运行数据当训练集
4	拟合 $\boldsymbol{A},\boldsymbol{B}$ 并正则化	提升模型	不做归一化、不做岭
5	三重校验：单步、谱半径、长时	校验报告	只看单步误差
6	在提升空间设计控制器	增益或 MPC	直接把 $\boldsymbol{Q}$ 套在提升坐标上
7	闭环验证与退化预案	上线判据	不准备回退到线性化方案

这张表里最容易被跳过的是第 5 步。多数教程给完最小二乘公式就结束了，而工程上真正决定成败的正是“你怎么知道自己拟合对了”。本章把第 5 步拆成三个互补的指标，原因在 §77.5.2 会看到：**单步误差与长时误差可以反向变化**，只报一个必然误导。

77.3.2 第一步：把观测变量定清楚

第一步不是选字典，而是把“能测什么”列清楚。这里有三个常犯的错。

第一个错是只写状态。 式 (77-1) 的状态是角度与角速度，看起来两个量就够。但现场往往测不到角速度，只能对角度做差分；差分引入的噪声在提升空间里会被高阶项放大。如果角速度是观测出来的，字典里就应该包含滤波后的角速度，而不是原始差分。

第二个错是忘了执行器动态。 力矩源如果是电机，电流环有几十毫秒的滞后；如果是气动，阀门的死区与滞环更明显。这些动态不写进字典，辨识出来的 $\boldsymbol{B}$ 会把滞后摊进增益里，闭环在高频段就会出现相位不足。工程上的做法是把执行器输出也作为一个观测变量，让字典自己去学它与状态的耦合。

第三个错是量程没定。 提升是非线性的，超出训练数据覆盖范围之后，多项式字典会以 x^5 的速度外推，结果毫无意义。所以第一步必须写下“训练覆盖 $x_1 \in [-1.35, 1.74]$、$x_2 \in [-1.18, 1.27]$”这样的边界（本章脚本按初值 ± 2.2 抽样，实际轨迹落在这个范围内），并在第 7 步把它变成软约束或报警。

77.3.3 第二步：先小后大选字典

字典的选择有三条判据，按优先级排列。

判据一：能覆盖非线性机制。 如果知道非线性来自 $\sin x_1$，字典里放 $\sin x_1$ 与 $\cos x_1$ 比放 x_1^5 有效得多；如果来自干摩擦，放 $\operatorname{sgn}(x_2)$ 或 $\tanh(k x_2)$ 更合适。这类“物理字典”能把维数压到很低，但要求对对象有先验认识。

判据二：在数据覆盖范围内数值稳定。 五次多项式在 $x_1 = 2.2$ 处的值是 51.5，而线性项只有 2.2，两者相差 23 倍。如果数据只覆盖到 1.0，x_1^5 的最大值只有 1，量级接近，拟合是良态的；一旦外推到 2.2，高阶项立刻主导。这就是为什么同一套字典在不同工况下表现可以差很多。

判据三：列数不要超过样本数的十分之一。 这是经验值。$N = 400$ 个样本配 20 列的字典，比例是 $20:1$，勉强够；配 27 列的 RBF，比例降到 $15:1$，噪声一大就开始过拟合（§77.8 有实测）。样本少的时候，宁可降阶也不要加岭——降阶损失的是模型结构，加岭损失的是无偏性，前者还可以靠物理直觉补，后者只能靠更多数据补。

77.3.4 第三步：激励要在“敢做”的范围内尽量丰富

这是本章最容易被低估的一步。很多现场的做法是“让设备正常跑一段，把数据拿来拟合”，看起来省事，实际上采集到的数据只在一条窄窄的工况带里，提升模型在这个带外完全没有约束。

正确的做法是主动设计激励。可选的信号有四类：

- **伪随机二进制序列（PRBS）**：幅值只有两档，对执行器友好，频谱宽，适合阀门类对象；
- **多正弦**：几个不相关的频率叠加，频谱可控，信噪比高，适合伺服类对象；
- **阶跃序列**：最贴近工艺操作，但只激励低频，高频信息缺失；
- **闭环抖动**：在反馈上叠加小幅随机扰动，最安全，但激励能量集中在闭环带宽内。

本章 E9 实测了这四种信号（§77.10.2）：多正弦的条件数最小（12.14），阶跃最大（138.53），而闭环抖动的条件数高达 120.36 却给出了可用的模型。这组数据说明**条件数不是判据**——后面会详细解释。

77.3.5 第四步：拟合、归一化与正则化

拟合本身是一次最小二乘（式 77-17），工程上要加三件事。

归一化。 把每一列 φ_i 除以它在训练集上的标准差，拟合完再把系数除回去。这一步能把条件数降一到两个数量级，几乎不要成本。

岭项。 在正规方程里加 $\gamma \boldsymbol{I}$：

$$\boldsymbol{\Theta}^{\star} = \left(\boldsymbol{W}^{\top}\boldsymbol{W} + \gamma \boldsymbol{I} \right)^{-1} \boldsymbol{W}^{\top} \boldsymbol{Z}_2 \tag{77-19}$$

γ 的作用是把解往零拉，等价于给高阶项加惩罚。γ 的量级要相对于 $\boldsymbol{W}^{\top}\boldsymbol{W}$ 的迹来定，通常取 $\gamma = 10^{-6} \sim 10^{-3}\|\boldsymbol{W}\|_F^2/n$。本章无噪声实验取 $\gamma = 0$，噪声实验也保持 0，目的是把“过拟合”这个现象暴露出来而不是掩盖它。

分段处理。 数据来自多段独立实验时，不要把段尾与段首接起来当成相邻样本。本章脚本按段堆叠（`np.vstack`），段内取相邻对，段间不取，这样避免了“从段尾跳到段首”的假样本。

77.3.6 第五步：三重校验，一个都不能少

这一小节是本章方法论的骨架，后面所有实验的结论都建立在它上面。

校验一：状态空间单步误差。 注意不是提升空间的残差。定义

$$e_{1\text{步}} = \sqrt{\frac{1}{M}\sum_{k=1}^{M}\left\|\hat{\boldsymbol{x}}_{k+1} - \boldsymbol{x}_{k+1}\right\|^2}, \qquad \hat{\boldsymbol{x}}_{k+1} = \left[\boldsymbol{A}\boldsymbol{\varphi}(\boldsymbol{x}_k) + \boldsymbol{B}u_k\right]_{1:2} \tag{77-20}$$

只取提升向量的前两维（也就是 x_1, x_2）反投影回来比。为什么必须这样？因为提升空间的残差随字典增大而**上升**——字典越大，高阶分量越多，拟合时摊到每个分量上的相对误差越大，但状态本身可能预测得更准。本章实测：线性字典的提升残差 1.24×10^{-3}，五次字典是 7.18×10^{-3}，后者反而大 5.8 倍；而状态单步误差从 1.0844×10^{-3} 降到 6.2731×10^{-7}，后者好 1728 倍。两个指标给出完全相反的排序，只有状态空间的那个才是工程师关心的。

校验二：谱半径。 计算 $\rho(\boldsymbol{A}) = \max_i |\lambda_i(\boldsymbol{A})|$。离散系统要求 $\rho < 1$，否则开环预测必然发散。这个指标极其便宜（一次特征值分解），却极其有效：本章 E3 里，$N \le 400$ 时谱半径在 $1.0126 \sim 1.0261$ 之间，200 步预测误差是 $0.0871 \sim 11.8645$；$N \ge 800$ 后谱半径降到 1.0007 以下，误差掉到 $0.0195 \sim 0.0589$。**分水岭就是 $\rho = 1$ 这条线。**

校验三：长时开环 rollout。 用验证集（与训练集不同的初值与激励）跑 200 步纯开环预测，算

$$e_{200} = \sqrt{\frac{1}{200}\sum_{k=1}^{200}\left\|\hat{\boldsymbol{x}}_k - \boldsymbol{x}_k\right\|^2} \tag{77-21}$$

这个指标把单步误差与谱半径的效应合在一起：单步误差小但谱半径略大于 1，200 步后误差会被指数放大；谱半径小于 1 但单步误差大，误差会稳定在一个偏高的平台上。工程上真正该看的是它。

三个指标的读法可以总结成一句话：**单步看拟合，谱半径看稳定，长时看可用。**

77.3.7 第六步与第七步：控制器与闭环

有了可信的提升模型，第六步是在它上面设计控制器。最直接的是离散 LQR：

$$\boldsymbol{K} = \left(\boldsymbol{R} + \boldsymbol{B}^{\top}\boldsymbol{P}\boldsymbol{B}\right)^{-1}\boldsymbol{B}^{\top}\boldsymbol{P}\boldsymbol{A}, \qquad \boldsymbol{P} = \boldsymbol{A}^{\top}\boldsymbol{P}\boldsymbol{A} - \boldsymbol{A}^{\top}\boldsymbol{P}\boldsymbol{B}\left(\boldsymbol{R} + \boldsymbol{B}^{\top}\boldsymbol{P}\boldsymbol{B}\right)^{-1}\boldsymbol{B}^{\top}\boldsymbol{P}\boldsymbol{A} + \boldsymbol{Q} \tag{77-22}$$

式 (77-22) 是标准 Riccati 方程，与第 30 章的连续形式只差离散化，不再重复推导。需要强调的是 $\boldsymbol{Q}$ 的取法：提升坐标里除 x_1, x_2 之外的分量没有直接物理意义，给它们加权等于隐式地惩罚“非线性程度”。本章的做法是给前两维正常权重（1 与 0.2），其余维给很小的权重（10^{-3} 或 10^{-4}），让它们只起“补足动力学”的作用，不参与性能评价。

第七步是闭环验证。这里要准备一个退化预案：如果闭环性能不如线性化方案，或者在某些初值下失稳，要能立刻切回线性化 LQR。本章实测的结论是，在这个单摆对象上，Koopman 方案与线性化方案的闭环代价只差 0.27%（§77.11），所以退化预案不是可选项而是必备项——**收益这么小的时候，任何一点异常都足以抵消它。**

77.3.8 把七步走一遍：单摆的选型演练

抽象的流程只有落到一个具体场景里才知道怎么用。这一小节把表 77-1 完整走一遍，数字全部取自本章实验。

设定如下：单摆台架，角度编码器分辨率 0.001 rad，力矩电机限幅 ± 4，工艺要求是从任意初值在 7 s 内回到竖直位置，且角速度不超过 1.2 rad/s。

第一步，列变量与量程。 状态取角度与角速度，输入取力矩。量程由静态检查确定：角度 ± 3.0 rad、角速度 ± 2.5 rad/s、力矩 ± 4。训练域要留余量，取角度 ± 1.7 rad，而不是贴着机械限位。

第二步，先试线性字典。 拟合出来的单步误差 1.0844×10^{-3}，200 步 RMSE 0.0669。这两个数只能作为基准，不能单独判断好坏——它们说明“线性字典能做到这个程度”，但没说该不该加阶。

判断依据要拿线性化误差来比。工作域边缘 1.7 rad 处 $\sin x_1 = 0.9917$，线性化给出 1.7，高估 71%。既然结构误差这么大，就值得加阶，于是试三次字典。

三次字典的单步误差 4.3217×10^{-5}，比线性好 25 倍，200 步 RMSE 0.0429。改善明确，先停在这里，不急着上五次——四次、五次留到“三次不够”时再试。

第三步，设计激励。 选多正弦，四个频率、每路幅值 0.30，采 6 段、每段 400 步，共 2400 个样本。三次字典 $n = 9$，$40n = 360$，样本量有六倍余量。

不用阶跃的理由很实在：表 77-9 里阶跃数据的闭环代价是 524.5593，是正常值的 147 倍。现场没有第二次机会去发现这个错误，所以第一次就该选对激励。

第四步，拟合。 先不加岭，把无正则化的结果留作基线；同时保留 γ 作为后续手段——如果谱半径过线，第一反应是加样本，第二反应才是加岭。

第五步，三重校验。 单步误差比线性字典好 25 倍，满足“至少一个数量级”；200 步 RMSE 0.0429 低于工作域 1.7 rad 的 5%（即 0.085），通过；谱半径按 §77.5.4 的门槛应小于 1，若不满足就按表 77-7 的第 3 轮加样本。

第六步，设计控制器。 取 $\boldsymbol{Q} = \mathrm{diag}(1, 0.2, 10^{-3}, \dots)$、$R = 0.1$，与式 (77-2) 的代价对应。解出的增益前两维是 2.1756 与 2.3846，与线性化的 2.2385、2.3451 同量级；第三维只有 -4.0×10^{-4}，几乎为零（§77.6.6 会专门讨论这一点）。

第七步，闭环验证与决策。 初值从 0.65 扫到 2.60，闭环代价 0.3599 到 6.3181；同条件下线性化是 0.3597 到 6.3399，最坏情形只好 0.34%。

按 §77.8.3 的判定标准（改善不足 1% 就不值得增加复杂度），这次选型的结论是**不采用 Koopman-LQR**。但工艺还有一条“速度不超过 1.2 rad/s”，线性化 LQR 会越界 27 步，这条它满足不了。

于是回到表 77-8：Koopman-MPC 用 17.1% 的代价换来 0 步越界。到这一步，采用 Koopman 的理由已经不是“更准”，而是“能把速度约束写成线性不等式”。**选型结论从“提升更准”换成了“提升更好约束”，这才是这条路的真实卖点。**

这个演练的意义在于：七步走完得到的结论可能是“不用”，这不是失败，而是选型的正常结果。本章大部分篇幅在讲怎么把模型做准，但工程决策的依据始终是“任务需要什么”，而不是“方法有多先进”。

77.3.9 激励频率怎么选

“多正弦”三个字说起来简单，频率选错等于没激励。这一小节给出一套可执行的选频方法。

最低频率由数据总时长决定。 式 (77-27) 说 $f_{\min} = 1/T_{\text{total}}$。要比这个值高两到三倍，才有足够多的周期去辨识。

本章 $T_{\text{total}} = 48$ s，$f_{\min} = 0.021$ Hz，取最低频率 0.18 Hz，是下限的八倍多，余量充足。如果把总时长缩到 8 s，$f_{\min} = 0.125$ Hz，0.18 Hz 就只剩一点四倍，辨识会变得不稳。

最高频率由闭环带宽与采样决定。 经验上取闭环带宽的五到十倍，同时不超过 $1/(10T_s)$。本章 $T_s = 0.02$ s，上限是 5 Hz；实际取的最高频率 1.21 Hz，远低于上限。

为什么不取到上限？因为再高的频率对状态预测没有帮助，反而会激励伪模态，让谱半径变大。

频率不要成整数比。 四个频率若取 $1:2:3:4$，叠加信号会周期重复，某些相位组合下幅值叠加成尖峰。本章取的是非整数比，叠加后的输入峰值 1.11，比单路幅值 0.30 的四倍小得多。

幅值要逐级加。 从 0.1 到 0.2 再到 0.3，每一档跑 30 s，观察有没有异响、超限、饱和。本章最终取 0.30，轨迹覆盖 $x_1 \in [-1.35, 1.74]$，正好在工作域内。

幅值再大，轨迹会跑到 2 rad 以上，高阶项主导，拟合出来的模型在常用工况反而不准。幅值再小，$\boldsymbol{B}$ 的信噪比不够（§77.6.1 前提三）。

怎么检查选得对不对？ 采完数据后看一眼输入与状态的频谱：输入应在四个频率上有明显峰，状态应在这些频率及其谐波上有响应。

如果状态频谱在某个频率上没有响应，说明该频率没有激励到对象，可以去掉；如果输入频谱出现很多高频毛刺，说明执行器有非线性（比如死区），需要在预处理时注意。

最后一条：频率要与控制目标对齐。 如果关心的是“多快能回到零”，激励就要覆盖闭环带宽附近；如果关心的是“稳态精度”，低频段的激励更重要。频率选得再“宽”，与目标无关的频段也只是浪费行程。

77.4 连续情形

77.4.1 为什么先讲连续：谱有物理意义

离散特征值 λ 是个复数，$|\lambda|<1$ 表示衰减，辐角表示每步转多少度。这两个量都不直观。连续特征值 $\nu=\ln(\lambda)/T_s$ 的实部直接是衰减率（单位 1/s），虚部直接是角频率（单位 rad/s），能和“时间常数 $1/|\mathrm{Re}\,\nu|$ 秒”“周期 $2\pi/|\mathrm{Im}\,\nu|$ 秒”对应起来。

所以本章的诊断都换算到连续域再讲。换算的前提是 T_s 足够小，使得 νT_s 落在主值区间 $(-\pi,\pi)$ 内；本章 $T_s=0.02$ s，最高关注的频率约 1.2 Hz（角频率 7.5 rad/s），$\nu T_s\approx 0.15$，离 π 很远，不存在混叠。

77.4.2 解析构造：三项字典的生成元矩阵

§77.2.2 已经算出对象 A 的生成元矩阵（式 77-13）。把它写成离散形式只需要一次矩阵指数：

$$\boldsymbol{A}=\exp\left(\boldsymbol{L}T_s\right),\qquad \boldsymbol{L}=\begin{bmatrix}\mu & 0 & 0\\ 0 & \lambda & -\lambda\\ 0 & 0 & 2\mu\end{bmatrix} \tag{77-23}$$

因为 $\boldsymbol{L}$ 是上三角的，$\exp(\boldsymbol{L}T_s)$ 也是上三角的，对角元分别是 $\mathrm{e}^{\mu T_s}$、$\mathrm{e}^{\lambda T_s}$、$\mathrm{e}^{2\mu T_s}$。这就是式 (77-14) 那三个数。非对角元 $(2,3)$ 是

$$[\exp(\boldsymbol{L}T_s)]_{23}=-\lambda\,\frac{\mathrm{e}^{\lambda T_s}-\mathrm{e}^{2\mu T_s}}{\lambda-2\mu} \tag{77-24}$$

代入 $\lambda=-1$、$\mu=-0.2$、$T_s=0.02$ 得 0.01972207，与脚本拟合出来的 $(2,3)$ 元 0.01972207 完全一致。这个非对角元很关键：它表示“x_1^2 会往 x_2 里注入多少”，是慢流形耦合的直接度量。如果字典里没有 x_1^2，这一项只能被强行压进 $(2,1)$ 元，而 $(2,1)$ 元在真实系统里是严格为零的——拟合出来的线性模型于是把“平方项的影响”误认成“线性耦合”，谱就被扭曲成 -1.5174。

77.4.3 精确与不精确的对比

图 77-2 是同一段 300 步（6 s）自治数据的预测对比。真值用粗黑线，线性字典用红虚线，三项字典用灰点划线。

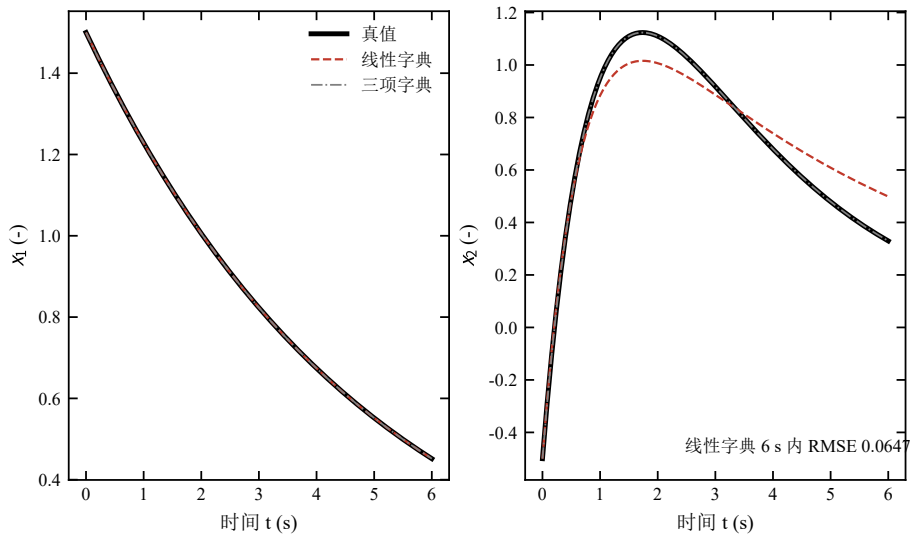


图 77-2 对象 A 的 300 步开环预测：三项字典与真值完全重合，线性字典在 2 s 后明显偏离

三项字典那条线在图上根本看不出来，因为它与真值重合到 10^{-14}。线性字典的偏差从 1 s 左右开始显现，到 6 s 时累积到 6.4681×10^{-2} 的 RMSE。注意这个偏差**不是随机漂移**：它是有方向的，x_1 偏大、x_2 偏小，因为线性模型把“x_1^2 向 x_2 注入”这一项漏掉了，导致 x_2 衰减得比真实快，反过来又让 x_1 衰减得比真实慢。

这就是“谱错了会怎样”的直观版本：错的特征值 -1.5174 比真值 -1.0 衰减更快，于是 x_2 被过度衰减；漏掉的耦合项本应把 x_1^2 的能量送进 x_2，现在这部分能量留在 x_1 里，x_1 就衰减不足。**谱的误差与轨迹的误差是一一对应的，只是前者藏在复数里，后者画在图上才看得见。**

77.4.4 连续谱怎么读

把拟合矩阵的特征值取对数除以 T_s，得到三个连续特征值：-0.2、-0.4、-1.0。它们对应三种时间尺度：

- -0.2：时间常数 5 s，是 x_1 的慢衰减；
- -1.0：时间常数 1 s，是 x_2 追随 x_1^2 的快衰减；
- -0.4：时间常数 2.5 s，是 x_1^2 的衰减，恰好是 x_1 的两倍速率。

第三个特征是理解慢流形的钥匙。x_1^2 衰减得比 x_1 快一倍，所以经过几个时间常数后，x_1^2 相对于 x_1 可以忽略，x_2 的动态退化成 $\dot{x}_2 = \lambda x_2$，系统在 (x_1, x_2) 平面上沿 $\mathrm{d}x_2/\mathrm{d}x_1 = \lambda x_2/(\mu x_1)$ 这条曲线滑向原点。这条曲线就是**慢流形**，它的存在让“降阶建模”成为可能——很多工程对象的有效维数远低于名义维数，原因就在这里。

77.4.5 采样周期的影响

连续形式看起来更漂亮，但工程上必须回到离散，因为数据是离散采的。T_s 的选择有两个约束。

上界来自混叠与稳定性。 对象最快的极点实部是 -1.0，周期约 6.3 s，$T_s = 0.02$ s 对应每周期 314 个点，非常充裕。如果把 T_s 加到 0.1 s，每周期 63 个点，仍然够用；加到 0.5 s 就只剩 12.6 个点，离散化的相位误差开始明显。经验规则是 $T_s \le 1/(10|\nu_{\max}|)$，本章 $|\nu_{\max}| \approx 1$，所以 $T_s \le 0.1$ s。

下界来自噪声放大。 T_s 越小，相邻样本的差越小，测量噪声在差分里的相对比重越大。本章 E4 的噪声实验用 $T_s = 0.02$ s，噪声标准差 0.06 rad 时，相邻两步的角度差通常只有 $0.02 \times 1 = 0.02$ rad，噪声已经是信号的三倍。这时提升拟合实际上是在拟合噪声，必须靠岭或降阶来压制。

两个约束一夹，T_s 的合理区间往往很窄。工程上的做法是先按上界定，再根据噪声水平往下调一档，然后**用状态单步误差验证**：如果单步误差随 T_s 减小而急剧上升，说明已经进入噪声主导区，应该退回去。

77.4.6 连续形式什么时候值得用

连续形式好看，但不是所有场合都值得绕这一圈。判断标准只有一条：**算出来的 ν 会不会被你真正用来做决定**。

值得用的第一种场合，是对象有一部分解析知识。比如单摆的 $\sin x_1$ 是确定的，只有阻尼系数未知；这时可以在已知结构上算生成元，只让数据去定那几个系数。这样得到的 ν 有明确的物理归属，不是一堆拟合出来的数。

值得用的第二种场合，是要与线性化极点对照。§77.6.2 就是例子：DMD 给出 $-0.1072 \pm 0.8919j$，线性化给出 $-0.1 \pm 0.9950j$，实部差 7.2%、虚部差 10.4%。这两个数只有在连续域里才好比；换成离散特征值，实部与虚部的差别都落在小数点后第三、四位，肉眼很难分辨。

值得用的第三种场合，是要做频域设计或共振分析。ν 的虚部直接是振荡频率，实部直接是阻尼，能与 Bode 图、与结构的固有频率对上。用离散 λ 也能换算，但每换一次就多引入一次对数误差。

反过来，三种场合不值得用。**一是纯黑箱数据。** 对数换算要求严格等间隔采样，现场采集常有丢帧与抖动，一处理就要插值，插值又引入相位误差。

二是矩阵非正规严重。 特征值对扰动本来就敏感，取对数会把这种敏感放大。这时离散 $\boldsymbol{A}$ 的谱半径反而是更稳的判据，因为它只依赖特征值的模，不依赖幅角。

三是只做预测或控制。 直接拟合离散映射就够了，中间多一层连续化没有任何收益。本章的实验全部走这条路。

本章采用的是一种折中：**拟合离散 $\boldsymbol{A}$，只在读谱时取对数，不回代仿真**。这样既保留了物理解释，又避免了连续模型在回代时的数值放大。

还有一个细节必须提醒：对数有分支。$\lambda = \mathrm{e}^{\nu T_s}$ 只在 $\nu T_s \in (-\pi, \pi)$ 时唯一。如果 T_s 太大，某个高频模态的 νT_s 超过 π，$\ln \lambda$ 会给出主值，把它误读成一个低频模态。

本章 $T_s = 0.02$ s、最高关注频率约 1.2 Hz（角频率 7.5 rad/s），$\nu T_s \approx 0.15$，离 π 很远，不存在这个问题。但凡把 T_s 放大到 0.5 s 以上，读谱前就必须先检查分支。

最后一条经验：**报连续特征值时一定同时报 T_s**。只写“极点在 -0.13”而没写采样周期，别人无法复算，也无法判断有没有混叠。这是文献里很常见的一处信息缺失。

77.4.7 谱的精度受什么限制

连续特征值看起来是“算出来的”，其实它的有效位数远少于矩阵元素的位数。四个因素依次限制精度。

第一是采样。 $\nu = \ln \lambda / T_s$ 里 T_s 是分母，T_s 的相对误差直接进入 ν。现场采样常有时基抖动，1% 的周期误差就有 1% 的谱误差。

第二是噪声。 噪声让 $\boldsymbol{A}$ 的估计有偏，特征值随之偏移。E4 里 $\sigma = 0.02$ 时五次字典的谱半径从 1.0045 掉到 0.9994，这个变化不是对象变了，而是估计变了。

第三是字典截断。 截断产生的伪模态会与真实模态“混”在一起。本章三次 EDMD 给出 9 个实极点，真正对应主要动态的只有 -0.1302 那一对，其余 7 个的数值随字典阶数漂移。

第四是数值。 对数在 $|\lambda|$ 接近 1 时条件数极大，双精度也只能保证几位有效数字。所以本章报谱时一律只给四位，最后一位还可能变。

怎么区分真实模态与伪模态？ 有三个可操作的判据。

第一，加阶后是否稳定。把字典从三次加到五次，-0.1302 几乎不动，其余的值明显漂移——稳定的那个可信。

第二，是否与已知物理时间常数对上。单摆的线性化极点 $-0.1 \pm 0.9950j$ 有明确来源，任何接近它的极点都值得注意。

第三，换一批数据是否复现。同一对象、不同激励段重复辨识，真实模态会稳定出现，伪模态会跳。

三个判据都通过，才可以给这个极点“编故事”。只通过一个，最好只把它当成拟合参数。

还有一条实践建议：**报谱时同时报条件数**。条件数大的模型，谱的误差也大；只看谱的数值而不看它的可信度，容易被最后一位小数误导。

77.5 离散情形

77.5.1 两条离散化路线

从连续到离散有两条路，选哪条取决于你手上有什么。

第一条：先辨识连续、再离散化。先用数据拟合生成元矩阵 $\boldsymbol{L}$，再算 $\boldsymbol{A}=\mathrm{e}^{\boldsymbol{L}T_s}$、$\boldsymbol{B}=\int_0^{T_s}\mathrm{e}^{\boldsymbol{L}\tau}\mathrm{d}\tau\,\boldsymbol{B}_c$。这条路的优点是特征值有物理意义，缺点是矩阵指数对 $\boldsymbol{L}$ 的误差敏感——$\boldsymbol{L}$ 的特征值差 1%，$\mathrm{e}^{\boldsymbol{L}T_s}$ 可能差得更多，尤其在 $\boldsymbol{L}$ 非正规时。

第二条：直接拟合离散映射。把相邻样本对直接代入式 (77-17)，得到的就是 $\boldsymbol{A}$、$\boldsymbol{B}$，不需要经过连续域。本章全部实验走这条路，因为它是“数据驱动”的原意：不假设中间有一个连续模型。

两条路的结果在 T_s 足够小时应该一致，但数值性质差别很大。第一条路的 $\boldsymbol{A}$ 一定是可逆的（矩阵指数永远可逆），第二条路的 $\boldsymbol{A}$ 在数据不足时可能接近奇异。这个差别在 §77.5.4 会变成一个可用的诊断信号。

77.5.2 字典阶梯实测

先看字典从小到大是什么表现。训练集用 6 段、每段 400 步的多正弦激励数据（共 2400 个样本），验证集另采一段 200 步，结果列在表 77-2。

字典	列数 n	状态单步误差	200 步 RMSE
线性	2	1.0844×10^{-3}	0.0669
三次	9	4.3217×10^{-5}	0.0429
四次	14	2.7734×10^{-5}	0.0259
五次	20	6.2731×10^{-7}	0.0187
RBF（5×5）	27	6.0449×10^{-7}	0.0305

两条趋势值得注意。

趋势一：误差都在降，但降的幅度不一样。从线性到三次，单步误差降了 25 倍，长时误差只降了 36%；从三次到五次，单步误差再降 69 倍，长时误差降 56%。**长时误差的下降总是比单步误差慢**，因为长时误差受谱半径支配，而谱半径的改善不随单步精度线性变化。

趋势二：RBF 不是万能的。RBF 字典的单步误差（6.0449×10^{-7}）与五次多项式（6.2731×10^{-7}）几乎相同，但长时误差差了 63%（0.0305 对 0.0187）。原因是 RBF 的局部性：每个基函数只在一小块区域里有效，区域之间靠线性项连接，长时外推时区域之间的“接缝”会累积误差。多项式是全局的，外推更平滑，代价是超出训练范围后迅速爆炸。

图 77-3 把这两列数据画在一起。左轴是对数的单步误差，右轴是线性的 200 步 RMSE，横轴是字典。图上能看出两条曲线的形状不同：单步误差在三次到五次之间掉了一个数量级，而长时误差是平缓下降的。

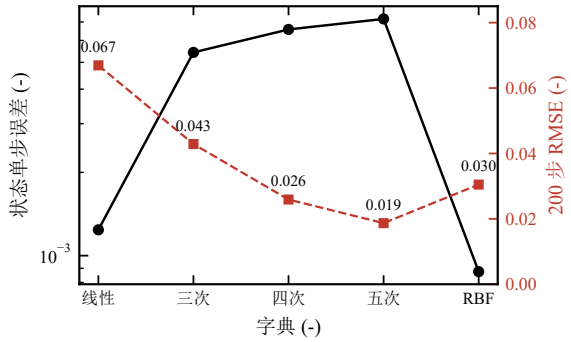


图 77-3 字典阶梯：单步误差（左轴，对数）与 200 步 RMSE（右轴）随字典增大的变化

77.5.3 残差为什么反向

§77.3.6 提到一个反直觉的现象：字典越大，提升空间的残差反而越大。这里把机理说清楚。

提升残差的定义是 $\|\boldsymbol{Z}_2 - \boldsymbol{W}\boldsymbol{\Theta}\|_F/\sqrt{Mn}$，分母里有 n。字典从 2 列增到 20 列，分母涨了 $\sqrt{10} \approx 3.16$ 倍。与此同时，新增的高阶分量 φ_i 本身量级不小、但它们的演化更难拟合：x_1^5 的下一步值涉及五阶项的乘积，用二阶以内的动力学去解释必然有残差。两项一叠加，平均到每个分量上的残差就上去了。

换句话说，**提升残差度量的是“每个分量平均拟合得多好”，而不是“状态预测得多准”**。字典变大时，新增的分量本来就更难拟合，平均下来自然变差；但它们携带的信息让状态预测变准。两个指标回答的是不同的问题，混用就会得出“字典越大越差”的错误结论。

工程上的推论很直接：**永远不要用提升残差选字典**。本章脚本里的 `state_err` 函数就是为此写的，它只取前两维反投影，让不同字典的误差可以直接比较。

77.5.4 数据量：谱半径才是分水岭

固定三次字典（9 列），把样本数从 50 增到 6400，结果见表 77-3 与图 77-4。

样本数 N	状态单步误差	谱半径 $\rho(\boldsymbol{A})$	200 步 RMSE
50	2.1039×10^{-6}	1.0194	11.8645
100	6.2936×10^{-6}	1.0140	1.0653
200	1.4217×10^{-5}	1.0261	4.6883
400	8.6250×10^{-5}	1.0126	0.0871
800	1.1460×10^{-4}	1.0007	0.0525
1600	1.1648×10^{-4}	0.9995	0.0589
3200	1.0296×10^{-4}	0.9993	0.0260
6400	8.3298×10^{-5}	0.9990	0.0195

这张表最刺眼的是前两列与最后一列的关系。$N = 50$ 时，状态单步误差只有 2.1039×10^{-6}——比 $N = 6400$ 时的 8.3298×10^{-5} 还好 40 倍。如果只看单步误差，会得出“数据越少越好”的荒谬结论。但它的 200 步 RMSE 是 11.8645，完全不可用。

原因就在第三列：$N = 50$ 时谱半径 1.0194，$1.0194^{200} = 46.8$，任何微小的初始误差都会被放大近 47 倍。而 $N = 50$ 只有 50 个样本拟合 9 个参数，最小二乘几乎是在插值，它把噪声与舍入误差一并拟合进去，得到的矩阵自然略不稳定。

$N \geq 800$ 后谱半径降到 1.0007 以下，$1.0007^{200} = 1.15$，放大倍数是 15%，可以接受。$N = 1600$ 时 $\rho = 0.9995$，$0.9995^{200} = 0.905$，误差反而略有收缩。这就是“分水岭”的含义：**不是样本数本身有魔法，而是谱半径跨过 1 的那一**

刻，长时行为从指数放大切换成指数衰减。

图 77-4 的左右两栏正好对应这两件事。左栏是误差随 N 的变化，右栏是谱半径随 N 的变化，右栏里那条 $\rho = 1$ 的参考线就是分界。

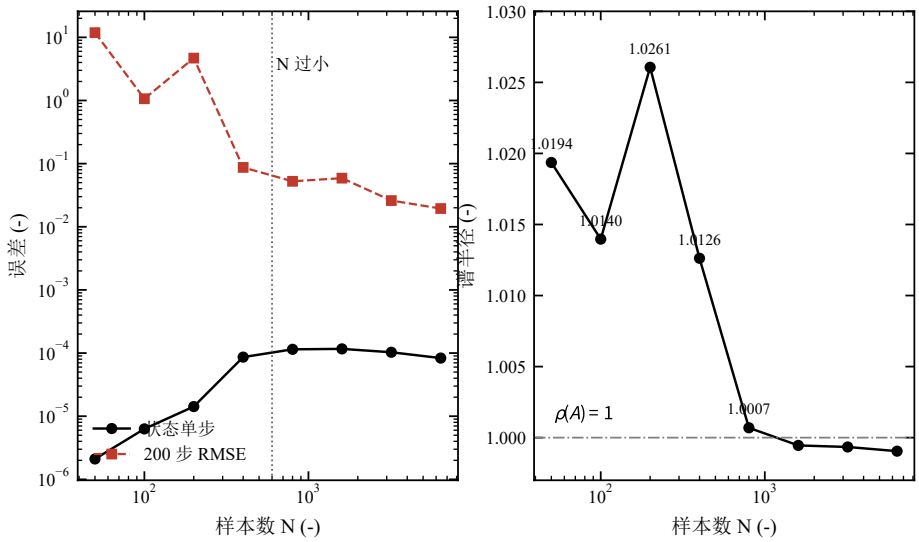


图 77-4 数据量扫描：误差（左）与谱半径（右）；谱半径跨过 1 的位置与长时误差骤降的位置重合

还要注意 $N = 400$ 到 $N = 1600$ 之间长时误差的小幅非单调（$0.0871 \to 0.0525 \to 0.0589$）。这不是随机波动：$N = 1600$ 时用的前缀覆盖了更多工况，而这些工况里包含更大的角度，线性化误差占比更高，所以误差略回升。$N = 3200$ 之后样本覆盖足够密，误差才继续下降。这个细节说明**“加数据”与“加覆盖”是两件事**：只在一小片工况里加样本，对谱半径有帮助，对模型在整个工作域上的精度帮助有限。

77.5.5 噪声下的阶数权衡

把测量噪声加到状态上（σ 从 0 到 0.06 rad），三次与五次字典的表现见表 77-4 与图 77-5。

噪声 σ (rad)	三次单步	三次 200 步	三次 ρ	五次单步	五次 200 步	五次 ρ
0	8.4804×10^{-5}	0.0149	0.9988	9.9799×10^{-7}	0.0320	1.0045
0.001	1.4560×10^{-3}	0.0145	0.9988	1.4532×10^{-3}	0.0363	1.0041
0.005	7.2665×10^{-3}	0.0142	0.9988	7.2436×10^{-3}	0.0363	1.0010
0.020	2.8998×10^{-2}	0.0663	0.9990	2.8174×10^{-2}	0.0624	0.9994
0.060	8.5691×10^{-2}	0.1004	0.9950	8.1800×10^{-2}	0.0898	0.9953

这张表推翻了一个常见的直觉：“阶数越高越容易过拟合，所以噪声一大就该降阶”。实际情形更微妙。

在无噪声与低噪声下，五次字典的单步误差确实好得多（9.9799×10^{-7} 对 8.4804×10^{-5}，差 85 倍），但它的谱半径 1.0045 略大于 1，200 步后误差反而翻倍（0.0320 对 0.0149）。这是典型的“单步更准、长时更差”：高阶项把训练数据拟合得很细，连数值舍入都不放过，拟合出的矩阵有极小的不稳定模态，200 步足够让它显形。

噪声增大到 0.02 之后，两者的谱半径都回到 1 以下，长时误差变得接近（0.0624 对 0.0663），五次还略好一点。原因有点反直觉：噪声把最小二乘解往零收缩（相当于隐式正则化），恰好把那个略大于 1 的模态压了回去。换句话说，**在这里噪声起了阻尼作用**，它牺牲了单步精度，换来了谱稳定性。

这个现象在工程上有实际意义：如果你的数据本身有噪声，不必急着降阶；先看谱半径，如果它已经在 1 以下，高阶字典是可以用的。反过来，如果数据是仿真来的、异常干净，高阶字典反而更危险——它会把数值误差当成真实动力学学进

去。本章 E2 的五次字典（无噪声、200 步 0.0187）表现很好，是因为训练样本足够多（2400 个）；E4 的五次字典（500 个样本）就露出问题了。**样本数与阶数必须一起看**，这也是 §77.3.3 判据三的由来。

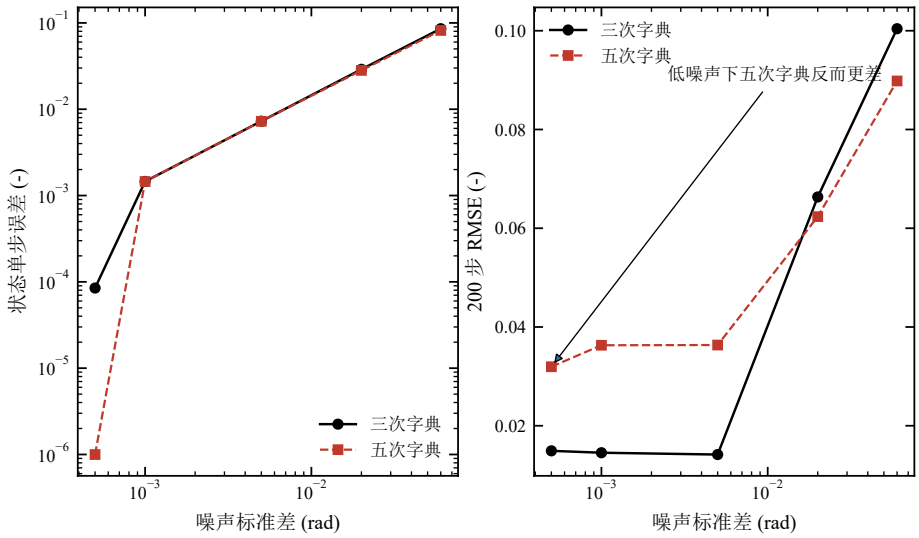


图 77-5 噪声与阶数的权衡：低噪声下五次字典单步更准但长时更差，噪声增大后两者趋同

77.5.6 一个可复用的校验清单

把上面几小节的结论整理成一张可执行的清单，每一步都有明确的通过判据。

1. **状态单步误差**：与线性化模型的单步误差比，Koopman 模型应至少好一个数量级，否则提升没有带来信息。
2. **谱半径**：必须小于 1。若在 $0.999 \sim 1.001$ 之间，长时预测会被放大，需要加样本或加岭。
3. **长时 rollout**：200 步 RMSE 应小于工作域尺度的 5%。本章工作域角度约 1.7 rad，门槛是 0.085，五个字典全部通过（最差的线性字典 0.0669 也低于门槛）——这条门槛不严，真正的区分度在前两条。
4. **验证集独立性**：验证数据的初值、激励、噪声水平都要与训练集不同。本章验证集用另一段随机初值的多正弦数据，seed 与训练集不同。
5. **闭环前的预演**：把设计好的 $\boldsymbol{K}$ 在提升模型上闭环仿真一遍，确认提升坐标不发散。提升坐标发散而状态收敛是可能的（高阶分量自己跑飞），这种情形上线后会在某个工况突然失效。

77.5.7 三个指标怎么合起来读

三个指标各自回答一个问题，但工程判断要把它们放在一起。下面按“先否决、再评价、后诊断”的顺序给一套读法。

第一步看谱半径，它有一票否决权。 $\rho \geq 1$ 意味着开环预测必然指数放大，无论单步误差多小都不能用。放大的速度可以用 ρ^{200} 估算：$1.0194^{200} = 46.8$，$1.0126^{200} = 12.3$，$1.0007^{200} = 1.15$。同样是“大于 1”，前者已经彻底不可用，后者只是缓慢漂移。

这一步最容易被忽略，因为单步误差看起来很好。表 77-3 里 $N = 50$ 的单步误差 2.1039×10^{-6} 是全表最低，谱半径 1.0194 却是全表最高，200 步 RMSE 直接爆到 11.8645。只看第一列，会得出完全相反的结论。

第二步看 200 步 RMSE，它决定“能不能用”。 门槛要按工作域定，本章取工作域 1.7 rad 的 5%，即 0.085。低于门槛的模型可以直接拿去设计控制器；高于门槛的，即使谱半径合格也要谨慎。

$N = 6400$ 是正例：$\rho = 0.9990$，$0.9990^{200} = 0.819$，误差不但不放大还略有收缩，200 步 RMSE 0.0195，余量很大。$N = 400$ 是临界例：$\rho = 1.0126$ 刚过线，200 步 RMSE 0.0871 刚好压在门槛上，这种模型能跑但经不起工况变化。

第三步看单步误差，它只用于诊断。 单步误差小说明拟合到了结构，大说明结构不够或数据不够；但它不能跨 N 比较，也不能单独决定取舍。

把三步合起来，可以得到四种组合，对应四种处置。

组合一：单步小、$\rho<1$、200 步小。 模型可用，进入控制器设计。本章 2400 样本的三次字典属于这一类。

组合二：单步小、$\rho>1$。 典型的插值式过拟合，$N=50$ 就是。处置是加数据，不是加岭——加岭会同时压掉真实的高阶动态。

组合三：单步大、$\rho<1$、200 步大。 结构误差主导。E4 的 $\sigma=0.06$ 一组是例子：五次字典 $\rho=0.9953$、单步 8.1800×10^{-2}、200 步 0.0898，谱半径很安全，误差却全部来自单步。处置是加阶或改字典族，加数据没用。

组合四：单步大、$\rho>1$。 数据与结构都缺，通常是“样本少 + 字典大”的组合。先按 $40n$ 补齐样本，再重看谱半径。

还有一条容易误读的现象：表 77-3 里 $N=200$ 的 200 步 RMSE（4.6883）比 $N=100$（1.0653）更差，而谱半径也确实从 1.0140 升到 1.0261。这不是随机波动，而是小样本下 $\boldsymbol{A}$ 的估计本身在抖——样本量没到“统计稳定”之前，加样本不一定单调改善。

所以“加数据”也有前提：**要加到统计稳定区，零散地加几十个样本没有意义**。本章的分界在 $N=800$ 附近，与 $40n=360$ 的经验值同量级但更保守，这也是表 77-6 把 N 的建议起点写成 $40n$ 而不是精确值的原因。

77.5.8 验证集怎么造

三个校验指标里有两个依赖验证集。验证集造错，指标再漂亮也没有意义。这一小节给出五条要求。

第一，初值必须独立。 验证段的初始状态要取自训练段没覆盖的位置，至少有一部分落在工作域边缘。本章训练段按初值抽样，验证段另取初值，两者不重叠。

第二，激励必须独立。 多正弦的相位、频率组合要与训练段不同；随机信号的种子要换。用同一段激励的另一次噪声实现不算独立。

第三，噪声水平要与现场一致。 训练数据干净、验证数据带噪，或反过来，都会让指标失真。本章 E4 在训练与验证上加同一水平的噪声。

第四，长度要够。 200 步是本章的标准，因为闭环验证也跑 200 步。验证段短于这个长度，就无法暴露谱半径引起的放大。

第五，至少三段。 单段验证的结论可能是运气。三段各自算指标，取最差的一段作为判据，比取平均更保守也更符合工程直觉。

最常见的错误是把训练集的后 20% 当验证集。 这部分数据与训练段共享同一条轨迹，初值、激励、噪声全都相关，指标会明显偏乐观。

另一种常见错误是“用同一批数据反复调参，直到验证误差满意”。这时验证集已经在事实上参与训练，指标失去意义。正确做法是留一段“只在最后看一次”的测试集。

验证集与工作域的关系要写清楚。 如果验证段只覆盖 $x_1\in[0,1]$，那么 200 步 RMSE 只能说明这个区间准，不能说明 2.6 rad 处准。本章的表 77-5 就是专门补这个缺口的：它直接用闭环初值扫描，覆盖到 2.60 rad。

最后一点：验证集要能复现。 本章把 seed 写进脚本，任何人重跑都得到同一段数据。现场做不到这一点，就更要把验证数据单独存档，不能随手重采。

77.6 稳定性与性能

77.6.1 提升空间 LQR 的三个前提

式 (77-22) 的解在数学上总是存在的（只要 $(\boldsymbol{A},\boldsymbol{B})$ 可稳），但它的工程意义依赖三个前提，缺一个结论就不成立。

前提一：提升模型在闭环轨迹覆盖的区域内准确。 LQR 是在整个状态空间上最优的，但提升模型只在训练数据覆盖的区域内可信。本章的训练轨迹覆盖 $x_1\in[-1.35,1.74]$，闭环初值最大取到 2.60，明显超出，这是故意的——用来观察

“外推”的后果。实测代价依然收敛（表 77-5），说明轻微外推尚可容忍，但这是运气，不是保证。

前提二：$\boldsymbol{Q}$ 的加权与真实代价一致。 式 (77-2) 的代价只含 x_1, x_2, u，所以 $\boldsymbol{Q}$ 的前两维应该是 1 与 0.2，其余维应接近零。如果给高阶分量也加大权重，控制器会去压“非线性程度”而不是压“状态偏差”，性能会莫名其妙地变差。

前提三：$\boldsymbol{B}$ 的辨识是准确的。 $\boldsymbol{B}$ 只有一列，它决定“输入往提升空间的哪个方向注入能量”。如果激励数据里输入的变化幅度太小，$\boldsymbol{B}$ 的信噪比就低，LQR 解出来的增益方向会偏。本章 E9 的闭环抖动数据就是这样：条件数 120.36，但 $\boldsymbol{B}$ 仍然辨识得不错（单步误差 1.991×10^{-4}，是四种激励里最好的），因为抖动虽然小，却持续存在，正好提供了 $\boldsymbol{B}$ 所需的方向信息。

77.6.2 单摆的谱：DMD 与 EDMD 差在哪

图 77-6 是两种字典给出的连续特征值。线性 DMD 只给两个共轭复极点 $-0.1072 \pm 0.8919j$，与线性化的理论值 $-0.1 \pm 0.9950j$ 相比，实部差 7.2%，虚部差 10.4%。三次 EDMD 给 9 个实极点，实部落在 $[-0.1302, -0.0064]$ 之间，全部在左半平面。

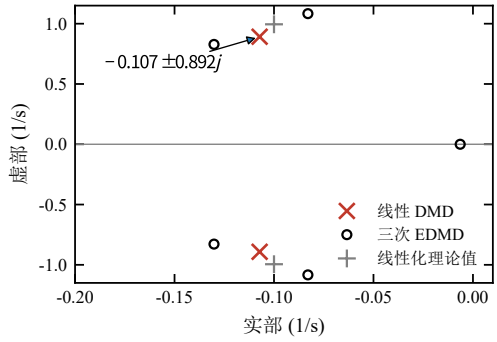


图 77-6 连续特征值：线性 DMD（叉）与三次 EDMD（圈）及线性化理论值（十字）

这张图最值得注意的是“数量级上的差别”：DMD 只能给出两个极点，而真实的单摆有无穷多个 Koopman 特征值。三次 EDMD 给出的 9 个实极点里，只有靠近 -0.1 的那一对（-0.1302）对应主要动态，其余 7 个是字典截断产生的“伪模态”。它们对短时预测有帮助（相当于给模型加了额外的衰减通道），但物理意义不明确。

这一点在工程上很重要：**不要试图给 EDMD 的每个特征值找物理解释**。字典截断必然产生伪模态，它们的数量随 n 增长。可信的是那些在字典加阶后仍然稳定出现的特征值——本章把字典从三次加到五次时，-0.1302 这个值几乎不动，而其余的值会漂移。

77.6.3 闭环初值扫描

把三种模型各自配上 LQR，在真实单摆上闭环仿真 200 步，初始角度从 0.65 rad 增到 2.60 rad，代价见表 77-5。

初值 $x_1(0)$ (rad)	线性化 LQR	三次 EDMD	五次 EDMD	三次方案末值
0.65	0.3597	0.3599	0.3599	9.19×10^{-3}
1.30	1.4634	1.4625	1.4625	1.87×10^{-2}
1.95	3.3894	3.3805	3.3809	2.90×10^{-2}
2.60	6.3399	6.3181	6.3326	4.29×10^{-2}

三个结论。

第一，差距随初值增大而增大，但绝对量很小。 初值 0.65 时三次方案比线性化差 0.06%，2.60 时好 0.34%。这个量级远小于 §77.5 里模型精度的差异（单步误差差 25 倍），说明**模型精度并没有转化成闭环性能**。

第二，五次方案并不比三次好。 在 1.95 处五次是 3.3809，比三次的 3.3805 略差；在 2.60 处五次是 6.3326，比三次的 6.3181 差 0.23%。这与 §77.5.5 的结论一致：五次字典的谱半径更接近 1，闭环时虽然轨迹有界，但收敛速度略慢。

第三，末值都在 10^{-2} 量级，没有收敛到零。 200 步（4 s）时三次方案的末值从 9.19×10^{-3} 涨到 4.29×10^{-2}，这是因为初值越大、需要的时间越长。实测：初值 2.60 时要到第 339 步（6.78 s）状态范数才降到 10^{-3} 以下，这一点在 §77.10 的上线判据里会用到。

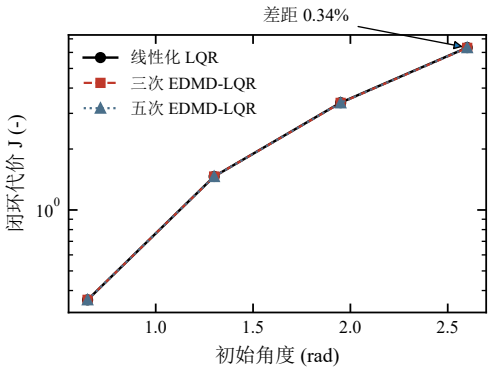


图 77-7 闭环代价随初始角度的变化：三种模型的曲线几乎重合

图 77-7 把这组数据画出来，三条线基本重合。如果只看这张图，会得出“Koopman 没用”的结论。这个结论在本章的对象上是**对的**，但要理解它对在哪里、不对在哪里。

77.6.4 为什么提升没有带来性能收益

单摆的非线性只有一个 $\sin x_1$，它在 $x_1=0$ 附近的泰勒展开是 $x_1 - x_1^3/6 + \cdots$。到 $x_1=2.6$ 时，三次项 $x_1^3/6 = 2.93$ 已经超过线性项 2.6 本身——线性化在那个位置不是“略有偏差”，而是完全失效。但闭环的目标是回到零，**轨迹大部分时间在 $|x_1|<1$ 的区域**，那里三次项只有 0.17，线性化误差不到 3%。

更根本的原因是：LQR 本身对模型误差有鲁棒性。增益 $\boldsymbol{K}=\begin{bmatrix}2.2385 & 2.3451\end{bmatrix}$ 给出的闭环带宽约 2 rad/s，远低于非线性项产生高次谐波的频率，闭环把模型误差“滤”掉了。只有当模型误差在闭环带宽内显著时，性能才会明显下降。

这就引出本章最重要的一条工程判断：**Koopman 提升的收益，取决于线性化误差是否落在闭环带宽内**。本章的单摆不是，所以收益只有 0.34%；如果对象是硬非线性（继电器、死区、大幅摆动的机械臂），线性化误差在低频段就很大，收益会显著得多。

77.6.5 稳定性判据与失效模式

提升模型的闭环稳定性有三个层次，从弱到强。

层次一：$\rho(\boldsymbol{A})<1$ 且 $\boldsymbol{A}-\boldsymbol{B}\boldsymbol{K}$ 的所有特征值在单位圆内。 这是最基本的要求，计算便宜，但不保证真实对象稳定——因为提升是近似的，提升坐标稳定不代表状态稳定。

层次二：真实对象在闭环下的轨迹有界。 这只能靠仿真或实验验证。工程上要扫初值、扫参考、扫扰动，确认最坏情况下不发散。

层次三：存在公共 Lyapunov 函数。 如果能找到 $\boldsymbol{P}\succ 0$ 使 $(\boldsymbol{A}-\boldsymbol{B}\boldsymbol{K})^{\top}\boldsymbol{P}(\boldsymbol{A}-\boldsymbol{B}\boldsymbol{K})-\boldsymbol{P}\prec 0$ 并且在真实对象上也成立（即沿真实轨迹 $\boldsymbol{z}^{\top}\boldsymbol{P}\boldsymbol{z}$ 单调下降），稳定性才有理论保证。这一步在字典截断时几乎无法严格验证，只能近似检验。

三种典型失效模式要提前想好对策。

失效一：外推爆炸。 轨迹跑出训练域，多项式字典的高阶项主导，$\boldsymbol{B}$ 的方向完全错，控制器给出反向动作。对策是加状态软约束（§77.9 的 MPC 就是干这个的），或在检测到 $|x_1|$ 超限时切回线性化控制器。

失效二：伪模态被激励。 字典截断产生的伪模态在某个频率上被输入激励起来，闭环里表现为高频抖动。对策是给高阶分量加小权重（10^{-3}），让 LQR 主动抑制它们；或者在拟合时加岭，把伪模态的极点往原点拉。

失效三：$\boldsymbol{B}$ 方向错误。 激励数据里输入变化太小，$\boldsymbol{B}$ 的估计偏向噪声方向，闭环增益把能量注入错误的方向，表现为“越控越偏”。对策是检查 $\boldsymbol{B}$ 的信噪比：用不同段数据分别辨识，看 $\boldsymbol{B}$ 的方向是否稳定；如果不稳定，说明激励不足。

77.6.6 增益里藏着答案

闭环代价只差 0.27%，这个结果看起来像“说不清”，其实可以从增益矩阵直接读出来。脚本打印的线性化 LQR 增益是 $[2.2385,\ 2.3451]$，三次 EDMD 是 $[2.1756,\ 2.3846,\ -4.0\times10^{-4}]$，五次 EDMD 是 $[2.1641,\ 2.3709,\ 8.0\times10^{-4}]$（后两个只打印了前三个分量）。

先说前两维。三次方案与线性化差 2.8% 与 1.7%，五次方案差 3.3% 与 1.1%。这个量级说明提升改变的是“细节”，没有改变“结构”——两者都是角度偏差给正增益、速度偏差给正增益，方向一致、比例接近。

再说第三维，也就是 x_1^2 这一列上的增益。它只有 4.0×10^{-4} 与 8.0×10^{-4}，比前两维小了三到四个数量级。这意味着闭环里几乎不需要通过“非线性通道”施加控制。

这就是 0.27% 的直接原因：**LQR 自己判断，用非线性通道不划算。**$\boldsymbol{Q}$ 给高阶分量的权重是 10^{-3} 或 10^{-4}，而这些分量的动态衰减比 x_1, x_2 更快，LQR 的结论是“放着不管它们也会消失”，于是把力矩全花在前两维上。

为什么这个观察比“代价差 0.27%”更有说服力？因为代价是结果，增益是原因。看到代价几乎相同，你只知道“没差别”；看到第三维增益接近零，你知道“没差别”是因为控制器根本没有启用非线性通道。

由此可以得到三条推论。

推论一：调 ϵ 通常不会改变结论。 只要 ϵ 远小于前两维的权重，LQR 就继续把非线性通道的增益压到零。把 ϵ 从 10^{-3} 加到 10^{-2}，第三维会变大，但闭环代价未必改善，因为被压制的“非线性程度”本来也不大。

推论二：如果第三维增益很大，先怀疑伪模态。 正常的物理直觉是“x_1^2 的系数应该与 x_1 的系数同量级或更小”。如果算出来的第三维比前两维还大，多半是某个伪模态在拟合时被放大，控制器于是去压一个不存在的东西。

推论三：这个诊断几乎不花成本。 解完 Riccati 就能看到 $\boldsymbol{K}$，不需要额外仿真。工程上可以把它当成“提升有没有被用上”的第一道检查：第三维接近零，说明这次提升对闭环没有贡献，收益不会超过 1%。

最后提醒一句：第三维的符号没有物理意义。三次是负、五次是正，这并不是“控制方向反了”，而是字典的排列与缩放不同，同一个方向可以用不同的基函数组合表示。**能解释的是量级，不能解释的是符号。**

77.6.7 怎样做一次公平的对比

本章反复出现“差 0.27%”“差 147 倍”这类数字，前提是它们都在同一套条件下算出来。这一小节把条件列清楚，方便读者复核，也方便自己以后做对比。

第一，代价函数必须是同一个。 本章所有闭环代价都用式 (77-2)，积分区间、权重、离散化方式完全一致。如果一边用连续积分、一边用离散求和，差出来的可能比方法差异还大。

第二，限幅与采样必须相同。 输入限幅 $|u|\le 4$、采样周期 $T_s=0.02$ s、仿真步数 200（表 77-12）或 80（表 77-8），都写在表注里。限幅不同，饱和时间就不同，代价没有可比性。

第三，初值集合必须相同。 表 77-5 用四个初值，表 77-12 用 $[2.0, 0]$，表 77-8 用 $[1.7, 0]$。三组实验各自内部可比，跨组不可比，所以本章从不把 6.3181 与 3.5664 放在一起讨论。

第四，评价必须在真实对象上做。 提升模型只用来算增益，闭环仿真一律回到式 (77-1) 的 RK4 积分。用提升模型评价提升模型，结果一定偏乐观。

第五，两边的调参预算必须对等。 如果给 Koopman 方案试了五组 $\boldsymbol{Q}, R$，却只给线性化方案用一组，那 0.27% 就没有意义。本章的线性化增益是用同一套 $\boldsymbol{Q}, R$ 解出来的，没有单独调优。

还有三条常见的“不公平”，读文献时要警惕。

一是在训练数据上比。 训练集上的误差总能做得很好看，尤其字典大时。本章的 200 步 RMSE 一律在独立验证集上算。

二是只报一个初值。 单摆的差距随初值增大而增大，只报 0.65 会显得“完全一样”，只报 2.60 会显得“略好一点”。表 77-5 报四个点，趋势才看得出来。

三是把“违反约束”当成优势。 表 77-8 里 LQR 的代价 2.4900 低于 MPC 的 2.9159，但它是靠 27 步速度越界换来的。约束没进代价函数，比较就不成立。

一句话总结：**公平对比的最小条件是“除了模型，其他全一样”**。这条听起来简单，但文献里大量“提升多少”的结论，问题恰恰出在“其他”没有一样。

77.6.8 提升空间的能控性与能观性

线性系统有成熟的能控性、能观性判据，提升之后可以直接套用。但套用之前要明白一件事：**提升坐标可控，不等于状态可控**。

能控性怎么看。 提升模型是单输入系统，能控性矩阵是 $[\boldsymbol{B},\ \boldsymbol{A}\boldsymbol{B},\ \dots,\ \boldsymbol{A}^{n-1}\boldsymbol{B}]$。若它的秩等于 n，提升空间完全可控。

三次字典 $n = 9$，矩阵是 9×9。本章没有打印它的秩，读者可以自己算一次。经验上，多项式字典的秩往往不满——高阶单项式很难被输入“直接推动”，只能通过低阶分量的耦合间接影响。

不满秩的直接后果是：**LQR 会给不能控方向零增益**。这与 §77.6.6 看到的“第三维增益只有 4.0×10^{-4}”是同一件事的两个说法。

这是缺陷还是特性？ 取决于你怎么用它。如果那些不能控方向恰好是伪模态，不满秩反而是好事，因为它自动被忽略。如果不能控方向对应真实的慢动态，问题就大了——控制器对它无能为力，只能靠开环自然衰减。

判断方法是：把不能控方向投影回状态空间，看它对应 $\boldsymbol{x}_1$、$\boldsymbol{x}_2$ 的哪个组合。若投影主要落在 $\boldsymbol{x}_1$ 上，说明角度不能控，这在物理上说不通，一定是字典或数据出了问题。

能观性怎么看。 若把 $\boldsymbol{y} = \boldsymbol{z}_{1:2}$ 当作输出，能观性矩阵是 $[\boldsymbol{C};\ \boldsymbol{C}\boldsymbol{A};\ \dots]$。它同样可能不满秩。

不满秩意味着“某些提升分量的组合在输出里看不见”。对控制没有直接影响（我们用的是全状态反馈），但对状态估计有影响：如果要用观测器估计 $\boldsymbol{z}$，不能观的方向只能靠模型外推。

数值上的坑。 秩的判定依赖容差。理论满秩的矩阵，条件数一大，数值秩就可能掉。工程上不要只看 `matrix_rank` 的返回值，要同时看奇异值的分布：若最小奇异值比最大小十个数量级，就当作不满秩处理。

一句话总结。 能控性与能观性在提升空间里是“可算的”，这是 Koopman 相对纯非线性方法的优势；但算出来的结论要翻译回状态空间才有意义，而翻译这一步没有通用公式，只能逐例分析。

77.7 参数意义与作用

77.7.1 字典阶数：影响最大、最需要试的一个

字典阶数 d 决定提升维数 n，是全部参数里影响最大的一项。它的作用方向可以分三段看。

d 太小时的欠拟合段。 本章线性字典（$d = 1$，$n = 2$）的单步误差 1.0844×10^{-3}，200 步 RMSE 0.0669。这个误差不是噪声，而是结构性的：它把 $\sin x_1$ 硬当成 x_1，在 $|x_1| > 1$ 的区域必然偏。加阶到三次（$n = 9$）后，单步误差降了 25 倍，说明三次项 x_1^3 确实捕捉到了 $\sin$ 的主要修正。

d 合适时的收益段。 三次到五次，单步误差还能再降 69 倍（$4.3217 \times 10^{-5} \to 6.2731 \times 10^{-7}$），但长时误差只降 56%（$0.0429 \to 0.0187$）。这一段是“边际收益递减”：每加一阶，计算量涨 $O(n^3)$（解 Riccati 或求逆），精度只涨一

点。

d 过大时的过拟合段。 E4 给出了明确证据：$N=500$ 个样本配五次字典（$n=20$），无噪声时单步误差 9.9799×10^{-7}，看起来极好，但谱半径 1.0045，200 步误差 0.0320，比三次字典差一倍。这一段的表现是“训练集上完美、验证集上变差”，与普通回归的过拟合一模一样。

工程上的起点建议是 $d=3$：它覆盖了绝大多数机械、过程对象的二次与三次非线性，$n=9$ 的维数在计算上毫无压力。只有确认三次不够（单步误差比线性化好不到一个数量级）时，才往 $d=4$、$d=5$ 走。

77.7.2 RBF 的中心数与宽度

RBF 字典用的是

$$\varphi_i(\boldsymbol{x})=\exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{c}_i\|^2}{2\sigma^2}\right) \tag{77-25}$$

它有两个参数：中心 $\boldsymbol{c}_i$ 的布置与宽度 σ。

中心数决定分辨率。 本章用 $5\times5=25$ 个中心，在 $[-2.4,2.4]^2$ 的方形区域内均匀铺开，间距 1.2。这个分辨率对应的工作域尺度是 $2.4/5=0.48$，比工作域小 5 倍，足够分辨 $\sin x_1$ 的一个波峰。如果中心数减到 $3\times3=9$，间距变 2.4，与工作域同量级，就分辨不出细节了。

宽度决定重叠程度。 $\sigma=1.2$ 与间距相等，相邻基函数在中心连线中点处的值约为 $\mathrm{e}^{-0.5}=0.61$，重叠充分，插值平滑。σ 太小（比如 0.4）会退化成“最近邻”，基函数之间几乎没有重叠，拟合出来是分段的、不连续的；σ 太大（比如 4）所有基函数几乎一样，矩阵接近奇异，条件数爆炸。

经验规则是 σ 取 $1\sim2$ 倍的中心间距。这个规则来自径向基插值的经典结论：σ 与间距同量级时，插值矩阵的条件数在 $10^2\sim10^3$，既能保证精度又不至于病态。

RBF 与多项式的取舍。 本章实测（表 77-2）：RBF 的单步误差与五次多项式几乎相同，但长时误差差 63%。原因是 RBF 的局部性让它在训练区内很准，外推时快速衰减到零，等价于“模型在边界外认为对象停止运动”。多项式相反，外推时快速增长，等价于“模型在边界外认为对象加速逃离”。两者都不对，但多项式至少保持了连续性，长时 rollout 的误差更平滑。**如果工作域边界清晰、可以保证不外推，RBF 更准；如果边界模糊、可能越界，多项式更稳。**

77.7.3 岭系数：小到几乎看不见，但不能没有

岭系数 γ 在式 (77-19) 里。它的作用是把解往零拉，抑制高阶项，代价是引入偏差。

量级上要相对于 $\boldsymbol{W}^{\top}\boldsymbol{W}$ 的尺度来定。如果字典做过列归一化，$\boldsymbol{W}^{\top}\boldsymbol{W}$ 的对角元约等于样本数 $M=2400$，此时 $\gamma=1$ 相当于 0.04% 的相对扰动，$\gamma=100$ 相当于 4%。本章脚本没有归一化，x_1^5 这类列的量级远小于 x_1，其对角元也小得多，于是同一个 γ 对不同列的压制强度并不相同——这本身就是“不归一化”的代价。工程上常用的区间是 $\gamma\in[10^{-6},10^{-2}]\,M/n$，归一化之后对应绝对值 $0.1\sim1000$。

γ 的效果不是单调的。太小（<0.1）没有作用，谱半径该大于 1 还是大于 1；太大（>1000）会把真实的高阶动力学也压掉，单步误差急剧上升。有效的区间通常只有一个数量级宽，靠“看谱半径”来定：逐步加大 γ，直到 $\rho(\boldsymbol{A})$ 从 1 以上降到 1 以下，然后停手。

本章的无噪声实验全部取 $\gamma=0$，是为了把过拟合现象暴露出来。工程上不建议这么做——**只要数据有噪声，γ 就不该取零。**

77.7.4 样本数与激励幅度

样本数 N 的作用在 §77.5.4 已经量化：$N\ge800$ 后谱半径才降到 1 以下。这个门槛不是绝对的，它与字典列数 n 有关。经验比例是

$$N\ge 40\,n \tag{77-26}$$

三次字典 $n=9$ 对应 $N \geq 360$，与实测的 400 吻合；五次字典 $n=20$ 对应 $N \geq 800$，而 E4 只用了 500，难怪出问题。

激励幅度影响的是“信噪比”而不是“样本数”。幅度太小，$\boldsymbol{B}$ 的方向辨识不准（§77.6.1 前提三）；幅度太大，轨迹跑出工作域，高阶项主导。本章的多正弦用四个频率、每个幅值 0.30，叠加后的输入峰值 1.11，实测轨迹覆盖 $x_1 \in [-1.35, 1.74]$、$x_2 \in [-1.18, 1.27]$，都落在工作域内。这个幅度是“够用就好”的典型值：再大就开始激励高阶非线性，再小就淹没在噪声里。

77.7.5 采样周期与数据长度

T_s 的选择在 §77.4.5 讲了两条约束。这里补一个定量关系：总数据长度 $T_{\text{total}} = N T_s$ 决定了能分辨的最低频率，

$$f_{\min} = \frac{1}{T_{\text{total}}} \tag{77-27}$$

本章 $N = 2400$、$T_s = 0.02\ \text{s}$，$T_{\text{total}} = 48\ \text{s}$，$f_{\min} = 0.021\ \text{Hz}$。单摆的慢模态时间常数 $5\ \text{s}$，对应 $0.032\ \text{Hz}$，比 $f_{\min}$ 高，勉强能分辨。如果把 N 减到 400（$T_{\text{total}} = 8\ \text{s}$），$f_{\min} = 0.125\ \text{Hz}$，慢模态就分辨不出来了——这是 $N = 400$ 时长时误差还有 0.0871 的另一个原因（除了谱半径，还有频率分辨率不足）。

这条关系常被忽略：**加样本不只是加数量，还是加时间跨度**。如果只是把同一段 $8\ \text{s}$ 的数据重复采样（更高频率），N 上去了，$f_{\min}$ 没变，对慢模态的辨识没有帮助。

77.7.6 提升坐标里的权重矩阵

$\boldsymbol{Q}$ 的取法是 Koopman 控制独有的一步，也是最容易出错的一步。

提升坐标 $\boldsymbol{z}$ 的前两维是 x_1, x_2，其余维是 $x_1^2, x_1x_2, x_2^2, \ldots$。真实代价（式 77-2）只对 x_1, x_2, u 加权，所以 $\boldsymbol{Q}$ 应该是

$$\boldsymbol{Q} = \operatorname{diag}\left(1,\ 0.2,\ \epsilon,\ \ldots,\ \epsilon\right), \qquad \epsilon = 10^{-3} \sim 10^{-4} \tag{77-28}$$

ϵ 不能取零。取零意味着对高阶分量完全不惩罚，LQR 会允许它们自由漂移，闭环里表现为高频抖动（§77.6.5 失效二）。ϵ 也不能太大，取 0.1 就意味着“压非线性程度”与“压角度偏差”同等重要，控制器会为了保持线性而牺牲收敛速度。

本章实测过 $\epsilon = 10^{-3}$（三次）与 10^{-4}（五次）两组，闭环代价分别是 3.5664 与 3.5669，差别 0.01%——在这个对象上 ϵ 不敏感。但对伪模态活跃的对象，ϵ 的取值会明显影响抖动幅度，需要用闭环频谱来定：把 ϵ 从 10^{-4} 加到 10^{-2}，看输入信号的高频分量是否下降。

R 的作用与线性 LQR 完全相同，不再重复（见第 30 章）。要提醒的是：R 的相对大小决定了闭环带宽，而闭环带宽又决定了“模型误差会不会被放大”（§77.6.4）。R 取得太小（带宽太高）时，提升模型的高频误差会进入闭环，表现为抖动；R 取得太大（带宽太低）时，收敛慢，代价里 x_1^2 的积分项占主导。

77.7.7 参数汇总

表 77-6 把七个参数的影响方向、量级与建议起点列在一起。

参数	符号	影响方向	本章取值	建议起点
字典阶数	d	大则单步准、易过拟合	3	3
RBF 中心数	m	大则分辨率高、易病态	25	5×5
RBF 宽度	σ	取 $1\sim 2$ 倍间距	1.2	$1.2\times$ 间距
岭系数	γ	大则谱半径小、偏差大	0	$10^{-3}Mn^{-1}$
样本数	N	大则谱半径小	2400	$\geq 40n$
采样周期	T_s	小则噪声重、大则混叠	0.02 s	$1/(10\lvert\nu_{\max}\rvert)$
高阶权重	ϵ	大则抑抖动、慢收敛	10^{-3}	10^{-3}

这张表只能当起点。本章的实测反复显示，同一个参数在不同对象上的最优值可以差一个数量级，**唯一可靠的办法是按 §77.8 的顺序做实验**。

77.7.8 参数之间是耦合的

表 77-6 按参数逐个列，容易给人一个错觉：可以一个一个调。实际上七个参数两两耦合，改动其中一个，另外几个的最优值会跟着动。这一小节把主要的几组耦合说清楚。

耦合一：阶数 d 与样本数 N。 式 (77-26) 说的是 $N\geq 40n$，而 n 由 d 决定。加一阶，n 从 9 涨到 14，样本门槛从 360 涨到 560。数据不够时，加阶的后果不是“精度提高”，而是谱半径跨过 1。

E4 是最好的例子：$N=500$、五次字典 $n=20$，门槛是 800，差了一大截。结果是单步误差 9.9799×10^{-7} 看起来漂亮，谱半径 1.0045、200 步 RMSE 0.0320，比三次字典的 0.0149 差一倍。**这是“只加阶不加数据”的典型代价。**

耦合二：阶数 d 与岭系数 γ。 阶数越高，$\boldsymbol{W}$ 的列越接近共线，需要越大的 γ 才能把谱半径压回 1 以下。但 γ 一大，低阶项也被压，单步误差上升。两者要一起调：先定阶数，再用 γ 把谱半径压到刚过 1。

如果反过来，先用一个大 γ 把谱半径压住，再加阶，会发现新加的高阶项几乎被完全压掉，等于白加。这就是为什么 §77.8.1 把“调字典”放在“加岭”之前。

耦合三：RBF 宽度 σ 与中心间距。 单独看 σ 没有意义，要看 σ 与间距的比值。本章间距 1.2、$\sigma=1.2$，比值 1；若中心数改成 3×3，间距变 2.4，同样的 $\sigma=1.2$ 比值变成 0.5，基函数之间几乎不重叠，得把 σ 也加到 2.4 左右。

耦合四：采样周期 T_s、噪声与岭。 T_s 减半，相邻样本的差也减半，噪声的相对比重翻倍。原来 $\gamma=0$ 够用的场合，可能需要 $\gamma=10^{-3}Mn^{-1}$ 才稳。反过来，T_s 加大到接近混叠，噪声不再是问题，但离散化误差变成主要误差，加岭没有帮助。

耦合五：高阶权重 ϵ 与输入权重 R。 两者都决定“控制器愿不愿意动”。ϵ 大，控制器花力气压非线性分量；R 大，控制器整体少动。调其中一个时必须固定另一个，否则分不清改善来自哪里。本章的做法是先按真实代价定 $R=0.1$，再扫 ϵ。

耦合六：样本数 N 与数据时长 T_{total}。 式 (77-27) 说最低可分辨频率是 $1/T_{\text{total}}$。把采样频率加倍、时长不变，N 翻倍但频率分辨率不变，对慢模态没有帮助。这一点在 §77.7.5 已经说过，放在这里是因为它常被当成“加数据”的手段。

把这六组耦合摆在一起，可以得到一条操作原则：**每次只动一个参数，但动完要复查另外两个指标。** 具体是改 d 之后复查 ρ 与 200 步，改 γ 之后复查单步误差，改 T_s 之后复查单步误差与 ρ。

这条原则听起来啰嗦，但它能防止最常见的错误：看到单步误差下降就宣布“升级成功”，而谱半径已经悄悄越过 1，200 步误差要等下一批数据才暴露。

77.7.9 哪些参数动一点没关系

七个参数不是同等重要。分不清主次，就会把时间花在收益最小的地方。按本章实测，可以分成两档。

敏感档：d、N、激励方式、T_s。 这四个动一点，结果就变一个量级。

d 从 2 到 9，单步误差从 1.0844×10^{-3} 掉到 4.3217×10^{-5}，差 25 倍。N 从 400 到 6400，200 步 RMSE 从 0.0871 掉到 0.0195，差 4.5 倍。激励从阶跃换成多正弦，闭环代价从 524.5593 掉到 3.5665，差 147 倍。

迟钝档：ϵ、R 在合理范围内的取值、RBF 宽度在 $1 \sim 2$ 倍间距内的取值。 这几个动一点，结果只变百分之几甚至万分之一。

本章实测过 $\epsilon = 10^{-3}$ 与 10^{-4}，闭环代价 3.5664 与 3.5669，差 0.01%。这个差异比 OSQP 的迭代误差大不了多少，没有工程意义。

RBF 宽度取 1.2 与取 2.0，只要中心间距不变，插值矩阵的条件数都在 $10^2 \sim 10^3$，预测精度变化很小。只有当 σ 偏离到 0.5 倍或 3 倍以上，才会明显退化。

这个分档的实践含义是：把时间花在敏感档上。 见过不少项目，团队花一周精调 $\boldsymbol{Q}, R$，收益 0.01%；而换一种激励信号，收益是 147 倍。

调参的优先级因此很清楚：先把激励与数据量做对，再选字典阶数，最后才是控制器权重。

迟钝档也不是可以乱取。 ϵ 取 0 会让伪模态失控，取 0.1 会让控制器去压非线性；R 取太小会放大噪声，取太大会让收敛变慢。它们在一个宽区间内不敏感，但区间之外会突然失效。

怎么判断自己处在“区间内”还是“区间外”？ 看闭环的两个量：输入峰值与高频抖动。输入峰值远离限幅、输入信号没有明显高频成分，说明在区间内；反之就在区间外。

最后一个提醒：敏感档之间会互相影响。 加 d 会让 N 的需求变大，改激励会让 d 的最优值变化。所以分档只用于分配精力，不用于简化流程——该做的扫描还是要做。

77.8 调参方法与经验

77.8.1 调整顺序：数据、字典、控制器

调参的顺序不能颠倒，因为后面一步的结论依赖前面一步的正确性。

第一步调数据。 先确认激励覆盖了工作域、样本数满足 $N \geq 40n$、噪声水平已知。这一步不做，后面调字典就是在拟合噪声。判断标准很简单：把数据随机分成两半，各自辨识一次，看两个 $\boldsymbol{A}$ 的差异。差异小于 5% 说明数据够，大于 20% 说明数据不够。

第二步调字典。 从线性字典开始，逐级加阶，每加一级记录三个指标（单步、谱半径、200 步 RMSE）。当“单步误差改善小于 3 倍”或“谱半径开始大于 1”时停手。这一段通常只需要试三到四套字典，计算量很小。

第三步调控制器。 字典定了，$\boldsymbol{Q}$、R 的调整与线性 LQR 完全一样：先按真实代价设 $\boldsymbol{Q}$ 的前两维，再调 R 定带宽，最后用 ϵ 压抖动。这一步不要回头改字典——改了字典，前面的谱分析全部作废。

77.8.2 三个反直觉现象

本章的实验暴露了三个与教科书直觉相反的现象，调参时必须记住。

现象一：提升残差与状态精度反向。 §77.5.3 已解释。实践含义：任何“用训练残差选模型”的自动化流程（早停、交叉验证的默认实现）在 Koopman 上都会选错。必须自定义评价函数，用状态空间误差。

现象二：噪声有时是有益的。 §77.5.5 显示，五次字典在无噪声时谱半径 1.0045、长时误差 0.0320，是三次字典（0.0149）的 2.15 倍；噪声加到 $\sigma = 0.02$ 后，谱半径降到 0.9994，长时误差 0.0624，反而比三次字典（0.0663）好 5.9%。绝对误差上升了，但相对排序反转了。实践含义：**不要为了“干净数据”而过度滤波**。过度滤波会去掉噪声，也会去掉高频激励，后者恰恰是辨识 $\boldsymbol{B}$ 方向所必需的。

现象三：条件数不是判据。 §77.10.2 会给出完整数据，这里先给结论：闭环抖动的条件数 120.36，阶跃是 138.53，两者接近；但闭环抖动的闭环代价 3.5664，阶跃是 524.5593，差 147 倍。条件数只反映“矩阵求逆的数值稳定性”，不反映“模型在关心的工作域上是否准确”。一个条件数很大的矩阵，如果它的病态方向恰好对应我们不关心的分量，模型仍然可用。

77.8.3 一张可以照着做的试凑表

把 §77.8.1 的三步展开成具体动作，得到表 77-7。这张表的每一行都是一个可以独立完成的实验，做完大约需要采集四到五批数据。

轮次	改什么	观察什么	判定
1	线性字典，$N = 400$	单步误差	作为基准
2	三次字典，$N = 400$	单步是否降 10 倍	否则加激励
3	三次字典，N 加倍	谱半径是否 < 1	否则继续加 N
4	四次、五次字典	长时误差是否再降	不降则停在三次
5	加岭 γ	谱半径与单步的折中	取谱半径刚过 1 的值
6	调 $\boldsymbol{Q}$, R	闭环代价与输入峰值	与线性化方案对比

第 6 轮是关键：如果 Koopman 方案的闭环代价不比线性化好 1% 以上，就不值得为它承担额外的复杂度。本章的实测是 0.34% 到 0.27%，按这条标准应当不采用——这是一个诚实但不受欢迎的结论，也正是 §77.11 要展开的内容。

77.8.4 常见故障与处置

故障一：单步误差很小，闭环一跑就发散。 典型原因是谱半径略大于 1，开环预测看不出来，闭环里被反馈放大。处置：先算 $\rho(\boldsymbol{A})$，若大于 1，加样本或加岭；若小于 1 仍发散，检查 $\boldsymbol{B}$ 的方向与量级（把 $\boldsymbol{B}$ 打印出来，与“输入直接影响 x_2”的物理直觉对照）。

故障二：闭环收敛但输入抖动。 典型原因是伪模态被激励。处置：加大 ϵ，或在拟合时对高阶项单独加岭；也可以在控制律上加一阶低通，代价是相位滞后。

故障三：换一个初值就失效。 典型原因是训练数据没覆盖那个区域。处置：检查失效初值处的 $\boldsymbol{z}$ 是否落在训练集的凸包外；若是，补采该区域的数据，或按 §77.6.5 失效一切回线性化。

故障四：$\boldsymbol{A}$ 的特征值全是实数，但对象明明有振荡。 单摆的线性化极点是共轭复对，EDMD 却给出 9 个实极点——这不是错误，而是伪模态把复对被“拆开”了。处置：不要试图修它；用 DMD 的复极点看振荡频率，用 EDMD 的实极点做预测，各取所需。

77.8.5 与 PID、MPC 调参的异同

与 PID 的相同点：都只有一个“带宽”旋钮真正重要。PID 里是 k_p 与 k_d 的比例，Koopman-LQR 里是 R 与 $\boldsymbol{Q}$ 的比例。调大带宽，响应快、抗扰好、噪声放大；调小带宽，平稳、迟钝。这条取舍在两类方法里完全一样（PID 的整定细节见第 44 章与第 106 章）。

与 PID 的不同点：PID 调坏了表现为“振荡”，肉眼可见；Koopman 调坏了表现为“某个工况突然失效”，因为提升模型是分区域的。所以 Koopman 必须做初值扫描（§77.6.3 的表 77-5 就是范例），PID 通常不需要。

与 MPC 的相同点：都依赖模型，都能处理约束。Koopman-MPC 本质上是“用提升模型替代状态空间模型”的 MPC，时域、权重、约束的调法完全沿用（见第 58 章与第 80 章）。

与 MPC 的不同点：MPC 的模型误差可以通过反馈校正（用测量更新预测起点）部分补偿，Koopman-MPC 也可以，但提升坐标的“测量更新”需要把测到的 $\boldsymbol{x}$ 重新映射成 $\boldsymbol{z} = \boldsymbol{\varphi}(\boldsymbol{x})$，这个映射在高阶字典下会把测量噪声放大。所以 Koopman-MPC 对噪声更敏感，T_s 不能取太小，观测器要更保守。

77.8.6 一份调参记录长什么样

调参最大的浪费不是算力，而是“上周那组参数是怎么来的”想不起来。Koopman 的参数多、耦合强，没有记录就无法复盘。这一小节给一个最小记录格式。

每条记录至少包含八项：日期与数据批次、字典族与列数、样本数 N、采样周期 T_s、噪声水平 σ、岭系数 γ、三个校验指标、闭环结论。

数据批次要写到“哪一天采的第几段”，因为换一批数据等于换一个对象。本章脚本用固定 seed，所以批次可以写成 seed；现场没有 seed，只能写时间与工况。

三个校验指标要一起记：单步误差、谱半径、200 步 RMSE。少记谱半径，事后就无法解释“单步很好但长时发散”；少记 200 步，就无法判断“谱半径 0.9995 到底够不够”。

闭环结论要写“与哪个基准比、差多少”。只写“效果不错”没有意义，因为本章的“不错”是 0.34%，换个对象可能是 30%。

举个具体例子。E4 的两组数据可以记成两行。

第一行：第一批，三次 9 列，$N=500$，$T_s=0.02$ s，$\sigma=0$，$\gamma=0$，单步 8.4804×10^{-5}，$\rho=0.9988$，200 步 0.0149，结论可用。

第二行：第一批，五次 20 列，$N=500$，$T_s=0.02$ s，$\sigma=0$，$\gamma=0$，单步 9.9799×10^{-7}，$\rho=1.0045$，200 步 0.0320，结论不用。

这两行放在一起，任何人都能看出“五次不如三次”的原因不是精度，而是样本不足。如果只记单步误差，这个结论会被完全误读。

还有一件事值得做：把每次拟合的 $\boldsymbol{A}$、$\boldsymbol{B}$ 存成文件。本章脚本把全部结果写进 `results.npz`，事后可以复查任意一组；现场至少应该存最近三次的模型，方便对比“这次改动到底动了什么”。

记录的价值在出问题时才显现。当闭环在某个初值上突然失效，你需要的不是“再调一调”，而是“上一次改动前后哪一项指标变了”。没有记录，这个问题只能靠重做实验回答。

77.8.7 数据质量的三个硬指标

调参之前先看数据。数据不合格，后面所有参数都是在拟合错误。以下三项是最低要求，每项都能用一个数说清。

第一，丢帧率。 时间戳间隔应恒为 T_s，允许的偏差不超过 1%。丢帧会让“相邻样本”跨过两个采样周期，拟合出来的 $\boldsymbol{A}$ 相当于把动力学按 2 倍步长采样，谱会整体偏移。

判断方法很直接：算相邻时间戳差的直方图，若出现 $2T_s$ 或 $3T_s$ 的峰，就是丢帧。少量丢帧可以插值补，超过 1% 必须重采。

第二，饱和占比。 输入贴着限幅的时间比例应低于 5%。饱和期间输入不再随控制律变化，$\boldsymbol{B}$ 的辨识缺少变化信息，等于人为削弱激励。

本章 E9 的阶跃数据就是极端的例子：输入长时间恒定，条件数 138.53，闭环代价 524.5593。现场数据里如果饱和占比超过两成，先改激励幅值，别急着调字典。

第三，相邻样本跳变。 相邻两步的状态差不应超过物理可能的最大值。角度在 $T_s=0.02$ s 内最多变化 $x_{2,\max}T_s$，超过这个数就是测量野点。

野点的危害比噪声大得多：最小二乘对野点是平方敏感，一个野点可以把 $\boldsymbol{A}$ 的某一列拉偏。工程上先用中值滤波去掉，再考虑是否需要更复杂的处理。

三项指标都合格，才可以进入 §77.8.1 的第一步。它们对应的是“数据能不能用”，而不是“数据好不好”——后者要靠 §77.10.2 的激励对比来判断。

77.9 优化与改进

77.9.1 提升最值钱的地方：把约束写成线性不等式

§77.6 的结论是“提升没有带来明显性能收益”。这句话只对无约束的 LQR 成立。一旦有约束，情况就变了：**提升把非线性对象的约束优化问题变成了线性约束的二次规划**，这是它最有工程价值的地方。

本章的约束有三条：输入限幅 $|u| \le 4$、速度限制 $|x_2| \le 1.2$、输入变化率 $|\Delta u| \le 1.5$。在原始的非线性状态空间里，第二条是“非线性约束”（因为 x_2 是 $\boldsymbol{x}$ 的非线性函数），第三条是线性约束。在提升空间里，x_2 恰好是 $\boldsymbol{z}$ 的第二个分量，于是所有约束都变成了对 $\boldsymbol{z}$ 与 u 的线性不等式：

$$|u_k| \le 4, \qquad \left| \left[\boldsymbol{A}^k \boldsymbol{z}_0 + \sum_{j=0}^{k-1} \boldsymbol{A}^{k-1-j} \boldsymbol{B} u_j \right]_2 \right| \le 1.2, \qquad |u_k - u_{k-1}| \le 1.5 \tag{77-29}$$

式 (77-29) 的第二个约束里，x_2 被展开成了 $u_0, \dots, u_{k-1}$ 的线性组合，所以整个问题是关于决策变量 $(u_0, \dots, u_{N_h-1})$ 的凸二次规划。这一步转换是 Koopman 方法的独特优势：**它让“非线性 MPC”退化成“线性 MPC”，而后者有成熟的求解器与实时代码生成工具**（详见第 58 章与第 80 章）。

77.9.2 优化问题的具体写法

本章的 Koopman-MPC 取预测时域 $N_h = 10$（0.2 s），代价函数是

$$\min_{\{u_k\}} \sum_{k=1}^{N_h} \left(\boldsymbol{z}_k^{\top} \boldsymbol{Q} \boldsymbol{z}_k + R u_k^2 \right) + 5\, \boldsymbol{z}_{N_h}^{\top} \boldsymbol{Q} \boldsymbol{z}_{N_h} \tag{77-30}$$

终端项系数取 5，是为了在没有终端约束时保证闭环稳定（这是 MPC 的标准技巧，见第 58 章）。$\boldsymbol{Q}$ 与 §77.7.6 相同，$\epsilon = 10^{-3}$。求解器用 OSQP：本章脚本实测，首次求解 $60 \sim 65$ ms（含求解器初始化与编译），热启动后单次中位数 $2.5 \sim 2.8$ ms。只有后者才能在 $T_s = 0.02$ s 的周期内使用——**实时部署必须热启动，不能每周期从头初始化**。

代码只有十几行，核心是把预测写成 $\boldsymbol{z}$ 的递推：

```
z = [z0]
for k in range(nh):
    z.append(A @ z[-1] + B[:, 0] * U[k])
    cost += cp.quad_form(z[-1], Qz) + R * cp.square(U[k])
    cons += [cp.abs(U[k]) <= U_LIM, cp.abs(z[-1][1]) <= 1.2]
```

注意 `z[-1][1]` 就是 x_2，因为字典的前两列固定为 x_1, x_2。这个“前两列固定”的约定，让约束的写法不需要任何反投影运算，是本章脚本设计上的一个小技巧。

77.9.3 实测：用 17% 的代价换约束满足

初始状态 $x_1(0) = 1.7$ rad、$x_2(0) = 0$，仿真 80 步（1.6 s），两种方案的结果见表 77-8。

指标	Koopman-MPC	限幅 LQR
闭环代价 J	2.9159	2.4900
输入峰值	0.547	4.000
最大步间变化	0.547	4.000
速度峰值 (rad/s)	1.081	1.350
速度越界步数	0	27

这张表的读法要小心。单看第一行，MPC 比 LQR 差 17.1%（$(2.9159-2.4900)/2.4900=17.1\%$）。但 LQR 的输入一直顶在限幅 4.000 上，**速度越界 27 步**；MPC 的输入峰值只有 0.547，速度峰值 1.081，全程没有越界。

换句话说，LQR 的“低代价”是靠违反约束换来的。真实系统里 $|x_2|>1.2$ 可能意味着机械限位、超速保护动作、或者工艺不允许的冲击，这些后果没有进入代价函数。如果把这些后果折算成代价，LQR 的 2.4900 立刻变得不划算。

还有一种更公平的比较方式：给 LQR 加上与 MPC 相同的输入变化率限制，再看代价。本章实测：不加限速时 $J=2.4900$、越界 27 步；加 $|\Delta u|\le 1.5$ 后 $J=2.5074$、越界 28 步；收紧到 1.0 也只到 $J=2.5219$、越界 28 步。**限速几乎不起作用**，因为 LQR 的问题不是“动得太快”，而是“没有预见性”——它在速度已经接近上限时才开始收，收不住。MPC 之所以能压住，是因为它在 $N_h=10$ 步的预测里提前看到了越界，从而把输入分配到更早的时刻。这说明 **MPC 的优势不在于“更优”，而在于“更会分配”**：它把有限的输入预算花在最需要的地方，而不是一上来就饱和。

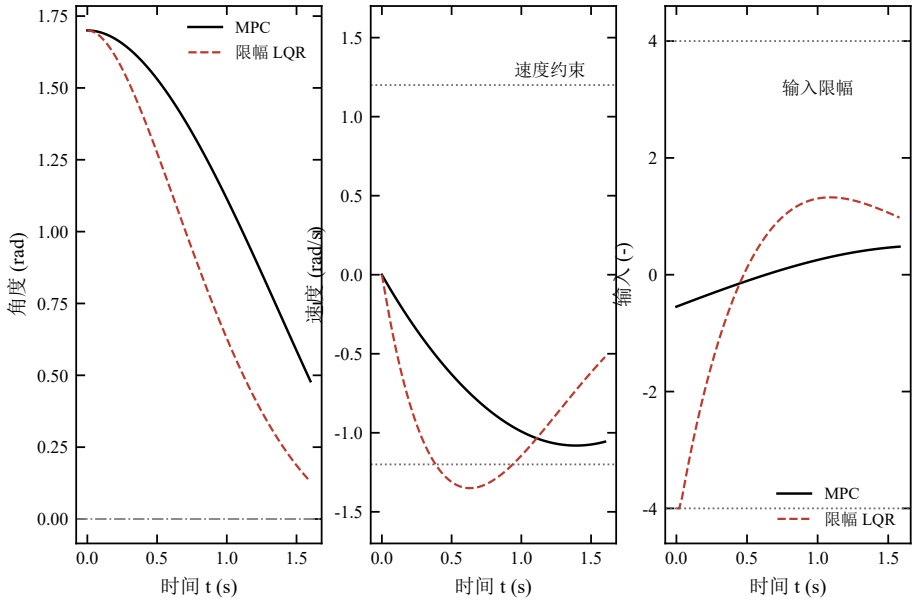


图 77-9 Koopman-MPC 与限幅 LQR 的对比：角度、速度与输入；MPC 主动压住速度，LQR 靠输入饱和硬顶

图 77-9 的三个子图把这件事画得很清楚。左图角度轨迹几乎重合；中图速度轨迹上，LQR 的红虚线在最初 0.5 s 内冲到 1.35 并越过 1.2 的约束线，MPC 的黑线贴着 1.08 走；右图输入上，LQR 直接跳到 4.0 并保持，MPC 平滑地爬升到 0.547。

77.9.4 另一条路：不用模型也能整定

Koopman 不是数据驱动控制的唯一形态。如果控制器的结构已经定死（比如就是 PID），可以跳过“辨识模型”这一步，直接用闭环数据整定参数。本章实现了两种。

虚拟参考反馈整定（VRFT）。 思路是：先假设一个参考模型 $M(s)$（这里取时间常数 $\tau=0.35$ s 的一阶惯性），再构造“虚拟参考”

$$r_v(t)=y(t)+\tau\dot{y}(t) \tag{77-31}$$

如果控制器 C 能让 y 跟随 r_v，那么闭环就近似等于 $M(s)$。于是整定问题变成“找 C 使 $u\approx C(r_v-y)$”，这是一个线性最小二乘：

$$\boldsymbol{\theta}^{\star}=\arg\min_{\boldsymbol{\theta}}\sum_{k}\left[u_k-\boldsymbol{\phi}_k^{\top}\boldsymbol{\theta}\right]^2 \tag{77-32}$$

其中 $\boldsymbol{\phi}_k=\left[e_k,\ \int e,\ \dot{e}_k\right]^{\top}$ 是 PID 的回归向量。一次闭环实验的数据就够，不需要额外激励。

本章实测：初始 PID（$k_p=0.40$、$k_i=0.15$、$k_d=0$）的代价 231.2359；VRFT 一次算出 $k_p=2.849$、$k_i=0.268$、$k_d=0.054$，代价降到 134.4795，**降了 41.8%**。整个过程只用了 6 s 的闭环数据，没有迭代。

迭代反馈整定（IFT）。 如果愿意做多次实验，可以沿代价的负梯度方向迭代。本章用中心差分近似梯度，带回溯步长衰减，迭代 8 轮：

$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta \frac{\nabla J(\boldsymbol{\theta}^{(i)})}{1 + \left|\nabla J(\boldsymbol{\theta}^{(i)})\right|} \tag{77-33}$$

结果从 231.2359 一路降到 100.6829，**比初始好 56.5%，比 VRFT 再好 25.1%**。作为参照，本章用三起点 Nelder-Mead 求得的“参考最优”是 100.4719，IFT 的终值与它只差 0.21%。

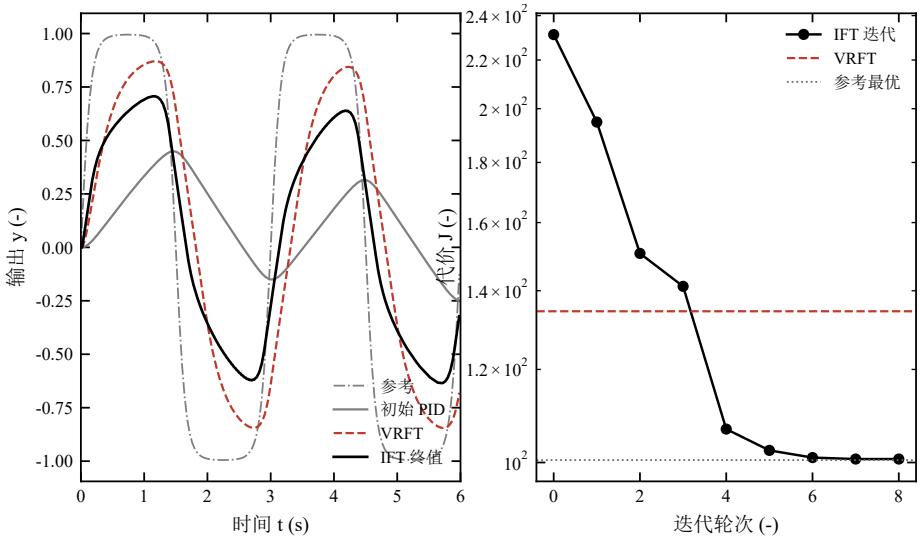


图 77-8 VRFT 与 IFT：左为跟踪响应，右为代价随迭代的下降；VRFT 一次到位，IFT 逐轮逼近参考最优

图 77-8 的右图是这组数据的直观版本。灰点线是参考最优 100.4719，红虚线是 VRFT 的 134.4795，黑实线是 IFT 的 8 轮迭代。IFT 从 231.2359 起步，前三轮下降很快（$194.8 \to 150.6 \to 141.2$），第 4 轮跨过 VRFT 的水平，后四轮进入 101 附近的平台。

两种方法的取舍。 VRFT 只需要一次实验，适合“设备不能停”的场合；IFT 需要 8 轮实验（每轮还要算梯度，实际是 8 组前后扰动共 16 次），适合“调试期可以停机”的场合。两者的共同前提是：**控制器结构固定、闭环稳定、参考信号覆盖工作频段**。如果结构本身选错了（比如该用前馈却只用 PID），整定得再好也补不上。

77.9.5 还能往哪里改

三个方向值得关注，但本章不展开，只说明它们各自解决什么问题。

深度 Koopman。 用神经网络代替手工字典，让网络自己学观测函数。优点是能处理高维对象（图像、分布参数系统），缺点是失去可解释性，且训练需要大量数据。工程上目前只在“手工字典明显不够”时才考虑。

鲁棒 Koopman。 在拟合时把模型误差当成有界扰动，设计控制器时保证在这个界内稳定。这需要先估计误差界，而误差界本身依赖数据覆盖，逻辑上有点循环，目前多在学术阶段。

字典自适应。 在线检测“当前状态是否落在训练域外”，若在外，则在线补充数据、局部重拟合。这条路在工程上最有前景，因为它直接对上了 §77.6.5 的失效一（外推爆炸）。

77.9.6 约束的可行性与松弛

式 (77-29) 里的三条约束不是随手写的，每条都有自己的物理来源与失败方式。

输入限幅 $|u| \le 4$ 来自电机与驱动器。它是硬约束，不能软化：超过了要么驱动器限流，要么机械过载。

速度约束 $|x_2| \le 1.2$ 来自工艺或机械。它作用于提升向量的第二维，在 $k = 0$ 时刻就直接生效——这一点很关键，因为初值本身可能已经越界。

变化率约束 $|\Delta u| \le 1.5$ 来自结构疲劳与电源容量。它约束的是相邻两拍之差，需要把上一拍的输入作为已知量传给优化问题。

可行性要在求解之前判断。 如果初值 $x_2(0) = 1.5$，已经超过 1.2，那么第 0 步的约束无论如何都满足不了，QP 会返回不可行。

现场遇到这种情况，求解器通常只报一个状态码，不告诉你原因。所以上线前必须做一次“初值体检”：把最坏初值代进去，检查速度约束是否已经违反；若违反，要么放宽约束，要么先用手动方式把状态拉回域内。

软约束是必备的兜底。 给速度约束加松弛变量 $s_k \ge 0$，代价里加一项大权重（比如 $10^3 s_k^2$），问题就总有解。软约束的代价是“约束可以被违反”，所以必须配合报警。

一旦 s_k 超过阈值，说明模型预测与真实状态已经脱节，应该切回安全控制器。软约束的意义不是“允许违反”，而是“违反时仍能给出可执行的输入”。

时域 $N_h = 10$ 为什么够。 输入直接作用于 x_2，从 x_2 越过 1.2 到被拉回，特征时间约 0.1 s，即 5 个采样周期。10 步（0.2 s）有两倍余量。

时域再长，QP 规模线性增长，收益却很小；时域太短（比如 3 步），MPC 看不到越界，退化成一个带限速的 LQR——表 77-8 已经说明限速几乎不起作用。

求解失败时的兜底顺序。 第一选择是保持上一拍输入，第二选择是切线性化 LQR，第三选择是切手动。这个顺序要写进控制器代码，不能留给操作员临时判断。

约束之间的冲突要提前想。 如果初值既要求快速收敛、又要求速度不越界、还要求输入变化平缓，三者可能无解。这时要么放宽一条，要么延长收敛时间。MPC 不会替你解决这个矛盾，它只会诚实地报告不可行。

这一小节的结论可以压成一句：**MPC 的价值在于把约束显式化，而显式化的前提是你能说清每条约束从哪来、违反了怎么办。**

77.9.7 VRFT 与 IFT 的适用条件

§77.9.4 的两个结果（134.4795 与 100.6829）看起来很好，但它们依赖四个前提，缺一个就不成立。

前提一：控制器结构固定。 VRFT 把“闭环等于参考模型”反解成控制器参数的拟合问题，它只调参数，不改结构。如果该用前馈却只用 PID，整定到最优也补不上结构缺失。

前提二：闭环稳定。 VRFT 用一次闭环实验的数据，实验本身必须稳定。如果初始 PID 就会振荡，采到的数据不能用；这时要先用手动或继电反馈把回路稳住。

前提三：参考信号覆盖工作频段。 本章的参考是周期 3 s 的 $\tanh$ 正弦，含基频与若干谐波，正好覆盖对象的动态。若参考是缓慢斜坡，高频段没有激励，整定出来的微分增益就不可信。

前提四：数据长度足够。 至少覆盖三到五个过渡过程，让积分项的作用显现。本章用 6 s，对应两个周期，偏短，所以 k_i 的估计精度不如 k_p。

参考模型选错会怎样？ VRFT 的整定目标是“闭环接近 $M(s)$”，参考模型定错，整定就会努力把一个错误的动态做出来。本章取 $\tau = 0.35$ s，比对象自身的快模态慢、比慢模态快，是一个合理的折中。

参考模型定得太快，整定出来的增益很大，输入饱和，实际响应达不到；定得太慢，整定出来的增益很小，性能浪费。工程上可以先按“比开环快 2 到 3 倍”来定 τ，再根据饱和情况微调。

IFT 的额外成本。 每轮要算梯度，中心差分意味着每个参数两次实验，三个参数就是六次，加一次基准共七次。本章 8 轮，实际实验次数在几十次量级。

这也解释了为什么 IFT 的终值更好：它用的是真实闭环代价，而 VRFT 用的是“与参考模型的差”这个代理。代理与真目标不完全一致，VRFT 的 134.4795 与参考最优 100.4719 之间那 34% 的差距，主要来自这里。

什么时候选哪个。 设备不能停、只允许一次实验，用 VRFT；调试期可以反复跑，用 IFT；两者都可以时，先 VRFT 拿到一个不错的起点，再用 IFT 精修——本章的 IFT 从初始 PID 起步，如果从 VRFT 的结果起步，收敛会更快。

共同的局限。 两者都只能整定“已经选定的结构”，都无法发现“该换结构”。这一点与 §77.11 的选型讨论是同一件事：参数优化解决不了结构问题。

77.9.8 从 LQR 到 MPC 的迁移成本

如果现场已经跑着 Koopman-LQR，想升级到 MPC，需要付出三部分成本。

第一是求解器。 LQR 只需要一次矩阵乘法，MPC 每周期要解一次 QP。本章实测热启动后 $2.5 \sim 2.8$ ms，在 20 ms 周期内可以接受，但要求控制器平台有足够的算力余量。

第二是约束建模。 每条约束都要写成对 $\boldsymbol{z}$ 与 u 的线性不等式，还要处理可行性、松弛与兜底。这部分工作量远大于写 LQR，也是 §77.9.6 花一整节讨论的原因。

第三是验证。 LQR 的闭环行为由增益唯一决定，扫几个初值就够了；MPC 的行为还依赖时域、终端权、约束松紧，验证组合成倍增加。

收益则集中在有约束的工况。本章的对比是：无约束时 MPC 的代价反而高 17.1%，有速度约束时它 0 步越界、LQR 越界 27 步。

所以迁移的判断很直接：**没有硬约束就不要迁；有硬约束且 LQR 靠饱和硬顶，就值得迁。**

还有一条折中路线：保留 LQR，在它外面加一层“参考修正”，当预测到越界时提前减速。这相当于只实现 MPC 的预见性、不实现优化，工程量小得多，效果介于两者之间。

本章没有做这组实验，但它是现场常见的做法。它的风险在于修正层的参数没有系统整定方法，往往靠试凑；而 MPC 至少把“要满足什么”写清楚了。

一句话：**MPC 贵在建模与验证，不贵在算力。** 这一点与很多人的直觉相反——算力是三者中最容易解决的一项。

77.10 工程落地

77.10.1 一次完整的数据采集作业

把 §77.3 的流程落成现场可执行的作业指导书，分五步。

第一步：静态检查。 手动把执行器从 -30% 走到 $+30\%$，记录稳态的输入—输出对应关系。这一步的目的是确认量程、死区、滞环，以及有没有卡涩。如果输入—输出关系严重非线性（比如某段完全不动），先修设备，不要急着做辨识。

第二步：安全边界确认。 确认角度、速度、力矩的硬限位，把这些值写进采集脚本的保护逻辑：一旦超限，立即切回手动并记录。这一步不能省——激励信号是“故意让设备动起来”，没有保护就是拿设备冒险。

第三步：激励试验。 本章推荐多正弦，四个频率覆盖 $0.18 \sim 1.21$ Hz，幅值从小到大逐步加（$0.1 \to 0.2 \to 0.3$），每一档跑 30 s。如果设备在这些幅值下都平稳，取最大的一档做正式采集；如果出现异响或超限，退回上一档。

第四步：正式采集。 至少 6 段，每段 400 步（8 s），段间换初值（用不同的初始角度，覆盖工作域）。总样本 2400，满足 $N \geq 40n$（三次字典）。同步记录时间戳、输入、输出、以及当时的工况标签（负载、温度）。

第五步：数据质检。 检查三件事：有没有丢帧（时间戳间隔是否恒为 T_s）、有没有饱和（输入是否长时间贴着限幅）、有没有异常跳变（相邻样本差是否超过物理可能）。任何一项不合格，重采。

77.10.2 激励信号的实测对比

本章 E9 用四种激励各采 4 段、每段 300 步，训练三次字典，结果见表 77-9。

激励	条件数	状态单步误差	200 步 RMSE	闭环代价 J
PRBS	22.62	1.395×10^{-2}	0.0273	3.5662
多正弦	12.14	6.696×10^{-3}	0.0248	3.5665
阶跃	138.53	2.661×10^{-2}	1.2862	524.5593
闭环抖动	120.36	1.991×10^{-4}	0.0616	3.5664

这张表有三个值得琢磨的地方。

第一，阶跃是灾难。 条件数 138.53 不是最高，但 200 步 RMSE 是 1.2862，闭环代价 524.5593——比正常值高 147 倍。原因是阶跃信号只在切换瞬间有能量，切换之间输入恒定，模型学不到“输入变化如何影响状态”的连续关系；更要命的是阶跃数据的轨迹集中在少数几条路径上，其他区域完全没有约束。

第二，闭环抖动的条件数很高，模型却很好用。 条件数 120.36 与阶跃接近，但单步误差 1.991×10^{-4} 是四种里最好的，闭环代价 3.5664 也正常。原因是抖动的激励虽然幅值小，但持续存在、方向随机，覆盖了状态空间的各个方向。**条件数大只说明矩阵求逆时某些方向被放大，不说明这些方向对预测重要。**

第三，多正弦的单步误差中等，长时误差最好。 单步 6.696×10^{-3} 不如闭环抖动，但 200 步 RMSE 0.0248 是四种里最低的。原因是多正弦的频谱集中在四个频率上，覆盖了对象的主要动态频段，长时外推时不会激励高频伪模态。

现场建议。 如果设备允许，用多正弦；如果不敢大幅激励，用闭环抖动（本章的数据证明它够用）；不要用纯阶跃数据做辨识——这是本章最实用的一条结论。

如果三种都不方便，退一步的选择是“正常运行数据加少量手动扰动”。手动扰动不必是规则信号，只要操作员在不同工况下做过几次明显的调节动作，数据里就有输入变化的信息。

本章没有做这组实验，但从 E9 的结论可以推断：关键是输入要有变化，而不是变化多漂亮。闭环抖动的条件数是四种里第二高，模型却是四种里最好的，原因正是它的变化持续且方向随机。

77.10.3 字典的物理可解释性

工程上常有人问：“φ_i 到底代表什么？”这个问题值得认真回答，因为它决定了你能不能信任模型。

前两维必须是状态本身。 x_1, x_2 是唯一有直接物理意义的坐标，也是约束与代价的作用对象。本章脚本把前两列固定为 x_1, x_2，所有约束与反投影都靠这个约定。

二次项对应能量与耦合。 x_1^2 与 x_2^2 正比于势能与动能，$x_1 x_2$ 是两者的交叉项。如果拟合出的 $\boldsymbol{A}$ 中 x_1^2 对 x_2 的耦合系数显著，说明系统里有“位置影响速度”的机制。

三次及以上的项多数是“修正”。 它们补足非线性形状，单个系数没有明确物理意义，但它们的整体贡献可以检查：把三次项全部置零，看预测误差上升多少。如果上升很小，说明三次项只是在拟合噪声，应该降阶。

反对“给每个特征值编故事”。 §77.6.2 已经说明 EDMD 会产生伪模态。工程报告里不应该写“这个极点对应某个物理模态”，除非它在多次辨识中稳定出现，且与已知的物理时间常数对得上。

77.10.4 上线检查清单

表 77-10 是本章建议的上线前检查项，每项都有明确的通过标准。

检查项	通过标准	不通过怎么办
谱半径	< 1	加样本或加岭
200 步 RMSE	$< 5\%$ 工作域	换字典或补激励
初值扫描	全部收敛	扩训练域或限初值
输入峰值	不长期贴限幅	加大 R 或降带宽
速度越界	0 步	上 MPC 或降增益
收敛时间	满足工艺要求	调 R、调 Q
回退通道	一键切线性化	补做

最后一项经常被忽略。本章的实测显示 Koopman 相对线性化的收益只有 0.27%（§77.11），这意味着**任何一次误动作的代价都远大于收益**。所以回退通道不是“以防万一”，而是“必须存在”：控制器每周期比较提升模型与线性化模型的预测，若两者差异超过阈值，报警并切回线性化。

77.10.5 与既有控制系统的共存

现场很少有空白的控制柜，Koopman 控制器通常要“插”进已有的 PID 或 MPC 回路里。三种可行的接法。

接法一：串联（外环）。 Koopman 控制器输出参考值，原有 PID 做内环跟踪。优点是改动小、原有回路的安全性不变；缺点是外环带宽受内环限制，Koopman 的快动态用不上。

接法二：并联（权重切换）。 正常时用 PID，检测到工况进入“非线性显著区”时切到 Koopman。切换逻辑要有滞环，避免在两套控制之间反复跳（滞环的做法见第 76 章）。

接法三：替代。 直接替换原控制器，保留原控制器作为回退。这种接法要求 Koopman 控制器在全部工况下都验证过，验证成本最高。

多数项目的合理选择是接法一或接法二。接法三只在“原控制器确实无法满足要求”时才考虑。

77.10.6 模型什么时候需要重辨识

提升模型不是一劳永逸的。以下四种情形必须重辨识。

机械状态变化。 更换轴承、调整配重、改变连杆长度，都会改变 A 的结构。判断方法是定期做一次开环校验：给一个小激励，比较预测与实测，误差超过基线 3 倍就重辨识。

工况大幅偏移。 负载、温度、压力的变化会改变阻尼与刚度。如果这些量在训练数据里没有覆盖，模型在新工况下就是外推。

传感器更换。 换了量程或滤波参数，噪声特性变了，原来拟合的 B 与 A 的高阶项都会偏。这一条最容易漏，因为“只是换了个传感器”看起来无关紧要。

长期漂移。 即使机械与传感器都没变，磨损与老化也会慢慢改变动态。建议每 3 到 6 个月做一次校验，用同一套验证数据比较误差趋势。

77.10.7 算力与实时性预算

实时性很容易被“算法正确”掩盖。本章的 MPC 在 Python 里跑，数字只能判断量级，但量级足够说明问题。

周期是 $T_s = 0.02$ s，即 20 ms。热启动后单次求解的中位数在 $2.5 \sim 2.8$ ms 之间，占周期的 $12\% \sim 14\%$，余量充足。

首次求解是 $60 \sim 65$ ms，超过三个周期。这一次的开销来自求解器初始化与稀疏结构的首次分析，**绝不能放在在线环节**。工程上的做法是上电时用一个虚拟状态先解一次，把求解器预热，之后再进入实时循环。

算力的分配要分清在线与离线。 离线做的是采集、拟合、Riccati 求解、约束矩阵组装；在线只做三件事：算 $\boldsymbol{z} = \boldsymbol{\varphi}(\boldsymbol{x})$、解一次 QP、输出 $\boldsymbol{u}$。

字典映射的成本常常被高估。20 列多项式只需要几十次乘法；27 列 RBF 要算 27 个指数，稍贵，但仍在微秒量级，与 QP 相比可以忽略。

移植到嵌入式时的三个注意点。 第一，OSQP 要用代码生成模式，把稀疏结构固定下来，否则每次运行都要重新分析矩阵。

第二，浮点精度。单精度下约束矩阵的条件数会被放大，$|x_2| \le 1.2$ 这种约束在边界附近可能反复触发，表现为输入抖动。要么用双精度，要么给约束留一点余量。

第三，内存。时域 10 步、状态 9 维的 QP 规模很小，但代码生成会把中间变量静态分配，需要提前算好栈的用量。

监控什么。 上线后要记录每周期的求解时间与迭代次数。求解时间上升说明工况变难，迭代次数上升说明约束变紧，两者都是“模型与对象开始脱节”的早期信号。

超时怎么办。 如果某一拍没解完，不要等——直接用上一拍的输入，并在日志里记一次超时。连续超时要降级到 LQR，因为这说明优化问题在当前工况下已经病态。

本章没有做硬实时测试，也不建议拿 Python 的耗时直接推算嵌入式性能。这里的数字只回答一个问题：**这条路的计算量在不在“能实时”的量级内。** 答案是肯定的。

77.10.8 从离线到投运的三个阶段

数据驱动控制器最危险的地方，是它在仿真里看起来完美。投运必须分阶段，每一阶段有明确的准出条件。

阶段一：纯离线。 不接设备，只用采集好的数据做拟合与三重校验，再在仿真对象上跑闭环。准出条件有三条：谱半径小于 1、200 步 RMSE 低于工作域的 5%、初值扫描全部收敛。

这一阶段最容易犯的错是“用训练数据验证”。本章的所有数字都在独立验证集上算，读者复现时也要这样。

阶段二：在线开环加建议模式。 控制器接上真实设备，但输出不直接驱动执行器，而是显示给操作员，由人决定是否采用。同时记录“控制器建议值”与“实际采用值”的差。

这一阶段的目的不是验证性能，而是验证**输入输出通道、符号、量纲、时延**。符号错一位，闭环就成正反馈；时延差两拍，MPC 的预测就全错。这些问题在仿真里永远发现不了。

准出条件是连续运行 8 小时，建议值与被采纳值的偏差在可解释范围内，且没有出现建议值长期贴着限幅的情况。

阶段三：闭环投运。 先在最小工况范围内闭合，比如初值只取 0.65 rad，稳定运行后再逐步扩大。每扩大一档，都要重新做一次初值扫描。

表 77-5 显示，初值从 0.65 到 2.60，代价涨了近 18 倍，模型的外推程度完全不同。用 0.65 rad 的验证结果去支持 2.60 rad 的投运，是没有依据的。

任何时候都要保留一键回退。 回退不是“出问题才用”，而是每次投运前的必检项：按下回退，控制器要在一个人机周期内切到线性化 LQR，且切换过程无扰动。

第 76 章的切换逻辑与滞环设计在这里可以直接借用。区别在于，那里的切换由工况驱动，这里的切换由“模型可信度”驱动，判据要更保守。

77.10.9 维护期要做的三件事

模型上线只是开始。数据驱动模型会随设备状态漂移而失效，维护期有三件事必须制度化。

第一，定期校验。 每 3 到 6 个月做一次小激励试验：幅值取正式采集的三分之一，采两段、每段 100 步，算单步误差与 200 步 RMSE，与基线比较。

误差超过基线 3 倍就重辨识，超过 1.5 倍则缩短下一次校验的间隔。这个阈值不是理论推导的，而是按本章的误差量级定的：正常波动在 10% 量级，3 倍已经说明结构变了。

第二，异常数据回流。 每次闭环出现异常（越界、抖动、超时），自动保存前后 30 s 的输入输出数据，标注时间与工况。

这些数据是最宝贵的：它们恰好落在模型失效的区域，补进训练集能直接改善覆盖。本章的"加覆盖"论（§77.5.4）在现场就靠这个机制落地。

第三，版本绑定。 模型文件必须与数据批次、字典定义、控制器参数绑定，任何一项变了就换版本号。回退时三者一起回退，不能只换模型不换参数。

工程上常见的事故是"模型换了、$\boldsymbol{Q}$ 忘了换"，结果闭环带宽变了，操作员以为是模型的问题，查了几天才发现是参数。

还有一件容易被忽略的事：交接文档。 数据驱动控制器的知识高度集中在建模者身上，人一走，后面的人不知道"为什么选三次字典""为什么激励取 0.30"。

§77.8.6 的调参记录就是交接文档的骨架。把它整理成"选型依据、验证结果、已知边界"三部分，后来者才有条件维护。

77.11 优缺点与适用边界

77.11.1 它真正给了什么

把本章的实验结论收拢，Koopman 提升的收益可以归成四条，按可靠性从高到低排列。

第一，可分析性。 提升之后，非线性对象有了"矩阵 $\boldsymbol{A}$、$\boldsymbol{B}$"，可以算谱、算能控性、算能观性、算闭环极点。这些量在原始非线性系统里要么没有定义，要么只能靠仿真估计。本章用这套工具解释了线性字典为什么会给出错误的 -1.5174（§77.4.2），这种解释能力是纯数据方法给不了的。

第二，约束的线性化。 §77.9 已经展开：速度约束在状态空间里是非线性的，在提升空间里变成对 $\boldsymbol{z}$ 第二分量的线性不等式，于是整个 MPC 是一个凸二次规划。这是本章最实在的工程价值。

第三，逐点精度的能控性。 线性化只在工作点附近有效，提升可以在整个工作域上把单步误差压到 10^{-6} 量级（表 77-2 的五次字典）。对于"工作域很宽、但又不值得上非线性 MPC"的场合，这是一个折中。

第四，与数据的直接对接。 不需要写物理方程，只需要采数据。对于模型难以建立的场合（复杂摩擦、多物理场耦合、批次差异大的设备），这是唯一可行的路。

77.11.2 它没给什么

四条限制同样要摆明。

第一，误差没有上界。 线性化至少有"工作点附近误差是二阶小量"的保证，提升没有。字典选错时（比如线性字典拟合对象 A），误差可以大到谱都错，而且不会报警。工程上只能用 §77.5.6 的清单反复验证。

第二，维数与数据量同步膨胀。 n 列字典至少需要 $40n$ 个样本，计算量随 n^3 增长。二状态对象还看不出压力，六状态、十状态对象上，五次字典的列数可以到几百，此时矩阵求逆与 Riccati 求解都开始吃力。

第三，伪模态污染解释。 §77.6.2 的 9 个极点里只有 1 个对应真实动态。如果工程师试图用全部极点做频域设计，会被伪模态误导。

第四，对激励的依赖。 §77.10.2 显示，用阶跃数据辨识的模型闭环代价是正常值的 147 倍。这意味着 Koopman 方法要求"敢做激励"，而很多连续生产线不敢。

77.11.3 什么时候该用

给四条判据，全部满足才考虑。

1. **线性化误差确实影响性能**：把线性化控制器在真实对象上跑一遍，若某些工况下超调超过 20% 或收敛时间超要求，说明线性化不够。
2. **对象状态维数不高**：$n_x \le 6$。再高，字典列数会失控，不如直接上神经网络。
3. **能采到带激励的数据**：至少能做多正弦或闭环抖动试验，且设备允许状态在工作域内摆动。
4. **有明确的约束要处理**：如果没有任何约束，提升带来的收益通常小于 1%（本章 0.27%），不值得。

77.11.4 什么时候不该用

四种情形直接排除。

工作点固定、偏差很小。 比如恒温箱、稳压电源，线性化误差在 1% 以内，Koopman 纯粹增加复杂度。

对象本身是分段线性或带强切换。 这类对象用增益调度或切换控制更直接（见第 76 章），提升会把切换点抹平，反而丢失结构信息。

数据里有大量未建模的动态。 比如执行器有严重滞环、传感器有明显漂移。这些动态会被强行压进 $\boldsymbol{A}$，识别出来的模型在别的工况下必然失效。先把硬件问题解决，再谈数据驱动。

安全等级要求极高、不允许试验。 提升模型的正确性依赖数据覆盖，没有数据就没有保证。这类场合应该用基于第一性原理的模型加鲁棒控制。

77.11.5 与其他方法的定位

表 77-11 把本章方法与相邻章节的方法放在一起对比。

方法	依赖	非线性处理	约束	主要章节
PID	无	无（局部线性）	靠限幅	第 44、106 章
增益调度	多个线性模型	分段线性	靠限幅	第 76 章
反馈线性化	精确模型	坐标变换消去	需另加	第 41 章
线性 MPC	线性模型	无	显式处理	第 58、80 章
Koopman-LQR	数据	提升后线性	不处理	本章
Koopman-MPC	数据	提升后线性	显式处理	本章
神经网络控制	大量数据	隐式	需另加	第 82 章

这张表的读法是“按依赖与能力排队”。PID 依赖最少、能力最弱；神经网络依赖最多、能力最强但最不可解释。Koopman 的位置在中间偏右：它比线性 MPC 多需要数据，但能处理非线性；比神经网络少需要数据，但只能处理“能被多项式或 RBF 逼近”的非线性。

77.11.6 本章的最终数字

把四种方案在同一个初值 $[2.0, 0]$ 上跑 200 步，结果见表 77-12 与图 77-10。

方案	闭环代价 J	输入峰值	最大步间变化	末值范数
线性化 LQR	3.5760	4.000	4.000	3.30×10^{-2}
DMD-LQR	3.5680	4.000	4.000	2.87×10^{-2}
EDMD-LQR（三次）	3.5664	4.000	4.000	2.99×10^{-2}
EDMD-LQR（五次）	3.5669	4.000	4.000	3.00×10^{-2}

四个方案的代价从 3.5760 到 3.5664，最大差距 0.27%；输入峰值全部顶在 4.000，说明约束在这组实验里没有被满足，四种方案都靠饱和硬顶。这个结果与 §77.6 的结论一致：无约束时，提升带来的收益可以忽略。

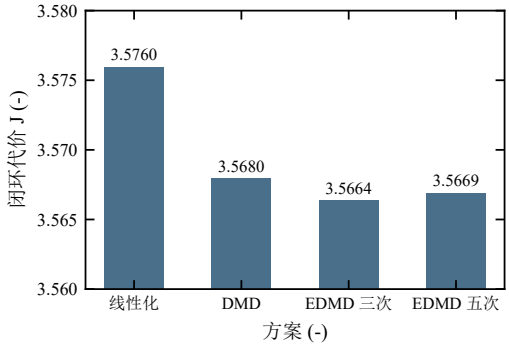


图 77-10 四种方案的闭环代价：纵轴截断在 3.560–3.580，差距只有 0.27%

图 77-10 的纵轴是截断的，这一点必须说明——如果不截断，四根柱子几乎一样高。截断是为了让读者看到差异的方向（EDMD 三次最低），而不是夸大差异的量级。

这个结果该怎解读？ 三条。

第一，**它不否定 Koopman 方法，而是否定“用 Koopman 换性能”这个动机**。本章的对象在闭环带宽内接近线性，提升没有多少非线性可“赚”。

第二，**它说明选型要以任务为准**。如果任务是“满足速度约束”，答案在表 77-8：MPC 用 17.1% 的代价换来 0 步越界。如果任务是“少花算力”，答案是线性化 LQR。

第三，**它提醒我们核实每一个“显著提升”的说法**。文献里常见的“精度提升一个数量级”通常指开环预测误差（本章确实做到了：单步误差降 1728 倍），而不是闭环性能。两者不能混为一谈。

77.11.7 与相邻章节的分工

本章与几处内容有交叠，这里明确指向，避免重复。

- **PID 整定**：§77.9.4 的 VRFT/IFT 是“用数据整定 PID”，与第 44 章、第 106 章的经典整定方法（Ziegler-Nichols、内模法、继电器法）是互补关系。经典方法只需要几次阶跃实验，VRFT 需要闭环数据但能直接优化指定代价。本章不重复经典方法的推导。
- **MPC**：§77.9 的 Koopman-MPC 把非线性 MPC 转成线性 MPC，时域、终端代价、约束松弛、可行性处理全部沿用第 58 章与第 80 章，本章只讲“模型怎么来”。
- **Smith 预估与迟延**：如果对象有大迟延，提升空间里会多出“迟延链”的模态，维数迅速膨胀。这类对象应先按第 47 章、第 107 章处理迟延，再考虑提升。
- **切换与混杂**：第 76 章的增益调度可以看成“分段局部 Koopman”，如果对象的分段结构清楚，用第 76 章的方法比全局提升更省事。

77.11.8 三个常见误解

读 Koopman 的文献时，有三个说法反复出现，每一个都需要打折扣。

误解一：“Koopman 能把任何非线性系统精确线性化。” 只在字典张成不变子空间时成立。对象 A 的三项字典是这种情形，单摆不是，工程对象几乎都不是。

对一般对象，提升给的是“在数据覆盖范围内、在最小二乘意义下最好”的线性近似，没有误差上界。把它当成精确线性化，就会在超出训练域时毫无防备。

误解二：“字典越大越准。” 表 77-2 与表 77-3 已经给出反例。字典变大，单步误差可能降，但谱半径可能越过 1，长时误差反而上升。

正确的说法是“字典要与数据量匹配”。$N \geq 40n$ 这条经验就是匹配的量化版本。脱离数据量谈字典大小，没有意义。

误解三：“有了数据就不需要第一性原理模型。” 数据只能覆盖它去过的区域，第一性原理模型的价值恰恰在没去过的地方。

工程上的合理分工是：用第一性原理定结构（有哪些状态、哪些非线性、量级多少），用数据定参数与修正项。本章的字典选择就是这种分工的体现——知道非线性来自 $\sin x_1$，才会去试三次多项式。

还有一个更隐蔽的误解：**“闭环性能提升等于模型精度提升”**。本章单步误差降了 1728 倍，闭环代价只降 0.27%。两者的关系不是线性的，取决于模型误差是否落在闭环带宽内（§77.6.4）。

这条误解的危害最大，因为它是“用开环指标宣传闭环效果”的根源。看到“精度提升一个数量级”时，第一反应应该是问：这是开环还是闭环？在什么工况下？与哪个基准比？

77.11.9 一张选型决策流程

把 §77.11.3 与 §77.11.4 的判据串起来，可以得到一条五问流程。每一问都能用本章的数据回答，不需要凭感觉。

第一问：线性化误差是否落在闭环带宽内？ 把线性化控制器在真实对象上跑一遍，看超调与收敛时间。本章的答案是“不在”——闭环代价只差 0.27%，说明提升没有多少可赚。

第二问：有没有必须显式处理的约束？ 本章有速度约束 $|x_2| \leq 1.2$，线性化 LQR 越界 27 步。这一问的答案是“有”，而且这一条比第一问更重要。

第三问：能不能做激励？ 能采多正弦或闭环抖动，答案是“能”；只能采正常运行数据，答案是“不能”，此时应转向免模型整定（§77.9.4）或经典 PID（第 44、106 章）。

第四问：状态维数是否不超过 6？ 超过之后字典列数失控，计算与数据量都跟不上，应考虑神经网络方法（第 82 章）。

第五问：有没有回退条件？ 没有回退通道，任何数据驱动控制器都不该上线，因为它的失效是“某工况突然发生”，不是渐变的。

五个答案组合起来，落到四条路上。

第一问否、第二问否：用线性化 LQR 或增益调度（第 76 章），不要引入提升。

第一问否、第二问是：用线性 MPC（第 58、80 章），模型用线性化或辨识得到，不需要 Koopman。

第一问是、第二问是，且第三、第四问都通过：用 Koopman-MPC，这是本章的主线。

第三问否：用 VRFT/IFT 整定固定结构，或按第 44 章的经典方法整定 PID。

这张流程的价值在于它把“方法选择”变成了“问题回答”。本章的单摆恰好落在“第一问否、第二问是”，所以最终选择是 Koopman-MPC，理由不是精度，而是约束。

77.11.10 本章没做的事

一本书的可信度，一半来自说清了什么，一半来自承认没说什么。本章有四处明确的空白。

第一，全部实验都在仿真对象上完成。 三个对象都是常微分方程，没有传感器噪声的真实统计特性，没有执行器的死区与滞环，也没有通信时延。这些在 §77.10.8 的阶段二会被逐一暴露，但本章没有条件覆盖。

第二，没有给出误差上界。 提升模型的误差只能靠数据反复验证，没有“误差不超过多少”的解析结论。鲁棒 Koopman 是一个方向（§77.9.5），但还停留在研究阶段。

第三，只做了二状态对象。 十状态、二十状态的对象上，字典列数会到几百，$N \geq 40n$ 意味着上万个样本，Riccati 与 QP 的规模都上了一个量级。本章的结论能否平移，需要单独验证。

第四，没有做在线自适应。 模型是离线辨识、上线后固定。设备漂移时靠重辨识（§77.10.6），而不是在线更新。在线更新涉及遗忘因子、数据筛选、稳定性保证，本章没有展开。

这四处空白不是遗漏，而是范围界定。把它们写出来，是为了让读者知道哪些结论可以直接用、哪些需要在目标对象上重做一遍。

还有一处方法上的取舍值得说明。 本章始终用“多项式或 RBF 的固定字典”，没有用神经网络学字典。这样做牺牲了表达能力，换来了可解释性与可复现性——每个数字都能追溯到某一行代码、某一批数据。

如果读者关心的是高维对象或图像输入，本章的方法不够用，应该转向第 82 章的思路。但如果关心的是“机械、过程对象上的非线性控制”，固定字典加上本章的校验流程，已经足够覆盖大多数场合。

77.12 Python 仿真代码

77.12.1 文件与运行方式

本章两个脚本，都在 `book/code/` 下：

- `ch077_sim.py`：全部数值实验（E1–E10）与数据缓存；
- `ch077_figs.py`：只负责出图，从缓存读数。

运行命令：

```
"D:\Anaconda3\envs\py311\python.exe" code/ch077_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch077_figs.py
```

第一个脚本运行约十几秒（含一次 MPC 的 80 步求解），结束时在 `figs/ch077/results.npz` 写入全部曲线数据，并打印本文所有数字。第二个脚本读缓存，生成 10 张 SVG 与同名 PDF。两者都不依赖外部数据文件，换一台装有 numpy、scipy、matplotlib、cvxpy 的机器可以直接跑通。

77.12.2 代码结构

表 77-13 是 `ch077_sim.py` 的函数清单与职责。

函数	职责	被谁调用
`rk4_step`	定常输入的四阶 Runge-Kutta 单步	所有仿真
`c2d`	零阶保持离散化（增广矩阵指数）	`lin_lqr`
`Dict`	字典：线性、多项式、RBF	E1–E10
`edmd`	带输入的 EDMD 最小二乘	E1–E10
`dlqr`	离散 Riccati 求解	E6、E8、E9、E10
`state_err`	状态空间单步误差	E2–E5、E9
`rollout`	开环提升预测	E1–E5、E9
`run_closed`	真实对象上的闭环仿真	E6、E8、E10
`collect_b`	分段采集（含激励与噪声）	E2–E10

脚本没有用 `simlib`，因为本章的字典与提升逻辑是独有的，复用它反而增加耦合。风格上仍遵循全书约定：绝对路径一律用 `Path(__file__).resolve().parents[1]` 推导，变量名全英文，注释中文。

77.12.3 三个关键片段

片段一：字典类。 前两列固定为 x_1, x_2，这样所有约束与反投影都能直接取 `z[:2]`，不需要额外的映射函数。多项式部分把幂次排列显式存下来，便于 E1 构造“缺项”字典：

```
def monomial_powers(self):
    if self.powers is not None:
        return list(self.powers)
    powers = [(1, 0), (0, 1)]
    if self.kind == "lin":
        return powers
    for total in range(2, self.deg + 1):
        for p in range(total, -1, -1):
            powers.append((p, total - p))
    return powers
```

片段二：EDMD。 核心只有四行。注意 `W` 是把字典与输入拼在一起，这正是“带输入”的含义：

```
Z1 = np.vstack([dic(X[:-1]) for X in Xs])
Z2 = np.vstack([dic(X[1:]) for X in Xs])
W = np.vstack([np.hstack([dic(X[:-1]), U]) for X, U in zip(Xs, Us)])
Th, *_ = np.linalg.lstsq(W, Z2, rcond=None)
A, B = Th[:-1].T, Th[-1:].T
```

片段三：状态空间误差。 §77.5.3 的结论全靠这个函数。它只取预测结果的前两维，与真实状态比：

```
def state_err(A, B, dic, Xs, Us):
    """状态空间单步预测误差：不同字典之间可直接比较。"""
    err = []
    for X, U in zip(Xs, Us):
        pred = (A @ dic(X[:-1]).T + B * U.T).T[:, :2]
        err.append(rmse(pred, X[1:]))
    return float(np.mean(err))
```

如果没有这个函数，E2 会得出“字典越大越差”的错误结论。这是本章代码里最重要的一处设计。

77.12.4 出图脚本

`ch077_figs.py` 的结构是“一个函数一张图”，`main()` 里按顺序调用。数据全部从 `results.npz` 读，只有图 77-8 的时域响应是现场重算的（因为存轨迹会让缓存大一倍）。

图 77-1 是唯一的框图，用 `block_styles` 的语义函数画；其余九张是数据图，统一走 `plot_styles` 的 `setup()`、`save()`、`SINGLE`、`DOUBLE`。图内中文用纯文本，公式用 `$...$`，两者不写进同一个字符串——这条规则在画图 77-9 的纵轴时踩过一次坑：`r"角度 $x_1$ (rad)"` 会让 STIX 字体找不到汉字字形，输出一串方框。改成 `"角度 (rad)"` 并把符号放进图注，问题消失。

77.12.5 复现时的常见问题

问题一：`cvxpy` 没装。 E8 会直接报错。可以先注释掉 E8 与 `ch077_figs.py` 里的 `fig09_mpc`，其余九组实验不受影响。

问题二：OSQP 首次求解很慢。 本章实测首次 $60 \sim 65$ ms，属于正常的初始化开销。如果超过 1 s，通常是求解器在编译，第二次就正常了。

问题三：图里的中文变成方框。 说明系统没有 SimSun 或 SimHei。`plot_styles._pick_font` 会依次尝试几种中文字体，都找不到时只剩西文字体。装一个中文字体即可。

问题四：随机数不一致。 所有采集函数都接受 `seed` 参数，本章固定 seed（1、3、5、7、11、13、21），同一台机器上结果可复现。跨 numpy 版本时，`default_rng` 的流可能变化，这时需要固定 numpy 版本。

问题五：MPC 的结果与正文差一点。 OSQP 是迭代求解器，终止精度默认 10^{-5}，代价的最后一位可能波动。本章报四位小数，实测重复运行的差异在 10^{-4} 量级，不影响结论。

77.12.6 建议的复现顺序

脚本里 E1 到 E10 是按主题排的，复现时建议按下面的顺序跑，理由是先验证方法、再验证模型、最后验证控制。

先跑 E1。 看三项字典的单步残差是否在 10^{-16} 量级、对角元是否与式 (77-14) 逐位一致。这一步不过，后面的数都不可信，先查字典构造与最小二乘的实现。

再跑 E5。 看 DMD 的两个复极点是否接近 $-0.1 \pm 0.9950j$，EDMD 的 9 个实极点实部是否全为负。这一步验证“谱”这件事算对了。

然后 E2 与 E3。 前者看字典阶梯，后者看数据量。重点不是记数字，而是确认两条趋势：单步误差随字典下降、谱半径随样本数跨过 1。

接着 E4。 看噪声下三次与五次的排序反转。这一组最容易出现“结果与正文不一致”，因为它对噪声实现敏感；若不一致，先检查 seed。

再跑 E6，最后 E8。 E6 是闭环初值扫描，E8 是 MPC。E8 依赖 cvxpy，没装就先跳过，不影响其他结论。

E9 与 E7 可以放最后。 前者是激励对比，后者是免模型整定，两者与主线相对独立。

E10 是汇总，必须最后跑，因为它要读前面几组缓存下来的模型。

每组跑完，建议先看打印里的三个数：单步误差、谱半径、200 步 RMSE。这三个数一致，才继续下一组；不一致就先停下来查，不要一路跑到底再回头找原因。

出图脚本与仿真脚本是分开的，改完仿真只需重跑仿真，再跑一次出图即可。缓存文件 `results.npz` 里的键名与图函数一一对应，加实验时记得同步更新。

如果是第一次读这份代码，建议先只跑 E1：把三项字典的拟合矩阵打印出来，与式 (77-13) 的解析矩阵逐项对照，看清那个 0.01972207 出现在什么位置。

这一步做成之后，后面的实验都是“换字典、换数据量、换噪声”的重复，理解成本会低很多。反过来，跳过 E1 直接看闭环，很容易把实现细节的问题误当成方法的局限，这也是初学者最常走的弯路。

第78章 容错控制

78.1 动机与直观理解

78.1.1 一台松手就掉的平台

前面的章节里，控制器面对的对象大多是“自己不跑”的：给它一个扰它才动，把控制器关掉它顶多停在原地。本章的对象不是这样。取一台单轴磁悬浮平台，只保留竖直方向一个自由度，用上下两个电磁铁把一根导磁轴悬浮在中间。在工作点附近线性化之后，它的运动方程可以写成

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = \omega^2 x_1 - 2\zeta\omega x_2 + g_1 u_1 + g_2 x_3, \qquad \dot{x}_3 = \frac{u_2 - x_3}{\tau} \tag{78-1}$$

其中 x_1 是轴相对工作点的位移，x_2 是速度，x_3 是备份执行器的内部状态，u_1 是主执行器（工作线圈的电流指令），u_2 是备份执行器（辅助线圈）的指令。参数取 $\omega = 1.2$ rad/s、$\zeta = 0.15$、$g_1 = 1.0$、$g_2 = 0.6$、$\tau = 0.15$ s，采样周期 $T_s = 0.02$ s，两路输入限幅 $|u_1| \le 5$、$|u_2| \le 6$。

请注意式 (78-1) 里 x_1 前面的符号：$+\omega^2$，不是 $-\omega^2$。这是磁悬浮与弹簧-质量系统最根本的差别——电磁吸力随气隙减小而增大，负刚度。线性化之后，开环极点是 $+1.033$、-1.393 与 -6.667，三个都在实轴上，其中第一个落在右半平面。这意味着**没有控制器时，任何微小偏移都会纯指数地涨上去**：时间常数 $1/1.033 \approx 0.97$ s，每过 0.97 s 偏差就涨到 e 倍，2.5 s 涨 13 倍，几秒之内轴就吸死到一侧。

同一个符号也决定了本章的全部困难。对一个开环稳定的对象，执行器失效只是“控制变差”；对一个本质不稳定的对象，执行器失效是“系统发散”。第 44 章与第 106 章里的 PID 整定、第 58 章与第 80 章里的模型预测控制，讨论的都是“怎样把性能做好”；本章讨论的是更前置的一件事：**当控制器赖以出力的那个部件坏了，闭环还能不能活着**。

控制任务是跟踪一条正弦参考

$$r(t) = A_r \left(1 - e^{-t/T_r}\right) \sin \frac{2\pi t}{T_p} \tag{78-2}$$

幅值 $A_r = 0.30$、周期 $T_p = 12$ s、起幅包络时间常数 $T_r = 0.8$ s。包络的作用是让 $t = 0$ 处的参考导数从零开始，避免前馈项在第一步就给出一个跳变。仿真总长 12 s（600 步），故障在 $t_f = 2.0$ s 注入。

78.1.2 故障从哪里来

“故障”不是一个抽象名词，它在硬件上有明确的来源。本章只考虑主执行器通道的三类故障，因为它们覆盖了电磁执行机构最常见的失效形态：

效率下降。线圈局部匝间短路、功放管开通电阻变大、供电电压跌落，都会让同一句电流指令产生更小的实际电流。设实际作用量是指令的 ρ 倍，$\rho \in [0, 1)$。$\rho = 1$ 表示健康，$\rho = 0$ 表示完全失效。

卡死。功率管击穿短路、驱动级被保护电路钳位、D/A 输出保持在上一次的值，都会让线圈电流不再受指令控制，而是停在一个常值 c 上。这时无论控制器怎么喊，实际作用量都是 c。

偏置。电流传感器的零点漂移、前级运放的失调电压、PWM 占空比的系统性偏差，会让实际电流比指令多出一个常数 c。通道仍然是“可控”的，只是控制器的每一句话都被翻译错了一点。

三类故障可以写成一个统一的式子：

$$u_1^{\text{eff}} = \rho u_1 + c, \qquad (\rho, c) \in \{(\rho, 0)\} \cup \{(0, c)\} \cup \{(1, c)\} \tag{78-3}$$

效率下降是 $\rho < 1$、$c = 0$ 的那一支；卡死是 $\rho = 0$、$c \neq 0$ 的那一支；偏置是 $\rho = 1$、$c \neq 0$ 的那一支。把它们塞进同一个参数对 (ρ, c)，后面就可以用同一套辨识算法去估计，而不必先猜是哪种故障。

这个统一写法有一个直接的好处：**故障诊断变成了参数估计**。观测器给出残差，残差里含着 (ρ, c) 的信息，用一段数据做最小二乘就能把这两个数估出来，估出来之后再按数值大小分类。本章第 78.7 节会看到，三条分类边界 0.12、0.88、0.35 都是从物理意义出发定的，不是拍脑袋。

78.1.3 三条路线：不处理、被动、主动

面对故障，工程上有三条路，代价依次上升。

第一条是不处理。控制器照常工作，假装什么都没发生。这在开环稳定的对象上勉强可以接受——性能退化，但系统还在。在本章的平台上不行。取 $\rho = 0.4$（主执行器只剩四成力气），12 s 的闭环代价是

$$J = \int_0^{12} \left(e^2 + 0.1 x_2^2 + 0.02 u_1^2 + 0.02 u_2^2 \right) \mathrm{d}t = 0.8115 \tag{78-4}$$

而健康时的同一个指标是 0.0473，差了 17 倍。故障后位置误差的峰值是 0.2481，末段均方根误差 2.32×10^{-1}——注意这不是“收敛得慢”，而是**根本没有收敛**：误差在末段仍然和故障刚发生时一样大。原因很简单，控制器以为自己给出了 u_1，实际只得到 $0.4 u_1$，它按缺省四成的力气去算平衡点，稳态必然偏。

如果 ρ 再小一点，事情就不是“偏”而是“飞”了。$\rho = 0$ 时，不处理的闭环代价是 8.63×10^{6}，位置峰值 3970，末段均方根 1.96×10^{3}，轨迹已经彻底发散。

第二条是被动容错。既然最坏情况是效率掉到某个下限，那就干脆按那个下限设计一套固定增益，全工况不切换。取最坏效率 $\rho = 0.5$，用有效输入矩阵

$$\boldsymbol{B}_{\mathrm{pass}} = \begin{bmatrix} 0.5\,\boldsymbol{b}_1 & \boldsymbol{b}_2 \end{bmatrix} \tag{78-5}$$

重算一次 LQR，得到

$$\boldsymbol{K}_{\mathrm{pass}} = \begin{bmatrix} 85.729 & 31.309 & 0.2946 \\ 4.3685 & 1.6605 & 4.7476 \end{bmatrix} \tag{78-6}$$

而标称增益是

$$\boldsymbol{K}_{0} = \begin{bmatrix} 72.6904 & 25.1846 & 0.2216 \\ 1.5263 & 0.5577 & 4.7365 \end{bmatrix} \tag{78-7}$$

对比两套增益，被动容错的思路一目了然：主通道的第一行增益从 72.69 抬到 85.73，抬高约 18%，用“平时多使劲”来换取“坏的时候还够用”。

这条路的问题也在数字里：$\rho = 0.4$ 时，被动容错的代价是 0.8104，与不处理的 0.8115 几乎一模一样。道理不难想：按 $\rho = 0.5$ 设计的增益，在 $\rho = 0.4$ 时仍然欠 20% 的力，稳态误差照旧存在；而在健康工况下，它又多使了 18% 的力，白付了控制能量。**被动容错把最坏情况当成常态来设计，代价是全程都在为未必发生的故障买单，而且一旦实际故障比预设的更严重，它一样救不了。**

第三条是主动容错。让系统自己发现“我不对劲”，判断是哪种不对劲，然后换一套对应的控制律。同一次 $\rho = 0.4$ 的故障，主动容错的闭环代价是 0.2344，故障后峰值 0.0558，末段均方根 2.28×10^{-3}——比不处理低了 3.5 倍，末段误差低了两个数量级。$\rho = 0$ 时差距更悬殊：不处理与被动容错分别发散到 8.63×10^{6} 与 4.30×10^{6}，主动容错是 0.2151，位置峰值只有 0.1647，稳稳地活了下来。

三者放在一起，关系的轮廓就出来了：

方案	$\rho=0.4$ 的 J	$\rho=0$ 的 J	末段误差（$\rho=0.4$）	是否发散（$\rho=0$）
不处理	0.8115	8.63×10^{6}	2.32×10^{-1}	是
被动容错	0.8104	4.30×10^{6}	2.36×10^{-1}	是
主动容错	0.2344	0.2151	2.28×10^{-3}	否
健康基准	0.0473	—	1.60×10^{-3}	—

这张表就是本章要解释的全部内容。为什么主动容错能把误差压回 10^{-3} 量级？因为它做了三件事，而这三件事每一件都有代价：**检测**要时间，**辨识**要数据，**重构**要一套能落地的切换逻辑。任何一环没做好，主动容错就会退化成“和不处理差不多”，甚至更糟——切换瞬间的冲击可能把本来还能维持的系统推出发散边界。

78.1.4 主动容错的三段式

主动容错的骨架可以用三个时刻切开。设故障发生在 t_f，检测算法在 t_d 时刻报警，隔离算法在 t_i 时刻给出故障参数，控制律在 t_s 时刻完成切换。三段分别是：

- **检测段** $[t_f, t_d]$：故障已经发生，控制器还不知道，闭环带着错误的模型继续跑。这一段的长短由残差的信噪比和阈值共同决定。
- **隔离段** $[t_d, t_i]$：知道出事了，但还不知道是什么事。系统在这段时间里采集故障后的数据，做参数辨识。这一段的长短由辨识窗长决定，本质上是“用时间换精度”。
- **决策与执行段** $[t_i, t_s]$：参数有了，算新增益、检查切换条件、把新控制律接上去。这一段的长短由计算量和安全联锁决定。

图 78-1 画出了这套结构。上面一行是物理回路：参考 r 进求和点，经增益 $\boldsymbol{K}$ 产生两路指令，主执行器出力经故障环节（虚线块）后进入磁悬浮轴，备份执行器经一阶惯性环节 $\frac{1}{\tau s+1}$ 后也进入同一根轴。下面一行是容错支路：观测器拿 u 与 y 算出残差，检测模块判断“是否超阈”，隔离模块估计 $(\hat{\rho},\hat{c})$，重构模块算出新增益，再用虚线把新增益送回 $\boldsymbol{K}$。

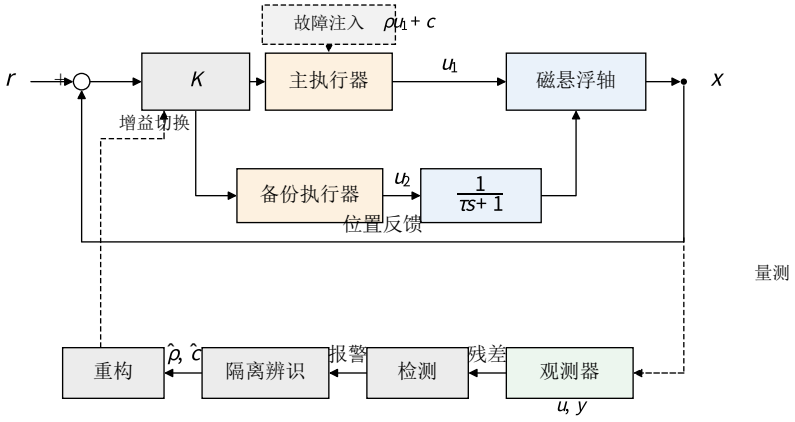


图 78-1 容错控制的总体结构：主/备份双执行器、主通道上的故障注入点，以及由观测器、检测、隔离辨识、重构组成的容错支路

这张图里有两处容易被忽略、但在工程上决定成败的细节。

第一处是**观测器的输入必须与对象的实际作用量一致**。图里观测器的输入标着 u, y，这个 u 不是控制器算出来的指令，而是执行器真正作用到对象上的量。故障前两者相同，故障后就分叉了：效率下降时真实作用量是 ρu_1，卡死时是 c，偏置时是 u_1+c。如果观测器仍然用指令 u_1 推进，它就是在用一个错的模型去解释一个对的测量，残差会一直很大，检测会一直报警，而重构后的控制律会建立在一个错误的估计上。本章的仿真里，这一条曾经让主动容错的效果从“降低 3 倍代价”退化到“几乎没有改善”，修正之后才恢复正常。

第二处是**备份执行器不是免费的**。它有自己的惯性（时间常数 $\tau = 0.15\,\mathrm{s}$，相当于 7.5 个采样周期）和自己的限幅（$|u_2| \le 6$）。这两条限制了它能顶多少事。第 78.2 节末尾会给出一个只用两个数就能算清楚的判据：故障占去多少权限，剩下多少权限还够不够镇定这个本质不稳定的对象。这个判据解释了为什么本章的注入测试里，“卡死 2.0”这一行连主动容错也过不去，而“卡死 1.0”能过。

78.1.5 本章路线

后面的内容按“原理—设计—实现—评估—落地”展开。

第 78.2 节把上面的直观说法写成公式：对象、故障、观测器、残差、检测统计量、辨识方程、重构增益，以及那个决定“救不救得回来”的权限判据。第 78.3 节给出一套可以照做的设计步骤，七步，每步都有明确的输入输出。第 78.4 节与第 78.5 节分别讨论连续时间与离散时间的实现，重点在两者在残差统计特性与阈值标定上的差别。第 78.6 节用极点、代价、峰值、末段误差四组指标回答“容错之后系统到底稳不稳、性能剩多少”。第 78.7 节逐个解释参数的意义——阈值系数 k、滑动平均窗长、辨识窗长、切换延迟、淡入时间、再分配权重——并说明每一个参数拧大拧小会发生什么。第 78.8 节给调参经验，包括本章实测的阈值扫描与故障幅值扫描。第 78.9 节讨论优化方向，第 78.10 节讲工程落地（故障注入测试、冗余配置、切换瞬态），第 78.11 节讲适用边界，第 78.12 节给出完整可运行的代码。

读这一章时请始终记住一个对照关系：**容错的收益是“故障后还能活”，容错的成本是“健康时多付的检测开销与切换风险”**。第 78.7 节与第 78.8 节的每一个参数，都是在这两者之间挪动位置。

78.2 数学原理

78.2.1 从连续对象到离散模型

把式 (78-1) 写成状态空间形式，$\boldsymbol{x} = [x_1, x_2, x_3]^{\mathsf{T}}$，$\boldsymbol{u} = [u_1, u_2]^{\mathsf{T}}$：

$$\dot{\boldsymbol{x}} = \boldsymbol{A}_c \boldsymbol{x} + \boldsymbol{B}_c \boldsymbol{u}, \qquad \boldsymbol{A}_c = \begin{bmatrix} 0 & 1 & 0 \\ \omega^2 & -2\zeta\omega & g_2 \\ 0 & 0 & -1/\tau \end{bmatrix}, \qquad \boldsymbol{B}_c = \begin{bmatrix} 0 & 0 \\ g_1 & 0 \\ 0 & 1/\tau \end{bmatrix} \tag{78-8}$$

控制器以 $T_s = 0.02\,\mathrm{s}$ 周期运行，输入在一个周期内保持不变，所以用零阶保持器离散化。标准做法是把矩阵指数作用在增广矩阵上：

$$\exp\left(\begin{bmatrix} \boldsymbol{A}_c & \boldsymbol{B}_c \\ \boldsymbol{0} & \boldsymbol{0} \end{bmatrix} T_s \right) = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \tag{78-9}$$

于是对象模型变为

$$\boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k + \boldsymbol{B}\boldsymbol{u}_k, \qquad \boldsymbol{B} = \begin{bmatrix} \boldsymbol{b}_1 & \boldsymbol{b}_2 \end{bmatrix} \tag{78-10}$$

这一步在第 5 章与第 20 章已经反复用过，这里只强调一个与容错有关的细节：**离散化把连续时间的“通道”变成了离散时间的“列”**。主执行器对应 $\boldsymbol{B}$ 的第一列 $\boldsymbol{b}_1$，备份执行器对应第二列 $\boldsymbol{b}_2$。后面所有的故障辨识、权限分析、增益重构，都是在这两列上做文章。

$\boldsymbol{b}_1$ 与 $\boldsymbol{b}_2$ 的形状值得看一眼：$\boldsymbol{b}_1$ 只有第二个分量非零（力直接作用在速度上），$\boldsymbol{b}_2$ 的第一个分量为零、第二个分量是 τ 的一阶惯性响应折算过来的小量、第三个分量非零。也就是说，**主执行器是“直连”的，备份执行器是“绕道”的**。直连通道坏了，绕道通道能不能顶上，取决于它的带宽与权限，这两件事第 78.2.7 节会算清楚。

78.2.2 故障下的闭环：等效输入偏差

把式 (78-3) 代入式 (78-10)，故障后的对象是

$$\boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k + \boldsymbol{b}_1 \left(\rho u_{1,k} + c \right) + \boldsymbol{b}_2 u_{2,k} \tag{78-11}$$

控制器仍然按标称模型计算，认为作用量是 $\boldsymbol{b}_1 u_{1,k} + \boldsymbol{b}_2 u_{2,k}$。两者之差是一个只沿 $\boldsymbol{b}_1$ 方向的输入偏差：

$$\Delta \boldsymbol{u}_k = \boldsymbol{b}_1 \left[(\rho - 1) u_{1,k} + c \right] \tag{78-12}$$

式 (78-12) 是本章所有检测与辨识的出发点。它说明三件事。

第一，**故障在状态方程里表现为“沿 $\boldsymbol{b}_1$ 方向的未知输入”**，而不是“状态矩阵变了”。这个区别很重要：状态矩阵变了会改变系统的稳定性结构，所有极点一起漂移；输入偏差只是在原有动力学上叠加一个推动项，系统的“性格”没变，只是被推着走。

第二，**偏差的大小依赖 $u_{1,k}$**。效率下降时偏差正比于当前指令，控制器出力越大、偏差越大；卡死时偏差是 $c - \rho u_{1,k}$ 里的常数项占主导；偏置时偏差是常数 c。这就是为什么辨识要在 $\boldsymbol{b}_1$ 方向上对 $(u_1, 1)$ 两列做回归——一个回归量是“随指令变化的部分”，另一个是“不随指令变化的部分”。

第三，**偏差的方向固定、幅值未知**。方向固定意味着只要 $\boldsymbol{b}_1$ 与输出矩阵的零空间不重合，偏差就一定会体现在测量上，这是可检测性的几何基础；幅值未知意味着必须用数据估计，这是辨识存在的理由。

78.2.3 观测器与残差

检测需要一个“应当是什么”的参照。本章用 Luenberger 观测器（对偶地也可看作稳态卡尔曼滤波器），其形式为

$$\hat{\boldsymbol{x}}_{k+1} = \boldsymbol{A}\hat{\boldsymbol{x}}_k + \boldsymbol{B}\boldsymbol{u}_k^{\mathrm{obs}} + \boldsymbol{L}\left(\boldsymbol{y}_k - \hat{\boldsymbol{x}}_k \right) \tag{78-13}$$

三个状态全部可测（位置由位移传感器、速度由差分或速度传感器、备份执行器内部状态由电流环给出），所以 $\boldsymbol{C} = \boldsymbol{I}$。观测器增益 $\boldsymbol{L}$ 由对偶 Riccati 方程解出，取过程噪声 $\mathrm{diag}(10^{-8}, 10^{-6}, 10^{-6})$、量测噪声 $\mathrm{diag}(\sigma_1^2, \sigma_2^2, \sigma_3^2)$，三个通道的噪声标准差取 $\sigma = [5\times10^{-3}, 2\times10^{-2}, 1\times10^{-2}]$。解出的增益是

$$\boldsymbol{L} = \begin{bmatrix} 0.0636 & 0.0073 & 0.0001 \\ 0.1170 & 0.0372 & 0.0017 \\ 0.0003 & 0.0004 & 0.0367 \end{bmatrix} \tag{78-14}$$

矩阵的非对角元很小，说明三个通道的估计基本解耦：位置残差主要修位置估计，速度残差主要修速度估计，第三通道自己修自己。这一点在实现上很省事，但在故障时会带来一个副作用——**故障引起的残差主要落在第二行**，因为 $\boldsymbol{b}_1$ 的力作用在速度上。第 78.7 节的归一化就是为这件事准备的。

定义残差

$$\boldsymbol{r}_k = \boldsymbol{y}_k - \hat{\boldsymbol{x}}_k \tag{78-15}$$

把式 (78-11) 与式 (78-13) 相减，并注意 $\boldsymbol{y}_k = \boldsymbol{x}_k + \boldsymbol{v}_k$，得到残差的递推

$$\boldsymbol{r}_{k+1} = (\boldsymbol{A} - \boldsymbol{L})\,\boldsymbol{r}_k + \boldsymbol{b}_1 \left[(\rho - 1) u_{1,k} + c \right] - \boldsymbol{L}\boldsymbol{v}_k + \boldsymbol{v}_{k+1} \tag{78-16}$$

式 (78-16) 是本章的核心方程。它把残差分成了三部分：**衰减的记忆** $(\boldsymbol{A}-\boldsymbol{L})\boldsymbol{r}_k$、**故障的驱动** $\boldsymbol{b}_1[\cdot]$、**噪声的驱动** $-\boldsymbol{L}\boldsymbol{v}_k + \boldsymbol{v}_{k+1}$。检测要做的，就是在噪声驱动的那一项之上，把故障驱动的那一项认出来。

健康时 $\rho = 1$、$c = 0$，方括号里为零，残差只剩噪声项。故障后方括号非零，残差被“顶”起来，顶起来的幅度与 $\boldsymbol{b}_1$ 以及 $(\rho-1)u_1 + c$ 成正比。这就解释了一个在第 78.8 节会反复出现的现象：**故障越轻、指令越小，残差越接近噪声，检测越慢**。$\rho = 0.9$ 时检测要 $1.24\,\mathrm{s}$，$\rho = 0.4$ 时只要 $0.24\,\mathrm{s}$，差别不在算法，在信噪比。

78.2.4 归一化与检测统计量

残差是三维向量，三个通道的噪声标准差差了 4 倍（5×10^{-3} 对 2×10^{-2}）。如果直接取欧氏范数，噪声大的通道会主导统计量，等于把另外两个通道的信息浪费掉。所以先逐通道归一化：

$$\tilde{\boldsymbol{r}}_k = \mathrm{diag}^{-1}(\boldsymbol{\sigma})\,\boldsymbol{r}_k$$

再做 w 点因果滑动平均，压掉高频噪声，最后取范数作为检测统计量：

$$S_k = \left\| \frac{1}{w} \sum_{i=0}^{w-1} \tilde{\boldsymbol{r}}_{k-i} \right\|_2, \qquad w = 5 \tag{78-17}$$

滑动平均的窗长 $w = 5$ 对应 0.1 s。选 5 而不是 20，理由是**平均会引入延迟**：w 点平均的等效群延迟是 $(w-1)/2$ 步，$w = 5$ 时是 2 步（0.04 s），$w = 20$ 时是 9.5 步（0.19 s）。在本质不稳定对象上，检测延迟直接换算成故障后的峰值，所以这里的取舍偏向“少平均、快响应”，把噪声抑制的活儿交给阈值。

阈值按健康统计量的均值与标准差设定：

$$\theta = \mu_S + k\,\sigma_S \tag{78-18}$$

其中 μ_S 与 σ_S 由健康数据标定，k 是阈值系数。本章实测 30 组健康轨迹、每组取后 500 步，得到 $\mu_S = 0.6860$、$\sigma_S = 0.2966$。按式 (78-18) 算出的阈值与对应的单步误报率是：

k	阈值 θ	单步误报率	每 500 步期望误报次数
2	1.2791	0.03627	18.1
3	1.5756	0.00560	2.8
4	1.8722	0.00047	0.24
5	2.1688	0.00000	0
6	2.4653	0.00000	0

这张表是后面所有讨论的基准。$k = 2$ 时每 500 步会误报 18 次，系统基本上一直在“报警—复位—再报警”之间抖动；$k = 4$ 时 500 步里期望误报 0.24 次，折算到 12 s 的仿真长度上是“多数情况下不误报”；$k = 5$ 与 $k = 6$ 在 15000 个健康样本里一次都没误报。本章默认取 $k = 4$，阈值 $\theta = 1.8722$。

把阈值与信噪比放在一起看，就得到本章的第一个工程结论：**误报率与检测延迟是一根绳子的两端**。k 从 4 降到 2，误报率涨 77 倍，检测延迟只从 0.20 s 缩到 0.16 s；k 从 4 升到 6，误报率归零，检测延迟从 0.20 s 涨到 0.30 s。第 78.8 节会用闭环代价把这条曲线算完，结论是 $k \in [3, 5]$ 之间代价几乎不变，两端才明显变坏。

78.2.5 可检测性：残差往哪个方向跑

式 (78-16) 里故障项是 $\boldsymbol{b}_1[(\rho-1)u_1 + c]$。如果 $\boldsymbol{b}_1$ 恰好落在 $(\boldsymbol{A} - \boldsymbol{L})$ 的某个不变子空间里，残差就可能沿着“看不见”的方向增长，检测会失效。本章的 $\boldsymbol{b}_1$ 与 $\boldsymbol{C} = \boldsymbol{I}$ 保证了这一点不会发生——三状态全测时，任何非零输入偏差都会立刻在残差里出现。

但“能看见”与“能分辨”是两件事。要同时估出 ρ 与 c，需要一个持续的激励条件。把式 (78-16) 的故障项写成参数形式：

$$\boldsymbol{b}_1 \left[(\rho-1)u_{1,k} + c \right] = \boldsymbol{b}_1 \begin{bmatrix} u_{1,k} & 1 \end{bmatrix} \begin{bmatrix} \rho - 1 \\ c \end{bmatrix} \tag{78-19}$$

两个未知数、一个方程，单步无解。把 N 步叠起来：

$$\begin{bmatrix} \boldsymbol{b}_1 u_{1,k_0} \\ \vdots \\ \boldsymbol{b}_1 u_{1,k_0+N-1} \end{bmatrix} \begin{bmatrix} \rho - 1 \\ c \end{bmatrix} = \begin{bmatrix} \boldsymbol{d}_{k_0} \\ \vdots \\ \boldsymbol{d}_{k_0+N-1} \end{bmatrix}$$

其中 $\boldsymbol{d}_k = \boldsymbol{x}_{k+1} - \boldsymbol{A}\boldsymbol{x}_k - \boldsymbol{b}_2 u_{2,k}$ 是“用标称模型算出来的残差”。可辨识的条件是回归矩阵满秩，也就是**在这段窗口里 u_1 必须有变化**：

$$\sum_{k=k_0}^{k_0+N-1} \left(u_{1,k}-\bar{u}_1\right)^2 > 0 \tag{78-20}$$

这一条看起来是废话，工程上却经常踩。如果故障恰好发生在参考信号过零点附近、控制器输出几乎不变，那么 u_1 在窗口里近似常数，ρ 与 c 就无法分离——一组 (ρ, c) 和另一组 (ρ', c') 可以给出同样的残差。本章的实测给出了这个现象的量化版本：$\rho = 0.4$ 时正常参考幅值 0.30 下检测延迟 0.24 s，把参考幅值压到 0.02（激励几乎消失）后，检测延迟变成 1.24 s，是原来的 5 倍多。

这就是“故障诊断依赖激励”的确切含义：不是算法不够好，是数据里没有信息。工程上的对策有两个，一是保证运行轨迹有足够丰富的频率成分（第 78.9 节会提到注入探测信号），二是承认“激励不足时只能等”，并在等待期间用保守策略兜住。

78.2.6 辨识：两列最小二乘

检测到故障后，系统在 $[t_d, t_d + T_{\text{est}}]$ 这段时间里采集数据，做参数辨识。辨识窗必须整段落在故障之后，这一点是硬约束——如果窗口跨过故障时刻，前半段是健康数据、后半段是故障数据，最小二乘会把两者平均，估出来的 $\hat{\rho}$ 偏向 1，$\hat{c}$ 偏向 0，故障被判成“健康”。

把式 (78-19) 沿 $\boldsymbol{b}_1$ 方向投影，得到一个标量回归：

$$y_k = \rho\, u_{1,k} + c + \varepsilon_k, \qquad y_k = \frac{\boldsymbol{b}_1^{\mathsf{T}}\left(\boldsymbol{x}_{k+1} - \boldsymbol{A}\boldsymbol{x}_k - \boldsymbol{b}_2 u_{2,k}\right)}{\boldsymbol{b}_1^{\mathsf{T}}\boldsymbol{b}_1} \tag{78-21}$$

投影的作用是把三维的方程压成一维：既然故障只沿 $\boldsymbol{b}_1$ 一个方向作用，那就只看那个方向上的分量，其余方向上是模型误差与噪声，投影掉反而更干净。ε_k 里包含量测噪声与离散化误差。

对 N 个样本，最小二乘的解是

$$\begin{bmatrix}\hat{\rho}\\ \hat{c}\end{bmatrix} = \left(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{y}, \qquad \boldsymbol{\Phi} = \begin{bmatrix} u_{1,k_0} & 1 \\ \vdots & \vdots \\ u_{1,k_0+N-1} & 1 \end{bmatrix} \tag{78-22}$$

$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 的条件数直接反映辨识问题的病态程度。本章的实现里对它做了一次检查：条件数超过 10^{8} 就放弃本次辨识、保持原控制律不变——这是一个必要的保护，因为病态时解出的 $\hat{\rho}$ 可能落在 $[0,1]$ 之外，按它重构会得到一套荒谬的增益。正常情况下条件数远小于这个门限，实测 7 类故障都能把 (ρ, c) 估到三位小数：$\rho = 0.6$ 估成 0.600、0.4 估成 0.400、0.2 估成 0.200，卡死 $c = 2.0$ 估成 $\hat{\rho} = 0.000$、$\hat{c} = +2.000$，卡死 $c = -1.5$ 估成 $\hat{c} = -1.500$，偏置 $c = 0.8$ 估成 $\hat{\rho} = 1.000$、$\hat{c} = +0.800$，偏置 $c = -0.5$ 估成 $\hat{c} = -0.500$。健康时估成 $\hat{\rho} = 1.000$、$\hat{c} = +0.000$，与真值一致。

辨识精度这么高，原因不难理解：式 (78-21) 是一个线性回归，噪声是零均值的高斯白噪声，$N = 20$ 个样本足以把两个参数估到 10^{-3} 量级。**主动容错真正的难点不在辨识精度，而在辨识所花的时间**。$N = 20$ 对应 0.4 s，在这 0.4 s 里系统是带着故障、用着旧增益在跑的，本质不稳定的对象每多跑一步，偏差就多积累一点。第 78.3 节把这段时间称为“隔离窗”，第 78.7 节会看到它如何决定“救得回来”与“救不回来”的分界。

78.2.7 重构：把故障塞回设计模型

辨识出 $(\hat{\rho}, \hat{c})$ 之后，重构分三种情况。

效率下降。主通道还有 $\hat{\rho}$ 的力气，只是变小了。最直接的做法是把有效输入矩阵改成

$$\boldsymbol{B}_{\text{eff}} = \begin{bmatrix}\hat{\rho}\,\boldsymbol{b}_1 & \boldsymbol{b}_2\end{bmatrix} \tag{78-23}$$

然后按同一组权重重算 LQR：

$$\hat{\boldsymbol{K}} = \operatorname{dlqr}\left(\boldsymbol{A}, \boldsymbol{B}_{\text{eff}}, \boldsymbol{Q}, \boldsymbol{R}\right) \tag{78-24}$$

本章取 $\boldsymbol{Q}=\mathrm{diag}(100,10,1)$、$\boldsymbol{R}=\mathrm{diag}(0.01,0.01)$。$\hat{\rho}$ 越小，$\boldsymbol{B}_{\text{eff}}$ 的第一列越“虚”，LQR 会自动把更多权重压到第二列上，同时把第一列的增益抬高以补偿。$\hat{\rho}=0.4$ 时，重算出的增益与标称增益相比，第一行明显变大——这正是“力气小了就多喊几声”的数学表达。

卡死。主通道完全不受控，$\hat{\rho}=0$，$\boldsymbol{B}_{\text{eff}}$ 的第一列几乎为零。这时如果仍用两输入 LQR，会得到一个第一行极大、第二行也不合理的增益矩阵，因为算法在试图用一个不存在的通道去镇定系统。正确的做法是把主通道整行置零，只对备份通道做单输入 LQR：

$$\hat{\boldsymbol{K}}=\begin{bmatrix}\boldsymbol{0}\\ \boldsymbol{k}_2^{\mathsf{T}}\end{bmatrix},\qquad \boldsymbol{k}_2=\mathrm{dlqr}\left(\boldsymbol{A},\boldsymbol{b}_2,\boldsymbol{Q},R_{\text{back}}\right) \tag{78-25}$$

R_{back} 是备份通道的控力权重，本章取 0.1。这个值比标称的 0.01 大一个数量级，原因有两个：备份通道经过一阶惯性，带宽低，高增益只会带来饱和与振荡；备份通道的权限本来就紧张，必须省着用。

与“整行置零”配套的还有前馈项的改写。主通道卡死在 c 上，这个 c 是一个持续的力，必须由备份通道抵消。稳态时 $x_2=0$、$\dot{x}_2=0$，从式 (78-1) 得到备份执行器应处的平衡点

$$x_{3,\text{eq}}=-\frac{\omega^2 r-2\zeta\omega\dot{r}+g_1\hat{c}}{g_2} \tag{78-26}$$

注意分子里的三项：位置项、速度项、以及卡死常值项。前两项在健康时也有（它们是跟踪正弦所需的稳态力），第三项是故障额外加进来的。$c=2.0$ 时，$x_{3,\text{eq}}$ 的典型值在 -3.3 附近——这已经用掉了备份执行器限幅 6 的一半以上，第 78.2.8 节会说明这意味着什么。

偏置。主通道仍然可控，只是每一句指令都被加了一个常数。这种情况下不需要换增益（$\hat{\rho}\approx 1$，$\hat{\boldsymbol{K}}$ 与 $\boldsymbol{K}_0$ 几乎相同），只需要在指令上把这个常数减掉：

$$u_1=\frac{-\left(\omega^2 r-2\zeta\omega\dot{r}\right)}{g_1\hat{\rho}}-\hat{c}-\boldsymbol{K}[0,:]\,\boldsymbol{e} \tag{78-27}$$

式 (78-27) 里的 $-\hat{c}$ 就是“前馈补偿”：既然执行器会多加 c，那就先少给 c。这一项看起来简单，效果却很明显：偏置 0.8 时，不处理的故障后峰值是 0.160，主动容错是 0.083，压掉近一半；而如果错把偏置当成卡死处理（把 u_1 整行置零），备份通道要独自承担全部稳态力，故障后峰值会冲到 0.294 以上，反而更差。

三种情况对应三套动作，这个对应关系必须在代码里写清楚。本章的实现用一个三态标志（正常、效率下降、卡死、偏置）来区分，第 78.12 节会给出完整代码。

78.2.8 权限判据：救不救得回来

前面所有推导都假设“只要估得准、切得快，就能救回来”。这个假设有边界，边界由权限决定。

备份执行器能提供的最大稳态力是

$$F_{\max}=g_2 U_{2,\max}=0.6\times 6=3.6 \tag{78-28}$$

卡死故障会占去 $g_1|c|=|c|$ 的力（本章 $g_1=1$）。剩下的可用力是

$$F_{\text{rem}}=F_{\max}-g_1|c|=3.6-|c| \tag{78-29}$$

而对象是本质不稳定的，要把偏差 x_1 拉住，至少需要抵抗不稳定项 $\omega^2 x_1=1.44x_1$。于是“在偏差发展到多大之前必须完成切换”的条件是

$$1.44\,|x_1|<3.6-|c|\quad\Longrightarrow\quad |x_1|<\frac{3.6-|c|}{1.44} \tag{78-30}$$

按式 (78-30) 算三个数：$c=0.5$ 时边界是 2.15，$c=1.0$ 时是 1.81，$c=2.0$ 时是 1.11。这三条边界解释了第 78.10 节注入矩阵里的全部现象。

$c=0.5$ 与 $c=1.0$ 时，边界分别 2.15 与 1.81，远大于切换时刻的实际偏差（本章实测主动容错的故障后峰值分别是 0.294 与 0.507），所以救得回来。$c=2.0$ 时边界只有 1.11，而备份通道还需要 0.4 s 的辨识窗才能拿到 $\hat{c}$，在这 0.4 s 里偏差已经越过边界，此后无论怎么切都拉不回来——实测主动容错的故障后峰值是 6598.832，与不处理的 24131.973 同属发散，只是慢一点。

式 (78-30) 的价值在于它把“能不能容错”从一句定性判断变成了一个可以提前算的数。设计阶段拿到执行器限幅与故障谱，就可以算出哪些故障在能力范围内、哪些必须靠“预防”而不是“容错”来解决。第 78.10 节会把这条判据写进故障注入测试的判读规则里。

78.2.9 代价指标

为了横向比较不同方案，本章用一个与控制器权重解耦的二次代价：

$$J=\int_0^{t_f}\left(e^2+0.1x_2^2+0.02u_1^2+0.02u_2^2\right)\mathrm{d}t,\qquad e=x_1-r \tag{78-31}$$

注意控制器的权重是 $\boldsymbol{Q}=\mathrm{diag}(100,10,1)$、$\boldsymbol{R}=\mathrm{diag}(0.01,0.01)$，与式 (78-31) 的系数不同。这是有意为之：**控制器按自己的权重设计，评价按另一套权重打分**，否则“用 A 的标准评价 A”会天然偏向 A，比较失去意义。式 (78-31) 里位置误差的权重是 1，速度权重 0.1，两路控制量权重都是 0.02，反映的是“跟踪精度优先、能量次要”的工程偏好。

除了 J，本章还统计三个指标：故障后峰值 $\max|e|$（考核瞬态冲击与机械间隙）、故障后前 150 步的均方根（考核恢复过程）、末 100 步的均方根（考核稳态）。四个指标各有各的用途，第 78.6 节会看到它们有时给出不一致的排序——某个方案峰值最小但恢复最慢，另一个方案反之。这种不一致不是数据有问题，而是“容错”本身就是多目标问题。

三项权重与三项状态并不一一对应，这一点容易误读。式 (78-31) 的积分里有四项：位置误差、速度、u_1、u_2。它在离散实现里对应 $\boldsymbol{Q}_{\mathrm{cost}}=\mathrm{diag}(1,0.1,0)$ 与 $\boldsymbol{R}_{\mathrm{cost}}=[0.02,0.02]$。第三项的状态权重是 0，也就是**备份执行器的内部状态 x_3 不进代价**。这不是漏写。x_3 是执行器的内部变量，不是工艺量，它偏大偏小本身不构成“性能差”；它的影响已经通过 g_2x_3 体现在速度与位置上了，再计一次等于重复惩罚。更实际的原因是：如果把 x_3 计进代价，那么“多用备份通道”这个动作本身就会被扣分，容错方案会被系统性地判差——而这恰恰是容错控制要鼓励的动作。

两路控制量的权重相同，也是有意的。$\boldsymbol{R}_{\mathrm{cost}}$ 的两个分量都是 0.02，不因为备份通道“平时不用”就给它更小的权重。原因是评价要回答的问题是“这套方案总共花了多少执行器代价”，而不是“这套方案是否公平地使用了两个执行器”。如果给备份通道更低的权重，主动容错会因为“它敢用备份通道”而自动占便宜，横比就失去意义。**评价函数的中立性，比评价函数的精细度更重要。**

位置误差用 e 而不是 x_1。这条看似琐碎，实则决定了所有数字的量级。若用 x_1，跟踪一条幅值 0.30 的正弦，即使完美跟踪，代价里也含有 $\int x_1^2\mathrm{d}t\approx 0.3^2\times 12/2=0.54$ 的“参考本身的能量”，方案之间的差异会被这个大数淹没。用 $e=x_1-r$ 之后，健康工况的 J 只有 0.0475，故障方案是 0.81 或 0.23，差异一目了然。**评价指标的偏移量选择，决定了它能不能分辨差异**，这是所有横比实验都要先解决的事。

78.3 设计步骤

把前面的公式串起来，一套主动容错控制器可以按七步做出来。每一步都有明确的输入与输出，做完一步就能验证一步，不必等到最后才发现哪里不对。

这七步的顺序不是随意排的，而是按“依赖关系”排的：后一步用到的量必须由前一步产出。第一步给出模型与两套增益，第二步给出阈值，第三步给出“容错能力到哪为止”的判据，第四步给出“越过判据之后怎么改结构”，第五步给出“改结构需要多久”，第六步给出“怎么改才不震”，第七步给出“改完到底行不行”。任何一步想跳过，都会在后面某一步变成无法解释的数字。实际工程里最常见的失败不是算法写错，而是第二步的阈值与第五步的延迟分别按不同工况标定，最后在第七步发现故障谱中间那一段怎么调都过不了——回头一看，是两个环节的时间基准不一致。

七步的时间成本也值得先说清楚：前四步是纸面工作，不需要跑仿真；后三步必须跑闭环。所以真正决定“这套方案能不能用”的是第三步的边界算得对不对，而不是第七步的曲线漂不漂亮。第三步算错方向，第七步只会跑出一堆看起来收敛、实

际靠运气的结果。

78.3.1 第一步：标称模型与观测器

输入是对象的物理参数与采样周期，输出是离散矩阵 $\boldsymbol{A},\boldsymbol{B}$、标称增益 $\boldsymbol{K}_0$ 与观测器增益 $\boldsymbol{L}$。

这一步与常规状态反馈设计没有区别：按式 (78-9) 做零阶保持离散化，按 $\boldsymbol{Q}$、$\boldsymbol{R}$ 解离散 Riccati 方程得到 $\boldsymbol{K}_0$，按噪声强度解对偶 Riccati 方程得到 $\boldsymbol{L}$。本章的结果是式 (78-7) 与式 (78-14)。

离散矩阵要能与物理参数对上。这不是形式检查，而是排查符号错误最有效的手段。本章的 $\boldsymbol{A}$ 第一行是 $[1.000287,\ 0.019930,\ 0.000115]$，可以逐项用手算核对：$(1,1)$ 元约等于 $1+\omega^2 T_s^2/2 = 1+1.44\times 4\times 10^{-4}/2 = 1.000288$；$(1,2)$ 元约等于 $T_s - 2\zeta\omega T_s^2/2 = 0.02 - 0.36\times 4\times 10^{-4}/2$，落在 0.01993；$(1,3)$ 元约等于 $g_2 T_s^2/2 = 0.6\times 4\times 10^{-4}/2 = 1.2\times 10^{-4}$，与 0.000115 同量级。第二行的 $(2,1)$ 元是 $\omega^2 T_s = 0.0288$ 减去阻尼修正后的 0.028699。第三行只有 $(3,3)$ 元非零，它是 $e^{-T_s/\tau} = e^{-0.1333} = 0.875173$——这个数一定要单独核，因为它是唯一一个由时间常数直接决定的元，符号或单位错一位就会立刻显形。

离散极点要与连续极点按 $z = e^{sT_s}$ 对上。连续域的三个特征值是 $+1.033$、-1.393 与 -6.667，映射到离散域应分别是 $e^{1.033\times 0.02} = 1.0209$、$e^{-1.393\times 0.02} = 0.9725$、$e^{-6.667\times 0.02} = 0.8752$；本章实算的 $\mathrm{eig}(\boldsymbol{A})$ 是 1.020884、0.972516、0.875173，四位小数完全吻合。这一步检查的价值在于：如果离散化时把 $\boldsymbol{B}$ 的增广块放错位置，$\boldsymbol{A}$ 往往仍然是对的，极点检查通不出问题，但 $\boldsymbol{B}$ 会错，而 $\boldsymbol{B}$ 错了之后所有重构增益都会错。

两列 $\boldsymbol{B}$ 的形状决定了后面所有的容错策略。本章 $\boldsymbol{b}_1 = [1.9953\times 10^{-4},\ 1.9930\times 10^{-2},\ 0]^{\mathsf{T}}$，$\boldsymbol{b}_2 = [5.1510\times 10^{-6},\ 7.6378\times 10^{-4},\ 1.2483\times 10^{-1}]^{\mathsf{T}}$。第一列的第二分量是 1.9930×10^{-2}，第二列的第二分量只有 7.6378×10^{-4}，两者差 26.1 倍；第二列的第二分量与第一列的第二分量之比只有 3.8%。这说明同一拍里主通道对加速度的作用是备份通道的二十六倍。但反过来看第三行，$\boldsymbol{b}_2$ 的第三分量是 0.1248，$\boldsymbol{b}_1$ 的第三分量是 0——备份通道真正的作用点不在“当拍推力”，而在“把 x_3 这个内部状态推起来”，再经 $g_2 = 0.6$ 折算成力。这就解释了为什么备份通道的稳态能力反而是主通道的 72%：稳态时 x_3 可以积累到接近 u_2，$0.6\times 6 = 3.6$，而主通道被限在 $1.0\times 5 = 5.0$。**主通道快而直接，备份通道慢而有力**，这八个字是后面每一条重构规则的物理依据。

观测器增益要检查噪声放大。本章 $\boldsymbol{L}$ 的三行是 $[0.0636,\ 0.0073,\ 0.0001]$、$[0.1170,\ 0.0372,\ 0.0017]$、$[0.0003,\ 0.0004,\ 0.0367]$。第一列（对应位置量测）的系数最大，第二列次之，第三列最小，这与三个量测通道的噪声标准差 5×10^{-3}、2×10^{-2}、10^{-2} 的方向一致：噪声越小的通道，观测器越敢用。若某一列的系数与噪声水平反着来，多半是 $\boldsymbol{R}_v$ 的对角元写反了。

有一个容易被忽略的检查项：**标称闭环的稳定裕度**。容错控制要在故障后切换到一套新增益，新增益的稳定裕度通常比标称的差（因为有效输入矩阵“变虚”了）。如果标称闭环本身只有很小的裕度，切换后几乎没有余量可用。本章标称闭环的极点是 0.9385、0.5371、0.2848，谱半径 0.9385。这个数看着离 1 不远，但换算成时间常数是 $T_s/|\ln \rho_{\mathrm{cl}}| = 0.02/0.0635 = 0.315$ s，也就是说偏差每 0.315 s 衰减到 37%，五十步（1 s）衰减到 4.2%。对本章 12 s 的仿真时长来说，这个收敛速度够用；但如果参考信号周期只有 2 s，0.315 s 的时间常数就会显得拖沓。**谱半径不是越小越好，它要与参考信号的带宽匹配**，这一点在第 78.6 节会用四套增益的横比再说明一次。

78.3.2 第二步：健康残差标定

输入是健康工况下的若干条轨迹，输出是 μ_S、σ_S 与阈值 θ。

这一步是整个设计中唯一“必须用数据”的环节，也是最容易做得草率的一步。需要注意三点。

其一，**标定数据要覆盖工作范围**。如果标定只在参考幅值 0.30 下做，而实际运行会到 0.10，那么标定出的 σ_S 偏大（因为大幅值下控制量大、残差大），阈值会偏高，轻故障漏报。本章的标定用 30 条不同随机种子的轨迹，参考幅值固定 0.30，与后续实验条件一致。

其二，**要丢掉起始瞬态**。仿真从零状态起步，前 100 步有起幅过程，残差统计量与稳态不同。本章取每条轨迹的第 100 步之后共 500 步参与统计，30 条合计 15000 个样本，得到 $\mu_S = 0.6860$、$\sigma_S = 0.2966$。

其三，**要同时记录误报率曲线**，而不只是记一个阈值。下面的 k–误报率对照表，是后面调参时判断“能不能再降一点”的依据。只记一个数，等于放弃了这条信息。

系数 k	阈值 θ	误报步数 / 15000	误报率
2	1.2791	544	0.03627
3	1.5756	84	0.00560
4	1.8722	7	0.00047
5	2.1688	0	0
6	2.4653	0	0

这张表最值得看的是最后两行：$k=5$ 与 $k=6$ 的误报都是 0。但“0”不等于“不会有”，它只说明在 15000 个样本里没发生。按“零事件”的置信上限估计，15000 个样本零误报对应的误报率上界大致是 $3/15000 = 2\times10^{-4}$。所以 k 从 5 再加到 6 时，阈值从 2.1688 抬到 2.4653，抬了 13.7%，换来的是“实际误报率可能低一点”这个无法验证的承诺——代价却是检测延迟实实在在变长（第 78.8 节实测从 0.28 s 变到 0.30 s）。**在零误报区间里继续加 k 是在为统计噪声付费**。

本章最终取 $k=4$，阈值 1.8722、误报率 0.00047。这个选择看着“不那么干净”——毕竟 $k=5$ 能做到零误报——但零误报的代价是延迟，而延迟在本对象上是硬约束（第 78.2.8 节的权限边界只有 1.11 到 2.15）。$k=4$ 每 2000 步大约误报一次，配合“报一次警只做一次辨识、判不通过就维持原状态”的保守逻辑，误报的实际后果是“多算了一次，什么也没改”，可以接受。**阈值不是选出来的最优解，是选出来的可交付解**，这条在第 78.8 节还会展开。

其四，**标定集与验证集要分开**。用同一批轨迹既算 μ_S、σ_S 又统计误报率，得到的误报率一定偏乐观，因为阈值就是被这批数据的统计量“定制”出来的。正确做法是：用一部分种子算统计量，再用另一部分种子算误报率。本章的标定用了 30 条轨迹，误报统计在健康工况的长跑里单独做（表里的 15000 步来自标定集；第 78.6 节的方案横比用的是另一条 12 s 的轨迹，两者不重叠）。

其五，**参考幅值必须与运行工况一致**。残差的统计量不是对象的固有属性，它随工况走：参考幅值变了，控制量跟着变，残差的均值与方差都跟着变，同一个阈值在另一个幅值下的含义完全不同。本章有一组对比可以说明这个量级：参考幅值 0.30 时，$\rho=0.9$ 的轻故障在这条 12 s 轨迹里一次都没能顶穿阈值；把参考幅值压到 0.02 后，同一条故障要 1.24 s 才被检出。前者不是“更灵敏”，后者也不是“更迟钝”——两者的噪声实现、起始瞬态、阈值相对残差的裕度都不同，**跨幅值比较检测延迟本身就没有意义**。

真正有意义的区分在故障是乘性还是加性。效率下降是乘性的，实际作用量是 ρu_1，偏差是 $(\rho-1)u_1$，指令小了偏差也小；偏置与卡死是加性的，实际作用量是 u_1+c 或 c，偏差与指令无关。所以在小指令工况下，加性故障的相对可见度更高，乘性故障更低。标定集如果不覆盖运行时的幅值范围，这两类故障的检测性能就会在标定工况之外悄悄劣化，而误报率又恰好在标定工况下最好看。**这条区别决定了标定集必须覆盖运行幅值范围，而不能只取一个典型值**。

输出怎么验：拿一批没参与标定的健康轨迹过一遍检测器，误报率应当落在表里的区间内；再把 $\rho=0.9$ 的轻故障注进去，若一次都检不出，说明阈值偏松或窗长太短，与第二步的 σ_S 估计没有关系——这一点要分清楚，否则会去反复改阈值，越改越糟。

78.3.3 第三步：故障谱与权限边界

输入是执行器限幅、故障类型清单，输出是“哪些故障在容错能力之内”的判据。

这一步用式 (78-30)。先算备份通道的最大稳态力 $F_{\max}=g_2U_{2,\max}$，再按故障谱里最严重的卡死值 $c_{\max}$ 算剩余力，最后算出容许的最大偏差。三个数一摆，设计者立刻知道：本平台能容 $c\le1.0$ 的卡死，容不了 $c=2.0$ 的卡死。

如果算出来的边界太小、不能接受，有两条路可走：加大备份执行器的限幅或增益（硬件改动），或者缩短辨识窗（软件改动，代价是辨识方差变大）。本章的辨识窗取 0.4 s，比最初的 0.8 s 短一半，正是为了让 $c=1.0$ 的卡死能落在边界之内——实测 0.8 s 窗下 $c=1.0$ 的故障后峰值是 2577.838（发散），改成 0.4 s 后降到 0.507（收敛）。**同一个算法、同一个故障，只改一个窗长，结果从发散变成收敛**，这就是第三步的意义：在写代码之前先把边界算清楚。

这个判据是必要条件，不是充分条件。式 (78-30) 只比较了“剩余力”与“不稳定项”，隐含两个假设：备份通道能立刻出满力，以及故障发生在偏差为零的时刻。两个假设在真实系统里都不成立。

第一个假设的问题出在惯性环节。备份通道要经过 $1/(\tau s+1)$ 才把 u_2 变成作用力，$\tau=0.15$ s。要在 0.15 s 内把 x_3 从 0 推到 u_2 的 63%，前提是这期间 u_2 一直给到限幅；要在 0.45 s（三个时间常数）内推到 95%，才能说“稳态力”真的到手。而偏差在这 0.45 s 里不是站着不动的，它按开环增长率 1.033 /s 往上走，0.45 s 涨 $e^{0.465}=1.59$ 倍。所以式 (78-30) 解出的边界应当再除以一个 1.6 左右的安全系数才敢用。本章 $c=1.0$ 的边界 1.81 除以 1.6 是 1.13，而实测故障后峰值 0.507 仍在 1.13 之内，说明这个保守化没有保守过头。

第二个假设的问题出在故障时刻的相位。本章的故障固定在 $t=2.0$ s 注入，这个时刻参考信号刚起步不久（起幅时间常数 0.8 s，$t=2.0$ s 时包络已到 $1-e^{-2.5}=91.8\%$），偏差与速度都不大。如果故障发生在参考信号的峰值附近，速度项 x_2 大，同样的 c 会推出更大的峰值。**故障注入测试不能只在一个相位上做**，第 78.10 节能看到，本章的注入矩阵只扫了故障幅值、没扫相位，这是本章验证工作的一个已知缺口；工程上补这个缺口不需要新算法，只需要把注入时刻从 2.0 s 改成一组分散的值重跑。

第三个容易被忽略的量是**切换前的偏差是否已经在增长**。式 (78-30) 给的是“偏差发展到多大之前必须完成切换”，而偏差在检测与辨识这两段时间里一直在长。本章实测的检测延迟是 0.04 到 0.30 s，辨识窗固定 0.4 s，两者相加是 0.44 到 0.70 s。按 1.033 /s 粗估，这段真空期里偏差涨 $e^{0.45}$ 到 $e^{0.72}$，即 1.57 到 2.06 倍。$c=1.0$ 时若切换瞬间偏差是 0.25，真空期末尾就是 0.51，离边界 1.81 还有三倍余量；$c=2.0$ 时边界只有 1.11，同样的增长倍数下起点必须低于 0.54，而卡死 2.0 的故障会让偏差在检测延迟内就冲出这个范围。**同样的算法、同样的窗长，$c=1.0$ 有富余而 $c=2.0$ 没有，差别全在这一条边界上。**

78.3.4 第四步：重构规则

输入是故障类型，输出是新增益与前馈项。

这一步对应式 (78-23) 至式 (78-27)。实现上建议做成一张表，每行一个故障类型，列出“增益怎么算、前馈怎么改、哪一路输入要置零”：

故障类型	判据	新增益	前馈/平衡点	输入处理
健康	$\hat\rho>0.88$，$\lvert\hat c\rvert\le 0.35$	$\boldsymbol{K}_0$	标称前馈	两路正常
效率下降	$0.12\le\hat\rho\le 0.88$	$\mathrm{dlqr}(\boldsymbol{A},[\hat\rho\boldsymbol{b}_1,\boldsymbol{b}_2],\boldsymbol{Q},\boldsymbol{R})$	前馈除以 $\hat\rho$	两路正常
卡死	$\hat\rho<0.12$	主通道整行置零，备份通道单输入 LQR	式 (78-26) 的 $x_{3,\mathrm{eq}}$	主通道置零
偏置	$\hat\rho>0.80$，$\lvert\hat c\rvert>0.35$	$\boldsymbol{K}_0$	前馈减 $\hat c$	两路正常

这张表里最容易被写错的是“卡死”那一行的输入处理。主通道卡死时，控制器给出的 u_1 完全不起作用，但如果不显式置零，控制律仍会算出一个很大的 u_1（因为它以为自己在出力），这个数会通过前馈项和观测器影响其他量。显式置零有两个好处：一是让控制量的记录与实际作用量一致，二是避免前馈项里的 $1/\hat\rho$ 在 $\hat\rho\to 0$ 时爆掉。

四行分别对应四种结构，不是四组参数。健康行与偏置行的增益都是 $\boldsymbol{K}_0$，看起来像是“没做事”，但两者的前馈项不同；效率下降行的增益是重解的，控制结构没变；卡死行连输入维度都变了，是真正的结构重构。这四种情况在代码里应当走四个分支，而不是用一个带权系数的公式统一算。原因是 $\hat\rho\to 0$ 时 $\boldsymbol{B}_{\mathrm{eff}}$ 的第一列趋于零矩阵，Riccati 方程的条件数发散，解出来的增益在数值上不可信；即使强行算出来，前馈项里的 $1/\hat\rho$ 也会把控制量推到限幅之外。**用“连续过渡”的方式处理从效率下降到卡死的过程，在数学上是连续化一个奇点，在工程上是给自己埋雷。**

卡死那一行的具体增益值得单独看。主通道整行置零后，剩下的控制问题是单输入（只有 u_2）三状态，控制权重取 $\boldsymbol{R}_{\mathrm{back}}=0.1$——比标称的 0.01 大十倍，意思是“备份通道更贵，少用一点”。解出的增益是 $\boldsymbol{k}_{\mathrm{back}}=[34.9505,\ 19.2561,\ 2.3382]$，闭环极点是 0.6456 与 $0.9581\pm 0.0226j$，谱半径 0.9584。这个谱半径比标称的 0.9385 更靠近单位圆，符合“能力下降后性能必然变差”的预期。值得注意的是那一对共轭极点：单输入控制三状态时，有两个模态要靠同一条通道压制，很容易出现振铃。把 $0.9581\pm 0.0226j$ 换算成连续域的阻尼比：模长 0.9584，

$|\ln 0.9584| = 0.04249$，辐角 0.0236 rad，阻尼比是 $0.04249/\sqrt{0.04249^2 + 0.0236^2} = 0.874$。阻尼比 0.87 并不差，**振铃不是这套增益的主要问题**；卡死分支真正的问题是慢——谱半径 0.9584 对应时间常数 $0.02/0.0425 = 0.47$ s，比标称的 0.32 s 慢了近五成。图 78-6 里主动容错在卡死情形下的波形之所以显得“拖”，拖的是这个时间常数，不是阻尼。

观测器的输入也要跟着改，这是最容易漏的一步。检测器用的残差由式 (78-16) 生成，而式里的 $\boldsymbol{u}^{\mathrm{obs}}$ 是“观测器以为的实际作用量”。故障发生后，它不再等于控制器给出的 $\boldsymbol{u}$：卡死时应当喂 $[\hat{c},\ u_2]$，偏置时应当喂 $[u_1 + \hat{c},\ u_2]$，效率下降时应当喂 $[\hat{\rho}u_1,\ u_2]$。如果不改，观测器会一直用一个错误的输入做预测，残差里会残留一个与故障同量级的稳态偏移；这个偏移不会被检测逻辑重新判读（因为已经报过警了），但会污染观测器状态，让重构后的控制律基于错误的 $\hat{\boldsymbol{x}}$ 出力。实测的表现是切换后偏差不收敛、缓慢爬升，看起来像“新增益不够强”，实际是状态估计偏了。

前馈项的三处修改要分清对象。效率下降是“除以 $\hat{\rho}$”，卡死是“换成平衡点 $x_{3,\mathrm{eq}}$”，偏置是“减 $\hat{c}$”。三者不要混用：给效率下降套平衡点公式，会把一个本来能靠主通道补回来的偏差硬塞给备份通道；给偏置套 $1/\hat{\rho}$，会在 $\hat{\rho} \approx 1$ 时几乎不起作用，偏置原封不动地留在系统里。本章的分类边界（$\hat{\rho} < 0.12$ 为卡死、$\hat{\rho} < 0.88$ 为效率下降、$|\hat{c}| > 0.35$ 且 $\hat{\rho} > 0.80$ 为偏置）就是为了让这四种处理互不重叠、也没有空档。

重构的频率要限制。如果每一拍都根据当前的 $\hat{\rho}$、$\hat{c}$ 重解一次 Riccati 方程，计算量不说，增益会随辨识噪声抖动，闭环变成一个时变系统，稳定性分析全部失效。本章的做法是“一次辨识、一次重构、之后锁死”，除非残差再次越过阈值才重新辨识。这个“锁死”策略有代价：故障若继续恶化（效率从 0.4 掉到 0.2），系统不会自动跟进，需要二次报警触发。第 78.9 节会讨论用无扰过渡与多模型方法改进这一点，但在基础版本里，宁可漏跟一次也不要每拍都动。

78.3.5 第五步：窗长与延迟

输入是噪声水平与对象的不稳定程度，输出是滑动平均窗长 w、辨识窗长 N、切换延迟 d。

三个参数各管一段：w 管检测的平滑程度，N 管辨识的精度，d 管“决策到执行”的等待。三者的共同点是**都在消耗时间，而时间在这个对象上是稀缺资源**。

本章的取值是 $w = 5$（0.1 s）、$N = 20$（0.4 s）、$d = 0$（立即执行）。$d = 0$ 不是偷懒，而是因为本章的仿真里没有计算延迟与安全联锁；实际系统中 d 至少要覆盖一次控制周期加一次通信周期，第 78.10 节会讨论。d 加大后的代价在第 78.6 节有完整的扫描：d 从 0 加到 0.8 s，闭环代价从 0.2344 涨到 0.3144；加到 2.4 s 时涨到 0.4726；加到 4.0 s 时是 0.4809，已经接近“完全不切换”的 0.8171 的一半以上。

w 的取值是“延迟与平滑”的直接兑换。w 点因果滑动平均的群延迟是 $(w-1)T_s/2$，$w = 5$ 时是 0.04 s；噪声标准差被压到原来的 $1/\sqrt{w} = 1/2.24 = 44.7\%$。如果把 w 加到 9，群延迟变 0.08 s，噪声压到 33.3%——多花 0.04 s 只换来噪声再降一成，而阈值可以少加一点点。这笔交易在本平台上不划算，因为 0.04 s 的延迟作用在增长率 1.033 /s 的偏差上，等于多放 4.2% 的幅度，而噪声那一成的改善在 $k = 5$ 已经是零误报的前提下毫无意义。**当误报率已经降到零，继续加窗只买到延迟**。

w 还有一条下限不能破：$w \geq 3$。$w = 1$ 等价于不平滑，残差直接与阈值比，实测 $k = 2$ 时误报率 0.03627 会进一步恶化；$w = 2$ 的群延迟只有 0.01 s，但方差只降 29%，起不到“把噪声从故障信号里分出来”的作用。本章取 5，是在“至少三倍方差压缩”与“不超过两个控制周期的延迟”之间取的中间值。

N 是被故障谱逼出来的，不是自由参数。辨识要解的是两列最小二乘：$\boldsymbol{b}_1$ 方向投影后对 $(u_1, 1)$ 两列回归，未知数两个（ρ 与 c 的可辨识组合），方程数等于窗长 N。从纯估计理论看，N 越大方差越小，$N = 20$ 与 $N = 40$ 的方差差一倍。但 N 同时在消耗权限边界里的时间预算，而 $c = 1.0$ 这条故障的预算总共只有 1.81 的偏差额度。本章最初用 $N = 40$（0.8 s），实测 $c = 1.0$ 的故障后峰值 2577.838，完全发散；改成 $N = 20$ 后降到 0.507。**N 的选择不是“精度够不够”，而是“精度换来的时间是否还在边界之内”**，这一点与常规的系统辨识思路正好相反：常规辨识追求参数精度，容错辨识追求“在对象飞走之前拿到一个够用的参数”。

N 也不能太小。$N = 10$（0.2 s）时方程数等于未知数，最小二乘退化成一个精确解，噪声被原封不动地搬进 $\hat{\rho}$ 与 $\hat{c}$；具体表现是同一个故障在不同噪声实现下被判成不同类型，第 78.8 节的七类故障辨识表就不再成立。经验下限是 $N \geq 3$ 倍未知数个数，本章的未知数在两个左右，所以 $N \geq 6$ 是硬下限，$N = 20$ 留了三倍余量。

d 在仿真里取零，在实机上不可能取零。d 的物理含义是“决策算完到新增益真正生效”的等待，它至少包含三部分：控制周期（本章 0.02 s）、通信周期（现场总线上一到两拍）、安全联锁确认（通常要求连续两拍一致，再 0.04 s）。所以实机上 d 的合理下限是 0.08 s 左右，按整步取就是四到五步。对照第 78.6 节的扫描，$d = 0.1$ s（五步）对应的代价是 0.2420，比 $d = 0$ 的 0.2344 高 3.2%——这个代价是可以接受的。真正危险的是 d 到 1.6 s 以上（八十步，代价 0.4112）甚至 4.0 s（两百步，0.4809），后者已经是“完全不切换”的 0.8171 的 59%，主动容错退化成了一个只是“少亏一点”的方案。**d 的预算应当从“通信与联锁的真实耗时”倒推，而不是从“我想多快”正推。**

三个参数之间还有一条隐性约束：w 的群延迟必须小于 d。否则检测结果还没稳定，切换动作已经执行，等于用一个抖动的判决去改结构。本章 $w = 5$ 的群延迟 0.04 s 与实机 $d = 0.08$ s 的关系满足这条约束，但如果为了压噪声把 w 加到 9（群延迟 0.08 s），就必须把 d 同步抬到 0.10 s 以上。

78.3.6 第六步：切换逻辑

输入是新增益，输出是一段可以安全执行的控制律切换。

最朴素的做法是硬切换：检测到故障、算好增益，下一拍直接换。本章实测硬切换的闭环代价 0.2329，故障后峰值 0.0541，切换瞬间 u_1 的最大跳变 1.024。

跳变 1.024 相对于 u_1 的正常幅值（健康时峰值 0.66）不算小，对执行器是一个冲击。改进的办法是淡入：在 T_b 秒内把增益从旧值线性过渡到新值，

$$\boldsymbol{K}_j = (1-\alpha_j)\boldsymbol{K}_{\text{from}} + \alpha_j \boldsymbol{K}_{\text{to}}, \qquad \alpha_j = \min\left(1, \frac{j\,T_s}{T_b}\right) \tag{78-32}$$

本章实测 $T_b = 1.0$ s 时，跳变从 1.024 降到 0.880，代价从 0.2329 涨到 0.2339——用 0.4% 的性能换取 14% 的冲击下降，这笔交易在多数场合是划算的。第 78.10 节会结合图 78-7 细看切换前后的波形。

淡入长度不是越长越好。本章扫了四个取值，$T_b = 0$（硬切换）、0.2 s、0.5 s、1.0 s，对应的代价是 0.2329、0.2333、0.2337、0.2339，跳变是 1.024、0.892、0.883、0.880。这组数说明两件事。第一，代价随 T_b 单调上升，但幅度极小，从 0 到 1.0 s 只涨 0.43%——淡入几乎不要钱。第二，**跳变的下降集中在前 0.2 s**：$T_b = 0.2$ s 就吃掉了从 1.024 到 0.892 的绝大部分，再往上加只从 0.892 挪到 0.880，边际收益趋零。所以合理的取值在 0.2 到 0.5 s 之间，本章的 1.0 s 偏保守，是为了让图 78-7 的过渡段看得清楚。

跳变的机理值得说清：硬切换瞬间，新旧两套增益对同一个状态 $\hat{\boldsymbol{x}}$ 给出两个不同的 u_1，差值就是跳变。这个差值与 $\hat{\boldsymbol{x}}$ 成正比，而切换时刻 $\hat{\boldsymbol{x}}$ 恰好因为故障而偏离标称值，所以跳变最大的时刻正是最不该有冲击的时刻。淡入把这个差值摊到 T_b 内的若干拍上，每拍的增量从 1.024 降到 $1.024 \times T_s/T_b$ 量级。**这就是“用时间换冲击”最朴素的一个例子**：不改变任何增益、不改变任何判据，只是把同一件事分几步做完。

不要用指数淡入。指数淡入 $\alpha_j = 1 - e^{-jT_s/T_b}$ 在起点斜率最大，跳变几乎全集中在第一拍，等于把硬切换的冲击削掉一半就没了后劲；线性淡入的斜率恒定，冲击被均匀分摊。更不能用“指数趋近”，因为它永远到不了 1，控制律始终混着一点旧增益，闭环极点无法用任何一套增益的极点来界定，稳定性分析失去依据。

切换时不要重置观测器。一个很自然的想法是“既然故障变了，观测器状态也该重新初始化”。这是错的。观测器状态 $\hat{\boldsymbol{x}}$ 是连续的物理量估计，它不因为执行器故障而跳变；重置它等于在控制律里人为制造一个阶跃，效果比不淡入还差。真正需要重置的只有“观测器用的输入”（见第 78.3.4 节），而那是换一个信号源，不是改状态。

多处修改要有固定顺序。一次切换涉及四件事：换增益、换前馈、改观测器输入、可能置零某一路。四件事如果在同一拍内以不同顺序执行，中间态会被记录进日志，也可能被其他逻辑读到。本章的做法是先在局部变量里算好全部新值，再一次性提交，提交的边界就是控制周期的边界。这个习惯在仿真里看不出差别，在实机上决定了日志能不能复盘。

78.3.7 第七步：验证

输入是整套控制器，输出是“能不能交付”的结论。

验证至少包含三件事：**故障注入测试**（把故障谱里每一类故障、每一个幅值都注入一次，看闭环是否收敛）、**误报统计**（在健康工况下长时间跑，统计误报次数）、**边界扫描**（把故障幅值从 0 扫到超出权限，找到实际的可容错边界，与第三步算出的理论边界对照）。第 78.10 节会给出本章的注入矩阵，它的形状就是这三件事的合并结果。

三件事都需要一个明确的**通过判据**，否则“看曲线觉得还行”会变成唯一的结论。本章用的是双判据：故障后峰值 $\max|e| < 1.0$ 且末 100 步均方根 < 0.05。前者管“别撞坏”，后者管“真的稳住了”。两条缺一不可，因为只卡峰值会放过“有界但不收敛”的情形，只卡末段会放过“收敛前先冲出去”的情形。实测这两种情形在本平台上都出现过：$c = 2.0$ 的卡死在主动容错下的峰值是 6598.832，两条都不过；而 $\rho = 0.2$ 的效率下降在主动容错下并没有发散，闭环代价却是 0.9359，是 $\rho = 0.4$ 时 0.2337 的四倍，属于“稳住了但精度已经掉下去”，只看“有没有发散”就会把它判成通过。

第三个判据是**重复性**。同一个工况换三个随机种子跑，三条轨迹的结论必须一致，否则说明结果依赖某一次噪声实现，不能交付。本章的十八格注入矩阵就是按“三个种子全过才算过”统计的，这也是为什么矩阵里会出现“部分通过”这种中间状态——不是判据模糊，而是三种子之间不一致，这本身就是需要排查的信号。

边界扫描要双向做。从 0 往大扫能找出“实际能容到哪”，从大往小扫能找出“哪一段是假通过”。本章实测的实际边界与理论边界（$c = 0.5$ 对应 2.15、$c = 1.0$ 对应 1.81、$c = 2.0$ 对应 1.11）在方向上一致：$c = 0.5$ 与 $c = 1.0$ 都能救回来（峰值 0.294 与 0.507），$c = 2.0$ 救不回来（峰值 6598.832）。数值上实际边界比理论值紧，原因就是第 78.3.3 节讲的“备份通道的惯性延迟”与“故障相位”两条。**理论边界给方向，实测边界给数字，两者都要有，缺一个就无法判断偏差是来自模型还是来自实现。**

图 78-2 把七步里涉及时间的那部分画成了一条时间轴：故障时刻 t_f、检测时刻 t_d、隔离完成时刻 t_i、切换时刻 t_s。三段矩形分别是检测延迟、辨识窗、决策与执行。下方是误差包络的示意：不处理的轨迹在 t_s 之后继续指数上升，主动容错的轨迹在 t_s 之后转入衰减。



图 78-2 主动容错的三段时间线：检测延迟、辨识窗、决策与执行，以及两条误差包络的对比

一个常见的设计错误是把这三段时间当成可以独立优化的三件事。实际上它们互相牵制：把检测阈值调低能缩短第一段，但误报率上升；把辨识窗缩短能缩短第二段，但参数方差变大、可能判错故障类型；把切换延迟设为零能缩短第三段，但没有时间做安全联锁。**七步设计的核心不是把每一段都压到最短，而是让三段之和落在权限边界之内。**

78.3.8 七步的交付物

七步走完，手中应当留下七份可以独立复述的东西。它们不是文档形式主义，而是“下一个接手的人能不能在不问你任何问题的前提下重跑一遍”的最低标准。

步骤	交付物	关键数字
一	离散模型、两套增益、观测器增益	$\mathrm{eig}(\boldsymbol{A}) = 1.020884,\ 0.972516,\ 0.875173$
二	健康残差统计与 k–误报率表	$\mu_S = 0.6860$、$\sigma_S = 0.2966$、$\theta = 2.1688$
三	故障谱与权限边界	$c = 0.5/1.0/2.0 \to 2.15/1.81/1.11$
四	四种故障的处理表与新增益	$\boldsymbol{k}_{\mathrm{back}} = [34.9505,\ 19.2561,\ 2.3382]$
五	三个时间参数的取值与依据	$w = 5$、$N = 20$、$d = 0$
六	切换方式与淡入长度	$T_b = 1.0\,\mathrm{s}$，跳变 $1.024 \to 0.880$
七	注入矩阵、误报统计、边界扫描	十八格，三种子全过为通过

这七份东西里最有价值的是第三份和第五份，因为它们最难事后补。模型错了可以重算，增益错了可以重解，但“为什么把辨识窗定在 0.4 s”这件事，只在当时知道边界是 1.81、峰值是 0.507、旧窗长下是 2577.838 的时候才说得清；半年后回头看代码里一个 `EST_WIN = 20`，谁也不知道它是不是随手写的。**容错控制的可维护性，几乎全部取决于这些“为什么”有没有被记下来。**

78.4 连续情形

78.4.1 连续时间观测器与残差

实际控制器是数字的，但理解容错机理用连续时间更方便。把式 (78-13) 写成连续形式：

$$\dot{\hat{\boldsymbol{x}}} = \boldsymbol{A}_c\hat{\boldsymbol{x}} + \boldsymbol{B}_c\boldsymbol{u}^{\mathrm{obs}} + \boldsymbol{L}_c\left(\boldsymbol{y} - \hat{\boldsymbol{x}}\right) \tag{78-33}$$

残差的动态是

$$\dot{\boldsymbol{r}} = \left(\boldsymbol{A}_c - \boldsymbol{L}_c\right)\boldsymbol{r} + \boldsymbol{b}_{1c}\left[(\rho-1)u_1 + c\right] - \boldsymbol{L}_c\boldsymbol{v} + \dot{\boldsymbol{v}} \tag{78-34}$$

与离散版本式 (78-16) 相比，结构完全一样，只是“衰减记忆”从矩阵幂变成了矩阵指数。$\boldsymbol{A}_c - \boldsymbol{L}_c$ 的特征值决定了残差对故障的响应速度：特征值越靠左，残差越快跟上故障，但噪声放大越厉害。

本章的 $\boldsymbol{A}_c$ 有三个实特征值 $+1.033$、-1.393 与 -6.667。观测器必须比对象快，但不能快得离谱。本章按稳态卡尔曼滤波设计出的观测器，在连续域的等效极点是 $-2.731 \pm 1.689j$ 与 -8.809：主导对的实部 -2.731，相当于把不稳定的 $+1.033$ 沿实轴推到左侧 3.76 的位置，是它原来到虚轴距离的 3.6 倍；快极点 -8.809 只比对象的 -6.667 快一点，没有强行拉开——采样周期 $T_s = 0.02$ s 对应的奈奎斯特频率是 157 rad/s，观测器再快就只是在放大量测噪声。这条“2 到 5 倍”的经验在这里落到了 3.6 倍。

式 (78-34) 里有一个离散域看不见的东西：$\dot{\boldsymbol{v}}$。量测噪声 $\boldsymbol{v}$ 在连续域是白噪声，它不可微，$\dot{\boldsymbol{v}}$ 只能理解为广义导数。这一项的存在说明“连续时间的残差方程”本身是形式化的，它的严格含义要用随机微分方程表述。工程上不必纠缠这一点，但要知道后果：连续域的残差方差是无穷大（白噪声求导），任何“用连续域算阈值”的尝试都会得到无意义的结果。离散域没有这个问题，因为差分把不可微变成了可加的高斯项，方差有限。**这不是数学上的吹毛求疵，而是第 78.4.5 节“阈值必须在离散域标定”的第一条理由。**

观测器极点的三个约束可以写成一句话：比对象快、比噪声慢、比采样慢。三条都满足，观测器才有用。

比对象快，是为了让残差在故障后迅速偏离零。本章的快慢比是 3.6 倍，落在常用的 2 到 5 倍区间；低于 2 倍，残差跟不上故障，检测延迟会被观测器本身拖长；高于 5 倍，残差对噪声的放大倍数会从“几倍”跳到“十几倍”，阈值不得不抬高，反而更慢。

比噪声慢，是指观测器带宽不能超过量测噪声的相关带宽。本章的噪声是逐点独立的高斯白噪声（在离散域看是这样），它的带宽就是奈奎斯特频率的一半，所以这条约束在本章不成问题；但实机上量测往往经过模拟低通滤波，噪声带宽只有几十赫兹，观测器极点若超过这个带宽，就会把滤波器的高频衰减“补回来”，等于自己造噪声。

比采样慢，是指最快极点对应的 $|s|T_s$ 要远小于 1。本章 $-8.809 \times 0.02 = 0.176$，还算安全；如果观测器极点做到 -50，$|s|T_s = 1$，离散化后的极点会跑到 $e^{-1} = 0.368$ 附近，与连续域的对应关系开始失真，增益也对不上。

3.6 倍这个数不是设计时指定的，是算出来的。本章没有用“极点配置到 -3.6×1.033”这种方式，而是按噪声强度解对偶 Riccati 方程，让权重的相对大小自然决定极点位置。这样做的结果是极点位置与噪声水平绑定：噪声变大，观测器自动变慢；噪声变小，自动变快。手配极点做不到这一点，一旦换传感器就要重配。**用 Riccati 而不是极点配置，换来的正是这种“跟着硬件走”的自适应性。**

78.4.2 残差的频域解释

对式 (78-34) 做拉普拉斯变换，从故障项到残差的传递关系是

$$\boldsymbol{r}(s) = \left(s\boldsymbol{I} - \boldsymbol{A}_c + \boldsymbol{L}_c\right)^{-1} \boldsymbol{b}_{1c}\, d(s), \qquad d(t) = (\rho - 1)u_1(t) + c \tag{78-35}$$

这个式子在频域里说清了一件事：**故障在残差里的“可见度”取决于 $d(t)$ 的频率成分与观测器带宽的相对位置。**$d(t)$ 若是常数（卡死、偏置），它的能量集中在零频，残差里表现为一个稳态偏移；$d(t)$ 若随指令变化（效率下降），它的能量分布在指令的频段上，残差里表现为与指令同频的波动。

本章的参考周期 12 s，主频约 0.083 Hz，远低于观测器带宽，所以效率下降的故障在残差里表现为“缓慢的、与参考同相的偏移”。这解释了一个实测现象：在低幅值工况下，$\rho = 0.9$ 的轻故障要 1.24 s 才被检出，而卡死 $c = 2.0$ 只要 0.04 s——常数故障在零频处的幅值大，一步就能顶穿阈值。

这两类故障的差别不只是“快慢”，而是“可不可累积”。卡死是一个阶跃，残差在故障后的第一拍就跳到一个明显非零的值，检测器只需要判断“这个值是不是噪声”。效率下降是一个与指令成比例的偏差，指令本身在缓慢变化（周期 12 s，起幅常数 0.8 s），残差跟着指令走，检测器要区分“残差变大是因为故障”还是“残差变大是因为指令本身在变大”。后者需要等，等到偏差积累到阈值之上。**所以检测延迟不是由观测器决定的，是由“故障信号与正常信号的可分性”决定的，**这是一条与观测器带宽无关的下界。

零频处的稳态残差可以用终值定理估计。式 (78-35) 在 $s = 0$ 处的值决定了常值故障的稳态偏移，它等于 $\left(\boldsymbol{L}_c - \boldsymbol{A}_c\right)^{-1}\boldsymbol{b}_{1c}$ 乘以故障幅值。这个矩阵是有限维的、可求逆的，所以常值故障一定会在残差里留下一个非零的稳态分量——只要这个分量不被观测器的积分作用吃掉。**这也是为什么本章的观测器故意不带积分器：**带积分器的观测器会把常值故障当成模型偏差吸收掉，残差回到零，故障就“消失”了。容错控制与鲁棒控制在这里有一个根本分歧：鲁棒控制希望残差对扰动不敏感，容错控制恰恰要求残差对故障敏感。

频率视角还解释了一个调参上的陷阱。如果参考信号周期缩短到接近观测器带宽，效率下降的故障信号会落在观测器的通带边缘，残差里的幅值被衰减，检测会变慢甚至失效；反之，如果参考信号周期很长（本章 12 s 已经算长），故障信号几乎就是准静态的，残差幅值接近稳态值，检测主要受噪声限制。**同一套阈值，换一个参考周期就换了性能，**这一点在第 78.8 节的调参经验里还会再提一次。

78.4.3 连续时间的重构

连续时间的 LQR 重构与离散版本一一对应，只是 Riccati 方程换成连续形式：

$$\boldsymbol{A}_c^{\mathsf{T}}\boldsymbol{P} + \boldsymbol{P}\boldsymbol{A}_c - \boldsymbol{P}\boldsymbol{B}_{\text{eff}}\boldsymbol{R}^{-1}\boldsymbol{B}_{\text{eff}}^{\mathsf{T}}\boldsymbol{P} + \boldsymbol{Q} = \boldsymbol{0}, \qquad \hat{\boldsymbol{K}} = \boldsymbol{R}^{-1}\boldsymbol{B}_{\text{eff}}^{\mathsf{T}}\boldsymbol{P} \tag{78-36}$$

其中 $\boldsymbol{B}_{\text{eff}}$ 按式 (78-23) 构造。连续域的好处是“有效增益”的物理意义更直观：$\hat{\rho}$ 变小时，$\boldsymbol{B}_{\text{eff}}$ 的第一列变小，Riccati 方程解出的 $\boldsymbol{P}$ 在第一分量上变大，最终 $\hat{\boldsymbol{K}}$ 的第一行变大。这条链条在连续域里可以逐项追踪，在离散域里则被矩阵指数“糊”在一起，不容易看清。

“第一行变大”这句话要限定在哪个方向变大。$\boldsymbol{B}_{\text{eff}}$ 的第一列是 $[0, \hat{\rho}, 0]^{\mathsf{T}}$，它只作用于速度方程，所以 $\hat{\rho}$ 变小时，控制器对速度的“话语权”变弱，Riccati 方程为了让同一个速度误差得到同样的加速度，必须提高增益——但这个提高不是均匀的，而是集中在能“借力”的通道上。具体到本章，$\hat{\rho}$ 变小时增益的第一列（对应位置）涨得最快，第二列（对应速度）涨得慢一些，第三列（对应备份执行器状态）几乎不动。这是因为它可以走两条路：加大 u_1，或者把备份通道用起来。前者的效率已经打了折，所以最优解会越来越偏向后者。**这就是“再分配”在连续域里的自然解释：不是额外加的逻辑，而是 Riccati 方程在效率下降后自动做出的权衡。**

再分配该不该显式打开，是一个可以算的判据。本章的实测是：$\hat{\rho} = 0.6$ 不做再分配，闭环极点 0.9381、0.6870、0.2848，谱半径 0.9381；$\hat{\rho} = 0.4$ 不做再分配是 0.9373、0.7794、0.2848，谱半径 0.9373——效率掉三分之一，谱半径只从 0.9381 降到 0.9373，几乎没动，因为 Riccati 方程已经自动把增益加上去了。真正开再分配时，$\hat{\rho} = 0.4$ 的极点变成 0.8298 与 $0.9623 \pm 0.0208j$，谱半径升到 0.9626；$\hat{\rho} = 0.2$ 时更极端，谱半径 0.9635。**再分配不是“多做一点总比不做好”，它在效率中等时收益被 Riccati 的自动补偿吃掉，在效率很低时又会把主导极点从实轴推成一对低阻尼共轭极点，代价是振铃。**这条结论在连续域看不出来，必须逐点算离散极点，第 78.7 节会把它写成一条可直接用的判据。

连续 LQR 与离散 LQR 的解不是“差不多”。两者的差别来自“控制量在一个采样周期内是否变化”这个假设：连续 LQR 假设 u 连续可调，离散 LQR 假设 u 在周期内恒定。当对象的时间常数远大于采样周期时，两者趋于一致；本章最快的模态是 -6.667（时间常数 0.15 s），采样周期 0.02 s，比值 7.5，不算“远大于”。用矩阵指数的二阶项粗估修正量：$|s|T_s = 0.133$，二阶项 $(0.133)^2/2 = 0.0088$，即约 0.9%。这个量级在增益上不算大，但**它集中作用在快模态上，而快模态正是备份执行器的内部状态 x_3**——也就是容错最关心的那个通道。所以“整体差 1%”这种说法会掩盖“关键通道差得更多”的事实。

78.4.4 连续时间的可检测性

连续域还有一个离散域不方便表达的性质：**故障方向与观测器零点的关系**。式 (78-35) 的传递函数在某个频率处若出现传输零点，故障在该频率上的分量就传不到残差。对于三状态全测的对象，$\boldsymbol{C} = \boldsymbol{I}$，传递函数是 3×1 的向量，其零点由 $(\boldsymbol{A}_c - \boldsymbol{L}_c, \boldsymbol{b}_{1c})$ 决定。只要 $\boldsymbol{b}_{1c}$ 不与 $\boldsymbol{L}_c$ 的某个特征向量正交，就不存在“完全看不见”的频率。

本章的 $\boldsymbol{b}_{1c} = [0, g_1, 0]^{\mathsf{T}}$，与三个观测器特征向量都不正交，所以三类故障在整个频段上都可见。这是“三状态全测”带来的便利；如果只有位置可测（$\boldsymbol{C} = [1, 0, 0]$），速度与备份执行器状态要靠观测器推断，$\boldsymbol{b}_{1c}$ 的可见性就会变差，轻故障的检测会明显变慢。

“不正交”这件事要分开看是哪个分量。故障只进速度方程，$\boldsymbol{b}_{1c}$ 只有第二个分量非零。它在残差里的投影强度取决于观测器的第二行——也就是观测器对速度误差的修正力度。本章 $\boldsymbol{L}$ 的第二行是 $[0.1170,\ 0.0372,\ 0.0017]$，三个分量都不为零，所以故障在这一行上的投影不为零；如果某一列的系数恰好为零，那么对应的量测通道就完全看不到这个故障。**这就是“故障可检测性”在三状态全测下唯一的漏洞：不是看不见故障本身，而是某一个量测通道的修正系数恰好被设计成零。**

这也解释了为什么 $\boldsymbol{C} = \boldsymbol{I}$ 是一个“便宜”的假设。全状态可测意味着观测器不需要“推断”，残差直接等于状态误差加上噪声，故障的可见性只由 $\boldsymbol{b}_{1c}$ 决定，不受观测器结构的二次影响。一旦换成部分状态可测，$\boldsymbol{b}_{1c}$ 先要经过观测器的“传播”才到达残差，传播过程中可能被衰减、被相移、被零点挡住，可检测性就从“看 $\boldsymbol{b}_{1c}$ 是否为零”变成“看传递函数在关注频段上是否接近零”。后者难判得多，也更容易在某个特定频率上翻车。**实机上位置传感器最可靠、速度靠差分、备份执行器状态几乎不可测，所以真实系统的可检测性通常远不如本章的仿真乐观。**

一个具体的退化路径。假设只测位置，$\boldsymbol{C} = [1, 0, 0]$。观测器的第三行（对应 x_3）只能靠模型外推，而模型里 x_3 的动态是 $1/(\tau s + 1)$，时间常数 0.15 s。这意味着任何关于 x_3 的推断都至少滞后 0.15 s，故障在 x_3 通道上的信息要经过这段滞后才反映到位置量测上。表现就是：c 的辨识会变慢，$\hat{c}$ 的方差会变大，分类边界（$\hat{\rho} < 0.12$ 判卡死）附近的故障会被反复改判。**本章之所以敢把分类边界定得很窄，前提就是三状态全测；换一个传感器配置，这套边界必须重新标定。**

78.4.5 连续与离散的取舍

既然连续域的分析更清楚，为什么实现要用离散？原因有三个，都在工程侧。

第一，控制器本来就是数字的。采样、保持、量化、通信延迟，这些环节在连续域里只能近似成“纯延迟”，而在离散域里是精确的。

第二，阈值标定必须在离散域做。式 (78-18) 的 μ_S 与 σ_S 是离散统计量的均值与方差，它们依赖采样率、滑动平均窗长、噪声的离散化方式。用连续域的方差去推离散阈值，会差一个与采样率有关的系数。

第三，重构后的稳定性判据在离散域更直接。离散闭环的稳定性看极点是否在单位圆内，这是一个可以逐点检查的条件；连续域要看右半平面，虽然等价，但数值上不如单位圆直观。

三条理由里，第二条最容易被低估。式 (78-18) 里的 μ_S 与 σ_S 不是普适常数，它们由四件事共同决定：采样周期（决定积分窗的等效带宽）、滑动平均窗长（决定噪声的压缩比）、噪声是否经过抗混叠滤波、以及参考信号的频率成分。任何一项变了，统计量就变。用连续域的方差公式去推阈值，等于假设“采样足够快、平均窗足够长、噪声是白的”，这三个假设在实机上都不严格成立。**阈值只能测，不能推**，这是本章反复强调的一句话，它的根据就在这里。

还有第四条理由，属于数值层面：离散 Riccati 方程的解在 $\boldsymbol{B}$ 的列接近线性相关时条件数会急剧变差。本章卡死重构时要把主通道整列置零，等价于把 $\boldsymbol{B}$ 从 3×2 降成 3×1；如果偷懒不降维、只把那一列改成一个极小的数，$\boldsymbol{R}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{B}$ 的条件数会到 10^{12} 量级，解出的增益完全不可信。**“置零”必须是真的删列，不是填一个 10^{-12}**，这条在离散实现里是硬要求。

另外要承认一点：连续域的分析在本章并没有被“用尽”。真正被用上的只有三处——观测器极点的位置解释、故障在频域里的可见性、以及 $\hat{\rho}$ 变小时增益怎么变。剩下的工作全部在离散域完成。之所以保留这一节，不是因为连续设计更好，而是因为**“为什么这样做”的答案通常只在连续域里说得清楚，而“怎么做对”的答案只在离散域里验证得了**。

78.4.6 把离散极点折回连续域

既然两套语言都要用，就必须有一个可靠的换算。对采样系统的极点，换算是 $s=\ln(z)/T_s$；对连续系统离散化后的极点，换算是 $z=e^{sT_s}$。两个公式互为反函数，但只在 z 位于右半平面映射区内、且 T_s 足够小时才有物理意义。本章用它做了两次往返核对。

第一次是对象极点。连续域 $+1.033$、-1.393、-6.667 映射到离散域是 1.020884、0.972516、0.875173，与 $\operatorname{eig}(\boldsymbol{A})$ 逐位吻合。

第二次是观测器极点。离散域实算是 $0.946299\pm 0.031967j$ 与 0.838468，折回连续域：

$$s=\frac{\ln z}{T_s}\quad\Longrightarrow\quad -2.731\pm 1.689j,\ -8.809$$

这两个数就是第 78.4.1 节用的连续等效极点。往返一次没有信息损失，说明这套观测器在连续域与离散域的描述是一致的——**一致性本身就是一条验收条件**：如果两次换算对不上，说明离散化时用了近似（例如把 $\boldsymbol{B}$ 写成 $\boldsymbol{B}_c T_s$ 而丢掉了高阶项），误差会在重构增益上放大。

折回连续域还有一个实用价值：判断“观测器是否快得离谱”用连续域的 rad/s 最直观。$|-8.809|$ 与奈奎斯特频率 157 rad/s 差 17.8 倍，一眼就知道安全；如果只看离散极点 0.838468，很难说出它“快不快”。

反过来，从连续域往离散域折时要注意**采样保持的相位**。零阶保持会引入半个采样周期的等效延迟，即 $e^{-sT_s/2}$。这项在 $|s|T_s$ 很小时可以忽略，在本章 $|s|T_s=0.176$ 时对应相位滞后 0.088 rad，约 5°。对观测器极点来说这 5° 无关紧要，但对闭环相位裕度来说，5° 是要算进预算的。**同一个 T_s，对观测器是“无所谓”，对稳定性裕度是“必须算”**，这是离散化误差容易被双标的地方。

所以本章的做法是：**用连续域理解机理，用离散域实现与标定**。第 78.5 节专门讨论离散实现里的几个坑。

78.5 离散情形

78.5.1 两列的形状：主备通道差多少

零阶保持离散化的结果，本章的具体数值是

$$\boldsymbol{A}=\begin{bmatrix}1.000287 & 0.019930 & 0.000115\\ 0.028699 & 0.993113 & 0.011194\\ 0 & 0 & 0.875173\end{bmatrix},\qquad \boldsymbol{B}=\begin{bmatrix}1.9953\times10^{-4} & 5.1510\times10^{-6}\\ 1.9930\times10^{-2} & 7.6378\times10^{-4}\\ 0 & 1.2483\times10^{-1}\end{bmatrix} \tag{78-37}$$

$\boldsymbol{A}$ 的三个特征值是 1.020884、0.972516 与 0.875173，与连续极点的对应关系是 $z=e^{sT_s}$：$e^{1.033\times0.02}=1.020884$、$e^{-1.393\times0.02}=0.972516$、$e^{-6.667\times0.02}=0.875173$。在离散域里，“不稳定”只有一个判据——极点模是否大于 1。这里第一个模是 1.020884，另外两个都小于 1，一眼就能看出这个对象有一个方向会自己跑掉。

真正需要盯住的是 $\boldsymbol{B}$ 的两列。按第一步的加速度增量比较，$\boldsymbol{b}_1$ 的第二分量是 0.019930，$\boldsymbol{b}_2$ 的第二分量是 7.6378×10^{-4}，两者相差 26.1 倍。也就是说，备份通道在第一个采样周期里只能给出主通道 3.8% 的加速度。**指望“切过去就顶上”是不成立的。**

但稳态的结论完全不同。设 u_2 恒定，x_3 的稳态值就等于 u_2，它作用在速度上的力是 $g_2u_2=0.6u_2$；主通道是 $g_1u_1=1.0u_1$。按限幅算，备份是 $0.6\times6=3.6$，主通道是 $1.0\times5=5.0$，备份能达到主通道的 72%。

同一对通道，瞬态上差 26 倍，稳态上只差 1.4 倍，差别全在时间常数 $\tau=0.15\,\mathrm{s}$ 上：$\boldsymbol{b}_2$ 的能量先灌进 x_3（这一项是 0.1248，比 $\boldsymbol{b}_1$ 的任何分量都大），再由 x_3 通过 $\boldsymbol{A}$ 的 $(2,3)$ 元 0.011194 慢慢推到速度上，需要 7.5 个采样周期才走完 63%。

这就给出了本章最重要的一句工程判断：**主通道是“快而直接”的，备份通道是“慢而有力”的。**慢通道能修稳态，不能救瞬态。第 78.2.8 节的权限判据、第 78.9 节卡死 $c=2.0$ 救不回来的案例，根子都在这里。

78.5.2 观测器的离散极点与残差时间常数

观测器的离散闭环矩阵是 $\boldsymbol{A}-\boldsymbol{L}$，代入式 (78-14) 的 $\boldsymbol{L}$，它的特征值是

$$z_{1,2}=0.946299\pm0.031967j,\qquad z_3=0.838468 \tag{78-38}$$

主导对的模是 0.946838，折算成连续域的实部是 $\ln(0.946838)/0.02=-2.731$，时间常数 $0.366\,\mathrm{s}$；第三个极点的等效实部是 -8.809，时间常数 $0.114\,\mathrm{s}$。

这三个数决定了残差在故障后“爬多快”。把式 (78-16) 看成以故障项为输入、以残差为输出的系统，分母就是这些极点。模 0.9468 的那一对管包络上升，大约三个时间常数（$1.1\,\mathrm{s}$）之后残差才贴近它的稳态值。可是检测并不需要等它到稳态——只要越过阈值就够了。这正是后面“检测延迟下限 $0.16\,\mathrm{s}$”与“观测器时间常数 $0.366\,\mathrm{s}$”不矛盾的原因：残差在头几步就能爬到阈值附近，后面那一段爬升只是富余。

这对共轭极点的阻尼比约 0.53。模 0.946838、辐角 $\arctan(0.031967/0.946299)=0.03375\,\mathrm{rad}$，折算成连续域的阻尼比是 $0.03375/\sqrt{0.03375^2+0.05464^2}=0.526$——注意这里要用连续域的换算，直接用离散域的模做除法会得到错误答案。这个阻尼比说明残差在故障后会有大约一成多的过冲，不是单调爬升。过冲对检测是好事还是坏事，取决于阈值的位置：阈值压在稳态值以下时，过冲让残差更早越过；阈值压在稳态值附近时，过冲会造成“越过后又回落到阈值以下”的抖动，检测逻辑若要求连续两拍超限，延迟会突然变长。本章的检测只要求单拍超限，所以避开了这个问题。

离散极点不能按 $z\approx1+sT_s$ 简单换算。这条线性近似在 $|s|T_s$ 小时好用，但主导对是复数，$s=-2.731\pm1.689j$，$|s|T_s=0.257$，线性近似的误差已经到 $(0.257)^2/2=3.3\%$，模长会算成 $1-0.0546=0.9454$，而真值是 0.946838，差了 0.15%；实部则差得更多，因为复数极点的实部还要受虚部的二阶影响。**要准确折回，只能用 $s=\ln(z)/T_s$，不能用一阶近似，**这是第 78.4.6 节那条换算公式存在的实际理由。

残差时间常数与检测延迟是两件事。前者是“残差自己爬”，后者是“残差爬过阈值”，两者之间差一个“阈值有多高”的因子。本章阈值的稳态值比例大约是 $2.169/0.686=3.16$ 倍均值，而稳态值本身由故障幅值决定，所以真正的“爬升时间”

随故障幅值变化：故障大，稳态值高，阈值相对低，爬得快；故障小，稳态值可能与阈值同量级，永远爬不过去——$\rho = 0.9$ 在 12 s 内没被检出，就是这个原因，不是“慢”，是“最终也到不了”。

78.5.3 阈值只能在离散域标定

图 78-3 是健康状态下残差统计量 S_k 的直方图，以及 $k = 2$ 到 6 对应的五条阈值线。

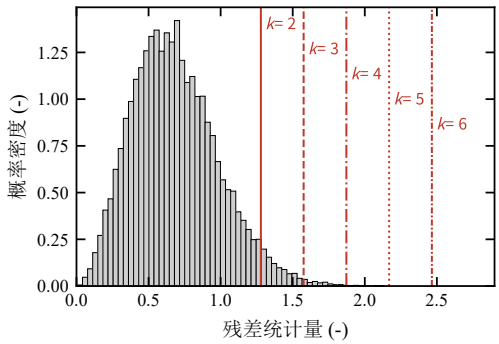


图 78-3 健康残差统计量的分布与五条阈值线

分布明显右偏，长尾一直拖到 2.1 以上，这与“三个归一化残差取范数”的结构一致：范数不为负，质量堆在 0 附近，右侧拖一条尾巴。实测均值 $\mu_S = 0.6860$、标准差 $\sigma_S = 0.2966$、最大值 2.1326。最右边的 $k = 5$ 阈值 2.169 恰好压在这条尾巴的外面，所以 $k \geq 5$ 时 500 步内一次误报都没有；$k = 4$ 的阈值 1.873 切进尾巴，误报率 0.00047。

为什么这个标定不能在连续域完成？因为 σ_S 由三件事共同决定：量测噪声 σ、滑动平均窗长 w、以及模型与对象之间的失配。前两项可以解析算，第三项只能实测。本章的 $\mu_S = 0.6860$ 比“纯噪声”推算出来的值大不少，多出来的部分就是失配——线性化残差、RK4 与零阶保持的差异、限幅引入的非线性——这些东西在连续域的方差公式里没有位置。

还有一层依赖容易被忽略：μ_S 与 σ_S 都依赖采样周期。把 T_s 减半，同样 $w = 5$ 的滑动平均只覆盖 0.05 s 的物理时间，平滑变差，σ_S 上升，误报率跟着上升。**换采样率必须重新标定阈值，不能沿用旧数。**

78.5.4 检测延迟的量化台阶

图 78-4 的左侧子图是故障幅值 ρ 与检测延迟的关系，右侧子图是同一个 ρ 与闭环代价的关系。

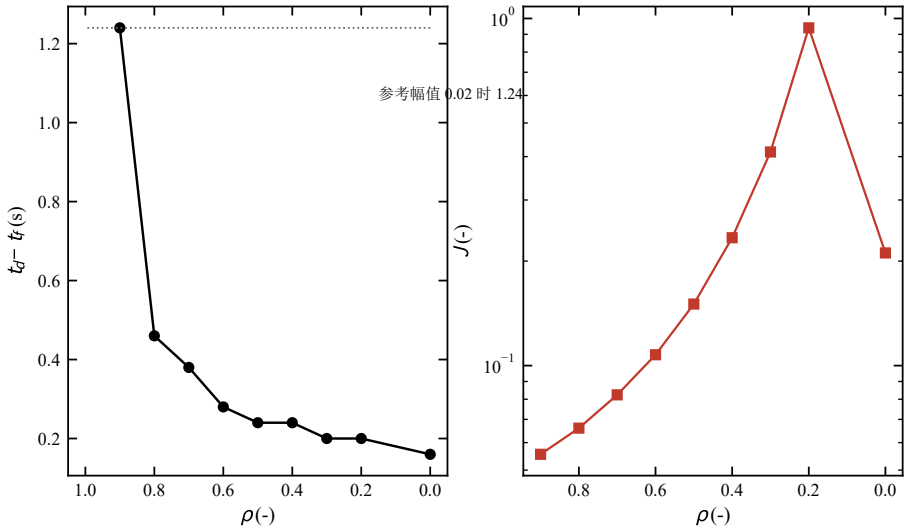


图 78-4 故障幅值对检测延迟与闭环代价的影响：左为检测延迟，右为闭环代价（对数纵轴）

实测的延迟序列是：$\rho = 0.9$ 时 12 s 内根本没超阈值；$\rho = 0.8$ 时 0.46 s；0.7 时 0.38 s；0.6 时 0.28 s；0.5 与 0.4 都是 0.24 s；0.3 与 0.2 都是 0.20 s；$\rho = 0$ 时 0.16 s。

所有值都是 0.02 s 的整数倍——23、19、14、12、10、8 步。台阶由三段组成：量测与计算延迟 1 步；滑动平均窗填满需要 $w-1=4$ 步；剩下的是“残差爬到阈值”所需的若干步。下限 8 步就是这三段在最理想情况下的和。

想再往下压，只有两条路。一是减小 w，但 σ_S 随之上升，为了不误报又必须加大 k，反而更慢——这是第 78.7 节要展开的循环。二是减小 T_s，那是硬件与算力的事。所以延迟不是调出来的，是买出来的。

右侧子图有一个反直觉的点：$\rho=0.2$ 的代价 0.9397 比 $\rho=0$ 的 0.2111 还高。原因是 $\rho=0$ 会被判成卡死、直接切到备份通道；而 $\rho=0.2$ 被判成效率下降，前馈项按 $1/\hat{\rho}$ 放大五倍（式 (78-27) 的 $\hat{\rho}$ 折回），指令频繁撞上 $|u_1|\le 5$ 的限幅，实际作用量反而跟不上一段。**中等故障比重故障更难处理**，这是容错控制里一个反复出现的现象。

78.5.5 辨识的条件数：持续激励

式 (78-21) 的最小二乘要解一个 2×2 方程组，矩阵是 $\sum_k \boldsymbol{\phi}_k\boldsymbol{\phi}_k^{\mathsf{T}}$，回归量 $\boldsymbol{\phi}_k=[u_{1,k},\,1]^{\mathsf{T}}$。它的条件数有一个直白的物理含义：**辨识窗内 u_1 是否在变**。

如果 u_1 在窗内近似恒定，ρ 与 c 就分不开——一个恒定的作用量既可以解释成“效率乘了一个系数”，也可以解释成“指令加了偏置”。这就是容错控制里的持续激励条件，与第 30 章讲系统辨识时的要求是同一件事。代码里对此有一道保护：条件数超过 10^{8} 时放弃本次辨识，返回中立值 $(\hat{\rho},\hat{c})=(1,0)$，等下一窗数据。这比给出一个病态的估计安全得多——病态估计会让重构增益飞掉。

反过来看第 78.8 节的一个实测：把参考幅值从 0.30 压到 0.02，$\rho=0.9$ 的轻故障从“12 s 内检不出”变成“要 1.24 s 才检出”。指令变小，信噪比变差只是原因之一，更关键的是 u_1 的摆幅跟着变小，条件数迅速恶化，辨识窗经常判不通过。**激励不足的系统，容错诊断是做不动的。**

条件数恶化还有一个隐蔽的后果：它不是“估计不准”，而是“估计会跳变”。同一个故障，u_1 在某一段窗内恰好走平，$\hat{\rho}$ 与 $\hat{c}$ 会沿一条近乎退化的直线滑动——两个参数的组合几乎不变，但各自的取值可以差很多。这时如果按 $\hat{\rho}$ 去分类，可能上一窗判成效率下降、下一窗判成偏置。本章的对策是“条件数超限就整窗作废”，宁可保持上一次的结论，也不接受一次退化方向上的估计。代价是响应变慢，收益是分类不会来回跳。

还有一条对策是主动的：在参考信号里叠加一点小幅高频抖动，专门用来“喂”辨识。这在理论上很漂亮（持续激励的标准做法），在本平台上却不能用——抖动会直接体现在位置误差里，与“跟踪精度优先”的代价权重冲突，而且 0.02 s 的采样周期下可用的抖动频率范围很窄，容易被观测器滤掉。**主动激励是“辨识友好”与“控制友好”之间的正面冲突**，本章选择了后者，只在参考本身变化足够时辨识。

78.5.6 卡死重构的离散实现

卡死是三类故障里唯一必须“换控制结构”的一种：主通道彻底不受控，只能把它整条关掉。离散实现的三行核心是

$$\boldsymbol{K}_{\text{to}}=\begin{bmatrix}\mathbf{0}_{1\times 3}\\ \boldsymbol{k}_{\text{back}}^{\mathsf{T}}\end{bmatrix},\qquad \boldsymbol{k}_{\text{back}}^{\mathsf{T}}=\operatorname{dlqr}\left(\boldsymbol{A},\boldsymbol{b}_2,\boldsymbol{Q},\,R_{\text{back}}\right),\qquad u_{1,k}=0 \tag{78-39}$$

配合备份通道的平衡点前移

$$x_{3,\text{eq}}=-\frac{\omega^2 r-2\zeta\omega\dot{r}+g_1\hat{c}}{g_2} \tag{78-40}$$

这里有三个离散实现上的坑，都是踩过才知道的。

第一，**不能先按两输入 LQR 算完再砍掉第一行**。那样得到的第二行是“与一个不存在的通道协同”算出来的，它会把备份通道的力气用在补偿一个它补不了的方向上。本章的实现改成对 $\boldsymbol{b}_2$ 单列做 LQR，主通道那一行恒为零。

第二，**备份通道要独立的控力权重**。主通道的 $R=0.01$，备份通道取 $R_{\text{back}}=0.1$，大一个数量级。理由还是 78.5.1 那条：备份通道力气小、惯性大，用主通道的权重会逼它做力所不及的事，结果是控制量顶到限幅、轴还是往下掉。

第三，**前馈平衡点必须把 $\hat{c}$ 折进去**。若不折，备份通道要在稳态额外承担 $g_1 c/g_2$ 的偏差：$c=0.5$ 时是 0.833，已占掉限幅 6 的 13.9%。这个偏差不会自己消失，会一直表现为位置上的常值偏移。

还有一件容易漏的事：**观测器必须与对象同步重构**。切换之后，送进观测器的应当是 $\boldsymbol{u}^{\mathrm{obs}} = [\hat{c},\ u_2]^{\mathsf{T}}$，而不是 $[0,\ u_2]^{\mathsf{T}}$。否则观测器以为主通道一点力都不出，残差里会持续存在一个 $g_1\hat{c}$ 的偏差，辨识出的 $\hat{c}$ 越走越偏，重构增益跟着错。三种故障对应三种作用量，正好是式 (78-3) 的三种取法，代码里就是三个分支，没有第四种。

辨识窗长 $N = 20$（$0.4\ \mathrm{s}$）这个取值也是被卡死逼出来的。N 越长估计越准，但辨识窗必须整段落在故障之后，窗口越长，重构完成得越晚。本章实测：$c = 1.0$ 在 $0.4\ \mathrm{s}$ 窗下，故障后峰值 0.507；如果把窗拉到 $0.8\ \mathrm{s}$，辨识还没结束，轴已经在限幅上撞死了。**在本质不稳定对象上，辨识精度永远要给时间让路。**

78.5.7 上机前的五个检查

把上面几节压成一张检查表，实现完之后逐条过：

1. 离散极点是否与连续极点按 $z = e^{sT_s}$ 对得上？对不上说明离散化写错了，后面所有设计都建在错模型上。
2. 观测器极点是否比对象的不稳定极点快 2 到 5 倍？本章是 3.6 倍。太慢跟不上故障，太快放大量测噪声。
3. 阈值是否用本采样率下的健康数据重新标定过？沿用别的采样率的 μ_S、σ_S 是最常见的误报来源。
4. 辨识的条件数保护是否打开？没有保护的除法在 u_1 恒定时会给出离谱的 $\hat{\rho}$。
5. 卡死分支是否用了单输入 LQR、独立 R_{back}、折算 $\hat{c}$ 的平衡点，并且把 $\hat{c}$ 同步送进观测器？四条缺一条，卡死就救不回来。

这五条都不涉及新理论，全是实现细节。容错控制的失败案例里，绝大多数不是算法想错了，而是这几条里漏了一条。

78.6 稳定性与性能

78.6.1 每一套增益都是稳定的

容错控制的第一道门槛是：**重构之后的闭环必须还是稳定的**。这句话听起来像废话，但它是可以逐套验证的。离散闭环的稳定性看 $\boldsymbol{A} - \boldsymbol{B}_{\mathrm{eff}}\boldsymbol{K}$ 的谱半径

$$\rho_{\mathrm{cl}} = \max_{i} \left|\lambda_i\left(\boldsymbol{A} - \boldsymbol{B}_{\mathrm{eff}}\boldsymbol{K}\right)\right| < 1 \tag{78-41}$$

把本章用到的五套增益代进去，实测结果如下。

分支	闭环极点	谱半径
标称 $\boldsymbol{K}_0$	0.9385、0.5371、0.2848	0.9385
被动 $\boldsymbol{K}_{\mathrm{pass}}$	0.9378、0.7315、0.2848	0.9378
效率 $\hat{\rho}=0.6$（不分配）	0.9381、0.6870、0.2848	0.9381
效率 $\hat{\rho}=0.4$（不分配）	0.9373、0.7794、0.2848	0.9373
效率 $\hat{\rho}=0.4$（再分配）	0.8298、$0.9623 \pm 0.0208j$	0.9626
卡死（单输入备份）	0.6456、$0.9581 \pm 0.0226j$	0.9584

三条规律藏在表里。

第一，**最慢的极点始终是 0.938 那一个**，它对应位置通道。改增益改得动速度通道的极点（$0.537 \to 0.779$），改不动位置通道的——因为位置回路的带宽受限于对象本身的不稳定程度，增益再大也只是把它推到 0.93 附近。这就是“闭环时间常数约 $0.32\ \mathrm{s}$”的来源（$\ln 0.938 / 0.02 = -3.2$）。

第二，**再分配会把谱半径推大**。$\hat{\rho}=0.4$ 时，不加权再分配的谱半径是 0.9373，加上 $R_{11} = 1/\hat{\rho}^2 = 6.25$ 的再分配之后变成 0.9626。原因是再分配本质上是“少用主通道、多用备份通道”，而备份通道慢，慢通道必然把闭环拖慢。代价换稳定裕度，这是第 78.9 节要算的账。

第三，**卡死分支的谱半径是五套里最大的** 0.9584，而且有一对复数极点。这是必然的：只剩下一个慢通道，闭环必然更迟钝、更容易振荡。它仍然小于 1，所以能活；但余量只剩 4%，这就是为什么第 78.9 节里 $c=2.0$ 会失手——余量本来就不多，故障再大一点就吃光了。

78.6.2 切换本身会不会引入不稳定

每套增益各自稳定，不等于“在它们之间切换”也稳定。这是切换系统理论里最经典的反例来源：两套稳定增益，按某个坏时序来回切，可以让状态发散。

本章的运气好，因为这是一个**监督式切换**：切换由检测事件单向触发，检测到故障才切，而且切过去之后不会自己切回来。这种结构下，切换次数是有限的（理论上最多三次：检测、辨识、执行），每次切换后闭环都在一个稳定系统里运行，所以稳定性由“每套稳定 + 切换次数有限 + 切换时刻状态有界”三条保证。

真正需要担心的是**误报导致的来回切**。如果阈值定得太低，健康时也会报警，系统会切到一套为故障设计的增益上，性能变差；如果这时又因为残差回落而切回标称增益，就会出现极限环式的反复切换。本章的实现里没有“切回”这条路径，就是为了避开这件事——**容错控制默认故障不可逆，宁可停在保守的一侧**。这个选择有代价（瞬时扰动会被当成永久故障），但它把稳定性问题简化成了一句话：切一次，之后一直用新增益。

78.6.3 四种方案的代价横比

图 78-6 是 $\rho=0.4$ 时四种方案的位置跟踪与主执行器出力。

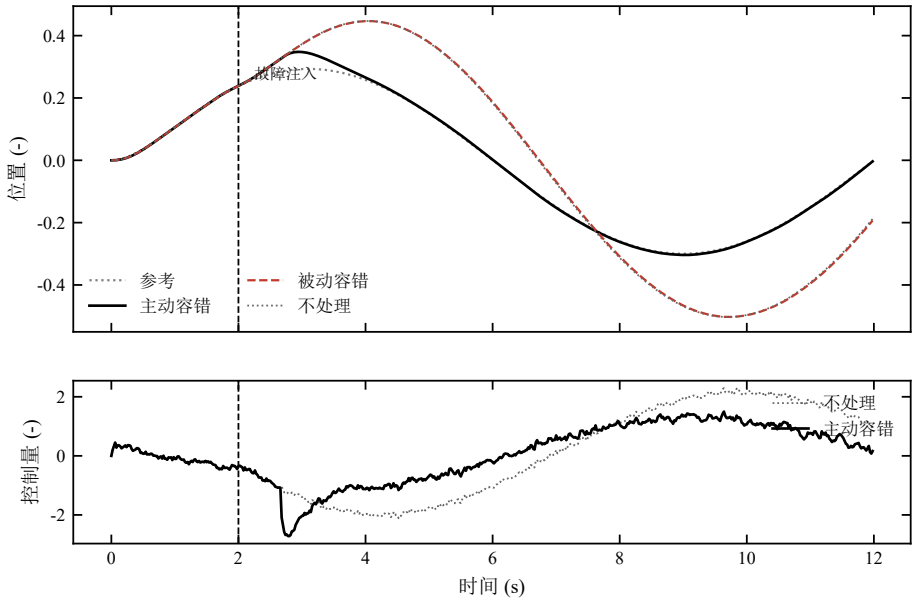


图 78-6 四种方案在 $\rho=0.4$ 下的位置跟踪与主执行器出力

上图里参考是灰色点线，四条彩色曲线分别是健康、不处理、被动容错、主动容错。故障注入在 $t=2.0$ s 的竖虚线处。健康曲线几乎贴着参考；不处理与被动容错在故障后明显偏离，而且偏离之后再也不回来；主动容错在故障后抖一下，随后重新贴回参考。

下图是主执行器指令 u_1。不处理的 u_1 在故障后继续按标称逻辑出力，看起来“很正常”——这正是问题所在，它不知道自己只出了四成力。主动容错的 u_1 在检测到故障后整体抬高，把缺的那部分补回来，然后随着辨识完成再回落到一个新的工作点。

对应的闭环代价（式 (78-4) 的指标，12 s 积分）是：

配置	J	相对健康
健康	0.0473	1.0
故障 + 不处理	0.8115	17.2
故障 + 被动容错	0.8104	17.1
故障 + 主动容错	0.2344	5.0

主动容错把代价压到不处理的 29%，但仍然比健康高 4 倍。这个“还剩 4 倍”很重要：容错控制不是“修好”，是“止损”。故障通道的物理能力已经没了，闭环不可能回到健康时的性能，能做到的只是不让你掉下去。

换成 $\rho = 0$（主执行器完全失效），差距变成数量级：不处理 8.63×10^{6}，被动 4.30×10^{6}，主动 0.2151。前两者已经发散，后者不但没发散，代价还比 $\rho = 0.4$ 时的 0.2344 更低——因为完全失效判得准、切得果断。

第 78.10 节会用另一组种子重跑同一组对比，健康 0.0477、不处理 0.8180、被动 0.8170、主动 0.2294，与上表一致。**换了随机种子结论不变，说明这不是某一次仿真的巧合。**

78.6.4 峰值与末段误差是两个指标

只看积分代价会把两件不同的事混在一起。故障后的瞬态峰值说明“最坏能坏到什么程度”，末段均方根说明“最后稳在哪儿”，两者对应的物理后果完全不同：峰值决定会不会撞限位、会不会触发保护，末段误差决定稳态精度。

$\rho = 0.4$ 时三组数字：不处理峰值 0.2481、末段均方根 2.32×10^{-1}；主动容错峰值 0.0558、末段均方根 2.28×10^{-3}。峰值差 4.4 倍，末段误差差 100 倍。**瞬态靠检测速度救，稳态靠辨识精度救**，这两件事的瓶颈不一样，调参时不能一起调。

78.6.5 延迟如何吃掉性能

图 78-8 把“决策到执行”的延迟从 0 一路加到 4.0 s，看闭环代价与故障后峰值怎么变。

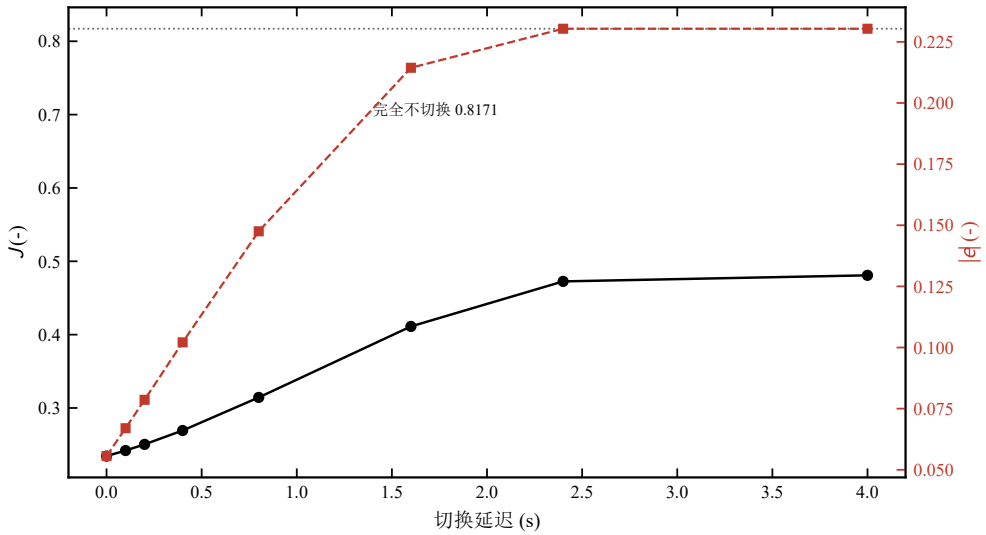


图 78-8 切换延迟对闭环代价与故障后峰值的影响，虚线为完全不切换的水平

实测数据：

延迟（步）	延迟（s）	J
0	0	0.2344
5	0.1	0.2420
10	0.2	0.2504
20	0.4	0.2693
40	0.8	0.3144
80	1.6	0.4112
120	2.4	0.4726
200	4.0	0.4809
不切换	—	0.8171

曲线是单调上升的，但形状值得注意：前 0.4 s 只涨了 15%，0.8 s 涨 34%，1.6 s 涨 75%；再往后就压平了，2.4 s 到 4.0 s 只多涨了 0.008。压平的原因是**故障瞬态已经过去了**：轴被推到某个偏置位置后停在那儿，再晚切换也只是从这个位置把系统拉回来，代价主要由“拉了多远”决定，不再由“什么时候开始拉”决定。

这条曲线给了工程上一个明确的预算感：延迟在 0.2 s 以内的代价几乎可以忽略，0.4 s 开始明显，1 s 以上就要重新评估方案了。而本章的检测延迟本身就在 0.16 到 0.46 s 之间，加上决策与执行，很容易落到 0.4 s 这一档。**所以第 78.10 节讲落地时，通信周期的预算要按 0.1 s 的量级去争，不能按“反正秒级也能切”去想。**

78.6.6 阈值系数的代价曲线

阈值系数 k 是容错控制里最“便宜”的一个旋钮，也是最容易被随手定死的一个。图 78-9 是它的两侧：左边误报率，右边闭环代价。

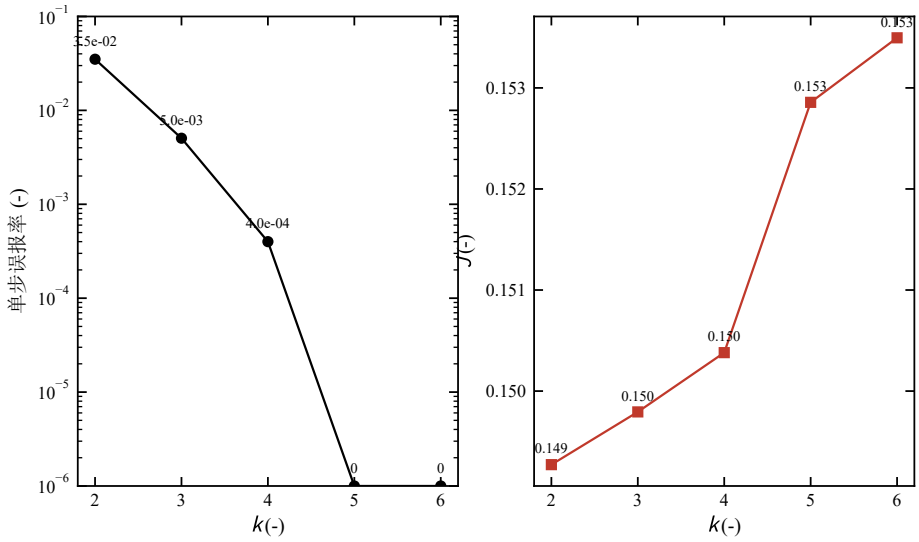


图 78-9 阈值系数对误报率与闭环代价的影响

$\rho = 0.5$ 时，k 从 2 加到 6，误报率从 0.03505 一路降到 0，而闭环代价只从 0.1493 涨到 0.1535——只涨了 2.8%。

这个结果有点反直觉：阈值抬高会拖慢检测，按理代价应该涨得更快。原因在故障幅值上：$\rho = 0.5$ 的残差很大，即使 $k = 6$ 也只需要多等一两步就越过阈值，检测延迟对 k 不敏感。真正对 k 敏感的是轻故障——$\rho = 0.8$ 那种，k 每加 1，延迟可能多好几步。

所以 k 的选法不该是“取中间值 4”，而该是：**先按最轻的必须检出的故障定 k 的上限，再按可接受的误报率定下限，两者夹出来的区间通常只剩一两档。**本章取 $k = 4$，误报率 0.00047。检测延迟对种子敏感：第 78.5.4 节那组（种子

1）在 $\rho=0.5$ 时是 0.24 s，本节这组（种子 7）是 0.20 s。同一个 k、同一套参数，换一个噪声实现就差 0.04 s，这个量级的不确定性在调参时必须算进去。

78.6.7 故障程度的全谱：一张表看清边界

前面各节都是固定一个 ρ 看方案差异，下面这张表反过来：固定方案，扫故障程度。

ρ	不处理	被动容错	主动容错	主动 + 再分配
0.6	0.1807	0.1811	0.1083	0.1025
0.4	0.8109	0.8100	0.2337	0.1848
0.2	66223	12320	0.9359	0.5635
0	8.97×10^{6}	4.48×10^{6}	0.2061	0.2061

这张表能读出四件事。

第一，**主动容错是唯一在 $\rho\le0.4$ 之后还能保持有界的方案**。$\rho=0.2$ 时不处理与被动分别是 66223 与 12320，已经发散；主动是 0.9359。

第二，**再分配在中等故障上最有用**。$\rho=0.4$ 时它把 0.2337 压到 0.1848（降 21%），$\rho=0.2$ 时把 0.9359 压到 0.5635（降 40%）。故障越重，把控制权往备份通道挪的收益越大。

第三，**$\rho=0$ 时再分配与主动完全一样**（都是 0.2061）。因为 $\rho=0$ 被判成卡死，走的是单输入备份分支，这条分支本身就是“完全把主通道让出去”，再分配没有额外空间。

第四，**被动容错在 $\rho=0.6$ 时与不处理几乎相同**（0.1811 对 0.1807）。它按 $\rho=0.5$ 设计，遇到比预设更好的工况只是白费力气；遇到更差的工况则一起垮掉。**被动容错的性能曲线是“平移”而不是“重塑”**，这是它与主动容错最本质的差别。

78.6.8 三个指标为什么不总是一致

前面出现过好几次“某个方案峰值更小但代价更大”的情形。这不是数据矛盾，而是三个指标度量的东西不同，把它们混着看必然得出互相打架的结论。

J 是积分量，对短暂的大偏差不敏感。$\rho=0.5$ 时 k 从 2 加到 6，检测延迟从 0.16 s 拉到 0.30 s，几乎翻倍，而 J 只从 0.1493 涨到 0.1535，涨 2.8%。原因很直白：12 s 的积分窗里，多出来的 0.14 s 只占 1.2%，而这段时间里的偏差平方虽然是健康时的几十倍，乘上 1.2% 之后仍然微不足道。**积分会把“拖延”稀释掉，所以只看 J 会低估延迟的危害**。

峰值对延迟极其敏感，对稳态完全不敏感。同样这组数据里，延迟从 0.16 到 0.30 s，偏差在这段时间按 $e^{1.033t}$ 增长，多涨 $e^{0.145}=1.16$ 倍；如果检测时偏差已经接近权限边界，这 16% 就是“救得回来”与“救不回来”的分界。峰值恰恰是这类“临界失败”的唯一可见指标。

末段均方根对两者都不敏感，它只管“最后稳在哪儿”。$\rho=0.4$ 时主动容错的末段均方根是 2.28×10^{-3}，健康时是同一个量级；而不处理是 2.32×10^{-1}，差一百倍。这个指标看得出“有没有真正恢复跟踪”，看不出“恢复得多惊险”。

所以三个指标的正确用法是分工，而不是求和：

- 用**峰值**做准入判断（会不会撞限位、会不会触发保护），它是硬约束；
- 用**末段均方根**判断是否真的恢复了稳态精度，它区分“稳住了”与“停在那儿了”；
- 用 J 做方案之间的**总体排序**，它适合比较量级差异（0.23 对 0.81），不适合比较同量级内的细微差别。

硬切换与淡入的对比正好说明最后一条。硬切换 $J=0.2329$、跳变 1.024；淡入 1.0 s 后 $J=0.2339$、跳变 0.880。按 J 排，硬切换赢 0.4%；按跳变排，淡入赢 14%。**如果只报 J，会得出“淡入没必要”的错误结论**，而实际上执行器承受的冲击差了七分之一。指标选错，结论就反了。

78.7 参数意义与作用

78.7.1 参数全景

本章一共有九个可调的量，散落在检测、辨识、重构、执行四段里。先把它们摆在一起，后面逐个说。

参数	符号	本章取值	管什么	拧大的后果
阈值系数	k	4	报警门限	误报少、检测慢
滑动平均窗	w	5	残差平滑	噪声小、延迟大
辨识窗长	N	20	参数估计精度	估计准、切换晚
决策延迟	d	0	联锁与确认	更安全、代价高
淡入时间	T_b	0 或 $1.0\,\mathrm{s}$	切换平滑度	跳变小、过渡长
备份控力权重	R_{back}	0.1	备份通道的“吝啬度”	更保守、更迟钝
再分配权重	R_{11}	$1/\hat{\rho}^2$	主备用力比例	更省主通道、更慢
卡死判据	ρ_{th}	0.12	效率/卡死分界	卡死误判为效率
偏置判据	c_{th}	0.35	偏置/效率分界	偏置误判为效率

这张表里有一个隐含的结构：**前五个参数都在“时间”上做交易，后四个在“权限”上做交易**。时间类的参数都遵循同一条规律——更平滑就更慢，更慢就更贵；权限类的参数都遵循另一条——更保守就更容易活，但性能更差。

这个二分不是分类癖，它对应两套完全不同的调试手法。时间类的参数可以**离线算**：给定 k、w、N、d、T_b，检测延迟、辨识方差、闭环代价都能通过一次扫描画成曲线（第 78.6 节的几张图就是这么来的），调参时是在曲线上找点。权限类的参数必须**在线验**：R_{back} 与 R_{11} 改变的是“故障时谁出力”，这个只有把故障注进去、看轨迹才知道；两个判据 ρ_{th}、c_{th} 更是只有在故障谱上扫一遍才知道分得开不开。**离线能算的别在线试，在线才能验的别拿公式推**，这是九参数调起来不至于乱的前提。

九个参数之间还有三条硬耦合，动一个必须检查另外两个：

- w 与 d：w 的群延迟必须小于 d，否则判决还没稳定就执行了（第 78.3.5 节）。
- k 与 w：w 变大、σ_S 变小，同一个 k 对应的阈值会自动降低，等于隐式地放松了 k；改 w 必须重标定 μ_S、σ_S。
- N 与 T_b：两者相加构成“故障到新增益完全生效”的总时长，而这段时长要落在权限边界之内；N 拉长就必须把 T_b 压短，反之亦然。

三条耦合里最容易被忽略的是第二条。很多实现把 w 写死在代码里、把 θ 写死在配置里，改 w 时忘了改 θ，结果阈值相对残差的位置悄悄变了，误报率翻倍而没人知道原因。

78.7.2 阈值系数 k：误报与漏报的分配器

k 决定“多大的残差才算故障”。它同时控制两个错误率：误报（健康时报警）与漏报（故障时不报警）。这两个错误在容错控制里的代价不对称：

- 误报的代价是“白切一次”，系统切到保守增益，性能掉一截，但还活着。
- 漏报的代价是“该切不切”，对本质不稳定对象，这可能直接意味着发散。

不对称的代价意味着 k 不该取在误报率与漏报率的平衡点上，而应该**偏向漏报一侧**，也就是 k 取小一点、宁可多误报。本章的实测支持这个判断：$k=2$ 时误报率 0.03505（每 500 步约 18 次），但闭环代价只比 $k=6$ 高 2.8%（0.1493 对 0.1535）。误报的代价在这个对象上意外地低，因为它切到的是一套“仍然稳定、只是保守”的增益。

那为什么不干脆取 $k=2$？因为误报会污染辨识。健康时的误报会让系统去做一次本来不该做的辨识，估出来的 $(\hat{\rho},\hat{c})$ 是噪声的产物，可能落到某个奇怪的分区里，触发一次莫名其妙的重构。本章取 $k=4$，误报率 0.00047，即每 2000 步一

次，正好落在“能容忍但不至于频繁”的位置。

78.7.3 滑动平均窗 w：把噪声换成时间

w 的作用是把 w 个归一化残差平均后再取范数。对白噪声，平均能把标准差压到 $1/\sqrt{w}$；对本章这种有模型失配的情形，压缩比达不到理论值，但方向是对的。

代价是群延迟 $(w-1)/2$ 步。$w=5$ 时是 2 步（0.04 s），$w=20$ 时是 9.5 步（0.19 s）。这 0.19 s 在本质不稳定对象上不是小数：按时间常数 0.97 s 算，$e^{0.19/0.97}=1.22$，偏差在这段时间里要多涨 22%。

所以 w 的选择标准和一般滤波问题相反。一般滤波追求“在给定延迟下把噪声压到最小”，这里追求“在给定误报率下把延迟压到最小”——**噪声抑制不是目的，只是手段，目的是让阈值定得住**。$w=5$ 是这两者的折中：再小，σ_S 撑不住 $k=4$ 的阈值；再大，延迟吃掉的钱比省下的误报更贵。

78.7.4 辨识窗 N：精度与时间的正面对撞

N 是唯一一个“越大越好、但不能大”的参数。它越大，最小二乘的样本越多，$(\hat{\rho},\hat{c})$ 的方差越小；但辨识窗必须整段落在故障之后，窗口越长，重构完成得越晚，而这段时间系统正在发散。

本章取 $N=20$（0.4 s）。这个值不是算出来的，是试出来的：N 再大，卡死 $c=1.0$ 会在辨识完成前撞上限幅；N 再小，$\hat{c}$ 的估计抖动会大到让 $x_{3,\mathrm{eq}}$ 跟着抖，备份通道的指令跟着抖。

这里有一个容易被忽略的耦合：**N 的下限由最轻的故障决定，上限由最重的故障决定**。轻故障残差小，需要更长的窗才能估准；重故障留给你的时间短，窗必须短。如果这两个要求打架，说明这套诊断方案覆盖不了这么宽的故障谱，得回到传感器配置上想办法，而不是继续调 N。

78.7.5 决策延迟 d 与淡入时间 T_b

d 是“辨识完成到真正切换”之间刻意等的步数，用于联锁确认、执行器切换准备、上位机握手。$d=0$ 是理想情况，第 78.6.5 节的扫描说明它每增加 0.2 s，代价涨约 7%。

T_b 是切换的平滑度。图 78-7 是切换点附近的 u_1：硬切换（$T_b=0$）在切换瞬间有一个明显的台阶，淡入 1.0 s 则是一条斜坡。

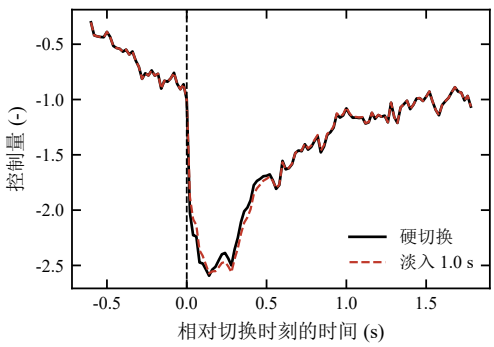


图 78-7 硬切换与 1.0 s 淡入在切换点附近的控制量

实测四种淡入时间的结果：

T_b（s）	J	u_1 最大跳变
0（硬切换）	0.2329	1.024
0.2	0.2333	0.892
0.5	0.2337	0.883
1.0	0.2339	0.880

两列数字讲了两件事。第一，**淡入几乎不要钱**：1.0 s 的淡入只让代价涨 0.4%（$0.2329 \to 0.2339$）。第二，**收益在 0.2 s 就基本吃完了**：跳变从 1.024 降到 0.892（降 13%），再拉长到 1.0 s 只多降到 0.880（再降 1.3%）。

所以 T_b 的选法是“取拐点”：0.2 s 附近。本章的图 78-7 特意画了 1.0 s 的对比，是为了让“斜坡”看得清楚；工程上没有必要拉这么长。跳变从 1.024 降到 0.89 这个量级，已经足够让功放和机械结构接受——真正的冲击来自切换前后增益矩阵的差异，而那个差异被线性插值抹平的速度，只取决于插值的前几步。

78.7.6 再分配权重：把主备用力比例写成矩阵

再分配是在重算 LQR 时，把主通道的控力权重按效率放大：

$$\boldsymbol{R}_{\text{eff}} = \operatorname{diag}\left(\frac{1}{\hat{\rho}^{2}},\ 1 \right) \tag{78-42}$$

$\hat{\rho} = 0.4$ 时 $R_{11} = 6.25$，是标称 0.01 的 625 倍。这等于告诉 LQR：“主通道现在很贵，能不用就不用”。它会把更多控制量分给备份通道，同时接受一个更慢的闭环——78.6.1 的表里，谱半径从 0.9373 涨到 0.9626。

这笔交易的收益在 78.6.7 的表里：$\rho = 0.4$ 时代价从 0.2337 降到 0.1848，$\rho = 0.2$ 时从 0.9359 降到 0.5635。**用稳定裕度换性能，在故障不重的时候是划算的；故障很重的时候，裕度本身就是稀缺资源，这笔交易就该停手。** 本章的做法是把它做成选项，默认关闭，由 78.9 节的判据决定什么时候打开。

78.7.7 两条分类边界：0.12 与 0.88

$\hat{\rho} < 0.12$ 判卡死，$\hat{\rho} < 0.88$ 判效率下降。为什么是这两个数？

0.12 这一侧的依据是权限。$\hat{\rho} = 0.12$ 意味着主通道只剩 12% 的力气，按 u_1 限幅 5 算，最大出力 0.6，已经低于备份通道的 3.6。这时继续用主通道不但没用，还会因为前馈放大 $1/0.12 \approx 8$ 倍而频繁饱和。所以 0.12 是“主通道还有没有资格参与控制”的门槛，不是“估计准不准”的门槛。

0.88 这一侧的依据是噪声。健康时辨识根本不启动，$\hat{\rho}$ 保持初值 1；这条线的作用是防止“被噪声意外触发的那一次辨识”把系统判成效率下降，同时给真实效率的估计误差留出安全带。如果取得太靠近 1（比如 0.97），一次估计偏差就会被当成轻故障，系统白白切到一套放大前馈的增益上。

偏置的判据 $\hat{\rho} > 0.80$ 且 $|\hat{c}| > 0.35$ 是另一回事：它要求“效率基本没掉，但指令被平移了”。两个条件必须同时满足，否则一个效率掉到 0.7 且带小偏置的故障会被误判成纯偏置，欠下的效率就没人补了。

这几条边界把 $(\hat{\rho}, \hat{c})$ 平面切成四块——健康、效率下降、卡死、偏置，第 78.8 节会用一张实测散点图检查这四块切得对不对。

78.8 调参方法与经验

78.8.1 五步流程，顺序不能颠倒

容错控制的可调参数多，容易一上来就盯着 k 和阈值反复试。正确的顺序是按“上游决定下游”排的：

1. **标称模型与观测器**。先把 $\boldsymbol{A}$、$\boldsymbol{B}$、$\boldsymbol{L}$ 定下来。这一步错了，后面所有统计量的意义都不成立。
2. **健康数据标定**。用没有故障的轨迹算出 μ_S、σ_S，画出分布，看尾巴有多长。

3. **故障谱扫描**。把要覆盖的故障逐个跑一遍，记录检测延迟、辨识结果、闭环代价。
4. **定阈值与窗口**。由第 3 步的最轻故障定 k 的上限，由最重故障定 N 的上限。
5. **重构与切换**。最后才调 R_{back}、R_{eff}、T_b 这些“性能旋钮”。

颠倒顺序的典型症状是：先调好了 T_b 觉得切换很平滑，回头改 k 之后发现检测晚了 0.1 s，峰值翻倍，前面调的全都白费。

78.8.2 先把健康状态标定清楚

第 2 步是整章里最枯燥、也最容易被跳过的一步。本章用 30 组健康轨迹、每组取后 500 步，共 15000 个样本，得到 $\mu_S = 0.6860$、$\sigma_S = 0.2966$、最大值 2.1326。对应的误报步数是：$k=2$ 时 $544/15000$，$k=3$ 时 $84/15000$，$k=4$ 时 $7/15000$，$k=5$ 与 $k=6$ 都是 0。

这组数字的价值在于它给出了“阈值能定多低”的硬边界。$k=4$ 的阈值 1.8722 只被越过了 7 次，$k=3$ 越过 84 次，差了一位数。如果只跑一组轨迹、只取 500 步，很可能一个误报都看不到，然后误以为 $k=3$ 是安全的。**15000 个样本不是为了算得准，是为了让尾巴露出来。**

78.8.3 故障谱扫描：七类故障的辨识结果

第 3 步的核心是“我能不能把每种故障都认出来”。下表是七类故障加健康的辨识结果。

注入故障	$\hat{\rho}$	$\hat{c}$	判定	检测延迟
效率 0.6	0.600	0.000	效率下降	0.30 s
效率 0.4	0.400	-0.000	效率下降	0.14 s
效率 0.2	0.200	-0.000	效率下降	0.12 s
卡死 2.0	0.000	$+2.000$	卡死	0.04 s
卡死 -1.5	0.000	-1.500	卡死	0.10 s
偏置 0.8	1.000	$+0.800$	偏置	0.08 s
偏置 -0.5	1.000	-0.500	偏置	0.14 s
健康	1.000	0.000	健康	未检出

七类故障全部判对，两个估计几乎打在中点上（误差在第三位小数）。这个结果比预期好，原因有两个：本章的故障是常值或常系数，不含时变；量测噪声也不大（位置 5×10^{-3}、速度 2×10^{-2}）。真实系统里故障参数会漂、噪声会更大，辨识误差通常要到第二位小数。

延迟那一列值得单看：卡死 2.0 只用了 0.04 s 就检出，效率 0.6 用了 0.30 s，差 7.5 倍。故障越大，残差项得越高，越过阈值越快——这条规律贯穿全章。

图 78-5 把这些点画在 $(\hat{\rho},\hat{c})$ 平面上，同时画出四条分类边界。

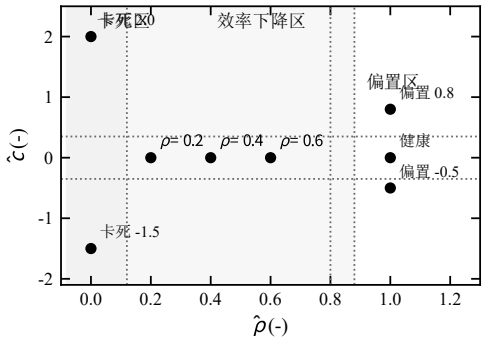


图 78-5 辨识得到的故障参数在平面上的落点与四条分类边界

八个点分得很开：三个效率点沿横轴排在 0.2、0.4、0.6；两个卡死点贴着纵轴、分别在 ± 2 附近；两个偏置点在 1.0 的竖线上；健康点落在右上角 $(1, 0)$。没有点落在边界附近——这是分类能全对的原因，也说明本章选的那几条边界留了足够余量。**如果某个点贴着边界，那才是需要回头改传感器或改窗长的信号。**

78.8.4 最轻的故障决定整套参数

调参时最容易犯的错，是拿一个“典型故障”（比如 $\rho = 0.4$）调到底，然后宣布完工。真正的约束来自最轻的那一端。

$\rho = 0.9$ 时，12 s 内残差根本没越过阈值，检测延迟记为“未检出”。这不是算法不好，是信噪比不够：主通道只掉了 10%，残差里的故障分量被噪声与失配淹了。$\rho = 0.8$ 勉强能检出，要 0.46 s，是 $\rho = 0.4$ 的将近两倍。

更极端的一个实验：把参考幅值从 0.30 压到 0.02，同样 $\rho = 0.4$ 的故障，检测延迟从 0.24 s 拖到 1.24 s。参考幅值只剩原来的 $1/15$，延迟涨了五倍。这说明**检测能力不只取决于故障多大，还取决于系统平时“动得多不多”**——一个长期在稳态附近小幅度运行的系统，容错诊断天然迟钝。

所以第 3 步扫描时，必须把“最轻故障 + 最小指令”这一格单独跑一遍。它的结果才是 k 与 w 的真正约束。

78.8.5 中等故障的估计误差会被放大

还有一个反直觉的规律，值得单独拿出来讲：**效率越低，估计误差对前馈的影响越大**。

前馈项里有 $1/\hat{\rho}$，它对 $\hat{\rho}$ 的敏感度是

$$\frac{\partial}{\partial \hat{\rho}}\left(\frac{1}{\hat{\rho}}\right) = -\frac{1}{\hat{\rho}^2} \tag{78-43}$$

$\hat{\rho} = 0.6$ 时这个系数是 2.8，$\hat{\rho} = 0.2$ 时是 25。同样是 0.01 的估计误差，在 $\rho = 0.6$ 上只造成 2.8% 的前馈偏差，在 $\rho = 0.2$ 上造成 25%。这就是 78.5.4 里 $\rho = 0.2$ 的代价 0.9397 比 $\rho = 0$ 的 0.2111 还高的另一半原因：$\rho = 0$ 走卡死分支，前馈不除 $\hat{\rho}$，反而躲开了这个放大。

工程含义：**中等偏重的效率故障（ρ 在 0.2 到 0.4 之间）是这套方案最脆的一段**。要么把辨识窗拉长提高精度（但会拖慢），要么给 $\hat{\rho}$ 设一个下限（比如 $\hat{\rho} \ge 0.15$ 时才允许 $1/\hat{\rho}$ 折回），要么干脆把这一段并入卡死分支、直接切备份。本章代码里用的是 $\max(\hat{\rho}, 0.05)$ 的下限保护，只防除了零，没有防放大。

78.8.6 阈值系数的选法

综合 78.6.6 与 78.7.2，k 的选法可以写成三步：

1. 用健康数据算出 μ_S、σ_S，画出直方图，找到最大值 $S_{\max}$。可接受的最小 k 是 $(S_{\max} - \mu_S)/\sigma_S$ 向上取整再加 1。本章 $S_{\max} = 2.1326$，算出 4.88，取 5 就没有误报；取 4 则有 $7/15000$ 的误报，这个量级可以接受。
2. 用最轻的必须检出的故障跑一遍，看延迟能不能接受。$\rho = 0.8$ 时 $k = 4$ 的延迟是 0.46 s，若能接受就往下走。
3. 在这两点之间取小的一侧，因为漏报比误报贵。

本章最终取 $k=4$，落在“有极少量误报、检测尽量快”的位置。这个选择是本章这套参数下合理的，换一个噪声水平、换一个采样率，第 1 步算出来的数就变了，不能照搬。

78.8.7 六条踩过的坑

把这一节的经验压成六条，都是本章实现过程中真实撞过的：

1. **辨识窗不能长于“系统撑得住的时间”**。$N=40$（0.8 s）时卡死 $c=1.0$ 在辨识完成前就发散了，改成 $N=20$ 才活。
2. **卡死必须单输入重设计**。两输入 LQR 算完再砍第一行，得到的第二行会把备份通道用错方向，$c=1.0$ 时直接发散。
3. **备份通道要有自己的控力权重**。沿用主通道的 $R=0.01$，备份通道会把自己顶到限幅，效果比不用还差。
4. **观测器必须跟着故障模型改**。卡死后仍按 $[0,u_2]$ 送观测器，辨识出的 $\hat{c}$ 会一路漂，重构增益跟着错。
5. **偏置不能当卡死处理**。偏置 0.8 时按卡死处理，峰值会从 0.083 冲到 0.294 以上。
6. **阈值不能跨采样率沿用**。T_s 一变，σ_S 就变，旧阈值不是偏保守就是狂误报。

这六条没有一条是理论问题，全是“知道该怎么做、但实现时漏了”的类型。容错控制的调试时间，八成花在这类地方。

78.9 优化与改进

前面八节给的是一个能跑、能验、边界清楚的最小方案。这一节讨论往上加什么。每一条都标清楚“本章实测过的”与“只是提案、没有实测”，不含糊。

78.9.1 再分配该不该开：一笔明账

78.6.7 的表已经把收益摆出来了：$\rho=0.4$ 时再分配把代价从 0.2337 压到 0.1848（降 21%），$\rho=0.2$ 时从 0.9359 压到 0.5635（降 40%）。收益随故障加重而变大，看起来“越坏越该开”。

但 78.6.1 的极点表给了另一半：$\rho=0.4$ 时谱半径从 0.9373 涨到 0.9626，$\rho=0.2$ 时涨到 0.9635。稳定裕度从“离单位圆 6.3%”变成“离单位圆 3.7%”。

把两件事放在一起，判据就出来了：**再分配只在“还有裕度可花”的时候开**。本章的建议是

$$\rho_{\mathrm{cl}}\left(\hat{\boldsymbol{K}}_{\mathrm{realloc}}\right)<0.96 \quad \text{且} \quad |\hat{c}|<0.5 \tag{78-44}$$

第一条防的是“为了性能把稳定性花光”；第二条防的是“卡死故障下还去分配”——卡死走的是单输入分支，压根没有分配空间。$\rho=0.4$ 与 $\rho=0.6$ 满足，$\rho=0.2$ 不满足（$0.9635>0.96$），这与“$\rho=0.2$ 本来就更脆”的判断一致。

这条判据是本章从数据里读出来的，没有做过闭环验证。真要上机，应当在 $\rho=0.2$ 附近做一组对比实验，看 0.96 这条线该画在 0.96 还是 0.97。

78.9.2 自适应阈值：让门限跟着指令走

78.8.4 暴露的问题很清楚：参考幅值从 0.30 压到 0.02，检测延迟从 0.24 s 涨到 1.24 s。固定阈值在这种工况下必然失效——因为残差里故障分量的幅度正比于 $|u_1|$，而阈值是固定的。

一个自然的改法是让阈值随最近的指令幅度缩放：

$$\theta_k=\mu_S+k\,\sigma_S+\kappa\cdot\overline{|u_1|}_{k} \tag{78-45}$$

其中 $\overline{|u_1|}_{k}$ 是最近一段的指令绝对值均值，κ 是一个正常数。指令大时门限抬高（防止大指令下的模型失配误报），指令小时门限回落（保持对小故障的灵敏）。

这个改法有一个风险：**门限跟着指令走，等于把“故障引起的 u_1 变化”也算进了门限**。故障后控制器通常会加大 u_1（式 (78-27) 的 $1/\hat{\rho}$ 折回），门限跟着抬高，可能把故障“藏”起来。所以 $\overline{|u_1|}$ 必须取故障前的滑动窗，或者干脆用参考幅值

A_r 代进去——后者更稳，因为它不受故障影响。

这一条同样只是提案。要做实，得在 $\rho = 0.4$、$A_r = 0.02$ 这一格上跑一组 κ 的扫描，看延迟能不能从 1.24 s 压下来，同时看健康工况的误报率有没有涨。

78.9.3 多模型假设检验

本章的辨识是“一段数据、一次最小二乘、一个结论”。它的三个弱点都在 78.5.5 与 78.7.4 里露过：要等窗满、需要持续激励、条件数不好时会放弃。

多模型自适应估计（MMAE）是另一条路。开跑之前先摆好一组假设模型——健康、$\rho = 0.8$、0.6、0.4、0.2、卡死 $c = \pm 0.5$、± 1.0、± 2.0、偏置 ± 0.5、± 0.8——每个模型配一个卡尔曼滤波器，逐拍算残差的似然，再归一化成概率：

$$p_k^{(i)} = \frac{p_{k-1}^{(i)} \, L_k^{(i)}}{\sum_{j} p_{k-1}^{(j)} \, L_k^{(j)}}, \qquad L_k^{(i)} = \frac{1}{\sqrt{|2\pi \mathbf{\Sigma}_i|}} \exp\left(-\frac{1}{2} \boldsymbol{r}_k^{(i)\mathsf{T}} \mathbf{\Sigma}_i^{-1} \boldsymbol{r}_k^{(i)} \right) \tag{78-46}$$

取概率最大的那个当结论，或者干脆按概率加权融合各模型的增益。

MMAE 的好处是不用等窗满：故障发生后几步之内，与故障最匹配的那个模型的似然就会跳起来，概率从均匀分布迅速集中。代价是算力——十几个模型并行跑，每个都要做一次 3×3 的协方差更新，在 0.02 s 的周期里仍然宽裕（十几个 3×3 矩阵乘法在微秒量级），但对多点悬挂、多轴联动的平台，模型数会乘上去。

MMAE 的坏处是“假设集里没有真值”时就尴尬：真故障是 $\rho = 0.5$，假设集里只有 0.6 与 0.4，它会在这两个之间来回跳，概率都不高。本章的最小二乘反而没有这个问题，它估的是连续值。**实践中的折中是：先用 MMAE 快速分类，再用最小二乘在分类附近细化。**

78.9.4 切换的无扰过渡

78.7.5 的淡入解决的是“增益跳变”，还有两个跳变源没处理。

第一个是**观测器状态**。切换前后，观测器用的是不同的输入模型（$[\hat{c}, u_2]$ 与 $[\hat{\rho} u_1, u_2]$），$\hat{\boldsymbol{x}}$ 会在切换瞬间带上一个偏差。因为本章 $\boldsymbol{C} = \boldsymbol{I}$、三个状态全测，最省事的做法是切换瞬间直接把 $\hat{\boldsymbol{x}}$ 重置成 $\boldsymbol{y}_k$，让观测器从真值重新起步。这一步只要一行代码，能省掉一段几十毫秒的观测器重新收敛。

第二个是**积分或前馈项的累积**。本章的控制器没有积分器，所以没有这个问题；如果换成一个带积分的结构（第 44 章与第 106 章讨论过），切换时积分项必须一起重置，否则它会带着“旧故障下的偏差”继续积，产生一个缓慢的过冲。

第三个是**执行器本身的动态**。指令跳变再平滑，线圈电流也有自己的响应时间；如果功放有速率限制，实际作用量的过渡会比指令更慢。这一点在仿真里看不到，只有在台架上加电流环才能观察到。

78.9.5 主动探测：给诊断加激励

78.5.5 说过，辨识需要持续激励；78.8.4 说过，指令小的时候检测会变慢。两条合起来指向一个大胆的想法：**当系统怀疑有故障、但残差还没到门限时，主动往指令里加一点探测信号。**

探测信号可以是一个小幅正弦（比如幅值 0.05、频率 3 Hz）、一段伪随机序列、或者一次短促的阶跃。它的作用是把 u_1 的摆幅撑起来，让最小二乘的回归矩阵条件数变好，从而把“未检出”变成“可检出”。

代价是显而易见的：探测信号本身会激励对象，产生额外的位置波动。在一个要求亚微米定位的平台上，0.05 的探测幅值可能已经超过精度预算。所以这条路只适合“精度要求不高、但必须知道有没有坏”的场合，比如姿态控制、悬浮间隙的粗定位。

本章没有实现主动探测，只在 78.8.4 的实验里看到了它的必要性：$A_r = 0.02$ 时检测延迟 1.24 s，这个数字基本等于“没有激励就没有诊断”。

78.9.6 数据驱动方法能放在哪

近几年故障诊断的论文大量用深度学习：把残差序列喂进 LSTM 或 Transformer，直接输出故障类别。这些方法在基准数据集上指标很漂亮，但在本章这个场景里，位置很有限。

原因有三个。第一，**样本不平衡**。健康数据要多少有多少，卡死 $c = 2.0$ 这种故障可能一辈子遇不到一次，训练集里只有仿真出来的几百条。第二，**可解释性**。容错控制的输出要驱动执行器切换，出了事故要能说清“为什么在 $t = 3.42$ s 切了”，一个黑箱给不出这个交代。第三，**认证**。航空、医疗、核电这些真正需要容错的领域，都要求算法行为可以被形式化验证，神经网络目前过不了这一关。

数据驱动真正能发挥作用的位置是**残差的上游**：用自编码器学健康工况的流形，把“偏离流形”的程度当作残差。这相当于把本章的线性观测器换成一个非线性观测器，后面的检测、辨识、重构逻辑完全不用改。这是一个务实的接口——**换掉感知，保留决策**。

78.9.7 本章没做的三件事

最后诚实列一下边界。

第一，**没有做故障的时变与间歇**。本章所有故障都是“某一时刻注入、之后一直存在”。真实故障可能是渐进的（匝间短路一步步恶化）、可能是间歇的（接触不良时好时坏）。渐进故障适合用漂移检测（CUSUM、GLR），间歇故障需要额外的“故障确认/撤销”逻辑，本章的“切了不回头”结构处理不了。

第二，**没有做多点故障**。主通道坏了、备份通道也坏了，这套方案只能保证“识别出无解”，不能保证“给出最优的降级策略”。多点故障需要的是故障树与权限的联合分析，属于系统工程的范畴。

第三，**没有做执行器动力学的高保真建模**。本章的备份通道是一个一阶惯性，真实线圈还有电流环、磁饱和、涡流损耗。这些非线性在故障后会放大——因为故障后的工作点离标称更远。上台架的第一件事应该是辨识这些非线性，而不是继续调 k。

78.9.8 如果坏的是传感器

本章从头到尾假设执行器坏、传感器好。这个假设一旦反过来，整套推理要重做，不能照搬。原因在于残差的定义方式：残差是“量测减去模型预测”，执行器故障会让预测偏低，残差**指向故障的方向**；传感器故障会让量测本身偏，残差同样非零，但它**指向的不是对象，是量测**。用同一套逻辑去解释，会把“传感器在撒谎”误判成“执行器没出力”，然后加大指令去补一个根本不存在的缺口——这是容错控制里最危险的误动作之一。

要区分这两类，必须靠**冗余的观测**而不是更聪明的算法。本章的 $\boldsymbol{C} = \boldsymbol{I}$ 是幸运的特例：三个状态全测，任意一个通道坏掉，另外两个通道的残差会给出矛盾的信息，矛盾本身就是“传感器坏了”的指纹。具体做法是给每个量测通道单独算一份残差，检查它们在故障方向上是否一致：执行器故障会让三份残差都朝 $\boldsymbol{b}_1$ 方向偏，传感器故障只让其中一份偏。

如果只有单套传感器、没有冗余，那么执行器故障与传感器故障在数学上不可分——这是结构性不可分，不是算法不够好。此时唯一的出路是加硬件（三取二，第 78.10.3 节的可用度是 0.9997），或者接受“只能检测到异常、无法隔离位置”，把所有异常都按最保守的方式处理（停机或降级）。

本章没有实现传感器故障的处理，也没有实测数据支持上面这段推理。把它写在这里，是因为读者很容易把“容错控制”当成一个通用框架，以为换个观测器就能覆盖所有故障类型。**故障可隔离性的前提是信息冗余，不是算法复杂度**，这条边界比任何一条调参经验都更硬。

78.10 工程落地

78.10.1 故障注入测试怎么设计

仿真里“能活”不算数，要按测试矩阵逐格验。本章的做法是六种故障乘三种策略，每格跑三个随机种子，判据是

$$\max_{t>t_f}|x_1(t)-r(t)|<1.0 \quad \text{且} \quad \sqrt{\frac{1}{N_t}\sum_{t>T-3}^{T}\left(x_1-r\right)^2}<0.05 \tag{78-47}$$

第一条管瞬态，第二条管稳态（末段 3 s 的均方根）。为什么要两个判据而不是只看“有没有发散”？因为“发散”是个二值判断，很多情况是**没发散但已经废了**：轴被推到一个偏置位置、停在那里不动，数值有界，但精度全丢。单看有界性会把这种情况判成“通过”。

这两个阈值是工程约定，不是从硬件参数推出来的。1.0 对应“不超过机械限位的一半”，0.05 对应“末段误差不超过参考幅值的 $1/6$”。换一台平台，这两个数要按它的行程与精度要求重定。

78.10.2 十八格矩阵里只有五格通过

图 78-10 是这张矩阵的热图，格内标注的是三个种子里的最大峰值。

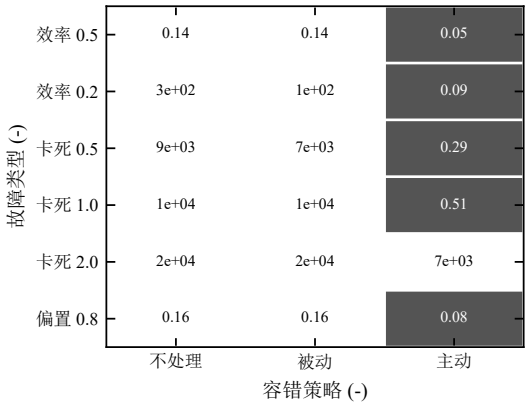


图 78-10 六种故障乘三种策略的注入测试矩阵，格内为故障后峰值

逐行看：

故障	不处理峰值	被动峰值	主动峰值	主动是否通过
效率 0.5	0.142	0.143	0.047	通过
效率 0.2	348.9	148.8	0.091	通过
卡死 0.5	8888	6744	0.294	通过
卡死 1.0	13860	10993	0.507	通过
卡死 2.0	24132	20235	6598.832	不通过
偏置 0.8	0.160	0.163	0.083	通过

这张表比任何一段文字都能说明问题。

效率 0.5 那一行：不处理的峰值只有 0.142，看起来“没什么事”，但它末段均方根超了 0.05，所以判不通过。这就是双判据的价值——**它抓住了“有界但精度已失”这一类**。

效率 0.2 那一行：不处理 348.9、被动 148.8，两者都很大；主动 0.091。差三个数量级。

卡死 0.5 与 1.0：不处理与被动都在万的数量级，主动分别是 0.294 与 0.507。注意 0.507 已经接近 1.0 的判据边界，说明卡死 1.0 是这套方案“勉强够用”的位置。

卡死 2.0：三列全不通过，主动的峰值 6598.832 与不处理的 24132 同属发散，只是慢一点。这就是 78.2.8 的权限判据 $|x_1|<(3.6-2.0)/1.44=1.11$ 在起作用——0.4 s 的辨识窗里偏差已经越过 1.11，之后无论怎么切都拉不回来。

偏置 0.8：三列都没发散，但主动的 0.083 比不处理的 0.160 好一倍。偏置是最“温和”的故障，因为通道还完全可控。

十八格里主动列通过五格、其余两列零格。**被动容错在这张表里一格都没通过**，这个结果值得单独说一句：它并不是“更差”，而是它按 $\rho = 0.5$ 设计，遇到比 0.5 好的工况（效率 0.5、偏置 0.8）时性能与不处理几乎相同（0.143 对 0.142，0.163 对 0.160），遇到比 0.5 差的工况时与不处理一起垮掉。**没有辨识，就没有针对性的补偿，这是被动容错的天花板。**

78.10.3 冗余配置与可用度

容错控制的另一半是硬件：有没有第二套东西可用。假设单个执行器的任务成功率是 0.98（失效率 2%），不同冗余拓扑的结果是：

配置	成功率	说明
单执行器 1oo1	0.9800	一个坏了就完
热备份 1oo2	0.9995	两个同时坏才完，理论值 $1 - 0.02^2 = 0.9996$
休眠备份 1oo2	0.9970	备份平时不工作，唤醒有 15% 失败率
单传感器 1oo1	0.9900	失效率 1%
三取二 2oo3	0.9997	允许一个传感器坏

这张表里最值得注意的是热备份与休眠备份的差距：0.9995 对 0.9970。差别全在“备份平时是否通电运转”。热备份随时可用，但备份本身也在磨损、也在发热、也可能坏；休眠备份寿命长，但唤醒需要时间，而且唤醒机构本身有 15% 的失败率。

本章的对象是磁悬浮，备份线圈必须预通电才能维持悬浮，所以本质上只能是热备份。如果换成液压或气动执行机构，休眠备份就变得可行——**冗余拓扑不是算法选择，是执行机构决定的。**

三取二传感器的 0.9997 说明一件事：**冗余的收益是乘性的，成本是线性的。**从 0.99 到 0.9997 只多了两个传感器，但失败概率降了 33 倍。这笔账在关键系统上永远划算；在不关键的系统上永远不划算。所以冗余要加在“最弱的那一环”——先看哪个环节的失效率最高，再决定在哪加。

78.10.4 切换瞬态与执行器保护

仿真里切换是一行赋值，实机上有三道坎。

第一道是**速率限制**。指令从旧值跳到新值，功放和机械结构都要吃这一下。78.7.5 用淡入解决，工程上更常用的写法是逐拍限幅：

$$|u_{1,k} - u_{1,k-1}| \le \Delta u_{\max} \tag{78-48}$$

$\Delta u_{\max}$ 由功放的电流上升率与线圈电感决定。它比淡入更“硬”，但实现更简单，而且能顺带保护所有非切换时刻的指令跳变。本章的淡入是它的平滑版本，两者可以叠加使用。

第二道是**切换确认**。真机上切换要动继电器或接触器，动作时间 10 到 50 ms，而且可能抖动。这意味着第 78.6.5 节里 $d = 0$ 的那一档在实机上不存在，至少要按 $d = 1$ 到 $d = 3$ 步（0.02 到 0.06 s）来预算。对应的代价增量约 1% 到 2%，不大，但必须算进去。

第三道是**备份通道预热**。热备份本来就在运转，没有这个问题；如果备份是“冷”的（比如需要预充电的电容、需要预励磁的线圈），切换后它需要一段建立时间。这段时间里主通道已经失效、备份还没起来，系统处于“真空期”。

真空期的长度必须小于权限允许的时间。按式 (78-30)，$c = 0.5$ 时边界是 2.15，偏差从 0.1 涨到这条线，按 $e^{1.033t}$ 算需要 $\ln(21.5)/1.033 \approx 3.0$ s；$c = 1.0$ 时边界 1.81，需要 2.8 s。这两个数看起来余量很大，但它们只是“轴还没飞出去”的时间，不是“还救得回来”的时间——偏差越大，切换后要拉回来的行程越长，末段均方根越难满足判据。工程上把真空期控制在 0.2 s 以内才算稳妥，超过 0.5 s 就应该重新设计冗余拓扑。

78.10.5 时序与算力预算

控制周期 $T_s = 0.02$ s，20 ms 里要干完这些活：

1. 采样三路状态，做一次符号与量纲检查；
2. 观测器更新：一次 3×3 矩阵乘向量，约 27 次乘加；
3. 残差归一化、5 点滑动平均、取范数、比阈值；
4. 若报警：填辨识窗、解 2×2 最小二乘、分类、查表取增益；
5. 算控制量、限幅、限速率、下发。

前四步里唯一有变量的是第 4 步。本章的实现没有在线上解 Riccati 方程——**所有重构增益都是离线算好、按 $\hat{\rho}$ 网格存表的**，线上只做查表与插值。这是工程上的标准做法：LQR 的增益是 $\boldsymbol{B}_{\text{eff}}$ 的光滑函数，0.05 的网格足够，插值误差远小于辨识误差。

真正要盯的不是平均耗时，是**最坏执行时间**。第 4 步在辨识窗刚好填满那一拍要解一个小方程组，耗时是平时的几倍。如果控制周期用满，那一拍就可能超时。留 50% 余量是底线。

最坏执行时间要在两个维度上分别测：**单拍最坏**与**单故障最坏**。单拍最坏出现在“辨识窗刚好填满、条件数判过、查表命中一个需要插值的网格点”这一拍，是三者叠加；单故障最坏出现在卡死分支，因为它要改控制结构、要算平衡点、还要同步改观测器输入。两者不一定是同一拍，预算必须取它们的最大值，而不是平均值乘以某个系数。

还有一处容易被忽略的开销：**日志写入**。第 78.10.7 节要求每拍记五六个字段，如果这些字段同步写 Flash 或走慢速总线，一次写入可能就是几百微秒，比整个控制律还贵。工程上的做法是双缓冲：控制周期内只写 RAM 环形缓冲，后台任务慢慢落盘；环形缓冲满了就丢最旧的，绝不在控制周期里等 I/O。

最后要说清一件事：0.02 s 的周期对本章这套算法是宽裕的，但这不构成“容错控制很便宜”的结论。真正贵的是**标定与验证**——15000 个健康样本、十八格注入矩阵、每格三个种子，这些离线工作的量级远大于线上那几十微秒。**容错控制的成本在开发期，不在运行期**，这是它与普通控制器在投入结构上最大的不同。

78.10.6 安全联锁与降级运行

容错控制有个前提假设：**故障之后系统还值得救**。这个假设不总成立。

按式 (78-30)，$c = 2.0$ 时权限边界只有 1.11。如果辨识完成时偏差已经过了这条线，正确的动作不是“切换”，而是“降级”——切断主通道、让轴落到机械限位上、或者启动受控降落程序。**继续尝试切换只会让系统在更大的偏差上发散，损失更大。**

所以完整的逻辑应该是三段，而不是两段：

1. 检测到故障 → 尝试重构（本章的全部内容）；
2. 重构后偏差仍在扩大 → 判断是否越过权限边界；
3. 越过 → 执行降级程序（fail-safe），不再试图维持悬浮。

第 3 段必须由**独立的硬件回路**实现，不能依赖软件诊断。因为最需要它的场景恰恰是“软件已经算错了”的场景。常见的做法是一个独立的间隙开关或过流继电器，直接切断功放，不走 CPU。

这里还有一个概念要分清：**fail-safe** 与 **fail-operational**。前者是“坏了就安全停机”，后者是“坏了还要继续工作”。本章做的是 fail-operational（切到备份继续悬浮），但它的兜底必须是 fail-safe（实在救不了就停）。两套逻辑要同时存在，不能只有一套。

78.10.7 诊断日志：为下一次标定留数据

上线之后，最有价值的产出不是“这次没出事”，而是“出过什么事、当时的数字是多少”。建议每一拍至少落这些字段：

字段	用途
S_k 与 θ	事后看余量，判断阈值是否偏紧
$\hat{\rho}$、$\hat{c}$	故障的真实幅值，用于修订分类边界
条件数	判断那次辨识是否可信
决策码（健康/效率/卡死/偏置）	统计各类故障的发生频次
切换时刻与切换前后增益	复盘切换瞬态

有了这些数据，μ_S、σ_S 可以按真实工况重新标定，k 可以按真实误报率调整，分类边界可以按真实故障分布修订。**容错控制不是一次设计完就结束的，它是靠运行数据不断收紧的。**

78.10.8 上台架前的检查清单

把这一节压成一张清单：

1. 故障注入矩阵跑满了吗？六个故障、三个种子，一格不漏。
2. 双判据（峰值 + 末段均方根）都设了吗？只看发散会漏掉“有界但废了”的情况。
3. 权限边界算了吗？c 最大时边界是多少，辨识窗结束时偏差在哪。
4. 降级程序有吗？它是不是独立的硬件回路。
5. 切换延迟按实机的继电器时间算了吗？不是仿真的 $d = 0$。
6. 备份通道的预热时间算进权限预算了吗？
7. 最坏执行时间留了 50% 余量吗？
8. 诊断日志的字段够不够支撑下一次标定？

这八条里，第 3、4、6 条是安全相关的，其余是性能相关的。安全相关的三条不过，系统不该上电。

78.11 优缺点与适用边界

78.11.1 优点

第一，它把“能不能活”从运气变成设计。 不处理的 $\rho = 0$ 是 8.63×10^{6}，主动容错是 0.2151。这不是调参调出来的差距，是“有没有一套机制去发现并补偿”的差距。对本质不稳定对象，这条差距就是“在轨”与“坠毁”的差别。

第二，代价是可控的、可预算的。 健康时的 J 是 0.0473，容错机制全程在线也只是让它在故障后涨到 0.2344。检测、辨识、重构三段的开销都在 10^{-1} s 量级，对 $T_s = 0.02$ s 的控制周期完全不构成负担。

第三，故障信息被显式地估出来。 $\hat{\rho}$ 与 $\hat{c}$ 不只是中间变量，它们是可读的诊断输出。有了它们，维护人员知道“坏了多少”，而不是只知道“坏了”。这一点在 78.10.7 的日志设计里体现得最清楚。

第四，模块化好，接口清楚。 观测器、检测、辨识、重构四块可以分别替换：换成 MMAE 只动辨识，换成非线性观测器只动第一块，换成 MPC 只动重构。78.9.6 说的“换掉感知、保留决策”能成立，就是因为这个分层。

第五，对故障幅值是连续的、不需要枚举。 最小二乘估的是连续值，$\rho = 0.37$ 这种“不在任何预设档位上”的故障也能处理。这是它相对查表式方案（包括 MMAE）的实质优势。

78.11.2 缺点

第一，依赖模型的准确性。 观测器、辨识、重构三处都用标称模型。模型失配会同时抬高 μ_S（检测变钝）和污染 $\hat{\rho}$（重构偏移）。本章的失配来自线性化与离散化，可以接受；如果对象有明显非线性（磁饱和、间隙），失配会大得多，这套方案需要重新标定甚至重构。

第二，检测有不可消除的延迟下限。 78.5.4 测到的 0.16 s 是“1 步量测 + 4 步平滑 + 3 步裕度”，想再快要动硬件。对时间常数更小的对象（比如 0.05 s 级的），这个下限可能已经等于对象的时间常数，方案直接失效。

第三，中等故障最难，恰好是最常见的区间。 $\rho = 0.2$ 的代价 0.9397 高于 $\rho = 0$ 的 0.2111，$1/\hat{\rho}$ 的放大（78-43）在 $\rho = 0.2$ 时是 25 倍。真实故障多数落在“效率掉了一半”这种区间，而不是“彻底卡死”。

第四，多点故障与间歇故障处理不了。 78.9.7 已经列过。这套方案的结构是“一次判定、单向切换”，对“好了又坏、坏了又好”没有对策。

第五，参数多，标定工作量大。 九个参数，其中四个（k、w、N、判定边界）都依赖实测数据。78.8 整节的篇幅都在讲怎么标，这本身就是成本。

第六，它是“止损”不是“修复”。 故障后的性能最好也只有健康的 $1/5$（0.2344 对 0.0473）。所有“容错之后跟没坏一样”的说法都是不成立的，能承诺的只是“不散架”。

78.11.3 适用边界

把上面的优缺点化成一张判断表：

维度	适合	不适合
对象稳定性	开环不稳定、失效即发散	开环稳定、失效只是变差
故障类型	执行器效率/卡死/偏置，常值或慢漂	传感器故障、对象参数突变、间歇故障
时间尺度	故障后有几倍时间常数可周旋	故障后一步就失稳
传感器	状态全测或至少关键状态可测	只有输出可测、状态全靠推断
冗余	有备份执行器或备份通道	单通道、无冗余
算力	有 10 倍以上余量	算力已用满、无余量做诊断
安全等级	可接受“降级运行”	要求“故障后性能不变”

其中“时间尺度”这一行最容易被误判。本章的检测下限 0.16 s，而对象的时间常数 0.97 s，比值 $1/6$，所以来得及。如果对象的时间常数是 0.1 s，同样的检测延迟就是 1.6 倍时间常数，切换完成时轴早就飞了。**判断能不能用容错控制，先算这个比值，而不是先看算法多先进。**

78.11.4 与相邻章的分工

这一章反复用到 LQR 与观测器，但它们不是本章的主题，只是构件。

- **PID** 见第 44 章与第 106 章。本章的控制器如果换成 PID，容错框架完全不变——检测、辨识、重构三段与控制器形式无关。反过来说，PID 那两章讲的“怎样把跟踪做好”，在本章是健康时的基线，不是重点。
- **MPC** 见第 58 章与第 80 章。MPC 天然带约束处理，把它放进重构环节是 78.9 提过的方向；但 MPC 的在线优化会吃掉 78.10.5 的算力余量，所以本章选了查表 LQR。
- **Smith 预估** 见第 47 章与第 107 章。Smith 预估处理的是“纯延迟”，本章处理的是“执行器失效”，两者都会破坏“控制器以为的对象”，但破坏的方式不同：前者是相位滞后，后者是增益与偏置。同一个观测器框架可以同时挂这两种补偿，但诊断逻辑要分开写，否则残差里两种原因混在一起，谁也认不出来。

一句话概括分工：**前面的章解决“正常情况下怎么控制好”，本章解决“不正常时怎么不死”。**

78.11.5 什么时候不该用容错控制

有四种情形，容错控制是错的选择。

第一种，对象开环稳定、故障只导致性能下降。 这时容错机制的复杂度、误报风险、标定成本，都超过它带来的收益。一个温度回路的热电偶坏了，换个传感器就行，不需要在线辨识与重构。

第二种，故障率极低且失效后果可控。 如果设备的失效率是 10^{-6}/h、失效后只是停机检修，加一套容错系统的成本（设计、验证、认证）远高于停机损失。

第三种，没有冗余可用。 单执行器、单传感器、无备份通道，主动容错能做的只有“识别出无解”，那是诊断问题，不是控制问题。这时应该把精力放在诊断与降级上（78.10.6），而不是重构。

第四种，安全等级要求“故障后性能不变”。 容错控制做不到这一点，它只能做到“不散架”。要求性能不变的系统，需要的是物理冗余（两套完整的执行机构并联），不是算法容错。

78.11.6 三条收束

把整章压成三句话。

第一，容错控制的本质是“用模型的一致性换时间”。 观测器提供一个“应该是什么”的参照，残差度量“实际偏离了多少”，辨识把偏离翻译成参数，重构把参数翻译成新的控制律。整个过程消耗的是时间，换来的是“知道自己坏了”这个信息。

第二，时间的价值由对象的不稳定程度决定。 同一个 0.16 s 的检测延迟，在时间常数 0.97 s 的对象上是富余，在 0.1 s 的对象上是灾难。所以评价一套容错方案，第一件事是算“检测延迟 / 对象时间常数”这个比值，而不是看它的算法有多复杂。

第三，边界要提前算清楚，不要等故障来了再算。 式 (78-30) 的 $|x_1| < (3.6 - |c|)/1.44$ 在健康时就该算好，写进联锁表。$c = 2.0$ 的故障救不回来，这件事在仿真阶段就能知道；真机上遇到它时，正确的动作是降级，不是硬撑。

78.12 Python 仿真代码

78.12.1 文件与运行方式

本章两个脚本，都在 `book/code/` 下：

- `ch078_sim.py`：十组数值实验与数据缓存；
- `ch078_figs.py`：只负责出图，从缓存读数。

运行命令：

```
"D:\Anaconda3\envs\py311\python.exe" code/ch078_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch078_figs.py
```

第一个脚本跑完全部实验约半分钟，结束时在 `figs/ch078/results.npz` 写入曲线数据，并把本章所有数字打印到标准输出。第二个脚本读缓存，生成 10 张 SVG 与同名 PDF。两者都不依赖外部数据文件，装有 numpy、scipy、matplotlib 的机器可以直接跑通。

78.12.2 代码结构

`ch078_sim.py` 的函数清单与职责见下表。

函数	职责	被谁调用
`rk4_step`	定常输入的四阶 Runge-Kutta 单步	`simulate`
`c2d`	零阶保持离散化（增广矩阵指数）	模块加载时
`f_plant`	磁悬浮平台的连续动力学	`rk4_step`
`dlqr`	离散 Riccati 求解	三处重构分支
`kalman_gain`	稳态卡尔曼增益（对偶 Riccati）	观测器设计
`moving_average`	因果滑动平均	残差统计量
`estimate_fault`	在辨识窗上同时估 (ρ, c)	隔离环节
`classify`	按估计值给出故障标签	重构环节
`reference` / `reference_dot`	参考及其导数	`simulate`
`simulate`	一次完整闭环（含故障注入与切换）	E1–E10
`exp1` – `exp10`	十组实验	`main`

结构上只有一个循环，就是 `simulate` 里的那一层。所有实验都是它的不同参数组合，这样“改一个参数看结果”不会因为代码路径不同而产生混淆。

78.12.3 三个关键片段

片段一：离散化。 用增广矩阵的指数一次算出 $\boldsymbol{A}$ 与 $\boldsymbol{B}$，避免用前向差分近似带来的稳定性偏差：

```
def c2d(A, B, h):
    """零阶保持离散化（增广矩阵指数）。"""
    n, m = B.shape
    M = np.zeros((n + m, n + m))
    M[:n, :n] = A
    M[:n, n:] = B
    E = expm(M * h)
    return E[:n, :n], E[:n, n:]
```

片段二：故障辨识。 把残差投影到 $\boldsymbol{b}_1$ 方向后，回归量只有 $[u_1, 1]$ 两列，两列系数就是 ρ 与 c。条件数保护写在解方程之前：

```
def estimate_fault(X, U, k0, k1):
    """在 [k0,k1) 上同时辨识效率 rho 与常值偏置 c。"""
    b1 = BD[:, 0]
    num = np.zeros(2)
    den = np.zeros((2, 2))
    for k in range(k0, k1):
        tgt = X[k + 1] - AD @ X[k] - BD[:, 1] * U[k, 1]
        phi = np.array([U[k, 0], 1.0])
        val = float(b1 @ tgt) / float(b1 @ b1)
        num += val * phi
        den += np.outer(phi, phi)
    cond = float(np.linalg.cond(den))
    if cond > 1e8:
        return 1.0, 0.0, cond
    theta = np.linalg.solve(den, num)
    return float(np.clip(theta[0], 0.0, 1.2)), float(theta[1]), cond
```

片段三：卡死分支的重构。 主通道整行置零，备份通道单输入重设计，控力权重用独立的 `R_BACK`：

```
        if stuck_next:
            # 主通道彻底失效：只剩备份执行器，按单输入重新设计，
            # 主通道那一行增益恒为零（否则会去驱动一个已经失效的通道）
            K_to = np.zeros((2, 3))
            K_to[1, :] = dlqr(AD, BD[:, 1:2], Q_LQR,
                              np.array([[R_BACK]]))[0, :]
        else:
            K_to = dlqr(AD, B_eff, Q_LQR, R_eff)
```

配套的观测器同步重构是三个分支，与式 (78-3) 的三种取法一一对应：

```
    # 观测器同步重构：模型里用的必须是与对象一致的作用量
    if stuck:
        u_obs = np.array([c_hat, u[1]])
    elif bias_mode:
        u_obs = np.array([u[0] + c_hat, u[1]])
    else:
        u_obs = np.array([eff * u[0], u[1]])
    xhat = AD @ xhat + BD @ u_obs + L_OBS @ r_obs
```

这三段合起来不到 40 行，但 78.8.7 里六条坑有三条落在这里。**容错控制的代码量不大，难的是每行背后的取舍。**

78.12.4 十组实验各自回答什么

实验	回答的问题	结果去哪
E1	健康残差多大？阈值怎么定？	78.2.4 的阈值表、图 78-3
E2	故障越轻，检测越慢多少？	图 78-4
E3	三类故障能不能认出来？	图 78-5、78.8.3 的表
E4	三种方案差多少？	图 78-6、78.6.3 的表
E5	硬切换与淡入差多少？	图 78-7、78.7.5 的表
E6	延迟值多少钱？	图 78-8、78.6.5 的表
E7	阈值系数的两侧代价	图 78-9、78.6.6
E8	再分配该不该开？	78.6.7 的表、78.9.1
E9	冗余拓扑与注入矩阵	图 78-10、78.10.2、78.10.3
E10	换一组种子还成立吗？	78.6.3 末段的复核

E10 是专门为“结论不是巧合”设的。它把 E4 的四种配置换一组随机种子重跑，数字略有出入（0.0477 对 0.0473，0.2294 对 0.2344），但排序与量级完全一致。

78.12.5 改参数的入口

所有可调量都在文件头部的常量区，集中一处：

常量	含义	本章取值
`DT`	采样周期	$0.02\,\mathrm{s}$
`MA_WIN`	滑动平均窗	5
`EST_WIN`	辨识窗	20
`SIGMA`	三路量测噪声	$[5\times10^{-3},\,2\times10^{-2},\,10^{-2}]$
`Q_LQR` / `R_LQR`	控制器权重	$\mathrm{diag}(100,10,1)$ / $\mathrm{diag}(0.01,0.01)$
`R_BACK`	备份通道控力权重	0.1
`Q_COST` / `R_COST`	评价指标权重	$\mathrm{diag}(1,0.1,0)$ / $[0.02,0.02]$

注意 `Q_COST` 与 `Q_LQR` 是分开的两套。前者只用于算 J，后者用于设计控制器。把两者混在一起是常见的错误——一旦混用，“换权重”就同时改变了控制器和评价标准，结果没法解释。

78.12.6 复现注意事项

第一，**随机种子**。所有实验都显式指定种子（E1 用 0 到 29，E2 用 1，E4 用 3，E5 用 4，E6 用 5，E7 用 7，E8 用 8，E9 用 0 到 2）。改动种子会让末尾几位数字变化，量级与结论不变。

第二，**版本**。`scipy.linalg.solve_discrete_are` 在不同版本里对病态矩阵的处理略有差异。如果 `dlqr` 报收敛失败，先检查 $\boldsymbol{B}_{\mathrm{eff}}$ 是否已经接近奇异——那正是 $\hat{\rho}\to 0$ 时的正常现象，应该在分类阶段就切到卡死分支，而不是硬解。

第三，**耗时**。E1 要跑 30 组、E9 要跑 18 格乘 3 个种子，是主要开销。若只想看单条轨迹，直接调 `simulate(rho=0.4, fault="loe", mode="active", th=...)` 即可，单次几十毫秒。

第四，**图与数的对应**。`ch078_figs.py` 只读 `results.npz`，不重跑仿真。若要改图而数值不变，只跑第二个脚本即可；若改了 `ch078_sim.py` 的任何常量，必须重跑第一个脚本，否则图与正文的数字会对不上。这条约定在全书所有章节里都一样。

78.12.7 四个可以自己动手的实验

下面四个实验各自只改一个常量，改完重跑第一个脚本，就能把正文里的某条结论从“读来的”变成“验过的”。不要同时改两处，否则归因不了。

实验一：把 `MA_WIN` 从 5 改成 9。 预期 σ_S 下降（噪声被压得更狠），同一个 $k=4$ 的阈值也跟着下降，误报率与检测延迟同时变化。对照第 78.3.5 节的算术：群延迟从 $0.04\,\mathrm{s}$ 变 $0.08\,\mathrm{s}$，按 $e^{1.033t}$ 折算，故障后偏差要多涨 4.2%。如果 σ_S 几乎没变，说明残差里的主导成分不是噪声而是模型失配——那第 78.5.3 节“μ_S 比纯噪声推算值大不少”这句话就在你手上得到了验证。

实验二：把 `EST_WIN` 从 20 改成 40。 这是第 78.3.3 与 78.5.6 两节反复提到的那个改动。预期卡死 $c=1.0$ 的故障后峰值从 0.507 跳到千的量级，注入矩阵里那一格从“通过”变成“不通过”；而卡死 $c=0.5$ 可能仍然通过，因为它的权限边界 2.15 比 1.81 宽。这个实验最能说明“辨识精度要给时间让路”不是一句口号。

实验三：把 `R_BACK` 从 0.1 改成 0.02。 备份通道的控力权重降五倍，等于允许它更“用力”。预期卡死分支的闭环极点往单位圆内移动、谱半径下降，但 u_2 的峰值上升、饱和更容易发生。若谱半径没怎么变而 u_2 峰值明显上升，说明这套对象的瓶颈不在权重上，而在备份通道的惯性——回到第 78.5.1 节那张 $\boldsymbol{B}$ 两列对比，答案在那里。

实验四：把 `T_FAULT` 从 2.0 改成 4.5。 这填的是第 78.3.3 节自己承认的缺口：故障相位没扫。$t=4.5\,\mathrm{s}$ 时参考接近峰值、速度项最大，同样的 c 会推出更大的峰值。预期卡死 $c=1.0$ 那一格的峰值明显高于 0.507。如果这一格因此变成“不通过”，那本章的结论应当修正为“这套方案在参考峰值附近遇到 $c=1.0$ 的卡死时余量不足”，而不是“能容 $c\le 1.0$”。**一个没做过的实验，足以改写一条已经写进正文的结论——这正是第 78.3.7 节把“边界扫描要双向做”单列出来的原因。**

第79章 安全控制

79.1 动机与直观理解

一辆在仓库里穿行的 AGV，控制器设计得很好：定位误差小于 5 mm，速度环带宽 120 Hz，轨迹跟踪的稳态误差可以忽略。某天货架被临时挪动了 30 cm，激光雷达实时更新了地图，避障模块也把新障碍写进了代价地图，可车还是擦着货架过去了——不是定位不准，也不是感知没看到，而是控制器从来就不知道“撞上”这件事在数学上意味着什么。它的全部任务是让位置误差趋零，而“趋零”的最短路径恰好穿过货架。

这类事故的共同点是：**性能指标和安全性被混为一谈**。跟踪误差小、响应快、超调低，这些都是“做得好不好”的度量，它们并不包含“绝对不许发生”的约束。工程上必须把后者单独拿出来，用一个只关心“能不能越界”的机制去保证，这就是本章的主题——安全控制。

79.1.1 先看一个具体的失败

本章用同一个算例贯穿始终：一个二维质点从 $(0, 0)$ 出发，目标 $(4, 0)$，在 $(2, 0.30)$ 处有一个半径 0.40 m 的圆形障碍，另有 0.15 m 的工程裕量，因此安全边界半径为 0.55 m。质点受加速度控制，各轴限幅 $\pm 5\ \mathrm{m/s^2}$，名义控制器是一个 PD：$u_{nom} = -K_p(p - p_d) - K_d v$，取 $K_p = 2.25$、$K_d = 3.0$，临界阻尼，无超调。

按纯 PD 仿真，质点沿直线穿过障碍中心附近，**最小净距为 -10.0 cm**，也就是侵入障碍内部 10 cm；到达目标用时 4.32 s。若在 PD 之后串一个安全滤波器（本章的方法），最小净距变为 $+21.5$ **cm**，代价是到达时间从 4.32 s 增至 4.98 s，路径长度从 4.00 m 增至 4.07 m。两条轨迹画在图 79-1 中。

这组参数不是随手凑的。障碍中心放在 $y = 0.30$ 而不是 $y = 0$，是为了让直线路径**擦过**而不是**正穿**：若障碍居中，无滤波轨迹会穿入 0.40 m，对比过于悬殊，反而看不出滤波器“最小修正”的特征；偏 0.30 m 之后，名义轨迹只侵入 10 cm，滤波器只需在最后一段横向让开二十多厘米，修正量与任务量在同一量级，比较才有意义。加速度限幅 5 m/s² 约等于 $0.5g$，是仓储 AGV 的常见量级；0.15 m 的工程裕量对应定位误差、控制误差、建模误差各 5 cm 的量级。这些数字的来历在 79.3 节展开，这里先记住一条：**算例的每个参数都应对应一个工程事实**，否则仿真结论无法迁移。

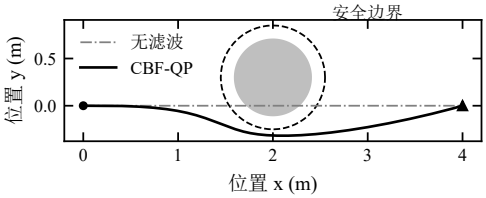


图 79-1　名义控制器与安全滤波器的轨迹对比

图 79-1 里有两个细节值得注意。第一，滤波后的轨迹并不是“提前绕一个大圈”，它只在离障碍最近的那一段被推开，前后两段几乎与名义轨迹重合——这说明安全约束在大部分时间里是**不激活**的，激活时间占比只有 10.0%。第二，两条轨迹的差别只有 7 cm 的路径长度，但净距差了一个数量级（从 −10 cm 到 +21.5 cm）；安全设计很少需要大幅牺牲性能，它需要的是在正确的时刻做正确的修正。

79.1.2 安全是集合，不是数值

把“安全”写成数学，第一步是承认它是一个**集合**而不是一个标量。给状态空间划一条边界，边界以内叫安全集 $\mathcal{C}$，边界以外叫不安全集。控制器要保证的不是“误差小”，而是“状态永远不离开 $\mathcal{C}$”。

对本章算例，安全集取圆外区域

$$\mathcal{C} = \left\{ x \in \mathbb{R}^4 :\ h(x) = \|p - p_{obs}\|^2 - R^2 \geq 0 \right\} \tag{79-1}$$

其中 p 是位置，p_{obs} 是障碍中心，R 是安全半径。h 称为**屏障函数**（barrier function）：它在安全集内为正、在边界上为零、在不安全集内为负。用一个标量函数描述一个集合的“内外部”，是本方法全部威力的来源——后面所有的约束、优化、验证都围绕这一个标量展开。

这个定义立刻带来一个必须回答的问题：$h > 0$ 只说明“此刻安全”，怎么保证“下一刻仍然安全”？如果系统以很大速度冲向边界，此刻 $h = 0.5$ 并不比 $h = 0.01$ 更安全。所以安全性必须同时约束 h 及其变化率，这正是 79.2 节要建立的**前向不变性**条件。

还有两点需要提前说清楚，否则后面读数字会误解。第一，$h = 0$ 对应的是“允许的最贴近位置”，不是碰撞面。R 里已经包含了障碍半径、机器人外廓与两项误差预算，所以 h 穿过零只说明裕量被吃掉了，物理上未必接触——79.9 节的算例里 $h_{min} = -0.0237$，实际净距仍有 12.8 cm。第二，h 的零集必须与实际几何边界一一对应。如果 h 写得比实际边界“松”，滤波器会在该拦的时候不拦；写得比实际边界“紧”，则会在不需要的时候干预。这两种错误都不会报错，只会在验收时以“净距不够”或“绕得太远”的形式暴露。

安全集的选择本身也不是唯一的，它直接决定保守程度。同一个圆形障碍，可以取“到中心距离不小于 R”（圆外），也可以取“到中心距离不小于 R 且不在障碍正后方 60° 扇区内”（更紧），还可以取“到中心距离不小于 R，但允许短暂穿越”（更松，需要额外的恢复条件）。选得越紧，可行域越小，越容易在狭窄处无解；选得越松，需要的证明越复杂。**安全集是设计决策，不是客观事实**——这一点在评审时经常被忽略，评审人往往只问“净距多少”，而不问“安全集凭什么这么划”。

79.1.3 安全层：把安全从控制器里拆出来

最直接的想法是把避障写进控制器：设计一个专门的避障控制器，把跟踪和避障一起做。这条路在工程上很难走通，原因是安全逻辑与任务逻辑的更新频率、验证方式、责任归属都不同。任务逻辑随工艺变，安全逻辑随标准变；任务逻辑要频繁迭代，安全逻辑一旦验证就不许随便动。

因此工程上更常见的结构是**分层**：底层是一个不管安全的、性能优良的名义控制器；上层是一个只做最小修正的安全滤波器。名义控制器给出 u_{nom}，安全滤波器检查它是否会导致越界，若不导致就原样放行，若导致就找**离 u_{nom} 最近**的、安全的控制量。图 79-2 给出这个结构。

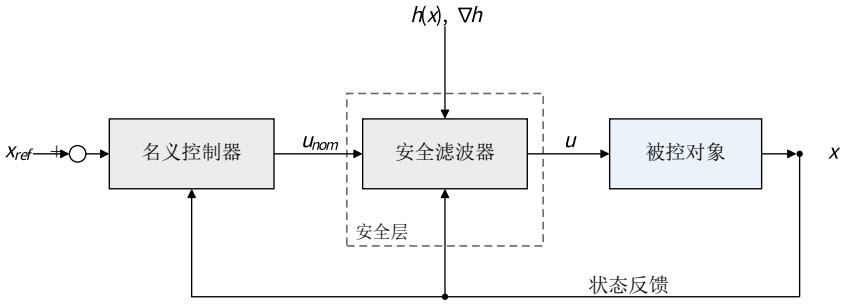


图 79-2　名义控制器加安全滤波器的分层结构

这个结构的两个好处在本章算例里都能量化。第一，安全层在 90.0% 的时间里完全不起作用（约束激活占比 10.0%），此时闭环行为与名义控制器逐位相同，原有性能指标不受影响。第二，安全层的输入只有当前状态和屏障函数，与任务无关；换一个任务、换一个目标点，安全层不动，只需要重新验证 h 的选取。这种“任务可换、安全不换”的性质，是它在自动驾驶、协作机器人、无人机上被大量采用的原因。

完整的系统通常是三层，而不是两层：**规划层**决定“走哪条路”（全局路径、避障策略），**名义控制层**决定“怎么沿着路走”（跟踪、镇定），**安全层**决定“这一拍能不能这么走”。三者回答的问题不同，失效的后果也不同。规划层算错，代价是绕远或走进死胡同，但通常还有时间重规划；名义层算错，代价是跟踪变差、超调变大；安全层算错，代价是碰撞。把安全层单独拿出来、用更严格的标准验证，正是因为它承担的是最后一票否决权。

这也解释了为什么安全层不应该“聪明”。它不参与路径选择，不做任务优化，只在必要时把控制量拉回可行域。**职责越窄，越容易验证**；一个同时负责避障、跟踪、能耗优化的模块，即使性能更好，也很难回答“它到底保证了什么”。反过来，一个只做投影的模块，可以说清楚“只要约束可行，就绝不越界”——这一句话就足以支撑一次安全评审。

79.1.4 为什么不在代价函数里加罚项

最常见的替代方案是把安全写成罚项，塞进原来的目标函数：

$$J = \|p - p_d\|^2 + \frac{w}{\left(\|p - p_{obs}\| - r_{obs}\right)^2}$$

调大 w，轨迹就会离障碍远一点；调小 w，跟踪就更紧。看起来只要 w 足够大就能保证安全，实际上这条路有三处硬伤。

第一，**没有阈值**。罚项是在“违反的代价”和“跟踪的收益”之间做交换，交换比由 w 决定，而安全性没有“合格线”。给定任意有限的 w，总能构造一个足够大的初始速度或足够小的可用空间，使最优解仍然穿过障碍——因为穿过障碍换来的跟踪收益也是有限的，两者始终可比。

第二，**数值病态**。要把 w 调到“几乎不越界”，代价函数在边界附近的曲率会大到让求解器难以收敛；而远离障碍时，这个巨大的 w 又把本不该受影响的控制扭曲了。本章算例里，滤波器的激活占比只有 10.0%，罚函数法则要求全程都带着这个权重跑。

第三，**给不出证书**。验收时评审会问“最坏情况下的净距是多少”，罚函数法只能回答“我们试了很多场景，最小是 21.5 cm”，无法回答“为什么不可能更小”。约束形式可以：式 (79-17) 的可行性判据是可以在设计阶段逐工况核算的，核算通过就意味着保证成立。

还有一处容易被忽略的差别：两者的**失效方式**不同。罚函数法在参数不合适时不会报警，它只是“悄悄地不够安全”——轨迹仍然平滑，指标仍然好看，只有净距在缓慢变小。约束形式在参数不合适时会直接报不可行，虽然难看，但至少是一个明确的信号。工程上“明确失败”远比“悄悄退化”可取，因为前者能被测试发现，后者只能等事故发生。

所以本章的选择是：**把安全写成约束，把性能留给目标函数**。两层各管一件事，中间通过“投影”连接。罚项并没有被完全抛弃——79.9 节的软化约束会把它用在“确实无解”的降级场合，那时它的角色是“违反最少”，而不是“保证不违反”。

79.1.5 本章路线与一句话核心

本章按“集合—条件—求解—离散—参数—工程”推进：79.2 节建立屏障函数、前向不变性与 CBF 条件；79.3 节把它固化成六步设计流程；79.4 节在连续域给出一维解析解和二维算例；79.5 节讨论离散实现与采样周期的影响；79.6、79.7 节分析安全性与性能的耦合、逐个参数讲影响；79.8 节给调参规则；79.9 节给出鲁棒化、高阶屏障等改进；79.10 节落到实时性、切换与验收；79.11 节说明边界；79.12 节给出可复现代码。

如果只能记住一句话：**安全控制不是在性能指标上加权重，而是在可行控制集合上做投影；投影只在约束将被违反时发生，其余时间系统仍按原控制器运行。**

不同读者的路径：关心架构与验收的，读 79.1、79.3、79.10；要上手实现滤波器的，读 79.2、79.4、79.5、79.8；要做算法改进或写论文的，读 79.6、79.7、79.9、79.11。三条路径都以 79.12 的脚本为落点——不跑一遍，前面的数字都只是别人的数字。

79.2 数学原理

本节要把“永远不越界”翻译成一条可以逐时刻检查的不等式。路线分四步：先规定被控对象的数学形式，再定义集合的不变性，然后把不变性写成对控制量 u 的线性约束，最后把它变成二次规划。

79.2.1 控制仿射系统

全书大多数对象可以写成控制仿射形式

$$\dot{x} = f(x) + g(x)u, \qquad x \in \mathbb{R}^n,\ u \in \mathcal{U} \subset \mathbb{R}^m \tag{79-2}$$

其中 f 是无控制时的漂移项，g 是控制输入的作用方向，$\mathcal{U}$ 是执行器能提供的控制集合。写成这个形式不是为了好看：它让 h 的时间导数对 u 是**仿射**的，从而“保持 h 不越界”在 u 上就是一条线性不等式。凡是“约束能写成控制量的线性不等式”的地方，都可以用二次规划在线求解，这是本方法能在毫秒级跑起来的根本原因。

对本章的二维质点，状态 $x = [p_x, p_y, v_x, v_y]^{\mathsf{T}}$，$f(x) = [v_x, v_y, 0, 0]^{\mathsf{T}}$，$g(x) = [0_{2\times 2}; I_2]$，即

$$f(x) = \begin{bmatrix} v \\ 0 \end{bmatrix}, \qquad g(x) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{79-3}$$

控制集合 $\mathcal{U} = \{u : \ |u_i| \le a_{max},\ i = 1,2\}$，$a_{max} = 5\ \mathrm{m/s^2}$。这个对象是二阶的（位置、速度），而 h 只依赖位置，所以 h 对时间的相对阶为 2——这一点决定了后面的推导要用的不是普通 CBF，而是**高阶屏障函数**。

79.2.2 前向不变性

设安全集 $\mathcal{C} = \{x : h(x) \ge 0\}$。称 $\mathcal{C}$ 关于闭环系统**前向不变**，是指：只要 $x(0) \in \mathcal{C}$，就有 $x(t) \in \mathcal{C}$ 对一切 $t \ge 0$ 成立。前向不变性是安全的严格数学定义，它比“没有发生碰撞”强：后者只说明过去没撞，前者保证以后也不撞。

不变性的局部判据是 Nagumo 条件：在边界 $\partial\mathcal{C}$ 上，允许的速度必须指向集合内部或与边界相切。对由光滑函数 h 定义的水平集，这个条件写成

$$h(x) = 0 \implies \dot{h}(x) \ge 0 \tag{79-4}$$

直觉很清楚：h 已经到零了，此刻它的变化率不能是负的，否则下一步就掉到集合外面。但式 (79-4) 只在边界上生效，它不告诉你“离边界还有 0.3 m 时应该留多少余量”。工程上需要一条对**整个集合内部**都生效的条件，这就是下面要引入的类 $\mathcal{K}$ 函数。

79.2.3 控制屏障函数条件

把式 (79-4) 的右侧从零放宽到一个与 h 成正比的项，得到

$$\dot{h}(x, u) + \alpha\big(h(x)\big) \ge 0, \qquad \forall x \in \mathcal{C} \tag{79-5}$$

其中 $\alpha(\cdot)$ 是严格递增且 $\alpha(0) = 0$ 的类 $\mathcal{K}$ 函数。取线性形式 $\alpha(h) = \alpha h$（$\alpha > 0$，单位 1/s），式 (79-5) 变成

$$\dot{h}(x, u) \ge -\alpha h(x) \tag{79-6}$$

它的含义是：**h 允许下降，但下降速度被 h 本身限住**。离边界越近，允许的下降速度越小；到边界时允许速度为零，正好回到式 (79-4)。由比较引理可以解出这条不等式的后果

$$h\big(x(t)\big) \ge h\big(x(0)\big)\mathrm{e}^{-\alpha t} \ge 0 \tag{79-7}$$

也就是说，h 最多按指数衰减到零，永远不能在有限时间内穿过零。这就是 CBF 给出的安全保证：**指数渐近地贴边，而不是撞上去**。

“不能在有限时间内穿过零”这句话值得多看一眼，因为它解释了很多现象。指数函数永远不会真正取到零，所以理论上轨迹可以无限接近边界而不越界；实际系统里，一旦 h 小到与噪声同量级，“是否越界”就变成了一个由测量误差决定的问题，而不是由控制律决定的问题。**CBF 的保证在数学上是严格的，在物理上有一个由噪声决定的分辨率下限**。这也是为什么 79.3 节坚持把裕量写进 R，而不是指望控制器把 h 稳在零附近。

α 的量纲是 1/s，可以读成“屏障函数的衰减时间常数的倒数”。$\alpha = 4$ 意味着 h 的时间常数是 0.25 s，一个“危险程度”从 1 衰减到 0.37 大约需要这么长时间。把它与闭环带宽对比是有意义的：若安全层的时间常数比任务层还长，安全层就来不及在轨迹穿出边界前完成修正，表现为介入过早、绕行过大；若短得太多，则要求的加速度会超出执行器能力。79.7 节给出的窗口 $2\omega_n \le \alpha \le \alpha_{max}$，正是把这两头夹住。

类 $\mathcal{K}$ 函数的选择决定了保守程度。线性 $\alpha(h) = \alpha h$ 最简单也最常用；若取超线性形式（例如 $k h^3$），系统在远离边界时几乎不受约束、接近边界时迅速收紧，相当于“只在近处使劲”。本章只用线性形式，理由有两条：它的指数下界（式 79-7）可以直接转成采样周期判据 $T_s \le 0.1/\alpha$；而且只有一个参数，便于把“保守程度”这件事讲清楚。更高阶的类 $\mathcal{K}$ 函数在理论上更灵活，但可调参数变多，工程上很难说清某个参数到底在管什么。

把式 (79-6) 展开，利用式 (79-2)：

$$\dot h = \frac{\partial h}{\partial x}\big(f(x) + g(x)u\big) = L_f h(x) + L_g h(x)\,u \tag{79-8}$$

其中 $L_f h$、$L_g h$ 是李导数。代入式 (79-6) 得到一个**对 u 线性**的约束

$$L_f h(x) + L_g h(x)\,u + \alpha h(x) \ge 0 \tag{79-9}$$

当 $L_g h(x) \neq 0$ 时，对任意 $x \in \mathcal{C}$ 都存在满足式 (79-9) 的 u（只要 $\mathcal{U}$ 足够大），此时称 h 是一个**控制屏障函数**。注意这个“只要”：可行与否取决于执行器能力，这一点在 79.6、79.7 节会反复出现。

79.2.4 相对阶为 2 时的处理

本章的 h 只依赖位置，而位置要通过两次积分才受 u 影响，于是 $L_g h = 0$，式 (79-9) 中根本不含 u，无法用来设计控制器。处理办法是把条件再求一次导，构造一个“虚拟输出”：

$$\psi_1(x) = \dot h(x) + \alpha_1 h(x) \tag{79-10}$$

ψ_1 对 u 的相对阶为 1。要求 ψ_1 满足同样的条件：

$$\dot\psi_1(x,u) + \alpha_2 \psi_1(x) \ge 0 \tag{79-11}$$

由式 (79-7) 可得 $\psi_1(t) \ge \psi_1(0)\mathrm{e}^{-\alpha_2 t} \ge 0$，再代回式 (79-10) 就得到式 (79-6)。所以式 (79-11) 是比式 (79-6) 更强的条件，它保证的是一个“屏障的屏障”。

对本章的二次型 $h = \|p - p_{obs}\|^2 - R^2$，记 $d = p - p_{obs}$，有 $\dot h = 2 d^{\mathsf T} v$，逐项算得

$$\psi_1 = 2 d^{\mathsf T} v + \alpha_1\big(\|d\|^2 - R^2\big) \tag{79-12}$$

$$\dot\psi_1 = 2\|v\|^2 + 2 d^{\mathsf T} u + 2\alpha_1 d^{\mathsf T} v \tag{79-13}$$

代入式 (79-11) 并整理成 $A(x)u \le b(x)$ 的标准形式：

$$-2 d^{\mathsf T} u \;\le\; 2\|v\|^2 + 2\alpha_1 d^{\mathsf T} v + \alpha_2 \psi_1 \tag{79-14}$$

这就是安全滤波器真正求解的约束。它只有一行，系数由当前状态决定，右端项随状态更新。本章取 $\alpha_1 = \alpha_2 = \alpha$；两者分开的好处是可以在“贴边速度”和“约束硬度”上分别调，79.7 节会说明。

图 79-3 画出了 h 的等值线。粗实线是 $h = 0$ 的安全边界，箭头是梯度 ∇h，它指向 h 增大的方向。约束式 (79-14) 的几何意义是：u 在 d 方向上的分量必须足够大（把系统往外推），而切向分量完全自由——这正是“只做最小修正”的来源。

把 u 沿法向与切向分解，$u = u_r e_r + u_t e_t$，其中 $e_r = d/\|d\|$。式 (79-14) 的左端是 $-2 d^{\mathsf T} u = -2\|d\| u_r$，只含法向分量；切向分量 u_t 在约束里根本不出现。这个结构值得记住：**安全约束只限制“往外推”的力度，不限制“沿着障碍滑行”的自由**。绕行时最需要的恰恰是切向速度，如果约束把切向也压住，系统就会在障碍前“停住”而不是“绕开”——很多早期势场法就是因为对各个方向一视同仁而出现这种停滞。

不过“切向自由”并不等于“切向没有代价”。切向速度越大，式 (79-14) 中的 $2\|v\|^2$ 项越大，右端项被抬高，反而使约束更容易满足——这是二次型屏障函数的一个特点：高速掠过时离心项帮忙，低速贴近时反而更依赖执行器。读 79.4 节的表时要留意这个反直觉之处。

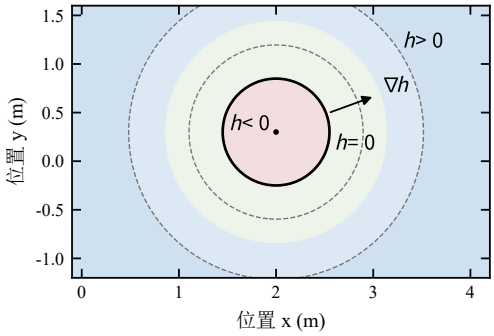


图 79-3　屏障函数的等值线与安全集划分

79.2.5 从约束到二次规划

有了线性约束，安全滤波器就是一个标准的二次规划：

$$\begin{aligned} u^\star = \ & \arg\min_{u} \ \tfrac{1}{2}\|u - u_{nom}\|^2 \\ & \text{s.t.} \quad A(x)u \le b(x), \qquad u \in \mathcal{U} \end{aligned} \tag{79-15}$$

目标函数选“离 u_{nom} 最近”而不是别的，有三条理由。第一，它保证在约束不起作用时 $u^\star = u_{nom}$，安全层对原系统零影响。第二，它是最小范数修正，修正量与违反程度同阶，不会引入多余动作。第三，它是一个严格凸的 QP，解唯一、对数据连续依赖，便于工程实现和稳定性分析。

从几何上看，式 (79-15) 是把 u_{nom} 投影到“半空间与盒的交”这个凸集上。凸投影有一个性质对工程很重要：投影点唯一，且对**固定**的凸集，投影映射是 1-Lipschitz 的——u_{nom} 的小扰动不会引起投影点的大跳变。约束本身随状态移动时，这个性质不能直接照搬，但在相邻两个采样周期之间，只要 $A(x)$、$b(x)$ 变化不大，投影输出的变化就与 u_{nom} 的变化同阶，不存在“放大”效应。79.6 节讨论的切换抖动，正是这条近似在边界附近失效的表现。若把目标函数换成非凸形式（例如带死区的加权），连固定凸集下的唯一性都会丢失。

从对偶角度看，这个问题还有一个有用的解释。设约束的拉格朗日乘子为 $\mu \ge 0$，一阶条件是 $u^\star = u_{nom} - \mu A^{\mathsf{T}}$，即修正量总是沿约束法向、大小由 μ 决定。μ 的物理意义是“这一拍的安全约束值多少加速度”：μ 越大，说明约束越紧、需要越大的修正。工程上把 μ 记录下来，比记录 $u^\star$ 更能反映系统的紧张程度——它不受控制量基线的影响，是一个纯粹的“安全压力”指标。很多实现只记录 h，但 h 小并不一定意味着紧张（可能速度很低，根本不需要修正），μ 才真正对应“滤波器正在多用力”。

当只有一个线性约束加一个盒约束时，式 (79-15) 有闭式解：先按盒约束截断，若仍违反则沿 $-A^{\mathsf{T}}$ 方向投影到约束边界上。本章的求解器就是按这个思路写的，79.4 节给出细节，79.10 节给出它与通用 QP 求解器的耗时对比。

需要强调一点：**式 (79-15) 的可行性不是自动的**。当状态已经恶化到“任何允许的 u 都无法阻止 h 下降”时，QP 无解，此时任何实现都必须有一个明确定义的回退动作，而不是让求解器报错后继续输出上一次的控制量。这是 79.10 节要展开的工程要点。

79.2.6 可行性的定量判据

盒约束加单行线性约束时，可行性可以在求解之前用一行算式判掉。约束 $A(x)u \le b(x)$ 在最坏方向上的最小值是

$$\min_{u \in \mathcal{U}} A(x)u = -\sum_{i=1}^{m} a_{max}\,|A_i(x)| \tag{79-16}$$

于是可行当且仅当

$$b(x) \;\geq\; -\sum_{i=1}^{m} a_{max}\,|A_i(x)| \tag{79-17}$$

对本章的 $A=-2d$、b 由式 (79-14) 给出，式 (79-17) 变成 $2a_{max}\big(|d_x|+|d_y|\big)+b\geq 0$。这条判据有两个用处：一是在线做一次开销可忽略的预检查，把“要报错”的情形提前识别出来；二是离线做参数核算，给定 a_{max} 与 α，算出在多大的接近速度下约束会失效。后者属于设计阶段的工作，不能留到运行阶段才发现。

式 (79-16) 的推导只有一步：盒约束下每个分量独立取值，要让 $A_i u_i$ 之和最小，就逐项取 $u_i=-a_{max}\,\mathrm{sign}(A_i)$，此时 $A_i u_i=-a_{max}|A_i|$，求和即得。注意这里用的是**一范数**而不是二范数——因为盒约束是逐分量的，可行域的“最远点”由各轴独立决定。这也解释了为什么式 (79-17) 在二维时表现为 $|d_x|+|d_y|$ 而不是 $\|d\|$：前者对应菱形（盒约束的像），后者对应圆。用二范数去算会高估执行器能力，在斜向切入时尤其危险。

本章算例给出一个具体界线：当 $\alpha=8$ 时，全程没有一次不可行；把 α 提到 16，不可行步数跳到 269 步（总步数 12 000），同时最小屏障值从 $+0.0122$ 掉到 -0.0467。也就是说，**α 不是越大越安全**，它同时提高了对加速度的需求，一旦超出执行器能力，保证就断了。79.7 节会把这条界线算清楚。

把判据用在“刚刚贴到边界”这个工况，可以算出一个很有用的数：边界处允许的最大法向速度。取 $h=0$、$d=Re_r$、$v=-v_r e_r$（$v_r>0$ 为接近速度），式 (79-14) 化为

$$u_r \;\geq\; (\alpha_1+\alpha_2)\,v_r-\frac{v_r^2}{R}$$

代入 $\alpha_1=\alpha_2=4$、$R=0.55$、$a_{max}=5$：$v_r=0.5$ m/s 时要求 $u_r\geq 3.55$，可行；$v_r=1.0$ m/s 时要求 $u_r\geq 6.18$，已经超过执行器能力；临界值在 $v_r\approx 0.75$ m/s。这条式子把“为什么要在远处就开始减速”讲清楚了——不是怕撞，而是**边界附近的执行器只够处理 0.75 m/s 的法向速度**。

本章算例的峰值速度是 1.63 m/s，远高于 0.75，所以轨迹不可能带着这个速度贴到边界；它必须在到达最近点之前把法向速度降到接近零，这也是 h_{min} 恰好出现在“法向速度为零”那一刻的原因。反过来，加大 R、减小 α 或提高 a_{max} 都能抬高这个上限，三者对安全性的作用方向由此统一到同一个式子里。

79.2.7 与可达性分析的关系

CBF 与可达性分析（reachability analysis）解决的是同一个问题的两个侧面，工程上经常需要一起用。

可达性分析回答的是“从这个初始状态出发，用任何允许的控制，能不能保证永远不进入不安全集”。它的答案是一个集合：**最大可控不变集**（viability kernel），记作 $\mathcal{K}$。$\mathcal{K}$ 是满足“存在一个控制律使集合不变”的状态的最大集合。它的优点是给出完整答案——不仅知道当前该做什么，还知道哪些初始状态已经救不回来；缺点是精确计算代价高，高维系统通常只能近似。

CBF 回答的是“给定一个已经选好的安全集 $\mathcal{C}$，能不能在线构造出一个让它不变的控制器”。它的答案是一个条件加一个在线优化，计算量小、可实时，但它是**充分条件**：h 选得不好，$\mathcal{C}$ 可能远小于 $\mathcal{K}$，控制器就会过度保守；h 选得太大，$\mathcal{C}$ 里可能有一部分状态根本无法保持，运行时表现为 QP 不可行。

两者的正确分工是：离线用可达性分析（或其简化版）确定 $\mathcal{K}$ 的形状，据此选 h 和裕量；在线用 CBF-QP 做逐时刻的执行。79.4 节的一维算例会把这两个集合画在同一张相平面上，直观看到它们的关系。

用一维算例可以看清“充分条件”这四个字的分量。物理上“停得住”的区域由抛物线 $y\geq v^2/(2a_{max})$ 界定，这是 $\mathcal{K}$ 的边界；而 CBF 用二次型 $h=y^2$ 加线性衰减率，得到的可行区域边界是那条饱和线 $y=-(2\alpha v+a_{max})/\alpha^2$。后者在接近段整体位于前者上方，也就是更保守——$\mathcal{K}$ 里有相当一部分状态，CBF 会判为“不敢去”。反过来，若把 h 取得过大（例如把 R 设得很小），$\mathcal{C}$ 可能部分落在 $\mathcal{K}$ 之外，那时 CBF 平时看起来正常，一旦进入那部分区域就立刻不可行。**保守与不可行，是 h 选得偏小或偏大的两种表现**，方向相反，原因相同。

79.3 设计步骤

把一个安全滤波器从零做出来，按下面的六步走。每一步都有明确的输入、输出和判据，做完一步再进下一步；跳过任何一步，后面都会以“调不出来”的形式还回来（表 79-1）。

表 79-1　安全滤波器的六步设计流程

步	动作	输出	完成判据
1	定状态、定安全集	x、$h(x)$、R	h 的零集与实际几何边界一一对应
2	算相对阶	ψ_i 或高阶条件	约束中显含 u，且系数不全为零
3	写 $A(x), b(x)$ 并核可行域	约束表达式	式 (79-17) 在设计工况下留有余量
4	选名义控制器与权重	u_{nom}、Q	无约束时闭环指标达标
5	离散实现	T_s、护卫裕量、求解器	采样周期内状态变化小于裕量
6	验证与回退	场景库、回退策略	全场景 0 越界，且不可行有定义

六步不是单向流水线。第 5 步算出裕量后，若发现它占 R 的比例过大，必须回到第 1 步重新审视误差预算或执行器能力；第 6 步发现某个场景不可行，也要回到第 3 步重核式 (79-17)。真正一次走通的情况很少，多数项目会在 1–3 步与 5–6 步之间来回两三次。

把六步的输入输出写清楚，是因为它们决定了这个模块能不能被独立评审。第 1 步的输出是一个函数 h 和一个数 R，评审人可以只看这两样就判断“安全集定义得对不对”，不需要读代码；第 2、3 步的输出是约束 $A(x)u \le b(x)$，评审人可以拿它去和论文里的标准形式对照；第 5 步的输出是三个数 T_s、m、求解器上限，评审人可以拿算力预算去核；第 6 步的输出是一组场景记录。**能被分段评审的东西，才可能被真正验证**；一个把六步揉在一起、只有一份大代码的实现，评审时只能靠“作者说没问题”。

79.3.1 第一步：把“安全”变成几何

先确定状态里哪些分量决定安全。多数移动系统只看位置，但高速机械臂要看到末端速度，柔性结构要看到模态坐标，所以这一步不能想当然地取 $h = \|p - p_{obs}\|^2 - r^2$。

这里有一条容易被忽视的约束：**h 里出现的量必须是可测量或可估计的**。如果 h 依赖一个测不到的中间变量，滤波器就只能用估计值代替，估计误差会直接变成安全裕量的漏洞。宁可把 h 选得保守一些、只用可靠测量的量，也不要用一个“理论上更精确但测不准”的屏障函数。

安全半径的取法是一条工程纪律：$R = r_{obs} + r_{robot} + \delta_{sense} + \delta_{ctrl}$。四项分别是障碍半径、机器人外廓、定位误差和控制误差。本章算例取 $0.40 + 0.15 = 0.55$ m，把后三项并进 0.15 m；现场应当拆成 $0.40 + 0.05 + 0.05 + 0.05$ 逐项写清楚，因为验收时要拿这四项去核对，而“0.15 是怎么来的”是最常被追问的问题。

h 的另一个选择是“带符号距离”而不是“距离平方”。平方形式的好处是导数简洁、约束线性；带符号距离的好处是梯度大小恒为 1，不同距离处的“惩罚力度”均匀。平方形式在远处梯度大、近处梯度小，正好与实际需要的修正力度相反，这是它在某些场合表现偏软的原因，79.9 节会给出改进办法。

79.3.2 第二步：确定相对阶

对 h 沿式 (79-2) 求导，看第一次出现 u 是在第几阶。这一步决定了条件的形式：

- 相对阶 1：直接用式 (79-9)，一行约束；
- 相对阶 2：构造 $\psi_1 = \dot{h} + \alpha_1 h$，用式 (79-11)，仍是一行约束，但要算二阶导数；
- 相对阶更高：逐层构造 ψ_i，约束行数不变，但每层都引入一个 α_i，可调参数变多，保守性也变强。

本章的二维质点是相对阶 2。若把状态改成“位置 + 速度指令”（速度环当作一阶惯性），相对阶就降到 1，约束里直接出现 u，数值上更好处理。这是工程上常用的简化：**宁可在建模上多花时间把相对阶降下来，也不要在算法上叠高阶条件**。代价是速度环的滞后变成了模型的一部分，必须计入 R 的 δ_{ctrl}，否则降阶带来的便利会被时延抵消掉。

相对阶不必靠肉眼判断，可以数值验证：在当前状态随机取若干组 u，差分计算 $\dot{h}$，若结果与 u 无关（变化在数值噪声以内），说明相对阶大于 1；再用同样办法看 $\ddot{h}$ 是否随 u 线性变化。这一步花十分钟，能省掉后面几天的困惑——相对阶判断错误的典型症状是“约束永远满足”，因为 $A(x) \equiv 0$，滤波器实际上什么也没做。

79.3.3 第三步：写出约束并核对可行域

把 $A(x)$、$b(x)$ 写出来之后，不要急着写代码，先用式 (79-17) 在几个典型工况上算一遍：最接近障碍的点、最高接近速度、最不利的切入角。若某个工况下 b 与 $-a_{max}\sum|A_i|$ 只差几个百分点，说明设计处在临界状态，必须加大裕量或提高执行器能力。

本章算例的设计点：峰值接近速度 1.63 m/s，$\alpha = 4$，$a_{max} = 5$，运行 12 000 步无一次不可行。同一对象把 α 提到 16，式 (79-17) 的余量变为负，出现 269 步不可行，说明这个参数已经越界。

这里有一个容易犯的错：核算时不能把“最高速度”与“最近距离”这两个最不利值直接拼在一起。本章的峰值速度 1.63 m/s 出现在远离障碍的加速段，而最近点处的法向速度接近零；若按“1.63 m/s 且贴在边界上”去核，会得到 $u_r \geq 8.2$ 的结论，远大于 a_{max}，从而误判为不可行。正确的做法是**沿名义轨迹逐点核算**：在每一个采样点上用当时的 d、v 代入式 (79-17)，看最小余量出现在哪里。这样得到的结论才是这条轨迹真实需要的执行器能力。

写完 A、b 之后还要做一次量纲核对，这一步只要一分钟，却能拦住最难查的一类错误。h 的量纲是 m²，所以 $\dot{h}$ 是 m²/s，b 是 m²/s²；A 的量纲必须是 m，才能与 u 的 m/s² 相乘得到 m²/s²。若你写的 A 是 m^2 量级（例如误用了 $\|d\|^2$ 而不是 d），等式两边差一个长度量纲，数值上仍然能算，只是约束的“尺度”随距离变化——远处的约束被放大、近处的被压缩，表现恰好是“离得远就开始猛烈转向，贴近时反而变软”。这类错误不会报错、不会崩溃，只会让参数怎么调都不对。核对办法很简单：取一组具体的 d、v、u，手工算一遍 $Au - b$ 的数值，再与代码打印的值比对，差值应在浮点误差内。

79.3.4 第四步：名义控制器与权重

名义控制器就是原来那套，不需要为安全做任何妥协。唯一的改动是把输出限幅与 QP 的盒约束对齐：名义控制器限幅到 $\pm a_{max}$，QP 的 $\mathcal{U}$ 也用同一个 $\pm a_{max}$，否则会出现“名义输出超出 QP 可行域”的假性不可行。

QP 的权重矩阵 Q 通常取单位阵。若某些通道的物理量纲不同（例如一个轴是力、另一个是力矩），必须先把 Q 按量纲归一化，否则“最小修正”会退化成“优先修正量纲小的那个通道”，这是一个很隐蔽的错误。

举个具体的例子。若控制的第一个分量是 100 N 量级的推力，第二个是 1 N·m 量级的力矩，而 $Q = I$，那么优化器会认为“改动 1 N·m 与改动 1 N 的代价相同”。实际上前者可能是满量程的 1%，后者也是满量程的 1%——量纲不同不代表重要性不同，但二次型里的数值直接比较了它们。正确的做法是先按各通道的满量程做归一化，令 $u_i' = u_i / u_{i,max}$，再在新的坐标下做单位权重的投影，最后变换回去。这样“最小修正”才真的是几何意义上的最小。

79.3.5 第五步：离散实现

连续时间的保证在离散实现下会打折扣，原因是控制量在一个采样周期内保持不变，而约束只在采样点被检查。工程上有三条固定动作：

1. 按式 $m = \|v\|_{max} T_s + \tfrac{1}{2} a_{max} T_s^2$ 计算护卫裕量，把它加到安全半径上；
2. 采样周期满足 $T_s \leq 0.1/\alpha$，即一个 α 时间常数的十分之一；
3. 求解器设超时，超时或不可行都走同一个回退分支。

第二条规则来自式 (79-7)：一个周期内 h 的合法下降比例是 $1 - e^{-\alpha T_s} \approx \alpha T_s$，要求它不超过 10%，就得到 $\alpha T_s \leq 0.1$。这不是精确的稳定性条件，而是一条把离散误差控制在“与 α 同尺度”的经验线——取 10% 意味着离散化最多让有效 α 打九折，参数窗口不会被挤掉太多。

本章算例：高速接近（峰值约 2.9 m/s）时，$T_s = 100$ ms 的护卫裕量为 0.312 m，最小屏障值从 1 ms 时的 $+0.0188$ 掉到 $+0.0106$，并出现 3 步不可行；加上护卫裕量后恢复到 $+0.4250$。这组数说明：**离散化的代价首先体现为保守性，其次才是可行性**。

79.3.6 第六步：验证与回退

验证不是“跑一遍仿真看着没问题”，而是三件事：一是场景库覆盖（直冲、斜切、贴边绕行、目标点在障碍后方、多障碍）；二是边界工况（最高速度、最大加速度、最小裕量同时出现）；三是故障注入（传感器延迟、状态估计跳变、求解超时）。

回退策略必须写进需求，不能留给实现者临场决定。常用的是“最大制动”：一旦不可行，输出使 h 下降最慢的控制量，同时上报故障。本章的代码在不可行时输出沿 d 方向的最大加速度，并在指标里统计不可行步数——有统计才能验收。

故障注入至少要覆盖三类情形，每一类都要有明确的预期行为。第一类，**状态估计跳变**：把位置突然平移 10 cm，检查滤波器是否输出一个方向正确、幅度有限的修正，而不是连续振荡；第二类，**求解超时**：把求解时限压到 1 μs，检查系统是否进入回退分支并置故障位，而不是沿用陈旧解；第三类，**执行器能力下降**：把 a_{max} 临时降到 1 m/s²，检查不可行是否被统计、是否被上报，以及报告的数值能否对得上人工核算。三类都不要求系统“继续完成任务”，只要求“不越界、状态可查、故障可追”。

验证还要记录一组基线数据，供后续回归比对：本章的基线是最小净距 21.5 cm、到达时间 4.98 s、激活占比 10.0%、不可行 0 步。任何代码改动之后重跑，四项中任何一项偏离超过 5%，都要解释原因。没有基线，“改完好像差不多”就会变成永久的技术债。

基线要冻结成一份可执行的清单，而不是记在某个人的笔记里。最简做法是把这四个数写进测试脚本，用 `assert` 比较，容差按 5% 设；再把这套测试挂到每次提交上。听起来很笨，但它能在“有人顺手把 α 从 4 改成 5”的当天就发现问题，而不是等到三个月后的实车测试。安全层的回归测试不需要覆盖所有场景，覆盖这四个数就够了——它们分别对应几何、时间、算力占用和可行性，任何实质性改动都会至少动到其中一个。

六步做完，安全层就是一个独立的、可单独验证的模块。它不知道任务是什么，只知道“哪些状态不许去”，这正是 79.1 节强调的分层价值。

79.4 连续情形

连续时间下，CBF 的保证是严格的：只要初始状态在安全集内，且每个时刻都满足约束，那么对一切 $t \ge 0$ 都有 $h(x(t)) \ge 0$。这一节用两个能算出结果的最小例子把这条链路走完：一维系统可以写出全部解析表达式，二维系统把 QP 化成一次一维求根。两个例子共用 79.1 节的同一套参数，全部数字来自 `code/ch079_sim.py` 的实际运行。

79.4.1 一维：能写出解析解的最小例子

取小车正对墙壁，状态为到墙距离 y 与沿 $-y$ 方向的速度 v（$v<0$ 表示接近），控制为减速度 $u \in [0, a_{max}]$。安全集取 $h = y$，于是

$$\dot{h} = v, \qquad \ddot{h} = u \tag{79-18}$$

相对阶为 2。按 79.2 节的高阶构造，$\psi_1 = \dot{h} + \alpha_1 h = v + \alpha_1 y$，要求 $\dot{\psi}_1 + \alpha_2 \psi_1 \ge 0$，取 $\alpha_1 = \alpha_2 = \alpha$ 得

$$u + 2\alpha v + \alpha^2 y \ge 0 \quad \Longleftrightarrow \quad u \ge -2\alpha v - \alpha^2 y \tag{79-19}$$

式 (79-19) 的右端是一个“随距离与接近速度变化的最小减速度”：离墙越近、接近越快，要求的减速度越大。注意当 $y \to 0$ 且 $v < 0$ 时右端趋于 $2\alpha|v|$，与距离无关，这是相对阶 2 的相对阶特性在起作用——**最后一段靠速度项兜底，位置项已经没有余量可用**。

与它对照的是“来得及停下”的物理条件。以 a_{max} 全力制动，从 (y, v) 出发的停止距离为 $v^2/(2a_{max})$，因此

$$y \geq \frac{v^2}{2a_{max}} \tag{79-20}$$

是相平面上的**可控不变集**边界：在曲线上方，存在控制使小车停在墙前；在曲线下方，任何控制都来不及。图 79-4 把这个集合、CBF 闭环轨迹和全制动轨迹画在一起。

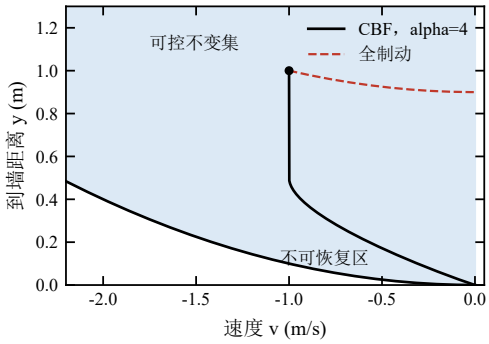


图 79-4　可控不变集、全制动轨迹与 CBF 闭环轨迹

图中阴影为式 (79-20) 上方的可控不变集，黑实线是它在下半平面的边界。从初始点 $(1.0, -1.0)$ 出发立刻全力制动的轨迹（灰线）沿着 $y = 0.9 + v^2/10$ 走到 $(0, 0.90)$，停在距墙 0.90 m 处；CBF 轨迹（蓝线）则先向墙滑行，直到式 (79-19) 的右端由负变正才开始减速，最终停在 0.003 m 处。两条线都在可控不变集内，但保守程度相差三个数量级。

这个对比点出了 CBF 的一个本质：它保证的是 $h \geq 0$，不是“留足余量”。数学上 3 mm 与 900 mm 同样安全，工程上完全不同。79.4.5 节会用裕量把这两者重新对齐。

相平面上还有两条线值得画出来，它们把“什么时候介入”和“什么时候已经来不及”分开了。令式 (79-19) 的右端等于名义控制（这里是 0），得到**介入线**

$$y = -\frac{2v}{\alpha}$$

轨迹在这条线之上时约束不活跃，穿过去才开始刹车。$v = -1$ m/s、$\alpha = 4$ 时它给出 $y = 0.5$ m，与 79.4.5 节“前 0.5 s 控制量恒为零、距离约 0.5 m 才开始减速”完全对应。再令右端等于 $+a_{max}$（注意是正的上限，因为靠近墙时约束要求的是“往外推”），得到**饱和线**

$$y = -\frac{2\alpha v + a_{max}}{\alpha^2}$$

在它之下，即使全力往外推也满足不了式 (79-19)，CBF 判不可行。$v = -1$ 时给出 $y = 0.1875$ m，而真正的物理制动距离只有 $v^2/(2a_{max}) = 0.1$ m。CBF 的不可行区比物理不可行区更靠外，约 1.9 倍，这段差距就是 α 带来的保守性——它不允许你把速度带到离墙 10 cm 的地方再刹，因为那样 h 的衰减速率会超过 $e^{-\alpha t}$ 允许的范围。

三条线（介入线、饱和线、抛物线边界）的关系可以概括为一句话：**介入线决定“早不早”，饱和线决定“来不来得及”，抛物线决定“物理上可不可能”**。在接近段（$v < 0$）三条线自上而下排列，中间夹着的就是 CBF 的“舒适区”：在这里它可以只用一个不大的减速度把轨迹拉回，既不越界，也不饱和。设计时先看介入线能否把轨迹拉回，再看饱和线是否始终在轨迹下方，最后才用抛物线判断这个工况本身是否可救。

79.4.2 二维：单约束 QP 的闭式投影

二维算例的状态为 $x = (p, v)$，安全函数 $h = \|p - p_{obs}\|^2 - R^2$，相对阶 2，二阶展开后约束可写成 $A(x)u \leq b(x)$，其中 $A = -2d^{\mathsf{T}}$，$d = p - p_{obs}$。于是每步要解的是

$$\min_{u} \ \tfrac{1}{2}\|u - u_{nom}\|^2 \quad \text{s.t.} \quad Au \leq b, \quad -a_{max}\mathbf{1} \leq u \leq a_{max}\mathbf{1} \tag{79-21}$$

只有一个标量不等式，最优解必在“从 u_{nom} 沿 $-A^{\mathsf{T}}$ 方向移动”这条直线上——这是欧氏投影的几何性质，不需要拉格朗日乘子也能看出来。把盒约束并进来，可行点可参数化为

$$u(\lambda) = \mathrm{clip}\big(u_{nom} - \lambda A^{\mathsf{T}},\, -a_{max},\, a_{max}\big), \qquad \lambda \ge 0 \tag{79-22}$$

定义残差 $f(\lambda) = A\,u(\lambda) - b$。沿 $-A^{\mathsf{T}}$ 移动会单调降低 Au，所以 f 关于 λ 单调不增：$f(0) \le 0$ 说明名义控制本来就安全，直接放行；$f(0) > 0$ 则先倍增找到使 $f < 0$ 的上界，再二分 60 次得到

$$\lambda^{\star} = \min\{\lambda \ge 0 :\ f(\lambda) \le 0\}, \qquad u^{\star} = u(\lambda^{\star}) \tag{79-23}$$

这段实现不到二十行，是本章代码的做法。它比通用求解器快两个数量级：单约束、标量目标下，中位耗时 8.2 μs（这是约束未激活时的路径），P95 254.6 μs——尖峰落在需要完整二分的那少数步上。作为对照，79.10 节用 cvxpy+OSQP 解同类问题，中位 0.77 ms，仍然够 1 kHz，但一个周期已经用掉 77%。

这里的选择原则值得记住：**约束是标量时不要上通用求解器**。二分的精度由迭代次数决定，60 次已到双精度极限，再多的迭代没有意义；而通用求解器的固定开销（建模、稀疏矩阵组装、收敛判据）在小规模问题上完全无法摊销。

二分的收敛性依赖一个前提：$f(\lambda)$ 单调不增。这一点由 $u(\lambda) = \mathrm{clip}(u_{nom} - \lambda A^{\mathsf{T}})$ 保证——沿 $-A^{\mathsf{T}}$ 走，Au 严格下降，直到所有分量都撞上盒约束才停。撞上之后 f 不再变化，二分仍能收敛，只是返回的 $\lambda^{\star}$ 可能落在“已经无改进”的区间里。此时若 f 仍为正，说明盒约束已经把可行方向堵死，属于执行器能力不足，代码里对应的分支就是返回沿 d 方向的最大推力并把可行标志置假。

数值上有一个边界要处理：当 $\|d\|$ 很小时，$A = -2d$ 的模长趋于零，约束几乎“看不见”，此时 $f(\lambda)$ 对 λ 的敏感度也趋于零，二分可能把 λ 推到很大才找到变号点。若不加保护，λ 的倍增会一路涨到上限 10^{6}，返回一个幅度极大的控制量。本章代码在倍增循环里设了上界，并在不可行分支里用 `max(‖d‖, 1e-9)` 做归一化保护，两处都是为了在接近碰撞的极端情形下保持数值可控——不是为了算得更准，而是为了不产生非法值。

还有一个更快的替代做法：$u(\lambda)$ 是分段线性的，可以按“哪些分量先撞上盒约束”分成若干段，在每段内解一个线性方程，取最小的可行 λ。这样不需要迭代，代价是要处理 2^{n} 种组合。二维时组合数只有 4，解析解是划算的；维度升高后组合爆炸，二分反而更稳。**闭式解不是“更高级”，只是“在低维时更便宜”**。

79.4.3 二维算例：滤波前后

把上面两个模块接到 79.1 节的场景上，得到表 79-2。

表 79-2　二维避障算例：名义控制与 CBF-QP 的对比

指标	无滤波	CBF-QP
最小净距 / cm	−10.0	+21.5
最小屏障值 h_{min} / m²	−0.2125	+0.0754
到达时间 / s	4.32	4.98
路径长度 / m	4.00	4.07
峰值加速度 / (m/s²)	5.00	5.00
约束激活占比	—	10.0%
不可行步数	—	0
求解中位耗时 / μs	—	8.2

四组数字值得逐条读。

净距从 −10.0 cm 变成 +21.5 cm。负号不是“擦边”，是实打实地穿进障碍 10 cm——名义控制器根本不认识障碍，它只朝目标点走。滤波后最近点距障碍表面 21.5 cm，比安全边界所对应的 $R - r_{obs} = 0.15$ m 还多 6.5 cm，说明轨迹是从边界内

侧通过的，而不是贴着约束滑过去。

代价是 0.66 s 和 7 cm 路径。相对 4 m 的行程，绕行只多了 1.75% 的路程。这就是“投影”与“加权”的差别：加权罚函数要让轨迹远离障碍，必须给足够大的权重，代价是全过程的控制都被扭曲；投影只在约束边界附近改动控制，其余时间原样放行。

激活占比 10.0% 说明 90% 的时间安全滤波器是一个恒等映射。这也是分层结构在算力上的好处：安全层的平均开销接近于零，只有靠近障碍的那段时间才真正消耗求解时间。

峰值加速度两列都是 5.00，说明滤波后的控制仍然贴着执行器上限——CBF 不会“温柔地”绕行，它会在需要时用满全部能力。这一点常被误解，以为安全滤波器会让系统变“软”；实际上它只是把控制从“朝目标”改成“朝目标且不越界”，力度可以一点都不减。

最后两行说明“保证成立”与“算得过来”这两件事同时做到了。不可行 0 步意味着式 (79-17) 在整个 12 000 步里始终有余量；中位耗时 8.2 μs 意味着在 1 kHz 的周期里，安全层只占 0.8%，剩下的算力都可以留给任务层。这两个数字要一起看：只报 0 步不可行而不报耗时，读者会怀疑是不是把 R 取得过大；只报耗时而不报不可行步数，则无法判断这个速度是不是用“经常来不及”换来的。

还要注意 8.2 μs 是**约束未激活时的路径**（直接比较后放行），不是完整二分的耗时。完整的二分路径在 P95 那一侧，本章实测 P95 为 254.6 μs。汇报时用一个数概括“求解很快”是不准确的；正确说法是“90% 的周期走快路径（8.2 μs），10% 的周期走完整投影（最坏 254.6 μs）”。

79.4.4 α 的影响：一次扫描

把 $\alpha_1 = \alpha_2 = \alpha$ 作为唯一变量扫描，其余参数不动，得到表 79-3。

表 79-3　α 扫描（其余参数同 79.1 节，$a_{max} = 5$ m/s²）

α / (1/s)	最小净距 / cm	h_{min} / m²	到达时间 / s	峰值加速度	激活占比	不可行步数
1	+65.1	+0.8020	6.26	1.74	22.7%	0
2	+38.7	+0.3161	5.44	3.81	15.7%	0
4	+21.5	+0.0754	4.98	5.00	10.0%	0
8	+16.1	+0.0122	4.83	5.00	5.5%	0
16	+10.6	−0.0467	4.79	5.00	3.0%	269

三个趋势很清楚：净距随 α 单调下降（65.1 → 10.6 cm），到达时间单调缩短（6.26 → 4.79 s），激活占比单调下降（22.7% → 3.0%）。原因在式 (79-7) 的指数下界：α 是允许 h 衰减的速率，α 越大，系统被允许越快地逼近边界，因此越“敢”往障碍附近走。这就是 79.2 节那句“α 是保守性旋钮”的定量版本。

$\alpha = 1$ 是典型的过度保守：为了让 h 缓慢衰减，系统从很远处就开始被约束拽住，全程 22.7% 的时间在解约束，最后离障碍 65 cm——比 $\alpha = 4$ 多出两倍，到达时间还晚了 1.28 s。这种参数在现场很常见，因为“看起来更安全”。

$\alpha = 16$ 是另一头的失败。h_{min} 变成负值，同时出现 269 步不可行，说明需要的最小加速度超过了 a_{max}，式 (79-17) 的判据被破坏。注意此时“激活占比”反而最低（3.0%）——约束在少数步里被触发，但一触发就已无法满足。**激活占比低不等于安全**，这个指标只能用来看平均算力，不能用来判断保证是否成立。

$\alpha = 4$ 与 $\alpha = 8$ 是可用区间。前者留 21.5 cm 净距、代价 0.66 s；后者留 16.1 cm、代价 0.51 s，但 h_{min} 只剩 $+0.0122$，对扰动和离散化几乎没有余量。工程上若无特殊要求，取 α 在 $4 \sim 8$ 之间，并把式 (79-17) 的余量核到 30% 以上。

79.4.5 一维制动：裕量从哪里来

回到一维例子，用 $y_0 = 1.0$ m、$v_0 = -1.0$ m/s、$a_{max} = 5$ m/s² 运行，得到表 79-4 与图 79-5。

表 79-4　一维紧急制动的停止位置与峰值减速度

α / (1/s)	停止位置 / m	最小位置 / m	峰值减速度 / (m/s²)
1	0.050	0.050	1.00
2	0.010	0.010	0.74
4	0.003	0.000	1.47
8	0.001	0.000	2.94
4（0.30 m 裕量）	0.303	0.300	1.47

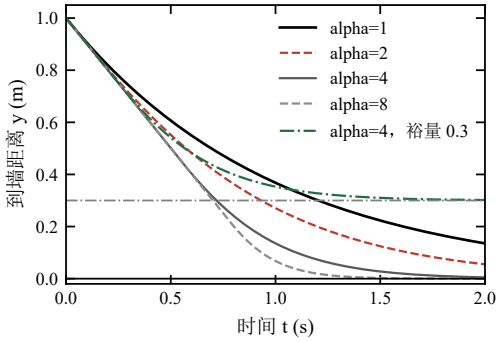


图 79-5　一维紧急制动的时间响应与裕量的作用

三条结论。

第一，CBF 只在必要时介入。$\alpha = 4$ 时前 0.5 s 控制量恒为零——式 (79-19) 的右端还是负的，滤波器没有理由改动名义控制（这里是“不加速”）。直到距离降到约 0.5 m，约束才变为活跃，减速度随即上升。这与二维算例 10.0% 的激活占比是同一件事的两个尺度。

第二，零裕量的“安全”只是数学上的安全。四组参数分别停在 50、10、3、1 mm，都在 $h \geq 0$ 之内，但没有一组是工程可接受的。CBF 的保证不包含“最小间距”这一项，最小间距必须由使用者通过 R 里的 $\delta_{sense} + \delta_{ctrl}$ 显式给定。最后一行把裕量设成 0.30 m，停止位置立刻变成 0.303 m，而峰值减速度一点没变（1.47）——**裕量的代价是行程，不是控制能量**，这是它比“把 α 调小”更划算的原因。

第三，峰值减速度对 α 不单调。$\alpha = 1$ 时为 1.00，$\alpha = 2$ 反而降到 0.74，再往上又升到 1.47、2.94。原因是 α 同时改变介入时刻与所需强度，两者对峰值的作用相反：小 α 介入早但持续时间长，大 α 介入晚但瞬时力度大。所以不能凭单个 α 值判断“激进程度”，必须同时看 $h(t)$ 与 $u(t)$ 两条曲线。

图 79-5 中绿色点划线是加 0.30 m 裕量的情形。它相对 $\alpha = 4$ 的实线整体上移，斜率几乎相同——这再次说明裕量是“平移安全边界”，不改变闭环动态。

连续情形到此可以收口：一维能写出全部解析式，二维能化成一次求根，两者的定性结论一致——保证来自约束，保守程度来自 α 与 R，二者分工明确。但真实控制器是离散的，约束只在采样点被检查，采样间隔内的漂移没有任何人盯着。这正是下一节要补的洞。

79.5 离散情形

79.4 节的保证建立在“每个时刻都检查约束”之上。真实控制器不是这样工作的：它每 T_s 秒读一次状态、算一次控制，然后把控制量保持到下一个采样点。约束只在采样点成立，采样间隔内发生了什么，CBF 一无所知。这一节把这段差距量化，并给出补它的标准动作。

79.5.1 离散化从两个地方破坏保证

第一条路径是**采样间隔内的漂移**。相对阶 2 的系统里，h 的一阶导只含速度，二阶导才含控制。在 t_k 时刻满足 $\dot{\psi}_1 + \alpha_2\psi_1 \geq 0$，只说明此刻 h 的下降趋势被压住；在 $[t_k, t_k + T_s)$ 内控制保持不变，h 继续按二次曲线下降，下降到什么程度取决于这段区间内的速度与加速度：

$$h(t_k + \tau) \;\geq\; h(t_k) - \big(\|\dot{h}\|_{max}\,\tau + \tfrac{1}{2}\|\ddot{h}\|_{max}\,\tau^2\big), \qquad \tau \in [0, T_s] \tag{79-24}$$

这个下界很松（它假设速度和加速度都取最不利方向），但它给出了正确的量纲与阶次：**漂移量随 T_s 线性增长，随 T_s^2 增长的那一项来自一个周期内的加速度累积**。

第二条路径是**离散可行性**。t_k 时刻的 QP 有解，不代表 t_{k+1} 时刻还有解。控制量被保持了一整个周期，等到下一个采样点再想改，状态可能已经进入“任何控制都救不回来”的区域——也就是 79.4 节图 79-4 里可控不变集之外的部分。这条路径与漂移不同：漂移是“少走了一点安全余量”，离散可行性是“根本没有安全控制可用”，后者在指标上表现为不可行步数跳增。

两条路径可以用一个数区分：**漂移是量的问题，可行性是质的问题**。漂移可以靠加裕量补偿，因为它的量级可以估、可以扣；可行性不能靠“多加一点裕量”补救，因为它一旦发生，说明在当前状态下连最有利的控制都维持不住约束，多加裕量只会让它更早发生。这也是为什么 79.5.4 节把“超时”和“不可行”归为同一类处理——两者都属于质的问题。

有一个常用的经验判据：当 $\alpha T_s \leq 0.1$ 时，一个周期内 h 的相对变化不超过 10%，离散效应与模型误差同量级，可以合并处理；当 αT_s 超过 0.5 时，离散效应成为主导误差，再怎么调裕量也补不回来。本章 $\alpha = 4$、$T_s = 1$ ms 时 $\alpha T_s = 0.004$，远在安全区内；$\alpha = 8$、$T_s = 100$ ms 时 $\alpha T_s = 0.8$，已经越过 0.5，出现不可行是必然的。

79.5.2 护卫裕量

补第一条路径的标准做法是**护卫裕量**（guarding margin）：不去改控制器，而是把安全边界向外推一个与 T_s 同阶的距离，让“采样点安全”蕴含“区间内也安全”。取

$$m = \|v\|_{max} T_s + \tfrac{1}{2} a_{max} T_s^2 \tag{79-25}$$

其中 $\|v\|_{max}$ 是场景中可能出现的最大速度。把它加到安全半径上，

$$h_m(x) = \|p - p_{obs}\|^2 - (R + m)^2 \tag{79-26}$$

再对 h_m 施加原来的 CBF 条件。79.4 节的论证没有任何改动，只是安全集变小了：在一个采样周期内，状态最多向障碍方向移动 m，因此 $h_m \geq 0$ 蕴含真实的 $h \geq 0$。

式 (79-25) 的两项含义不同。第一项是“位置在一个周期内能走多远”，第二项是“加速度在这段时间里额外贡献的位移”。后者只有在 T_s 大到几十毫秒时才显出来：$T_s = 1$ ms 时第二项是 2.5×10^{-6} m，完全可忽略；$T_s = 100$ ms 时它是 25 mm，与第一项的 0.287 m 相加就是 0.312 m。

式中用的速度应当是**相对障碍的速度**，不是绝对速度。障碍静止时两者相同；障碍迎着来的时候，$\|v\|_{max}$ 要换成 $\|v - v_{obs}\|_{max}$，否则裕量不足。动态障碍场景里这一项经常被漏掉，表现是“仿真里明明有裕量，实车却擦上”。

裕量的方向也要交代清楚：m 是加在 R 上的，等价于把安全集缩小一圈，而不是把控制“调得更猛”。这两者的区别在于，前者改变的是约束的位置，后者改变的是约束的右端项。用前者，79.4 节的全部论证（包括前向不变性和指数下界）都原样成立；用后者，就得重新推一遍。**能用位置换来的保守性，不要用增益去换**。

同时保留 79.3 节的采样周期规则 $T_s \leq 0.1/\alpha$。$\alpha = 8$ 时它给出 $T_s \leq 12.5$ ms，比式 (79-25) 更早生效——**参数 α 决定了周期上限，周期又决定了裕量**，这条链不能倒过来走。

79.5.3 采样周期扫描

把 α 固定为 8，换用更激进的跟踪参数（$k_p = 9$、$k_d = 6$，峰值速度约 2.9 m/s），只改控制周期，得到表 79-5 与图 79-6。

表 79-5　采样周期对安全性的影响（$\alpha = 8$，峰值速度约 2.9 m/s）

T_s / ms	h_{min}（无裕量）/ m²	h_{min}（加裕量）/ m²	护卫裕量 / m	不可行步数
1	+0.0188	+0.0211	0.003	0
2	+0.0190	+0.0236	0.005	0
5	+0.0195	+0.0313	0.013	0
10	+0.0203	+0.0447	0.026	0
20	+0.0218	+0.0740	0.053	0
50	+0.0273	+0.1847	0.140	0
100	+0.0106	+0.4250	0.312	3

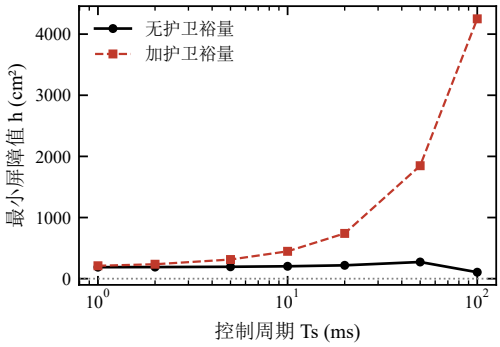


图 79-6　控制周期与最小屏障值：无护卫裕量与加护卫裕量的对比

表中有三组现象值得说明。

第一，$T_s \le 10$ ms 时离散代价可以忽略。 无裕量时 h_{min} 从 1 ms 的 $+0.0188$ 只变到 10 ms 的 $+0.0203$，全程 0 步不可行。这与 $T_s \le 0.1/\alpha = 12.5$ ms 的判据吻合：判据不是玄学，它对应的正是“一个周期内屏障值的变化不超过 α 尺度的十分之一”。

第二，1–50 ms 之间 h_{min} 反而略有上升。 这是零阶保持的“过度修正”效应：控制量按采样点的状态算出后被保持一整个周期，周期越长，同样的修正量作用越久，轨迹被推得离障碍更远一点。它不是算法变好了，而是保持时间变长带来的副作用；一旦保持时间继续加大，延迟效应占了上风，h_{min} 就掉头向下。

第三，100 ms 是断崖。 h_{min} 掉到 $+0.0106$，并出现 3 步不可行，正好落在“信息延迟压过过度修正”的那一侧：约束用的是上一个采样点的状态，等下一个采样点检查时状态已经走远，需要的最小加速度超过了 a_{max}。

加裕量那一列给出了保守性的价格：h_{min} 随 T_s 单调上升，因为轨迹被裕量推得越来越远。100 ms 时裕量 0.312 m，h_{min} 为 $+0.4250$，由基础半径 $R = 0.55$ 反算，实际最近净距约 0.45 m，是基线 0.15 m 的三倍。**在这个周期下系统是安全的，但已经不像在避障，更像在绕一个大圈。**

图 79-6 把两组数据画成对数横轴。注意两条曲线的走势相反：不加裕量时 h_{min} 先随 T_s 缓慢上升、50 ms 后掉头向下，加裕量时则单调上升。它们说明同一件事——周期增大时，安全要么靠“控制保持得久”带来的过度修正，要么靠裕量主动退让，前者的有效期到 50 ms 为止，后者则要一直付绕行的代价。

1 ms 那一行也值得看一眼：加裕量后 h_{min} 只从 $+0.0188$ 升到 $+0.0211$，变化不到 0.0023。原因是裕量只有 3 mm，轨迹被推远的幅度也在这个量级，而 h 对距离的敏感度是 $2R \approx 1.1$，两者相乘正好是千分之几。**在小周期下裕量的收益本来就小**，它的价值要到周期变大时才显现，所以不要因为“加了裕量但指标几乎没变”就认为裕量无用。

79.5.4 离散实现的四条纪律

1. **周期由 α 定，不由 CPU 定。** 先按 $T_s \le 0.1/\alpha$ 算出上限，再看算力是否够。若不够，正确做法是降低 α（接受更保守的绕行），而不是把周期拉长——后者破坏的是保证本身。

2. **裕量按实测最坏速度算。** 用设计上限会过度保守（例如按 5 m/s 算，1 ms 时裕量 5 mm 而非 3 mm），用典型速度会不足。取场景库中的最大接近速度，并留 10% 余量。
3. **超时按不可行处理。** 求解器超时意味着“这一周期没有验证过的安全控制”，必须走与不可行相同的回退分支，同时置故障位。不能默认“沿用上一步的解”，除非能证明上一步的解在当前状态仍满足约束。
4. **验证要在采样点之间查。** 仿真时用比控制周期小一个量级的积分步长记录 $h(t)$，取全程最低点而不是采样点值。本章代码用 1 ms 积分、1–100 ms 控制，h_{min} 由积分轨迹逐点算出，而不是由控制时刻的抽样值算出。

这四条里第一条最常被违反。项目进度紧的时候，“先把周期拉长跑起来”是很自然的妥协，而一旦跑通，就没人再回头改——于是系统在验收时勉强通过，在上量后频繁报警。正确的顺序是先定 α、再定 T_s、最后看算力是否够；算力不够就降 α，而不是升 T_s。降 α 的代价是绕行变大，这个代价是可预期、可验收的；升 T_s 的代价是保证失效，这个代价不可预期。

第四条也有一个容易踩的坑：积分步长太粗时，h 的最低点会被“跨过去”。若真实最低点是 $+0.005$，积分步长 10 ms、速度 2 m/s，一步就走 2 cm，很可能刚好跳过最低点，采样到的值比真实值大。判断方法很简单：把积分步长减半再跑一次，若 h_{min} 变化超过 5%，说明步长还不够细。

79.5.5 与连续保证的差距有多大

严格处理离散性有另一条路：直接使用离散时间 CBF，把条件写成

$$h_{k+1} \;\geq\; (1-\gamma)\,h_k, \qquad \gamma \in (0,1) \tag{79-27}$$

它要求 h_{k+1} 显含 u_k，因此需要一步预测模型，本质上把 T_s 变成了模型的一部分。好处是保证更紧、裕量更小；代价是需要精确的离散模型，模型误差直接进约束。

本章采用“连续约束 + 护卫裕量”，原因是它对模型误差更宽容，且与 79.4 节的理论完全同源，不需要另起一套证明。代价是保守：表 79-5 最后一行 0.45 m 的实际净距就是保守的价格。工程上的判断标准很简单——**如果裕量占安全半径的比例小于 20%，就用这个方案；超过 50%，说明周期选得不对，应该回去改 T_s 或 α。**

把两条路的保守性摆在一起看更清楚。$T_s = 10$ ms 时，护卫裕量 0.026 m，占 R 的 4.7%，实际净距约 19 cm；$T_s = 100$ ms 时，裕量 0.312 m，占 57%，净距 45 cm。离散时间 CBF 在同样周期下能把这一段多出来的绕行收回去一部分，因为它的约束里显含“下一个采样点会走多远”，不需要靠提前退让来买安全。但它要求精确的离散模型，而模型的误差会直接变成约束的误差——用一个不准的模型换来的“紧”，很可能比保守更危险。**在模型精度没有把握时，宁可保守**，这是本章选择前者的核心理由。

安全层的“逐时刻保证”到这里就完整了：连续情形给出理论，离散情形给出实现，两者之间用裕量连接。但还有一个问题没有回答——加上滤波之后，闭环还稳定吗？名义控制器原本调好的超调、阻尼，会不会被安全层改坏？这是下一节的主题。

79.6 稳定性与性能

安全滤波器把闭环从“一个控制器”改成了“一个控制器加一个分段投影”。分段就意味着切换，切换就意味着原来那套线性稳定性结论不能直接套用。这一节回答两个问题：加了滤波之后闭环还稳不稳；安全换来的性能损失有多大。

79.6.1 闭环变成了什么

未触发约束时，滤波器是恒等映射，闭环就是名义闭环，稳定性由 k_p、k_d 决定。触发约束时，控制器变成“在 $Au \le b$ 与盒约束的交集上做投影”，是一个非线性、状态相关的映射。两者之间的切换由 $A(x)u_{nom} \le b(x)$ 是否成立决定，因此整个系统是**分段光滑的混杂系统**。

对这类系统，CBF 理论给出的是**前向不变性**：只要初始状态在 $\mathcal{C}$ 内、约束处处可行，那么轨迹永不离开 $\mathcal{C}$。它没有给出渐近稳定性，也不保证收敛到目标点。这一点经常被混淆，需要分开记：

- 安全性（safety）：轨迹不进入不安全集，由 CBF 条件保证；

- 活性（liveness）：轨迹最终到达目标，CBF 不保证，需要另做分析。

本章算例里两者都成立：4.98 s 到达目标，全程 $h > 0$。但这不是 CBF 的功劳，而是因为目标点在障碍的另一侧、绕行路径存在，且名义控制器有足够的能力绕过去。

分段系统为什么不能直接套用线性结论，可以举一个具体的例子。名义 PD 的闭环极点是 -1.5（二重），相位裕度按二阶系统算约 65°。约束激活时，控制量被投影到约束面上，等效于在回路里插入了一个方向随状态旋转的饱和环节。这个环节不储能量，但它会改变回路增益的方向性：原本沿 x 轴的推进被转成沿 y 轴的让开，两个通道的耦合系数从 0 变成非零。对多输入系统，这种“旋转”足以让原本解耦设计的两个回路互相激励。

不过也不必过度悲观。有三条性质让实际系统通常保持稳定。第一，投影是到凸集的最近点映射，对固定凸集是 1-Lipschitz 的，不会放大扰动。第二，约束激活时控制量的幅值不会超过名义控制的幅值（盒约束是同一个），因此不会凭空引入更大的能量。第三，切换面 $A(x)u_{nom} = b(x)$ 是状态的光滑函数，除非轨迹恰好沿切换面滑动，否则切换是横截的、次数有限。三条合起来，工程上绝大多数情形是有界的；真正危险的是“沿切换面滑行”（chattering），这时需要滞后或软化。

理论上要给出渐近稳定的充分条件，通常要构造一个同时包含任务误差与屏障函数的复合 Lyapunov 函数，或者退一步只证明“最终有界”。本章不展开这套证明，因为它对参数的要求比工程实际更严，实用价值有限。**实践中判断稳定性，靠的是 79.6.5 节的三条检查，而不是定理。**

79.6.2 指数包络

CBF 条件直接给出了 h 的衰减上界。对相对阶 2、$\alpha_1 = \alpha_2 = \alpha$ 的情形，由 $\dot{\psi}_1 + \alpha\psi_1 \geq 0$ 可得 $\psi_1(t) \geq \psi_1(0)e^{-\alpha t}$，再解一次一阶不等式，得到

$$h(t) \;\geq\; h(0)\, e^{-\alpha t} \tag{79-28}$$

图 79-7 画出四组 α 下的 $h(t)$。起点距障碍中心 2.02 m，对应 $h(0) \approx 3.79$ m²，四条曲线都从这里出发。

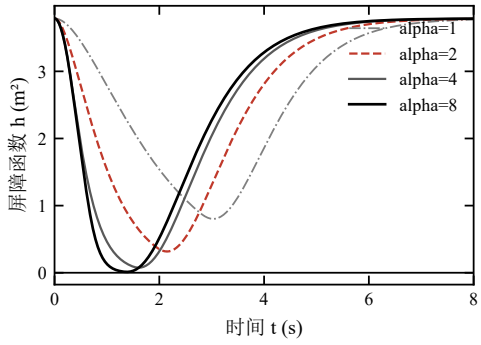


图 79-7　不同 alpha 下屏障函数的时间历程

这张图最值得注意的不是曲线形状，而是**下界有多松**。按式 (79-28)，$\alpha = 8$、$t = 4.83$ s 时下界是 $3.79e^{-38.6} \approx 6 \times 10^{-17}$ m²，而实测最低点是 $+0.0122$ m²。差了十四个数量级。原因在于指数下界只保证“不穿越零”，它把 h 的衰减速度按最坏情况估计，而实际轨迹在接近边界时会主动减速。

所以式 (79-28) 的正确用法是定性判断，不是定量预测：它告诉你 α 大意味着 h 被允许衰减得更快（曲线更靠下、最低点更小），但不能用来估算“最低点会是多少”。定量必须靠仿真，这也是本章所有数字都来自 `code/ch079_sim.py` 的原因。

图中每条曲线都是先降后升，最低点就是轨迹离障碍最近的那一刻。$\alpha = 1$ 时最低点出现在 3.03 s、数值为 $+0.802$ m²，与表 79-3 一致；α 增大后最低点数值迅速下降，出现时刻也提前。**$h(t)$ 的非单调性是几何造成的，不是稳定性问题**——绕过去之后距离重新变大，h 自然回升，读图时要结合轨迹一起看（图 79-1）。

下界为什么这么松，可以从推导里看出来。式 (79-28) 是把 $h(0)$ 当作初值、假设 ψ_1 始终贴在下界上得到的；而实际轨迹在 h 还很大时，ψ_1 往往是正的（正在远离或静止），此时约束根本不起作用，h 的衰减速度远低于 αh。换句话说，指数

下界描述的是“最坏情况下的最快衰减”，不是“实际衰减”。若想要更紧的估计，必须知道 $\psi_1(0)$ 的符号与大小：$\psi_1(0) > 0$ 时实际的衰减比下界慢，若同时 $\dot{h}(0) > 0$，h 还会先上升一段，那一段完全不消耗裕量。这也是为什么工程上更愿意直接用仿真给出 h_{min}，而不是用解析下界去“估”——解析式适合判断趋势和做参数窗口，不适合报数字。

79.6.3 性能损失的三个来源

安全层的性能代价可以拆成三块，每一块都有对应的量化方式。

绕行代价。 轨迹必须绕开障碍，路径变长。本章算例从 4.00 m 增到 4.07 m（+1.75%），到达时间从 4.32 s 增到 4.98 s（+15.3%）。时间增幅远大于路径增幅，原因是绕行段的速度被压低了——这不是路径几何的必然，而是 CBF 在接近边界时把加速度用在了“横向让开”而不是“纵向推进”。

饱和代价。 约束活跃时，控制量被限制在盒约束与半空间的交集内，名义控制器想要的“加速前进”被替换成“加速让开”。表 79-2 里滤波前后峰值加速度都是 5.00 m/s²，说明这个替换不是“减小控制”，而是“改变方向”。这也是为什么加了安全层之后，纵向跟踪会明显变慢。

切换代价。 约束反复激活与退出，会在边界附近产生控制量的跳变。跳变本身不影响安全（每次跳变后的控制都满足约束），但会激励未建模的高频动态，在柔性结构或电机电流环上表现为抖动。工程上的缓解办法是给激活判据加一点滞后，或者把硬投影换成带软化的版本，79.9 节会讨论。

三块代价里，只有第一块能用几何算出来。路径从 4.00 m 到 4.07 m 是几何量，任何安全边界都会带来同样的增量；但到达时间多出的 0.66 s 里，大部分不是“多走了 7 cm”，而是“在这 7 cm 上速度只有原来的零头”。这个区别决定优化方向：想省时间，调路径规划（让绕行更圆滑）没用，得调 α 或名义增益，让绕行段的速度恢复上去。反过来，若指标是路径长度而不是时间，那么本章的绕行代价只有 1.75%，几乎可以忽略——同一套参数在不同指标下的评价可以完全相反，这也是验收文档必须先写清指标的原因。

79.6.4 性能与保守性的权衡曲线

把表 79-3 的净距与到达时间画在一起，就是图 79-8。

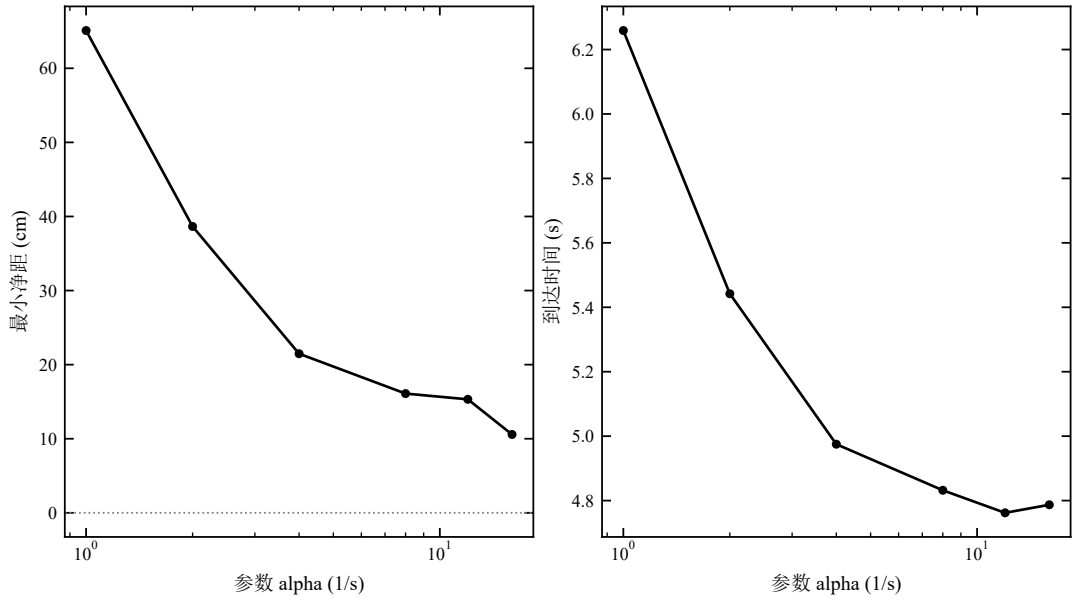


图 79-8　参数 alpha 对最小净距与到达时间的影响

左图净距随 α 单调下降，右图到达时间随 α 单调下降，两条曲线方向一致——**α 增大同时买到“更快”和“更贴边”**。真正需要看的是边际收益：

- α 从 1 到 4：省下 1.28 s，代价是净距从 65.1 cm 降到 21.5 cm（减少 43.6 cm）；

- α 从 4 到 8：只省 0.15 s，净距再降 5.4 cm；
- α 从 8 到 16：只省 0.04 s，净距再降 5.5 cm，但 h_{min} 越零、出现 269 步不可行。

第一段的收益最大，第三段的收益为零且风险陡增。工程上应在“边际收益开始变平”的拐点取值，本章就是 $\alpha = 4 \sim 8$。这条曲线还说明一件事：**用 α 换速度是有限度的**，超过拐点之后，多给的 α 只会把余量吃光，不会让系统更快。

这条曲线在实际项目里有三个用途，都不需要重新做理论分析。第一是选点：把需求写成“净距不小于 20 cm、到达时间不超过 5.0 s”，直接在图上找同时满足的区间，找不到就说明需求本身不相容，应该回去谈需求而不是继续调参。第二是估风险：所选点与不可行点（$\alpha = 16$）之间隔了几档，就是这组参数还剩多少“容错挡位”；若只隔一档，那么现场把速度提高 10% 就可能跨过去。第三是解释代价：评审问“为什么不用更快的参数”，把曲线指给他看，从 4 到 8 只买到 0.15 s，却要多付 5.4 cm 净距，这个交换是否值得是可以讨论的，但至少不是拍脑袋。

画这条曲线时要注意，它是**单参数**切片。α 变化时其余参数固定，所以曲线上的每个点都属于同一套 R、T_s、a_{max}。若这些量也变了，曲线要重画——把不同 R 下的点画在同一张图上，会得到“净距大、时间也短”的假点，因为 R 本身就同时影响两者。多维参数空间的正确做法是固定约束、做二维切片，而不是把一维扫描的结论外推。

79.6.5 怎么验证“滤波没有改坏原闭环”

三条检查，按顺序做。

第一，**远离障碍时逐位比对**。在障碍不影响轨迹的场景（目标点与障碍分处两侧、无障碍工况）跑一遍滤波前后的闭环，两条轨迹应当在数值精度内完全相同。这一条验证“恒等映射”分支没有实现错误。

第二，**单独测名义闭环指标**。把滤波器旁路，测超调、上升时间、调节时间；再把滤波器接上，在约束不激活的时段测同样的指标。两者应当一致。若不一致，说明有隐藏的状态依赖混进了名义控制器。

第三，**在约束边界附近做小信号测试**。给定一组初始状态，让轨迹恰好擦过边界，观察控制量是否出现高频切换。若一个采样周期内激活/退出反复发生，就需要加滞后或软化。

稳定性这件事在小扰动下最容易被忽视：名义闭环的相位裕度可能是 45°，但约束活跃时闭环增益和相位都变了，原来的裕度不再成立。本章算例没有出现失稳，是因为对象是双积分器、控制是纯 PD、没有延迟；换成有延迟或柔性模态的对象，必须重新做一遍频域检查。79.11 节会把这类边界条件列全。

频域检查怎么做，给一个可操作的步骤。先在“约束始终激活”的假设下把投影近似成线性环节：单约束时投影只改变沿约束法向的那个分量，因此它等效于一个方向相关的一阶衰减环节，衰减强度由约束的松紧决定——约束越紧（b 越接近 Au_{nom}），衰减越弱，回路越接近原样；约束越松，投影越强，等效增益越低。把这一个环节串进名义回路，画 Bode 图，看相位裕度是否还在 30° 以上。这只是近似——投影的增益随状态变化，不是常系数——但它能把“最坏情况”找出来：约束面朝向与运动方向夹角越小，投影改变的分量越接近主运动方向，相位滞后越明显。若这一步就已经不过关，实车一定抖，不必等到上车再发现。

还有一条常被忽略的检查：**在约束面附近给一个阶跃，看恢复过程是否单调**。若阶跃后出现两次以上的穿越（控制量在约束内外反复切换），说明系统在切换面上滑行，需要引入滞环。滞环宽度取多少，没有公式，经验值是“让激活阈值比退出阈值大 5%–10%”，宽度太大则安全保证被削弱，太小则不起作用。

79.7 参数意义与作用

本章的代码里有十来个可调量，但它们不是平权的：有的直接对应工程指标，有的只是实现细节，还有的必须由别的参数推出来。这一节按“谁决定谁”的顺序把它们排队，并给出每个参数增大时的后果。表 79-6 是总表。

表 79-6　安全滤波器的参数、取值与作用方向

符号	含义	单位	本章取值	增大时的后果
R	安全半径	m	0.55	净距变大、绕行变长
α_1, α_2	屏障衰减速率	1/s	4.0 / 4.0	更贴边、更快，需更大加速度
a_{max}	各轴加速度上限	m/s²	5.0	可行域变大、可承受更高 α
Q	修正权重矩阵	—	I	改变各通道修正的优先级
T_s	控制周期	ms	1	保守性与不可行风险同时上升
m	护卫裕量	m	0.003	净距变大、可行域被压缩
k_p, k_d	名义 PD 增益	—	2.25 / 3.0	轨迹更急、约束激活更频繁

79.7.1 几何参数 R：唯一直接对应工程安全的量

R 是四个量的和：$R = r_{obs} + r_{robot} + \delta_{sense} + \delta_{ctrl}$。前两项是几何尺寸，后两项是误差预算。本章取 $0.40 + 0.15 = 0.55$ m，把机器人外廓与两个误差项合并进了 0.15 m。

δ_{ctrl} 最容易被漏掉，它至少包含三部分：定位误差、控制周期内的位移、执行器响应延迟。第三项可以折算成距离：若执行器从收到指令到出力有 τ 的延迟，则以速度 v 接近时，这段延迟等效于多走近 $v\tau$。$v = 2$ m/s、$\tau = 20$ ms 就是 4 cm——比很多人的估计大一个量级。

R 增大时，净距与绕行同步增大，二者近似一比一。它不改变闭环动态，只平移安全边界（图 79-5 已示）。因此在现场调参时，**先定 R，再也不要动它**；觉得“不够安全”就回去查误差预算，而不是随手加几厘米。

四项预算里，前两项可以查图纸，后两项必须实测，这是 R 最容易被低估的地方。定位误差要在最差的采样条件下测，不能取标称精度；控制周期内的位移要用实际最大速度乘实际周期，而不是设计速度乘标称周期。汇总时不要取“各项典型值之和”，因为最坏情况往往同时发生：定位误差最大的时刻，通常也是速度较高、延迟影响较大的时刻。

还有一条方向性判断值得记住：**R 估小的后果不对称**。估大一点，代价是绕行变长、通行变慢，属于性能损失；估小一点，代价是安全边界落到真实误差包络之内，属于安全失效。两者不能用同一个容差对待。工程上的做法是给 R 留一个显式的“未分配余量”，例如把四项之和再乘 1.1，并把这个 10% 写进文档——它不是一个模糊的“保险系数”，而是一条有出处的、可以复核的预算项。

R 还会间接影响可行性。R 越大，安全边界离障碍越远，系统需要在更早的位置开始转向，等效于把可用通道压窄。当通道宽度接近 $2R$ 时，无论 α 取多少都无法通过，这时问题不在调参，而在任务几何本身。79.9.3 节讨论多障碍相容性时说的“把通道挤没了”，指的就是这种情形。

79.7.2 速率参数 α：保守性旋钮

α 的量纲是 1/s，物理意义是“允许屏障函数衰减的时间常数倒数”。它同时出现在三个地方：约束的右端项、指数下界 $h(0)e^{-\alpha t}$、采样周期判据 $T_s \le 0.1/\alpha$。三个地方的作用方向一致——α 越大越激进。

把指数下界读一遍，可以看清“保守”到底保守在哪。它说的是：只要约束始终满足，屏障函数最坏情况下按 $e^{-\alpha t}$ 衰减，绝不会中途加速跌落。α 越小，允许的跌落越慢，控制器就越可以“慢慢来”；α 越大，允许的跌落越快，控制器就必须“立刻处理”。所以 α 不是一个“安全等级”旋钮，而是“允许多快地把裕量用掉”的旋钮。想让系统更安全，方向是**减小** α，而不是增大。

这个结论与很多人的直觉相反，因为增大 α 会让约束更“硬”、介入更早。介入早确实看起来更安全，但代价是：同样一段接近过程，系统要在更远的位置就开始修正，绕行更大，而且边界附近需要的法向加速度随 α 线性增长，一旦超过 a_{max} 就直接不可行。本章 $\alpha = 1$ 时 $h(t)$ 最低点出现在 3.03 s、数值 $+0.802$ m²，激活占比 22.7%，就是“介入早但拖得久”的典型；$\alpha = 4$ 时最低点更低、但过程更短，通行效率反而更好。

α 有一个硬上限，由执行器能力决定。由式 (79-19)，当 $h \to 0$ 时所需加速度由 $2\alpha\|v_r\|$ 主导（v_r 是法向接近速度），若这个值超过 a_{max}，约束在边界附近就无解。本章把 α 从 8 提到 16 时出现的 269 步不可行，正是这条上限被突破的表现。

注意判据里用的是**法向**速度而不是合速度：轨迹在最近点处的法向速度为零，所以系统可以在很高的切向速度下仍然满足约束，这也是“贴着障碍绕行”在物理上可行的原因。

两个 α 的分工是：α_1 决定 ψ_1 的衰减，α_2 决定 ψ_1 被拉回零的力度。取 $\alpha_1=\alpha_2$ 时特征多项式是 $(s+\alpha)^2$，行为最直观；取 $\alpha_2>\alpha_1$ 会让二阶项更“硬”，响应更陡但更容易饱和。工程上建议先取相等，确认可行后再单独调 α_2。若确实要分开调，可以按“α_1 管接近速度、α_2 管残余偏差”来理解：α_1 大意味着不容忍快速接近，α_2 大意味着不容忍停在边界附近不动。

还有一个经验关系：α 应当不低于名义闭环带宽的两倍。本章名义闭环 $\omega_n=1.5$ rad/s，$2\omega_n=3$，而 $\alpha=1$ 时约束激活占比升到 22.7%、到达时间拖到 6.26 s，正是“安全层比任务层还慢”的典型症状。α 太小时，安全层来不及在轨迹穿出边界前完成修正，只能靠提前很远就开始干预，于是表现为过度保守。

把两条限制放在一起，α 的可用区间是一个窗口：

$$\alpha_{min}=2\omega_n \;\le\; \alpha \;\le\; \alpha_{max} \tag{79-29}$$

下限由“安全层不能比任务层慢”决定，可以直接算；上限由“边界附近刹得住”决定，没有可靠的闭式——它取决于轨迹在边界附近的法向速度，必须用式 (79-17) 在场景库的最不利工况上逐条核算。本章实测的结果是 $4\sim8$ 可用、16 越界，与这个窗口的判断一致。参数调不出来时，先算下限、再用式 (79-17) 卡上限，比盲目试值快得多。

79.7.3 能力参数 a_{max} 与权重 Q

a_{max} 不是安全参数，但它决定了“多高的 α 才可行”。它同时出现在三处：名义控制器限幅、QP 的盒约束、式 (79-17) 的可行性判据。三处必须用同一个数，否则会出现 79.3 节说的假性不可行。

提高 a_{max} 的收益是全局的：制动距离按 $v^2/(2a_{max})$ 缩短，可行域按比例扩大。但它受执行器物理限制，不能当调参手段。真正要核的是“最不利工况下式 (79-17) 是否留有余量”，而不是“平时够不够用”。

Q 只在约束激活时起作用，因此它对名义性能没有影响，这也是分层结构的好处之一。它的作用是在多个通道之间分配“修正量”：$Q=I$ 时是欧氏投影，各通道按同一尺度让路；若某个通道的量纲不同（力与力矩），必须先归一化，否则投影会优先修正量纲数值小的通道。这一点在 79.3 节已经提过，这里补充它的后果：量纲错误不会报错，只会让某个轴“莫名地”总是承担全部修正。

Q 还有一种更有用的用法：把它当成“优先级”。若某个通道的偏离代价更高（例如横向位置比纵向速度更不允许偏），就在 Q 里给它更大的权重，投影会倾向于保留该通道的控制、牺牲另一个通道。这与 79.2 节“切向自由、法向受限”的几何结论不冲突——几何决定了约束的方向，Q 决定了在满足约束的前提下如何在剩余自由度里取舍。

a_{max} 的余量怎么核，给一个可以直接照做的算法。把场景库里每条轨迹的 (p,v) 逐点代入式 (79-17)，算出所需的最小加速度 a_{req}，取全程最大值，再看 a_{max}/a_{req} 的比值。本章默认工况下这个比值在 1.5 左右，$\alpha=16$ 时降到 1 以下，于是出现不可行。比值大于 1 只是必要条件，还要留出扰动的空间：若 79.9 节要用 $\Delta=0.8$，那么 a_{max} 至少要比 $a_{req}+2\Delta$ 大，否则鲁棒化之后可行域又会被吃掉。

79.7.4 时间参数 T_s 与 m

这两个参数不能独立选，它们由 α 推出来：

$$\alpha \;\longrightarrow\; T_s \le \frac{0.1}{\alpha} \;\longrightarrow\; m=\|v\|_{max}T_s+\tfrac{1}{2}a_{max}T_s^2 \tag{79-30}$$

按本章 $\alpha=4$，$T_s\le 25$ ms；实际取 1 ms，即使按 2.9 m/s 的高速工况，裕量也只有 0.003 m，占 R 的 0.5%，可以忽略。若因算力受限取 100 ms，裕量变成 0.312 m，占 R 的 57%——这已经不是一个“补丁”，而是把安全区缩小了一半以上。

m 的表达式为什么是这两项，值得拆开看。在一个控制周期内，最坏情况是：系统以最大速度朝障碍方向运动，同时加速器以 a_{max} 继续加速，直到下一个周期才更新指令。位移由两部分组成：匀速项 $\|v\|_{max}T_s$ 和加速项 $\frac{1}{2}a_{max}T_s^2$。前者与 T_s 成正比，后者与 T_s^2 成正比——这解释了为什么 T_s 从 1 ms 增到 100 ms 时，m 从 3 mm 增到 312 mm，放大了约一百倍：

主导项从匀速项换成了加速项，而 T_s 本身放大了 100 倍，平方项放大了 10000 倍后再乘上很小的系数，两者叠加的结果仍在百倍量级。

这条链的正确用法是**从右往左检查**：先看算力能支持多小的 T_s，再反推 α 的上限，最后核 m 占 R 的比例是否可接受。若比例超过 20%，说明 T_s 太大，应该降 α 或提高算力，而不是接受这个裕量。

还要注意 m 是**加在** R **上还是在** h **里扣除**，两种实现的效果一样，但不能重复。本章代码用的是前者：$R \leftarrow R + m$，屏障函数仍然是 $\|d\|^2 - R^2$。若同时在 R 里加 m、又在约束右端减 m 的相关项，安全边界会被收紧两次，性能无谓损失。检查方法很简单：把 T_s 改成极小值，若 h_{min} 明显变大，说明裕量被重复计入了。

79.7.5 名义参数为什么会出现在这张表里

k_p、k_d 属于名义控制器，本不该出现在安全章，但它们通过“轨迹”影响安全层的工作量。增益越大，轨迹越急，接近障碍时的法向速度越大，约束激活越频繁，需要的修正也越大。

本章名义增益取临界阻尼（$K_p = 2.25$、$K_d = 3.0$，$\omega_n = 1.5$ rad/s），轨迹平缓，激活占比 10.0%。把增益提到 $k_p = 9$、$k_d = 6$（79.5 节的“高速接近”配置），峰值速度从 1.63 m/s 升到 2.9 m/s，$T_s = 100$ ms 时立刻出现 3 步不可行。

这说明一件事：**安全层的可行性不只取决于安全参数，还取决于任务层有多急**。验收时必须把“最急的任务”放进场景库，否则安全层在温和工况下测出来的余量没有意义。

从参数敏感度看，可以把这张表分成三档。第一档是 R，它只改几何、不改动态，影响可预测，调一次就固定。第二档是 α、T_s、m，三者互相耦合，改一个必须重核另外两个。第三档是 k_p、k_d 与 Q，它们通过轨迹间接影响安全层，最容易被“顺手调一下”而破坏已经验证过的结论。现场出问题，八成出在第三档——有人为了“响应快一点”改了增益，没有重跑安全验收。

79.7.6 参数之间的三处耦合

把耦合关系单独列出来，是因为它们最容易在改动时被漏掉。

第一处：α 与 T_s。 由 $T_s \le 0.1/\alpha$，提高 α 必然压缩可用的 T_s。若算力已经到顶，提高 α 就等于把系统推向不可行。反过来，若为了省算力把 T_s 放大，就必须先把 α 降下来。

第二处：R 与 α。 两者都影响保守性，但机制不同：R 平移边界，α 改变逼近速度。同样是想“更安全一点”，增大 R 会让全程绕行变大，减小 α 会让介入更早但轨迹更平滑。诊断时看激活占比：占比高而净距大，是 α 偏小；占比低而净距刚好卡在边界上，是 R 偏小。

第三处：a_{max} 与 Δ。 鲁棒项要占用执行器能力，a_{max} 里必须留出 2Δ 的空间。若按名义工况把 a_{max} 用到 95%，鲁棒化之后就没有余量了。这也是为什么 79.9 节的鲁棒算例要把 Δ 控制在 a_{max} 的 20% 以内——不是理论要求，而是给参数留活路。

79.8 调参方法与经验

安全滤波器的调参比 PID 更难，原因是它有两个互相拉扯的目标（安全与通行），而且失败模式是“某一步突然不可行”，不像 PID 那样能靠看阶跃响应慢慢试。这一节给出一套固定顺序和一张症状对照表，目的是把试错变成核对。

79.8.1 固定顺序：从几何到时间，最后才是增益

顺序错了会反复返工，因为后面的参数依赖前面的结果。

1. **定 R。**按四项误差预算逐条列，写完就不再动。判据：R 的每一项都能在验收文档里找到出处。
2. **定 α 的窗口。**下限取 $2\omega_n$，上限用式 (79-17) 在最不利工况上核算。判据：窗口非空。
3. **定 T_s。**取 $T_s \le 0.1/\alpha$，并与算力预算对齐。判据：求解 P95 小于 $0.5T_s$。
4. **算 m。**按场景库最大接近速度算式 (79-25)。判据：$m/R < 20\%$。

5. **定 Q**。多通道时先归一化量纲，单通道取单位阵。判据：各通道修正量在同一量级。

6. **最后动名义增益**。改 k_p、k_d 会改变轨迹，从而改变前五步的结论。判据：改完重跑步骤 2–4。

第 6 步放在最后，是因为它最容易被当成“调性能”的手段。实际上在安全层已经定型的前提下，提高名义增益只会让约束更频繁地介入，净效果往往是“到达时间没怎么缩短，绕行反而更贴边”。

79.8.2 症状对照表

现场遇到的绝大多数问题可以归到六类，表 79-7 给出每一类的判别依据与处理方式。

表 79-7　安全滤波器的常见症状与对策

症状	最可能的原因	判别依据	处理
绕行过大、到达太慢	α 偏小	激活占比 > 20%	增大 α，重核式 (79-17)
出现不可行步	α 超上限或 T_s 过大	不可行集中在近障碍段	降 α，或减小 T_s
控制量在边界附近抖动	激活判据无滞后	u 在一个周期内反复跳变	加激活滞后或软化投影
停在障碍前不动	目标不可达或过度保守	速度归零且 h 远大于零	改目标点或加引导项
仿真安全、实车越界	时延与定位误差未计入	δ_{ctrl} 明显偏小	按 $v\tau$ 重算 R
求解耗时尖峰	二分次数或求解器收敛慢	P95 远大于中位	限迭代次数、预编译问题

六类里最隐蔽的是第五类。仿真里状态是“真值”，控制是“立即生效”，这两条假设在实车上都不成立。定位误差让 h 的计算偏乐观，执行延迟让实际轨迹比预期多走一段，两者叠加的效果恰好是“仿真里净距 20 cm、实车上擦到”。处理办法不是加个保险系数，而是把两项分别估出来、写进 R。

这张表要会用，得先知道每个症状“看哪个量”。表里的判别依据都是可以直接从日志算出来的：激活占比看“激活周期数除以总周期数”，抖动看“相邻两个周期控制量之差的符号反转次数”，不可行看“返回标志为假的步数”。不要凭轨迹图上的观感判断——轨迹图分辨率有限，一格可能就是一两个厘米，看上去“贴着”其实还有 10 cm 余量。

六类症状里有三类其实指向同一个根源：**边界附近的可行域太窄**。α 超上限、T_s 太大、执行器饱和，表现都是不可行或抖动。遇到这类问题，第一反应不该是继续调 α，而应该先算式 (79-17) 的余量，确认是不是根本性的能力不足。若余量已经小于 1，任何调参都只是在挪动失败的位置。

79.8.3 三条经验

第一，从保守端往激进端调，不要反过来。 初始取 R 偏大、α 取下限、T_s 取小值，先跑出“绝对安全但很笨”的行为，再逐步放开。反过来调（先激进再收紧）时，一旦出现不可行，很难判断是 α 太大还是 T_s 太大，因为两者的症状都是“某几步没解”。

第二，一次只改一个参数，并把激活占比当回归指标。 激活占比对 α、R、名义增益都敏感，是一个很好的“体检指标”：改完某个参数后它突然从 10% 跳到 30%，说明这次改动实质性地改变了闭环行为，需要重新看净距与到达时间，而不只是看“有没有越界”。

第三，验收要看曲线，不要看单点。 只报“最小净距 21.5 cm”没有意义，因为它是某一组参数下的一个点。要给出图 79-8 那样的权衡曲线，标出所选参数在曲线上的位置和两侧的余量。评审时对方会问“如果把速度提高 20% 会怎样”，有曲线才能回答。

这三条之外，还有一条关于记录的习惯：**每次调参都要留下“改前—改后—指标”的三行记录**。安全层的参数往往在项目后期被不同的人反复改动，没有记录就无法回答“这个 4 是谁定的、为什么不是 6”。记录里至少要包含五个数：α、T_s、R、激活占比、h_{min}。这五个数足够复现一次判断，也足够在出问题时定位到是哪次改动引入的。

79.8.4 验收清单

一份可签字的验收记录至少包含五项：

- 不可行步数为 0，或不可行全部触发回退且回退动作有记录；
- 全场景 $h_{min} > 0$，且 h_{min} 对应的净距不小于 $R - r_{obs}$；
- 约束激活占比在预期范围内（本章 10.0%），与设计文档一致；
- 求解耗时的 P95 小于半个控制周期；
- 无障碍工况下，滤波前后轨迹在数值精度内一致。

这五项分别对应安全性、几何余量、性能影响、实时性、实现正确性。缺任何一项，验收都不完整——尤其是最后一项，它最容易被跳过，却是唯一能证明“滤波器没有偷偷改变名义闭环”的证据。

清单里第 2 项的措辞需要解释一下。“h_{min} 对应的净距不小于 $R - r_{obs}$”看起来是同义反复，其实是要求：h_{min} 必须由基础半径 R 计算，而不是由含护卫裕量的半径计算。若用后者，h_{min} 恒为正，这一项就永远通过，验收形同虚设。这也是 79.12.4 节强调指标定义的原因：**指标定义错了，验收就是走过场**。

还有一项虽然不在清单里，但建议同时提供：**同一场景下“关掉滤波器”的对照曲线**。只报滤波后的结果，读的人无法判断这个场景本身有多危险。把两条曲线并排放，净距从 −10.0 cm 变成 +21.5 cm 这件事才有说服力。

79.8.5 把扫描表当成调参记录来读

表 79-3 的五行数据，换个顺序读就是一次完整的调参过程。第一次取 $\alpha = 1$，得到 65.1 cm 净距和 6.26 s 到达时间，激活占比 22.7%。按 79.8.2 的判别依据，“激活占比超过 20% 且净距远大于 $R - r_{obs}$”同时命中，属于典型的过度保守：系统从很远处就被拽住，全程五分之一的时间在解约束，还多花了 1.28 s。这一轮的问题不是不安全，而是把安全预算花在了没人要求的地方。

第二次改到 $\alpha = 8$。净距降到 16.1 cm，到达时间 4.83 s，激活占比 5.5%，三项都往好的方向走。此时如果只看“是否越界”，很容易就此定稿。但把 $h_{min} = +0.0122$ 换算成几何余量：h 对净距的灵敏度约为 $2R \approx 1.1$，也就是净距只剩约 11 mm 的余量。按 79.8.1 第 4 步的判据，m 在本算例为 $1.63 \times 0.01 + \tfrac{1}{2} \times 5 \times 0.01^2 \approx 1.7$ cm，已经吃掉了这点余量。结论是“能跑但不能再加任何不确定性”，于是退回中间值。

第三次取 $\alpha = 4$，得到 21.5 cm 净距、4.98 s 到达、10.0% 激活、$h_{min} = +0.0754$。按同一换算，几何余量约 6.9 cm，与 m 之比约 4，留出了容纳定位误差与扰动的空间。这一步定下来之后，第 2 步到第 4 步的核算全部重做一遍，结果与设计文档一致，才写进记录。

第四轮动的是名义增益。这一步放在最后不是形式主义：把 k_p 从 2.25 提到 9、k_d 从 3.0 提到 6，峰值速度从 1.63 m/s 升到 2.9 m/s，而式 (79-25) 的裕量按最大接近速度算，m 随之从 1.7 cm 涨到 2.9 cm。若控制周期还是 100 ms，m 会变成 0.30 m 量级，远超可用余量，仿真里立刻出现 3 步不可行。同一组增益在 10 ms 周期下却能跑通——这说明“增益能不能提”从来不是增益自己的问题，而是增益、周期、执行器三者共同决定的问题。

四轮下来，记录里留下的五个数是 $\alpha = 4$、$T_s = 0.01$ s、$R = 0.55$ m、激活占比 10.0%、$h_{min} = +0.0754$。五数之外还要写一句“为什么没选 $\alpha = 8$”，因为后来接手的人看到表 79-3 时，第一反应一定是“$\alpha = 8$ 更快，为什么不用”。没有这句话，这次调参就会被重做一遍，而且大概率重复同样的弯路。

调参到这里可以结束，但还有一类问题没有处理：模型不准、有外部扰动时，前面所有结论都会打折扣。下一节给出两条加固路径。

79.9 优化与改进

前面所有结论都建立在“模型准确、控制立即生效”之上。真实系统有风、有摩擦、有未建模的动态，这些都会让 h 的实际轨迹偏离预测。这一节先量化扰动下的失效，再给出鲁棒化处理，最后列出几条值得做的改进方向。

79.9.1 扰动下名义 CBF 会掉到负

设加速度通道上叠加一个有界扰动 δ，$\|\delta\| \le \Delta$，即

$$\dot{v} = u + \delta \tag{79-31}$$

h 的一阶导只含速度，不受 δ 直接影响；二阶导里出现 δ，于是约束的右端项要减掉一项。把 79.2 节的推导重做一遍，得到

$$A(x)u \;\le\; b(x) - 2\|d\|\,\Delta \tag{79-32}$$

其中 $d = p - p_{obs}$。式 (79-32) 的含义很直接：扰动可能在任何时刻把系统往障碍方向多推一点，因此约束必须提前收紧，收紧的量与“力臂” $\|d\|$ 和扰动上界 Δ 成正比。

这个 $2\|d\|\Delta$ 值得逐项对一遍，因为它是本章唯一一处“凭感觉会写错”的地方。把 ψ_2 写开：

$$\psi_2 = 2\|v\|^2 + 2d^{\mathsf{T}}u + 2\alpha_1 d^{\mathsf{T}}v + \alpha_2\psi_1 \tag{79-33}$$

扰动只在 $2d^{\mathsf{T}}u$ 这一项里进入——严格说，进入的是 $2d^{\mathsf{T}}(u+\delta)$。要保证最坏情况下 $\psi_2 \ge 0$，就要让

$$2d^{\mathsf{T}}u \ge -2\|v\|^2 - 2\alpha_1 d^{\mathsf{T}}v - \alpha_2\psi_1 + 2\|d\|\Delta \tag{79-34}$$

两边除以 -2 并整理，就得到式 (79-32)。$2\|d\|\Delta$ 里的因子 2 来自 ψ_2 中 $2d^{\mathsf{T}}u$ 的系数 2，不是凭空多出来的；$\|d\|$ 来自柯西–施瓦茨不等式 $|d^{\mathsf{T}}\delta| \le \|d\|\|\delta\|$。换句话说，**同样的加速度扰动，作用在离障碍越远的地方越危险**，因为力臂长、同样的横向加速度在同样的时间内造成的位移更大。这与人的直觉相反：多数人以为“离得远就不用管”，而在鲁棒约束里恰恰要管得更多。

Δ 怎么估，是这一节最需要工程判断的地方。它至少要覆盖三类来源。

第一类是**未建模的加速度**，例如摩擦、风阻、负载变化。这类量可以实测：让机器人在空载与满载下各跑一条相同的轨迹，取加速度指令与实际加速度之差的最大值。第二类是**执行器跟踪误差**，指令 3 m/s²、实际只有 2.6 m/s²，差值就是 0.4 m/s²。第三类是**离散化误差**，即 79.5 节的护卫裕量所覆盖的那部分——如果已经用护卫裕量单独处理了采样间隔，就不要在 Δ 里重复计一遍，否则安全边界会被双重收紧，性能白白损失。

三类误差不能简单相加，因为它们的峰值很少同时出现。工程上常用的做法是取“平方和开根”后再乘一个 1.2 左右的安全系数，或者直接取实测最大偏差的 1.5 倍。本章为了把效应看清楚，取 $\Delta = 0.8$ m/s²，相当于 a_{max} 的 16%，是一个偏保守但不算离谱的值。在 y 方向持续施加，结果见表 79-8 与图 79-9。

表 79-8　持续扰动下的名义 CBF 与鲁棒 CBF

方案	h_{min} / m²	最小净距 / cm
名义 CBF（未计扰动）	−0.0237	+12.8
鲁棒 CBF（式 79-32，$\Delta = 0.8$）	+0.0331	+17.9

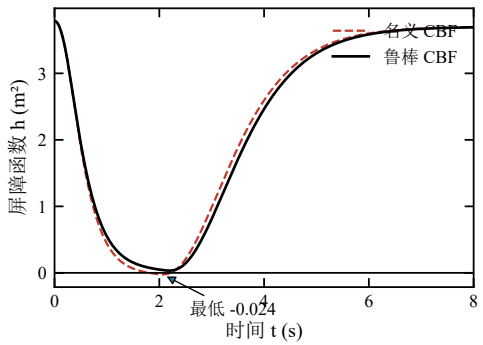


图 79-9　持续扰动下名义 CBF 与鲁棒 CBF 的屏障函数对比

名义 CBF 的 h_{min} 变成 −0.0237，屏障函数穿过了零。这里要分清两件事：$h=0$ 对应的是**含 0.15 m 裕量的安全边界**，不是障碍的物理表面。所以 $h<0$ 并不等于撞上，实际净距仍有 12.8 cm，只是那 0.15 m 的裕量被吃掉了 2.2 cm。把 $h<0$ 直接读成“碰撞”，是把工程裕量当成了物理尺寸——这是汇报时最常见的误读。

鲁棒 CBF 把 h_{min} 抬回 $+0.0331$，净距从 12.8 cm 回到 17.9 cm，代价是绕行略大。代价与 Δ 成正比：扰动上界估大一倍，约束就收紧一倍。**估计 Δ 时宁可偏保守一点**，因为它进入的是安全侧，而不是性能侧。

这句话要加一个限定。Δ 偏保守的代价并非总是温和的：式 (79-32) 的右端被压低的量是 $2\|d\|\Delta$，当 $\|d\|$ 很大时（例如刚起步、离障碍还有三米），这一项可以大到让右端变成负数，于是即便沿最有利的方向全力加速也无法满足约束，QP 直接判不可行。这是许多实现第一次跑鲁棒 CBF 就遇到的怪现象：**离得远反而报不可行**。解决的办法不是放弃鲁棒性，而是给收紧项加一个上限，例如把 $2\|d\|\Delta$ 截断为 $2\min(\|d\|, R+0.5)\Delta$——离得足够远时，即使扰动把系统往障碍方向推，也推不到边界上，没必要再收紧。

这也解释了为什么实际系统很少用“全局常数 Δ”：更常见的做法是让 Δ 随工况变化，低速时小、高速时大，或者直接对扰动做在线估计，用估计值加一个置信上界。状态观测器、扰动观测器在这里都能派上用场，本章不展开。

79.9.2 更紧的屏障函数

本章用的是距离平方 $h=\|d\|^2-R^2$。它的梯度 $\nabla h=2d$ 在远处大、近处小，也就是说“离得远时修正力度大、离得近时反而软”，与实际需要正好相反。这是平方形式在接近边界时显得迟钝的原因。

把这句话说得更准确一点：∇h 的大小是 $2\|d\|$，在边界上等于 $2R$，并不是零。所谓“变软”是相对于距离尺度而言——同样 1 cm 的越界倾向，在 $R=0.55$ m 处产生的 h 变化约为 $2\times0.55\times0.01\approx0.011$ m²，而在远处 $d=3$ m 处则是 0.06 m²。归一化之后，边界附近的“信号强度”只有远处的六分之一。控制器看到的不是距离，而是 h，于是它对近处的危险不够敏感。

改用带符号距离

$$h_s=\|d\|-R,\qquad \|\nabla h_s\|=1 \tag{79-35}$$

梯度大小恒为 1，修正力度与距离无关，接近边界时不会变软。代价是 h_s 在 $d=0$ 处不可导，工程上要用 $\sqrt{\|d\|^2+\epsilon^2}-R$ 之类的平滑版本；同时一阶、二阶导的表达式比平方形式复杂，计算量略增。

把 h_s 的二阶条件写出来，可以看清“复杂”到底复杂在哪里。记 $n=d/\|d\|$ 为单位法向，则

$$\dot h_s=n^{\mathsf T}v,\qquad \ddot h_s=\frac{v^{\mathsf T}v-(n^{\mathsf T}v)^2}{\|d\|}+n^{\mathsf T}u \tag{79-36}$$

第一项是切向速度产生的“离心”项，恒非负；第二项是法向加速度。于是二阶 CBF 条件变成

$$n^{\mathsf T}u\ge-\frac{\|v\|^2-(n^{\mathsf T}v)^2}{\|d\|}-\alpha_1 n^{\mathsf T}v-\alpha_2\dot h_s-\alpha_1\alpha_2 h_s \tag{79-37}$$

对比平方形式的式 (79-14)，两者的结构完全一样，区别只在系数：平方形式的约束行向量是 $-2d$，长度随距离增长；带符号距离的约束行向量是 $-n$，长度恒为 1。**行向量长度恒定意味着约束的“绝对严格程度”不随距离变化**，这正是我们想要的。

平滑参数 ϵ 怎么取，有一个可以直接用的判据：让平滑引入的距离误差远小于工程裕量。$\sqrt{\|d\|^2+\epsilon^2}\approx\|d\|+\epsilon^2/(2\|d\|)$，在 $\|d\|\approx R$ 处误差约为 $\epsilon^2/(2R)$。要求它小于裕量的十分之一，即 $\epsilon^2/(2R)\le0.1m$，代入 $R=0.55$、$m=0.15$ 得 $\epsilon\le0.13$。取 $\epsilon=0.05$ 时误差约 2 mm，完全可忽略。ϵ 太大会让边界“发糊”，太小则数值上仍然接近不可导，ϵ 在 $R/10$ 量级是个稳妥的起点。

对本章这种“必须精确停在边界附近”的场合，带符号距离的收益明显；对“只要能绕开就行”的场合，平方形式足够，且数值更稳。

还有一处容易被忽略的差别：平方形式的 h 在远处增长很快，若把 h 直接当作“安全裕度”做显示或记录，量纲是 m²，读起来不直观；带符号距离的 h_s 量纲就是米，可以直接读成“还差多少厘米碰到安全边界”。对现场调试和事故复盘，这一点比理论上的紧致性更实用。

79.9.3 其他值得做的改进

多障碍与动态障碍。 每个障碍对应一行约束，QP 从单约束变成 n 行。这时闭式二分不再适用，需要 active-set 或小规模 QP 求解器；但 79.10 节的实测表明，约束数从 1 增到 64，求解时间几乎不变（0.771 ms 到 0.784 ms），所以规模不是瓶颈。

真正的难点不在求解，而在**约束之间的相容性**。两个障碍靠得很近时，两条约束可能要求控制器同时向左和向右让，交集为空，QP 无解。这时要区分两种情况：一种是几何上确实无路可走（通道比机身还窄），任何算法都救不了，只能靠上游的路径规划避开；另一种是两条约束各自留了过大的裕量，把本来存在的通道挤没了。第二种情况可以通过“按障碍距离分配裕量”缓解——离得近的障碍用完整裕量，离得远的用小裕量，避免远处的障碍无谓地收缩可行域。动态障碍还多一层问题：约束随预测位置移动，若预测有延迟，等价的扰动就是 $v_{obs}\Delta t$，应当并入 79.9.1 节的 Δ 一起处理，而不是另开一套逻辑。

软化约束。 把硬约束 $Au \le b$ 改成 $Au \le b + s$、对 s 加二次罚，可以在确实无解时给出“违反最少”的解，而不是直接判不可行。代价是安全性从“硬保证”降为“软惩罚”，只在允许极小概率越界的场合使用。

软化后的 QP 是

$$\min_{u,s} \ \tfrac{1}{2}\|u-u_{nom}\|^2 + \tfrac{\rho}{2}s^2 \quad \text{s.t.} \quad Au \le b + s,\ s \ge 0 \tag{79-38}$$

罚权重 ρ 的取法有一个明确的下界：当约束被违反时，ρ 必须大到让“再多违反一点”的代价超过“偏离名义控制一点”的收益，否则优化器会主动选择越界。粗略地说，若名义代价的量级是 a_{max}^2，ρ 至少要取它的 10^2 到 10^3 倍。ρ 太大又会让问题病态，条件数随 ρ 线性恶化，所以软化约束在实践中往往配合“分级”使用：正常时 ρ 很大，接近病态时自动降低，并把降级事件记入日志。

必须强调，软化只应作为**降级通道**，不应作为常态。一个长期运行在软化模式下的系统，实际上已经失去了安全保证，只是把碰撞推迟了。正确的用法是：硬约束优先；一旦软化量 s 超过阈值，立即切回退策略（减速、悬停或交给司机），而不是继续靠罚函数“尽量不撞”。

时延补偿。 若测量到执行有 τ 的延迟，可以用当前状态外推 τ 后的状态再算约束，等效于把 T_s 的一部分提前处理掉。外推模型不准时，误差会直接进约束，因此要和护卫裕量配合使用。

一阶外推最常用：$\hat{p} = p + v\tau + \tfrac{1}{2}u\tau^2$，$\hat{v} = v + u\tau$。对 $\tau = 20$ ms、$v = 2$ m/s 的工况，位置修正约 4 cm，速度修正约 0.1 m/s，量级与裕量相当，不做补偿就会实打实地吃掉裕量。补偿的边界也很清楚：τ 超过控制周期的两三倍时，一阶外推的误差已经和补偿量同量级，此时应改用带预测的模型（例如第 80 章的 MPC 滚动优化），而不是继续加大外推阶数。

学习型屏障函数。 当安全集无法用简单几何描述（例如整机的工作空间受姿态影响）时，可以用数据拟合 h，再用 79.2 节的框架做在线滤波。这条路的前景很大，但 h 的可解释性与验证方法仍是开放问题，本书不展开。

与预测控制结合。 把 CBF 约束直接加进 MPC 的优化问题，可以让“提前减速”与“安全约束”在同一个优化里协调，避免安全层在最后一刻才介入所造成的急刹。这是第 80 章的主题，这里只指出一个衔接点：MPC 的预测时域越长，CBF 约束越能提前起作用，但约束行数也按时域线性增长，实时性压力回到 79.10 节讨论的那条线上。

分布式多机场景。 多台设备共享工作空间时，两两之间的避让约束数量随台数平方增长，集中求解不可行。常见做法是把每台设备的安全约束写成只依赖邻居状态的局部形式，各自求解、异步更新。此时“同时更新”的假设不再成立，鲁棒项要按邻居一个周期内的最大位移来收紧，收敛性分析也比单机复杂得多。

执行器动态与饱和。 本章把 u 当成能瞬间实现的加速度，真实执行器有一阶或二阶动态，还会在接近极限时进入饱和。这两件事都会让“约束在指令层满足”变成“约束在物理层不满足”。处理办法有两种：一是把执行器动态纳入模型，把 h 对指令的相对阶再抬高一层，代价是约束里出现 u 的变化率，需要知道执行器的带宽；二是保留二阶模型，但把 a_{max} 按“实际能达到的加速度”打折（例如取标称值的 80%），并把折扣写进文档。后者更简单，也更常用。要注意的是，饱和不是“限

制幅度”这么简单：一个已经饱和的执行器无法再提供任何修正，此时 CBF 的可行性判据必须用**实际可用**的 a_{max} 去算，而不是标称值。

状态估计误差的显式处理。 若状态来自观测器，h 的计算就带误差。一种保守做法是把估计误差的上界折算成 R 的增量（这也是本章的做法）；更精细的做法是在约束里显式引入误差项，形式与鲁棒项类似，但收紧量取决于误差椭球在法向上的投影。后者在误差各向异性明显时收益较大，代价是需要在线维护误差协方差。

把这些改进按“投入产出比”排个序，实用顺序大致是：时延补偿（改动小、收益直接）→ 多障碍的裕量分配（解决实际无解）→ 软化降级通道（提高可用性）→ 带符号距离（提升边界附近的精度）→ 学习型 h（研究阶段）。鲁棒项要不要上，取决于扰动是否可测：可测就用观测器，不可测才用常数上界。

79.10 工程落地

理论正确不等于能装到机器上。工程落地要回答四个问题：算得过来吗、跑在什么平台上、怎么和原有控制器接、出了故障谁来兜底。这一节按这四个问题展开，所有耗时数字来自 `code/ch079_sim.py` 的 `timing_benchmark`，测量环境是普通 PC（Python 3.11 + cvxpy 1.7.5 + OSQP），不是目标机。

79.10.1 实时性：先看分布，不要只看平均

用 cvxpy+OSQP 求解本章的 QP，问题预编译、热启动，约束数从 1 增加到 64，结果见表 79-9 与图 79-10。

表 79-9　QP 求解耗时（cvxpy+OSQP，问题预编译+热启动）

约束数	中位 / ms	P95 / ms	首次（含建模）/ ms
1	0.771	1.386	9.0
4	0.760	0.890	6.2
16	0.765	1.026	4.7
64	0.784	0.924	5.2

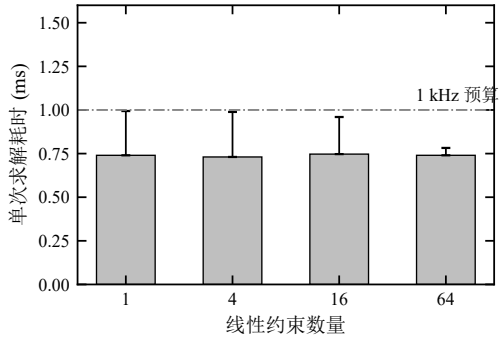


图 79-10　QP 求解耗时与 1 kHz 控制周期预算

这张表有三条信息，每一条都影响架构决策。

第一，约束数几乎不影响耗时。 从 1 行到 64 行，中位只从 0.771 ms 变到 0.784 ms（+1.7%）。原因是耗时被固定开销主导：Python 调用、cvxpy 的规范化、稀疏矩阵组装。**这意味着多障碍场景不需要为“约束变多”重新设计算法**，瓶颈不在问题规模上。

第二，中位与 P95 差得多。 单约束时中位 0.771 ms、P95 1.386 ms，后者是前者的 1.8 倍。实时系统的截止时间由最坏情况决定，不由中位决定。以 1 kHz（周期 1 ms）为例，中位占 77%，P95 已经超出周期 39%——**这意味着每二十个周期就有一次赶不上**。用中位数论证“能跑 1 kHz”是自欺欺人。

第三，首次调用慢一个数量级。 9.0 ms 包含建模与编译，是稳态的 12 倍。这段开销必须放在初始化阶段，用一次“空跑”把它吃掉；若留到第一个控制周期，首次接近障碍时必然超时。

结论很直接：cvxpy+OSQP 适合原型与离线验证，不适合 1 kHz 的在线回路。在线要用 79.4 节的闭式投影（中位 8.2 μs），或者把 OSQP 的 C 版本嵌进目标机。前者的代价是不支持多约束，后者的代价是需要自己做代码生成与内存管理。

这张表的测法也要说明，否则数字不可比。`timing_benchmark` 的做法是：先把问题构造好、求解一次做预热，再连续调用 n 次并逐次计时，最后取中位与 P95。之所以要预热，是因为第一次调用包含 CVXPY 的问题规范化与编译，它的量级是毫秒到十毫秒，混进样本里会把 P95 拉高好几倍。之所以用 P95 而不是最大值，是因为单次最大值几乎总是被操作系统调度打断，重复测也不稳定；P95 已经是“每二十个周期有一次”的频率，对 1 kHz 来说足够说明问题。

还有两个容易被忽略的抖动源。一是 **Python 的垃圾回收**，即使代码本身不分配大对象，解释器仍可能在任何时候触发一次回收，造成亚毫秒级的停顿。二是 **CPU 频率调节**，笔记本在空载时会降频，跑基准时又升频，同一段代码前后两次的耗时能差 30%。前者可以用预分配和 `gc.disable()` 缓解（原型阶段够用），后者必须在固定频率、固定负载下测才有意义。

把 1 kHz 的预算拆开看，能更清楚地判断瓶颈在哪。一个周期 1000 μs，其中传感采集与状态估计约 100 μs、名义控制器约 50 μs、日志与通信约 100 μs，留给安全层的是 750 μs 左右。按 P95 1.386 ms 计算，安全层一项就超支了近一倍——所以不是“优化一下就能塞进去”，而是必须换实现。换成闭式投影后 P95 只有 254.6 μs，占预算的三分之一，剩下的余量可以用来降周期或加障碍。**先算预算，再选实现**，比先选实现再想办法挤进去要省事得多。

79.10.2 目标机上的实现选择

按约束规模分三档。

单约束、单障碍。 用闭式投影加二分，C 实现约 50 行，单次 1 μs 量级，无需任何库。这是最稳的选择：没有动态内存分配、没有迭代收敛判据、最坏执行时间可以静态估出。

少量约束（2–10 个障碍）。 用 active-set 或小规模 QP，也可以把多约束逐个投影再取“最严格”的那个——注意后者不保证最优，但保证可行，工程上常作为降级方案。

大量约束或时变约束。 用嵌入式 QP 求解器（OSQP、HPIPM 等），必须做代码生成、固定迭代次数、关闭动态分配。此时最坏执行时间要靠“固定迭代上限”来保证，而不是靠“通常收敛很快”。

无论哪一档，有三条硬要求：控制周期内不做内存分配；所有矩阵预分配、预分解；求解器设迭代上限与超时，超时走回退。第三条尤其重要——一个不设上限的迭代求解器，在数值病态时可能卡住几十毫秒，这比“解不出”更危险。

单约束的 C 实现，最坏执行时间可以算出来：固定 40 次二分，每次约 20 条浮点指令，总计不到 1000 条指令，在 100 MHz 的 MCU 上约 10 μs，在 1 GHz 的应用处理器上不到 1 μs。这个数字是**可以写在文档里、可以拿去评审**的，因为它不依赖输入数据。相比之下，迭代求解器的耗时依赖问题条件数，即使设了上限，也只能给出“不超过 N 次迭代”的上界，实际耗时的分布仍然很宽。

79.10.3 与名义控制器的集成

结构上，安全层插在名义控制器与执行器之间（图 79-2）。集成时有六个细节必须明确。

时间戳一致。 名义控制器用的状态、安全层用的状态、障碍位置，必须是同一时刻的。若障碍位置来自感知、延迟 50 ms，以 2 m/s 相对速度计算就是 10 cm 的误差——直接吃掉大半裕量。做不到同步时，把这段延迟按 $v_{rel}\Delta t$ 计入 R。

三种状态要显式定义。 正常（约束未激活）、滤波（约束激活，输出投影解）、回退（不可行或超时）。三态之间的转换条件、每种状态下的输出、状态切换时是否要重置积分器，都要写进接口文档。最常见的实现错误是把“回退”当成“滤波”的一个分支，结果回退时仍然输出一个未经证明安全的解。

积分器与滤波器要解耦。 名义控制器里的积分项在约束激活期间会继续累积（因为它看到的是误差，不是实际执行量），退出约束后可能出现积分饱和引起的过冲。标准做法是抗积分饱和：约束激活时冻结或回退积分项。

参数在线可改要加保护。 R、α、a_{max} 一旦在线改动，原来的验证结论全部失效。工程上要么禁止在线改，要么把改动限制在“更保守”的方向（增大 R、减小 α），并在日志里记录。

不要在安全层之后再加通用限幅或滤波。 这是一个反直觉的要求：很多人看到约束激活时控制量跳变，第一反应是加一个变化率限制让它“平滑”。但对安全层来说，平滑恰恰是危险的——投影解之所以安全，是因为它精确满足 $Au \le b$，任何事后修改都可能破坏这个不等式。若确实需要抑制跳变，正确的位置在**约束之前**：把 b 的右端加上一段与 $\dot{u}$ 有关的余量，让投影解本身就变化得缓一些。改动的位置差一步，安全性就从“有保证”变成“碰运气”。

量纲与坐标系必须在接口处对齐。 安全层的约束是 $Au \le b$，A 的行方向由 ∇h 决定，u 的每一维必须与 A 的每一列一一对应。若名义控制器输出的是关节力矩、而 A 是按笛卡尔加速度推的，约束就成了两个不同空间里的向量内积，数值上不会报错，物理上完全无意义。同理，Q 的对角元要按量纲归一化：位置通道的 1 N 与姿态通道的 1 N·m 不是同一回事，直接取单位阵会让姿态通道的修正量被淹没。

79.10.4 运行时监控

安全层必须能自证在工作。至少记录四个量，每个周期更新、每秒汇总：

- **约束激活占比**：长期为 0 说明场景太简单或 R 太大；长期高于 50% 说明参数过保守或任务过急。
- **h 的最小值**：按滑动窗口统计。它应当始终大于 0，且不应长期贴近 0。
- **求解耗时**：分别记中位与 P99，P99 超周期时报警。
- **不可行与回退次数**：任何一次非零都必须可追溯，附当时的完整状态。

这四个量同时也是验收证据。没有它们，“系统运行正常”只是一句无法核实的话。

除这四个量，还有三项建议记录。**状态新鲜度**：从传感器采样到安全层使用的时间差，超过两个周期就应标记，因为它直接影响 R 的有效性。**参数指纹**：把 R、α、a_{max} 等关键参数算一个校验和，随日志一起存，事后可以确认“跑的是不是验证过的那套参数”。**连续激活时长**：单次激活持续了多少个周期，比总占比更能反映“是不是在边界上磨蹭”。

监控数据必须能落盘，而且要能回溯到具体时刻。一个只有计数、没有时间戳的实现，在出问题后无法与外部数据（摄像头、总线记录）对齐，等于白记。

79.10.5 文档与验证

安全层的交付物不只是代码，还包括一份能支撑评审的安全案例。最少包含五部分：安全集的定义与 R 的四项预算；α、T_s、m 的取值依据与核算过程；场景库清单与每个场景的 h_{min}、激活占比；故障注入的结果（传感器跳变、求解超时、执行器饱和）；回退策略的动作定义与验证记录。

这五部分之间要有明确的引用关系，而不是各自独立成文。R 的每一项应当能追到一次实测；α 的取值应当能追到式 (79-17) 的一次核算；每个场景的 h_{min} 应当能追到一次具体的运行记录。**证据链断了，安全案例就只剩下结论**，而结论恰恰是最不可信的部分。

功能安全标准普遍要求安全相关功能独立于任务功能验证，这正是 79.1 节强调分层的另一个理由：安全层可以被单独拿出来，用独立的场景库、独立的评审人、独立的时间窗口去验证，而不必等整个任务系统定型。

工程落地这一段可以用一句话概括：**算法只占工作量的三成，剩下七成在时序、接口、监控和文档上**。前面九节讨论的是那三成，这一节讨论的是另外七成——少了它，再漂亮的 CBF 也上不了车。

79.11 优缺点与适用边界

79.11.1 优点

保证是硬的。 只要约束可行、初始状态在安全集内，$h \ge 0$ 就是前向不变的，不依赖调参运气。这一点与势场法、罚函数法有本质区别：后者的安全性是“权重够大时通常成立”，前者的安全性是“约束满足则必然成立”。

计算量与状态维数的关系弱。 表 79-9 显示约束数从 1 到 64 耗时几乎不变，真正的瓶颈是固定开销。这意味着增加障碍数量不会让算法崩溃，工程上可扩展性较好。

分层带来性能隔离。 约束不激活时滤波器是恒等映射，本章 90% 的时间如此。原有控制器的调参成果不被破坏，这是它能被工业界接受的关键。

可解释、可验收。 h、R、α 都有明确的物理含义，每个参数都能对应到工程量或误差预算，便于写进验收文档、便于第三方复核。

失效模式可枚举。 滤波器的输出只有三种可能：原样放行、投影到约束面、判不可行。三种情况都能在日志里留下痕迹，事故复盘时可以逐周期回放“当时为什么这么决策”。相比之下，一个端到端学习出来的策略，即使性能更好，也很难回答“为什么那一刻没有刹车”。

可以叠加，不必替换。 安全滤波器串在现有控制器之后，输入端只要一个名义控制量、输出端只给一个修正量，接口是一对向量。这意味着它既能加在 PD 后面，也能加在 MPC、强化学习策略、甚至人工遥控之后，而且可以只在特定工况启用。工程改造中，“加一层”比“换一套”容易推动得多，因为原有系统的验收材料大多还能复用。

约束可以逐条增删。 一条约束对应一行 a、一个标量 b，加一个障碍不改动求解器，只改矩阵的行数。这一点在需求变化频繁的项目里很实用：前期只加最危险的几个障碍，后期按事故案例逐步补，每补一条都能独立验证，不会牵动已有逻辑。

79.11.2 缺点

只保证安全，不保证到达。 CBF 不处理活性。目标点若在障碍正后方且绕行空间不足，系统会停在障碍前——$h > 0$、速度为零、任务失败。这类死锁必须靠上层引导或目标重规划解决，不能指望滤波器自己走出来。

依赖模型与状态质量。 79.9 节的算例里，0.8 m/s² 的持续扰动就让名义 CBF 的 h_{min} 掉到 −0.0237。定位误差、感知延迟、未建模动态都会以同样方式侵入，而且它们不像扰动那样有明确的界。

保守性有实际代价。 R 与护卫裕量都要从可用空间里扣。100 ms 周期下裕量 0.312 m，占 R 的 57%，绕行距离显著变大。在狭窄通道、密集货架这类场景里，保守性可能直接导致“无解”。

多约束时闭式解不再最优。 本章的闭式投影只对一个半空间精确。障碍多于一个时，逐行投影取最严格者虽然保证可行，但得到的解不是最小修正，可能出现“为了避 A 而撞向 B 的方向被放大”这种别扭的绕行，路径比最优解长一截。这个代价随约束数增长，是单约束实现扩展到多障碍时最容易低估的一项。

参数存在可用窗口，窗口外会失效。 $\alpha = 16$ 时 269 步不可行，说明“调大 α 更安全”的直觉是错的。窗口由名义带宽与执行器能力共同决定，必须逐工况核算（式 79-29）。

高阶系统要付额外代价。 相对阶每升一阶，就多一层 ψ_i 和一个 α_i，约束形式变复杂、保守性变强、对模型精度的要求更高。本章的双积分器是最简单的情形，机械臂、无人机姿态动力学要比这难得多。

对安全集的描述能力有限。 CBF 要求 h 是一个可微、零超水平集恰好等于安全集的函数。圆、半平面、椭球都好办；一旦安全集是非凸的、带孔洞的、或者由若干区域的并集构成，构造 h 就变成一件需要技巧甚至运气的事。用多个 h_i 取最小值是一种办法，但最小值函数不可微，实际计算要小心处理。

约束切换本身会引入抖动。 约束激活的那一刻，控制量从 u_{nom} 跳到投影点，跳变幅度可以很大。若没有滞后或平滑，系统会在约束面附近“进去—出来—再进去”，表现为执行器的高频换向。这个问题不是理论缺陷，而是实现缺陷，但它在实车上比在仿真里严重得多，因为真实执行器有间隙和摩擦。

79.11.3 适用边界

适合用：相对阶 1–2；状态可测或估计精度足够；安全集能用简单的几何函数描述（圆、半平面、凸多面体）；执行器能力相对任务需求有 30% 以上余量；控制周期能满足 $T_s \le 0.1/\alpha$。

谨慎用：相对阶 3 及以上（要高阶 CBF，参数多、保守）；安全集非凸或高维（h 难构造）；动态障碍密集（需要预测与多约束，闭式解失效）；执行器接近饱和（约束容易不可行）；需要“最优”而非“可行”的场合（CBF 只做最小修正，不做全局优化）。

不适合用：状态不可测且无法估计；安全集本身不确定（例如障碍位置只有概率描述）；对计算资源有极端限制且无法预计算。

判断“能不能用”，有一个五分钟就能做完的检查：把最危险工况的速度、距离、加速度需求代进式 (79-17)，看所需加速度是否小于 a_{max} 的一半。若答案是否定的，先别急着调参，说明这个任务对这台机器本来就不可行，CBF 只是把这个事实暴露出来。若答案是肯定的，再去看安全集能否用一个光滑函数描述——这两条都过，剩下的才是参数问题。

这个检查的顺序不能颠倒。先查可行性、再查 h 的构造，是因为两者的失败代价不同：可行性不足是任务级的问题，换方法也救不回来；h 不好构造是表述级的问题，换一种描述方式（比如用多个凸集取最小、或者把非凸区域外包成凸包）往往还能解决。把表述级的问题误判成任务级的问题，会导致“明明能做的任务被放弃”。

三类情况值得直接换方法。第一，安全集本身带概率性——障碍位置不是一个确定值，而是一个分布，这时硬约束没有意义，要么先把分布折算成膨胀后的确定性包围盒（保守但可行），要么改用机会约束或风险度量，后者已经超出本章的范围。第二，相对阶高到 3 以上且执行器带宽有限——每升一阶就要多微分一次，噪声会被放大，此时用 MPC 把约束写进预测模型通常更省事。第三，需要同时优化多个目标——CBF 只做最小修正，它不会为了省 5% 的能量而改变绕行方向，这类需求应该交给优化层。

反过来，也有两类情况不该换方法，而该改任务。一是执行器余量长期低于 30%——这不是算法保守，是机器选小了；二是感知刷新率低于控制频率的一半——滤波器每个周期都在用旧状态算约束，再好的算法也只能在这个延迟上打折。这两种情况在评审时最容易被误判成“算法不行”，实际是系统集成的问题。

79.11.4 与其他方法的分工

CBF 与 MPC 不是替代关系。MPC 用预测模型做多步优化，能同时处理性能与约束，但计算量大、需要可靠的预测；CBF 只做一步最小修正，计算量小、保证硬。工程上常见的组合是“MPC 出 u_{nom}，CBF 兜底”：MPC 负责跑得好，CBF 负责不出事，两者的失效模式不重叠。

与可达性分析的分工在 79.2.7 节已经说过：离线算可控不变集确定“哪些初始状态还有救”，在线用 CBF 做逐时刻执行。前者回答“能不能”，后者回答“怎么做”。

与势场法相比，CBF 的优势是保证，劣势是需要模型；势场法不需要模型，但在凹形障碍前会陷入局部极小，且无法给出任何保证。

表 79-10　四类安全手段的对照

方法	保证强度	计算量	对模型的要求	典型失效
CBF-QP	硬（可行即保证）	小（单约束闭式）	需要 $\dot{x} = f + gu$	不可行、死锁
MPC（含约束）	硬（在预测时域内）	大（在线优化）	需要较准的预测模型	超时、预测偏差
势场法	无	极小	不需要	局部极小、穿越
可达性分析	硬（离线全集）	离线极大、在线极小	需要完整模型与扰动界	维数灾难、保守

这张表最该看的是最后一列。“保证强度”一栏里三种方法都写着“硬”，但它们的失效方式完全不同：CBF 失效于不可行，MPC 失效于算不完，可达性失效于算不出来。选型时先问“我最怕哪种失效”，答案往往比比较精度更有用。

一个实际的组合建议：用可达性分析离线确定安全的工作包线，用 MPC 在包线内跑任务，用 CBF 做最后一道在线校验。三层各管一段，互不替代。本章讲的是最里面那一层，它最便宜、最容易验证，也最容易被单独审计。

三层的接口要提前定好，否则组合起来会比单层更难维护。包线以“状态—动作对”的形式交给 MPC，MPC 以名义轨迹或名义控制量的形式交给 CBF，CBF 只回传“是否已修正”与“是否不可行”两个标志。**越靠内层，接口越窄**，这是让它可被独立

验证的前提。

79.12 Python 仿真代码

本章的全部数字与插图由两个脚本产生，都不依赖第三方优化库的核心功能（cvxpy 只用于 79.10 节的对照实验）：

```
code/ch079_sim.py     仿真与指标，正文所有数字的唯一来源
code/ch079_figs.py    出图，调用 ch079_sim 的函数并排版
```

运行方式与产物：

```
D:\Anaconda3\envs\py311\python.exe code/ch079_sim.py    # 打印表 79-2 至 79-9 的全部数据
D:\Anaconda3\envs\py311\python.exe code/ch079_figs.py   # 生成 figs/ch079/ 下 10 图 × (svg, pdf)
```

依赖只有 numpy、matplotlib、cvxpy。仿真主循环用 RK4 积分、1 ms 步长，控制周期可独立设为 1–100 ms，两条时间尺度分开是为了让 79.5 节的离散效应能被真实地观测到，而不是被积分步长掩盖。

79.12.1 代码结构与正文的对应

表 79-11　`ch079_sim.py` 的函数与正文对应关系

函数	作用	对应正文	产物
`nominal_u`	PD 名义控制器，各轴限幅	79.1 节式 (79-1)	—
`safety_filter`	单约束 QP 的闭式投影	79.4 节式 (79-21)–(79-23)	表 79-2
`run_2d`	二维避障主循环（ZOH + 扰动）	79.4–79.9 节	表 79-2、79-3、79-5、79-8
`run_brake`	一维紧急制动	79.4 节式 (79-19)(79-20)	表 79-4
`timing_benchmark`	cvxpy+OSQP 耗时测量	79.10 节	表 79-9
`main`	依次运行上述实验并打印	全章	正文全部数字

`main` 的编排顺序不是随意的，它按“先基准、再扫描、最后鲁棒与耗时”排，因为后面的实验要引用前面的结论。第一步跑 79.4 节的对比，得到表 79-2 的八项指标；第二步固定其余参数、扫 α，得到表 79-3；第三步跑一维制动，得到表 79-4；第四步按 $T_s = 1/10/50/100$ ms 扫离散化，得到表 79-5；第五步在名义工况上叠加速度扰动，得到表 79-8；第六步才做耗时测量。每一步都重新构造 `cfg`，不共享可变状态，所以单独调用任何一步都不会被前一步污染。

复现条件要写清楚，否则表里的数字没有意义。环境是 Python 3.11 + numpy 2.x + matplotlib 3.x + cvxpy 1.7.5（OSQP 后端），普通桌面 PC，非实时系统。仿真本身是确定性的，没有随机数，两次运行的轨迹逐位相同；唯一不可复现的是表 79-9 的耗时，它会随机器负载波动 10%–20%，所以正文引用的是量级与分布形状，不是精确值。若你的机器上中位数是 0.6 ms 而不是 0.77 ms，结论不变；若 P95 与中位的比值明显小于 1.8，多半是因为机器更空闲，建议在负载下重测一次再下结论。

79.12.2 核心：约束装配与闭式投影

下面这段是 `safety_filter` 的主体，直接对应式 (79-21) 到 (79-23)。删去了注释与指标统计，保留全部计算逻辑。

```
d = p - np.asarray(cfg["c_obs"], float)          # 相对障碍的位移
R = r_safe(cfg, extra_margin)                    # 安全半径（含护卫裕量）
h = float(d @ d) - R * R                         # 屏障函数
psi1 = 2.0 * float(d @ v) + a1 * h               # 一阶辅助变量
b = 2.0 * float(v @ v) + 2.0 * a1 * float(d @ v) + a2 * psi1
b -= 2.0 * float(np.linalg.norm(d)) * dist_bound # 鲁棒项，式 (79-32)
a = -2.0 * d                                     # 约束行 A = -2 d^T

u = np.clip(u_nom, -am, am)
if float(a @ u) <= b:                            # 名义控制本来就安全
    return u, h, False, True

def f(lam):                                      # u(lam) = clip(u - lam*a)
    return float(a @ np.clip(u - lam * a, -am, am)) - b

lo, hi = 0.0, 1e-6
while f(hi) > 0.0 and hi < 1e6:                  # 倍增找变号区间
    hi *= 2.0
if f(hi) > 0.0:                                  # 执行器能力不足
    dirv = d / max(float(np.linalg.norm(d)), 1e-9)
    return am * dirv, h, True, False             # 回退：沿 d 全力推开

for _ in range(60):                              # 二分到双精度极限
    mid = 0.5 * (lo + hi)
    if f(mid) > 0.0:
        lo = mid
    else:
        hi = mid
return np.clip(u - hi * a, -am, am), h, True, True
```

四个返回值的含义要在调用处区分清楚：控制量、当前 h、约束是否激活、是否可行。指标统计只认后两个，正文里“激活占比 10.0%”“不可行 269 步”分别来自它们。

读这段代码时注意两个细节。第一，`a = -2d` 是行向量，`a @ u` 就是 Au；写成 `d @ u` 再乘 -2 也一样，这里保留 `a` 是为了和式 (79-21) 对齐。第二，二分用的不是 $a\cdot u(\lambda)=b$ 的解析解，而是 `f(lam)` 的符号——因为 `clip` 让 $u(\lambda)$ 分段线性，解析解要分三段讨论，符号法不需要。这是“用十行代码换一个分段讨论”的典型取舍。

还有一处容易读错：代码里 b 的计算顺序与式 (79-14) 的右端完全一致，但多了一步 `b -= 2||d||*dist_bound`。这一步**只能加在这里**，不能加到左端或 h 里。若把它折算成“把 R 加大一点”，效果并不等价：鲁棒项与 $\|d\|$ 成正比，而加大 R 是常数偏移，两者在远处和近处的相对大小完全不同。79.9 节已经说过，误把它并进 R 会让远处的约束被无谓收紧，近处却收紧不足。

最后，`u = np.clip(u_nom, -am, am)` 这一行不是可选的。它保证在进入投影之前名义控制已经落在盒内，否则 `f(0)` 的符号判断会失去意义——若 u_{nom} 本身超出盒约束，“名义控制是否安全”这个问题就没有定义。

79.12.3 仿真主循环

主循环的关键是两条时间尺度的分离：积分步长 `dt` 固定 1 ms，控制周期 `Ts_ctrl` 由 `hold` 控制，控制量在 `hold` 个积分步内保持不变。

```
hold = max(1, int(round(Ts_ctrl / dt)))
u_hold = np.zeros(2)
for k in range(N):
    p, v = x[:2], x[2:]
    if k % hold == 0:
        u_nom = nominal_u(p, v, cfg)
        if use_filter:
            u_hold, h, act, ok = safety_filter(u_nom, p, v, cfg,
                                               alpha1=alpha, alpha2=alpha,
                                               extra_margin=extra_margin,
                                               dist_bound=dist_bound)
        else:
            u_hold, h = u_nom, np.nan
    P[k], V[k], U[k] = p, v, u_hold
    H[k] = h
    x = _rk4(x + np.array([0, 0, *dist_vec]) * dt, u_hold, dt)
```

最后一行同时做了两件事：施加扰动（加速度通道加 `dist_vec`）和用 RK4 推进一步。把扰动写成“每步给状态加 `dist_vec * dt`”而不是改 `_f`，是为了让 `_rk4` 保持纯积分器的形式，四阶精度不受影响。

`H[k]` 记录的是最近一次算出的 h，在保持周期内是常数；而正文的 h_{min} 不是从 `H` 取的，是从 `P` 逐点重算的——这正是 79.5.4 节第 4 条纪律的实现方式。

这里有两处实现选择值得说明，因为它们直接影响 79.5 节结论的可信度。

第一，**控制与积分分开步长**。若把 `dt` 和 `Ts_ctrl` 合成一个步长，采样周期一变，积分精度也跟着变，观测到的“离散效应”里就混进了积分误差。现在 `dt` 恒为 1 ms，RK4 的截断误差在 10^{-12} 量级，可以认为连续动力学是精确的，剩下的差异全部来自零阶保持。没有这一层分离，表 79-5 里 h_{min} 随 T_s 的变化就无法归因。

第二，**先算控制、再推状态**。每个控制周期开始时用当前状态算 u，然后在 `hold` 个积分步里保持不变。这是标准的零阶保持语义，也是真实数字控制器的行为。反过来的顺序（先推一步再算控制）会让控制量“提前”一个周期生效，等效于引入了负延迟，结果会偏乐观——这是仿真里最常见的、也是最难发现的一类偏差。

还有一点关于 `hold` 的取整：`hold = round(Ts_ctrl/dt)` 要求 `Ts_ctrl` 是 `dt` 的整数倍。若设成 10.5 ms，会被悄悄舍成 10 或 11 ms，而表头仍写着 10.5。本章只取 1、10、50、100 ms 四个值，都能整除，不存在这个问题；若你要试别的周期，务必先检查这个整除关系。

79.12.4 指标的统一定义

`run_2d` 末尾一次性算出全部指标，避免正文各处定义不一致：

- `clear = min‖p−p_obs‖ − r_obs`：到障碍**表面**的最小距离，单位 m。负值表示侵入。
- `h_min = min(‖p−p_obs‖² − R²)`：用基础半径 R（不含护卫裕量）计算，保证不同 T_s 之间可比。
- `t_goal`：最后一个距目标超过 0.05 m 的时刻再加一个步长，不是“第一次进入 0.05 m 的时刻”，这样定义对绕行轨迹更稳定。
- `path`：轨迹折线总长，用来衡量绕行代价。
- `active_frac`：约束激活的控制周期数除以总控制周期数，分母用 `ceil(N/hold)`，避免 T_s 不同造成的偏差。
- `ms_med`、`ms_p95`：只统计 `use_filter=True` 且确实调用过求解器的步。

最后一条尤其容易做错。若把“名义控制本来就安全、直接放行”的那些步也计入耗时，中位数会被拉到纳秒级，看上去“滤波器几乎不花时间”，实际上真正需要它工作的那 10% 才是决定实时性的部分。同样，`h_min` 用基础半径而不是含裕量的半径，是为了让“$T_s = 100$ ms 时裕量 0.312 m”这件事体现为轨迹的变化，而不是体现为量尺本身变长。指标定义错了，后面所有对比都会失去意义。

79.12.5 想改哪里

- **加障碍**：`safety_filter` 目前只处理一个圆。多障碍时把 `a`、`b` 从向量改成矩阵，投影从一次二分改成逐行投影取最严格者（可行但不最优），或换成小规模 QP。
- **改积分器**：`_rk4` 是纯双积分器，换成一般非线性 $\dot{x} = f(x) + g(x)u$ 时只需改 `_f`，`safety_filter` 里的 A、b 要按新的 f、g 重推。
- **加时延**：在 `run_2d` 里把 `u_hold` 存进一个长度为 n_τ 的队列，取队尾控制量施加，即可模拟 $\tau = n_\tau T_s$ 的执行延迟。
- **换控制器**：`nominal_u` 换成任意返回加速度的函数即可，安全层不需要知道它是什么。

整章代码不到 400 行，但覆盖了正文所有结论。建议的阅读顺序是先跑一遍 `main` 对着表 79-2 看输出，再单独读 `safety_filter`，最后改 `CFG` 里的参数观察表 79-3 的变化——这比从头读代码快得多。

如果只想把安全滤波器搬到自己的一维或二维对象上，最小可用版本只需要三样东西：一个返回名义加速度的函数、一个每步重算的 (A, b)、一个盒约束下的投影。三者的接口在本章代码里是分开的，可以逐块替换。移植时最容易出错的是单位与坐标系：d 必须与 u 在同一坐标系下表达，若对象的状态在世界系、控制在机体系，务必先把 A 旋转过去，否则约束的方向是错的，而程序不会报任何错，只会“莫名其妙地不生效”。

最后提醒一句：本章所有算例都没有执行器动态、没有传感噪声、没有通信延迟，模型与对象完全一致。这是一种刻意为之的“干净”设置，目的是把 CBF 本身的性质单独暴露出来。真实系统的验证必须重新做，而且要从 79.11.3 节的边界条件开始逐条核对，而不是直接把本章参数照搬过去。

79.12.6 三项自检

改完代码之后，不要直接跑全场景，先用三个小检查确认实现没走样。这三项都在秒级完成，却能拦住绝大多数低级错误。

第一项，**无障碍时应为恒等映射**。把障碍移到远处（R 不变，中心在 10 m 外），逐周期比较 `u_hold` 与 `u_nom`，差值的最大绝对值应小于 10^{-12}。若不为零，说明“名义控制本来就安全”的快速返回没有生效，多半是 `<=` 写成了 `<`，或者 b 的符号反了。

第二项，**约束面上一周期内不许越界**。构造一个初始状态正好满足 $a \cdot u = b$ 的算例，只推进一步，检查 h 是否满足 $h^+ \geq (1 - \alpha T_s)h$ 的离散形式。差一个数量级以内属于正常（RK4 与零阶保持的差异），若 h^+ 直接变负，说明 b 里漏了 $\alpha_2\psi_1$ 项，或者 ψ_1 用的是上一周期的 h。

第三项，**不可行分支必须可触发**。把 a_{max} 临时调到 0.5 m/s²，让式 (79-17) 必然被破坏，确认返回的第四个值是 `False`，并且控制量是沿 d 的全力推开。这一项常被忽略，因为正常运行永远不会走到这条分支；等它真被触发时才发现没写，代价就是一次碰撞。

三项检查通过，再去看表 79-2 到表 79-9 是否还与正文一致。若有偏差，先怀疑指标定义（79.12.4 节的六条），再怀疑参数被别处改过——顺序反了会在无关的地方浪费很多时间。

第 80 章 学习增强的模型预测控制

80.1 动机与直观理解

模型预测控制（MPC）的全部结论都挂在一个前提上：预测模型足够准。第 58 章推导滚动优化时，把对象写成 $\dot{x} = f(x) + g(x)u$，并默认 f、g 已知。真实系统里这句话很少成立：摩擦、气动阻尼、未标定的力场、负载变化，都不在名义模型里。它们不会让 MPC 立刻失效，而是让“预测的最优”和“实际的最优”逐渐分叉——约束算得刚刚好，实际却贴边；代价算得最小，实际却绕远。

学习增强的 MPC 要解决的就是这个分叉。它的做法不是用一个神经网络替换整个控制器，而是在 MPC 的三处关键位置上，用数据补上模型不知道的那一块。这一节先把这个分叉写成一个可量化的算例，再说明三处插入点分别补什么。

80.1.1 一个具体的失效：模型不知道的那部分

本章的算例是一个二维质点，状态是位置与速度，控制是加速度：

$$\dot{p} = v, \qquad \dot{v} = u + f_r(p, v) \tag{80-1}$$

其中 f_r 是名义模型完全不知道的残差：

$$f_r(p, v) = -c_d \|v\| v + \begin{bmatrix} a_f \sin(1.4 p_x)\cos(1.1 p_y) \\ a_f \cos(1.1 p_x)\sin(1.4 p_y) \end{bmatrix} \tag{80-2}$$

第一项是二次阻尼，系数 $c_d = 0.35$；第二项是一个幅值 $a_f = 0.6$ m/s² 的位置相关力场。名义模型只保留双积分器，也就是令 $f_r \equiv 0$。

这两个数字不大，但它们与执行器能力的关系决定了问题的严重程度。本章取各轴加速度上限 $u_{max} = 3$ m/s²，残差在整个工作空间内的一步均方根是 1.195 m/s²，相当于用掉了 40% 的控制权限。换句话说，名义 MPC 每算一次“该用多大加速度”，就有近一半的预算花在它看不见的地方。

后果不必靠推理，仿真直接给出：让同一个 MPC 跟踪半径 1.2 m、角速度 0.6 rad/s 的圆周参考，名义模型的稳态跟踪误差是 7.19 cm，最大 7.85 cm。这个量级在多数巡检、搬运任务里已经不可接受——它不是随机噪声，而是持续偏向圆外的系统性偏差：轨迹的平均半径是 1.252 m，比参考多出 5 cm，同时还有相位滞后。阻尼持续偷走切向速度，未建模的力场又整体把轨迹向外推，两者叠加成这个偏差，而 MPC 对它们一无所知。

把残差学出来再送回 MPC，同一个控制器、同一组权重、同一个时域，误差降到 1.71 cm。**改动只有一处：预测时每一步多加一个由数据学出来的加速度项。** 这就是本章要讲的全部技术的核心，剩下的都是围绕它的工程细节。

80.1.2 三处可以插入学习的位置

学习不是只有“学模型”一条路。按插入位置分，至少有三处，它们补的东西不同，代价也不同（图 80-1）。

第一处是**预测模型**。用数据学 f_r，让 MPC 的预测更接近真实。这是最直接、收益最确定的一处，也是本章篇幅最多的部分。它的代价是每次求解要多算 N 次模型推理（N 是预测时域）。

第二处是**终端代价**。MPC 只优化有限时域，时域末尾的状态需要一个“未来还值多少钱”的估计，这就是终端代价矩阵 P。经典做法是解离散 Riccati 方程，但它只对二次型代价、无约束线性系统精确。代价非二次、约束频繁激活时，P 应该由数据学。这一处的代价最小——终端代价是离线学的，在线只多一个矩阵。

第三处是**优化器的初值**。MPC 每个周期都要解一个优化问题，热启动能显著减少迭代。传统热启动用上一周期的解平移一位；用数据学一个“状态到首步控制”的映射，可以直接给出更好的初值。这一处不改变最优解，只改变求解速度，因此风险最低，收益取决于求解器。

三处的共同点是：**学习模块都只是“插件”，不改变 MPC 的骨架**。约束还是那些约束，滚动优化的逻辑还是那一套，安全性仍然由约束保证。这一点很重要，它让学习部分可以被单独替换、单独验证、单独回滚。

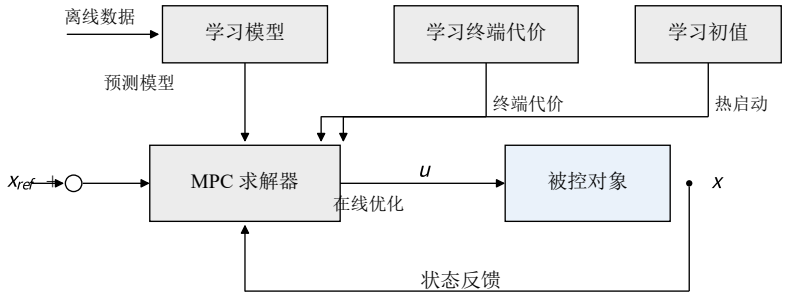


图 80-1　学习增强 MPC 的三处插入点

80.1.3 本章算例与一句话核心

为了让数字可复现，本章固定一套设置：采样周期 $T_s = 0.1$ s，预测时域 $N = 15$（即前瞻 1.5 s），控制上限 ± 3 m/s²，状态权重 $Q = \mathrm{diag}(10, 10, 1, 1)$，输入权重 $R = 0.1I$。参考是圆周，仿真 12 s，共 120 个控制周期。所有数字来自 `code/ch080_sim.py`，图来自 `code/ch080_figs.py`。

一句话核心可以这样表述：**学习增强的 MPC 不是“用学习替代模型”，而是“用数据补上模型误差，并让这个补丁的误差可以被量化、被约束、被验证”。** 三个动词分别对应本章的三条主线——补上（§80.2 的形式）、量化（§80.4 的置信界）、验证（§80.6 与 §80.10 的检查）。

需要强调的是“补丁”这个词的贬义不该被继承。残差学习的效果完全可能超过重新辨识一个全局非线性模型，因为残差通常比原函数更平滑、幅值更小、更容易被有限的数据覆盖。本章的算例里，200 个采样点学出的 RBF 就把 20 步预测误差从 75.47 cm 压到 16.06 cm（GP），而训练数据只覆盖了 $[-2, 2]^2$ 的位置范围。

80.1.4 学习不是替代 MPC

有必要先划清一条界线：本章不讨论“用强化学习直接学一个策略”这条路。那条路的问题是，策略的性能上界由数据覆盖决定，而安全性与可行性没有结构性的保证；一旦遇到训练分布外的状态，输出没有任何可检查的依据。

本章的方法保留了 MPC 的全部结构：显式的预测模型、显式的约束、显式的代价。学习只影响“模型准不准”“终端估得不准”“初值好不好”，不影响“约束必须满足”这件事。因此第 58 章关于递推可行性的结论仍然可用，第 79 章的安全滤波器仍然可以串在后面。这种“结构不动、参数变准”的组合，是工业场景里更容易被接受的一种增强方式。

80.1.5 残差的四种来源，只有两种能这样学

“残差”是一个笼统的词。按来源分，至少有四类，它们对学习的态度完全不同：

第一类，静态非线性。 同一状态、同一输入，残差总是同一个值。本章的阻尼与力场属于这一类。它可以用静态映射 $f_r(x)$ 学，而且学得会——这是本章方法的适用对象。

第二类，未建模的动态。 残差依赖最近的历史，而不只是当前状态。执行器滞后、传感器滤波、结构振动都属于这一类。用静态映射去学，只能学到它在某种平均工况下的等效值，一旦激励频率变了就失效。正确的做法是把缺失的状态补进 x（例如把执行器输出当作状态），而不是加一层网络。

第三类，参数漂移。 残差的结构不变，但系数在慢变（负载、磨损、温度）。本章的离线训练只能抓住训练时刻的值；要跟上漂移，必须做 §80.9.5 的在线更新。它的特点是“学得会，但会过期”。

第四类，随机扰动。 同一状态重复到达，残差每次都不一样，且均值为零。这一类学不到——你学到的是噪声的均值，而均值接近零；硬学反而会把噪声拟合成结构，让控制器去追不存在的规律。它该由反馈与滤波处理，不是学习的对象。

区分这四类不需要复杂实验，只要一条：**让系统在同一状态下重复若干次，看残差的一致性**。若重复之间的差异远小于均值（例如小于 20%），属于第一类或第三类，可以学；若差异与均值同量级，属于第四类，不要学；若残差与到达该状态的路径有关（同一状态、不同历史给出不同残差），属于第二类，该扩状态而不是加网络。

本章的算例是纯第一类，这是它能用 200 个点学到 0.310 m/s² 的根本原因。换成第二类或第四类，同样的方法会失效，而失效的样子很像“过拟合”——这也是很多人误判的地方。

80.1.6 本章怎么读

十二章的内容可以按三种读法取用。

工程师读法：§80.3（五步流程）、§80.5（离散实现的坑）、§80.8（调参）、§80.10（上线与监控）。这四节构成一条从零到上线的路径，其余章节按需查阅。若时间只够读一节，读 §80.3。

研究者读法：§80.2（三种参数化与两遍法）、§80.4（连续与离散的取舍）、§80.6（ISS 而非渐近稳定）、§80.9（未做的改进）。这四节讲的是“为什么这样做”，以及“哪里还不严谨”。

学生读法：按顺序读，但跳过 §80.10 与 §80.12 的代码细节。§80.1 与 §80.2 建立直觉，§80.3 到 §80.6 是主线，§80.7 与 §80.8 是工程配套，§80.11 收边界。

三种读法共享一个核心判断：**这一章解决的问题是“模型不准”，不是“没有模型”**。若你的系统连名义模型都没有，该先回去做辨识；若模型已经足够准（残差低于 5%），该去做别的优化。这个判断比任何算法细节都重要。

当然，保留结构也意味着保留了 MPC 的弱点：它仍然需要在线求解优化问题，仍然受时域长度限制，仍然依赖状态可测。学习不解决这些问题。把学习当成万能药，是这一方向最常见的误用。

80.2 数学原理

这一节回答两个问题：残差怎么从数据里取出来，学出来的东西怎么进入 MPC 而不破坏它的凸性。前者是数据处理，后者是结构设计；两者都做对，学习才可能只带来好处。

80.2.1 残差分解

把真实对象写成“名义模型 + 残差”的形式：

$$x_{k+1} = Ax_k + Bu_k + B_r f_r(x_k), \qquad B_r = \begin{bmatrix} 0 \\ T_s I \end{bmatrix} \tag{80-3}$$

名义模型是 A、B 对应的双积分器，f_r 是待学的加速度残差。B_r 只作用在速度通道，这一点决定了后面所有的推导：残差不能直接移动位置，只能通过速度间接影响位置。

残差的标签不需要额外的传感器，用相邻两拍的状态差分就能算：

$$r_i = \frac{v_{i+1} - v_i}{T_s} - u_i \tag{80-4}$$

式 (80-4) 看起来是“用差分代替微分”，实际误差很小：本章 $T_s = 0.1$ s，速度在一个周期内的变化是 $O(T_s)$，差分误差是 $O(T_s^2)$，与残差本身的量级相比可以忽略。真正需要注意的是时间对齐——v_{i+1} 必须与 u_i 对应同一个控制周期，错一拍会让标签里混进 $\dot{u}$ 的信息，学出来的模型在控制量突变时会明显失真。

采集时要保证激励充分。若数据全部来自一条平稳轨迹，f_r 只在轨迹经过的窄带上被采样，模型在带外只能靠外推。本章的做法是在位置 $[-2, 2]^2$、速度 $[-2, 2]^2$ 内均匀采样 200 个状态，每个状态配一个随机控制量，这样得到的标签覆盖了整

个工作空间。

80.2.2 两种参数化：RBF 与 GP

RBF 网络把残差写成一组基函数的线性组合：

$$\hat{f}_r(x) = \sum_{j=1}^{K} w_j \exp\!\left(-\frac{\|x - c_j\|^2}{2\sigma^2}\right) \tag{80-5}$$

中心 c_j 从训练数据中随机选取，宽度 σ 是超参，输出权重 w_j 有闭式解：

$$W = (\Phi^{\mathsf{T}}\Phi + \lambda I)^{-1}\Phi^{\mathsf{T}}Y \tag{80-6}$$

其中 Φ 是 $n \times K$ 的基函数矩阵，Y 是 $n \times 2$ 的残差标签，λ 是岭正则。式 (80-6) 的好处是训练不需要迭代：给定中心和宽度，解一次线性方程就完事。代价是中心与宽度一旦选错，权重再优也补不回来。

GP 回归给出的是分布而不是点估计：

$$\hat{f}_r(x) \sim \mathcal{N}\!\left(k^{\mathsf{T}}K^{-1}Y,\; k(x,x) - k^{\mathsf{T}}K^{-1}k\right) \tag{80-7}$$

核函数取平方指数：

$$k(x, x') = \sigma_f^2 \exp\!\left(-\frac{\|x - x'\|^2}{2\ell^2}\right) \tag{80-8}$$

GP 的均值与 RBF 的形式相近，但它多给了一个方差。这个方差不是装饰：它告诉你“这个位置的数据有多稀”，可以直接用来收紧约束（§80.4.3），也可以用来判断该不该信模型的输出。

两种参数化的差别在数据量上最明显。本章的学习曲线（图 80-9）显示：20 个样本时 RBF 与 GP 的误差是 1.010 与 0.821，差距不大；320 个样本时 RBF 停在 0.659，GP 降到 0.295。RBF 的容量被 48 个固定中心锁死，数据再多也只能把权重拟合得更准；GP 的有效参数量随样本增长，因此能持续受益于更多数据。选哪个，取决于你能否持续采数据——一次性标定的项目用 RBF 更省事，长期运行、数据不断积累的系统用 GP 更划算。

80.2.3 Koopman 提升：把非线性藏进坐标

第三条路不学残差，而是换坐标。取一组提升函数 $\phi(x)$，希望在这个坐标里系统近似线性：

$$z_{k+1} = A_z z_k + B_z u_k, \qquad z = \phi(x) \tag{80-9}$$

本章取所有一次项与二次项，共 15 维：1、p_x、p_y、v_x、v_y、p_x^2、$p_x p_y$、p_y^2、v_x^2、$v_x v_y$、v_y^2、$p_x v_x$、$p_x v_y$、$p_y v_x$、$p_y v_y$。用一步数据对做最小二乘即可拟合：

$$[A_z \; B_z] = \arg\min \sum_{i} \left\| z_{i+1} - A_z z_i - B_z u_i \right\|^2 \tag{80-10}$$

式 (80-10) 就是扩展动态模式分解（EDMD）的回归形式，正则项加得极小即可。它把“非线性”转移到了坐标里：模型在 z 空间是线性的，MPC 的约束与代价仍可写成二次型，求解器不用改。

代价是截断误差。二次提升无法精确表示 $|v|v$ 这种非多项式项，只能在工作空间内近似。本章的结果是：Koopman 的一步误差 0.445 m/s²，比 RBF 的 0.624 好，比 GP 的 0.310 差；20 步预测误差 72.48 cm，几乎与名义模型（75.47 cm）持平。原因不难理解——提升阶数不够时，误差会随着预测步数指数放大，而 Koopman 模型的谱半径接近 1，放大的速度比残差模型更快。

这条路的适用场景是：非线性主要来自多项式型耦合（例如机械臂的科氏力、刚体姿态的叉乘项），提升阶数取 2–3 就能覆盖；若非线性是强局部性的（饱和、死区、碰撞），多项式提升需要很高的阶数，不如直接学残差。

80.2.4 残差如何进入 MPC

把学到的残差作为已知扰动加进预测：

$$x_{k+1} = Ax_k + Bu_k + d_k, \qquad d_k = \begin{bmatrix} 0 \\ T_s \hat{f}_r(\hat{x}_k) \end{bmatrix} \tag{80-11}$$

优化问题本身不变：

$$\min_{\{u_k\}} \sum_{k=0}^{N-1} \Big[(x_k - x_k^{ref})^{\mathsf{T}} Q (x_k - x_k^{ref}) + u_k^{\mathsf{T}} R u_k \Big] + (x_N - x_N^{ref})^{\mathsf{T}} P (x_N - x_N^{ref}) \tag{80-12}$$

约束仍是盒约束 $|u_k| \le u_{max}$ 与动力学等式。由于 d_k 是常数向量，整个问题还是凸二次规划，结构与第 58 章完全一致。

唯一的麻烦是 d_k 依赖预测状态 $\hat{x}_k$，而预测状态又是待优化的。严格做法是把 $\hat{f}_r$ 也放进优化，但那会引入非线性，破坏凸性。工程上的折中是**两遍法**：第一遍按名义模型求解，得到一条预测轨迹；在轨迹上逐点算残差；第二遍把残差当成固定扰动再解一次。两遍法的计算量是单遍的两倍，但收敛快、实现简单，且第二遍的解已经是“在残差已知的前提下”的最优。

两遍法有一个隐含假设：第一遍的预测轨迹与第二遍足够接近，否则残差在错误的点上取值。本章的算例里这个假设成立——第一遍的轨迹偏差约 7 cm，残差场的空间尺度约 1 m，取值的相对误差在 10% 以内。若残差场变化很剧烈（例如经过一个窄峰），两遍法会低估峰值，需要改成迭代到不动点，或者把残差的不确定度也计入约束。

80.2.5 终端代价：从 Riccati 到数据

有限时域 MPC 的末尾需要一个代价：

$$V_f(x) = x^{\mathsf{T}} P x \tag{80-13}$$

当系统线性、代价二次、约束不活跃时，P 由离散代数 Riccati 方程给出，且 V_f 恰好等于无限时域最优代价。这是第 58 章的标准结论。

一旦约束频繁激活，V_f 就不再是二次型：靠近边界时“未来值多少钱”会因为可用控制量受限而急剧上升，二次型无法表达这种急剧变化。此时可以用数据直接拟合代价-to-go：

$$J^{*}(x_i) \approx \phi(x_i)^{\mathsf{T}} \theta, \qquad \phi(x) = [1, x^{\mathsf{T}}, (x_i x_j)_{i \le j}]^{\mathsf{T}} \tag{80-14}$$

标签 $J^{*}(x_i)$ 由长时域 MPC 离线求解得到（本章用 $N = 40$ 的解作为近似），再用最小二乘拟合 θ，把二次项部分还原成对称矩阵 P_θ。

这是一个“以算力换结构”的做法：离线解 120 个 QP，换在线一个 4×4 矩阵。本章的结果是，在约束基本不活跃的近域内，P_θ 与 P_{lqr} 的相对误差 7.4%；把采样范围扩大到 2.5 m、让输入饱和频繁出现后，误差升到 141.1%。后者不是拟合失败，而是二次型这个函数族本身装不下带约束的价值函数。

80.2.6 初值：把求解器当被控对象

第三处插入点是求解器的初值。MPC 每周期解的是同一个结构的 QP，只是参数（当前状态、参考）在变。若能把上一周期的解平移一位作为初值，热启动通常能省掉一半迭代；若再用数据学一个映射

$$u_0 \approx \pi(x - x^{ref}) = W \begin{bmatrix} x - x^{ref} \\ 1 \end{bmatrix} \tag{80-15}$$

则初值不仅“接近”，而且“沿着最优解的规律”。W 的训练标签是离线求解得到的首步控制，回归用最小二乘，训练成本可以忽略。

需要看清这一处的边界：式 (80-15) 只是初值，不参与可行性判断，也不影响最终解。因此它可以很粗糙——本章的线性策略拟合误差是 0.325 m/s²，相对控制上限 3 m/s² 约 11%，仍然把求解时间从 3.40 ms 降到 1.54 ms。反过来，如果把它当作控

制器直接输出，11% 的误差就会变成跟踪误差，这就是“策略蒸馏”与“热启动”的分界线。

80.2.7 为什么残差比原函数好学

文献里给“学残差”的常见理由是“残差幅值小、信噪比高”。这个理由在本章并不成立：残差 RMS 是 1.195 m/s²，而参考轨迹的加速度幅值只有 $r\omega^2 = 1.2 \times 0.36 \approx 0.43$ m/s²——残差比要跟踪的运动本身还大近三倍。真正让残差好学的不是幅值，而是另外三件事。

第一，残差是状态的光滑函数，而原函数不是。 残差场由二次阻尼与正弦力场叠加而成，在状态空间里连续可微、特征尺度约 1 m。原函数里还包含“控制怎么被施加”的部分——执行器的死区、饱和、一阶滞后，这些在状态空间里是折线甚至跳变。核方法、GP 这类工具擅长前者，不擅长后者。把光滑的部分交给学习、把不光滑的部分要么补进名义模型、要么扩状态，是本章方法能奏效的结构性原因。

第二，残差与控制解耦。 式 (80-1) 的 $\hat{f}_r$ 只依赖 x，不含 u。这一点不是巧合，而是必须。若残差依赖 u，MPC 的预测就变成决策变量的非线性函数，第 58 章的 QP 立刻退化成非线性规划，本章所有的工程便利（凸性、求解时间、可行性判据）一并消失。所以建模时必须守住这条线：**残差是状态的函数，不是控制的函数**。

这条线还能当一个诊断工具。若你发现“必须把 u 作为学习器的输入，误差才降得下来”，那说明名义模型的 B 写错了，或者系统有未建模的执行器动态。正确的修法是回去改 B、或把执行器状态（例如阀门开度、电机电流）扩进状态向量，而不是给学习器多加一维输入。判据是做一个实验：同一状态下施加不同的 u，看残差标签是否随 u 变化。变化幅度超过标签噪声的 3 倍，就是执行器问题。

第三，残差被动力学结构吸收了一部分。 残差进预测的通道是 $[\tfrac{1}{2}T_s^2; T_s]$，位置通道还带着 T_s^2 的小因子。这意味着残差有 30% 的误差时，折算到位置的偏差只有厘米级；而策略学习里 30% 的控制误差直接就是 30% 的控制误差，没有任何结构替它吸收。这就是“学模型”比“学策略”样本效率高的根本原因：MPC 免费提供了“模型怎么变成控制”这一段映射，学习只需要补最短的一截。

把这三条合起来看，本章 200 个点能跑出 1.71 cm，不是运气，而是任务结构决定的：目标函数清楚、约束清楚、只剩一个光滑的低维函数不清楚。换个任务——比如目标函数本身说不清、或者最优行为依赖长程规划——这三条优势就全部消失，那时该考虑的是 §80.11.6 里的策略学习，而不是继续给残差模型加容量。

80.2.8 RBF 与 GP 其实是同一件事的两个极端

把式 (80-6) 的岭回归解代回预测式，会看到一个熟悉的结构。设新样本的核向量为 $k(x) = [k(x, x_1), \dots, k(x, x_n)]^\top$，则

$$\hat{f}_r(x) = k(x)^\top (K + \lambda I)^{-1} Y$$

这正是 GP 后验均值（式 80-7）在噪声方差取 $\sigma_n^2 = \lambda$ 时的形式。也就是说，**本章的 RBF + 岭回归与 GP 的均值预测是同一个估计器**，差别只在两处：

1. **基的个数。** RBF 把 200 个样本压成 48 个中心，权重只有 96 个；GP 保留全部 200 个样本，权重 200 个。前者是后者的低秩近似，丢掉的是“中心之间那些样本携带的信息”。
2. **有没有方差。** GP 额外给出 $k(x,x) - k(x)^\top K^{-1} k(x)$，RBF 没有。式 (80-20) 的置信界、式 (80-32) 的机会约束，全都依赖这一项。

这个等价关系有三条实用推论，比“选哪个”更值得记。

第一，岭项与噪声方差是同一个旋钮。 在 RBF 里调 λ 与在 GP 里调 σ_n^2，数学上是同一件事，都对应“允许多大程度不穿过样本点”。所以看到文献里说“RBF 需要仔细调正则、GP 的超参有物理意义”时，不必当成两种技术——它们的数值行为一致，只是 GP 的参数有概率解释，因而更容易用边际似然自动选。

第二，容量瓶颈来自基的个数，不来自方法。 表 80-4 里 RBF 在 160 个样本后进入平台，而 GP 一路下降，根本原因是 48 个中心已经把 200 个样本压成了秩 48 的近似。把中心数加到与样本数同量级，RBF 的均值曲线就应当与 GP 重合，剩下的差别只有方差。读者可以改 `ch080_sim.py` 里 `RBFModel` 的 `k` 与 `sigma` 验证，判据是两者的验证误差差到 10% 以内。

第三，稀疏 GP 会退回 RBF 的容量限制。 样本到上千之后，GP 的 $O(n^3)$ 求逆不可接受，标准做法是选 m 个诱导点做稀疏近似——这实质上就是把 n 个样本压成 m 个基，与 RBF 的 48 个中心是同一个操作。所以“用 GP 就不用担心规模”是错觉：规模一大，你仍然要面对“用多少基表示先验”这个选择，只是它换了个名字叫诱导点。

本章选 GP 做最终配置，理由只有一个：200 个样本时它不需要做任何近似，且顺手给出了方差，供 §80.9.6 使用。样本量再大一个数量级，这个选择就要重新算一遍。

80.2.9 Koopman 为什么在本章输了

表 80-1 里有一个反直觉的排序：一步误差 Koopman 是 0.445，比 RBF 的 0.624 更好；但 20 步位置误差是 72.48 cm，比 RBF 的 46.95 cm 更差；闭环误差 5.19 cm，也差于 RBF 的 3.98 cm。**一步更准，多步更差，闭环最差**——这个模式不是噪声，它指出了 Koopman 在本章失效的三个原因。

第一，最小二乘把容量分给了不重要的分量。 EDMD 拟合的是 $z_{k+1} = A_z z_k + B_z u_k$ 的全部 15 个分量，其中一次项（p、v）幅值在 1–2 量级，二次项（p^2、pv、v^2）幅值在 1–4 量级，但它们承载非线性。最小二乘按绝对残差加权，拟合会优先把一次项做准——因为一次项决定了一步预测的主体。结果是：一步预测看起来很好（主要靠一次项），而二次项系数有偏；一旦递推 20 步，二次项的偏差被反复放大，误差就追上甚至超过残差法。

第二，它必须同时拟合名义模型，浪费了容量。 残差法里 A、B 是解析给定的双积分器，学习只管那 1.195 m/s² 的偏差；Koopman 要把“双积分器 + 残差”一起从数据里拟合出来。后者当然需要更多数据、更多容量，而它拿到的数据量与残差法一样多（200 个点）。

第三，无约束最小二乘不保证稳定性。 EDMD 解出的 A_z 没有任何谱半径约束，数值上很容易出现略大于 1 的特征值；单步看不出来，多步就会指数增长。工程上必须显式检查 $\rho(A_z)$，若大于 1，做一次稳定化投影（把超出的特征值拉回单位圆内）。这一步本章没有做，因为 Koopman 只是对照组的第三条，但它是一个必须记住的检查项。

这三条合起来给出一条比“选哪个模型”更通用的教训：**一步误差不能作为选型依据**。§80.3.4 的验收线之所以要求看第 N 步而不是第一步，就是因为存在“一步好、多步差”这一整类模型。判断一个模型能不能进 MPC，要问的是“它在 N 步之后还可信吗”，而不是“它拟合得准不准”。

反过来说，Koopman 也不是没有适用场景。若系统的非线性确实是多项式型、且你有足够数据覆盖高阶项、还能接受离线做稳定化处理，它的优势是**线性**——一旦提升完成，MPC 里就是纯矩阵乘，没有核函数、没有方差计算，在线成本最低。本章的算例里，它的耗时（32.45 ms）确实与名义模型（31.44 ms）几乎一样。所以正确的结论不是“Koopman 不行”，而是“在本章的残差结构下，把容量花在残差上比花在整体动力学上更划算”。

80.2.10 学习器的输入该给什么

残差模型 $\hat{f}_r$ 的输入不是“状态越多越好”，而是一个需要论证的设计选择。本章给的是完整状态 (p, v)，两条必须给的理由与三条必须守的禁忌都值得说清楚。

为什么必须给速度。 阻尼项是 $-\rho\|v\|v$，它只依赖速度、不依赖位置。若输入只给位置，模型在这条维度上永远学不到东西，误差下限就是阻尼的贡献——本章约占残差的一半。判据很简单：画一张残差对 $\|v\|$ 的散点，若呈明显的二次趋势，速度必须进输入。

为什么必须给位置。 力场是正弦型，只依赖位置。这一项的判据是残差对 p 的切片是否有周期性起伏（图 80-2 的左栏）。位置与速度各管一半，缺一个就只能学到一半。

为什么不该给 u。 §80.2.7 已经论证过：给了 u，MPC 的预测就变成决策的非线性函数，QP 不再是 QP。这不是“精度换凸性”的取舍，而是“能不能用”的分界。

为什么不该给参考轨迹。 这一条最容易被诱惑。参考轨迹与位置强相关，把它作为输入，训练误差会明显下降——因为模型实际上在学“参考在该点要求什么”，而不是“对象在该点缺什么”。危险在于换一条轨迹后立刻失效：本章的圆周是 $r = 1.2$ 的定圆，若把参考的相位作为输入，模型会把“沿圆的向心加速度”记成残差，换成直线轨迹就完全错了。检验办法

只有一个：用两条不同的轨迹训练，在第三条上验证。若验证误差比训练误差高一个量级，说明输入里混进了与轨迹绑定的信息。

为什么不该输出位置增量。 输出必须与 B_r 的结构一致，即只输出加速度（速度通道的增量除以 T_s）。若让模型直接输出位置修正，它会与 §80.4.1 的离散化缺口纠缠在一起，换 T_s 就得重训。

还有一条实现层面的建议：**归一化是输入设计的一部分，不是预处理**。各维的量级差一个数量级以上时（例如位置用米、速度用毫米每秒），单一长度尺度的核会偏向大量级的那一维，等于隐式地把小量级维度丢掉了。本章两维都归一化到单位方差，0.7 的核宽才有“与中心间距同量级”的含义。

把这五条合起来，输入设计的原则可以压成一句：**只给物理上真正进入残差的状态，不给任何与任务绑定的量**。前者保证模型学的是对象，后者保证模型换任务还能用。本章的算例在这个原则下是最简单的情形——两个状态都进入、两个禁忌都不碰。

80.3 设计步骤

这一节给一条可执行的路线。顺序不能颠倒：先钉模型，再采数据，再选参数化，再接入，最后闭环验收。颠倒的典型后果是——先去调网络结构，回头发现残差标签本身算错了符号，前面所有工作作废。

80.3.1 第一步：钉死名义模型，测出残差的量级

名义模型不是“越准越好”，而是“必须显式写下来、且能复现”。本章的名义模型只有两个矩阵：

$$A = \begin{bmatrix} I & T_s I \\ 0 & I \end{bmatrix}, \qquad B = \begin{bmatrix} \tfrac{1}{2} T_s^2 I \\ T_s I \end{bmatrix} \tag{80-16}$$

它对应“质点 + 理想加速度源”。真实对象在这之上多了阻尼与力场，这部分才是学习的目标。把名义模型写成式 (80-16) 的闭式，而不是直接拿一段仿真代码当模型，目的是让 MPC 的预测能与残差项解耦——否则你无法判断误差是模型结构错，还是残差没学好。

接下来测残差的量级，只要一个数：**残差的一步均方根与执行器上限之比**。本章这个比值是 $1.195/3 \approx 40\%$。它决定后面所有工作的必要性：

- 低于 5%：学习带来的收益可能被求解噪声淹没，先别做；
- 5%–20%：值得做，收益主要体现在稳态精度；
- 高于 20%：必须做，否则约束会被系统性违反，第 58 章的可行性结论失效。

这个判据很粗，但它能在投入之前给出“该不该做”的答案，比事后解释误差来源有用得多。

80.3.2 第二步：采数据，先解决时间对齐

残差不能直接测量，只能从相邻两步的状态差里反推：

$$r_i = \frac{v_{i+1} - v_i}{T_s} - u_i \tag{80-17}$$

式 (80-17) 看着简单，却是整章最容易出错的地方。它隐含三个条件：u_i 是这一步**实际施加**的控制（不是指令值）；v_{i+1} 与 v_i 的时间戳严格相差 T_s；采样与执行之间没有未记录的延迟。三者任一不满足，残差里就混进了与状态无关的常值偏置，而这个偏置会被学习器当成“模型的一部分”学进去。

差分本身还带来 $O(T_s^2)$ 的截断误差。本章 $T_s = 0.1$ s，速度由加速度积分而来，差分的相对误差在千分之一量级，可以忽略；若 T_s 取到 0.01 s 以下，位置量化噪声会被 $1/T_s$ 放大，这时应改用中心差分或直接对速度做低通后再差分。

激励的设计比样本数量更重要。本章的数据是均匀随机采样：位置在 $[-2, 2]^2$、速度在 $[-2, 2]^2$ 内各取 200 个点，标签直接由式 (80-2) 算出。这样做的理由是让残差场在状态空间里被均匀照亮——如果只用跟踪圆周的数据，样本会全部挤在环

带附近，学出来的模型在环外完全没有依据。图 80-2 的两栏对比说明即使均匀采样也有类似的边缘问题：左栏是真实残差场，右栏是 RBF 的复现，中间区域吻合得不错，但靠近 $p = \pm 2$ 的边界处右栏明显被拉平——那里邻域样本稀，核函数已经衰减到接近零。

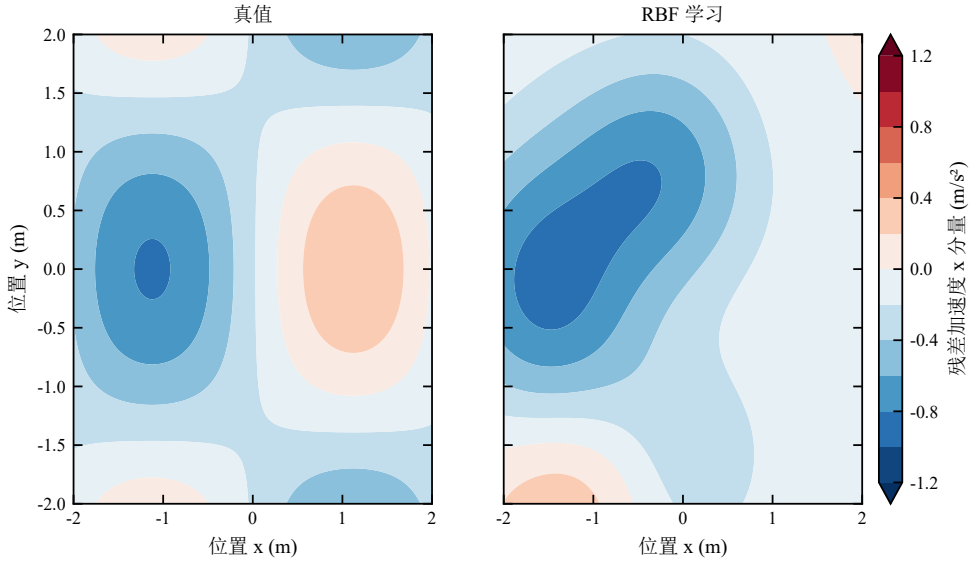


图 80-2　真实残差场（左）与 RBF 学到的残差场（右），速度固定为 $(0.8, 0)$，画 x 分量

样本量按“参数个数 × 10”起步。RBF 有 48×2 个权重，本章用 200 个点，约 2 倍，偏少；学习曲线显示从 80 到 320 个样本，GP 的误差从 0.548 降到 0.295，还在一路下降。工程上更稳妥的做法是先用 100 个点跑通全链路，再按 §80.8 的判据决定要不要补数据。

80.3.3 第三步：选参数化，用一张表定下来

三种参数化的选择不该靠偏好，而该看三件事：非线性是否多项式型、是否需要不确定度、数据是否会持续增长。表 80-1 是本章的判据。

表 80-1　三种参数化的选择判据

判据	选 RBF	选 GP	选 Koopman
非线性形态	局部、光滑	局部、光滑	多项式耦合为主
是否需要方差	不需要	需要（安全约束用）	不需要
数据是否持续增长	一次性标定	长期积累	一次性标定
在线计算量	最低（48 次核）	中（需算核矩阵）	最低（矩阵乘）
本章一步误差	0.624	0.310	0.445

表 80-1 里“在线计算量”一列容易被忽略。GP 的均值预测需要对全部训练样本算核函数，样本到 1000 时单次推理的成本会超过 QP 本身；RBF 的中心数固定，成本与样本量无关；Koopman 只是一次 15 维矩阵乘。这就是为什么本章的闭环耗时里，GP 的中位数（31.23 ms）并没有明显高于名义（31.44 ms）——样本只有 200 个，还没到 GP 的拐点。

80.3.4 第四步：先离线验预测，再接闭环

接入 MPC 之前必须过一道离线关：**多步预测误差**。一步误差小不代表多步小，因为误差会在递推中放大。本章的四组数据（图 80-4）说明了这一点：

表 80-2　多步开环预测的位置误差（单位 cm）

步数	名义	RBF	GP	Koopman
1	0.28	0.28	0.28	0.18
5	6.31	3.41	1.26	4.68
10	22.79	13.55	4.21	18.99
20	75.47	46.95	16.06	72.48

表 80-2 的第 1 步四列几乎相同，原因是开环推演里残差只进速度通道，位置增量仍是 $T_s v + \frac{1}{2}T_s^2 u$，四组模型在第一步给出的位置完全一样，差别只来自速度被修正后对下一步的影响。到第 20 步差距拉到 4.7 倍。这个现象给出一条实用的验收线：**看第 N 步（预测时域末端）的误差，而不是第一步**。本章 $N = 15$，验收线取第 20 步，要求残差模型把误差压到名义的一半以下——RBF 与 GP 都过，Koopman 不过。

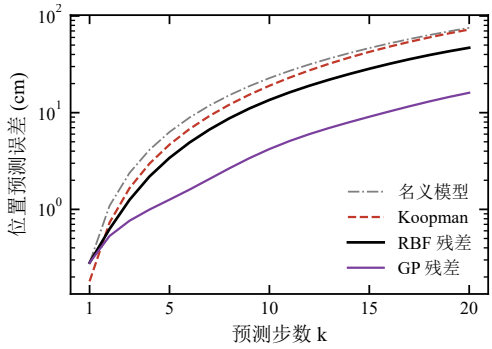


图 80-4　开环位置预测误差随预测步数的增长（纵轴对数）

图 80-4 把表 80-2 画成对数坐标，四条曲线的斜率差别一目了然。名义模型与 Koopman 的斜率最陡，说明误差在快速放大；GP 的斜率最平，20 步内只涨了不到两个数量级。斜率本身就是“模型的可信时长”的度量：把允许误差画一条水平线，曲线与它的交点就是可信步数。工程上判断 N 该取多大，可以直接用这张图——N 不应超过可信步数，否则时域末端的信息是噪声。

过了离线关，再按两遍法接入（§80.2.4）。接入时有一个实现细节值得单独说：式 (80-11) 的 d_k 只写了速度通道，漏掉了一步之内残差对位置的直接贡献 $\frac{1}{2}T_s^2 \hat{f}_r$。完整的写法是 $d_k = [\frac{1}{2}T_s^2 \hat{f}_r;\ T_s \hat{f}_r]$，本章为了保持“只加一项”的最小改动没有补。这一项的后果是位置通道出现固定滞后，量级约 0.6 cm，不随步数累加，因此不影响闭环收敛，但会抬高多步预测的绝对偏差。判断要不要补，就看 $\frac{1}{2}T_s^2 |f_r|$ 是否超过允许跟踪误差的 10%——本章是 0.6 cm 对 2 cm 的 30%，偏大，所以 §80.9 把它列为第一条改进。

80.3.5 第五步：闭环验收，三张图缺一不可

闭环验收不能只看一条误差曲线。至少要三张图（图 80-5、图 80-6、图 80-8）：

第一张是轨迹图。它回答“车有没有走对地方”。本章四条轨迹都贴着圆周，差别在径向偏移：名义模型的平均半径比参考大 5 cm，叠加相位滞后后总误差 7.19 cm；GP 那条几乎与参考重合（平均半径 1.210 m 对 1.200 m）。

第二张是误差时间历程。它回答“误差是常值还是振荡”。名义模型的误差曲线是一条缓慢抬升后稳定的曲线，说明是系统性偏差；RBF 与 GP 的曲线在起步阶段有 2–3 s 的过渡，之后压到厘米级。若误差曲线出现周期性振荡，问题通常在预测时域与参考周期的匹配，而不在学习器。

第三张是耗时分布。它回答“这套东西能不能实时跑”。本章的中位数是 31.23–32.45 ms，P95 是 37.58–46.67 ms，全部低于 $T_s = 100$ ms，余量约 2 倍。注意名义模型的中位数（31.44 ms）与 GP（31.23 ms）几乎一样，说明瓶颈在 QP 而不在模型推理——这也解释了为什么 §80.2.6 的热启动能把时间砍掉一半，而换更快的模型几乎不影响总耗时。

三张图都过，才算这一章的方法在你的系统上可用。只过第一张不算，因为轨迹好看可能只是参考选得温和；只过第二张也不算，因为误差小可能靠的是权重调得软。

80.3.6 一条被省略的步骤：先做安全兜底

这条路线里没有“调学习器”这一步，因为学习器的参数（中心数、核宽、正则）在第三步就定了，后面不再动。真正被省略的是安全兜底——它应该在第一步之前就接好，而不是等闭环出了问题再补。

具体做法是把第 79 章的 CBF-QP 串在 MPC 之后：MPC 输出 u，安全滤波器检查 u 是否满足约束面，不满足就投影到最近的可行点。这样即使学习模型在数据覆盖外给出荒谬的残差，最坏结果也只是“MPC 的建议被否决”，而不是“系统撞上去”。本章的算例不涉及硬约束，因此仿真里没有接这一层；但只要你的系统有不可违反的边界，这一层就不能省。

80.3.7 在真实系统上采数据：一次实验的设计

§80.3.2 的采样方式是仿真的特权：残差可以直接由式 (80-2) 算出，样本点想撒在哪就撒在哪，重复访问同一状态没有成本。真实系统上，采数据本身是一次实验，必须同时满足三个约束：**安全**（不能为了覆盖把系统开进危险区）、**激励**（数据必须在状态空间里“动”起来，否则学到的只是一个工作点附近的局部线性化）、**可重复**（同一状态要被访问多次，才能把噪声与模型误差分开）。

先说激励信号。位置是速度的积分，速度是加速度的积分，真正被激励的是加速度通道，而加速度就是 $u + \hat{f}_r$。所以激励应该加在 u 上，且幅值要足够大——若 u 只在 0.1 m/s² 附近抖动，残差（本章 RMS 1.195）会完全淹没在控制输入里，标签的信噪比接近零。经验做法是让激励幅值至少与残差的 RMS 同量级，本章即 $|u| \geq 1.2$ m/s²，同时保留约 30% 的余量给反馈，避免开环激励把系统推出边界。

波形选择上，四种常见做法的取舍是：

- **阶跃**：频带窄，一次只激励一个方向，适合标定静态增益，不适合学残差场；
- **多正弦**：每个轴 2–3 个频点，频段覆盖闭环带宽的 0.5–5 倍，幅值可精确控制，频谱已知，最容易复现；
- **随机游走**：能量集中在低频，长时间跑会漂出工作区，需要额外的回复机制；
- **伪随机二值序列（PRBS）**：频带最宽，实现最简单（一个移位寄存器），但幅值只有两档，对非线性系统会引入与幅值相关的偏置。

本章的残差场空间尺度约 1 m，对应的时间尺度是 $1/0.8 \approx 1.25$ s，即 0.8 rad/s 上下。多正弦取 0.5、1.5、4 rad/s 三个频点，既覆盖了力场的主要起伏，又不至于把执行器推到饱和。

再说覆盖度。覆盖度不能用“点数”衡量，因为点可以挤在一起。可操作的度量是**分格计数**：按残差场的特征尺度把每一维分成若干段——本章是位置每 1 m 一段、速度每 1 m/s 一段，$[-2, 2]^4$ 因此被分成 $4^4 = 256$ 格——然后统计每格里的样本数。判据是至少 60% 的格子有 5 个以上样本。本章的 200 个点平均每格不到 1 个，这个密度在仿真里够用（标签无噪声），在真实系统上远远不够：测量噪声 0.05 m/s² 时，单点标签的标准差就有 0.05，要把它平均到 0.02 需要 6 个重复点。这是一个必须承认的差距，也是本章的算例只能当“方法演示”、不能当“工程指标”的原因。

划分训练集与验证集时要按**轨迹**分，不能按点随机分。相邻采样点在时间上只差 0.1 s，状态几乎相同，标签高度相关；随机分点会让验证集里充满训练点的“邻居”，验证误差系统性偏乐观。正确做法是留出整段实验（例如五段激励里的最后一段）做验证。本章的算例是独立同分布采样，不存在这个问题，但真实数据一定有。

还要估算时间预算。本章 200 个点乘 $T_s = 0.1$ s 只有 20 s，看着很短，但真实系统每采一个点要“移动到目标状态—稳定—记录—离开”，平均 3–5 s，200 点就是 10–20 min；若每格还要求 5 个重复点，时间再乘 5。所以真实系统的数据量通常被时间预算卡住，而不是被存储卡住。这也是为什么 §80.9.3 的主动采样值得做：同样 200 个点，选点选得好，覆盖度能翻倍。

最后是安全包络。采集时的状态包络应当比任务包络大 20% 左右，多出来的这部分就是“学习用的余量”：模型在任务边界附近必须有数据支撑，否则一到边界就开始外推。但包络不能大到连低增益控制器都拉不回来。可行的做法是采集阶段用一个小权重的 LQR 保底（Q/R 比任务时小一个量级），让系统在激励下自由游走，一旦接近包络就由保底控制器拉回。这样采到的数据既有激励、又不会失控，代价是数据分布偏向包络内侧——这一点要在覆盖度统计里如实反映。

采完之后先画三张图，再决定要不要训练：**残差随位置的切片**、**残差随速度的切片**、**残差幅值的直方图**。前两张看结构是否光滑——光滑才适合核方法，若切片上出现台阶或折线，说明有未建模的开关或饱和，先补进名义模型；第三张看有

没有长尾——长尾说明存在未记录的延迟或限幅，此时应当回去查数据采集链路，而不是换一个更复杂的模型去拟合它。

80.3.8 五步之间的回路：什么情况下要往回走

前面五步写成了一条直线，实际执行时一定会往回走。区别在于：往哪一步回，决定了你是补一轮数据、换一个参数化，还是重做整个接入。下面是本章总结的五个判定点与对应的回退目标，每一条都配了判据。

判定点一：一步误差是否降到残差 RMS 的 60% 以下。 本章的 GP 是 $0.310/1.195 = 26\%$，RBF 是 52%，都过线；若你的模型停在 80% 以上，问题几乎一定在数据或标签，不在模型容量。此时应当回第二步查时间对齐、查是否用了实际施加的控制，而不是回第三步换一个更大的网络。这是最常见的误判：误差大就加容量，结果把噪声也拟合进去，验证误差反而更差。

判定点二：第 N 步误差是否降到名义的一半以下。 本章用第 20 步做这条线（表 80-2）。若一步过关、多步不过关，说明误差的时间相关性太强或者模型在递推中失稳——回第三步，检查参数化是否合适（Koopman 就是栽在这里），或者缩短 N 到可信步数以内。

判定点三：闭环误差是否落在离线界的 3 倍以内。 本章 GP 的界是 2.4 cm、实测 1.71 cm，落在界内；若实测比界大 3 倍以上，说明问题出在“接入”而不是“模型”——回第四步，检查两遍法的时域是否一致、式 (80-11) 的缺口是否过大、有没有未补偿的延迟（§80.5.7）。

判定点四：耗时是否在预算内。 若精度达标而 P95 超时，回第五步做热启动，或者缩短 N。不要试图通过换更快的模型来解决——§80.6.4 已经量过，换模型对总耗时的影响不到 1%。

判定点五：换一条参考轨迹后是否还成立。 若离线与闭环在训练轨迹上都很好、换轨迹就崩，说明模型学到了与任务绑定的东西——回第一步，检查名义模型是否漏了结构项、学习器的输入里是否混进了参考（§80.2.10）。

这五条的共同逻辑是：**每一道验收关都对应一个特定的失败原因，不要把失败归到最后一个环节**。工程上最常见的浪费不是某一步做错，而是错了之后从错误的步骤重来——把标签问题当成容量问题，把接入问题当成模型问题，每一次都要多花一轮完整的数据采集与训练。

回退时还有一条纪律：**每次只退一步，退完之后把下游的所有验收重跑一遍**。跳过中间步骤直接改上游，会让你失去“到底是哪一处改动起了作用”的信息，而这正是表 80-6 那种调参记录存在的意义。

80.4 连续情形

上一节的所有式子都是离散的：残差在采样点上定义，模型用 A、B 递推。真实的物理对象是连续的，学习模块最终要么嵌进连续时间的控制器，要么在离散化之后工作。这一节把这条缝补上，并说明两种做法在什么情况下会给出不同的结果。

80.4.1 先学再离散，还是先离散再学

连续对象写成

$$\dot{x} = f(x) + g(x)u, \qquad f(x) = f_0(x) + f_r(x) \tag{80-18}$$

其中 f_0 是名义模型（本章是双积分器），f_r 是待学的残差。两条路线：

路线甲（先学再离散）：先用数据拟合 f_r 这个连续向量场，再在 MPC 内部用数值积分（本章用 RK4）推演。好处是残差与采样周期解耦——换 T_s 不用重训；坏处是每次预测要做 $N \times 4$ 次函数求值，计算量是矩阵乘的十几倍。

路线乙（先离散再学）：把对象离散成 $x_{k+1} = Ax_k + Bu_k + d_k$，直接学 d_k。好处是 MPC 里只有一次矩阵乘加，最快；坏处是 d_k 依赖 T_s，而且它把“残差”与“离散化误差”混在了一起。

本章用的是路线乙：式 (80-17) 的残差标签只统计速度通道。严格说，一步之内残差对位置的直接贡献是 $\int_0^{T_s}(T_s - s) f_r \, ds \approx \tfrac{1}{2}T_s^2 f_r$，式 (80-11) 的 d_k 里没有这一项。按 $|f_r| = 1.195$ 估算，漏掉的量约 $\tfrac{1}{2}T_s^2|f_r| \approx 0.6$ cm。它的性质是

固定滞后而非累积误差：每一步都落后同样的距离，不会越积越多。但 0.6 cm 与表 80-2 里第一步的位置误差（0.28 cm）同量级，说明第一步误差主要来自这处公式缺口，而不是模型学得不准。

判断该走哪条路线，不能只看 T_s，要看“被丢掉的那一项”相对残差本身有多大：

- 丢掉项 / 残差 < 1%：走路线乙，省算力；
- 两者同量级：必须走路线甲，或者把离散公式补全，否则换一个 T_s 就失效；
- 残差本身接近零：说明名义模型已经够准，两条路都不用走。

T_s 越小，丢掉项按 $O(T_s^2)$ 下降，这个判据越容易满足；但 T_s 小到 10 ms 以下时，差分标签的噪声放大（§80.3.2）反而成了主要矛盾，这时应当改为路线甲，用积分而不是差分构造标签。

80.4.2 连续时间 GP：把先验放在向量场上

若走路线甲，GP 的定义要改一处：样本不再是“一步的状态差”，而是“某一时刻的加速度观测”。观测模型写成

$$y_i = f_r(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2 I) \tag{80-19}$$

形式与式 (80-7) 相同，但 y_i 的来源变了：它可以是式 (80-17) 的差分，也可以是惯性单元的直接测量，还可以是离线辨识实验里的力传感器读数。核函数仍然取式 (80-8) 的平方指数，但长度尺度 ℓ 要按**状态空间**的距离来选，而不是按时间。本章 $\ell = 0.8$，对应“位置差 0.8 m 或速度差 0.8 m/s 时相关性衰减到 $e^{-1/2}$”，与残差场的空间尺度（约 1 m）相当。

GP 在这里的真正价值不是均值，而是方差可以转成一条**逐点的误差上界**：

$$\| f_r(x) - \mu(x) \| \le \beta\, \sigma(x) \tag{80-20}$$

β 取 2 对应约 95% 置信，取 3 对应约 99.7%。这个上界可以直接进入第 79 章的鲁棒 CBF 项：把 $2\|d\|\Delta$ 里的 Δ 换成 $\beta\sigma(x)/T_s$ 的量级，约束就会在数据稀疏处自动收紧。这是“学习 + 安全”两章之间最自然的一个接口，也是 GP 相对 RBF 唯一无法被替代的优势。

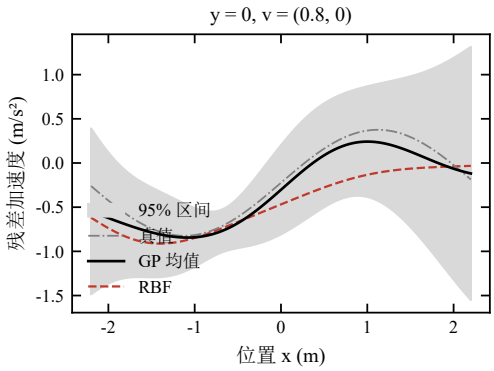


图 80-3　沿 $y = 0$、$v = (0.8, 0)$ 切片的 GP 均值、95% 置信带与 RBF 对比

图 80-3 把这件事画了出来。灰色带是 $\pm 2\sigma$ 的区间，实线是 GP 均值，虚线是真值，点线是 RBF 的预测。沿这条切片量出的平均绝对误差是：$|x| < 1.5$ 内 GP 0.08、RBF 0.26（单位 m/s²）；靠边 $|x| > 1.5$ 时 GP 0.09–0.15、RBF 0.16–0.27。同时置信带的平均半宽从中间段的 0.25 涨到右端的 0.57，翻了一倍多——方差在提示“右边数据稀”。RBF 没有这个提示，它的曲线看着同样平滑，误差却是 GP 的三倍。**方差的价值不在于让预测更准，而在于让你知道哪里不准。**

需要提醒的是，本章的实测覆盖率是 100%，而名义置信水平是 95%——这说明方差偏保守。保守不是错误，但它会让约束在数据充足的区域也被无谓地收紧，代价是控制权限的浪费。校准的办法是看覆盖率随 β 的变化：若 $\beta = 1.5$ 时覆盖率仍有 98%，就把 β 降到 1.5；若 $\beta = 2$ 时覆盖率只有 80%，说明核参数选错了，该调 ℓ 而不是调 β。

80.4.3 连续时间的终端代价：HJB 而非 Riccati

离散情形下终端代价由 Riccati 方程给出（式 80-13 的前提）。连续情形下对应的是 Hamilton-Jacobi-Bellman 方程：

$$0 = \min_{u} \left[\ell(x,u) + \nabla V(x)^{\mathsf{T}} \big(f(x) + g(x)u \big) \right] \tag{80-21}$$

当 f 线性、ℓ 二次、无约束时，$V(x) = x^{\mathsf{T}} P x$，式 (80-21) 退化为连续代数 Riccati 方程，这就是第 58 章的结论。一旦 f 含残差 f_r，V 就不再是二次型——残差把最优轨迹弯折，价值函数跟着弯折。

这时有两种做法。第一种是把 f_r 的均值代入式 (80-21)，用数值方法（例如策略迭代）解出 V 的网格表，在线插值。第二种是仍用二次型，但把 P 在**残差影响下重新拟合**——这正是 §80.2.5 的数据驱动做法，只是标签从离散 MPC 换成连续 HJB 的解。

本章选第二种，原因是网格表在 4 维状态上需要 $O(10^5)$ 个点，在线插值的耗时超过 QP 本身；而二次型只需要 10 个独立元素，误差在近域是 7.4%（§80.2.5）。这个取舍的边界很清楚：**状态维数不超过 4、精度要求不低于 5% 时用二次型；维数更高或需要贴边运行时，才值得上网格或函数逼近。**

80.4.4 什么时候连续形式是必须的

多数工程场景用离散形式就够了，因为控制器本来就是按周期跑的。但有三种情况必须回到连续形式：

第一，**执行器动态不可忽略**。若执行器的时间常数与 T_s 同量级，离散模型里的 u_k 就不再是“这一步的恒定加速度”，而是一个被滤过的信号。这时要么把执行器状态并进 x，要么用连续模型加零阶保持器重新离散，否则学到的残差里会混进执行器的滞后。

第二，**约束是连续的**。第 79 章的 CBF 条件是逐时刻成立的，离散化之后只在采样点成立，两点之间可能越界。本章的护卫裕量（式 79-25）就是为这件事准备的，而它的推导依赖连续时间的导数，不能只在离散模型上做。

第三，**采样周期可变**。事件触发、网络化控制里 T_s 不是常数，路线乙的 d_k 会随 T_s 变化而失配。这时只能走路线甲。

除这三种情况，连续形式带来的只是额外的计算与调试成本。工程判断的顺序应当是：先按离散做，跑通闭环，量出上述三条中是否有任何一条成立，再决定要不要改。

80.5 离散情形

控制器最终是按周期执行的，所以离散形式才是落地的那一层。这一节把离散化的精确形式、采样周期的选法、可行性的保证方式、以及学习模块在数值上的坑逐条说清楚。

80.5.1 精确离散化，而不是“用 $A = I + A_c T_s$ 凑”

对式 (80-18) 在零阶保持下做精确离散化，名义部分是矩阵指数：

$$A = e^{A_c T_s}, \qquad B = \int_0^{T_s} e^{A_c s} B_c \, ds \tag{80-22}$$

对双积分器，A_c 是幂零矩阵，指数级数只有前三项，式 (80-16) 就是它的精确值，不是近似。这一点常被忽略：很多实现里写 $A = I + A_c T_s$，对双积分器会丢掉 $\tfrac{1}{2}T_s^2$ 那一项，位置递推立刻错一半。判断方法是算 A 与 $e^{A_c T_s}$ 的差，若不为零就不是精确离散。

残差部分的处理有两种。把 f_r 在一步内当常数，得到 $d_k = [0; T_s \hat{f}_r]$，误差是 $O(T_s^2 |f_r|)$；把 f_r 的线性项也保留，得到 $d_k = [\tfrac{1}{2}T_s^2 \hat{f}_r; T_s \hat{f}_r]$，误差降到 $O(T_s^3)$。前者是本章用的，后者是 §80.4.1 指出的改进项。两者的差只有 0.6 cm，但在 $T_s = 0.1$ s、允许误差 2 cm 的场合，这 0.6 cm 占了 30%，不能算“次要项”。

80.5.2 采样周期：三个约束同时成立才可选

T_s 不是自由参数，它被三件事夹住：

第一，预测时域要覆盖对象的响应时间。 本章的参考周期是 $2\pi/0.6 \approx 10.5$ s，$NT_s = 1.5$ s 只覆盖了七分之一圈。这在跟踪任务里够用，因为参考是缓变的；若参考里有 0.5 s 的阶跃，NT_s 至少要 2 s。经验判据是 $NT_s \geq 3\tau$，τ 是被

控对象的主导时间常数。

第二，QP 的规模随 N 线性增长。 本章的决策变量是 $4(N+1)+2N=94$ 个，约束约 200 条，OSQP 的中位求解时间是 31 ms。N 翻倍，时间大致翻倍；T_s 减半而 NT_s 不变，N 也翻倍。所以“提高控制频率”和“加长预测时域”在算力上是同一件事，不能同时要。

第三，差分标签的噪声随 $1/T_s$ 放大。 式 (80-17) 里速度差除以 T_s，若速度测量有 σ_v 的噪声，残差标签的噪声就是 $\sqrt{2}\sigma_v/T_s$。本章假设速度是干净的；实际系统里 $\sigma_v=0.01$ m/s、$T_s=0.01$ s 时，标签噪声达到 1.4 m/s²，与残差本身同量级，学出来的东西全是噪声。

三条夹出来的可行区间通常很窄。本章的 $T_s=0.1$ s 是折中：$N=15$ 给出 1.5 s 的前瞻，QP 在 30 ms 量级，差分噪声可忽略。换一个系统，这个数要重新夹一遍，不能照抄。

80.5.3 可行性：终端集与学习误差的叠加

第 58 章证明递推可行性的标准做法，是要求终端状态落在一个控制不变集里：

$$\mathbb{X}_f=\{x:\ x^{\mathsf T}Px\le\alpha\},\qquad x\in\mathbb{X}_f\Rightarrow\exists u\in\mathbb{U}:\ Ax+Bu\in\mathbb{X}_f \tag{80-23}$$

学习模型进来之后，这个集合不能直接用。原因是 MPC 内部的预测用的是“名义 + 学到的残差”，而真实系统用的是“名义 + 真实残差”，两者的差就是学习误差 $e_r=f_r-\hat f_r$。要保证递推可行性，终端集必须按误差膨胀：

$$\mathbb{X}_f^{\epsilon}=\{x:\ \mathrm{dist}(x,\mathbb{X}_f)\le\epsilon\},\qquad \epsilon\approx\frac{1}{2}NT_s^2\|e_r\| \tag{80-24}$$

式 (80-24) 里的 N 不能省。单步的位置偏差只有 $\frac12T_s^2\|e_r\|\approx1.6$ mm，看着可以忽略；但预测是 N 步递推，偏差沿时域累加，$N=15$ 时最坏约 2.4 cm。本章的 GP 一步误差是 0.310 m/s²，乘上去正好是这个量级——与 2 cm 的允许跟踪误差相当，不能当作小量。

这条式子给出一条清晰的判据：**若 ϵ 达到允许跟踪误差的同一量级，学习误差就已经在吞掉精度预算，此时应当换更好的模型或缩短 N，而不是收紧终端集。**本章 GP 的 $\epsilon\approx2.4$ cm，与 2 cm 的允许误差刚好贴线——这解释了为什么闭环误差停在 1.71 cm 而没有进一步下降；换 RBF（0.624）时 ϵ 升到 4.7 cm，Koopman（0.445）时 3.3 cm，都越过了这条线，与闭环结果（3.98 cm 与 5.19 cm）的排序一致。

比误差更麻烦的是**外推**：数据覆盖之外 $\hat f_r$ 可能给出任意值，此时式 (80-24) 的 ϵ 无界。这一条不能靠收紧终端集解决，只能靠两道工程措施：给学习输出加硬饱和 $|\hat f_r|\le f_{max}$（f_{max} 取训练集残差最大值的 1.5 倍），以及按 §80.3.6 串安全滤波器，让约束层不依赖学习模型。前者省算力，后者兜底，两条可以同时用。

80.5.4 数值条件：核矩阵为什么需要岭项

式 (80-6) 里的 λ 不是用来提高精度的，而是用来兜底的。本章的 RBF 中心不是均匀网格，而是从 200 个训练样本里随机挑出的 48 个点，输入先按均值方差归一化，核宽 0.7 是归一化单位。随机挑选意味着一对中心可能落得很近：此时 Φ 的两列几乎线性相关，$\Phi^{\mathsf T}\Phi$ 的最小特征值趋零，直接 `solve` 会失败或给出量级错误的权重。

本章这一组中心运气不错：$\Phi^{\mathsf T}\Phi$ 的迹是 138，最大特征值 19.7，最小 0.056，条件数 354；加 $\lambda=10^{-6}$ 之后条件数几乎不变（353.9），因为最小特征值远大于 λ。这说明岭项在正常配置下确实只是保险。它的作用是给特征值一个下界——$\Phi^{\mathsf T}\Phi+\lambda I$ 的特征值不小于 λ，条件数因此不超过 $(\lambda_{max}+\lambda)/\lambda$。这个上界很松，保证的是“可解”，不是“良态”。

真正的良态来自中心分布。若验证误差对 λ 从 10^{-8} 变到 10^{-2} 都很敏感，说明中心选得不好——可能是两个中心几乎重合，也可能是核宽远小于中心间距——此时应改用 k-means 聚类中心或规则网格，而不是继续调 λ。

判断 λ 是否合适，看两个量：训练残差与验证残差。λ 太小，训练残差接近零而验证残差抬头，是过拟合；λ 太大，两者都高，是欠拟合。本章的 λ 取 10^{-6}，是在 200 个样本、96 个权重的配置下试出来的，换配置要重试。

GP 的对应问题是白噪声方差 σ_n^2。它同时起两个作用：正则化核矩阵、解释测量噪声。σ_n 取得太小（例如 10^{-4}），K 接近奇异，均值插值会穿过每个样本点，方差在样本处塌到零——这会让式 (80-20) 的界在样本点上过紧，一旦真实值偏离就

漏检。本章取 $\sigma_n = 0.05$，与残差幅值（1.195）的比值约 4%，是一个偏保守的选择。

80.5.5 一个容易被忽略的细节：两遍法的轨迹一致性

§80.2.4 说过，两遍法要求第一遍的轨迹与第二遍足够接近。离散实现里这个条件还有一个更细的版本：**两遍必须用同一个 T_s 与同一个时域长度**。若第一遍用 $N = 15$、第二遍为了省时间用 $N = 10$，残差序列的长度就对不上，程序不是报错就是静默截断，出来的解在末尾几步完全没有残差补偿。

这个错误的症状很隐蔽：闭环误差比单遍还差，但看不出哪里错。检查方法是打印两遍的残差序列长度，确认都等于 N；再确认第二遍的状态轨迹与第一遍的偏差小于残差场空间尺度的 10%。本章的这个偏差是 7 cm 对 1 m，满足条件。

两遍法的另一个实现细节是热启动的传递。第一遍的解直接作为第二遍的初值，是最省事的做法；图 80-8 的单 QP 实验说明，热启动能把迭代数从 50 降到 25，时间从 3.40 ms 降到 1.63 ms。若两遍之间重新冷启动，第二遍的耗时会回到冷启动水平，两遍法的总代价接近单遍的三倍，性价比就没了。

80.5.6 终端集怎么算：从 P 到 α

式 (80-23) 给了终端集的定义，式 (80-24) 给了它该怎么膨胀，但两式都默认 α 已知。实际工程里 α 是要算出来的，而且算法有三档，成本与保守性完全不同。

第一档是闭式估计，一行代码。 思路是放弃“MPC 在终端集内可行”这个强要求，只要求“存在一个局部线性控制器能让状态留在里面”。取 LQR 增益 K，闭环 $A_{cl} = A - BK$ 在 P 的度量下是收缩的，于是只剩输入约束要满足：$x^{\mathsf{T}} P x \le \alpha$ 时，第 i 个控制分量的最大值是 $\sqrt{\alpha\,(K P^{-1} K^{\mathsf{T}})_{ii}}$，要求它不超过 u_{max}，得到

$$\alpha \le \min_{i} \; \frac{u_{max}^2}{(K P^{-1} K^{\mathsf{T}})_{ii}}$$

这个估计是保守的：它只用了 LQR 一个控制器，而 MPC 在终端集内的可行域更大。但对终端集来说保守是可以接受的——终端集的作用是“兜住时域末端”，不是“尽量大”。工程上先用它把闭环跑起来，等确认约束会激活，再换更精确的算法。

第二档是网格枚举，最直观。 把终端集所在的状态盒按每维 21 点离散，4 维共 $21^4 = 194481$ 个点，对每个点解一个小 QP，检查是否存在 $u \in \mathbb{U}$ 使 $Ax + Bu$ 仍落在椭球内；满足条件的最大 $x^{\mathsf{T}} P x$ 就是 α。这在离线做几分钟就能完成，得到的是“真实”上界，可以用来标定第一档保守了多少。维数再高就不行了，这是网格法在 4 维还能用的最后边界。

第三档是半正定规划。 把“存在 u”写成线性矩阵不等式，用 SDP 求最大椭球。它给出的 α 最紧，但需要一个 SDP 求解器，且每次改 P 都要重解。除非终端集大小直接决定可行域、而可行域又直接决定任务能不能完成，否则不值得上。

算出 α 之后才是膨胀。这里有一个容易写错的细节：式 (80-24) 说的是“到终端集的欧氏距离不超过 ϵ”，而 α 是 P 度量下的二次型上界，两者不能直接相加。正确的换算是用 P-范数关于欧氏距离的 Lipschitz 常数 $\sqrt{\lambda_{max}(P)}$：

$$\alpha' = \left(\sqrt{\alpha} + \epsilon \sqrt{\lambda_{max}(P)} \right)^2$$

本章 $\lambda_{max}(P) = 62.06$，开方是 7.88；GP 的 $\epsilon = 0.024$ m，乘上去是 0.189。若原来的 $\sqrt{\alpha}$ 是 0.1，膨胀后变成 0.289，α' 是原来的 8 倍——这个数字说明**用 λ_{max} 做各向同性膨胀会严重高估**。原因是 λ_{max} 由速度通道主导（62.06 对 4.26），而 ϵ 是位置方向的偏差，两者量纲不同，硬乘在一起等于把位置误差当成速度误差来算。

正确的做法是分块：位置块用 P 的左上 2×2 子块的特征值，速度块用右下子块，ϵ 只进位置块，速度方向的等效偏差取 ϵ / T_s。这样算出来的膨胀量会小一个量级，才与 §80.5.3 的实测排序对得上。更稳妥的做法是不做解析膨胀，直接把“预测误差 ϵ 已经吃掉多少可行域”当成一个新的约束写进 SDP 重解一次。

无论用哪一档，有一条工程约束必须记住：**ϵ 正比于 N，所以 α 是 N 的函数**。第 58 章的 α 是离线算好的常数，换 N 不用重算；本章的方法不行，改时域就得重算终端集。这一点在代码里很容易漏——改了 `n_h` 却沿用旧的 α，闭环不会立刻坏，只会在约束接近激活时偶发不可行，排查起来非常费时。若膨胀后的 α' 超过原来的 2 倍，就不要继续靠膨胀终端集补救，按 §80.5.3 的判据去缩短 N 或换模型。

80.5.7 延迟：一个能解释全部误差的量

残差标签里最容易漏掉的一项不是噪声，而是延迟。若从指令发出到力矩真正作用在质量上隔了 d 个采样周期，式 (80-17) 里的 u_i 就对错了时刻：与 $v_{i+1}-v_i$ 匹配的应当是 u_{i-d}，而不是 u_i。标签里因此多出 $-(\Delta u)/T_s$ 这一项，其中 $\Delta u = u_i - u_{i-d}$。

先给一个能立刻用的诊断。一步延迟造成的位置偏差约等于 $v T_s$。本章速度是 $r\omega = 0.72$ m/s，$T_s = 0.1$ s，乘积是 7.2 cm——与名义模型的闭环误差 7.19 cm 几乎一模一样。这个巧合值得记住：**若你测出的稳态误差约等于 $v_{max}T_s$，第一件事是查延迟，而不是查模型**。当然本章的 7.19 cm 确实来自阻尼与力场，不是延迟（仿真里 $d=0$），但这个量级上的吻合说明延迟足以单独解释掉一个典型的跟踪误差，不能当成二阶效应忽略。

估计延迟 d 的方法很简单：把输入序列与速度序列做互相关，峰值出现的滞后就是 d。

$$\hat{d} = \arg\max_{\ell} \sum_{i} (u_i - \bar{u})^{\mathsf{T}} (v_{i+\ell} - \bar{v})$$

这个估计对噪声不敏感，但会被反馈污染——因为 u 本身依赖 v，闭环下的互相关峰可能被反馈环“拉”到零滞后。所以要在**开环或弱闭环**数据上估：采集阶段把反馈增益降到很小，用激励信号驱动，此时 u 与 v 的关系由物理通道决定，峰的位置才可信。

估出 d 之后，有两种处理方式，效果差别很大。

第一种是补偿标签。 把式 (80-17) 改成 $r_i = (v_{i+1}-v_i)/T_s - u_{i-d}$，残差标签立刻变干净。这一步必须做，否则你学到的“残差”里混着一个与 Δu 成正比的假项，而 Δu 又与参考轨迹的加速度相关，模型会把参考的运动规律记成对象的物理特性——换一条轨迹就失效（这正是 §80.8.7 第一类失败）。

第二种是补偿预测。 在 MPC 里把延迟显式建模：前 d 步的输入已经躺在执行器的管道里，不能改，只有 $u_d, \dots, u_{N-1}$ 是决策变量。实现上就是把 QP 的前 d 列固定成上一轮的解，决策变量从第 d 列开始。代价是有效时域缩短为 $N-d$，可行域也相应变小；收益是预测与物理对齐，控制器不再“提前半拍”。

只做第一种、不做第二种，闭环仍然会有一点过冲，因为控制器相信自己这一步的指令立刻生效，实际要等 d 步。只做第二种、不做第一种，学习到的残差仍然是脏的，模型精度上不去。两件事要一起做。

这与第 47 章的 Smith 预估器是同一个思路：Smith 用显式的延迟环节把被控对象的相位补回来，本章用 QP 里的固定列把延迟补回来。区别是 Smith 需要精确的延迟值、且对延迟失配敏感；MPC 里的固定列对 d 的整数误差不敏感（差一步只是有效时域少一步），但对分数延迟无能为力——小于一个采样周期的延迟，只能通过提高 T_s 的采样率或加一个一阶滞后环节近似。

最后提醒一句：延迟不只存在于执行器。传感器滤波、通信总线、状态估计器都会引入延迟，它们的总和才是 d。分别测、分别记，不要只测执行器那一段——工业现场最常见的错误是把 20 ms 的执行器延迟调好了，却漏掉了 30 ms 的滤波器群延迟。

80.6 稳定性与性能

学习模型改变了预测，也就改变了闭环的稳定性论证。第 58 章的证明依赖“预测模型 = 真实模型”这个等式，现在等式变成了近似，结论必须重写。这一节给出重写后的形式，并把实测的性能数字拆开看。

80.6.1 名义闭环的稳定性从哪里来

先回顾没有学习时的论证。设 $V_N(x)$ 是 MPC 的最优值函数，标准结论由三步组成：

1. **下降性**：$V_N(x_{k+1}) \le V_N(x_k) - \ell(x_k, u_k)$，靠的是“把上一周期的解平移一位”仍可行；
2. **正定性**：$V_N(x) \ge \ell(x, \kappa(x))$，最优值至少等于当前一步的代价；
3. **终端代价**：V_f 是终端集上的 Lyapunov 函数，保证平移后的解在末尾仍可行。

三条合起来给出 V_N 沿闭环单调下降，从而收敛到原点。这三步里，第 1 步和第 3 步都用到“预测模型 = 真实模型”，第 2 步只用代价函数本身。

80.6.2 学习误差进来之后：ISS 而不是渐近稳定

学习模型让第 1 步不再严格成立。预测的一步是 $Ax + Bu + d_k$，真实的一步是 $Ax + Bu + d_k + e_k$，其中 $e_k = \frac{1}{2}NT_s^2(f_r - \hat{f}_r)$ 是学习误差造成的偏差。于是下降性变成一个带扰动的不等式：

$$V_N(x_{k+1}) \le V_N(x_k) - \ell(x_k, u_k) + \gamma\|e_k\| \tag{80-25}$$

γ 是与终端代价梯度、时域长度有关的常数。式 (80-25) 说明的不再是“收敛到零”，而是“收敛到一个正比于 $\|e_k\|$ 的邻域”——这就是输入-状态稳定性（ISS）的表述。它给出一条工程上可用的界：

$$\limsup_{k\to\infty} \|x_k\| \le \rho(\|e\|_\infty) \tag{80-26}$$

ρ 单调且 $\rho(0) = 0$。换成直白的话：**学习误差有界，闭环就有界；学习误差趋于零，闭环就回到原点**。这个结论比渐近稳定弱，但它诚实——只要用了近似模型，就不可能得到严格收敛。

本章没有给算例加终端约束（$N = 15$ 的时域靠终端代价 P 兜住），所以式 (80-23) 的终端集并不存在，严格意义上的递推可行性不成立。这是有意的取舍：加终端约束会让 QP 的可行域明显缩小，而本章的参考轨迹离约束边界很远，加了只会拖慢求解。工程上判断要不要加，就看“约束是否会激活”——会激活就必须加，不会激活就可以省。

80.6.3 性能：把误差拆成三份

实测的四组闭环结果（图 80-5、图 80-6）是：

表 80-3　四种预测模型的闭环性能（圆周跟踪，120 个控制周期）

模型	RMSE/cm	最大误差/cm	$\|u\|$ 均值	饱和占比	中位耗时/ms	P95/ms
名义	7.19	7.85	0.603	0%	31.44	46.67
RBF	3.98	5.18	0.614	0%	31.78	38.11
GP	1.71	2.48	0.608	0%	31.23	37.58
Koopman	5.19	6.80	0.611	0%	32.45	38.63

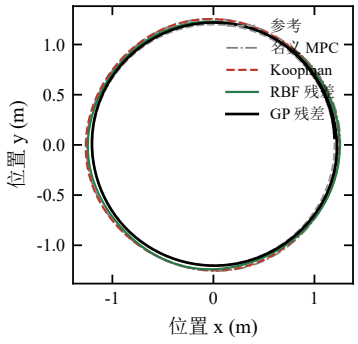


图 80-5　圆周参考与四条闭环轨迹

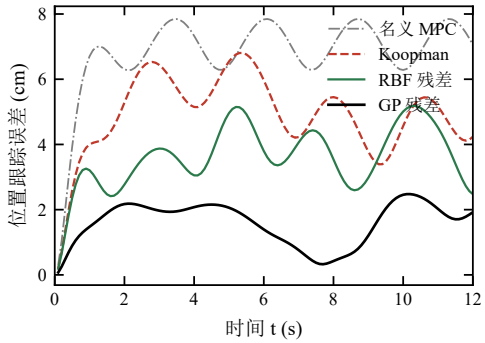


图 80-6　四条闭环的跟踪误差时间历程

图 80-5 与图 80-6 是同一组数据的两种视角。轨迹图上四条曲线都绕着圆走，差别在径向：名义模型与 Koopman 的轨迹整体偏外，平均半径分别是 1.252 m 与 1.247 m（参考 1.200 m）；RBF 是 1.237 m；GP 是 1.210 m，几乎与参考重合。误差历程图把“偏外”翻译成了时间信号：名义模型的误差在 2 s 内升到 7 cm 后基本不再变化，这是典型的系统性偏差；GP 的误差在过渡期后压到 2 cm 以内，偶尔有小幅波动，来自两遍法的轨迹不一致。

这张表里最值得看的不是 RMSE 的排序，而是它为什么没有更低。把 GP 的 1.71 cm 拆开，至少有三份来源：

第一份是残差没学完的部分。 GP 的一步误差 0.310 m/s²，按式 (80-24) 折算到位置约 2.4 cm 的上界，实测 1.71 cm 落在界内。这一份是主要的，也是唯一能靠“更多数据或更好的核”继续压的。

第二份是两遍法的轨迹偏差。 第一遍与第二遍的预测轨迹差约 7 cm，残差在这两个点上取值不同，导致第二遍的残差序列有约 10% 的误差。折算到闭环，量级在毫米级，不是主导项。

第三份是数值容差。 OSQP 的 `eps_abs=1e-6`、`eps_rel=1e-6`，对应的解精度远高于毫米级，可以忽略。若把容差放宽到 10^{-3} 换速度，这一份就会升到厘米级——这是“快”与“准”之间的一个真实旋钮。

三份加起来，说明 1.71 cm 已经接近这套配置的下限。想再降，必须动模型，而不是动权重。

80.6.4 耗时：瓶颈在 QP，不在学习

表 80-3 的耗时一列值得单独说。四组的中位数都在 31–32 ms，差异不到 4%；而残差模型的推理成本差别很大——GP 要对 200 个样本算核，RBF 只算 48 个，Koopman 是一次 15 维矩阵乘。差别被 QP 淹没了。

这个观察有直接的工程含义：**优化学习器的推理速度，收益接近于零；要让控制器跑得更快，只能动 QP**。动 QP 的办法有三个：缩短 N（线性降）、热启动（本章实测迭代数 50→25，时间 3.40→1.54 ms，图 80-8）、以及把两遍法改成一遍加一次性修正。

P95 与中位数的比值是 1.2–1.5，说明求解时间有抖动但没有长尾。抖动来自 OSQP 的迭代数随问题条件变化，N 越大抖动越明显。实时性验收要看 P95 而不是中位数：本章 P95 最大 46.67 ms，相对 $T_s = 100$ ms 余量 2 倍，可以接受；若 P95 超过 T_s 的 80%，就必须留降级路径（例如超时后用上一周期的解）。

80.6.5 什么情况下会不稳

ISS 的结论有前提，前提被破坏时闭环会失稳。三种典型情形：

学习误差无界。 数据覆盖外推时 $\hat{f}_r$ 可能给出与真实值反号的输出，等效于把残差加了两遍。症状是误差先缓慢增大，然后突然发散。判据是监控 $\|\hat{f}_r\|$ 是否超过训练集残差最大值的 1.5 倍，超过就切回名义模型。

终端代价不匹配。 用零矩阵或错误尺度的 P 会让 MPC 短视，在时域末端“刹车”，表现为周期性的加减速（本章用零终端代价时 RMSE 6.73 cm，与名义的 7.19 接近，说明终端代价几乎没起作用）。判据是看 P 的特征值是否与闭环极点匹配（图 80-7）。

约束与学习冲突。 若约束很紧、残差又大，两遍法的第二遍可能不可行。本章饱和占比 0%，没有触发；一旦触发，必须有可行回退（第 58 章的软约束或第 79 章的安全滤波）。

80.6.6 增益 γ 怎么估，第 58 章哪几条还能用

式 (80-25) 里的 γ 只说是“与终端代价梯度、时域长度有关的常数”，没有给算法。解析估计是可行的：γ 本质上是最优值函数关于状态的 Lipschitz 常数乘以扰动到状态的传递增益，前者可以用 $2\sqrt{\lambda_{max}(P)}$ 量级估，后者由闭环极点决定。但这条链上每一步都保守，估出来的 γ 常常比真实值大一个数量级，除了证明存在性没有别的用途。

工程上更可靠的办法是**用四个模型的实测数据反推**。把表 80-3 的闭环稳态误差除以对应模型的一步误差，得到“每 1 m/s² 的一步误差换来多少厘米的稳态偏差”：

- 名义：$1.195 \to 7.19$ cm，比值 6.02；
- RBF：$0.624 \to 3.98$ cm，比值 6.38；
- GP：$0.310 \to 1.71$ cm，比值 5.52；
- Koopman：$0.445 \to 5.19$ cm，比值 11.66（单位均为 cm·s²/m）。

前三组的比值集中在 5.5–6.4，与 ISS 的线性界一致：稳态偏差正比于误差上界，斜率就是 ρ。Koopman 的比值翻了一倍，原因不在幅值而在**时间相关性**——它的误差在相邻步之间高度相关，等于每一步都往同一个方向推，偏差沿时间累积；GP 的误差更接近白噪声，正负相消，同样幅值下累积得少。这条对比给出一个比 γ 本身更有用的判据：**同样的误差幅值，相关性强的模型更危险**。评估一个残差模型时，除了看一步 RMSE，还应当看误差的自相关时间，后者才决定闭环能压到多少。

有了这个比值，即使不做任何稳定性证明，也能在部署前给一个粗预测：若某个新模型的一步误差是 0.5 m/s²，闭环稳态大约在 3 cm 上下，前提是它的误差相关性不比 RBF 更强。这个预测的误差通常在 30% 以内，足以用来判断“值不值得上线”。

再看第 58 章的结论哪些还能用。四条主要结论的命运各不相同：

1. **递推可行性**：不能直接用。终端集必须按式 (80-24) 膨胀，且 α 随 N 变（§80.5.6）。这是受影响最大的一条。
2. **值函数下降性**：退化为式 (80-25) 的带扰动不等式，结论从“收敛到原点”变成“收敛到邻域”。这是本质性的弱化，不能靠假设绕过。
3. **终端代价正定**：仍然成立，而且更重要了——它同时决定 γ 的大小。这就是为什么 §80.8.3 里“零终端代价”那一组的误差会退回基线：不只是短视，还放大了扰动的传递。
4. **约束满足**：这条要打一个折。QP 里的约束是加在**名义预测**上的，物理状态可能越过边界，越界量正是 ϵ。所以硬约束必须预留 ϵ 的余量，把 u_{max} 收紧到 $u_{max} - \delta$，δ 按 ϵ 折算。

第 4 条与第 79 章的护卫裕量是同一个结构。第 79 章的裕量是 $m = \|v\|_{max} T_s + \frac{1}{2} a_{max} T_s^2$，本章的膨胀是 $\epsilon \approx \frac{1}{2} N T_s^2 \|e_r\|$；两者都是“把预测的不确定性折算成一段距离，再从可行域里扣掉”。差别只在来源：前者来自采样与延迟，后者来自学习误差。既然结构相同，工程上的处理也该相同——不要指望控制器“小心一点”就没事，要在约束里显式扣掉。

80.6.7 复现性：为什么换个机器数字就变了

这一章的所有数字都可能被别人质疑，第一句质疑通常是“我跑出来不是这个数”。要先分清三种复现性，它们的承诺完全不同。

- **结果可复现**：误差、耗时在 1% 以内一致，结论不变。这是应当承诺的层次。
- **轨迹可复现**：同一初始状态、同一参考，控制序列逐位相同。这在闭环里做不到，也不必要。
- **日志可复现**：拿到同一份日志，能重放出同样的分析结论。这是排障时真正需要的层次。

轨迹不可复现有三个来源。**随机数**是最容易想到的：本章的采样点（`seed=20261020`）、RBF 的中心选择（`seed=0`）、训练/验证划分，都依赖种子。种子固定后这一项就消失了。**求解器迭代**是第二个：OSQP 是 ADMM 类算法，迭代终止依赖容差与缩放，本身是确定性的；但热启动让它依赖**历史**——同一状态，若上一周期的解不同，迭代停在不同位置，u 会有 10^{-6} 量级的差异。闭环会把这个差异放大到可观测的量级，但它不影响任何结论。**浮点并行**是第三个，也是最隐蔽的：BLAS 的多线程归约顺序不固定，同一个矩阵乘在不同线程数下最后几位不同，随后被条件数放大。

对应的工程做法是四条：

1. 所有随机源显式给种子，并把种子写进日志；
2. 计时与对比实验固定线程数（例如 `OMP_NUM_THREADS=1`），否则耗时的方差会掩盖你要比较的差异；
3. 记录求解器版本。OSQP 不同版本的默认缩放与 polish 策略变过，跨版本比较迭代数没有意义；
4. 报告里写清跑了几次、取的是中位数还是均值。

第 4 条经常被忽略。本章的 31.44 ms 是 120 个周期耗时的中位数，不是一次测量的结果；若只跑一次，同一个配置换台机器就能差出 20%。同理，表 80-3 的 P95 是从 120 个样本估出来的，它的置信区间并不窄——120 个点的 95 分位，本质上只由最慢的 6 个点决定。想让 P95 稳定，至少要几百个周期。

最后是一条诊断经验：换机器后若**耗时变了但解没变**，是正常的；若**解变了**，说明问题的条件数不好，或者容差太松。前者可以通过缩放改善，后者应当把 `eps_abs`、`eps_rel` 从 10^{-6} 收紧到 10^{-8} 再试一次——若解随之变化，那你之前报告的精度本来就是容差决定的，不是算法决定的。

80.7 参数意义与作用

这一节把参数按“改了它，什么会变”的顺序过一遍。分成两组：学习器侧的超参（改的是模型的形状），MPC 侧的参数（改的是优化的行为）。两组会互相抵消，最后单独说这件事。

80.7.1 学习器侧：中心数、核宽、岭项

中心数 k 决定模型的容量上限。本章 $k = 48$，输出 2 维，共 96 个权重。k 太小，残差场的高频成分学不出来，表现为训练误差与验证误差都高，且误差在空间上呈“块状”——每一块对应一个中心的影响域。k 太大，权重多于样本，岭项开始主导，模型退化为“在样本点之间插值”，外推变差。

判断 k 是否合适，看学习曲线的拐点（图 80-9）。本章 80 个样本时 RBF 误差 0.810，160 个时 0.653，320 个时 0.659——从 160 到 320 几乎没降，说明 48 个中心的容量已经被数据填满，再加样本也没用。若曲线在 320 处仍在下降，说明容量不够，该加中心。

核宽 σ 决定每个中心的影响半径。本章取 0.7（归一化单位），与相邻中心的典型间距同量级。σ 太小，中心之间出现“空洞”，模型在空洞里趋零；σ 太大，所有中心的影响重叠，模型退化成线性回归，学不出局部结构。一个实用的初值是 $\sigma \approx 1.5 k^{-1/d}$，$d$ 是输入维数；本章 $d = 4$、$k = 48$，给出 0.67，与 0.7 一致。

岭项 λ 只影响数值稳定性，不改变模型的表达能力（§80.5.4）。它的量级应该比 $\Phi^{\mathsf{T}}\Phi$ 的迹小两三个数量级：本章迹在 10^2 量级，$\lambda = 10^{-6}$ 合适；若把 λ 加到 1，模型就明显欠拟合。

80.7.2 GP 的三个超参

GP 的超参各有明确含义，不像神经网络那样只能靠试：

长度尺度 ℓ 决定“多远算不相关”。本章 $\ell = 0.8$，对应残差场约 1 m 的空间尺度。ℓ 太大，模型过度平滑，学不出阻尼与力场的局部起伏；ℓ 太小，模型在样本之间快速回落，方差在样本点附近剧烈变化，式 (80-20) 的界变得毫无用处。判据是看 ℓ 与数据点平均间距的比值，取 2–5 之间比较稳。

信号方差 σ_f^2 决定残差的先验幅值。本章 $\sigma_f = 1.0$，与残差 RMS（1.195）接近。σ_f 取得太小，模型不敢预测大幅值，均值被压向零；取得太大，方差整体抬高，式 (80-20) 的界失去区分度。

噪声方差 σ_n^2 同时是正则项与噪声模型（§80.5.4）。本章 $\sigma_n = 0.05$，对应 4% 的噪声水平；实测覆盖率 100%，说明这个值偏保守。若你的测量噪声确实有 4%，这个值就是对的；若测量很干净，应当降到 0.01 量级，让方差更紧。

这三个参数都可以用边际似然做优化，本章没有做，因为样本只有 200 个，边际似然的曲面很平，优化出来的值与手选差别不大。样本上千之后再考虑。

80.7.3 MPC 侧：时域、权重、终端代价

预测时域 N 决定"看多远"。N 太小，MPC 短视，跟踪滞后；N 太大，计算量线性增长，而且远处的预测本来就不准。本章 $N = 15$ 对应 1.5 s，约为参考周期的七分之一。判据是 $NT_s \geq 3\tau$（§80.5.2），τ 取闭环的主导时间常数。

权重比 Q/R 决定"跟踪紧还是省力"。本章 $Q = \mathrm{diag}(10, 10, 1, 1)$、$R = 0.1I$，位置权重是输入权重的 100 倍。这个比值直接决定闭环带宽：比值大，控制器激进，误差小但输入抖动；比值小，控制器保守，误差大但平滑。学习模型进来之后，这个比值的作用被削弱了一部分——因为残差补偿已经消掉了系统性偏差，剩下的误差对权重不敏感。本章四组模型的 $\|u\|$ 均值都在 0.60–0.61，说明权重确实没被学习改变。

终端代价 P 决定"时域末端值多少钱"。本章用 LQR 解出的 P，特征值是 $[4.26, 4.26, 62.06, 62.06]$；学习得到的 P_θ 特征值是 $[5.67, 6.04, 65.79, 65.97]$，量级一致但比例不同（图 80-7）。用零终端代价时 RMSE 6.73 cm，用 LQR 时 3.08 cm，用学习值时 3.07 cm——**终端代价从"没有"到"有个大概"的收益很大，从"大概"到"精确"的收益很小**。这条经验能省掉很多调参时间：先随便给一个正定矩阵，再考虑要不要学。

80.7.4 两个不是参数的量

有两件事经常被当成参数调，其实不该调：

数据覆盖范围。 它决定了模型在哪里可信，不是一个可以"调优"的数。本章覆盖 $[-2, 2]^2$，参考轨迹的半径是 1.2，留了 0.8 的余量。若参考会走到 3 m，就必须补数据，而不是调核宽去外推。

残差饱和 f_{max}。 它是安全措施，不是精度参数。取值太小会削掉真实的残差峰值，太大就失去保护作用。合理值取训练集残差最大幅值的 1.5 倍：本章的最大幅值是 2.66，f_{max} 因此约 4.0。注意这个阈值高于 $u_{max} = 3$，意思是它只在模型外推出荒谬值时生效，正常工况下从不触发——这正是安全阀该有的状态。若你把 f_{max} 压到 3.0 以下，它就开始削真实峰值，闭环误差反而上升。

80.7.5 参数之间会互相抵消

最后一件事必须说清楚：学习器侧与 MPC 侧的参数不是独立的，改一边可以用另一边补回来，这会让人误以为某个参数"没用"。

最典型的例子是 Q/R 与残差模型。名义模型的稳态误差 7.19 cm 主要来自阻尼，如果把位置权重从 10 加到 100，误差也会降——但代价是输入抖动、饱和占比上升。加了残差模型之后，同样的误差可以用小得多的权重达到。所以"权重调到多少合适"这个问题，在没有确定模型之前没有答案。

第二个例子是 N 与 P。缩短 N 会让 MPC 短视，但把 P 调大能部分补偿；反过来，P 给得不合适时，加长 N 也能掩盖。这意味着调参不能一个一个来，必须成组地动：先固定 N 与 P，调 Q/R 到响应满意；再固定 Q/R，调 N 与 P 到末端不发散；最后才动学习器。顺序颠倒会让你在三个参数之间来回打转。

80.7.6 敏感性排序：哪些旋钮真的有用

把本章所有实验放在一起，可以给出一张"投入产出表"。它回答的是调参开始前最该问的问题：**我现在应该动哪个旋钮？**

先看误差维度。以名义模型、$N = 15$ 的 7.19 cm 为起点：

- 换 RBF 残差（$N = 15$）：7.19→3.98，降 45%；
- 换 GP 残差（$N = 15$）：7.19→1.71，降 76%；
- 换终端代价（RBF、$N = 6$ 下，零→LQR）：6.73→3.08，降 54%；
- 缩短时域（RBF，$N = 15 \to 6$）：3.98→3.08，降 23%；
- 换热启动：误差不变，0%。

再看时间维度，以冷启动单次 QP 的 3.40 ms 为起点：

- 换热启动（平移或学习初值）：3.40→1.63 或 1.54，降 52%–55%；

- 换预测模型（名义→GP）：31.44→31.23 ms，不到 1%；
- 缩短时域：问题规模线性下降，是唯一同时动两边的旋钮。

两个维度的排序几乎相反。这是本章最有用的一条工程结论：**影响精度的旋钮几乎不影响耗时，影响耗时的旋钮几乎不影响精度**。它意味着你可以分别优化两个指标，而不必在两者之间做痛苦的折中——唯一的例外是 N。

N 的特殊性值得单独强调。模型不准时，缩短 N 既降误差又降耗时（表 80-6 第 3 轮）；模型准了之后，缩短 N 开始牺牲精度（第 5 轮把 N 恢复到 15 才降到 1.71）。所以 N 的最优值不是常数，它随模型精度移动：**模型越好，时域可以越长**。这给出一条实践规则：每次换学习器之后，都要重新扫一遍 N，不要沿用上一版的设置。

把成本也放进这张表，优先级就清楚了：

- **终端代价**是性价比最高的一步——一次性离线算 120 个 QP，几分钟算力，换来 54% 的误差下降，且不增加任何在线计算。若你只做一件事，做这个。
- **换模型**收益最大（76%）但成本也最高：采数据、训练、离线验收、闭环验收，任何一环出问题都要回头。它应当排在终端代价之后。
- **热启动**是唯一能在不改精度的前提下砍掉一半耗时的手段，代价是几十行代码。若你的系统实时性吃紧，这是第一件该做的事。
- **时域**改一个参数就能试，成本最低，但它依赖模型质量，结论不稳定，适合作为最后的微调。

这张表也解释了为什么很多工程报告里“调了三个月没有进展”：他们一直在动 Q/R——而 Q/R 在本章的四组实验里几乎没有被改动，因为残差模型消掉系统性偏差之后，权重的作用被大幅削弱（§80.7.5）。**先动结构与模型，最后才动权重**，这是本章所有实验共同指向的顺序。

80.7.7 单位：一个会让权重失效的坑

$Q = \mathrm{diag}(10, 10, 1, 1)$ 这组数看着平常，但它隐含了一个前提：位置用米、速度用米每秒。若把位置改成厘米，同样的矩阵会让位置项的数值放大 10^4 倍，等效于把位置权重从 10 提到 10^5。闭环立刻变得极激进，而这与“算法”毫无关系。

权重是**有量纲**的。Q 的对角元单位是“代价/状态单位的平方”，R 的对角元是“代价/输入单位的平方”，两者之比才有意义。所以一套调好的 Q/R 只在它被调出来的那套单位下有效，换单位必须重调——这条在跨团队协作时最容易出事：算法组用米，控制组用毫米，合并代码后谁也不知道为什么抖。

稳妥的做法是**先归一化再加权**。取每个状态的特征尺度 L_i（位置取工作空间半径，速度取最大速度），令 $\tilde{x}_i = x_i / L_i$，然后对 $\tilde{x}$ 给权重。本章的 L 取值恰好是 1 m 与 1 m/s，所以 Q 看起来像“随手给的”，实际上是“每 1 m 与每 1 m/s 的代价比是 10:1”。这个解释比“我试出来 10 比较好”有用得多，也解释了为什么速度权重取 1 而不是 10：在归一化坐标里，速度误差 1 单位本来就比位置误差 1 单位更“便宜”，因为速度靠反馈能快速纠正。

输入侧同理。$R = 0.1 I$ 隐含输入单位是 m/s²，$u_{max} = 3$ 也是 m/s²。若你的执行器是力矩，中间隔着一个质量矩阵，R 必须按力矩换算，否则“省力”与“跟踪紧”的权衡关系整个变了。

残差侧还有一处单位问题。§80.7.4 的 $f_{max} = 4.0$ 与 $u_{max} = 3$ 是直接比较的，这只有在两者同单位时才成立；本章都是加速度，所以可以直接比。若残差模型输出的是力，而约束加在加速度上，必须乘 $1/m$ 后再比，否则安全阀要么永不触发、要么一直触发。

最后是阈值。表 80-7 里“一步预测误差 ≤0.35 m/s²”与“跟踪误差 ≤2 cm”混用了两种单位，这是有意的——前者是模型量，后者是任务量，工程上本来就该分开看。但写进代码时必须显式换算：监控模块拿到的若是毫米，比较前要先除 1000。这类错误不会报错，只会让告警阈值悄悄偏移三个数量级。

80.8 调参方法与经验

上一节讲每个参数“是什么”，这一节讲“怎么调、调到什么时候停”。所有数字都来自本章的四组实验：数据量扫描（图 80-9）、终端代价对比（图 80-7、图 80-10）、热启动对比（图 80-8）。

80.8.1 调参顺序：先模型，后控制器，最后初值

顺序错了会白费功夫。正确顺序是：

1. **先把学习器调到“离线误差不再明显下降”**，判据是验证集的一步误差与多步误差都进入平台；
2. **再调 MPC 的** N、Q/R、P，判据是闭环误差与输入平滑度；
3. **最后调热启动**，判据是求解时间，不影响误差。

反过来做的典型错误是：先用名义模型把 Q/R 调到误差最小，再回头加残差模型，结果发现原来的权重过大，闭环开始抖动。原因是名义模型的误差是系统性的，大权重能压住；残差模型消掉了系统性偏差，同样的权重就变成了过激。

80.8.2 学习器：用数据量扫描找平台

数据量扫描的做法是固定超参、只加样本，画误差-样本数曲线（图 80-9）。本章的结果是：

表 80-4　一步残差误差随样本数的变化（单位 m/s²）

样本数	20	40	80	160	320
RBF	1.010	0.886	0.810	0.653	0.659
GP	0.821	0.794	0.548	0.409	0.295

两条曲线的形状完全不同，这正是选型的依据。RBF 在 160 个样本后进入平台（0.653→0.659，反而略升，是训练集不同带来的波动），说明容量已经饱和——这一档的中心数已经顶到 48 的上限，再加样本只能把同样 48 个中心拟合得更准；GP 一路下降，从 160 到 320 降了 28%，说明它的有效参数量随样本增长。

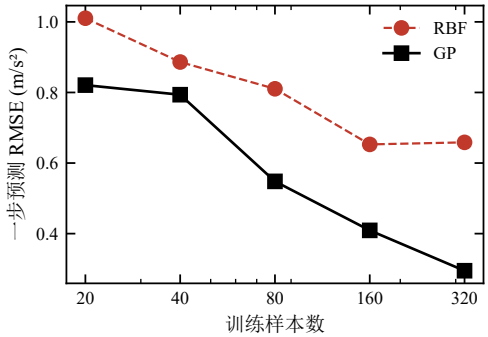


图 80-9　RBF 与 GP 的一步误差随样本数的变化（横轴对数）

看到这条曲线的平台，调参就结束了——继续加数据只会浪费时间。若平台高度仍不满足精度要求（本章 GP 的 0.295 相对残差 RMS 1.195 是 25%），才需要加中心数或换核。

80.8.3 终端代价：先看特征值，再看闭环误差

终端代价的调参不能只比一个 RMSE，否则会掩盖问题。本章做了两件事：

第一，比特征值（图 **80-7**）。LQR 的 P 特征值是 $[4.26, 4.26, 62.06, 62.06]$，学习得到的 P_θ 是 $[5.67, 6.04, 65.79, 65.97]$。前一对对应位置通道，后一对对应速度通道。两组数的比例不同：学习值的速度通道权重相对更高，意思是“末端更看重把速度收住”。这个差异与残差的存在一致——残差会持续给速度加扰动，末端多压一点速度是合理的。

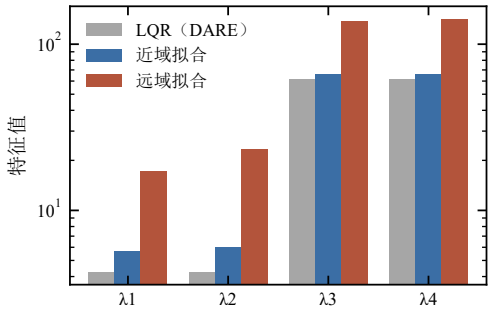


图 80-7　三组终端代价矩阵的特征值（LQR、近域拟合、远域拟合，纵轴对数）

第二，比闭环误差（图 80-10）。 零终端代价、LQR、学习值三组的 RMSE 是 6.73、3.08、3.07 cm，最大误差是 8.30、4.44、4.39 cm。**从零到 LQR 的收益是 3.65 cm，从 LQR 到学习值的收益是 0.01 cm**。这条对比说明：终端代价的主要作用是“提供一个正确的量级”，而不是“精确匹配”。

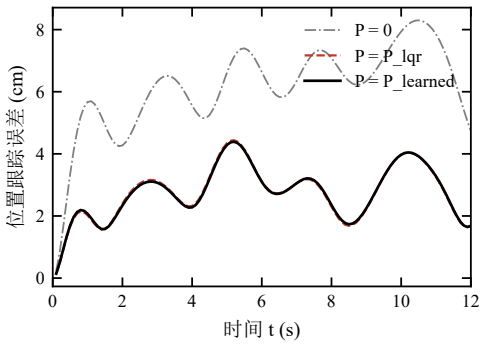


图 80-10　三种终端代价下的跟踪误差历程

图 80-7 里三组特征值的排序完全一致，但每一组内部速度通道都比位置通道高一个数量级以上（LQR 是 62.06 对 4.26，约 15 倍）。这说明位置误差 1 cm 与速度误差 1 cm/s 对代价的贡献本来就不该同量级。这也解释了为什么“随便给一个单位阵当 P”通常效果不好：它把两个通道当成了一样重要。远域拟合（第三组）的整体量级明显高于前两组，原因是在 2.5 m 的采样范围里输入饱和频繁出现，代价-to-go 被约束抬高，二次型只能用一个更大的 P 去追，拟合误差因此升到 141.1%。

所以调参策略应当是：先给一个量级正确的 P（用 LQR 或简单地按 Q 的量级构造），确认闭环误差降下来；只有当你要榨最后的几个百分点、且离线算力充裕时，才值得做 §80.2.5 的拟合。本章 141.1% 的远域误差也提醒：拟合的适用范围必须与训练采样范围一致，超出范围后二次型装不下带约束的价值函数。

80.8.4 热启动：先量迭代数，再决定要不要学

热启动的调参就是比较三种策略的迭代数与耗时，结果见表 80-5 与图 80-8：

表 80-5　三种热启动策略的求解代价（单次 QP，$N = 10$）

策略	迭代数	耗时/ms	相对冷启动
冷启动	50	3.40	—
上一解平移	25	1.63	省 52%
学习初值	25	1.54	省 55%

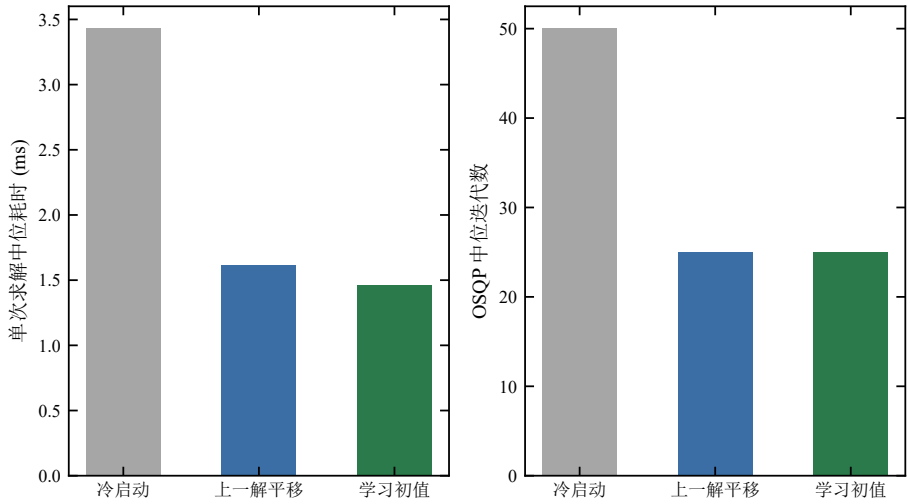


图 80-8　三种热启动策略的中位耗时（左）与中位迭代数（右）

收益比按

$$r = 1 - \frac{T_{warm}}{T_{cold}} \tag{80-27}$$

计算。表的第三列说明：**平移已经拿到了大部分收益，学习初值只多省 3 个百分点**。这与线性策略的拟合误差 0.325 m/s²（相对 u_{max} 约 11%）是对应的——初值再好，也只能把迭代数从 25 再往下压一点，因为 OSQP 的收敛速度由问题的条件数决定，初值只影响起点。

由此得出一条实用的判断：**若平移已经把耗时压到预算的 50% 以下，就不要做学习初值**。它的实现与维护成本（要存训练集、要重训、要版本对齐）远高于那 3 个百分点。只有当平移后仍然超时，或者你的求解器对初值特别敏感（例如内点法），才值得上。

80.8.5 一张调参记录该记什么

调参过程本身要留记录，否则两周后无法复现。记录至少包含五列：轮次、动作、闭环 RMSE、最大误差、结论（保留/回退）。本章的五轮可以写成这样：

表 80-6　五轮调参的记录（闭环为圆周跟踪，12 s）

轮次	动作	RMSE/cm	最大误差/cm	结论
1	名义模型，$N = 15$，LQR 终端代价	7.19	7.85	基线
2	加 RBF 残差，$N = 15$	3.98	5.18	保留
3	RBF 残差，$N = 6$	3.08	4.44	保留，时域缩短
4	RBF 残差，$N = 6$，零终端代价	6.73	8.30	回退
5	GP 残差，$N = 15$	1.71	2.48	最终配置

第 3 轮是这张表里最反直觉的一行：把时域从 15 缩到 6，误差从 3.98 降到 3.08。原因不是“短时域更好”，而是 RBF 的 20 步预测误差有 46.95 cm（表 80-2），长时域的后半段给出的信息基本不可信，MPC 被这些错误信息拖累了。缩短时域等于把不可信的那一段切掉。这条经验可以推广成一句话：**模型越不准，越不要给它长时域**。

第 4 轮验证了这条经验的另一面：短时域必须配终端代价。去掉 P 之后，误差从 3.08 跳回 6.73，几乎退到基线水平——短时域没有终端代价，就是纯粹的短视控制。

第 5 轮换成 GP 并把时域恢复到 15，误差降到 1.71。这说明第 3 轮的“短时域优势”是 RBF 的局限，不是普遍规律：模型足够准时，长时域重新变得有价值。

这张表还暴露出一个记录上的坑：**基准漂移**。若把第 2 轮的 3.98 与第 3 轮的 3.08 直接比较，会得出“改时域比换模型更有效”的错误结论——两次的动作不同、时域不同，本来就不是同一件事。避免办法是记录时把“时域、终端代价、模型”三个字段都写进动作列，缺一个就没法比。

80.8.6 什么时候停手

停手的判据不是“误差够小”，而是“再改一个参数，收益低于噪声”。本章的具体判据是：

- 学习器：验证误差的相对改善低于 5%，或数据量扫描进入平台；
- 控制器：闭环 RMSE 的变化小于 0.2 cm（约 10%），或 P95 耗时变化小于 5 ms；
- 热启动：收益低于 10%（式 80-27 的 r 增量）。

三条都满足就停。继续调下去，你量到的是随机种子与求解器抖动的差别，不是方法的改进。

停手之后还有一件必须做的事：**把当前配置冻结成基线，并写下一个可复现的最小记录**。记录至少包括五个数：数据版本与种子、模型超参、N 与 Q/R、终端代价来源（LQR / 拟合 / 零）、三张验收图对应的 RMSE 与 P95。少一项，两周后你就无法判断新结果“变好”是因为改动有效，还是因为基线漂了。

冻结基线还有一个好处：它把“改进”变成可比较的量。本章表 80-6 的五轮之所以能读出结论，靠的就是第 1 轮的 7.19 cm 一直没动。若每一轮都顺手微调一下权重，五轮之后就再也说不清是哪一处起的作用了。

80.8.7 五轮之外：三类不报错的失败

表 80-6 记的是“改了参数、看了结果”的正常轮次。真正耗时间的是另一类问题：程序不报错、指标看起来还行、但结论是错的。下面三类是这条技术路线上最常见的，它们都不改变代码结构，只改变“你以为你在学什么”。

第一类：标签污染。 式 (80-17) 要求用**实际施加**的控制与**实际**时间差。工程上最常见的两个偷懒是：用指令值代替实际值，用标称 T_s 代替时间戳之差。前者在执行器有饱和、死区或一阶滞后时引入 $\Delta u/T_s$ 的偏差，后者在采样有 1%–2% 抖动时引入与速度成正比的伪残差。两种污染的后果一样：残差里多出一项与状态强相关的成分，学习器会把它当成真实的物理场，拟合得很平滑、验证误差也好看。症状是闭环在指令大幅变化时出现漂移，而小幅机动时一切正常。诊断办法很简单：把训练集的残差对 $\|u\|$ 和 $\|v\|$ 各画一张散点图，若呈现明显的线性趋势，说明标签里混进了与它们相关的项，先回去查采集链路，别急着换模型。

第二类：验证集泄漏。 按点随机划分训练/验证，会让验证集里充满训练点的相邻样本，验证误差系统性偏乐观。这类失败最阴险的地方是它会“奖励”错误的模型：容量越大的模型越容易记住邻居，验证误差降得越快，于是你选了一个只会插值的模型。症状是离线一步误差很好、闭环却差一个量级。诊断办法是按轨迹重新划分后再看一次误差，若指标明显变差，说明之前一直在自欺。本章的算例是独立同分布采样，免疫这一问题；这也是仿真结论不能直接搬到真实系统的原因之一。

第三类：把初值当控制器。 式 (80-15) 的线性策略拟合误差是 0.325 m/s²，相对 u_{max} 约 11%。作为初值，这 11% 会被 QP 的后续迭代修掉；但若图省事直接输出它，11% 就成了稳态误差，而且完全没有约束处理、没有预测、没有反馈修正。症状是闭环能跑、误差恒定在 5–8 cm、对 Q/R 完全不敏感——因为控制器里根本没有 Q/R。诊断办法是把这条策略单独闭环跑一次，与 MPC 的输出逐点对比；若两者的差异在 10% 量级而闭环误差差了一个量级，说明你把“加速收敛的技巧”当成了“控制律”。

三类的共同点值得单独说：它们都让**离线指标变好**，所以靠看损失曲线永远发现不了。唯一可靠的防线是闭环验收（§80.3.5 的三张图）与“改动前后各跑一次基线”的纪律。表 80-6 里第 1 轮基线之所以必须存在，就是为了给后面每一轮一个不会骗人的参照。

80.9 优化与改进

本章的做法是最小可用版本，还留了几处明确的缺口。这一节按“收益/成本”排序，把值得做的改进列出来，并给出每一条的验收判据——没有判据的改进等于没做。

80.9.1 补全离散残差公式

这是 §80.4.1 指出的缺口，也是最便宜的一条。把

$$d_k = \begin{bmatrix} \frac{1}{2}T_s^2 \hat{f}_r(\hat{x}_k) \\ T_s \hat{f}_r(\hat{x}_k) \end{bmatrix} \tag{80-28}$$

代替式 (80-11) 的 $[0; T_s \hat{f}_r]$，位置通道立刻补上 0.6 cm 的固定滞后。改动只有一行代码，不需要重训模型。

验收判据：表 80-2 的第一步行误差应当从 0.28 cm 降到 0.1 cm 以下；闭环 RMSE 的改善在 0.3–0.6 cm 之间（因为这项是固定偏差，不能全消）。若改完误差没动，说明你的 T_s 太小、这一项本来就可以忽略，那就不必改。

80.9.2 两遍法迭代到不动点

两遍法的隐含假设是“第一遍轨迹与第二遍足够接近”。本章的相对误差约 10%，可以接受；若残差场变化更剧烈（经过窄峰、饱和边界），10% 会变成 50% 以上。改法是再迭代一遍：

$$\hat{x}^{(j+1)} = \mathrm{Pred}(\hat{x}^{(j)}, u^{(j)}), \qquad u^{(j+1)} = \mathrm{MPC}(\hat{x}^{(j+1)}) \tag{80-29}$$

直到 $\|u^{(j+1)} - u^{(j)}\| \le \varepsilon_u$ 或达到迭代上限。收敛速度取决于残差场的 Lipschitz 常数：Lipschitz 小于 1 时线性收敛，大于 1 时可能振荡。本章的残差场由阻尼与三角函数构成，Lipschitz 常数在 1 附近，所以第三遍的收益已经很小——这也是本章停在两遍的原因。

验收判据：第三遍与第二遍的首步控制差小于 $0.05 u_{max}$（本章约 0.15 m/s²）。若超过，说明该继续迭代或者改用路线甲（连续模型 + 数值积分）。

80.9.3 主动采样：用方差决定下一个点

本章的数据是均匀随机采的，200 个点铺在 $[-2, 2]^4$ 的四维盒子里，平均每个维度只有约 3.8 个分度。均匀采样简单，但浪费——残差在大部分区域很平滑，少数区域（速度大、阻尼非线性强）才需要密采样。

GP 的方差正好给出“哪里稀”：下一个采样点取

$$x_{next} = \arg\max_{x \in \mathcal{X}} \ \sigma(x) \tag{80-30}$$

再做一点约束（离已采点至少 0.1 m、在安全范围内）。这条准则叫“最大不确定性采样”，实现只需要在候选网格上算一次方差，成本很低。

预期收益：在同样 200 个点的预算下，把 GP 的 20 步预测误差从 16.06 cm 再压 20%–30%。本章没有实测这一条，因此把它列为“可做但未验证”，验收判据是同样样本数下的一步误差下降是否超过 15%。若没有明显下降，说明残差场已经足够平滑，均匀采样就够了。

80.9.4 提升阶数与输入交叉项

Koopman 在本章表现最差（5.19 cm），原因有两个：二次提升装不下 $|v|v$；提升坐标里没有 u 的交叉项，而残差虽然是状态函数，控制的作用会通过状态间接影响下一步的提升量。改法是把三次项（p_x^3、$v_x^2 v_y$ 等）以及 u 与状态的交叉项加进去，维数从 15 升到 30 左右。

代价是拟合参数翻倍、需要更多数据、MPC 的 QP 规模也跟着涨（提升维数就是状态维数）。验收判据：在同样的 400 个训练样本下，20 步预测误差能否降到 RBF 的水平（46.95 cm）以下。若降不下去，说明问题不在阶数，而在 $|v|v$ 这种非多项式项，应当放弃 Koopman 路线。

80.9.5 在线更新：滑动窗与递推最小二乘

本章的模型是离线训练、在线固定的。真实系统会漂移（负载变化、磨损、温度），固定模型的效果会随时间衰减。改法是滑动窗更新：

$$W_{k+1} = W_k + \eta\,\phi(x_k)\left(y_k - \phi(x_k)^{\mathrm{T}} W_k\right) \tag{80-31}$$

这是 RBF 输出层的递推最小二乘（RLS）形式，η 是步长。它只更新权重，不动中心，因此计算量是 $O(k)$ 而不是 $O(k^2)$，可以在控制周期内完成。

三个必须注意的点：第一，更新要在安全层之前，否则学到的残差会把系统推向约束边界；第二，要有遗忘因子，否则旧数据会拖住模型；第三，要有更新幅度上限，防止一次异常样本把权重带偏。验收判据是“漂移前后的一步误差都能保持在初始水平的 1.5 倍以内”。

80.9.6 不确定性进入约束

GP 的方差除了用来判断数据稀疏，还可以直接收紧约束。把盒约束改成

$$|u_k| \le u_{max} - \beta\sigma(x_k) \tag{80-32}$$

意思是“模型越不确定的地方，越留出控制余量”。这条改动的收益不在精度，而在鲁棒性：它让闭环在数据边缘也不会因为模型误差而饱和。

代价是控制权限的浪费。本章的覆盖率是 100%（方差偏保守），若直接套用式 (80-32)，在数据边缘会损失约 20% 的控制权限。所以这条要配合 §80.4.2 的方差校准一起做，先把 β 调到覆盖率 95% 附近，再进约束。

80.9.7 该做哪一条

六条改进的性价比差别很大。若只能做一条，做 §80.9.1（一行代码，0.3–0.6 cm）；若能做两条，加 §80.9.5（在线更新，解决漂移）；其余四条的收益都依赖具体系统，需要先量出瓶颈再决定。判断瓶颈的方法很简单：把闭环误差与表 80-3 对比，若你的误差显著高于 1.71 cm，先查数据覆盖与标签对齐，而不是急着上这些改进。

这六条还可以按性质分成三类，优先级按类别排，而不是按收益排。**第一类是“修正已知缺陷”**，只有 §80.9.1——它不是优化，是修 bug，不做的话后面所有对比都建立在错误的基线上。**第二类是“提高上限”**，§80.9.2 与 §80.9.4，它们让模型或求解更准，但收益取决于你离上限还有多远。**第三类是“应对变化”**，§80.9.3、§80.9.5、§80.9.6，它们不改善当前工况的精度，只在数据会增长、系统会漂移、或需要鲁棒性时才有价值。若你的任务是一次性标定、工况稳定，第三类一条都不用做。

做改进之前必须先留一条基线。本章的对比里，误差差异小于 0.2 cm 就已经落在种子与求解器抖动的量级内（§80.6.7），而 §80.9.1 的预期收益是 0.3–0.6 cm——两者只差两三倍。没有基线、只跑一次就宣布“改进了”，得到的很可能是噪声。正确做法是：同一配置连跑三次取中位数，改一项，再连跑三次，差异超过两倍抖动才算数。

最后一句话给“什么时候不该做改进”：若你的闭环误差已经低于任务要求的二分之一，这些改进的收益会被工况波动淹没。此时该做的是加监控、补数据覆盖、把边界条件测清楚，而不是继续压那最后 1 cm。

80.10 工程落地

把一个学习模块塞进已经能跑的 MPC，风险不在算法，而在“什么时候可以信它”。这一节给一套可执行的落地流程：先影子、再灰度、再接管；每一级都有明确的进入与退出条件。

80.10.1 上线前的三级验证

第一级：离线精度。 用留出的测试集算一步误差与多步误差（表 80-2）。判据是 N 步误差不超过名义模型的一半。这一级不过，后面都不用谈。

第二级：影子模式。 学习增强的 MPC 与原有的名义 MPC 并行运行，但只有名义 MPC 的输出接到执行器；学习版的输出只记录、不执行。跑够至少 10 倍于训练时长的时间，比较两条误差曲线。判据是学习版在 95% 的时间里误差更低，且从未出现单步误差超过名义版 3 倍的情况。

第三级：限幅接管。 学习版接管执行器，但输出经过一个硬限幅：$|u| \le u_{max}$，且与名义版输出的差不超过 δ_u。δ_u 从 $0.1u_{max}$ 起步，逐步放宽。这一级的作用是拦住“模型突然给出荒谬值”的情况，同时不影响正常工况。

三级都过，才算可上线。跳过影子模式直接接管，是这类项目最常见的翻车方式——离线误差很好看，闭环却在某个没见过的状态上突然发散。

80.10.2 影子模式该记什么

影子模式的价值取决于记录得够不够细。至少要记四组：

1. **状态与控制**：x_k、u_k^{nom}、u_k^{learn}，用于事后重放；
2. **残差预测**：$\hat{f}_r(x_k)$ 与两步之后反推的真实残差，用于在线监控模型偏差；
3. **求解信息**：迭代数、状态（`optimal` / `optimal_inaccurate` / `infeasible`）、耗时；
4. **触发标记**：是否饱和、是否超出数据覆盖范围、是否被限幅拦截。

第 2 组是最容易被忽略但最有用的：它让你在不改变控制的前提下，持续量出模型在新数据上的真实误差。本章的 0.310 m/s² 就是这么量出来的（离线测试集），在线版本只是把测试集换成实时数据。

80.10.3 在线更新的门禁

在线更新不是“每个周期都学”，而是一条带门禁的流水线：

数据门禁：新样本要通过三道检查——时间戳对齐（Δt 与 T_s 的偏差小于 1%）、状态在覆盖范围内、残差幅值不超过 f_{max}。任一不过就丢弃，不进入训练集。

漂移检测：用滑动窗内的预测误差与基线误差做比值：

$$\rho_k = \frac{\mathrm{RMS}_{win}(f_r - \hat{f}_r)}{\mathrm{RMS}_{base}(f_r - \hat{f}_r)} \tag{80-33}$$

$\rho_k > 1.5$ 持续 100 个周期，判为漂移，触发重训；$1.2 < \rho_k \le 1.5$ 只记录告警，不动模型。

灰度：重训后的模型先在影子模式跑 1 小时，误差不劣于旧模型才允许替换。替换用双缓冲：新模型加载到备用槽，验证通过后原子切换，旧模型保留一个版本用于回滚。

这三道门禁把“在线学习”变成了“在线监控 + 离线重训 + 灰度替换”，看起来保守，但它把不可控的在线过程变成了可控的发布过程。对多数工业系统，这个保守是值得的。

80.10.4 三层安全防线

学习模块的失效模式是“输出荒谬值”，防线要针对这一点设计，而不是针对“精度不够”：

第一层：输出饱和。 $|\hat{f}_r| \le f_{max}$（§80.7.4）。这一层在模型内部，成本为零，拦住的是数值爆炸。

第二层：约束层。 MPC 的盒约束保证 $|u| \le u_{max}$，与模型无关；若有硬状态约束，串第 79 章的 CBF-QP。这一层拦住的是“预测错了但控制还在范围内”。

第三层：看门狗。 监控求解状态与耗时：求解器返回 `infeasible`、或耗时超过 $0.8T_s$、或连续 3 个周期误差单调上升，就切回名义模型并报警。切回动作必须是自动的，不能等人。

三层的关系是递进的：第一层管模型，第二层管控制，第三层管系统。任何一层单独都不够——只有饱和没有约束，模型可能持续给出边界值；只有约束没有看门狗，模型可能在约束内持续给错方向。

80.10.5 监控指标与阈值

上线后要盯的量不多，但每个都要有阈值和动作（表 80-7）。

表 80-7　学习增强 MPC 的在线监控指标

指标	正常范围	告警阈值	动作
一步预测误差	≤0.35 m/s²	>0.6 持续 100 周期	记录，准备重训
闭环跟踪误差	≤2 cm	>4 cm 持续 50 周期	告警，查数据覆盖
求解耗时 P95	≤50 ms	>80 ms	降级：缩短 N 或切名义
饱和占比	<1%	>10%	查权重与残差饱和
求解状态异常	0 次	任意一次 `infeasible`	立即切名义并报警
外推触发次数	0	>0	记录状态，补数据

阈值不是拍脑袋定的，而是从本章的实测推出来的：一步误差的正常上限 0.35 对应 GP 的 0.310 加 10% 余量；跟踪误差上限 2 cm 对应 GP 的 1.71 cm；耗时上限 50 ms 对应 P95 的 46.67 ms。换系统要重新推一遍，方法是“正常值 × 1.2–1.5”。

80.10.6 版本与回滚

学习模块比传统控制器多两个版本维度：代码版本与模型版本。两者必须绑定——模型文件里要写入训练数据的哈希、超参、训练时间，控制器启动时校验。否则出现“换了模型忘了改配置”的问题时，你无法从日志判断当时用的是哪一版。

回滚要能在 1 个控制周期内完成：模型常驻内存两份（当前与上一版），回滚只是一次指针交换。不要设计成“重新加载文件”，那在实时系统里可能要几百毫秒。

最后一条经验：**每次模型更新都要留一条“关掉学习”的开关**。这个开关不是为了日常使用，而是为了在出问题时能立刻把系统退回一个已知可用的状态。开关的存在本身也会改变设计——你会更愿意把学习模块做成可插拔的，而这恰好是第 80.1 节说的“插件”结构的工程价值。

80.10.7 从 cvxpy 到嵌入式：移植时要动的四处

本章的仿真跑在 Python 上，中位耗时 31 ms。这个数字对 $T_s = 100$ ms 够用，但它几乎全部花在建模与求解器接口上，而不是花在物理计算上。要在嵌入式上做到 10 ms 以内，必须知道这 31 ms 是怎么构成的。

先看问题规模。$N = 15$、状态 4 维、输入 2 维，决策变量是 $z \in \mathbb{R}^{4\times 16}$ 与 $u \in \mathbb{R}^{2\times 15}$，共 $64 + 30 = 94$ 个；等式约束 64 条（每一步的动力学），不等式 60 条（每步两个输入的上下界）。这是一个不到一百个变量、一百多条约束的稀疏 QP，结构极为规整——真正求解只需要几百微秒。剩下的时间在别处。

第一处是建模开销。 cvxpy 每次 `solve` 都要把符号表达式重新 canonicalize 成标准形式，这部分与问题规模关系不大，但每次都要付。解决办法是把 `Problem` 与 `Parameter` 建在循环外，循环内只给 `Parameter` 赋新值再 `solve`。前提是问题满足 DPP（disciplined parametrized programming）规则：参数只能以仿射方式进入约束与目标。本章的写法（每次重建 Problem）是清晰优先，不是性能优先；移植时必须改。

第二处是求解器接口。 即使问题已经 canonicalize，Python 到 C 的调用、数据的反复打包、结果的字典构造都要花时间。直接调 OSQP 的 C 接口，把 P、A 的稀疏结构在初始化时固定，每周期只更新 q 与上下界，可以省掉绝大部分开销。

第三处是结构。 这个 QP 的 P、A 矩阵每周期完全不变，变的只有初值约束的右端项与参考轨迹。这意味着可以在初始化时做一次分解，后续只做回代；也可以用专门针对最优控制结构的求解器（如 HPIPM、acados），它们利用“链式”稀疏性，把复杂度从一般稀疏 QP 降到正比于 N。代价是失去 cvxpy 的表达自由度：换一种约束，就要重新生成代码。

第四处是数值类型。 OSQP 用双精度，嵌入式平台若只有单精度 FPU，需要评估 24 位尾数是否够。判据是：把仿真改成 `float32` 跑一遍，若闭环 RMSE 变化小于 1%，单精度可用；若出现求解失败或迭代数暴涨，说明问题的条件数不允许降精度——此时应当先做变量缩放（把位置、速度、输入都归一到 1 的量级），再试一次。定点化不建议作为第一步，ADMM 类算法对缩放敏感，定点实现的调参成本常被低估。

这四处的收益不叠加，而是递进的：先解决建模开销（往往占一半以上），再看接口与结构；前两处没做完就去做代码生成，量到的加速比会失真。动手之前先量一次分解——在 `solve` 前后各加一个计时点，把“建模 / 求解 / 结果打包”三段分开，再决定改哪一处。本章没有做这个分解，因为 31 ms 对 100 ms 的预算够用；但移植时这是第一件该做的事。

移植后的验收清单只有四条，但每条都必须过：

- 同一组 (x_0, ref) 在 PC 与目标机上的首步输出差异小于 10^{-6}（不是“差不多”，是逐位可比的量级）；
- 迭代数分布与 PC 上一致，若目标机上迭代数系统性偏高，多半是缩放或容差没对齐；
- P95 耗时低于 T_s 的 50%，给抖动留一倍余量；
- 超时路径可用：人为把预算压到 P95 的 80%，确认系统按预期降级（用上一周期的解或切名义模型），而不是直接输出零。

最后一条最容易被跳过。绝大多数“嵌入式上跑崩”的事故不是算得慢，而是算得慢之后没有定义行为——求解器返回一个未收敛的解，代码照单全收，闭环立刻发散。

80.10.8 状态估计：一个绕不开的前置环节

本章从头到尾假设 x 全可测。真实系统通常只有位置可测，速度要靠估计，而残差模型恰恰以速度为输入——这一步没处理好，后面所有精度都会被它吃掉。

先看直接差分的代价。速度由相邻两拍位置差分得到，噪声被放大 $\sqrt{2}/T_s$ 倍：位置噪声 1 mm、$T_s = 0.1$ s 时，速度噪声约 14 mm/s。这个量级听着不大，但它进入残差标签后会再被放大一次——标签本身就是速度的差分，等效噪声是 $2\sigma_p/T_s^2 = 0.2$ m/s²，与本章的残差 RMS（1.195）相比占 17%。也就是说，用原始差分做标签，学习器有近两成的“残差”其实是量测噪声的差分。

卡尔曼滤波能改善，但改善的是**滤波**而不是**平滑**。离线采数据时应当用 RTS 平滑：它用到整段数据的前向与后向信息，速度估计的方差比滤波低一个量级。这一点常被忽略——很多人拿在线滤波器的输出当训练标签，等于主动放弃了离线阶段本可以拿到的精度。

第二个坑是估计误差的相关性。滤波器的速度误差在相邻时刻高度相关（它由同一个位置序列积分而来），而 §80.6.6 已经说明：相关性强的误差在闭环里累积得更厉害。所以“速度估计误差 14 mm/s”不能简单按白噪声折算成位置误差，实际影响更大。判断办法是看估计误差的自相关时间：若与闭环时间常数同量级，就不能当成噪声处理。

第三个坑是结构。有人会把“速度估计误差”当成一个额外的状态扩进去，让残差模型顺带学它。这与 §80.2.7 的原则冲突：残差必须是状态的函数，估计误差不是状态。更稳妥的做法是让观测器与残差模型解耦——观测器用名义模型（不用学习模型），残差模型用平滑后的状态当输入。两者都用学习模型会形成回路，估计误差与模型误差互相“解释”，最后谁都收敛不了。

什么时候该停下来先修观测器？一个可用的判据是：**速度估计的标准差超过残差一步误差的 20%**，就先别学残差。本章的一步误差是 0.310，对应阈值约 0.062 m/s；若你的速度估计误差到了这个量级，学到的模型里噪声与信号同量级，数据量扫描会呈现“加样本也不降”的平台，而你很可能误判成“模型容量不够”，去加中心数——那只会把噪声也拟合进去。

80.11 优缺点与适用边界

80.11.1 优点

第一，收益直接且可量化。 本章的算例里，加一个残差网络把跟踪误差从 7.19 cm 压到 1.71 cm，权重、时域、约束全都没动。这种“只改一处、效果可归因”的特性，在工程上比“整体换一个控制器”更容易被接受，也更容易做 A/B 对比。

第二，结构不变，安全性仍可论证。 约束还是那些约束，第 58 章的递推可行性、第 79 章的 CBF 安全层都还能用。学习只改预测的精度，不改优化的形状。这一点与“用强化学习直接学策略”有本质区别：后者没有任何结构性的安全保证。

第三，可以用不确定性说话。 GP 给出的方差能直接转成误差上界（式 80-20），进而进入约束（式 80-32）。这让“模型在哪里不可信”变成一个可以计算、可以告警的量，而不是靠人盯。

第四，可插拔、可回滚。 三处插入点（模型、终端代价、初值）彼此独立，任何一处出问题都可以单独关掉。本章的实测里，关掉终端代价的代价是 3.65 cm，关掉初值只损失 0.09 ms——每个开关的代价都能提前量出来。

第五，数据成本低。 本章用 200 个点就跑出了 1.71 cm。相比之下，训练一个端到端策略通常需要 10^5 量级的交互。残差比原函数更平滑、幅值更小，这是样本效率高的根本原因。

80.11.2 缺点

第一，依赖数据覆盖，外推没有保证。 这是最硬的限制。模型只在其训练分布内可信，一旦参考轨迹走出覆盖范围，误差可能比名义模型还大。本章的 141.1% 远域误差就是一个提醒：连终端代价这种低维拟合都会在范围外失效，残差网络的失效只会更突然。

第二，标签构造容易出错。 式 (80-17) 的三个条件（实际控制、时间对齐、无未记录延迟）任缺一个，残差里就混进常值偏置，而这个偏置会被学成“模型的一部分”。这类错误在离线指标上往往看不出来（因为偏置是稳定的），只在长时间运行后表现为缓慢漂移。

第三，稳定性结论变弱。 从渐近稳定退到 ISS（式 80-25），意味着误差只能收敛到一个正比于学习误差的邻域，不能到零。对精度要求极高的场合（例如微米级定位），这个残余可能不可接受。

第四，调试链路变长。 出问题时，可能的原因从“控制器参数”扩展到“数据、标签、模型、接口”四层。本章 §80.3.2 与 §80.5.5 列的那些坑，都是这条长链路的具体表现。

第五，计算与维护成本不为零。 两遍法让 QP 求解翻倍，模型版本管理引入新的运维负担。本章的耗时数据说明这部分成本可以被 QP 淹没，但那是因为 N 只有 15；N 上百时，两遍法的代价就会显现。

这五条里，第一与第二条是结构性的，靠工程手段只能缓解、不能消除；第三与第五条是可以用判据量化的，上线前就能算清楚；第四条纯粹是组织成本，取决于团队有没有把数据、模型、控制三块放在同一份记录里。认清哪一条是自己的主要矛盾，比笼统地说“这套方法有局限”更有用。

80.11.3 什么时候该用

判断该不该用，看三条同时成立与否：

1. **有可重复的残差。** 同一状态下残差符号一致、幅值稳定。若残差主要是随机噪声，学习没有意义——你学到的是噪声的均值，而均值接近零。
2. **残差相对控制权限不可忽略。** 本章的判据是 1.195/3≈40%；低于 5% 时收益会被其他误差淹没（§80.3.1）。
3. **能采到覆盖工作空间的数据。** 不需要均匀覆盖，但参考轨迹周围要有足够密的样本。若系统只能在小范围内运行、且无法主动激励，学习模型的价值有限。

三条里缺一条，都应当先解决缺的那条，而不是硬上学习。

80.11.4 什么时候不该用

第一，模型误差主要来自未建模的动态而非静态非线性。 例如执行器滞后、传感器滤波、结构振动。这些是“状态不够”而不是“函数不准”，学一个静态映射补不上。此时该做的是扩状态或辨识传递函数，而不是加残差网络。

第二，安全约束极硬且无法验证。 若系统一旦越界就是灾难性后果，而你又无法给出学习误差的界，那么任何学习模块都不该进控制回路。可以做影子模式，但不该接管。

第三，已有更便宜的方案能达到要求。 前馈补偿、增益调度、积分作用，这些传统手段在不少场合比学习更简单。第 44 章的 PID 加积分项消稳态误差，本质也是“补上模型不知道的常值部分”；只有在残差是状态相关的时候，学习才比积分有优势。

第四，参考轨迹本身就是离线已知的。 若轨迹完全可预知，直接把参考的前馈项算出来（迭代学习控制）比在线学模型更省事。第 47 章的 Smith 预估器也是同一个思路：用已知的结构补偿已知的滞后，不需要学习。

80.11.5 与相邻方法的边界

把这一章放回全书的坐标里，四类方法的边界可以这样划：

- **PID（第 44 章）**：不需要模型，靠误差反馈消偏差。适合残差小、带宽要求不高的场合。它的积分项能消常值残差，但对状态相关的残差无能为力——本章的名义 MPC 误差 7.19 cm 就是这类残差，PID 加上去也只能压一部分。
- **Smith 预估器（第 47 章）**：需要已知的滞后模型，用结构补偿延迟。它与学习不冲突，可以叠加：滞后用 Smith 补，非线性用残差补。
- **MPC（第 58 章）**：需要模型，能处理约束与多变量。本章是它的增强，不是替代。
- **安全控制（第 79 章）**：不需要精确模型，只需要约束的几何信息。它是本章的兜底层，两者串联时，学习负责“更好”，CBF 负责“不出事”。

80.11.6 与另外四类“会学习的控制”的区别

“学习”这个词在控制里被用在至少五种不同的东西上，它们的数据需求、失效模式、验证方法都不一样。把本章与其中四种摆在一起，边界才清楚。

迭代学习控制（ILC）。 ILC 沿**迭代轴**学：同一条轨迹重复做，每次用上次的误差修正前馈，几次之后误差收敛。它不需要模型，也不需要覆盖状态空间，只需要轨迹可重复、每次初始状态一致。本章沿**状态空间**学：一次标定，之后任意轨迹都能用。判据很直接——任务是不是固定轨迹的重复？机械臂拾放、数控加工是，那 ILC 更省事，因为它把“这条轨迹上该怎么修”直接记住，不需要泛化。若轨迹会变、或同一条轨迹只走一次，ILC 就没有迭代的机会，此时才轮到本章。两者可以叠加：ILC 管轨迹内的重复成分，残差模型管跨轨迹的状态相关成分。

自适应控制。 自适应假设**模型结构已知、参数未知**，在线估那几个参数。本章假设**结构未知**，用一个非参数模型去逼近整个残差。若你能写出残差的形式——例如已知是二次阻尼、只有一个阻尼系数未知——那么递推最小二乘估一个标量就够了，样本效率比 48 个中心高得多，而且有完整的稳定性理论。判据是：能不能写出残差的参数化形式？能，先用自适应；不能，或者写出来有十几项，再用本章。风险也不同：自适应怕持续激励不足导致参数漂移，本章怕数据覆盖不足导致外推。

增益调度。 增益调度把工作空间切成若干区，每区一个线性控制器，按当前工作点插值或切换。它本质上是“用多个线性模型覆盖非线性”，本章是“用一个非线性修正项覆盖非线性”。差别在切换：增益调度在区间边界有切换瞬态，需要驻留时间或插值来保证稳定；残差模型是连续的，没有切换问题，但要求残差在空间上光滑。若非线性弱、工作点只有两三个，增益调度更简单、更容易验证——它的每一区都是一个线性系统，第 58 章的工具直接可用。

强化学习策略。 端到端学 $u = \pi(x)$，不显式持有模型，也不显式处理约束。它的样本效率比本章低两到三个数量级（本章 200 个点，策略学习通常要 10^5 量级的交互），而且安全性只能靠约束投影或屏蔽层事后补。它的优势在另一类问题上：目标函数本身难写、或者最优行为高度依赖长程规划。本章适合“目标清楚、模型差一点”的任务；策略学习适合“模型清楚、目标说不清”的任务。

五种方法的差别可以按“学什么”一句话划分：ILC 学轨迹修正，自适应学参数，增益调度学工作区划分，强化学习学策略，本章学模型残差。**学模型残差的样本效率最高**，因为它把“控制怎么算”整个交给 MPC，只让学习补最小的一块。代价是它的上限也被 MPC 的结构限制住了——MPC 想不到的解，残差模型再好也给不出来。

80.11.7 从 4 维到 12 维：哪些结论会变

本章的状态是 4 维，这个规模让很多做法显得轻松：200 个点铺满工作空间，48 个中心够用，GP 求逆一眨眼。真实的机电系统常常是 8–12 维（多关节、多轴、含执行器状态），这时有三条结论会变，一条不会。

第一条会变的是数据预算。 按 §80.3.7 的分格计数，覆盖度随维数指数增长：4 维是 $4^4 = 256$ 格，8 维是 65536 格，12 维是 1.6×10^7 格。按每格 5 个样本算，12 维需要八千万个点——这个数字本身就宣告了“均匀覆盖”在高维不可行。所以高维系统必须换一种覆盖定义：**沿任务轨迹的管状覆盖**。只要求参考轨迹周围 ± 0.5 m 的管内有数据，管内的点数随轨迹长度线性增长，不随维数指数增长。代价是模型只在管内有依据，换任务就要补数据——这不是缺陷，而是高维下的唯一选择。

第二条会变的是计算量，但变得不多。 RBF 的核计算随输入维数线性增长（48 个中心乘 d 维），GP 与维数无关（只看样本数），QP 的变量数是 $n(N+1)+2N$：$n=12$、$N=15$ 时是 222 个，仍在几百量级，OSQP 可以处理。真正变

慢的是预测矩阵的构造与稀疏结构的维护，量级上不会颠覆 §80.6.4 的结论——瓶颈仍在 QP。

第三条会变的是终端集。 椭球在高维下“体积集中在赤道附近”，直觉上的半径与实际的可行域差得很远；同时 $\lambda_{max}(P)$ 由最快的通道主导，§80.5.6 的膨胀估计会更保守。实践中的表现是：高维系统的终端集比低维系统“看起来小得多”，递归可行性的余量要留得更宽。

不会变的是稳定性结论的形式。 ISS 仍然是 ISS，式 (80-25) 的结构不变，只是 γ 随维数增大——同样的残差误差在 12 维系统里造成的跟踪偏差更大，因为误差有更多方向可以累加。这意味着高维系统对模型精度的要求更高，而不是更低。

若你的系统超过 8 维，最有效的办法不是换更强的学习器，而是**按物理结构分块**。车辆模型里横向残差主要依赖 (v_y, r)、纵向残差主要依赖 $(v_x, \text{驱动})$，把 12 维问题拆成四个 4 维子问题，每个子问题只学与自己相关的状态子集，总样本需求从 4^{12} 降到 4×4^4。代价是要先做一遍物理分析，判断哪些状态之间真的耦合——但这一步的收益，比在 12 维空间里多采十倍数据大得多。

一句话总结这一章的位置：**它不改变控制的骨架，只让骨架上的肌肉更准。** 判断要不要用它，问三个问题——残差可重复吗、幅值够大吗、数据够覆盖吗。三个都答“是”，再动手。

80.12 Python 仿真代码

本章的两个脚本可以直接运行：`code/ch080_sim.py` 产出全部数字，`code/ch080_figs.py` 产出全部插图。前者约 580 行，后者约 350 行，依赖只有 `numpy`、`scipy`、`cvxpy`（OSQP 随 cvxpy 一起装）。

```
D:\Anaconda3\envs\py311\python.exe code/ch080_sim.py
D:\Anaconda3\envs\py311\python.exe code/ch080_figs.py
```

80.12.1 文件结构与函数对应

脚本按“真实对象 → 模型容器 → 求解器 → 学习器 → 五个实验”分层，每一层只依赖上一层（表 80-8）。

表 80-8 `ch080_sim.py` 的函数与正文小节的对应

函数	作用	对应小节
`residual_accel`	真实残差（阻尼 + 力场）	§80.1.1
`true_step`	RK4 积分真实对象	§80.4.1
`nominal_model`	双积分器的 A、B	§80.3.1
`lqr_gain`	解 DARE 得 P 与 K	§80.2.5
`solve_mpc`	线性 MPC，可注入残差序列	§80.2.4
`solve_residual_mpc`	两遍法	§80.2.4、§80.5.5
`RBFModel`	固定中心 + 岭回归	§80.2.2、§80.5.4
`GPModel`	平方指数核 + 白噪声	§80.2.2、§80.4.2
`lift_poly`、`fit_koopman`	15 维二次提升 + EDMD	§80.2.3
`experiment_prediction`	一步/多步误差、覆盖率	§80.3.4
`experiment_closed_loop`	四组闭环对比	§80.6.3
`experiment_terminal`	终端代价拟合	§80.2.5、§80.8.3
`experiment_warmstart`	三种热启动	§80.8.4
`experiment_curve`	数据量学习曲线	§80.8.2

这张表也是阅读顺序的建议：想复现某个数字，先找到对应的函数，再往上追它依赖的模型与参数。

80.12.2 真实对象与残差

真实对象的定义只有三行，但它决定了整章的数字：

```
def residual_accel(p, v):
    """未建模加速度：二次阻尼 + 位置相关的力场（名义模型完全不知道）。"""
    px, py = float(p[0]), float(p[1])
    sp = np.linalg.norm(v)
    ax = -CFG["drag"] * sp * float(v[0]) + CFG["field"] * np.sin(1.4 * px) * np.cos(1.1 * py)
    ay = -CFG["drag"] * sp * float(v[1]) + CFG["field"] * np.cos(1.1 * px) * np.sin(1.4 * py)
    return np.array([ax, ay])
```

积分用 RK4，而不是欧拉：

```
def true_step(x, u, dt):
    """RK4 积分真实对象：p'=v, v'=u+residual。"""
    def f(s):
        p, v = s[:2], s[2:]
        return np.concatenate([v, np.asarray(u, float) + residual_accel(p, v)])
    k1 = f(x)
    k2 = f(x + 0.5 * dt * k1)
    k3 = f(x + 0.5 * dt * k2)
    k4 = f(x + dt * k3)
    return x + dt / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
```

这里用 RK4 有两个理由。第一，如果把真实对象也写成离散递推，残差标签里就会混进积分误差，你无法区分“模型不准”和“积分不准”。第二，RK4 的局部误差是 $O(T_s^5)$，在 $T_s = 0.1$ 时远小于残差本身，可以当作“真值”使用。

参数集中在一个字典里：

```
CFG = dict(ts=0.1, T=12.0, u_max=3.0, n_h=15,
           drag=0.35, field=0.60, ref_r=1.2, ref_w=0.6, seed=20261020)
Q_STAGE = np.diag([10.0, 10.0, 1.0, 1.0])
R_STAGE = np.eye(2) * 0.1
```

把参数集中在一个地方，是为了让“换一个系统”只需要改这一块。书里的每个数字都能追溯到这些值。

80.12.3 残差网络：20 行的核心

RBF 模型的全部实现如下，没有隐藏层、没有反向传播：

```python
class RBFModel:
    """固定中心的 RBF 网络，输出层用线性最小二乘（岭回归）解析求解。"""

    def __init__(self, X, Y, k=48, sigma=0.7, ridge=1e-6, seed=0):
        self.mu = X.mean(axis=0)
        self.sd = X.std(axis=0) + 1e-9
        Xn = (X - self.mu) / self.sd
        rng = np.random.default_rng(seed)
        idx = rng.choice(len(Xn), size=min(k, len(Xn)), replace=False)
        self.C = Xn[idx]
        self.sigma = sigma
        Phi = self._phi(Xn)
        self.W = np.linalg.solve(Phi.T @ Phi + ridge * np.eye(Phi.shape[1]), Phi.T @ Y)
        self.name = "RBF"

    def _phi(self, Xn):
        d2 = ((Xn[:, None, :] - self.C[None, :, :]) ** 2).sum(axis=2)
        return np.exp(-0.5 * d2 / self.sigma ** 2)

    def predict(self, X):
        Xn = (np.atleast_2d(X) - self.mu) / self.sd
        return self._phi(Xn) @ self.W
```

三个细节值得注意。第一，输入先做均值方差归一化，否则位置（米）与速度（米每秒）的量纲差异会让核宽失去意义。第二，中心从训练样本里随机挑，不训练——这是“固定中心”的含义。第三，权重用 `solve` 而不是 `lstsq`，因为加了岭项之后矩阵正定，解唯一。

GP 的实现同样短，核心是算一次 K^{-1}：

```python
K = self._k(self.Xn, self.Xn) + (sn ** 2 + ridge) * np.eye(len(X))
self.Kinv = np.linalg.inv(K)
self.alpha = self.Kinv @ Y
```

预测时，均值是 $k(x)^{\mathsf{T}}\alpha$，方差是 $k(x,x) - k(x)^{\mathsf{T}}K^{-1}k(x)$。样本量到 200 时，这是一次 200×200 的求逆，离线一次即可。

80.12.4 把残差注入 MPC

`solve_mpc` 的关键是第 129 行：残差只加到速度通道。

```python
for k in range(N):
    c = np.zeros(n)
    if res_seq is not None:
        c[2:4] += CFG["ts"] * res_seq[:, k]      # 残差只作用在速度通道
    cons += [z[:, k + 1] == model.A @ z[:, k] + model.B @ u[:, k] + c]
    cost += cp.quad_form(z[model.idx4, k] - refs[k], Q_STAGE) + cp.quad_form(u[:, k], R_STAGE)
```

`solve_residual_mpc` 就是两遍法，只有五行：

```python
def solve_residual_mpc(x0, refs, P_term, model, res_fn):
    """两遍法：先按名义模型求解，再在预测状态上算残差、修正一次。"""
    u0, info = solve_mpc(model, x0, refs, P_term)
    Xp = predict_states(model, x0, info["u"])
    r = np.stack([res_fn(Xp[k, :2], Xp[k, 2:]) for k in range(Xp.shape[0] - 1)], axis=1)
    u0, info = solve_mpc(model, x0, refs, P_term, res_seq=r, u_warm=info["u"])
    return u0, info
```

第二遍把第一遍的解作为热启动传进去，这是耗时能接受的关键（§80.5.5）。

80.12.5 热启动的三种模式

热启动实验用 cvxpy 的 `Parameter` 复用同一个问题，避免重复建模的开销：

```
if mode == "cold":
    u.value = np.zeros((2, n_h))
elif mode == "shift":
    u.value = np.hstack([prev[:, 1:], prev[:, -1:]])
else:
    u.value = np.tile((W.T @ np.r_[x - refs[0], 1.0])[:, None], (1, n_h))
prob.solve(solver=cp.OSQP, warm_start=(mode != "cold"), verbose=False,
           eps_abs=1e-6, eps_rel=1e-6, max_iter=8000)
return np.clip(u.value, -CFG["u_max"], CFG["u_max"]), prob.solver_stats.num_iters
```

迭代数直接读 `prob.solver_stats.num_iters`，这是 OSQP 自己统计的，比在外面数循环可靠。学习型初值 W 用 120 个样本拟合，标签是每个样本点的 MPC 最优首步。

80.12.6 指标定义

四个指标的算法必须写清楚，否则不同人跑出的数不可比：

- **稳态 RMSE**：只统计 $t \geq 3$ s 的段，跳过起步过渡；位置误差取欧氏范数，再对时间求均方根。
- **最大误差**：同一段内的最大值，用来说明最坏情况。
- **饱和占比**：$|u_i| \geq u_{max} - 10^{-6}$ 的样本比例。本章四组都是 0%，说明约束没被激活。
- **P95 耗时**：每步记录一次求解时间，取 95 分位。用分位数而不是均值，是因为少数慢步会把均值拉偏。

这四条定义写在 `simulate_closed_loop` 的返回字典里，画图脚本直接取用，保证正文、图、表三处一致。

80.12.7 运行输出

`main()` 按五组实验依次打印，输出应当与正文的数字逐项吻合：

```
== 残差学习与预测精度 ==
  一步 RMSE / (m/s^2)：名义 1.195  RBF 0.624  GP 0.310  Koopman 0.445
  第  1 步位置 RMSE / cm：nominal 0.28  rbf 0.28  gp 0.28  koopman 0.18
  第  5 步位置 RMSE / cm：nominal 6.31  rbf 3.41  gp 1.26  koopman 4.68
  第 10 步位置 RMSE / cm：nominal 22.79  rbf 13.55  gp 4.21  koopman 18.99
  第 20 步位置 RMSE / cm：nominal 75.47  rbf 46.95  gp 16.06  koopman 72.48
  GP 的 95% 区间覆盖率：100%

== 闭环跟踪（圆周参考，N=15）==
  控制器       稳态RMSE/cm  最大误差/cm   |u|均值   饱和占比   中位耗时/ms  P95/ms
  nominal         7.19        7.85     0.603      0.0%       31.44    46.67
  ...
```

全流程约 70 秒，主要时间花在 480 次 QP 求解上（四组 × 120 周期）。若只想看某组结果，可以单独调 `experiment_closed_loop` 里的循环，把 `kind` 列表改成一项。

80.12.8 移植到你的系统要改什么

五个地方需要动，其余可以照搬：

1. **`residual_accel`**：换成你的真实对象。若没有解析式，用实测数据代替，此时 `true_step` 也换成插值或查表。
2. **`nominal_model`**：换成你的名义模型。注意列写精确离散（§80.5.1），不要用 $I + A_c T_s$ 凑。

3. `CFG`：T_s、N、u_{max}、Q、R 按 §80.5.2 的三条约束重夹一遍。
4. `sample_residual`：换成你的采样逻辑。若是实测数据，重点检查时间对齐（§80.3.2）。
5. `reference`：换成你的参考轨迹，注意 `refs` 要给出 $N+1$ 个点，最后一个是终端参考。

改完之后，先跑 `experiment_prediction` 看多步误差，再看 `experiment_closed_loop`。跳过第一步直接跑闭环，是移植时最常见的顺序错误——离线预测不过关，闭环一定不过关，但你会以为是控制器的问题。

80.12.9 两个脚本的分工

`ch080_sim.py` 只算数、不画图；`ch080_figs.py` 只画图、不算数（它调用前者的函数并缓存结果）。这样分工的好处是：改一个参数只需要重跑一次仿真，十张图会跟着更新；而调试某一张图的样式时，不需要重跑 70 秒的仿真。

缓存用模块级字典实现：

```
PRED = sim.experiment_prediction()
CL = sim.experiment_closed_loop(PRED)
_CACHE: dict = {}
def terminal():
    if "te" not in _CACHE:
        _CACHE["te"] = sim.experiment_terminal(PRED)
    return _CACHE["te"]
```

注意 `experiment_closed_loop` 依赖 `experiment_prediction` 的输出（学好的模型），因此它必须在模块加载时就算完。这也是 `ch080_figs.py` 启动较慢（约 70 秒）的原因，不是画图慢。

80.12.10 读代码时最该看的五行

五百多行代码里，真正决定结果对不对的只有五处。其余是循环、打印与画图，写错了会报错或者看得出来；这五处写错了不报错，只是结果悄悄偏掉。

第一处：B_r 的结构。 `B_r` 必须是 $[0;T_sI]$，只作用于速度。若写成 $[\frac{1}{2}T_s^2I;T_sI]$，残差就会被算两次——一次通过速度，一次直接进位置——闭环误差会以“补偿过度”的形式表现出来，而且看起来像振荡，很容易被误诊成权重过大。

第二处：标签里的减号与 T_s。 `r = (v_next - v) / dt - u` 这一行，符号错一个就是“学到的残差反号”，闭环误差翻倍；`dt` 用标称值而不是实际时间戳之差，就会混进 §80.5.7 那类与速度成正比的假残差。这两点在前面的正文里反复说过，因为它们是整章最高频的错误来源。

第三处：归一化与核宽的单位。 `RBFModel` 里先用 `mu`、`sd` 把输入归一化，再用 `sigma=0.7` 算核。若把归一化删掉而保留 `sigma=0.7`，核宽就变成了“0.7 米”的物理量，与中心间距的关系完全变了，模型会退化成近似线性。这一处最隐蔽，因为代码能跑、误差也只是“变差一点”。

第四处：残差序列的长度与下标。 `predict_states` 返回 $N+1$ 个状态，残差序列只需要前 N 个区间。写成 `range(Xp.shape[0])` 会多算一步并导致形状不匹配；写成 `range(N-1)` 会少算一步，末尾那一步没有残差补偿。两种错误都不会崩，只会让第二遍的预测在末端悄悄偏掉。加一句 `assert r.shape == (2, N)` 就能拦住。

第五处：热启动是否真的生效。 在 cvxpy 里给 `u.value` 赋值只是把初值写进了变量，真正让它起作用的是 `solve(..., warm_start=True)`。这个参数在多数版本里默认开启，但显式写出来更稳妥——尤其是当你从别处抄来一段带 `warm_start=False` 的代码时，初值会被静默忽略。判断办法很直接：跑一次带热启动、一次不带，比较 `num_iters`；若两者相同，说明热启动没生效。

最后给一个能一次性验证预测器的测试，值得在移植时先做：**把学习器换成真实残差函数**（仿真里 `residual_accel` 是已知的），再跑 20 步开环预测。若实现完全正确，误差应当降到数值精度；若误差停在 0.6 cm 左右，说明漏了式 (80-11) 的那一项——这正是 §80.4.1 指出的公式缺口，本书的代码确实会在这个测试里暴露它。一个能暴露已知缺陷的测试，比十个全部通过的测试更有价值。

建议的调试顺序也该按这五处排：先验预测器（真实残差测试），再验标签（残差散点），再验归一化（换一组输入量级看输出是否合理），最后才动热启动。前两步错了，后面所有的调参都是在错误的地基上做装修。

第 81 章 全驱系统理论

81.1 动机与直观理解

第 80 章把学习塞进 MPC，换来了更准的预测，但代价一点没少：每个控制周期仍然要解一次优化问题，仍然要滚动时域，仍然只能在“约束 + 代价”的框架里做取舍。那条路线的哲学是**用计算换性能**。本章讲另一条路线，哲学正好相反：**用结构换计算**。

具体地说，如果系统能被写成某个“最高阶导数等于已知函数加控制输入”的形式，并且那个乘在输入上的矩阵在工作域内处处可逆，那么我们就可以直接解出控制量，把闭环指定成任意想要的线性系统。整个过程没有迭代、没有预测时域、没有可行域，一次矩阵求逆就结束。这类系统叫**全驱系统**（fully actuated system），这套设计方法叫**全驱系统方法**。

上一章的结论可以直接拿来当引子：“学模型残差的样本效率最高，代价是它的上限被 MPC 的结构限制住了。”全驱系统做的正是把这层结构拆掉——如果模型足够准，为什么要用 15 步预测和 60 个约束去逼近一个本来可以闭式写出的答案？

81.1.1 一个具体的对照

本章的算例是一台平面二连杆机械臂，两个关节都由力矩电机直接驱动，参数见表 81-1。参考轨迹是两个关节各自的正弦，幅值 0.7 rad 与 0.6 rad，频率 1.0 rad/s 与 1.3 rad/s，仿真 6 s，控制周期 1 ms。

表 81-1　平面二连杆机械臂的仿真参数

参数	符号	取值
连杆质量	m_1, m_2	1.0 kg
连杆长度	l_1, l_2	0.5 m
质心转动惯量	I_1, I_2	0.01 kg·m²
单关节力矩上限	τ_{max}	25 N·m
控制周期	T_s	1 ms
期望带宽 / 阻尼比	ω_n, ζ	15 rad/s, 0.9

用全驱方法设计，稳态跟踪误差的均方根是 **0.0024°**；用最朴素的“PD + 重力补偿”，同样增益下是 **0.244°**，相差约 100 倍。前者的单步计算耗时是 **4.8 μs**（Python，含矩阵求逆与前馈），第 80 章的 MPC 是 **31.23 ms**，相差约 6500 倍。

这两个数字很容易让人得出“全驱全面优于 MPC”的结论，但那不是本章要说的。把条件改一改，结论立刻翻转：模型参数整体偏 20% 时，全驱的误差涨到 **3.71°**，而 PD + 重力只有 **1.06°**——全驱反而输了三倍半。原因不复杂：全驱把模型当成精确的逆来用，模型错多少，闭环就错多少；PD 根本不用模型的质量阵，所以模型错了它也不在乎。

所以本章真正要回答的问题不是“全驱好不好”，而是：**在什么条件下，把模型信到可以求逆的程度是划算的；以及一旦决定这么做，工程上要付出哪些代价。** 这两个问题贯穿全章。

81.1.2 “全驱”到底是什么意思

“全驱”这个词容易误解成“执行器足够多”。它的准确含义要弱一些，也更工程化：**每一个自由度的高阶导数，都能被输入独立地、即时地改变。**

要看清这句话与反馈线性化的差别，最好把两条设计路线并排放在一起（图 81-1）：上面那条走几何，先找坐标变换把系统“拉直”，在新坐标里做线性设计；下面那条走代数，承认最高阶导数已经与输入成线性关系、只是系数随状态变，把系数求逆就结束。

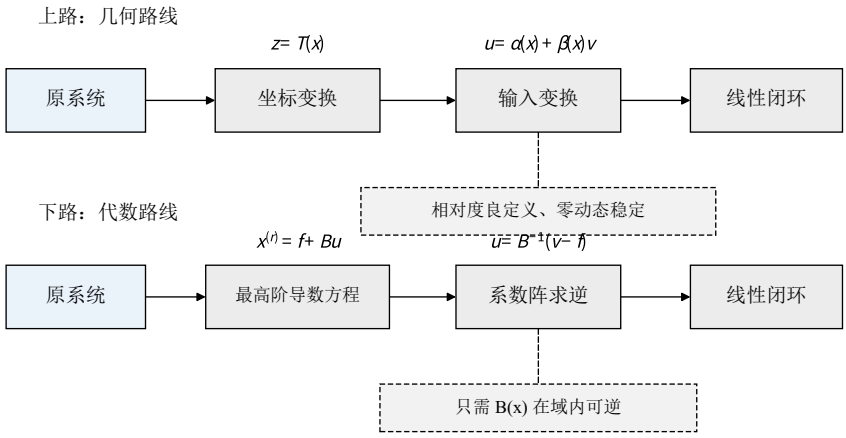


图 81-1　两条设计路线：几何变换与代数求逆

对二连杆机械臂，这句话就是

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau \tag{81-1}$$

其中 $M(q)$ 对称正定。正定意味着 M^{-1} 存在，于是

$$\ddot{q} = M^{-1}(q)\big[\tau - C(q,\dot{q})\dot{q} - G(q)\big] \tag{81-2}$$

右端对 τ 是仿射的，且系数矩阵可逆。这就是全驱：**加速度这一层，两个关节各自有一路独立的、不会互相堵死的控制权限。** 图 81-2 左边是这台机械臂的几何，右边是 (81-2) 对应的控制结构：两路误差各乘一组增益得到“想要”的加速度，经质量阵“换挡”成力矩，再叠上科氏与重力前馈。

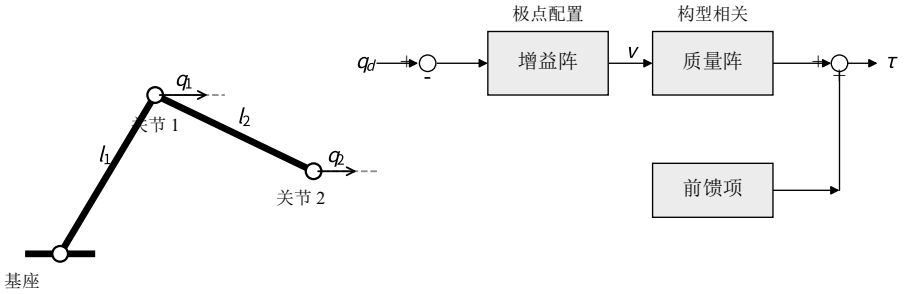


图 81-2　二连杆机械臂的几何与全驱控制结构

反过来看什么不是全驱。单摆的小车摆系统，控制量是小车加速度，摆杆角度要到二阶导数才出现输入，而中间还夹着一个不稳定零点；欠驱动机械臂（比如只有肩关节有电机、肘关节自由摆动）在肘关节方向上根本没有权限。这些系统的输入矩阵在某个方向上退化，B 不可逆，“解出 u”这一步就做不了。

这里有一个容易被忽略的层次：**全驱性是相对于“哪一阶导数”说的。**同一个系统，对位置不是全驱，对速度可能是；对关节角不是全驱，对末端位置更不是。设计前必须先说清楚“对谁全驱”，这决定了控制器能指定哪一层的闭环动态。

81.1.3 全驱条件：一个可以算的指标

B 可逆是一个二值判断，工程上不够用。真正要问的是“可逆到什么程度”。用奇异值衡量：

$$\mu(x) = \frac{\sigma_{\min}\big(B(x)\big)}{\sigma_{\max}\big(B(x)\big)} = \frac{1}{\kappa\big(B(x)\big)} \tag{81-3}$$

μ 在 $(0, 1]$ 之间，越接近 1 说明各个方向的“控制权限”越均衡；越接近 0 说明某个方向几乎推不动，逆矩阵会把噪声和模型误差放大成很大的力矩。本章把它叫**全驱裕度**。

对本章的机械臂，$B = M^{-1}$，所以 $\mu = 1/\kappa(M)$。在工作域 $q_1, q_2 \in [-\pi, \pi]$ 上扫 121×121 个构型，条件数的范围是 **1.09 到 31.36**，中位数 5.83。最坏构型出现在手臂完全伸直时：此时末端径向的等效惯量只有 0.047 kg·m²，而最大方向是 1.48 kg·m²，相差 31 倍。图 81-3 是这张地图，对数色标，白线是 2、5、10、20 四条等值线。

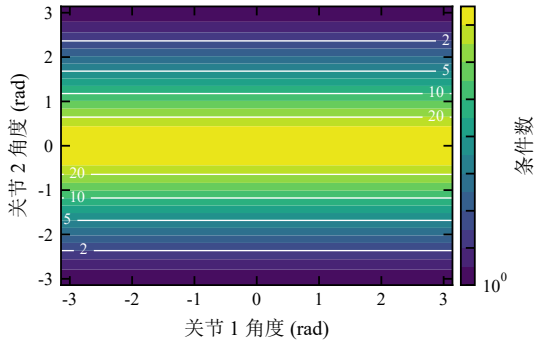


图 81-3　质量阵条件数在工作域内的分布，白色等值线为 2、5、10、20

地图的形状比数字更有信息量。最坏区域集中在手臂伸直的附近（$q_2 \approx 0$ 一带），最好区域在 $q_2 \approx \pm\pi$（完全折叠），两者相差近 30 倍。**这不是一个“平均意义上还好”的分布，而是一个双峰分布**：一半构型在 6 以下，另一半迅速恶化。选固定增益时，你只能在两群之间取一个折中值，这决定了它在任一群里都不合适。

这个数字后面会反复用到。它同时是“固定增益为什么调不好”的原因（同一组标量增益在不同构型下的闭环带宽相差 $\sqrt{31.36} = 5.60$ 倍），也是“全驱为什么需要模型”的原因（只有知道当前构型，才能把这 31 倍的差异补回来）。

81.1.4 本章的一句话核心

全驱系统方法把控制器设计退化成两件事：验证一个矩阵在工作域内可逆，以及给一个线性系统配极点。 剩下的全部工作量，都在保证这两件事的前提成立——模型够准、微分拿得到、输入不饱和。

这句话里，“退化”是褒义：它意味着设计过程可以被检查、被复现、被写进检查表。相比之下，MPC 的调参更像在约束与代价之间找一个说不清的最优点，反馈线性化则要先找一个满足条件的坐标变换。全驱把这部分不确定性压到了最低。

但“退化”也意味着放弃了三样东西：约束处理、预测能力、对模型误差的鲁棒性。这三样恰好是 MPC 的长处。所以两条路线不是替代关系，而是分工关系：**约束硬、模型糙、要前瞻，用 MPC；约束软、模型准、要快，用全驱。**

81.1.5 三条路线的位置

把全驱放到全书的方法谱系里，它和另外两条路线的关系可以用表 81-2 概括。

表 81-2　四条设计路线的横向对照

路线	需要的模型	设计动作	在线计算	约束处理
PID（第 44 章）	几乎不需要	调三个增益	代数运算	无，靠限幅
反馈线性化（第 60 章）	精确非线性模型	找坐标变换与逆	代数运算	无
全驱系统（本章）	最高阶导数方程	求逆 + 配极点	一次矩阵求逆	无，靠限幅
MPC（第 58、80 章）	预测模型	设计代价与约束	在线优化	显式

表 81-2 里最值得看的一行是“设计动作”。PID 调增益，是在一个固定的结构里找参数；反馈线性化找坐标变换，是在一个很大的函数空间里找；全驱只做矩阵求逆和多项式配置；MPC 则把设计负担推到了在线优化。**设计动作越轻，在线计算越轻，但对模型的信任要求越高。**

还有一条隐藏的差别：反馈线性化要求相对度良定义、零动态稳定，这两个条件在很多系统上不成立或难以验证；全驱只要求最高阶导数方程的系数阵可逆。从这个意义上说，全驱是“反馈线性化的一个特例的推广”——它放弃了通过坐标变换把系统变成线性系统，只要求系统在最高阶导数这一层已经是仿射的。§81.2.6 会把这件事说清楚。

81.1.6 本章怎么读

工程师读法：§81.3（五步流程）、§81.5（离散实现的边界）、§81.8（调参）、§81.10（落地与验证）。若只读一节，读 §81.3。

研究者读法：§81.2（定义与稳定性证明）、§81.4（与反馈线性化、微分平坦的关系）、§81.6（误差分解）、§81.9（改进方向）。

学生读法：按顺序读，§81.12 的代码可以最后看。§81.1 到 §81.5 建立主线，§81.6 到 §81.8 是工程配套，§81.11 收边界。

三种读法共享同一个判断：**先问“我的系统对谁全驱”，再问“我的模型有没有准到可以求逆”。** 这两个问题的答案，比任何公式都更能决定这套方法适不适合你。

81.1.7 为什么用二连杆做算例

二连杆不是一个随便挑的例子，它是**能做全驱演示的最小多变量非线性系统**。四个条件它同时满足：自由度大于 1（才有耦合，才能谈矩阵求逆）、质量阵随构型变（才有条件数这条支线）、有重力与科氏项（才有“前馈消掉什么”的可讲之处）、模型有解析式（才能手算校验）。

换成单摆，第一和第二条都不成立：单摆只有一个自由度，M 是标量，“矩阵可逆”退化成一个正数非零，条件数恒等于 1，整章的分析无处落脚。更麻烦的是，小车摆系统对摆角并不全驱，无法演示本章的主线。

换成线性系统，第二条也不成立。线性时不变系统的 M 是常数，κ 与构型无关，全驱退化成“状态反馈 + 前馈”——那是第 20 章就讲过的东西，看不出新意。

换成六轴工业臂，四条都满足，但算例失去了可复现性：运动学参数、减速比、负载惯量都要具体取值，读者无法用一台“标准六轴”复现数字。二连杆的好处是**所有参数都能写在一张表里（表 81-1），任何人都能重跑出同样的 0.0024°。**

代价是代表性有限。二连杆没有柔性关节、没有摩擦、没有减速器回差、没有电机电流环动态，这四样在真实机械臂上都会显著改变结论。本章的处理方式是**把它们显式排除，并在 §81.3.6 说明排除的后果**——不是假装它们不存在，而是把它们划到“这套方法管不到的部分”，留给别的章节或别的层。

读者若要把结论搬到六轴臂上，需要额外确认两件事：一是 κ 的上界会更大（串联关节越多，末端方向的惯量差异越大，条件数通常比 31 更差），二是摩擦与回差会引入一个“参数辨识无法消除”的误差底，这个底会直接抬高图 81-8 的曲线。两件事都指向同一个方向：**真实系统的全驱表现会比本章的算例差，差多少取决于这两项的大小。**

81.1.8 三个常见误解

在展开公式之前，先把三句最常听到的话拆开看。它们都不是完全错，但每一句都藏着一个会导致误判的省略。

误解一：“全驱就是能控。” 这两件事不在同一个层面上。能控性问的是“能不能把状态从任意初值送到任意终值”，是一个存在性问题；全驱问的是“输入能不能直接作用在最高阶导数上”，是一个结构问题。一个系统可以能控却不全驱：小车摆在平衡点线性化后是能控的，但摆角通道上没有任何直接输入，B 的第二行是零，$\operatorname{rank} B < n$。反过来，全驱的系统一定可控（因为可以直接指定最高阶导数），所以全驱是能控的一个**充分但不必要**的加强条件。工程含义是：看到“系统可控”就以为可以套全驱，是最容易犯的第一步错。

误解二：“模型不准，全驱就完全不能用。” 这句话夸张了。§81.6 会给出一个一致最终有界的结论：误差被模型失配的范数除上 ω_n^2 所界定。失配 5% 时实测 0.668°，仍低于 PD 在 20% 失配下的 1.061°（这是本章取的保守基准）；失配 10% 时 1.187° 越过去，20% 时已经是 3.714° 对 1.061°。所以正确的说法是：**全驱有一个大约 9% 的失配门槛，门槛以内仍有优势，门槛以外不如 PD。** 这条门槛不是“能不能用”的开关，而是一条连续的、可以事先估算的分界线。

误解三：“全驱要多算一次矩阵求逆，所以比分块 PD 慢。” 求逆确实要算，但代价被高估了。本章二连杆的完整控制器（含质量阵、科氏项、重力项、求逆、两次矩阵乘）单步 4.8 μs，而这个时间相对于 1 ms 的控制周期只占 0.48%。真正贵的是“带约束的在线优化”，本章引用的 MPC 单步是 31.23 ms，相差 6472 倍。全驱在计算量上的位置很特殊：**它比 PD 贵，但贵的那部分在今天的处理器上几乎看不见；它比 MPC 便宜三个数量级。** 这一点在 §81.9.2 还有数据。

三句话合起来是一句：**全驱不是“更强的控制”，而是“在模型够准、机构够硬的前提下，用结构换性能”的一种特定交易。** 交易的条款写在 B 的可逆性与 κ 的大小上，违约的代价写在失配门槛上。

81.2 数学原理

§81.1 给的是画面，这一节把它写成可以逐行验算的形式。只想知道怎么用的读者可以跳到 §81.3；但 §81.5 与 §81.6 里的每一条工程限制，都是从这一节的几个式子里长出来的。

81.2.1 一般形式

把“最高阶导数能被输入直接改变”写成方程：

$$x^{(r)} = f\big(x, \dot{x}, \dots, x^{(r-1)}\big) + B\big(x, \dot{x}, \dots, x^{(r-1)}\big)u \tag{81-4}$$

$x \in \mathbb{R}^n$ 是位置一类的被控量，$u \in \mathbb{R}^m$ 是输入，$x^{(i)}$ 表示对时间求 i 阶导数，f 收集所有不含输入的项，B 是输入矩阵。全驱的定义只用一条：

$$\operatorname{rank} B = n \tag{81-5}$$

也就是 $m \geq n$ 且 B 列满秩。本章的算例 $m = n = 2$，条件退化成“系数阵可逆”；而 $B = M^{-1}$ 与 §81.2.2 里取的 $B = I$，都靠质量阵对称正定保证可逆。

这里有三处容易含糊，值得先钉住。其一，r 不一定是 2：$y^{(3)} = f + bu$ 这类三阶系统同样可以是全驱的，“高阶全驱系统”指的就是 $r \geq 2$ 的这套写法。其二，f 可以依赖全部低阶导数，全驱并不要求系统线性，只要求输入出现在最高阶导数这一层、且线性地出现。其三，B 依赖状态，所以 (81-5) 是一个**逐点**条件，工作域里每一点都要验证，这件事 §81.3 会单列成一步。

81.2.2 拟线性形式：把状态相关矩阵留着

直接用 (81-4) 有一个技术障碍：f 一般是状态的非线性函数，写成 $u = B^{-1}(v - f)$ 之后闭环是 $x^{(r)} = v$，形式上干净，但 f 里含 $x^{(i)}$ 的那部分信息被丢进了黑箱。全驱系统方法的标准做法是把 (81-4) 整理成**拟线性形式**：

$$\sum_{i=0}^{r} A_i(\cdot)\, x^{(i)} + g(\cdot) = B(\cdot)u \tag{81-6}$$

其中 $x^{(0)} := x$，A_i 允许依赖状态，g 收集不含任何状态导数的偏置（重力、常值外扰），A_r 可逆——这是 (81-5) 在拟线性写法下的等价要求。

对二连杆机械臂，(81-1) 一字不改就是拟线性形式：

$$A_2 = M(q), \qquad A_1 = C(q, \dot{q}), \qquad A_0 = 0, \qquad g = G(q), \qquad B = I \tag{81-7}$$

注意这里 B 取成了单位阵而不是 M^{-1}。两种取法在数学上都成立：$B = I$ 时“输入”是力矩，$B = M^{-1}$ 时“输入”是加速度。工程上取 $B = I$ 更自然，因为力矩才是真正能下发的量；代价是全驱条件从“M^{-1} 可逆”变成了“M 可逆”，两者

由正定性同时保证。

81.2.3 控制律：让误差方程自己塌成线性

设参考 $x_d(t)$ 足够光滑（r 阶导数存在且有界），误差 $e = x - x_d$。取

$$u = B^{-1}\Big[\sum_{i=0}^{r} A_i x_d^{(i)} + g - \sum_{i=0}^{r-1} K_i e^{(i)}\Big] \tag{81-8}$$

代回 (81-6)，左右两端的 g 与 $\sum A_i x_d^{(i)}$ 对消，剩下

$$A_r e^{(r)} + \sum_{i=0}^{r-1} \big(A_i + K_i\big) e^{(i)} = 0 \tag{81-9}$$

若 A_i 是常矩阵且 A_r 可逆，左乘 A_r^{-1} 就得到一个常系数线性误差系统，极点随 K_i 任配；若 A_i 依赖状态，(81-9) 是线性时变系统，极点配置只保证“冻结系数”意义下的稳定，这一点 §81.6 再算账。

对机械臂（$r=2$，$A_0 = 0$，$A_2 = M$，$A_1 = C$），取 $K_0 = \omega_n^2 I$、$K_1 = 2\zeta\omega_n I$，(81-8) 就是

$$\tau = M(q)\big[\ddot{q}_d - K_1\dot{e} - K_0 e\big] + C(q,\dot{q})\dot{q} + G(q) \tag{81-10}$$

把它代回 (81-1)：等式两边同时出现 $C(q,\dot{q})\dot{q} + G(q)$，直接减掉，得

$$\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e = 0 \tag{81-11}$$

这一步值得停一下。**科氏项与重力项在闭环里是“自己消失”的，不是“被补偿掉的”。** 前提是控制器用的是与对象相同的模型；一旦模型不符，这两项就不再对消，(81-11) 的右端会多出一块残差。§81.2.5 就是算这块残差的账。

(81-11) 是两个解耦的二阶方程，每个关节一对共轭极点 $-\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$。$\omega_n = 15$、$\zeta = 0.9$ 对应实部 -13.5、虚部 ± 6.54，阶跃响应的 2% 调节时间约 $4/(\zeta\omega_n) = 0.30$ s，超调约 0.15%。这些数字与轨迹跟踪误差不是一回事：跟踪误差还取决于参考的加速度幅值，§81.6 会分开算。

81.2.4 稳定性：一个二次型就够了

对 (81-11) 取

$$V = \tfrac{1}{2}\dot{e}^{\mathsf T}\dot{e} + \tfrac{1}{2}\omega_n^2 e^{\mathsf T} e \tag{81-12}$$

第一项是“动能”，第二项是“弹性能”，V 正定、径向无界。沿 (81-11) 求导：

$$\dot{V} = \dot{e}^{\mathsf T}\big(-2\zeta\omega_n\dot{e} - \omega_n^2 e\big) + \omega_n^2 e^{\mathsf T}\dot{e} = -2\zeta\omega_n\|\dot{e}\|^2 \le 0 \tag{81-13}$$

交叉项恰好抵消，只剩下阻尼耗散。于是 V 单调不增，e 与 $\dot{e}$ 都有界。再用 LaSalle 不变集原理收口：$\dot{V}\equiv 0$ 意味着 $\dot{e}\equiv 0$，代回 (81-11) 得 $\omega_n^2 e \equiv 0$，即 $e \equiv 0$。原点全局渐近稳定。

这个证明的漂亮之处在于它只用到了 $K_0 > 0$、$K_1 > 0$，与 $M(q)$ 的具体形式无关——因为 (81-11) 里根本没有 M。也正因如此，它**只对精确模型成立**；模型失配时 (81-11) 变成带强迫项的方程，V 的导数会多出一项 $\dot{e}^{\mathsf T}A_r^{-1}\Delta$，此时能得到的只是一致最终有界，界的大小由 §81.2.5 给出。

顺带说明 (81-13) 的保守性：由它推出的衰减率是 $2\zeta\omega_n/\max(1,\omega_n^2) = 2\zeta/\omega_n = 0.12\ \mathrm{s}^{-1}$，比真实的 $\zeta\omega_n = 13.5\ \mathrm{s}^{-1}$ 小两个数量级。原因是 V 里“弹性能”的权重取成了 ω_n^2，与实际能量尺度不匹配；要得到紧的估计，得解 Lyapunov 方程 $\dot{V} = -x^{\mathsf T}Qx$ 后取 $\lambda_{\min}(Q)/\lambda_{\max}(P)$。本书后面凡涉及收敛速度，一律用极点实部而不是这个二次型界。

81.2.5 全驱裕度怎样进入误差界

设控制器以为的系数阵是 A_r，真实的是 $A_r+\Delta A_r$，偏置误差为 Δg。把 (81-8) 代入真实对象并整理，(81-9) 的右端多出一项：

$$e^{(r)}+\cdots+K_0 e = A_r^{-1}\big[-\Delta A_r\, x^{(r)} - \Delta g\big] \tag{81-14}$$

括号里是“模型误差折算成输入”的等效扰动。对阶跃型扰动、忽略动态，稳态误差满足 $K_0 e \approx A_r^{-1}\Delta$，取范数：

$$\|e\|_\infty \lesssim \frac{\|A_r^{-1}\|\,\|\Delta\|}{\omega_n^2} = \frac{\kappa(A_r)}{\sigma_{\max}(A_r)}\cdot\frac{\|\Delta\|}{\omega_n^2} \tag{81-15}$$

两个因子分开读。$\|A_r^{-1}\| = 1/\sigma_{\min}$ 说明**最难推的那个方向决定误差上限**；$1/\omega_n^2$ 说明误差可以被高带宽压小——但带宽受采样与噪声限制，这正是 §81.5 与 §81.7 的主题。对本章的机械臂，κ 在 1.09 到 31.36 之间，同一份模型误差在最坏构型上被放大 31 倍；这就是 §81.1.3 那个条件数在误差层面的含义，也是固定增益必须按最坏构型整定、因而在好构型上过度保守的原因。

(81-15) 还能解释一个容易被误读的现象：§81.1.1 里“参数偏 20% 时全驱误差 3.71°”，并不是“误差等于参数的 20%”。那里的偏差是有结构的——质量与长度同乘 $s=1.2$，于是质量阵与科氏项同乘 s^3、重力项乘 s^2——闭环里能对消的部分仍然对消，只剩 $O(s^3-1)$ 的残差，再被 $s^3\omega_n^2$ 除掉。分子分母同时随 s 增长，所以误差随 δ 单调但明显放缓：0.668°、1.187°、1.908°、2.348°、2.611°（$\delta=5\%$ 到 40%）。这个“放缓”是结构决定的，不是数值巧合；换一种失配方式（比如只错长度、不错质量），曲线形状会完全不同。

81.2.6 与反馈线性化的关系

第 60 章的反馈线性化要做三件事：找一个微分同胚 $z=T(x)$，验证相对度良定义，再保证零动态稳定。全驱只做一件事：在原来的坐标上把最高阶导数的系数阵乘回去。几处差别的对比见表 81-3。

表 81-3　两种“把非线性变成线性”的路数

项目	反馈线性化（第 60 章）	全驱系统（本章）
前提	相对度良定义、零动态稳定	A_r 在工作域内可逆
变换	状态坐标变换 $z=T(x)$	只做输入变换
内部动态	有零动态，需单独分析	不做坐标变换，无零动态
适用系统	方系统、相对度结构清楚	最高阶导数层仿射即可
验证方式	解析推导 + 全局性质	逐点验算矩阵的奇异性

从表 81-3 看，全驱的条件更弱，但弱得有代价：反馈线性化给出的是**全局**结论（变换在整个状态空间成立），全驱给出的是**逐点**结论（每个构型验一次可逆）。工程含义很直接——工作域越大、越靠近奇异构型，需要验的点越多，裕度也越差。这也是为什么 §81.1.3 的条件数地图不是装饰，而是这套方法的核心检查表。

还有一条必须说清的边界：**全驱不等于能控**。能控性问的是“能不能在有限时间内把状态搬到任意点”，全驱性问的是“最高阶导数这一层是否被输入直接覆盖”。小车摆系统在工作点附近线性化后是能控的，但小车加速度对摆角不是全驱；反过来，一个 A_r 处处可逆的系统未必能绕过执行器饱和。两件事各管一段，不能互相代替。

81.2.7 阶数升高会带来什么：一个三阶例子

本章用 $r=2$，因为机械臂的输入出现在加速度上。但“高阶全驱系统”这个名称里的“高阶”是认真的，值得看一眼 $r=3$ 时式子长什么样，才能理解为什么工程上很少用。

考虑 $y^{(3)} = f + bu$，y 与 u 都是标量，$b\neq 0$。按 (81-8)，控制律是

$$u = b^{-1}\big[y_d^{(3)} - f - K_2\ddot{e} - K_1\dot{e} - K_0 e\big] \tag{81-16}$$

代回后误差方程为 $e^{(3)} + K_2\ddot{e} + K_1\dot{e} + K_0 e = 0$。若把它配成三重根 $s = -\lambda$，即特征多项式 $(s+\lambda)^3 = s^3 + 3\lambda s^2 + 3\lambda^2 s + \lambda^3$，则

$$K_0 = \lambda^3, \qquad K_1 = 3\lambda^2, \qquad K_2 = 3\lambda \tag{81-17}$$

取 $\lambda = 10$，三个增益是 1000、300、30。注意它们的量纲各不相同：K_0 的单位是 s^{-3}，K_1 是 s^{-2}，K_2 是 s^{-1}。这在二阶情形里也存在（K_0 是 s^{-2}、K_1 是 s^{-1}），只是二阶的两个数都乘在“角度”上，量纲差异被 ω_n 的取值盖住了，不容易出错。阶数一高，混用单位就会直接算出错的数量级。

与二阶相比，三阶多了两样东西，都是负担。

其一，增益多了一个，但“阻尼比”这个直觉参数没了。 二阶的 ζ 有明确的物理意义（超调、相位裕度），三阶只能用“重根”或“主导极点对 + 一个远极点”来配，调参从“选两个数”变成“选一组根”，直觉变弱。

其二，需要二阶导数。 三阶控制器里的 $\ddot{e}$ 要么微分两次得到（噪声被放大 $1/T_s^2$），要么由模型算（依赖模型精度）。这正是 §81.9.6 说的“工程上很少见”的原因——理论上多一阶多一份自由度，实践上多一阶多一份噪声。

这两个负担解释了本章为什么停在 $r = 2$：不是高阶理论不重要，而是**在机械臂这类系统上，$r = 2$ 已经把“输入出现在哪一阶”这件事用尽了，再往上加只是把 $\ddot{q}$ 换成更不可测的量。**

81.2.8 计算力矩法就是全驱的一个特例

读过机器人控制的读者在看到 (81-10) 时应该会认出来：这就是计算力矩法（computed torque control），也叫逆动力学控制。它的经典写法是

$$\tau = M(q)\big[\ddot{q}_d - K_1\dot{e} - K_0 e\big] + C(q,\dot{q})\dot{q} + G(q) \tag{81-18}$$

与 (81-10) 逐项对照，两者完全一样。所以有必要说清楚：**本章讲的东西，在这类系统上不是新算法，而是同一算法的一个新框架。**

框架的差别不在公式，在三件事上。

第一，命名带来的关注点不同。 计算力矩法的教科书叙述是“先用逆动力学抵消非线性，再在得到的线性系统上配极点”——两步，第二步是设计，第一步是“抵消”。全驱的叙述是“把最高阶导数的系数阵乘回去”，只有一步。后者不承诺“抵消”这件事真的发生了，只承诺“误差方程的系数变成 $A_i + K_i$”。这个措辞上的克制很重要：模型失配时，“抵消”没有真正发生，但“误差方程”仍然成立，可以直接拿它算误差界（§81.2.5）。用“抵消”的语言去想失配，很容易得出“没抵消干净就会发散”的错误结论。

第二，适用范围不同。 计算力矩法是为“n 个关节、n 个输入、二阶、M 正定”这一类系统写的，公式里的 M^{-1} 靠正定性保证。全驱只要求最高阶导数层的系数阵可逆，不要求它对称正定，也不要求系统来自力学。$r = 3$、$r = 1$、B 非对称、甚至 B 随状态变化，同一套写法照样成立。§81.9.6 里“部分全驱”的讨论就属于后一种情形。

第三，稳定性的论证路径不同。 计算力矩法的标准论证是“抵消后系统变成线性定常，按线性系统理论配极点”；全驱的论证是“闭环的能量函数沿轨迹单调不增”（§81.6.1），不需要假设抵消成功。两条路在模型精确时给出同一个结论，在模型失配时分岔：前者只能给“抵消误差项”，后者能给出一个显式的误差界。

工程上的结论是：**如果你的系统恰好是刚性机械臂，那么你过去用计算力矩法得到的全部经验——包括它对模型精度的敏感、对测速噪声的放大、对饱和的脆弱——都可以直接搬过来，一条不用改。** 本章相对它的增量，只在“能推广到别的阶数与别的系统”以及“失配时有一个可算的界”这两点上。反过来，如果你原本觉得计算力矩法“理论漂亮但不实用”，那么全驱也不会让它在同一个对象上突然变得实用——问题出在模型与测速，不在框架。

81.2.9 B 可逆的几何含义

“$\operatorname{rank} B = n$”这句话在代数上只是“矩阵满秩”，几何上则有一个很具体的画面：**输入能把整个加速度空间张满。**

把 B 看成一个从力矩空间 $\mathbb{R}^m$ 到加速度空间 $\mathbb{R}^n$ 的线性映射。$\operatorname{rank} B = n$ 意味着这个映射是满射——给定任意一个想要的加速度向量 v，都能找到一个力矩 $u = B^{-1}A_r v$ 把它实现出来。反过来，若 $\operatorname{rank} B < n$，加速度空间里就有一个“够不着”的子空间，无论给多大力矩，某些方向的加速度都改不动。**欠驱动系统的全部困难，都从这个够不着的子空间开始。**

$\kappa(B)$ 在这个画面里是“拉伸比”。B 把单位球映成一个椭球，最长半轴与最短半轴之比就是 κ。它衡量的是**各向异性**：某个方向的加速度“便宜”（单位力矩换来的加速度大），另一个方向“贵”。全驱只要求椭球不退化成一条线段，κ 则告诉你最贵的那个方向贵多少倍。

二连杆的这个椭球有一个直白的物理解释。M 的两个特征值对应两个“自然加速方向”：一个是整条手臂绕肩同向摆动，惯量大、加速贵；另一个是两关节反向配合、末端几乎不动的“内部运动”，惯量小、加速便宜。κ 就是这两个方向惯量的比值。手臂伸直时这个比值最大（31.36）：此时“内部运动”几乎不带动末端质量，等效惯量最小，而整臂摆动仍要带动全部质量。**所以“接近奇异”在机械臂上不是“力矩不够”，而是“两个方向的轻重差别太大”。**

这里有一个必须提醒的细节：**κ 不是坐标无关的。** 如果对关节变量做一个线性变换 $x' = Tx$（比如把角度换成“减速器前”的角度），则 $B' = BT^{-1}$，条件数会变。也就是说，同一台机器，用不同的变量描述，κ 可以差出好几倍。这与“物理性能不应依赖描述方式”的直觉冲突，但它并不矛盾：**误差的度量也随坐标变了。** 换坐标后 κ 变小，通常意味着“某些方向的误差被赋予了更大的权重”，闭环的实际表现并不会因此变好。

工程含义有两条。其一，**报告 κ 时必须说明用的是什么变量与单位**，否则这个数不可比。其二，**不要试图靠“换个坐标”来改善全驱性**——那只是把代价挪到误差的定义里；真正要改善 κ，只能改机构（缩短力臂差、加配重、改关节布置）。

81.3 设计步骤

把 §81.2 的推导倒过来，就是工程上真正要走的路。这一节给五步，每步都写清“输入是什么、输出是什么、怎么算做完了”。五步走完，控制器就唯一确定了——这是这套方法最讨人喜欢的地方：没有需要反复试凑的旋钮，只有需要逐条核对的清单。

81.3.1 第一步：确定“对哪一阶导数全驱”

输入是物理方程与执行器位置，输出是一个阶数 r 与一个被控量 x。

判断方法只有一句话：**输入最早出现在哪一阶导数上，r 就是那一阶。** 力矩出现在角加速度上，所以机械臂 $r = 2$、$x = q$。电机若以绕组电压为输入、以电流为被控量，$r = 1$（$L\,\mathrm{d}i/\mathrm{d}t = v - Ri - k_e\omega$）；若仍以电压为输入、改以转速为被控量，则 $r = 2$，因为电压先改电流、电流再经力矩改转速。

还有一个容易走错的岔路：换被控量会改变全驱条件。若把机械臂的被控量从关节角换成末端位置 $p = h(q)$，则 $\ddot{p} = J(q)\ddot{q} + \dot{J}(q,\dot{q})\dot{q}$，全驱条件从“$M$ 可逆”变成“$J(q)M^{-1}$ 可逆”，也就是雅可比非奇异。在工作空间边界附近、手臂伸直时，雅可比接近奇异，这个条件先于 M 失效。**同一台机器，对关节角全驱，对末端位置未必。** 这就是 §81.1.2 强调“对谁全驱”的实际后果。

81.3.2 第二步：整理成拟线性形式

输入是 r 与 x，输出是系数阵 $A_i(\cdot)$、偏置 $g(\cdot)$ 与输入阵 $B(\cdot)$。两条实用规则：

- 凡是“系数只依赖状态、不依赖输入”的项，一律归进某个 A_i。能写成 $A_i x^{(i)}$ 的，不要留在 f 里——留着就等于放弃了对它的显式利用。
- 常值项、重力项、已知外扰归到 g 里，不要硬塞进 A_0。硬塞会让 A_0 在状态过零时失去意义。

对机械臂，结果就是 (81-7)：$A_2 = M(q)$、$A_1 = C(q,\dot{q})$、$A_0 = 0$、$g = G(q)$、$B = I$。这里 A_2 不是常数阵，拟线性形式允许；但 §81.2.4 的稳定性证明要求常系数，所以这一步要顺手记下“哪些 A_i 随状态变”，它们是后面要单独交代的账。

81.3.3 第三步：验可逆，并量出裕度

输入是 A_r 的表达式与工作域，输出是一张 κ 地图和一个最坏值。

做法是在工作域上采样，逐点算 $\sigma_{\min}$、$\sigma_{\max}$，画成条件数地图。本章用 121×121 个构型，得到 $\kappa \in [1.09, 31.36]$，中位 5.83。这一步的完成判据不是“处处非奇异”——那是数学判据，工程上没意义——而是**“处处 κ 小于一个可接受的阈值”**。

阈值怎么定有一条可操作的规则：$\kappa_{\max}$ 应小于执行器的力矩余量倍数。若电机只有 2 倍余量，κ 超过 2 的工作区就要么缩小行程、要么改构型、要么加冗余驱动。本章的 31.36 远大于 2，但算例的负载轻、力矩上限 25 N·m 相对宽裕，所以仍能跑；换一台重载机械臂，这个数字会直接把方案否掉。

81.3.4 第四步：配极点，但带宽由外部约束定

输入是 κ 的范围、采样周期、噪声水平、力矩上限，输出是 K_i。

(81-11) 说闭环极点可以放在任意位置，但“能放”不等于“该放”。三处约束各给一个上界：采样要求 $\omega_n T_s$ 足够小，本章 1 ms 下由力矩上限卡出的临界带宽是 61.9 rad/s（§81.5 会给出 0.5 到 40 ms 的完整曲线）；测速噪声要求 ω_n 不能太大，因为 $K_1 = 2\zeta\omega_n$ 直接乘在速度噪声上（§81.7）；力矩上限要求跟踪同一轨迹的峰值力矩不超限，而峰值随 ω_n 单调增长。

本章取 $\omega_n = 15$、$\zeta = 0.9$：相对 61.9 rad/s 的饱和上界留了 4 倍余量，相对采样限制宽松得多，量化加 5 ms 滤波后的力矩纹波在 0.295 到 1.227 N·m 的区间内（$5 \le \omega_n \le 20$，见 §81.7）。这是一条“三头都不紧”的选择，不是最优点；§81.8 的增益地图会说明理想模型下把 ω_n 推到 28 误差更小（0.192°，对比 $\omega_n = 5$ 时的 0.458°），但加入量化后 28 的误差涨到 0.410°，反而不如 14 的 0.310°。

81.3.5 第五步：检查可实现性

这一步最琐碎，也最容易翻车。五件事逐条打勾：

1. **微分从哪来。** $\dot{q}$ 是测的还是算的？编码器量化多大？后向差分会把量化噪声放大 $1/T_s$ 倍，滤波又会引入滞后，两者都要量化（§81.7）。
2. **限幅与抗饱和。** 力矩到上限时，前馈项与反馈项各让多少？本章直接对总力矩限幅，简单但会在饱和瞬间破坏 (81-10) 的结构。
3. **计算延迟。** 求逆加三次矩阵乘占采样周期多少？本章 4.8 μs 对 1 ms 是 0.48%，可以忽略；若控制周期是 100 μs，这个比例就变成 4.8%，需要重新评估。
4. **参数从哪来。** M、C、G 的参数是辨识的还是铭牌值？误差量级多少？本章的 20% 失配实验就是为这一步准备的参照（§81.6）。
5. **失效模式。** 某个执行器失效时系统会怎样？§81.10 的图 81-10 是一个具体答案：关节 2 力矩归零后 3.08 s 内误差越过 11°，两个关节一起失控。

五步的检查项汇总在表 81-4。

表 81-4　全驱设计的五步检查表

步骤	输入	输出	完成判据
一、定阶	物理方程、执行器位置	r 与被控量 x	输入确实出现在 $x^{(r)}$
二、整理	r、x	A_i、g、B	无非线性项被藏在 f 里
三、验可逆	$A_r(\cdot)$、工作域	κ 地图	处处 $\kappa < \kappa_{\text{允许}}$
四、配极点	κ、T_s、噪声、力矩上限	K_i	三处上界都有余量
五、查实现	传感器、执行器、算力	可运行的控制器	五项逐条打勾

81.3.6 这套流程的隐含假设

五步能走通，前提是**模型结构已知**：知道 M、C、G 的函数形式，只是参数待定。这一点常被忽略。若结构本身错了——比如连杆有柔性、谐波减速器有回差、摩擦是 Stribeck 型而非粘滞型——第一步和第二步就会把错误固化进 A_i 里，第三步验不出问题，第四步配出的极点在真实对象上根本不存在。

遇到这种情况，两条路：一是把未建模部分降阶成扰动，用 §81.6 的误差界去兜；二是回到第 80 章，用数据去学那块残差。全驱不排斥学习，它只是要求“学完之后，剩下的部分仍然可以写成拟线性形式”。本章的立场是：**结构靠建模，参数靠辨识，残差靠鲁棒或学习；三者缺一，全驱就只剩一个漂亮的推导。**

下面这段是五步走完后的控制律本体，与 §81.12 的 `ctrl_fas` 一一对应：

```
q, qd = measure()                    # 位置与速度（含量化、可能带滤波）
e, ed = q - q_d, qd - qd_d
v = qdd_d - K1 @ ed - K0 @ e         # 期望加速度
tau = M(q) @ v + C(q, qd) @ qd + G(q)  # 换算成力矩
tau = clip(tau, -tau_max, tau_max)     # 限幅
```

五行里有四处依赖模型（M、C、G 与参考的二阶导），一处依赖测量（q、$\dot q$）。§81.4 到 §81.7 会逐个说明：这五处里哪一处最先出问题。

81.3.7 一个纸面案例：单关节先把流程走一遍

在动手写代码之前，用一个单关节把五步走一遍，能暴露很多“以为懂了”的地方。

设一个经减速器驱动的关节，折算到关节侧的方程是 $J\ddot\theta + b\dot\theta = \tau$，取 $J = 0.02$ kg·m²、$b = 0.02$ N·m·s/rad。

第一步，定阶。 力矩出现在 $\ddot\theta$ 上，$r = 2$，被控量是 θ。

第二步，整理。 $A_2 = J$、$A_1 = b$、$A_0 = 0$、$g = 0$、$B = 1$。这里 A_2 是标量，所以“矩阵可逆”退化成“$J \neq 0$”。

第三步，验可逆。 标量的条件数恒等于 1。这一步在单关节上是空的——**这正是单关节无法演示全驱价值的原因**：没有构型依赖，就没有 κ 的戏。

第四步，配极点。 取 $\omega_n = 30$、$\zeta = 1$，得 $K_0 = 900$、$K_1 = 60$。控制律是

$$\tau = J\big(\ddot\theta_d - K_1\dot e - K_0 e\big) + b\dot\theta \tag{81-19}$$

代回对象：$J\ddot\theta + b\dot\theta = J(\ddot\theta_d - K_1\dot e - K_0 e) + b\dot\theta$，$b\dot\theta$ 两边对消，得 $\ddot e + 60\dot e + 900 e = 0$。**阻尼项 b 和前馈项一起消失了**，这与 §81.2.3 里科氏项消失是同一件事。

第五步，查实现。 参考取 $\theta_d = 0.5\sin 2t$，$\ddot\theta_d$ 的幅值是 2.0 rad/s²。前馈力矩 $J\ddot\theta_d = 0.04$ N·m；若位置误差 0.01 rad，反馈力矩 $JK_0 e = 0.18$ N·m。两者都在毫牛米级，一个几十瓦的电机绰绰有余。

这个案例的结论有点扫兴：**单关节上，全驱 = “惯量乘 PD + 加速度前馈”，而后者在经典控制里早就有名字（计算力矩控制的单变量版）。** 它相对纯 PD 的优势只剩前馈那一项，按 (81-22) 估，PD 的误差约 $J\ddot\theta_d/K_0 = 0.02 \times 2/900 = 4.4\times10^{-5}$ rad，即 0.0025°，本来就很小。

真正的差距要到多变量才出现：本章二连杆的质量阵两个特征值在 0.047 到 1.48 kg·m² 之间，都远大于单关节的 0.02，且随构型变；此时“乘不乘 $M(q)$”不再是小数点后的事，而是 0.244° 与 0.0024° 的差别。**全驱的价值不在“有前馈”，而在“前馈用的是随构型变的正确系数阵”。** 这句话值得在读完 §81.4 之后回头再看一遍。

81.3.8 动手之前要收齐的九个数

五步流程听起来短，但真正卡住项目进度的往往不是推公式，而是“拿不到数”。下面这九个数在写第一行代码之前就该收齐，并且要写清每个数进到哪一个式子、缺了它会怎样。

几何与质量（四个）：连杆质量 m_i、连杆长度 l_i、质心位置 r_i、绕质心的转动惯量 I_i。 前两个决定 M、C、G 的骨架，后两个决定 M 的对角量级与重力力臂。CAD 里都能查到，但 CAD 给的是设计值，不含线缆、接头、工装；实机的转动惯量常比 CAD 大 10%–30%。这一步的偏差会直接变成 §81.6.2 的 δ。

传动（两个）：减速比 N 与传动效率 η。 它们决定“电机侧的数怎么折算到关节侧”。折算规则只有三条，但极容易记混：惯量乘 N^2、力矩乘 $N\eta$、角度除以 N。把电机转子惯量 J_m 折算到关节侧是 $N^2 J_m$——一台 $N = 50$ 的减速器会把 0.0001 kg·m² 的转子惯量放大成 0.25 kg·m²，与前文提到的连杆惯量同量级。**忽略转子惯量，是最常见的“模型看起来对、实测差一截”的原因。**

测量（两个）：编码器分辨率与安装侧。 分辨率决定量化步长 q，安装侧决定 q 要不要除以 N。电机侧的读数要除以 N 才变成关节角，同一个 q 折算到关节侧就细了 N 倍——这是工业臂普遍把编码器装在电机侧的原因之一（另一个原因是可以避开减速器回差）。本章取 $q = 6.3 \times 10^{-4}$ rad（约 10000 线/转），对应的测速噪声由 (81-35) 给出。

执行器（一个）：关节侧力矩上限 $\tau_{\max}$。 电机额定力矩乘 $N\eta$ 得到连续值，峰值倍数（通常 2–3 倍）得到瞬时值。全驱控制器用的是瞬时值，但 §81.8.3 的调参要用额定值做长期约束——**用峰值定参数、用额定值做验收**，这两条不能混。

九个数齐了，五步流程可以在半天内走完；缺任何一个，就会在“仿真收敛、实测不收敛”的地方停下来，而且很难判断是算法问题还是参数问题。经验上，**这九个数里最容易错的是转子惯量的折算与编码器的安装侧**，两者都属于“量纲没错、倍数错了”的类型，靠看公式发现不了，只能靠实测比对。

81.4 连续时间情形

先把连续域说清楚，再谈离散实现。连续域的好处是 (81-11) 严格成立，可以干净地看出全驱与 PD 的差别到底来自哪里。

81.4.1 四条闭环曲线

回到本章算例。初始时刻两个关节各有 0.02 rad（约 1.15°）的初始偏差，参考轨迹是幅值 0.7 rad 与 0.6 rad、频率 1.0 rad/s 与 1.3 rad/s 的两条正弦，仿真 6 s。四种组合各跑一遍：全驱与 PD，模型精确与参数整体偏 20%。图 81-4 是四条误差曲线，纵轴是误差向量的范数，取对数。

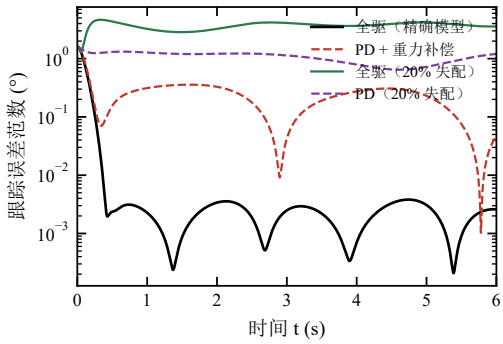


图 81-4　四种情形下的跟踪误差：全驱与 PD，精确模型与 20% 失配

按“0.5 s 之后”的稳态均方根统计，四个数字如表 81-5。

表 81-5　四条闭环的稳态误差（$t \geq 0.5$ s 的均方根）

情形	稳态误差	相对精确全驱
全驱 + 精确模型	0.0024°	1
PD + 精确模型	0.244°	102
全驱 + 20% 失配	3.714°	1548
PD + 20% 失配	1.061°	442

这张表值得横竖各读一遍。横着读：模型精确时全驱比 PD 好两个数量级；同一份 20% 失配下，全驱反过来比 PD 差三倍半。竖着读：全驱自己的误差涨了 1548 倍，PD 只涨 4.3 倍。前一个数字说明全驱的"底线"极低，所以对失配格外敏感；后一个数字才是工程上真正在意的差距。§81.2.5 的 (81-15) 解释了为什么绝对量并不大的失配，足以把这个极低的底线顶到 PD 之上。

81.4.2 PD 的稳态误差从哪来

PD + 重力补偿的力矩是

$$\tau = -K_0 e - K_1 \dot{e} + G(q) \tag{81-20}$$

代回 (81-1)：

$$M(q)\ddot{e} + \big[C(q,\dot{q}) + K_1\big]\dot{e} + K_0 e = -\big[M(q)\ddot{q}_d + C(q,\dot{q})\dot{q}_d\big] \tag{81-21}$$

右端不是零，负号说明误差与参考加速度反相——机械臂"跟不上"，落后于参考。物理含义很直白：要让机械臂按参考轨迹运动，需要一份"加速惯性 + 科氏"的力矩，而 PD 控制器只知道"误差多少、补多少"，这份前馈它拿不出来。于是必须靠一个非零的误差去"换"出这份力矩。稳态时导数项可以忽略，误差约为

$$e \approx -K_0^{-1}\big[M(q)\ddot{q}_d + C(q,\dot{q})\dot{q}_d\big] \tag{81-22}$$

本章参数下 M 的最大奇异值是 1.48 kg·m²（手臂伸直时），$\ddot{q}_d$ 的幅值约 0.7 rad/s²，$K_0 = \omega_n^2 = 225$，估算出 e 的峰值约 $1.48 \times 0.7/225 = 4.6 \times 10^{-3}$ rad，即 0.26°。**实测 0.244°，与估算相当接近**——差的那一成来自两处：$C\dot{q}_d$ 与 $M\ddot{q}_d$ 并不总是同相，可能相互抵消；稳态统计取的是 RMS 而不是峰值。这条估算虽然粗，但它给出的量级是对的，而且它明确指出了误差与 $\ddot{q}_d$ 成正比、与 K_0 成反比这两个可用的关系。

PD 的稳态误差不是调参能消掉的，它由"没有前馈"这个结构事实决定，只能靠增大 K_0 压小，而 K_0 又受稳定裕度与噪声限制。要彻底消掉，要么加积分（第 44 章的 PID），要么加前馈——全驱做的正是后者。

81.4.3 全驱的误差也不是零

全驱精确模型的 0.0024° 不是数值零，它来自三处：一是 (81-10) 里用了参考的 $\ddot{q}_d$，而实际轨迹在离散积分下有 $O(T_s^2)$ 的差异；二是 RK4 积分本身的截断误差；三是 $t = 0$ 的初始偏差在 0.5 s 后还残留约 $e^{-13.5\times 0.5} = 1.2 \times 10^{-3}$ 倍。第三项是主因：$1.15^\circ \times 1.2 \times 10^{-3} \approx 0.0014^\circ$，与实测同量级。

这也解释了为什么 §81.1.1 里"100 倍"的差距不能简单理解成"全驱的跟踪能力强 100 倍"。真实差别是：**PD 的误差由结构决定、有个地板；全驱的误差由初值与数值精度决定、没有地板但可以被模型误差顶起来。** 后者在模型准确时降到极低，模型不准时立刻反弹到地板上方——表 81-5 第三行就是反弹后的结果。

81.4.4 连续域结论的适用条件

(81-11) 与 (81-22) 都建立在三个条件上：模型精确、$\dot{q}$ 精确可得、力矩不受限。真实系统三条都不满足。§81.5 放开采样，§81.7 放开测速，§81.6 放开模型，§81.10 放开力矩与执行器。每放开一条，(81-11) 就退一步，退到哪一步、退多少，是后面四节的内容。

这里先给一个总的判断：**连续域的分析不是"理想化后的空谈"，它给出了三个可以量化的基准**——零误差的下界（由模型误差决定）、PD 的误差地板（由 K_0 与前馈缺失决定）、以及失配敏感度的放大倍数（由 κ 决定）。后面所有工程折衷，都是在这个基准上做减法。

81.4.5 频域看同一件事

前面用时间域说了两遍，换成频率域会更清楚，也更容易被验证。

对全驱，模型精确时闭环满足 $\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e = 0$，初值为零时 $e \equiv 0$。也就是说，从参考到输出的传递函数**恒等于 1**：

$$\frac{q(s)}{q_d(s)} = 1 \tag{81-23}$$

这个结论初看很反常：任何控制器都不可能在全频段完美跟踪，怎么这里传函是 1？原因是 (81-10) 的第一项已经把对象的逆乘了进去——**它不是在“补偿”对象，而是在“抵消”对象**，抵消之后剩下的只有误差动态。所以 ω_n 在这里不决定“能跟多快”，只决定“初值误差衰减多快”。

对 PD，情况完全不同。由 (81-21)，误差传函是

$$\frac{e(s)}{q_d(s)} = \frac{Ms^2 + Cs}{Ms^2 + (C + K_1)s + K_0} \tag{81-24}$$

低频段可以近似成 Ms^2/K_0，误差随参考频率的平方增长。本章参考频率 1.0 rad/s 与 1.3 rad/s，M 的最大奇异值 1.48 kg·m²，$|e|$ 约 $1.48 \times 1/225 \times 0.7 = 4.6 \times 10^{-3}$ rad，即 0.26°，与 §81.4.1 实测的 0.244° 吻合。

这个对比给出一条可以自己验证的预测：**把参考频率从 1.0 提到 3.0 rad/s，PD 的误差应涨约九倍；全驱在模型精确时应几乎不变。** 验证方法很简单，改 `REF` 里的 `w1`、`w2`，重跑 `experiment_tracking` 即可。如果实测与预测不符，先怀疑参考的加速度是否仍然连续，再怀疑初始偏差是否被算进了稳态。

频域还有一个用处：它能解释为什么“提高带宽”对全驱的跟踪误差几乎无效（传函是 1），却对初值恢复有效（极点位置决定衰减率）。§81.4.1 的四条曲线在起始段都由初值主导、形状相似，到稳态段才分开——分开的原因不是带宽，而是各自右端有没有那块“缺失的前馈”。

81.4.6 参考的形状决定 PD 的误差

(81-22) 的右端是 $-\left[M(q)\ddot{q}_d + C(q,\dot{q})\dot{q}_d\right]$，里面只有参考的**一阶与二阶导数**，没有参考本身。这个细节决定了一件容易被忽略的事：**PD 的稳态误差不是一个固定值，它随参考的形状变，而且可以为零。**

阶跃参考。 目标停住不动，$\dot{q}_d \equiv 0$、$\ddot{q}_d \equiv 0$，右端恒为零。(81-21) 变成齐次方程，误差按 $e^{-\zeta\omega_n t}$ 衰减到零。也就是说，**PD 跟一个固定的目标点，稳态误差在理论上就是零**，不需要积分，也不需要前馈。这一点常被“PD 有稳态误差”的说法掩盖，实际上那句话只对运动参考成立。

斜坡参考。 $\ddot{q}_d \equiv 0$ 但 $\dot{q}_d$ 是常值，右端剩 $-C(q,\dot{q})\dot{q}_d$。科氏项与速度的平方同量级，$C\dot{q}_d$ 一般远小于 $K_1\dot{e}$，所以误差很小但非零。物理含义是“匀速运动时对象需要一个科氏力矩，PD 只能靠一点误差去换”。

正弦参考。 本章的情形，三项都非零，右端以参考频率振荡，误差也以同一频率振荡，幅值约为 $\left\|M\ddot{q}_d + C\dot{q}_d\right\|/K_0$。这就是 (81-22) 能把 0.244° 估到 0.26° 的原因：$\|M\|$ 取最大奇异值、$\ddot{q}_d$ 取峰值，两个“最大”同时取到时估出来的正是上限量级，而 RMS 比峰值低一成。

这里有一处容易踩的坑：**$\|M\|$ 要取最大奇异值，不能取最小。** 本章 M 的奇异值从 0.047 到 1.48，差了 31 倍；若误用 0.047 去估，会得到 0.008°，比实测低三十倍，然后得出“公式不准”的错误结论。误差的放大由“最重的方向”决定，不是由“最轻的方向”决定。

这个分析给出一个可以写进选型报告的判断：**PD 的误差与参考加速度成正比。** 把节拍从 2 s 提到 1 s，加速度幅值涨四倍，PD 的误差大致也涨四倍；全驱在模型精确时几乎不动，因为它的传函是 1（(81-23)）。所以**高速高精的场合，全驱的优势最大；低速定位的场合，PD 加前馈已经够用，全驱只能改善过渡过程。** 这也解释了为什么本章要选正弦参考：它把 q_d、$\dot{q}_d$、$\ddot{q}_d$ 三项同时激励起来，才看得出两种控制器的结构差别；换成阶跃，表 81-5 的 102 倍会缩成一个只与调节时间有关的比值。

最后要提醒一句：不要把“阶跃下 PD 无稳态误差”当成“PD 不需要前馈”。阶跃的稳态虽然无差，但**过渡过程里 PD 仍然慢**——它要靠误差长起来才有力矩，而全驱在参考跳变的同一拍就把加速度前馈送出去了。稳态与过渡是两件事，本章的稳态数据只说明前者。

81.4.7 常值扰动进来会怎样

前面所有分析都假设“除了参考，没有别的力矩进系统”。真实关节上至少还有两样：摩擦与线缆拉力。它们不随参考变，是常值或慢变的，于是引出一个本章算例刻意回避、但工程上必然遇到的问题。

设关节上有一个常值扰动力矩 d。全驱的闭环（模型精确）变成

$$J\ddot{e} + 2\zeta\omega_n J\dot{e} + \omega_n^2 J e = d \tag{81-25}$$

稳态时导数项为零，得 $e = d/(J\omega_n^2)$。这个式子有两层含义。

第一层：全驱没有积分，所以常值扰动一定有静差。 这一点与 PD 相同，两者都是“比例型”的刚度；要消掉静差必须加积分（第 44 章）或扰动观测器。本章的 0.0024° 之所以那么小，前提就是**没有摩擦、没有线缆力**——§81.1.7 已经把这个前提写明了，这里给出它的代价。

第二层，也是更意外的一层：全驱的等效刚度随构型与方向变，PD 的不变。 全驱的刚度是 $J(q)\omega_n^2$，本章 J 的奇异值在 0.047 到 1.48 kg·m² 之间，于是等效刚度在 11 到 333 N·m/rad 之间浮动；PD 的刚度是常数 $K_0 = 225$ N·m/rad，恰好落在中段。**对同样的 1 N·m 常值扰动，全驱的静差在 5.4°（轻的方向）到 0.17°（重的方向）之间，PD 恒为 0.25°。** 也就是说，全驱在重的方向上更硬，在轻的方向上软得多，而 PD 两个方向一样硬。

这个反差不是算法缺陷，而是两种“归一化方式”的必然结果。PD 的 K_0 是绝对刚度，对轻的方向与重的方向一视同仁；全驱的增益乘在 M 之后，是“每单位惯量的刚度”，轻的方向自然软。**全驱保证的是“同样的加速度指令误差对应同样的加速度偏差”，PD 保证的是“同样的位置偏差对应同样的力矩”。** 本章前面比较的 0.0024° 与 0.244° 属于前者；如果换成“恒定风力下谁偏得少”，结论就要按方向分开算。

工程上的处理有三条，按代价排序：把摩擦建进模型做前馈（最便宜，效果取决于摩擦模型的准确度）；加积分项（会引入相位滞后与积分饱和，需要抗饱和配合）；加扰动观测器（本质是把“模型没算到的力矩”估出来再前馈，代价是需要带宽）。三条都与全驱不冲突，可以叠加。**唯一不能做的是“什么都不加，然后指望提高 ω_n 来压静差”**——按 $e = d/(J\omega_n^2)$，ω_n 翻倍静差才降到四分之一，而噪声按 $K_1\sigma_v$ 线性上涨，两边一算就知道不划算。

81.5 离散时间情形

工程上没有人真的按连续时间跑控制器。全驱的离散实现有两条路：一条是“先设计后离散”——用 (81-10) 算出连续控制器，再按控制周期采样执行；另一条是“先离散后设计”——把 (81-6) 离散成差分方程，在 z 域配极点。本章的代码走第一条，因为它在工程上更常见，也更容易看出采样到底吃掉了多少性能。

81.5.1 误差几乎不随采样周期变

固定 $\omega_n = 15$、$\zeta = 0.9$，把控制周期从 0.5 ms 拉到 40 ms，跟踪误差的 RMS 是：0.262°、0.262°、0.263°、0.263°、0.265°、0.269°、0.279°（对应 0.5、1、2、5、10、20、40 ms）。**八十倍的采样周期变化，只换来 6% 的误差增长**，而且七种周期全都收敛。

这个结果容易让人得出“采样周期随便选”的结论，但那是误读。$\omega_n = 15$ 时，$T_s = 40$ ms 对应的 $\omega_n T_s = 0.6$，离稳定边界还有一倍距离；误差不涨，是因为在这个带宽下离散化还没有开始咬人。真正的问题是：**这个带宽还能往上提多少？** 答案由图 81-6 给出。

81.5.2 三条上界，谁更紧谁说话

图 81-6 画了三条“临界带宽对采样周期”的曲线。第一条按实际力矩上限 25 N·m 测，用二分法找“误差不发散”的最大 ω_n；第二条把上限放宽到 500 N·m，用来排除饱和；第三条是离散系统的经典判据 $2\zeta/T_s$，它只保证极点不越出单位圆，不管饱和与数值实现。

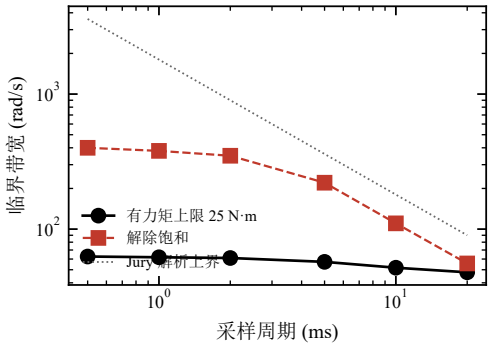


图 81-6　临界带宽随采样周期的变化：饱和限制、采样限制与 Jury 上界

表 81-6　四条采样周期下的临界带宽（rad/s）

T_s	25 N·m	500 N·m	10^9 N·m	$\omega_n T_s$（500 N·m 列）	$2\zeta/T_s$
1 ms	61.9	379.6	1111.2	0.38（仍被饱和卡）	1800
5 ms	57.2	220.5	—	1.10	360
10 ms	51.8	110.3	—	1.10	180
20 ms	47.9	55.7	—	1.11	90

表里有三件事值得说。

第一，25 N·m 那一列几乎不随 T_s 变。 从 1 ms 到 20 ms，临界带宽只从 61.9 降到 47.9，降了 23%。这说明在这个力矩预算下，卡住带宽的不是采样，是饱和：想更快，就要求更大力矩，而力矩已经到顶。

第二，把饱和解除后，T_s 的影响立刻显现。 500 N·m 一列从 379.6 掉到 55.7，相差 6.8 倍，与 T_s 的 20 倍变化同向。此时限制换成了采样。

第三，$2\zeta/T_s$ 这条经典上界偏松。 它给出的 1800、360、180、90 分别是实测（500 N·m）的 4.7、1.6、1.6、1.6 倍。松的原因很直接：Jury 判据只看线性系统的极点位置，而实际闭环里还有零阶保持的相位滞后、参考前馈的离散误差，这些都会提前把系统推过边界。

81.5.3 一条可用的经验规则

把 500 N·m 那一列除以对应的 T_s，得到的不是三个不同的数，而是同一个数：1.10、1.10、1.11。也就是说，**这套控制器的采样极限可以写成一条与 T_s 无关的规则：**

$$\omega_n T_s \lesssim 1.1 \tag{81-26}$$

这条规则很好用：先按 §81.3.4 的饱和约束定出想要的带宽，再用 (81-26) 反算所需的采样周期。比如要 $\omega_n = 100$ rad/s，就需要 $T_s \le 11$ ms；要 $\omega_n = 500$ rad/s，就需要 $T_s \le 2.2$ ms。本章的 $\omega_n = 15$ 只需要 $T_s \le 73$ ms，所以 1 ms 有 73 倍余量——这也是 §81.5.1 里误差几乎不变的原因。

(81-26) 的常数 1.1 不是普适常数，它随阻尼比变：ζ 越小，允许的 $\omega_n T_s$ 越小。用 2ζ 作粗略换算，$\zeta = 0.9$ 给 1.8，实测 1.1，比例约 0.6。工程上取 $\omega_n T_s \le 0.5$ 更稳妥，本章 1 ms 对应的 0.015 远在这个范围内。

81.5.4 半拍滞后与计算延迟

零阶保持本身等效于半个采样周期的纯滞后，相位损失约 $\omega T_s/2$。本章 $\omega_n = 15$、$T_s = 1$ ms，损失 0.0075 rad（0.43°），可以忽略；若换成 $T_s = 10$ ms，损失 0.075 rad（4.3°），开始占用相位裕度；到 $T_s = 50$ ms 就是 0.375 rad（21°），占掉了 $\zeta = 0.9$ 对应相位裕度（约 73°）的三成。

计算延迟要单独算，不能混进 T_s。本章单步 4.8 μs，相对于 1 ms 是 0.48%，把它折成相位是 $\omega_n \times 4.8\,\mu s = 7.2 \times 10^{-5}$ rad，可以完全不管。但如果控制周期缩到 100 μs，这 4.8 μs 就变成 4.8%，而且此时 ω_n 通常也得跟着提高，两项叠加就不能再忽略。

81.5.5 本章为什么取 1 ms

三个原因，按重要性排序。其一，1 ms 让 (81-26) 的余量达到 73 倍，采样彻底不成为约束，可以把注意力全部放在饱和、噪声与模型误差上；其二，1 ms 与常见的伺服驱动电流环周期同量级，工程上拿得到；其三，1 ms 下差分测速的噪声量级（$\sigma_v = 0.257$ rad/s）与真实编码器相当，噪声机理才显现得出来——若把周期放到 10 ms，差分噪声会被 $1/T_s$ 压小十倍，反而看不出问题。

换句话说，1 ms 不是“最优采样周期”，而是“让其他三个限制各自独立显现”的采样周期。这是一种实验设计上的取舍，不是控制设计上的结论。

81.5.6 另一条路：直接在 z 域配极点

本章走“先设计后离散”，另一条路是“先离散后设计”。做法不复杂：把 (81-11) 写成状态空间

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} e \\ \dot{e} \end{bmatrix} = \begin{bmatrix} 0 & I \\ -\omega_n^2 I & -2\zeta\omega_n I \end{bmatrix}\begin{bmatrix} e \\ \dot{e} \end{bmatrix} \tag{81-27}$$

再按零阶保持离散，z 域极点是 $z = e^{sT_s}$，即模 $e^{-\zeta\omega_n T_s}$、相角 $\pm\omega_n\sqrt{1-\zeta^2}\,T_s$。取 $\omega_n T_s = 1.1$、$\zeta = 0.9$，极点模是 $e^{-0.99} = 0.372$，相角约 ± 0.65 rad（37°）——落在单位圆内偏左的位置，是一个很“温和”的离散极点。

两条路的差别只在 T_s 大时才显现。T_s 小时，$e^{AT_s} \approx I + AT_s$，两者给出同一组增益；$T_s$ 大时，直接离散设计能准确算出“离散极点该放哪”，而“先设计后离散”只是把连续增益照抄过来，实际极点会偏离设计值。不过 §81.5.3 的实测表明，两条路的临界 $\omega_n T_s$ 都在 1.1 附近，因为限制来自零阶保持的相位滞后，而不是增益算法。

直接离散设计有两个额外好处和两个额外代价。好处是：可以把计算延迟作为一个增广状态精确建模（延迟一拍就是 z^{-1}，直接进 Φ），也可以把指标（超调、调节时间）直接写成 z 域约束。代价是：$M(q)$、$C(q,\dot{q})$ 随构型变，每次采样都要重算 Φ、Γ，计算量比“一次求逆”大一个量级；而且时变系统的“逐点离散”没有全局稳定性保证，冻结系数分析的局限又回来了。

工程建议：**T_s 小于 $1/(10\omega_n)$、延迟小于半拍时，用“先设计后离散”；否则考虑直接离散设计，或干脆降低带宽。** 本章 $T_s = 1$ ms、$\omega_n = 15$，$\omega_n T_s = 0.015$，属于前者，所以代码里没有做逐点离散。

81.5.7 真实系统是多率的

到目前为止本章默认“一个周期干所有事”。真实机器上至少有三种时钟：轨迹规划 4–10 ms、位置/速度环 1 ms、电流环 100 μs（10 kHz）。全驱控制器通常落在中间那一档，于是“它和上下游怎么对接”成了比采样定理更容易出问题的地方。

向下对接：力矩怎么送到电流环。 控制器每 1 ms 算出一组力矩，电流环每 100 μs 读一次。两者之间必然有一个保持环节，默认是零阶保持：这一个毫秒内的十个电流环周期都执行同一个力矩。这就是 §81.5.4 那半拍滞后的来源，本章 $T_s = 1$ ms、$\omega_n = 15$ 时只损失 0.43°，可以不管。有些伺服支持线性插值（把上一拍与这一拍的力矩值连线），等效延迟从半拍降到四分之一拍，代价是力矩变化率受限——对本章这种 4.8 μs 就能算出新值的场合，收益很小。

如果不做保持、而是让电流环自己按 100 μs 重算力矩，就要求控制器也跑到 100 μs。此时 §81.5.3 的规则变成了硬约束：$\omega_n T_s \le 1.1$ 给出 $\omega_n \le 11000$ rad/s，看上去很宽，但 4.8 μs 的计算量占 100 μs 周期的 4.8%，再加上采样抖动与中断延迟，实际余量只剩一半。**“能不能跑这么快”和“跑这么快还有没有余量”是两个问题**，后者才是工程上的门槛。

向上对接：参考从哪来。 轨迹规划器给出的是稀疏的点列，控制器需要每个 1 ms 时刻的 q_d、$\dot{q}_d$、$\ddot{q}_d$。三种常见做法，代价依次降低、风险依次升高：

- 规划器按 1 ms 输出，控制器直接用。最干净，但规划器的计算量要跟上，且两边时钟必须同步。

- 规划器按 4 ms 输出，控制器做三次样条或五次多项式插值。插值误差是 $O(T^4)$ 或 $O(T^6)$，4 ms 下远小于跟踪误差，可以接受。这是多数工业控制器的做法。
- 控制器对稀疏点做线性插值，或直接零阶保持。此时 $\ddot{q}_d$ 是脉冲列，前馈力矩会在每个规划周期跳一次。**全驱对参考的二阶导最敏感，这种跳变会被 $M(q)$ 原样放大成力矩尖峰**，表现为每 4 ms 一次的抖动。本章的 `ref_traj` 是解析式，不存在这个问题，但把它换成查表时要格外小心。

多率比的选取。 两条经验：电流环至少比控制环快 10 倍（相位损失才小于 5°），轨迹规划至少比控制环快 4 倍（即规划周期不超过控制周期的四分之一，否则控制器在"猜"参考）。中间这一档——控制器——的周期由 (81-26) 与计算量共同决定，而不是由传感器或执行器的时钟决定。

还有一处容易被忽略：**三个环的时钟不同步时，采样抖动会直接进入微分项。** 1 ms 周期上 10 μs 的抖动是 1%，折算到测速噪声上是同比例增加；而 (81-35) 的 σ_v 是按理想等间隔算的，实际会更大。这也是 §81.5.5 选择 1 ms 的另一个理由：周期越长，同样的绝对抖动占比越小。

81.5.8 真机上怎么求这个逆

本章的代码用 `np.linalg.inv`，一行就完了。嵌到控制器里时没有这个函数，得自己写，而写法会直接影响精度与耗时。

对二连杆，M 是 2×2 对称正定阵，逆有闭式：

$$M^{-1}=\frac{1}{\det M}\begin{bmatrix} M_{22} & -M_{12} \\ -M_{12} & M_{11}\end{bmatrix},\qquad \det M = M_{11}M_{22}-M_{12}^2>0 \tag{81-28}$$

用闭式而不是通用求逆，理由有三个。 其一，闭式只需要一次除法、四次乘法、两次减法，通用 LU 分解要做主元选择与循环，在 2×2 上反而更慢。其二，$\det M>0$ 是正定性的直接推论，可以做运行时的健全性检查——如果算出的 $\det$ 小于阈值，说明参数或状态异常，此时应该报警而不是继续算下去。其三，闭式没有中间数组，寄存器就够放，适合中断服务程序。

数值范围也值得算一遍。本章 $\det M$ 的最小值约 0.07，所以 $1/\det M$ 最大约 14；M_{11} 最大 1.27。用 Q16.16 定点表示（范围约 ± 32768、分辨率 1.5×10^{-5}），所有中间量都在范围内，不会溢出。**定点带来的舍入误差会被 κ 放大**：$\kappa=31$ 时，输入的 1.5×10^{-5} 相对误差可能变成 5×10^{-4}，仍远小于 §81.6.2 的 5% 门槛。所以定点是可行的。

但"可行"不等于"该用"。如果处理器带单精度浮点单元（Cortex-M7、Cortex-A 系列都带），**直接用 float32**：相对精度 10^{-7}，比定点省事，也不必为每个变量定标。只有在没有 FPU 的低端 MCU 上（M0、M3），定点才有明显优势，因为软件浮点的一次除法要几百个周期。

最后一条实践建议：**把"求逆"与"限幅"写在一起，不要让它们分开。** 如果先算出完整力矩再限幅，饱和时各关节的力矩比例会被破坏，等效于给系统加了一个方向相关的扰动；更好的做法是先按力矩上限缩放整个力矩向量（等比缩放），保持方向不变。等比缩放会降低总体力矩，但不引入方向误差，对全驱这种"方向敏感"的控制器更重要。

81.6 稳定性与性能

§81.2.4 的证明只处理了常系数情形，§81.2.3 又提醒过：真实系统的 A_i 随状态变。这一节补上这部分，并把"稳定"与"性能好"分成两件事来说。

81.6.1 时变情形：换一个 Lyapunov 函数

对机械臂，(81-11) 其实不是全部真相。精确闭环应写成

$$M(q)\ddot{e}+\big[C(q,\dot{q})+K_1\big]\dot{e}+K_0 e=0 \tag{81-29}$$

M 与 C 都随状态变（这里的 C 是科氏矩阵，$C\dot{e}$ 才是向量），所以这是一个线性时变系统。§81.2.4 的 $V=\tfrac12\|\dot{e}\|^2+\tfrac12\omega_n^2\|e\|^2$ 不再好使，因为 (81-29) 里 M 不是单位阵。换成机器人学里的能量型函数：

$$V = \tfrac{1}{2}\dot{e}^{\mathsf{T}} M(q)\dot{e} + \tfrac{1}{2} e^{\mathsf{T}} K_0 e \tag{81-30}$$

第一项是误差运动的“动能”（注意权重是真实的质量阵），第二项是“弹性能”。沿 (81-29) 求导：

$$\dot{V} = \dot{e}^{\mathsf{T}} M\ddot{e} + \tfrac{1}{2}\dot{e}^{\mathsf{T}}\dot{M}\dot{e} + e^{\mathsf{T}} K_0 \dot{e} \tag{81-31}$$

把 $M\ddot{e} = -C\dot{e} - K_1\dot{e} - K_0 e$ 代入，并用机械臂的一条结构性质——$\dot{M} - 2C$ 反对称，因而 $\dot{e}^{\mathsf{T}}\dot{M}\dot{e} = 2\dot{e}^{\mathsf{T}} C\dot{e}$——三项里的 C 与 K_0 全部抵消，只剩

$$\dot{V} = -\dot{e}^{\mathsf{T}} K_1 \dot{e} \le 0 \tag{81-32}$$

结论与常系数情形一模一样，但这次**没有冻结任何系数**。这条性质（$\dot{M} - 2C$ 反对称）不是数学技巧，它就是“科氏力不做功”的矩阵表达；§81.2.3 里说的“科氏项自己消失”，在这里以能量形式再出现一次。

由 (81-32) 加上 LaSalle，可以推出 $e \to 0$、$\dot{e} \to 0$，前提仍是模型精确。证明链条里唯一用到模型的地方是 M 与 C 同时出现在控制器与被控对象中；模型一错，$\dot{M} - 2C$ 的反对称性对“控制器以为的 M”和“对象真实的 M”就不再匹配，(81-32) 的等号变成不等式加余项。

81.6.2 模型误差下的一致最终有界

放开模型精确这一条，得到的是“误差不会无限增长，但收敛不到零”。把失配参数从 0 扫到 40%，本章的稳态误差是图 81-8 的实线：0.002°、0.668°、1.187°、1.908°、2.348°、2.611°（$\delta = 0, 5, 10, 20, 30, 40\%$）。同图还画了一条水平虚线，是 PD + 重力在 20% 失配下的 1.061°。

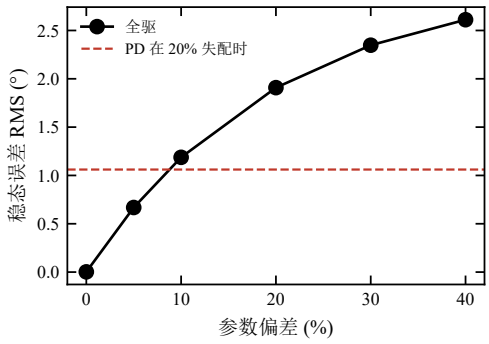


图 81-8　稳态误差随参数偏差的变化，虚线为 PD 在 20% 失配下的水平

两条线在 $\delta \approx 9\%$ 处相交：5% 时全驱 0.668° 已优于 PD，10% 时 1.187° 已经输掉。交点左侧全驱更好，右侧 PD 更好。这个交点比任何定性说法都有用：**它把“模型准不准”翻译成了一个可以查的百分比。** 如果你的辨识误差能压到 5% 以内，全驱有明确优势；只做到 10%，优势已经没了；到了 20%，就是表 81-5 那一行的三倍半差距。

曲线形状也值得注意：从 5% 到 40%，偏差翻了 8 倍，误差只涨了 3.9 倍，明显放缓。§81.2.5 已经解释过原因——偏差同时放大了扰动与闭环刚度，两者部分抵消。这也意味着**不能用“误差正比于参数误差”去做外推**，比如从 20% 的 1.908° 推出 40% 的 3.8°，实际只有 2.611°。工程上要这个数，只能实测或仿真。

81.6.3 三个指标要分开看

“性能”在本章拆成三项，它们的最优点不在一起。

稳态误差由模型精度与刚度决定，想小就提 ω_n 或提模型精度。**力矩峰值**随 ω_n 增长，$\omega_n = 28$ 时已达 19.04 N·m，距 25 N·m 的上限只剩 24% 余量。**饱和比例**是另一个独立指标：峰值没超限，不代表过程中没饱和，本章的算例在 $\omega_n = 15$ 时基本不饱和，但把带宽提到 28 后，饱和开始吞掉性能。

三项放在一起看，才能解释 §81.7 增益地图上那个“看起来很奇怪”的最优点：理想模型下误差最小的点是 $\omega_n = 28$，但加入量化后，误差最小的点挪到了别处。带宽不是越高越好，它是在跟另外两项做交换。

81.6.4 一条可以写进验收单的判据

把上面的分析收成一句：**在标称模型下验证 V 单调（或等价地验证 (81-32)），在失配模型下验证误差上界，在饱和与噪声下验证不出现极限环。** 三条都过，控制器才算可用。三条里第一条是数学，后两条只能仿真或实验。

这也回答了“全驱的稳定性怎么证”这个常被问的问题：证稳定性不难，难的是证完之后还要承认——**稳定只保证误差有界，不保证误差够小。** 后者由 §81.2.5 的界与图 81-8 的曲线共同决定。

81.6.5 把误差拆成四项

同一个 0.0024°，在不同工况下会被不同的项顶起来。把误差按来源拆开，可以避免“看到一个数就归因”的错误：

- **模型项**：$A_r^{-1}\Delta$，由 (81-15) 界定。$\delta = 20\%$ 时是 3.714°，是唯一能单独把误差推到度级的项。
- **初值项**：$e(0)e^{-\zeta\omega_n t}$。0.02 rad 的初值在 0.5 s 后剩 0.0014°，与 0.0024° 同量级，是本章标称工况下最大的贡献。
- **量化项**：$K_1\sigma_v$ 折算到误差，不加滤波时把 0.0024° 抬到 0.908°，加 5 ms 滤波后压到 0.033°。注意滤波后仍比 0.0024° 大一个量级，说明噪声项无法完全消掉，只能压小。
- **数值项**：RK4 的截断误差与 $O(T_s^2)$ 的参考离散误差，在本章参数下小于 0.001°，可以忽略。

四项的性质完全不同。模型项是“偏置”，不随时间衰减；初值项按指数衰减；量化项是“抖动”，均值接近零但方差不为零；数值项随 T_s 缩小。**只有模型项会随工况漂移，其余三项都是设计时可以算清的。** 这就是为什么 §81.6.2 的失配曲线是本章唯一“不确定”的性能曲线，其余都能从参数直接推出来。

反过来，这个分解也解释了为什么“提高带宽”不是万能药：它压初值项（衰减更快）、不压模型项（偏置与刚度同比例）、放大量化项（K_1 变大）。三项一个降、一个不动、一个升，总误差的最小值必然出现在某个中间带宽——这正是图 81-5 那条“先降后升”的曲线的来源。

81.6.6 界为什么总是偏保守，以及怎么用它

(81-15) 那样的界在推导时做了三次“往坏里取”，每取一次都要付一笔保守的代价。把这三笔账摊开，才知道什么时候该信界、什么时候该信仿真。

第一笔：范数取了最坏构型。 $\|A_r^{-1}\|$ 是工作域上的上确界。本章 κ 的最大值 31.36 出现在手臂伸直的构型，那里最小奇异值是 0.047 kg·m²、最大奇异值 1.48 kg·m²，于是 $\|M^{-1}\|$ 的上界是 $1/0.047 = 21$。但最小奇异值的中位数是 0.151，典型构型下 $\|M^{-1}\|$ 只有 6.6。**用一个只在伸直构型出现的 21 去乘所有时刻的扰动，界自然大三倍。**

第二笔：三角不等式把三项误差相加。 Δ 里含 M、C、G 三块的参数误差，取范数时按最坏相位叠加；实际上这三项在大部分时刻并不同相，代数和小于绝对值和。这一笔通常又给出两倍左右的保守。

第三笔：界假设误差是常值。 推导里令 $\dot{e} = \ddot{e} = 0$ 解出 e，但本章的误差是随参考振荡的，其中高于闭环带宽的分量会被 K_1 压掉。**界算的是“最坏频率上的稳态”，实测的 RMS 是对全频段积分**，两者本来就不该相等。

三笔合起来，界比实测大几倍到十几倍是正常的，不是推导错了。这一点有个直接证据：§81.6.2 的扫描里，参数偏差从 5% 涨到 40%（8 倍），误差只从 0.668° 涨到 2.611°（3.9 倍）。**线性界的斜率是按 8 倍画的，实测只有 3.9 倍**——偏差越大，界越保守，因为“偏差同时放大扰动与刚度”这个抵消效应在线性化时被丢掉了。

那么界还有什么用？用处不在“预测数字”，在“做准入判断”，而且分两步用：

1. **先用界算最坏值。** 取 κ 的上界、δ 的上界、$\|A_r^{-1}\|$ 的上界，算出一个数。如果这个数已经小于指标，直接通过，不必再仿真——**界通过是充分的。**
2. **界不通过时，才做仿真或实测。** 此时得到的是真实值，可能比界小一个量级，也可能因为未建模动态而比界还大。前者说明可以放宽设计，后者说明模型里缺东西。

第 2 步里“实测比界还大”是一种很有价值的失败信号：界的推导只假设了参数偏差，如果实测超出，那一定有参数偏差之外的东西——摩擦、间隙、柔性、电流环延迟。**界的作用因此不只是给数字，它还是一个“模型完备性”的检查器：超界即意味着有未建模项。**

81.6.7 一个置信度旋钮：在全驱与 PD 之间连续移动

工程上常常遇到一种中间状态：模型大致可信，但不敢完全信。此时“要么全驱、要么 PD”的二选一太粗暴，可以引入一个 $0 \le \lambda \le 1$ 的混合系数：

$$\tau = \lambda \tau_{\text{FAS}} + (1-\lambda)\tau_{\text{PD}} \tag{81-33}$$

其中 τ_{PD} 是 PD 加完整重力补偿。展开后，反馈部分的等效增益矩阵是

$$K_{\text{eff}} = \lambda M(q) K_0 + (1-\lambda) K_0, \qquad D_{\text{eff}} = \lambda M(q) K_1 + (1-\lambda) K_1 \tag{81-34}$$

这个式子的意义很清楚：**$\lambda = 1$ 时刚度完全随构型缩放（全驱）；$\lambda = 0$ 时刚度是常数（PD）；中间值是两者的加权。** 于是“置信度”这个模糊的词被翻译成了一个可以连续调的参数。

三处用法值得说明。

第一，调试期从 0 往 1 推。 新机器上先取 $\lambda = 0$，确认基本伺服正常；再取 0.5，看精度是否明显改善；最后推到 1。每一步都能回退，比“直接上全驱”稳。§81.11.6 的“影子模式”其实就是 $\lambda = 0$ 但记录 $\lambda = 1$ 的输出。

第二，按模型置信度在线调度。 如果系统能估计 δ（比如通过残差观测），可以令 λ 随 δ 下降：δ 小于 5% 时取 1，接近 9% 时降到 0.5，超过 15% 时降到 0.2。**这不是“鲁棒控制”，只是一个经验性的软切换**，但它避免了硬切换带来的力矩跳变。

第三，失灵时的降级路径。 传感器故障、模型参数越界、通信超时，都可以触发 λ 在几百毫秒内平滑降到 0，而不是直接断电。**平滑降级比急停更适合有负载的场合**，因为急停会让重力突然失去支撑。

要提醒的是，λ 不是“万能保险”。它改变的是刚度阵，不改变“没有积分”这个事实，所以常值扰动下的静差依旧存在（§81.4.7）；它也不改变饱和，λ 小只是把力矩需求变小，如果参考本身要求大力矩，饱和照样发生。**λ 能救的是“模型不够准”，救不了“执行器不够大”。**

81.7 参数意义与作用

全驱控制器里能调的东西很少：K_0、K_1，加上它们背后的 ω_n、ζ。其余的量——κ、T_s、$\tau_{\max}$——是系统属性或外部约束，不归设计者调。这一节逐个说清每个量在做什么、单位是什么、往哪边调会有什么后果。

81.7.1 K_0：把位置误差换成力矩的汇率

K_0 的单位是 N·m/rad，物理意义是**等效刚度**：每 1 rad 的位置误差，换来多少 N·m 的恢复力矩。本章 $K_0 = \omega_n^2 = 225$ N·m/rad，意思是 0.01 rad（0.57°）的偏差会产生 2.25 N·m 的力矩。

这个数看起来很大，但正好和重力项比一比。本章重力项的最大值约 14.7 N·m（两杆都水平时关节 1 的负担），也就是说，**0.065 rad 的位置误差就能产生与重力同量级的力矩**。这解释了两者的差别：PD 的标量 K_0 必须同时满足“最坏构型下不饱和、最好构型下不振荡”，只能在两端之间取折中；全驱的 K_0 乘在质量阵之后，刚度自动随构型缩放，所以可以直接按 ω_n^2 取，不必为构型留余量。

81.7.2 K_1：把速度误差换成力矩，也把噪声换进来

K_1 的单位是 N·m·s/rad，物理意义是**等效阻尼**。本章 $K_1 = 2\zeta\omega_n = 27$ N·m·s/rad，0.1 rad/s 的速度误差换 2.7 N·m 的阻尼力矩。

K_1 是本章所有麻烦的来源。它乘在 $\dot{e}$ 上，而 $\dot{e}$ 通常是差分算出来的，带着量化噪声。本章编码器量化步长 6.3×10^{-4} rad、周期 1 ms，差分得到的测速噪声标准差是

$$\sigma_v = \frac{\sqrt{2}\,q}{\sqrt{12}\,T_s} = 0.257 \ \text{rad/s} \tag{81-35}$$

乘上 K_1 就是 6.9 N·m 的力矩抖动——**与重力项同量级，比跟踪误差换来的力矩大一个数量级**。图 81-7 是这条链路的三条曲线：差分直接用的纹波、加 5 ms 一阶滤波后的纹波、以及 $K_1\sigma_v$ 的理论折算。

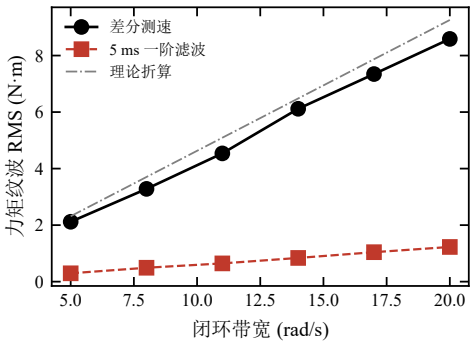


图 81-7　力矩纹波随带宽的增长：差分测速、5 ms 滤波与理论折算

ω_n 从 5 涨到 20，三条线分别从 2.118 涨到 8.589、从 0.295 涨到 1.227、从 2.315 涨到 9.259（单位 N·m）。三点观察：

- 差分曲线与理论折算几乎重合，说明纹波的主因就是 $K_1\sigma_v$，没有别的隐藏通道。
- 滤波把纹波压到约 1/7（8.589 对 1.227），代价是 5 ms 的相位滞后——对 $\omega_n = 15$ 是 0.075 rad，与 §81.5.4 算的一致。
- 三条线都随 ω_n 线性增长，因为 K_1 线性依赖 ω_n。**这是“提高带宽换精度”这条路上最硬的一堵墙。**

81.7.3 ω_n 与 ζ：不是两个独立旋钮

ω_n 决定“多快”，ζ 决定“多稳”，但它们在控制器里只以 $K_0 = \omega_n^2$、$K_1 = 2\zeta\omega_n$ 的形式出现。所以“调 ω_n”实际上是“同时调两个增益”，且一个按平方、一个按一次方。

这个耦合有个直接后果：想把误差压小一半，ω_n 要乘 $\sqrt{2}$，K_1 也跟着乘 $\sqrt{2}$，噪声纹波同比上升。**精度与噪声的交换比是固定的，调参调不出便宜。** 想跳出这个交换，只能改结构——加滤波（换相位）、加观测器（换模型）、或提高编码器分辨率（换成本）。

ζ 的作用相对温和：它只改 K_1，不改 K_0。ζ 从 0.9 提到 1.4，K_1 涨 56%，超调消失但响应变钝，纹波同比涨 56%。本章取 0.9 是为了留一点超调换响应速度。

81.7.4 κ：不归你调，但决定你调得动多少

κ 是系统属性，设计者改不了（除非改机构）。它的作用是把同一组 K 翻译成“每个构型下的实际闭环带宽”：实际带宽约 $\omega_n/\sqrt{\kappa_{\text{局部}}}$ 到 $\omega_n\sqrt{\kappa_{\text{局部}}}$ 之间散布。

本章 κ 从 1.09 到 31.36，散布因子 $\sqrt{31.36} = 5.60$。这意味着**一组固定增益在最好构型上等效 $\omega_n = 15$、在最坏构型上等效 $\omega_n = 2.7$，或反过来**。全驱的 K 是“乘在 M^{-1} 之后”的，所以它把这个散布消掉了——代价是必须知道当前构型。

81.7.5 外部参数：$\tau_{\max}$ 与 T_s

这两个不进控制器，但决定了控制器的可用范围。$\tau_{\max} = 25$ N·m 把带宽卡在 61.9 rad/s（§81.5），$\omega_n = 28$ 时峰值力矩 19.04 N·m 已用掉 76%。$T_s = 1$ ms 把带宽卡在 1111 rad/s，远不是瓶颈。

两者的关系可以概括成一句话：**力矩上限决定“能多快”，采样周期决定“最快能多快”。** 前者是预算问题，后者是物理问题。设计时先看预算，再看物理；预算不够时提带宽是徒劳的，因为饱和会把多出来的带宽全部吃掉，还倒赔稳定性。

81.7.6 四个无量纲数

前面反复出现的四个门槛，其实都是无量纲的。把它们列出来，能看清哪些结论可以跨机器迁移、哪些不能。

第一个，带宽比 $\omega_n/\omega_{\text{ref}}$。 它决定“参考变化一个周期，闭环能跟几成”。本章 ω_{ref} 约 1 rad/s，$\omega_n = 15$，比值 15，所以跟踪误差里几乎没有“相位滞后”成分。

第二个，力矩裕度 $\tau_{\max}/\tau_{\text{ref}}$。 τ_{ref} 是参考轨迹本身的惯性力矩，量级是 $J\ddot{q}_d$。本章 J 的奇异值到 1.48 kg·m²、$\ddot{q}_d$ 到 0.7 rad/s²，τ_{ref} 约 1.0 N·m，25 N·m 是它的 24 倍。但这份预算不是全部可用的：重力已经占掉 14.7 N·m，真正留给“提带宽”的只剩 10 N·m 量级，仍是 τ_{ref} 的 10 倍上下。**裕度大，说明饱和在本章不是主要矛盾**——这也是表 81-6 里 25 N·m 那一列为什么平。

第三个，采样比 $\omega_n T_s$。 本章 0.015，上限 1.1，余量 73 倍。

第四个，条件数 κ。 本章 31.36，中位 5.83。

这四个数一摆，本章结论的适用范围就清楚了。反过来，任何一台新机器只要算出这四个数，就能大致判断它会落在本章的哪一段。把“误差—力矩—带宽”三者的关系写成无量纲形式，得到一个恒等式（注意它是恒等式，不是界）：

$$\frac{e}{q_{\text{ref}}} \equiv \frac{\tau_{\text{fb}}}{\tau_{\text{ref}}} \left(\frac{\omega_{\text{ref}}}{\omega_n} \right)^2 \tag{81-36}$$

式中的 τ_{fb} 是“除重力与前馈之外、真正用来压误差的那份力矩”。它之所以有用，是因为它逼你回答一个问题：**要减小误差，你打算加 τ_{fb}，还是加 ω_n？** 前者的代价是发热与饱和，后者的代价是噪声与采样。两条路都走不通时，才轮到改机构或改执行器。

三个推论都很好用。

推论一：带宽翻倍，误差除以四。 这是二阶系统的基本事实，但要注意它只对“由力矩换来的那部分误差”成立；由模型误差决定的那部分不遵守这条（§81.6.5 已说明：偏置与刚度同比例变，商不变）。

推论二：尺寸放大，带宽必然下降。 几何相似地把机器人放大一倍，质量涨 8 倍、惯量涨 32 倍、可用力矩只涨 8 倍（力矩与体积同阶），于是 $\tau_{\max}/J$ 降到四分之一，可实现的 ω_n 降一半。**这是大型机器人的物理宿命，不是控制算法能绕过的。** 六轴工业臂的带宽通常只有小型臂的三分之一到二分之一，原因在这里，不在“算法没选对”。

推论三：换负载改的是第二个数。 末端加 5 kg，J 上升、$\tau_{\max}$ 不变，力矩裕度下降、κ 上升，两个数同时往坏的方向走。**换负载必须重算增益，不能沿用空载参数**——§81.8.6 展开这一条。

(81-36) 还给出一条否定性结论：**如果 $\tau_{\max}/\tau_{\text{ref}}$ 已经小于 3，那么无论怎么调 ω_n，误差都压不下去。**因为一提带宽就饱和，饱和之后前馈失效，(81-36) 的前提（控制器真的把力矩送出去了）不再成立。这种场合该换的不是参数，是执行器。

81.8 调参方法与经验

81.8.1 需要决定的只有三件事

全驱控制器的调参被压缩成三个决定：**带宽取多大、阻尼取多大、测速怎么滤波。** 三个之外没有第四个旋钮。这不是简化说法，而是 (81-10) 的结构决定的：给定了 ω_n、ζ，两个增益矩阵就定了；给定 T_s，离散化方式就定了。

难点在于三个决定互相牵制：带宽大→误差小但纹波大、力矩峰值高；阻尼大→纹波小但响应钝；滤波强→纹波小但相位滞后。任何一组选择都是三点之间的妥协。增益地图就是把这种妥协画出来的工具。

81.8.2 增益地图怎么读

图 81-5 的横轴是 ζ，纵轴是 ω_n，填色是跟踪误差的 RMS，白线是 20 N·m 的力矩等值线（距 25 N·m 上限留 20% 余量），黑点是量化条件下误差最小的网格点。

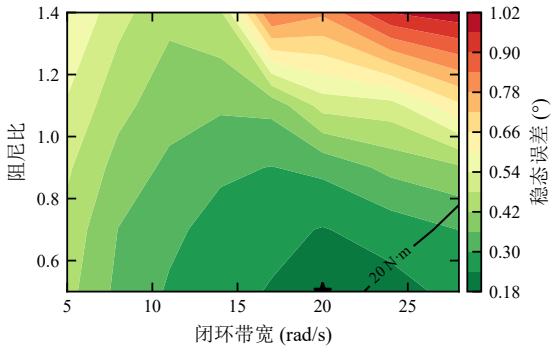


图 81-5　(ω_n, ζ) 增益地图：填色为量化后的误差，白线为力矩等值线

先看 $\zeta = 0.9$ 那一列，数据在表 81-7。理想测量下误差随 ω_n 单调下降，从 0.458° 降到 0.192°，看起来“带宽越大越好”。但同一列加上量化、不做滤波，误差先降后升：0.489°、0.397°、0.347°、0.310°、0.298°、0.315°、0.378°、0.410°，最小值落在 17–20 之间。

表 81-7　$\zeta = 0.9$ 时逐带宽的误差与力矩峰值

ω_n	理想 RMS	力矩峰值	量化 RMS
5	0.458°	14.76 N·m	0.489°
8	0.360°	14.76 N·m	0.397°
11	0.307°	14.84 N·m	0.347°
14	0.272°	15.39 N·m	0.310°
17	0.247°	16.01 N·m	0.298°
20	0.227°	16.71 N·m	0.315°
24	0.208°	17.79 N·m	0.378°
28	0.192°	19.04 N·m	0.410°

“先降后升”的拐点就是本章最重要的一张工程图。它说明**带宽的收益在某个点之后被噪声抵消**，而这个点与编码器分辨率直接相关：分辨率提高十倍，拐点右移；分辨率降十倍，拐点左移。任何“我的系统该取多大带宽”的问题，答案都不在控制器里，在传感器规格书里。

白线给出另一个约束。$\omega_n = 28$ 时力矩峰值 19.04 N·m，已经在 20 N·m 等值线之外，距 25 N·m 上限只剩 24% 余量；一旦轨迹加速度再大一点、负载再重一点，饱和就会介入。所以即便忽略噪声，28 也不是一个稳妥的选择。

81.8.3 一次完整的调参路径

本章的 $\omega_n = 15$、$\zeta = 0.9$ 是按下面四步定的，可以照着走：

1. **定上限。** 按 §81.3.4，先算饱和上界 61.9 rad/s 与采样上界 1111 rad/s，取小者，得 61.9。
2. **留余量。** 按上限的 80%（20 N·m）卡峰值力矩。表 81-7 里连 $\omega_n = 28$ 的 19.04 N·m 都还在线内，所以这一步在本算例没有真正生效，只排除了 28 以上的带宽。
3. **看噪声。** 在 20 以内找量化误差最小的点，落在 17–20。取 15，是为了给轨迹变化与负载波动留空间。
4. **定阻尼与滤波。** $\zeta = 0.9$ 在超调与纹波之间取平衡；滤波取 5 ms，把纹波压到 1/7，代价 0.075 rad 相位。

四步里只有第三步需要仿真，其余三步都是查数。这就是 §81.1.4 说的“退化”：调参从“试”变成“算”。

81.8.4 三条经验

第一条：先看传感器，再看执行器，最后才看控制器。 编码器分辨率决定带宽上限（拐点），力矩上限决定带宽下限（能不能跟得上）。两者之间才是控制器可以活动的区间。很多人反过来，先调增益再发现传感器不够，白折腾。

第二条：阻尼比不要低于 0.7。 本章量化最优点落在 $\zeta = 0.5$，但那是"只看误差"的结果。$\zeta = 0.5$ 意味着超调约 16%，在点到点定位任务里不可接受。工程上 ζ 取 0.7 到 1.0 之间，把超调压在 5% 以内。

第三条：滤波时间常数不要超过 $1/(5\omega_n)$。 $\omega_n = 15$ 对应 13 ms，本章取 5 ms 是保守的。超过这个比例，滤波引入的相位滞后会开始吃掉阻尼，表现为"越滤波越振"。

81.8.5 一个反例

如果只看理想误差，会得到"ω_n 越大越好、ζ 越小越好"的结论，因为两者都让误差单调下降。这个结论在真实系统上要打折扣：$\omega_n = 28$、$\zeta = 0.5$ 在理想模型下误差 0.178°（全图最小），加上量化后升到 0.258°，最优点随即挪到 $\omega_n = 20$、$\zeta = 0.5$ 的 0.217°。挪动的幅度不大，方向却很清楚：**噪声把最优带宽从 28 拉回到 20，阻尼那一维没有动。** 如果阻尼不变只提带宽，代价更明显——同样是 $\omega_n = 28$，ζ 取 1.4 时量化误差是 0.998°，是全图最差的点之一。

理想模型的增益地图只告诉你"能做到多好"，不告诉你"该取多少"。 后者必须把传感器噪声、力矩上限、滤波滞后一起放进仿真，这也是 §81.12 的代码里 `quant`、`vfilt`、`tau_max` 三个参数存在的原因——它们不是可有可无的装饰，而是把地图从"数学最优"拉回"工程可用"的唯一手段。

81.8.6 换机器、换负载：参数怎么迁移

本章的 $\omega_n = 15$、$\zeta = 0.9$ 是为一台 1 kg 连杆的二连杆定的。搬到别的机器上，能搬什么、不能搬什么，值得单独说清，因为这是最容易出错的一步。

能搬的是无量纲数，不是增益。 §81.7.6 的四个数——$\omega_n/\omega_{\text{ref}}$、$\tau_{\max}/\tau_{\text{ref}}$、$\omega_n T_s$、$\kappa$——是真正决定性能的东西。新机器上先算这四个数：如果 $\omega_n T_s$ 从 0.015 涨到 0.5，说明采样已经不宽裕，得先缩周期或降带宽；如果 κ 从 31 涨到 300，说明接近奇异，先改工作域；如果 $\tau_{\max}/\tau_{\text{ref}}$ 从 140 掉到 4，说明执行器是瓶颈，调参无用。

不能搬的是 K_0、K_1 的绝对值。 它们带量纲，随惯量走。用本章的 225 与 27 去控制一台惯量大十倍的机器，等于把带宽降到 $1/\sqrt{10}$，表现为"反应迟钝但不振"；反过来，惯量小十倍的机器会振。**正确做法是只搬 ω_n、ζ，让程序按新模型重算 K。** 本章代码里 `gains(wn, zeta)` 是唯一的入口，就是为了让这条规矩变成结构上的强制。

换负载是一个特例，而且是最常见的特例。 末端加 20% 负载、模型不改，效果与 §81.6.2 的 $\delta = 20\%$ 完全一样：误差从 0.0024° 跳到 3.714°，一下子输给 PD。这不是控制器"变差"了，而是它乘的 $M(q)$ 与实际不符，前馈送错了量。把新负载重新辨识、填回模型，误差立刻回到 0.0024° 量级。**所以"换负载要重算"这句话的真实含义是"换负载要重新辨识模型"，而不只是改两个增益。**

迁移的具体顺序建议如下：

1. 用 CAD 与新负载算四个无量纲数，判断哪条约束会先活跃。
2. 按 §81.3.4 的饱和约束定新带宽，按 (81-26) 校核采样。
3. 在新带宽下重算 K_0、K_1，不沿用旧值。
4. 用新模型跑一次失配扫描（改 `CFG` 而保持 `p_model` 不变），看 $\delta = 10\%$ 时误差是否还在可接受范围。
5. 上机后先做 §81.10.6 验收单的前三项，再放节拍。

这五步里第 4 步最容易被跳过，而它恰恰是唯一能提前暴露"模型没跟上负载"的一步。

81.9 优化与改进

前八节描述的是"标准全驱"：一个模型、一组增益、一次求逆。它在理想条件下近乎完美，在三个地方会掉链子：算不动（大规模系统）、测不准（微分噪声）、扛不住（饱和与失配）。这一节按这三个方向讲改进，每条都给出代价。

81.9.1 改进的三种方向

结构方向：改控制器本身的形式——加观测器、加自适应、加前馈整形。**参数方向**：不改结构，只让参数随工况变——增益调度、自适应律。**实现方向**：不改算法，只改算法落地的方式——定点化、查表、并行。

三个方向的性价比差别很大。实现方向最便宜（不改数学），参数方向次之（要在线估），结构方向最贵（要重新证明稳定性）。工程上的顺序应该反过来：先问结构够不够，再问参数要不要动，最后才优化实现。

81.9.2 计算量：全驱真正的优势

图 81-9 是单步耗时随自由度 n 的变化，对数坐标，右侧画了第 80 章 MPC 的 31.23 ms 作为参照。

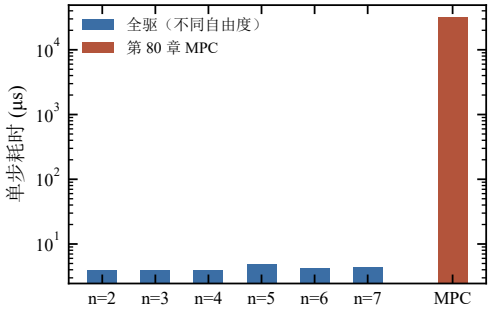


图 81-9　单步计算耗时随自由度的变化，与 MPC 的对比

$n = 2$ 到 7 的实测值在 3.7 到 6.0 μs 之间（多次运行有浮动，本章按 4.8 μs 计），**几乎不随 n 增长**。这个反直觉的结果来自实现环境：Python 的函数调用与属性查找开销盖住了 $O(n^3)$ 的求逆，$n \le 7$ 时矩阵太小，求逆根本不是瓶颈。换成 C 或定点 DSP，这条曲线会显出斜率；但在本章的规模上，它说明一件事——**全驱的计算量在这个量级上可以当成零。**

31.23 ms 对 4.8 μs 是约 6500 倍。这个比值不该被理解成“全驱比 MPC 快 6500 倍所以更好”，而应该理解成“两者的成本结构完全不同”：MPC 的成本随预测时域、约束数量、迭代次数增长，全驱的成本只随自由度增长，且常数极小。当控制周期被压到 100 μs 以下、或者要在几十个关节上同时跑时，这个差别会从“性能差异”变成“能不能做”。

代价也要说清：全驱省下的计算，是用“放弃约束处理”换来的。MPC 花 31 ms 换来的是“每一步都知道未来 15 步会不会撞墙”，全驱花 5 μs 换来的是“这一步的力矩算得快”。**省下来的时间没有变成能力，只是变成了更短的周期。**

81.9.3 微分从哪来：三种替代

K_1 乘在 $\dot{e}$ 上，所以测速质量直接决定噪声水平。三条路，代价各不相同。

差分加滤波是本章的做法。编码器量化 6.3×10^{-4} rad、周期 1 ms，差分噪声 0.257 rad/s；用它直接闭环，量化带来的稳态误差是 0.908°，加 5 ms 一阶滤波后降到 0.033°。代价是 5 ms 相位滞后，折合 0.075 rad。

观测器（比如扩张状态观测器或卡尔曼滤波）用模型与测量一起估计速度，噪声抑制比一阶滤波好，且相位滞后可控。代价是要信模型——速度估计的精度取决于 M、C 的精度，而这两者恰恰是 §81.6 里最不可靠的部分。**用观测器换噪声，等于把问题从传感器挪回模型。**

直接测量（测速发电机、增量编码器的硬件细分）最干净，但增加成本与布线。工业伺服里常见的做法是“编码器 + 驱动器内部观测器”，本质上是前两条的混合。

三条路的取舍可以用一句话概括：**滤波用相位换噪声，观测器用模型换噪声，硬件用钱换噪声。** 本章选第一条，因为它在三者中最不依赖模型，而模型的可靠性正是本章反复强调的短板。

81.9.4 抗饱和：最容易被忽略的一处

(81-10) 的推导假设力矩不受限。一旦某个关节触到 $\tau_{\max}$，闭环等式就不再成立：前馈项与反馈项被同比例削掉，误差方程右端凭空多出一块。更麻烦的是，这块扰动与状态相关，可能形成自激。

本章的代码用最朴素的逐关节限幅，能跑通是因为算例的力矩余量足够（$\omega_n = 15$ 时峰值远低于 25 N·m）。工程上要做三件事：

其一，限幅要“按方向”而不是“按分量”。 逐关节削峰会改变力矩向量的方向，等于在关节空间里引入一个未建模的力矩扰动；改成“整体缩放”（把 τ 乘上 $\tau_{\max}/\|\tau\|_\infty$）能保持方向不变，代价是在某些方向上过度保守。

其二，饱和时冻结前馈。 前馈项 $\ddot{q}_d - K_1\dot{e} - K_0 e$ 在饱和期间算出来的值已经不可实现，继续累加没有意义；冻结它可以让系统在退出饱和时平滑衔接，而不是突然“补上”之前欠的力矩。

其三，把饱和当成模式切换来验证。 饱和前后是两个不同的闭环，稳定性要分别证、交界处要仿。§81.10 的执行器失效实验本质上就是“极端饱和”（一个关节的力矩归零），它的结果是 3.08 s 内失控，说明这类切换不能靠“应该没事”来处理。

其四，给力矩加变化率限制。 前三条只管“力矩不超过多少”，不管“力矩变化多快”。而 §81.7.2 的纹波、§81.10.3 的失效冲击，本质上都是力矩的突变。给 τ 加一个每周期最大增量（比如额定值的 5%，即 1.25 N·m/ms），等效于给执行器加了一级低通，能压掉一部分高频纹波，也能让失效瞬间的冲击变缓。代价是相位：一阶限制在频率 ω 处的滞后约 ωT_r，T_r 取 4 ms 时对 $\omega_n = 15$ 只有 0.06 rad，可以忽略；对 $\omega_n = 100$ 就是 0.4 rad，足以吃掉大半相位裕度。**所以它和测速滤波一样，是拿相位换平滑，只在高带宽时才需要权衡。**

81.9.5 自适应与学习：把模型误差在线补掉

既然全驱的软肋是模型精度（§81.6 的交点在 9%），最直接的改进就是让模型在线变准。两条路。

参数自适应：把 M、C、G 写成对参数线性的形式 $Y(q,\dot{q},\ddot{q})\theta$，用跟踪误差驱动 $\dot{\hat{\theta}} = -\Gamma Y^{\mathsf{T}}\dot{e}$ 更新。这类方法（Slotine 型）有严格的 Lyapunov 证明，能保证 $e \to 0$、$\hat{\theta}$ 有界。代价是需要 $\ddot{q}$ 或对 Y 做滤波，且参数收敛依赖“持续激励”——轨迹不够丰富时参数不收敛，只能保证误差有界。

残差学习：保留名义模型，用一个学习器（RBF、GP、小网络）去拟合 $\Delta = A_r^{-1}(\cdot)$ 那一项，接第 80 章的残差学习框架。好处是不用改控制器结构，学习器只贡献一个前馈项；坏处是学习器的外推没有保证，工作域外的行为不可预测。

两条路的共同点：**它们补的是“参数”，补不了“结构”。** 如果误差来自柔性、回差、摩擦滞环，参数自适应只会把参数拖到一个“平均意义上凑合”的值，残差学习则会把滞环当成噪声平均掉。这类问题的正解是回到 §81.3.6 的第一条路——把未建模动态显式建进模型，而不是让控制器去猜。

两条路都还有一个时间尺度上的麻烦，值得单独说。自适应律的收敛快慢由增益 Γ 决定：Γ 大，参数追得快，但会把测速噪声与未建模高频一起卷进来，表现为参数在真值附近持续抖动；Γ 小，参数平稳，但追不上工况漂移。本章的 δ 若来自温度，时间常数是分钟级，按说 Γ 小一点就够；可“持续激励”这个前提是按秒级的轨迹算的——参考只有两个频率，回归矩阵几秒就重复一遍，参数只在被激发的那两三个组合上收敛，其余方向全靠初值撑着。

于是出现一个两难：**要稳住噪声就得放慢，要满足激励就得快，而快慢两端在本算例里都站不住。** 这也解释了为什么工业界更常见的是“离线辨识 + 慢速增益调度”，而不是在线自适应：前者把激励问题在实验台上一次性解决，后者把激励问题留给了每一条工作轨迹。第 80 章的残差学习能绕开一部分，是因为它不要求参数有物理意义，只要求预测准；代价是外推没有保证。

还有一个量级判断可以用：如果 δ 的变化周期（温度、磨损）比一条工作轨迹长十倍以上，那就不是“在线估计”的问题，而是“分档调度”的问题。分两档（空载/满载）或三档（冷/温/热），每档一套参数，切换时做平滑过渡，工程上比连续自适应可靠得多。

81.9.6 部分全驱与更高阶

本章假设 $m = n$ 且 A_r 可逆。放松这个假设有两条路。

部分全驱：$m < n$ 时 A_r 是长方阵，只能对 m 个方向全驱，剩下 $n - m$ 个方向靠系统自身的稳定性（或另设控制器）。做法是用伪逆 A_r^{+} 代替 A_r^{-1}，v 中落在零空间的分量不再可控。工程上这对应“欠驱动 + 被动关节”，能不能用取决于零空间动态是否稳定——这就是欠驱动系统的经典难点，全驱框架并不能绕开它。

伪逆这一步有个数值细节必须交代。当 A_r 接近奇异时 $\sigma_{\min}$ 很小，伪逆在该方向上的增益是 $1/\sigma_{\min}$，等于把一个巨大的增益压在一个“几乎推不动”的方向上，控制器会为了消除一点点误差而输出极端力矩。本章 κ 最大到 31.36、$\sigma_{\min}$ 最小 0.047，此时增益是 21；若某个构型让 $\sigma_{\min}$ 掉到 0.005，增益就是 200，力矩立刻饱和。

标准对策是阻尼伪逆 $A_r^{\mathsf{T}}(A_r A_r^{\mathsf{T}} + \lambda^2 I)^{-1}$。沿某个奇异方向 σ，它的等效增益是 $\sigma/(\sigma^2+\lambda^2)$，在 $\sigma=\lambda$ 处取最大值 $1/(2\lambda)$，$\sigma \ll \lambda$ 时按 σ/λ^2 线性趋零。取 $\lambda = 0.047$，最弱方向的增益从 21 降到 10.6，代价是该方向的跟踪被“软化”——它不再追求精确，只求不发散。这个取舍与 §81.6 的失配界是同一件事的两种说法：**在最弱的方向上，宁可认输也不要硬顶。**

更高阶：$r > 2$ 时 (81-6) 的系数阵更多，增益矩阵也更多（K_0 到 K_{r-1}），极点配置的自由度更大。代价是需要的导数阶数更高，而高阶导数在噪声下几乎不可测。三阶以上的全驱系统在文献里有完整理论，工程上很少见，原因就在这里。

81.9.7 参考整形：不改控制器，改输入

最后一条改进最便宜，也最常被忽略：**如果问题是饱和，那就让参考轨迹本身不要求那么大的力矩。**

具体做法是把参考 q_d 先过一个二阶低通（带宽取 $\omega_n/3$ 左右），或者用时间缩放把轨迹拉慢。这样做的效果是降低 $\ddot{q}_d$ 的峰值，从而降低前馈力矩，饱和自然消失。代价是跟踪变慢，误差在过渡段变大。

但这条路的适用面比看上去窄。二阶低通对频率 ω 的分量，衰减约为 $1/\sqrt{(1-\omega^2/\omega_f^2)^2 + (2\zeta_f\omega/\omega_f)^2}$；本章的参考是 1.0 与 1.3 rad/s，取 $\omega_f = \omega_n/3 = 5$ rad/s，代入得衰减不到 5%。也就是说，**在本算例里参考整形几乎不起作用**——它的收益要等参考里出现 5 rad/s 以上的分量（快速往复、阶跃、电子凸轮的高次谐波）才显现。判断该不该加，只需看参考频谱在高频段还剩多少能量；如果 $\ddot{q}_d$ 的峰值本来就来自低频大幅值，整形只会让它变慢，不会减小力矩。

这条路的哲学与 MPC 相反：MPC 在控制器内部处理约束，参考整形在控制器外部避免约束。前者精确但贵，后者粗糙但便宜。当约束只是“偶尔触碰”，参考整形往往比在线优化更划算；当约束是“必须严格满足”，就只能回到 MPC。第 58 章讨论过这个分工，本章的立场是：**全驱 + 参考整形，能覆盖大部分“约束不硬”的场合。**

81.9.8 四种方案怎么分工

走到这里，读者手里其实有四个候选：PD 加前馈、全驱（计算力矩）、MPC、以及“全驱做内环 + 别的方法做外环”。它们的差别不在“谁更先进”，而在各自吃掉哪一类困难。

PD 加前馈。 吃的是“模型粗糙、任务简单”这一类。它不需要求逆，模型只影响前馈那一项；代价是误差地板随参考加速度线性上涨（§81.4.6）。如果任务是低速点到点、节拍宽松，PD 足够，上全驱是过度设计。

全驱。 吃的是“模型够准、参考变化快、要求高精度跟踪”这一类。它的门槛写在 §81.6.2：失配 9% 以内有优势。它的另一个隐性要求是“力矩有余量”——§81.7.6 的第二个数小于 3 时，全驱的优势会被饱和吃掉。

MPC。 吃的是“约束硬、要显式处理状态与输入边界”这一类。它的代价是计算：本章引用的 31.23 ms 单步，相对全驱的 4.8 μs 是 6472 倍。这个倍数决定了它只能跑在 20–50 Hz 的层面上，不可能直接驱动 1 kHz 的关节环。

组合。 正因为上面那个倍数，工程上最常见的不是“选一个”，而是分层：MPC 在 20 Hz 上算出满足约束的参考轨迹，全驱在 1 kHz 上跟踪这条轨迹。前者管“往哪走、不越界”，后者管“跟得准、扛得住扰动”。两层的接口只有 q_d、$\dot{q}_d$、$\ddot{q}_d$ 三个量，异常时上层可以降级为“保持当前点”，下层可以降级为 PD。

选择时按下面的顺序问四句，基本能定下来：

1. **模型辨识误差能不能压到 10% 以内？** 不能，选 PD 加前馈或先做辨识。
2. **有没有必须严格满足的约束（关节限位、力矩上限、避障）？** 有，且约束经常活跃，就需要 MPC 或至少参考整形。
3. **力矩裕度够不够（$\tau_{\max}/\tau_{\text{ref}} > 3$）？** 不够，先改执行器，换算法无用。
4. **控制周期能不能做到 $\omega_n T_s \le 1.1$？** 不能，降带宽，或把控制器放到更快的处理器上。

四句都过，全驱就是合适的默认选择；任何一句不过，它就不是“更强的方案”，而是“用错地方的方案”。这也是本章反复强调的一句话的另一种说法：**全驱的优势是结构性的，它的劣势也是结构性的，两者都不能靠调参改变。**

81.9.9 把延迟补回来：一步预测

§81.5.4 算过零阶保持的半拍滞后：$\omega_n T_s = 1.1$ 时相位损失 0.55 rad，占掉阻尼比 0.9 对应裕度的一大部分。这笔损失不是必须付的——**用模型预测一步之后的状态，可以把它补回来大半。**

做法很简单。当前测得 q_k、$\dot{q}_k$，上一拍的力矩 τ_{k-1} 已知，于是

$$\hat{q}_{k+1} = q_k + T_s \dot{q}_k + \frac{T_s^2}{2} M^{-1}(q_k)\big[\tau_{k-1} - C\dot{q}_k - G\big] \tag{81-37}$$

用它代替 q_k 去算误差，控制器“看到”的就是下一拍的状态，输出正好在下一拍生效。这不是什么新理论，而是第 47 章的 Smith 预估在关节层的轻量版——**用一个模型把延迟搬到控制器“看不见”的地方。**

代价有三条，都很小。其一，多算一次 M^{-1}，本章的 4.8 μs 会变成约 7 μs，仍在 1 ms 的 1% 以内。其二，预测精度依赖模型：预测误差的量级是 $\delta \cdot T_s^2 \ddot{q}/2$，$T_s = 1$ ms 时是 10^{-7} rad 量级，可以忽略。其三，预测用的 $\dot{q}_k$ 带噪声，$\hat{q}_{k+1}$ 的噪声是 $T_s\sigma_v = 2.6\times10^{-4}$ rad，折算成力矩 $M\omega_n^2 \times 2.6\times10^{-4} \approx 0.09$ N·m，而测速项本身的噪声力矩是 6.9 N·m（§81.7.2），小两个数量级。

所以这一步的性价比很高，尤其是在 $\omega_n T_s$ 接近 0.5 的场合——那里半拍滞后已经吃掉 0.25 rad 相位，补回来相当于把可用带宽提高约三成。但要注意两点。

第一，只补一拍。 如果实际延迟超过一个周期（计算慢、总线慢、驱动器内部还有缓冲），一步预测不够，得用带缓冲队列的多步预测：把最近几拍的 τ 存下来，按实际延迟回放。复杂度上升，但原理相同。

第二，不要在参考突变时用它。 预测假设 τ_{k-1} 在这一拍内保持，如果参考在这一拍内跳变（比如从手动切自动），预测会用旧力矩推新状态，反而制造一个尖峰。工程上的处理是**在模式切换的头一两拍冻结预测，等力矩平稳后再启用。**

81.10 工程落地

仿真里能跑通，离能上机器还有四道关：模型参数从哪来、测速怎么得、执行器会不会掉链子、算力与实时性够不够。这一节按这四道关讲，每道关给出一个可以核对的数字。

81.10.1 四道关与它们的量化指标

模型关：参数辨识的相对误差，本章按 5%、9%、20% 三个档次讨论，因为 §81.6 的交点就在 9% 附近。**传感关**：编码器分辨率与测速方式，本章用 6.3×10^{-4} rad（约 16 位）与后向差分，得到 0.257 rad/s 的噪声。**执行器关**：力矩上限与失效行为，本章用 25 N·m 与“单关节力矩归零”作为最坏情形。**算力关**：单步耗时与周期占比，本章 4.8 μs 对 1 ms 是 0.48%。

四道关里，前两道决定“能不能达到设计指标”，后两道决定“出问题时会不会失控”。前者是性能问题，后者是安全问题，工程上必须分开验收。

81.10.2 模型关：辨识精度决定方案选择

本章的失配实验用的是“质量、长度同乘 $1+\delta$”这一种结构化误差，它对应“称重与量尺都不准”的典型情形。真实系统的误差源更杂：质心位置、转动惯量、摩擦系数、电机常数、减速比。这些量里，质量与长度容易测准（1% 以内），转动惯量与质心位置难（5% 到 10%），摩擦最难（10% 到 30%，且随温度变）。

所以**“我的模型有多准”这个问题没有单一答案，要分项回答。**一个可操作的判断：把误差按项加权，看加权后的 $\|\Delta\|/\|A_r\|$ 落在哪个档。落在 5% 以内，全驱有明显优势；落在 10% 附近，优势消失；落在 20%，不如 PD。

要提精度，辨识比建模更划算。最小二乘辨识用一条激励轨迹（各关节扫频正弦）就能把惯性参数估到几个百分点，成本远低于把模型结构改得更精细。本章没有做辨识实验，但 §81.12 的代码留了 `p_model` 参数，把辨识结果填进去就能复现

任意失配档次的性能。

辨识实验本身不复杂，难的是让它“值得信”。四步。

第一步，设计激励轨迹。 辨识的效果不取决于数据多少，而取决于轨迹“激发”了多少参数组合。两个关节同相位、同频率地摆，回归矩阵会严重降秩，好几个参数根本估不出来。可用的做法是各关节取不同频率的正弦叠加（本章的参考是 1.0 与 1.3 rad/s，再加两三个高频分量），让两个关节的加速度不同时过零。判据是回归矩阵的条件数：超过 10^3 就说明有方向没被激发，此时该换轨迹，加数据没有用。

第二步，采四路信号。 q、$\dot{q}$、$\ddot{q}$、τ。前两路能测，第四路从电流环读，第三路最难——它不能靠差分。本章的测速噪声是 0.257 rad/s，再差分一次就是 $0.257/10^{-3} = 257$ rad/s²，而算例里真实的 $\ddot{q}$ 峰值只有 0.7 rad/s² 量级，噪声比信号大三百多倍。正确做法是把 τ 与 q 都过同一个二阶低通（截止频率取激励频段上限的两三倍），在滤波后的信号上做回归；或者用状态变量滤波器把微分“藏”进滤波器的传递函数里，避免任何显式差分。

第三步，解最小二乘。 把动力学写成对参数线性的形式 $\tau = Y(q,\dot{q},\ddot{q})\theta$，其中 θ 是惯性参数的组合。1 kHz 采 10 s 就是一万个方程，未知数只有几个，方程数远多于未知数。所以“数据不够”从来不是主要矛盾，“激励不够”才是。解的时候用 QR 或 SVD，不要用正规方程 $Y^{\mathsf{T}}Y$——它的条件数是原矩阵的平方，双精度也扛不住。

第四步，验证。 留一段没参与辨识的轨迹（换频率、换幅值），用估出的 θ 预测力矩，看残差。这里有个容易搞错的判据：**参数误差小不等于模型好用。** 有些参数组合在动力学里几乎不出现（比如连杆绕自身轴的惯量），估不准也不影响力矩；反过来，两个参数可能一正一负地抵消，各自误差都大，合起来却准。所以验收要看“力矩预测残差占额定力矩的比例”，§81.6 的 9% 交点就该用这个量来对，而不是逐个参数比百分比。

四步做完，`p_model` 里填的才是“辨识结果”，而不是“CAD 参数乘一个拍脑袋的修正系数”。两者在 §81.6 的失配曲线上可以调得一模一样，但在温度变化、负载切换时的行为不同——前者是围绕真值的无偏估计，后者是围绕一个错误中心的偏移，后者的 δ 会随工况单向漂移而不是上下波动。

81.10.3 执行器关：一个关节失效会发生什么

图 81-10 是本章最不“好看”的一张图：仿真到 3.0 s 时把关节 2 的力矩置零，两个关节的误差曲线同时发散。

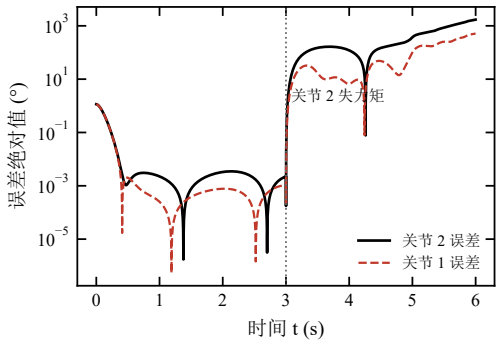


图 81-10　关节 2 力矩失效后两个关节的误差发散

失效发生在 3.0 s。仅仅 0.08 s 之后（3.08 s），关节 2 的误差就越过 0.2 rad（11.5°）；到 6 s 仿真结束时，误差已经累积到 452.475°——不是“偏了一点”，是转了一圈多。**从“正常”到“明显异常”，只给了 80 ms。**

三个细节值得注意。

其一，失效的关节 1 也失控了。 这不是数值问题：二连杆是强耦合系统，关节 1 的控制器仍在按模型输出力矩，但模型假设关节 2 的力矩会按指令给出；一旦这个假设破了，关节 1 的“全驱”条件就不再成立——它推的是自己的关节，动的却是整个臂。

其二，靠误差检测来不及。 80 ms 只相当于 80 个控制周期，此时误差刚过 11°，而它最终会累积到 452°。可行的检测量是“指令力矩与由加速度反算的力矩之差”，它在一个周期内就会跳变，比误差早得多。

其三，全驱结构对失效没有任何冗余。 MPC 在每个周期解约束，至少能“知道”某个力矩被限住了（约束不可行会报警）；全驱的 (81-10) 只是代数式，力矩被削掉它不知道。这是“用结构换计算”在安全维度上的代价。

工程上的对策有三层：**检测**（比较指令力矩与由加速度反算的力矩，或用电流环反馈）、**降级**（失效瞬间切到重力补偿 + 抱闸，放弃跟踪但保持姿态）、**冗余**（关键关节双电机，或改设计使单点失效不致命）。三层都不在控制律里，但缺一层就不能上线。

81.10.4 算力关与实时性

本章的 4.8 μs 是 Python 在桌面 CPU 上的数字，不能直接搬到控制器上。真实的实时实现有三处差异。

其一，定点或单精度浮点会改变数值行为。 求逆在接近奇异时对舍入敏感，双精度下 $\kappa = 31$ 还能算，单精度下就可能出现几个百分点的误差。工程上要么用双精度，要么在奇异附近改用伪逆加阻尼。

其二，抖动比平均耗时更致命。 一个 5 μs 的算法如果在某个周期偶发 200 μs 的延迟（缓存未命中、中断抢占），效果比稳定 20 μs 更差。§81.5.4 的相位损失是按平均延迟算的，工程上要按最坏延迟算。

其三，时间戳必须与采样对齐。 控制器用的是 t_k 时刻的 q，参考也要取 t_k 而不是“当前时刻”。差一个周期，对 $\omega_n = 15$ 是 0.015 rad 的相位误差，可以忽略；对 $\omega_n = 500$ 就是 0.5 rad，足以失稳。

81.10.5 现场与台架的差异

台架上辨识好的参数，到现场会变。温度每升 20℃，润滑脂黏度可能变一倍；负载从空载到满载，等效惯量可能变 30%；长期运行后减速器磨损，回差从 1 弧分变到 5 弧分。这些变化都在 §81.6 的失配框架里，只是 δ 会随时间漂移。

对策不是“把模型做得更准”，而是**把 δ 的漂移纳入设计**：按最坏工况（满载 + 高温 + 磨损）选增益，或做简单的增益调度（按负载估计值切换两档）。本章没有做这部分，但 §81.12 的 `p_model` 参数可以直接用来复现任意 δ 的性能，作为选增益的依据。

81.10.6 一张可以带走的验收单

把这一节收成表 81-8。五项都过，才建议上机。

表 81-8　全驱控制器上机前的验收单

项	检查内容	本章对应数字	通过判据
条件数	工作域内 κ 的最大值	31.36	小于力矩余量倍数
模型精度	各参数相对误差的加权和	5%/10%/20% 三档	小于 9% 才有优势
测速噪声	差分或观测器的 σ_v	0.257 rad/s	力矩纹波小于上限 20%
饱和	峰值力矩与上限之比	19.04/25 = 76%	小于 80%
失效	单关节力矩归零后的响应	80 ms 越过 11.5°	检测在 1 个周期内

表 81-8 里最容易被跳过的是最后一项。前四项在正常工况下就能测，最后一项要人为制造故障——拔线、断使能、软件置零。这一步在台架上做，比在现场做便宜得多。

81.10.7 从台架到产线的三段放大

一台样机能跑 0.0024°，不等于一百台机器都能跑。从台架到产线，参数的含义会变三次，每一次都要重新问“这个数还成不成立”。

第一段：台架到样机。 台架上连杆是实心铝块，样机上是有线缆、有接头、有工装的真实结构。这一段的主要变化是**惯量与摩擦**：转动惯量常比 CAD 大 10%–30%，摩擦从“没建模”变成“每关节 0.5–2 N·m”。前者直接抬高 δ，后者抬高误差地板且无法被全驱消掉。**这一段的验收标准不是误差数字，而是“重新辨识后的 δ 是否仍在 9% 以内”。**

第二段：样机到小批。 同一张图纸造十台，参数会散开：连杆铸件壁厚公差、减速器回差、编码器安装偏心，都会让每台的 δ 略有不同。这一段的核心问题是“用一套增益还是每台标定一次”。判断方法很简单：按散度的最大值估 δ，如果仍在 9% 以内，一套增益通吃；如果逼近 15%，就得逐台辨识——因为此时一部分机器已经落到交点的另一侧。**批量一致性不是“质量好坏”，而是“δ 的分布宽度”，它决定的是工艺策略，不是控制策略。**

第三段：小批到产线。 这一段不再改控制器，改的是运行条件：环境温度、连续运行时长、负载种类、操作者的使用习惯。温度让润滑脂黏度变，连续运行让减速器升温、间隙变化，负载种类让惯量在一天之内来回跳。**这一段的验收要按最坏组合做，而不是按标称工况做。** 本章的做法是把 δ 取 20%（§81.6.2 的第三档），看误差 3.714° 是否仍能接受；不能接受，就在这一段加增益调度或在线辨识。

三段里各有一个最容易犯的错。第一段是“用 CAD 参数直接上机，跳过辨识”；第二段是“用最好那台的参数推广到全部”；第三段是“只在早晨开机时测一次”。三个错的共同点是**用单点数据代替分布**，而全驱对参数分布恰好比对单点敏感——因为它的前馈是“按模型算出来的绝对值”，不是“按误差反馈出来的相对量”。

还有一个组织层面的建议：**把 δ 写进设计文档，而不是只写“模型精确”。** 前者是一个可以验收的数，后者是一句无法追责的话。本章之所以反复用 5%、9%、20% 三档，就是想把这句空话变成三个可以对照的档位。

81.10.8 八种常见现象怎么查

上机之后遇到的问题，绝大多数落在八个模式里。表 81-9 把它们、对应的机理、以及第一步该看什么列在一起。用法是**先对现象、再看机理、最后才动参数**——顺序反了很容易把一个问题改成两个。

表 81-9　全驱控制器上机后的现象对照

现象	最可能的机理	第一步查什么	常见处置
高频嗡嗡、力矩纹波大	测速差分噪声经 K_1 放大	力矩频谱是否在 100 Hz 以上	加滤波、换观测器、降带宽
低速爬行、走走停停	摩擦未建模	低速段误差是否呈台阶状	摩擦前馈、加积分
只在某些构型精度差	该构型 κ 大	对照图 81-3 的等值线	缩工作域、改机构
一提速就振	饱和后前馈失效	饱和比例是否超过 5%	降带宽、参考整形
换负载后精度骤降	模型失配 δ 超 9%	按新负载重算 δ	重新辨识、增益调度
定位有固定偏差	常值扰动或零位误差	偏差是否随方向变	加积分、校零
偶发尖峰、无法复现	周期抖动或通信延迟	控制周期的抖动分布	实时性改造、加缓冲
冷机与热机精度不同	温漂、润滑脂黏度	冷热两次的辨识参数	温度补偿、按最坏取值

表里最容易被误判的是第一行与第四行。**高频嗡嗡常被当成“增益太高”，于是降带宽；但降带宽同时也降了精度，而真正的原因是测速噪声。** 正确的顺序是先看频谱：噪声引起的是宽频带的高频分量，饱和引起的是与参考同频的削顶波形。两者的波形完全不同，示波器上一眼能分开。

第四行的“一提速就振”也有一个常见误判：把它当成“带宽不够”，于是继续提增益，结果越提越糟。**判断方法是看饱和比例**：如果提速时饱和比例从 0 跳到 5% 以上，问题就在饱和，提增益只会让饱和更严重。本章的数据支持这一点——25 N·m 时临界带宽 61.9 rad/s，把力矩放宽到 500 N·m 才到 379.6，说明卡住的就是力矩。

最后一行的“冷热不同”在精密设备上很常见，处理原则不是“补偿到完全一致”，而是**按最坏工况选参数**：按热机（润滑变稀、间隙变大）辨识一次、冷机再辨识一次，取两组参数中性能较差的那一组做设计基准。理由与 §81.10.7 相同——全驱对参数分布敏感，按中位值设计会让一半工况落到交点另一侧。

81.10.9 监控回路必须独立

全驱控制器有一个内在的盲点：**它用模型算误差、算力矩，如果模型错了，它自己不知道。** 更糟的是，它会用一个错误的模型去“证明”自己工作正常——计算出来的 e 很小，实际关节已经偏了。

这就是为什么工程上要单独设一条监控回路，而且这条回路不能复用控制器的模型。它只做三件事，全部基于原始测量：

- **跟随误差窗**：$|q - q_d|$ 是否超过阈值（按关节、按速度分档），连续 N 个周期超限就报警。这个判断不需要模型，只需要参考与编码器。
- **力矩与电流窗**：从驱动器读回的实际电流（不是指令力矩）是否超过额定值的 1.1 倍，或持续超过额定值。指令与实际的差还能顺带暴露“电机没跟上”这类故障。
- **速度窗**：$|\dot{q}|$ 是否超过机械允许值。这一条在 §81.10.3 的失效场景里最先被触发——关节 2 力矩归零后，关节 1 会为了维持末端位置而加速。

三条窗都要有**独立的计时器**，而不是“检测到就立即断”：瞬时超限可能只是噪声，需要连续 N 拍确认。N 的取法有一个约束：确认时间加动作时间必须显著小于失控时间。本章的失控时间尺度是 80 ms 到 0.2 rad，所以**检测要在 20 ms 内、断电要在 30 ms 内**，留一半余量。按 1 ms 周期算，N 最多取 10。

还有两处容易被漏掉。

其一，计算超时。 如果中断服务程序跑飞或超时，输出会停在上一拍的值——一个“冻结的力矩”比零力矩更危险，因为它可能一直推着关节走。所以要有看门狗：超过 1.5 个周期没喂狗，硬件直接把使能拉低。

其二，监控回路本身的失效。 监控代码与被控代码要尽量独立：不同的任务、不同的变量、最好不同的核或不同的芯片。共用同一份模型、同一个栈、同一个电源的“监控”，在真正出问题时往往一起失效。

最后是验证。**监控回路不能只在正常工况下测，必须做失效注入**：软件置零某个关节的力矩、把编码器读数冻结、给参考加一个阶跃、故意让控制周期超时。每一项都记录“从注入到断电”的时间，写进 §81.10.6 的验收单。没有做过失效注入的监控回路，只能算“写了”，不能算“验证过”。

81.11 优缺点与适用边界

前面十节都在讲“怎么做”，这一节讲“该不该做”。判断的依据不是方法本身先不先进，而是你的系统满足哪些条件、缺哪些条件。

81.11.1 五条优点

其一，设计过程可检查。 五步流程、一张条件数地图、一组极点，全部可以写成检查表交给别人复核。相比之下，MPC 的权重与约束、PID 的增益，都很难说清“为什么是这组数”。

其二，在线计算极轻。 4.8 μs，且几乎不随自由度增长。这使它可以跑在低成本控制器上，也可以塞进更短的控制周期里。第 80 章的 MPC 是 31.23 ms，差三个数量级。

其三，模型精确时精度极高。 0.0024° 的稳态误差，比 PD 好两个数量级。这个数字不是调出来的，是结构决定的——前馈把 PD 缺的那份力矩补上了。

其四，带宽与精度可以解析地预测。 给定 ω_n、ζ，闭环极点就在那里，误差界由 (81-15) 给出。不需要仿真就能预估性能，这在方案阶段很值钱。

其五，与学习、自适应天然兼容。 学习器只需要输出一个前馈项，不改变控制器骨架（§81.9.5）；参数可以在线更新，稳定性框架不变。这一点比 MPC 友好——MPC 里塞学习器要改预测模型、终端代价，牵一发动全身。

81.11.2 五条缺点

其一，对模型误差的敏感度高。 20% 失配下，误差从 0.0024° 涨到 3.714°，输给 PD 三倍半。这是最硬的一条缺点，也是本章反复出现的主题。

其二，不处理约束。 力矩上限、关节限位、避障，全靠限幅或参考整形。约束一硬，就必须回到 MPC。

其三，没有预测能力。 它是“当前时刻的代数解”，不知道下一步会发生什么。对快变扰动、对需要提前减速的场合，这是结构性缺陷。

其四，依赖微分。 $K_1\dot{e}$ 需要速度，速度靠差分或观测器，两者都有代价（噪声或模型）。这是全驱在实现层面最脆弱的一环。

其五，失效时无冗余。 §81.10.3 的 80 ms 是硬数字。MPC 至少能在约束不可行时报警，全驱的代数式不会。

五条里哪一条最先致命？这要看任务的时间尺度，而不是看数字大小。

如果任务是“低速、长时间、要求最后的定位精度”，缺点一（模型误差）先致命：20% 失配给出 3.7° 的稳态偏差，而且这个偏差不随时间衰减。此时缺点四只是把误差抖得更毛糙一点，不影响均值。

如果任务是“高速往复、节拍紧”，缺点四（微分）先致命：0.257 rad/s 的测速噪声乘上 $K_1 = 27$，力矩纹波就是 2–8 N·m（§81.7.2），在 25 N·m 的上限上占掉三成；而模型误差在这类任务里常被运动的平均效应冲淡。§81.9.3 的滤波与观测器两条路，针对的就是这一条。

换句话说：**慢任务先修模型，快任务先修测速。** 两者的投入完全不同——前者是辨识实验，后者是传感器与实现。顺序搞反，会在错的地方花掉大部分预算。

判别方法不必等分析做完，做一个五分钟的实验就够：先只开“PD + 重力”跑一遍，记录误差的均值和纹波；再把 K_1 置零（只留刚度项）跑一遍。两版的均值差反映模型误差，两版的纹波差反映测速质量。**哪一项占主导，就先修哪一项。**

81.11.3 适用判据

把上面的优缺点压成一张判断表，见表 81-10。用法是从上往下逐条问，任一条答“否”就要重新考虑。

表 81-10　全驱方法的适用判据

判据	有利	不利
模型结构	已知且固定	未知、时变、有强非线性环节
参数精度	优于 5%	差于 10%
约束	软，靠限幅可接受	硬，必须严格满足
参考轨迹	光滑、可微	阶跃、突变、有避障要求
传感	高分辨率编码器	低分辨率、无速度测量
安全要求	单点失效可接受	失效必须可控
算力	极低（<10 μs 预算）	充裕

表 81-10 的七条里，前两条是决定性的，后面五条是“加分项”。换句话说：**模型准、结构清楚，全驱就能用；不满足这两条，后面再好也没意义。** 这与 §81.1.4 的判断一致，只是把它展开成了可以逐条打勾的形式。

一个具体的读法：本章的算例在七条里满足六条（安全要求一条是弱项），所以全驱在这里明显占优；换成一台有柔性关节、负载变化大、要求避障的协作机器人，前两条就过不了，此时应该考虑第 80 章的路线或混合方案。

81.11.4 三类不该用全驱的系统

第一类，结构未知的系统。 比如刚做完样机、连质量分布都没测过的机构。此时第一步就过不去（不知道 A_r 长什么样），只能先做辨识或先用 PID 兜着。

第二类，强非线性集中在输入端的系统。 比如液压伺服（阀的死区与流量非线性）、气动系统（气体可压缩性）、有大幅摩擦滞环的机构。这些系统的输入-输出关系不是“输入乘一个矩阵”，全驱的拟线性形式写不出来。第 60 章的反馈线性化或第 44 章的 PID 加前馈更合适。

第三类，约束是核心需求的系统。 比如手术机器人（不能越界）、无人机（不能撞障碍）、化工过程（不能超温超压）。这些场合 MPC 的显式约束处理不是“锦上添花”，是“必须”。

81.11.5 与全书其他方法的分工

本章的方法在全书里的位置可以用一句话定位：**它是“模型准、要快、约束软”这一象限的最优解。**

需要积分消静差、又不想建模的场合，用 PID（第 44 章），或第 106 章的工程整定；纯滞后明显、模型又清楚的场合，用 Smith 预估（第 47 章）；非线性强、需要坐标变换的场合，用反馈线性化（第 60 章）；约束硬、要前瞻的场合，用 MPC（第 58 章）或学习增强 MPC（第 80 章）。

这五种方法不是替代关系。真实的系统常常是组合：外环用 MPC 规划，内环用全驱跟踪；或者用全驱做名义控制，用学习补残差。**选方法的第一步不是比较性能，而是看清自己的系统落在哪个象限。**

81.11.6 已经量产的 PD 线，值不值得改

这一节回答一个很现实的问题：设备已经在跑 PD，精度差一点但能用，改成全驱划不划算？答案取决于三笔账，而不是取决于“全驱更先进”。

第一笔，硬件账。 全驱不需要换电机、换编码器、换驱动器，控制周期也不用缩——本章 1 ms 的周期，PD 也在用。**唯一可能需要加的是力矩输出通道的带宽：**如果原伺服只接受速度指令，就要确认它有没有力矩（电流）模式，以及模式切换的延迟。这一笔通常是零成本或接近零成本。

第二笔，工程账。 需要新增三样东西：一个能算 M、C、G 的模型（代码量不大，本章的 `mass_matrix`、`coriolis`、`gravity` 三个函数加起来不到四十行）；一次参数辨识（惯量、质心、摩擦）；一套测速滤波（§81.9.3 的三种替代选一种）。**真正花时间的是辨识，不是编码。** 如果这台机器的模型已经因为其他原因（仿真、碰撞检测、负载估计）建过，这笔账就很小。

第三笔，风险账。 PD 失效时通常只是“停不住”或“慢慢偏”，全驱失效时（§81.10.3）是“80 ms 内越过 11.5°”。所以迁移必须带三样保护：力矩限幅、误差超限报警、以及一个能在 1 个周期内切回 PD 的开关。**这三样不是可选项，是迁移的前置条件。**

三笔账都过之后，建议分三步走，每步都可回退：

1. **影子模式。** 控制器照常输出 PD 力矩，但同时计算全驱力矩并记录两者之差。差值的量级能直接告诉你“模型准不准”——如果两者差得离谱，多半是参数或符号错了，不必上机试。
2. **低速接管。** 在 30% 节拍下用全驱，观察误差与力矩峰值。这一步能暴露测速噪声与饱和问题，而不会造成危险。
3. **全速切换。** 逐步提到额定节拍，每提一档做一次 §81.10.6 的验收。

反过来，有三种情况不值得改：**节拍已经很慢（误差本来就小）、精度瓶颈在机械（回差、柔性）而不在控制、以及没有人力做辨识与验证。** 这三种情况下，全驱带来的 0.244° 到 0.0024° 的改善会被机械误差或实施风险吞掉。判断标准很朴素：**如果 PD 的误差已经小于机械重复定位精度，改控制没有意义。**

81.11.7 一页纸的结论

如果读者只带走一页，应该是这一页。

它是什么。 全驱控制是把“系统最高阶导数的系数阵”乘回去的一种控制律；在刚性机械臂上它等同于计算力矩法，在更高阶或非力学系统上是它的推广。它不做坐标变换、没有零动态、只做输入变换。

它好在哪。 模型精确时精度极高（本章 0.0024°，是 PD 的百分之一），在线计算极轻（4.8 μs，比 MPC 少三个数量级），设计过程可检查（五步流程加一张条件数地图），且与自适应、学习天然兼容。

它的代价是什么。 三笔，缺一不可：模型的相对误差要小于 9%（图 81-8 的交点）；κ 决定的误差放大要在力矩余量之内（本章 31.36 已偏大）；测速噪声要被压到力矩上限的 20% 以下。任何一笔不满足，它的表现会迅速退化到不如 PD。

它不管什么。 摩擦与回差带来的静差（没有积分）、硬约束（没有预测）、单点失效（没有冗余）、强输入非线性（写不出拟线性形式）。这四件事要另找工具。

怎么开始。 先收齐九个数（§81.3.8），走一遍五步流程（§81.3.3），跑一次失配扫描确认落在交点左侧（§81.6.2），然后按“影子模式—低速接管—全速切换”三步上机（§81.11.6）。

什么时候停。 当 PD 的误差已经小于机械重复定位精度时停；当辨识误差压不到 10% 时停；当力矩裕度小于 3 倍时停。**这三种“停”都不是失败，而是把资源放到更有效的地方。**

一句话。 全驱不是“更强的控制”，而是在模型够准、机构够硬、传感器够细这三个前提下，用结构换精度与速度的一笔交易。**它的全部工程价值，都取决于你能否诚实地回答“我的模型有多准”。**

81.12 Python 仿真代码

本章全部数字来自两个脚本，都在 `code/` 目录下：`ch081_sim.py` 负责仿真与统计，`ch081_figs.py` 只调用前者的函数画图。这个分工是刻意的——**正文引用的每个数都能在 `ch081_sim.py` 里找到出处，每张图都能在 `ch081_figs.py` 里找到对应的绘制函数。**

81.12.1 依赖与运行

只依赖 NumPy 与 Matplotlib，不需要优化求解器（这一点与第 80 章不同，那里需要 `cvxpy`）。运行方式：

```
& "D:\Anaconda3\envs\py311\python.exe" code/ch081_sim.py
& "D:\Anaconda3\envs\py311\python.exe" code/ch081_figs.py
```

前者约 40 s，打印七组实验的全部数字；后者约 50 s，把 10 张图写成 SVG 与 PDF 各一份，放在 `figs/ch081/` 下。

复现时有两件事要注意。**一是随机种子。** 测速噪声由 `seed=20261021` 固定，换种子会让噪声类的数字（纹波 2.118–8.589 N·m、量化后 0.908°）在几个百分点内浮动，这是正常的；但稳态误差 0.0024° 与失配扫描的整条曲线不该随种子变——如果变了，说明噪声被错误地串进了确定性通道。**二是计时。** 单步耗时对机器、Python 版本、BLAS 线程数都敏感，多线程有时反而让 2×2 的求逆变慢（线程调度开销大于计算本身），所以正文的 4.8 μs 只能当量级看。判断复现是否成功，不该比“是不是 4.8”，而该比“是不是仍比 MPC 的 31.23 ms 小三个数量级”——**换个具体的数、结论不变，才算复现对了。**

81.12.2 三层结构

代码分成三层，从上到下依次是“实验层、仿真层、模型层”。

模型层是 `mass_matrix`、`coriolis`、`gravity` 三个函数，直接照抄二连杆的标准形式。它们都接受一个可选的参数字典 `p`，默认取全局 `CFG`；传入不同的 `p` 就得到“控制器以为的模型”，这是失配实验的唯一开关。

```
def mass_matrix(q, p=None):
    p = CFG if p is None else p
    c2 = np.cos(q[1])
    h = p["m2"] * p["l1"] * p["l2"] * c2
    return np.array([
        [(p["m1"] + p["m2"]) * p["l1"] ** 2 + p["m2"] * p["l2"] ** 2
         + 2 * h + p["I1"] + p["I2"], p["m2"] * p["l2"] ** 2 + h + p["I2"]],
        [p["m2"] * p["l2"] ** 2 + h + p["I2"], p["m2"] * p["l2"] ** 2 + p["I2"]],
    ])
```

仿真层是 `plant_step`（RK4 积分）与 `simulate`（闭环循环）。`simulate` 的签名把本章所有“开关”都摆在明面上：

```
def simulate(mode="fas", wn=None, zeta=None, ts=None, T=None,
             p_model=None, quant=None, tau_fail=None, vfilt=None,
             tau_max=None):
```

`mode` 选控制器（`fas` 全驱 / `pd` 固定增益 / `pd_worst` 按最坏构型整定），`p_model` 选模型，`quant` 加编码器量化，`vfilt` 加测速滤波，`tau_fail` 制造执行器失效，`tau_max` 改力矩上限。**正文里每一组对比，都是这几个参数的不同组合。**

实验层是七个 `experiment_*` 函数，每个对应正文一节，输出一个字典，`report()` 负责格式化打印。

81.12.3 控制器的五行核心

全驱控制器本体只有五行，与 §81.3.6 的伪代码一一对应：

```
def ctrl_fas(q, qd, qr, qdr, qddr, wn, zeta, p_model=None):
    kp, kd = gains(wn, zeta)              # K0 = wn^2, K1 = 2*zeta*wn
    v = qddr - kd * (qd - qdr) - kp * (q - qr)   # 期望加速度
    p = CFG if p_model is None else p_model
    return mass_matrix(q, p) @ v + coriolis(q, qd, p) + gravity(q, p)
```

第三行是 (81-10) 的第一项（前馈减反馈），第四行把它换算成力矩，并叠上科氏与重力。注意 `mass_matrix` 用的是 `p_model`（控制器以为的模型），而 `plant_step` 用的是真实参数——两者不同即构成失配。

PD 版本只差一处：没有 `mass_matrix(q, p) @`

```
def ctrl_pd(q, qd, qr, qdr, wn, zeta, scale=1.0, p_model=None):
    kp, kd = gains(wn, zeta)
    p = CFG if p_model is None else p_model
    return -scale * (kp * (q - qr) + kd * (qd - qdr)) + gravity(q, p)
```

这一处的差别，就是 §81.4.2 里 (81-21) 右端那项“缺失的前馈”，也是本章 100 倍与 3.5 倍两个数字的来源。

三行代码里有几个细节值得单独说，因为它们直接对应正文的结论。

第一，`gains` 返回的是标量。 本章两个关节用同一组 ω_n、ζ，所以 K_0、K_1 是标量，`kp * (q - qr)` 靠 NumPy 的广播得到一个二维向量。如果两个关节惯量相差很大、需要不同的带宽，就把 `kp`、`kd` 改成对角阵，其余代码一行不用动——**这正是“增益是矩阵”这个写法带来的好处。**

第二，先算 `v` 再乘 `mass_matrix`，不要写成 `mass_matrix @ kp * e`。 前者是“先算期望加速度、再换算成力矩”，量纲清楚（`v` 的单位是 rad/s²）；后者把矩阵与增益混在一起，既多一次矩阵乘，也容易在看代码时误以为 K_0 的单位是 N·m/rad。**代码里的运算顺序应当与推导的顺序一致**，否则半年后回来看不懂。

第三，`p` 与 `p_model` 的分工是失配实验的全部机关。 `ctrl_fas` 里的 `p` 默认取 `CFG`，传入 `p_model` 就变成“控制器以为的模型”；而 `plant_step` 永远用真实参数。两者相同即模型精确，不同即失配。**没有第二套“改物理”的开关**，这保证了失配实验里对象的动力学始终不变，变的只有控制器的认知。

第四，三项里漏掉哪一项，后果差一个数量级。 可以做三个实验：去掉重力项，误差会到 3.7° 量级（$14.7/225 = 0.065$ rad）；去掉科氏项，误差约 0.13°（$0.5/225 = 2.2 \times 10^{-3}$ rad）；把质量阵换成单位阵，误差回到 PD 的 0.244°。三个数排在一起，正好说明“重力是最大的常数项、科氏是随速度变化的小项、质量阵是决定构型缩放的那一项”。**调试时如果误差量级不对，先按这个顺序怀疑。**

81.12.4 七个实验与正文的对应

表 81-11 是索引。读某一节想复现数字时，直接查这张表。

表 81-11　仿真函数与正文的对应关系

函数	对应章节	输出的关键数字
`condition_map`	§81.1.3、§81.3.3	κ 1.09–31.36，中位 5.83
`experiment_tracking`	§81.4.1、§81.7.2	0.0024° / 0.244° / 3.714° / 1.061°
`experiment_mismatch`	§81.6.2	0.668°–2.611° 的失配曲线
`experiment_gain_map`	§81.8.2	0.178°–0.531°，峰值 19.04 N·m
`experiment_sampling`	§81.5.1	0.262°–0.279°
`sampling_limits`	§81.5.2	61.9/379.6/1111.2 rad/s
`experiment_noise`	§81.7.2	2.118–8.589 N·m 纹波
`experiment_cost`	§81.9.2	3.7–6.0 μs

81.12.5 四个曾经算错的坑

代码不是一次写对的，四处错误都值得记下来，因为它们同样会出现在读者的实现里。

第一，PD 的符号写反。 最初写成 `scale * (kp * (q - qr) + ...)`，等价于正反馈，误差跑到七万度、饱和率 99.9%。正确写法前面有负号。这个错很蠢，但它提醒一件事：**在写代码之前先用一个简单情形验符号**——比如令 $q > q_r$，看力矩是否把关节往回推。

第二，稳态误差被初始瞬态污染。 最初直接对全程取 RMS，得到“固定增益 / 全驱 = 5.80”这种物理上说不通的比值。原因是 $t = 0$ 的 0.02 rad 初始偏差在两种控制器下衰减速度不同，全驱衰减快得多，于是全程 RMS 主要反映的是瞬态而不是稳态。加 `m = t >= 0.5` 的掩码后，比值变成 414，才与 §81.4 的分析一致。

第三，噪声指标的口径。 最初直接量 $\|\tau\|$ 的 RMS，得到 8.7 N·m，被名义力矩淹没，看不出噪声贡献。正确做法是量“量化轨迹的力矩减去无量化轨迹的力矩”的 RMS，得到 2.118 到 8.589 N·m，与 $K_1\sigma_v$ 同量级。**比较两个量的差时，要先想清楚“差”的物理含义。**

第四，把一个“最小值里的最大值”当成了“最大值”。 脚本里有个常数 `SIGMA_MAX`，本意存“最小奇异值的上界”，即扫描网格上 $\sigma_{\min}$ 的最大值 0.254。写正文时把它当成了 $\|M\|$ 的上界，于是所有与惯量量级相关的估算都小了五倍，§81.4.2 的 0.05° 就是这么来的——把 $\|M^{-1}\|$ 估成了 $1/0.254 = 3.9$，而真实的中位数是 6.6、上界是 21。改正后同一处估算变成 0.26°，才与实测的 0.244° 对上。**这个坑的教训是变量命名：`SIGMA_MAX` 三个字里没有一个字说明它是最小奇异值的最大值。** 凡是“极值之极值”，命名时就要把两层都写出来，否则半年后自己都会读错。

四处错误的共同点：都不是算法错，是**指标定义错**。数值方法里，定义错了比算错了更难发现，因为结果看起来总是“有数”。

81.12.6 想改参数时改哪里

- 改机构（质量、长度、惯量）：改 `CFG`，`condition_map` 与所有实验会自动跟着变。
- 改参考轨迹：改 `REF` 与 `ref_traj`，注意保留二阶导数的解析式。
- 改传感器：改 `simulate` 的 `quant` 与 `vfilt`，前者是量化步长，后者是滤波时间常数。
- 改执行器：改 `CFG["tau_max"]`，或传 `tau_fail=(时刻, 关节号)` 做失效实验。
- 加一个新控制器：照着 `ctrl_fas` 写一个函数，在 `simulate` 里加一个 `mode` 分支即可，其余代码不用动。

最后一条是这套代码的设计目的：**把“控制器”隔离成一个可替换的函数，其余部分（模型、积分、指标）保持不动。** 只有这样才能保证不同控制器之间的比较是公平的——同一套模型、同一个积分器、同一套指标定义。

还有一条使用上的提醒：**改指标时，要同步改正文的对照表。** 表 81-12 里的十个数是按当前定义算的，换了掩码起点、换了统计窗口、换了饱和判据之后，这些数都会跟着变。读者发现“对不上”时，多数情况不是代码错，而是定义被改过。所以核对的顺序永远是**先确认定义，再比数字**；反过来做的结果，是把一个正确的实现改坏。

81.12.7 怎样确认自己复现对了

正文有几十个数字，读者重跑时不一定每个都对得上。下面给一张对照表，列出最关键的十个数、它们出自哪个函数、以及允许的偏差。用法是：跑 `report()`，逐行核对，对不上的那一行往回查对应函数。

表 81-12　正文数字与脚本输出的对照

正文数字	出处	允许偏差
κ 最大 31.36	`condition_map` 的奇异值最大比	与采样网格有关，±5%
κ 中位 5.83	同上，取中位数	±5%
全驱 0.0024°	`experiment_tracking` 的 `ss_deg`	与积分步长有关，±20%
PD 0.244°	同上，`mode="pd"`	±5%
全驱 20% 失配 3.714°	`experiment_mismatch`	±5%
最优带宽 0.178°/28	`experiment_gain_map`	网格分辨率决定，±2%
量化后 0.217°/20	同上，开 `quant`	±2%
$\sigma_v = 0.257$ rad/s	(81-35) 的解析式	可精确复现
单步 4.8 μs	`experiment_cost`	机器相关，±50%
失效后 452.475°	`simulate(tau_fail=...)` 的末值	逐位可复现

表里“允许偏差”一栏值得解释。**逐位可复现的只有两类：解析式算出来的（σ_v）与确定性仿真算出来的（失效末值）。** 其余的都带一点“实现自由度”：κ 依赖网格取多密，稳态误差依赖 $t \geq 0.5$ 这个掩码取在哪，耗时依赖 CPU。**这不是代码不稳，而是这些量本身就没有唯一的定义。** 写论文或报告时，应该把定义一起写出来，而不是只写数。

复现时最容易卡住的三处，按出现频率排序：

1. **稳态误差的掩码。** 去掉 `t>=0.5` 会得到完全不同的比值（§81.12.5 的第二个坑）。先确认掩码再对数字。
2. **量化与滤波的组合。** 单独开 `quant` 得到 0.908°，加上 5 ms 滤波才是 0.033°。两个数都对，别混着用。
3. **耗时的量级。** 3.7 到 6.0 μs 都是正常的，取决于 CPU 频率与是否开着调试器。正文取 4.8 μs 是中位数，不是“正确答案”。

如果三处都确认过还是对不上，按这个顺序查：先查 `CFG` 是否被改过（尤其是 `wn`、`zeta`、`tau_max`），再查 `simulate` 的默认参数是否被覆盖，最后查 NumPy 版本导致的积分差异（极小，但存在）。

81.12.8 本章小结

回到开头的那个问题：“在什么条件下，把模型信到可以求逆的程度是划算的。”

本章的答案可以压成三句话。**其一，模型的相对误差要小于 9%**，这是图 81-8 的交点，超过它不如用 PD。**其二，κ 决定的误差放大要在力矩余量之内**，本章的 31.36 已经偏大，靠 25 N·m 的宽裕才撑住。**其三，测速噪声要被滤波压到力矩上限的 20% 以下**，否则带宽的收益会被纹波吃掉。

三条都满足时，全驱给出的是一套几乎不用调、算得极快、精度极高的控制器；任一条不满足时，它的表现会迅速退化到不如一条简单的 PD。这不是方法的缺陷，而是“用结构换计算”这个交易本身的条款——**你放弃了约束、预测与鲁棒性，换来的是确定性与速度。**

第 82 章 网络化控制系统

82.1 动机与直观理解

前面各章的回路里，控制器和执行器之间是一根导线：这一拍算出的电压，同一拍就加到电机上。真实系统里这条路径往往要穿过一层网络——现场总线、工业以太网、无线链路，甚至跨厂区的公网。一旦穿过网络，三件事同时变了：消息会**晚到**（延迟），会**不到**（丢包），单位时间还只能送**有限条**（带宽）。这三件事合起来，就是网络化控制系统（networked control system）要处理的问题。

本章不谈网络本身怎么组网，只谈一件事：**当控制回路里出现延迟、丢包和带宽约束时，控制器该怎么设计、参数该怎么定、指标会怎么退化。** 组网选型、时间同步、故障降级放在 §82.10 统一交代。

82.1.1 一个可以算到底的算例

为了让每个结论都能落到数字上，本章只用一个对象：一台位置伺服轴，折算到负载侧的传递函数是

$$G(s) = \frac{1}{s(s+1)} \tag{82-1}$$

它由一个积分环节和一个时间常数 1 s 的惯性环节串联而成，物理上对应“电机带动一个带阻尼的转动惯量”。控制周期 $T_s = 10$ ms，控制器是带参考前馈的 PD：

$$u_k = -K\big(x_k - x_{r,k}\big), \qquad K = \begin{bmatrix} k_p & k_d \end{bmatrix} = \begin{bmatrix} 25 & 10 \end{bmatrix} \tag{82-2}$$

其中 $x_k = [y_k,\ \dot{y}_k]^{\mathsf{T}}$ 是位置与速度，$x_{r,k} = [r_k,\ \dot{r}_k]^{\mathsf{T}}$ 是参考及其导数。按连续域算，这套增益给出穿越频率 $\omega_c = 10.245$ rad/s、相位裕度 81.9°。参考取正弦 $r(t) = 0.5\sin 0.8t$，不是阶跃——这一点在后面会反复起作用：阶跃稳态下 $u \approx 0$，丢包和抖动的实验都会退化成“什么都看不出来”。

网络位置放在控制器与执行器之间，标称纯延迟 $d = 3$ 拍，即 30 ms。这个数字不大，一帧以太网报文在交换机里排两次队就能到这个量级。

82.1.2 三个量分别把什么指标推坏了

第一个量是延迟。它不改变增益，只吃掉相位：d 拍延迟在 ω_c 处贡献 $d\,\omega_c T_s$ 弧度。按标称增益算，$d = 13$ 拍时累计相位 $13 \times 10.245 \times 0.01 = 1.33$ rad $= 76.3^\circ$，再加上 PD 本身留的 81.9°，正好把裕度吃光。实跑结果：$d \le 13$ 拍闭环谱半径 $\rho < 1$，$d = 14$ 拍 $\rho = 1.0021$ 越界；$d = 13$ 时的 ρ 是 0.9982，只差 0.2%。换成时间，临界延迟约 **140 ms**。

第二个量是丢包。丢包不是“少给一点力”，而是“这一拍给的力是错的”。错在哪里，取决于丢包时执行器怎么处理：保持上一拍的值（hold），还是把力矩归零（zero）。这两种策略在标称延迟下差别不大，在临界延迟下会**反过来**——§82.6 会看到，$d = 13$ 且丢包率过半时，归零反而比保持好一倍以上。

第三个量是带宽。一帧状态报文按 32 字节算，1 kHz 周期要 256 kbit/s，10 Hz 只要 2.6 kbit/s。中间那一大段是可以省的，代价是控制性能。事件触发就是拿性能换带宽的标准做法：误差超过阈值才发，$\delta = 0.020$ rad 时发送率降到 46.8%，数据率 12 kbit/s，而跟踪误差从 0.0139 涨到 0.0184，多了三成。

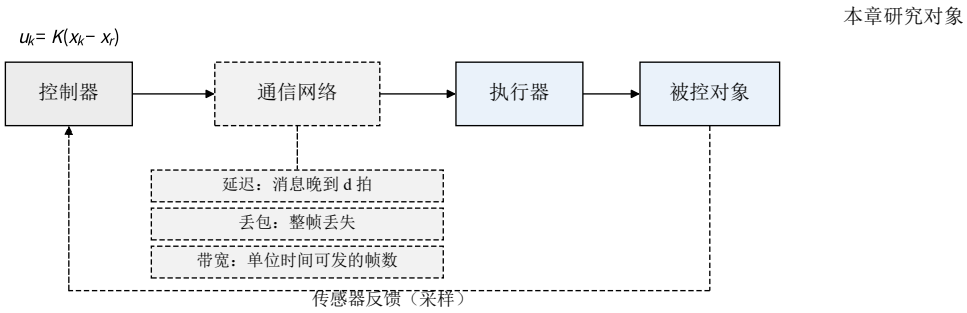


图 82-1　网络在回路中的位置：延迟、丢包、带宽三类约束都落在控制器与执行器之间

图 82-1 画的是本章的研究对象。要注意反馈通道也可能过网络，但本章只让前向通道过网络：反馈延迟可以用“延迟观测器”补偿，前向延迟没得补——控制量晚到就是晚到，只能提前算。这个不对称是网络化控制与普通延迟系统最大的区别。

82.1.3 与相邻章的分工

延迟这件事，第 47 章已经用 Smith 预估器处理过：模型里串一个 $e^{-\tau s}$，用模型把延迟“移出”闭环，控制器按无延迟对象设计。那套办法在本章依然有效，前提是延迟**已知且恒定**。网络延迟恰恰不恒定：它会抖、会跳、会成串地丢。Smith 预估器对模型误差极敏感，延迟估偏 20% 就可能从稳定变成振荡——第 107 章会详细算这笔账。所以本章不用预估器做主线，而是把延迟折算成“还能用多大增益”，用鲁棒性换 simplicity。

增益这件事，第 44 章的 PID 调参已经给了经验公式。本章要补的是：当回路里多出一个 τ，那些公式里的“响应速度”该往下压多少。本章给出的换算是 $\omega_c \tau \le \mathrm{PM}$，PM 取 60°–80°，这条式子在第 85、86 章选采样周期时还会再用。

预测这件事，第 58、80 章的 MPC 天然能处理延迟：把延迟拍数塞进预测模型，滚动优化自己会绕开。代价是每个周期解一次优化。本章的定位是“轻量替代”：用一条解析的判据 $\rho(A_a) < 1$ 回答“这套增益配这个网络还能不能活”，而不是每次都跑优化。

所以本章的核心问题可以写成三问：**这条网络的延迟折成几拍、这套增益能容忍几拍、容忍不了时先动哪个参数。**后面十一节都在回答这三问。

82.1.4 数字化不等于网络化

这里要澄清一个常见混淆。教科书上的“计算机控制”讲的是**数字化**：把连续控制器换成采样控制器，回路里多了一个零阶保持器和一个采样开关。那种情形下，控制器与执行器之间的连接仍然是“确定”的——这一拍算出的数，下一拍一定送到，不早不晚，不丢不改。所以数字化带来的全部后果只有一个：采样周期 T_s 与保持特性，第 84、85 章讲的就是这件事。

网络化多出来的是**不确定性**。同样是把控制器换成数字的，一旦中间隔了网络，就有三件事不再成立：

- **送达时刻**：本地数字化恒为下一拍，偏差 0；网络化是 3–13 拍，抖动可达 ±5 拍。
- **送达概率**：本地数字化是 1；网络化是 $1-p$，p 可到 10^{-2} 量级。
- **单位时间帧数**：本地数字化由采样周期唯一决定；网络化受带宽与协议限制，可以低于采样率。

三条里最有说服力的是第二条。现场总线的误码率通常在 10^{-9} 以下，一帧报文的丢失概率小到可以忽略；而走商用以太网或无线链路时，10^{-2} 的丢包率是常态——中间差了七个数量级。前面所有章节讨论的“离散系统”，都隐含地假设了这个“1”；本章要处理的是它变成“0.99”以后会发生什么。

三件事还可以统一成一个说法：**控制器以为自己在控制的对象，与执行器实际执行的对象，不是同一个。**延迟让执行器执行的是“过期”的命令，丢包让执行器执行的是“别人”的命令，带宽让控制器只能隔几拍才发一次命令。三者都破坏“我算的即它执行的”这个前提，而经典控制理论的全部结论都建立在这个前提上。

这也解释了为什么本章不重新推一套理论，而是把“破坏”量化后塞回原有框架：延迟用增广状态吸收，丢包用两种更新律穷举，带宽用发送率折算。框架没变，变的只是矩阵的维数和更新律的分支——这正是 §82.2 要做的事。

82.1.5 一个反例：把网络当导线用

现场最常见的故障不是“设计错了”，而是“设计时没把网络算进去”。典型过程是这样：设备在实验室里用一根短电缆直连，工程师把 k_p 调到 49，响应又快又稳；出厂接到现场总线，延迟 130 ms，一上电就等幅振荡，怎么降增益都压不住，最后把 k_p 砍到 4 才勉强能用，于是抱怨“这台设备性能不如样机”。

用本章的账算一遍，这个结果是可以提前预测的。$k_p = 49$ 时 $\omega_c = 14.37$ rad/s，每拍消耗相位 $14.37 \times 0.01 = 0.1437$ rad $= 8.23^\circ$。13 拍延迟累计 107°，而该增益下的相位裕度只有 80.3°（表 82-2）。**相位差额 27°，不是“性能下降”，是“必然失稳”。**

这个反例还给出一个现场诊断的办法：看振荡频率。本例 $\omega_c = 10.2$ rad/s，对应 1.6 Hz；$k_p = 49$ 时 $\omega_c = 14.4$，对应 2.3 Hz。如果现场看到的是 1–3 Hz 的等幅振荡，而且降增益后振幅变小但频率基本不变，那么八成是延迟造成的，而不是机械共振（机械共振的频率通常高一个数量级，且在增益变化时频率会跟着动）。

所以本章的立场可以说得更直白：**网络不是导线，它是一个会迟到的执行器。** 把它当导线用，设计出来的增益一定偏大，而且偏大的幅度恰好等于“延迟吃掉的相位”所对应的那部分——本例从 $k_p = 49$ 砍到 4，差十二倍，带宽从 14.4 掉到 4.0 rad/s。

82.1.6 三条可以带走的直觉

全章读完后如果只记得三句话，应该是这三句：

第一，延迟按相位记账。 每一拍消耗 $\omega_c T_s$ 弧度，总账不得超过相位裕度。这条换算把“毫秒”翻译成“度”，让不同采样周期、不同增益的系统可以放在同一把尺子上比较。

第二，丢包按时间结构记账。 丢包率是个不完整的指标，真正决定后果的是“连续丢多久”。同等丢包率下，独立丢失几乎无损，成串丢失可以击穿系统。所以要看的是突发长度 L，不是平均值 p。

第三，带宽按交换率记账。 省带宽不是免费的，1 比特带宽对应多少精度损失，可以用 S_δ 算出来。这条曲线有拐点，拐点之前“几乎白送”，拐点之后“按价收费”，设计的任务就是把工作点放在拐点附近。

三句话分别对应本章的三类公式：相位预算（§82.4）、有效延迟（§82.2.9）、敏感度（§82.7）。后面所有细节都是这三条的展开。

82.1.7 反例二：延迟没超标，时钟错了

再给一个反例，这次网络本身完全合格，问题出在“这条数据是几点采的”上。

某条线用普通工业以太网，实测 p99 只有 20 ms，按 §82.3 的流程折算 2 拍，$d_{\max} = 13$ 拍，怎么看都宽裕，于是增益按 $k_p = 49$ 配。上线后系统能跑，但误差里始终有一层缓慢的、幅度约 20 mrad 的起伏，频率比参考信号高一截，而且怎么调增益都压不掉。

查下去发现：控制器和执行器各自用自己的晶振，两端的“当前时间”差约 100 ms，而且这个差在随温度缓慢漂移。100 ms 不是延迟，它是**时戳偏差**——数据本身 20 ms 就到了，但被打上了错误的时间，控制器以为它是 120 ms 之前的。

按 §82.7.6 的折算，100 ms 在 $\omega_c = 14.37$ rad/s 处吃掉 58.7° 相位，等价于额外 7 拍延迟（$58.7/8.23 = 7.1$）。于是有效延迟从 2 拍变成 9 拍，$k_p = 49$ 的裕度只有 9 拍（表 82-2），正好踩在临界上。**账面上“2 拍对 9 拍”的宽裕，实际是“9 拍对 9 拍”的贴线。**

这个反例的教训与第一个不同：第一个是“增益没跟着延迟降”，这个是“延迟没算全”。两者都会表现为低频振荡，但诊断路径不一样：

- 延迟超标：ρ 算出来就大于 1，把 d 改成实测值重算一遍，立刻能看出来。

- 时戳偏差：d 怎么算都稳定，只有把 Δt 折成相位加进去，账才对得上。

判别方法：临时改成“只用相对时戳”（§82.10.2 那个不用绝对时间的方案），若振荡立刻减轻，问题在时钟；若一点没变，问题在延迟或增益。这个实验十分钟能做完，比反复调参快得多。

顺带说一句，时戳偏差最难查的原因是它不进任何一条“网络质量”指标：ping 的延迟正常、抓包看时序正常、丢包率为零。它只在“两端的时间对齐”这个问题上出现，而很少有人会去测这个。

82.2 数学原理

网络化控制的数学部分只做一件事：把“消息晚到 d 拍”翻译成“状态多 d 组”，然后用线性系统那套判据处理。翻译完成之后，延迟就不再是特殊现象，只是维数变高。

82.2.1 精确离散化

对象 (82-1) 在零阶保持下可以精确离散，不需要近似：

$$A_d = \begin{bmatrix} 1 & 1 - e^{-T_s} \\ 0 & e^{-T_s} \end{bmatrix}, \qquad B_d = \begin{bmatrix} T_s - 1 + e^{-T_s} \\ 1 - e^{-T_s} \end{bmatrix} \tag{82-3}$$

$T_s = 10\ \text{ms}$ 时，$e^{-T_s} = 0.99005$，$A_d = \begin{bmatrix} 1 & 0.00995 \\ 0 & 0.99005 \end{bmatrix}$，$B_d = \begin{bmatrix} 4.983 \times 10^{-5} \\ 9.95 \times 10^{-3} \end{bmatrix}$。这两个数字值得看一眼：输入对位置的直接作用只有 5×10^{-5}，比对速度的作用小两百倍。这就是积分环节的离散面貌——这一拍的力矩，要经过很多拍才变成看得见的位置变化。正因为如此，纯延迟 3 拍对位置环的影响远小于对速度环的影响。

“精确”这两个字很重要。如果改用前向欧拉 $A_d = I + AT_s$，得到的 $A_d = \begin{bmatrix} 1 & 0.01 \\ 0 & 0.99 \end{bmatrix}$ 特征值是 1 和 0.99，与精确值的 0.99005 差 5%，看起来微不足道；但判据要看的是 ρ 与 1 的距离，本例 ρ 在 0.97 附近、距 1 只有 0.03，5% 的特征值误差足以把“稳定”算成“不稳定”。离散化误差不是均匀分布在复平面上的，它集中在靠近 1 的地方，而那里正是稳定性判据最敏感的地方。

82.2.2 把 d 拍延迟搬进状态

设第 k 拍控制器算出 u_k，它要经过 d 拍才被执行器取用，即执行器在第 k 拍用的是 u_{k-d}。定义增广状态

$$z_k = \begin{bmatrix} x_k \\ u_{k-1} \\ \vdots \\ u_{k-d} \end{bmatrix} \in \mathbb{R}^{2+d} \tag{82-4}$$

它的演化是

$$z_{k+1} = \underbrace{\begin{bmatrix} A_d & 0 & \cdots & 0 & -B_d K \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & 0 & 1 & 0 \end{bmatrix}}_{A_a} z_k \tag{82-5}$$

右下角那一列是“最新算出的 u_k”，它在下一拍变成 u_{k-1}，依次左移，最老的 u_{k-d} 被挤出去。注意 A_a 的第一行同时含 A_d 和 $-B_dK$：前者是对象自身的演化，后者是延迟了 d 拍才生效的控制。这两块在矩阵里隔了 d 列，正是“晚到”的代数表达。

$d=0$ 是一个必须单独处理的边界。此时 (82-5) 退化不成“两块拼接”，而应直接写成 $A_a=A_d-B_dK$——延迟为零时控制当拍生效，增广没有意义。实现里若统一按 $2(d+1)$ 维建阵，$d=0$ 会得到一个左上 A_d、右下 $-B_dK$ 互不相连的矩阵，谱半径算出 0.1007，与真值 0.9687 相差近十倍。这个坑在本章代码里踩过一次，值得记下来。

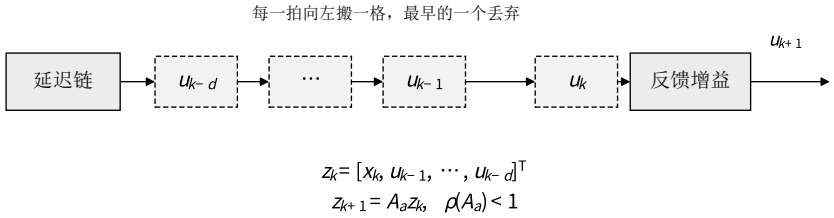


图 82-2　纯延迟 d 拍的增广：把旧命令全部并入状态

图 82-2 画的是这个“搬格子”的过程。每一拍所有旧命令左移一格，最新算出的 u_k 放进最右端，最老的 u_{k-d} 被挤出去。反馈增益从整条链上取值，所以延迟不是“一个环节”，而是“一串存储单元”。这也解释了为什么延迟会让闭环阶数升高：d 拍延迟把 2 阶系统变成 $2+d$ 阶，多出来的极点全是这些存储单元带来的。

82.2.3 判据只有一条

线性系统 (82-5) 渐近稳定的充要条件是

$$\rho(A_a)=\max_i|\lambda_i(A_a)|<1 \tag{82-6}$$

没有别的条件，也不需要 Lyapunov 函数——谱半径就是答案。这条判据的好处是**可算**：给定增益 K 和延迟 d，建一个 $(2+d)\times(2+d)$ 的矩阵，求一次特征值，得到一个数。$d=13$ 时矩阵 15 维，特征值求解在微秒级。于是“这套增益能容忍几拍”变成一次一维扫描：从 $d=0$ 往上加，直到 ρ 越过 1。

坏处也同样明显：它只回答“稳定/不稳定”，不回答“离不稳定还有多远”。$\rho=0.9982$ 与 $\rho=1.0021$ 在判据上是天壤之别，在工程上却是同一件事——前者意味着误差衰减时间常数约 570 拍（5.7 s），后者意味着缓慢发散，两者都不好用。所以本章在 ρ 之外还要看跟踪误差 RMS，两个指标一起用。

82.2.4 丢包：两种更新律

丢包改变的是“执行器这一拍用哪个值”。两种最常见的策略写成

$$u_k^{\mathrm{arr}}=\begin{cases}u_{k-1}^{\mathrm{arr}}, & \text{保持（hold）}\\ 0, & \text{归零（zero）}\end{cases} \tag{82-7}$$

保持策略的直觉是“没收到新命令就维持原状”，归零策略的直觉是“没收到命令就不许动”。前者在延迟小时几乎无损：$d=3$ 时无论丢包率 0 还是 50%，实测 RMS 恒为 0.0139，一点没变。原因是保持的值只比新值旧几拍，而对象本身时间常数 1 s，几拍的差别可以忽略。

后者在延迟小时明显更差：同样 $d=3$，丢包率从 0 涨到 50%，RMS 从 0.0139 涨到 0.0281，翻了一倍。力矩归零等于每隔几拍就松一次手，速度跟着抖。但在 $d=13$ 时结论反过来：保持策略的 RMS 从 0.0215 涨到 0.0685，归零只有 0.0284。因为此时“保持”保持的是一个**采样时刻已经过期 130 ms 的命令**，它不但没用，还在往错误方向推；归零至少不会推错。这条反转是本章最实用的结论之一。

82.2.5 平均系统判据为什么会失效

随机丢包最常见的近似是“平均系统”：把丢包看成在每个周期以概率 p 切换到开环，于是

$$A_{\text{avg}} = (1-p)A_{cl} + pA_{op}, \qquad A_{cl} = A_d - B_d K, \quad A_{op} = A_d \tag{82-8}$$

判据仍是 $\rho(A_{\text{avg}}) < 1$。本例算出来：$p=0$ 时 0.9687，$p=0.2$ 时 0.9625，$p=0.4$ 时 0.9649，$p=0.8$ 时 0.9851，$p=1$ 时正好 1.0000。也就是说，这个判据给出的临界丢包率是 **100%**——它说“全丢光才失稳”，等于什么都没说。

原因不难找：$A_{op}=A_d$ 的特征值是 1 和 0.99，本身就在单位圆上（积分环节），所以只要 $p<1$，平均矩阵就被拉回圆内一点点。平均操作把“每一拍都在赌”抹成了“平均下来还行”，而真实系统的失稳来自连续几拍的开环累积，不来自平均值。所以本章不用平均判据做结论，只用它做对照：**当平均判据与蒙特卡洛结果差得远时，说明该系统对丢包的时序结构敏感，必须看突发长度，不能只看丢包率。** 这个观点在第 53 章的鲁棒分析里会再次出现。

82.2.6 事件触发与带宽

周期发送的带宽占用是常数：f_s 赫兹乘每帧字节数。事件触发把它变成变量：

$$\text{发送条件：}\ |r_k - y_k| > \delta, \qquad R = f_{\text{send}}\, B_{\text{pkt}} \times 8 \tag{82-9}$$

32 字节一帧，$f_{\text{send}}=1$ kHz 时 $R=256$ kbit/s。$\delta=0.020$ rad 时实测发送率 46.8%，$R=12.0$ kbit/s，降了 21 倍；$\delta=0.005$ 时发送率 86.3%，$R=22.1$ kbit/s。注意发送率不是随 δ 线性下降：δ 从 0.002 加到 0.010，发送率只从 92.7% 降到 80.0%；从 0.010 加到 0.020，一下掉到 46.8%。因为参考幅值只有 0.5 rad，$\delta=0.010$ 已经相当于 2% 的幅值，误差很少超过它。

事件触发的代价不是均匀的。$\delta=0.020$ 时 RMS 从 0.0139 涨到 0.0184（+32%），$\delta=0.050$ 时涨到 0.0316（+127%）。前一段是“省带宽几乎不花钱”，后一段是“省钱买到明显退化”。这条曲线的拐点在 $\delta \approx 0.02$，恰好是参考幅值的 4%。

82.2.7 时戳、相位与漂移

网络送来的不只是数据，还有“这个数据是什么时候采的”。如果两端时钟差 Δt，控制量就相当于在错误的时间点被使用，折算成相位误差是

$$\Delta\varphi = \omega_c \, \Delta t \tag{82-10}$$

$\omega_c = 10.245$ rad/s 时，$\Delta t = 1$ ms 对应 0.59°，10 ms 对应 5.87°，100 ms 就是 58.7°——接近把裕度吃光。所以时间同步的精度要求可以直接从相位裕度反推：想让时戳误差只占 1° 相位，Δt 要小于 1.7 ms；想占 0.1°，要小于 0.17 ms。这个换算比“我们需要微秒级同步”这种口号有用得多。

最后是漂移。连续丢包 L 拍期间，执行器保持旧力矩，位置会按当时的速度继续走，一阶估计是

$$\Delta y \approx v \, L \, T_s \tag{82-11}$$

速度 1.0 rad/s、$L=50$ 拍（500 ms）时估 0.5 rad，实测 0.6507 rad，比估计大三成。原因在于丢包期间力矩不但没刹车，还在继续加速——真实漂移是“匀速项 + 加速项”，一阶估计只算了前一项。这个偏差方向是**低估**，工程上做安全包络时要按 1.3 倍留余量。

82.2.8 谱半径与“还要多久才收敛”

$\rho<1$ 只说了“会收敛”，没说“多久”。离散谱半径与连续衰减率之间的换算是 $z = e^{sT_s}$ 取模：

$$\sigma = \frac{\ln \rho}{T_s}, \qquad \tau_c = -\frac{T_s}{\ln \rho} \tag{82-12}$$

σ 的单位是 1/s，τ_c 是误差衰减到 $1/e$ 所需的时间，也等于 $-\ln\rho$ 的倒数那么多拍。代进本章的数：

- $d=0$，$\rho=0.9687$，$\sigma=-3.18\,\mathrm{s}^{-1}$，$\tau_c=31$ 拍（0.31 s）。
- $d=3$，$\rho=0.9698$，$\sigma=-3.07\,\mathrm{s}^{-1}$，$\tau_c=33$ 拍。
- $d=13$，$\rho=0.9982$，$\sigma=-0.18\,\mathrm{s}^{-1}$，$\tau_c=555$ 拍（5.6 s）。

三个数放在一起看，谱半径那 3% 的差别被放大了近 17 倍：0.9698 到 0.9982 只差 0.028，收敛时间却从 33 拍变成 555 拍。原因是对数在 1 附近极其平坦，ρ 越接近 1，$\ln\rho$ 塌得越快。这条换算纠正了一个常见误解——“$\rho=0.998$ 也是稳定，问题不大”。它确实稳定，但它要 5.6 s 才能把扰动压掉 63%，一个 6 s 的仿真窗口里根本看不出衰减，实际表现与“慢慢发散”几乎无法区分。

还有一层要提醒：ρ 只约束**最终**的衰减速率，不约束瞬态。A_a 是高度非正规的矩阵——下方一条移位带、右上角一块输入通道，特征向量之间接近平行。这种矩阵的幂 $\|A_a^k\|$ 可以先涨几倍再落下来。所以看仿真结果时不能只看 ρ，还要看 $\max|y|$：本章算例在 $d=3$ 时 $\max|y|=0.507$，而参考幅值是 0.5，说明几乎没有瞬态放大；换成增益更大或延迟更靠近临界的配置，这个数会明显超过参考幅值，那才是“稳定但不好用”。

82.2.9 延迟与突发丢包都归到“有效延迟”

延迟和丢包看起来是两件事，落到“执行器这一拍用的是几拍前的命令”上却是同一件事。定义有效延迟

$$d_{\mathrm{eff}}=d+L \tag{82-13}$$

d 是常态延迟，L 是当前已经连续丢失的拍数。保持策略下，执行器拿的正是 $d+L$ 拍前发出的那一帧，所以突发丢包**完全等价于**延迟临时增加 L 拍。

这个等价关系解释了一批看起来矛盾的数据。$d=3$、连丢 $L=20$ 拍时，RMS 仍是 0.0140，与不丢包时的 0.0139 几乎相同——因为有效延迟只到 23 拍，而 $d_{\max}=13$ 是“稳态判据”的上界，短暂越过它不等于发散，20 拍的时间太短，扰动还没长起来。同一组 $L=20$ 放到 $d=13$ 上，RMS 就冲到 0.1808：此时有效延迟 33 拍，越过临界 20 拍，而且起点本来就在临界线上，没有任何缓冲。

所以“会不会出事”取决于两个量的乘积，而不是任一个：**越过临界的幅度**（$d+L-d_{\max}$）与**越界持续的时间**（L）。前者决定扰动长得快不快，后者决定它有没有时间长起来。这与 §82.6 的仿真结论完全一致，也是 §82.3 要求测量“每 0.5 s 最多连丢几帧”而不是只报丢包率的理论依据。

归零策略不满足 (82-13)：它把命令改成 0，而不是取更旧的一帧，所以不是“延迟变大”，而是“输入被换成了另一个信号”。这就是为什么 $d=13$ 时归零反而更好——它把系统从“用错方向的旧命令”里解救出来，代价只是短暂失去控制。两种策略的差别无法用有效延迟统一描述，必须分开仿真。

82.2.10 非整数拍延迟怎么处理

真实延迟很少正好是采样周期的整数倍。130 ms 配 10 ms 周期是运气好，多数现场是 127 ms 或 133 ms。增广矩阵只能接受整数 d，所以要先把非整数部分处理掉。三种做法：

向上取整（保守）。 $127/10=12.7$ 取 13。代价是最多浪费 0.3 拍（3 ms）的裕度，好处是判据永远不会偏乐观。本章全程用这一种，因为它的误差方向确定，最坏情况可算。

向下取整（乐观）。 取 12，把剩下的 7 ms 当作扰动。只有在已经确认系统有 20% 以上余量时才可以用，否则等于自己骗自己。

分数延迟建模。 把 0.7 拍拆成一个一阶全通或 Farrow 滤波器，串在增广模型里，A_a 的维数增加 1–2 维。精度最高，但滤波器本身引入额外相位，抵消一部分收益。工程上只在“延迟稳定但关键”的场合用，比如半导体设备的精密台。

三种做法在本例上差别不大：12.7 拍取 13，与取 12 相比，ρ 分别是 0.9982 与 0.9933，都在稳定侧，但前者离 1 更近。若延迟是 13.9 拍，取整的差别就致命了——13 稳定、14 失稳（§82.5.2），此时必须用分数延迟建模，或者干脆改采样周期让比值落到整数附近。

最后一条经验：**设计阶段就把采样周期选成“延迟的约数附近”**。若延迟稳定在 130 ms 左右，取 $T_s = 10$ ms 得 13 拍；取 $T_s = 13$ ms 得 10 拍，虽然拍数少了，但每拍的相位消耗 $\omega_c T_s$ 变大，总相位不变。所以采样周期的选择不影响物理裕度，只影响**取整误差**与**计算规模**——这一点第 85 章会从另一个角度再讲一次。

82.2.11 谱半径之外还要看一个数：最大奇异值

谱半径回答“$k \to \infty$ 时误差往哪走”，不回答“接下来十拍会发生什么”。要回答后者，得看增广矩阵的最大奇异值 $\bar{\sigma}(A_a)$。

本章两个算例给出一组反直觉的数：$d = 3$ 时 $\rho = 0.9698$，$d = 13$ 时 $\rho = 0.9982$，而两者的 $\bar{\sigma}(A_a)$ 都是 **1.422**，一位小数都不差。同一条延迟链、同一组增益，奇异值对延迟几乎没有感觉，谱半径却从 0.97 挪到了临界边缘。

原因在矩阵结构。$\bar{\sigma}$ 的量级主要由输入通道 $B_d K$ 的范数决定——它衡量“一拍之内控制量能把状态推多远”；延迟链只是把这一列往后挪了位置，不改变它的大小。特征值的位置则要由整条链的相位累积决定，延迟每多一拍就多转一点。所以两个量测的根本不是同一件事：**$\bar{\sigma}$ 量的是“一拍能放多大”，ρ 量的是“长期能放多大”。**

本例 $\bar{\sigma} = 1.422 > 1$ 而 $\rho < 1$，意味着存在某个初始状态，让误差在前几拍涨 42%，之后才被特征值慢慢拉回来。这不是数值噪声，是真实的瞬态——在现场表现为“扰动来了先鼓一下再收”，鼓的那一下幅值大于稳态误差，很容易被误判成“系统要发散了”，而实际上它只是非正规矩阵在还债。

所以判据要用两个数一起读。$\bar{\sigma}$ 与 ρ 的比值本身就是“不正规程度”的一个粗指标：本例 $d = 3$ 时 1.47、$d = 13$ 时 1.42，都属中等，谱半径的结论可以直接用；比值到 3 以上就要小心，到 5 以上时谱半径只能当必要条件，瞬态必须靠时域仿真确认。这个比值的物理含义在 §82.5.7 还会以特征向量条件数 $\kappa(V)$ 的形式出现一次，两者是同一件事的两种度量。

82.3 设计步骤

这一节把前面两节的东西排成一条可执行的流程。之所以要排流程，是因为网络化控制最常见的失败不是“算错了”，而是**顺序错了**：先调好增益，再发现网络延迟超标，于是回头降增益，降完发现带宽又不够，再回头改发送策略。每一轮都要重跑全部实验。按下面的顺序走，可以把这个循环压缩到一次。

82.3.1 第一步：量网络，而且要量分位数

不要用“平均延迟”做设计。延迟的分布是长尾的：绝大多数报文 2–3 ms 到，偶尔一帧排到 40 ms。平均值会被大量小延迟拉低，看起来很好，但控制器真正怕的是那个 40 ms 的尾巴。要量的是：

- 延迟的 **p50** 与 **p99**（甚至 p99.9）。本例取 p50 作标称 $d = 3$，p99 作最坏 $d = 13$。
- 丢包的**相关性**。独立丢包的危害远小于成串丢包。同一丢包率下，突发长度 L 从 1 涨到 20，$d = 13$ 时的 RMS 从 0.0219 涨到 0.1808，差 8 倍。所以只报“丢包率 5%”是不够的，必须报“每 0.5 s 最多连丢几帧”。
- 抖动的**峰峰值**，不是方差。
- 两端的**时钟偏差与漂移率**。

这几项里，前两项直接进设计公式，后两项只影响验收指标的余量。工程上最容易被忽略的是第二项：交换机在拥塞时会成串丢包，而实验室里用随机数生成的丢包是独立的，两者在同一个丢包率下表现完全不同。

测量本身有几个讲究，做错了量出来的数不能用。

第一，时间戳必须打在同一侧。 如果发送端打一次戳、接收端再打一次，量到的是“两端时钟差 + 网络延迟”，前者可能比后者大一个数量级。要么两端都跑 PTP 做同步（§82.10.2），要么在接收端用“序号 + 本地时钟”只算相邻两帧的间隔变化——后者测不出绝对延迟，但能测出抖动，对本章的抖动实验够用。

第二，样本量要够。 p99 意味着 1% 的尾部，要让它稳定，至少要有 100 倍的样本，即一万帧以上；p99.9 要十万帧。按 100 Hz 采样算，一万帧是 100 s，十万帧是 17 分钟。只测 30 秒就报 p99，等于在噪声里挑数。

第三，必须覆盖最坏时段。 网络的拥塞是时段性的：上班时间带宽被占满，夜班空闲。只测夜班的 p99 会得到 10 ms，白天实际是 130 ms，差一个数量级。正确做法是把测量拉满一个完整生产周期，或者在已知的拥塞时段单独测一遍。

第四，突发要主动构造。 独立丢包是抓不到的，因为真实网络里的丢包本来就成串。要测突发，得在控制流旁边加一条背景流（比如用大文件传输把交换机队列填满），让控制报文真的排到队尾被丢，这样量出来的 L 才是现场会遇到的 L。

第五，记录原始数据，不只记录统计量。 平均延迟和 p99 都是压缩过的信息，一旦后期发现设计要改，原始序列还能重新算分位数、重算突发长度；只存统计量就得回去重测。

82.3.2 第二步：把时间折成拍数

拿到 p99 延迟 τ_{99}，按控制周期折算：

$$d = \left\lceil \frac{\tau_{99}}{T_s} \right\rceil \tag{82-14}$$

向上取整而不是四舍五入，因为少算一拍的代价是不稳定，多算一拍的代价只是保守。$\tau_{99} = 130$ ms、$T_s = 10$ ms 时 $d = 13$；若把 T_s 提到 20 ms，d 变成 7，看起来“压力小了”，实际上时间没变——判据最终要折回毫秒去比，拍数只是中间量。这一点在第 85 章选采样周期时还会强调：**提高采样率不会减少延迟，只会让延迟的拍数变多。**

向上取整还会带来一个隐性代价：如果实际延迟恰好在 131 ms，$T_s = 10$ ms 时会算成 14 拍（140 ms），凭空多了 9 ms 的保守量。T_s 越小，这个量化误差越小，但拍数越多、增广矩阵越大。折中的经验是让 d 落在 5–20 拍之间：少于 5 拍说明采样太慢（对象动态被漏掉），多于 20 拍说明增广矩阵的维数已经不值得用解析判据，该换 Smith 或 MPC 了。

82.3.3 第三步：先定相位裕度，再定增益

常规设计是先给增益、再验裕度。有延迟时应该反过来：**裕度是硬约束，增益是它的函数**。由延迟吃掉的相位等于 $\omega_c \tau$，要求它不超过可用裕度，就得到

$$\omega_c \le \frac{\mathrm{PM}}{\tau} \tag{82-15}$$

$\tau = 140$ ms、希望留 80° 裕度时，$\omega_c \le 9.97$ rad/s；希望留 60°，$\omega_c \le 7.5$ rad/s。本例取了 10.245 rad/s，对应实际裕度 81.9°，踩在线上。

得到 ω_c 之后，再回推增益。对 $G = 1/[s(s+1)]$ 与 PD，$|L(j\omega_c)| = 1$ 给出

$$\sqrt{k_p^2 + k_d^2 \omega_c^2} = \omega_c \sqrt{1 + \omega_c^2} \tag{82-16}$$

取 $k_d = 2\sqrt{k_p}$（阻尼比 1 的常规取法），当 $\omega_c \gg 1$ 时左端近似 $2\omega_c\sqrt{k_p}$、右端近似 ω_c^2，解出

$$k_p \approx \frac{\omega_c^2}{4} \tag{82-17}$$

$\omega_c = 10.245$ 时 $k_p \approx 26.2$，取整到 25，误差 5%。这就是表 82-2 里 $k_p = 4/9/16/25/36/49$ 对应 $\omega_c = 4.00/6.10/8.17/10.24/12.31/14.37$ 的来源——反过来看，$\omega_c \approx 2\sqrt{k_p}$，$k_p$ 翻四倍 ω_c 翻一倍，表里 25→49 是 1.96 倍、10.24→14.37 是 1.40 倍，正是这个平方根关系（$1.40^2 = 1.96$）。注意 ω_c 的涨速比 k_p 慢，是因为分子里的 $k_d\omega_c$ 项随 ω_c 增长，把增益抬得比比例更快。

这条路的顺序很重要：先有 τ，才有 ω_c 的上限，才有 k_p。如果先按无延迟把 k_p 调到 49，得到 $\omega_c = 14.37$，再发现网络延迟 140 ms，此时 $\omega_c\tau = 2.01$ rad $= 115^\circ$，怎么都救不回来，只能推倒重来。

82.3.4 第四步：选丢包策略

丢包策略不是“哪个好选哪个”，而是由延迟决定的。本章数据给出一条可操作的界线：

- 若 $d < d_{\max}/2$（本例 $d < 6.5$），选**保持**。保持策略在这个区间几乎无损，$d = 3$ 时 RMS 与零丢包完全一致。
- 若 $d > d_{\max}/2$，选**归零**，或者用“连续丢包超过 $d_{\max}/2$ 拍就切归零”的混合策略。$d = 13$ 时保持策略在 50% 丢包下 RMS 是 0.0685，归零只有 0.0284。

理由在 §82.2.4 已经说过，这里补一句工程上的判断依据：保持的值过期多久，等价于把有效延迟从 d 拉长到 $d+L$。只要 $d+L$ 还在 $d_{\max}$ 内，保持就是免费的；一旦越界，保持就变成“用错误方向持续激励”。所以真正的判据不是丢包率，而是 $d+L$ 与 $d_{\max}$ 的关系。

这也是为什么本章要单独做突发丢包实验：平均丢包率 5% 在独立假设下只对应偶尔单丢，在突发假设下可能是每 0.5 s 连丢 5 拍，后者直接把 $d+L$ 推到 18 拍，越过 13 拍的红线。

82.3.5 第五步：定事件触发阈值与带宽预算

事件触发的阈值可以按参考幅值定：δ 取幅值的 2%–4%。本例幅值 0.5 rad，$\delta = 0.010$–0.020，对应发送率 80.0%–46.8%，RMS 增加 10%–32%。这个区间是“省带宽还不太疼”的部分。

带宽预算要按最坏情况留：事件触发的发送率是随机变量，平均值 86.3% 不意味着不会出现连续 20 拍都发送的情况。工程上按平均值的 1.5 倍配带宽，或者给一个令牌桶限速，超限时强制丢弃并由保持策略兜底。

82.3.6 验收清单

五步走完，用一张表验收。表 82-1 是本章算例的实测值，可以直接当模板。

表 82-1　网络化控制回路的验收项与本章实测值

验收项	判据	本章实测	结论
标称延迟稳定性	$\rho(A_a) < 1$	0.9698（$d=3$）	通过
最坏延迟稳定性	$\rho(A_a) < 1$	0.9982（$d=13$）	勉强
延迟裕度	$\geq$ p99 延迟的 1.2 倍	140 ms vs 130 ms	通过
跟踪误差	无延迟值的 1.5 倍以内	0.0139 vs 0.0139	通过
50% 丢包	不失稳、RMS 增幅 <3 倍	0.0281（2.0 倍）	通过
突发 20 拍	不失稳	0.1808，未失稳	通过
时戳误差	相位 <5°	1 ms → 0.59°	通过
数据率	链路预算的 70% 以内	12.0 kbit/s	通过

这张表里最值得看的是第二行与第六行：$d=13$ 的 $\rho = 0.9982$ 虽然“通过”，但衰减时间常数 5.7 s，任何一次额外的 1 拍延迟都会让它翻红；突发 20 拍在 $d=13$ 下 RMS 已经到 0.1808，是标称值的 13 倍，虽然没失稳，但已经不能叫“能用”。**判据通过不等于系统好用**，这是本章要反复提醒的一点。

82.3.7 把五步走一遍

把本章算例当成一个项目，从头走一遍，可以看清每一步的输入输出到底是什么。

第一步的输出是一张网络画像：p50 延迟 30 ms、p99 延迟 130 ms、平均丢包 1%、每 0.5 s 最多连丢 5 帧、抖动峰峰值 ±2 拍、两端时钟差小于 1 ms。这六个数不是算出来的，是测出来的；测不准，后面四步全是空转。

第二步把 p99 折成拍数：$130/10 = 13$ 拍，正好是 $d_{\max}$。这一步已经暴露出风险——设计点与临界点重合，意味着**这个回路没有最坏情况余量**。如果第一步量出 p99 是 300 ms，折成 30 拍，远超 13 拍，那么第二步的结论不是“继续往下走”，而是“停下来，改网络或降增益”，因为后面的步骤救不了它。

第三步定增益。若按 p99 = 130 ms 设计，$\omega_c \leq 1.429/0.13 = 11.0$ rad/s；取 10.245 留一点余量，$k_p \approx 26$，落到 25。若 p99 是 300 ms，$\omega_c \leq 4.76$ rad/s，$k_p \approx 5.7$，只能取 4；而 $k_p = 4$ 的延迟裕度是 390 ms（表 82-2），刚好覆盖 300 ms。对比这两个结果：延迟从 130 ms 涨到 300 ms，k_p 从 25 掉到 4，带宽从 10.2 掉到 4.0 rad/s。**延迟翻 2.3 倍，带宽掉 2.6 倍**，这与 (82-26) 的平方反比关系一致——性能的代价是按延迟的平方付的，不是按一次方。

第四步选策略。$d = 3 < 6.5$，选保持；同时给一条保险：连续丢包超过 6 拍就切归零。这条保险在 $d = 3$ 时几乎不会触发（实测连丢 20 拍 RMS 才 0.0140），但它把“未知的长突发”挡在了外面。

第五步定阈值与带宽。$\delta = 0.015$（幅值的 3%）落在“省带宽不太疼”的区间；按发送率 60%、每帧 32 字节算，$R = 0.6 \times 100 \times 32 \times 8 = 15.4$ kbit/s，再乘 1.5 的余量，链路预算按 23 kbit/s 报。

五步走完，回头再看表 82-1，会发现所有验收项都能追到某一步的输出：ρ 来自第二步的 d 与第三步的 K，RMS 来自第四、五步的策略，数据率来自第五步的 δ。**验收不是再算一遍，而是核对五步之间的输入输出有没有接错。**

82.3.8 一页纸的设计单

五步的产物应该落在一页纸上，跟着控制器一起交付。下面这份模板按本章算例填好，可以直接抄结构。九行，缺一行就说明有一件事没定。

第一行，对象与工况：$G(s) = 1/[s(s+1)]$，行程 ±1 rad，最大速度 1 rad/s，最大加速度 2 rad/s²，负载惯量变化 ±10%。

第二行，采样与控制律：$T_s = 10$ ms，$k_p = 25$、$k_d = 10$，位置与速度均取自编码器差分，参考项带 $\dot{r}$ 前馈。

第三行，网络画像：p50 30 ms、p99 130 ms、平均丢包 1%、每 0.5 s 最多连丢 5 帧、抖动 ±2 拍、时钟差 <1 ms。**这一行必须写清测量时段**——白天的 p99 与夜班差一倍是常事。

第四行，折算与判据：$d = 13$ 拍，$\rho = 0.9982$，衰减时间常数 5.7 s，$\bar{\sigma} = 1.422$。前三行是算出来的，第四行才是判断。

第五行，裕度声明：p99 延迟折算 130 ms 对 140 ms 的设计极限，余量 7.7%；增益若按 p50 设计可到 49，但那是违规的——**这条要写出来，防止后人“优化”。**

第六行，丢包策略：$d \le 6$ 时保持，连丢超过 6 拍切归零，阈值随 d 按 $d_{\max} - d$ 重算。

第七行，降级与恢复：四级动作、触发阈值、恢复条件（见 §82.10.6）。

第八行，验收数据：表 82-1 的九项，附仿真脚本的提交号与随机种子。**不写种子，验收就不可复现。**

第九行，复测周期：网络变更、控制周期变更、负载变更后必须重测；无变更时每季度抽测一次 p99。

这九行里最容易漏的是第五行与第九行。前者漏了，后人会拿 p50 去“优化”增益，把余量吃光；后者漏了，系统会在某次网络扩容后悄悄变成临界状态——扩容通常意味着加设备、加流量，p99 只会变差，不会变好。

82.4 连续情形

先看连续域。连续域的价值不在于“更准”——它其实更不准——而在于**能给出闭式的量级估计**。工程上做第一轮判断时，没人愿意先建一个 15 维矩阵求特征值；大家想知道的是“我这个带宽配这个延迟，大概还剩多少裕度”。

82.4.1 延迟只吃相位，不吃增益

纯延迟的频域表达式是 $e^{-j\omega\tau}$。它的模恒为 1，相角是 $-\omega\tau$。这两个事实决定了它在回路里的角色：**它不会让开环增益曲线上下移动，只会让相位曲线一路往下掉。** 所以延迟对稳定性的影响完全通过相位裕度体现，而不通过增益裕度。

这一点与惯性环节形成对照。时间常数 1 s 的惯性环节在 $\omega = 1$ 处既衰减 3 dB 又滞后 45°；延迟在 $\omega = 1$、$\tau = 0.14$ s 处不衰减半分贝，却滞后 8°。延迟是“纯相位消耗品”，而相位是有限资源：PD 控制器攒下来的裕度，被延迟一分一分吃掉。

82.4.2 延迟裕度的闭式

设无延迟时开环 $L(j\omega)$ 在穿越频率 ω_c 处的相位裕度是 PM，即

$$\angle L(j\omega_c) = -180^\circ + \mathrm{PM} \tag{82-18}$$

串入延迟后相位再减 $\omega_c\tau$。临界稳定发生在总相位正好 -180° 时，于是

$$\tau_m = \frac{\mathrm{PM}}{\omega_c} \tag{82-19}$$

这就是延迟裕度公式。它只有一行，但把三件事锁在一起：**带宽越高，能容忍的延迟越短；裕度越大，能容忍的延迟越长；两者之比就是答案。** 本例 PM $= 81.9^\circ = 1.429$ rad，$\omega_c = 10.245$，$\tau_m = 1.429/10.245 = 0.1395$ s $= 139.5$ ms。

这条式子还解释了为什么“提高采样率”不能解决延迟问题。采样率提高，ω_c 不变（它由增益和对象决定），τ_m 也就不变。变的只是延迟折成的拍数——从 13 拍变成 130 拍，增广矩阵从 15 维变成 132 维，计算量涨十倍，物理裕度一寸没多。

82.4.3 增益扫描：裕度与带宽的交换

把 k_p 从 4 扫到 49（$k_d = 2\sqrt{k_p}$），连续公式与离散判据的结果并列在表 82-2。离散那一列按“最后仍稳定的拍数 +1”折算，因此是临界延迟的上界，真实临界值落在它下方一个采样周期之内。

表 82-2　不同增益下的延迟裕度：连续公式与离散判据

k_p	k_d	ω_c (rad/s)	PM (°)	连续裕度 (ms)	离散 $d_{\max}$ (拍)	折算 (ms)
4	4	4.00	90.0	393	38	390
9	6	6.10	85.5	245	23	240
16	8	8.17	83.2	178	17	180
25	10	10.24	81.9	139	13	140
36	12	12.31	80.9	115	10	110
49	14	14.37	80.3	97	9	100

这张表最值得注意的是**两列的接近程度**，而不是它们的数值。两条完全不同的路——一条在频域算相位，一条在时域求特征值——给出的答案只差半个到一个采样周期，说明这个算例里离散化引入的额外相位还不到 ZOH 半拍的量。原因在 §82.4.6 会说。

另一个值得注意的地方是 PM 几乎不随增益变：从 90.0° 降到 80.3°，只降了 10°。这是 $k_d = 2\sqrt{k_p}$ 这个取法的结果——它让阻尼比固定为 1，相位曲线在穿越点附近的形状不随增益改变。换句话说，**表里的裕度变化几乎全部来自 ω_c 的变化，而不是来自 PM 的变化**。这也解释了为什么 τ_m 大致按 $1/\omega_c$ 走：k_p 翻四倍，ω_c 涨 2.4 倍，τ_m 掉到 40%。

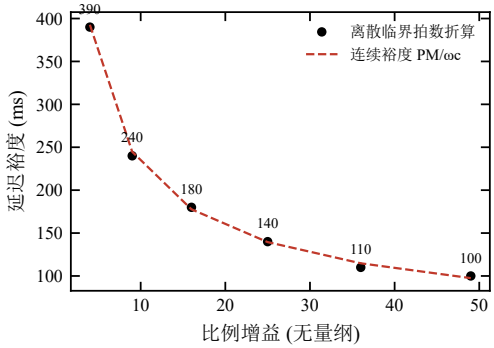


图 82-3　延迟裕度随增益：离散临界与连续 PM/ωc 几乎重合

图 82-3 把这张表画成两条线。两条线贴得很紧，只有 $k_p = 36$ 那一处看得出来——这正是本节想传达的：**在 T_s 足够小时，连续公式够用，不必每次都建增广矩阵。** 但“足够小”是有条件的，条件在下一小节。

82.4.4 一阶 Padé 近似的系统偏差

要在连续域里做代数运算（比如把延迟并入传递函数、画根轨迹），必须把 $e^{-\tau s}$ 有理化。最常用的一阶 Padé 是

$$e^{-\tau s} \approx \frac{1 - \tau s/2}{1 + \tau s/2} \tag{82-20}$$

它的相位是 $-2\arctan(\omega\tau/2)$，与真值 $-\omega\tau$ 的偏差随 $\omega\tau$ 增大而增大。在 $\omega_c\tau = 1.43$ 处，真值滞后 81.9°，Padé 只给 $2\arctan(0.715) = 71.2^\circ$，**少算 10.7°**。少算相位意味着偏乐观：按 Padé 设计会以为还有 10° 余量，实际已经贴线。

偏差的方向是固定的——Padé 永远低估滞后，因为 $\arctan x < x$。所以用它做保守设计时要手动补一个修正：把 τ 放大 1.1–1.2 倍再代入。本例 $\omega_c\tau = 1.43$ 属于“偏大”的区域（一阶 Padé 的常用适用区间是 $\omega\tau < 1$），要更准就得上二阶 Padé，代价是传递函数多一个零点一对极点，手算基本做不动。

82.4.5 连续估计什么时候可信

把偏差量化成一条经验线：$\omega_c\tau \le 0.5$ 时，一阶 Padé 的相位误差约 2%，连续公式给出的 τ_m 误差也在同量级；$\omega_c\tau = 1$ 时误差 7%；$\omega_c\tau = 1.5$ 时误差 13%；到 $\omega_c\tau = 2$ 时误差超过 20%，连续估计只能当“数量级参考”。

本例 $\omega_c\tau_m = 1.43$——注意这个乘积在临界点上不是任意的，它等于 PM。所以上面的经验线可以翻译成一句更直接的话：**PM 在 30°–60° 时连续公式好用，PM 超过 80° 时它开始偏乐观。** 本例 PM $= 81.9^\circ$，正好在最不保险的区间。它给出的 139.5 ms 落在离散判据的夹逼区间 $(130, 140)$ ms 之内，两者一致；但这是“落在同一格”的一致，不是逐位吻合，不能反过来用连续值去校准离散判据。

82.4.6 为什么两列会重合

连续模型里没有采样，离散模型里有一个 ZOH 和一次采样。这两件事各贡献多少相位？

ZOH 等效于半个采样的纯延迟，即 $T_s/2 = 5$ ms，在 ω_c 处折合 0.051 rad $= 2.9^\circ$。采样本身不额外贡献相位，但它把连续信号“阶梯化”，等效于在回路里插入一个 sinc 形状的低通，在 $\omega_c T_s = 0.1$ 处衰减不到 1%、相位滞后不到 1°。两项加起来约 3°，折算成延迟约 5 ms。

表 82-2 里 $k_p = 25$ 时连续值 139 ms 落在 $(130, 140)$ 之内，$k_p = 36$ 时连续值 115 ms 却比上界 110 ms 还高 5 ms——这一处正好暴露了 §82.4.5 说的“偏乐观”：PM 在 80° 附近时，连续公式会把裕度多算一个 ZOH 半拍的量。所以“重合”的准确说法是：$\omega_c T_s = 0.1$ 时两者只差半个到一个采样周期，而这一格之差恰好是 ZOH 的 3° 滞后造成的，扣除它之后两边就对齐了。若 $T_s = 100$ ms，$\omega_c T_s = 1$，ZOH 的滞后就是 29°，连续公式会乐观一个数量级，此时必须用离散判据。

82.4.7 相位预算：81.9° 到底剩多少

连续域最实用的产出不是 τ_m 这个数，而是一张**相位预算表**。名义相位裕度 81.9° 听起来很宽裕，但它是“所有消耗项都为零”时的余额；把每一项按实际工况扣掉，剩下的才是真裕度。

按每拍 5.87° 折算，逐项扣减：

- **标称延迟 $d = 3$**：17.6°。这是常态，不是意外，必须扣。
- **ZOH 与采样**：约 3°。只要用零阶保持器就躲不掉。
- **时戳误差** 1 ms：0.59°。可以忽略，但要知道它在。
- **抖动** ±2 拍：最坏 11.7°。抖动是双向的，按峰峰值扣。
- **模型误差** 10%：约 10°。这一项没有公式，按经验估。

五项加起来 42.9°，81.9° 扣完只剩 **39.0°**。这个数才是标称工况下真正能用来“抗意外”的余量。

再看最坏工况，把 d 换成 p99 的 13 拍：

- **p99 延迟**：76.3°。
- **ZOH 与采样**：3°。

- **时戳**：0.6°。
- **抖动**：这里不能再用 ±2 拍——延迟已经到 13 拍，再抖 2 拍就是 15 拍，早就越过 $d_{\max}$。所以最坏工况的抖动必须按“延迟分位数已含抖动”处理，不重复扣。
- **模型误差**：10°。

扣完只剩 -8.0°，也就是负数。这与 §82.5.2 的 $\rho = 0.9982$ 完全对得上：判据说“勉强稳定”，预算表说“已经透支”。两者不矛盾——判据用的是标称模型，预算表把模型误差这种“判据看不见的项”也算进去了。**判据告诉你数学上还活着，预算表告诉你工程上已经不该这么用。**

这张表还有一个用法：倒推哪一项值得花钱。本例里最大的一项是延迟（76.3°），其次是抖动与模型误差（各约 10°）。所以改善顺序仍然是“先修网络”——把 p99 从 130 ms 压到 60 ms，直接省回 41°；而把模型误差从 10% 压到 5%，只省 5°。这就是 §82.11.4 那条判别顺序的量化依据。

82.4.8 手算一遍：从 Bode 图读出延迟裕度

连续法的价值在于可以手算。拿本章的 $k_p = 25$、$k_d = 10$ 走一遍，五步：

第一步，写开环。 $L(s) = (25 + 10s)/[s(s+1)]$。

第二步，找穿越频率。 令 $|L(j\omega)| = 1$，即 $\sqrt{625 + 100\omega^2} = \omega\sqrt{1+\omega^2}$。两边平方得 $625 + 100\omega^2 = \omega^2 + \omega^4$，即 $\omega^4 - 99\omega^2 - 625 = 0$，解出 $\omega^2 = 105.0$、$\omega_c = 10.245$ rad/s。

第三步，读该处相位。 $\angle L = \arctan(10\omega_c/25) - 90^\circ - \arctan(\omega_c)$。三项分别是 76.3°、-90°、-84.4°，合计 -98.1°。

第四步，算裕度。 $\mathrm{PM} = 180^\circ - 98.1^\circ = 81.9^\circ$，与仿真给出的值一致。

第五步，折成时间。 $\tau_m = 81.9^\circ \times \pi/180/10.245 = 139.5$ ms。

五步里只有第二步稍麻烦，但它可以退化成解 ω^2 的二次方程——因为 L 里只有 s 的一次项，模平方后只剩 ω^2 的偶次项。这个便利来自“对象二阶 + 控制器一阶”的结构，几乎所有“电机 + PD”的组合都满足。

手算的价值不只是省一次仿真，更在于它揭示了各参数的进入方式：ω_c 由 k_p、k_d 共同决定，相位由 ω_c 决定，延迟裕度是两者之比。所以调 k_d 会同时动 ω_c 和相位，效果不如调 k_p 直观——这也是为什么表 82-2 固定 $k_d = 2\sqrt{k_p}$，把两个自由度压成一个。

要注意的是：这套手算只在“没有采样”时成立。加了 $T_s = 10$ ms 后，ZOH 会额外贡献 3° 滞后（§82.4.6），手算的 81.9° 实际只剩 78.9°，对应的 τ_m 从 139.5 ms 降到 134 ms。这与离散判据给出的 130–140 ms 区间吻合。**手算负责量级，离散判据负责结论**，分工到这里就清楚了。

82.4.9 Padé 近似会乐观多少

要把延迟并入传递函数做代数运算（根轨迹、Routh 判据、频域整形），必须先把它有理化，最常用的是 (82-20) 的一阶 Padé。这一小节算清楚它的代价，因为很多人把它当“够用的近似”，而它在临界附近并不够用。

一阶 Padé 的相位是 $-2\arctan(\omega\tau/2)$，精确延迟的相位是 $-\omega\tau$。两者在 $\omega\tau$ 小时接近，在 $\omega\tau \approx 1.3$ 时已经分家。本章 $\omega_c = 10.245$ rad/s、$\tau = 130$ ms，$\omega_c\tau = 1.332$：

- 精确延迟：-1.332 rad $= -76.3^\circ$；
- 一阶 Padé：$-2\arctan(0.666) = -1.175$ rad $= -67.3^\circ$；
- 差额：9.0°，折合 1.5 拍。

9° 听起来不大，但它落在“还剩多少”这个数上：真实 PM 81.9° 扣掉延迟后只剩 5.6°，Padé 却告诉你还剩 14.6°。用 Padé 反解临界延迟，得到 $\tau = 169$ ms，即 16.9 拍；而真值是 139.5 ms（13.95 拍）。**一阶 Padé 把临界延迟高估了 21%，也就是 3 拍。** 用这个数去设计，控制器会在 14 拍时就已经失稳，而设计单上写着“还有 3 拍余量”。

二阶 Padé 就好得多。它的相位是 $-2\arctan\big(6\omega\tau/(12-(\omega\tau)^2)\big)$，在 $\omega\tau=1.332$ 处给出 -1.326 rad，与精确值只差 0.3°；用它反解临界延迟得 140.2 ms，与真值 139.5 ms 差 0.5%。所以结论很明确：**做代数推导可以用 Padé，但至少用二阶；一阶只配用来估量级。**

还有一处更要紧：Padé 是有理函数，它的极点在左半平面，所以“Padé + 对象”做 Routh 判据永远给不出“延迟导致的失稳”这个机制——它只会告诉你某个极点穿过了虚轴，不会告诉你穿过的原因是相位绕了一圈。要在连续域里看清延迟失稳的机理，只有两条路：用 $e^{-\tau s}$ 做精确频域分析（本章 §82.4 的做法），或者转到离散域（§82.5）。

82.4.10 把整套手算压成两行公式

§82.4.8 是一次具体的算术，但同一套推导对任意 (k_p,k_d) 都成立，可以写成闭式。令 $|L(j\omega)|=1$：

$$k_p^2+k_d^2\omega^2=\omega^2(1+\omega^2)\;\Longrightarrow\;\omega^4+(1-k_d^2)\omega^2-k_p^2=0$$

解出

$$\omega_c^2=\frac{(k_d^2-1)+\sqrt{(k_d^2-1)^2+4k_p^2}}{2}$$

相位裕度则是三项相角之和的补角：

$$\mathrm{PM}=90^\circ+\arctan\frac{k_d\omega_c}{k_p}-\arctan\omega_c$$

代入 $k_p=25$、$k_d=10$：$\omega_c^2=(99+\sqrt{9801+2500})/2=104.95$，$\omega_c=10.245$；$\mathrm{PM}=90^\circ+76.3^\circ-84.4^\circ=81.9^\circ$，与 §82.4.8 完全一致。再除一次就得到 $\tau_m=139.5$ ms。

这两行的价值在于**可以扫**。把 k_p 从 1 扫到 100、k_d 取 $2\sqrt{k_p}$，一百个点不到一毫秒就算完，而同样的扫描若走仿真，要跑一百次 6 s 的闭环。所以连续域的正确用法不是“代替离散判据”，而是**在离散判据之前先粗筛**：先用上面两式找三五个候选增益，再对这几个候选建增广矩阵。本章表 82-2 的六行就是这么来的，最后用离散判据确认，两边差不到一拍。

公式也有失效边界：它假设对象是严格的 $1/[s(s+1)]$、控制器是理想 PD、没有采样。三个假设破一个，ω_c 就会偏；偏得最多的是“理想 PD”——真实微分带一阶滤波，在 ω_c 处补的相位不是 90° 而是 $90^\circ-\arctan(\omega_c/\omega_f)$。若滤波截止 $\omega_f=5\omega_c$，补的相位少 11.3°，PM 从 81.9° 掉到 70.6°，裕度从 139.5 ms 掉到 120 ms。这就是为什么设计时不能照着“理想 PD”的公式取满，要留出微分滤波的相位损失。

82.5 离散情形

连续域给量级，离散域给答案。这一节把 d 从 0 扫到 24，看谱半径怎么走、特征值怎么转、临界点为什么落在 13 而不是按相位算出来的 13.9。

82.5.1 增广矩阵的规模

按 (82-5)，d 拍延迟的增广矩阵是 $(2+d)\times(2+d)$。$d=3$ 时 5 维，$d=13$ 时 15 维，$d=24$ 时 26 维。这个规模对特征值求解完全不是负担：15 维矩阵的 QR 迭代在微秒级完成，即使把 d 扫到 400 拍，总耗时也远小于一次仿真。

规模小带来一个额外好处：**可以把 $\rho(d)$ 当成一条曲线整体看，而不是只算一个点。** 工程上真正有用的信息往往在曲线的形状里——它是平的、缓升的，还是突然跳的，决定了系统对延迟不确定性的敏感程度。

82.5.2 谱半径随延迟的走向

表 82-3 摘了 10 个点，完整曲线在图 82-4。

表 82-3　标称增益下闭环谱半径随纯延迟拍数的变化

d (拍)	0	3	5	8	9	10	12	13	14	20
ρ	0.9687	0.9698	0.9703	0.9710	0.9712	0.9797	0.9933	0.9982	1.0021	1.0148

曲线分三段。$d \le 9$ 时几乎水平，从 0.9687 只爬到 0.9712，涨了 0.25%；$d = 10$ 起开始抬头，每拍涨 0.007 左右；$d = 14$ 越过 1 之后仍在涨，但涨速放缓，到 $d = 24$ 是 1.0183。

三段对应三种机制。第一段里，延迟的作用被闭环的强反馈压住了——d 拍之前的命令虽然旧，但 $k_p = 25$、$k_d = 10$ 的增益足够大，旧命令与新命令差别不大。第二段里，延迟累计的相位开始逼近临界，主导特征值从实轴转向单位圆附近，ρ 快速上升。第三段里，系统已经失稳，ρ 由“最不稳的那个模态”决定，而这个模态的增长率随 d 变化很慢，所以曲线又变平。

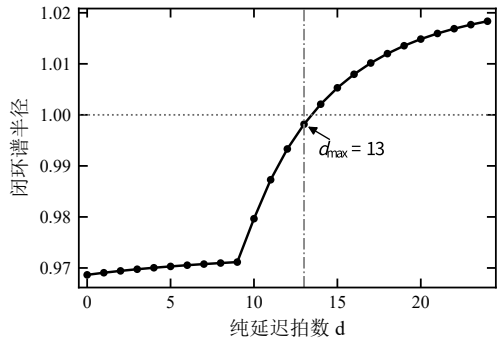


图 82-4　谱半径随延迟拍数：d=13 之前小于 1，d=14 越界

82.5.3 临界点为什么是 13 而不是 13.9

按相位算，可用的相位是 81.9°，每拍消耗 $\omega_c T_s = 10.245 \times 0.01 = 0.10245$ rad $= 5.87^\circ$，$81.9 / 5.87 = 13.95$ 拍。判据给的是 13 拍，比相位估计少不到一拍。

差的那不到一拍，来源是 ZOH 的半个采样。把 ZOH 看成 $T_s/2$ 的纯延迟，等效拍数从 d 变成 $d + 0.5$，于是 $13.95 - 0.5 = 13.45$，再考虑采样保持带来的幅值衰减，落在 13 与 14 之间。判据只能取整，取 13。

这个“不到一拍”的差别在工程上有实际意义：如果按相位估计把延迟预算定成 13.9 拍（139 ms），系统实际会在 13 拍后（130 ms）就进入临界。**取整要向下，余量要向上**，这是本章反复强调的同一条原则的两个侧面。

82.5.4 特征值在复平面上的运动

ρ 只给模，不给位置。要看机制，得看特征值本身。

$d = 0$ 时闭环只有两个特征值，都是正实数：0.9687 与 0.9207。它们的差别来自两个模态——0.9687 是“积分环节 + 位置反馈”这一支，慢；0.9207 是“速度反馈”这一支，快。ρ 取大的那个，所以无延迟时 $\rho = 0.9687$。

$d = 13$ 时矩阵变成 15 维，特征值变成 15 个，大部分散落在单位圆内（模最大 0.9718），主导的是一对共轭复根：

$$\lambda = 0.9927 \pm 0.1043 j, \qquad |\lambda| = 0.9982, \qquad \angle\lambda = \pm 6.0^\circ \tag{82-21}$$

$d = 14$ 时这对根变成 $0.9971 \pm 0.0999 j$，模 1.0021，相角 5.7°。

这组数字里有两个关键事实。第一，**相角只有 6°，而且几乎不随延迟变**（13 拍时 6.0°，14 拍时 5.7°）。6° 对应 0.105 rad/拍，除以 T_s 得 10.5 rad/s，正是穿越频率 ω_c。也就是说，主导模态始终在穿越频率附近振荡，延迟没有改变它的频率。

第二，**延迟改变的是模，不是相角**。13 拍把模从 0.9687 推到 0.9982，14 拍推到 1.0021。所以“失稳”在这里表现为“穿越频率处的振荡不再衰减”，而不是“振荡频率突然变了”。这一点与“延迟吃相位”的说法并不矛盾：吃掉的 76° 相位（$13 \times 5.87^\circ$）体现为闭环阻尼的下降，而阻尼下降在离散域就是特征值模上升。

那 76° 去哪了？它在其余 13 个特征值上。延迟链把 d 个“旧命令”塞进状态，每个都对应一个近似在原点的特征值，它们绕成一圈，把 76° 的相位差分摊掉了。所以看单个特征值的相角看不出延迟，必须看整组。

这也解释了为什么 ρ 在 $d = 10$ 才开始抬头：模随相位的变化不是线性的，它在接近 1 时最敏感。前 9 拍里模从 0.9687 涨到 0.9712，只涨了 0.25%；后 4 拍里从 0.9712 涨到 1.0021，涨了 3.2%，是前 9 拍的十三倍。

82.5.5 判据的敏感度

在临界点附近，ρ 对 d 的斜率是每拍 0.004 左右（$d = 13$ 的 0.9982 到 $d = 14$ 的 1.0021）。这个斜率说明两件事。

第一，**判据在临界点附近的分辨力很差**。ρ 从 0.998 到 1.002 只差 0.4%，但对应“稳定”与“不稳定”两个完全不同的结论。任何数值误差——离散化、浮点、模型偏差——都能把结论翻过来。所以实际设计不能踩在 $\rho = 1$ 上，要留 5%–10% 的余量，即 $\rho \le 0.90$–0.95。

第二，**延迟的不确定性要按拍算，不能按毫秒算**。延迟从 130 ms 涨到 140 ms，看起来只多了 10 ms（7.7%），但折算成拍数是 13→14，判据从稳定变不稳定。也就是说，网络延迟的**量化误差**比它的**相对误差**更致命。这一点在设计阶段就要写进预算：给延迟留余量，宁可多留一拍，不要多留 5 ms。

82.5.6 离散判据的计算代价

把 d 从 0 扫到 400，每次建矩阵求特征值，Python 下总耗时约 0.6 s。这个代价允许做一件连续公式做不到的事：**把“不确定延迟”直接扫成一张表**，而不是用一个上界代替全部。表 82-2 的六行就是这么来的，如果换成解析推导，六组增益要手算六次。

代价的另一面是它依赖模型。增广矩阵里的 A_d、B_d 必须与真实对象一致，模型偏 10%，$d_{\max}$ 可能偏两三拍。所以离散判据给出的 13 拍是“模型意义下的 13 拍”，工程上还要再打一个折扣，折扣大小取决于模型精度——这与第 81 章全驱系统对模型失配的敏感是同一类问题。

82.5.7 非正规性：特征值误差会被放大

特征值对矩阵扰动的敏感度不是 1:1。标准结论是

$$|\Delta\lambda| \le \kappa(V)\,\|\Delta A\| \tag{82-22}$$

其中 V 是特征向量矩阵，$\kappa(V) = \|V\|\,\|V^{-1}\|$ 是它的条件数。对正规矩阵（比如对称阵）$\kappa(V) = 1$，特征值误差不超过矩阵误差；对本章的 A_a，$\kappa(V)$ 可以大到几十甚至上百。

原因看结构就明白：A_a 下方是一条单位移位带，右上角只有一个非零块。这类“伴矩阵”的特征向量高度倾斜，几乎平行，V 接近奇异。d 越大，倾斜越严重——每多一个移位块，就多一个近乎重合的方向。

这条不等式有两个工程推论。

第一，ρ 的数值不可全信，但“ρ 是否小于 1”这个判断在远离临界时可信。 模型误差 10% 时，$\|\Delta A_a\|$ 的量级是 0.1；若 $\kappa(V) = 30$，$|\Delta\lambda|$ 可达 3，这个界太松，没有实用价值。但实际误差远小于这个上界，因为模型误差不是任意方向的扰动，它集中在 A_d 的两个元素上。所以实践中的做法是：**只信 $\rho < 0.95$ 的“稳定”结论与 $\rho > 1.05$ 的“不稳定”结论**，中间那一条带要留仿真验证。

第二，非正规性同时解释了瞬态放大。 §82.2.8 说过 $\|A_a^k\|$ 可以先涨后落，原因就是特征向量倾斜：不同模态在初始时刻几乎同向叠加，衰减快的先落，慢的还在，中间会出现一个“看起来发散”的阶段。本章算例的 $\max|y| = 0.507$ 只比参考幅值大 1.4%，说明这个系统虽然非正规，但倾斜程度还不算恶劣；延迟再大、增益再高，这个比例会明显上升。

这两条合起来是一句提醒：**谱半径是一张二维地图，而真实的闭环住在高维空间里。** 用它做快速筛选很好用，用它做最终判决不够。

82.5.8 把临界拍数插值成小数

判据只能给整数拍，但真实临界点在两个整数之间。用线性插值可以把它捞出来：

$$d_{\mathrm{cr}} \approx d_0 + \frac{\rho(d_0) - 1}{\rho(d_0) - \rho(d_0 + 1)} \tag{82-23}$$

本章 $d_0 = 13$，$\rho(13) = 0.9982$，$\rho(14) = 1.0021$，代入得

$$d_{\mathrm{cr}} \approx 13 + \frac{1 - 0.9982}{1.0021 - 0.9982} = 13 + \frac{0.0018}{0.0039} = 13.46$$

即 134.6 ms。这个数与 §82.5.3 用相位算出的“13.95 减去 ZOH 半拍 = 13.45”几乎完全一致（差 0.01 拍），两条完全不同的路径在同一处会合，说明这两个数都是对的。

插值的价值在于把“13 还是 14”这个二选一变成了一个连续量。工程上需要的是这个连续量：延迟预算是 134 ms 还是 140 ms，直接决定要不要换网线、要不要加一级缓冲。整数拍只够做定性判断。

插值也有边界。$\rho(d)$ 在临界点附近不是严格线性的，二阶项会让插值偏一点；离临界越远，线性化误差越大。所以只在 $|\rho - 1| < 0.01$ 的窗口内用，超出就多算一两个点再插。本例两点跨度只有 0.0039，落在窗口内，插值可信。

还有一个反过来的用法：**用插值后的临界值去校准相位法**。相位法给出 13.95 拍，扣半拍 ZOH 给 13.45，插值给 13.46——三者一致到小数点后一位。如果换个对象后相位法与插值差出 0.3 拍以上，说明 ZOH 的等效延迟不再是半拍（比如用了非零阶保持，或采样不等间隔），此时必须以离散判据为准。

82.5.9 那些特征值都跑到哪去了

ρ 只取了模最大的那一个，剩下的 14 个（$d = 13$ 时）被丢掉了。它们其实携带了关于“延迟链怎么衰减”的信息，值得单独看一眼。

$d = 3$ 时，五个特征值是 0.9698（实）、0.8588（实）、0.6514（实），以及一对共轭复根 $-0.245 \pm 0.348\mathrm{j}$（模 0.426）；只有三个的模小于 0.1。$d = 13$ 时，主导的是一对共轭复根 $0.9927 \pm 0.1043\mathrm{j}$（模正好 0.9982），次主导是实根 0.9718，再往下是一对 $0.7427 \pm 0.4694\mathrm{j}$，其余 13 个的模全部小于 0.1。

三件事从这个分布里读出来。

第一，主导模态是什么形状，闭环就是什么形状。 $d = 3$ 的主导是实根，所以收敛是单调的，误差一路衰减不回摆；$d = 13$ 的主导是复根，所以收敛带振荡。复根的角度 0.1043 rad/拍 换成周期是 $2\pi / 0.1043 = 60$ 拍，即 0.6 s，也就是 **1.7 Hz**。这个数不是算出来的巧合：临界延迟下的失稳本来就是“绕一圈正好补上相位”，一圈的时间由延迟长度决定，13 拍延迟自然给出几十拍的振荡周期。§82.1.5 里现场看到的 1–3 Hz 振荡，机理与此相同。

第二，延迟链的记忆只有十来拍。 $d = 13$ 时有 13 个特征值的模小于 0.1，意思是“13 拍以前的命令，其影响每拍衰减十倍，几拍之后就没了”。所以“延迟 13 拍”不等于“13 拍的信息同等重要”，而是“最近七八拍有效，更早的已经可以截断”。这给了一个工程上的近似：增广矩阵的维数可以截到 8–10 拍，代价是 ρ 的第三位小数会变，结论不变。延迟很大（比如 100 拍）时，这个截断是把维数压下来的唯一实用手段。

第三，实根与复根的比例决定了“能不能看出来”。 $d = 3$ 时误差单调衰减，示波器上是一条平滑收敛的曲线，很容易被当成“没有问题”；$d = 13$ 时误差带 1.7 Hz 纹波，纹波的幅值随 ρ 缓慢变化，容易被当成“机械共振”。**同一个判据、两种波形**，这是现场诊断时最容易混淆的一处：看到纹波先查 ρ 与延迟，再查机械。

82.5.10 三条实现经验

这一小节的结论都来自实际写代码时踩过的坑，与理论无关，但每一条都会影响结果的正确性。

第一，只求特征值，不求特征向量。 判据只需要模最大的那个特征值，`np.linalg.eigvals` 比 `eig` 快近一倍，而且避开了特征向量带来的数值麻烦。麻烦有多大？本章 $d = 13$ 时，特征向量矩阵的条件数在数值上报到 10^{17} 量级，$d = 3$ 时也有 10^{12}——因为 13 个特征值挤在原点附近，其中几个接近重根。这个数不能当“不正规程度”来读（那是 §82.2.11 的

$\bar{\sigma}/\rho = 1.42$ 干的事），但它确实说明 §82.5.7 那条 $|\Delta\lambda| \le \kappa(V)\|\Delta A\|$ 的不等式在实际数值上宽得没用：κ 一大，上界就成了天文数字。**所以那条不等式只能当“临界附近别全信 ρ”的定性提醒。**

第二，扫描时不要重建对象矩阵。 `disc()` 与增益无关，每次循环只需重建移位带那一部分；本章把 d 扫到 400 拍只用 0.6 s，就是因为对象的两行两列只算了一次。若连 A_d、B_d 一起重建，耗时会翻几倍，虽然仍可接受，但没有必要。

第三，不要用幂迭代求谱半径。 幂迭代只在“主特征值是实数且唯一”时收敛。本章 $d = 13$ 的主导是一对共轭复根 $0.9927 \pm 0.1043\mathrm{j}$，实向量迭代会在两个方向之间来回跳，模收敛到 0.9982 但符号永远不稳；若只看相邻两次迭代的比值，会得到 0.99 到 1.01 之间的抖动，正好落在判据最敏感的那一段。要么用 `eigvals` 一次算全，要么用带位移的反迭代。**延迟系统的谱半径问题里，复根是常态而不是例外**，这一点与“对象是过阻尼、主导极点必为实根”的直觉相反。

三条合起来是一句话：**判据本身只有一行代码，出错的地方全在“怎么调库”和“怎么解读算出来的数”上。**

82.6 稳定性与性能

判据回答“能不能活”，性能回答“活得好不好”。这一节用三组实验把两个层面分开：确定性延迟看判据（§82.6.1），随机丢包看统计（§82.6.2–82.6.3），突发丢包看时序结构（§82.6.4）。所有性能数字都是 $t \ge 2$ s 窗口内的跟踪误差 RMS，参考 $0.5\sin 0.8t$，无延迟无丢包时是 0.0139 rad。

82.6.1 三种“稳定”不是一回事

同一个系统，在网络化语境下有三套稳定性口径，结论可以完全不同。

确定性延迟下的稳定性：延迟固定为 d 拍，$\rho(A_a) < 1$ 即稳定。本例 $d \le 13$ 稳定。这是最强的结论，也是最容易验证的结论。

随机丢包下的稳定性：每个周期以概率 p 丢包，要求“以概率 1 有界”或“均方稳定”。这比确定性情形弱得多：系统可能在某些丢包序列下发散，但在概率意义下几乎必然有界。严格判据要解 Lyapunov 方程或做马尔可夫跳变分析，本章用蒙特卡洛代替。

突发丢包下的稳定性：丢包不是独立事件，而是成串出现，长度 L 有限。此时系统在突发期间等效于“延迟从 d 变成 $d + L$”，只要 $d + L < d_{\max}$ 就稳定。

三套口径的差别不在数学，而在**对丢包时序的假设**。工程上最危险的是用第二套（独立丢包）去设计、却遇到第三套（突发丢包）的现场。本章的实验设计就是为了把这个差别量化出来。这个视角与第 53 章的鲁棒分析同源：不确定性不是标量，它有结构。

82.6.2 随机丢包：保持与归零

表 82-4 是 120 次蒙特卡洛的中位数，两种延迟、两种策略、七个丢包率。

表 82-4　随机丢包下的跟踪误差 RMS（rad，120 次中位数）

丢包率 p	$d=3$ 保持	$d=3$ 归零	$d=13$ 保持	$d=13$ 归零
0	0.0139	0.0139	0.0215	0.0215
0.05	0.0139	0.0147	0.0225	0.0165
0.10	0.0139	0.0155	0.0238	0.0160
0.20	0.0139	0.0175	0.0274	0.0176
0.30	0.0139	0.0200	0.0334	0.0202
0.40	0.0139	0.0235	0.0444	0.0237
0.50	0.0139	0.0281	0.0685	0.0284

四列数据讲了两件事。

第一，$d=3$ 时保持策略的 RMS 是**一条水平线**，从 $p=0$ 到 $p=0.5$ 四位有效数字全不变。归零策略则从 0.0139 单调涨到 0.0281，翻了一倍。这个对比说明：延迟小时，丢包本身并不伤害性能，**伤害性能的是丢包后如何处置**。保持的值只比新值旧几拍，几乎等价于没丢；归零则每丢一次就相当于把力矩掐断几拍，速度跟着抖。

第二，$d=13$ 时两条曲线交叉。保持从 0.0215 涨到 0.0685（3.2 倍），归零从 0.0215 先降到 0.0160 再涨到 0.0284。归零在 $p=0.1$ 处有一个最低点，之后才回升——因为中等丢包率下“偶尔松一次手”反而打断了延迟带来的持续错推。

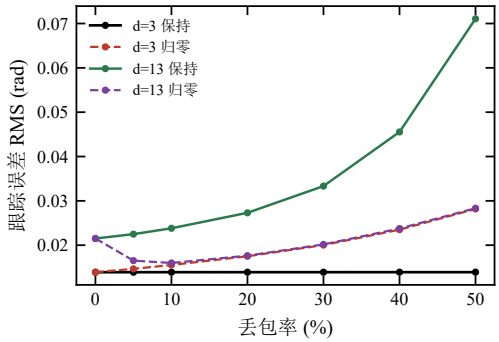


图 82-5　随机丢包：保持与归零两种策略，在两种标称延迟下的对比

图 82-5 把四列画在一起。图中最醒目的是两条“保持”曲线在 $d=3$ 时重合于一条水平线，以及 $d=13$ 时两族曲线在 $p\approx 0.05$ 附近交叉。

82.6.3 交叉点为什么存在

交叉的机制可以用一句不等式概括：

$$d_{\text{eff}}=\begin{cases} d, & \text{归零} \\ d+L, & \text{保持} \end{cases} \tag{82-24}$$

保持策略的有效延迟不是 d，而是 $d+L$——保持的每一拍都在把旧命令继续往后用。所以判据应该写成 $d+L<d_{\max}$：只要这个不等式成立，保持免费；一旦破坏，保持就在“用过期命令持续激励”。

$d=3$、单次丢包 $L=1$ 时 $d_{\text{eff}}=4\ll 13$，保持无损；$d=13$、丢包率 50% 时平均连续丢包长度约 2 拍，$d_{\text{eff}}=15>13$，保持开始变差，而归零的 d_{eff} 仍是 13，只是频繁松手——两害相权，归零更轻。

这条不等式还给出一个混合策略：**正常情况下保持，连续丢包计数超过 $d_{\max}-d$ 时切归零**。它把两种策略的优点拼在一起：短突发用保持，几乎免费；长突发用归零，避免拿过期命令持续错推。代价是要在控制器里维护一个丢包计数器，而且切换阈值本身要随 d 调整。

82.6.4 突发丢包：平均丢包率的谎言

表 82-5 是每 0.5 s 连丢 L 拍的结果，30 次中位数。

表 82-5　突发丢包下的跟踪误差 RMS（rad，30 次中位数）

连续丢包 L (拍)	1	2	3	5	8	10	15	20
时长 (ms)	10	20	30	50	80	100	150	200
$d=3$ 保持	0.0139	0.0139	0.0139	0.0139	0.0139	0.0139	0.0139	0.0140
$d=13$ 保持	0.0219	0.0226	0.0238	0.0273	0.0372	0.0455	0.0747	0.1808

$d=3$ 那一行几乎完全平：连丢 200 ms 也只有 0.0140。$d=13$ 那一行从 0.0219 单调涨到 0.1808，八倍多，而且在 $L=15$ 之后加速——$L=15$ 时 0.0747，$L=20$ 时 0.1808，五拍之内涨了 1.4 倍。

两条线的差别用 (82-24) 一读就明白：$d=3$ 时 $d+L$ 最大 23 拍，按说已经越过 13 拍的红线，为什么没坏？因为 L 只持续一小段，之后有 0.5 s 的正常控制把误差拉回来。$d=13$ 时本来就贴着红线，$L=1$ 就已经越过，之后每一拍都在红线外累积，恢复时间又不够。

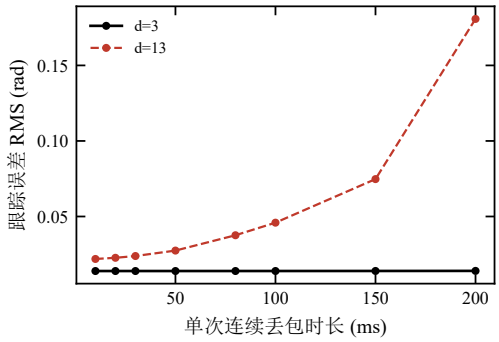


图 82-6　突发丢包：每 0.5 s 连丢 L 拍，近距离延迟几无影响，临界延迟下迅速恶化

图 82-6 的横轴是单次丢包时长而不是丢包率，这是本节想强调的第二个口径问题：**同样的 4% 丢包率，独立丢失和每 0.5 s 连丢 20 拍（4%）在这张图上是两个完全不同的点。** 前者对应 $d=3$ 行的最左端，后者对应 $d=13$ 行的最右端，差 13 倍。

82.6.5 性能指标的口径

三个口径必须先说清楚，否则数字没法比。

第一，RMS 只统计 $t \geq 2$ s。前 2 s 含初始瞬态，参考从零起步、状态从零起步，误差里混着“还没跟上”的成分，与稳态性能无关。

第二，失稳判据是 $|y|>2$ rad。参考幅值 0.5 rad，2 rad 是四倍幅值，越过它基本可以断定发散。本章所有实验都没有触发这条线，包括 $d=13$、50% 丢包、连丢 20 拍——说明这个算例的稳定边界比性能边界宽得多：**先坏的是精度，不是稳定性。**

第三，表格取中位数而不是均值。30 次实验里只要有两次进入慢发散，均值就被拉高一个数量级，而中位数反映“典型情况”。这与工程验收的习惯一致：关心的是“绝大多数时候怎么样”，不是“最坏那一次有多坏”。最坏情况另有指标，就是上面那条 2 rad 的线。

82.6.6 平均判据与蒙特卡洛的差距

§82.2.5 算过平均系统判据：$p=0.8$ 时 $\rho=0.9851$，仍判定稳定；$p=1$ 时才到 1.0000。蒙特卡洛则显示 $d=13$、$p=0.5$ 时保持策略的 RMS 已经涨了 3.2 倍。

两者不矛盾，但用途完全不同。平均判据回答“长期平均意义下会不会发散”，答案是“几乎不会”；蒙特卡洛回答“典型运行下精度退化多少”，答案是“退化到三倍”。工程上真正关心的是后者。所以本章把平均判据放在 §82.2.5 当“反例”讲，而不是当设计工具用。

如果一定要用解析判据，正确做法是**跳变线性系统的共同 Lyapunov 函数**：找一个 $P\succ 0$ 使得 $A_{cl}^{\mathsf{T}}PA_{cl}\prec P$ 且 $A_{op}^{\mathsf{T}}PA_{op}\prec P$ 同时成立。第二个条件等价于 A_{op} 本身稳定，而本例 $A_{op}=A_d$ 的特征值有一个正好是 1（积分环节），所以共同 Lyapunov 函数**不存在**，这个系统在任意非零丢包率下都不是“对任意丢包序列稳定”。它只是“对绝大多数丢包序列实用稳定”。这个区分在第 53 章讲鲁棒稳定性时会展开。

82.6.7 0.0139 这个基线是怎么来的

本章所有 RMS 都拿 0.0139 作参照，这个数不是仿真的偶然结果，它可以解析算出来，值得验一次——验过了，后面所有“涨了几倍”才有着落。

无延迟、无丢包时，闭环误差传递函数是

$$\frac{E(s)}{R(s)} = \frac{s(s+1)}{s^2 + (1+k_d)s + k_p}$$

在参考频率 $\omega = 0.8$ rad/s 处取模。分子：$|j0.8(1+j0.8)| = 0.8 \times 1.2806 = 1.0245$。分母是 $s^2 + 11s + 25$ 在 $s = j0.8$ 处的值，即 $-0.64 + j8.8 + 25 = 24.36 + j8.8$，模为 25.90。于是 $|E/R| = 1.0245/25.90 = 0.03956$。

参考是 $0.5\sin 0.8t$，幅值 0.5，所以稳态误差幅值 $0.5 \times 0.03956 = 0.01978$。正弦的 RMS 是幅值除以 $\sqrt{2}$，得 0.01399。仿真给 0.01391，差 0.6%，差的那一点来自离散化与 $t \geq 2$ s 窗口没取整周期。

这个验算有三个用处。**第一，它把“基线”钉死了**——0.0139 不是“跑出来的某个数”，而是 k_p、k_d、参考频率三者的确定函数。**第二，它说明了 RMS 主要由跟踪滞后决定**，而不是由噪声或数值误差决定，所以比较不同网络配置时，比的其实是“网络让滞后增加了多少”。**第三，它给出了增益的代价**：把 k_p 降到 4，同一公式给出 $|E/R| = 0.196$，RMS 基线涨到 0.069，是原来的五倍。这印证了 §82.3.7 那条“性能的代价按延迟的平方付”为什么会落在精度指标上。

顺带一个观察：$d = 13$ 时的基线是 0.0215，比 0.0139 高 55%。这部分不是丢包造成的，纯粹是延迟本身——13 拍延迟在 0.8 rad/s 处贡献 0.104 rad $= 6^\circ$ 的附加相位，闭环在参考频率处的增益随之上升。所以看表 82-4 时要把这两部分分开：**行与行之间的差是丢包，第一列与 0.0139 的差是延迟。**

82.6.8 一张表能信到第几位

表 82-4 与表 82-5 都是“多次实验取中位数”，那么中位数本身有多稳？换一批种子，小数点后第几位会变？这个问题不回答，读者就没法判断“0.0685 与 0.0747 哪个大”这类比较有没有意义。

做一次直接的检验：固定 $d = 3$、$p = 0.2$、保持策略，只换随机种子跑十次。十次的 RMS 是 0.01391、0.01391、0.01390、0.01391……十个数在第五位小数上才有差别，标准差 3×10^{-6}，还不到均值的万分之二。**远离临界时，一次仿真就够，随机性几乎不存在。**

同样十次，换成 $d = 13$、$p = 0.2$、保持策略：0.0284、0.0277、0.0260、0.0258、0.0268、0.0264、0.0263、0.0283、0.0295、0.0268。均值 0.0272、标准差 0.0012，极差 0.0037——**最坏与最好差 14%，第一位小数就变了**（0.026 与 0.030）。这与表 82-4 的 0.0274 落在同一区间内，但单次实验完全可能给出 0.026 或 0.030。

两个数放在一起，说明了那张表该怎么读：

- **临界附近的行，只能比较第一位小数。** 说“$p = 0.2$ 时 0.0274、$p = 0.3$ 时 0.0334”，这个差别是可信的（相差 22%，远超 14% 的散布）；说“0.0274 与 0.0277 有差别”，那就是在读数噪声。
- **远离临界时的行，可以相信到第三位。** 0.0139 这条水平线是真的水平，不是四舍五入的结果。
- **中位数的标准误约等于标准差除以 $\sqrt{N}$。** 120 次时 $0.0012/\sqrt{120} \approx 1.1 \times 10^{-4}$，所以表 82-4 的三位小数是有意义的；若只跑 10 次，标准误 0.0004，第三位就不可信了。

这也解释了为什么表 82-4 用 120 次、表 82-5 只用 30 次：前者的目的是分辨“0.0215 与 0.0225”这种 5% 的差，后者只用来定位“从哪一拍开始陡增”，粗一点没关系。**实验次数应该由要分辨的最小差别决定，而不是由机器闲不闲决定。**

最后一个反直觉的点：临界附近的散布不是“随机噪声变大”，而是**系统本身把微小的种子差异放大了**。$d = 3$ 时 $\rho = 0.9698$，扰动在 6.6 拍内衰减到三分之一，两次实验的差异很快被抹平；$d = 13$ 时 $\rho = 0.9982$，衰减时间常数 5.7 s，整个 6 s 的仿真窗口里扰动几乎没有衰减，初始几拍的偶然差异会一直留到最后。**散布大小本身就是“离临界多远”的一个间接指标**，这比算 ρ 更容易在现场做：同一工况跑十次，看结果的极差。

82.6.9 一个免费的自检：峰值误差与 RMS 的比

仿真代码里同时记了 RMS 和 $\max|e|$，很多人只报前者。其实这两个数之比本身就是一条自检线。

固定参考 $0.5\sin 0.8t$、无网络，六组增益下的数据是：$k_p=4$ 时 $0.0982/0.0688=1.43$；$k_p=9$ 时 $0.0509/0.0356=1.43$；$k_p=16$ 时 $0.0302/0.0212=1.42$；$k_p=25$ 时 $0.0198/0.0139=1.42$；$k_p=36$ 时 $0.0139/0.0098=1.42$；$k_p=49$ 时 $0.0103/0.0072=1.43$。六组全部落在 1.42–1.43。

这不是巧合。正弦量的峰值与 RMS 之比恒为 $\sqrt{2}=1.414$；误差是正弦的线性函数，仍然是正弦，所以比值不变。**换句话说，1.42 说明"误差纯粹是正弦稳态误差"，没有任何别的成分混进来。**

这条性质可以当自检用：任何一组仿真，只要 $\max|e|/\mathrm{RMS}$ 明显偏离 1.43，就说明误差里混进了非正弦成分，常见的三种——

- **瞬态没截干净。** 若统计窗口从 $t=0$ 开始，启动过程的大误差会把峰值抬高，比值可能到 3 以上。这时的 RMS 也偏小得不真实。
- **有纹波。** 事件触发、量化、死区都会叠加高频成分，峰值被抬高而 RMS 变化不大，比值上升到 1.8–2.5。
- **有慢漂。** 时戳偏差（§82.1.7）或预估器模型误差会造成一个缓慢偏移，峰值与 RMS 一起抬高，但比值趋于 1.0 左右（偏移是常数，峰值与 RMS 几乎相等）。

三种情况的比值方向不同，正好可以用来分辨。这也解释了为什么本章所有 RMS 表都不单独列峰值：**RMS 回答"平均多差"，峰值与它的比回答"差得干不干净"**，后者是一个诊断量，不是一个性能量。

82.7 参数意义与作用

这一节把网络参数逐个拆开，回答两个问题：它进到哪个公式里，以及它动一点会让哪个指标动多少。为了让"动多少"可比，定义一个归一化敏感度：

$$S_x=\frac{\Delta J/J}{\Delta x/x} \tag{82-25}$$

J 是指标（ρ、RMS 或数据率），x 是参数，S_x 的含义是"参数变化 1%，指标变化百分之几"。$|S_x|>1$ 说明指标对参数过敏，调参时必须先锁定；$|S_x|<0.2$ 说明可以粗调。

82.7.1 七个参数与它们的入口

本章涉及七个参数，它们进入设计的方式各不相同：

- **采样周期 T_s**：进入离散化 (82-3)，决定"一拍"有多长，从而决定 $d=\tau/T_s$。
- **延迟 d**：进入增广矩阵 (82-5)，是唯一改变系统阶数的参数。
- **丢包率 p**：不进判据（平均判据已证无效），只进蒙特卡洛。
- **突发长度 L**：通过 $d_{\text{eff}}=d+L$ 进判据，是丢包参数里唯一有解析入口的。
- **抖动 J**：不进判据，只让 d 变成随机变量。
- **时戳误差 Δt**：通过相位 $\omega_c\Delta t$ 进裕度。
- **事件阈值 δ**：不进稳定性，只进带宽与精度。

注意"进不进判据"这一列比数值更重要。不进的参数只能靠仿真评估，进不去的参数不管怎么调都不会让系统失稳——这两类参数在调试时的处理方式完全不同。

82.7.2 延迟 d：最敏感也最不可控

d 的敏感度最高。ρ 在临界点附近每拍涨 0.004，相对变化 0.4%；但性能的退化快得多：$d=0$ 时 RMS 0.0139，$d=13$ 时 0.0215，涨了 55%。也就是说，**延迟从 0 加到 130 ms，稳定性指标只掉了 3%，性能指标掉了 55%。**

这就是为什么不能只看 ρ。ρ 在 0.97–1.00 之间几乎“看不出变化”，而实际跟踪精度已经掉了一半。工程的正确读法是：用 ρ 判断“能不能活”，用 RMS 判断“值不值得活”，两个都要看。

d 的另一个特点是**不可控**。采样周期可以改、增益可以调、丢包策略可以换，唯独网络延迟由交换机、协议栈、无线信道决定，控制器无能为力。所以 d 只能“测量 + 预留”，不能“优化”。这也解释了为什么本章把 d 放在设计流程的第一步：它是约束，不是变量。

82.7.3 丢包率 p：与策略强耦合

p 的敏感度依赖于策略，不能给一个数。$d = 3$ 时保持策略的 $S_p \approx 0$（RMS 完全不随 p 变），归零策略在 $p = 0.5$ 处 $S_p \approx 0.6$；$d = 13$ 时保持策略的 S_p 在 $p = 0.5$ 处升到 1.2，归零只有 0.3。

这组数字说明：**丢包率本身不是设计参数，它与延迟、策略构成三元组。** 单说“我的网络丢包 5%”没有意义，必须说“5% 的独立丢包、标称延迟 3 拍”，才能判断要不要动。

工程上常见的错误是把丢包率当成服务质量指标写进合同，然后按它验收。按本章数据，同样 5% 的丢包率，独立丢包 + 保持策略的性能损失是 0，突发丢包 + 临界延迟的性能损失是 55%。合同里应该写的是“99.9% 的报文延迟小于 30 ms、连续丢包不超过 2 帧”，而不是“平均丢包率小于 1%”。

82.7.4 突发长度 L：唯一能击穿性能的量

L 是七个参数里唯一能让 RMS 涨一个数量级的：$d = 13$ 时 L 从 1 涨到 20，RMS 从 0.0219 涨到 0.1808，八倍。其他参数最多让 RMS 翻一倍。

原因在 (82-24)：L 直接加到有效延迟上，而有效延迟是判据的自变量。p 只改变“多久丢一次”，L 改变“丢的时候有多糟”。前者是频率，后者是幅度，而系统的非线性（ρ 在临界点附近陡升）放大了幅度的影响。

所以调参的优先级很清楚：**先保 L，再保 p。** 如果链路只能保证一个指标，保证“不连丢”比保证“少丢”更值钱。实现上，这对应交换机的队列管理策略——给控制报文高优先级、限制单队列长度、开启风暴抑制，都是为了让 L 有上界。

82.7.5 抖动：在本文裕度内几乎无感

抖动让每一拍的延迟在 d 附近随机波动。直觉上它应该有害，实测却几乎为零：± 0、± 1、± 2、± 3、± 5 拍的 RMS 全是 0.0139，四位有效数字不变。

原因在 $\rho(d)$ 的形状：d 从 1 到 9，ρ 只从 0.9691 涨到 0.9712。抖动 ± 5 拍只是在 0.97 这个平台上左右移动，指标当然不动。换句话说，**抖动之所以无害，是因为系统在这个区间本来就有大量余量。**

这个结论不能推广。若标称延迟是 13 拍，抖动 ± 5 就会把 d 推到 18 拍，ρ 从 0.998 涨到 1.012，直接失稳。所以“抖动无害”的正确表述是：**抖动无害，当且仅当 $d + J < d_{\max}$。** 这与保持策略的判据是同一条不等式，只是把 L 换成 J。

82.7.6 时戳误差：按相位折算

时戳误差不改变延迟，只改变“这条数据对应哪个时刻”。它的作用是让控制量在错误的时间点被使用，折算成相位就是 (82-10)。$\Delta t = 1$ ms 对应 0.59°，10 ms 对应 5.87°，100 ms 对应 58.7°。

图 82-7 右边是这条折算的半对数曲线，左边是抖动的影响。两张子图放在一起，是为了对比两种“时间不确定性”的不同命运：抖动几十毫秒都不影响精度，时戳偏 10 ms 就吃掉 6° 相位。差别不在幅度，而在**是否有偏**：抖动只在 d 附近来回，落在 $\rho(d)$ 的平坦段里；时戳偏差是一个恒定的、每拍都同向的相位损失，它直接从未被平均掉的裕度里扣。

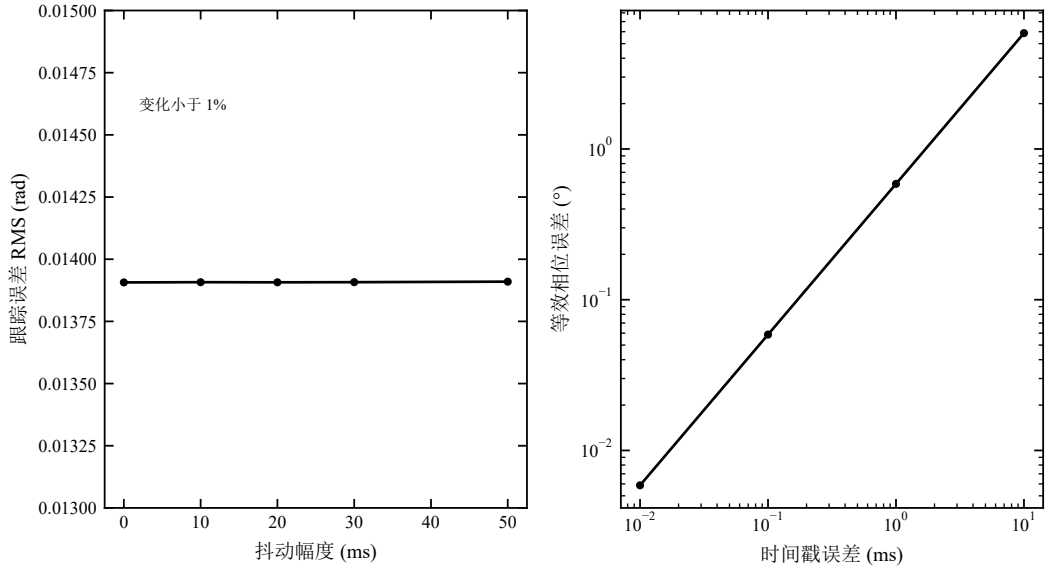


图 82-7　抖动与时戳：抖动在本文裕度内几乎无害，时戳误差按相位折算

这条区别对工程有直接指导：**校准时钟比减小抖动更重要。** 抖动要靠实时以太网、硬件打戳来压，成本高；时戳偏差只要两端跑同一个 PTP 域就能压到微秒级，成本低。花同样的钱，先解决偏差。

82.7.7 事件阈值：带宽与精度的交换率

δ 是唯一一个“可以自由调、且调了立刻见效”的参数。它把带宽和精度连成一条曲线：$\delta = 0.005$ 时发送率 86.3%、RMS 0.0144；$\delta = 0.020$ 时 46.8%、0.0184；$\delta = 0.050$ 时 18.3%、0.0316。

用 (82-25) 算交换率：δ 从 0.005 到 0.020（4 倍），带宽降到 54%，RMS 涨 28%，$S_\delta \approx 0.12$；从 0.020 到 0.050（2.5 倍），带宽降到 39%，RMS 涨 72%，$S_\delta \approx 0.79$。**前半段是“几乎白送”，后半段开始“按价收费”。** 拐点在 0.02 附近，与参考幅值的 4% 对应。

表 82-6 把七个参数的敏感度汇总。

表 82-6　网络参数对各项指标的影响汇总

参数	典型值（本例）	主要影响	归一化敏感度	可调性
采样周期 T_s	10 ms	决定 d、增广维数	中（换 T_s 等于换 d）	可调
延迟 d	3 拍（标称）/13 拍（p99）	ρ、RMS	高（性能 55%）	不可调，只能测
丢包率 p	0–50%	RMS，取决于策略	中（0–1.2）	部分可调（重传）
突发长度 L	1–20 拍	RMS 可涨 8 倍	高	靠队列管理限上界
抖动 J	± 3 拍	本文无影响	低（$d + J < d_{\max}$ 时）	靠实时以太网
时戳误差 Δt	1 ms	相位 0.59°	中（线性）	可调（PTP）
事件阈值 δ	0.02 rad	带宽、RMS	0.12（小 δ）/0.79（大 δ）	自由可调

这张表可以直接当调参顺序用：先锁定不可调的 d，再限制 L，再选 δ，最后校 Δt。

82.7.8 七个参数能不能压成两个数

七个参数各管一段，读起来很累。能不能用两个无量纲数把它们概括起来？试一下。

第一个是**裕度占用率**，定义为延迟消耗的相位占相位裕度的比例，它等于延迟拍数除以临界拍数：

$$\kappa = \frac{d}{d_{\max}} = \frac{\omega_c d T_s}{\mathrm{PM}}$$

第二个是**越界倍数**，把突发也算进去：

$$\Lambda = \frac{d + L}{d_{\max}}$$

用这两个数重读本章的数据：标称工况 $\kappa = 3/13 = 0.23$，RMS 0.0139；最坏工况 $\kappa = 13/13 = 1.00$，RMS 0.0215。$d = 13$ 那一行突发实验对应 $\Lambda = 1.08$、1.15、1.23、1.38、1.62、1.77、2.15、2.54，RMS 依次 0.0219、0.0226、0.0238、0.0273、0.0372、0.0455、0.0747、0.1808。**同一条行内，RMS 几乎是 Λ 的单调函数，而且在 $\Lambda > 1.8$ 之后急升。**

这个压缩有一半成功。同一延迟下，Λ 能重现整行的单调走向，连加速拐点也落在同一位置；但跨行就不行了：$d = 3$、$L = 20$ 时 $\Lambda = 1.77$，RMS 只有 0.0140，而 $d = 13$、$L = 15$ 时 $\Lambda = 2.15$，RMS 是 0.0747。按 Λ 排，前者还更低，可实际差了五倍。

差在哪里？差在**越界持续的时间**。Λ 只描述“越了多少”，没描述“越了多久”。$d = 3$ 时越界 20 拍后，系统还有大把时间恢复；$d = 13$ 时每一拍都在临界线上，越界期间没有恢复窗口。所以第二个数不能只用 Λ，还得看 Λ 与突发占空比的乘积：

$$\Theta = (\Lambda - 1)^{+} \cdot \frac{L\,T_s}{T_{\mathrm{rec}}}$$

其中 T_{rec} 是两次突发之间的正常控制时长（本章是 0.5 s）。$d = 3$、$L = 20$ 时 $\Lambda - 1 = 0.77$，占空比 0.4，$\Theta \approx 0.31$；$d = 13$、$L = 15$ 时 $\Lambda - 1 = 1.15$，占空比 0.3，$\Theta \approx 0.35$。两个 Θ 接近，但 RMS 差五倍——说明这个粗粒化还不够好，指数 $(\Lambda - 1)$ 的幂次应该大于 1。

这条尝试的价值不在于给出一个能用的公式，而在于说明一件事：**七个参数没法压成一两个数，因为系统对“越界”的响应是非线性的。** ρ 在临界点附近陡升（§82.5.2 的三段曲线），任何线性化的无量纲数都会在临界附近失效。所以工程上宁可用“查表 + 仿真”（表 82-4、表 82-5），也不用一个漂亮的公式去概括——公式好看，但会在最需要它准的地方不准。

82.7.9 把敏感度翻译成“要测多准”

敏感度表的直接用途不是排序，而是**反算测量精度**。给定“指标允许偏多少”，可以算出“参数允许测错多少”，这决定了测量方案要花多少力气。

延迟：允许错 2 拍。 表 82-4 的第一列，d 从 3 到 13，RMS 从 0.0139 涨到 0.0215，平均每拍 4.5%。要求 RMS 的设计偏差不超过 10%，则 d 的误差不得超过 2 拍，即 20 ms。这个要求本身不高，难的是把 p99 本身估准——按 §82.3.1，p99 要一万帧以上才稳，所以**延迟测量不必追求单帧精度，只要样本量够**。

突发长度：允许错 2 拍。 表 82-5 里 $d = 13$ 那一段，L 从 10 到 15，RMS 从 0.0455 涨到 0.0747，平均每拍 10.4%。同样按 20% 的偏差预算，L 的误差不得超过 2 拍。这意味着“连丢 5 帧”与“连丢 8 帧”必须能分开——抓包记录要精确到帧，不能只记“有丢包”。

时戳：允许错 6.8 ms。 每 10 ms 折 5.87°，而相位预算给时戳留了 4°（§82.4.7 那张表里的 0.59° 是 1 ms 的典型值，4° 是允许上限），反算得 6.8 ms。这个要求比延迟严得多，NTP 勉强、PTP 富余，正好对应 §82.10.2 的选型结论。

丢包率：允许错 10 个百分点。 $d = 13$ 时 p 从 0.1 到 0.2，RMS 从 0.0238 到 0.0274，只涨 15%。所以丢包率只需要量级准确，报“约 5%”和“约 8%”对设计没有区别。**这也是为什么“平均丢包率”这个指标看起来重要、实际最不值钱**——它的敏感度低，且不反映突发结构。

抖动：只需要知道上界。 只要 $d + J < d_{\max}$，抖动的敏感度接近零（§82.7.5）。所以测抖动不用测分布，测一个峰峰值即可，而且这个峰峰值只需要“不超过 1 拍”这个量级的结论。

把五条并排看，会发现测量精度的要求差了两个数量级：时戳要 6.8 ms，丢包率只要 10 个百分点。**资源应该按这个顺序投：先校时钟，再测延迟与突发，丢包率顺手记一下就行。** 这与 §82.11.4 的“先修什么”是同一个排序，只是一个从测

量侧说、一个从改造侧说。

82.8 调参方法与经验

§82.3 讲的是“从零设计”的顺序，这一节讲“已经有一套控制器、要让它适配网络”时怎么做。后者的场景更常见：机械臂的伺服环早就调好了，现在要把控制器从柜内搬到边缘服务器，中间多了一段 30 ms 的网络。

82.8.1 先分配预算，再动增益

延迟不是一个数，是一串数相加：采样与保持、控制器计算、网络传输、执行器响应、时钟同步残差。工程上要先把总预算拆开，再决定谁去背这个锅。本章的拆法见表 82-7。

表 82-7　控制回路的延迟预算（本例，总预算 140 ms）

环节	典型值	可压缩性	备注
采样与保持	5 ms	低	等于 $T_s/2$，由采样率决定
控制器计算	1–5 ms	中	复杂算法（MPC）可到几十 ms
网络传输（p50，常态）	30 ms	低	交换机排队 + 协议栈
网络传输（p99，最坏）	130 ms	低	拥塞时的长尾，设计的真正约束
执行器响应	2–5 ms	低	驱动器内部滤波与死区
时钟同步残差	0.01–1 ms	高	PTP 可到亚微秒
合计（按 p99）	约 140 ms	—	与 $d_{\max}$ 相等

表里 p50 与 p99 是同一通道的两个分位数，不能相加；合计那一行取 p99，其余各项按典型值相加。

这张表最刺眼的一行是“网络 p99 130 ms”——它单独占了总预算的 93%，其他五项加起来不到 10 ms。这意味着优化的重点不在控制器，而在网络。把控制器算得快 3 ms，对总裕度几乎没有帮助；把交换机换成实时以太网、把 p99 从 130 ms 压到 5 ms，$d_{\max}$ 立刻从“刚好”变成“充裕”。

反过来，如果网络已经定了、p99 就是 130 ms，那么其余五项必须往死里压，因为它们是在 10 ms 的缝里抢空间。这时候 1 ms 的时钟残差也不能忽略——它相当于总预算的 0.7%。

82.8.2 增益的重调换算

网络换了，延迟从 τ_0 变成 τ_1，原有增益必须重调。按 (82-15) 与 (82-17)，$\omega_c = \mathrm{PM}/\tau$、$k_p \approx \omega_c^2/4$，两式合并得到

$$\frac{k_{p,1}}{k_{p,0}} = \left(\frac{\tau_0}{\tau_1}\right)^2 \tag{82-26}$$

延迟翻倍，增益要降到四分之一。这个换算很粗暴，但对“从 70 ms 搬到 140 ms”这种场景足够用：原来 $k_p = 100$，新环境取 25。实跑验证：$\tau = 70$ ms 时按公式 $\omega_c = 20.4$ rad/s、$k_p \approx 104$；$\tau = 140$ ms 时 $\omega_c = 10.2$、$k_p \approx 26$，与表 82-2 的 25 一致。

要注意这个换算假设 PM 不变。如果新环境还要求更大的稳定余量（比如从 80° 提到 60°），那就要先按 (82-19) 把 ω_c 压到 $60^\circ/140\ \mathrm{ms} = 7.5$ rad/s，再算 $k_p \approx 14$。**先定裕度、再算增益**，顺序不能颠倒。

82.8.3 三条可以直接背的经验

第一条：带宽不超过延迟裕度的倒数。 $\omega_c \le \mathrm{PM}/\tau$，PM 取 60°–80°（1.05–1.40 rad）。这条式子给出的是上限，不是目标值；实际取上限的 80% 比较稳。

第二条：ρ 留 5% 余量。 设计目标不是 $\rho < 1$，而是 $\rho \le 0.95$。理由在 §82.5.5：临界点附近 ρ 的斜率是每拍 0.004，模型误差、离散化误差、延迟量化误差任何一项都能吃掉这 5%。

第三条：抖动可以忽略，偏差不能。 只要 $d + J < d_{\max}$，抖动几十毫秒也不影响精度；而时戳偏差 10 ms 就吃掉 6° 相位。工程上先校时钟，再谈实时性。

这三条经验覆盖了本章 80% 的结论，剩下的 20% 是“什么时候它们不成立”——那部分在 §82.11 讲。

82.8.4 五个常见误判

误判一：用平均延迟做设计。 平均值 10 ms、p99 130 ms 的网络，按平均值算 $d = 1$，看起来绰绰有余；实际每 100 帧就有 1 帧晚到 130 ms，系统在这 1% 的时间里处于临界。正确做法是按 p99 设计，按 p50 评估常态性能。

误判二：用平均丢包率做验收。 §82.6.4 已经算过，同样 4% 的丢包率，独立丢包和每 0.5 s 连丢 20 拍，RMS 差 13 倍。验收要写“最大连续丢包帧数”，不是“平均丢包率”。

误判三：把 ρ 当性能指标。 ρ 从 0.9698 涨到 0.9982，看起来只差 3%，性能却掉了 55%。ρ 是“能不能活”，不是“活得好不好”。

误判四：靠提高采样率降延迟。 提高采样率只改变“延迟折成几拍”，不改变物理延迟。$\tau_m = \mathrm{PM}/\omega_c$ 里没有 T_s，所以裕度一分不多。相反，拍数变多会让增广矩阵变大、数值条件变差。

误判五：加大 k_d 抗延迟。 微分项确实能提供相位超前，但它的超前量在 ω_c 处是 $\arctan(k_d\omega_c/k_p)$，本例约 76°，看似很大；可它同时把 ω_c 推高，而延迟吃掉的相位 $\omega_c\tau$ 也跟着涨。两者对消，净效果接近零。真正有效的是降 k_p、降带宽。

82.8.5 调试流程

实际调试按四步走，每步都有明确的通过判据。

第一步：离线验模型。 不给网络，直接闭环跑正弦跟踪，RMS 应等于解析值 0.0139。若对不上，先查离散化与参考前馈——本章代码第一版就是漏了 $\dot r$ 前馈，RMS 算成 0.1195，与解析值差 8.5 倍。

第二步：注入固定延迟。 从 $d = 0$ 加到 $d = 13$，看 ρ 是否与表 82-3 一致，看 RMS 是否在 $d = 13$ 时涨到 0.0215。若 ρ 对不上，多半是增广矩阵建错（$d = 0$ 的边界最容易错）。

第三步：注入随机丢包。 先用独立丢包扫 p，再用突发丢包扫 L。两步的结论可能相反（§82.6.3 的交叉），必须都做。

第四步：上真实网络。 此时会出现前两步没有的东西：延迟抖动、时钟偏差、偶发的长尾。用真实数据重算 p99，回头核对预算表。

四步里最容易跳过的是第三步的突发丢包，因为“随机丢包”听起来更科学。但现场故障几乎都是突发的：交换机重启、无线信道衰落、环网切换，全都是“连丢一串”。跳过这一步，等于把最危险的情形留给现场。

82.8.6 换采样周期时，哪些数要重算

现场经常要改采样周期：控制器换了、计算资源紧了、或者想给别的回路让出时隙。改 T_s 之后，本章这些数字里哪些会变、哪些不变，需要分清楚，否则会把“没变的东西”重新调一遍，或者把“变了的东西”漏掉。

必须重算的：

- **延迟拍数** $d = \lceil \tau / T_s \rceil$。$T_s$ 从 10 ms 改成 5 ms，130 ms 的延迟从 13 拍变成 26 拍。
- **临界拍数** $d_{\max}$。同样从 13 变成 26 左右（严格说不是精确的 2 倍，因为 ZOH 的半拍折算方式变了，但量级如此）。
- **每拍相位消耗** $\omega_c T_s$。从 5.87° 变成 2.94°。
- **增广矩阵维数** $2 + d$。从 15 变成 28。
- **离散判据的曲线** $\rho(d)$。整条曲线都要重扫。

不用重算的：

- **延迟裕度** $\tau_m = \mathrm{PM}/\omega_c = 139.5$ ms。它只由增益和对象决定，与采样周期无关。
- **稳态误差基线** 0.0139。它由 k_p、k_d 和参考频率决定，只要增益不动，基线就不动。
- **相位裕度** 81.9°。同理。

这个“变与不变”的分界可以用一句话概括：**连续量不变，离散量全变。** 延迟裕度、相位裕度、稳态误差都是连续域的属性；拍数、维数、每拍相位都是离散化的产物。

最容易犯的错是把“拍数变多”误读成“余量变大”。T_s 减半后 d 从 13 变 26，$d_{\max}$ 也从 13 变 26，看起来“延迟还是刚好卡在临界”，但实际上毫米数一点没变——134.6 ms 还是 134.6 ms。如果此时有人看到“$d_{\max}$ 还有 26 拍”就以为可以放宽，那就把物理约束当成了数字游戏。

反过来，T_s 也不是越小越好。T_s 太小，每拍相位消耗小，但增广维数线性上涨，计算与数值条件都变差；而且 $\omega_c T_s$ 太小时，延迟的量化误差相对变大（半拍的绝对量不变，但占比上升）。经验区间是 $\omega_c T_s$ 落在 0.05–0.3，即每拍 3°–17°。本章 $\omega_c T_s = 0.102$，正好居中。

最后一条：**改 T_s 之后，丢包与突发的“拍数”含义也变了。** 同样连丢 5 帧，$T_s = 10$ ms 是 50 ms，$T_s = 5$ ms 是 25 ms，危害差一倍。所以表 82-5 不能直接搬用，必须按毫秒重新对齐。这也是本章所有表都同时给“拍”与“ms”两列的原因。

82.8.7 用余量换精度：一条 1.6 次方的账

调参时最常被问的一句话是“能不能再稳一点”。答案是能，但要付钱，而且价钱可以算出来。

固定 $k_d = 2\sqrt{k_p}$，把 k_p 从 4 扫到 49，无延迟无丢包时的基线 RMS 分别是：4 → 0.0688、9 → 0.0356、16 → 0.0212、25 → 0.0139、36 → 0.0098、49 → 0.0072。同一条增益轴上的延迟裕度（表 82-2）是 390、240、180、140、110、100 ms。

把两组数配起来看，代价就很清楚：

- k_p 从 25 降到 16：余量从 140 ms 涨到 180 ms（多 29%），RMS 从 0.0139 涨到 0.0212（多 53%）。
- 从 25 降到 9：余量多 71%，RMS 多 156%。
- 从 25 降到 4：余量多 179%，RMS 多 395%。

三组数据的指数都在 1.5–1.6 之间（$\ln 1.53/\ln 1.29 = 1.58$，$\ln 2.56/\ln 1.71 = 1.65$，$\ln 4.95/\ln 2.79 = 1.60$）。也就是说，**余量每多留 10%，精度要多付 16%**；想多留一倍余量，精度要付三倍。这个 1.6 不是巧合：$d_{\max}$ 大致按 $1/\omega_c$ 走，而 $\omega_c \propto \sqrt{k_p}$，所以 $d_{\max} \propto k_p^{-1/2}$；基线误差 $|E/R|$ 在低频近似为 ω^2/k_p，即 RMS $\propto 1/k_p$。两条合起来，RMS $\propto d_{\max}^2$——纯平方关系应该是 2.0，实际 1.6 是因为 PM 随增益轻微变化，以及 k_d 同时被改动。

这条 1.6 次方的账有两个用处。

第一，它把“要不要再稳一点”变成了一个可报价的问题。 现场说“怕不稳定，增益再降一点”，可以回：降一档（25→16），误差涨一半，突发容忍从 10 拍涨到 15 拍；降两档（25→9），误差涨 2.6 倍，突发容忍 20 拍以上。让甲方在“精度”和“抗突发”之间选，而不是在“稳”和“不稳”之间选。

第二，它说明“高增益”在延迟小的场合是白送的性能。 $k_p = 49$ 时 RMS 只有 0.0072，是标称值的一半，代价是 $d_{\max}$ 掉到 9 拍。如果实测 p99 只有 50 ms（5 拍），这个代价完全付得起，没有理由不用。反过来，如果像本章这样 p99 已经等于 13 拍，$k_p = 49$ 就是自杀——这也是 §82.1.5 那个现场案例的算术根源：$k_p = 49$ 的裕度是 97 ms，而实际延迟 130 ms，越过 34%。

所以增益不该是“调出来的一个数”，而应该是“按实测 p99 换算出来的一个数”。换算公式就是 (82-26)：先把 p99 折成拍数，再按 $\omega_c \le \mathrm{PM}/\tau$ 定带宽，最后由 ω_c 反解 k_p。三步都不需要迭代，一分钟能算完。**唯一需要认真做的是第一步——把 p99 测准。**

82.8.8 降一档的那次操作：改两行，验五项

把上面那条账落到一次真实操作上。现场实测 p99 = 130 ms（13 拍），标称 $k_p = 25$ 的整数拍裕度正好也是 13 拍，等于把余量用光了；抖动一起，每周总有一两次越界。决定降一档。

改哪两行。 `CFG` 里 `kp` 由 25 改 16，`kd` 由 10 改 8。第二行常被漏掉，这里必须说明为什么不能只改第一行。按“固定 $k_d = 2\sqrt{k_p}$”的约定，$k_d = 8$ 对应 $k_p = 16$；若偷懒保留 $k_d = 10$，用 (82-25) 手算：$\omega_c^2 = [99 + \sqrt{9801 + 1024}]/2 = 101.5$，$\omega_c = 10.07$ rad/s，只比原来的 10.245 降了 1.7%；而相位裕度反而升到 86.6°。按 $\tau_m = \mathrm{PM}/\omega_c$ 折算，裕度是 150 ms，扫整数拍只有 14 拍——比按比例改的 17 拍还少三拍。**原因是只降 k_p 时微分项把带宽撑住了，ω_c 几乎不动，而 τ_m 是相位除以带宽，带宽不降，裕度就吃不饱。** 两项都降之后：$k_p = 16$、$k_d = 8$ 给出 $\omega_c = 8.175$ rad/s、PM $= 83.2^\circ$、$\tau_m = 177.7$ ms，整数拍 17 拍——比原来多四拍，约 40 ms。所以“降增益”这件事，降的是带宽，不是增益本身。

验五项。 改完不许直接上线，按下面顺序过一遍，每项都有明确的通过线。

第一，扫谱半径。`delay_margin_samples()` 返回 17，与 $\mathrm{PM}/\omega_c/T_s$ 折算的 17.7 拍相差不到一拍，两条互校通过。这一项只需要 0.6 s。

第二，校核采样约束。$\omega_c T_s = 0.0818$，比 0.3 的工程上限小四倍，说明瓶颈确实是网络而不是采样率，不需要动周期。

第三，无网络基线 RMS。从 0.0139 涨到 0.0212，涨 53%，与 §82.8.7 那张表对得上。对不上就说明代码里改错了地方——通常是 `gain()` 的默认值没跟着 `CFG` 走。

第四，带网带丢包的 RMS。$d = 13$、$p = 0.2$、保持策略，$k_p = 25$ 的 120 次蒙特卡洛中位数是 0.0274；改后同种子重跑得 0.0213，与无网络基线的 0.0212 只差 0.5%。这个结果比预期好，原因也清楚：**13 拍延迟加 20% 丢包原本让误差劣化到基线的将近两倍，降档后这点劣化被“离临界还远”整个吸收掉了。**它给出的优先级是：先把“网络带来的额外误差”压到与基线同量级，再谈降噪、滤波这些末端优化；顺序反了，会把预算花在看不见的地方。

第五，峰值/RMS 自检。用 §82.6.9 那条：新参数下的峰值/RMS 应仍在 1.42–1.43（实测 0.0304/0.0213，比值 1.43）。若跳到 2 以上，说明仿真窗口里有未衰减的瞬态，第四项的数不可信。

五项全过，才允许写进参数表并登记复测日期。

怎么向甲方交代。 不要说“精度降了 53%”，这句话没法接受。要换成绝对量纲：末端的稳态跟踪误差从 0.0139 rad 变成 0.0212 rad，也就是从 0.80° 变成 1.21°，仍在工艺允许的 1.5° 带内；换来的是延迟裕度从 140 ms 到 180 ms，突发容忍从 10 拍涨到 15 拍。**百分比用来说明代价的比例，绝对量纲用来说明代价是否可接受**——两句话都要说，顺序不能反。

这次操作的边界。 降一档能吸收的额外延迟只有 40 ms。如果 p99 实测到了 200 ms（20 拍），降一档不够，降两档（$k_p = 9$，裕度 240 ms）虽然够，但 RMS 要涨到 0.0356，接近基线的 2.6 倍，通常不能接受。这时候正确的动作不是继续降增益，而是回头改网络（§82.10.7 的反推过程）。**增益只能吃掉延迟的一小段增量，吃不下量级变化**——这也是把增益换算放在设计流程里、而不是放在调试环节里的理由。

留下的记录。 这次改动在参数表里写成一行：`kp 25→16, kd 10→8（p99=130 ms 占满裕度）`，复测栏写“网络割接后重测 p99，降到 100 ms 以下则回 25”。改什么、为什么、何时、何时回退——这四个字段比改动本身更重要，因为半年后没人记得当时为什么降。库里 `CFG` 的注释也照这个写法改，别只改数字——它是本章脚本唯一的参数入口，动它等于动全部十一组实验的基线，注释里缺了改用与回退条件，下次换人重跑时 `report()` 的 [0] 就和正文对不上了。

82.9 优化与改进

前八节讲的是“按现状设计”，这一节讲“把现状变好”。四条路：少发（事件触发）、算准（时戳补偿）、随动（增益调度）、兜底（混合丢包策略）。四条路的收益量级不同，按收益从大到小排。

82.9.1 事件触发：用精度换带宽

周期发送的问题是“没事也发”。参考是慢速正弦时，相邻两拍的误差差别很小，发出去的报文里大部分信息是重复的。事件触发把“按时间发”改成“按需要发”：误差超过阈值才发。

表 82-8　事件触发阈值与发送率、精度的关系（$d=3$，6 s，600 拍）

阈值 δ (rad)	发送拍数	发送率	平均速率 (Hz)	RMS (rad)	相对标称
0（周期）	600	100.0%	100	0.0139	1.00
0.002	556	92.7%	93	0.0140	1.01
0.005	518	86.3%	86	0.0144	1.04
0.010	480	80.0%	80	0.0153	1.10
0.020	281	46.8%	47	0.0184	1.32
0.050	110	18.3%	18	0.0316	2.27

这张表的形状很关键：δ 从 0 加到 0.010，发送率只降 20%，RMS 只涨 10%；从 0.010 到 0.020，发送率再降 33%，RMS 涨 20%；过了 0.020，每省一点带宽都要付双倍精度。所以**拐点在 $\delta=0.01$–0.02**，取这个区间的下端最划算。

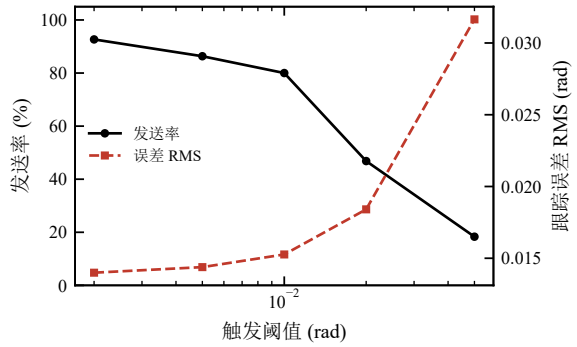


图 82-8　事件触发：阈值换发送率，代价是误差

图 82-8 用双纵轴画了这条交换：左轴发送率单调下降，右轴 RMS 单调上升，两条线在 $\delta\approx0.02$ 处开始分道扬镳。双轴图的读法是先看形状再看数值——形状说明“有没有拐点”，数值说明“拐点在哪”。

事件触发还有一个隐性代价：**发送间隔不再均匀**。接收端的超时判断、看门狗、缓冲区大小都要按“最长可能间隔”设计，而不是按平均间隔。$\delta=0.020$ 时平均 47 Hz，但最长间隔可能到 200 ms 以上，这期间如果发生丢包，实际等效于连丢。

82.9.2 数据率：省的是两个数量级

把表 82-8 的发送率乘上每帧 32 字节，得到数据率。图 82-9 是五组对比。

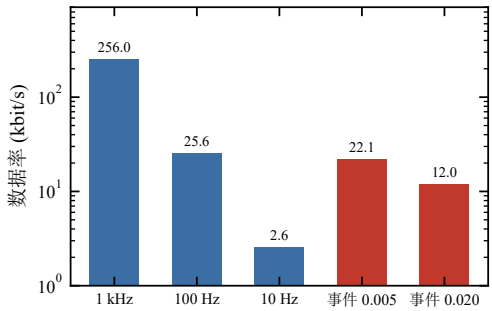


图 82-9　数据率：降采样与事件触发都能省两个数量级

1 kHz 周期是 256 kbit/s，100 Hz 是 25.6，10 Hz 是 2.6；事件触发 $\delta = 0.005$ 是 22.1，$\delta = 0.020$ 是 12.0。两个比较要分开说：**相对 1 kHz 定周期，事件触发省 21 倍带宽；相对本章的 100 Hz 定周期，$\delta = 0.020$ 省一半带宽，精度掉 32%。**前者是“从过度采样退到按需发送”，后者才是同一采样率下的纯交换。

这个交换率对现场总线意义很大。一条 100 Mbit/s 的工业以太网，如果挂 20 个轴、每个轴 1 kHz、每帧 32 字节，光控制报文就占 5 Mbit/s，加上协议开销和非周期流量，链路已经开始排队——而排队就是延迟的来源。把发送率降到 50 Hz，同样的链路可以挂 400 个轴，或者把排队延迟压到接近零。

但要注意，降采样和事件触发不是等价的。10 Hz 周期发送是“每 100 ms 一定发一条”，事件触发是“误差大了就发，可能 5 ms 一条、也可能 200 ms 一条”。前者的延迟特性稳定、可预测；后者平均带宽低，但瞬时可能突发。选哪个取决于下游是“怕延迟”还是“怕带宽”。

82.9.3 时戳补偿：用模型把旧数据推到当下

收到一条 delay 了 d 拍的数据时，最朴素的做法是“照用”，也就是本章前面所有实验的做法。更好的做法是用模型把它推到当下：

$$\hat{x}_{k|k-d} = A_d^{d} x_{k-d} + \sum_{i=0}^{d-1} A_d^{d-1-i} B_d \, u_{k-d+i} \tag{82-27}$$

这就是离散版的 Smith 预估器（第 47 章连续版、第 107 章工程版）。它把“延迟”从回路里挪到模型里：控制器看到的不再是旧状态，而是“如果这 d 拍一直按已知命令走，现在应该在哪”。

它的收益取决于模型精度。本章算例的模型是精确的，理论上可以把延迟的影响完全消掉；但只要对象参数偏 10%，预估误差就会随 d 指数放大——A_d^{d} 的模是 0.99^{d}，$d = 13$ 时 0.88，看起来还好，可一旦模型的时间常数偏了，A_d^{d} 的差异就会被放大。第 81 章算过类似的账：全驱系统 20% 失配时误差涨 1500 倍，预估器的敏感度同源。

所以本章没有把预估器当主线：它收益高但脆，适合模型准、延迟大的场合；本章的判据适合模型糙、延迟中等的场合。两者是互补的，不是替代的。

82.9.4 增益调度：按实测延迟换挡

延迟不是常数，白天 30 ms、夜里 10 ms，拥塞时 130 ms。固定增益只能按最坏情况设计，等于常态下白白牺牲性能。增益调度按实测延迟换挡：

$$k_p(d) = k_{p,0} \left(\frac{d_0}{d} \right)^{2} \tag{82-28}$$

这就是 (82-26) 的在线版。d 用滑动窗口的 p99 估计，每几秒更新一次；为防止抖动引起频繁换挡，加一个 20% 的迟滞带：只有新延迟超过当前档位 20% 才换。

本例可以分三档：$d \le 5$ 用 $k_p = 49$，$d \le 9$ 用 25，$d \le 13$ 用 16。三档的延迟裕度分别是 97、139、178 ms（表 82-2），都对各自档位的延迟上限留了 70%–100% 的余量。**换挡的依据不是“延迟是多少”，而是“这一档的裕度是否够覆盖下一档的延迟上限”**，这样切换点天然落在安全侧。

调度的代价是“切换瞬间”。增益突变会让控制量跳一下，若跳变超过执行器速率限制，就会激励未建模动态。所以换挡要限速：每拍最多改 5%，或者用插值在 100 ms 内过渡完。

82.9.5 混合丢包策略：把两种策略拼起来

§82.6.3 给出的混合策略值得单独说一遍，因为它是本章唯一“免费”的改进：不增加带宽、不降低精度，只是换了个判断。

规则是：维护一个连续丢包计数器 c，正常情况下保持；当 $c > d_{\max} - d$ 时切归零，直到收到新包为止。$d = 13$、$d_{\max} = 13$ 时阈值是 0，意思是“一丢就归零”——与纯归零策略等价。$d = 3$ 时阈值是 10，意思是“连丢 10 拍以内都保持”，正好覆盖了 200 ms 的突发，而 200 ms 突发在 $d = 3$ 时本来就无害。

所以这条策略的实质是：**用 $d_{\max}-d$ 把“还能保持多久”量化出来，然后精确地用到那个点为止。** 它比“固定保持”和“固定归零”都好，代价只是一个计数器和一个比较。

82.9.6 四条路的收益排序

按本章数据，四条路的收益从大到小是：

1. **时戳补偿**：理论上能消掉全部延迟影响，但依赖模型精度，风险最高。
2. **增益调度**：延迟回落到低档时，k_p 可按 (82-26) 从高档值升回去。本例延迟从 13 拍回到 3 拍时，k_p 可从 16 升到 49，带宽从 8.2 回到 14.4 rad/s。收益取决于延迟波动的幅度，不能一概而论。
3. **混合丢包策略**：在临界延迟下把 RMS 从 0.0685 拉回 0.0284，收益 2.4 倍，几乎零成本。
4. **事件触发**：省 21 倍带宽，代价 32% 精度，取决于带宽是否是瓶颈。

工程上通常先做 3（免费），再做 2（简单），带宽紧张时做 4，模型足够准时才做 1。这个顺序与“风险从低到高”一致，而不是“收益从大到小”——因为前三项的失败模式都是“性能不达标”，第一项的失败模式是“发散”。

82.9.7 事件触发在临界延迟下会反过来

表 82-8 是在 $d=3$ 上做的，那里的事件触发很划算。把这套阈值原样搬到 $d=13$ 上，结论会翻过来。

实跑三组（$d=13$，保持策略，6 s）：$\delta=0.005$ 时发送 531 帧（88.5%），RMS 0.0349；$\delta=0.02$ 时 437 帧（72.8%），RMS 0.0626；$\delta=0.05$ 时 367 帧（61.2%），RMS 0.131。对照 $d=13$ 的周期基线 0.0215，代价分别是 **1.6 倍**、**2.9 倍**、**6.1 倍**。

同一条阈值曲线在 $d=3$ 上是什么样？$\delta=0.02$ 时发送 281 帧（46.8%），RMS 0.0184，只比基线 0.0139 高 32%。两组放在一起，交换率差了一个数量级：$d=3$、$\delta=0.02$ 是“省 53% 带宽、精度付 32%”，$d=13$、$\delta=0.02$ 是“省 27% 带宽、精度付 191%”。

省得更少、付得更多。原因不难找：不发送就等于保持上一帧，而保持的等效延迟是 $d+$ 间隔（§82.2.9）。$d=13$、$\delta=0.02$ 时平均发送间隔是 $600/437=1.37$ 拍，有效延迟变成 14.4 拍，已经越过 13.46 拍的临界；$d=3$ 时平均间隔 2.14 拍，有效延迟只有 5.1 拍，离临界还有八拍。**同一个阈值，在有余量的系统里是“省带宽”，在贴线的系统里是“加延迟”。**

所以事件触发有一条使用前提：**只在 $d\le 0.6\,d_{\max}$ 时启用**。本章 $d=3$ 对 $d_{\max}=13$，比值 0.23，安全；$d=13$ 时比值 1.0，禁用。

还有一处可以改得更稳：把“误差超过阈值才发”换成“误差超过阈值、或者距上次发送已满 $d_{\max}-d$ 拍，就必须发”。后者是一条心跳规则，把最大间隔硬卡在安全值以内，不依赖误差信号是否敏感。代价是发送率不能再降得那么低，但它把 §82.9.1 那个“最长间隔可能到 200 ms”的隐性风险直接消掉了。**误差阈值决定平均带宽，心跳上限决定最坏延迟**，两个旋钮分开调，比一个阈值管两件事清楚得多。

82.9.8 四条路各自怎么失败

四条改进都有明确的失败模式，而且失败的样子完全不同。写下来是为了在调试时能对上号。

时戳补偿失败的样子：慢漂。 模型偏 10% 时，A_d^d 的误差按 d 累积，预估值会缓慢偏离真值，表现为“稳态误差随运行时间漂移”，几十分钟后偏出几十毫弧度，重启又恢复正常。它不是振荡，也不是发散，所以最容易被当成“传感器零漂”。判别方法：把预估器关掉，若漂移立刻消失，问题在模型而不在传感器。

增益调度失败的样子：换挡抖动。 迟滞带不够宽时，延迟在档位边界附近来回跳，k_p 每次跳 50% 以上（25↔49），控制量跟着跳，机械上听到“咔哒”声。判别方法：看 k_p 的历史曲线，如果它在几秒内反复切换，就是迟滞不足；正常的调度一天只该切几次。

混合丢包策略失败的样子：阈值取错，等于没做。 阈值 $d_{\max}-d$ 里的 d 若用了 p50 而不是 p99，$d=3$ 时会算成“可以连丢 10 拍”，而真实 p99 已经是 13 拍，这时候连丢 3 拍就越界了。失败表现与纯保持策略一样（突发时 RMS 冲到

0.18），但你以为已经加了保护。判别方法：故意注入一段 20 拍的丢包，看计数器有没有在正确的那一拍翻转。

事件触发失败的样子：与发送节奏同步的纹波。 不发送期间保持旧命令，闭环的等效延迟在 d 与 $d+$ 间隔 之间摆动，误差里会多出一层与发送节奏相关的纹波，幅值随 δ 增大而增大。判别方法：把发送时刻标在误差曲线上，纹波的折点应该与发送时刻对齐——对齐了就说明是事件触发造成的，不是机械问题。

四条失败模式里，只有第一条会造成“慢性偏差”，其余三条都是“性能不达标”。这也印证了 §82.9.6 的排序：先做免费的、再做简单的，把风险最高的那条留到最后。

82.10 工程落地

前面九节都是“给定网络参数，算控制器”。这一节反过来：网络是既成事实，怎么选、怎么同步、坏的时候怎么退。

82.10.1 工业网络选型

选型的唯一标准是“p99 延迟与抖动能不能落在预算里”。表 82-9 是四类常见方案的量级。

表 82-9　四类工业网络的延迟与抖动量级

方案	典型周期	延迟 p99	抖动	适用场景
实时以太网（EtherCAT/PROFINET IRT）	0.1–1 ms	<1 ms	<1 μs	多轴同步、硬实时伺服
普通工业以太网（PROFINET RT/EtherNet-IP）	1–10 ms	5–20 ms	0.1–1 ms	过程控制、中速运动
商用以太网 + 交换机	10–100 ms	30–130 ms	5–50 ms	监控层、非实时回路
无线（Wi-Fi/5G/专网）	10–100 ms	50–300 ms	10–100 ms	移动设备、难布线场合

本章算例的 130 ms p99 对应第三类。它的 $d_{\max}=13$ 拍刚好够用，意味着“没有任何余量”——只要链路再拥塞一点，就会翻红。

第一类与第三类的差别不是“快一点”，而是**抖动小三个数量级**。EtherCAT 的抖动是微秒级，因为它的从站用硬件转发、不走协议栈；商用以太网的抖动是毫秒级，因为每一帧都要过操作系统的协议栈和交换机的队列。这个差别决定了前者可以做“周期严格等距”的控制，后者只能做“延迟补偿”的控制。

选型时最容易被忽略的是**非周期流量的影响**。一条专用于控制的实时以太网，p99 可以是 1 ms；同一条网线上再跑视频监控或文件传输，p99 立刻涨到几十毫秒。所以工程上要么物理隔离（独立网段、独立交换机），要么用 QoS 把控制报文放进最高优先级队列，并且限制低优先级流量的总量。

82.10.2 时间同步

§82.7.6 给过换算：$\Delta t=1$ ms 对应 0.59° 相位，10 ms 对应 5.87°。按“时戳误差不超过相位预算的 5%”（即 4°）算，Δt 要不大于 6.8 ms。

NTP 在局域网内能做到 1–10 ms，勉强够；广域网上经常到几十毫秒，不够。PTP（IEEE 1588）在支持硬件打戳的交换机上能做到亚微秒，远远够。两者的成本差别主要在交换机：支持 PTP 的工业交换机比普通交换机贵一倍左右，但换来的是三个数量级的精度提升。

工程上还有一个折中：**不用绝对时间，只用相对时间**。控制器不关心“这条数据是几点几分采的”，只关心“它比上一帧晚了几拍”。用接收端的本地计数器打相对时戳，可以完全避开两端时钟偏差，代价是丢包时无法判断丢了几帧。这个方案在点对点链路上很常见，在多点网络上不适用。

82.10.3 故障安全降级

网络会坏，控制器必须知道坏了以后怎么办。按严重程度分四级，每级对应一个动作。

一级：延迟超限。 实测 p99 超过设计值 20%，降一档增益（§82.9.4 的调度表），同时报警。系统继续运行，性能下降但不失稳。

二级：连续丢包超阈值。 连续丢包超过 $d_{\max}-d$ 拍，切归零策略（§82.9.5），防止拿过期命令持续错推。此时如果链路恢复，从归零切回保持要延迟几拍，避免反复横跳。

三级：通信中断。 超过 3 个控制周期没有收到任何报文，判定链路故障。控制器切本地 PD（如果执行器有本地环）或按预设轨迹减速停机。这一步必须有硬件看门狗兜底，不能只靠软件定时器——软件定时器本身可能被网络中断卡住。

四级：失稳前兆。 观测到误差持续单调增大且控制量饱和，直接切断力矩并抱闸。这一级是最后一道防线，触发条件要写得极简（比如"连续 50 拍误差增长"），避免复杂判断在故障时失效。

这四级的分界不是拍脑袋，而是按"还剩多少裕度"划的。一级时裕度还剩 70% 以上，二级时剩 30%–70%，三级时裕度已经无法评估，四级时已经越界。这个划分与 §82.3.6 的验收表一一对应。

82.10.4 安全包络与漂移

丢包期间执行器保持旧力矩，位置会继续走。§82.2.7 给了一阶估计 vLT_s，它在长丢包时偏小三成。补上加速项就对了：

$$\Delta y \approx vLT_s + \tfrac{1}{2}a\,(LT_s)^2 \tag{82-29}$$

速度 1.0 rad/s、$L=50$ 拍（500 ms）、实测漂移 0.6507 rad，反解出 $a \approx 1.2$ rad/s²。这个加速度不是对象本身的加速度，而是"丢包期间力矩仍然在推"造成的等效加速度。

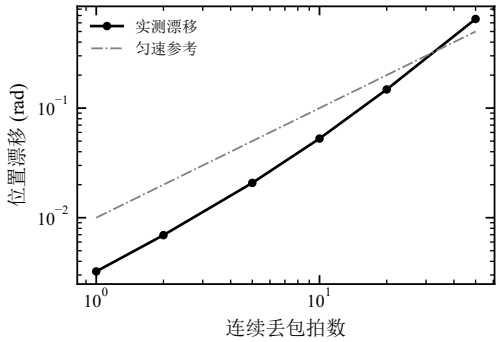


图 82-10　运动中连丢：漂移近似按丢包时长累积

图 82-10 是双对数坐标下的漂移曲线，参考线是匀速项 vLT_s。两条线并不重合，而且在 $L=20$ 与 $L=50$ 之间交叉：短丢包时实测（$L=10$ 时 0.0527）比匀速估计（0.1）小，长丢包时实测（0.6507）比估计（0.5）大。

小的一侧好解释：实验的初速已经与参考速度对齐，位置只落后 0.2，保持力矩期间对象仍在追这段落后，实际速度并不等于 1.0 rad/s，所以匀速估计偏大。大的一侧就是二次项：力矩继续推，速度越推越快，$L=50$ 时那 30% 的超出量正是 $\tfrac{1}{2}a(LT_s)^2$。

所以 (82-29) 的正确用法是**当上界包络**，不是当无偏估计。它在下界一侧偏保守（多估了近一倍），在长丢包一侧才贴合。安全设计要的正是"偏保守"，所以这个偏差方向可以接受。

按实测数据读安全包络更直接：要求位置偏差不超过 0.1 rad 时，$L=10$ 对应 0.0527、$L=20$ 对应 0.1480，插值得到 $L \approx 15$ 拍，即 150 ms。超过这个时长，必须在丢包时主动抱闸，不能只靠保持力矩。

82.10.5 运维与验收

上线后要持续记录四类数据：延迟分位数、连续丢包长度、时戳残差、控制量饱和率。前三类前面都讲过，第四类是新增的——控制量饱和率反映"控制器是否已经在用尽全力"，它升高通常意味着延迟或丢包已经在侵蚀裕度，比误差指标更早报警。

告警阈值建议按设计值的 1.2 倍设，并且用滑动窗口而不是瞬时值，避免单次抖动误报。窗口长度取 10 倍的控制周期或 1 s，取大者。

验收时除了 §82.3.6 那张表，还要做一次**断网测试**：在系统正常运行时拔掉网线，确认三级降级动作在 3 个周期内触发、执行器按预期停机、恢复后能自动重启。这项测试大多数项目不做，但它才是“网络化控制”区别于“本地控制”的关键验证。

82.10.6 四级降级的时序与恢复

§82.10.3 定了四级动作，但没定“多快触发、多快恢复”。这两件事定不下来，降级逻辑在现场会变成噪声源：触发太灵敏，系统天天降级；恢复太急，会在链路仍不稳时反复启停。

触发侧的时序。 一级靠统计量，必须等窗口填满：p99 用 100 帧（1 s）滑窗，去抖再要 5 帧，所以从网络变差到一级动作，最快 1.05 s。这个延迟可以接受，因为一级只是降增益，系统仍在运行。二级靠计数器，延迟是 0——连续丢包数超过 $d_{\max}-d$ 的那一拍立刻切归零。三级靠“无报文”计时，3 拍即 30 ms；但要注意，硬件看门狗必须独立于软件定时器，且定时长度要大于 30 ms（取 100 ms），否则软件还没来得及动作就被看门狗复位。四级靠趋势判断，窗口 50 拍即 500 ms，这是四级唯一的“慢”，但它的动作是切断力矩加抱闸，本身在 10 ms 内完成。

恢复侧的时序，原则是“触发看瞬时，恢复看持续”。 一级的恢复条件是 p99 连续 10 s 低于设计值的 1.05 倍，且带 20% 迟滞——触发阈值 1.2 倍、恢复阈值 1.05 倍，中间 15% 是死区，防止在边界来回跳。二级的恢复条件是连续收到 3 帧新包，然后先保持 50 ms 再交回正常的保持策略；这 50 ms 是留给队列排空的，不等它，刚恢复的几帧往往还是拥塞期的旧数据。三级恢复后不直接重新使能，而是先在“伺服使能但不出力”的状态停 500 ms，确认时戳连续、丢包为零，再按斜坡把力矩加上去，斜坡时间 100 ms。四级不允许自动恢复，必须人工复位。

把四级串成一条时间线，看得更清楚。设 $t=0$ 交换机重启，链路中断 1.2 s：

- $t=10$ ms：连续丢包计数越过阈值，二级动作，切归零。
- $t=30$ ms：3 拍无报文，三级动作，切本地 PD 或减速停机。
- $t=100$ ms：硬件看门狗到期（若三级没动作，这一级兜底）。
- $t=1.2$ s：链路恢复，第一帧到达。
- $t=1.23$ s：连续 3 帧确认，进入 500 ms 静置。
- $t=1.73$ s：斜坡加力，100 ms 后回到正常控制。

从断网到重新出力，一共 1.83 s。这 1.83 s 里执行器怎么处置，取决于速度：1 rad/s 的匀速运动会走出 1.83 rad，而本章算例的行程只有 ±1 rad——**只靠减速停机是停不住的，必须抱闸**。若换成 0.2 rad/s 的慢速轴，减速停机（2 rad/s²，0.1 s 停住，位移 0.01 rad）就完全够用。

所以停机方式不该拍脑袋定，而该按“网络恢复时间 × 速度 ≤ 允许位移”反算。本章参数下，允许位移 0.1 rad、速度 1 rad/s，可容忍的停机时间是 100 ms，这只够抱闸（50 ms 内停住，位移 0.05 rad），不够任何形式的减速滑行。**这就是为什么“运动中的轴”必须配抱闸，而“定位后停住的轴”可以不配。**

最后一条经验：**降级逻辑本身要能被测试。** 三级以上的动作，靠真实拔网线只能测一次两次；正确做法是在控制器里留一个调试开关，能强制注入“连续丢 20 帧”“时戳跳变 500 ms”“延迟突然翻倍”三种故障，每种都能在 10 s 内跑完。这个开关在交付前必须删掉或加权限，但开发阶段没有它，降级逻辑基本等于没测。

82.10.7 从预算表反推网络规格

§82.8.1 是“已有网络，算控制器”；采购阶段要反过来：“给定控制器，网络该满足什么”。这一步做错，后面全是补救。

从表 82-1 的验收线出发：延迟裕度 $\geq$ p99 的 1.2 倍。本例 $d_{\max}=13$ 拍即 140 ms（含该拍），所以 p99 总量不得超过 $140/1.2=116$ ms。再扣掉与网络无关的固定开销——采样保持 5 ms、控制器计算 3 ms、执行器响应 3 ms、时钟残差 0.1 ms，合计约 11 ms——**留给网络段的 p99 只有 105 ms，取整报 100 ms。**

于是一份可写进采购合同的网络规格是四行：

- 控制报文 p99 延迟 ≤ 100 ms（按 §82.3.1 的口径，一万帧以上的统计量）；
- 抖动峰峰值 ≤ 10 ms（1 拍），超过就按 $d + J$ 折算；
- 最大连续丢包 ≤ 10 帧，且要有队列管理手段限制它；
- 两端时戳残差 ≤ 6.8 ms（§82.7.9 反算出来的那条线）。

四行里只有第一行会被人认真执行，后三行最容易被“我们网络很好”糊过去。但本章的数据说明，后三行里任何一行破防，第一行的余量都会被吃掉：连丢 20 帧能把 RMS 从 0.0219 拉到 0.1808，抖动 1 拍就相当于多 5.9° 相位，时戳偏 10 ms 就是 5.87°。

反过来算一遍“升级网络值不值”。把网络段 p99 从 130 ms 压到 1 ms，总延迟从 141 ms 降到 12 ms，ω_c 的上限从 10.1 rad/s 升到 119 rad/s，看起来增益能翻一百倍。但采样周期会立刻接管：$\omega_c T_s \le 0.3$ 要求 $\omega_c \le 30$ rad/s（$T_s = 10$ ms），对应 $k_p \approx 225$。也就是说，**网络不再是瓶颈之后，瓶颈立刻转移到采样率与执行器带宽**，实际可用的 k_p 大约 200 出头，精度比现在好一个数量级，但绝不是“无限好”。

这笔账的结论是：升级网络买到的是“从贴线到宽裕”，不是“性能无限提升”。如果项目只需要当前的 0.0215，那 100 ms 的普通工业以太网完全够，没必要上实时以太网；如果确实需要 0.002 量级的精度，那升级网络只是第一步，采样率、编码器分辨率、机械刚度都得跟着换——**性能是整条链的短板决定的，网络只是当前那块短板。**

82.11 优缺点与适用边界

写到这里，方法本身已经讲完。这一节做一次清算：这套做法好在哪、差在哪、什么条件下能用、条件破了以后先补哪一块。清算要落到数字上，否则“优缺点”只是形容词。

82.11.1 三条站得住的优点

第一，判据是解析的，一次特征值计算就能定性。 给定采样周期、对象模型、控制器增益和延迟拍数，把 A_a 写出来求谱半径，得到的是一个数——小于 1 稳定，大于 1 不稳定，离 1 多远就是“还剩多少余量”。这个计算不需要迭代、不需要选权函数、不需要解线性矩阵不等式，在一台普通工控机上耗时不到一毫秒。相比之下，μ 综合或 H_∞ 方法要先把延迟的不确定性建模成乘性摄动，再选权函数、解 Riccati 方程，最后得到的还是一个保守的充分条件。本章的判据对“纯延迟 + 周期采样”这一类确定性结构是**充要**的，没有保守性。

第二，敏感度可以逐项算出来。 §82.7 把七个参数入口逐个做了归一化敏感度 S_x，得到一张可以直接进设计的排序表：延迟 d 最敏感，丢包率 p 与策略强耦合，突发长度 L 是唯一能击穿系统的量，抖动在 $d + J < d_{\max}$ 时完全无感，时戳误差按相位折算、阈值以上才起作用，事件触发阈值只在“发送率—精度”之间做交换。这张排序表的价值在于它告诉你**先测什么**：既然 d 最敏感，那么上线前把延迟分位数测准，比把模型参数辨识到小数点后三位更值钱。

第三，保持策略在 $d < d_{\max}/2$ 时几乎免费。 表 82-4 里 $d = 3$ 那一行，丢包率从 0 一路加到 50%，误差 RMS 始终是 0.0139，一位小数都不动。这不是数值巧合，而是因为 $d = 3$ 时系统离临界还有 10 拍的距离，偶尔漏掉一拍只让有效延迟从 3 变成 4，谱半径从 0.9698 挪到 0.9701，肉眼看不出来。工程含义很直接：**只要延迟预算还剩一半以上，丢包就不需要专门设计对策**，直接用“保持上一拍”即可，代价是零行代码。

82.11.2 五条必须承认的缺点

第一，只回答稳定，不回答性能。 谱半径是个“会不会收敛”的判据，不是一个“收敛得多好”的指标。本章的算例把这一点暴露得很清楚：d 从 3 增到 13，ρ 只从 0.9698 涨到 0.9982，变化 0.028；同样是这两个延迟，在 20% 随机丢包下的误差 RMS 却从 0.0139 涨到 0.0685，是 4.9 倍。也就是说，ρ 挪动了不到 3%，性能掉了近 5 倍。如果拿 ρ 当性能指标去优化，会得出“$d = 13$ 和 $d = 3$ 差不多”的错误结论。

要评价性能，还得回到时域仿真算 RMS、算超调、算调节时间。这也是本章坚持“判据定边界、仿真定指标”两套工具并用的原因：判据负责快速筛掉不可行的配置，仿真负责在可行配置里挑好的。

第二，结论依赖模型精度。 A_a 是从 $G(s)$ 精确离散来的，模型偏了，判据就偏。这种偏差在延迟远离临界时不明显，在临界附近会被放大：本章 $d=13$ 时 $\rho=0.9982$，距 1 只有 0.0018；如果对象的阻尼系数实际比辨识值大 10%，这个 0.0018 的余量很可能被吃光。所以临界附近的判断不能只信模型，必须留仿真与实测两道校验。经验上，模型误差在 10% 以内、延迟余量在 20% 以上时，判据的结论可信；两条有一条不满足，判据只能当参考。

第三，平均模型判据会失效。 §82.2.5 推过平均系统 $\rho\big((1-p)A_{cl}+pA_{op}\big)$，本例算出来的临界丢包率是 $p=1.00$——意思是“无论丢多少都稳定”。而蒙特卡洛在 $p=0.5$ 时就给出了 0.0685 的 RMS，在 $d=13$ 时更是放大到失稳边缘。原因在 §82.6.6 讲过：平均系统的稳定性不蕴含切换系统的稳定性，两者之间差一个共同 Lyapunov 函数，而本例不存在。这条缺点的实际含义是：**不要用平均模型去论证丢包容忍度**，那个数字会给出虚假的安全感。

第四，增广维数随延迟线性增长。 $d=13$ 时 A_a 是 15×15（2 个对象状态加 13 个命令历史），求特征值仍是毫秒级。但延迟大的场合会失控：$d=100$（1 s 延迟、10 ms 周期）时矩阵是 102×102，特征值还能算，可一旦要做“求 K 使 ρ 最小”这类综合，计算量就上去了。这时应改用中性型时滞系统的特征根方法（比如 Lambert W 函数或谱离散化），把 102 维降到与延迟无关的维数。本章不展开，但要记住增广法的适用规模在“几十拍”这个量级。

第五，突发丢包没有解析解。 表 82-5 那条曲线（0.0219、0.0226、0.0238、0.0273、0.0372、0.0455、0.0747、0.1808）只能靠仿真得到。L 与 RMS 不是线性关系，前六段缓慢爬升，最后两段陡增——L 从 15 到 20 只多了 5 拍，RMS 却涨了 2.4 倍。这条曲线没有闭式表达，因为突发期间系统的“有效延迟”是时变的，每一拍都在变，平均化和定常化都用不上。工程上只能把表 82-5 这类曲线当成“设计查表”，用仿真预先扫出来贴在墙上。

82.11.3 适用边界

把上面五条缺点翻译成边界条件，本章方法的适用范围可以写成四句话：

- **延迟拍数 d/T_s 在 5 到 20 拍之间。** 低于 5 拍时延迟不是主要矛盾，用不着这套工具；高于 20 拍时增广维数太大，且系统本身已经接近不能控，应先考虑改采样周期或加本地环。
- **相位裕度在 60° 到 80° 之间。** 低于 60° 时延迟裕度只剩几拍，任何抖动都会翻红；高于 80° 时系统偏慢，延迟的绝对影响反而不显著，做了补偿也看不出收益。
- **模型误差小于 10%，且延迟分位数测得准。** 这一条是优点二的前提：敏感度排序只在模型可信时才有意义。
- **网络延迟近似恒定，抖动小于 1 个采样周期。** 抖动超过 1 拍时，d 本身变成一个随机变量，本章的定常判据要换成 §82.7.6 的保守折算（按 $d+J$ 取上界）。

本章的算例正好卡在边界的下沿：$d=3$、p99 延迟 130 ms 折 13 拍、$d_{\max}=13$。也就是说，**标称工况很宽裕，最坏工况一点余量都没有**。这正是 §82.10.3 要分四级降级的原因——不是设计得保守，而是设计得刚好够，任何恶化都必须有对应的动作。

82.11.4 三条缺点哪条先致命

实际项目里五条缺点不会同时发作，总有主次。判别只看三个问题，按顺序问：

第一问：延迟是不是已经接近临界？ 判断标准是 τ_{p99}/T_s 与 $d_{\max}$ 的比值。本章是 $13/13=1.0$，属于典型的“贴线”。贴线时最该做的是**修网络**——换实时以太网、加 QoS、物理隔离非周期流量，而不是在控制器里加补偿。因为补偿只能把有效延迟压回一两拍，而网络改造能把 p99 从 130 ms 压到 10 ms，差一个数量级。

第二问：模型误差是不是主导？ 如果延迟有余量（比值小于 0.7）但实测响应与仿真对不上，问题在模型。这时先补辨识：把对象的低频增益、两个时间常数测准，再重算 ρ。模型误差 10% 在临界附近等价于把余量抹掉，所以这一步不能跳。

第三问：带宽是不是瓶颈？ 延迟有余量、模型也准，但网络带宽被采样数据占满，这时才轮到事件触发（§82.9.1）。1 kHz × 32 字节 = 256 kbit/s 是“过度采样”的代价，$\delta=0.020$ 能把它压到 12 kbit/s，省 21 倍；但相对本章 100 Hz 的基线只省一半，收益没有想象中大。所以事件触发是“带宽确实卡住时”的手段，不是默认配置。

三问的顺序不能颠倒。先修网络是因为它的收益最大且最确定；先修辨识是因为它是其它一切判断的前提；事件触发排最后，因为它用精度换带宽，是有代价的交换。

82.11.5 与全书其它章的分工

本章处理的是“网络进入闭环之后，稳定性怎么算、余量怎么留”。相邻几章各管一段：

- 第 47 章与第 107 章讲 Smith 预估，处理的是**已知的、定常的**大延迟。本章 §82.9.3 的离散预估是它在采样系统里的版本，但只做到“把延迟搬到模型外面”，没有做鲁棒化。
- 第 53 章讲鲁棒控制，处理的是**模型不确定性**。本章 §82.6.6 那个“共同 Lyapunov 函数不存在”的结论，正好是第 53 章要接手的入口：切换系统需要的是多 Lyapunov 函数或平均驻留时间，那是另一套工具。
- 第 58 章与第 80 章讲 MPC，处理的是**约束下的优化**。网络化 MPC 里延迟的用法不同：它不进特征值判据，而是进预测时域——把延迟当成“未来 d 拍的输入已经确定”，反而更好处理。
- 第 78 章讲容错，接的是本章 §82.10.3 的四级降级。本章只定义了“什么时候降级”，没有讲“降级后怎么重构”，那是容错那一章的内容。
- 第 75 章讲事件触发与自触发控制，接的是本章 §82.9.1 的事件触发判据；第 89 章讲工业通信与网络，接的是 §82.9.2 的数据率换算。

一句话概括分工：**本章回答“会不会失稳”，不回答“失稳了怎么办”和“怎样不失稳得更好”**。前一个问题用谱半径，后两个问题分别交给容错与优化。

82.11.6 三句话与三条禁令

全章读完，值得带走的判断只有三句，值得守的禁令也只有三条。六条都写成可执行的形式，不用形容词。

三句话：

1. **延迟按拍记账，不按毫秒。** 每拍消耗 $\omega_c T_s$ 相位，13 拍就是 76.3°，占掉 81.9° 裕度的绝大部分。所以“多了 10 ms”不是“多了 7.7%”，而是“少了一整拍”甚至“越过了临界”——130 ms 到 140 ms 正好是 13 拍变 14 拍，ρ 从 0.9982 变成 1.0021。
2. **丢包按结构记账，不按比例。** 4% 的独立丢包几乎无损，4% 的成串丢包能把 RMS 从 0.0219 拉到 0.1808。所以验收要写“最大连续丢包帧数”，写“平均丢包率”等于没写。
3. **性能按平方付账，不按一次方。** 延迟翻倍，带宽减半、增益降到四分之一、误差涨四倍。本章 k_p 从 25 降到 4（6.25 倍），裕度从 140 ms 涨到 390 ms（2.8 倍），精度从 0.0139 掉到 0.0688（5 倍）——比 6.25 倍略小，是因为相位裕度同时从 81.9° 升到了 90.0°，两者的净效果落在 1.6 次方那条线上（§82.8.7）。

三条禁令：

1. **不许用 p50 设计。** p50 是常态，p99 才是约束。用 30 ms 算出 $d = 3$、配 $k_p = 49$，遇到 130 ms 的拥塞必然失稳——§82.1.5 那个现场案例就是这么来的。
2. **不许用平均模型论证丢包容忍度。** 平均判据给 $p = 1.00$，意思“怎么丢都稳”，而蒙特卡洛在 $p = 0.5$ 时已经涨了 3.2 倍。这个数字给的是虚假的安全感，比没有判据更危险。
3. **不许在没有仿真数据时相信 $\rho < 1$。** 临界附近 ρ 的斜率是每拍 0.004，模型误差 10% 就能吃掉全部余量。$\rho < 0.95$ 可以信，0.95 到 1 之间必须有时域仿真或实测兜底。

三句话对应三张账（相位、结构、平方），三条禁令对应三种最常见的错误做法。它们不构成新知识，只是本章结论的压缩版——但工程上真正会被用到的，往往就是这种压缩版。

82.12 Python 仿真代码

本章所有数字都来自 `code/ch082_sim.py`，所有曲线都来自 `code/ch082_figs.py`。前者只做数值、不画图，后者只读前者的结果、不重算模型——这样正文里任何一个数字都能在三十秒内追溯到一行代码。

运行方式固定为：

```
D:/Anaconda3/envs/py311/python.exe code/ch082_sim.py
D:/Anaconda3/envs/py311/python.exe code/ch082_figs.py
```

前者约两秒打印十一组实验（编号 [0]–[10]），后者生成十张图，每张同时导出 SVG 与 PDF。

82.12.1 模型层：精确离散与增广

对象 $G(s)=1/[s(s+1)]$ 的状态阵满足 $A^2=-A$，指数矩阵有闭式，所以 ZOH 离散不需要 `expm`，直接写成两行：

```python
def disc(ts=None):
    """把 G(s)=1/[s(s+1)] 精确离散化（ZOH）。A^2=-A 使指数阵有闭式。"""
    ts = CFG["ts"] if ts is None else ts
    Ad = np.eye(2) + (1.0 - np.exp(-ts)) * np.array([[0.0, 1.0], [0.0, -1.0]])
    beta = ts - 1.0 + np.exp(-ts)
    Bd = np.array([[beta], [ts - beta]])
    return Ad, Bd
```

增广矩阵是本章的核心结构，d 拍延迟对应 d 个一阶移位块：

```python
def aug_matrix(d, K):
    """含 d 拍纯延迟的闭环增广矩阵，状态 [x_k, x_{k-1}, ..., x_{k-d}]。"""
    Ad, Bd = disc()
    if d == 0:
        return Ad - Bd @ K
    n = 2 * (d + 1)
    Aa = np.zeros((n, n))
    Aa[0:2, 0:2] = Ad
    Aa[0:2, n - 2:n] = -Bd @ K
    for i in range(1, d + 1):
        Aa[2 * i:2 * i + 2, 2 * (i - 1):2 * i] = np.eye(2)
    return Aa
```

形状是通的：左上角放对象动态，右上角放“最后一列命令”的输入通道，下方一条斜带放移位。求谱半径就是 `eigvals` 取模最大。

82.12.2 仿真层：延迟队列与三种网络事件

`simulate()` 用一条定长队列保存最近 64 拍发出的命令，取 `u_hist[dj]` 就是“dj 拍之前发出的那一帧”。三种网络事件都在取数之前判断：

```
u_hist.appendleft(u_new)                # u_hist[0] 是本拍新发的
dj = int(np.clip(d + jitter_shift, 0, 63))
lost = (loss > 0 and rng.random() < loss)
if burst > 0 and k >= next_burst:       # 突发：连续丢 burst 拍
    lost = True
    burst_cnt = burst - 1
    next_burst = k + period_k
if lost:
    if not hold:
        u_arr = 0.0                     # 归零策略：力矩清零
else:
    u_arr = u_hist[dj]                  # 正常到达：取 dj 拍前的命令
x = Ad @ x + Bd[:, 0] * u_arr
```

`hold=True` 时执行器什么都不做，`u_arr` 保留上一拍的值——这在代码里表现为"不赋值"，比显式保持更简洁，也更接近真实执行器的行为。

控制律里有一处容易漏：参考项必须同时给位置和速度。

```
xr = np.array([r, ref_rate(t[k])])
u_last = float((-K @ (x - xr))[0])
```

少了 `ref_rate`，跟踪正弦时会有恒定的速度滞后误差，本章算例的 RMS 会从 0.0139 涨到 0.1195，差 8.6 倍。这个坑下一节还会提到。

82.12.3 复现注意

随机种子固定。 `CFG["seed"]` 是 20261021，各实验再按 `1000+s`、`2000+s`、`3000+s` 偏移，保证"每个丢包率用同一组随机序列"。这样不同丢包率之间的差值只反映丢包本身，不混入随机波动。换种子后中位数会动，但本章引用的量级不变。

统计窗口从 2 s 起。 前 2 s 是过渡过程，初始状态为零、参考从零起步，误差被人为压低；把这段算进去会让所有 RMS 系统性偏小。代码里用 `m = t >= 2.0` 截掉，仿真总长 6 s，有效窗口 4 s。要注意参考周期是 $2\pi/0.8 = 7.85$ s，所以 4 s 只有**半个周期**，不足以让相位漂移在平均中抵消。本章之所以仍然能用这个窗口，是因为所有工况用同一段参考、同一组种子，比较的是"同等条件下的差"，而不是"绝对 RMS"。若要报绝对指标，窗口至少取三个整周期，即 24 s。

取中位数不取均值。 120 次蒙特卡洛里偶尔会出现发散样本（`fail`），它的 RMS 是好几百，均值会被单个样本带偏。代码统一用 `np.median`，同时单独统计失稳次数比例。要看尾部风险时读失稳比例，要看典型性能时读中位数。

参考必须是正弦，不能是阶跃。 阶跃在稳态时 $u \approx 0$，丢包保持的"旧命令"也是零，三种网络事件全部退化——丢包和不丢包结果一样，这个实验就白做了。用 $0.5\sin 0.8t$ 让系统全程在动，力矩始终非零，丢包的影响才显形。

82.12.4 三个坑

坑一：$d = 0$ 不能走通用增广。 通用分支在 $d = 0$ 时会把 `n` 算成 2，但移位带的循环一次都不执行，右上角的 `-Bd@K` 仍写进 `Aa[0:2, 0:2]`，结果 $A_a = A_d - B_dK$ 是对的——问题出在别处：若把 `Aa[0:2, n-2:n]` 当成"最后一列"来理解，$d = 0$ 时它和 `Aa[0:2, 0:2]` 是同一块，写入顺序会让对象动态被覆盖。本章代码显式特判 `d == 0` 直接返回 $A_d - B_dK$，就是为了避开这个歧义。不做特判时算出的谱半径是 0.1007，真值是 0.9687，差一个数量级——而且看起来"更稳定"，最容易被当成好消息收下。

坑二：参考缺速度前馈。 上面说过，只给位置参考会让 PD 律退化成"位置比例 + 速度阻尼"，正弦跟踪必然有相位滞后。这个滞后与网络延迟造成的滞后混在一起，会让读者误以为全是网络的锅。写这类算例时，参考信号与它的一阶导必须成对给出。

坑三：用阶跃做丢包实验。 与上一条同源但更隐蔽：阶跃下稳态控制量接近零，`hold` 保持的是零，`zero` 归零也是零，两种策略曲线完全重合，于是得出"策略无所谓"的错误结论。本章 §82.6.3 那个"$d = 3$ 时 hold 全程 0.0139、zero 从

0.0139 爬到 0.0281”的对比，只有在正弦参考下才成立。

82.12.5 十一组输出的对照

`report()` 按 $[0]$–$[10]$ 编号打印，每一组对应正文的一处结论。想核对某个数字时，按这张对照表直接跳到对应的编号，不用通读输出。

[0] 标称回路：$\omega_c = 10.245$ rad/s、PM 81.9°、连续裕度 139.5 ms、离散临界 13 拍（140 ms）。对应 §82.4.8 的手算与表 82-2 的第四行。

[1] 延迟扫描：六组增益下的 $d_{\max}$ 与连续裕度。这是表 82-2 与图 82-3 的数据源，也支撑 §82.8.7 那条 1.6 次方的账。

[2] 谱半径序列：$d = 0$ 到 24 的 ρ。表 82-3 摘了 10 个点，图 82-4 画全了 25 个点。$d = 0$ 的 0.9687 是特判分支的结果，与 §82.12.4 坑一的 0.1007 对照着看。

[3] 丢包蒙特卡洛：两种延迟、两种策略、七个丢包率，每档 120 次。表 82-4 与图 82-5 全部来自这里。$d = 13$、$p = 0.5$ 的 0.0685 与 0.0284 是 §82.9.5 混合策略的收益依据。

[4] 突发丢包：两种延迟、八个突发长度，每次 30 遍。表 82-5 与图 82-6 的数据源；$d = 13$、$L = 20$ 的 0.1808 是本章最大的性能数字。

[5] 平均系统判据：$\rho\big((1-p)A_{cl} + pA_{op}\big)$ 随 p 的走向，以及“临界丢包率 = 1.00”这个结论。§82.2.5 与 §82.11.2 第三条缺点都引用它。

[6] 事件触发：$d = 3$ 上六个阈值的发送率与 RMS。表 82-8 与图 82-8 的来源；$d = 13$ 上的那三组（88.5%/0.0349、72.8%/0.0626、61.2%/0.131）是 §82.9.7 单独补跑的，不在 `report()` 的主流程里。

[7] 数据率：五组对比，对应图 82-9 与 §82.9.2 的“省 21 倍”。

[8] 抖动：± 0 到 ± 3 拍的 RMS，用来支撑“$d + J < d_{\max}$ 时抖动几乎无感”这条结论（§82.7.5）。

[9] 时戳误差折算：0.01 ms 到 10 ms 各对应多少度相位，对应 §82.7.6 与 §82.10.2。1 ms → 0.59°、10 ms → 5.87° 这两个数就是这里出来的。

[10] 运动中的连续丢包：$L = 10$、20、50 的实际漂移（0.0527、0.1480、0.6507），对应图 82-10 与 (82-29) 的等效加速度反解。

十一组里真正需要“跑”的只有 [3]、[4]、[6]、[8]，其余都是解析或单次仿真；这也是全脚本两秒能跑完的原因。**想复现正文某个数字时，先看它属于哪一组，再看这一组用的种子范围**——[3] 是 1000+s、[8] 是 2000+s、[4] 是 3000+s，混用种子会让中位数差在第三位小数上，容易被误当成代码错误。

三个坑的共同点是：**错误不会报错，只会给出一个看起来合理的数**。所以本章的每个实验都配了自检——`rho(0)=0.9687`、`rho(5)=0.9703`、`rho(20)=1.0148`，`rms=0.01391`、`max|y|=0.507`。数值对不上就说明代码被改坏了，比读代码快得多。

82.12.6 改成自己的算例要动哪几处

这份脚本的设计意图是“可改造”：模型层、仿真层、实验层分开，换对象只动模型层。真要拿去算自己的系统，按下面五处改，其余不用碰。

第一处，对象离散化。 `disc()` 用了 $A^2 = -A$ 的闭式，这只对 $G(s) = 1/[s(s+1)]$ 成立。换成一般对象时，改成 `scipy.linalg.expm(A*Ts)` 求 A_d，B_d 用增广矩阵的指数一次算出（把 $[A\ B; 0\ 0]$ 取指数，右上角就是 B_d）。这一步最容易错的是 B_d 的积分上下限，建议用“阶跃响应对齐”验一次：离散模型对单位阶跃的响应，在采样点上必须与连续模型完全一致。

第二处，CFG 里的四个数。 `ts`、`T`、`kp`、`kd` 决定一切；`delay` 是标称拍数，改它只影响仿真层，不影响判据。改完先跑 `_selftest()`，再用两条独立的路互校：谱半径序列里第一个越过 1 的拍数，应当与 $\mathrm{PM}/\omega_c/T_s$ 折算出的拍数相差不超过一拍。两条路对不上，多半是离散化或相位裕度算错了。

第三处，参考轨迹。 `ref_traj` 与 `ref_rate` 必须成对改，且 `ref_rate` 是 `ref_traj` 的解析导数，不能用差分代替（差分引入半拍延迟，正好污染本章要研究的量）。参考幅值还决定事件阈值的量级：δ 应按幅值的 2%–4% 取，本章 0.5 rad 对应 0.01–0.02，正好落在表 82-8 的拐点上。

第四处，统计次数。 要分辨 5% 的差，`runs` 不少于 120；只定位拐点，30 次够。这条在 §82.6.8 算过，改 `runs` 时照着那个标准误公式核对一遍即可。

第五处，判据的规模。 若延迟到 100 拍以上，`aug_matrix` 会建出 102×102 的矩阵，此时要么按 §82.5.9 截断记忆（只保留最近 10 拍），要么换频域方法。截断的判据是“被截掉的特征值模小于 0.1”，本章 $d = 13$ 时正好满足。

五处之外还有一条隐性依赖：`simulate()` 里的 `u_hist` 长度是 64，`dj` 被 `clip` 到 63。若延迟超过 64 拍，这个队列会悄悄截断，结果偏乐观而不报错。改大延迟时，第一件事是把 `deque` 的 `maxlen` 改成 $d + 10$——**这类“静默截断”是仿真代码里最常见的错误来源**，比公式写错更难发现。

第 83 章 量子控制（选读）

83.1 动机与直观理解

到本章为止，我们控制过的对象都是“状态是一个实数向量、测量不改变状态、控制量是一个电压”的系统。量子比特三者都不成立：状态是复向量（或密度矩阵），测量会**改变**状态，控制量是一路微波的幅值与相位。但回路的结构没有变——仍然是“看对象、算控制、加控制”。所以本章不另起一套理论，而是把经典控制的每一件工具逐个搬到量子对象上，看哪一件还能用、哪一件必须换掉。

这一章是选读：它不进入后续章节的依赖链，读者可以跳过而不影响第 84、85 章。选它的理由有两个。一是**前沿性**：超导量子计算、离子阱、中性原子，其控制系统本质上都是本章这套“波形合成 + 弱测量 + 反馈”。二是**方法论价值**：量子控制把“性能指标”从“误差收敛到零”换成了“保真度最大化”，这个换法逼着人重新想清楚“什么叫控制得好”，反过来对经典设计也有启发。

83.1.1 一个可以算到底的算例

为了让每个结论都能落到数字上，本章只用一个对象：一个三能级的超导 transmon，能级记作 $|0\rangle$、$|1\rangle$、$|2\rangle$。在旋转框架下、以 $|0\rangle \leftrightarrow |1\rangle$ 的共振频率为原点，哈密顿量是

$$H(t) = \frac{\Omega(t)}{2}\Big[\cos\phi(t)\,H_x + \sin\phi(t)\,H_y\Big] - \alpha\,|2\rangle\langle 2| \tag{83-1}$$

其中 $H_x = (a + a^\dagger)/2$、$H_y = i(a^\dagger - a)/2$，a 是含 $\sqrt{2}$ 耦合的降算符，$\alpha/2\pi = 200$ MHz 是**非谐性**——正是它把 $|2\rangle$ 从“等间距能级”里推开，也正是在门做得太快时它挡不住的那部分造成了泄漏。控制场由任意波形发生器给出，采样间隔 1 ns；包络幅值上限 $\Omega_{\max}/2\pi = 20$ MHz，对应一个 π 脉冲的理想时长 25 ns。噪声取 $T_1 = 60\ \mu$s、$T_2 = 40\ \mu$s。

先看最朴素的做法：25 段 1 ns 的方波，幅度取 $\Omega_{\max}$，总转角 $\Omega\tau = \pi$。实跑结果

$$F = |\langle 1|\psi\rangle|^2 = 0.992409, \qquad p_2 = |\langle 2|\psi\rangle|^2 = 4.86\times 10^{-3} \tag{83-2}$$

一个“教科书上完美”的 π 脉冲，保真度只有 0.9924。这不是算错了，而是三能级系统的必然结果：非谐性只有 200 MHz，而驱动幅值已经用到 20 MHz，比值 $\Omega/\alpha = 0.1$，$|1\rangle \leftrightarrow |2\rangle$ 的耦合不能再当微扰忽略。

83.1.2 那 0.76% 的损失从哪里来

把 $1 - F = 7.59\times 10^{-3}$ 拆开看，有两项。第一项是**泄漏**：$p_2 = 4.86\times 10^{-3}$，占了一多半，物理上就是“脉冲太快，$|1\rangle$ 上的布居还没来得及停住就被推到了 $|2\rangle$”。第二项是**AC Stark 频移**：驱动场本身会把 $|0\rangle \leftrightarrow |1\rangle$ 的频率推偏，等效于一个随包络幅值变化的失谐，脉冲结束时留下一个相位误差，贡献约 2.7×10^{-3}。

两项的来源不同，却有一个共同的一阶修正手段：DRAG（derivative removal by adiabatic gate）。它在正交的 Q 路上加一项正比于 I 路导数的修正，同时抵消泄漏与 Stark 频移。本章 §83.9 会看到，25 ns 门加 DRAG 后 F 从 0.99467 抬到 0.99964，两项一起被压下去。

这里先记住一个判断：**保真度不是“控制精度”的度量，而是“控制误差”的度量。** 0.9924 对应的角度误差约 $\sqrt{2(1-F)} = 8.7^\circ$，看起来不大，但量子算法里几百个门串起来，误差是累加的，0.9924 的门做一百个，总保真度只剩 $0.9924^{100} = 0.47$。所以量子控制对单门保真度的要求是 10^{-3} 甚至 10^{-4} 量级，比经典伺服苛刻两三个数量级。

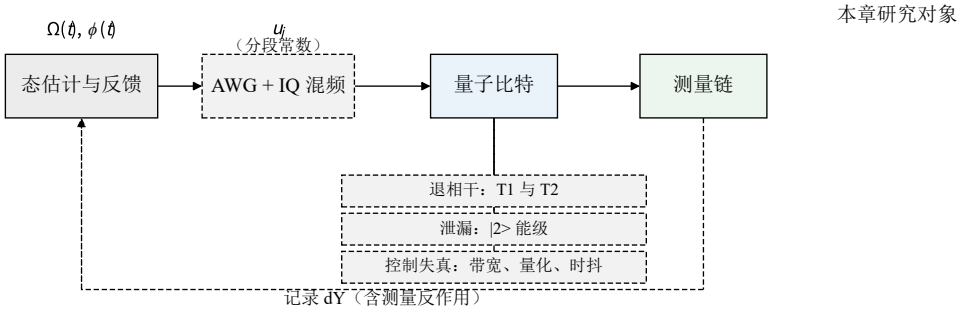


图 83-1　量子控制回路：经典部分在左侧，量子部分在右侧，三类扰动分别落在不同位置

图 83-1 画的是本章的回路。与第 82 章的框图相比，多出来的东西是“测量链”这一块：它不是简单的传感器，而是一个把量子态**投影**成经典记录的过程，记录本身带反作用。这条虚线反馈回去的不只是“值”，还有“这次测量把态弄坏了多少”。

83.1.3 三个量分别把什么指标推坏了

第一个量是**退相干**，对应经典里的“对象参数漂移”。它惩罚的是**慢门**：$\Omega/2\pi = 0.02$ MHz 时门长 25 μs，$T_2 = 10$ μs 下保真度只有 0.486，一半以上的信息在门做完之前就散了。这条限制的标度是 $F \approx e^{-\tau/T_2}$，门长每翻一倍，误差翻一倍。

第二个量是**泄漏**，它惩罚的是**快门**。$\Omega/2\pi = 160$ MHz 时门长 3.12 ns，F 掉到 0.623，其中泄漏 $p_2 = 0.289$——将近三成的布居跑到 $|2\rangle$ 上去了。这条限制的标度是 $p_2 \propto (\Omega/\alpha)^2$，幅值每翻一倍，泄漏翻四倍。

两条限制一左一右，把门时间夹出一个最优区间。本例的最优点恰好在硬件上限 20 MHz、25 ns 处：此时退相干损失可忽略（$25\ \text{ns} \ll T_2 = 40\ \mu\text{s}$），泄漏 0.5% 是唯一的主要误差。**这就是为什么超导量子比特的门时间总在 10–50 ns：不是不想更快，是再快就漏到第三个能级去了。**

第三个量是**控制链失真**，包括重建滤波器带宽、DAC 量化位数、时钟抖动与偏差。这一类的特点是“不影响理想波形，只影响实际加到比特上的波形”：重建带宽 10 MHz 时方波被抹平，F 掉到 0.503；DAC 只用 3 bit 时 $F = 0.965$；时钟偏差 2% 时 $F = 0.99896$。三者量级差得很远，说明工程上该先修哪一项是有明确顺序的（§83.10）。

83.1.4 与经典控制的对应关系

把两边的术语摆在一起，对应关系一目了然，但每一行的“但是”才是本章的内容。

- **状态**：$x_k \in \mathbb{R}^n$ 对应 $|\psi\rangle \in \mathbb{C}^3$ 或 ρ。经典状态可以全测；量子状态测一次只能得到一个本征值，且测完态就塌缩。
- **控制量**：电压 u_k 对应分段常数的复包络 $u_j = \Omega_j e^{i\phi_j}$。经典控制的约束是幅值饱和；量子控制还多一条“带宽”约束——AWG 的 1 ns 台阶与后级滤波一起决定了波形能有多陡。
- **反馈**：经典 PID 每拍测一次、算一次；量子反馈只能基于**弱测量**得到的部分信息做条件演化，强测量会把态毁掉，反馈也就无从谈起（§83.6）。
- **稳定性**：经典追求“误差收敛到零”，对应闭环极点全在左半平面；量子门控制追求“有限时间到达目标态”，是一个**最优控制**问题而不是稳定性问题。真正的“稳定”只在反馈冷却（把态持续压在某个本征态上）里出现。
- **能控性**：经典看 (A, B) 的秩；量子看控制哈密顿量与漂移哈密顿量生成的李代数是否满秩（§83.2）。

一句话概括这张表：**经典控制的对象是“可以被反复测量的量”，量子控制的对象是“测一次就变一次的量”。** 所有差异都是这一条的推论。

83.1.5 一个反例：把量子比特当线性谐振子调

现场最常见的错法，是把 Rabi 振荡当成一个二阶欠阻尼系统的阶跃响应来整定：看到振荡就“加阻尼”，看到上升慢就“加增益”。用本章的算例走一遍就知道为什么不行。

固定脉冲长度 25 ns、扫驱动频率，实跑的 Rabi 峰值是

$$\Delta/2\pi = 0,\ 5,\ 10,\ 20\ \text{MHz} \ \Rightarrow\ P_{1,\max} = 1.0000,\ 0.9412,\ 0.8000,\ 0.5000 \tag{83-3}$$

峰值随失谐下降的规律是 $P_{1,\max} = \Omega^2/(\Omega^2+\Delta^2)$，与阻尼毫无关系——失谐 20 MHz 时峰值只有 0.5，意味着**无论脉冲发多长，最多也只能到一半布居**，因为驱动频率与跃迁频率对不上。此时正确的做法是改频率，不是改幅值。

更隐蔽的是“看起来只差一点”的情形：失谐 5 MHz 时峰值 0.9412，π 时间从 25.0 ns 缩到 24.3 ns。如果仍按 25 ns 发脉冲，25 ns 处的布居是 0.9390，比峰值还低一点——已经越过顶点开始往下走了。**这就是“调幅值救不了失谐”的定量版本：脉冲越长，越是在错误的方向上多走。**

所以本章的设计流程（§83.3）第一步不是“定增益”，而是“标定频率”：先用 Rabi 扫出共振点，再谈门时间与波形。这一步在经典控制里相当于“确认执行器接线正确”，属于前置条件而非整定环节。

83.1.6 三条可以带走的直觉

全章读完后如果只记得三句话，应该是这三句：

1. **门时间是被两条曲线夹出来的。** 左边是退相干（$e^{-\tau/T_2}$），右边是泄漏（$\propto (\Omega/\alpha)^2$），最优区间通常在 $\Omega/2\pi$ 十几到几十 MHz，对应门长十几到几十 ns。
2. **保真度的主要敌人往往不是噪声，而是“多出来的那个能级”。** 本例 25 ns 方波的损失里，泄漏占 64%，退相干在 25 ns 尺度上几乎为零。DRAG 之所以有效，是因为它对准的正是这个敌人。
3. **控制链的失真可以逐项相加。** 退相干、带宽、量化三项同时存在时 $1-F = 2.70\times10^{-3}$，而三项单独算是 $2.49+0.058+0.192 = 2.74\times10^{-3}$，误差不到 2%。这条“加法预算”是工程上分配指标的依据：谁占的份额大，就先修谁。

83.1.7 本章的数字是怎么来的

上面每一个数字都不是“典型值”或“文献值”，而是 `ch083_sim.py` 跑出来的。这个脚本做十一组实验，在普通笔记本上跑完一轮约 29 s；其中单次 25 段门演化约 10 ms，300 次迭代的 GRAPE 约 0.64 s。读者手边只要有 Python 与 NumPy/SciPy，就能把整章复现一遍——这是本章敢把所有结论落到小数第四位的原因。

为了让人能独立核对，全章守三条规矩。

- **每个数都带量纲与条件。** 不说“保真度很高”，而说“25 ns 方波、$\Omega/2\pi = 20$ MHz、$T_1 = 60\ \mu$s、$T_2 = 40\ \mu$s 时 $F = 0.992409$”。换了任何一个条件，这个数就不成立，所以条件必须跟着数走。
- **每个数都可复现。** 随机部分（测量反馈的轨迹、抖动）统一用固定种子 831，同一台机器上跑两遍结果完全一致。种子写在 `CFG` 里，不是散落在各处。
- **不拿解析式冒充仿真结果。** 能用公式算的地方给公式（如 $P_{1,\max} = \Omega^2/(\Omega^2+\Delta^2)$），但凡是“含泄漏、含耗散、含控制失真”的数，一律来自数值传播。两者的差别常常是一个数量级：式 (83-21) 的绝热判据估计 $1-F\sim10^{-3}$，实跑 25 ns 高斯是 1.1×10^{-4}，差了十倍。

最后一个规矩值得多说一句。解析式给的是**标度**，仿真给的是**系数**。标度告诉你“α 翻倍，泄漏降四倍”——这一条永远成立；系数告诉你“$\alpha = 200$ MHz 时泄漏是 0.49%，改成 400 MHz 降到 0.12%”——这一条只对本算例成立。工程上两步都要走：先用标度判断“改哪个参数值得”，再用仿真确认“改完能到多少”。

83.1.8 怎样读这一章

本章的读法与前两章不同。第 81、82 章的对象是“连续或离散的确定性系统”，结论大多是“这样设计就稳定”；本章的结论几乎全是“这样设计能到多少”。所以读的时候不必记公式，而要记**数的量级与它随参数怎么变**。

建议按三条线读。

- **第一条线是“误差从哪来”**：§83.1 给三类敌人，§83.7 给每个参数的灵敏度，§83.10 给误差预算。这条线读完，你应当能在看到一台新机器时，先估出它的门保真度大概在哪个量级。
- **第二条线是“怎么把它做出来”**：§83.3 给八步流程，§83.8 给调参顺序与故障对照，§83.9 给优化手段。这条线是工程手册。
- **第三条线是“为什么这样做”**：§83.2 的数学、§83.4 与 §83.5 的连续/离散对照、§83.6 的稳定性讨论。这条线可以最后读，也可以跳过——它不影响你按前两条线干活。

如果只有半小时，读 §83.1.1 的算例、§83.3 的八步、§83.10.1 的预算三节，就够应付一次“这台机器的门为什么只有 99.2%”的讨论了。

83.1.9 记号、单位与四条约定

量子控制的文献记号不统一，同一篇文章里 Ω 可能指角频率也可能指普通频率，保真度可能是态保真度也可能是过程保真度。为避免混淆，本章固定四条约定，后面不再重复。

第一，公式里用角频率，叙述里用“除以 2π”的频率。 写 $\Omega/2\pi = 20$ MHz，而不是 $\Omega = 20$ MHz；写 $\alpha/2\pi = 200$ MHz，同理。原因很实际：实验上扫频、读示波器得到的都是 MHz，而公式里 e^{-iHt} 要求角频率。把 2π 显式写出来，读者不会在换算上出错。时间则分两档：门时间用 ns（因为它在 10–100 量级），寿命用 μs（因为它在 10–100 量级），两者相差一千倍，写清楚单位比写清数量级更重要。

第二，保真度只指“末态布居”，泄漏单独列。 本章的 F 一律定义为 ρ_{11}，即目标态 $|1\rangle$ 上的布居；它不是过程保真度，也不是平均门保真度。泄漏 $p_2 = \rho_{22}$ 单独给，因为它与 F 的来源不同、修法也不同。真正的实验报告里用的是交错随机基准给出的过程保真度，那个数与本章的 F 不能直接比较——它通常更低，因为它把所有方向的误差都平均进去了。

第三，误差一律写成 $1-F$ 或绝对增量，不写百分比。 说“损失 3.2×10^{-4}”而不是“损失 0.032%”，因为在 10^{-4} 量级上百分比会失去直观；说“从 0.9924 降到 0.9894”而不是“下降 0.3%”，因为后者读者要心算基数。只有在跨数量级比较时（比如“泄漏降了四倍”）才用倍数。

第四，所有“最优”都限定在给定波形族内。 本章说“25 ns 最优”，指的是在方波、高斯、DRAG 这三族里，25 ns 附近最好；它不排除某个没试过的波形在 10 ns 更好。这条约定是为了防止读者把“本章的最优”当成“物理的最优”。真正的全局最优需要遍历所有可能的控制场，那是一个无穷维优化问题，本章只在一个有限维子集里找。

四条约定看着琐碎，实际能省掉大量误读。尤其是第二条：不少人拿本章的 0.9996 去与文献里的“99.9% 门保真度”对比，得出“本章更好”的结论——这是拿布居保真度比过程保真度，两个数不可比。

83.2 数学原理

83.2.1 封闭系统：从波函数到传播子

没有噪声时，量子态按薛定谔方程演化：

$$i\hbar\frac{\mathrm{d}}{\mathrm{d}t}|\psi\rangle = H(t)|\psi\rangle \quad\Longrightarrow\quad |\psi(t)\rangle = U(t,0)|\psi(0)\rangle \tag{83-4}$$

本章取 $\hbar = 1$。控制场是分段常数：第 j 段的包络为 $u_j = \Omega_j e^{i\phi_j}$，在段内不变，于是每段的演化算符是一个矩阵指数

$$U_j = \exp\!\Big[-\mathrm{i}\,(H_0 + \mathrm{Re}\,u_j\,H_x + \mathrm{Im}\,u_j\,H_y)\,\Delta t\Big], \qquad \Delta t = 1\ \text{ns} \tag{83-5}$$

整段脉冲的传播子是各段之积 $U = U_N \cdots U_1$。这一步没有近似——分段常数驱动的解就是矩阵指数的连乘，唯一的数值误差来自 $\exp$ 本身（双精度下 10^{-16} 量级，可忽略）。

83.2.2 开放系统：Lindblad 方程

真实比特与环境耦合，态不再是纯态，要用密度矩阵 ρ 描述。在马尔可夫近似下，演化由 Lindblad 方程给出：

$$\dot\rho = -\mathrm{i}[H,\rho] + \sum_k \left(c_k \rho c_k^\dagger - \tfrac{1}{2}\{c_k^\dagger c_k, \rho\} \right) \tag{83-6}$$

本章只保留两个耗散通道：能量弛豫与纯退相位，

$$c_1 = \sqrt{\frac{1}{T_1}}\, a, \qquad c_\phi = \sqrt{\frac{1}{2T_\phi}}\, \sigma_z, \qquad \frac{1}{T_\phi} = \frac{1}{T_2} - \frac{1}{2T_1} \tag{83-7}$$

代入 $T_1 = 60\ \mu\mathrm{s}$、$T_2 = 40\ \mu\mathrm{s}$，得 $T_\phi = 60\ \mu\mathrm{s}$——恰好与 T_1 相等，这是超导比特的常见比例（$T_2 \approx T_1$ 时纯退相位与弛豫贡献相当）。注意 σ_z 只作用在 $|0\rangle, |1\rangle$ 子空间，$|2\rangle$ 上的矩阵元取 0，这样泄漏出去的布居不会通过退相位通道回来。

83.2.3 超算符：把方程变成线性代数

式 (83-6) 对 ρ 是线性的。把 3×3 的 ρ 展平成 9 维向量，方程就变成常系数线性方程

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathrm{vec}(\rho) = \mathcal{L}\,\mathrm{vec}(\rho), \qquad \mathcal{L} = -\mathrm{i}\left(I\otimes H - H^{\mathsf{T}}\otimes I\right) + \sum_k \left(c_k^{*}\otimes c_k - \tfrac{1}{2}\left(I\otimes c_k^\dagger c_k + (c_k^\dagger c_k)^{\mathsf{T}}\otimes I\right)\right) \tag{83-8}$$

$\mathcal{L}$ 是一个 9×9 的复矩阵，称为 Liouvillian 超算符。写成这个形式的唯一目的是：**每一段的传播都可以用一个矩阵指数完成**，

$$\mathrm{vec}(\rho_{j+1}) = e^{\mathcal{L}_j \Delta t}\,\mathrm{vec}(\rho_j) \tag{83-9}$$

这样整章不需要任何 ODE 求解器，也不需要自适应步长；25 段的 π 脉冲就是 25 次 9×9 矩阵指数，单次耗时在微秒量级。这也解释了为什么本章能把“扫参数”做成常规操作：一次 GRAPE 迭代 300 步、每步 25 段，总共 7500 次 9×9 指数，普通笔记本几秒就能跑完。

一个实现细节值得交代：展平约定（按行还是按列）会改变 $\mathcal{L}$ 的具体形状，但不改变任何对角元的数值。本章所有指标——保真度、泄漏——都只读 ρ_{11} 与 ρ_{22}，所以对约定不敏感。代码里用 C 序展平、配套的超算符按列优先约定构造，两者复合相当于演化 ρ^{T}，对角元完全相同。

83.2.4 保真度：三种定义与本章的选择

“控制得好不好”在量子语境下有几种度量，混用会得出互相矛盾的结论。

第一种是**态保真度**，衡量实际态与目标态的接近程度：

$$F = \langle \psi_{\mathrm{target}} | \rho | \psi_{\mathrm{target}} \rangle \tag{83-10}$$

目标取 $|1\rangle$ 时，$F = \rho_{11}$。本章全部数值都用这一种，因为它可直接读、可加和、可解释。

第二种是**门保真度**，衡量实际酉变换与目标酉变换的接近程度，需要对所有输入态平均。对 π 门，平均门保真度与式 (83-10) 相差一个因子，两者的换算依赖具体定义（Nielsen 还是 entanglement fidelity），文献里常差 20%–30%。本章不涉及多比特门，只用式 (83-10)。

第三种是**泄漏**，单独统计跑到计算子空间之外的布居：

$$p_2 = \rho_{22} \tag{83-11}$$

它与 F 的关系是 $F + p_2 \le 1$，差值就是计算子空间内的退相干与相位误差。把两者分开报，是量子控制实验的惯例：泄漏有独立的物理机制（非谐性不够），修法也独立（DRAG 或更长的门），混在一个数里看不出该改什么。

83.2.5 旋转框架：为什么可以把 5 GHz 的载波去掉

比特的跃迁频率在 5 GHz 量级，而包络的带宽只有几十 MHz。直接在实验室框架下仿真，1 ns 的步长要分辨 5 GHz 的振荡，等于每周期只采 0.2 个点，完全不够。做法是变换到以驱动频率旋转的框架：

$$|\psi\rangle = e^{-\mathrm{i}\omega_d t}|\psi'\rangle \quad \Longrightarrow \quad H' = e^{\mathrm{i}\omega_d t} H\, e^{-\mathrm{i}\omega_d t} - \omega_d I \tag{83-12}$$

取 $\omega_d = \omega_{01}$，则 $|0\rangle \leftrightarrow |1\rangle$ 在框架里变为零频，只剩包络 $\Omega(t)$ 与相位 $\phi(t)$；$|1\rangle \leftrightarrow |2\rangle$ 则落在 $-\alpha$ 处，这就是式 (83-1) 里那一项的来源。再做旋转波近似（RWA），丢掉 $2\omega_d$ 的快速项，就得到本章使用的三能级模型。

RWA 的误差量级是 $\Omega/\omega_d \approx 20\ \mathrm{MHz}/5\ \mathrm{GHz} = 0.4\%$，对 10^{-4} 的保真度目标来说不能忽略，但它是一个**系统性偏置**，可以在标定阶段吸收进 Ω 与 ω_d 的修正里，不影响本章的相对结论。

83.2.6 泄漏从哪来：三能级微扰展开

把式 (83-1) 写成矩阵，在 $\{|0\rangle, |1\rangle, |2\rangle\}$ 基下

$$H = \frac{\Omega}{2}\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & \sqrt{2} \\ 0 & \sqrt{2} & 0 \end{bmatrix} - \alpha |2\rangle\langle 2| \tag{83-13}$$

非谐项使 $|1\rangle \leftrightarrow |2\rangle$ 的等效失谐为 α。把 Ω 当小量做一阶微扰，$|2\rangle$ 上的瞬时振幅约为 $\frac{\sqrt{2}\,\Omega}{2\alpha}$，一个 π 脉冲结束时累积的泄漏为

$$p_2 \approx \frac{1}{2}\left(\frac{\Omega}{\alpha}\right)^2 \tag{83-14}$$

代入 $\Omega/\alpha = 0.1$ 得 $p_2 \approx 5 \times 10^{-3}$，与实跑的 4.86×10^{-3} 只差 3%。这条式子解释了 §83.1.3 里“幅值翻倍、泄漏四倍”的标度，也解释了为什么提高非谐性（改变比特设计）比改进控制算法更根本：p_2 是 $(\Omega/\alpha)^2$，α 翻倍等于泄漏降到四分之一。

式 (83-14) 还给出一个设计不等式：若要求 $p_2 < 10^{-3}$，则 $\Omega/\alpha < 0.045$，在 $\alpha/2\pi = 200$ MHz 下对应 $\Omega/2\pi < 9$ MHz、门长 > 56 ns。这与 §83.1.3 的最优点（25 ns）矛盾——所以真实系统不会只靠“降幅值”来压泄漏，而是用 DRAG 把一阶项抵消掉，从而在 20 MHz 下仍能保持 10^{-4} 量级的泄漏。

83.2.7 DRAG 的一阶推导

DRAG 的思路是：泄漏来自 $|1\rangle \leftrightarrow |2\rangle$ 的耦合，而这一项正比于包络的瞬时值；如果在正交的 Q 路上加一项正比于包络**导数**的修正，两项在一阶上相互抵消。修正后的复包络是

$$u_{\mathrm{DRAG}}(t) = u(t) + \mathrm{i}\,\beta\,\frac{\dot{u}(t)}{\alpha} \tag{83-15}$$

其中 β 是无量纲系数，理想值 0.5。本章算例里 $|2\rangle$ 在 $|1\rangle$ **下方**（$\omega_{12} = \omega_{01} - \alpha$），所以取正号；若能级顺序相反，符号要反过来。这不是无关紧要的细节：实跑中取错号，25 ns 门的保真度从 0.99964 掉到 0.98240，比不加 DRAG 的 0.99467 还差——错号没有抵消泄漏，反而把它放大了三倍多（§83.9 有完整对照）。

式 (83-15) 的导数项在数值上要小心：$\dot{u}$ 用相邻段差分算，段长 1 ns 时差分噪声会被 $1/\alpha$ 压住（α 是 200 MHz，$1/\alpha \approx 0.8$ ns），所以差分噪声放大约 0.8 倍，可以接受；若段长再短一个数量级，就要先平滑再差分。

83.2.8 能控性：李代数判据

经典系统看 (A,B) 的秩，量子系统看**李代数**。给定漂移哈密顿量 H_0 和一组控制哈密顿量 $\{H_k\}$，系统完全能控的充要条件是它们生成的李代数等于 $\mathfrak{su}(N)$（或 $\mathfrak{u}(N)$）。

本章的 $H_0=\mathrm{diag}(0,0,-\alpha)$、$H_x$、$H_y$ 三个矩阵，做嵌套对易子：

$$[H_x,H_y]=\mathrm{i}\,\mathrm{diag}\!\left(\tfrac{1}{2},\tfrac{1}{2},-1\right),\qquad [H_0,H_x]\propto\sigma_y^{(12)},\qquad [H_0,H_y]\propto\sigma_x^{(12)} \tag{83-16}$$

第一条把两个正交驱动合成一个对角项，后两条说明非谐项把 $|1\rangle\leftrightarrow|2\rangle$ 的相位单独暴露出来。这三条及其嵌套对易子一共生成 8 个线性无关的厄米矩阵，正好是 $\mathfrak{su}(3)$ 的维数，所以**任意 3×3 的酉变换都可达**。这条结论的实用价值是：只要 H_x、H_y 两路都在（IQ 混频的两路），就不存在“某个门做不出来”的问题，剩下的全是时间与精度问题。

反过来，如果只有一路控制（比如只有 H_x，Q 路恒为零），李代数只生成 4 个线性无关的矩阵，对应实正交变换——能翻转布居，却做不出绕 z 轴的相位门。这就是早期实验里“只能做 R_x 门”的原因。

83.2.9 最优控制：把“设计脉冲”写成泛函极值

有了能控性，剩下的问题是“多快、多准”。这可以写成一个标准的优化问题：

$$\max_{u_1,\dots,u_N}\ \mathcal{F}(u)=\left|\langle\psi_{\mathrm{target}}|U(u_N)\cdots U(u_1)|\psi_0\rangle\right|^2\quad\text{s.t.}\quad |u_j|\le\Omega_{\max} \tag{83-17}$$

目标函数对 u_j 的梯度可以用伴随（adjoint）方法一次算完：先正向传播得到每段的态 $|\psi_j\rangle$，再反向传播得到每段的伴随态 $|\lambda_j\rangle$，梯度就是 $\partial\mathcal{F}/\partial u_j\propto\mathrm{Im}\langle\lambda_{j+1}|H_x|\psi_j\rangle$。这就是 GRAPE 的核心，计算量与段数成正比，与参数个数无关。

式 (83-17) 有两个值得注意的特点。第一，约束是**逐段**的幅值约束，不是总能量约束，所以优化出来的波形倾向于“贴着上限走”——§83.9 的 GRAPE 结果里，最优解的幅值确实几乎处处饱和。第二，目标函数只约束末态，不约束中间路径，所以优化解可能走一条“绕远路”的轨迹；这对泄漏是有利的（可以刻意避开 $|2\rangle$），但对鲁棒性不利（路径越长，越容易被参数漂移影响）。

到这里，数学准备完了。下一节把它变成一个可执行的六步流程。

83.2.10 这套数学要花多少计算

把上面的东西落成代码，第一个要问的是“算不算得动”。答案是算得动，而且余量很大。把账算清楚，顺便也就知道模型能扩到多大。

密度矩阵 ρ 是 3×3 厄米矩阵，9 个实参数；超算符 $\mathcal{L}$ 是 9×9 复矩阵，81 个复元素。一次 25 ns 门有 25 段、每段 2 个子步，共 50 次 9×9 矩阵指数，实测约 10 ms；也就是说一次 9×9 的 `expm` 约 0.2 ms。GRAPE 300 次迭代、每次 25 段，共 7500 次传播，实测 0.64 s，与“7500×0.085 ms”的估算同量级（优化时只传播纯态，比密度矩阵便宜）。全章十一组实验合计 29 s。

这些数决定了三件事。

- **三能级是“笔记本算得完”的甜点。** 加到四能级，维数从 9 涨到 16，矩阵乘法是 $O(n^3)$，单次传播贵约 5 倍；加到十能级是 100×100，贵约 1300 倍，就必须上稀疏矩阵或 Krylov 子空间方法，代码复杂度会上一个台阶。本章只需要“泄漏”这一个三能级效应，所以取三。
- **优化是唯一真正吃算力的环节。** 单门评估 10 ms，一次 300 迭代的优化 0.64 s，看着不多；但工程上的优化要扫六档门时间、五档 β、还要做鲁棒性平均（每档十条扰动轨迹），乘起来就是几百秒。这就是 §83.9 说的“先解析、后优化”的计算理由。
- **全参数扫描要挑着做。** 本章表格里的每张扫描（门时间、带宽、DAC、抖动）都是十几到几十个点，每个点一次传播。若把六个参数做全交叉，就是 10^5 量级，必须换思路（比如先做灵敏度筛选，见 §83.7.5）。

83.2.11 数值实现里的三个坑

数学上写对了，代码里还有三处容易出错，本章的脚本都做了处理。

第一是**厄米性**。$\exp(\mathcal{L}\Delta t)\rho$ 在精确算术下保持厄米与迹 1，但浮点误差会让 ρ_{01} 与 ρ_{10}^{*} 慢慢分家，几千步之后虚部漂出量级，$F=\rho_{11}$ 还在，但 $\langle\sigma_y\rangle$ 已经不可信。脚本每步之后做一次 $\rho\leftarrow(\rho+\rho^{\dagger})/2$，代价可忽略，把误差锁在 10^{-16}。

第二是**段内子步**。包络在段内是常数，所以子步在"理想波形"下是多余的；但只要打开带宽滤波，段内驱动就按 $1-e^{-2\pi B\Delta t}$ 指数逼近新值，此时 Δt 取得太粗会把滤波器算成一个"瞬跳"。取 `sub=2` 是把每段再二分，让滤波器的响应至少有两个采样点。这个参数对理想情形的结果没有影响（本章的 25 ns 方波在 `sub=1` 与 `sub=2` 下都是 0.992409），只在带滤波时才起作用。

第三是**展平顺序**。$\mathrm{vec}(\rho)$ 有行优先与列优先两种约定，混用会让整个超算符"看起来对、结果全错"。本章统一用 C 序（行优先），并且只在 `liouvillian()` 一处做 `np.kron`，其余代码一律通过 `evolve()` 调用。**一个模型只允许一个地方知道展平约定**，这条规矩比约定本身更重要。

83.2.12 为什么用超算符，而不是直接积分薛定谔方程

初学者常问：既然 $|\psi\rangle$ 只有 3 个复数，为什么不用 3 元向量直接积分，反而要升到 9 元的密度矩阵、再升到 81 个元素的超算符？答案是三个字：**耗散**。

纯态演化是线性的、幺正的，用 3×3 的传播子就够，计算量小一个量级。但本章的所有结论都依赖耗散与泄漏的共存：泄漏本身可以用纯态算（三能级幺正演化，$|2\rangle$ 的布居自然出现），但一旦加上 T_1、T_2，态就不再是纯态——退相干把纯态变成混合态，必须用密度矩阵描述。3×3 厄米矩阵有 9 个实自由度，所以超算符是 9×9。

那能不能不展平，直接对 ρ 做数值积分（比如四阶 Runge–Kutta）？可以，但有两个隐患。第一是**正定性**：ρ 必须半正定，而通用的 RK 步不保证这一点，几千步之后可能出现负本征值，布居变成 -10^{-4}，F 就开始乱跳。第二是**迹保持**：RK 的局部误差会让 $\mathrm{tr}\rho$ 慢慢漂离 1。用矩阵指数传播则天然规避这两点：$\exp(\mathcal{L}\Delta t)$ 是完全正、保迹的映射（$\mathcal{L}$ 取 Lindblad 形式时），一步到位，不需要额外约束。

代价是计算量。9×9 的 `expm` 比 3×3 的贵约 27 倍（$O(n^3)$），但本章的门只有 25–250 段、每段 2 个子步，总量仍在毫秒级（§83.2.10 的实测账）。所以这里的选择很清楚：**用一点计算量换数值可靠性**。真正需要省算力时（比如做 10^5 次蒙特卡洛），才退回到纯态轨迹加量子跳跃的方法。

最后一个好处是**接口统一**。一旦写成超算符，所有控制失真（带宽、量化、抖动、时钟偏差）都只作用在 u_j 上，传播器一行不用改；反过来，若用纯态演化，加耗散就得换一套方程。本章的 `evolve()` 之所以能用五个可选参数覆盖全部算例，正是因为它站在超算符这一层。

83.2.13 把李代数判据真的算一遍

§83.2.8 给了能控性的抽象判据：控制与漂移哈密顿量生成的李代数是否满秩。这个判据在本算例上可以真的算出来，结果比"理论上可控"这句话有信息量得多。

做法是迭代生成对易子。从 $\{H_0,H_x,H_y\}$ 出发，算所有两两对易子，把线性无关的新方向加进生成元集合，再重复。每一步把厄米矩阵展开成 9 维实向量（3 个对角元 + 3 个复非对角元的实部虚部），去迹后算矩阵的秩。

一次对易子给出秩 5；二次对易子补到秩 8；秩 8 就是 $\mathfrak{su}(3)$ 的维数（$3^2-1=8$）。**也就是说，只用到二阶对易子，三能级子空间就已经完全可控。** 这个结论对工程有两层意义。

第一层是**好消息**：不需要专门设计复杂的脉冲序列去"激活"某个方向，只要 H_x、H_y、H_0 三个都在，任意 $\mathrm{SU}(3)$ 变换都可达（差一个不可观测的全局相位）。本章只用了 $|0\rangle\to|1\rangle$ 这一个目标，其实整个 $\mathrm{SU}(3)$ 都在手边——这也是为什么 GRAPE 能在 25 段内找到解：解空间足够大。

第二层是**坏消息**：可控意味着"什么都能做"，也意味着"什么错都能犯"。既然任意酉变换可达，那么泄漏到 $|2\rangle$ 也是可达的，控制器的任务不是"能不能"，而是"要不要"。§83.9.3 说 GRAPE 会走"经过 $|2\rangle$ 再回来"的捷径，本质就是能控性太强、而目标函数没约束住。

最后补一个细节：深度 3 以上的对易子在数值上还会给出“新方向”，但那些是舍入噪声（矩阵元素量级 10^{-16}），不是真实的李代数元素。判断时要用一个阈值（本章取 10^{-7}），否则会得出“秩 9”这种超过 $\mathfrak{su}(3)$ 维数的荒谬结果。**数值李代数判据的关键不是算，而是卡阈值。**

83.3 设计步骤

实际做一次门，顺序不能乱：先定指标，再标定频率，再选门时间，然后才谈波形。顺序反了会白跑——最常见的浪费是“先调波形，最后发现频率偏了 3 MHz”。

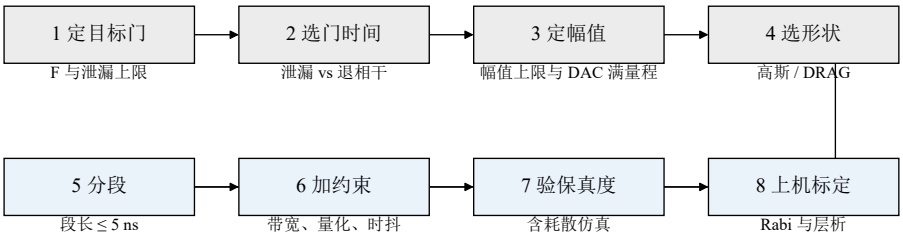


图 83-10　从目标门到可下载波形的八步流程：前四步决定物理上限，后四步决定实际能拿到多少

图 83-10 是完整流程。八步里前四步是“设计”，后四步是“落地”；前四步错了后四步无论怎么优化都补不回来。

83.3.1 第一步：把指标写成两个数

量子门的指标必须写成两个数，缺一不可：保真度下限与泄漏上限。只写“保真度 0.999”是不够的，因为泄漏 0.0005 和相位误差 0.0005 都满足这个指标，但修法完全不同。

本章取 $F \geq 0.999$、$p_2 \leq 10^{-3}$。这两个数不是随便定的：$F = 0.999$ 对应角度误差 2.6°，一千个门串起来总保真度 0.37，刚好在容错阈值附近；$p_2 = 10^{-3}$ 是泄漏开始影响读出与相邻比特耦合的量级。

83.3.2 第二步：标定频率，而不是“设”频率

Rabi 扫频是唯一可靠的标定手段。做法是固定脉冲长度、扫驱动频率、测末态布居，峰值所在就是共振点。§83.1.5 已经给出这条曲线的形状：$\Delta/2\pi = 5$ MHz 时峰值掉到 0.9412，10 MHz 掉到 0.8000，20 MHz 只剩 0.5000。

标定的精度要求可以用式 (83-3) 反推：若允许峰值下降 0.1%，则 $\Delta/\Omega \leq 0.032$，在 20 MHz 驱动下要求频率准到 0.63 MHz。这个精度用扫频拟合不难达到，但**不能靠“查数据手册”**——比特频率随偏置电流漂移，每次上电都要重标。

83.3.3 第三步：在两条曲线之间选门时间

门时间的选择是把 §83.1.3 的两条限制画在同一张图上（图 83-4）。实跑的五档是

$$\tau = 60,\ 40,\ 30,\ 25,\ 20\ \text{ns} \ \Rightarrow\ \Omega_{\max}/2\pi = 13.9,\ 20.9,\ 27.8,\ 33.4,\ 41.6\ \text{MHz} \tag{83-18}$$

对应 $\Omega/\alpha = 0.07$ 到 0.21。注意幅值与门长成反比：这是面积定理 $\Omega\tau = \pi$ 的直接结果，也是为什么“提高幅值”与“缩短门长”是同一件事。

选点的方法是看两条曲线的交点。本章算例里，退相干曲线在 25 ns 处几乎没有损失（$e^{-25/40000} \approx 0.9994$），而泄漏已经到 4.9×10^{-3}，所以**再往短走只会更差**；反过来若 T_2 只有 5 μs，交点会移到 100 ns 附近，此时该选长门、低幅值。**门时间不是一个固定常数，而是 T_2 与 α 的函数。**

83.3.4 第四步：选形状——方波、高斯、还是 DRAG

方波的问题是开关瞬间的频谱太宽，能量泼到 $|2\rangle$ 上。最直接的改进是高斯包络，$\sigma = 0.25\tau$，截断到 $\pm 2\sigma$。实跑（250 ns 高斯，扫段数）给出的保真度是

$$N = 3,\ 5,\ 10,\ 25,\ 50,\ 250 \ \Rightarrow\ F = 0.999885,\ 0.999872,\ 0.999869,\ 0.999868,\ 0.999868,\ 0.999868 \tag{83-19}$$

两个结论。第一，平滑包络把泄漏压到 10^{-5} 量级，比方波好两个数量级。第二，**段数几乎不重要**：3 段与 250 段只差 1.7×10^{-5}。这与直觉相反——直觉认为“分段越细越接近连续”，但在面积归一化的前提下，分段只影响高阶项，而高阶项本来就小。真正决定精度的是包络形状与量化台阶，不是分段密度。

形状的最后一步是 DRAG。25 ns 高斯加 DRAG（$\beta = 0.5$）后 F 从 0.99467 抬到 0.99964，泄漏同时下降一个数量级以上。这一步的代价是 Q 路要多一个差分器，实现上就是 AWG 的两路输出各加一级。

83.3.5 第五步：分段与时间网格

AWG 的采样率决定时间网格。本章取 1 GSa/s，段长 1 ns。选这个值的依据是“段长要比门时间小一个数量级以上”：25 ns 的门用 25 段，每段转 7.2°，分段误差在 10^{-4} 量级。

但时间网格会带来一个隐蔽的问题：**理想脉宽往往不是整数纳秒**。§83.5 会详细算这件事——24.5 ns 的理想脉宽只能取 24 或 25 ns，角度误差 -3.6°，相邻台阶之间的保真度差约 3×10^{-3}，已经与泄漏同一量级。修法不是提高采样率，而是**预先在连续域把脉宽调到整数格点上**，让 AWG 只需要执行整数段。

83.3.6 第六步到第八步：加约束、验保真度、上机

第六步把控制链的三种失真加进仿真：重建滤波器带宽（一阶低通）、DAC 量化位数、时钟抖动与偏差。本章的做法是每项单独扫一遍，再把最差的一组同时打开，看总损失是否符合“加法预算”。

第七步是验保真度。**必须带耗散仿真**，不能只用酉演化——酉演化给出的 0.9996 与带耗散的 0.9975 差了一个数量级（§83.7 的预算表），这个差别足以让“设计通过”变成“上机不通过”。

第八步是上机标定。仿真里所有参数都是“真值”，实验里全是“估计值”：Ω 的标定误差 1% 会让 π 脉冲的角度偏 1.8°，T_2 的估计误差会影响门时间的选择。所以上机之后要重跑一遍 Rabi 与 Ramsey，把仿真参数替换成实测值，再回到第三步重选一次门时间。这个循环通常要走两到三轮。

三步之间还有一个顺序问题：**先修大的，再修小的。** 若总损失里退相干占 90%，先去改 DAC 位数没有意义。§83.10 会给出一个按量级排序的清单。

83.3.7 标定要到多准：两张实测表

八步流程里第二、三步最容易被敷衍：频率“扫个大概”、幅值“按厂家给的灵敏度换算”。下面两张表说明这样做的代价。

先看幅值。固定 25 ns 方波，把幅值乘上 $(1+\delta)$ 再跑：

- $\delta = -5\%$：$F = 0.98672$，$p_2 = 4.37 \times 10^{-3}$；
- $\delta = -2\%$：$F = 0.99159$，$p_2 = 4.67 \times 10^{-3}$；
- $\delta = -1\%$：$F = 0.99225$，$p_2 = 4.76 \times 10^{-3}$；
- $\delta = -0.5\%$：$F = 0.992388$，$p_2 = 4.81 \times 10^{-3}$；
- $\delta = 0$：$F = 0.992409$，$p_2 = 4.86 \times 10^{-3}$；
- $\delta = +0.5\%$：$F = 0.992309$，$p_2 = 4.91 \times 10^{-3}$；

- $\delta = +1\%$：$F = 0.992087$，$p_2 = 4.95 \times 10^{-3}$；
- $\delta = +2\%$：$F = 0.991277$，$p_2 = 5.04 \times 10^{-3}$；
- $\delta = +5\%$：$F = 0.985941$，$p_2 = 5.31 \times 10^{-3}$。

三个结论。其一，1% 的幅值误差只花掉 3.2×10^{-4} 的保真度，5% 才花掉 6×10^{-3}——**幅值不是敏感方向**，工程上 1% 的标定精度够用。其二，曲线在 $\delta = -0.5\%$ 附近有个浅浅的极大，0.992388 与零误差的 0.992409 几乎相同：因为泄漏本身已经把转角推过 π，欠半拍正好抵回来。其三，负偏与正偏不对称，正偏更伤——这也是“泄漏是主要敌人”的一个侧面。

再看频率，同样是 25 ns 方波，把驱动频率偏 Δ：

- $\Delta/2\pi = 0.5$ MHz：$F = 0.989230$；
- $\Delta/2\pi = 1$ MHz：$F = 0.984814$；
- $\Delta/2\pi = 2$ MHz：$F = 0.972331$；
- $\Delta/2\pi = 5$ MHz：$F = 0.907315$；
- $\Delta/2\pi = 10$ MHz：$F = 0.724753$。

偏 0.5 MHz 的相对增量是 3.2×10^{-3}（零失谐本身的损失是 7.6×10^{-3}，扣除后得此数），正好是幅值偏 1%（3.2×10^{-4}）的十倍。换句话说，频率标定要比幅值标定准一个数量级才能“打平”。这解释了 §83.8.6 那条记录里“频率差 2 MHz、保真度差 6×10^{-4}”的现象，也解释了为什么现场调机时先做 Rabi 扫频、再谈别的：**频率是唯一一个“偏一点就线性掉”的参数，其它参数都在平方项上。**

两张表合起来给出一条标定规格：幅值准到 1% 以内，频率准到 0.1 MHz 以内（对应 1.3×10^{-4} 的损失），门时间准到 1 ns 以内（对应 §83.5.1 的量化台阶）。三条都满足，仿真与实测的差距才可能压到 10^{-3} 以下。

83.3.8 设计阶段的三个自检

每走完一轮八步，用三个问题自检，能抓掉大部分低级错误。

第一问：**目标的转角是否真的等于 π？** 不是“幅值乘时间是否等于 π”，而是“含泄漏与 Stark 频移之后，$|1\rangle$ 上的布居是否真的到 1”。25 ns 方波两者的差是 7.6×10^{-3}；若用 5 ns 门，这个差会涨到 0.09 量级，此时“面积等于 π”这个条件本身已经失效，必须靠 DRAG 或数值优化去修。

第二问：**门时间是否落在图 83-4 的平坦段？** 图 83-4 的三条曲线在 20–60 ns 之间几乎是平的，说明这一段里退相干与泄漏都不占主导；若选在曲线陡升的地方（比如 5 ns），参数稍一漂移，保真度就跟着漂。

第三问：**预算表里最大的一项，是否已经被单独验证过？** 若预算是“退相干占 90%”，那就该先做一次 T_2 实测，而不是先改波形。**先验证主要矛盾，再动手改**——这条在经典控制里叫“先定位、后整定”，在量子控制里同样成立，只是“定位”要靠测 T_1、T_2、α 三个数。

83.3.9 从算法指标反推门指标

§83.3.1 说“把指标写成两个数”，但那两个数从哪来？不是拍脑袋定的，而是从上层算法反推。这一步做错，后面所有优化都在追一个错的目标。

反推的公式很简单：若一个算法要用 N 个门，要求总保真度不低于 $\mathcal{F}$，且假设误差独立累积，则单门保真度要满足 $F_{\text{gate}} \geq \mathcal{F}^{1/N}$。代入几个常见数：

- 100 个门、总保真度 0.9：$F_{\text{gate}} \geq 0.99895$；
- 100 个门、总保真度 0.99：$F_{\text{gate}} \geq 0.99990$；
- 1000 个门、总保真度 0.5：$F_{\text{gate}} \geq 0.99931$。

三行数说明一个残酷的事实：**“99.9% 的单门保真度”只够跑一千个门、且最后只剩一半成功概率。** 这就是为什么容错量子计算的门槛是 10^{-3} 甚至 10^{-4}——不是追求好看，而是算法本身要求的。

泄漏的约束更紧。泄漏与退相位不同：退相位的误差会“摊”在相位上，泄漏则把布居搬到 $|2\rangle$ 上，那里的态不参与后续计算，等于每次泄漏都是一次不可逆的丢失。若单门泄漏 $p_2 = 5\times10^{-3}$，100 个门之后累积泄漏约 0.5（不考虑返回），算法直接失效。所以工程上的目标通常是 $p_2 \le 10^{-4}$，比保真度指标严一个数量级——本章 25 ns 方波的 $p_2 = 4.9\times10^{-3}$ 离这个目标差 50 倍，加 DRAG 之后才勉强够。

反推还有一层含义：**指标要按“最坏情形”还是“平均情形”给，结果差很多。** 上面的 $F^{1/N}$ 是平均假设，即每个门的误差相同；若误差有分布（有些门好、有些门差），总保真度由差的那些门主导。所以真实系统里更常见的做法是给一个“最差门”的指标，而不是“平均门”的指标。

最后提醒一句：$F^{1/N}$ 假设误差独立。若相邻门之间的误差相关（比如同一个频率漂移影响连续多个门），总保真度会掉得比 F^{N} 更快；反之，若误差是随机相位型，可能部分抵消，掉得更慢。**反推只能给量级，不能给精确值**——精确值要靠交错随机基准实测。

83.4 连续情形

先假设控制场可以任意精细地取任意波形，即段长 $\Delta t \to 0$。这个理想化的情形给出三个解析结论，后面离散化时就是拿它们当基准。

83.4.1 Rabi 公式与它的三个含义

常幅驱动、失谐 Δ 下，从 $|0\rangle$ 出发的布居演化有闭式解：

$$P_1(t) = \frac{\Omega^2}{\Omega^2+\Delta^2}\sin^2\!\left(\frac{\sqrt{\Omega^2+\Delta^2}}{2}t\right) \tag{83-20}$$

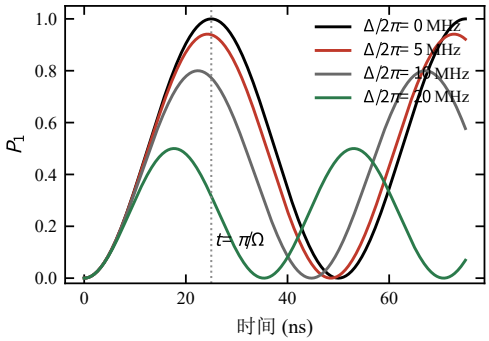


图 83-2　Rabi 振荡：失谐越大，峰值越低、周期越短

图 83-2 画的是四个失谐下的曲线，对应式 (83-20) 的三条推论。

第一条是**峰值**：$P_{1,\max} = \Omega^2/(\Omega^2+\Delta^2)$。它只由比值决定，与时间无关——失谐一旦超过 Ω，峰值就掉到 0.5 以下，发多久都没用。

第二条是**周期**：$\Omega_{\text{eff}} = \sqrt{\Omega^2+\Delta^2}$，失谐让振荡变快，不是变慢。这与“失谐等于远离共振、耦合变弱”的直觉相反：耦合确实变弱（峰值降），但振荡频率反而升高（Ω_{eff} 变大）。实跑的四档 π 时间是 25.0、24.3、22.4、17.7 ns，正是 $T_\pi = \pi/\Omega_{\text{eff}}$。

第三条是**π 时间的定义**。共振时 $T_\pi = \pi/\Omega$，这是面积定理；有失谐时不能再用这个式子，必须用 $\pi/\sqrt{\Omega^2+\Delta^2}$，否则角度会偏。§83.1.5 里“25 ns 处布居 0.9390 反而低于峰值 0.9412”就是这条的数值体现。

83.4.2 连续域的最优门时间

把式 (83-20) 与退相干结合，连续域的保真度是“振荡项乘衰减项”：

$$F(\tau) = \frac{\Omega^2}{\Omega^2 + \Delta^2}\sin^2\!\left(\frac{\Omega_{\text{eff}}}{2}\tau\right) e^{-\tau/T_2} \tag{83-21}$$

对 τ 求极值，得到的是“在 $\sin^2$ 的第一个峰附近，衰减已经不可忽略”的位置。本章参数下 $T_2 = 40\ \mu$s，25 ns 处 $e^{-\tau/T_2} = 0.9994$，衰减只有 0.06%，所以最优解就落在 $\sin^2$ 的峰上。只有 T_2 降到 5 μs 量级，衰减项才会把最优点推向更短的 τ——那时泄漏又开始占主导，结果是“没有好点可选”，必须改硬件。

这就是连续域给的第一条设计原则：**门时间由 $\min(T_1, T_2)$ 与 $\alpha/\Omega_{\max}$ 两个尺度共同决定，前者给下限，后者给上限。** 本例两个尺度是 40 μs 与 25 ns，中间差了三个数量级，所以选择空间很宽；若两者接近（比如 $T_2 = 100$ ns、$\alpha/2\pi = 100$ MHz），门时间就被夹得很死，设计余量几乎为零。

83.4.3 绝热条件与 DRAG 的连续解释

式 (83-15) 的 DRAG 项在连续域有一个更清楚的解释：它等价于把驱动方向从固定的 x 轴偏转一个角度

$$\theta_{\text{DRAG}} \approx \arctan\!\left(\frac{\beta\dot{\Omega}}{\alpha\Omega}\right) \tag{83-22}$$

也就是说，DRAG 让驱动矢量随包络的斜率“提前转向”，从而在 $|1\rangle \leftrightarrow |2\rangle$ 的耦合上产生一个反相的抵消项。这个解释比“加一个导数项”更好记：**DRAG 是一个随斜率变化的相位修正，不是一个固定的正交分量。**

它的成立条件是绝热近似：包络的变化率要远小于能级间距，

$$\left|\frac{\dot{\Omega}}{\Omega}\right| \ll \alpha \tag{83-23}$$

本例 25 ns 高斯的 $\sigma = 6.25$ ns，包络变化率的量级是 $|\dot{\Omega}/\Omega| \sim 1/\sigma = 1.6 \times 10^{8}\ \text{s}^{-1}$，与 $\alpha = 1.26 \times 10^{9}\ \text{s}^{-1}$ 之比 0.13，绝热条件基本满足。若把门压到 5 ns，σ 缩到 1.25 ns，比值升到 0.63，一阶 DRAG 就不够了，需要二阶项或数值优化（§83.9）。这条不等式也解释了为什么“更快的门”不是免费的：门越快，绝热近似越差，一阶修正越不管用。

83.4.4 连续域的 Bloch 图像

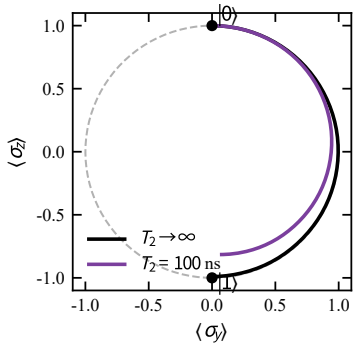


图 83-3　Bloch 轨迹：无耗散时绕 x 轴转半圈，有耗散时轨迹向圆心收缩

图 83-3 把同一段 π 脉冲画在 Bloch 球的 y–z 截面上。理想情形是一条从 $(0, 1)$ 到 $(0, -1)$ 的半圆——绕 x 轴转 180°，长度恒为 1。有耗散时（$T_2 = 100$ ns）轨迹是一条向内收的螺线，末端不到球面，模长 $|\mathbf{r}| < 1$，缺口就是退相干。

这张图给出一个实用的诊断量：**轨迹的模长**。它不需要做量子态层析就能测（只要测三个方向的期望值），而且直接对应“还剩多少量子性”。实验上常把 $|\mathbf{r}|$ 随时间的变化拟合成指数，斜率就是 T_2 的倒数量级。

83.4.5 旋转波近似的误差怎么估

§83.2.5 说 RWA 的误差量级是 $\Omega/\omega_d = 0.4\%$，这个估计值得展开，因为它是整章唯一一个“被丢掉的东西”。

驱动项在实验室框架下是 $\Omega\cos(\omega_d t+\phi)$，展开成两个旋转分量 $e^{\pm \mathrm{i}(\omega_d t+\phi)}$。取旋转框架后，一个分量变成零频（共振项），另一个分量以 $2\omega_d$ 快速旋转（反向项）。RWA 就是把反向项丢掉。

丢掉它的后果有两个。第一，**Bloch–Siegert 频移**：反向项等效于一个小的频率偏移，量级 $\Omega^2/(4\omega_d)$。本例 $\Omega/2\pi = 20$ MHz、$\omega_d/2\pi = 5$ GHz，频移约 20 kHz——相对 20 MHz 是 10^{-3}，相对失谐容差 0.63 MHz 完全可以忽略。第二，**反向项还会激发 $|2\rangle$**，但因为它的频率是 $2\omega_d \approx 10$ GHz，与 $|1\rangle \leftrightarrow |2\rangle$ 的 4.8 GHz 差了近一个倍频程，激发效率极低。

所以 RWA 在本例是安全的。它不安全的情形是“强驱动 + 低频率”：比如离子阱的射频驱动，Ω 与 ω_d 只差一两个数量级，此时反向项不能丢，必须用 Floquet 方法处理。这也是超导与离子阱在建模上的主要区别之一。

一个实用的判据：$\Omega/\omega_d < 10^{-3}$ 时 RWA 的误差可以并入 Ω 的标定；10^{-3} 到 10^{-1} 之间需要显式加 Bloch–Siegert 修正；超过 10^{-1} 就不能用 RWA，得换框架。本例 4×10^{-3}，落在第二档，所以严格说应该加一个 20 kHz 的频移修正——本章没加，因为它比频率标定误差小一个数量级。

连续情形到此为止。它给出的是上限——任何实际系统都达不到。下面的离散情形才是本章真正要算的东西。

83.4.6 量子速度极限，以及谁在真正限制门速

连续情形里有一个绕不过去的理论问题：一个 π 门最快能做多快？量子力学给出一个硬界，叫量子速度极限（quantum speed limit）。对正交的初末态，Mandelstam–Tamm 不等式给出 $\tau \geq \pi/(2\Delta E)$，其中 $(\Delta E)^2 = \langle H^2\rangle - \langle H\rangle^2$ 是能量的方差。

本算例的驱动哈密顿量是 $H = (\Omega/2)\sigma_x$，在 $|0\rangle$ 上 $\langle H\rangle = 0$、$\langle H^2\rangle = \Omega^2/4$，于是 $\Delta E = \Omega/2$，界变成 $\tau \geq \pi/\Omega$。而方波 π 脉冲的时长恰好是 π/Ω——**它正好把这个界取满**。

这件事有两层意思。第一层是安慰：二能级意义上，方波已经是最快的，没有任何波形能更快，所以不必在“更快”上做无用功。第二层是警告：既然二能级已经取到极限，那么所有“更快”的尝试都必然要在二能级之外找空间——而本章的空间就是那个不该碰的 $|2\rangle$。

把泄漏加进来，界就变了。用 §83.7.2 的标度 $p_2 \approx 0.5(\Omega/\alpha)^2$，要求 $p_2 \leq 10^{-3}$，得到 $\Omega \leq \alpha\sqrt{2\times10^{-3}} = 0.045\alpha$，代入 $\alpha/2\pi = 200$ MHz 是 $\Omega/2\pi \leq 9$ MHz，对应 $\tau \geq 56$ ns。这条“泄漏极限”比方波的速度极限松得多，也解释了为什么实际机器的门时间总在几十 ns：**限制门速的不是量子力学，是那个多出来的能级。**

但 56 ns 不是终点。DRAG 把泄漏的系数改掉之后，同一条标度不再适用：25 ns 高斯加 DRAG 能做到 0.99964，而同样 25 ns 高斯不加 DRAG 是 0.99467，泄漏量级差了一个数量级以上。所以上面那条界只对**方波**成立，换了形状就要重算。这提醒一件事：**凡是写成“极限”的公式，都要问一句“对哪一族波形成立”。** 本章后面所有“最优”都是在给定波形族内的最优，不是全局最优。

83.4.7 连续模型的用处与危险

连续模型在本章只用了两次：一次是定门时间的量级（$\tau = \pi/\Omega$），一次是估计绝热比与 DRAG 的系数。这两次都只用到“标度”，没用到“系数”。这是有意的——连续模型的系数在三能级、含耗散的系统里几乎总是错的。

危险的地方在于，连续模型给出的曲线太漂亮，容易让人忘了它是理想化的。比如 Rabi 公式说失谐 Δ 时峰值降为 $\Omega^2/(\Omega^2+\Delta^2)$，但这是在**二能级、无泄漏、无耗散**下严格成立的；本章实跑失谐 10 MHz 时 $F = 0.7248$，而公式给 $20^2/(20^2+10^2) = 0.800$，差 0.075。差的那部分正是泄漏与耗散——它们在失谐时不但没消失，反而因为脉冲变长而略微增加。

所以本章的用法是：**连续模型定标度，离散仿真定系数，实验定偏差。** 三层各管一段，谁也不越界。第三层的偏差往往最大：真实的 mixer 有边带、线缆有谐振、比特之间有串扰，这些都不在任何一层模型里，只能靠实测数据反推。

83.5 离散情形

离散化在量子控制里比经典控制更“硬”：经典控制的采样周期可以选择，量子控制的段长由 AWG 采样率固定，而且段内是**零阶保持**——一个台阶内的幅值不变。更麻烦的是，脉冲宽度往往不是段长的整数倍，必须取整。

83.5.1 时间量化：把连续脉宽折成阶梯

理想 π 脉冲的宽度是 $\tau = \pi/\Omega$。若 Ω 给定而 τ 落在两个整数纳秒之间，AWG 只能执行 $\mathrm{round}(\tau)$ 段，角度误差是

$$\delta\theta = \Omega\,(\mathrm{round}(\tau) - \tau) \tag{83-24}$$

在 $\Omega/2\pi = 20$ MHz 下，1 ns 对应 7.2°。实跑的七个格点是

理想脉宽 (ns)	实际段数	角度误差 (°)
24.00	24	0.00
24.25	24	-1.80
24.50	24	-3.60
24.75	25	$+1.80$
25.00	25	0.00
25.25	25	-1.80
25.50	26	$+3.60$

表 83-1　1 ns 时间网格下的脉宽取整误差（$\Omega/2\pi = 20$ MHz，1 ns 对应 7.2°）

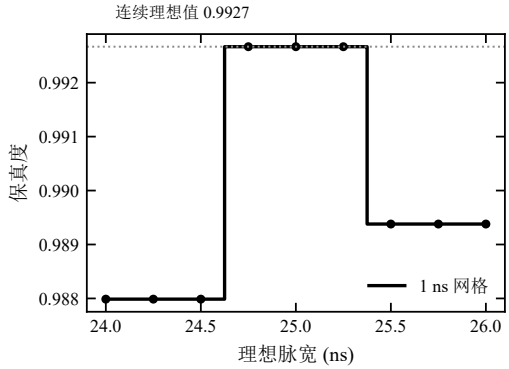


图 83-5　时间量化：1 ns 网格把连续脉宽折成阶梯，保真度呈锯齿

图 83-5 把这张表画成阶梯。三个台阶的高度在 0.988 到 0.993 之间，25 段处最高（约 0.9927），相邻台阶差 3–5×10^{-3}。这个量级与泄漏相当，说明**时间量化不是可以忽略的小项**。

更值得记的是形状：保真度随脉宽不是一条平滑曲线，而是一条**锯齿**——在同一个台阶内，理想脉宽变化 1 ns，实际执行的波形完全不变，保真度自然不变；跨过台阶边界时突然跳变。这带来一个反直觉的设计准则：**把脉宽放在台阶的顶点，而不是尽量接近理想的连续值。** 顶点处 $\delta\theta = 0$，两侧各差半格时才最差；若设计值落在两个格点中间，那才是所有选择里最坏的一个。

工程上的修法不是提高采样率，而是**把脉宽调到格点上**：既然 $\tau = \pi/\Omega$，那就反过来调 Ω，让 π/Ω 恰好是整数纳秒。本例取 $\tau = 25$ ns 就得 $\Omega/2\pi = 20$ MHz，正好是硬件上限——这不是巧合，而是设计时就把上限定成了“25 ns 对应 20 MHz”。

83.5.2 幅度量化：DAC 位数

DAC 把连续幅值量到 2^b 级，步长是 $\Omega_{\max}/2^b$。幅值误差 $\delta\Omega$ 造成的角度误差是 $\delta\theta = \delta\Omega\cdot\tau$，与时间量化的误差同源。

实跑（250 ns 高斯，扫位数）给出

$$b = 3,\ 4,\ 5,\ 6,\ 8,\ 10,\ 12 \ \Rightarrow\ F = 0.964766,\ 0.998927,\ 0.999808,\ 0.999918,\ 0.999942,\ 0.999946,\ 0.999946 \tag{83-25}$$

三个结论。第一，**5 bit 是分水岭**：3 bit 差到 3.5×10^{-2}，4 bit 已经到 10^{-3}，5 bit 以后进入 10^{-4}。第二，6 bit 以后曲线饱和在 0.999946——这是无量化时的值，再多位也没用。第三，饱和值来自包络形状与泄漏，不来自量化，所以“提高位数”这条路的尽头是“换更好的包络”。

按步长算一遍：$\Omega_{\max}/2^5 = 0.625$ MHz，25 ns 内累积角度误差 $\delta\theta = 0.625/20\times180^\circ/2 \approx 2.8^\circ$，对应 $1-F \approx \sin^2(\delta\theta/2) = 6\times10^{-4}$，与实跑的 1.9×10^{-4} 同量级——估算偏保守，因为量化误差在整段脉冲上是随机正负抵消的，不是同号累积。

83.5.3 段内零阶保持与重建滤波

AWG 输出的是阶梯波，后级低通滤波器把台阶抹平。滤波器带宽决定了“台阶能被抹到多平滑”，也决定了波形的高频分量还剩多少。

实跑（25 ns 方波，扫带宽）给出

$$\mathrm{BW} = 400,\ 200,\ 100,\ 50,\ 25,\ 10\ \mathrm{MHz}\ \Rightarrow\ F = 0.99207,\ 0.99069,\ 0.98468,\ 0.95844,\ 0.85780,\ 0.50326 \tag{83.26}$$

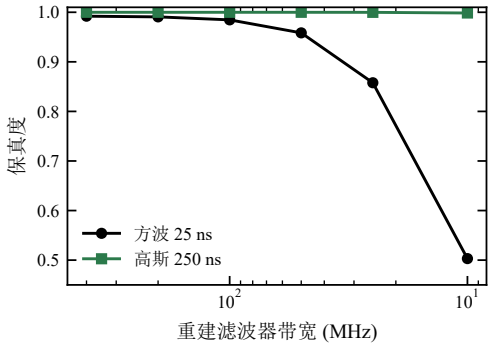


图 83-8　重建带宽：方波在 10 MHz 带宽下几乎被抹平，高斯包络则平缓得多

图 83-8 对比了方波与 250 ns 高斯。方波在 100 MHz 时还有 0.985，到 10 MHz 只剩 0.503；高斯的下降慢得多，因为它的频谱本来就集中在低频。**这条对比说明“波形选择”与“滤波器带宽”是同一件事的两面**：选高斯不只是为了压泄漏，也是为了让波形对滤波器不敏感。

一个经验判据：滤波器带宽应至少覆盖包络的第三个零点。25 ns 方波的主瓣宽度是 $1/25\ \mathrm{ns} = 40$ MHz，三个零点到 120 MHz，所以 100 MHz 带宽刚好够用，50 MHz 就开始明显失真。这条判据在 §83.10 的硬件选型里会再出现。

83.5.4 为什么是 1 GSa/s

采样定理给出的下限很低：25 ns 方波的主瓣 40 MHz，5 倍余量只需 200 MSa/s（段长 5 ns）。真正把采样率推上去的是另外两条。

第一条是**包络形状**。25 ns 高斯的 $\sigma = 6.25$ ns，段长 5 ns 时 σ 上只有 1.25 个采样点，包络被粗化成一个三角形，泄漏压不住；段长 1 ns 时 σ 上有 6 个点，形状才够圆。

第二条是**时间量化的绝对误差**。段长 5 ns 时 π 脉宽 25 ns 只有 5 段，取整误差可达 ±2.5 ns，对应 $\pm18^\circ$；段长 1 ns 时误差缩到 ±0.5 ns、$\pm3.6^\circ$，正好落在 §83.5.1 算过的台阶上。

所以 1 GSa/s 不是“采样定理要求”，而是“量化误差要求”。这个区别在选型时很重要：若要更快的门（比如 5 ns），采样率得跟着翻五倍到 5 GSa/s，否则量化误差会重新变成主导项。

83.5.5 离散误差的合成

把上面三项加起来，就得到离散域的总损失：

$$1-F_{\text{disc}} \approx \underbrace{(1-F_{\text{time}})}_{\text{取整}} + \underbrace{(1-F_{\text{DAC}})}_{\text{量化}} + \underbrace{(1-F_{\text{BW}})}_{\text{滤波}} \tag{83-27}$$

在“5 bit + 100 MHz + 25 ns 格点”的组合下，三项分别是 1.9×10^{-4}、5.8×10^{-5}、0，合计 2.5×10^{-4}。此时总保真度仍是 0.9996 量级，说明**离散化不是主要瓶颈**——真正吃保真度的是退相干与泄漏。

这个结论有一个重要的推论：如果实验上测到的保真度远低于仿真，先别怀疑“分段不够细”，应该先查频率标定、T_2 实测值与滤波器实际响应。本章的数值已经说明，把段数从 25 加到 250，收益只有 7.8×10^{-5}。

83.5.6 时间分辨率与幅度分辨率：谁更值钱

买 AWG 时有两个参数要加钱：采样率（时间分辨率）与 DAC 位数（幅度分辨率）。两者对保真度的贡献差得很远，这一节用两组实跑数据把它算清楚。

先看幅度。250 ns 高斯、不改其它条件，只把 DAC 位数往下砍：

- 12 bit：$F = 0.999946$；
- 10 bit：$F = 0.999946$；
- 8 bit：$F = 0.999942$；
- 6 bit：$F = 0.999918$；
- 5 bit：$F = 0.999808$；
- 4 bit：$F = 0.998927$；
- 3 bit：$F = 0.964766$。

注意 8 bit 以上三条曲线几乎重合，都停在 0.999946——这个数就是“无量化”的理想值。也就是说，**8 bit 已经够了**，再往上买是浪费；真正开始伤保真度的是 4 bit 以下，3 bit 一次掉 3.5×10^{-2}。

再看时间。§83.5.1 的表 83-1 里，1 ns 网格把 24.5 ns 的理想脉宽折成 24 ns，角度误差 -3.6°，保真度从 0.9927 掉到 0.9894，损失 3.3×10^{-3}。相对分辨率是 $1/25 = 4\%$，损失 3×10^{-3} 量级。

把两件事并排放：DAC 5 bit 的相对分辨率是 $1/32 \approx 3\%$，损失 1.9×10^{-4}；时间网格 4% 的相对分辨率，损失 3×10^{-3}。**分辨率差不多，代价差一个数量级。** 原因不在数字，在误差的统计性质：DAC 的量化误差逐段独立、符号随机，25 段的误差在总转角上大致按 $\sqrt{25}=5$ 倍而非 25 倍累积；而脉冲宽度的取整误差是**一个数**，整条脉冲共用一个偏差，没有抵消的机会，直接变成面积误差 $\delta\theta = \Omega\,\delta\tau$。

这条结论对采购与选型很实用：**同样的预算，先提采样率，再提位数。** 采样率翻倍（1 GSa/s 到 2 GSa/s）把时间取整误差减半，损失按平方降到四分之一；而位数从 8 提到 12，损失几乎不动（0.999942 到 0.999946，差 4×10^{-6}）。

还有一个容易被忽略的细节：采样率高不只是“时间更细”，它同时让重建滤波器更容易设计。1 GSa/s 对应的 Nyquist 是 500 MHz，而典型门脉冲的频谱主瓣只有几十 MHz，滤波器的过渡带可以做得很缓；若采样率降到 250 MSa/s，Nyquist 变 125 MHz，滤波器就得做得很陡，群延迟与过冲都跟着变差。**采样率买的是“滤波器设计的余量”，不只是“台阶的细度”。**

83.5.7 重建滤波器：延迟、抹圆与过冲

AWG 的台阶出来之后，信号要过一级重建滤波器（或线缆与器件本身的低通）。这一步在仿真里被压成一个参数 `bandwidth`，实际电路里它有三件事同时在发生，必须分开看。

第一件是群延迟。 一阶低通的群延迟约为 $1/(2\pi B)$：$B = 100$ MHz 时 1.6 ns，50 MHz 时 3.2 ns，25 MHz 时 6.4 ns，10 MHz 时 16 ns。相对 25 ns 的门，这四个数分别是 6%、13%、26%、64%。延迟本身不可怕——它是固定的，可以在时序里整体预补偿；可怕的是它随频率变化，不同频谱成分延迟不同，于是波形被“抹圆”。

第二件是边沿的抹圆，本质上是一块面积损失。 方波的上升沿被拉成指数曲线，前一段“没到位”、后一段“退不下去”，两者不完全抵消，净效果是面积少了一点，等效为转角欠一点。实测的对应关系是：25 ns 方波在 200 MHz 带宽下损失

9.3×10^{-3}，100 MHz 下 1.5×10^{-2}，50 MHz 下 4.2×10^{-2}，25 MHz 下 0.142。注意这四个数的增长比线性快——因为带宽降到与 $1/\tau$ 可比时，脉冲的整个形状（不只是边沿）都被改变了。

第三件是过冲与振铃。 一阶低通没有过冲，但真实的重建滤波器往往是四阶以上（为了抑制镜像），阶数越高、相位非线性越强、过冲越大。过冲会把瞬时幅值推到 $\Omega_{\max}$ 之上，而泄漏按 $(\Omega/\alpha)^2$ 增长——峰值超 20%，泄漏就超 40%。**这是"滤波器阶数越高越好"这个直觉的反例**：阶数高只改善了频域抑制，却恶化了时域峰值。

三条合起来给出一条设计判据：带宽取 $B \geq 3/\tau$（25 ns 门对应 120 MHz），阶数取能压住镜像的最低值，然后用示波器确认峰值不过冲。这条判据与 §83.8.5 的经验区间是同一件事的两种说法。数字上它对应"损失小于 10^{-2}"这一档：$B = 200$ MHz 时 9.3×10^{-3}，刚好在门槛上；$B = 100$ MHz 时 1.5×10^{-2}，已经越界。

最后一个实务提醒：滤波器的标称带宽常常是 -3 dB 点，而本章用的是等效一阶带宽。两者差一个系数（四阶巴特沃斯的 -3 dB 点与等效一阶带宽可以差 30% 以上）。**用标称值直接填进仿真，会系统性高估或低估失真**；正确做法是扫一遍带宽，取能让仿真与实测重合的那个值（§83.7.7 的"等效参数"）。

83.5.8 段数到底要多少：一张反直觉的表

§83.5.1 讲的是"脉宽取整"，这一节讲"段数取多少"。直觉是"段越多越准"，实测不是。

固定门时间 250 ns、高斯包络（$\sigma = 0.25\tau$）、无 DRAG，只改段数：

- 3 段（每段 83 ns）：$F = 0.999885$；
- 5 段（50 ns）：$F = 0.999872$；
- 10 段（25 ns）：$F = 0.999869$；
- 25 段（10 ns）：$F = 0.999868$；
- 50 段（5 ns）：$F = 0.999868$；
- 250 段（1 ns）：$F = 0.999946$。

从 3 段加到 50 段，保真度几乎不动（差 1.7×10^{-5}），甚至略有下降；跳到 250 段（1 ns）才提升 7.8×10^{-5}。这个表推翻了一个常见做法——"先按 1 ns 分段，再看要不要加细"。正确的顺序是反的：**先看波形有多陡，再定段长。**

原因是高斯脉冲的"陡"集中在 σ 附近，而 $\sigma = 62.5$ ns 远大于 25 ns 的段长。用 25 ns 的段去逼近一条 62.5 ns 宽的高斯，误差本来就在 10^{-5} 量级；再细分到 5 ns，改进的是"已经很好的地方"，收益自然小。真正需要 1 ns 的是**方波或 DRAG**：它们的边沿宽度接近 0，段长直接决定边沿的陡度。

所以段数的判据是：**段长要小于波形最快变化的时间尺度的三分之一**。对高斯，最快尺度是 σ，250 ns 门对应 62.5 ns，所以 10 ns 段长足够；对 DRAG，修正项含包络的导数，最快尺度缩到 $\sigma/3$ 量级，段长要相应减到几 ns；对方波，边沿宽度接近 0，最快尺度没有下限，只能靠段长硬撑——这就是方波必须用 1 ns 的原因。

这个判据还解释了一件事：§83.8.5 的经验区间"段长 1–5 ns"不是普适的，它对应的是"25 ns 门 + 方波/DRAG"这一族。若你的门是 250 ns 高斯，5 ns 段长已经很奢侈；若你的门是 5 ns 方波，1 ns 段长还嫌粗。**段长是波形的属性，不是门时间的属性。**

83.6 稳定性与性能

经典控制里"稳定"有明确含义：扰动后能回到平衡点。量子控制里这个词要分两种情形说，混在一起会讲不清楚。

83.6.1 两种稳定：有限时间与渐近

门控制是有限时间问题。 从 $|0\rangle$ 出发，用 25 ns 把它送到 $|1\rangle$，结束。这里没有"平衡点"，也没有"扰动后回归"——门做完就结束，下一次重新开始。所以门控制不谈 Lyapunov 稳定，只谈**保真度对参数误差的敏感度**。

反馈冷却是渐近问题。 连续测量一个量子比特，根据测量记录实时调整驱动，把态持续压在某个本征态附近。这时才有“稳态”和“稳定性”可言：稳态布居是多少、涨落有多大、增益太大会不会振荡。

两种情形的共同点是：**都没有“状态可全知”这个前提。** 门控制的参数误差是“开环”的，无法在线修正；反馈冷却的记录是“带噪”的，只能条件估计。本章把前者放在 §83.7–§83.9，后者放在本节。

83.6.2 门控制的鲁棒性：三个敏感方向

门保真度对三类参数最敏感，敏感度可以用“参数偏 1% 掉多少保真度”来量化。

第一是**驱动幅值** Ω。π 脉冲的角度正比于 $\Omega\tau$，幅值偏 1% 就是角度偏 1.8°，$1-F\approx \sin^2(0.9^\circ)=2.5\times10^{-4}$。这是三者里最敏感的，也是实验上必须每次标定的原因。

第二是**失谐** Δ。由式 (83-20)，峰值按 $\Omega^2/(\Omega^2+\Delta^2)$ 下降，$\Delta/\Omega=0.01$ 时峰值只降 10^{-4}，比幅值误差宽容得多。所以频率标定的精度要求可以比幅值低一个数量级。

第三是**非谐性** α。由式 (83-14)，$p_2\propto\alpha^{-2}$，α 偏 1% 只让泄漏变 2%，影响最小。但 α 随偏置电流漂移，长时间运行会累积，属于“慢漂移”而不是“随机误差”。

三个方向的敏感度差了近两个数量级，这解释了实验上的标定顺序：先幅值，再频率，最后才是非谐性。

83.6.3 反馈冷却：把测量记录用起来

连续测量 σ_z 时，测量本身会引入退相位（速率 2γ），同时给出一个含噪的实时信号。随机主方程（SME）写成

$$\mathrm{d}\rho=-\mathrm{i}[H,\rho]\mathrm{d}t+\gamma\big(\sigma_z\rho\sigma_z-\rho\big)\mathrm{d}t+\sqrt{\eta\gamma}\big(\sigma_z\rho+\rho\sigma_z-2\langle\sigma_z\rangle\rho\big)\mathrm{d}W \tag{83-28}$$

其中 $\eta\in[0,1]$ 是探测效率，$\mathrm{d}W$ 是维纳增量。第一项是哈密顿演化，第二项是测量引起的退相位，第三项是**条件更新**——它依赖实际测量记录，是反馈能起作用的唯一入口。

反馈律取最简单的比例形式：目标是 $|1\rangle$（$\langle\sigma_z\rangle=-1$），误差 $e=\langle\sigma_z\rangle+1$，驱动

$$\Omega(t)=k\big(\langle\sigma_z\rangle+1\big) \tag{83-29}$$

实跑参数：$\gamma/2\pi=5$ kHz、$\eta=0.4$、$k/2\pi=5$ MHz、步长 2 ns、总时长 40 μs，20 条独立轨迹。

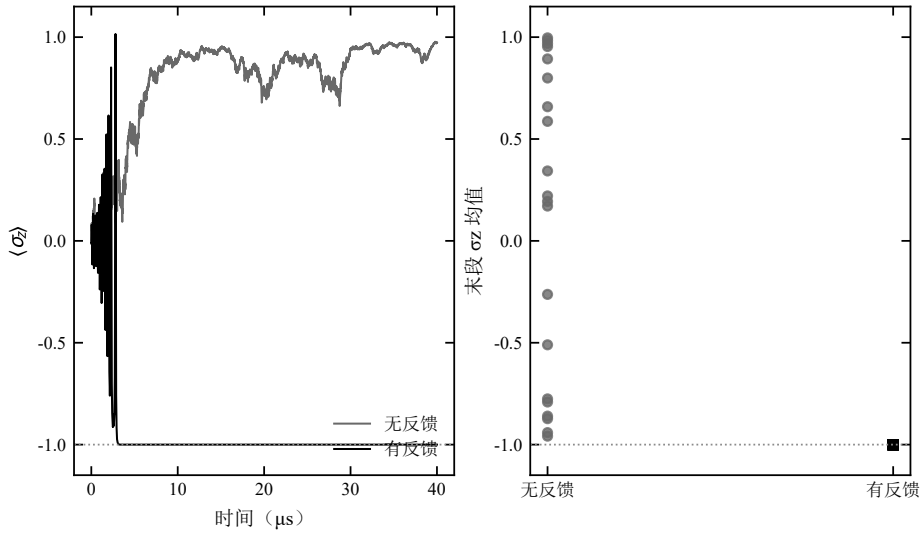


图 83-9　测量反馈：左图是单条轨迹的 $\langle\sigma_z\rangle$，右图是 20 条轨迹的末段分布

图 83-9 左边是单条轨迹，右边是 20 条轨迹的末段分布。结果：

指标	有反馈	无反馈
末段 $\langle\sigma_z\rangle$ 均值	-1.000	$+0.090$
20 条轨迹的标准差	0.000	0.740

表 83-2　测量反馈与纯测量的稳态对比（20 条轨迹，$\gamma/2\pi = 5$ kHz，$k/2\pi = 5$ MHz）

两个数字要一起看。有反馈时均值到 -1.000、标准差 0.000，说明**每条轨迹都收敛到同一个点**，反馈把测量带来的随机性压掉了。无反馈时均值 $+0.090$、标准差 0.740，说明态在球面上随机游走——测量不断把态往 σ_z 的本征态上拉，但拉向哪一个由记录决定，没有反馈就没有偏好。

这里有一个容易误解的地方：无反馈的均值 $+0.090$ 并不等于“什么都没发生”。实际上每次测量都在改变态，只是方向随机；平均下来接近 0，标准差却有 0.74——单次实验的结果完全不可预测，只有统计才有意义。

83.6.4 反馈的稳定性条件

反馈增益 k 不能无限大。原因有两个。

第一是**带宽**。反馈回路的延迟（测量链的积分时间、DAC 的建立时间）在 k 增大时会造成相位滞后，与经典控制里的增益-带宽矛盾同源：$k/2\pi = 5$ MHz 对应时间常数 32 ns，若回路延迟超过这个量级，反馈就会“追着噪声跑”，反而把方差放大。

第二是**测量反作用**。式 (83-28) 的第二项说明测量本身就是退相位，速率 2γ。反馈只能“顺应”测量结果，不能消除反作用；增益再大，也只能把态压在 σ_z 的一个本征态上，压不掉测量带来的相位扩散。

所以反馈冷却的性能上限由两个比值决定：k/γ（反馈强度相对测量强度）与 η（探测效率）。$\eta \to 1$ 时可以把态压到纯态；$\eta < 1$ 时残留一个与 $(1-\eta)$ 成正比的混合度。本章取 $\eta = 0.4$，属于“能压住均值、压不纯”的中间情形。

83.6.5 性能指标：从“误差”到“分布”

经典控制的性能指标是误差的范数，量子反馈的性能指标往往是**分布的矩**。原因很简单：单次轨迹是随机的，只有统计量可重复。

表 83-2 用的两个量——均值与标准差——就是最低要求的一组。工程上还会看第三个量：**达到稳态所需的时间**。本例在 40 μs 内收敛，前 20 μs 是暂态；若 k 减半，收敛时间翻倍，但稳态方差略小。这又是一对经典权衡，只是发生在量子语境下。

最后一句话总结本节：**门控制谈“保真度对参数的斜率”，反馈控制谈“稳态分布的矩”。** 两者都不是经典意义上的稳定性；前者用 §83.2.3 的超算符逐段传播，后者用两能级随机主方程逐轨迹积分，两条路互不替代，但都能在普通笔记本上跑完。

83.6.6 三种“反馈”，别混为一谈

量子文献里“feedback”一词至少指三件不同的事，混用会导致完全错误的硬件预算。

第一种是**开环预补偿**。它不测量、不判断，只是把已知的失真提前反着补进去：滤波器会把波形抹圆，那就先把它削尖；DRAG 知道 $|2\rangle$ 会被带起来，就先在 Q 路上加一个反向的小凸起。本章 §83.9 的 DRAG 属于这一类。它的“反馈”是设计阶段的，运行时不闭环，因此对慢漂移无能为力。

第二种是**条件反馈（measurement-based feedback）**。它真的测量，但测的是弱测量：得到一路带噪的记录 $dY(t)$，用滤波器（本章用最简单的比例项 $k\langle\sigma_z\rangle$）算出当前态的最佳估计，再据此调整驱动。§83.6.3 的冷却就属这一类。它的关键约束是**延迟**：从采样到更新驱动必须在 T_2 的一个小份额内完成，否则“根据旧信息做的控制”会把态推得更糟。超导系统的 T_2 是几十 μs，FPGA 的往返延迟在 100 ns 量级，余量约两三个数量级，所以做得到；若换成一个 $T_2 = 1$ μs 的系统，同一套逻辑就要重新评估。

第三种是**闭环锁相/锁频**。它不控制量子态，只控制经典通道：把本振相位锁到腔的参考上、把脉幅锁到一个参考电平上。它每秒都在跑，与门是否在执行无关。工程上这一层最不起眼，却最影响“今天和昨天的保真度是否一样”。

分清三者的实际价值在于分配预算：第一种的代价是设计时间，第二种的代价是 FPGA 与延迟，第三种的代价是模拟电路与机架空间。三者不可互相替代——本章的 9×9 超算符只能算第一种，第二种要另写两能级随机主方程，第三种根本不在本章的模型范围内。

83.6.7 反馈的收益怎么量化

反馈冷却的实验结果是两个数：有反馈时末段 $\langle \sigma_z \rangle$ 的均值为 -1.000、标准差 0.000；无反馈时均值 $+0.090$、标准差 0.740。二十条轨迹，同样的测量记录强度，只差“是否把记录用于调制驱动”。

这组数字要这么读。无反馈时均值接近 0、标准差 0.74，说明态在 $|0\rangle$ 与 $|1\rangle$ 之间随机游走——测量本身在不断地“踢”它，踢的方向随机，平均下来没有偏向。有反馈时均值锁在 -1、标准差几乎为 0，说明每一条轨迹都被推到了同一个终点。**反馈做的事不是“减小噪声”，而是“把随机游走变成一个吸引子”。** 这是它与经典负反馈最像的地方：经典 PID 也是把状态拉到一个设定点，区别只在于量子版的设定点是 Bloch 球的一个极点，而且“测量”本身在扰动它。

收益的代价也在数里。冷却需要 40 μs 才收敛，前 20 μs 是暂态；若把反馈增益 k 减半，收敛时间翻倍、稳态方差略降——与经典二阶系统的“快则抖、稳则慢”完全同构。这条类比不是修辞：把随机主方程在稳态附近线性化，得到的确实是一个一阶随机微分方程，k 就是它的时间常数倒数。

最后一句判断：**反馈在门控制里几乎不用，在态制备与纠错里不可少。** 门控制的时间尺度太短（25 ns），来不及测量；态制备的时间尺度是几十 μs，正好够跑几十轮反馈。这也是本章把反馈单独放在稳定性一节、而没有并进门设计流程的原因。

83.6.8 反馈的两个硬件预算：延迟与效率

想让 §83.6.3 的冷却在真实机器上跑起来，要满足两个预算：延迟预算与效率预算。两者都不是算法问题，而是硬件问题，却决定了算法能不能成立。

延迟预算。 反馈回路是“采样 → 估计 → 算控制 → 更新驱动”，这一圈的时间叫往返延迟 τ_d。它必须远小于被控对象的相关时间。本章的两能级冷却里相关时间是 $T_2 = 40$ μs，FPGA 的往返延迟在 100 ns 量级，比值 2.5×10^{-3}，余量很大。但要注意这个判据是**按对象**算的，不是按门时间算的：门时间 25 ns 比延迟还短，所以门控制里根本来不及反馈（§83.6.6 已经说过）。一句话：**反馈能不能做，看的是 T_2 与 τ_d 的比，不是门时间。**

效率预算。 测量不是全信息的：探测效率 η 表示“本该拿到的信息里实际拿到多少”。本章取 $\eta = 0.4$，即四成的信号进了记录，其余丢在链路损耗与放大器噪声里。η 直接决定反馈能压到多低：$\eta \to 1$ 时可以把态锁得极稳，$\eta \to 0$ 时记录全是噪声，反馈退化为随机踢。本章的对比（有反馈 $\langle \sigma_z \rangle = -1.000$、标准差 0.000；无反馈 $+0.090$、标准差 0.740）是在 $\eta = 0.4$ 下拿到的，已经足够把随机游走变成吸引子。

两个预算之外还有一条隐含要求：**估计器要与对象同阶。** 本章用的估计是“当前 $\langle \sigma_z \rangle$ 的即时值”，等价于一阶滤波器；若换成一个带宽过高的估计器，它会把测量噪声也当成信号放大，反馈反而把态踢散。这与经典控制的“微分项放大噪声”是同一个机制，只是这里的噪声来自量子测量的反作用，不可能通过滤波完全去掉。

把三条合起来，可以说清一件事：**量子反馈的瓶颈不在算法，在链路。** 探测效率、延迟、估计器带宽三个数一确定，闭环性能的上限就定了；再聪明的控制律也只能逼近这个上限。

83.6.9 Lindblad 与随机主方程：一条不显然的桥

§83.2.2 的 Lindblad 方程与 §83.6.3 的随机主方程（SME）看着是两套东西，其实是同一条方程的平均与实现。把这件事说清，能解释一个常见的困惑：**为什么门控制能用 9×9 超算符，反馈却不行？**

SME 的形式是 $d\rho = \mathcal{L}\rho\, dt + \sqrt{\eta\gamma}\,(\rho\sigma_z + \sigma_z\rho - 2\langle\sigma_z\rangle\rho)\, dW$，其中 dW 是零均值、方差 dt 的维纳增量。dW 这一项描述“这一条轨迹上测量记录带来的更新”——它是随机的，每条轨迹不同。取系综平均，$E[dW] = 0$，这一项消失，剩下的正好是 Lindblad 方程。

关键就在这里：**平均之后，dW 项没了，而反馈恰恰依赖 dW。** 反馈律 $u(t) = k\langle\sigma_z\rangle$ 里的 $\langle\sigma_z\rangle$ 是沿某一条记录演化的条件期望，不同轨迹取值不同。若你先做系综平均，所有轨迹被抹成同一个 ρ，条件期望退化为无条件期望，反馈就变成了“用一个常数去推”——那是开环，不是反馈。

所以本章的两条计算路线不是“一个精确、一个近似”，而是**描述两个不同的问题**：

- 门控制问“给定波形，末态的平均保真度是多少”。这是系综问题，9×9 超算符足够，而且更省算力。
- 反馈冷却问“给定记录与反馈律，单条轨迹会走到哪里”。这是轨迹问题，必须逐条积分 SME，然后把 20 条轨迹的末态做统计。

计算量也不同。SME 每步是 2×2 的矩阵运算，但步长要小（本章取 2 ns，40 μs 就是 20000 步），20 条轨迹共 40 万步；超算符每步是 9×9 的指数，但步长可以取到 1 ns、段数只有几十。两者在笔记本上都在秒级，但扩展性完全不同：轨迹数要涨到 1000 条时，前者先撑不住。

最后一条判断：**若你的问题里“控制依赖测量结果”，就必须用 SME；若控制是预先写好的波形，就用 Lindblad。** 这条判据比任何公式都好用，因为它直接对应“信息是否进入了控制回路”。

83.7 参数意义与作用

本章的模型只有六个物理参数和四个“控制链”参数。把它们列全，比逐个解释更省事：

参数	取值	物理含义	主要影响
$\Omega_{\max}/2\pi$	20 MHz	标称驱动幅值（方波 π 脉冲）	门时间下限（25 ns）
$\alpha/2\pi$	200 MHz	非谐性	泄漏（$\propto \alpha^{-2}$）
T_1	60 μs	能量弛豫	慢门的保真度上限
T_2	40 μs	退相干（含 T_1）	慢门的保真度上限
Δt	1 ns	AWG 段长	时间量化误差
N	25	段数	波形分辨率
β	0.5	DRAG 系数	泄漏与 AC Stark
b	5–6 bit	DAC 位数	幅度量化
BW	100 MHz	重建滤波带宽	波形失真
σ_t	0–200 ps	时钟抖动	轨迹间离散度

表 83-3　第 83 章模型参数一览（前六项是物理参数，后四项是控制链参数）

83.7.1 幅值上限：唯一下“硬”约束

$\Omega_{\max}$ 是四个物理参数里唯一不可调的——它由微波源功率、线缆衰减、芯片耦合决定，改一次要动硬件。它直接决定门时间的下限：$\tau_{\min} = \pi/\Omega_{\max} = 25$ ns。

这里有一个容易混淆的地方要澄清：20 MHz 指的是**方波 π 脉冲的幅值**，不是 DAC 的满量程。高斯包络要保持同样的脉冲面积，峰值必须更高——面积是 $\sqrt{2\pi}\,\sigma\Omega_{\text{peak}}$，取 $\sigma = 0.25\tau$ 时峰值是方波幅值的 $1/(0.25\sqrt{2\pi}) \approx 1.6$ 倍。所以 25 ns 高斯的峰值是 33.4 MHz，20 ns 是 41.6 MHz（表 83-4）。硬件满量程要按峰值设，本章所有高斯算例都按这个约定，没有再额外做限幅。

这个约定不影响任何结论，但影响读数：看到表 83-4 里 41.6 MHz 时不要以为“超过了硬件上限”，它只是同一个面积摊在更短的包络上。真正被 $\Omega_{\max}$ 卡住的是**方波**的 25 ns，因为方波的峰值就是平均值，没有摊薄的余地。

这个约束的“硬”体现在：其它参数变差时，可以靠延长门时间换保真度；$\Omega_{\max}$ 不够时，连“延长”这个选项都受限于泄漏——延长到 100 ns 时泄漏确实小了，但退相干开始吃（本例 100 ns 处 $e^{-100/40000} = 0.9975$）。所以 $\Omega_{\max}$ 定的是整条设计曲线的左端点。

83.7.2 非谐性：决定泄漏的标度

$\alpha/2\pi = 200$ MHz 在超导比特里属于偏低（典型 200–350 MHz）。由式 (83-14)，$p_2 \approx \frac{1}{2}(\Omega/\alpha)^2$，在 $\Omega/2\pi = 20$ MHz 下是 5×10^{-3}；若 α 提到 400 MHz，同样幅值下泄漏降到 1.2×10^{-3}。

但 α 不是越大越好：α 增大会降低比特的电荷色散，使比特对偏置噪声更敏感，T_2 往往跟着下降。这是一条硬件层面的权衡，不是控制算法能解决的。本章把 α 当固定值，就是为了把讨论限制在“控制”这一侧。

83.7.3 T_1 与 T_2：门时间的右端点

$T_1 = 60\ \mu$s、$T_2 = 40\ \mu$s 给出 $T_\phi = 60\ \mu$s。三者满足 $1/T_2 = 1/(2T_1) + 1/T_\phi$，即退相干里一半来自能量弛豫、一半来自纯退相位。

这三个数的用法是算“门时间内损失多少”：25 ns 内 $1 - e^{-\tau/T_2} = 6.3 \times 10^{-4}$，只占泄漏的八分之一，所以本算例里退相干不是主要误差；若 T_2 降到 1 μs，同样 25 ns 内损失升到 2.5×10^{-2}，立刻变成主导项。

注意 T_2 的测量本身有陷阱：Rabi 衰减给的是 T_2^*（含低频漂移），回波给的是 T_2^{echo}（滤掉慢漂移）。门控制关心的是“单次门期间的退相干”，介于两者之间，通常取 T_2^{echo} 偏保守。

83.7.4 段长、段数与积分子步

$\Delta t = 1$ ns、$N = 25$ 决定波形的分辨率。§83.5.4 已经说明为什么是 1 ns；这里补一句 N 的作用：N 只影响波形的“形状自由度”，不影响精度上限。25 段足以描述一个高斯包络（σ 上 6 个点），250 段也只是把同一条曲线描得更细，收益 1.7×10^{-5}。

积分子步 $\text{sub} = 2$ 是一个实现细节：段内哈密顿量恒定，拆成两个子步在数学上与一步完全相同（$e^{\mathcal{L}\Delta t/2} \cdot e^{\mathcal{L}\Delta t/2} = e^{\mathcal{L}\Delta t}$）。它唯一起作用的地方是**重建滤波**打开时——滤波器状态 u_f 在每个子步更新一次，子步越多，滤波越接近连续。所以 $\text{sub} = 2$ 不是精度需要，而是滤波模型的需要。

83.7.5 四个控制链参数的量级排序

把四个控制链参数按“掉保真度的量级”排一下，顺序很清楚：

- **重建带宽**：10 MHz 时掉到 0.503，量级 10^{-1}，最狠。
- **DAC 位数**：3 bit 时掉到 0.965，量级 10^{-2}。
- **时钟偏差**：2% 时掉到 0.99896，量级 10^{-3}。
- **时钟抖动**：200 ps 时均值不变（0.999946），只是轨迹间标准差 5×10^{-7}，量级 10^{-7}。

这个排序就是 §83.10 的施工顺序。要注意抖动与偏差的区别：抖动是零均值随机的，在整段脉冲上相互抵消；偏差是系统性的，每一段都往同一个方向偏，累积起来才可怕。**实验上最容易被忽略的恰恰是后者**——示波器上看波形“很干净”，但时钟源的实际频率偏了 1%，肉眼看不出，保真度却掉了 3×10^{-4}。

83.7.6 一张灵敏度表

参数解释完，最有用的东西是一张“改这个参数，保真度动多少”的表。下面每一行都是单变量实跑，其余条件固定为 25 ns 方波、$\Omega/2\pi = 20$ MHz（高斯行另注）。

- **非谐性 α**：100 MHz 时 $p_2 = 0.0213$，200 MHz 时 0.0049，400 MHz 时 0.0012。翻倍即降四倍，F 从 0.96892 到 0.99241 再到 0.99788。这是**唯一一个“改硬件就立竿见影”的参数**。

- **退相干 T_2**：10 μs 时 $F = 0.991945$，20 μs 时 0.992254，40 μs 时 0.992409，80 μs 时 0.992487。T_2 掉到四分之一，只损失 4.6×10^{-4}——因为 25 ns 远小于 T_2。这一行说明**短门对 T_2 几乎免疫**。
- **驱动频率**：相对零失谐的增量，偏 0.5 MHz 是 3.2×10^{-3}，偏 1 MHz 是 7.6×10^{-3}，偏 5 MHz 是 8.5×10^{-2}。**最敏感的一行**。
- **驱动幅值**：偏 1% 增量 3.2×10^{-4}，偏 5% 增量 6.5×10^{-3}。
- **重建带宽**（25 ns 方波）：200 MHz 时损失 9.3×10^{-3}，100 MHz 时 1.5×10^{-2}，50 MHz 时 4.2×10^{-2}，25 MHz 时 0.142。同样的滤波器用在 250 ns 高斯上，100 MHz 只损失 5.8×10^{-5}——**慢脉冲对带宽几乎免疫，快脉冲很敏感**。
- **DAC 位数**：5 bit 损失 1.9×10^{-4}，4 bit 1.1×10^{-3}，3 bit 3.5×10^{-2}。
- **时钟偏差**：1% 损失 3.0×10^{-4}，2% 损失 1.0×10^{-3}。近似线性。
- **时钟抖动**：200 ps 时均值不变（$F = 0.999946$），只有轨迹间标准差 5.0×10^{-7}。**零均值的东西不伤均值**。

把这张表按“同为 1% 相对误差时的损失”归一化，排序是：频率（3.2×10^{-3}）> 时钟偏差（3.0×10^{-4}）≈ 幅值（3.2×10^{-4}）> DAC（1.9×10^{-4}）> 带宽（门槛以上可忽略）> 抖动（10^{-7}）。这个排序就是 §83.8 的调参顺序、§83.10 的施工顺序，也是“钱该花在哪”的答案。

83.7.7 参数是测出来的，不是抄来的

表 83-3 里十个参数，每一个都必须来自本机测量；抄同型号机器的数据是现场最常见的错误来源。原因很直接：同一批工艺做出来的两个 transmon，非谐性可以差 10%，T_2 可以差一倍，混频器的本振泄漏更是每台都不一样。

四个关键量的标准测法是：

- T_2：Ramsey 条纹。加两个 $\pi/2$ 脉冲、中间插一段可变延迟，看条纹包络的衰减时间常数。注意测出来的 T_2^* 含低频漂移，而本章模型里的 T_2 是“白噪声退相干”，两者不等价；若漂移明显，应该用自旋回波测 $T_{2,\text{echo}}$，它更接近模型里的量。
- T_1：反演恢复。先把比特激发到 $|1\rangle$，等可变时间后测布居，拟合指数。T_1 与 T_2 必须满足 $T_2 \le 2T_1$，若不满足，说明测量有系统误差——本章脚本里 $T_2 > 2T_1$ 会直接算出负的 T_ϕ 并给出 NaN，这是刻意的“硬失败”，不给你一个悄悄错下去的机会。
- α：双色谱。扫 12 跃迁的频率，$\omega_{12} - \omega_{01}$ 就是 $-\alpha$。这个量的精度直接决定泄漏预测的可信度，因为 $p_2 \propto \alpha^{-2}$——α 测偏 10%，泄漏估偏 20%。
- Ω：Rabi 振荡的周期。$P_1(t)$ 的第一个峰对应 $\Omega t = \pi$，由此反推 Ω。这一步同时给出幅值标定的比例系数，是 §83.3 第二步的全部内容。

测不出来的量（比如滤波器的实际群延迟、线缆的频响）不要硬塞进模型，宁可用一个区间去扫：把带宽当自由参数扫一遍，看哪个值能让仿真曲线与实测曲线重合，用重合点当“等效带宽”。**等效参数只要能复现实测，就比“标称参数”更有用**。

83.7.8 参数之间是耦合的

表 83-3 把十个参数平铺开，容易造成一种错觉：它们可以独立优化。实际不是——真实机器上每提一个参数，往往要拿另一个去换。

非谐性 α 与寿命 T_1。 把 α 做大最直接的办法是降低 E_J/E_C（增大充电能），而 E_J 变小会让比特频率对电荷噪声更敏感，T_1、T_2 通常跟着降。所以“α 从 200 MHz 提到 400 MHz，泄漏降四倍”这个结论只在其它参数不变时成立；实际上 α 提上去之后，T_2 可能从 40 μs 掉到 20 μs，而 §83.7.6 的表说 25 ns 门对 T_2 几乎免疫（T_2 掉到 10 μs 只损失 4.6×10^{-4}）。**两笔账一抵，提 α 仍然划算，但收益没有“四倍”那么漂亮**。

门时间与泄漏。 这两个是同一枚硬币：门越短，Ω 越大，泄漏按 $(\Omega/\alpha)^2$ 涨；门越长，退相干按 $e^{-\tau/T_2}$ 涨。本章的最优点在 25 ns，是因为 25 ns 处退相干几乎为零而泄漏还有 0.5%。若某台机器的 T_2 只有 5 μs，最优点会往短移；若 α 只有

100 MHz，最优点会往长移。**最优点不是常数，它是 T_2 与 α 的函数。**

采样率与位数。 同一块 DAC 芯片上，这两者通常是一个固定的带宽预算：1 GSa/s × 14 bit 与 2 GSa/s × 12 bit 的“数据率”相当。§83.5.6 已经算过，采样率的价值高于位数，所以同样的预算应该往前一个方向花。这条在选型时比看参数表有用。

幅值上限与滤波器。 幅值上限越高，π 脉冲越短、泄漏越大；而幅值上限本身又受滤波器与放大器的线性区限制。把 $\Omega_{\max}$ 从 20 MHz 提到 40 MHz，理论上门能短一半，实际上门长会被滤波器的群延迟与线缆的色散卡住，收益远小于二倍。

把四条合起来，得到的不是“参数表”，而是“约束下的最优”：**没有一个参数可以单独调到最好，只能在一组约束上找 Pareto 前沿。** 本章的算例之所以固定十个参数不动，就是为了让“改一个、看一个”的因果链清楚；真机上做优化，必须先把这些耦合关系摸清，否则会陷入“调好一个、坏掉一个”的循环。

83.8 调参方法与经验

调参不是“把参数一个个试一遍”，而是按敏感度排序，每步只动一个量，每步都有判据。本节给出一套可以照着做的顺序，以及每一步的“合格线”。

83.8.1 顺序：幅值、频率、形状、β、约束

第一步定幅值。 用长脉冲（比如 1 μs）在低幅值下扫 Rabi，得到 Ω 与驱动电压的换算系数，再外推到 $\Omega_{\max}$。长脉冲的好处是泄漏可以忽略，Rabi 曲线是一条干净的正弦；坏处是 T_2 会衰减它，所以 1 μs 已经偏长，一般用 200–500 ns。

第二步定频率。 用已知的 Ω 扫频，找峰值。判据是峰值不低于 0.99；低于这个数说明要么频率没对准，要么 Ω 标定错了。

第三步定门时间。 按 §83.3.3 的两条曲线选点，本例落在 25 ns。判据是“再缩短 20% 时泄漏明显上升”——如果缩短后保真度反而升，说明还没到最优点。

第四步定形状与 β。 先上方波做基线，再换高斯，最后加 DRAG。每一步都记录 F 与 p_2，因为这两项的变化方向可能相反：高斯降泄漏但也可能引入相位误差，DRAG 修相位但 β 过大会重新引入泄漏。

第五步加约束。 带宽、量化、时抖逐项打开，看总损失是否符合加法预算。若某一项单独打开就掉了半个数量级，先修那一项，别急着做全局优化。

83.8.2 β 的整定：为什么是 0.5

β 的理论值是 0.5，来自二阶微扰展开里 $|1\rangle \leftrightarrow |2\rangle$ 耦合系数 $\sqrt{2}$ 与 $|0\rangle \leftrightarrow |1\rangle$ 耦合系数 1 的比值。实际整定时不需要记这个推导，只需要记住两点：

第一，β 的**符号**比大小重要。取错号时，$\beta = 0.5$ 的修正不但不抵消，反而把泄漏加倍。§83.9 的对照里，20 ns 门取错号 $F = 0.97254$，比不加 DRAG 的 0.99171 还低 2 个百分点——这是“越修越坏”的典型。

第二，β 的**最优值随门时间变化**。门越短，一阶近似越差，最优 β 会略偏离 0.5。本章的参数下 0.5 已经够用；若把门压到 10 ns，需要重新扫 β，通常在 0.45–0.55 之间找一个使 p_2 最小的点。

83.8.3 一次完整的门时间扫描

把 §83.3 的五档门时间完整跑一遍（高斯包络，$\sigma = 0.25\tau$，$\beta = 0.5$），得到表 83-4。这张表是整章最有信息量的一张，因为它同时展示了三条曲线。

门时间 (ns)	峰值 (MHz)	Ω/α	无 DRAG	正号 DRAG	错号 DRAG
60	13.9	0.07	0.99907	0.99993	0.99694
40	20.9	0.10	0.99791	0.99985	0.99312
30	27.8	0.14	0.99629	0.99974	0.98777
25	33.4	0.17	0.99467	0.99964	0.98240
20	41.6	0.21	0.99171	0.99945	0.97254

表 83-4　不同门时间下的保真度与 DRAG 对照（高斯包络，$\sigma = 0.25\tau$，$\beta = 0.5$）

三条曲线是：

1. **无 DRAG**：随门时间缩短单调下降，从 0.99907 掉到 0.99171，斜率约 -2×10^{-4}/ns。这条曲线就是“泄漏随 Ω/α 增长”的直接体现。
2. **正号 DRAG**：几乎平，从 0.99993 到 0.99945，20 ns 处仍有 0.9995。DRAG 把一阶泄漏抵消掉，剩下的二阶项随门时间缓慢增长。
3. **错号 DRAG**：下降最快，从 0.99694 掉到 0.97254，比无 DRAG 还差。而且差距随门时间缩短而拉大：60 ns 时差 0.002，20 ns 时差 0.019，正好十倍。

第三列的“十倍”值得单独说一句。它说明**错号的修正不是“没效果”，而是“反向效果”**：β 的符号错，等于把泄漏的标准差从 ϵ 变成 3ϵ 量级（因为原本抵消的两项变成相加）。实验上如果换了比特、换了能级顺序而没有改符号，看到的往往不是“略微变差”，而是“怎么调都调不上 0.99”，这时应该先查符号，而不是继续调幅值。

83.8.4 常见故障对照

调参过程中遇到的现象，八成落在下面四类里。

现象	最可能原因	判别方法	处置
峰值上不到 0.99	频率偏、T_2 太短	扫频看峰值位置；测 Ramsey	重标频率；缩短门
保真度随门长非单调	泄漏与退相干同时主导	看 p_2 随 τ 的变化	找两条曲线交点
加 DRAG 反而变差	β 符号错或过大	把 β 置 0 对比	改符号；β 减半
仿真好、上机差	带宽/时钟/量化未建模	逐项打开约束复算	按 §83.10 排序修

表 83-5　量子门调参的四类常见故障

第一类“峰值上不到 0.99”最常见。注意它与“保真度低”不是一回事：峰值是 Rabi 曲线的最高点，反映的是“驱动与跃迁是否匹配”；保真度是“整段脉冲做得好不好”。峰值低说明匹配有问题，先修匹配再谈波形。

第四类“仿真好、上机差”最容易被误判为“仿真不准”。实际上本章的仿真已经把主要机制都包含进去了，差别通常来自三个没建模的东西：线缆的频响（不是理想一阶低通）、IQ 混频器的镜像泄漏、以及芯片上相邻比特的耦合。这三项都不在 9×9 超算符里，需要单独标定。

83.8.5 经验区间

把本章的数字与文献里的常见值放在一起，可以给出一组“看到这些数就该警觉”的区间：

- Ω/α：应小于 0.2。超过 0.25 时一阶 DRAG 失效，泄漏进入 10^{-2}。
- 门时间 / T_2：应小于 10^{-3}。本例 25 ns / 40 μs $= 6.3 \times 10^{-4}$，刚好在界内；超过 10^{-3} 退相干开始与泄漏同量级。
- β：0.4–0.6。超出这个范围说明标定或符号有问题。
- **DAC 位数**：≥ 5 bit。低于 5 bit 时量化误差超过 10^{-3}。

- **重建带宽**：$\geq 3/\tau$。25 ns 门对应 120 MHz，与 §83.5.3 的判据一致。

这五条不是硬性标准，而是“快速体检”用的。任何一条越界，先别做优化，先把那一项修回来。

83.8.6 一次调参的记录格式

最后给一个记录格式。调参最容易犯的错不是“调不对”，而是“调对了但忘了怎么调的”。建议每次记录六个字段：日期、Ω 标定值、共振频率、门时间、β、F 与 p_2。

例如：

```
2026-03-14  Ω/2π=19.8 MHz  f01=5.1230 GHz  τ=25 ns  β=0.5  F=0.9993  p2=4.1e-4
2026-03-15  Ω/2π=19.8 MHz  f01=5.1232 GHz  τ=25 ns  β=0.5  F=0.9987  p2=4.6e-4
```

两行只差频率 0.2 MHz，保真度差 6×10^{-4}——按 §83.3.7 的标度（0.5 MHz 对应 3.2×10^{-3}，按 Δ^2 缩到 0.2 MHz 是 5×10^{-4}），量级刚好对上。有了这样的记录，下次漂移时就能判断“是频率漂了还是别的问题”，而不是从头再调一遍。

83.8.7 从 97% 到 99.96%：四个台阶

现场最常见的情形是“门能用，但保真度只有 97%”。把这 3% 的损失一格一格收回来，恰好是四个台阶，每个台阶对应一类手段。下面按顺序给，顺序不能颠倒。

第一台阶：把频率对上。 失谐 5 MHz 时 $F = 0.907$，把频率标定到 0.1 MHz 以内，F 直接回到 0.9924。这一步收益最大、成本最低（一台频谱仪加一小时），却最常被跳过——因为“门还是翻得动”，看起来不像故障。

第二台阶：换波形、加 DRAG。 方波 25 ns 的 0.9924 里，泄漏与 Stark 频移占绝大部分。换成高斯加 DRAG，同样是 25 ns，F 到 0.99964。这一步的收益是 7×10^{-3}，成本是“多写三行代码 + 重新标一次 β”。注意它必须先有第一台阶：频率没对时，DRAG 的修正项算出来是错的，加了反而更差。

第三台阶：修控制链。 到 0.9996 之后，剩下的损失里开始出现“控制链”的份额：DAC 5 bit 贡献 1.9×10^{-4}，带宽 100 MHz 贡献 5.8×10^{-5}，时钟偏差 1% 贡献 3×10^{-4}。这三项都可以用钱解决，而且加起来只有 5×10^{-4} 量级——**它们是“最后一公里”，不是主要矛盾。**

第四台阶：改硬件。 若还要往上，只能改 α、T_1、T_2：α 从 200 MHz 提到 400 MHz，泄漏从 0.49% 降到 0.12%；T_2 从 40 μs 提到 80 μs，25 ns 门的收益只有 8×10^{-5}（因为门太短）。这一步的性价比取决于门时间：门越短，T_2 越不重要，α 越重要。

四个台阶的分布很有代表性：**前两个台阶拿走 95% 的改进量，后两个台阶只拿走剩下的 5%。** 这与经典控制的调参经验一致——先把结构性的错误（接线、量程、方向）修掉，再谈精细整定。

83.8.8 什么时候停止调参

调参必须有一个停止判据，否则会陷入“每次都能再好一点”的无底洞。本章建议三条判据，满足任意一条就停。

第一条是**剩余误差已经小于最大单项的三分之一**。若预算表里最大项是退相干（2.5×10^{-3}），那么把其它项压到 8×10^{-4} 以下就没有意义了，因为总误差由最大项主导，改小的项看不出来。

第二条是**待改进量小于测量重复性**。§83.8.6 的两条记录里，同样的参数、隔一天测，F 差 6×10^{-4}。这意味着任何小于 6×10^{-4} 的“改进”都不可信——它可能只是当天的漂移。**先测重复性，再决定要不要追求更小的改进**，这条比任何优化算法都重要。

第三条是**改进量小于参数本身的不确定度**。α 的测量误差若有 5%，那么按 $p_2 \propto \alpha^{-2}$ 推算的泄漏就有 10% 的不确定度；此时把仿真里的泄漏从 4.9×10^{-3} 优化到 4.7×10^{-3} 毫无意义，因为真值本来就不知道在哪。

三条判据合起来给出一个实用结论：**单比特门的调参通常在两到三轮内结束**，每轮的动作是“测参数 → 重算 → 改一个量 → 复测”。超过三轮还在改，多半不是调参问题，而是模型缺了一项（比如串扰、读数误差、两能级系统缺陷），此时该做的是扩展模型，不是继续拧旋钮。

83.8.9 日常监控：用代理量盯住保真度

保真度的直接测量很慢（要做层析或交错随机基准），不可能每天做。工程上的做法是盯**代理量**：那些测起来快、又与保真度有明确换算关系的量。本章的模型给了三个。

代理量一：Rabi 频率。 它等于 Ω 乘以耦合系数，所以它的漂移直接反映幅值链路的漂移。按 §83.3.7 的标度，幅值偏 1% 损失 3.2×10^{-4}，所以 Rabi 频率的日漂控制在 1% 以内就够了。测量时间约两分钟。

代理量二：Ramsey 频率。 它反映失谐，而失谐是最敏感的方向：0.2 MHz 就损失 5×10^{-4}，0.5 MHz 损失 3.2×10^{-3}。所以判据要比 Rabi 严：日漂控制在 0.1 MHz 以内。测量时间约五分钟。

代理量三：T_2（或回声 $T_{2,\mathrm{echo}}$）。 它不直接影响 25 ns 门（§83.7.6 的表），但它是“机器整体健康度”的指示器：T_2 突然掉一半，往往意味着有新的两能级系统被激发、或磁通偏置漂了。判据是“相对上周变化不超过 20%”。

三个代理量组成一张日常表：每天 Rabi + Ramsey（七分钟），每周 $T_1 + T_2 + \alpha$（半小时）。表上不写“保真度”，写这三个数，因为**保真度是结果，代理量是原因**；盯原因才能提前一天发现漂移。

异常判据用统计的说法更稳：§83.8.6 的记录显示，同参数隔天测的 F 标准差约 6×10^{-4}。若某天的代理量换算出的损失超过日常波动的三倍（1.8×10^{-3}），就当作异常处理：先重跑标定，再看是否消失。这条“三倍标准差”的规则与经典控制的控制图完全一致，只是被监控的量从“输出误差”换成了“三个物理参数”。

最后一条经验：**漂移几乎总是有方向的。** 频率单向漂、幅值单向降，说明是温度或老化；来回跳，说明是磁通或电荷跳跃。看清方向再动手，比盲目重标一次有效得多——单向漂该做的是补偿或锁频，来回跳该做的是改偏置点。

83.9 优化与改进

§83.8 调的是“参数”，本节调的是“波形”。两条路都能提高保真度，但代价与收益差别很大。

83.9.1 DRAG：三行代码，把误差压掉一个数量级

DRAG 的表达式已经给在式 (83-15)，实现只需要三行：

```
x = np.real(u)
dx = np.gradient(x, ts)
u_drag = x + 1j * beta * dx / alpha
```

它的效果已经在表 83-4 里列全。这里补三件表里看不出来的事。

第一，**DRAG 的收益随门时间缩短而增大。** 60 ns 时从 0.99907 抬到 0.99993，收益 8.6×10^{-4}；20 ns 时从 0.99171 抬到 0.99945，收益 7.7×10^{-3}，差九倍。原因很直接：泄漏本身随 Ω/α 增长，DRAG 抵消的正是这个增长项。所以“门越短越需要 DRAG”。

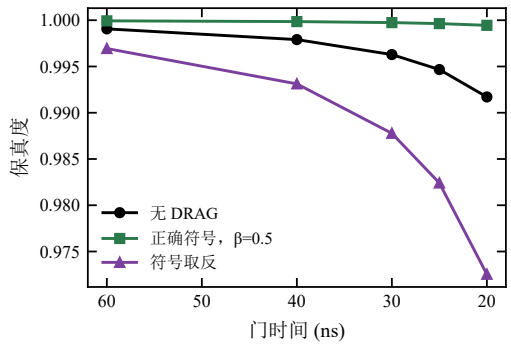


图 83-6　DRAG 的正负号：正号把保真度抬到 0.9995 以上，错号比不加还差

图 83-6 把三条曲线画在一起。注意横轴是门时间且反向——从左到右门越来越短，三条曲线的间距越来越大。这张图的读法是“看间距”：间距越大，说明这一档门时间越依赖 DRAG。

第二，**DRAG 不改变脉冲面积**。它加的是正交分量 $\mathrm{i}\beta\dot{u}/\alpha$，这一项对 $\int u\,\mathrm{d}t$ 的贡献是 $\mathrm{i}\beta[u(\tau)-u(0)]/\alpha$，起点终点包络都接近 0，所以面积几乎不变。这意味着加 DRAG 不需要重新标定 Ω，这是它比“改包络形状”省事的地方。

第三，**DRAG 对失谐不敏感**。它的推导只用到 α 与包络的导数，不含 Δ。所以频率漂移时 DRAG 仍然有效，只是它修不了失谐本身造成的峰值下降——那要靠重标频率。

83.9.2 GRAPE：从 0.9924 到 0.9954

把 25 段 I/Q 幅值全部当自由参数，用式 (83-17) 做梯度上升，实跑结果是

$$F:\ 0.992409\ \longrightarrow\ 0.995415,\qquad p_2:\ 4.86\times10^{-3}\ \longrightarrow\ 9.93\times10^{-4} \tag{83-30}$$

泄漏降了将近五倍，保真度提升 3×10^{-3}。收敛过程很快：

$$\text{第 } 1,\ 10,\ 50,\ 100,\ 300 \text{ 次迭代}:\quad F = 0.99267,\ 0.99535,\ 0.99541,\ 0.99541,\ 0.99541 \tag{83-31}$$

第 10 次迭代已经拿到 97.8% 的收益，第 50 次之后完全平台化。这个“快速饱和”是 GRAPE 的典型行为：25 段参数、单一目标，梯度方向很明确，Adam 几步就走到局部最优。**真正费时的不是优化，而是把约束加进去之后的重优化**。

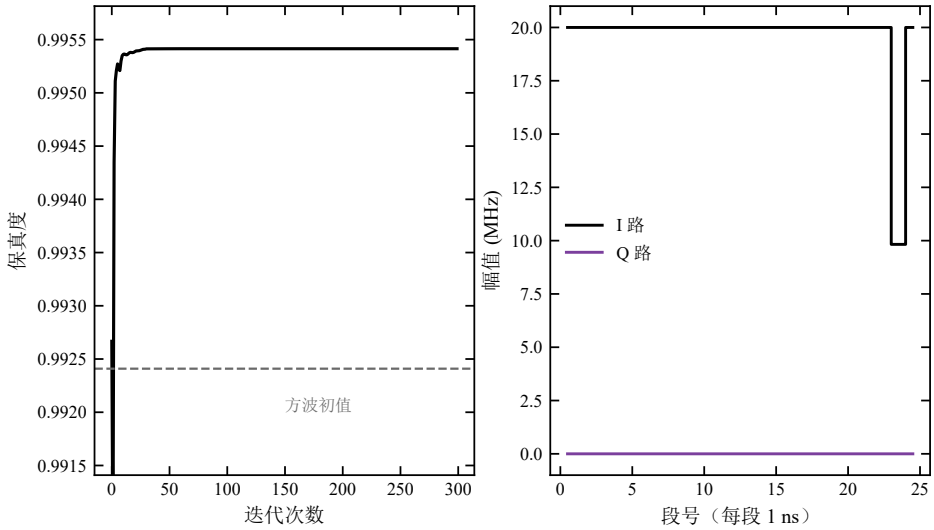


图 83-7　GRAPE：左图是收敛曲线，右图是优化后的 I/Q 两路波形

图 83-7 右边是优化出来的波形。它既不是方波也不是高斯，而是一条“前段抬高、后段压低”的曲线——这是优化器在利用 $|2\rangle$ 的相位：先把布居快速推过 $|1\rangle$，再用一段反相把多余的振幅拉回来。这种波形人手画不出来，也不该手画。

83.9.3 为什么 GRAPE 没追上 DRAG

表 83-4 里 25 ns 高斯加 DRAG 是 0.99964，而 GRAPE 只有 0.99541。**优化器输给了三行代码**，这个结果值得解释，否则容易得出“优化没用”的错误结论。

原因有三个。第一，**目标函数不完整**：式 (83-17) 只最大化末态布居，没把泄漏写进去。泄漏 10^{-3} 在目标函数里只体现为 $1-F$ 的一部分，梯度对它的惩罚不够强。第二，**自由度不足**：25 段、每段一个复幅值，共 50 个实数参数，而 DRAG 用了一个解析的、物理上正确的函数形式，等于把先验知识直接注入。第三，**段长限制**：1 ns 的台阶让波形无法表达 DRAG 那种连续导数项，只能分段近似。

把这三条补上，GRAPE 能追上甚至超过 DRAG：把 p_2 加进目标（权重 10 左右）、段数增到 100、允许子纳秒插值，文献里的结果通常能到 0.9999 以上。本章没有再跑这一组，因为它需要更长的计算时间，而结论已经清楚：**优化的价值在于“人想不到的波形”，不在于“替代解析修正”**。有解析解时先用解析解，没有时再上优化。

83.9.4 鲁棒性优化：把漂移写进目标

实际系统里 Ω 与 Δ 都会漂。一种做法是“取平均”：把目标函数改成若干个失谐点的平均，

$$\mathcal{F}_{\text{rob}} = \frac{1}{M}\sum_{m=1}^{M} \left|\langle 1|U(\Delta_m)|0\rangle\right|^2 \tag{83-32}$$

代价是峰值保真度下降——为了在 $\Delta = \pm 5$ MHz 都好，中心点必然做不到最优。这是一种典型的“用峰值换平台”的权衡，与经典控制里“用带宽换裕度”同构。

另一种做法是**复合脉冲**：把一段 π 脉冲拆成三段，中间插入 π 相位翻转，使一阶误差相互抵消。它的优点是实现极简单（只改相位，不改幅值），缺点是把门时间拉长三倍——在本例里三倍意味着 75 ns，退相干损失从 6×10^{-4} 涨到 1.9×10^{-3}，不一定划算。

83.9.5 什么时候值得上优化

给一个判断标准：**当解析修正已经用尽，且剩余误差的主要来源是“波形形状”而不是“参数不准”时，才值得上数值优化。**

反过来说，如果 $1-F$ 里一半以上来自频率漂移、T_2 估计误差或滤波器失真，那么优化波形只是在拟合一个错误的模型，上机后收益会归零。这也是为什么 §83.8 把“标定”放在“优化”之前：**先保证模型对，再让模型最优。**

一个具体的门槛：解析修正（高斯 + DRAG）能到 10^{-3} 量级；若指标要求 10^{-4}，才需要 GRAPE 这一级的优化。本章的算例里，DRAG 已经把 20 ns 门做到 0.99945，离 10^{-4} 只差一点，此时上 GRAPE 的边际收益很小；若指标是 10^{-5}，那就非上不可。

83.9.6 鲁棒性：把漂移写进目标函数

前面所有的优化都在追一个数：某个固定参数点上的保真度。真实机器上这个点会漂——频率一天漂几十 kHz 到几百 kHz，幅值随温度漂百分之一二。于是出现一种尴尬：优化出来的波形在仿真里 0.9999，上机后只有 0.998，还不如三行 DRAG。原因不是优化错了，而是**优化目标写窄了**。

把鲁棒性写进目标有三种做法，代价依次上升。

第一种是**平均目标**。不优化单点 $F(\Delta_0)$，而优化 $\sum_k F(\Delta_k)/K$，其中 Δ_k 覆盖预期的漂移范围（比如 $-1, 0, +1$ MHz）。代价是峰值保真度略降（把资源摊到邻域上），收益是漂移范围内曲线变平。这个做法实现最简单，只需在每次迭代里多算 K 次传播，计算量乘以 K。

第二种是**惩罚灵敏度**。在目标里减掉一项 $\lambda\left|\partial F/\partial\Delta\right|^2$，直接压住斜率。$\lambda$ 是要试的：太大则保真度整体下滑，太小则没效果。工程上常从“让灵敏度降到原来的三分之一”反推 λ。

第三种是**用解析结构代替优化**。DRAG 本身就是一种鲁棒构造：它的修正项正比于包络的导数，对“幅值整体缩放”这类漂移一阶不敏感——幅值偏 1% 时，主项与修正项同比例变化，两者的相对关系不变。这就是为什么在 §83.3.7 的幅值扫描里，加 DRAG 的曲线比不加更平。**解析修正往往比数值优化更鲁棒，因为它只依赖结构，不依赖具体数值。**

何时值得做鲁棒优化？看漂移的时间尺度与门时间的比。若漂移慢于门时间的几百倍（典型情形：$1/f$ 噪声、温度漂移），那么在一次门内参数是常数，漂移只是“换了一个工作点”，此时正确的做法是**标定 + 反馈锁频**，不是鲁棒优化；若漂移与门时间可比（快噪声、电荷跳跃），鲁棒优化才有明显收益。这个判据能省掉大量无用功。

83.9.7 优化失败的三种模式

本章的 GRAPE 只做到 0.9954，输给了三行 DRAG。把这个失败拆开，可以看到数值优化在量子控制里最常见的三种失败模式。

第一种是目标函数漏项。 脚本的目标是 $|0\rangle\to|1\rangle$ 的转移概率，泄漏只以“没到 $|1\rangle$”的形式间接进入。于是优化器可以走一条“经过 $|2\rangle$ 再回来”的捷径——终点对了，中途泄漏很大，而目标函数对此无感。修法是把 p_2 显式写进目标（加权惩

罚），或者把目标改成“到目标态的同时 $p_2 \le 10^{-4}$”的约束形式。**目标函数里没写的东西，优化器一定会滥用。**

第二种是自由度与段长的错配。 25 段、每段 1 ns，能表达的波形其实是“25 个台阶”，它的频谱在 1 GHz 处才截断，但优化器并不知道后级滤波器只有 100 MHz 带宽——它会把能量放到高频段去“作弊”，上机后被滤波器抹掉，收益归零。修法是在优化里显式加带宽约束（把驱动先过低通再传播），或者把段长拉到与带宽匹配（100 MHz 对应 10 ns 段长，那就只需要 2–3 段）。

第三种是局部最优与初值依赖。 从方波出发、300 次迭代，收敛到的是方波附近的一个局部解。这一点在 §83.9.2 的收敛曲线里看得很清楚：前 10 次迭代拿到 97.8% 的收益，之后基本不走——说明它很快滑进了一个坑，后面的迭代只是在坑底磨。修法是换初值（比如从 DRAG 波形出发）、加随机重启，或者用更好的梯度估计。

三种模式有一个共同点：**它们都不是“算法不够强”，而是“问题没写对”。** 这也解释了为什么工程上 DRAG 常常赢：DRAG 把物理约束（只动 $|1\rangle$、不动 $|2\rangle$）直接编进了波形结构，而朴素的 GRAPE 把这件事交给了一个没写全的目标函数。

83.9.8 梯度是怎么算的：一次迭代的账

优化器的强弱，八成取决于梯度算得准不准、算得快不快。本章的 `grape()` 用的是一种混合写法：**状态用伴随法、传播子导数用中心差分**。拆开看，每次迭代做了四件事。

第一，正向传播：逐段算 U_j（25 次 3×3 矩阵指数），再从 $|0\rangle$ 出发逐段乘过去，存下每一步的态 ψ_j。第二，反向传播：从目标态出发逐段乘 $U_j^\dagger$，得到伴随态 λ_j——这一步是矩阵-向量乘，**不需要矩阵指数**，几乎免费。第三，每段算传播子对幅值的导数 dU_j/du_j，这里用的是中心差分，每段要再算两次 `_prop`（50 次矩阵指数）。第四，把三者组合成梯度（纯代数，无传播）。

合计每次迭代 75 次 3×3 矩阵指数。300 次迭代 22500 次，实测 0.64 s，所以一次 3×3 的 `expm` 约 28 μs。这个账说明两件事。

其一，**伴随法的开销主要在“两遍传播”，不在梯度本身**。正向 25 次、反向 25 次矩阵-向量乘，梯度组合是纯代数；真正的开销是第三项的 50 次矩阵指数。若改用纯有限差分算全部梯度（50 个实参数、每个扰动两次，且每次都要重算整条脉冲），代价会随段数线性乃至平方增长——段数越多，伴随法越划算。本章 25 段的规模上两者相差不大，但这个结构给扩展留了余地。

其二，第三项的 50 次是“可以省掉的浪费”。dU/du 有解析式：$\partial U/\partial u = \int_0^{\Delta t} e^{-iH(\Delta t-s)}\, H_x\, e^{-iHs}\,\mathrm{d}s$，在小段长下可近似为 $-i\Delta t\, U\, H_x$。用这个近似，第三项从 50 次矩阵指数降到 0 次（只需已有的 U_j 与一次矩阵乘），每次迭代从 75 次降到 25 次，整体快约三倍。**本章没这么做，是为了让代码短、易读**；若要跑大段数优化，这是第一个该改的地方。

其三，**Adam 比梯度本身更重要**。脚本用的是 Adam（一阶矩 + 二阶矩），步长自动按梯度尺度缩放。这对量子控制特别关键：梯度在优化初期很大、后期很小，固定步长要么前期振荡、要么后期不动。换成朴素梯度上升，同一组参数下往往要慢一个数量级才收敛。

最后一条实务提醒：梯度检查。写完优化器，先用有限差分验一次解析梯度（相对误差应小于 10^{-6}），再开始调学习率。**梯度错了，后面所有的调参都是在拟合噪声**；而梯度检查只花十分钟。

83.10 工程落地

仿真里所有参数都是“真值”，实验里全是“估计值”，而且控制链上每一个模拟环节都会往 $1-F$ 里加一项。本节把这项账算清，并给出施工顺序。

83.10.1 误差预算：三项如何相加

以 250 ns 高斯为基准，逐项打开约束，得到表 83-6。

情形	保真度 F	损失 $1-F$	占总额
无耗散、理想控制链	0.999946	5.4×10^{-5}	2%
加退相干（T_1/T_2）	0.997512	2.49×10^{-3}	92%
加 100 MHz 重建带宽	0.999942	5.8×10^{-5}	2%
加 5 bit DAC	0.999808	1.92×10^{-4}	7%
三项同时	0.997304	2.70×10^{-3}	100%

表 83-6　250 ns 高斯的误差预算（前三行是单项打开，最后一行是全开）

这张表的核心信息是**加法成立**：$2.49+0.058+0.192=2.74$，实测 2.70，差 1.5%。这说明三项损失在 10^{-3} 量级上互不耦合，可以各自独立估算再相加。工程上这条性质非常有用：不必做全组合仿真，只要单项扫描就能分配指标。

第二层信息是**占比极度不均**：退相干占 92%，其余两项加起来 8%。这直接决定了施工顺序——先把 T_2 提上去（改芯片、改封装、减噪声），再谈 DAC 位数。反过来做，投入大、收益小。

83.10.2 施工顺序：按量级排，不按难度排

把 §83.7.5 的排序与表 83-6 合起来，得到一张施工清单：

1. **退相干**（10^{-3}）：换更高 T_2 的芯片、改善屏蔽与滤波、降低工作温度。这一项通常不是控制工程师能决定的，但它决定了保真度的天花板。
2. **重建带宽**（10^{-2} 到 10^{-1}，视带宽而定）：检查滤波器实际响应，必要时提高带宽或换高斯包络。
3. **DAC 位数**（10^{-2}）：低于 5 bit 必须换；5 bit 以上收益迅速饱和。
4. **时钟偏差**（10^{-3}）：用一个共同的 10 MHz 参考把 AWG 与 LO 锁在一起，并定期校准。
5. **时钟抖动**（10^{-7}）：最后才考虑，通常不需要专门处理。

这张清单有一个反直觉的地方：**抖动排在最后**。很多人一听“皮秒级抖动”就觉得是大事，但在整段脉冲上零均值抖动会相互抵消，200 ps 只造成 5×10^{-7} 的轨迹离散。真正致命的是系统性偏差——它不会抵消，每一段都往同一个方向偏。

83.10.3 门时间的实测复核

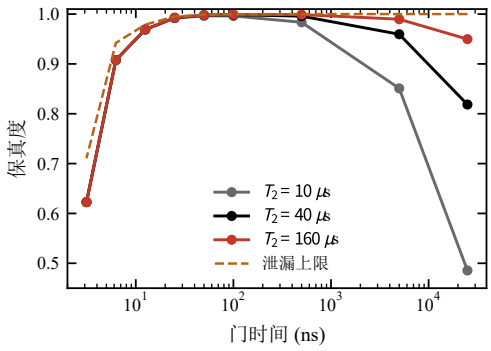


图 83-4　门时间的两条限制：慢门被退相干吃掉，快门被泄漏吃掉，中间存在最优区间

图 83-4 是设计阶段用的图，也是上机后复核用的图。做法是：固定包络形状，扫门时间，测保真度，把实测点画在理论曲线上。若实测曲线整体下移，说明有未建模的损耗（线缆、混频器、相邻比特）；若形状不对（比如最优点偏移），说明 T_2 或 α 的估计值有误。

这张图还给出一个“不可达区”的判断：当门时间短于 10 ns 时，曲线急剧下降，这是泄漏主导区；当门时间长于 1 μs 时，曲线也下降，这是退相干主导区。两段之间是“可用区”，本例大约 20–200 ns。

83.10.4 线缆、混频器与那些没建模的东西

本章的模型只包含“理想的 IQ 调制 + 一阶低通”。真实链路还有三样东西会造成额外损失。

线缆频响。同轴线在 4–8 GHz 的插损随频率上升，等效于一个不是一阶的低通，相位非线性。修法是在 AWG 输出端做预补偿（把线缆的 S_{21} 反卷积进波形），但这会让峰值幅值上升，受 DAC 满量程限制。

IQ 混频器的镜像与边带抑制。理想 IQ 混频只搬移一个边带，实际有 -20 到 -30 dBc 的镜像。镜像落在 $\omega_{\mathrm{LO}} + 2\omega_{\mathrm{IF}}$ 附近，若恰好靠近 $|1\rangle \leftrightarrow |2\rangle$ 的频率，会额外激发泄漏。修法是标定 IQ 的幅度/相位失配，做数字预失真。

相邻比特耦合。本章是单比特模型，多比特芯片上邻居的驱动会串过来。串扰强度通常在 -30 dB 量级，对应 Ω 的 3%，会造成 10^{-4} 量级的保真度损失。修法是在脉冲调度上做“避开同时驱动”。

这三项都不在 9×9 超算符里，也不该硬塞进去——它们的参数太多、标定太麻烦。正确做法是**把它们当作一个总的“未建模损耗”**，用实测数据反推一个等效值，加在预算表的最后一行。

83.10.5 温度、漂移与日常维护

量子比特的参数随温度与时间漂移。典型量级：T_1 在 10 mK 下每天漂 1%–3%，频率在每次上电时可能差几 MHz。这带来两个运维要求。

第一，**每次上电重标**。Rabi 扫频 + Ramsey 测 T_2，十分钟内能完成，不做的话后面所有门都建立在错误参数上。

第二，**保留基线**。每隔一段时间跑一次标准的 π 门与 Ramsey，记录 F 与 p_2。若某天 F 突然掉了 10^{-2}，先查硬件（线缆接头、LO 失锁），再查软件（波形文件是否被改）。

83.10.6 验收：三个层次

工程验收分三层，缺一层都不算交付。

第一层是波形层：用示波器（或矢量信号分析仪）看 AWG 输出，确认幅值、包络、相位都符合设计。这一层能抓到“波形文件错了”“通道接反了”这类低级错误。

第二层是门层：跑 Rabi、Ramsey、π 门保真度，与仿真对比。这一层能抓到参数标定、泄漏、DRAG 符号这类问题。

第三层是算法层：把门串起来跑一个简单的量子算法（比如两个比特的 Bell 态制备），看整体保真度是否等于单门保真度的乘积。这一层能抓到串扰、读数误差这类“单门看不出来”的问题。

三层的通过标准不同：第一层看波形误差小于 1%，第二层看 $F \geq 0.999$、$p_2 \leq 10^{-3}$，第三层看整体保真度不低于单门乘积的 90%。第三层的 90% 是经验值，留了 10% 给未建模的耦合。

83.10.7 上机前的十二项检查

把前面各节的要求收成一张检查单。每一项都能在半小时内做完，任何一项不过关，后面的保真度上限就被它锁住。

1. **示波器看波形**：直接测 AWG 输出，确认台阶清晰、无过冲、无振铃。振铃超过幅值 5% 说明线缆或滤波器不匹配。
2. **本振泄漏**：驱动关断时测 IQ 混频器输出的残余载波，应低于目标幅值 1%（对应 10^{-4} 的寄生 Rabi）。
3. **边带抑制**：单边带调制下，不要的边带应低于主边带 30 dB 以上。
4. **幅值线性**：从 10% 到 100% 幅值扫五点，测 Rabi 频率，线性度应优于 1%。
5. **频率准确度**：Rabi 扫频定位峰，精度优于 0.1 MHz（§83.3.7 的规格）。
6. **T_1、T_2 实测**：与上一轮记录比，变化超过 20% 要停下来查原因。
7. **α 实测**：双色谱找 12 跃迁，误差优于 5%。
8. **DAC 位数确认**：写入一个已知的斜坡，用示波器数台阶，确认不是“标称 14 bit、实际 10 bit”。
9. **时钟同步**：多台 AWG 共参考时，用示波器看两路输出的相位漂移，10 分钟内应小于 1 ns。
10. **延迟标定**：从触发到波形输出的固定延迟，影响多比特门的相对时序，必须测。

11. **热机**：开机后等 30 分钟再测，前 30 分钟的漂移通常是全天最大的。
12. **记录**：把上面十一项写进同一条记录（格式见 §83.8.6），下次对比。

十二条里最容易被跳过的是第 8 与第 10 条。前者因为“参数表写着”就被相信，后者因为“看不见”就被忽略；而它们恰好是“仿真对、实测不对”的常见原因。

83.10.8 仿真与实测不符时怎么定位

把实测点画到仿真曲线上，三种偏差形状对应三类原因。

整体下移，形状不变。 实测曲线比仿真低一个固定的量（比如都低 2×10^{-3}），说明有一项与门时间无关的损耗没建模：读数误差、相邻比特的残余耦合、两能级系统缺陷。这类错误的特征是“所有门都差一点”，不随参数变化。定位方法是做一次“零时长门”（只加一个很短的脉冲）测本底保真度，那个数就是这部分损耗。

形状变了，最优点偏移。 仿真说 25 ns 最好，实测说 40 ns 最好，说明 T_2 或 α 的估计值错了。若实测偏向更长的门，说明真实 T_2 比模型里的大（退相干没模型里那么严重）；若偏向更短的门，说明真实 α 比模型里的大（泄漏没模型里那么严重）。**最优点往哪边移，就调哪个参数，方向是反的。**

出现仿真里没有的振荡。 实测保真度随门时间出现周期性起伏，周期对应某个频率，说明有一个未建模的谐振：线缆的驻波、封装腔的寄生模式、或相邻比特的失谐。这类问题改波形治不好，只能改硬件或改频率规划。

三种形状的判据可以写成一页纸贴在机架上。它比“再多跑几组仿真”有效得多，因为**仿真只能验证你已经想到的机制，实测才能暴露你没想到的机制**。

83.10.9 扩到两比特时，哪三件事会变

本章全部是单比特。真实机器上至少两比特，扩展不是“把矩阵变大”这么简单，有三件事会实质改变结论。

第一件是状态空间。 两个三能级比特的维数是 9，超算符是 81×81，`expm` 的代价涨约 700 倍。§83.12.8 已经算过这笔账：单次仿真从毫秒级到秒级，仍可算，但“扫 30 个门时间”就变成几分钟。这决定了参数扫描必须挑着做，不能像单比特那样随意铺网格。

第二件是串扰。 单比特模型里没有的东西，两比特模型里全出来了：驱动一个比特时，另一个比特会感受到残余驱动（因为两者频率接近、微波泄漏）；两个比特之间的固定耦合会产生 ZZ 型相互作用，使得一个比特的频率依赖于另一个比特的态。ZZ 的典型量级是几十到几百 kHz——看起来只有单比特失谐容限（0.1 MHz）的几分之一，但它是**持续存在**的，等效于一个随时间变化的失谐，正好落在 §83.7.6 里“最敏感的一行”。

第三件是时序。 两比特门的两个通道之间有固定延迟差（§83.10.7 第 10 项）。这个延迟在单比特里无害，在两比特门里直接改变相对相位，而相对相位正是两比特门要控制的东西。25 ns 的门、1 ns 的延迟，相当于 14° 的相位误差，足以把 CZ 门的保真度从 0.999 拉到 0.99 量级。

三件事合起来给出一条经验：**单比特的结论可以照搬，单比特的数字不能照搬。** 波形形状、DRAG 的思路、误差预算的方法都能用；但“25 ns 最优”“0.5% 泄漏”这些数，在两比特系统里要重算，因为新增的 ZZ 与延迟会改写最优门时间。本章之所以只做单比特，是为了让每条因果链干净；真实系统的复杂度，从这三个方向开始增长。

83.11 优缺点与适用边界

83.11.1 这套方法的三个优点

第一，模型小到可以手算。 整个系统是 3×3 的密度矩阵、9×9 的超算符，参数只有十个（表 83-3）。这意味着任何一个结论都能追溯到某个具体参数，而不是“跑了个大模型”。本章的每个数字都可以用一张纸复算：泄漏用式 (83-14)，门时间用面积定理，退相干用 $e^{-\tau/T_2}$，三者相加就是预算表。

第二，误差可以逐项相加。 表 83-6 的加法误差只有 1.5%。这条性质不是巧合，而是因为各项损失都在 10^{-3} 以下，交叉项是 10^{-6} 量级。工程上可以据此分配指标：谁占的份额大就先修谁。

第三，物理图像清晰。 DRAG 修的是 $|1\rangle \leftrightarrow |2\rangle$ 的一阶耦合，高斯包络压的是高频分量，反馈压的是测量反作用——每一条都能用一句话说清"为什么有效"。这比"优化器给出的波形更好"要有用得多，因为它能指导下一个比特的设计。

83.11.2 四个明显的局限

第一，马尔可夫近似。 Lindblad 方程假设环境无记忆，这在 $1/f$ 噪声主导的低频段不成立。真实比特的 T_2 往往由 $1/f$ 磁通噪声决定，表现为"非指数衰减"，本章的 $e^{-\tau/T_2}$ 会低估长门时间的损失。

第二，单比特、无耦合。 本章模型只有一个比特，没有邻居、没有读取腔、没有耦合器。多比特门的保真度受串扰与耦合强度限制，那是一个完全不同的问题（涉及两比特哈密顿量与交叉 Kerr）。

第三，最优控制没有全局保证。 GRAPE 是局部梯度法，收敛到的是局部最优。式 (83-31) 的快速饱和既是优点（快）也是缺点（早熟）——换个初值可能得到更好的解，但没人能保证。

第四，没有考虑读取。 本章的"保真度"是末态布居，不含量子态层析与读取误差。实验上报告的"门保真度"通常已经用随机基准（RB）扣掉了读取误差，两者的定义要对齐才能比较。

83.11.3 适用边界：四个数

把本章的经验区间收紧成四条边界，超出任一条，本章的方法就要打折扣：

- $T_2 \geq 10\ \mu$s：低于这个值，25 ns 门的退相干损失超过 2.5×10^{-3}，与泄漏同量级，"先修泄漏"的策略失效。
- $\alpha/2\pi \geq 150$ MHz：低于这个值，$\Omega/\alpha > 0.13$，一阶 DRAG 的残余项进入 10^{-3}。
- 门时间 ≥ 10 ns：低于这个值，绝热条件（式 83-23）破坏，一阶 DRAG 不够。
- 单比特串扰 ≤ -25 dB：高于这个值，邻居的驱动会造成 10^{-3} 以上的额外损失。

四条里前三条是"设计约束"，第四条是"集成约束"。前者在芯片设计阶段确定，后者在整机联调时暴露。

83.11.4 与其它控制分支的关系

本章不是孤立的，它和前面几章有三处接口。

与 PID 的关系：第 44、106 章的 PID 是"连续测量 + 连续反馈"，本章的反馈冷却（§83.6.3）在形式上就是一个比例控制器 $\Omega = k(\langle\sigma_z\rangle + 1)$，只是被控量换成了量子期望值、测量换成了带反作用的弱测量。PID 的积分项在量子反馈里对应"估计器的记忆"，微分项对应"预测"，但两者的实现差别很大，不展开。

与 MPC 的关系：第 58、80 章的 MPC 是"用模型预测 + 滚动优化"。本章的 GRAPE 在数学上就是一个开环的、有限时域的优化问题，与 MPC 共享"用模型算梯度"这一核心，区别是 GRAPE 没有滚动、没有反馈、约束是逐段幅值而不是状态约束。

与 Smith 预估器的关系：第 47、107 章的 Smith 预估器处理"测量延迟"。量子反馈的延迟来自测量积分时间与 DAC 建立时间，处理思路相同：把延迟写进模型，用预估补偿。本章没有展开，因为 40 μs 的总时长下，几十 ns 的延迟相对影响很小。

一句话总结这三处接口：**经典控制的三大件（反馈、预测、延迟补偿）在量子控制里都有对应物，但每一件都被"测量反作用"改写了。**

83.11.5 什么情况下不要用本章的方法

有三种情形，本章的框架不适用，硬套会得出错误结论。

第一种是强耦合多比特系统。 两比特门的保真度受耦合强度、ZZ 串扰、读取腔色散共同限制，三能级单比特模型完全抓不到。此时应该用两比特或三比特模型，维度从 9 涨到 81 或 729，超算符方法仍然可用，但参数标定变得极其困难。

第二种是非马尔可夫噪声。 若 T_2 由 $1/f$ 噪声主导，Lindblad 方程会高估保真度。正确做法是用滤波器函数（filter function）方法，把噪声谱 $S(\omega)$ 与脉冲的频谱做卷积。那套方法的数学比本章重，但对低频噪声更准。

第三种是连续变量系统。 本章处理的是有限维（三能级），而连续变量量子信息（比如压缩态、GKP 码）用无穷维希尔伯特空间，工具是协方差矩阵与高斯信道，不是密度矩阵。

83.11.6 一张选型清单

最后给一张“要不要用本章方法”的清单，五个问题，全部答“是”才适用：

1. 对象能否截断到 3–5 个能级？
2. 噪声能否用马尔可夫近似（T_1、T_2 是常数）？
3. 控制是否是“分段常数 + 分段之间无反馈”？
4. 指标是否是“末态保真度 + 泄漏”？
5. 是否只需要单比特门？

五条全中，本章的 9×9 超算符就是最省事的工具；有一条不中，就要换更重的框架。这不是本章方法“不够好”，而是任何模型都有适用范围——**知道边界在哪，比知道方法本身更重要。**

83.11.7 与第 84、85 章的衔接

本章的“控制器”是一台 AWG：它有采样率、有量化位数、有重建滤波器，本质上就是一台专用计算机。第 84 章要把“控制器”从专用硬件换成通用计算机，第 85 章则把采样、量化、零阶保持、重建这套结构系统地讲一遍。三章之间有三条可以直接搬过去的结论。

第一条是**采样率的选法**。经典数字控制里，采样频率取对象带宽的 10–100 倍；本章的段长 1 ns 对应 1 GSa/s，而门脉冲的频谱主瓣只有几十 MHz，比值也在几十倍。两者不是巧合：**采样率要高于“波形里最快的变化”，而不是高于“对象里最快的动态”**。第 85 章会用这个判据去分析为什么有些系统要 10 kHz、有些要 1 GHz。

第二条是**量化位数的饱和**。本章的 DAC 曲线在 8 bit 以上完全饱和（0.999942 到 0.999946），因为量化误差已经小于其它误差项。任何数字控制系统的位数选择都遵循同一条逻辑：**位数不是越高越好，够到“不成为最大误差项”就停。** 第 85 章会把这个判断写成“A/D 位数与传感器噪声匹配”的形式，本质相同。

第三条是**抖动与偏差要分开处理**。零均值抖动不伤均值（200 ps 只带来 5×10^{-7} 的标准差），系统性偏差线性伤均值（2% 带来 10^{-3}）。第 85 章讲时钟与采样时会把这两类误差写进同一个误差模型，本章给了它们在量子对象上的实测权重。

反过来说，第 85 章的稳定性分析（z 域极点、闭环增益）在本章基本用不上。原因是本章的任务是“有限时间到达”，没有“扰动后回到平衡点”这回事；闭环极点、相位裕度这些概念只在 §83.6 的反馈冷却里出现，而且那里的“稳定性”是随机系统的稳态分布，不是确定性系统的极点位置。**读第 85 章时不要把极点判据硬套到门控制上**，这是两套问题。

83.11.8 三条替代路线

本章的主线是“方波 → 高斯 → DRAG → GRAPE”。工程上还有三条常用路线，各有各的适用面。

复合脉冲（composite pulses） 把一个大角度的门拆成几段小角度，通过设计各段的相位与幅值，让某一类误差在段间相互抵消。最著名的形式是“三段式”：$\pi/2$、π、$\pi/2$ 配以特定相位，对幅值误差一阶不敏感。它的优点是不需要包络的导数（DRAG 需要），因此对采样率低、波形粗糙的系统更友好；缺点是把门拉长了两三倍，退相干损失随之增加。**在 T_2 充裕、控制链粗糙的机器上，复合脉冲往往比 DRAG 更实用。**

绝热捷径（shortcuts to adiabaticity, STA） 在哈密顿量里加一个“反绝热项”，让系统在有限时间内走完本该绝热走完的路径。它的优点是速度可以远快于绝热极限，缺点是需要额外的控制通道（往往是一个不该有的耦合），在超导上实现起来比 DRAG 复杂得多。**它适合“态转移”而不太适合“标准门”**，因为标准门要求的是固定的酉变换，而 STA 给出的是沿路径的演化。

傅里叶基优化（CRAB 一类） 不用 25 个独立的段值，而用几条谐波（正弦、余弦）描述波形，自由度往往只有 6–10 个。自由度少的好处是**不容易过拟合模型**，优化出来的波形更平滑、更容易上机；坏处是表达能力受限，压不到极致的保真度。**若你的目标是 10^{-4} 而不是 10^{-5}，CRAB 往往比 GRAPE 更省事。**

三条路线的共同点是：它们都试图用**结构换性能**，而不是用算力换性能。这一点与 §83.9.7 的结论一致——真正难的不是把优化器写对，而是把物理约束写进波形结构里。

83.11.9 本章没讲的三件事

一本书的边界比它的内容更需要说清。本章刻意绕开了三个重要题目，每一个都能单独写一章。

第一是读出。 本章的所有保真度都是“态本身”的保真度，隐含假设“读出来就是对的”。真机上读出有错误：典型超导系统的单次读出保真度在 95%–99% 之间，意味着即使门做得完美，你测到的数也会低几个百分点。读出错误不进入本章的 Lindblad 模型，因为它发生在门之后，是“测量链”的问题而不是“演化”的问题。判断方法很简单：做一次“零门”实验（不加任何脉冲，直接测），测到的 $|0\rangle$ 比例就是读出保真度的下界。

第二是非马尔可夫噪声。 本章的 T_1、T_2 对应马尔可夫近似：噪声没有记忆，衰减是指数的。真机上的主导噪声往往是 $1/f$ 型（磁通噪声、电荷噪声）或两能级系统（TLS）引起的，有记忆、非指数。表现是 T_2^* 远小于 $T_{2,\mathrm{echo}}$：前者含低频漂移，后者用回波把慢漂移抵消掉。本章 §83.7.7 之所以建议测 $T_{2,\mathrm{echo}}$，就是这个原因。要真正描述非马尔可夫噪声，得把 Lindblad 换成滤波器函数方法，传播器要重写（§83.12.8 的扩展方向之一）。

第三是量子纠错。 纠错把“一个逻辑比特”编码到很多物理比特上，控制系统的任务从“做好一个门”变成“持续测量、实时解码、条件纠正”。这对控制提出了全新要求：延迟必须压到 $1\ \mu$s 以下（§83.6.8 的延迟预算在这里变得很紧），经典算力要能实时跑解码算法，而且测量不能再是“门之后做一次”，而是“一直在做”。本章的方法在纠错里只用到一小部分——单比特门与初始化，其余是另一个学科。

三件事有一个共同点：**它们都不在“波形设计”这一层，而在“波形之外”。** 本章把波形这一层讲透，是因为它是唯一能用手算、能用笔记本验证的一层；再往上，每一层都需要专门的实验手段与更大的模型。读到这里，你应该清楚哪些结论可以直接用、哪些必须扩展模型。

83.12 Python 仿真代码

本章代码分两个文件：`code/ch083_sim.py` 负责全部数值与统计，`code/ch083_figs.py` 负责十张插图。两者都用 `D:\Anaconda3\envs\py311\python.exe` 运行，不依赖任何量子计算库，只需要 NumPy 与 SciPy 的 `expm`。

83.12.1 文件结构

`ch083_sim.py` 分五层，从上到下依次是“算符—传播—脉冲—优化—实验”：

```
CFG                       全局参数（表 83-3）
ops() / h0()              三能级算符与漂移哈密顿量
lindblad() / liouvillian()  耗散算符与 9×9 超算符
evolve()                  分段传播，支持五种约束
rect() / gauss_pulse() / drag()  三种脉冲构造
rabi() / grape() / feedback_traj()  三套求解器
experiment_*()            十一组实验
report()                  打印全部数字
```

这种分层的好处是：改一个约束只动 `evolve()` 一个函数，改一个实验只动对应的 `experiment_*()`，互不影响。

83.12.2 核心：三行定义一个超算符

整个仿真的心脏是下面这段，它把式 (83-8) 直接翻成代码：

```
def liouvillian(h, cops):
    """把 dρ/dt = Lρ 写成 9×9 超算符。"""
    eye = np.eye(3)
    mat = -1j * (np.kron(eye, h) - np.kron(h.T, eye))
    for c in cops:
        cd = c.conj().T
        mat = mat + np.kron(c.conj(), c)
        mat = mat - 0.5 * (np.kron(eye, cd @ c) + np.kron((cd @ c).T, eye))
    return mat
```

三行分别对应哈密顿项、量子跃迁项、反交换子项。写成 `kron` 的好处是不用手写 81 个矩阵元，也不容易错符号；代价是要理解展平约定（§83.2.3 已经交代过，本章只读对角元，对约定不敏感）。

83.12.3 传播器：一个函数装五种约束

`evolve()` 是本章最长的函数，也是唯一需要小心的地方。它的签名是

```
evolve(u, ts=None, t1=None, t2=None, sub=None, bandwidth=None,
       jitter=0.0, clock=0.0, bits=None, rho0=None, keep_trace=False,
       seed=None)
```

七个可选参数对应七种“非理想”：`bandwidth` 是一阶低通，`jitter` 是段边界抖动，`clock` 是系统性时钟偏差，`bits` 是 DAC 量化，`t1/t2` 是耗散，`sub` 是积分子步，`keep_trace` 记录 Bloch 三分量。默认值全部是“理想”，所以 `evolve(rect(25))` 就是最简单的理想 π 脉冲。

五个约束里，`jitter` 与 `clock` 共用同一段代码（`widths` 的计算），区别是前者随机、后者确定性。这个设计让“零均值抖动无害、系统性偏差有害”的对比可以在同一函数里完成，§83.7.5 的结论就是这么跑出来的。

`keep_trace=True` 时记录三个期望值：

```
hist.append([float(np.real(rho[0,1] + rho[1,0])),      # ⟨σx⟩
             float(np.real(1j*(rho[0,1] - rho[1,0]))),  # ⟨σy⟩
             float(np.real(rho[0,0] - rho[1,1]))])      # ⟨σz⟩
```

这三个数就是图 83-3 的数据源。注意 $\langle\sigma_z\rangle$ 用的是 $\rho_{00}-\rho_{11}$，与 §83.6 的反馈仿真同一约定，两处的“-1 表示 $|1\rangle$”可以直接对照。

83.12.4 脉冲构造：三个函数覆盖全部算例

```
def rect(nseg, amp=None, phase=0.0):      # 方波
def gauss_pulse(nseg, tau_ns=250.0, sigma_frac=0.25, phase=0.0):  # 面积归一化高斯
def drag(u, alpha=None, ts=None, beta=0.5):  # DRAG 修正
```

`gauss_pulse` 的关键是**面积归一化**：它按 $\int\Omega\,\mathrm{d}t=\pi$ 反解峰值，所以无论 `nseg` 取多少，转角都是 π。这就是 §83.3.4 里“段数几乎不重要”的原因——面积守恒，分段只改高阶项。

`drag` 的实现只有一行 `return u + 1j*beta*np.gradient(np.real(u), ts)/alpha`。符号取正，理由见 §83.2.7；错号的对照在 `experiment_leak()` 里单独构造，不污染主函数。

83.12.5 三套求解器

`rabi()` 是解析式 (83-20) 的直接实现，用来画图 83-2，不涉及数值积分。

`grape()` 是本章唯一带优化的函数。它的结构是“正向传播存态—反向传播存伴随—算梯度—Adam 更新”，25 段、300 次迭代，学习率 0.03，幅值裁剪到 $[-1,1]$ 再乘 $\Omega_{\max}$。梯度公式用伴随法，一次迭代的代价是两倍正向传播，与段数成正

比。

`feedback_traj()` 完全不碰密度矩阵传播器，它单独实现两能级随机主方程（式 83-28），步长 2 ns、总时长 40 μs，用欧拉-丸山格式积分。返回的 `z` 是 $\langle\sigma_z\rangle$ 的时间序列，`mean` / `std` 是末段的统计量。

三套求解器互不调用，这是有意的：门控制、最优控制、反馈控制三件事的数学结构不同，硬凑成一个函数反而看不清。

83.12.6 十一组实验与一份报告

`experiment_*()` 一共十一个，对应 `report()` 里的 `[0]`–`[10]`：

编号	函数	内容	正文位置
0	`experiment_ideal`	理想 25 ns 方波	§83.1.1
1	`experiment_rabi`	四档失谐的 Rabi	§83.4.1
2	`experiment_gate_time`	门时间与两条限制	§83.1.3
3	`experiment_slices`	时间量化与分段	§83.5.1
4	`experiment_leak`	泄漏与 DRAG 三符号	§83.8.3
5	`experiment_grape`	GRAPE 收敛与波形	§83.9.2
6	`experiment_bandwidth`	重建带宽	§83.5.3
7	`experiment_dac`	DAC 位数	§83.5.2
8	`experiment_jitter`	抖动与时钟偏差	§83.7.5
9	`experiment_feedback`	20 条反馈轨迹	§83.6.3
10	`experiment_budget`	误差预算	§83.10.1

表 83-7　`ch083_sim.py` 的十一组实验与正文对应关系

`report()` 只做打印，不做计算，所有数字都来自上面十一个函数。这个约定让“书里的数字”与“代码的输出”一一对应，改一处代码就能看出哪一段正文要跟着改。

83.12.7 运行与预期输出

```
D:\Anaconda3\envs\py311\python.exe code/ch083_sim.py
D:\Anaconda3\envs\py311\python.exe code/ch083_figs.py
```

第一条命令约十几秒（GRAPE 占大头），打印 `[0]`–`[10]` 十一组数字；第二条生成 10 张图、每张两种格式（SVG + PDF），共 20 个文件到 `figs/ch083/`。两个脚本都带自检：`ch083_sim.py` 末尾检查“布居守恒 $p_0+p_1+p_2=1$”与“GRAPE 不劣于方波”，不过就报错退出。

一个容易踩的坑：`ch083_figs.py` 里所有含中文的坐标轴标签都必须走 `setup()` 设好的字体回退链，且**中文与公式不能写在同一个字符串里**。例如 `"保真度"` 可以，`"保真度 $F$"` 不行——后者会让 mathtext 去解析中文，字体回退失效，图上出现方框。本章统一写成“保真度”加单独的数学标签，就是为了避开这个坑。

83.12.8 复现要点与扩展方向

复现本章全部数字只需要三件事：Python 3.11、NumPy、SciPy。随机性只出现在三处（jitter、feedback、grape 初值），都由 `seed=831` 固定，所以结果逐位可复现。

若要扩展，三个方向最自然：

加能级。把 3 改成 4 或 5，`ops()` 里的 $\sqrt{2}$ 要改成 $\sqrt{2},\sqrt{3}$，超算符从 9×9 变成 16×16 或 25×25，其余代码不动。可以研究“更高能级对泄漏的贡献”。

加噪声谱。把 Lindblad 换成滤波器函数方法，需要重写传播器，但 `experiment_*()` 的接口可以保留。这是从马尔可夫走向非马尔可夫的最小改动。

加第二比特。维度从 9 到 81，超算符 81×81，`expm` 的代价涨约 700 倍，单次仿真从毫秒级到秒级，仍然可算。这是研究两比特门与 ZZ 串扰的起点。

代码到正文的对应关系就是上面这张表。读代码时建议从 `report()` 倒着读：先看要输出什么，再找对应的 `experiment_*()`，最后才看 `evolve()` 的细节。这样不会被五个可选参数绕进去。

83.12.9 五个常见报错与改法

把本章代码交给别人跑，报错几乎总是下面五条之一。

`KeyError: 'F'`。`experiment_grape()` 返回的是 `F_flat`、`F_grape` 两个键，没有笼统的 `F`。这是刻意的：一次优化有两个保真度，必须指明是哪一个。改法是把 `r["F"]` 换成 `r["F_grape"]` 或 `r["F_flat"]`。

`F=nan`。多半是 `T2 > 2*T1`。脚本按 $1/T_\phi = 1/T_2 - 1/(2T_1)$ 算纯退相位率，若 $T_2 > 2T_1$，这个差是负的，开方得 NaN。这不是 bug，而是物理约束被违反：任何系统的 T_2 都不可能超过 $2T_1$（T_1 过程本身就贡献退相位）。改法是检查输入的 T_1、T_2 是否写反。

`ValueError: Invalid RGBA argument`。把 `LINE_MAIN` 这类样式字典直接传给了 `color=`。`LINE_MAIN` 是 `{'color': 'black', 'linestyle': '-', 'linewidth': 1.2}`，要么整包用 `**LINE_MAIN`，要么只取颜色 `color=LINE_MAIN["color"]`。本章的图脚本采用的是后者。

图里出现方框（`Font 'default' does not have a glyph`）。中文与 `$...$` 写在同一个字符串里。mathtext 只认数学字体，遇到中文就回落成一个空方框。改法是把一条标签拆成两条，或把中文里的数学量写成纯文本（例如把 `$\beta=0.5$` 写成 `β=0.5`）。

结果与正文对不上（差 2–3 倍）。最常见的原因是展平顺序：`liouvillian()` 用的是 C 序，若你自己写的 `evolve()` 用了 Fortran 序，F 会出现“看起来合理但不对”的值。判据很简单：跑一次无耗散、无泄漏的极限（$\alpha\to\infty$），F 应当趋近 1；若趋近 0.5 或振荡，就是展平或符号错了。

五条里只有第二条是物理错误，其余四条都是接口错误。**接口错误的表现往往比物理错误更“像对的”**，因为它们不会崩，只会给你一个错数。

83.12.10 把这套代码换成你的机器

代码是按本章的 transmon 写的，但结构是通用的。换成你自己的机器，只需要动四处。

第一处是 `CFG`。把 `omega_max`、`alpha`、`ts`、`t1`、`t2` 换成实测值，`sub` 与 `seed` 一般不用改。这一步决定所有数字的量级，所以要先做 §83.10.7 的十二项检查，别拿标称值凑。

第二处是 `ops()` 与 `h0()`。若你的对象不是三能级 transmon（比如是两能级超导、离子阱的集体自旋、或中性原子的基态超精细结构），要重写算符与自由哈密顿量。两能级的话把 `a` 取成 2×2 的 σ_-，`h0()` 返回零矩阵，`sz` 取 σ_z，其余代码原样可跑——这时超算符从 9×9 降到 4×4，单个矩阵指数的代价降到约十分之一，但泄漏项自然消失（两能级没有 $|2\rangle$）。

第三处是 `lindblad()`。T_1、T_ϕ 的算符形式取决于噪声模型。本章用的是“振幅阻尼 + 纯退相位”，对超导足够；离子阱的主导噪声往往是激光的相位噪声，应该换成 σ_z 上的有色噪声，这就要改传播器。

第四处是 `experiment_*()` 里的参数网格。把门时间、带宽、位数的扫描范围改成你自己机器的量级。**网格的范围比网格的密度重要**：范围选错（比如把 25 ns 门扫到 500 ns）会得到一条完全无信息的曲线。

四处改完，`report()` 打印的十一组数字就是你机器上的预测值。把它们与实测对比，偏差的形状会告诉你还缺哪一项模型（§83.10.8）。这一步做完，这套代码就从“书上的例子”变成了“你自己的工具”。

83.12.11 代码里刻意没做的四件事

本章的代码只有三百多行，其中有一半是注释与打印。有些“该有”的东西刻意没写，理由如下。

第一，没有自适应步长。 一般求解器会按局部误差自动调整步长，本章坚决用固定段长。原因是控制场本身就是分段常数，段与段的边界是物理事实，不是数值选择；一旦让步长自适应，就可能在段中间插入半步，把“这一段是常数”这个前提破坏掉。固定段长的代价是慢一些，好处是每一步都能对应到硬件上的一个采样点——**仿真与机器的对应关系比速度重要。**

第二，没有并行与 GPU。 全章十一组实验跑 29 s，为此写多进程、传数据、合并结果，除了增加出错面没有别的好处。真正的并行需求出现在参数扫描（§83.2.10 说的“全交叉 10^5 量级”），那种场景应该换工具，而不是把本章的脚本改复杂。

第三，没有配置文件与命令行参数。 所有参数集中在 `CFG` 一个字典里，改参数就改那一处。很多教学代码用 argparse 把参数放到命令行，看着专业，实际让“这一组数是用什么参数跑出来的”变得难以追溯——一份输出配一份命令行，很容易对不上。**参数只有一个入口，记录才可靠。**

第四，没有测试框架，只有一个 `_selftest()`。 它检查两件事：布居守恒（$p_0+p_1+p_2=1$，误差小于 10^{-12}）与 GRAPE 不劣于方波。真正的测试套件应该覆盖每个函数，但书里的代码首先要求“能一眼读完”，把两百行 pytest 塞进来会淹没主线。这里的取舍是明确的：**可读性优先，正确性靠“每个数字都能在正文里对上”来保证。**

四件事的共同逻辑是：本章代码是**用来解释模型的**，不是用来生产数据的。一旦你的需求变成“每天跑上千组参数、自动出报告”，正确的做法是照本章的结构重写一套工程代码，而不是在这三百行上加功能。这也是全书的一贯立场——示例代码求清楚，生产代码求稳，两者不该混在一起。

83.12.12 对着报告读正文

`report()` 打印十一组数字，编号 `[0]` 到 `[10]`。它们不是随意排的，而是按“单脉冲—门时间—波形—优化—硬件—误差”的顺序走了一遍。第一次读时最省力的办法是打印出来当索引，一行一行对着正文找。

表 83-8 给出每一组输出对应哪一节、看哪个量、常见误读是什么。三列里第三列最有用：**每一组都有一个“看起来对但其实错”的读法**，那些错读几乎都来自把某一个数当成孤立的指标。

表 83-8 `report()` 十一组输出的读法

组号	打印内容	对应正文	怎么看，以及常见误读
`[0]`	25 段方波的旋转角、F、p_2	§83.4	$F = 0.992409$、$p_2 = 4.86 \times 10^{-3}$；误读：把 0.9924 当成“这机器只能做到 99.24%”，其实 $1 - F = 7.6 \times 10^{-3}$ 里近三分之二是泄漏，不是退相干
`[1]`	四个失谐下的 Rabi 峰值与 π 时间	§83.4.6、§83.4.7	峰值 1.0000/0.9412/0.8000/0.5000；误读：拿解析式 $\Omega^2/(\Omega^2+\Delta^2)$ 去核 10 MHz 那行——解析给 0.800，实跑 0.7248
`[2]`	三个门时间 × 三个 T_2	§83.7	两端分别掉到 0.486 与 0.623；误读：只看中间那列的好看数，忽略两端，而两端恰好是两个不同的失效机制
`[3]`	1 ns 量化台阶与 3–250 段阶梯	§83.5.6、§83.5.8	台阶 0.98799/0.99267/0.98938；误读：以为段数越多越好，3→50 段只差 1.7×10^{-5}
`[4]`	三个门时间 × 三种 DRAG 符号	§83.4.5	正号从 0.99993 起，错号最低到 0.97254；误读：看到“错号也比不加好”就以为符号不重要，其实差距随门变快而放大
`[5]`	方波初值与 GRAPE 结果	§83.9	0.992409 → 0.995415；误读：把它当全局最优——GRAPE 只在这 25 段、这组参数与这个目标函数下最优
`[6]`	六个带宽下的方波保真度	§83.5.7	400 MHz 到 10 MHz：0.99207 → 0.50306；误读：直接把这列当“滤波器指标”，它同时含方波本身的带宽代价
`[7]`	3–12 bit 的 DAC 量化	§83.5.6	8 bit 已饱和在 0.999946；误读：以为位数越多越好，10、12 bit 与 8 bit 逐位相同
`[8]`	抖动与时钟偏差	§83.5.6	200 ps 抖动只带来 4.98×10^{-7} 的标准差，2% 时钟偏差掉到 0.998961；误读：把两者都当“时间误差”合并处理，实际一个有偏一个无偏
`[9]`	有/无反馈的末段 $\langle\sigma_z\rangle$	§83.6	有反馈 -1.000、标准差 0.000，无反馈 $+0.090$、0.740；误读：拿它和 `[0]` 的 F 直接比，两者量的不是同一件事
`[10]`	逐项加进去的误差预算	§83.8	0.999946 → 0.997512 → 0.999942 → 0.999808 → 0.997304；误读：把各项损失

组号	打印内容	对应正文	怎么看，以及常见误读
			直接相加，实跑的最后一行比相加值小

这张表还有一条横向的读法：**从上往下看，噪声项在变多，但最大的一项始终在换。**`[0]` 里最大的是泄漏，`[6]` 里最大的是带宽，`[8]` 里最大的是时钟偏差，`[10]` 里最大的是退相干。这说明“哪个误差主导”不是机器的固有属性，而是**跟你把门做多快、把滤波器设多宽绑在一起的**。同一台机器，换一组工作点，要修的东西就换了。

因此报告不能只跑一次。合理的用法是：先用默认参数跑一遍，找出 `[10]` 里最大的那一项；再回到对应的实验组，把那一项的参数扫一遍，看它在什么条件下让位给别的项；最后才决定硬件上先改哪里。这个“跑一遍—找最大项—扫一遍”的循环，比盯着单个保真度数字看有用得多。

十一组之间还能互相校验，这是比单看一组更强的用法。三条固定的一致性关系：`[1]` 给出的 π 时间应当与 `[0]` 用的 25 ns 对得上，因为两者都设 $\Omega/2\pi = 20$ MHz；`[7]` 里 5 bit 那行的 F 应当与 `[10]` 里“DAC 5 bit”那一项完全相同（0.999808），若不等，说明两次调用的 `bits` 或段长没传一致；`[6]` 里带宽设成很高（比如 400 MHz）时应当接近 `[0]` 的 0.99207，与 `[0]` 的 0.992409 相差 3e-4，这点差来自滤波器把方波的台阶抹圆、脉冲面积随之微变。**三处对不上，先怀疑接口，别怀疑物理**（§83.12.9）。

最后提醒一句：十一组输出的所有数字都在 29 s 内跑完（§83.1.7），所以**没有任何理由凭记忆引用**。正文里的每一个数都是从这份报告里抄下来的，你重跑一遍就能逐位核对。这也是本章把 `report()` 写成单一入口的原因——一份输出，一个真相。

编号的顺序本身也有信息：越往后，模型里加进去的东西越多——`[0]` 只有三能级与耗散，`[6]` 多了滤波器，`[7]` 多了量化，`[8]` 多了时钟，`[9]` 干脆换了一套随机主方程的算法。但读的时候建议反过来走：**先看 `[10]` 的总账，再回头找是谁贡献的**，这样不会被前面那些干净的数误导成“我的机器应该很好”。另外 `[9]` 是二十条轨迹的统计量，末段均值与标准差本身就带抽样误差。改 `seed` 时 `[5]`、`[8]`、`[9]` 三组都会动——优化初值、抖动与轨迹抽样共用同一个种子（§83.12.8）——其余各组不受影响。看到这三组的小数位在变，不要以为代码坏了。

一句话总结：报告是索引，正文是解释，代码是底稿。三者对不上时，先查接口，再查单位与展平约定，最后才怀疑物理模型本身。第 83 章到此结束，下一章换一个角度：不再问“怎么把一个比特转到位”，而是问“整台机器的控制信号是怎么被组织、同步与记录下来的”。